TIS Project Reference Booklet

Documentation version: 3.1

Generated: 2026-09-25 12:26 Arab Standard Time

Branch: dev

Git commit SHA: 7372b6b

Source of truth: Markdown files under docs/ are authoritative. This PDF is a generated snapshot and must not be edited manually.

Use the table of contents and PDF bookmarks to move directly to each source document. Major document headings are available as child bookmarks.

How to Use This Handbook

Start with the TIS KMS Navigation Guide to choose the smallest useful reading path for your role or task. Use this booklet when a portable, reviewable snapshot is more convenient than browsing the Markdown sources.

Navigate

Use the table of contents for document-level page numbers. In PDF viewers that support outlines, open the bookmarks panel to browse every source document and its major headings.

Verify

The title page records the documentation version, generated timestamp, branch, and Git commit SHA. The generated manifest records source hashes and each document's starting PDF page.

Source Of Truth

Markdown files under docs/ remain authoritative. This PDF is generated and must never be edited manually. When source documents change, run the approved KMS synchronization command to regenerate this snapshot and its manifest.

Table of Contents

TIS Documentation Index . . . 9

TIS KMS Navigation Guide . . . 12

TIS AI Project Context . . . 17

TIS Documentation Update Policy . . . 75

TIS Master Context . . . 78

TIS Project State . . . 124

TIS Change History . . . 215

TIS Architecture Decision Records . . . 314

TIS Engineering Handbook . . . 316

TIS Module Map . . . 318

TIS Repository Architecture . . . 359

TIS User And System Flows . . . 371

TIS Database Architecture Overview . . . 417

Independent Permission Closure Review . . . 435

TIS Development Standards . . . 445

TIS UI UX Design Philosophy . . . 449

TIS Product Roadmap . . . 454

TIS Rejected Architectural Decisions . . . 462

TIS Visual Documentation Guide . . . 465

TIS AI Optimization Guide . . . 469

TIS Project Governance . . . 473

TIS Knowledge Lifecycle . . . 478

TIS Documentation Automation . . . 481

TIS Knowledge Impact Assessment Standard . . . 486
TIS Self-Evolving Workflow . . . 489
TIS Documentation Dependency Map . . . 492
TIS AI Coding Workflow . . . 495
TIS Future Automation Roadmap . . . 499
Separate Next.js Landing Website . . . 502
Separate SaaS Identity And Operational Users . . . 503
Paddle Payment Architecture . . . 504
Webhook-Only Payment Confirmation . . . 506
Delayed Tenant Provisioning After Verified Payment . . . 508
Documentation As Source / Knowledge Management System . . . 509
Landing Page Visual System Strategy . . . 511
Workspace Classification Foundation . . . 512
Commercial State And Entitlement Resolution . . . 514
Review-Only SaaS Demo Requests Before Provisioning . . . 516
Demo Workspace Provisioning And Commercial Source Links . . . 517
Seven-Day Demo Lifecycle And Access Enforcement . . . 518
Demo-To-Paid Workspace Conversion . . . 519
Demo Customer Communications And Approval Orchestration . . . 520
Centralized AI Entitlements And Usage Accounting . . . 521
ADR 0016 - Pre-Deploy Database Migration Boundary . . . 522
Platform Owner Demo Operations And Access Profiles . . . 524
Unified Commercial Access And Capacity Authority . . . 526
Secure Promo Code Definition And Governance . . . 528
Promo Redemption And Commercial Grants . . . 530

Existing Workspace Conversion Read-Only Audit . . . 532

Controlled Existing Workspace Owner Alignment And Activation-Required Conversion . . . 533

Existing Workspace Paid Activation Through Paddle . . . 535

Promo Expiry Recovery And Paid Continuation . . . 537

Capacity-Based Packaging And Common Customer Feature Baseline . . . 538

Versioned, Constraint-Based Smart Timetable Generation . . . 540

B10 Longitudinal Organization Intelligence Contract . . . 545

B11 Production Qualification Policy . . . 549

M10 Organization Analytics REPEATABLE READ Transaction Boundary . . . 554

M10 Organization Analytics Multi-Worker Qualification Minimum . . . 557

Student Learning Style V1 . . . 559

Draft Talent Program Hard Delete (Scoped Exception) . . . 564

ADR 0033: Additive Roster Synchronization While A Talent Cycle Is Open . . . 566

Talent Student Assessment Delete (Zero-Evidence Exception) . . . 568

ADR 0035: Talent Assessment Eligibility Uses Live Academic Placement . . . 570

Talent Rubric Re-evaluation Attempts . . . 572

Talent Overall Program Result . . . 576

Talent Evaluation-First Assessment and Review Workspace . . . 581

ADR 0039: Talent Competency -> KPI -> Level Simplification . . . 585

Per-User Permission Override Precedence, Deny-over-Allow, and Platform-Actor Scope Resolution . . . 588

Promo Grant Replacement (Promo-to-Promo) . . . 591

Student Learning Style Four-Dimension Profile (Schema Foundation) . . . 593

TIS Student Number Global Managed-Identifier Invariant (Schema Foundation) . . . 596

Talent Evaluation Progress Analytics (M4) . . . 598

Al-Andalus Section Display - Workspace Authorization (Governance Only) . . . 603

Academic Calendar History . . . 605
AI Entitlements History . . . 606
M8B9 Demo Operations, Notifications, And Testing . . . 607
Common Customer Demo Feature Baseline . . . 608
KMS v3 Engineering Handbook . . . 609
KMS v3 Phase 3B Engineering Layers . . . 611
KMS v3 Phase 3C Governance And AI Traceability . . . 613
KMS v3 Phase 3D Lifecycle Foundation . . . 615
Production Memory Stability Guardrails . . . 617
Automatic KMS Synchronization Enforcement . . . 619
KMS Phase 7A Navigation Foundation . . . 621
KMS Phase 7B Professional PDF Navigation . . . 622
KMS Phase 7D Navigation And Catalog Enforcement . . . 623
Claude Code KMS Configuration . . . 625
Cline KMS Configuration And Three-Agent Orchestration . . . 626
M10 B11-C PostgreSQL Profiling Evidence Remediation . . . 627
M10 B11-D PostgreSQL Concurrency And Consistency Qualification . . . 648
M10 B11-E Integrated Production Qualification (REPEATABLE READ Implementation) . . . 660
M10 B11-E F2 PostgreSQL Qualification (Independent Re-Verification, Bounded) . . . 668
M10 B11-E Live Multi-Worker, Performance & Memory Qualification (Direct Execution) . . . 671
Engineering Handbook History . . . 675
Landing CTA Consolidation And Demo Domain Policy . . . 677
M8 Landing Integration Open Account . . . 678
Landing Page History . . . 680
Platform Owner Knowledge Center . . . 681

KMS Phase 7C Knowledge Center Navigation . . . 683
Platform Knowledge Center History . . . 685
KMS Foundation . . . 686
Provisioning History . . . 688
TIS Module History . . . 689
SaaS Onboarding History . . . 690
Subscription History . . . 699
Smart Timetable Stage 2 Version Foundation . . . 705
Smart Timetable Stage 3 Readiness And Slot Semantics . . . 706
Smart Timetable Stage 3.5 Composed Timeline . . . 707
Smart Timetable Stage 4 Version Publication . . . 708
Smart Timetable Stage 5.1 Generator . . . 709
On-Demand Timetable Generation Workflow . . . 710
Smart Timetable Stage 5.2 Simplified Workflow And Published Visibility . . . 711
Explicit Planning Subject Demand Foundation . . . 712
Planning Subject Demand Consumer Transition . . . 713
Atomic Curriculum Adjustment Apply . . . 714
Read-Only Curriculum Adjustment Preview . . . 715
Curriculum Transfer and Subjects Display Correction . . . 716
Guided Curriculum Adjustment UI . . . 717
Reduce-Only Curriculum Adjustment . . . 718
Teacher Scheduling Rules . . . 719
Advanced Timetable Release 1 Reconciliation And Solver Adapter . . . 721
Structured Timetable Conflict Evidence . . . 722
Timetable Lesson Requirement Projection . . . 723

Professional Timetable Lifecycle UX 724

Planning And Subject Delete Dependency Guards 725

Return-To Reopen-On-Load UI Pattern 726

Planning Subject Requirement Removal (Single And Bulk) 727

Workforce Planning History 729

Workspace Classification History 731

TIS Landing Page Source of Truth 732

TIS Landing Page Master Content 733

TIS Location Data Roadmap 741

Students + Talent & Potential Update Package Release Handoff (M1-M13) 743

TIS Documentation Index

docs/README.md

TIS Documentation

This folder is the source of truth for the TIS Knowledge Management System (KMS).

Markdown files are authoritative. The PDF booklet is a generated snapshot and must never be edited manually.

Repository enforcement begins with root `AGENTS.md` and `.kms-impact.yml`. `scripts/check_kms_impact.py` validates the declaration, changed paths, declared Markdown updates, and generated artifacts. It never rewrites documentation.

Choose A Reading Path

Open the [KMS Navigation Guide](KMS_NAVIGATION.md) to select a focused reading path by role or task. It covers human and AI onboarding, SaaS and subscriptions, operational modules, database work, Platform Owner tools, the public landing website, location data, design, architecture decisions, and review/KIA work.

For a fast baseline, read:

1. [AI Project Context](AI_PROJECT_CONTEXT.md)
2. [TIS Master Context](TIS_MASTER_CONTEXT.md)
3. [Project State](PROJECT_STATE.md)
4. [Documentation Update Policy](DOCUMENTATION_UPDATE_POLICY.md)

Then follow the relevant task path rather than loading the entire handbook.

Core Documents

1. [KMS Navigation Guide](KMS_NAVIGATION.md): role-based and task-based routes through the KMS.
1. [AI Project Context](AI_PROJECT_CONTEXT.md): compact onboarding file for future Codex and ChatGPT conversations.
1. [TIS Master Context](TIS_MASTER_CONTEXT.md): long-term product, architecture, SaaS, workflow, roadmap, and critical-rules source of truth.
1. [Project State](PROJECT_STATE.md): living status for branch strategy, priorities, milestones, known issues, and next work.
1. [Documentation Update Policy](DOCUMENTATION_UPDATE_POLICY.md): non-negotiable KMS and Knowledge Impact Assessment policy.
1. [Change History](CHANGE_HISTORY.md): chronological summary of meaningful changes.

Engineering Handbook

Use the [Engineering Handbook Index](engineering/README.md) for the full engineering document map.

Primary engineering references:

1. [TIS Module Map](engineering/TIS_MODULE_MAP.md)
1. [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)

- [User and System Flows](engineering/USER_AND_SYSTEM_FLOWS.md)
- [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)
- [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)
- [UI/UX Design Philosophy](engineering/UI_UX_DESIGN_PHILOSOPHY.md)
- [Product Roadmap](engineering/PRODUCT_ROADMAP.md)

Decision And History Documents

- [ADR Index](adr/README.md): Architecture Decision Records for major accepted decisions.
- [Rejected Decisions](engineering/REJECTED_DECISIONS.md): significant alternatives and why they were declined.
- [Module History Index](history/README.md): module-based history preserving deeper before/after context.

Current module history areas:

- [Subscriptions](history/subscriptions/README.md)
- [Landing Page](history/landing-page/README.md)
- [Academic Calendar](history/academic-calendar/README.md)
- [Workforce Planning](history/workforce-planning/README.md)
- [SaaS Onboarding](history/saas-onboarding/README.md)
- [Provisioning](history/provisioning/README.md)
- [Platform Knowledge](history/platform-knowledge/README.md)
- [Engineering Handbook](history/engineering-handbook/README.md)

Supporting Documents

- [Location Data Roadmap](location-data-roadmap.md): location data roadmap and related implementation notes.
- [Students + Talent & Potential Update Package Release Handoff (M1-M13)](releases/2026-09-23-students-talent-m1-m13-release-handoff.md): release-readiness closeout, migration/deployment plan, and post-deploy smoke matrix for the bounded Student ID/Learning Style/Al-Andalus/roster/Talent-cleanup/Evaluation-Progress/Results-and-Analytics package.
- [Landing Page Source of Truth](marketing/landing_page_source_of_truth.md): boundary between the public Next.js landing website and the FastAPI application portal.
- [Landing Page Master Content](marketing/tis_landing_page_master_content.md): approved marketing foundation and landing page content direction.

Generated Snapshot

Generated files:

- `static/docs/TIS_Project_Reference_Booklet.pdf`
- `static/docs/docs_manifest.json`

The PDF is generated from approved Markdown docs. It includes a handbook-use page, a page-numbered table of contents, source-document bookmarks, and child bookmarks for major headings. The manifest records the generated timestamp, documentation version, branch, commit SHA, included sources, source hashes, and each source

document's starting PDF page.

Platform owners can use the protected Knowledge Center at `/platform/knowledge` to search the manifest-backed library, filter by category, module, or freshness, and open a source at its recorded booklet page. The owner utility remains read-only and never links directly to the static PDF path.

KMS Synchronization

Validate KIA, regenerate the PDF and manifest, and verify freshness with:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Complete read-only validation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

The command reuses the dependency-light generator and strict impact checker. It must not require LaTeX, Playwright, Chromium, network calls, or system fonts.

Knowledge Impact Assessment

Every future implementation report must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed. If included source docs change, regenerate the PDF.

Update `.kms-impact.yml` during every task. CI validates it on pull requests and `dev`; deployment from `master` depends on the same KMS gate.

Phase Boundary

Phase 2A and Phase 2B establish KMS governance, ADRs, module history, AI context, and PDF/manifest generation.

KMS v3.0 Phase 3D completes the KMS v1.0 lifecycle foundation with self-evolving documentation workflow, KIA standard, dependency map, AI coding workflow, and future automation roadmap. The Platform Owner Knowledge Center is implemented as a read-only owner utility. Do not add a regenerate button or app-side Markdown rewriting unless explicitly approved.

TIS KMS Navigation Guide

docs/KMS_NAVIGATION.md

Use this guide to choose the smallest useful reading path for the work in front of you. Markdown under `docs/` remains authoritative; this file organizes that knowledge but does not replace module, architecture, decision, or history documents.

Start Here

For any engineering task, begin with:

1. [AI Project Context](AI_PROJECT_CONTEXT.md) for the compact system orientation and current guardrails.
2. [TIS Master Context](TIS_MASTER_CONTEXT.md) for durable product, architecture, SaaS, and workflow truth.
3. [Project State](PROJECT_STATE.md) for current priorities, completed milestones, known issues, and next work.
4. [Documentation Update Policy](DOCUMENTATION_UPDATE_POLICY.md) for KIA and completion requirements.
5. [Development Standards](engineering/DEVELOPMENT_STANDARDS.md) before changing implementation code.

Then select the task path below. Read relevant ADRs and module history before editing.

New Human Developer

Read in this order:

1. [Engineering Handbook Index](engineering/README.md)
2. [TIS Module Map](engineering/TIS_MODULE_MAP.md)
3. [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)
4. [User and System Flows](engineering/USER_AND_SYSTEM_FLOWS.md)
5. [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)
6. [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)
7. [Project Governance](engineering/PROJECT_GOVERNANCE.md)

New AI Coding Conversation

Read in this order:

1. [AI Project Context](AI_PROJECT_CONTEXT.md)
2. [AI Optimization Guide](engineering/AI_OPTIMIZATION_GUIDE.md)
3. [AI Coding Workflow](engineering/AI_CODING_WORKFLOW.md)
4. [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)
5. [TIS Module Map](engineering/TIS_MODULE_MAP.md)
6. Relevant [ADRs](adr/README.md) and [module history](history/README.md)

Also follow the repository-level instructions in root `AGENTS.md`.

SaaS Accounts And Onboarding

Read:

1 [User and System Flows](engineering/USER_AND_SYSTEM_FLOWS.md)

1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

1 [ADR 0002: Separate SaaS Identity and Operational Users](adr/0002-separate-saas-identity-and-operational-users.md)

1 [ADR 0005: Delayed Tenant Provisioning](adr/0005-delayed-tenant-provisioning-after-verified-payment.md)

1 [SaaS Onboarding History](history/saas-onboarding/README.md)

1 [Provisioning History](history/provisioning/README.md)

Guardrail: SaaS accounts, platform identities, and operational users are separate security and ownership concepts.

Subscriptions, Billing, And Paddle

Read:

1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)

1 [User and System Flows](engineering/USER_AND_SYSTEM_FLOWS.md)

1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

1 [ADR 0003: Paddle Payment Architecture](adr/0003-paddle-payment-architecture.md)

1 [ADR 0004: Webhook-Only Payment Confirmation](adr/0004-webhook-only-payment-confirmation.md)

1 [ADR 0005: Delayed Tenant Provisioning](adr/0005-delayed-tenant-provisioning-after-verified-payment.md)

1 [Subscriptions History](history/subscriptions/README.md)

Guardrail: provider-confirmed state remains authoritative for paid entitlements, payment completion, proration, invoices, and lifecycle reconciliation.

AI Entitlements

Read:

1 [ADR 0015: Centralized AI Entitlements And Usage Accounting](adr/0015-centralized-ai-entitlements-and-usage-accounting.md)

1 [AI Entitlements History](history/ai-entitlements/README.md)

1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)

1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

Guardrail: routes never manipulate AI usage directly; permissions, commercial state, registry policy, and successful-use consumption remain separate.

Operational Academic Modules

For teachers, subjects, sections, workforce planning, timetables, calendars, observations, and reports, read:

1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)

1 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)

1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

1 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)

1 [Workforce Planning History](history/workforce-planning/README.md)

1 [Academic Calendar History](history/academic-calendar/README.md)

Guardrail: preserve tenant isolation, branch scope, role permissions, and active academic-year context.

Database, Migrations, And Tenant Isolation

Read:

1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

1 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)

1 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)

1 [ADR 0002: Separate SaaS Identity and Operational Users](adr/0002-separate-saas-identity-and-operational-users.md)

1 [Project State](PROJECT_STATE.md)

Guardrail: do not modify models, migrations, tenant boundaries, or `tis.db` without explicit approval and focused validation.

Platform Owner And Knowledge Center

Read:

1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)

1 [Project Governance](engineering/PROJECT_GOVERNANCE.md)

1 [ADR 0006: Documentation as Source](adr/0006-documentation-as-source-knowledge-management-system.md)

1 [Documentation Automation](engineering/DOCUMENTATION_AUTOMATION.md)

1 [Knowledge Impact Assessment Standard](engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md)

1 [Knowledge Lifecycle](engineering/KNOWLEDGE_LIFECYCLE.md)

1 [Platform Knowledge History](history/platform-knowledge/README.md)

1 [Engineering Handbook History](history/engineering-handbook/README.md)

Guardrail: KMS Markdown is reviewed source material, the PDF is generated, and protected documentation must remain owner-only.

Public Landing Website

Read:

1 [Landing Page Source of Truth](marketing/landing_page_source_of_truth.md)

1 [Landing Page Master Content](marketing/tis_landing_page_master_content.md)

1 [ADR 0001: Separate Next.js Landing Website](adr/0001-separate-nextjs-landing-website.md)

1 [ADR 0007: Landing Page Visual System Strategy](adr/0007-landing-page-visual-system-strategy.md)

1 [UI/UX Design Philosophy](engineering/UI_UX_DESIGN_PHILOSOPHY.md)

1 [Landing Page History](history/landing-page/README.md)

Guardrail: the public website lives in `tis-landing-website/`; legacy FastAPI landing files are not its source of truth.

Location Data

Read:

1 [Location Data Roadmap](location-data-roadmap.md)

1 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)

1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

Guardrail: preserve runtime memory limits, local dataset boundaries, and record-level manual fallbacks.

UI And Visual Design

Read:

1 [UI/UX Design Philosophy](engineering/UI_UX_DESIGN_PHILOSOPHY.md)

1 [Visual Documentation Guide](engineering/VISUAL_DOCUMENTATION_GUIDE.md)

1 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)

1 the relevant module or landing-page path above

Architecture Decisions And Alternatives

Use:

1 [ADR Index](adr/README.md) for accepted architectural and product decisions.

1 [Rejected Decisions](engineering/REJECTED_DECISIONS.md) for significant alternatives that should not be reopened without new evidence.

1 [Documentation Dependency Map](engineering/DOCUMENTATION_DEPENDENCY_MAP.md) to determine which KMS records a change should propagate through.

Review, Release, And KIA

Read:

1 [Project State](PROJECT_STATE.md)

1 [Change History](CHANGE_HISTORY.md)

1 [Project Governance](engineering/PROJECT_GOVERNANCE.md)

1 [Self-Evolving Workflow](engineering/SELF_EVOLVING_WORKFLOW.md)

1 [Knowledge Impact Assessment Standard](engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md)

1 [Documentation Automation](engineering/DOCUMENTATION_AUTOMATION.md)

Use `python scripts/kms.py sync` after reviewed documentation changes and `python scripts/kms.py check` for complete read-only validation.

Document Types

1 **Core context:** durable truth, current state, policy, and chronological change summary.

1 **Engineering handbook:** modules, repository boundaries, flows, data architecture, standards, design, governance, and roadmap.

- **ADRs:** accepted architectural and product decisions.
- **Rejected decisions:** important alternatives and why they were declined.
- **Module history:** deeper before-and-after records for a specific area.
- **Marketing documents:** approved public positioning and landing implementation boundaries.
- **Generated artifacts:** the PDF snapshot and manifest; never edit them manually.

Using The Generated Booklet

The generated PDF includes a "How to Use This Handbook" page and a page-numbered table of contents. Open the PDF viewer's bookmarks or outlines panel to browse every source document and its major headings. Source starting pages are also recorded in `static/docs/docs_manifest.json`.

Markdown remains authoritative. PDF navigation is generated from the fixed source order and must be refreshed through `python scripts/kms.py sync` whenever included source documents change.

Navigation Integrity

Every Markdown link in this guide must:

- use a relative POSIX-style path,
- resolve to a Markdown file inside `docs/`,
- point to an existing authoritative document included in the booklet source list,
- avoid external URLs, root-relative paths, parent-directory escapes, and direct generated-artifact links.

`python scripts/kms.py check` enforces these rules. References to repository files outside `docs/`, such as root `AGENTS.md`, must remain code paths rather than clickable links in this guide.

TIS AI Project Context

docs/AI_PROJECT_CONTEXT.md

Talent Chart Types, Background Refresh And Progress Over Time (Production Follow-up Part 2, 2026-09-25)

Read before touching Talent charts or dashboard filters. Chart-type selectors may only switch materially different visualization types (`talent-charts.js modes()`); never add a mode that draws the same picture, a circular type for a time series, high-cardinality (more than eight) or ordinal data, or a percentage-only toggle. Values stay visible in every type. Dashboard filter changes must go through `refreshDashboard()` (background fetch, newest-token-wins, no root replacement, no `scrollIntoView`, no navigation); do not reintroduce a root-level loading replacement. Selected Comparisons is one bottom section. Progress Over Time is a kept, real per-Program longitudinal view over the ADR 0027 API with honest gaps. Presentation only; browser behavior is not verified.

Talent Branch Authority, Start Assessment And Classification Honesty (Production Follow-up Part 1, 2026-09-25)

Owner-directed amendment. Read before touching Talent Branch scope, Start Assessment or aggregate Classification copy.

1 **Branch model.** The sidebar Branch selector lists REAL Branches only (no Talent

"All Branches" entry; `branch_scope=all` cookie and `user.scope_all_branches` are removed and any old cookie is ignored, it grants nothing). For an organization- authorized actor the sidebar Branch is only the DEFAULT page-level Branch (`tp-config.branch`); the Talent Branch filter offers All Branches (client marker `branch_scope=all` in the URL, no `branch_id` sent) and every authorized Branch. For a Branch-limited actor the ceiling stays their authorized Branch (`tp-config.branchLocked`, enforced by `auth.get_accessible_branch_query`). `talent_branch_scope.talent_branch_ceiling` is now always `None`; an omitted `branch_id` means every Branch the actor is authorized for; an explicit `branch_id` is validated server-side. This supersedes the "hard ceiling" section below for organization-authorized actors only.

1 **Start Assessment.** The backend flow is sound (POST then workspace reads succeed).

The production banner "This context cannot be displayed" was the generic mapping of a stable, safe 400 code (chiefly `assessment_tool_unavailable`: the Student's Grade has no saved criteria in the Program). `talent-api-errors.js CODE_COPY` now carries fixed curated copy for the Start Assessment codes (never backend detail) and the reason is shown inline beside the row without scrolling.

1 **Classification "Unavailable".** Not a bug: each band cell is protected by the

approved Release 1 provider (minimum cohort 5, per band, zero counts included, then complementary suppression). The UI adds one uniform, value-free sentence. No threshold, suppression or privacy rule changed; making small cohorts visible requires an owner decision plus an ADR 0028/0044 amendment.

Agent 3 privacy and aggregate authority (2026-09-25)

Read the Agent 3 amendments in ADRs 0031 and 0044 before changing summaries. Individually authorized Assessment roster rows are NOT cohort-suppressed. Classification aggregates still use the existing privacy provider. A Classification-filtered Learning Style aggregate reflects that same population, but returns a protected, value-free state if the selected Classification bucket or denominator is protected in the original projection. Never ignore Classification, emit a protected denominator via another chart, or invent a frontend threshold. Eight styles plus Unassigned remain categorical.

Results & Analytics is aggregate-only. One backend projection owns shared Branch/Grade/Section/Program/Period/Classification filters and selected comparisons. Rubric results are Program/framework-bound. Chart switching changes presentation only. Keep per-view bundles, the hard Branch ceiling, request batching and repeatable snapshot reads. This is Web-only work, with no schema, migration, permission, local database or timetable worker changes.

The generic "Protected for privacy" badge stays removed (M18a). Non-visible cells read "Unavailable"; only the Classification-cohort explanation may use the words "protected for privacy". Visual work in Talent is structurally verified, not browser verified, unless a reviewer states otherwise.

Global Branch = Hard Talent Scope (Batch 1 Closure, 2026-09-25)

> Amended 2026-09-25 (Part 1 production follow-up): for organization-authorized actors the > sidebar Branch is only a default and All Branches is chosen inside Talent; the marker cookie is > removed. The ceiling below still holds for Branch-limited actors. See the section above.

Read before adding any Talent route or Branch filter. The global Branch is a hard ceiling, NOT the "default" described in the Batch 1 section below (that wording is superseded). Never accept an explicit `branch_id` in a Talent route without going through the ceiling: use `talent_branch_scope.visible_branch_ids(_or_none)` / `branch_scope_unrestricted` / `branch_within_ceiling` (or `resolve_access_context` for M10 org routes) instead of `auth.can_access_all_branches`, which alone would let an organization actor widen past the active Branch. An omitted Branch must resolve to the ceiling. The ceiling is lifted only by an explicit global All Branches (`user.scope_all_branches`, cookie `branch_scope=all`, set via `POST /scope/branch` with `branch_id=all`; other modules ignore it). Client: `config.branch` is the ceiling (`qs()` clamps `branch_id`; no All Branches option when set). Tests: `tests/test_talent_branch_hard_scope.py`.

Talent Current-Student Data Integrity, Branch Scope And Loading Cost (Deployment Acceptance Batch 1, 2026-09-24)

Read before changing any Talent population query, analytics label, Student deletion path or the Talent workspace Branch/loader code.

1 **Grain discipline.** "Students" means DISTINCT current Students in the

authorized scope. `frozen_eligible*` counts are participations (one row per Student per Cycle/Program) and are labelled "Program participations"; assessment/ result counts (completion, started, Classification, Talented, Period results) keep their own denominators and honest labels. Never label a membership or result count as Students. The 63-vs-9 defect was exactly this.

1 **Current-Student rule.** Compose `talent_current_students.current_student_exists`

into any new Talent population read (it is already in `population_query`, `frozen_membership_query`, Evaluation Progress reads and `list_assessments`). Student `status` is deliberately not a filter (ADR 0039).

1 **Deletion.** `force_delete_student_history` deletes `STUDENT_OWNED_MODELS` (ten

tables) in one transaction and retains nothing; a metadata test fails if a new Student-referencing table is not listed. Permission gates are unchanged.

1 **Branch scope.** The active global Branch is only the workspace DEFAULT

(`tp-config.branch`); every API still authorizes its own `branch_id` and a foreign or unauthorized Branch is rejected. `reconcileBranchScope` drops stale Branch/ Grade/Section from a URL minted under another active Branch.

1 **Per-row cost.** Inside `talent_read_batch.read_batch(db)` (list/aggregate

endpoints only) Student-independent configuration reads are memoized per request; never enter it in write paths and never cache across requests. Resolve permissions once per request with `talent_request_permissions.request_permission_checker`.

1 **Loader.** Talent assets are `?v=<content hash>`; an inline watchdog ends a

still-untouched server loader in an explicit error after 15 s.

Student Assessment Editor Redesign (Deployment Acceptance Correction D, 2026-09-24)

Read before touching `static/js/talent-operations.js` or `static/css/talent-experience.css` assessment rules. The Student Assessment editor is one competency per row (single-column `.tp-assessment-grid`), each with adaptive, full-width selectable rubric tiles (`tp-rubric-tile`: rank "1 / N", level title, readable descriptor on its own line) and a full-width Evidence textarea bound to its competency. The whole tile is the radio `<label>` target (native keyboard/focus, visible focus ring, check marker, non-colour-only selected state). Presentation only: scoring, automatic Classification (Exceptional-only Talented), authorization and persistence are unchanged and still come from the backend. Structural verification only; browser visual acceptance is still pending.

Learning Style Distribution Correction (Deployment Acceptance Correction C, 2026-09-24)

Read before touching `student_learning_style_analytics.py`, the Students Learning Style panel or the Results & Analytics Learning Style section. The distribution is authorized Student-domain aggregation, **not** Talent scoring output: it has no Talent P3/small-cell or complementary suppression and no privacy-provider dependency (`build_distribution(students)` takes no policy). Denominator = every authorized Student in the selection including Unassigned; nine categories in stable order (eight values then Unassigned) with key/label/count/percentage plus `total_population`; zero is `0/0%`; empty selection is state `empty`. There is no per-Student percentage and the four `learning_style_*_percentage` columns are operationally deprecated and excluded from current Learning Style analytics and normal product presentation (legacy compatibility paths may still read/write the stored values for preservation, never as analytics authority). Authorization is unchanged (`resolve_population`, `students.view`, SchoolGroup/Branch/Grade scope). Never extend this exception to Classification, Talented, competency or organization metrics - their suppression is unchanged. The Results frontend renders backend `levels` and never computes a percentage. See ADR 0031 "Acceptance C Amendment".

Talent Current Workflow Correction (Deployment Acceptance Correction B, 2026-09-24)

Read before touching Talent Student surfaces. The current workflow ends in the automatic backend Classification; Exceptional is the only Talented state. Review Candidate / Official Identification are legacy history: do not reintroduce them as navigation peers, roster columns/actions, filters, current Student status, or Talented determinants (use "Legacy Review & Identification History", `/talent/reviews`, only). Render Students through `TalentStudentIdentity` (name + Learning Style + Classification + Talented) - never re-derive a band client-side, never call a per-Student Learning Style or classification endpoint, and take Learning Style from the existing authorized projection (`context.student_learning_style`, eligible-students `learning_style`, learner-profile `student.learning_style`, Student Drill `learning_style`). Applicable current result = `completed` AND `is_current`. Web Service only; not deployed. Acceptance C (Learning Style aggregates) and D (assessment entry editor) have since been implemented (see their sections).

Follow-up ClinePass remediation (same day): the legacy-history projection claim was overbroad - `talent_review_workspace` did not actually return the current Classification until it was added from `assessment_classification(..., overall=)` (reusing the computed Overall Result). Learning Style absence

copy was also normalized to "Unassigned" in `templates/_learning_style.html` to match Talent surfaces.

Talent & Potential Runtime Loading Reliability (Deployment Acceptance Correction A, 2026-09-24)

Frontend reliability fix on `dev` (not deployed). Root cause: M16's per-view script gating omitted `talent-rubric-visual.js` on most views while `talent.js` dereferenced its global at evaluation time, so those views never initialized and kept the "Loading your authorized workspace" placeholder. Fixed by loading the module everywhere, resolving it defensively, and adding an outermost `boot()` guard, bounded requests (25 s / 15 s / 20 s deadline), and independent per-section loading, error and Retry states for Organization Overview and Results & Analytics. When changing Talent client code, keep each view's script dependencies in `tests/test_talent_ui.py` accurate and use the stub harness in `tests/talent_runtime_harness.cjs`. No backend, schema, permission, privacy or analytics-semantics change; Web Service only.

Follow-up ClinePass remediation (same day): the Acceptance A closeout overclaimed "no duplicate requests" and "sanitized errors" - at 507dc59 raw backend `data.detail` could reach users and the rubric-distribution endpoint was fetched by both `talent.js` and `talent-experience.js`. Fixed via shared curated error mapping (`static/js/talent-api-errors.js`) and shared rubric read ownership (`static/js/talent-rubric-request.js`); the runtime harness now executes both `talent.js` and `talent-experience.js` together to prove single-request deduplication.

M14-M18 Correction Program Closeout (M18b-3, 2026-09-24)

Read this first for Talent work: the M14-M18 correction program is functionally closed on `dev` (owner-acceptance/release planning only; not deployed or merged). Do not reintroduce: the four Learning Style percentage dimensions (Student Learning Style is one of eight categories plus Unassigned in aggregates); percentage-band or client-side classification (backend `talent_classification_service` only, only Exceptional is Talented); Review Candidate/Official Identification as current Talent indicators (legacy history only); averaging of Branch rates for Organization values (sum raw counts); visible "Protected for privacy" copy or any suppressed numeric value in HTML/JS/ARIA; per-request permission fan-out on the common Talent page (bound: 3 projections) or any cross-user/global permission cache. Current applicable result = `completed` AND `is_current`. Before changing Student Drill queries note the pre-existing fixed-budget test failure and that per-row `overall_program_result` is passed into `assessment_classification` (M18b-3). No AI provider, entitlement, or execution architecture changed.

Results & Analytics IA reorder + Classification filter + accessibility/responsive audit + candidate_membership_count decision (M18b-2b)

Bounded completion pass over M18b-2's own explicit open-item list. Before touching the Results & Analytics page again, read this:

1 The in-content page header (`lede()` inside the `analytics` view render)

now reads "Results & Analytics" with a dedicated subtext. The shared workspace chrome title (`VIEWS["analytics"]` = "Organization Overview") is deliberately unchanged - it is pinned by `tests/test_talent_organization_analytics_providers.py`.

1 The 9 required sections now render in order: header -> summary cards

(now includes one backend-sourced "Talented (Exceptional) Students" fact) -> Learning Style -> Classification -> Current Talent -> Competency Analysis (Program result + competency averages + rubric distributions, now grouped together) -> Results/Evaluation Progress (Grade + Evaluation Period progression) -> Branch/Organization comparison

-> navigation links.

1 A real, progressive `Classification` filter (`<select name="classification">`)

was added, restricted to the 5 backend `CLASSIFICATION_LABELS`, narrowing only the `/results-analytics/.../classification` request, hidden/cleared until a Program is selected. Evaluation Period/Competency filters were deliberately NOT added - they need new supporting list endpoints beyond this pass's scope. Learning Style's own filters (`branch_id`/ `grade_level`/`section_name`) are already fully covered by the existing Branch/Grade selectors; no new control was needed for it.

1 No chart-type selector exists anywhere in `static/js/talent.js` (bar-only

compatibility matrix) - confirmed directly, not added.

1 `candidate_membership_count`'s Results & Analytics summary-card label is

now `"Legacy: Meets Program Criteria"` (previously unqualified) so it can never be read as a current-Talent figure; the metric/data/permission are unchanged. `candidate_count` (Program Portfolio/Branch pages) is untouched.

1 Full broad M14-M18 regression/performance/KMS closeout remains M18b-3,

still pending.

Results & Analytics Frontend Rebuild - current-Talent indicator + new-family consumption (M18b-2)

Bounded FRONTEND-ONLY sub-phase consuming the M18b-1 backend contract. Before touching the Talent Results & Analytics page/rendering, read this:

1 `static/js/talent.js`'s `analytics` view now calls all three M18b-1

`/api/talent/results-analytics/...` families (Learning Style, Classification, Talented) and renders them with new reusable helpers (`bucketBars`/`bucketTable`/`distributionSection`/`talentedSection`) - the frontend never derives a band, classification, or Talented state itself; every value rendered is exactly what the backend returned.

1 The previous legacy `identifiedIndicator` (Official Identification /

`identified_of_eligible`) primary "how many Students are talented" indicator is REMOVED from this view and replaced by `talentedSection` (backed by the new M17-classification-derived Talented family). The underlying legacy Review Candidate/Official Identification services and workspace are unchanged; `identified_of_eligible`/`candidate_of_eligible` remain valid options only in the separate, still-supported legacy Branch-comparison metric selector - never in the current-Talent section.

1 `routers/talent_ui.py` additively surfaces `"students.view"` in

`talent_permissions` (presentation-gating only, no new authorization scope) so the new Learning Style section can be gated client-side on the same permission its backend route requires.

1 Competency and Progress/Evaluation-Period/Branch-comparison sections were

confirmed already wired to the correct pre-existing governed endpoints and were left unchanged - do not assume they were rebuilt here.

1 A full 9-section information-architecture reorder, progressive filter

bar, and explicit accessibility/responsive audit of the whole page was NOT completed in this task - it remains open follow-up work (M18b-2b or M18b-3). Do not assume the Results & Analytics page is in its final target shape.

Results & Analytics Backend Contract + Correct Aggregation Authority (M18b-1)

Bounded BACKEND-ONLY sub-phase of M18b (the visible Results & Analytics page/chart rebuild is the separate, still-pending M18b-2). Before touching Talent Results & Analytics backend, Talented aggregation, or the Learning Style/Classification API contract, read this section:

1 `talent_org_intelligence_contract.MetricCode` is STILL frozen - the new

current-classification/Talented aggregate grain is served by a new module, `talent_results_analytics_service.py`, reusing the existing M9 `talent_analytics_service` context/filter/scope architecture and generic `Cell`/`Group` privacy primitives, NOT by extending `MetricCode`.

1 One coherent router, `routers/talent_results_analytics.py`

(`/api/talent/results-analytics/...`), serves three families: Learning Style (thin reuse of `student_learning_style_analytics.py`), Classification (current M17 five-band distribution, reusing `talent_classification_service.assessment_classification` - never a duplicated band table), and Talented (Exceptional-only).

1 **Organization/Branch aggregation is always a raw-count sum, never an

average of Branch percentages** - `sum_raw_counts_across_branches` is the only rollup rule; the exact proof case (Branch A 1/2=50%, Branch B 9/90=10% -> Organization 10/92, not the naive 30% average) is regression-tested.

1 Competency and Evaluation Period/Overall Result analytics are NOT

reimplemented here - the pre-existing `/api/talent/analytics/.../ rubric-distribution` and `/api/talent/evaluation-progress/...` routes already serve those two families correctly and are consumed directly.

1 The dead `learning_style_dimension` query parameter (inert since M14) is

removed outright from `routers/talent_evaluation_progress.py`/ `talent_evaluation_progress_service.py`.

1 M18b-2 (the visible page/chart rebuild consuming this contract) is

separate, still-pending work - do not assume the Results & Analytics templates/charts were touched here.

Current Talent Authority Alignment + Learning Style Cleanup + Privacy UX (M18a)

Bounded first half of M18 (M18b, the full Results & Analytics page rebuild, remains separate and out of scope). Before touching current Talent authority, Learning Style API exposure, or Talent privacy copy, read this section:

1 **Applicable-current-result authority** (resolved from existing evidence,

not invented): a `TalentStudentAssessment` row governs a Student's current classification exactly when `status == 'completed'` AND `is_current == True` - the pre-existing ADR 0036 reassessment authority.

1 The Learner Profile and B9 Student Drill now expose the same M17

classification already on the Talent Assessment API - reuse `talent_classification_service.assessment_classification`, never duplicate it, never source current Talented state from `TalentReviewCandidate`/`TalentOfficialIdentification`.

1 Do NOT add a new org/Branch aggregate "Talented count" `MetricCode` -

`talent_org_intelligence_contract.MetricCode` is frozen at 14 values (ADR 0044). That is explicitly M18b (Results & Analytics rebuild) scope.

1 The four `learning_style_*_percentage` fields are fully removed from the

current normal Student API/UI projection (`routers/students.py`/ `routers/students_ui.py`); stored columns remain untouched. There is no current four-dimension Learning Style analytics authority anywhere.

1 The literal phrase "Protected for privacy" is removed from Talent &

Potential UI; the underlying suppression/privacy contract is unchanged - do not reintroduce the literal phrase, and do not weaken suppression when writing replacement copy.

Automatic Assessment Classification + New Normal Talent Workflow (M17)

A Completed Student Assessment is automatically classified by the backend into exactly one of five fixed, owner-approved bands (1.00-1.99 Needs Improvement, 2.00-2.99 Developing, 3.00-3.74 Meets Expectations, 3.75-4.49 Advanced, 4.50-5.00 Exceptional); only Exceptional is Talented. These bands are never a percentage conversion and never AI-derived. A manual Review Candidate/Official Identification step is no longer required for a Student to become Talented in the current normal workflow, though both remain fully preserved, independently governed legacy/history surfaces.

The classification input is a deterministic linear projection from each Talent Program's own configured 1..N rubric-average (ADR 0037's existing Overall Program Result, unchanged) onto the fixed 1.00-5.00 classification scale - not a system-wide five-level rubric mandate, because real Programs in this repository configure varying rubric level counts. See ADR 0037's 2026-09-23 Amendment and `talent_classification_service.py` before touching Talent classification, Talented-state, or Review/Identification code.

Talent Module Performance And Student Action Layout (M16)

Talent HTML authorization now resolves the effective permission projection once at the route gate and reuses it for the Talent action payload and shared shell. The independent commercial feature checks inside the shell remain authorized, so runtime instrumentation bounds a Talent page to at most three permission projection calls instead of the former per-action loop (roughly 28-31 calls on common views). Every HTTP request still performs authentication, permission, tenant, and Branch-scope enforcement.

`templates/talent/workspace.html` loads only surface-relevant operational bundles. All views retain the common `talent.js` and `talent-experience.js`; Programs/Evaluation Plans load Program tooling, Evaluation Plans alone loads its standalone scheduler, and Assessments/Review load operational tooling. Analytics and Overview no longer download those unrelated bundles. No SPA, framework, API, business-rule, privacy, or analytics-semantic change was made. Student list action groups now use a shared wrapping flex row so Add/Delete and row Open/Delete controls sit side-by-side when space permits and wrap without overlap on narrow screens.

Learning Style Correction + Aggregate Distribution (M14)

Owner-directed correction (see ADR 0031's/ADR 0042's/ADR 0044's M14 amendment sections and `docs/PROJECT_STATE.md`'s M14 entry for the full record). Two corrections: (1) Learning Style is ONE categorical selection per Student, extended from four to eight values (Visual, Auditory, Read/Write, Kinesthetic, Verbal, Non-verbal,

Quantitative, Spatial) - the four added values were previously, mistakenly, modeled as an independent four-dimension percentage profile; those four `learning_style_*_percentage` columns are now operationally deprecated (no longer written, displayed, or read as authority anywhere), with existing stored values preserved untouched and physical removal separately gated on a later data-occupancy verification. (2) "Percentage" means population/aggregate distribution, never a per-Student dimension score - the Students page's existing categorical distribution engine now covers all eight values plus "Unassigned" (always part of the denominator), and the M4/M10 Talent Evaluation Progress "Learning Style" Branch-comparison metric (which had averaged the four deprecated columns) is REMOVED (six approved Branch-comparison metrics remain, not seven - see the correction to the M10 section below).

Student Roster Import / Export Frontend (M11)

The Students list now exposes compact, independently permission-projected **Import Students** and **Export Students** actions backed exclusively by the existing M6 roster API. Export downloads the backend-produced `.xlsx` bytes and preserves its response filename; the browser never rebuilds roster data.

Import is a transient `.xlsx`-only dialog. It uploads the selected workbook to the stateless preview route, renders backend summary/row results and safe messages, then requires explicit confirmation before uploading the same file to the apply route. Apply re-parses and independently revalidates the workbook and remains create-only and atomic. The browser never parses Excel, submits a trusted valid-row list, performs partial apply, decides Student-ID uniqueness, resolves tenant/Placement identity, or infers conflict ownership.

Canonical `STD` plus ten-digit Student IDs remain textual with leading zeroes. Same-tenant identity appears only when supplied by the privacy-safe backend; cross-tenant conflicts stay generic. Canonical Section name remains import identity, and no client `section_display` formatter exists. There is no CSV or `.xls` support, persistent batch, background job, backend semantic change, schema change, or migration.

Student Integrity & Roster Round-Trip (M15)

A permanently force-deleted Student is removed from every Student-owned Talent table and cannot appear as a current/live Student in any Talent & Potential surface (assessment roster, Talent lists, Review, Learner Profile navigation, Results drill). The Students-vs-Talent count difference the owner observed is a population-scope distinction - the Branch-scoped current Students list versus frozen multi-Branch Talent membership plus the status-agnostic eligible-students roster - not a deletion defect. No schema or cascade change was required.

The Student roster export is round-trip safe. `section_display` is an accepted-but-ignored display-only import column (never identity; canonical `section` remains identity), and the canonical `STD` prefix is normalized back to the bare ten-digit value on import. Import stays create-only: an unchanged exported row classifies **NO_CHANGE**, a new row **CREATE**, and any mutated existing row stays a blocked `student_id_conflict` (never a silent update). Apply writes only CREATE rows, skips NO_CHANGE rows, and remains atomic. The export workbook gains a bold header, frozen header row, deterministic column widths, and autofilter. Permissions, tenant/Branch scope, Student-ID uniqueness, and audit rules are unchanged.

Results & Analytics Branch Comparison Frontend (M10, Students Product Sequence)

Organization Overview now contains one primary Program-scoped Branch comparison chart backed by the existing M4 `branch-comparison` endpoint. Its selector was originally limited to seven governed metric families: Evaluation Period Result, Overall Result, Assessment Completion, Assessments Started, permission-projected Meets Program Criteria and Officially Confirmed, and Learning Style (which used only the current Verbal, Non-verbal, Quantitative, and Spatial dimension parameter defined by the backend). **Current state (M14 correction):** the Learning Style metric is removed - it averaged the now-deprecated four percentage columns, which is semantically wrong under the M14-corrected model. Exactly six metric families remain selectable.

The chart preserves backend row and Period order and uses the existing privacy-safe Branch-label projection. Only a `visible` backend state may create a numeric bar; zero remains a real `0%`, while suppressed, restricted, coarsened, and no-data states stay categorical and disclose no hidden count or value. Framework mismatch produces an explanation rather than an Overall Result bar. The browser performs no Branch/Organization mean, metric conversion, Learning Style aggregation/normalization, privacy-threshold decision, suppression reconstruction, current-Placement regrouping, or cross-framework calculation. No Organization summary is added because the comparison contract does not expose one consistently across all seven metrics. M11 roster UI, analytics/backend semantics, schema, and migrations remain unchanged.

Student Evaluation Progress Frontend (M9, Students Product Sequence)

Student Profile's Talent tab now renders compact Evaluation Progress for each authorized Program and Academic Year from the existing M4 Student endpoint. The browser preserves the backend period order and configured labels, displays an explicit backend result percentage or textual unavailable state for every active/opened Evaluation Period, and distinguishes Pending from a real `0%`. It displays **Overall Result** only when the backend supplies a comparable numeric `current_overall_result`.

When the backend reports `framework_changed`, individual Evaluation Period results remain visible, the combined numeric Overall Result is absent, and the page explains the comparability boundary without treating historical evidence as corrupt. Frontend code does not select active Periods, read nominal weights, sort Periods, average results, decide framework compatibility, or reconstruct aggregate values. Existing frozen historical Grade/Section context and M5 `section_display` remain unchanged. This milestone adds no Branch/Organization progress UI, M10 comparison chart, M11 roster UI, backend semantic change, schema, or migration.

Talent Frontend Cleanup (M8, Students Product Sequence)

The normal Talent assessment frontend no longer exposes **Reload Saved Rubric** or **Educator Input**. The obsolete reload click flow and Educator Input fetch/create/amend/history handlers were removed from normal UI orchestration, and the Student Talent-profile presentation no longer requests or renders Educator Input. Existing assessment/rubric evidence and persisted Educator Input records, lineage, permissions, audit, services, and API compatibility remain unchanged and are not deleted or rewritten.

This bounded cleanup does not change deterministic result/KPI semantics, Review Candidate or Official Identification behavior, reassessment rules, privacy, frozen historical attribution, or M5 `section_display`. Although the four-dimension Student Learning Style profile may be informational context in a future Talent presentation, current authoritative roadmap material does not assign that presentation to this M8 cleanup, so it is not implemented here. M9 Evaluation Progress UI, M10 Branch analytics charts, and M11 roster UI remain out of scope. No schema or migration is introduced.

Students M7 Frontend

The existing FastAPI/Jinja and vanilla JS/CSS Students surfaces now consume the authoritative M2 managed Student-number and M3 four-dimension Learning Style contracts. New Student creation requires the ten business-facing digits while the UI presents a fixed, non-editable `STD` prefix; leading zeroes remain text. The Students list and profile show the canonical managed value, while legacy Students without one show a neutral unavailable state and remain editable. Authorized number assignment/replacement continues through `students.manage_identifiers`, and duplicate disclosure reuses the one privacy-safe backend conflict projection.

Create/edit/profile now present independent nullable Verbal, Non-verbal, Quantitative, and Spatial integer percentages. Blank remains null and `0` remains an assessed zero; values are never normalized or converted from the deprecated categorical field. Profile bars include visible values and ARIA semantics, including an explicit unavailable state. Existing current Placement, historical Placement, tenant/Branch/RBAC boundaries, and the M5 server-derived Al-Andalus `section_display` projection remain unchanged. M7 adds no schema, migration, API redesign, backend business rule, or roster import/export UI; the latter was out of scope for M7 and was implemented in M11 (see "Student Roster

Import / Export Frontend (M11)" above).

Talent & Potential M4 - Authoritative Evaluation Progress Analytics (Backend)

Per ADR 0044, M4 adds the backend-authoritative Evaluation Progress contract for Student, Branch, and Organization scope, plus the seven approved Branch comparison metrics and the Learning Style Branch aggregate, in `talent_evaluation_progress_service.py` and one new router, `routers/talent_evaluation_progress.py` (`/api/talent/evaluation-progress/...`), strictly inside one Talent Program + one Academic Year exactly like every M9/M10 module. The canonical Evaluation Period numerical input is ADR 0037's Overall Program Result `normalized_percent`; the active/opened predicate is confirmed (not new) as `TalentPlannedEvaluationPeriod.status == 'planned'` AND its linked `TalentAssessmentCycle.status IN ('open', 'closed')`, with nominal weight `1/A` for A active Periods, derived only and never persisted.

Two previously open design questions are now owner-ratified and recorded in ADR 0044: (1) Evaluation Periods combine into one Overall Result only when every contributing active Period shares one identical governed `framework_version_id` - a mixed-framework active set returns every Period individually with `comparability_state="not_comparable"` / `"framework_changed"` and a `null` combined Overall, never a cross-framework delta or equivalence inference; (2) an individually authorized Student's own Evaluation Progress (`GET /api/talent/evaluation-progress/programs/{id}/academic-years/{id} /students/{id}`, gated by the existing `talent_learner_profiles.view` permission plus frozen-historical-Branch scope) is governed by normal Student/Talent authorization, never aggregate cohort-size privacy suppression - it has no `policy` dependency at all, by construction.

`BranchPeriodResult`/`OrganizationPeriodResult` are the direct mean of valid governed Student `normalized_percent` results attributed to that scope via frozen `TalentAssessmentCyclePopulationMember.branch_id` attribution; `OrganizationPeriodResult` is computed directly from every Student in scope, never as an average of `BranchPeriodResult` values (proven by a dedicated unequal-Branch-size regression test). Pending/ unavailable results are excluded, never zero. Every Branch/Organization aggregate (and the Learning Style Branch aggregate) reuses only the M9 generic `Cell`/`Group`/`apply_primary_privacy`/ `run_complementary_suppression` primitives - never the M10 `talent_org_intelligence_contract.py` `MetricCode`/`CellIdentity` vocabulary, which remains frozen and unmodified. The bounded Branch comparison dispatcher (`branch_comparison_metric`) supports exactly seven metrics - Evaluation Period Result, Current/Overall Progress, Assessment Completion, Assessments Started, Meets Program Criteria, Officially Confirmed (the last four reusing the existing M9 breakdown-by-dimension providers unchanged), and Learning Style - and adds no `MetricCode` value.

No schema, migration, new permission, or frontend change. The existing client-side Review-Candidate average in `talent-operations.js` is unrelated to this new backend contract and is untouched; a future frontend milestone may adopt these new endpoints. Focused coverage is `tests/test_talent_evaluation_progress.py`; see `docs/PROJECT_STATE.md` for the full implementation-truth summary and ADR 0044 for the complete governance record, including both owner-ratified decisions and a disclosed non-blocking characteristic inherited from the reused M9 breakdown providers (an all-zero-cohort total may label as `suppressed` rather than `no_data` - no raw value is ever leaked either way).

Owner Video Acceptance Correction

Student Assessments now treats the configured Program card inside each Evaluation Period as the direct operational selector. If its internal Cycle does not exist yet, authorized UI orchestration creates and links it through the existing Cycle/Period contracts and remains in Student Assessments. Start Assessment opens the returned Assessment and that Assessment resolves its exact persisted Program/Framework/Grade criteria; no client-side rubric substitution or cross-Program inference is permitted. Action errors are surfaced visibly.

An immutable Framework may expose Delete Competency only as a versioned future change: clone a new Draft, remove there under the dedicated permission/revision contract, and leave historical Assessments unchanged. The actual numeric KPI configuration remains separate and has a visible expand/collapse disclosure.

Students cross-Branch browsing is controlled by `students.view_all_branches` plus organization/global scope. The managed-role policy keeps it Administrator-only. Learning Style aggregation remains ADR 0031 privacy-closed; missing privacy configuration may be explained to the user but never causes raw aggregate values to be rendered.

Talent Operational Recovery And UX Correction

Talent's application sidebar tree is now the only primary module navigation; the page-level duplicate navigation is removed. Program setup no longer uses a separate Ready step. Readiness is a derived Program-summary state and is positive only when Program/Grade, assessment criteria, and Evaluation Plan configuration are complete. Grade-first authoring displays one selected Grade at a time.

Completed evidence remains immutable. Administrator receives the new default permission `talent_assessments.reset_for_reassessment`, which may mark a Completed current Assessment non-current so the same visible Evaluation can be started again. This is an audited pointer/state recovery only: competency results, Candidate/Identification state, Educator Input, placement provenance, and all history remain intact. Physical delete remains the ADR 0034 zero-evidence exception.

Student Assessments visibly retains the selected Evaluation Period and Program and carries that context into Start Assessment, whose workspace resolves the exact saved Grade-scoped competencies and assessment criteria. Talent Review supports authorized Branch/Grade/Section filtering and does not show the internal deterministic "Meets criteria" outcome as a primary classification; the Candidate mechanism itself remains deterministic and separate from Official Identification.

Per ADR 0039, competency-owned rubrics remain rubric domain objects internally (`TalentRubric`/`TalentRubricLevel`, unchanged) but the user-facing Program-setup/assessment hierarchy is presented as **Competency -> KPI -> Level** ("Assessment Criteria"/"Rubric" retired as normal-user-facing text for this hierarchy). This is a presentation-layer rename only; `talent_program_service.upsert_rubric` still writes to the same `TalentRubric` table, and its `name` is now optional and auto-derived from the owning Competency's label when omitted. **Known open terminology collision, not yet resolved**: the separate, pre-existing numeric `TalentKpiConfiguration` feature is also presented as **Key Performance Indicator (KPI)** with its own Add/Edit/Delete controls (including a `remove-kpi` UI action) - the two features now share the "KPI" label in the UI even though they remain functionally and architecturally distinct. ADR 0039 explicitly does not authorize merging or renaming either feature to resolve this; a follow-up Owner naming decision is required. Analytics errors surface specific governed backend reasons when available instead of generic environment-unavailable wording.

Talent Corrective Batches 3 And 4

Talent rubric levels use one shared, order-derived visual system across Program setup, assessment entry, Talent Review, Student/Learner Profile, and analytics. Visual strength follows `display_order` for any labels or level count and is always paired with label plus position; an interactive selection also has check/border state. Rubric level, Meets Program Criteria, and Official Identification remain three independent facts.

Talent Overview is the operational landing page. Organization Overview owns selective executive KPIs, Branch/Grade distribution, optional Program rubric distribution, and Evaluation Period progression; Program Results and Branch Results own detailed cards, while Talent Map alone owns the privacy-safe Program matrix. Learning Style distribution remains a Students-domain view over the existing endpoint and governed suppression provider, with Planning-derived Organization/Branch/Grade/Section context.

Evaluation Plan Scope And Local Test Identity

Evaluation Plan and Evaluation Period mutations require both the matching canonical permission and durable organization/global access scope. A visible or selected Branch remains working context and does not reduce an organization-scoped manager's authority. Truly Branch-scoped users receive a read-only Evaluation Plan workspace and are not offered mutation controls.

The sanctioned Talent local-test seed defines `local_test_admin` as organization-scoped while assigning North Campus as its working Branch. If an older disposable `.local_test_data/talent_local_test.db` contains that identity with `BRANCH` access scope, it is stale and must be rebuilt with the guarded local-test reseed command; application authorization must not bypass or elevate it at runtime.

Talent Program Operational Mode And Evaluation Plan Terminology

An active Program whose annual configuration, active assessment setup, achievement descriptions, and Evaluation Periods are complete opens in Operational/Summary mode by default. It shows the Program identity, real Academic Year label, enablement, Grades, competency/assessment-criteria/Period counts, assessment setup status, and permission-gated Edit Program, Edit What we assess, Manage Evaluation Plan, Open Assessments, and View Results actions. Explicit setup hashes re-enter the relevant focused edit panel; there is no separate Ready panel. For a fully configured Draft Program, Finish Setup is the governed activation boundary: Program activation runs first, then the reviewed Draft Framework is activated with expected revision/fingerprint protection, and the same Program opens in operational summary. Already-active state is not redundantly activated.

The approved user-facing annual concept is **Evaluation Plan**, containing **Evaluation Periods**. Program setup keeps the Evaluation Plan embedded in its focused setup panel. Backend `EvaluationPlan` identifiers and routes remain unchanged. Branch-scoped payloads no longer advertise Plan management that the API's existing organization-authority gate would reject, and any such API error is translated into an action-specific access message rather than exposing scope-policy jargon.

Talent Evaluation Grouping And Legacy Reassessment Compatibility

Student Assessments presents Evaluation Period -> unique Programs -> Students. Repeated Evaluation labels across Programs share one user-facing section, with configured sequence used to order label groups when available; multiple physical Cycles for one Program/Period never duplicate that Program. Evaluation Plan data keeps a configured Program/Period visible even before the internal Cycle is materialized.

Re-evaluation remains append-only. In addition to the normal newer-Framework trigger, legacy data created before consistent assessed-Framework immutability is checked against persisted completed competency-result rubric/level bindings. If those bindings no longer match a complete competency-owned rubric now present on the same Framework, the Student is shown as Re-evaluation required. The old completed attempt becomes non-current and a fresh current in-progress attempt starts with zero results in the same visible Evaluation. Untouched legacy or incomplete current rubrics do not trigger this compatibility rule.

Normal Grade-level Competency authoring does not fabricate ASCII-only codes in the browser. Unicode names are submitted directly and the backend generates the existing unique Program-scoped internal code.

Talent & Potential now expands as a permission-aware child tree in the main application sidebar. Overview, Programs, Student Assessments, Talent Review, and Results & Analytics are clean top-level destinations: moving between them retains Academic Year only and drops stale Program/Cycle/Assessment and analytics filter context. Programs therefore reopens its collection view unless the user explicitly opens a Program, while Student Assessments owns its own Program selector instead of inheriting the Program previously being edited. Program-context buttons inside an opened Program intentionally keep that Program when opening Assessments or Results.

Talent Guided Configuration Owner Recheck

Program creation and editing stay on `/talent/programs` through four hash-backed panels: Program Basics, What we assess, Evaluation Schedule, and Ready. Program Basics combines logo, draft name/description, selected

Academic Year, annual enablement, and Planning-derived Grades in one panel and one Save & Continue action. What we assess has its own Competencies, Rubric Levels, Achievement Descriptions, and Review substeps; only the selected substep is rendered and add/edit forms remain collapsed until requested. Assessment setup history and creation of an editable successor setup are disclosed under advanced history rather than shown as the normal workflow.

The Program wizard embeds the existing Evaluation Schedule renderer and its permission-, revision-, population-preview-, and lifecycle-aware Plan/Period/Cycle calls. The separate Evaluation Schedule route remains available for later operations, but setup navigation no longer redirects there. Ready is a compact count/check summary whose action returns to the embedded schedule. The Students collection uses an explicit desktop table/mobile cards breakpoint, so both representations are never visible together; normal Active status is accessible but visually neutral.

Talent Program Setup Wizard Visual Acceptance

Program setup is hash-backed for its editable panels but no longer has a separate Ready step. The Program identity remains visible, What we assess and Evaluation Plan stay focused, and readiness is projected on the operational Program summary. Positive Ready appears only when all required Program/Grade, assessment-criteria, and Evaluation Plan prerequisites are complete.

The wizard continues to call the existing permission-gated, revision-aware Program, annual configuration, framework, rubric, descriptor, Evaluation Plan, Period, and Cycle contracts. Program removal is intentionally absent because the governed Program lifecycle is activate/retire; supported logo, draft setup item, schedule-period, and in-progress assessment-result removal actions retain their existing permission and lifecycle checks.

Talent Program Identity And Planning Context Completion

Talent Program Identity uses nullable Program logo metadata, the existing organization branding storage safeguards, `talent_programs.manage`, and organization-scoped upload/replace/remove routes. Compact badges use the uploaded asset or derived initials across Program, evaluation, assessment/review, profile, and results contexts; organization branding remains separate and no Branch override exists.

Talent Grade choices and direct annual-configuration validation use operational Current/New `PlanningSection` data. Longitudinal filtering cascades Academic Year -> authorized Branch -> configured Grade -> Planning Section and fails closed for empty or invalid Planning scope. In-progress competency results may be cleared through the existing `talent_assessments.manage`, expected-revision DELETE contract; completed/incomplete/insufficient-evidence assessments remain read-only.

Talent Owner Visual Acceptance Corrections

Following real Owner visual acceptance of the M11 workspace, four presentation defects the prior code-level trace missed were corrected. The shared `tp-filters` context selector (Academic Year/Program/Branch/Grade/Metric/Dimension) no longer requires a separate "Apply context" confirm click on any Talent page that reuses it (Programs, Evaluation Schedule, Organization Overview, and every other view sharing the one `templates/talent/workspace.html` filter form and `static/js/ talent.js` apply logic): a `change` listener on the form now calls the same `applyContext()` function the button's `submit` handler already used, debounced 250ms so rapid multi-dropdown changes collapse into one reload; the existing URL-query-parameter/`history.replaceState` context-threading mechanism is unchanged and unextended by a new storage mechanism. The visible button is relabeled "Refresh" (no longer styled as a required primary action) and remains only as an immediate, non-debounced manual fallback.

The Talent primary navigation previously rendered a full analytics-family link list (Organization Overview, Talent Map, Program Results, Students Across Programs, Progress Over Time) and then, on every analytics-family page, an additional sub-navigation duplicating most of the same links directly beneath it. The primary nav now collapses that family into exactly one "Results & Analytics" entry; the existing sticky sub-nav (`.tp-results-nav`) continues to list the

individual analytics pages, including Students, only while the user is actually inside that family - it retains genuine value there since six distinct analytics pages are more than the primary nav should enumerate.

Both corrections are presentation-only: no schema, migration, permission, backend authority, analytics computation, or `tis.db` change. See `docs/PROJECT_STATE.md` for the file-level detail and the accompanying Evaluation-Schedule/Programs-page implementation-truth summary.

Talent Program And Evaluation UX Simplification

The M11 operational workspace now presents Program setup as four user-facing steps: Basics, What we assess, Evaluation schedule, and Ready. The desktop Program index is a searchable compact table; the Create-Program form is not permanently expanded on that landing screen and instead opens as a collapsed panel via a "New Program" action, matching the Students "Add Student" separate-entry-point precedent. What we assess joins each competency to its rubric levels and achievement descriptions in one view and keeps Framework version history inside the existing edit path.

Evaluation scheduling accepts a free-text, user-defined evaluation name (for example Term 1, Audition, or Spring Review - never a fixed Baseline/Term 1/ Term 2/Final picklist). Per ADR 0035, the normal path no longer exposes or requires Draft/Open Cycle, frozen-population, synchronization, reconciliation, or an "Open Evaluation" step before assessment. An Evaluation Period is simply the context for Student Assessments. The UI opens Student Assessments directly; current effective Academic Placement supplies the eligible Students, and historical placement/framework context is captured when each Assessment starts. Legacy Cycle/Population structures remain internal compatibility/provenance implementation details.

This change adds no schema, migration, backend domain mutation, pricing, AI, analytics, timetable, Student UI, or `tis.db` change.

Student/Talent PostgreSQL Migration Prerequisite Repair

Render pre-deploy exposed a PostgreSQL ordering defect before application startup: baseline `metadata.create_all()` attempted `student_academic_placements` against legacy `branches` and `academic_years` tables that lacked explicit `(id, school_group_id)` unique keys. PostgreSQL therefore rejected the composite tenant-scoped foreign key. Branch was the first error; Academic Year would have been the next legacy-parent blocker.

The migration runner now defers all Student/Talent migration-owned tables to the ordered ledger. New additive migration `20260904_000_student_talent_parent_scope_prerequisites` runs before `001` on a fresh deployment and later on an already-migrated environment. It detects any equivalent unique key and otherwise adds the governed keys for `branches`, `academic_years`, and an existing `students` table. Duplicate preflight fails the transaction without rewriting rows or recording the migration. The M4 Cycle migration also omits its later M8 Period foreign key at initial creation; the existing M8 migration installs that exact key after the Period parent exists. All tenant-scoped foreign keys remain intact.

Dedicated PostgreSQL 16 validation applied the complete 63-entry ledger and nine-step Student/Talent chain from a pre-Talent baseline, verified every composite target key, reran with no pending work, and proved failed old-runner DDL rolls back without partial Student tables. No production database was read or changed; the Render failure state conclusion is based on the transactional runner and reproduced PostgreSQL behavior.

B11-E F1 Production Providers

The seven M10 Organization Intelligence routes now resolve production-capable, fail-closed providers. Privacy uses an externally configured, approved minimum cohort of 5 for every P1-P7 class and retains the existing primary-then-complementary suppression engine. Commercial availability uses the registered `feature.organization_intelligence` key through the canonical entitlement and feature-registry path; route permissions remain a separate check. Breadth uses externally configured ceilings of 1000 matrix cells, 1000

relationship results, and 1000 Program-pair results and rejects excess work without truncation.

Missing, malformed, mismatched, or exceptional configuration fails closed. A deterministic development path is enabled only when `auth.is_production_environment()` is false and `DATABASE_URL` resolves exactly to `.local_test_data/talent_local_test.db`; it cannot activate for `tis.db`, an in-memory database, another SQLite file, or any production environment name. No schema, migration, new permission, plan/pricing rule, production data, or HTTP response contract changed. F1 resolves only the provider implementation gate. As of 2026-09-09 (ADR 0028/ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED, and the system is not declared production ready.

Talent & Potential Phase C Results Experience

The M11 Talent Results & Analytics workspace now presents all seven approved M10 projections through a visual, plain-language experience. Organization Overview combines governed headline metrics, Program progress cards, non-ranked Branch entry cards, and a completion Talent Map preview. Program Results and Branch Results use visible backend percentages for progress bars and preserve independent Candidate and Official Identification permission projection. Talent Map supports Program-by-Branch and Program-by-Grade views with keyboard-focusable cells, explicit zero/no-data/privacy categories, and Branch drill-down.

Participation Overlap is presented to users as Students Across Programs, with an accessible symmetric matrix and a plain explanation of distinct shared participation. Student Drill is presented as Students, keeps frozen historical context, shows Candidate/Identification state only when returned, and links to the canonical Student Profile when the API supplies the capability hint. Longitudinal is presented as Progress Over Time, stays within one Program and Academic Year, and translates every governed comparability reason without calculating changes or making improvement claims.

All magnitude visuals read only already-visible backend percentages. Protected results never influence text, width, height, intensity, opacity, tooltip, focus help, or accessible names. The backend, privacy closure, permission rules, tenant isolation, schema, migrations, production providers, and `tis.db` are unchanged. Focused UI and all seven M10 route regressions pass; the isolated persistent local dataset returned HTTP 200 for each route and page. Browser visual acceptance remains pending because no browser surface was available.

Talent & Potential Phase B Operational Workflow

The M11 Talent workspace now operates the approved M2-M8 APIs instead of remaining a read-only stakeholder surface. Authorized users can create and configure Program-specific workspaces, eligible Grades, Framework versions, competencies, ordered rubric levels, achievement descriptors, optional bounded KPI settings, and deterministic Review Candidate rules. The Evaluation Plan view reuses M8 Plan/Period capabilities and M4 Cycle lifecycle to prepare and link an evaluation, freeze its Student population, and open real M5 assessment entry. Result saves remain one API request per competency and consume each returned Assessment revision before the next write; a stale response stops the sequence and preserves unsaved entries.

Assessment, Candidate, and frozen-population projections add authorized display context (Student, Program, Academic Year, Cycle, exact version, and frozen Branch/Grade/Section labels) without changing authorization or using current Placement. Review remains distinct from append-only Official Identification, and Educator Input remains a separately permissioned append-only lineage that does not alter results, candidate evaluation, or identification. The existing Framework read addition exposes stable Framework-competency and rubric-level row IDs required by M5 writes; the public configuration projection also exposes descriptor IDs for the existing delete route. These IDs are explicitly excluded from the internal semantic-fingerprint projection.

No schema, migration, new permission, commercial rule, AI behavior, analytics privacy behavior, or `tis.db` change was made. Real HTTP acceptance proves the qualitative Performing Arts path through Program, version/rubric/rules, Plan/Period/Cycle, frozen Student, assessment/evidence, stale conflict, review, one human identification decision, and Educator Input amendment. Browser acceptance remains pending because no browser connection was available. The

Candidate read API does not expose its stored rule-by-rule evaluation snapshot, so the UI truthfully shows outcome/context without inventing detailed reasons.

Student Profile History Read Endpoint (Partial Phase A Core Students Slice)

A delegated "Phase A - Core Students" slice added one small, additive backend read endpoint, `GET /api/students/{student_id}/audit`, exposing the existing append-only `StudentAudit` rows for the canonical Student Profile's future History section. No schema, migration, permission, or write behavior changed; see `docs/PROJECT_STATE.md` for full detail. The much larger requested UI scope (Students navigation area, list/add/edit, Academic Placement UI, the four-section Student Profile, and its read-only Talent & Potential summary) was NOT implemented in this task after a live concurrent-agent process was found mid-edit on the exact same new `routers/students_ui.py` file, already independently building that same surface; the remaining implementation was deferred pending reconciliation rather than risk a destructive overwrite race.

B11-E Integrated Production Qualification

ADR 0029 governs and implements a dedicated M10-only REPEATABLE READ session/ dependency for the seven Organization Intelligence routes only, strictly backend-conditional (PostgreSQL only), with no server-wide or other-route isolation change. All seven routes were live-re-tested under the permanent implementation and confirmed consistent, including the five routes B11-D had only structurally inferred. Suppression/reconstruction concurrency was tested live with the existing non-production `DeterministicSuppressionTestPolicy` (primary and complementary suppression); no suppressed value became reconstructable. Index Candidate B is finalized NO CHANGE; statistics freshness is finalized as operational/runbook guidance. F1 subsequently implements the approved production privacy, availability, and breadth providers with fail-closed configuration and entitlement behavior. This resolves the provider-code blocker only. The earlier independent review returned PASS WITH NON-BLOCKING OBSERVATIONS. **As of 2026-09-09 (ADR 0028, ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED. Production readiness is NOT achieved (no merge to `master` and no production deployment).**

B11-D PostgreSQL Concurrency Qualification Closure

B11-D is CLOSED after independent review returned PASS WITH NON-BLOCKING OBSERVATIONS. PostgreSQL 16.15 / READ COMMITTED qualification proved two same-request mixed-snapshot inconsistencies: Overview Candidate total changed 314 to 315, and Student Drill gate/page population changed 705 to 706. Other multi-statement projections remain structurally vulnerable; the Participation Overlap core matrix is partly safer because one statement drives the intersection, while surrounding response inputs remain multi-statement.

A non-permanent REPEATABLE READ experiment resolved both reproduced cases. Permanent M10 REPEATABLE READ was then PROPOSED - GOVERNANCE REQUIRED and NOT IMPLEMENTED. B11-E subsequently governed and implemented that boundary, which starts before the first database statement and span access/context resolution, aggregate and frozen population reads, Candidate/Identification and Student Drill reads, longitudinal inputs, and privacy/reconstruction inputs. B11-E subsequently measured suppression and reconstruction under concurrency with the non-production `DeterministicSuppressionTestPolicy`.

Tenant isolation remained clean. Candidate B now trends NO CHANGE; no index, schema, migration, permission, entitlement, privacy semantic, production threshold, breadth, SLO, or memory value was approved. B11-E implementation is PASSED and checkpointable. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

B11-C PostgreSQL Qualification Closure

B11-C is CLOSED after independent re-review returned PASS WITH NON-BLOCKING OBSERVATIONS. Real PostgreSQL 16.15 profiling covered all seven M10 Organization Intelligence routes. Query counts remained bounded across tested scales and no N+1 behavior was found. Index Candidate A is NO CHANGE; Candidate B requires more evidence, so no index, schema, or migration is approved. The durable evidence is `docs/history/engineering-handbook/2026-09-07-b11c-postgresql-profiling-evidence-remediation.md`.

The memory measurements are directional local Windows development evidence only. No production privacy threshold, breadth limit, commercial mapping, SLO, or memory value was selected, and no permission, privacy, or entitlement semantics changed. B11-D subsequently completed and is CLOSED as recorded above. B11-E implementation subsequently PASSED. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

B11 Production Qualification Governance

ADR 0028 approves the DECIDE-NOW B11 policy boundaries without implementing code or selecting production values. Privacy thresholds and breadth limits are external provider-owned configuration with no hard-coded or permissive fallback; missing/exception/reject outcomes retain existing fail-closed HTTP behavior. M10 remains plan-agnostic while commercial availability consumes the existing entitlement/feature-registry decision pattern. B11-C collects PostgreSQL performance and memory evidence before any numeric SLO, limit, or ceiling is approved. B11-D tests current READ COMMITTED under controlled concurrency before any stronger isolation or evidence-backed index ADR.

B11-C and B11-D subsequently completed and are CLOSED as recorded above. B11-E implementation subsequently PASSED. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING; B11 overall is CLOSED on that same basis; B12 is CLOSED. See ADR 0028 for the complete entry criteria, release gate, required evidence, and explicitly deferred Owner decisions.

Organization Intelligence M10 B0-B9

M10 B0 adds only an executable, non-functional conformance boundary in `talent_org_intelligence_contract.py` and `tests/test_talent_org_intelligence_contract.py`. It separates canonical analytical `CellIdentity` from value/privacy/presentation state, restricts `RelationshipTerm` coefficients to exactly `+1/-1`, represents only the approved additive `sum(coefficient * Cell) = constant` topology, preserves the exact M9 privacy-state family, rejects cross-tenant relationship/graph membership, and tracks every QA vector V01-V25 with explicit B0/B1/B2 ownership and implementation status.

B0 performs no graph solving or complementary graph closure, exposes no M10 route or aggregate, and introduces no privacy threshold, entitlement key, production breadth limit, formula language, Talent Score, Potential Rate, or cross-Program normalization. V01-V03 structural contracts are implemented; `talent_analytics_relationship_graph.py` now implements B1's tenant-bound canonical Cell registry, value-independent canonical Relationship identity, Cell-to-Relationship adjacency, structural validation, shared-Cell reuse, and deterministic bipartite connected-component discovery. V04, V20, and V22 are implemented by B1.

Pre-B2 hardening and the approved B5 contract extension close the three identity vocabularies with string-backed enums: metric additionally includes `programs_configured` and `active_programs`, alongside `frozen_eligible`, `completed`, `completion_coverage`, `assessment_started`, `started_coverage`, `required_period_execution`, `candidate_count`, `candidate_of_eligible`, `identified_count`, `identified_of_eligible`, `participation_overlap`, or `student_drill_population`; measure component is exactly `count`, `numerator`, or `denominator`; membership grain is exactly `frozen_membership`, `distinct_student`, `program_participation`, `program_configuration`, `review_candidate_membership`, `identification_membership`, or `period_execution`. `rate/percentage` are derived only after future privacy closure and are not Cell identity components. `RelationshipTerm` now requires

`type(coefficient) is int` plus value `+1`/`-1`, rejecting bool, float, `Fraction`, `Decimal`, strings, and other equality-compatible values.

`talent_analytics_privacy_closure.py` implements B2 without route exposure. It uses `fractions.Fraction` RREF to detect inconsistent systems and determine uniqueness per protected coordinate, including unique coordinates in otherwise underdetermined systems. Its explicit pipeline applies M9 primary privacy before component-local graph closure. Closure is monotonic, bounded by the initially eligible exact-visible Cells, and selects complementary victims using only structural rank/exposure, privacy class, depth, provider preference, and canonical identity. Analyzer failures and inconsistent components restrict the affected component; untrusted graph partitioning fails the whole operation. `no_data` excludes a non-authoritative equation rather than becoming zero, coarsened replacements never substitute for hidden raw coordinates, and restricted Cells remain opaque. Exact derived rates use `Fraction` only when all sources are exact-visible; an all-or-nothing payload projection prevents raw numerator/denominator/percentage/total/metadata/insight sibling leakage. V01-V25 are executable across B0-B2.

There is no schema change, migration, new permission, production privacy provider, UI, deployment, or `tis.db` change.

M10 B3/B4 adds only internal access-context and set-based query primitives in `talent_org_intelligence_service.py`. Its immutable context resolves active authentication, selected SchoolGroup, injected commercial availability, existing `talent_analytics.view`, tenant-bound Academic Year, canonical historical Branch scope, and independent Candidate/Identification/Student- drill/Learner-Profile permissions before analytical aggregation. No M10 permission or entitlement key is introduced. Production availability and matrix breadth providers are intentionally unconfigured/fail-closed; tests inject explicit versioned allow/reject providers.

The authorized Program universe uses enabled Program/Academic-Year configuration and, for Branch actors, Open/Closed frozen Cycle membership in authorized historical Branches. The population query filters only frozen `TalentAssessmentCyclePopulationMember` context and never joins current Student Placement. One grouped-query family supplies Program×Branch, Program×Grade, Program totals, Branch totals, and Organization totals with Branch authorization inside the population subquery before aggregation. Candidate and Identification functions return before SQL execution unless their independent view permission is present. M8 required execution remains Program-grain `executed`/`cancelled`/`outstanding`.

`PrivacyClosedProjectionSet` is the mandatory future serialization seam: it accepts only a B2 `PrivacyClosureResult`, carries no raw query rows, and delegates derived rates and payloads to B2 guards. Context fingerprints remain context evidence, never authorization, freshness, or ETag proof. The accepted `no_data` rule is unchanged: a relationship containing `no_data` is non-authoritative for exact reconstruction in that response; `no_data` is not zero. B5 adds the first bounded real M10 route, `GET /api/talent/organization-analytics/overview`. It counts configured annual Program rows and their exact `TalentProgram.status == active` subset, frozen eligible memberships, completion coverage, and required Period execution; Candidate and `decision == identified` metrics are omitted unless their own permissions are present. It constructs canonical Cells/Relationships, applies M9 primary privacy before B2 closure, and serializes only a `PrivacyClosedProjectionSet`.

B6 is implemented, independently reviewed, and closed after the targeted sparse-matrix re-review passed. It adds `GET /api/talent/organization-analytics/talent-map` for canonical Program-by-Branch and Program-by-Grade factual matrices. `branch_program` is only a post-closure presentation transpose. Supported metrics are the approved frozen/completed/started/Candidate/identified counts and eligible rates; Program-level `required_period_execution` is rejected. Backend row, column, and authorized-scope totals share canonical Cells and additive Relationships before M9 primary privacy and B2 closure. There is no ranking, universal score, KPI/rubric comparison, or B7+ route. The initial B6 review found a sparse-matrix reconstruction blocker: totals excluded `no_data` arithmetically while their Relationships included those coordinates, causing B2 to discard the equation. The builder now derives both each total and its Relationship from one identical authoritative-numeric child set. Empty totals remain `no_data`; a single populated child retains the valid `Total - Child = 0` dependency. Global B2 `no_data` semantics are unchanged.

B7 adds `GET /api/talent/organization-analytics/program-portfolio` and `GET /api/talent/organization-analytics/branches/{branch_id}`. Portfolio emits stable Program rows with common factual participation/coverage, permission- projected Candidate/identified counts, and Program/AY required-Period execution. Branch Intelligence is constrained to one authorized historical frozen Branch, shares B6 Program/Branch identities, and does not attribute execution to a Branch. Both use breadth, set-based queries, M9 primary privacy, B2 closure, and closed-only serialization. No ranking, score, or Grade breakdown is added.

B8 adds `GET /api/talent/organization-analytics/participation-overlap`. The closed contract adds only `participation_overlap`, mapped to `count` at `program_participation` grain with governed P2 privacy. One set-based query deduplicates frozen Open/Closed Cycle membership to `(program_id, student_id)` inside authorized historical Branch scope, then aggregates canonical symmetric Program pairs. The diagonal is the Program's distinct participant count. Pair Cells have no fabricated additive row/column relationships; the full matrix mirrors each already-closed canonical pair projection. Candidate and Identification overlap, row sums, ranking, Talent Breadth/scores, Student data, schema, migration, UI, AI, and B9+ remain absent. Cross-request differencing remains a production privacy-provider responsibility. The first independent review found one implementation-only pair-index defect: name/id-divergent Program presentation order could create `(B,A)` dictionary keys while lookup required `(A,B)`, yielding HTTP 500. One explicit numeric `canonical_pair_key` now governs construction, raw aggregate lookup, mirrored lookup, and canonical output. Targeted independent re-review passed; B8 is CLOSED. Its committed/pushed implementation remains part of current dev.

B9 adds `GET /api/talent/organization-analytics/students`, the first M10 route exposing Student-identifiable output. It requires BOTH `talent_analytics.view` AND `talent_analytics.view_students` (true AND composition, matching the M9 `analytics_students` precedent) - `context. student_drill_allowed` is checked explicitly before any identifiable query. Candidate (`talent_review_candidates.view`) and Official Identification (`talent_official_identifications.view`) fields remain independently permissioned and query-skipped (not merely response-filtered) without their own permission; the key is absent, never `null`/`false`. Learner Profile access (`talent_learner_profiles.view`) only exposes an advisory `can_view_learner_profile` capability hint - the real `/api/talent/learner-profiles/{student_id}` route still independently authorizes. Student inclusion is derived exclusively from frozen `TalentAssessmentCyclePopulationMember` context (Branch/Grade/Section) inside the requested Academic Year and the actor's authorized historical Branch scope, reusing the same `frozen_membership_query`/`resolve_filters`/ `authorized_program_universe` primitives every other M10 route uses; current `StudentAcademicPlacement` never decides inclusion or supplies displayed context. Every identifiable Student row is P7 regardless of narrow scope, page size, or a Candidate/Identification field's own P5/P6 class. Following independent review, B9 builds exactly one gate-level P7 `CellIdentity` using `student_drill_population/count/distinct_student`, fed by `COUNT(DISTINCT student_id)`, through the same `apply_primary_privacy_and_close`/`PrivacyClosedProjectionSet` pipeline every other M10 route uses - mirroring M9's `analytics_students` eligibility-gate pattern rather than Cell-wrapping every row - because B9 has no authoritative additive cross-Student relationship. `talent_org_student_drill.py`'s `StudentDrillClosedProjection` is the sole accepted serialization input; its constructor and `serialize_projection` raise `TypeError` for a raw SQL row, raw Student/Candidate/Identification ORM object, or a non-visible gate, mirroring B8's closed-wrapper discipline. Response grain is one top-level Student (id/name and permission-gated Learner Profile hint) with deterministic frozen Program/Cycle contexts containing Branch/Grade/Section, assessment state, completed KPI result, and permission-gated Candidate/Identification state; there is no Student Talent Score, ranking, AI field, or Educator Input. Pagination is over distinct Students: default limit 25, max 100, deterministic name/id ordering, `has_more`, and no `total_count`. Breadth is evaluated (`projection_family="student_drill"`) before any identifiable query. No direct Student-ID filter is exposed - the existing M10 filter resolver validates `program_ids`/`branch_id`/`grade_level` but has no governed Student-identity filter contract, so one was deliberately not invented rather than risking an existence oracle. B9 adds no schema, migration, permission, entitlement, longitudinal/B10, UI, or AI capability. The distinct-Student remediation passed targeted independent re-review; B9 is CLOSED. Its committed/pushed implementation remains part of current dev; B10 is not implemented.

B10-A Longitudinal Organization Intelligence Governance Decision (Approved Architecture)

Following an independent governance review that returned "approve with required amendments," ADR 0027 records the approved B10 "Longitudinal Organization Intelligence" architecture contract. This governance-closure task itself authorized no code, route, schema, migration, permission, or entitlement. B8 and B9 remain CLOSED; this decision does not reopen them. B10-B has since implemented this approved contract - see "B10-B Longitudinal Organization Intelligence (Implemented)" below and `docs/PROJECT_STATE.md` for the implementation-truth detail; the remainder of this section is the unmodified B10-A architecture record.

B10 MVP is bounded to one Talent Program, one Academic Year, aggregate/ non-identifiable output, and the ordered M8 `TalentPlannedEvaluationPeriod` slots within that Academic Year, exposed through exactly one future route, `GET /api/talent/organization-analytics/programs/{program_id}/longitudinal`, requiring `academic_year_id` and exactly one approved metric, with only `branch_id`/`grade_level`/`planning_section_id` as optional filters reusing existing M10 B3/B4 frozen-provenance semantics. B10 MVP explicitly excludes multi-Academic-Year chronology, matched-cohort analytics, Student growth analytics, Student-level longitudinal history, cross-Program performance normalization, ranking, and AI; the M7 Learner Profile remains the governed individual-Student historical surface.

`AcademicYear.year_name`/label is descriptive only and must never be parsed, numerically interpreted, lexically sorted, or treated as chronology authority; multi-Academic-Year longitudinal ordering is deferred pending a future, separately governed Academic Year chronology contract. Because B10 MVP is single-AY, authorization reuses the existing single-AY `resolve_access_context` for exactly the requested Academic Year; no new multi-AY access resolver is approved or required.

The approved time-point model corrects the original proposal's lean toward treating `TalentAssessmentCycle` as the ordering authority: one longitudinal point is one M8 `TalentPlannedEvaluationPeriod` slot, ordered only by its governed `sequence` - the Period is the presentation/order authority, and its optional linked Open/Closed Cycle supplies the factual evidence for that point. This is evidenced by the existing database constraint `uq_talent_assessment_cycles_period`, which enforces at most one Cycle per Period; `TalentAssessmentCycle.status` is exactly `draft`/`open`/`closed` and `TalentPlannedEvaluationPeriod.status` is exactly `planned`/`cancelled` (both verified in `models.py`). Point states are, using those exact values: a linked Open/Closed Cycle is authoritative; a missing linked Cycle is `no_data`/`missing_cycle`; a linked Draft Cycle is `no_data`/ `cycle_not_authoritative`; a cancelled Period is `no_data`/ `cancelled_period`; an authoritative point with no frozen population is `no_data`/`no_frozen_population`. An unlinked/ad-hoc Cycle (no governed M8 Period) is excluded from B10 MVP entirely, and a missing or non-authoritative annual Plan never fabricates series facts.

The approved metric allowlist is exactly nine of the fourteen existing `MetricCode` values (verified in `talent_org_intelligence_contract.py`): unconditional `frozen_eligible`, `completed`, `completion_coverage`, `assessment_started`, `started_coverage`; Candidate-permission-conditional (`talent_review_candidates.view`) `candidate_count`, `candidate_of_eligible`; Identification-permission-conditional (`talent_official_identifications. view`) `identified_count`, `identified_of_eligible`. Excluded, with reasons: `required_period_execution` (Plan-wide execution grain, not a Cycle- population point), `programs_configured`/`active_programs` (organization configuration headline metrics), `participation_overlap` (a cross-Program pair metric incompatible with single-Program B10), and `student_drill_population` (the P7 Student Drill disclosure gate, not a longitudinal metric). No new `MetricCode` is approved; the contract remains exactly 14 `MetricCode` / 3 `MeasureComponent` / 7 `MembershipGrain` values. Exactly one metric is selected per response - a multi-metric B10 query is not approved.

Adjacent Period-point pairs for the selected metric are evaluated as exactly `comparable` or `not_comparable`, with approved reason codes `missing_cycle`, `cycle_not_authoritative`, `cancelled_period`, `no_frozen_population`, `metric_unavailable`, `framework_changed`, and `privacy_protected`.

"Comparable" means only that two cross-sectional factual points may be neutrally shown side by side - never same-cohort, growth, improvement, decline, or progress. Framework sensitivity is metric-specific: the five unconditional metrics remain comparable across a Framework-version change, while Candidate/Identification metrics become `not_comparable`/ `framework_changed` when adjacent points use different Framework versions; no KPI/rubric normalization or universal performance scale is approved.

B10 MVP returns no server-computed longitudinal arithmetic between points - no `delta`, `change`, `percent_change`, or `current = previous + delta` relationship - because an exact derived difference could reconstruct a protected endpoint; only individually privacy-closed points plus safe comparability metadata are returned. Future UI may use neutral factual language ("higher/lower than previous Period," "period-to-period difference," "side-by-side result") only when both points are already safely visible and comparable, never `growth`/`improvement`/`decline`/ `progress` language, since no matched-cohort authority exists.

The privacy architecture reuses the existing M9/M10 P1-P7 classes with no new privacy class: authorization -> frozen scope -> canonical Cell -> primary privacy -> B2 closure -> strict closed wrapper -> serializer, with no arithmetic from raw pre-closure values and no fabricated cross-time additive Relationship (Period populations are not presumed disjoint, and no `annual total = sum of Period populations` identity is asserted). Rate metrics derive each Period point's percentage only from that Period's own compatible numerator/denominator after closure, never by averaging percentages across Periods. Candidate/Identification metrics remain query-skipped (not response-filtered) before any analytical SQL when their respective permission is absent, matching the existing M10 discipline.

Deferred capabilities: multi-Academic-Year chronology and longitudinal series, canonical Academic Year ordering, a multi-AY authorization resolver, matched Student cohorts, retained-Student growth, Student movement decomposition, per-Student organization longitudinal analytics, KPI/rubric normalization across Framework versions, cross-Program performance normalization, server deltas, and AI interpretation. Open production gates: production privacy provider/threshold, commercial availability mapping, production breadth configuration, PostgreSQL consistent-snapshot and performance/concurrency qualification, future Academic Year chronology governance, and final M10 security/release qualification. See `docs/adr/0027-b10-longitudinal-organization-intelligence-contract.md` for the complete recorded decision.

B10-B Longitudinal Organization Intelligence (Implemented)

B10-B implements the ADR 0027 contract exactly as governed above, in `talent_org_longitudinal.py` plus one new route in `routers/talent_organization_analytics.py`: the seventh Organization Intelligence route, `GET /api/talent/organization-analytics/programs/ {program_id}/longitudinal`. It reuses `resolve_access_context/ authorized_program_universe/resolve_filters/frozen_membership_query` unchanged, fetches the single governed M8 Plan and its Period+linked-Cycle series in one bounded query, and aggregates population/completed/started/ Candidate/identified counts grouped by Cycle id in one bounded query each - never one query per Period. Every Cell/component is built with `relationships=()` (no same-point or cross-time additive Relationship anywhere in the module), so no `delta`/`change`/`percent_change` field is ever computed; rates are derived only through the unchanged `derive_exact_rate`/`project_safe_derived_payload` helpers. Comparability follows the ADR's exact precedence (a missing/cancelled/no-population reason, then `privacy_protected`, then `framework_changed`, then `comparable`) with an explicit regression test proving a missing/non- authoritative reason overrides `framework_changed` even when the Framework versions also differ. Candidate/Identification SQL is skipped entirely unless the selected metric requires it AND the actor holds the matching secondary permission - requesting an unrelated metric never triggers that SQL merely because the actor holds the permission. Breadth (`projection_family="program_longitudinal"`) is enforced before the aggregate queries run. See `docs/PROJECT_STATE.md` for the full implementation-truth summary and `tests/test_talent_organization_longitudinal.py` for the complete test matrix. Independent security/privacy review passed with non-blocking observations; B10 is CLOSED. B11, B12 (closeout), and every ADR 0027 "Deferred capabilities"/"Production gates" item remain open.

B11-A/B11-B Organization Analytics Observability

B11-A (read-only qualification, completed) confirmed that the privacy, commercial-availability, and breadth-policy provider seams used by all 7 M10 Organization Intelligence routes exist with no production implementation and correctly fail closed today. B11-B adds safe, non-sensitive operational observability around all 7 existing routes (`overview`, `talent-map`, `program-portfolio`, `branches/{branch_id}`, `participation-overlap`, `students`, `programs/{program_id}/longitudinal`) through a new `talent_organization_analytics_observability.py` module (a dedicated, non-exported stdlib logger, `tis.talent.organization_analytics .observability`, deliberately kept separate from the immutable business/ security audit trail in `audit.py`) plus telemetry wiring in `routers/talent_organization_analytics.py`. Recorded signals are bounded and allowlisted only: `projection_family`, `outcome` (`success`/`rejected`/`unavailable`/`failed`), route latency, safe structural counts already computed for each route's own `OrganizationAnalyticsBreadthPolicy.allows(...)` call (`program_count`/`row_count`/`column_count`/`prospective_cell_count`/ `prospective_pair_count`/`relationship_estimate`, plus route-specific `emitted_pair_count`, `period_count`/`component_count`/ `authoritative_cycle_count`/`comparison_count`, and Student Drill's `page_limit`/`page_returned_count`/`has_more`), and each governed provider's coarse outcome (`provider_missing`/`available`/`unavailable`/ `provider_exception` for availability; `provider_missing`/`evaluated`/ `provider_exception` for privacy; `policy_missing`/`allowed`/`rejected`/ `policy_exception` for breadth) - never a Student identifier, raw analytical value, privacy threshold, suppressed value, or Candidate/ Identification decision, and never `delta`/`change`/`percent_change`/ `total_count`. Every existing fail-closed HTTP outcome (provider missing/ false/exception, breadth reject/exception, privacy provider missing/ exception) is unchanged byte-for-byte; observability only observes an existing decision after the fact and never becomes a new oracle or alters a response body. A telemetry emission failure is swallowed internally and cannot affect the HTTP response or weaken a fail-closed decision. Query-count instrumentation is explicitly deferred to a future "B11-C profiling" phase - no new SQLAlchemy event-listener instrumentation was added outside existing test-only usage. B11-B is implemented and unit- tested (`tests/test_talent_organization_observability.py`, 22 tests, plus the full pre-existing B5-B10 regression suites green). Independent security/ privacy review passed with non-blocking observations; B11-B is CLOSED. No schema, migration, permission, entitlement, privacy threshold, or production breadth limit was added; PostgreSQL performance/concurrency/ memory qualification and every ADR 0027 production gate remain open; B11 overall (including any frontend work) is not CLOSED and B12 remains unimplemented.

Deterministic Talent Analytics (M9, Committed/Pushed On dev)

M9 adds a read-only, aggregate-only `/api/talent/analytics/programs/{program_id}/academic-years/{academic_year_id}/...` API family (`context`, `overview`, `rubric-distribution`, `kpi-distribution`, `competencies`, `breakdowns/{branch|grade|section}`, `period-comparison`, `students`) composed strictly inside one Program + one Academic Year. It never reinterprets current Student Placement for historical scope (frozen `TalentAssessmentCyclePopulationMember` fields only), never re-runs M6 Review Candidate policy evaluation, and never recomputes M5 KPI. There is no Talent Score/Index/Potential Rate, no cross-Program analytics, no ranking field, no matched-cohort growth, and no KPI mean/median/percentile/numeric bins - `kpi-distribution` explicitly reports `distribution_mode: unavailable` / `reason_code: numeric_binning_rule_unapproved` rather than inventing a bin rule. `talent_analytics.view` and `talent_analytics.view_students` are new Administrator-only-by-default permissions; analytics permission never implies access to the raw Plan/Cycle/Assessment/Candidate/Identification/Learner- Profile APIs, which remain independently gated. Candidate and Identification analytics are query-skipped (not merely response-filtered) without their own existing view permission, and Student-level drill additionally requires `talent_analytics.view_students` composed with `talent_analytics.view` via true AND composition.

Every privacy-sensitive metric is a `Cell` (identity-only, value-independent `key`, `privacy_class` P1-P7, `raw_value`, `state`, `value`) and a single-dimension breakdown is a `Group` (one parent-total = sum(children) reconstruction relationship). The canonical pipeline order is: authorized raw aggregate query -> metric construction -> `apply_primary_privacy` (invokes the injected `TalentAnalyticsPrivacyPolicy.evaluate_cell` per cell) -> `run_complementary_suppression` (reconstruction-breaking fixed point, up to 8 passes, identity/value-independent tie-break, fails to `restricted` on non-convergence) -> comparability -> privacy-safe insights -> serialization. Every one of the eight routes calls `apply_primary_privacy` on every privacy-sensitive cell/group before any `run_complementary_suppression` call in the same function; a structural regression test statically asserts this ordering for every `run_complementary_suppression` call site in `routers/talent_analytics.py` so a future omission fails CI rather than silently publishing an unevaluated `state=visible` cell. That ordering guard proves call-site sequence only, not that every independently disclosed derived quantity was itself privacy-evaluated - a second, distinct defect of the same reconstruction-risk class was found and fixed in a later pass (see below).

A route that privacy-evaluates only one headline/total `Cell` (for example `frozen_eligible`) and then serializes an entire raw per-status bundle computed independently of that decision is the same bug class as the call-ordering BLOCKER above, just relocated past the ordering check: the per-status counts, the derived `assessment_started`, and the derived percentages were never themselves evaluated or protected by complementary suppression, so a suppressed/coarsened/restricted status could still be reconstructed from its visible siblings or read directly off the raw bundle. This was found in `/rubric-distribution`'s and `/period-comparison`'s coverage side-bundles (both called `svc.coverage_metrics()` on the raw `svc.raw_coverage_counts()` dict once a single total `Cell` passed privacy) and fixed by extracting the already-correct `/overview` coverage pattern into one shared authority, `talent_analytics_service.build_privacy_safe_coverage_bundle`: it builds one `Group` of `frozen_eligible` + the 5 per-status `Cells`, runs `apply_primary_privacy` then `run_complementary_suppression` over the whole group, and only returns the full `coverage_metrics()` shape (with derived `assessment_started`/percentages) when EVERY cell in the group is independently `visible`; any other outcome collapses to a compact `{"state": ...}` (or `{"state": "coarsened", "value": ...}`) projection with no raw sibling field ever present. `/rubric-distribution` and `/period-comparison` now call this shared helper instead of `svc.coverage_metrics()` directly; `/overview` already implemented the identical all-or-nothing pattern independently and is unchanged. A recursive, runtime regression test walks the fully serialized response bodies of all three routes under a suppressive test policy and asserts no dict anywhere carries a non-visible privacy state next to a raw coverage-shaped sibling field, which is a stronger, empirical proof than the call-ordering AST guard alone. "Every privacy-sensitive metric is a `Cell`" (above) is accurate only as of this fix; before it, the coverage side-bundles briefly serialized raw dict values that were never wrapped in a `Cell` at all.

`coarsened` is implemented exactly per Product Owner decision: it is valid ONLY when the injected policy's own `PrivacyDecision` supplies an explicit safe replacement value alongside `state=coarsened`; `apply_primary_privacy` fails closed to `suppressed` if a policy requests `coarsened` with no replacement, so a `coarsened+null` cell can never be serialized as if coarsening succeeded. `run_complementary_suppression`'s own reconstruction-breaking step always demotes its tie-broken victim to `suppressed`, never `coarsened` - it has no channel to obtain a policy- supplied replacement for an arbitrary victim, so it never fabricates one. There is no generic bucket-merge/coarsening algorithm anywhere in this module. Production `TalentAnalyticsPrivacyPolicy` construction is configuration-driven: `resolve_privacy_policy_provider()` delegates to `talent_organization_analytics_providers.build_privacy_provider()`. The provider is created only when the deployment environment matches the approved Release 1 configuration; otherwise routes fail closed (`analytics_query_failed`, no threshold value in the response). Test policy overrides remain test-only. This is an intentional open gate, not a defect.

The `competency_id` filter is validated against both tenant scope and the resolved `framework_version_id` (must be a `FrameworkCompetency` on that exact Framework Version) and narrows `/competencies` output; a foreign or wrong-Framework `competency_id` is rejected as `invalid_filter`, matching the existing non-enumerating filter-rejection pattern. `request_context_fingerprint` is a request-parameter/permission-projection/policy-version hash - explicitly not a data-freshness or ETag proof (confirmed by test: identical requests against mutated underlying

rows still produce an identical fingerprint).

Period comparison implements `framework_changed`, `missing_cycle`, `no_frozen_population`, `no_completed_result`, and `metric_unavailable` reason codes, each independently tested. `candidate_policy_changed` exists in code but is dead/unreachable under the current R1 data model: it only re-evaluates when both sides already share an identical `framework_version_id` (a precondition of reaching `comparable` at all), and a Framework Version's Review Candidate Policy is immutable once Active (M3), so two same-framework Cycles always see an identical policy-enablement result. This is documented rather than resolved by inventing a new code path. `incompatible_kpi_semantics` and `no_valid_kpi_result` from the original aspirational vocabulary do not exist anywhere in the implementation and are not exposed in any response.

M9 is committed and pushed on `dev` at `23ade9a7c6166197140b48a3edbfac849396d580` (commit `feat: add deterministic talent analytics`); it is not deployed, not production-released, and not merged to `master`. At the M9 checkpoint no production privacy policy was approved; F1 subsequently approves and implements the cohort-5 provider. PostgreSQL execution/performance had not been validated at M9 (open gate, consistent with every prior milestone) - only SQLite-backed pytest coverage exists, including a query-count regression baseline proving no route loops per-row/per-dimension. Focused coverage is in `tests/test_talent_analytics.py`. UI, export, AI, Learning Style, and M10 Talent Map execution remain out of scope and unimplemented; only B0-B5 including the bounded Overview described above exists.

Annual Evaluation Plans And Periods (M8, Complete)

M8 adds `TalentAnnualEvaluationPlan` as the one annual planning authority for an exact `TalentProgramAcademicYearConfiguration`, and ordered `TalentPlannedEvaluationPeriod` identities beneath it. Plans are SchoolGroup/Program/AcademicYear scoped and move one-way `draft -> active -> closed`; Periods move `planned -> cancelled`. Period identity uses persisted NFKC/case-folded/whitespace-normalized label and optional short-code keys, dense positive sequence, bounded dates/notes, and database uniqueness. Draft and Active planning edits use expected Plan revision. Linked Periods are historical anchors; only the tail after the last linked Period may reorder.

A Cycle may carry one nullable `planned_evaluation_period_id`; pre-M8/ad-hoc Cycles remain NULL and behave exactly as before. Link/unlink is Draft-Cycle only, requires BOTH Plan-manage and Cycle-manage plus organization/global scope and both expected revisions, increments both aggregates, and follows Plan -> Period -> Cycle lock order. New links require an Active Plan and Planned Period. Cycle Open still requires only the established M4 govern permission, but a linked Cycle revalidates locked Active/Planned/same-Program/ same-year context before population freeze. A Closed Plan permits only the approved optional-Period + Draft-Cycle unlink exception.

Activation requires a non-empty valid Period set and enabled annual config. Cancellation requires governance, an Active Plan, no linked Cycle, and a bounded reason. Closure resolves required Periods as executed only through a Closed linked Cycle or cancelled only with no Cycle; optional unresolved Periods do not block. Rollover copies stable labels/order/required flags/short codes into a new Draft in a different enabled annual config, resetting dates, notes, cancellation, Framework, Cycle, and execution context.

Administrator-only defaults are `talent_evaluation_plans.view/manage/govern`. All mutations require organization/global scope; Branch actors are read-only. Cycle projection is present only with separate Cycle-view permission and is bounded to id/title/status/revision/Framework Version. Without it, no cycle key, placeholder, cycle-derived warning, relationship action, or execution detail is emitted. Plan/Period APIs contain no Student or analytics data. Advisory warnings are recomputed (`period_window_overlap`, `cycle_outside_planned_window`, `chronological_inconsistency`). M9 Deterministic Talent Analytics is now implemented (see above); M10, Learning Style, AI, and Student evidence remain out of scope and unimplemented.

Longitudinal Learner Profile And Placement Read Scope (M7, Complete)

M7 adds a read-only `/api/talent/learner-profiles/{student_id}` aggregation, not a new persistence authority. It returns current canonical Student identity and only independently authorized historical records: Academic Placement intervals, Program/Academic-Year/Cycle/Framework-version grouped Assessments and exact Competency Results, optional persisted KPI provenance, Review Candidate history, and Official Identification history. It never turns candidate and identification records into one status, recomputes KPI, or compares Frameworks/Programs as a universal score. The optional timeline is a bounded user-facing summary with deterministic timestamp/type/id ordering, not an audit-log export.

M7 adds Administrator-only-by-default `talent_learner_profiles.view`, separate from every Student, Assessment, Program, Candidate, and Identification key. It authorizes only base longitudinal evidence: Candidate, Official Identification, and Educator Input each additionally require their own `.view` permission. Branch-scoped profile reads filter each Placement by its stored historical Branch and every Talent record by frozen/persisted historical Branch; a profile with no visible historical record is non-enumerating not-found. Organization/ global scope can read same-tenant history. Current Student lifecycle metadata appears only after that historical-record access check and never grants access. Without the relevant secondary permission, that entire sensitive data class and its timeline events are omitted with no object, id, status, count, timestamp, or placeholder. Authorized readers still receive only records in their persisted historical Branch scope. Ordinary profile reads do not write audit events.

The same approved policy reconciles M1 direct placement reads: Branch-scoped `GET /api/students/{student_id}/placements`, `/effective`, and `/{placement_id}` now authorize each stored historical `branch_id`; unauthorized individual or effective rows return the existing non-enumerating 404 shape. No current Placement fallback exists. M7 adds no Annual Evaluation Plan/Periods, Learning Style, analytics/Talent Map, AI, Development and Support, export, notification, or profile materialization. M8 Annual Evaluation Plan & Periods remains deferred.

Talent Review, Official Identification & Educator Input (M6, Complete)

M6 implements the full "Review, Official Identification & Educator Input" milestone following 18 approved Product Owner governance decisions (see "M6 Governance Review: Decisions" below) that resolved the 9 previously open questions this section used to document as deferred.

Review Candidate evaluation/materialization (unchanged from the original partial slice): `talent_review_candidate_service.py` evaluates the exact M3 `TalentReviewCandidatePolicy`/`TalentReviewCandidateRule` attached to a Completed Assessment's exact Framework Version, using `rubric_level_at_or_above` (compared by `TalentRubricLevel.display_order`, never `numeric_value`) and `kpi_at_or_above` (compared against the Assessment's persisted `kpi_result`, never recomputed), with `all`/`any` composition. No policy attached means no candidate is ever inferred. Evaluation only reads `TalentStudentAssessment`, `TalentStudentCompetencyResult`, and `TalentAssessmentCyclePopulationMember`; it never mutates them. A qualifying evaluation persists one `TalentReviewCandidate` row per Assessment (`uq_talent_review_candidates_assessment`), starting `status="pending_review"`, with policy identity, match mode, a deterministic SHA-256 evaluation fingerprint, and a full rule-by-rule evaluation snapshot. Re-evaluating an already-materialized Assessment is idempotent.

Decision 1 - non-qualifying evaluation: still creates no `TalentReviewCandidate` row, but now writes one `TalentAssessmentAudit` row per evaluation call (`resource_type="review_candidate"`, `action="evaluate_non_qualifying"`) with assessment/Policy/Framework identity, `outcome="not_qualified"`, and the evaluation fingerprint - a closed, deterministic, free-text-free shape, not a durable negative entity. Repeated non-qualifying evaluation of the same immutable Completed Assessment appends one audit event per call (append-only event log, not idempotent state).

Decision 2/14 - Review workflow: `TalentReviewCandidate` gained `status` (`pending_review`/`reviewed`, CHECK-constrained), `reviewed_by_user_id`, `reviewed_at`. `POST /api/talent/review-candidates/{id}/review` (`talent_review_candidates.manage`) transitions `pending_review` -> `reviewed` only - no reverse transition, no other state. It never alters assessment

evidence/competency results/KPI and never auto-identifies. `.manage` grants only this transition, never Official Identification authority (that is exclusively Decision 6's separate permission+scope combination).

Decisions 3-7, 17 - Official Identification: new append-only `TalentOfficialIdentification` (`talent_official_identifications` table), bound via the identical composite-scoped-FK chain `TalentReviewCandidate` itself uses (assessment/cycle-population-member composite FKs) plus a direct FK to the Review Candidate. `decision` is exactly `identified`/`not_identified` (CHECK-constrained) - `deferred`/`revoked`/`superseded`/`re-identified`/ `reinstated`/`expired` are explicitly deferred, not implemented. Recording requires the candidate's `status` to already be `reviewed` (`candidate_not_reviewed` otherwise - a clean business error, not a crash). Exactly one decision per candidate: `uq_talent_official_identifications_candidate` (unique constraint) plus a service-layer check that raises `already_decided` (409) rather than letting a raw `IntegrityError` surface, matching M2's layered stale-write enforcement discipline. `not_identified` is exactly as durable as `identified` - never a failed-assessment/KPI/low-score signal, and there is no update/revoke/supersede function anywhere in `talent_official_identification_service.py`. `POST /api/talent/official-identifications` requires the dedicated `talent_official_identifications.record` permission AND organization/global access scope (`auth.can_access_all_branches`, the same organization-authority gate M4's Cycle `.govern` established) - a Branch-scoped actor is denied even when explicitly granted `.record`. Reads (`talent_official_identifications.view`) use the same frozen Cycle Population Member Branch discipline as Review Candidate/M4/M5.

Decisions 8-13 - Educator Input: new append-only-with-lineage `TalentEducatorInput` (`talent_educator_inputs` table) - explicitly not a generic note, private diary, assessment score, Official Identification, Review Candidate state, AI output, or messaging/chat. Required binding: SchoolGroup, Student, Program, AcademicYear, `observed_at`, and a historical Placement/Branch/Grade/Section snapshot. Optional binding: Cycle, frozen Cycle Population Member, Assessment, Review Candidate - when supplied, the service validates exact Student/Program/AcademicYear alignment rather than trusting the caller. If a frozen Cycle context is supplied, its snapshot wins (mirroring M4's Open-time freeze exactly, applied here at input-creation time); otherwise the Student's canonical `StudentAcademicPlacement` effective AT `observed_at` within the AcademicYear is resolved using the same half-open `[effective_from, effective_to)` logic as M4's `derive_eligible_population` - no valid historical Placement is a clean `no_historical_placement` rejection, never a silent fallback to current Placement. `category` is closed (`observation`/`context`/`supporting_evidence`); `content` is plain bounded text, non-empty, capped at 2,000 characters (`talent_educator_input_service.MAX_CONTENT_LENGTH` - no existing bounded-text constant fit this exactly, so M6 sets this cap directly, reusing the same clean-then-length-check validation *pattern* already used for `summary`/ `evidence` elsewhere in this milestone chain). Amendment is append-only via self-referential `supersedes_educator_input_id`; the original row is never edited in place. Cycle prevention walks the chain exactly like M3's `_validate_supersedes` for Framework Version supersession. Default reads (`list_inputs`) return only the current/latest version in each chain; an explicit `GET /{id}/history` returns the full chain. Cross-Student/cross- Program/cross-tenant supersession is rejected. The free-text `content` body is stripped from `TalentAssessmentAudit.before_json`/`after_json`, reusing M5's exact evidence-stripping pattern. New permissions `talent_educator_inputs.view/add/amend` are independent of every other Talent permission family; Branch access uses the row's own PERSISTED historical `branch_id`, never current Student Placement.

Migrations: `20260904_006_talent_review_candidate_foundation` (original `talent_review_candidates` table + `review_candidate` resource_type). `20260904_007_talent_review_workflow_identification_educator_input_foundation` adds the review-workflow columns, `talent_official_identifications`, `talent_educator_inputs`, widens `resource_type` further (`review_candidate_review`, `official_identification`, `educator_input`), and relaxes `talent_assessment_audits.cycle_id/framework_version_id` to nullable (Educator Input's Cycle binding is optional, so an Educator Input audit row may have no Cycle/Framework context - the composite FK is simply unenforced whenever a referencing column is NULL, standard SQL MATCH SIMPLE semantics). Both follow the established dialect-branched widening pattern (PostgreSQL `NOT VALID`/`VALIDATE`, SQLite

`_sqlite_rebuild_from_current_metadata`).

Independent-review closure: M6 write authorization validates the historical Branch resolved for every Educator Input create/amend operation, not only subsequent reads. Direct out-of-scope Review Candidate, Official Identification, and Educator Input IDs return the repository's uniform 404. Official Identification is constrained to the Review Candidate's exact Assessment/frozen-member/Student/Program/AcademicYear/Framework/tenant tuple, and Educator Input amendment lineage is database-unique as well as service-validated, preventing concurrent forks. These constraints are present in both ORM metadata and the idempotent migration path.

Review Candidate, Official Identification, and Educator Input remain structurally distinct tables with no shared mutable status column - a current Student transfer never reinterprets any of their frozen/persisted historical Branch access.

M6 Governance Review: Decisions (Resolved)

The 9 questions this section previously listed as open (Official Identification decision vocabulary and access-scope requirement, whether a non-qualifying evaluation is durably recorded, exact Review workflow states and the mandatory review gate, Educator Input's binding context/category taxonomy/amendment model, and negative-decision durability/re-identification) were resolved by 18 explicit Product Owner governance decisions and are now fully implemented as described above. Explicitly and permanently DEFERRED by those same decisions, not merely unimplemented by omission: Official Identification revocation, supersession of an Official Identification, a second identification decision, re-identification, a `deferred` decision state, a generic free-text review-note/case-management system, assessor assignment, Learner Profile, Development and Support, analytics/Talent Map, AI/AI Suggested Review, and Educator Input analytics/export/AI/attachments.

Talent Student Assessment And Competency Results

M5 implements the canonical `TalentStudentAssessment` authority for exactly one frozen Cycle Population Member/Student per Open Assessment Cycle. The Assessment persists its exact SchoolGroup, Cycle, frozen-member, Student, Program, Academic Year, and immutable Framework Version context; current Placement can never alter its authorization or historical interpretation. `TalentStudentCompetencyResult` rows bind each assessment to the exact Framework Competency and Framework-owned Rubric Level. All competency results use expected-revision stale-write protection, and only an In Progress Assessment in an Open Cycle is editable.

Completed requires a valid result for every exact Framework Competency. It is read-only, as are the explicit final non-complete Incomplete and Insufficient Evidence states; no correction/reopen workflow exists. A qualitative Program can complete without KPI or numeric rubric values. When an exact Framework enables the approved `weighted_level_average` KPI, completion calculates it using integer numerator contributions and denominator 10,000, rounds to the nearest integer with ROUND_HALF_UP, and atomically persists the Framework result scale, method, numerator, SHA-256 input fingerprint, and calculation timestamp. This is Framework-specific evidence, never a universal or cross-Program Talent Score. No KPI result exists for In Progress, Incomplete, Insufficient Evidence, or no-KPI Assessments.

M5 adds Administrator-only-by-default `talent_assessments.view`, `.manage`, and `.complete` permissions under `/api/talent/assessments`. Permission and canonical scope govern assessment work; assessor assignment remains deferred. Branch access resolves the frozen member Branch, not current Student Placement. `TalentAssessmentAudit` remains the bounded append-only operational audit, now recording assessment/result identifiers and transitions while excluding free-text evidence. Deferred: Review Candidate instances/evaluation, Official Identification, Educator Input, Learner Profile, analytics/Talent Map, AI, Development and Support, late population exceptions, assessor assignment, and corrections. Live PostgreSQL execution remains a follow-up.

M5/M3 Framework Read-Contract Addition (Additive, For A Future Assessment-Entry UI)

The M3 Framework read routes previously never exposed the stable identifiers the M5 write route requires. `GET /api/talent/programs/{id}/frameworks/{id}` returned each competency only as `{competency_id, display_order, label, description}`, where `competency_id` is the parent `TalentCompetency` catalog FK (`FrameworkCompetency.talent_competency_id`) - not the row's own `FrameworkCompetency.id`. Separately, `get_framework_configuration`'s `levels[]` (used by `.../frameworks/{id}/configuration`) omitted `TalentRubricLevel.id` entirely. `PUT /api/talent/assessments/{id}/competency-results/{framework_competency_id}` (`talent_student_assessment_service.set_competency_result`) is FK-constrained on exactly those two real ids (`FrameworkCompetency.id`, `TalentRubricLevel.id`), so no UI could have been built purely from the existing read payloads. Both routes now additively include the row's own `id` alongside the pre-existing fields; no permission, schema, migration, or write-route contract changed. Regression coverage: `tests/test_talent_rubric_kpi_candidate_policy.py::test_rubric_level_order_is_the_semantic_rank_independent_of_numeric_value` and the new `tests/test_talent_student_assessment_competency_results.py::test_framework_read_ids_are_accepted_end_to_end_by_the_write_route`, which proves a write built purely from the new read `id` fields succeeds.

Talent Assessment Cycle And Frozen Population Foundation

Milestone M4 implements `TalentAssessmentCycle` as one SchoolGroup-wide canonical authority bound to a Talent Program, Academic Year, and exact Framework Version. A Cycle deliberately has no `branch_id`. Its lifecycle is one-way `draft -> open -> closed`: there is no reopen, rollback, deletion, population removal, or assessor assignment. ADR 0033 permits only audited, additive synchronization of newly eligible Students while the Cycle is Open. Draft metadata uses expected-revision stale-write protection. The explicit `population_effective_at` may change only in Draft and is required for preview and Open; no hidden current-time rule is used.

Draft preview dynamically resolves Students with an effective-dated `StudentAcademicPlacement` at that exact instant, in the Cycle Academic Year and the enabled Program annual configuration's eligible KG/1-12 grades. `Student.status` is current mutable state with no effective-dated history, so it never gates this historical-instant derivation - only stable tenant/ identity alignment between Student and Placement is enforced; a Student's present-day status change never silently alters an already-defined historical population. Opening requires an Active Program and exact Active Framework, locks and revalidates the Cycle, derives the complete SchoolGroup population, inserts frozen members, stores the full count and SHA-256 population fingerprint, and marks the Cycle Open in one transaction. Each member preserves Student and Placement identity plus Academic Year, frozen Branch, grade, section, nullable PlanningSection provenance, and effective/frozen timestamps. Current Placement, annual configuration, PlanningSection, and later Framework retirement cannot reinterpret an Open/Closed population. After an authorized Student Placement save, ADR 0033 synchronization locks each matching Open Cycle, adds only a missing member with the new Placement snapshot, and updates revision, count, fingerprint, and audit in the placement transaction. Opening an existing Open assessment also invokes a dedicated organization-governed, expected-revision reconciliation before its population is read. That operation derives canonical Placement eligibility at the synchronization instant and adds only missing members, covering Students eligible before ADR 0033 deployment. Existing members and evidence are never changed; Closed Cycles remain final.

M4 adds dedicated Administrator-only-by-default permissions: `talent_assessment_cycles.view`, `.manage`, `.view_population`, and `.govern`. Branch-scoped `.manage` holders may author Draft metadata but cannot Open or Close; lifecycle governance requires `.govern` plus organization/global scope and always freezes the complete SchoolGroup population. Identifiable preview and frozen reads require `.view_population`. Branch-scoped readers receive only members whose resolved preview Branch or frozen historical Branch is authorized, an explicitly filtered subset count, and no full count or fingerprint. Organization/global population readers receive the complete population, count, and fingerprint. Metadata reads never disclose integrity metadata merely through `.view`.

Operational create/update/Open/Close events use the bounded append-only `TalentAssessmentAudit`, separate from Framework configuration audit, with actor, scope, Program/Year/Framework context, before/after JSON, correlation id, and Open population provenance. APIs are under `/api/talent/assessment-cycles`; no UI exists.

Deferred: Student Assessment, assessor assignment, competency results, Review Candidate instances/evaluation, Official Identification, Educator Input, Learner Profile, analytics/Talent Map, AI, development plans, and late population exception workflow. Live PostgreSQL execution remains a follow-up.

Talent Rubric, KPI, And Review Candidate Policy Foundation

Milestone M3 completes the deterministic semantic configuration subordinate to each `TalentProgramFrameworkVersion`. Migration `20260904_003_talent_rubric_kpi_candidate_policy_foundation` adds one optional Framework rubric, configurable ordered rubric levels, exact Framework- competency/level descriptors, optional KPI configuration and weighted competency inputs, and an optional bounded Review Candidate Policy with ordered declarative rules. Rubric level names and codes are Program/Framework configuration, never a global vocabulary. A qualitative Performing Arts Framework can activate without KPI or numeric rubric values.

KPI configuration supports only `weighted_level_average`: enabled inputs are exact Framework Competencies with positive basis-point weights totaling 10,000, all levels require explicit integer values inside an explicit result scale, and interpretation text is required. Review Candidate Policy supports only `all`/`any` composition of `rubric_level_at_or_above` rules and, when an enabled KPI exists, `kpi_at_or_above` rules. These are stored policy semantics only; M3 does not evaluate evidence or create candidate records. No script, SQL, Python expression, generic JSON logic, universal score, or cross-Program averaging is accepted.

M3 Governance Closure formally approves `weighted_level_average` as one bounded, optional, Framework-specific KPI calculation primitive: it is never a universal Talent Score, is never cross-Program normalized, is enabled only through explicit Framework configuration, and requires numeric rubric values only while it stays enabled. Additional calculation primitives require future governed Product Owner approval and an explicit model/constraint change; the `calculation_method` CHECK constraint stays a closed enum, never a generic expression/scripting mechanism. Qualitative Programs (for example Performing Arts) remain fully valid and activatable with no KPI and no numeric rubric levels.

`rubric_level_at_or_above` candidate-policy rules are defined by the configured Rubric Level order (`TalentRubricLevel.display_order`, lowest proficiency first) - a rule targeting a level means that level or any level ordered higher. This ordering never depends on `numeric_value`, so qualitative Programs with no numeric levels still get correct, meaningful thresholds. `display_order` is the single authority for both presentation order and this semantic proficiency rank: it is dense, unique per rubric, and fully reindexed on every add/remove/reorder, so no divergence between a level's displayed position and its proficiency rank is possible. Reordering levels is Draft-only and always bumps the Framework revision/semantic fingerprint since order is part of the M3 semantic payload; Active/Retired order is immutable. Candidate-policy rule evaluation itself remains deferred to a later assessment milestone.

Every M3 mutation requires `expected_revision`, is restricted to a Draft Framework, increments its revision, recomputes its semantic fingerprint over the complete ordered M2+M3 configuration, and appends an M3-specific action to the existing `TalentConfigurationAudit`. Audit identity now targets the specific M3 child resource that changed - `rubric`, `rubric_level`, `rubric_descriptor`, `kpi_configuration`, `kpi_component`, `review_candidate_policy`, or `review_candidate_rule` - matching the M2 `framework_competency` precedent, instead of only the containing `framework_version`. Create operations audit the new child row's post-flush id; `reorder_rubric_levels` (the one M3 bulk operation) appends one audit row per reordered level so each level's own audit identity is preserved. The widened `resource_type` CHECK constraint is applied by the M3 migration itself (both the SQLite table-rebuild and PostgreSQL `NOT VALID`/`VALIDATE` pattern already used elsewhere in `db_migrations.py`), since the audit table was originally created by the separate, already-applied M2 migration. Active and Retired Framework M3 state is immutable. Framework cloning creates independent rubric, level, descriptor, KPI, component, policy, and rule rows, mapping Framework Competencies through durable `TalentCompetency` lineage. Composite scoped foreign keys enforce SchoolGroup, Program, Framework, FrameworkCompetency, rubric, and level alignment.

The bounded JSON routes remain under `/api/talent/programs`: reads use `talent_programs.view`, Draft mutations use `talent_programs.manage`, and the existing `talent_programs.govern` plus organization/global scope requirement continues to control activation/retirement. No new permission or entitlement logic was introduced.

Not implemented: assessment/evidence, scoring/results, Review Candidate instances, candidate-policy evaluation, Official Identification, Educator Input, Learner Profile, analytics/Talent Map, AI, Development & Support, Learning Style, or UI. Live PostgreSQL execution remains a follow-up; SQLite migration, FK, lifecycle, service, API, permission, and regression coverage is in `tests/test_talent_rubric_kpi_candidate_policy.py` plus M1/M2 suites.

Student & Academic Placement Foundation

`Student` is a canonical SchoolGroup-owned identity distinct from `User`: `models.py`/migration `20260904_001_student_academic_placement_foundation` add `students` (first/father/last name, gender, status, audit timestamps), `student_external_identifiers` (namespace/value/source cross-system linking with tenant-scoped uniqueness and an activate/deactivate lifecycle), effective- dated `student_academic_placements`, and append-only `student_audits`. There is no separate Enrollment entity; `StudentAcademicPlacement` is the authoritative enrollment record, resolved by half-open `[effective_from, effective_to)` interval semantics with overlap rejection and deterministic point-in-time resolution. `academic_grade.py` (extracted from `routers/planning.py`'s prior inline normalization, existing callers unchanged) is the shared canonical KG/1-12 grade vocabulary used by both Planning and Student.

Every placement carries an immutable historical `grade_level`/`section_name` snapshot captured from the linked `PlanningSection` at creation or transition time (or supplied directly when no section link exists). A later Planning section rename/renumber never rewrites already-recorded placement history; deleting the linked `PlanningSection` only nulls its FK on referencing placements and preserves the snapshot. `transition_placement` closes the current placement and opens a new one atomically, preserving full history; `correct_placement` revalidates scope/snapshot/overlap and governs an in-place correction of the same row rather than creating parallel history. Every mutation writes an append-only `StudentAudit` row (before/after JSON, actor, action) in the same transaction; audits are never updated or deleted. Tenant/branch-scoped composite foreign keys reject cross-tenant Branch, AcademicYear, or PlanningSection references (`invalid_scope`).

`routers/students.py` (`/api/students`) is backed by `student_academic_service.py`. Branch-scope enforcement always runs after the permission check: an in-scope Student ID with an out-of-scope existing or target placement branch is rejected 403 without revealing existence; a foreign-tenant Student ID returns a uniform 404. The final canonical permission mapping (Checkpoint A) replaces the interim implementation, which gated all Academic Placement mutation through the broad, commonly-granted `planning.edit_section` permission alone with no Student-specific permission at all. `permission_registry.py` now has a dedicated `students` group: `students.view` (Student and Placement-history reads), `students.create`, `students.edit` (identity, non-status fields), `students.activate_deactivate` (status-only updates), `students.manage_identifiers` (External Identifiers), and `students.manage_placements` (create/end/transition/correct Academic Placement). See "Student And Talent Program Permission Governance" below for the shared default-assignment rationale.

Not implemented: Student UI, import/merge, Talent/Rubric assessment linkage, and analytics. PostgreSQL validation (constraints, concurrent placement writes) has not been executed against live PostgreSQL; only SQLite-backed `tests/test_student_academic_foundation.py` coverage exists.

Talent Program & Framework Foundation

`TalentProgram` is a durable SchoolGroup/organization-owned identity, not a Subject: migration `20260904_002_talent_program_framework_foundation` adds `talent_programs` (draft/active/retired lifecycle), `talent_program_academic_year_configurations` (per-year enable flag plus eligible KG/1-12 grade set), `talent_program_framework_versions` (Framework Version lifecycle with deterministic sequential version allocation, a revision counter, and a semantic fingerprint), `talent_competencies` (organization-scoped competency lineage), `talent_framework_competencies` (version-specific competency membership snapshots -

label/description/order captured per Framework Version independent of later competency edits), and append-only `talent_configuration_audits`.

Exactly one Active Framework Version exists per Program: activating a new version requires explicit supersession of the current active version (`supersession_required` otherwise) and atomically retires it; an active Framework's content becomes immutable (`immutable_framework`). Every Draft mutation and activation call is stale-write protected: the caller's `expected_revision` (and, for activation, `expected_fingerprint`) must match current state or the call fails `stale_framework`.

Governance is split into two tiers. Draft authorship - Program create/edit, Program annual configuration, Framework Draft edits (including competency membership add/update/reorder/remove within a Draft), and Competency create/edit - is permission-gated and organization-scope-independent. Framework activate/retire and Program lifecycle transitions additionally require organization/global access scope (`auth.ACCESS_SCOPE_ORGANIZATION`/ `ACCESS_SCOPE_GLOBAL`, checked through `_organization_authorized()` in `routers/talent_programs.py`) on top of the permission check; a Branch-scoped actor can never activate, retire, or transition a Program/Framework regardless of permission grant. This organization-scope gate was already correct in the interim implementation and is unchanged by Checkpoint A.

Checkpoint A fixed a real defect: Draft authorship was gated only by the same broad `planning.edit_section` permission already default-granted to Editor/User roles, so any Branch-scoped actor holding that common permission received full organization-wide authorship of Programs, Framework Drafts, and Competencies with zero Branch-relationship constraint (`TalentProgram` has no branch dimension to constrain against). `permission_registry.py` now has a dedicated `talent_programs` group: `talent_programs.view` (Program, Framework

1 including history - and Competency reads), `talent_programs.manage` (all

Draft authorship listed above), and `talent_programs.govern` (Framework activate/retire, Program lifecycle transition; checked in addition to, not instead of, the unchanged organization-scope gate).

`routers/talent_programs.py` (`/api/talent/programs`) is backed by `talent_program_service.py`; foreign-tenant Program IDs return a uniform 404.

Not yet implemented: Student Assessment, Review/Identification workflow, Learner Profile, analytics/Talent Map, and AI. PostgreSQL validation has not been executed against live PostgreSQL; only SQLite-backed `tests/test_talent_program_framework_foundation.py` coverage exists.

Student And Talent Program Permission Governance

Both new `students.*` and `talent_programs.*` permission keys follow the same established default-assignment pattern already used for the sensitive `users` group: `permission_registry.DEFAULT_ROLE_PERMISSIONS[ROLE_ADMINISTRATOR]` grants every permission key not in `DEVELOPER_ONLY_PERMISSION_KEYS` automatically, while Editor and User roles only receive the curated `_EDITOR_LIKE_PERMISSIONS` subset. Neither new group was added to that subset, so - exactly like `users.*` today - Student and Talent Program management are Administrator-only by default across every tenant; Limited remains read-only-elsewhere and gets neither group. No migration or seed change was needed: a newly introduced permission key with no per-tenant `RolePermission` override row simply falls back to the code-level default at read time (`auth.get_allowed_permission_keys`), matching how every prior permission addition (e.g. `timetable.manage_teacher_rules`, `curriculum.adjust`) was introduced as a pure `permission_registry.py` edit. Whether either group should ever be extended to Editor/User by default is an explicit Product Owner decision to be made later through the existing Role Permissions configuration UI, not a default this closure task changed.

Planning Subject Requirement Removal (Single And Bulk)

A Planning requirement is classified `removable` (untouched explicit `PlanningSubjectDemand`), `permanent` (Curriculum-Adjustment-touched - `updated_by_user_id` set - shown with a "Protected" explanation, never a hidden action), `fallback` (no explicit row at all - resolved only from the Subject catalog), or `not_found`. Both `removable` and `fallback` are now removable: an explicit row is deleted, while a `fallback` requirement is removed by creating an explicit `is_active=False/weekly_periods=0` suppression row. Unlike a Curriculum Adjustment retirement (which stamps `updated_by_user_id`), this suppression row only sets `created_by_user_id` (the acting admin, for audit) and leaves `updated_by_user_id` NULL - so it stays classified `removable`/setup-only rather than becoming permanent. A row only becomes `permanent` by genuinely being touched by Curriculum Adjustment, never merely by being suppressed from this action. When its section is otherwise dependency-free, section deletion automatically removes only these inactive zero-period setup suppressions in the same transaction; admins do not need to expose or clear an implementation artifact. This closed a real gap where "Remove demand" only ever appeared for explicit-row requirements, so the same teacher's subjects could inconsistently show the action for reasons that had nothing to do with the teacher.

`POST /planning/subject-requirements/remove` (`routers/planning.py`) handles both single removal ("Remove Subject Requirement") and checkbox-selected bulk removal (supported across every section on the page at once) atomically. Each expanded Planning section exposes a section-scoped **Select all** control beside a trash-icon action whose label reflects the current selection (singular or the exact plural count); the page-level action mirrors that state. Every target is validated and scope-checked before anything is mutated, and any protected/invalid/ out-of-scope target blocks the whole request with a named reason. Teacher assignments, timetable data, and Curriculum Adjustment audit history are never touched. The prior round's `GET /planning/subject-demand/delete/{demand_id}` route remains in place for removing active setup requirements.

Cline And Three-Agent Development Pool

Codex, Claude Code, and Cline/ClinePass are approved TIS development agents. Cline's repository entry points are `.clinerules/tis.md` and `.cline/skills/tis-kms-developer/SKILL.md`. They require governing KMS onboarding, implementation inspection, isolation/RBAC and database safety, minimal diffs, proportional tests, task-level KIA, KMS sync/check, and deployment reporting.

The master conversation balances tasks by complexity, specialization, risk, independent-review value, and current usage/load, without permanent assignments. See [Project Governance](engineering/PROJECT_GOVERNANCE.md) for the allocation policy and [AI Coding Workflow](engineering/AI_CODING_WORKFLOW.md) for Cline setup/invocation. This configuration changes no application, schema, or production behavior.

Claude Code KMS Configuration

Claude Code uses root `CLAUDE.md` as its repository entry point, `.claude/skills/tis-kms/SKILL.md` as the reusable TIS execution workflow, and `.claude/agents/tis-kms-developer.md` as the bounded specialized subagent. These files mirror, but do not replace, root `AGENTS.md` or the authoritative KMS. They require the same first reads, worktree preservation, tenant and deployment boundaries, task-level `.kms-impact.yml`, proportional validation, KMS sync/check, and no-commit/no-push default.

Return-To Reopen-On-Load UI Pattern

TIS is server-rendered FastAPI/Jinja, not an SPA, so a redirect after an action normally collapses any open `<details>`/accordion. The sanctioned fix reuses the existing `return_to` convention: `redirect_utils.py` (moved out of `main.py`, existing callers unchanged) provides `safe_redirect_path()` - rejecting `http(s)://` and `//` targets - plus fragment-preserving `redirect_with_notice/redirect_with_error`. A globally-loaded shared script, `static/js/reopen-on-load.js`, reads a target element id from `window.location.hash` (or an inline `window.__tisReopenTargetId` for a non-redirect response) on page load and reopens it if it is a `<details>`, scrolling it into view; a missing/stale id is a harmless no-op. Planning's "Remove demand" action is the first consumer - do not invent a page-local variant of this pattern; give the element a stable id and pass `return_to` through the existing

convention instead.

Planning And Subject Delete Dependency Guards

Deleting a Planning section or a Subject now runs a dependency check before any mutation and blocks with a customer-safe, per-category explanation instead of raising an unhandled `IntegrityError` or deleting silently. Planning section delete checks `TeacherSectionAssignment`, `PlanningSubjectDemand`, `TimetableEntry`, `TeacherSchedulingRuleTarget`, `CalendarEvent`/`CalendarEventSectionTarget`, and section-scoped `SubjectDistributionRule`; `TeacherSectionAssignment` is now a blocker like every other category rather than being auto-deleted. The sole cleanup exception is an inactive zero-period `PlanningSubjectDemand` with no `updated_by_user_id`: this setup-only Remove Subject Requirement suppression is not a blocker and is deleted atomically with an otherwise deletable section. Active setup demand, Curriculum-Adjustment-touched history, and every other dependency still block. Subject delete and Bulk Delete consolidate the equivalent check across `Teacher.subject_code`, `TeacherSubjectAllocation`, `TeacherSectionAssignment`, `PlanningSubjectDemand`, `TimetableEntry`, `CurriculumAdjustmentAudit`, and `SubjectDistributionRule`; Bulk Delete is atomic, so any blocked Subject in the selection deletes none of the batch and names every blocked Subject with its reason.

Planning subject demand is split into removable and permanent, using the existing `PlanningSubjectDemand.updated_by_user_id` column as the signal: Curriculum Adjustment always stamps it, the one-time setup backfill never does, so an untouched row is pure scaffolding and can be hard-deleted through the new `GET /planning/subject-demand/delete/{demand_id}` action ("Remove demand" on the Planning page), after which the section or subject becomes deletable. A row Curriculum Adjustment has ever touched, active or retired, is genuine history and remains a permanent blocker. Published/archived timetable placements follow the same principle without a new action: only mutable Draft entries can be removed. No archive/closed/inactive lifecycle, broad schema change, migration, or cascade deletion was introduced.

Professional Timetable Lifecycle UX

The timetable workspace presents Draft, Validate, Approve, and Publish without a new durable state. Validation and approval remain one backend transition to `publication_ready`; the active pointer identifies the immutable current published version. Version facts expose origin, source, created/generated, validation, approval, publication, mutability, and active state. Authorized users can create a working Draft from current published or immutable history without changing the active publication. Lifecycle and mutation failures retain legacy messages and add canonical conflicts, with an explicit refresh action for stale revisions.

Structured Timetable Conflict Evidence

`timetable_conflicts.py` defines the canonical internal conflict contract used by readiness, feasibility payloads, generation terminal payloads, and the independent validator. Stable machine codes coexist with legacy source codes/messages for API and UI compatibility. Conflicts classify HARD/SOFT severity and DURABLE, RECALCULABLE, or TRANSIENT evidence without adding a universal conflict table.

Public serialization is authorization-safe: it exposes only already-authorized same-scope entity references, redacts unauthorized references, never emits internal Lesson Requirement identities or solver clauses, and represents internal correlation only as presence flags. Infeasible, timeout, cancellation, stale authority, and solver/execution failure remain distinct outcomes. Existing safe human-readable generation and feasibility messages remain unchanged.

Timetable Lesson Requirement Projection

`timetable_requirement_projection.py` is the read-only domain boundary from Planning authority to timetable scheduling requirements. It resolves exact SchoolGroup/Branch/Academic Year scope, Planning section and subject identity, explicit-first `PlanningSubjectDemand`, legacy `Subject.weekly_hours` fallback only when no explicit row exists, and the existing Planning/HRT teacher authority. Explicit zero or inactive rows remain authoritative suppression

evidence.

Each projected requirement has an internal deterministic identity over its scope and governing Planning source plus a source fingerprint covering effective periods, active state, and teacher assignment authority. Schema-v5 immutable snapshots capture both values. Readiness and snapshot creation consume the same projection; the problem builder carries it to generation and the independent validator verifies the captured requirement contract. Planning remains the only editable academic-demand authority, and no Lesson Requirement table or CRUD API exists.

Timetable Solver Adapter Boundary

The timetable Workflow worker consumes the solver-neutral `TimetableSolver` contract in `timetable_solver.py`. Release 1 binds that contract only to the existing OR-Tools CP-SAT implementation; this boundary does not authorize a second solver, relaxed constraints, or a parallel generation architecture. Infeasibility diagnosis is part of the same adapter contract. The independent domain validator and atomic fingerprint/revision persistence gates remain outside the solver and continue to decide whether a candidate may be saved.

Teacher Scheduling Rules

Administrators with `timetable.manage_teacher_rules` configure teacher-specific timing policy in Timetable Settings without changing Planning demand or teacher allocation. The normal form asks only for teacher, rule, days, numbered periods, and optional Current/New sections with Select All. **Schedule within these periods** constrains existing eligible lessons to an allowed window without creating occupancy or demand; **Must teach these periods** requires occupancy in every selected slot; **Unavailable** prohibits all teacher lessons in those slots. Existing first/last, grade-target, and preference rules remain compatible but are not exposed in the simplified normal form.

Schema-v5 immutable snapshots capture canonical resolved teacher rules in the constraint fingerprint. Rule changes mark unpublished Drafts stale, clear Draft approval, and require regeneration while published history remains untouched. Readiness and the problem builder reject deterministic workload, allowed-window capacity, target, lock, slot, and contradictory hard-rule conflicts; CP-SAT enforces hard timing and scores existing preferences; the independent validator repeats hard-rule and window checks. With no rules, generation behavior is backward compatible.

When more than one Schedule-within rule targets the same teacher demand, their configured slots form one allowed-window union for that demand. They are not independent whole-workload restrictions and must not be intersected. The problem builder computes this combined authority before CP-SAT and performs exact teacher-slot matching plus deterministic checks for window/unavailability capacity, Must-teach compatibility, daily coverage, hard maximum-per-day, minimum-day, and double-block feasibility. Proven conflicts appear as readiness blockers with required-versus-available guidance instead of reaching a generic solver-infeasible result. Grouped activities count one teacher occupancy.

If CP-SAT still proves the complete model infeasible after deterministic checks, the Workflow reruns bounded diagnostic profiles without changing the real run. It distinguishes base demand/collisions, locks, grouped activities/resources, Subject Distribution Rules, Teacher Scheduling Rules, and subject/teacher-rule interaction. The proven family, current lock count, and grouped-activity count are stored in the Generation Run's customer-safe failure details. A diagnostic timeout is reported as inconclusive rather than mislabeling a constraint family. Diagnostic profiles use the dedicated `TIS_TIMETABLE_DIAGNOSTIC_TIMEOUT_SECONDS` Workflow setting, defaulting to 60 seconds per attempted profile and independent of the main solver timeout. Profiles for locks or grouped activities are skipped when their captured counts are zero; successful isolation also reports safe model/rule/window counts where available.

The configuration UI keeps section selection literal: Select All synchronizes every visible Current/New section, editing restores the exact saved targets, and saved summaries name selected grade-section labels instead of collapsing them to an any-class label. Manual Draft placement rejects hard Unavailable and Schedule-within violations immediately. Complete Draft validation additionally requires every Must-teach slot to contain an eligible lesson in its configured section scope; the same validation blocks approval and publication. Soft preferences remain non-blocking.

Guided Curriculum Adjustment UI

Administrators with `curriculum.adjust` can enter a five-step, page-based workflow from Planning or Subjects. The UI selects Current/New scope, prepares the existing read-only preview, separates blockers from warnings, requires an explicit eligible teacher decision for every section, and submits the reviewed fingerprint to the atomic apply endpoint. A changed Planning state refreshes the preview without exposing revision terminology. Success links to Timetable and regeneration, but never starts generation automatically.

The entered transfer period count is authoritative: preview computes each section's source and target result from its effective `PlanningSubjectDemand`, rejects non-positive or excessive transfers, and apply persists exactly that reviewed result. Subjects presents those same Current/New Planning values for the active branch/year: a uniform grade-subject demand is shown directly, while section differences display as **Varies** with a section breakdown. `Subject.weekly_hours` remains a catalog/default value and is not rewritten by scoped adjustments.

The same workflow supports `reduce_only`: no target subject or teacher reassignment is required, the requested reduction is applied to each selected source demand, and all existing preview, fingerprint, transaction, Draft invalidation, lock, rule, and publication safeguards remain in force. Subject Details shows current effective Planning periods and offers a permission-gated **Adjust Weekly Periods** link prefilled for that grade-specific subject. Multi-grade reduction is intentionally not combined because subject codes and the atomic review/audit contract are single-subject identities.

Atomic Curriculum Adjustment Apply

Stage 4 adds `curriculum_adjustment_apply_service.py` and permissioned `POST /planning/curriculum-adjustments/apply`. The service requires a Stage 3 fingerprint and an explicit target-teacher decision, including intentional unassigned, for every affected section. In one owned transaction it locks and revalidates exact-scope Current/New Planning authority, rejects active generation, stale previews, qualification/capacity failures, rule conflicts, and invalid Draft locks, then updates source/target demands and assignments and writes one durable audit. Zero source demand is retired and its active section rule is inactivated. The current unpublished Draft is preserved but marked stale, approval is cleared, and later regeneration is required. Published versions and the active pointer are never mutated. `curriculum.adjust` is a dedicated apply permission.

Read-Only Curriculum Adjustment Preview

Stage 3 adds `curriculum_adjustment_preview_service.py` and permissioned JSON route `POST /planning/curriculum-adjustments/preview`. The service previews source-demand reduction and transfer to a target subject for one grade, selected Current/New sections, or every active source use in one exact SchoolGroup/branch/year. It reads explicit-first `PlanningSubjectDemand`, current Planning/HRT teachers, teacher loads and capacity, normalized Subject Scheduling Rules, grouped legacy configuration, and the current mutable Draft Timetable revision. It returns per-section before/ after periods, teacher options, blockers/warnings, Draft stale/regeneration impact, and a deterministic fingerprint that can be rebuilt before a future apply action. It writes nothing, never assigns a teacher, and never changes timetable history.

Explicit Planning Subject Demand Foundation

Migration `20260828_004_planning_subject_demands_foundation` adds `planning_subject_demands`, an explicit branch/year and Planning-section scoped weekly subject-demand foundation. The migration idempotently backfills matching grade subjects for Current/New Planning sections only. Composite foreign keys prevent section or subject scope drift, while a partial unique index permits at most one active demand for a section and subject and preserves retired rows. `planning_subject_demand_service.py` resolves explicit active or retired rows before using legacy `Subject.grade + Subject.weekly_hours` fallback. Operational Planning, teacher required/allocated/remaining load, timetable readiness/workspace/snapshot/ generation input, Subject Scheduling Rule arithmetic, and required-hours reports consume that section demand. An explicit inactive or zero row suppresses demand. Legacy fallback remains only when a section-subject has no explicit row. Published timetable history remains

immutable and unchanged.

Tenant Report Branding

Tenant-facing operational reports and exports use `tenant_report_branding.py` as their shared logo authority. It resolves only configured branch/organization logo slots through the existing tenant-scoped branding resolver. A tenant with no configured logo receives a clean text header with no image; tenant artifacts must never fall back to a TIS product logo. Timetable PDF/XLSX exports, academic-calendar PDF exports, and observation PDFs consume this rule. TIS branding remains valid in the application shell, login/product surfaces, public/platform-owned materials, and explicitly internal platform-owner reports.

Subject Scheduling Rules UI

Timetable Settings now presents a grade-first Subject Scheduling Rules section: `planning_scope_service.py` is the reusable operational selector authority. The main and copy-rule grade selectors, timetable workspace section filters, and calendar/grouped section choices derive only from Current/New `PlanningSection` rows in the selected branch and academic year, never from the global grade range or a catalog-only Subject row. An administrator selects one planned grade, can search its compact subject list, and sees each Planning weekly total, current pattern, and configured/default status without raw subject codes. The native dialog stays closed on page load and opens only after Edit has populated the selected Subject + Grade and read-only Planning total; Cancel/Close restores the closed state. Its compact Session Structure keeps the aligned double-block and single-session steppers above one full-width live total. Two-period Separate Sessions and Consecutive Double Block quick choices remain available. Multiple primary conditions can be combined in a balanced grid; minimum teaching days, strictness, and section overrides remain progressively disclosed. Legacy subject-code mappings and Swimming JSON remain available only inside a secondary Advanced / Legacy area. `subject_distribution_rules_ui.py` re-validates the fixed two-period block arithmetic authoritatively on save using the Stage 1 validator. Saving creates or updates only the intended Grade+Subject (`scope_level="grade"`) row, or a specific section override when a section is selected from the panel's Section Overrides list. Reset To Default removes only the grade-level row so resolution falls back to the next hierarchy tier; Copy Rules From Grade matches subjects across grades by name (since subject codes are grade-specific) and skips any subject whose arithmetic does not fit the target grade instead of copying it blindly. No changes were made to the CP-SAT solver, independent validator, snapshot immutability, or grouped-activity/Swimming JSON authority.

Smart Timetable Academic Quality Rules

Branch/year Timetable Settings store explicit subject-code mappings for core, spread, ICT, double-period, and grouped Swimming behavior. Schema-v3 snapshots carry the normalized rules into the problem builder. CP-SAT requires daily core coverage when weekly demand reaches the teaching-day count, maximizes distinct-day spread for shorter core and configured spread subjects, supports hard ICT one-per-day, and keeps configured Swimming sections simultaneous without weakening section, teacher, or shared-resource protection. The independent validator repeats every new hard check. Regeneration diversity remains configurable with a 25 percent default.

Subject Distribution Rules Generation Wiring

Schema-v3 snapshots now resolve and embed the effective Subject Distribution Rule (section over grade over branch default, `None` for legacy fallback) on every Planning demand at snapshot creation time, so a later rule change never alters an already-created snapshot. The problem builder carries that resolved rule per demand, exposes true physical period adjacency on every slot, and runs the arithmetic/feasibility validator as a final pre-solve defense, failing cleanly with `distribution_rule_invalid` rather than reaching CP-SAT. CP-SAT enforces an exact partition into intentional two-period blocks and single-period sessions. Explicit block-start and single-session decisions channel every occupied period to exactly one session: sessions may touch, so three consecutive periods can be one double plus one single and four can be two touching doubles, but session membership never overlaps. Physical timeline interruptions still break a double. Daily session/load channeling strengthens hard daily-coverage propagation without relaxing its

meaning. CP-SAT also enforces generalized daily-coverage/spread/max-per-day/min-teaching-days hard-or-soft behavior per resolved rule, and exempts declared blocks from the consecutive-avoidance penalty; demands without a resolved rule keep the exact legacy `quality_rules_json` code-list behavior. The independent validator mirrors every new hard check (existence of the same exact non-overlapping session partition via true adjacency, hard daily coverage, hard max/day, hard min teaching days) alongside the unchanged grouped-activity and collision checks. Readiness now blocks genuinely invalid or infeasible normalized configurations before generation while keeping the existing core-subject warning path for legacy-only scopes. Regeneration is unchanged: the same constraint-building path applies during regenerate, so hard distribution rules remain valid while the diversity requirement reshuffles unlocked placements.

Subject Distribution Rules Foundation

A normalized `subject_distribution_rules` table (migration `20260828_003_subject_distribution_rules_foundation`) now exists alongside `quality_rules_json` without replacing it. Rows are scoped `branch_default`, `grade`, or `section` per branch/academic year, with block/single counts, min teaching days, max periods per day, daily-coverage/spread/consecutive preferences, an optional min day gap, and a hard/soft strictness flag. `subject_distribution_rules.py` resolves section-over-grade-over-branch-default precedence with field-level inheritance for nullable fields, and returns `None` when no normalized row exists so the caller keeps today's `quality_rules_json` behavior unchanged. `subject_distribution_validator.py` is a pure arithmetic and feasibility check confirming `block_count * block_length + single_count` equals the authoritative Planning weekly total and that day/period limits are feasible. `timetable_slot_service.py` marks each composed teaching period with true physical adjacency to its next period, false whenever a Break, Prayer, or other non-teaching item is composed between them, so intentional-block enforcement can rely on genuine continuity rather than raw period-index arithmetic.

Smart Timetable Stage 5.2

Stage 5.2 presents Configure, Generate Draft, Review/Edit Draft, and Publish to Users while retaining the Stage 2-5.1 version and solver internals. The primary workspace resolves one exact-scope mutable unpublished Draft Timetable, including a freshly created draft, and uses Approve Draft, Draft Approved, Published, and Timetable History language. Generation never approves a draft. Approval records the reviewing administrator and exact draft state; placement, lock, regeneration, and stale-authority changes invalidate it. Publication requires approval and repeats validation before atomically changing the active pointer. Generated Draft workflow actions are grouped together on the primary workspace; delete, archive, comparison, and version-specific exports remain in History.

Delete Draft Timetable permanently removes an eligible never-published draft and preserves the active pointer. Timetable History may permanently delete every never-published, non-active candidate regardless of origin or internal draft/archive state; dependent unpublished versions are removed child-first while snapshots, generation runs, and audit history are preserved. Protected published lineage and active generation are reported as blockers. The assignable `timetable.delete_versions` permission controls these actions and is granted to Administrators by default. Mutable working lessons may be moved or atomically swapped between canonical teaching slots, with lock, class, and teacher conflicts failing the whole operation. Publication uses explicit first-publish or replacement confirmation and customer-facing Published to Users language. Authorized managers may copy the current Published Timetable into an editable draft or create a fresh empty draft from current Planning/configuration authority. Neither action changes the active published pointer before explicit publication. `timetable_visibility_service.py` resolves official reads strictly through `TimetableActiveVersion`. `/my-timetable` matches scoped `User.user_id` to `Teacher.teacher_id` and shows only that teacher's published lessons. View-only users are redirected away from management drafts and history.

Smart Timetable Stage 5.1

Stage 5.1 implements automatic branch/year timetable generation with Google OR-Tools CP-SAT 9.15.6755 in a separate task dependency environment. HTTP requests capture schema-v3 immutable inputs, commit durable

PostgreSQL `TimetableGenerationRun` rows, and use the server-side Render client to start one on-demand Workflow task with only the run public ID. The task claims that exact row, leases and heartbeats it, supports cooperative cancellation, solves hard constraints, and submits candidates to an OR-Tools-independent validator. Render automatic retry is disabled so TIS terminal state and Generate Again remain the only retry authority.

Generate creates a new unpublished `publication_ready` generated version only in one atomic transaction after current-fingerprint revalidation. Regenerate preserves locked placements, excludes the exact source, and requires 25 percent of unlocked placements to differ, rounded up. An infeasible diversity target fails explicitly rather than returning a nearly identical result. Neither flow changes published history or the active pointer. The generated result remains the logical Draft successor of the fresh source draft; the source is retained for provenance but omitted from normal History presentation. Freshness fingerprints cover Planning, canonical timetable configuration, stable generation constraints, and locks; generation-only source metadata is retained in snapshots but excluded from authority comparison. Safe component-level diagnostics identify real mismatches. `timetable.generate` is independent of publish authority. Stage 5.2 The old polling worker is optional/local only; production requires no always-on solver service. Immediate dispatch failure is terminal and customer-safe, while delayed task start is reported as waiting for compute. Stage 5.2 supplies the simplified workflow, Delete Working Timetable, and teacher My Timetable.

Smart Timetable Stage 3.5

`timetable_slot_service.py` is the single clock-time authority. For every working day it composes the configured shift start, teaching-period count/duration, and applicable non-teaching blocks into ordered teaching/block items. After-period blocks are the default and shift every later period; legacy fixed-time blocks are inserted only at a valid live boundary and otherwise surface a correction blocker. The calculated end, settings preview, timetable grid, readiness, assignment validation, snapshots/fingerprints, and XLSX/PDF exports consume this projection. The additive placement columns preserve existing blocks as `fixed_time`. Stage 3.5 itself added no solver, generation endpoint, availability, or resources; Stage 5.1 now consumes that canonical timeline.

Smart Timetable Stage 4

The timetable page now exposes exact-scope version history and distinguishes Draft, Publication Ready, Active/Published, Superseded, and Archived views. Opening a historical version never changes the active pointer. Operational default resolution remains: newest mutable draft derived from active, otherwise active, otherwise newest mutable scope draft. Immutable versions require an explicit copied draft.

Draft lesson locks are explicit future-regeneration commitments. Placement and lock edits use `edit_revision`; lock edits refresh the version snapshot/fingerprint. Solver-independent draft validation is separate from Stage 3 generation readiness. Atomic publication revalidates freshness/completeness, locks the selected version and active pointer, checks pointer revision, supersedes the old active version, and makes the selected version immutable through the active pointer. Stage 4 itself added no solver or worker; Stage 5.1 now builds generation on this boundary.

Smart Timetable Stage 3

`timetable_slot_service.py` is the single composed-timeline authority for teaching, non-teaching, and invalid slots. Full coverage disables a slot; a block wholly between periods removes no slot; partial overlap is invalid; all-day and every-day rules expand deterministically. Blocks never shift or shorten teaching periods.

`TimetableReadinessService` is read-only, solver-independent, and exact-scope. It combines configuration, Planning demand, explicit/HRT teacher resolution, existing teacher capacity, slot sanity, locks, and Stage 2 fingerprints. Generation-ready means only that a future solve may be attempted, never that feasibility is proven.

Capacity-Based Packaging And Common Customer Features

Starter, Professional, and Enterprise AI now represent organization scale, not progressive access to normal customer modules. Their ceilings remain 1/5/25, 5/20/100, and 25/100/500 for active branches, operational staff users, and

teachers. Paid, active promo, and active customer-demo authority share the normal customer baseline only after source/lifecycle, permission, tenant, branch, and academic-year checks succeed.

`saas/customer_feature_policy.py` classifies that baseline and explicitly excludes the Developer-only audit export. `saas.entitlement_service.evaluate_feature_access` is the shared source-aware decision used by Advanced Reporting. Promo activation and migration `20260819_001_capacity_based_customer_feature_baseline` persist the same baseline without changing exact promo capacities or immutable promo history. Standard, Full, and Custom demos retain the baseline; Custom controls only legitimate scope, safety, experimental, and consumption policy.

Enabled AI features are commercially available across paid tiers, promo, and active demo with `ai.use`. `saas/ai_consumption_policy.py` separately resolves execution allowance. Existing reservations, counters, idempotency, and operation events remain unchanged, and globally disabled AI definitions remain unavailable.

Stage 5 SchoolGroup Authorization And Capacity Presentation

Direct operational SchoolGroup creation and deletion are global platform actions. They require a Platform identity plus `schools.manage_all_schools` and the corresponding `schools.create` or `schools.delete` permission. Tenant roles cannot receive those create/delete permissions, and platform identity is checked again in the handlers so stale permission state cannot cross the boundary. Tenant users with `schools.edit` may update only their linked SchoolGroup; arbitrary IDs fail before validation or mutation. Approved paid, demo, promo, and conversion provisioning remain separate and unchanged.

School Management presents branch usage through `saas/commercial_authority_service.py`, so paid and promo workspaces use the same authoritative source-aware counts without exposing source internals. Branch create, promo assignment, and entitlement remain atomic under the existing SchoolGroup lock. The individual last-active-branch safeguard now counts only the target SchoolGroup. `scripts/audit_schoolgroup_provenance.py` is a PostgreSQL-only, repeatable-read, read-only report for active internal sandboxes that have neither tenant provenance nor explicit workspace entitlement; it never remediates candidates.

M4C Existing Workspace Paid Activation

M4C lets a verified tenant owner activate an existing `customer` workspace in `provisioning` through Paddle without recreating onboarding or tenant data. The durable `ExistingWorkspacePaidActivation` aggregate is anchored to SchoolGroup, workspace UUID, SaaS account, tenant-owner link, selected plan, billing interval, branch snapshot, quote fingerprint, checkout lineage, payment attempt, and a SchoolGroup-anchored subscription contract. Checkout and payment rows use strict pending-organization versus paid-activation contexts.

Professional and Enterprise AI quotes include every active operational branch; Paddle quantity remains active branch count. Starter remains unavailable for existing workspaces until restricted-branch enforcement is proven across all operational entry points. Preparation creates no PendingOrganization, ProvisioningJob, SchoolGroup, or pre-payment entitlement. Organization Account's **Choose a Plan** action always opens plan selection while the activation is editable. A saved `draft` or `checkout_ready` selection is highlighted but may be replaced by another currently eligible plan, which rebuilds the authoritative quote. Plan browsing and draft replacement make no Paddle API call and create no PaymentAttempt. Once checkout is `checkout_started`, `payment_processing`, or in a manual-review/inconsistent state, plan replacement fails closed. Billing identity is stored against SchoolGroup and explicitly associates any reused Paddle customer with the workspace. The association also snapshots the provider address and business used by that workspace, so a shared SaaS account cannot make another workspace's mutable customer defaults authoritative.

Only a verified matching `transaction.completed` event can activate access. In one transaction TIS revalidates identity, quote, capacity, selected branches, provider customer/address/business, price, quantity, currency, interval, transaction, and subscription identity; then it confirms the contract and subscription, creates paid workspace and branch entitlements plus the paid TenantProvisioningLink, and changes lifecycle to `active`. Classification remains `customer`.

`transaction.paid` is processing only, browser return grants nothing, and conflicting or stale evidence fails closed. Returned and reused transactions must be billed, automatic-collection transactions whose item, price, quantity, subtotal, interval, currency, customer, address, business, checkout URL, and full custom-data lineage match the current quote. PostgreSQL lock acquisition refreshes mapped state before duplicate webhook decisions. Existing promo, demo-to-paid, and PendingOrganization checkout paths remain separate.

Organization Account presentation distinguishes commercial activation from payment processing. A coherent `activation_required` workspace with no current payment attempt displays **Activation required**. **Payment processing** requires a current, unexpired existing-workspace `PaymentAttempt` in `checkout_started` or `payment_processing`; failed, cancelled, and expired attempts display recovery states. Workspace lifecycle `provisioning` alone is never payment evidence.

M4B Controlled Existing Workspace Conversion

M4B converts an explicitly audited internal sandbox into a customer workspace that still requires commercial activation. The durable conversion operation is the ownership-claim record: it stores the exact workspace tuple, normalized intended-owner email, approved M4A hash, canonical parameter hash, immutable branch/commercial snapshots, setup state, actors, stage, status, and failure code. Append-only conversion events contain only allowlisted redacted details.

Preparation creates no account and sends no email. The intended owner registers and verifies through the normal TIS Account flow, explicitly claims the approved workspace, and is aligned through the shared operational-owner identity rules. No tenant ownership is granted before verification. The setup review accepts only legal name, an IANA timezone, and educational program; existing display name, country, branches, academic and operational records remain unchanged.

Final execution is PostgreSQL-only and locks the operation and SchoolGroup. It re-audits immediately, requires exact identity and unchanged branch/dependency and sandbox-entitlement evidence, then ends the internal entitlement and sets `customer / provisioning` in one commit. It creates no active entitlement or tenant source. M1 resolves this coherent state as `activation_required`, keeps Organization Account access, and blocks operations until M3 promo activation. Existing-workspace Paddle activation is deliberately deferred; new-onboarding Paddle behavior is unchanged.

M4A Existing Workspace Conversion Audit

M4A adds a generic read-only evidence boundary before any existing internal sandbox can be considered for customer conversion. The service resolves one explicit `SchoolGroup.id`, workspace UUID, exact name, and normalized intended owner email. It inventories workspace metadata, identities, account links, tenant/provisioning evidence, entitlements, paid/demo/promo records, and every reflected branch foreign-key descendant. Logical branch references without a database foreign key are reported separately for manual review.

The production CLI requires PostgreSQL and starts a repeatable-read, read-only transaction. It outputs deterministic sanitized JSON or text, records the transaction settings, and provides a stable SHA-256 snapshot hash. Exit `0` means the audit is coherent, exit `2` requires manual review, exit `3` means the supplied workspace identity does not match, and exit `1` is an execution or configuration failure. Provider references remain presence flags. The CLI always rolls back and performs no conversion, branch change, owner alignment, entitlement or tenant-link mutation, Paddle call, or email. M4A never approves hard deletion or write conversion; incomplete traversal, identity mismatch, duplicate ownership, non-sandbox commercial authority, or missing schema evidence fails closed for manual review. Recommended archival IDs include only active branches with zero direct, transitive, logical, or soft-deleted dependencies and are withheld when unmodeled branch foreign keys exist.

M3 Promo Redemption And Commercial Grants

M3 turns an approved promo definition into commercial authority only after an authenticated, verified organization owner completes a resume-safe activation. The workflow supports either a completed `PendingOrganization` or a

separately aligned existing `SchoolGroup` with a valid tenant-owner account link; it never fabricates pending, payment, subscription, contract, or demo records.

Successful activation creates one immutable `PromoRedemption`, one immutable capacity/plan/scope snapshot in `PromoGrant`, one promo `WorkspaceEntitlement`, explicit branch entitlements and assignments, and a promo-sourced `TenantProvisioningLink`. The link requires exactly one source: paid contract, approved demo request, or promo grant. M1 resolves promo access from that chain. Pending sessions grant nothing, expired or ambiguous grants fail closed, and no Paddle API or billing record participates.

When eligible branches exceed promo capacity, the owner selects exactly the covered branches. Every preserved workspace branch receives explicit evidence: selected branches receive an assignment and active entitlement, while every unselected branch, including an operationally inactive branch, receives an inactive entitlement without consuming grant capacity or changing operational status. Reactivation locks and rechecks M1 capacity, then activates the branch, assignment, and entitlement in one commit. A PostgreSQL-only, dry-run-first reconciliation command can add only deterministically missing inactive evidence; ambiguous or contradictory chains require manual review. Staff and teacher records are never selected or deactivated, and their authoritative counts must fit the grant.

Organization Account resolves the authoritative commercial source before choosing its customer presentation. Paid authority continues through the Paddle-backed subscription portal, demo authority keeps its dedicated journey, and promo authority uses `commercial_portal_service` to compose the central commercial authority with only attributable immutable promo metadata. The promo page shows the grant plan, safe status and dates, masked reference, and authoritative branch/system-user/teacher capacity. Operational promo access ends exactly at `PromoGrant.effective_to`; `grace_period_days` is only a recovery window for Organization Account and paid continuation and never extends operational access.

An expired or recovery-period promo owned by a verified tenant owner can enter the existing-workspace paid activation flow. Checkout preparation and pending payment do not change promo authority. Only verified provider completion locks and revalidates the tenant, quote, capacity, promo source, and paid evidence, then atomically ends the promo entitlement, relinks the existing tenant source to the paid contract, creates paid workspace/branch entitlements, and preserves the SchoolGroup, workspace UUID, branches, users, teachers, data, branding, and immutable promo history. Active-promo early conversion remains blocked because promotional value is not paid credit.

`commercial_badge_service` converts centralized commercial access into one reusable, source-aware Promo, Demo, or Paid badge view model. Organization Account and commercial templates render that prepared identity and never infer authority from a historical contract, grant, plan, or subscription row.

M2 Promo Code Foundation

M2 adds secure, definition-only Starter, Professional, and Enterprise AI promo administration under `/saas-admin/promo-codes`. Exact positive branch, system-user, and teacher capacities must pass the existing SubscriptionPlan ceilings and shared plan-capacity validator. Scope supports global, organization, pending organization, account email, and email domain targets, with coherent reinforcing restrictions and optional existing-branch snapshots.

Raw codes use at least 100 bits of secure randomness and are shown exactly once in a `Cache-Control: no-store` response. TIS stores only an HMAC-SHA256 lookup hash produced with the dedicated `TIS_PROMO_CODE_HMAC_SECRET`, a key ID, and safe display prefix/suffix. Missing promo security configuration fails only promo generation/future lookup, with no insecure fallback. Platform Developers need `promo_codes.view` or `promo_codes.manage`; only Platform Owners activate or terminally revoke. Row locks and allowlisted durable audit protect lifecycle transitions. The M2 definition remains non-authoritative until M3 completes a valid redemption; definition creation itself creates no entitlement, tenant access, Paddle object, or M1 commercial-authority source.

Subscription Capacity Presentation And Billing Entry

Customer subscription pages distinguish confirmed paid branch quantity from the active plan's branch ceiling. Every active branch remains a Paddle quantity unit; the plan ceiling describes only the largest organization the plan can support. System-user and teacher counts remain included eligibility ceilings and never become provider quantity. Review Capacity shows current paid branches, required active branches, additional billed branches, the plan ceiling, and the resulting minimum eligible plan without describing unused plan ceiling as prepaid capacity. The approved commercial feature matrix is common across the three self-service plans, so customer plan cards continue to show plan identity, three capacity ceilings, current/eligibility state, and permitted plan-change actions rather than a feature ladder.

An authorized operational organization owner or user with `subscriptions.manage_billing` sees Billing & Subscription under System Configuration. The bridge revalidates the operational user, selected tenant, SaaS account-user link, and billing authority, then opens the existing Organization Account subscription page. If SaaS account authentication is required, only an allowlisted internal continuation is preserved through password or social sign-in. Multiple-organization selection remains explicit, restricted customers retain permitted recovery access, and external return destinations fail back to Organization Account. The public landing action is labelled Organization Sign In and continues to use centralized `/saas/login`; it never defaults an organization owner into operations.

Organization Account Sign-In Boundary

Public and onboarding SaaS Sign In authenticate through `/saas/login` and then resolve the customer journey centrally. An activated organization owner or a linked operational user with account-management permission lands on `/saas/account`, which is the Organization Account Overview. That overview exposes only permitted Organization Profile, Branches, Billing & Subscription, and Account & Security sections. `/login` is never the default destination for an authorized organization account manager; only the explicit Enter TIS Platform action enters the operational authentication flow, where commercial access and operational permissions are checked again.

Incomplete onboarding still resumes its authoritative setup step. Restricted or suspended account managers remain in Organization Account with permitted billing/recovery access and no active Enter TIS action. A linked operational user without account-management permission follows the existing role-based operational destination and receives no organization-owner or billing controls. When one SaaS identity manages multiple organizations, `/saas/account` requires organization selection before rendering the selected account overview. The selected organization UUID is retained in an HTTP-only cookie, revalidated against the account's tenant links and permissions on every request, and then supplied to the existing entitlement resolver so Subscription Management uses the selected tenant without weakening isolation. Password login, social login, already-authenticated sign-in, SaaS root restoration, and post-verification sign-in all use this same resolver.

Initial Checkout Retry And Lineage Safety

The diagnosed Secure Payment failure was a local readiness rejection before any Paddle API request. `_ensure_checkout_launchable()` rejected the legacy `ready_for_checkout` pre-checkout billing state even though it could be prepared safely. The verified Professional Annual catalog mapping remains USD 790 per active branch; two branches therefore produce Paddle quantity 2 and an authoritative annual total of USD 1,580.

Initial Secure Payment authority is the current server-built quote plus its plan selection, checkout session, and payment attempt lineage. A plan, interval, branch quantity, capacity estimate, provider price, or quote fingerprint change supersedes unfinished local sessions and attempts. Late webhooks for those attempts are retained for review but cannot activate a subscription or workspace.

Retry Secure Payment revalidates otherwise eligible unpaid onboarding state, normalizes legacy pre-checkout billing status, and creates a fresh checkout when the prior transaction is missing, incomplete, non-billed, or mismatched. Compatible active Paddle customer addresses are reused by country. Only an automatic, billed, launchable transaction for the current customer and quote may be released to the public payment launcher. Provider diagnostics and readiness failures remain logged server-side with tracebacks; customers receive one structured safe alert without provider or internal details.

Organization Billing Identity

Organization billing identity is explicit persisted organization state, not an alias for the authenticated user's login identity. `OrganizationBillingProfile` owns the confirmed billing email, legal/billing name, contact, optional company and tax identifiers, and the supported billing address fields. Initial checkout requires this profile. Authorized organization owners or users with `subscriptions.manage_billing` may update it from Subscription Management; changing it does not change login credentials, and changing login credentials does not change billing identity.

`PaymentCustomer` persists the mapped Paddle customer plus `provider_address_id` and `provider_business_id`. TIS synchronizes explicit billing changes to the mapped customer, reuses or updates one tenant-attributable active Paddle Business, updates the active subscription's customer/address/business identity, and includes the mapped business in new initial transactions. Ambiguous or failed synchronization fails closed with customer-safe guidance and server-side diagnostics. Existing financial documents are never silently rewritten; updated details primarily apply to future billing documents.

Billing Contact is read-only until an authorized user explicitly selects Edit. Valid local changes remain saved if Paddle synchronization fails. A dedicated, tenant-scoped retry reuses the stored profile and existing customer, address, and business mappings; an already synchronized retry is a provider-call-free no-op. Provider failures are logged by safe step (`customer`, `address`, `business`, or active-subscription identity) with provider status and error code, while the customer sees no provider identifiers or diagnostics. Once a saved profile is pending or failed, new plan and quantity mutations fail closed until synchronization succeeds; cancellation and legacy subscriptions that have no saved billing profile retain their existing behavior.

Paddle transaction states remain distinct. `paid` records that money was received and is displayed as `Payment received - processing`; it does not complete subscription reconciliation. `completed` remains the authoritative processed-payment signal and is displayed as `Paid`.

Organization Profile Save And Pending Logo Safety

Organization Profile accepts National, International, and Both through a controlled selector and normalizes those values to the existing uppercase codes. Missing or invalid customer input returns the same page with preserved form values. Pending organization logos reuse the established image decoder and 4 MB limit, allow PNG/JPG/WEBP only, enforce minimum dimensions, and use an opaque unique filename plus atomic file replacement. A database or later save failure rolls back organization changes and removes the newly written file; an empty upload retains the current logo.

Pending logos currently use the application-local `static/uploads/saas/pending_logos` directory and store only a relative public path in `PendingOrganization`. This is not durable on an ephemeral production filesystem. Persistent disk or object storage requires a separate deployment and storage-architecture decision; this correction does not introduce one. The initial Account Setup state is compact: one setup title, one explicit POST start action, and the existing eight-step journey.

Branch Setup never totals nullable estimates in Jinja. The service normalizes legacy/missing display values to zero in Python, while browser saves still require explicit non-negative whole numbers for every active branch. New pending branches receive explicit zero defaults.

After a pending logo is saved, Organization Profile and the shared School Workspace identity area show it beside the organization name while retaining the official TIS logo as platform branding. Checkout keeps the pending reference. Paid and demo provisioning already converge on `create_workspace_records()`, which copies the validated pending image into the established primary `SchoolGroupLogo` slot. The operational shell renders that organization-owned asset through its protected route and keeps the TIS logo separate. A referenced pending file that is missing at activation now fails provisioning instead of silently activating without the customer logo.

Post-Verification Sign-In Safety

Successful email verification redirects by HTTP 302 to the GET `/saas/login` page. The form submits by POST to `/saas/auth/login`. A newly verified account with no School Workspace Setup record continues to the GET `/saas/account` dashboard, where the existing explicit start action may create the setup record by POST. Login continuations accept only known customer GET destinations; POST-only, malformed, traversal, and external targets fall back to Account Setup. A defensive GET request to `/saas/auth/login` redirects to the normal sign-in page instead of exposing FastAPI's raw 405 response.

Public Subscription Entry

The landing hero Subscribe Now action scrolls to the public pricing section. Starter, Professional, and Enterprise AI then enter the public TIS Account signup route with an allowlisted preferred-plan code. That preference is non-authoritative: it creates no plan selection, checkout, payment attempt, or Paddle object, and it is applied only after School Workspace Setup confirms the plan remains eligible across branches, system users, and teachers. Invalid or undersized preferences are ignored or cleared safely. Custom remains a contact-only path.

Three-Dimension Subscription Capacity

Self-service subscription eligibility is organization-wide across three independent persisted limits: branches, tenant operational staff users, and teacher records. Branch Setup stores required non-negative system-user and teacher estimates on each active onboarding branch; legacy organization totals are assigned to the primary branch only when no branch estimates exist, after which organization totals are derived summaries.

Before confirmed payment, capacity authority is the active onboarding branch count plus the greater of each branch-estimate total and the same workspace's actual active count. After activation, actual active tenant operational users and active teacher records are authoritative. Every distinct active tenant User counts as staff regardless of role, title, position, or internal-test attribution. An operational teacher-user with an active Teacher record consumes one staff slot and one teacher slot. Platform users and account-only SaaS identities do not consume tenant staff capacity. Starter is 1/5/25, Professional 5/20/100, and Enterprise AI 25/100/500. Exceeding any Enterprise AI limit requires the contact-only Custom path and cannot create Paddle checkout.

Active Subscription Management uses that same three-dimension authority. The Review Capacity flow accepts proposed branch, system-user, and teacher totals, selects the minimum eligible plan from all three, and records the capacity snapshot on a generated plan-change preview. Branch growth may therefore produce one combined plan-and-quantity upgrade, while system-user and teacher growth affects plan eligibility only. Paddle quantity always remains the branch quantity; user and teacher counts never become billable units.

Plan downgrades remain scheduled for the next billing boundary and are checked against all three live operational counts both before submission and again when provider evidence says the downgrade is effective. If capacity no longer fits, local activation stops in manual review and the current safe entitlement state is preserved. Quantity reductions receive the same effective-date capacity check. Cancellation remains provider-scheduled at paid-period end, preserves tenant data and access through that date, and may be reversed while provider lifecycle rules allow it.

M1 enforces post-activation growth through `saas/commercial_authority_service.py`. The facade composes, rather than duplicates, workspace classification/lifecycle, WorkspaceEntitlement, TenantProvisioningLink, SubscriptionContract, PaymentSubscription, commercial state/access, demo lifecycle, and plan capacity. Paid branch capacity is the confirmed subscription quantity capped by the plan branch ceiling. Known teacher IDs are normalized and deduplicated; each blank legacy teacher row counts independently. Customer Demo and Internal Sandbox remain explicitly unmetered in M1, while missing or contradictory authority fails closed.

Capacity-increasing branch, user, teacher, academic-year, and provisioning paths lock the owning SchoolGroup, recount, evaluate the proposed final state, mutate, and commit in one transaction. Existing over-capacity workspaces keep their data and access but cannot further increase an exceeded dimension.

Returning Customer Journey And Commercial Expiry

Returning SaaS-account login resolves authoritative onboarding, demo request, account-to-workspace, workspace classification, lifecycle, and subscription evidence. Incomplete onboarding resumes, pending demos open request status, active demo or paid tenants enter operational login, unpaid customers reach subscription setup, and expired customers receive branded expiry guidance. Operational login and protected requests run the commercial guard before branch, academic-year, or workspace page work; expected expiry never becomes a 500.

Paid-workspace access uses the same tenant-link and SubscriptionContract-linked `PaymentSubscription` selected by `saas.entitlement_service`. A newer stale, orphaned, or unrelated organization-level subscription row cannot replace that authority, and Subscription Management and operational access consume that same resolution. An unresolved, pending, failed, expired, canceled, or abandoned plan upgrade remains a separate change request and does not revoke the currently active plan. For example, active Professional access remains available while an Enterprise AI upgrade is `payment_pending`. The specialized plan-change `subscription.updated` handler synchronizes provider subscription status while retaining the existing two-signal rule before the target plan becomes authoritative.

For a production subscription whose provider state is authoritative but whose local status is stale, replay the attributable stored, signature-verified Paddle webhook through the existing webhook reconciliation path. Do not repair commercial status fields manually. Confirm that `subscription.updated` synchronizes the current `PaymentSubscription.status`; a pending plan change must still wait for its separate required provider payment signal before the target plan becomes authoritative.

Commercial access distinguishes active, trialing, payment processing, past due, paused, canceled within a paid period, expired, suspended, archived, and inconsistent states. Canceled subscriptions retain current entitlements only through their confirmed current-period end. Ambiguous evidence fails closed and uses verification guidance rather than falsely claiming expiration. Renewal guidance appears only for a genuinely expired commercial state.

Expired demos select real public plans and intervals against the existing organization and actual active operational branches. Checkout and confirmed payment reuse the existing Paddle and conversion authorities, preserving the SchoolGroup, workspace UUID, tenant link, users, branches, permissions, and data. The Next.js landing derives Request Demo, Subscribe, Sign In, and Open TIS App from `NEXT_PUBLIC_TIS_APP_BASE_URL`. Customer communications name the TIS team; internal role names and audit evidence remain unchanged.

M8B9 Demo Operations And Access Profiles

M8B9 adds Platform Owner-only orchestration in `saas/demo_operations_service.py` and policy resolution in `saas/demo_access_service.py`. Owners can expire, reactivate, set an unbounded future expiry, send final-day reminders, invoke the existing lifecycle processor, and choose Standard, Full, or Custom access at workspace or tenant-validated branch scope. M8B8 usage history is never reset; permissions, commercial lifecycle, and feature entitlement remain separate fail-closed checks. Material changes reuse M8B7 communications and every attempt is durably audited.

M8B8 AI Entitlement Foundation

M8B8 centralizes AI access in `saas.ai_entitlement_service`. A controlled registry owns stable feature identifiers and temporary reviewed plan mapping. Customer Demo receives two successful uses per feature; pre-execution reservations block concurrent excess attempts, while failed/no-result work releases capacity without consuming. Durable counters and operation events are scoped by SchoolGroup, feature, and separate internal/demo/paid metric context. Existing tenant resolution, `ai.use` permission, commercial state, demo expiry, and paid `module.ai` entitlement remain distinct authorities. No AI business tool or M8B9 operational control is included.

M8B7 Demo Customer Journey

M8B7 makes normal Platform Owner approval orchestrate the existing independently retryable provisioning service. It adds a durable branded demo email outbox, deduplicated Platform Owner Notification Center events, a shared-shell

active-demo indicator, atomic Day 6/expiry communication intents, and coherent expired-demo conversion after authoritative confirmed payment. The same PendingOrganization and then exactly one SchoolGroup are preserved. Production must schedule `python scripts/process_demo_lifecycle.py --apply`; dry-run remains the default. M8B8 and M8B9 are not included.

This is the first file future Codex or ChatGPT coding conversations should load. It is a compact project onboarding reference; detailed source of truth remains in the other Markdown docs.

What TIS Is

TIS is Teacher Information System, a developing SaaS academic operations platform for schools and school groups. It connects teacher information, staffing and workload planning, academic calendars, observations, branch context, SaaS onboarding, billing, provisioning, and future AI-assisted academic decision support.

Public URLs:

- Public website: `https://tisplatform.com`
- Application portal: `https://app.tisplatform.com`
- Render must set `TIS_PUBLIC_BASE_URL=https://app.tisplatform.com` so transactional emails and background Workspace Activation emails generate production login and static asset URLs instead of local-development fallbacks.

Important routes:

- Operational login: `/login`
- SaaS signup: `/saas/signup`
- SaaS login: `/saas/login`
- SaaS account: `/saas/account`
- Platform console: `/platform`

Current Architecture

The operational app is a FastAPI application at the repository root.

The FastAPI web process never creates tables or runs pending migrations during import, startup, middleware, or background threads. Render must run `python scripts/run_migrations.py` as its Pre-Deploy Command; only after that command succeeds may Render activate the new web version. The Start Command is `uvicorn main:app --host 0.0.0.0 --port $PORT`, which constructs the app, performs only lightweight process-local startup checks, and binds without PostgreSQL DDL. This boundary applies consistently to production, local development, and tests.

Key files and folders:

- `main.py`: primary FastAPI app and many route handlers.
- `auth.py`: authentication, roles, platform identity helpers, permissions, sessions.
- `authorization.py`: protected route rules and access-denied handling.
- `permission_registry.py`: permission keys, groups, defaults, and developer-assignable permissions.
- `location_service.py`: global location picker lookup/validation. Must stay memory-conscious and scoped; do not restore full unbounded dataset parsing for normal picker requests.
- `ui_shell.py`: shared app shell/navigation/page metadata.
- `models.py`, `database.py`, `db_migrations.py`: data model, DB setup, local schema repair/migration logic.

- `routers/`: modular operational routes.
- `saas/`: SaaS account, onboarding, payment, billing, and provisioning services/routes.
- `saas/entitlement_service.py`: provider-confirmed commercial entitlement and paid branch-capacity resolution.
- `saas/commercial_authority_service.py`: unified operational commercial access, capacity counters, structured decisions, and tenant row locking.
- `saas/commercial_portal_service.py`: source-aware customer commercial presentation for promo-backed workspaces without duplicating commercial decisions.
- `saas/subscription_portal_service.py`: customer subscription portal view model.
- `saas/subscription_change_service.py`, `saas/subscription_plan_change_service.py`, and `saas/subscription_cancellation_service.py`: provider-authoritative quantity, plan, and cancellation workflows.
- `saas/subscription_lifecycle_service.py`: centralized lifecycle state and allowed-action policy.
- `saas/billing_history_service.py`: provider-sourced billing history and short-lived invoice download resolution.
- `saas/billing_identity_service.py`: explicit organization billing profile validation and Paddle customer/address/business synchronization.
- `saas/payment_lifecycle_reconciliation_service.py`: guarded reconciliation for attributable finalized payment evidence.
- `scripts/sync_paddle_price_ids.py`: environment-specific Paddle initial checkout price mapping sync into `subscription_plan_prices.provider_price_id`.
- `templates/`: Jinja templates.
- `static/`: static assets and generated documentation output.
- `tests/`: pytest coverage.

The public marketing website is separate:

- `tis-landing-website/`
- Next.js / Node runtime
- Source of truth for public landing implementation
- M8 landing integration exposes two public conversion paths through `NEXT_PUBLIC_TIS_APP_BASE_URL`: Request a Demo routes to `/saas/signup?intent=demo`, while Subscribe Now routes to `/saas/signup?intent=subscribe`.
- The selected intent is preserved through TIS Account signup and School Workspace Setup, then emphasized on the customer-safe commercial-choice page without removing the customer's ability to choose either path.
- Customer demo eligibility is reserved once per normalized organization domain. Existing Customer Demo history, including expired or converted demos, remains ineligible for a second demo; internal sandbox history does not consume customer eligibility.

Legacy FastAPI landing files are not the source of truth:

- `templates/landing.html`
- `static/landing/landing.css`

Engineering Handbook

For deeper onboarding, read:

```
docs/engineering/README.md
docs/engineering/TIS_MODULE_MAP.md
docs/engineering/REPOSITORY_ARCHITECTURE.md
docs/engineering/USER_AND_SYSTEM_FLOWS.md
docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
docs/engineering/DEVELOPMENT_STANDARDS.md
docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
docs/engineering/PRODUCT_ROADMAP.md
docs/engineering/REJECTED_DECISIONS.md
docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md
docs/engineering/AI_OPTIMIZATION_GUIDE.md
docs/engineering/PROJECT_GOVERNANCE.md
docs/engineering/KNOWLEDGE_LIFECYCLE.md
docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
docs/engineering/SELF_EVOLVING_WORKFLOW.md
docs/engineering/AI_CODING_WORKFLOW.md
```

These files explain module ownership, repository boundaries, end-to-end flows, and what must not be changed casually.

Paddle Initial Checkout Configuration

Initial subscription checkout uses Paddle price IDs stored in `subscription_plan_prices.provider_price_id`. Use `scripts/sync_paddle_price_ids.py` with a structured sandbox or production mapping JSON to configure these values. Paddle credentials and endpoints remain environment variables. Real mapping files are ignored by Git; keep sandbox and live provider price IDs separate. If a mapping is missing, checkout fails closed before Paddle is called and customers receive a support-oriented Secure Payment message while internal diagnostics keep plan code, billing interval, and currency context.

Paddle transaction checkout uses a dedicated public SaaS payment launcher at `/saas/payment`. Configure `PADDLE_CHECKOUT_BASE_URL` to that page so Paddle returns transaction checkout URLs with `_ptxn` appended to the launcher instead of the operational app root. The launcher uses Paddle.js with `PADDLE_CLIENT_TOKEN` and `PADDLE_ENVIRONMENT`; never expose `PADDLE_API_KEY` in HTML or JavaScript.

Domains And Routing

The public website lives at `https://tisplatform.com`. The app portal lives at `https://app.tisplatform.com`.

SaaS account routes are under `/saas`. Platform-owner SaaS administration routes are under `/saas-admin`. Operational tenant workflows use routes such as `/dashboard`, `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, and `/observations`.

Completed M1-M5 Milestones

M1: SaaS identity foundation and separation between platform, tenant, and SaaS account identities.

M2: SaaS onboarding flow for organization, contacts, branches, academic setup, and review.

M3: Billing and plan foundation with plan catalog, checkout, billing status, and payment service boundaries.

M4: Tenant provisioning foundation with pending organizations, provisioning jobs, retry/run actions, and platform owner oversight.

M5: Platform access and owner controls, including platform owner/developer identities, permissions, and platform console behavior.

Platform Owner pending-organization views use a centralized owner lifecycle projection. The pending queue includes only records still in draft/setup, review, subscription checkout/payment, or incomplete/recoverable workspace activation. An active tenant link plus one coherent confirmed subscription, payment, contract, and active SchoolGroup resolves as Active Tenant and is excluded from pending counts. Completed-provisioning evidence that does not reconcile across those records is excluded from the normal queue and shown as Lifecycle Review Required in retained Organization Records. Historical onboarding fields are not rewritten by this projection.

Completed M7 Subscription Management

M7 includes a normalized entitlement foundation, a customer Subscription Management portal, paid branch-quantity management, upgrades and scheduled downgrades, Paddle-authoritative previews and proration, scheduled cancellation and reversal, a centralized lifecycle/action policy, provider-sourced billing history, protected invoice downloads, and webhook/reconciliation safeguards.

Commercial state fails closed when ownership, provider evidence, or local relationships are ambiguous. TIS does not independently calculate replacement monetary values. Immediate changes require provider-confirmed outcomes; scheduled changes remain pending until provider/webhook evidence reaches the effective boundary.

M8B-1 Workspace Classification Foundation

M8B-1 adds metadata-only workspace classification without changing customer behavior. `SchoolGroup` now owns an immutable workspace UUID, classification (`internal_sandbox`, `customer_demo`, or `customer_paid`), and lifecycle (`provisioning`, `active`, `suspended`, or `archived`). `PendingOrganization.workspace_intent`, `SaaSAccount.account_purpose`, and `User.is_internal_test_identity` preserve the corresponding pre-provisioning and identity intent.

All records that predate M8B-1 are confirmed test data. The controlled backfill classifies them as internal sandbox/test records; it is dry-run by default, transactional, idempotent, and records completion in `schema_migrations`. The separate diagnostic is read-only and reports tenant, onboarding, and Paddle relationship presence without exposing provider identifiers or secrets. Platform Owners can inspect workspace UUID, classification, and lifecycle in `/platform`; Platform Developers and tenant users do not receive that metadata block.

M8B-1 does not use classification for billing, entitlements, permissions, tenant isolation, reset eligibility, or customer routing. Demo workflows, conversions, Al-Andalus migration, memberships, and commercial-state resolution remain later work.

M8B-2 Commercial State And Entitlement Foundation

M8B-2 adds a read-only commercial decision layer without changing customer access or billing behavior. `WorkspaceEntitlement` is the workspace-level entitlement envelope, `WorkspaceEntitlementValue` reuses the existing normalized entitlement catalog for features and limits, and `BranchEntitlement` optionally records whether a branch inherits, is explicitly active, or is commercially inactive.

The commercial resolver combines persisted workspace classification/lifecycle metadata with one coherent effective workspace entitlement. Customer-paid workspaces additionally require the existing M7 confirmed subscription entitlement resolution and matching `PaymentSubscription`; M8B-2 does not calculate billing state or contact Paddle.

Missing, ambiguous, cross-tenant, orphaned, or invalid relationships fail closed to manual review.

These results are visible only to Platform Owners in `/platform`. They are not wired into tenant authorization, feature restrictions, branch behavior, onboarding, Paddle, demo expiration, or customer workflows. Internal sandbox workspaces created outside the migration retain a compatibility-only implicit entitlement so existing development flows remain unchanged.

M8B-3 Demo Request Workflow

M8B-3 adds the first customer-facing commercial choice after completed onboarding: Request Demo or Subscribe Now. Subscribe Now continues into the existing plan-selection and Paddle workflow unchanged. Request Demo validates the verified account and completed organization, contact, academic, and branch setup before creating a `SaaSDemoRequest` in Pending Review.

The request records the requester, pending organization, submission time, customer-demo classification intent, provisioning commercial-state snapshot, and a fail-closed pre-provisioning entitlement snapshot. It does not create a SchoolGroup, workspace entitlement, subscription, checkout, or Paddle record. The workflow is separate from the legacy public marketing `DemoRequest` lead table.

Platform Owners have a searchable and sortable review queue. Approval records a review decision only; rejection requires a reason; owner cancellation and customer withdrawal are limited to Pending Review. Every transition creates durable audit and internal-notification events, but M8B-3 sends no email and performs no demo provisioning or activation.

M8B-4 Demo Workspace Provisioning And Activation

M8B-4 adds a Platform Owner-only provisioning action for an approved SaaS demo request. The service fails closed unless the approval review, organization, customer-demo intent, pre-provisioning commercial snapshot, and entitlement snapshot remain coherent and no workspace has already been provisioned.

Demo provisioning reuses the paid provisioning engine's shared workspace-record builder for the SchoolGroup, branches, academic year, owner user, account link, role permissions, and branding. It creates no Paddle, payment, subscription-contract, payment-subscription, checkout, or billing record. A demo-backed `TenantProvisioningLink` identifies the operational tenant without fabricating a paid contract.

Workspace records, the demo entitlement, tenant link, request linkage, and activation are committed atomically. A failed attempt rolls the workspace changes back, leaves the request Approved and unprovisioned, and records a retryable failure outcome. Successful activation sets the SchoolGroup and demo entitlement active, records activation metadata and audit/internal events, and prevents duplicate provisioning. M8B-4 sends no email and implements no expiration, scheduler, login restriction, or conversion behavior.

M8B-5 Standard Customer Demo Lifecycle

Every customer-demo workspace lasts exactly seven days from the M8B-4 `activated_at` timestamp. Day 6 reminder and Day 7 expiration timestamps are derived from that single authority, calculated with timezone-aware UTC values, and displayed in the onboarding organization's IANA timezone. Missing or inconsistent timestamps fail closed.

`saas.demo_lifecycle_service` is the authoritative read-only resolver and independently callable processor. It resolves Active, Reminder Due, Expired, Suspended, or Manual Review; creates idempotent internal Day 6 notifications for the requesting SaaS account and active Platform Owners; and atomically ends the demo entitlement and suspends the SchoolGroup at expiration. Workspace users, branches, and all tenant data remain preserved.

The operational authentication middleware checks customer-demo commercial access on every protected request, including existing sessions. Active and reminder-due demos continue normally. Expired or ambiguous demos are redirected to the subscription activation experience, while protected API/download requests receive a customer-safe 403. Platform users, paid workspaces, internal sandboxes, public/authentication routes, secure logout, and SaaS

subscription routes are unaffected. Access-block auditing is deduplicated.

Run the lifecycle processor safely with `python scripts/process_demo_lifecycle.py --dry-run`; use `--apply` only for scheduled execution after validating the report. M8B-5 sends no email and adds no extension, conversion, archive, deletion, or read-only expired mode.

M8B-6 Demo-To-Paid Conversion

An active Customer Demo may enter the existing subscription checkout without closing or recreating its operational tenant. After `transaction.completed` establishes one confirmed active M7 subscription for the same pending organization, `saas.demo_conversion_service` atomically converts the existing SchoolGroup from `customer_demo` to `customer_paid`.

The conversion preserves the workspace UUID, SchoolGroup, tenant link row, organization, branches, users, permissions, academic data, and all demo/request/audit history. It ends the demo entitlement, creates a paid entitlement linked to the confirmed `PaymentSubscription`, moves branch entitlement links, and replaces the tenant link's demo source with the confirmed `SubscriptionContract`. Existing M7, workspace-entitlement, and commercial-state resolvers must then resolve the same tenant as Customer Paid Active.

Conversion Requested, Processing, Completed, and Failed states are durable and retryable. Any conversion failure rolls back workspace mutations while retaining confirmed provider payment records. Expired, invalid, ambiguous, cross-tenant, internal-sandbox, paid, or already-converted workspaces fail closed. Completed conversions no longer enter demo reminder or expiration processing.

Test Workspace Reset Dependency Rule

The Platform Owner-only test workspace reset keeps its existing preflight and single-transaction rollback behavior. Before it deletes scoped operational `User` records, it deletes `SubscriptionChangeRequest` rows scoped by the selected workspace's authoritative `school_group_id`. It also deletes selected workspace-entitlement values and branch-entitlement children before the selected `WorkspaceEntitlement`, which is removed before the final `SchoolGroup`. This prevents user, account, contract, subscription, and entitlement foreign-key references from blocking removal while preserving records belonging to every other workspace. Pre-analysis and structured deletion diagnostics report the same scoped record set.

The same Platform Owner-only test reset is a clean-room exception for internal M8 testing only. Within its existing single transaction, it deletes the selected pending organization's linked demo request, domain-eligibility reservation, review and audit history, demo provisioning/lifecycle records, and demo-to-paid conversion history before removing their parents. It also resolves the selected organization's domain through the same customer-demo resolver and may remove a detached reservation only when no other organization, request, workspace, or customer account uses that domain and the reservation has no historical manual-review evidence. Analysis exposes a separate list of safe detached reservation IDs; deletion consumes that exact list, flushes before parent deletion, and verifies that none of those IDs remain. A mismatch, remaining row, conflicting ownership, or historical ambiguity blocks or rolls back the reset. This allows the same test email and organization domain to complete a new internal journey after a reset. The normal customer one-demo-per-domain reservation policy is unchanged; global plans, prices, Paddle records, platform configuration, and records from other organizations remain preserved.

Historical detached demo-domain reservations whose owning organization and account were already removed are managed separately through the Platform Owner-only `/saas-admin/demo-eligibility-maintenance` workflow. It scans eligibility rows, reuses the authoritative domain normalization/resolution rules, and fails closed when any matching organization, account, request, tenant-profile workspace, provisioning record, subscription evidence, conversion, or manual-review marker remains. A confirmed deletion locks and re-analyzes one exact eligibility ID, deletes only that ID, flushes, verifies absence, and commits through the owner route. Durable audit records identify the owner, eligibility ID, domain, previous status, timestamp, and maintenance reason. This administrative repair path does not alter normal Customer Demo eligibility or the clean-room reset.

Current SaaS Account Verification State

Phase 1 TIS Account email verification recovery is accepted. Valid verification links now mark the SaaS account email verified/active and redirect to the TIS Account login page with a professional success notice so the customer can continue school workspace setup.

Expired or invalid verification links no longer dead-end. They show a recovery page with a resend verification form. Resend verification handles unverified accounts, already verified accounts, and unknown email addresses with safe customer-facing messaging that does not reveal account existence. Password-based accounts that remain unverified are blocked from starting or continuing school workspace setup.

This Phase 1 verification recovery work did not change payment, billing, provisioning, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work and was not implemented.

Current SaaS Customer-Facing Language State

Phase 2 TIS Account wording cleanup is accepted for customer-facing account and school workspace setup pages. Customer-visible account/setup pages now avoid presenting "SaaS" and technical identifiers as product language, while internal `/saas` routes, modules, models, and stored statuses remain unchanged.

The customer journey should use professional labels such as "TIS Account", "Account Dashboard", "School Workspace Setup", "Organization Profile", "Branch Setup", "Academic Setup", "Subscription Setup", "Secure Payment", and "Workspace Activation". Customer templates should label internal billing, payment, onboarding, and activation statuses through customer-safe display labels instead of raw database statuses such as `tenant_active`, provisioning states, checkout session states, provider identifiers, plan IDs, school group IDs, attempt UUIDs, or provider subscription/transaction IDs.

The shared TIS Account customer shell uses an official TIS logo image so customer account/setup forms inherit official branding. The light account shell uses the full-color horizontal logo variant, and transactional account emails use an existing official dark-blue wordmark asset. This wording/logo pass did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work and was not implemented.

Current TIS Account Guided Setup Framework State

Phase 3A shared guided setup framework is accepted for the TIS Account dashboard only. The customer account shell now supports a shared setup console with the official TIS logo, an 8-step journey stepper, a current-step/status area, one primary next action, and concise guidance. The account page uses this framework to answer "What should I do next?" without dashboard statistics.

The 8 customer-facing journey steps are TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform. The display state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

This Phase 3A work did not redesign onboarding forms, subscription/payment pages, or billing/status pages. It did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal `/saas` route names, or admin views.

Phase 3B redesigned the five School Workspace Setup onboarding pages on top of the Phase 3A shared shell: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup. The pages now use consistent guided-wizard sections, a single shared-shell primary CTA, secondary Back/Save Draft actions, concise guidance, and lower visual clutter while preserving all existing form actions, field names, validation behavior, draft behavior, and onboarding state transitions.

Phase 3B did not redesign subscription/payment/billing/status pages and did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal `/saas` route names, or admin views.

Phase 3C redesigned the customer-facing Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages on top of the Phase 3A shared shell and Phase 3B guided style. These pages now use one shared-shell primary CTA, customer-safe payment/activation labels, clearer browser-return messaging, and explicit guidance that TIS Platform access becomes available after Workspace Activation.

Phase 3C did not change payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal `/saas` route names, stored statuses, or admin views.

Current Priority

Current priority is automatic KMS synchronization enforcement and reliable post-M7 engineering context:

1 Markdown is source of truth.

1 PDF is generated snapshot.

1 Change history preserves chronological change context.

1 ADRs preserve major decisions.

1 Module history preserves deeper area-specific evolution.

1 Platform Owner Knowledge Center is implemented as a read-only owner utility with manifest-backed document titles, summaries, logical groups, client-side search, category/module/freshness filters, and latest-activity ordering.

1 The Knowledge Center uses protected routes for PDF view/download and source-specific `pdf_page` deep links; it does not link directly to static PDF paths.

1 KMS v3.0 Phase 3A adds a true engineering handbook with module map, repository architecture, workflows, and developer onboarding.

1 KMS v3.0 Phase 3B adds database architecture, development standards, UI/UX philosophy, product roadmap, and stronger human/AI developer guidance.

1 KMS v3.0 Phase 3C adds rejected decisions, visual documentation framework, AI optimization guidance, project governance, and decision traceability.

1 KMS v3.0 Phase 3D completes KMS v1.0 lifecycle standards, dependency mapping, AI coding workflow, and future automation roadmap.

1 Root `AGENTS.md` makes KMS onboarding mandatory for Codex tasks.

1 `.kms-impact.yml` is the machine-readable task declaration.

1 `scripts/kms.py sync` is the single local synchronization command; it validates KIA before writing, regenerates the PDF/manifest, then verifies freshness.

1 `scripts/kms.py check` is the single read-only local and CI validation command.

1 `scripts/check_kms_impact.py` validates major-change classification, declared Markdown updates, and generated-artifact freshness.

1 GitHub Actions block pull-request integration and production deployment when KMS validation fails.

Smart Timetable Architecture Baseline

ADR 0026 governs timetable evolution. Planning remains authority for section demand, subject requirements, assigned teachers, and HRT fallback; one placement means one teaching period, and `Subject.weekly_hours` remains the compatibility weekly-period authority. Timetables are durable SchoolGroup/Branch/Academic-Year versions. Active selection is a separate unique exact-scope pointer, not a version Boolean. Active, superseded, and archived history is immutable; drafts are mutable.

Stages 2-5.1 provide models, migrations, snapshots/fingerprints, lock persistence, version services, readiness, comparison, validation/publication, durable generation, and version-aware views/exports. Stage 3.5 supplies one composed per-day timeline and solver-ready teaching slots. Stage 5.1 adds worker-isolated CP-SAT, independent candidate validation, Generate/Regenerate actions, and atomic unpublished results. Existing and published placements remain unchanged unless the separate publication flow is used. Availability, room/resource, preferences, quality scoring, and broader Stage 5.2 UX remain future work.

Critical Rules

1. Do not touch SaaS flows unless explicitly approved.
1. Do not touch operational logic unless required by the approved task.
1. Do not touch database migrations or `tis.db` unless explicitly approved.
1. Do not weaken tenant isolation.
1. Do not bypass permissions or platform owner checks.
1. Do not merge platform user, SaaS account, and tenant user concepts.
1. Do not change landing page implementation unless explicitly approved.
1. Do not add a KMS regenerate button until explicitly approved.
1. Do not push or commit unless explicitly requested.
1. Treat production memory as a hard budget. Do not add unbounded full-dataset caches, duplicate production template renders, startup-heavy work, or warning-level debug spam on normal requests.

KMS Policy

Every implementation must include a Knowledge Impact Assessment:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

If included docs change, synchronize:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Development Workflow

1. Read this file first.
2. Read root `AGENTS.md`, `docs/TIS_MASTER_CONTEXT.md`, and `docs/PROJECT_STATE.md`.
3. Read `docs/engineering/README.md`.
4. Read `docs/engineering/DEVELOPMENT_STANDARDS.md`.
5. Read `docs/engineering/AI_OPTIMIZATION_GUIDE.md`.

6 Read `docs/engineering/AI_CODING_WORKFLOW.md`.

7 Read relevant engineering docs, ADRs, module history, and supporting docs.

8 Inspect code before editing.

9 Keep changes scoped.

10 Update `.kms-impact.yml` and affected KMS docs when meaningful behavior, architecture, product state, module map, repository ownership, data model, design philosophy, roadmap, governance, decision traceability, automation, lifecycle, or workflow changes.

11 Run `scripts/kms.py sync` if included source docs changed.

12 Run `scripts/kms.py check` for final read-only validation.

13 Run implementation validation.

14 Report KIA in final response.

Landing Page Situation

The public landing implementation is in `tis-landing-website/`. Marketing docs live under `docs/marketing/`. Do not modify legacy FastAPI landing files unless explicitly approved.

Next Planned Work

Review the automatic KMS enforcement baseline, keep M7 subscription documentation synchronized as follow-up fixes evolve, and later consider an explicit owner-only regenerate action. Any regenerate action may rebuild artifacts from reviewed Markdown only and must not rewrite source prose.

Timetable feasibility authority (2026-08-30)

Deterministic timetable readiness ends at `configuration_complete`. The `timetable_feasibility_verifications` ledger stores a hard-only CP-SAT result by SchoolGroup, branch, academic year, immutable snapshot, and full input fingerprint. Generate is permitted only for an exact verified fingerprint. The validated placements are reused as an optimization hint and as the independently validated timeout fallback; soft objectives do not participate in verification. `stale_input` is an orthogonal Draft lifecycle state, not structural incompleteness: when it is the only finding, current inputs remain eligible for a fresh feasibility verification. The old Draft stays stale and unpublishable; after the current fingerprint is verified, Regenerate creates a new version from that protected source. Historical runs from older authority fingerprints remain auditable but do not drive the current action state. A regeneration source is the current populated, mutable, unpublished Draft with a usable placement arrangement, regardless of whether its origin is `manual`, `generated`, or `regenerated`. An empty starter Draft remains a Generate candidate; active, historical, superseded, and archived versions are never source candidates.

Talent & Potential Owner correction pass (2026-09-12)

UI/UX-only correction pass over ADR 0039's Competency+KPI+Level model, no eligibility-gate or architecture change. Fixed a `rubricGradeSections` rendering bug (`static/js/talent-program-workspace.js`) that hid a no-Grade Competency's entire row - Delete action included - on a genuinely new draft Framework; the fix keys the "no Grade = all Grades" behavior on `framework.in_use_by_assessments` so an already-used Framework's legacy no-Grade Competency still stays out of new-Grade authoring. Competency cards are now independently collapsible (`<details>`/`<summary>`), Delete Competency uses one unified confirmation across the direct and version-safe paths, the Student Assessment editor (`static/js/talent-operations.js`) now has a "Back to Students" action that preserves Academic Year/Program/Cycle context, and several Talent counters/headings were corrected to Title Case ("N Competencies", "N Levels", "KPI Overview", "Assessment Status"). The Owner-supplied symbol-only Ghars asset is now present as `static/img/talent-ghars-symbol.png` and is used by the Talent & Potential sidebar entry

through the existing icon-sized brand slot; the visible module label remains unchanged.

Talent & Potential UI action-placement correction (2026-09-12)

Edit/Delete for every Competency, KPI, and Level item in the builder (`static/js/talent-program-workspace.js`) are now compact icon-only buttons (pencil/trash via a new `iconButton()` helper) beside each item instead of visible "Edit/Delete Competency|KPI|Level" text, each with an `aria-label`/ `title` naming the exact item so there is never ambiguity about which row it affects. Applied consistently to both the primary Grade->Competency->KPI-> Level accordion and the older parallel flat-table wizard view; the Competency header also gained an Edit icon it previously lacked, reusing an existing edit capability (no new backend behavior). Add actions ("+ Add Competency", "+ Add KPI", "+ Add Level", "Copy Levels From...") are unchanged and stay in their structural positions, not beside every item. No hierarchy/CRUD/eligibility change; no backend files touched.

Talent & Potential Grade-to-Grade Assessment Criteria Copy (2026-09-12)

New backend `talent_program_service.copy_grade_criteria` + `POST /api/talent/programs/{id}/frameworks/{id}/grades/copy` (`talent_programs.manage`) copies a Grade's complete Competency/KPI/Level/ descriptor structure into an empty destination Grade as fully independent new records (new `TalentCompetency`/`FrameworkCompetency`/`TalentRubric`/ `TalentRubricLevel`/descriptor rows) - never shared identity with the source. An already-populated destination is always rejected (`target_grade_occupied`); no merge/Replace flow was implemented in this pass, per explicit Owner instruction to prefer safety over a hidden merge. Reuses `_require_mutable_draft` unchanged, so the frontend (`static/js/talent-program-workspace.js`) clones an immutable/active Framework into a new draft first (the same pattern as versioned Delete Competency) before copying. The UI action ("Copy Criteria From Grade...") is a structural, always-visible Grade-level action, not an inline icon. Same SchoolGroup/Program/Framework scoping as every other Program mutation.

CRITICAL: Production DB connection-pool exhaustion fixed (2026-09-12)

Root cause of a confirmed production `QueuePool limit of size 5 overflow 10 reached, connection timed out` release blocker: `main.py`'s `inactivity_timeout_middleware` (global, runs on every authenticated request app-wide) opened its own `SessionLocal()` and held it checked out for the entire `await call_next(request)` downstream request/response cycle - on top of the separate connection the route itself acquires via `Depends(get_db)` - doubling real per-request pool consumption. Not a leak (it always closed eventually); a long-held/duplicated connection. Talent Student Assessments' several concurrent API calls per load surfaced it first; the pool then stayed globally exhausted for the whole app until a Render restart reset it. Fix: the middleware's session is now opened, used for the auth/idle-timeout/commercial-access/permission checks, and closed before `call_next` is ever awaited - behavior- preserving, no Grade or annual-configuration gate reintroduced. See `tests/test_db_connection_pool_lifecycle.py` for the regression proof (ordering assertions plus a real small-QueuePool concurrency reproduction of both the old doubled-checkout shape exhausting the pool and the fixed shape not exceeding it).

Start Editing accidental empty-draft recovery + mandatory clone (2026-09-12)

Start Editing's old "Copy the current rubric structure (optional)" checkbox could be left unchecked, producing a genuinely empty new Draft while the Program's real Competencies stayed safe in the Framework being edited. Fixed permanently: the checkbox is removed and the new-version form submission always sends `clone_from_id` (`static/js/ talent-program-workspace.js`). New governed recovery path for any Program already caught by the old defect: `talent_program_service. recover_accidental_empty_draft` clones from the empty Draft's own `supersedes_framework_version_id` after validating it is genuinely empty, via the existing `create_framework_draft(clone_from_id=...)` mechanism - never hand-reconstructs data, never touches the

source Framework or historical Assessment evidence. Operator CLI: `scripts/recover_talent_accidental_empty_draft.py` (dry-run by default). No live Talent data was reachable from this environment (local `tis.db` has no talent tables; the only local Postgres is a stale, empty test schema) - the recovery function is proven against a synthetic 9-Competency reproduction in `tests/test_talent_program_start_editing_recovery.py`.

Talent Student Assessment empty-attempt rubric refresh + context cleanup (2026-09-12)

The learner-evidence boundary is now persisted Competency Results, not existence of an Assessment row. `continue_empty_assessment_on_current_rubric` replaces only a current In Progress attempt with zero `TalentStudentCompetencyResult` rows when a newer assessable Framework exists. The prior empty row becomes non-current; the replacement remains attached to the original visible Evaluation. Any saved Competency Result and every terminal Assessment remain bound to their exact historical Framework.

`GET /api/talent/assessments/contexts` omits private derived physical Cycles whose Assessment `cycle_id` differs from `evaluation_context_cycle_id`, while retaining those rows internally for provenance. The roster selects the newest matching current attempt if legacy/bad data exposes more than one, so a newer Completed attempt cannot be masked as Continue Assessment by an older In Progress row. No Grade or annual Program-configuration gate was reintroduced.

The Talent & Potential sidebar entry now uses the shared inline-SVG `sparkles` icon from the `icon()` macro in `templates/_app_icons.html`, like every other module (2026-09-19 UI polish; the per-item `brand_logo` override was removed from `ui_shell.py`). The earlier Owner-supplied Ghars PNG (`static/img/talent-ghars-symbol.png`) is no longer referenced by the sidebar; earlier entries in this file describing it in the sidebar are historical. The visible "Talent & Potential" label is unchanged.

Organization Analytics availability correction (2026-09-12)

Direct Owner decision removes the commercial runtime gate previously implemented through `feature.organization_intelligence`. The seven M10 Results & Analytics routes now resolve a permission-scoped availability provider: valid tenant/year shape passes availability and the existing `talent_analytics.view` plus secondary sensitive-data permissions remain authoritative. No plan, subscription, WorkspaceEntitlement, or feature-registry lookup participates in Organization Analytics availability. Privacy and breadth providers remain fail-closed and unchanged.

Talent Owner Adjustment Package - Current Grade/UX Contract (2026-09-12)

Owner amendment supersedes ADR 0039's prior cross-Grade fallback. Grade remains non-gating for roster membership, but a current Assessment can use only criteria aligned to the Student's Grade plus intentionally unscoped criteria. No other Grade's Competency/KPI/Level set may substitute. Start blocks with a Grade-specific setup message if none applies; zero-result In Progress reload follows only a newer applicable Grade build. Persisted evidence and terminal history remain immutable.

Presentation additions are Student Assessment Grade/Section/status filters, Talent Review identification classification filtering, responsive KPI/Level assessment cards, ordered blue low-to-high coloring for visible analytics magnitudes, categorical Learning Style indicators retaining style colors without revealing suppressed magnitude, Students nested under Talent for Talent-capable users with students-only fallback, natural Student table page flow, and Overview-only full transparent Ghars wordmark. No permission, privacy threshold, scoring, schema, migration, or write-contract change.

Talent Owner Adjustment Package - Correction (2026-09-12, follow-up)

Two corrections to the same-day package above. First, the Talent Overview header wordmark is **blocked pending the Owner-supplied asset file**, not delivered: the prior pass had wired the header to a locally traced/reconstructed SVG (`static/img/talent-ghars-logo.svg`), which is not an Owner-supplied file and must not be presented as the delivered logo. The header container/CSS now points at a documented, not-yet-present path (`static/img/ghars-full-wordmark-dark.png`) and renders empty until the Owner supplies that exact asset file; no logo is fabricated from a description. Second, Student Assessments' Evaluation Period cards mark the selected Evaluation in place (`is-selected` + `aria-current`); the separate duplicated "Selected Evaluation" panel that still rendered under the cards has been removed, completing the single-card-per-Evaluation-per-Program presentation rule. No schema, permission, privacy, or scoring change.

TIS Documentation Update Policy

docs/DOCUMENTATION_UPDATE_POLICY.md

This policy defines the non-negotiable rules for keeping the TIS Knowledge Management System reliable.

Source Of Truth

Markdown files under `docs/` are the source of truth.

The PDF booklet at `static/docs/TIS_Project_Reference_Booklet.pdf` is a generated snapshot. It must never be edited manually. It must be regenerated whenever included Markdown source files change.

Required Knowledge Impact Assessment

Every approved implementation must end with a Knowledge Impact Assessment (KIA).

Required final report template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until the KIA is included, even when no documentation changes are needed.

Every task must also update `.kms-impact.yml`. The declaration is the machine-readable form of the KIA and records `knowledge_impact`, a summary, affected areas, KMS files updated, a no-impact reason, and the explicit major-change override. `scripts/check_kms_impact.py` validates it against the actual Git diff.

`knowledge_impact: yes` requires changed authoritative Markdown. `knowledge_impact: no` requires a specific reason. When a path is conservatively classified as major but the change is genuinely non-behavioral, `major_change_override: yes` is allowed only with that written reason.

When Documentation Must Be Updated

Update documentation when a task changes:

- product vision or positioning,
- architecture or module boundaries,
- SaaS signup, login, onboarding, billing, payment, or provisioning flows,
- platform owner or permission behavior,
- operational workflows such as calendar, planning, timetable, teachers, subjects, or observations,
- deployment assumptions,
- branch/release/project status,
- landing page source-of-truth or visual strategy,
- roadmap or known issues,
- documentation system behavior.

Which Docs To Update

Use this guide:

- `docs/AI_PROJECT_CONTEXT.md`: compact onboarding context for future AI coding conversations; update when the high-level project situation changes.
- `docs/TIS_MASTER_CONTEXT.md`: durable product, architecture, workflow, roadmap, and critical rules.
- `docs/PROJECT_STATE.md`: current branch strategy, priorities, milestone status, known issues, and next work.
- `docs/CHANGE_HISTORY.md`: chronological summary of meaningful changes.
- `docs/adr/`: major architectural or product decisions.
- `docs/history/`: deeper module-specific before/after history.
- Supporting docs under `docs/marketing/` or other folders when those areas change.

ADR Rule

Create or update an ADR when a change affects a major long-term decision, including identity boundaries, payment architecture, provisioning strategy, deployment architecture, public website architecture, documentation governance, or visual system strategy.

Do not create ADRs for routine bug fixes unless they introduce or reverse a significant decision.

Module History Rule

Update `docs/history/<module>/` when a meaningful module area changes and the previous documented state should be preserved.

`docs/CHANGE_HISTORY.md` remains the chronological summary. Module history stores deeper context.

KMS Synchronization

Validate KIA, regenerate the PDF and manifest, and verify freshness with:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

The generator must remain dependency-light:

- use existing `reportlab`,
- no LaTeX,
- no Playwright or Chromium,
- no external network calls,
- no system font dependency.

Complete read-only validation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

The `check` command must never write documentation. GitHub Actions run it for pull requests and `dev`; the production deployment workflow requires it before triggering deployment from `master`. The `sync` command writes generated artifacts only and never rewrites authoritative Markdown.

Navigation And Catalog Integrity

Every included Markdown source must have a usable front-matter `title` or level-one Markdown heading. Placeholder, empty, punctuation-only, or excessively long titles are invalid.

The approved document categories are:

- `core`
- `engineering`
- `decisions`
- `history`
- `marketing`
- `supporting`

The approved module values are maintained in `kms_catalog.py`. Declared front-matter taxonomy must use the exact approved slug, and any declared category must match the source path's category. New modules require a reviewed catalog update and focused validation.

`docs/KMS_NAVIGATION.md` may link only to existing, listed Markdown sources inside `docs/` through relative POSIX-style paths. External URLs, parent-directory escapes, root-relative links, non-Markdown targets, missing targets, and unlisted authoritative documents fail validation.

The manifest source inventory must exactly match the normalized generator source list in deterministic order. Every source must have a positive integer `pdf_page`, source pages must increase in generator order, and no source page may exceed the generated PDF page count.

Prohibited KMS Content

KMS documentation describes system design only. Never include customer or organization information, personal data, subscription rows, invoices, transactions, real production identifiers, real webhook payloads, credentials, secrets, environment values, database row contents, or test-customer personal details.

Reporting Rule

Every final implementation report must mention:

- whether knowledge was impacted,
- which docs changed,
- whether change history changed,
- whether an ADR was needed,
- whether module history changed,
- whether the PDF was regenerated,
- whether `AI_PROJECT_CONTEXT.md` changed,
- validation results.

App Behavior Rule

The application may later detect documentation freshness and expose docs through owner-only protected routes. It must not silently rewrite Markdown source docs. Source docs are edited through reviewed development work.

TIS Master Context

docs/TIS_MASTER_CONTEXT.md

Talent Branch Authority Amendment (Production Follow-up Part 1, 2026-09-25)

Owner decision: the global (sidebar) Branch selector contains real Branches only. Inside Talent, an organization-authorized actor chooses All Branches or any authorized Branch in the page-level Branch filter; the sidebar Branch is only the default selection. A Branch-limited actor can never leave their authorized Branch (server-enforced). The Batch 1 closure wording "the global Branch is a hard Talent ceiling" is superseded for organization-authorized actors only. Tenant isolation, permissions and privacy are unchanged. Aggregate Classification remains governed by the approved privacy provider; groups it withholds read "Unavailable" with one uniform explanatory sentence.

Agent 3 Talent product transformation (2026-09-25)

Overview is the executive entry point; Results & Analytics is the six-section aggregate workspace, not another Student roster. A bounded backend projection provides shared filters, raw-population completion, canonical M17 Classification, eight-category Learning Style plus Unassigned, Program-bound Rubric Indicators, selected comparisons and compatible Evaluation Period context. Results across different Programs/frameworks/scales never become one Talent score.

The owner clarified the privacy boundary: individually authorized operational Student records retain exact permitted fields; aggregate Classification retains primary, complementary, fail-closed and reconstruction protection. Classification-filtered Learning Style is protected whenever it could disclose the protected cohort count through totals, buckets or percentages. The backend privacy provider decides publishability; the browser has no threshold authority. This supersedes the unconditional aggregate exception for this Classification-filtered case only. See the Agent 3 amendment in ADR 0031.

Global Branch Is A Hard Talent Scope (Batch 1 Closure, 2026-09-25)

> Amended 2026-09-25: see "Talent Branch Authority Amendment" above (organization-authorized > actors choose the Talent Branch on the page; the marker cookie is removed).

The active global Branch bounds Talent & Potential: it is an upper ceiling, not a default. The ceiling is the intersection of the actor's authorization and the active global scope (`talent_branch_scope.talent_branch_ceiling`, from the authenticated request user), composed into the existing visible-Branch resolution of every Talent read route, so a stale or hand-crafted `branch_id` naming another Branch is rejected and an omitted Branch resolves to the ceiling. "All Branches" inside Talent means all Branches permitted by the current global scope; only an explicit global "All Branches" (the `branch_scope=all` marker, `user.scope_all_branches`, offered on `/talent` pages only) exposes All Branches, the Organization Talent Map and cross-Branch comparison. Supersedes the Batch 1 "default only" wording. Details and the coverage table are in `docs/PROJECT_STATE.md`.

Talent Current-Student Integrity, Branch Scope And Loading (Deployment Acceptance Batch 1, 2026-09-24)

Talent & Potential counts only Students that currently exist. The Overview "Students participating" figure is the DISTINCT current-Student count (`distinct_students`); the previous 63-vs-9 discrepancy was the frozen-membership row count (one per Student per Cycle/Program) labelled as Students. Membership figures are "Program participations". Permanent Student deletion removes every Student-owned Talent row (ten tables, one transaction) and every current analytics read additionally requires the Student to exist, so an orphan can never inflate a

figure. The active global Branch is the default Branch scope of Talent (backend authorization unchanged); Talent list endpoints no longer scale per Assessment row; Talent assets are cache-busted and a loader watchdog prevents an endless generic loader. See PROJECT_STATE for the metric grain table, the FK audit, measurements and the open owner decisions (delete permission model, inactive Students, Talented Student headline). Supersedes the M15 reading that the Student-vs-Talent count difference was only a population-scope distinction.

Learning Style Distribution (Deployment Acceptance Correction C, 2026-09-24)

Learning Style is Student-domain categorical profile data (one of eight values, or Unassigned); there is no per-Student percentage and the four deprecated `learning_style_*_percentage` columns are operationally deprecated and excluded from current Learning Style analytics and normal product presentation (legacy compatibility paths may still read/write the stored values for preservation, never as analytics authority). The aggregate distribution (Students panel, `/api/students/analytics/learning-style-distribution`, and `/api/talent/results-analytics/academic-years/{id}/learning-style`) is authorized Student-domain aggregation: denominator = every authorized Student in the selection including Unassigned; nine categories always returned with count and percentage; zero categories are `0` / `0%`; empty population is an explicit empty state. By owner decision (ADR 0031 "Acceptance C Amendment") it is **not subject to Talent small-cell suppression**, and this does not weaken any other Talent privacy class. SchoolGroup/Branch/Grade scope, Student visibility, tenant isolation and `students.view` still bound the population; only aggregates are returned. No schema or migration change.

Current Talent Workflow And Student Identity (Deployment Acceptance Correction B, 2026-09-24)

Current normal workflow: Student -> Program -> Teacher Assessment -> Complete Assessment -> deterministic backend Automatic Classification (`talent_classification_service`, ADR 0037 2026-09-23 amendment). Only `Exceptional` (4.50-5.00 on the fixed classification scale) is Talented; the other bands are Needs Improvement, Developing, Meets Expectations, Advanced. Review Candidate and Official Identification are legacy historical data (preserved, permission-gated, audit-only, never current Talent state) reachable only as "Legacy Review & Identification History". Wherever an identifiable Student appears in Talent the shared identity presentation (`static/js/talent-student-identity.js`) shows name, canonical `Student.learning_style` (eight categories, "Unassigned" if none) and, for a `completed` + `is_current` assessment only, the backend Classification and (for Exceptional) a Talented badge. The legacy-history review workspace (`routers/talent_review_candidates.py` `talent_review_workspace`) includes this same current Classification via `assessment_classification(..., overall=)`, reusing the already-computed Overall Program Result; Classification never comes from Review Candidate / Official Identification. No classification is persisted or derived in the browser; there is no schema change. Not deployed.

Talent & Potential Runtime Loading Contract (Deployment Acceptance Correction A, 2026-09-24)

Authority for the Talent workspace client lifecycle (`static/js/talent.js`, `templates/talent/workspace.html`), implemented on `dev` only and not deployed:

1 No Talent view may stay indefinitely on a generic loader. The page shell is

server-rendered; JS replaces the placeholder deterministically at boot; one bounded essential request/context follows; every additional section then loads, and fails, independently with its own Retry.

1 Every script global that `talent.js` or a delegate needs must be loaded by

the template for that view before `talent.js` (defer scripts run in document order); `talent-rubric-visual.js`, `talent-api-errors.js` and `talent-rubric-request.js` are loaded on all views. A future change that gates a script per view must update the per-view dependency test in `tests/test_talent_ui.py`.

1 Read requests are bounded (25 s; 15 s for selector lookups; 20 s boot context

deadline); the stale-generation guard, AbortController, no-store and 250 ms debounce are preserved; the current generation always reaches a terminal state. User-facing errors use a curated status/code mapping (`talent-api-errors.js`) and never expose raw backend `detail`, exception text, stack, database or endpoint detail (raw-`detail` disclosure was corrected by the ClinePass remediation). Authorization, tenant scope, privacy policy and analytics semantics are unchanged.

M14-M18 Correction Program Closed On dev - Final Authority Summary (M18b-3, 2026-09-24)

The M14-M18 correction program is functionally implemented on `dev` and closed for owner-acceptance/release planning only; it is NOT deployed, merged to `master`, or claimed live. Final authority: (1) Learning Style is one categorical Student value from exactly eight categories, aggregated as those eight plus Unassigned; the four legacy percentage columns are operationally deprecated (stored, never exposed or aggregated). (2) Student vs Talent count differences are scope/population semantics; force-deleted Students never appear on current Talent surfaces. (3) Assessment classification is automatic and backend-only: the Program-native 1..N result is deterministically projected to 1.00-5.00 and banded Needs Improvement / Developing / Meets Expectations / Advanced / Exceptional; only Exceptional is Talented; Review Candidate and Official Identification are preserved legacy history, not current Talent authority. (4) An applicable current result is `status == 'completed' AND is_current == True`. (5) Results & Analytics owns a backend contract (`/api/talent/results-analytics/...`) for Learning Style, Classification, and Talented; Organization values are raw-count sums (10/92, never averaged rates); the UI renders backend values only, with a Program-bound narrow-only Classification filter and privacy states that never expose suppressed numbers. (6) Performance boundary: at most 3 effective permission projections on the common Talent page, per-request (never shared) authorization, and surface-specific script bundles. Known pre-existing issues (Student Drill fixed query budget/N+1, stale JS/Python test expectations, PostgreSQL FK-order tests, SaaS permission-pattern test, the historic unscoped-suite hang) are documented in `docs/PROJECT_STATE.md` and are not correction-program regressions. M18b-3 made one bounded fix: the Student Drill no longer recomputes the overall Program result a second time per row for classification.

Results & Analytics IA reorder + Classification filter + accessibility/responsive audit + candidate_membership_count decision (M18b-2b)

Bounded completion pass over M18b-2's own explicit open-item list; no backend semantic change, M18b-1 contract untouched. The `analytics` view's in-content page header now reads "Results & Analytics" with a dedicated subtext (the shared workspace chrome title stays "Organization Overview", pinned by `tests/test_talent_organization_analytics_providers.py`). Its 9 sections now render in the required order (header -> summary cards, including one backend-sourced Talented count fact -> Learning Style -> Classification -> Current Talent -> Competency Analysis, now grouped together -> Results/Evaluation Progress -> Branch/Organization comparison -> navigation). A real, progressive `Classification` filter (`<select name="classification">`, the 5 backend `CLASSIFICATION_LABELS`) narrows only the classification request and is hidden/cleared until a Program is selected; Evaluation Period/Competency filters were deliberately not added (would need new supporting list endpoints); Learning Style's own filters are already covered by the existing Branch/Grade selectors. No chart-type selector exists (bar-only compatibility matrix, confirmed not added). `candidate_membership_count`'s summary-card label is now `"Legacy: Meets Program Criteria"` so it is never read as current-Talent data; metric/data/ permission unchanged. `static/css/talent.css` was independently

re-checked against every new class the M18b-2 markup uses (`.tp-grade-chart/ .tp-grade-row/.tp-table-wrap/.tp-primary-indicator/.tp-filters label`) - all already responsive at the 680px breakpoint, no CSS gap found, no CSS change made. Regression: `tests/talent_results_experience.test.cjs` 26/28 passed (4 new tests, 2 identical pre-existing failures, git-stash- verified against the unmodified `5aa13cb` baseline); `tests/ talent_branch_comparison_frontend.test.cjs` + `tests/ talent_experience_adjustments.test.cjs` + `tests/talent_ui.test.cjs` 38/41 passed (identical 3 pre-existing failures, git-stash-verified); `tests/ test_talent_results_analytics.py` + `tests/test_talent_ui.py` 51/51 passed; `tests/test_talent_organization_analytics_providers.py` 22/22 passed; no schema/migration change; `tis.db` byte-identical (SHA-256 verified). Full broad M14-M18 regression/performance/KMS closeout remains M18b-3, still pending.

Results & Analytics Frontend Rebuild - current-Talent indicator + new-family consumption (M18b-2)

Bounded FRONTEND-ONLY sub-phase consuming the already-approved M18b-1 backend contract; no backend semantic change. `static/js/talent.js`'s `analytics` view now fetches and renders all three M18b-1 families - Learning Style (Student-domain-wide, gated on `students.view`), Classification (Program-bound, exactly 5 bands), and Talented (Program-bound, Exceptional-only, with an optional per-Branch breakdown) - using new reusable chart+accessible-table helpers (`bucketBars`/ `bucketTable`/`distributionSection`/`talentedSection`) that reuse the existing `tp-grade-chart`/`table()` CSS/markup, so no charting library or CSS change was needed. The legacy `identifiedIndicator` (Official Identification) primary "how many Students are talented" indicator is removed from this view and replaced by `talentedSection`; the underlying legacy Review Candidate/Official Identification services, data, and the separate Talent Review workspace are completely unchanged, and `identified_of_eligible`/`candidate_of_eligible` remain valid options only in the separate, still-supported legacy Branch-comparison metric selector. `routers/talent_ui.py` additively surfaces `"students.view"` in `talent_permissions` (presentation-gating only). Competency and Progress/Evaluation-Period/Branch-comparison sections were confirmed already correctly wired to the pre-existing governed endpoints and were left unchanged. Regression (git-worktree-verified against unmodified M18b-1 baseline `2bf9b8c`): `tests/talent_*.test.cjs` 170/154/16 versus baseline 165/149/16, identical 16 pre-existing failures by exact name, zero new JS failures; `tests/test_talent_results_analytics.py` + `tests/test_talent_ui.py` 51/51 passed; no schema/migration change; `tis.db` byte-identical (SHA-256 verified). A full 9-section information-architecture reorder, progressive filter bar, and explicit accessibility/responsive audit remain open follow-up work (M18b-2b/M18b-3) - this page is not yet in its final target shape.

Results & Analytics Backend Contract + Correct Aggregation Authority (M18b-1)

Bounded BACKEND-ONLY sub-phase delivering the org/Branch aggregate "current Talented count" analytics M18a deferred, plus a Learning Style/Classification backend contract (M18b-2's visible page/chart rebuild remains separately scoped and pending). `talent_org_intelligence_contract.MetricCode` stays frozen and unmodified - the new module `talent_results_analytics_service.py` instead reuses the existing M9 `talent_analytics_service` Program+ AcademicYear context/filter/scope architecture and generic `Cell`/`Group` privacy primitives (a new opaque privacy class `"P4"`, never the legacy Candidate/Identification metrics) behind one coherent route, `routers/talent_results_analytics.py` (`/api/talent/results-analytics/...`), for three families: Learning Style (thin reuse of `student_learning_style_analytics.py`, no new computation), Classification (current M17 five-band distribution over the M18a-governed completed+current grain, reusing `talent_classification_service. assessment_classification` for every band - never a duplicated band table), and Talented (Exceptional-only count/denominator/rate).

Correct Organization aggregation (CRITICAL, regression-tested exactly): Organization/Branch totals are always the raw sum of individual Branch counts (`sum_raw_counts_across_branches`), never an average of Branch percentages - proven with the exact Branch A 1/2 (50%) + Branch B 9/90 (10%) -> Organization 10/92 (~10.87%, not the naive 30% average) case.

Competency and Evaluation Period/Overall Result analytics are deliberately NOT reimplemented in this contract: the pre-existing `/api/talent/analytics/.../rubric-distribution` and `/api/talent/evaluation-progress/...` routes already provide governed, correct, Program-bound/Period-bound backend analytics for those two families and are consumed directly by M18b-2 instead of being duplicated.

The dead `learning_style_dimension` query parameter (inert since M14 - the "learning_style" Branch-comparison metric it parameterized was never a member of `APPROVED_BRANCH_METRICS`) is removed outright from `routers/talent_evaluation_progress.py`/`talent_evaluation_progress_service .py`.

Current Talent Authority Alignment + Learning Style Cleanup + Privacy UX (M18a)

Bounded first half of M18 (the full Results & Analytics rebuild is M18b, out of scope here). Applicable-current-result authority for M17 classification: the `TalentStudentAssessment` row where `status == 'completed'` AND `is_current == True` (the pre-existing ADR 0036 reassessment authority already consumed by M9 analytics and the B9 Student Drill - not a new rule).

The Learner Profile and B9 Student Drill now additively expose the same backend M17 classification (`classification`/`classification_score`/ `is_talented`) already on the Talent Assessment API, reusing the single classification authority (`talent_classification_service .assessment_classification`) - never a duplicated derivation, never from `TalentReviewCandidate`/`TalentOfficialIdentification`. A NEW org/Branch AGGREGATE "current Talented count" metric was evaluated and explicitly deferred to M18b: `talent_org_intelligence_contract.MetricCode` is a frozen 14-value enum (ADR 0044: "no MetricCode extension"), and the existing Talent-adjacent aggregate metrics are grounded in the legacy Review/ Identification membership grains, not an M17-classification grain - that is new analytics infrastructure, not cleanup. `static/js/talent.js`'s `candidate_of_eligible`/`identified_of_eligible` labels (previously "Talent share"/"Officially confirmed share", which read as current Talent-status shares though their data is the legacy Review/Identification workflow) are relabeled "Legacy review share"/"Legacy identification share" - data, permissions, and computation unchanged.

M14's Learning Style deprecation is now complete for read exposure: `routers/students.py`/`routers/students_ui.py` no longer serialize the four `learning_style_*_percentage` fields in the current normal API/UI projection (stored columns untouched), and `talent_evaluation_progress_service.py`'s confirmed-unreachable `learning_style_branch_aggregate`/ `_learning_style_values_by_branch`/`_LEARNING_STYLE_COLUMNS` dead code is removed. There is no current four-dimension Learning Style analytics authority anywhere after M18a.

The literal visible phrase "Protected for privacy" is removed from Talent & Potential / Learning Style UI and replaced with context-appropriate neutral copy (presentation only - the underlying suppression/privacy contract is completely unchanged).

Automatic Assessment Classification Authority (M17)

Completing a Student Assessment automatically derives its classification and `Talented` state; no manual Review Candidate/Official Identification step is required for the current normal workflow. Classification is always backend-computed, deterministic, and derived only from the ADR 0037 Overall Program Result of a Completed Assessment - never from AI, never from the frontend, never from `normalized_percent`, and never expressed as a percentage. The five owner-approved bands (Needs Improvement, Developing, Meets Expectations, Advanced,

Exceptional) are fixed on a governed 1.00-5.00 classification scale; only Exceptional is Talented.

Because real Talent Programs configure varying rubric level counts (not universally five), the classification scale is reached by a deterministic linear projection from each Program's own 1..N rubric average onto 1.00-5.00 (ADR 0037's 2026-09-23 Amendment), not by mandating a five-level rubric system-wide. `talent_classification_service.py` is the single classification authority; `talent_student_assessment_service.overall_ program_result` is unchanged except for an additive `average_tenths` field used for exact Decimal projection math.

Legacy `TalentReviewCandidate`/`TalentOfficialIdentification` records remain fully preserved and independently reachable for history/audit, but no longer govern current `Talented` state and are never rewritten to match a later automatic classification. No schema migration, permission removal, or tenant/privacy boundary change was required.

Talent Performance And Student Action Layout Authority (M16)

Talent page authorization must compute an effective permission set as a bounded request-scoped operation and reuse that already-authorized set for shell and action presentation. This reuse never crosses requests, users, SchoolGroups, or Branch scopes and does not replace per-request authorization. Talent surfaces load only the operational JavaScript bundles they use; `talent.js` and `talent-experience.js` remain common. The application remains server-rendered with vanilla JavaScript, and no API or product semantics changed.

Students action containers are wrapping horizontal flex rows. Add/Delete and Open/Delete therefore remain adjacent on desktop/tablet where space permits and wrap safely on narrow mobile without changing accessible labels or permission projection.

Learning Style Correction + Aggregate Distribution (M14)

Owner-directed correction (`docs/PROJECT_STATE.md`'s M14 entry and ADR 0031/0042/0044's M14 amendment sections are the full governance record). `Student.learning_style` is the single authoritative Learning Style field, now accepting eight values (Visual, Auditory, Read/Write, Kinesthetic, Verbal, Non-verbal, Quantitative, Spatial). The four `learning_style_*_percentage` columns (previously an independent per-Student four-dimension profile) are operationally deprecated: no longer written, displayed, or read as Learning Style authority anywhere, including in Talent Student context or the M4/M10 Learning Style Branch-comparison metric (removed). Existing stored percentage values are preserved untouched; physical column removal is a later, separately gated cleanup.

Aggregate Learning Style distribution (`student_learning_style_analytics.py`, extended rather than parallel-built) now covers all eight values plus an "Unassigned" bucket for the authorized Student population, reusing the same privacy/suppression contract and Branch/Organization scope ADR 0031 already required. The denominator is always every authorized Student in scope, including Unassigned Students.

Student Roster Frontend (M11)

The Students list owns a bounded `.xlsx` roster workflow over the existing M6 API. `students.export` projects a compact export action that downloads the server workbook unchanged. `students.import` projects a transient import dialog for file selection, stateless validation preview, structured row feedback, and explicit apply confirmation. UI projection is not authorization; each M6 API route continues to enforce its own permission and SchoolGroup/Branch scope.

The selected workbook itself is submitted to both preview and apply. The frontend never treats preview as a write authorization or sends a generated approved-row list. Apply remains independently revalidated, all-or-nothing, and create-only. Student-ID uniqueness, privacy-safe conflict disclosure, canonical Placement/Section matching, workbook parsing, and transaction atomicity remain backend responsibilities. M5 `section_display` stays presentation-only. No CSV, `.xls`, persistent import batch, background job, schema, or migration is added.

Student Integrity & Roster Round-Trip (M15)

M15 (isolated `m15-student-integrity-roster` feature branch) verifies and documents the permanent-delete boundary rather than changing it: a force-deleted Student is removed from every Student-owned Talent table and can never appear as a current/live Student in Talent & Potential. The Students vs Talent/Review count difference is a deliberate population-scope distinction (Branch-scoped current Students list vs frozen multi-Branch Talent membership and status-agnostic eligible roster), not a deletion defect.

The Student roster export is now round-trip safe: `section_display` is accepted-but-ignored on import (display-only, never identity; canonical `section` remains identity) and the `STD` prefix is normalized back to the bare 10-digit value. Import remains create-only with deterministic NO_CHANGE classification for unchanged exported rows and blocked conflicts for any mutated existing row. Apply writes only CREATE rows and skips NO_CHANGE rows atomically.

Students Frontend Identity And Learning Style (M7)

Students create/list/profile/edit now consume the canonical managed TIS Student ID and four-dimension Learning Style backend contracts. The browser accepts only the 10-digit portion beside a fixed `STD` prefix, preserves leading zeroes, and does not construct authority or duplicate-conflict details. New Students require the ID; legacy Students may remain without one and can still edit other fields. ID replacement remains separately permissioned by `students.manage_identifiers`.

Verbal, Non-verbal, Quantitative, and Spatial were independent nullable 0-100 integer values as originally shipped here. Blank and zero were distinct, no sum was enforced, and the original single-select categorical Learning Style field was neither displayed as the current profile nor auto-converted. Student Placement identity/history and M5 Section presentation remain unchanged. Roster import/export UI was out of scope for M7 and was implemented in M11 (see "Student Roster Frontend (M11)" above).

Current state (M14 owner correction): the above four-percentage model was a misinterpretation and is now operationally deprecated - see "Learning Style Correction + Aggregate Distribution (M14)" below for the corrected model (one categorical `learning_style` field, eight values, with the four percentage columns no longer written/displayed/read anywhere). Existing stored percentage values described above remain preserved, untouched.

Current Permission Resolution Ownership Boundary

Current-user permission evaluation remains canonical through `auth.get_allowed_permission_keys`, with ADR 0040's unchanged default/global/ tenant/user precedence and Deny-over-Allow rule. Non-platform actors must have durable SchoolGroup ownership (stored or branch-derived); contradictory stored and branch ownership, absent ownership, or foreign selected/explicit evaluation scope fails closed. Abstract role-reference resolution remains separate and may legitimately evaluate global policy without a current-user identity. Target-user management and user-override writes also reject contradictory target ownership, including when the actor is platform-level; ordinary cross-tenant platform management remains available for targets with valid ownership.

Teacher and Planning edit actions do not inherit authority for separately permissioned fields: teacher qualifications, subject/section assignments and capacity, and Planning homeroom/teacher assignments retain their dedicated keys. Read-only omitted controls preserve persisted values and relationship identities. Notification automatic mark-read is a mutation requiring its dedicated key; inactive platform identities receive no helper capability. Teacher bulk deletion uses `teachers.bulk_delete`, independently of single-record deletion.

Role policy logical keys are protected by global/tenant partial unique indexes and a duplicate-preflight migration; this does not authorize automatic cleanup. The independent review's remaining registry-policy and verification blockers are recorded in `docs/engineering/PERMISSION_CLOSURE_REVIEW.md`; do not equate a green focused test suite with whole-system permission approval.

Role Permission Management - Direct Assignment vs Effective Authorization

The Role Permissions editor (`/system-configuration/role-permissions`) manages the four tenant roles (Administrator, Editor, User, Limited) at either global-default scope or a selected SchoolGroup scope. `role_permission_service.py` is now the single resolver for both UI payloads and backend authorization (`auth.py` delegates to it), replacing previously duplicated default/global/tenant merge logic.

A role's **effective** permission for a key is the ordered merge of built-in role defaults, global (SchoolGroup-null) `RolePermission` override rows, then the selected tenant's override rows; the more specific write is authoritative (tenant overrides global, global overrides default). The editor previously rendered only this effective set as one checkbox, conflating "directly assigned here" with "inherited by default/global". The payload now also carries per-permission `direct`, `inherited`, and `source` so the editor labels a default- or global-granted permission as inherited instead of implying one checkbox represents both.

The administrator Save path persists only intentional scope-level overrides: a key whose submitted state equals its inherited baseline is left with no scope-level row, so a no-op Save never converts inherited/default/global grants into direct rows, and later default/global changes still propagate to any role without an explicit local override.

Owner-only controls (`system_owner.manage_ownership`, `system_owner.transfer_ownership`) and developer-only controls are classified as `PLATFORM_ONLY` and are locked, non-assignable, and excluded from Administrator defaults; they remain owner/developer identity-bound and never enter a tenant role's editable model. No schema change: `RolePermission` still stores explicit allow/deny rows, and a permission with no override row falls back to its default.

Per-User Permission Overrides (UI + Route Wired - Allow-Over-Deny Not Approved)

A per-user override layer (`UserPermissionOverride`, `user_permission_service.py`) sits above the existing role-resolved set. See ADR 0040 for the full recorded decision (precedence chain, Deny-over-Allow rationale, uniqueness-rule justification, and platform-actor scope resolution); this section summarizes it. The approved precedence is built-in role default -> global (SchoolGroup-null) `RolePermission` override -> tenant `RolePermission` override -> per-user override -> effective permission, with exactly three per-user states: Inherit (no row), Allow, Deny. Uniqueness is `user_id + permission_key` (not `school_group_id + user_id + permission_key`) because a TIS user belongs to exactly one `school_group_id` at a time (`User` has a single `school_group_id` column, not a membership table), so the SchoolGroup is not an independent dimension of the override identity; `apply_user_override` still records and validates `school_group_id` for tenant-isolation enforcement and audit context.

Allow-over-Deny is intentionally NOT implemented. No prior ADR or this document approved letting an explicit per-user Allow override grant access that the resolved role currently denies (whether by default, global, or tenant-scope `RolePermission` override). `user_permission_service .apply_overrides_to_keys` therefore applies Deny-over-Allow precedence: a per-user Deny always removes a key, but a per-user Allow only has effect when the key is already present in the role-resolved set - it can never add a key the role denies. A stored Allow override on a role-denied key is preserved (so re-granting the key at the role level immediately reactivates it) but is inert until then. Enabling true Allow-over-Deny is an open product/security decision requiring explicit Owner sign-off before implementation.

The Phase 3 whole-application qualification (see `docs/engineering/PERMISSION_CLOSURE_REVIEW.md`) classifies all 175 registered keys (143 active-enforced, 14 platform-only enforced, 5 alias/composite, 13 dormant/reserved, 0 unresolved) and pins that classification in a machine-checked test; platform users additionally need the canonical all-school capability (`_can_manage_all_school_scopes`) to select an organization context.

Every consumer of "the current user's effective permissions" must resolve through `auth.get_allowed_permission_keys` (the sole integration point that folds role resolution and user overrides together via `role_permission_service` and `user_permission_service`). The abstract role-level set used to render the four managed roles' reference summaries (e.g. "Dashboard: 4/5") is read from

`role_permission_service` (`get_allowed_permission_keys` / `build_role_permission_payload`; the Phase 3 audit deleted the unused duplicate `_get_role_permission_rows`/`_get_allowed_permission_keys` helpers formerly in `auth.py`, `main.py`, `routers/users.py`, and `ui_shell.py`, so no second role-policy reader remains). A role-level summary is never a substitute for a specific user's effective permissions; a UI that displays a specific user's access must use `user_permission_service.build_user_permission_payload` instead, otherwise an administrator can be misled by a role-level summary that does not reflect that user's per-user override.

`build_user_permission_payload` is consumed by the Edit User page (`templates/edit_user.html`, rendered from `routers/users.py` `_render_edit_user_page`), which shows a "User Permission Exceptions" section: a user summary and, per permission, Effective, Source (User Exception / School Override / Standard Role Package) and the direct exception. The UI is binary Allow / Deny plus **Reset to Role Settings** (shown only when an exception exists); "Inherit" is not a user-facing choice, although internally no row still means follow-the-role and `apply_user_override(..., "inherit")` still deletes the row. A stored Allow under a role-level Deny is shown as Effective Deny and "stored, currently ineffective", is never deleted silently, and applies again if the role re-grants. Each action posts a single `change="<key>|<allow|deny|reset>"` to `/users/permissions/{user_pk}` (`routers/users.py` `update_user_permissions`; the legacy paired-list form remains supported), which is gated by the existing `configuration.manage_permissions` key (no new permission key), resolves the override's `school_group_id` from the acting admin's own active SchoolGroup (never client input), and re-verifies tenant boundary via the existing `_get_user_for_management` / `_can_manage_target_user` guard before calling `user_permission_service.apply_user_override`. Platform-only and non-`assignable` keys render locked with no Allow/Deny buttons, consistent with the service-layer rejection; a stored exception that became non-assignable after a role change shows only "Reset to Role Settings" (deletion, no authority). Normal UX is Allow / Deny + Reset to Role Settings; "inherit" remains only the internal, backward-compatible API form of clearing an override. Regressions: `tests/test_user_exceptions_ui.py` and `tests/test_edit_user_template_render.py`.

The route resolves the override's `school_group_id` from the acting admin's own active SchoolGroup for an ordinary tenant administrator. For a platform-level actor (owner/developer) -- who has no tenant SchoolGroup of their own (`auth.get_user_school_group_id` always returns `None` for a platform user) and whose own session-selected branch/SchoolGroup scope may legitimately differ from an arbitrary target's tenant -- the route instead derives scope unconditionally from the already server-loaded target user's own verified `school_group_id`, mirroring `_build_user_permission_context`'s identical read-side resolution. This is never derived from client-supplied request input; it relies only on the codebase's existing, pre-established cross-tenant "platform manages any tenant user" authority (`_can_manage_target_user`/`can_manage_target_user_account`), and `apply_user_override` independently re-validates the target's own `school_group_id` regardless, so no other SchoolGroup can ever be written. See ADR 0040 for the full rationale and the rejected client-supplied-scope alternative.

Talent Guided Setup And Scheduling Authority

The normal Program journey is Basics -> What we assess -> Evaluation schedule, with readiness projected directly on the Program summary rather than through a separate Ready step. **Ready** with a positive check appears only when Program details/eligible Grades, the assessment criteria, and at least one Evaluation Period are complete. What we assess is a joined presentation of one exact Framework version's competencies, rubric levels, and achievement descriptions; editing continues to use draft versions and preserves active/historical immutability internally. Program discovery uses a searchable desktop table with Program, Grades, Type, Current Year, Status, and actions; the Create-Program form is not permanently expanded there and opens as a collapsed panel behind "New Program".

Evaluation names are free-text and user-defined (for example Term 1, Audition, or Spring Review), never a fixed Baseline/Term 1/Term 2/Final picklist, with states Setup, Ready to start, In progress, and Complete. "Add evaluation" and Start Evaluation orchestrate the existing M8 Plan/Period and M4 Cycle APIs - adding the first evaluation creates the underlying Plan/Period state implicitly; they do not replace domain services, and none of that internal Plan/Period/Cycle/Link vocabulary is rendered in this normal path. Before open, the authorized population preview supplies the eligible Student count and effective date. Cycle open remains organization-governed and freezes Academic Placement-derived history. Permission projection is advisory presentation only; API authorization, tenant

scope, revisions, lifecycle validation, and frozen history are final.

The shared `tp-filters` context selector applies selection changes automatically (debounced) instead of requiring a separate "Apply context" click. Talent's main application sidebar tree is the sole primary module navigation; the page no longer duplicates Overview/Programs/Student Assessments/Talent Review/Results & Analytics. Results & Analytics retains its own bounded sub-navigation inside that family. Program assessment authoring shows one selected eligible Grade at a time (defaulting to the first configured Grade) instead of repeating the same Grade in every Competency row.

Video Acceptance Corrections - Student Assessments And Students

Owner video acceptance on 2026-09-11 confirms that a configured Program shown under an Evaluation Period is itself the selectable operational entry point. If the Period does not yet have its internal Cycle, an authorized selection creates and links that Cycle through the existing M8/M4 APIs and remains in Student Assessments. The user is never redirected to Evaluation Plan merely because that internal row is absent. **Start Assessment** must create/open the Student's current Assessment and render the exact saved Program Framework for the Student's Grade: Competencies -> competency-owned assessment criteria -> ordered Levels/descriptions. Any start failure must surface visibly as an actionable error rather than appearing to do nothing.

For immutable/assessed Program setup, Delete Competency never mutates historical Framework evidence. When the actor has the dedicated delete/manage permissions, the UI may offer Delete Competency by cloning a new editable Framework Version and deleting the Competency there; completed evidence remains bound to the old Framework. The numeric KPI disclosure has an explicit expand/collapse affordance.

Students list management actions use compact icon-only controls with accessible labels. Cross-Branch Student browsing uses the dedicated `students.view_all_branches` permission and also requires organization/global scope; the managed role policy keeps it Administrator-only. Other actors are fixed to their assigned authorized Branch and do not see **All branches**. Learning Style aggregates formerly failed closed under ADR 0031 (unavailable privacy configuration, suppressed cohorts); **superseded 2026-09-24 by the ADR 0031 Acceptance C Amendment**: the distribution is authorized aggregation with no Talent small-cell suppression.

Talent Assessment Recovery And Review UX Authority

Completed Assessment evidence remains immutable. A new Administrator-default permission, `talent_assessments.reset_for_reassessment`, provides an evidence-preserving operational recovery path when a completed current Assessment must be assessed again but automatic rubric-change detection does not apply. The action marks the prior attempt non-current and audits the reset; it does not delete competency results, Review Candidate/Official Identification records, Educator Input, placement provenance, or other history. The Student then returns to **Not started / Start Assessment** in the same visible Evaluation, and the normal start path creates the new current attempt. ADR 0034 hard-delete remains limited to zero-evidence Assessments.

Student Assessments keeps the Evaluation Period visibly selected while the user works, nests configured Programs inside that Period, and opens the exact Program/Grade assessment criteria when Start/Continue/View Assessment is used. Talent Review's primary table no longer presents the internal deterministic "Meets criteria" classification as a user-facing category; internal Review Candidate policy remains intact for review actions and Official Identification remains a separate human decision. Review can be narrowed by authorized Branch, Grade, and Section. "All Branches" is offered only when the actor has more than one authorized Branch. Analytics failures surface the backend's governed reason/action instead of a generic unavailable message.

User-facing Program setup uses **Assessment Criteria** for the competency-owned rubric structure so it is not falsely conflated with the separate numeric `TalentKpiConfiguration`. The real numeric feature is explicitly labeled **Key Performance Indicator (KPI)** and exposes Add/Edit/Delete KPI controls when editable. Internal rubric API/schema terminology remains unchanged.

Student/Talent Migration Ordering Authority

Student and Talent persistence is owned by the ordered migration ledger, not pre-ledger baseline metadata creation. `scripts/run_migrations.py` excludes all `students/student_*` and `talent_*` tables from its baseline set. Migration `20260904_000_student_talent_parent_scope_prerequisites` precedes `001` and requires PostgreSQL-valid unique parent keys on `(id, school_group_id)` for `branches`, `academic_years`, and `students` when that table already exists. Equivalent keys are accepted; duplicate legacy data aborts without mutation or a migration marker.

Migration `004` creates the Cycle table without requiring the future M8 Period parent. Migration `20260905_001` remains the authority that creates Plans and Periods and then adds the nullable composite Cycle-to-Period foreign key. This ordering preserves every composite tenant-isolation constraint and supports both fresh production execution and safe late application of `000` to an environment where `001` already ran.

B11-E F1 Provider Authority

ADR 0028 now approves and F1 implements three Organization Intelligence provider decisions. Privacy requires external configuration equal to the approved minimum cohort of 5 and applies it to P1-P7 through the existing primary and complementary suppression pipeline. Availability uses only the registered `feature.organization_intelligence` key and the canonical active commercial-state/feature-entitlement path; `talent_analytics.view` and all secondary permissions remain independent authorization checks. Breadth requires external configuration equal to 1000 for matrix cells, relationship results, and Program-pair results; an over-limit request is rejected and never silently truncated.

Every missing, invalid, mismatched, rejected, or exceptional production input fails closed. The only automatic deterministic provider path is non-production with `DATABASE_URL` resolving exactly to `.local_test_data/talent_local_test.db`. Production environment names always disable that path, and `tis.db`, memory databases, and other SQLite paths never qualify. This implementation adds no schema, migration, new permission, plan-specific packaging, price, or HTTP response field. F1 does not close the remaining B11 release gates or B12 and is not a production-readiness claim.

Talent Results Experience Authority

The M11 Results & Analytics UI is a presentation layer over the seven M10 privacy-closed projections. Public navigation uses Organization Overview, Talent Map, Program Results, Branch Results, Students Across Programs, Students, and Progress Over Time. It does not expose internal projection names or provider metadata. Organization, Program, and Branch visuals may use only returned factual counts and already-visible percentages; there is no composite score, Branch ranking, cross-Program averaging, or client-derived metric.

Talent Map color intensity and progress/chart dimensions may be computed only from an already-visible backend percentage. Zero is a visible result. No data, privacy protection, unavailable analytics, and permission denial remain separate categorical outcomes. A protected result must not affect text, percentage, range, width, height, opacity, intensity, tooltip, animation, hidden DOM data, or accessible name. Students preserve the P7 gate and frozen historical context, and Progress Over Time stays within one Program and Academic Year with governed comparability states and no delta or improvement claim.

Talent Operational Workspace Authority

The Talent & Potential operational workspace is a presentation/orchestration layer over the existing M2-M8 APIs. It never owns domain state or frontend-only authorization. Program setup remains Program-specific and versioned; qualitative rubrics and Performing Arts remain first-class, while the optional numeric KPI is the approved bounded Framework-specific calculation only. No universal rubric, Talent Score, or cross-Program normalization exists.

The user journey is Program -> Annual Evaluation Plan -> Period -> Cycle -> frozen Students -> Assessment -> Review. Assessment editing resolves stable FrameworkCompetency and RubricLevel row IDs from authorized reads and writes each competency separately. Every subsequent write uses the revision returned by the preceding write. A stale conflict stops remaining writes and retains unsaved input; the UI never represents this sequence as an atomic bulk save.

Display labels for operational Assessment and Candidate screens are enriched only after the existing tenant/Branch authorization gates. Student and Program names are current display labels; Branch, Grade, and Section interpretation is always from the frozen Cycle population member. Review Candidate, review state, append-only Official Identification, and append-only-with-lineage Educator Input remain separate permission and data boundaries. Identification is never inferred or automatically recorded. UI action visibility mirrors permissions and organization-scope requirements, while backend authorization remains final.

B11-E Integrated Production Qualification Authority

ADR 0029 governs and implements a dedicated M10-only REPEATABLE READ transaction boundary (`database.M10OrganizationAnalyticsSessionLocal`, `dependencies.get_m10_organization_analytics_db`), applied only to the seven M10 Organization Intelligence routes, strictly backend-conditional (PostgreSQL only), with no server-wide or other-route isolation change. All seven routes were live-re-tested under the permanent implementation: the two B11-D-proven mismatches are resolved, and the five previously structurally-inferred routes were independently reproduced live for the first time and confirmed resolved.

Suppression/reconstruction concurrency was tested live with the existing non-production `DeterministicSuppressionTestPolicy` for both primary and complementary (relationship-linked) suppression; no suppressed value became reconstructable under a concurrent write. Index Candidate B is finalized as NO CHANGE (the isolation change has no mechanism to affect query planning); statistics freshness is finalized as operational/runbook guidance. F1 subsequently implements the approved fail-closed privacy, commercial- availability, and breadth providers described above. The earlier independent review returned PASS WITH NON-BLOCKING OBSERVATIONS. **As of 2026-09-09 (ADR 0028, ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED. Production deployment has not occurred.**

B11-D PostgreSQL Concurrency Qualification Authority

B11-D is CLOSED after independent review passed with non-blocking observations. READ COMMITTED produced same-request mixed snapshots for Overview (Candidate 314 to 315) and Student Drill (gate/page population 705 to 706). An isolated, non-permanent REPEATABLE READ experiment resolved both. Permanent adoption was then PROPOSED - GOVERNANCE REQUIRED and NOT IMPLEMENTED; B11-E subsequently governed and implemented the complete M10 request boundary from the first database statement through privacy/reconstruction inputs.

B11-E subsequently completed the multi-statement route reproduction, suppressing-policy concurrency, statistics-freshness, and final scale/index disposition. Candidate B is NO CHANGE and no index is approved. Production provider, memory-ceiling, multi-worker, and SLO gates were subsequently resolved. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

B11-C PostgreSQL Qualification Authority

B11-C is CLOSED after a PASS WITH NON-BLOCKING OBSERVATIONS independent re-review. PostgreSQL 16.15 evidence covers all seven M10 routes, bounded query counts across tested scales, no N+1, Candidate A = NO CHANGE, and Candidate B = MORE EVIDENCE REQUIRED. No index, schema, migration, permission, privacy, or entitlement semantic change followed. Memory results are directional local Windows development evidence only, and

no production policy value was chosen.

B11-D subsequently completed and is CLOSED. B11-E implementation subsequently PASSED. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

B11 Production Qualification Policy Authority

ADR 0028 governs B11 production qualification. Privacy thresholds and breadth limits remain external configuration with fail-closed provider resolution and no hard-coded, implicit, or permissive fallback. Organization Analytics stays plan-agnostic; commercial availability uses the existing entitlement/feature- registry architecture while exact packaging and any new semantic feature key remain Owner decisions. B11-C must gather PostgreSQL query/latency/memory/ privacy-closure and EXPLAIN evidence before numeric SLOs or limits. B11-D tests READ COMMITTED first; stronger isolation or an index requires demonstrated evidence and its own governed ADR.

B11-C and B11-D subsequently completed and are CLOSED. B11-E implementation subsequently PASSED. B10 and B11-B remain CLOSED; B11-A is complete. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

Talent Organization Overview Authority

M10 B5 exposes one bounded route: `GET /api/talent/organization-analytics/overview`. Its Program configuration grain has exactly two P1 count metrics: `programs_configured` counts enabled Program Academic-Year configurations in the authorized tenant/year universe; `active_programs` is exactly the subset whose durable `TalentProgram.status` is `active`. It does not infer active from Framework, Cycle, dates, or activity. Frozen population metrics use historical frozen Branch context, and Candidate and Official Identification counts require their independent permissions; Official Identification counts only `decision == identified`. HTTP output is constructed only after M9 primary privacy and B2 reconstruction closure through the `PrivacyClosedProjectionSet` boundary. No production privacy or commercial availability provider is approved, so production requests remain fail-closed.

Talent Map Authority

M10 B6 provides a factual Program-by-Branch or Program-by-Grade matrix over frozen Cycle membership. `branch_program` is presentation-only transpose after privacy closure. Matrix facts and backend row/column/authorized-scope totals are canonical Cells in compatible additive Relationships; rates derive only from closed numerator and denominator Cells. Candidate and identified metrics retain independent permissions, and identified means only `decision == identified`. Required Period execution, ranking, Program-native rubric/KPI comparison, and universal Talent scores are unsupported. Missing production privacy, availability, or breadth providers fail closed. The B6 sparse-matrix rule derives a total and its additive terms from the same authoritative numeric child collection. Rectangular `no_data` coordinates remain displayable but are absent-not zero-in the equation. Empty totals are `no_data`; a single authoritative child uses `Total - Child = 0`. The targeted independent sparse-matrix re-review passed, and B6 is closed.

Program Portfolio And Branch Intelligence Authority

M10 B7 Program Portfolio summarizes configured Programs for one tenant/year using common participation and coverage facts plus Program-grain required- Period execution. Branch Intelligence is one authorized historical frozen Branch context and never attributes Program/AY execution to that Branch. Shared Program/Branch facts reuse B6 canonical identities. Candidate and identified fields retain independent permissions. Both projections require breadth and traverse M9 primary privacy, B2 closure, and closed-only serializers. B7 itself adds no ranking, universal score, Grade breakdown, or Participation Overlap.

Participation Overlap Authority

M10 B8 exposes `GET /api/talent/organization-analytics/participation-overlap`. Participation is a distinct `(program_id, student_id)` fact from Open/Closed frozen Cycle population membership in one tenant/year and authorized historical Branch scope. Pair values are distinct Student intersections; the diagonal is the Program participant count. `participation_overlap` is exactly a P2 `count` at `program_participation` grain, and normalized Program pairs share one Cell and one privacy decision even when presented symmetrically. There are no additive row/column overlap equations or totals. Candidate/Identification overlap, Talent Breadth, ranking, scoring, and Student-identifiable output are absent; stateful cross-request controls remain a production privacy-provider gate.

Student Drill Authority

M10 B9 exposes `GET /api/talent/organization-analytics/students`, the first Student-identifiable M10 route. It requires `talent_analytics.view` AND `talent_analytics.view_students` (true AND composition), and derives Student inclusion only from frozen `TalentAssessmentCyclePopulationMember` context within the authorized tenant/Academic-Year/historical-Branch/Program scope; current `StudentAcademicPlacement` never decides inclusion or supplies displayed Branch/Grade/Section. Every identifiable Student row is P7 regardless of narrow scope, page size, or a Candidate/Identification field's own P5/P6 class - B9 evaluates one gate-level `student_drill_population/count/distinct_student` Cell, fed by distinct authorized Student count, through the identical B2 closure pipeline (`apply_primary_privacy_and_close/ PrivacyClosedProjectionSet`) every other M10 route uses, since there is no authoritative additive relationship across Student rows. Candidate (`talent_review_candidates.view`) and Official Identification (`talent_official_identifications.view`) fields are independently permissioned and query-skipped, not response-filtered - the key is absent, never `null`/`false`. Learner Profile access (`talent_learner_profiles.view`) exposes only an advisory `can_view_learner_profile` hint; the real Learner Profile route still independently authorizes. One top-level row exists per Student with explicit frozen Program/Cycle contexts. Pagination (limit 25/max 100, `has_more`, no `total_count`) is over distinct Students in deterministic name/id order - no Student Talent Score, ranking, AI field, or Educator Input. No direct Student-ID filter is exposed, to avoid an existence oracle. B9 adds no schema, migration, permission, entitlement, longitudinal/B10, UI, or AI capability. The distinct-Student remediation passed targeted independent re-review; B9 is CLOSED. Its committed/pushed implementation remains part of current dev.

B10-A Longitudinal Organization Intelligence Governance Authority (Approved) / B10-B Implemented

ADR 0027 governs B10 as an approved architecture decision, and B10-B has implemented it exactly as governed: `GET /api/talent/organization-analytics/ programs/{program_id}/longitudinal` (`talent_org_longitudinal.py`, the seventh Organization Intelligence route) is implemented and independently security/privacy reviewed with PASS and non-blocking observations; B10 is CLOSED. B9's own closure status above is unchanged, and no schema, migration, permission, or entitlement was added. B10 MVP is bounded to one Program, one Academic Year, and the ordered M8 `TalentPlannedEvaluationPeriod` slots within it, presented through the now-implemented route above. The Period is the presentation/order authority (ordered only by its governed `sequence`); its optional linked Open/Closed `TalentAssessmentCycle` supplies the factual evidence, per the database's `uq_talent_assessment_cycles_period` one-Cycle-per-Period constraint. The approved metric allowlist is exactly nine of the fourteen existing `MetricCode` values: `frozen_eligible`, `completed`, `completion_coverage`, `assessment_started`, `started_coverage` unconditionally, plus `candidate_count/candidate_of_eligible` and `identified_count/identified_of_eligible` under their existing permissions. `AcademicYear.year_name` remains descriptive only, never chronology authority; multi-Academic-Year longitudinal ordering is deferred. Comparability between adjacent Period points is exactly `comparable/ not_comparable`, never growth/improvement language, and B10 returns no server-computed delta/percent-change between points - an intentional privacy decision reusing the existing P1-P7 privacy classes and B2 closure pipeline with no new privacy class. See `docs/adr/0027-b10-longitudinal-organization-intelligence-contract.md` for the

complete decision, deferred-capability list, and open production gates.

B11-A/B11-B Organization Analytics Observability

B11-A (read-only qualification) is completed: the privacy, commercial- availability, and breadth-policy provider seams used by all 7 M10 routes existed with no production implementation and correctly failed closed at that checkpoint. F1 subsequently implements the governed providers. B11-B adds safe, non-sensitive operational observability (`projection_family`, `outcome`, latency, safe structural counts reused from each route's own breadth-policy inputs, and each provider's coarse missing/allowed-or- available/rejected-or-unavailable/exception outcome - never a Student identifier, raw value, privacy threshold, or Candidate/Identification decision) around all 7 routes, through a new dedicated logger in `talent_organization_analytics_observability.py`, kept separate from the immutable business/security audit trail. Every fail-closed HTTP outcome is unchanged byte-for-byte and a telemetry failure can never affect the HTTP response. Query-count instrumentation is deferred to a future "B11-C profiling" phase. B11-B is implemented and unit-tested; independent security/ privacy review passed with non-blocking observations and B11-B is CLOSED. No schema, migration, permission, entitlement, privacy threshold, or breadth limit was added by B11-B; F1 later adds the governed provider decisions. Every PostgreSQL/production ADR 0027 gates were subsequently resolved. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

Talent Annual Evaluation Planning Authority

The canonical planning chain is `TalentProgram -> Program Academic Year Configuration -> TalentAnnualEvaluationPlan -> TalentPlannedEvaluationPeriod -> optional TalentAssessmentCycle`. Plan is annual planning authority, Period is stable ordered planning identity, Cycle remains execution authority, and Framework Version remains owned by the exact Cycle. One Plan exists per annual configuration; Plans move `draft -> active -> closed`, Periods move `planned -> cancelled`, and neither Plan nor Period owns Framework or Student data.

Cycle linkage is nullable, unique in both directions, same-tenant/Program/year, and mutable only while the Cycle is Draft. Link/unlink requires Plan-manage AND Cycle-manage, organization/global scope, both expected revisions, and Plan -> Period -> Cycle locking. Existing/ad-hoc Cycles stay unlinked. M4 Cycle govern still opens a Cycle; linked context is revalidated but M8 govern is not added to Cycle Open. Plan reads disclose bounded Cycle projection only with Cycle view permission, and otherwise omit all relationship placeholders, derived actions, and cycle-derived warnings.

Planning Subject Requirement Removal (Single And Bulk)

`routers/planning.py:_get_planning_requirement_removal_status` classifies every displayed Planning requirement using the same explicit-first/legacy- fallback authority the rest of Planning already renders through (`resolve_section_subject_demands`), never a separate derivation: an untouched explicit `PlanningSubjectDemand` row is `removable`; a row Curriculum Adjustment has ever created or modified (`updated_by_user_id` set) is `permanent`, shown with an explicit "Protected (Curriculum Adjustment history)" explanation rather than a silently hidden action; a requirement resolved only from the Subject catalog with no explicit row at all is `fallback`; anything else is `not_found`. Both `removable` and `fallback` are removable through the same action - an explicit row is deleted, while a `fallback` requirement is removed by creating an explicit `is_active=False/weekly_periods=0` row. That row deliberately does NOT match a Curriculum Adjustment retirement stamp-for-stamp: it sets only `created_by_user_id` (the acting admin, for audit) and leaves `updated_by_user_id` NULL, so it stays classified `removable`/setup-only rather than immediately becoming permanent. Only a row Curriculum Adjustment itself has touched is ever `permanent`; a leftover setup-only row is automatically cleaned when its otherwise dependency-free section is deleted, preserving the remove requirement -> remove section -> remove subject workflow without exposing an implementation artifact. This closed a genuine gap: previously only explicit-row requirements ever showed a removal action, so identical-looking subject rows for the same teacher could differ in removability for reasons unrelated to the teacher.

`POST /planning/subject-requirements/remove` handles single and checkbox-selected bulk removal atomically, with selection supported across every section on the page in one request. Each expanded section provides a section-scoped **Select all** control and an adjacent trash-icon action. Its label reflects the actual selection as "Remove Subject Requirement" or "Remove N Subject Requirements"; the page-level action mirrors the same state. Every target is scope-validated (SchoolGroup/branch/academic year) and status-classified before any mutation; if any target is protected, invalid, or out of scope, nothing is removed and every blocked target is named with its reason - the same all-or-nothing contract as Subject bulk delete. Confirmation is required before both single and bulk removal. `TeacherSectionAssignment`, `TimetableEntry`, and `CurriculumAdjustmentAudit` rows are never touched. The prior round's `GET /planning/subject-demand/delete/{demand_id}` route remains functional for active untouched setup rows. Section delete itself now removes only inactive zero-period rows with `updated_by_user_id IS NULL`, in the same transaction and exact branch/year scope; active demand, Curriculum Adjustment history, and every other protected dependency still block.

Cline And Three-Agent Development Pool

Codex, Claude Code, and Cline/ClinePass are approved TIS development agents. Cline's repository entry points are `.clinerules/tis.md` and `.cline/skills/tis-kms-developer/SKILL.md`. They require governing KMS onboarding, implementation inspection, isolation/RBAC and database safety, minimal diffs, proportional tests, task-level KIA, KMS sync/check, and deployment reporting.

The master conversation balances tasks by complexity, specialization, risk, independent-review value, and current usage/load, without permanent assignments. See [Project Governance](engineering/PROJECT_GOVERNANCE.md) for the allocation policy and [AI Coding Workflow](engineering/AI_CODING_WORKFLOW.md) for Cline setup/invocation. This configuration changes no application, schema, or production behavior.

Claude Code Repository Configuration

Claude Code is governed through three repository-native instruction layers: root `CLAUDE.md` provides the concise entry point, `.claude/skills/tis-kms/SKILL.md` defines the reusable KMS-governed working method, and `.claude/agents/tis-kms-developer.md` defines a specialized bounded subagent. They defer to root `AGENTS.md` and the authoritative KMS rather than duplicating project architecture as a competing source of truth. Every Claude Code task must still preserve the existing worktree, update `.kms-impact.yml`, synchronize the KMS when knowledge changes, run final validation, and avoid commits, pushes, deployments, production mutations, and `tis.db` changes unless explicitly approved.

Return-To Reopen-On-Load UI Pattern

`redirect_utils.py` centralizes the open-redirect guard (`safe_redirect_path`) and fragment-preserving redirect helpers previously private to `main.py`; `main.py` now imports them under their original names so every existing caller is unaffected. `static/js/reopen-on-load.js`, loaded globally from `base.html`, reopens whatever `<details>` element matches a target id taken from `window.location.hash` or an inline `window.__tisReopenTargetId`, scrolling it into view, and is a silent no-op for a missing or stale id. Planning's "Remove demand" action (`routers/planning.py: delete_planning_subject_demand`) is the reference implementation: the section's `<details>` gets a stable id, the link passes `return_to` to that section's fragment, and the route's in-place blocked-render paths set the same reopen target without a redirect. This is not an SPA conversion and adds no browser storage. Subjects and Teachers were not changed; this establishes the pattern for later, incremental adoption elsewhere.

Planning And Subject Delete Dependency Guards

`routers/planning.py:delete_planning_section` and `routers/subjects.py`'s single/Bulk Delete perform a read-only dependency scan before any mutation and block with a specific, customer-safe message naming every blocking category, instead of raising an unhandled database error. Planning section delete checks

`TeacherSectionAssignment`, `PlanningSubjectDemand`, `TimetableEntry`, `TeacherSchedulingRuleTarget`, `CalendarEvent`/ `CalendarEventSectionTarget`, and section-scoped `SubjectDistributionRule`. `TeacherSectionAssignment` is now a blocker like every other dependency rather than being silently deleted alongside the section, since no documented product rule required that cascade. Subject delete/Bulk Delete additionally check `TimetableEntry.subject_code`, `CurriculumAdjustmentAudit` source/target codes, and `SubjectDistributionRule.subject_code`, none of which had an application-level check before, so a Subject referenced only by one of those could previously be deleted and silently orphan that historical reference.

Planning subject demand is classified removable or permanent using the existing `PlanningSubjectDemand.updated_by_user_id` column, set only by Curriculum Adjustment (`curriculum_adjustment_apply_service._set_demand`) and never by the one-time setup backfill migration. A row with no `updated_by_user_id` has never been acted on by an admin - pure setup scaffolding - and can be hard-deleted through the new, permission-gated `GET /planning/subject-demand/delete/{demand_id}` action ("Remove demand" on the Planning page), after which the section or subject becomes deletable. A row Curriculum Adjustment has ever touched, active or retired, is genuine history that TIS preserves permanently and remains an unresolvable blocker with an honest explanation. Timetable placements follow the same principle without a new action: only entries in a mutable Draft can be removed, while published/active/superseded/archived placements are permanent history. Bulk Subject delete is atomic: any blocked Subject in the selection stops the whole batch, and every blocked Subject is named with its specific reason and, where applicable, the Remove-demand action that resolves it. No archive/ closed/inactive lifecycle, broad schema change, migration, or cascade deletion was introduced.

Guided Late-Stage Curriculum Adjustment

The administrator workflow at `/planning/curriculum-adjustments` is gated by `curriculum.adjust` and consumes the established preview/apply contracts. It supports grade, selected-section, and all-active-use scopes, displays section demand, teachers, capacity, scheduling rules, grouped configuration, and Draft impact, then requires explicit teacher selections and confirmation. Stale review responses return the user to a refreshed preview. Applying does not regenerate the Draft or change published history.

`requested_transfer_periods` is the preview/apply request authority. Per affected section, source after equals source before minus that count and target after equals target before plus it. Partial transfers retain active source demand; only zero retires it. The Subjects list resolves exact-scope Current/New section demand through the explicit-first service and shows either the uniform effective value or **Varies** with section values, without mutating catalog defaults.

`adjustment_type` distinguishes `transfer` from `reduce_only`. Reduce-only omits the target, changes only source demand, reduces the assigned source teacher's calculated load, and requires no teacher decision payload. The workflow retains one grade, selected sections, or all active uses of one exact subject code; it does not merge grade-specific subjects into one multi-grade transaction.

Atomic Late-Stage Curriculum Adjustment Application

`curriculum_adjustment_apply_service.py` owns the Stage 4 write transaction. Apply requires the exact Stage 3 request and fingerprint plus one explicit target-teacher decision per affected section. Duplicate successful fingerprints resolve through the audit ledger without applying twice. The service locks scoped Planning demand, assignments, rules, timetable versions, and active generation state before rebuilding the preview; any drift or blocker aborts before mutation.

Successful apply changes only the selected Current/New section demands. Source zero is retained as inactive retirement evidence, target demand becomes explicit, source assignment is removed only when retired, and target assignment follows only the confirmed teacher or unassigned decision. Chosen teachers must be exact-scope, target-allocated/qualified, and within final international capacity. Active source section rules are retired with zero demand; unresolved target rule arithmetic fails closed. Invalid source/teacher locks block apply. The unpublished Draft

retains its snapshot and placements but becomes stale, loses approval, advances revision, and requires later regeneration. Published history and `TimetableActiveVersion` remain immutable.

Read-Only Late-Stage Curriculum Adjustment Preview

`curriculum_adjustment_preview_service.py` is the Stage 3 analysis boundary for future late-stage subject retirement/reduction/transfer workflows. It accepts only an exact SchoolGroup, branch, and academic year and selects Current/New Planning sections by grade, explicit section IDs, or active source-subject use. The preview uses `PlanningSubjectDemand`, retains missing-row fallback through the existing resolver, and performs no mutation.

Each section reports source/target before and proposed periods, current explicit or HRT-resolved teachers, ranked but uncommitted teacher options, projected capacity, Subject Scheduling Rule arithmetic, grouped legacy warnings, and blockers. A mutable Draft is reported as becoming stale and requiring regeneration only after a future apply; published history is never selected for mutation. The preview fingerprint covers scope, request, resolved section analysis, teacher/rule/configuration inputs, and Draft identity/revision/authority so later confirmation can fail closed after authority changes. The API requires `planning.edit_section`.

Explicit Section Subject Demand Transition

`PlanningSubjectDemand` is the normalized future authority for one section's subject and weekly-period requirement. It is isolated by branch and academic year, supports retained inactive/retired rows, and database constraints prevent cross- scope section or Subject references and duplicate active section-subject demand. Migration `20260828_004_planning_subject_demands_foundation` seeds only Current/New Planning sections from the existing grade-aligned Subject catalog and is idempotent.

Stage 2 makes the explicit-first resolver authoritative for live Planning demand, teacher workload arithmetic, timetable readiness/workspace/snapshot/generation input, Subject Scheduling Rule validation, and required-hours reporting. An explicit inactive or zero row suppresses fallback and represents durable retirement for that section. `Subject.grade + Subject.weekly_hours` remains a transitional fallback only where no explicit section-subject row exists. Existing teacher assignment identity, solver behavior, timetable UX, and published history are not rewritten.

Smart Timetable Customer And Published-Visibility Boundary

Stage 5.2 keeps internal versions but presents Configure ? Create New Timetable ? Draft ? Generate/Review/Edit ? Publish. The newest mutable non-active exact-scope version is the current Draft Timetable; generated results continue that logical Draft while source rows remain internal provenance. Technical selection, comparison, historical exports, archive, and permanent cleanup actions live in Timetable History.

Publishing remains the atomic exact-scope active-pointer swap. Deleting an eligible never-published draft removes its placements and version while preserving snapshots, runs, and published history; History bulk cleanup uses dependency-safe child-first deletion and reports protected remnants. Official non-management reads use `timetable_visibility_service.py`, which accepts only `TimetableActiveVersion`. Teacher identity follows the existing scoped `User.user_id == Teacher.teacher_id` convention.

Smart Timetable Generation Authority

Academic scheduling quality is configured per branch and academic year using authoritative subject codes. Core English, Mathematics, and Science receive hard daily coverage when demand permits and distinct-day soft spread otherwise. Art, Well-being, Social Studies, Reflection, and ICT mappings receive configurable spread and non-consecutive preferences; ICT may enforce one session per day. Explicit Swimming groups synchronize selected sections and account for one common teacher and an optional capacity-limited resource. Independent validation repeats all hard rules.

Stage 5.1 makes Planning-resolved section/subject/teacher demand, including subject-specific Grades 1-2 HRT fallback, the only lesson authority for automatic generation. Schema-v3 snapshots freeze exact

SchoolGroup/Branch/Academic-Year scope, canonical composed teaching slots, demands, locks, source revision and regeneration arrangement. OR-Tools is task-runtime-only; web processes commit a run, dispatch only its public ID to an on-demand Render Workflow task, and poll TIS state. The task claims the exact run under PostgreSQL locking and retains lease/heartbeat protection against duplicates and late persistence. Render automatic retries are disabled, and the old infinite worker loop is an optional local fallback only.

Hard-valid candidates must pass an independent validator and a current-input check before atomic persistence as unpublished generated/regenerated publication-ready versions. Generation never changes `TimetableActiveVersion`. Freshness authority uses stable Planning, canonical timetable configuration, generation constraints, and locks; runtime generation metadata is retained in snapshots but excluded from the freshness comparison. Regeneration preserves locks and requires a calculated minimum difference; random seed alone is not diversity authority. Publishing remains a separate permission and transaction.

Capacity-Based Commercial Packaging

ADR 0025 establishes one normal customer feature baseline for coherent active paid, promo, and customer-demo workspaces. Starter, Professional, and Enterprise AI differ by organization capacity and billing context, not by ordinary product modules. Their established 1/5/25, 5/20/100, and 25/100/500 branch/staff/teacher ceilings remain unchanged; Custom remains contact-only above Enterprise.

Commercial source and lifecycle decide whether the workspace may operate. Permissions and tenant/branch/academic-year scope decide what a user may do. Normal feature policy is resolved centrally, and branch entitlement is required for branch-scoped actions. Platform, Developer, System Owner, global audit, promo-admin, and demo-admin capabilities are excluded. AI availability uses the common baseline, while AI consumption uses a separate policy boundary and the existing durable accounting model.

Stage 5 SchoolGroup Management Boundary

Top-level SchoolGroup creation and deletion from operational configuration are platform-global actions, not tenant administration. Each requires Platform identity, `schools.manage_all_schools`, and the matching create/delete permission. Tenant Administrators no longer receive those permissions by default, and stale permission state cannot bypass the identity guard. Tenant SchoolGroup updates remain available only for the user's linked SchoolGroup; foreign IDs fail before mutation.

School Management consumes the unified commercial authority facade for branch capacity presentation. Promo and paid tenants therefore see authoritative used, allowed, and remaining branch capacity while branch mutations retain the existing locked, source-aware enforcement and atomic promo evidence. A read-only PostgreSQL provenance audit can identify active internal sandboxes with missing tenant-link and workspace-entitlement evidence for Platform Owner review; it performs no repair.

Controlled Existing Workspace Conversion Boundary

M4B is the only supported bridge from an audited internal sandbox to a customer workspace awaiting commercial activation. A durable operation binds the exact SchoolGroup identity, normalized intended owner, approved M4A evidence hash, canonical parameters, branch/dependency snapshot, sandbox entitlement snapshot, required setup fields, actors, lifecycle stage, and idempotency key. Conversion events are append-only and redacted. A partial unique database index permits at most one active `tenant_owner` account link per SchoolGroup; migration stops for manual review if legacy duplicates exist.

The operation is also the ownership claim. It does not fabricate an account or password. The intended owner must use ordinary registration and verification, then explicitly claim the workspace. Alignment rejects duplicate, suspended, unverified, platform, or cross-tenant identities and reuses an existing same- tenant operational User when safe. An existing different owner requires a separate Platform Owner transfer approval.

Only legal organization name, IANA timezone, and educational program are collected. Final execution locks and revalidates the operation and workspace, performs a fresh M4A audit, rejects branch or commercial drift, retires the one active internal-sandbox entitlement, and transitions to `customer / provisioning` without creating a replacement entitlement or TenantProvisioningLink. M1 calls that state `activation_required`; Organization Account remains available while operational access fails closed. M3 promo activation is available. Paddle activation for an already existing workspace is a deferred milestone and must not be inferred from normal new-onboarding checkout support.

Existing Workspace Conversion Audit Boundary

Before a legacy internal sandbox may enter any controlled customer conversion, M4A requires an explicit read-only audit keyed by SchoolGroup ID, workspace UUID, exact organization name, and normalized intended-owner email. The audit reflects the deployed schema, traverses branch foreign-key descendants, identifies unconstrained branch references, and inventories ownership, provisioning, entitlement, subscription, demo, and promo evidence. It emits counts and allowlisted metadata only; provider identifiers are reduced to presence flags and secrets or private payloads are never included.

`scripts/audit_existing_workspace_conversion.py` runs only on PostgreSQL in a repeatable-read, read-only transaction and always rolls back. Deterministic JSON and text formats expose the observed transaction mode, a stable SHA-256 snapshot, conservative archival candidates, and explicit exit codes: `0` coherent, `1` execution/configuration failure, `2` manual review, and `3` workspace identity mismatch. Soft-deleted rows still count as dependencies; an unmodeled branch foreign key blocks archival recommendations. Its result is evidence for a later design decision, not conversion authority. It cannot archive a branch, align an owner, change workspace metadata, create a tenant or commercial source, call Paddle, send email, approve deletion, or perform the future conversion. Any identity conflict, commercial conflict, incomplete schema coverage, or uncertain relationship requires manual review.

Promo Redemption And Activation Authority

Promo access is an explicit third customer commercial source beside paid subscription and customer demo. A verified organization owner may redeem an active, approved, in-window definition from completed onboarding or from an existing workspace that a separate controlled process already classified as `customer`, left in `provisioning`, linked to that owner, and left without any commercial source. Active internal sandboxes are never converted implicitly.

Final activation locks the activation session, promo definition, and target workspace; revalidates scope, limits, source compatibility, people usage, and branch selection; then creates immutable redemption/grant evidence, explicit branch access, a promo workspace entitlement, and the promo tenant link in one commit. Exactly one tenant-link source is required: `SubscriptionContract`, `SaaSDemoRequest`, or `PromoGrant`. Pending activation gives no access and Paddle is not involved.

Branch selection is the only reversible capacity selection in M3. Existing branches above the grant require exactly the allowed count; fewer branches are all selected and unused capacity remains available for normal future branch creation. The final entitlement inventory covers every preserved branch: selected branches have an assignment and active entitlement; unselected active or inactive branches have an explicit inactive entitlement. Operational status is unchanged. Individual and bulk reactivation acquire the workspace lock, enforce grant capacity, and update status, assignment, and entitlement atomically. The dry-run-first `reconcile_promo_branch_entitlements.py` command may add only missing inactive evidence for a coherent active promo source and otherwise fails closed. Staff and teachers are never modified. Any organization-wide excess blocks activation until the owner changes operational usage and resumes.

Billing & Subscription is source-aware after organization selection and permission validation. Paid workspaces retain the existing Paddle subscription portal; demo workspaces retain their dedicated demo journey; promo workspaces render a dedicated Commercial Access page. Promo presentation composes M1 authority for plan, status, capacity usage, and remaining capacity with the attributable grant/redemption for effective dates and a masked reference. Normal operational access ends exactly at the grant's `effective_to`; grace days provide only a customer recovery window and do not extend access.

Expired and recovery-period promo owners may continue through the existing-workspace paid plan and Paddle checkout path. Promo remains the sole authority while checkout is pending, failed, abandoned, or unconfirmed. A verified completed transaction locks and revalidates the existing tenant and atomically replaces promo source evidence with paid contract, subscription, workspace entitlement, and branch entitlement evidence. The SchoolGroup, workspace UUID, branches, users, teachers, academic data, branding, ownership, and immutable promo records are retained. Active-promo early conversion is not supported. Shared commercial badges are generated from centralized authority for Organization Account and full commercial presentation. The normal operational shell does not expose commercial source, plan, or lifecycle identity.

Promo Definition And Governance Boundary

Platform Console now manages definition-only promo offers for the three active self-service plan tiers. A definition selects exact positive capacity within the existing plan ceilings, a controlled scope, validity and one access-expiry policy, redemption-policy metadata, and optional eligible existing branches. Draft, Active, Paused, and Revoked are persisted; Expired is derived. Active material edits require pause and return the versioned definition to Draft.

Raw promo codes are generated by TIS, shown once, and never persisted. Lookup authority is a deterministic HMAC-SHA256 hash under a dedicated secret and key identifier. Platform permission and owner-approval rules are separate from tenant permissions and capacity. M3 consumes only an approved active definition through the separate locked redemption path; M2 definition actions still do not grant access or call Paddle.

Organization Account Sign-In Boundary

Public SaaS Sign In resolves an activated organization owner or authorized account manager to `/saas/account`, the Organization Account Overview, rather than directly to the operational workspace. The overview separates organization profile, branch visibility, billing/subscription, and account security by the linked operational user's existing permissions. Enter TIS Platform is the only customer-account action that enters `/login`, and operational commercial-access and authorization checks still apply there. Restricted account managers retain safe account and billing recovery access without operational entry; incomplete accounts resume onboarding; non-management users retain their approved role-based destination; and multi-organization accounts select an organization before its overview is shown. That selection is an HTTP-only UUID hint that is accepted only after revalidation against the current account links and existing permissions; it cannot grant cross-tenant access.

Customer Journey Continuity

TIS treats demo or subscription expiry as an expected commercial state. Returning routing is derived from SaaS onboarding, request, durable workspace link, lifecycle, and subscription records. Expired access is intercepted before protected workspace services and shown with plan, renewal, support, and sign-out actions. Expired-demo checkout quotes the preserved workspace's active branches and must never create another organization or SchoolGroup. Customer-visible language uses "TIS team" or "TIS support team"; Platform Owner remains an internal role.

M8B9 Demo Operations, Notifications, And Testing

Customer Demo operations are generic and Platform Owner-only. Immediate expiry is reversible; reactivation and custom expiry reuse the same workspace and require a future date with no maximum. Manual reminders are limited to the final organization-calendar day, and manual lifecycle runs call the production M8B7 processor. Standard preserves M8B8 usage, Full remains permission/expiry bound, and Custom supports registry-controlled workspace and branch policy.

M8B8 AI Entitlements And Commercial Foundation

AI access resolves centrally after tenant scope and `ai.use` permission. Commercial state remains authoritative: active Internal Sandbox is unlimited, active Customer Demo receives its configured successful-use allowance per registered

feature, and expired/restricted demos fail closed. Enabled AI availability uses the common customer `module.ai` baseline for paid, promo, and active demo authority; globally disabled definitions remain denied. Usage is durable, auditable, idempotent by operation key, concurrency-safe, and separated into internal, demo, promo, and paid metric contexts. Availability and consumption policy are independent. No executable AI tool or M8B9 owner override/reset operation exists.

M8B7 Demo Customer Journey

M8B7 projects customer Pending, Active, Expired, and Declined presentation from the existing request, provisioning, and lifecycle sources. Approval invokes the separate retryable provisioning service and creates activation communications only after success. Six email types use the existing branded provider through a durable deduplicated outbox; demo events reuse the Platform Owner Notification Center. The shared tenant shell shows active demo status. A coherent expired demo may convert after authoritative confirmed payment by reactivating and relinking the same SchoolGroup and preserving its UUID and all tenant data. Sandbox, checkout-return, and unverified evidence remain non-authoritative.

Documentation version: 3.0

Last major context update: 2026-06-27

Product Identity

TIS stands for Teacher Information System. It is a developing SaaS academic operations platform for schools, school groups, academic leaders, supervisors, and platform owners who need one trusted place for academic staffing, teacher records, planning, calendars, observations, branch context, and future intelligence.

TIS is not only a teacher directory. It is intended to become the operational backbone for academic decision-making across a school or multi-branch organization.

Public product presence:

- Public website: `https://tisplatform.com`
- Application portal: `https://app.tisplatform.com`
- Operational login: `/login`
- SaaS signup: `/saas/signup`
- SaaS login: `/saas/login`
- SaaS account area: `/saas/account`

Product Vision

TIS helps schools move away from scattered spreadsheets and disconnected operational files. The product vision is to give academic leaders a structured, secure, and tenant-isolated platform where teacher information, teaching loads, staffing needs, observations, calendars, and branch-level context can be managed together.

As the platform matures, TIS should also become the trusted data foundation for AI-assisted academic operations. Future AI features should depend on verified school data, clear permissions, and careful subscription packaging.

Business Goal

The business goal is to establish TIS as a subscription-based SaaS platform for academic operations. The platform should support school onboarding, plan selection, payment, provisioning, account management, and scalable multi-tenant usage.

Near-term business priorities:

- Convert interested schools from public landing page traffic into SaaS accounts.
- Support clear onboarding from signup through school setup.
- Provide reliable billing and payment status visibility.
- Enable platform owners to manage pending organizations and provisioning.
- Preserve trust by keeping tenant data isolated and operational flows stable.

Educational Goal

The educational goal is to reduce administrative friction so schools can make better academic decisions. TIS should help leadership teams identify staffing gaps, workload problems, incomplete academic coverage, observation follow-up needs, and calendar conflicts earlier.

The platform should support educational quality by making academic operations easier to see, review, and improve.

Target Customers

Primary customers:

- Private schools
- Multi-branch school groups
- Academic leadership teams
- Principals and vice principals
- Department heads and supervisors
- School operations and HR-adjacent academic staff

Internal platform users:

- Platform Owner
- Platform Co-Owner
- Platform Developer

Tenant users:

- Administrators
- Coordinators
- Supervisors
- Teachers and academic staff, depending on enabled workflows

FastAPI App Architecture

The operational TIS application is a FastAPI application at the repository root. It uses Python, SQLAlchemy, Jinja templates, static assets, and modular routers.

Core app areas:

- `main.py`: primary FastAPI app, route registration, app-level workflows, startup checks, platform console routes, dashboard and configuration flows.
- `models.py`: main SQLAlchemy models.

1 `database.py`: database connection/session setup.

1 `db_migrations.py`: local schema migration and repair logic.

1 `auth.py`: authentication, role normalization, platform identity helpers, permission helpers, session handling.

1 `authorization.py`: route permission rules and access-denied response helpers.

1 `permission_registry.py`: permission groups, labels, defaults, developer-assignable permissions, and system owner permissions.

1 `role_permission_service.py`: role permission persistence and related helpers.

1 `ui_shell.py`: shared application shell context, navigation, visual identity, and page metadata.

1 `academic_grade.py`: shared canonical KG/1-12 grade vocabulary and normalization, consumed by Planning and by Student Academic Placement.

1 `student_academic_service.py`: Student identity, external identifiers, and effective-dated Academic Placement authority.

1 `talent_program_service.py`: Talent Program, annual configuration, Framework Version lifecycle, competency lineage, Framework-specific rubric/descriptors, optional bounded KPI, and deterministic Review Candidate Policy configuration authority.

1 `talent_assessment_cycle_service.py`: SchoolGroup-wide Assessment Cycle lifecycle, explicit-time Draft eligibility preview, atomic opening population, accepted ADR 0033 additive-only Open roster synchronization after confirmed Placement plus organization-governed reconciliation when an existing Open assessment is loaded, and population fingerprint authority; Closed membership remains final.

1 `talent_student_assessment_service.py`: exact frozen-member-bound Student Assessment lifecycle, expected-revision competency result mutations, deterministic completion, persisted Framework-specific integer KPI provenance, and bounded operational audit.

1 `talent_review_candidate_service.py`: deterministic Review Candidate evaluation/materialization from a Completed Assessment against its exact M3 Review Candidate Policy, plus the two-state (`pending_review`/`reviewed`) Review workflow transition; read-only against Assessment/Result/frozen-population data.

1 `talent_official_identification_service.py`: append-only Official Identification decision authority (`identified`/`not_identified`), gated on a Reviewed Review Candidate, exactly one decision per candidate, requiring organization/global access scope in addition to permission; an exact composite FK prevents its duplicated historical context from drifting from the candidate.

1 `talent_educator_input_service.py`: bounded qualitative Educator Input authority with historical Placement/Branch resolution (frozen Cycle context or effective-dated Placement at `observed_at`) and append-only amendment/supersession lineage; routers authorize the resolved historical Branch before commit and database uniqueness prevents concurrent lineage forks.

1 `talent_learner_profile_service.py`: read-only, per-Student aggregation of authorized M1-M6 historical evidence; it never materializes a profile or changes source records.

1 `talent_analytics_service.py` / `talent_analytics_privacy.py`: M9 read-only aggregate query/metric authority and the privacy-decision/complementary-suppression engine (`Cell`/`Group`, `apply_primary_privacy`, `run_complementary_suppression`) shared by every `/api/talent/analytics` route; no production `TalentAnalyticsPrivacyPolicy` is approved yet.

1 `routers/`: modular feature routers for users, teachers, subjects, planning, timetable, academic calendar, observations, students, Talent Programs, Assessment Cycles, Student Assessments, Review Candidates, Official Identifications, Educator Inputs, Learner Profiles, Annual Evaluation Plans, and Talent Analytics.

1 `templates/`: Jinja templates for the operational app and SaaS app pages.

- `static/`: CSS, JavaScript, images, branding assets, and generated public artifacts.
- `tests/`: pytest coverage for tenant isolation, SaaS phases, platform access, permissions, email, branding, and related workflows.

Important operational route families:

- `/login`: operational app login.
- `/dashboard`: tenant operational dashboard.
- `/platform`: platform console for platform identities.
- `/system-configuration`: branch, year, branding, role permissions, and configuration workflows.
- `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, `/observations`: core academic operations.
- `/api/students`: Student identity, external identifiers, and Academic Placement JSON API (`routers/students.py`, `students.*` permissions). No page/UI exists yet.
- `/api/talent/programs`: Talent Program, Framework Version, Competency, rubric/level/descriptor, optional KPI, and Review Candidate Policy JSON API (`routers/talent_programs.py`, `talent_programs.*` permissions). M3 semantic mutations are Draft-only and expected-revision protected; no page/UI exists yet. `GET .../frameworks/{framework_id}` additively exposes each competency's own `FrameworkCompetency.id` (alongside the pre-existing parent-catalog `competency_id`), and `GET .../frameworks/{framework_id}/configuration` additively exposes each `levels[]` entry's own `TalentRubricLevel.id`, so a future assessment-entry UI can read the exact identifiers the `/api/talent/assessments/{id}/competency-results/{framework_competency_id}` write route is FK-constrained on. Purely additive read fields; no permission, schema, migration, or write-route contract changed.
- `/api/talent/assessment-cycles`: internal Evaluation-context compatibility/provenance API plus the current-eligibility read used by Student Assessments. ADR 0035 supersedes Draft/Open/frozen-population state as an Assessment eligibility gate. Legacy Open/Close/synchronization endpoints remain for backward compatibility and historical analytics, but normal assessment work does not require them.
- `/api/talent/assessments`: Student Assessment and exact Competency Result API (`routers/talent_assessments.py`, `talent_assessments.view/manage/complete`). Starting an Assessment uses the Student's current effective Academic Placement in the selected Academic Year, Program eligible Grade, authorized Branch scope, and the exact assessable framework/tool already referenced by the Evaluation; Program/Framework lifecycle labels are not additional assessment-start gates. The referenced framework must contain at least one competency and one rubric level; the existing Cycle Population Member row is captured/reused only as historical placement/framework provenance. Results are editable only while the Assessment itself is In Progress and use expected-revision protection; Cycle Draft/Open status is not an editability gate. Completed requires every Framework Competency; it and Incomplete/Insufficient Evidence are read-only. An enabled KPI uses integer weighted contributions over denominator 10,000, nearest-integer ROUND_HALF_UP, and persists Framework-scale result provenance only at completion. Qualitative/no-KPI completion has no result sentinel. Corrections and assessor assignment remain deferred.
- `/api/talent/review-candidates`: deterministic Review Candidate evaluation/materialization from a Completed Assessment plus the Review workflow transition (`routers/talent_review_candidates.py`, dedicated `talent_review_candidates.view/manage`, distinct from `talent_assessments.*`/`talent_programs.*`). `POST .../evaluate` is read-only against Assessment evidence, idempotent, and persists a candidate row (starting `pending_review`) only when the exact M3 policy is satisfied - a non-qualifying evaluation is structurally audited instead. `POST .../{id}/review` (`.manage`) is the one-way `pending_review` -> `reviewed` transition. Branch authorization resolves the frozen Cycle Population Member.
- `/api/talent/official-identifications`: append-only Official Identification decision API (`routers/talent_official_identifications.py`, dedicated

talent_official_identifications.view/record). POST "" requires .record AND organization/global access scope (a Branch-scoped actor is denied even with the permission) and only succeeds once the target Review Candidate is reviewed; exactly one decision per candidate is enforced by both a unique constraint and a service check (409 on a second attempt). identified/not_identified are equally durable; there is no mutation/revocation route.

- /api/talent/educator-inputs: bounded qualitative Educator Input API (routers/talent_educator_inputs.py, dedicated talent_educator_inputs.view/add/amend). POST "" resolves and persists historical Placement/Branch/Grade/Section context (frozen Cycle context if supplied, otherwise the Student's Placement effective at observed_at); POST /{id}/amend creates a new append-only row referencing the one it supersedes. GET "" returns only the current/latest version per lineage chain; GET /{id}/history returns the full chain. Branch authorization uses the row's own persisted historical Branch. Revocation, supersession of Official Identification, re-identification, and Educator Input analytics/export/AI/attachments remain explicitly deferred (M6 Decision 17/18 and the M6 Governance Review in docs/AI_PROJECT_CONTEXT.md).

- /api/talent/learner-profiles/{student_id}: read-only longitudinal profile (routers/talent_learner_profiles.py, talent_learner_profiles.view). Base permission exposes authorized Placement and Assessment history only; Review Candidate, Official Identification, and Educator Input each additionally require their own M6 .view permission, otherwise the complete data class and events are omitted. Historical Branch filtering applies independently to every visible record. No profile table, current Talent status, cross-Framework score, audit read event, M8 periods, or Learning Style exists.

- /api/talent/analytics: M9 read-only, aggregate-only, privacy-gated analytics API (routers/talent_analytics.py, dedicated talent_analytics.view/talent_analytics.view_students) - context, overview, rubric-distribution, kpi-distribution, competencies, breakdowns/{branch|grade|section}, period-comparison, students. Every route runs authorized raw aggregate -> apply_primary_privacy -> run_complementary_suppression -> comparability -> serialization; Candidate/Identification analytics are query-skipped without their own .view permission; kpi-distribution reports distribution_mode: unavailable rather than any numeric bin. resolve_privacy_policy_provider() delegates to the configuration-driven Release 1 provider builder. Production returns a provider only when the governed analytics environment configuration matches the approved Release 1 values; missing/invalid configuration returns None and fails closed with analytics_query_failed without exposing threshold values. No Talent Score/Index/Potential Rate, Talent Map, ranking, export, or AI exists.

Next.js Landing Architecture

The public marketing website lives in tis-landing-website/. It is separate from the FastAPI operational portal.

Landing architecture:

- Runtime: Next.js / Node.
- Public website domain: https://tisplatform.com.
- Local development URL: http://localhost:3000.
- Main page: tis-landing-website/src/app/page.tsx.
- App layout: tis-landing-website/src/app/layout.tsx.
- Global styles: tis-landing-website/src/app/globals.css.
- Logo component: tis-landing-website/src/components/tis-logo.tsx.
- Public assets: tis-landing-website/public/.

- M8 landing integration uses `NEXT_PUBLIC_TIS_APP_BASE_URL` as the configurable deployed application base URL. Public Open Account CTAs build one shared registration destination at `/saas/signup`.
- The hero Subscribe Now action scrolls to `#pricing`. Starter, Professional, and Enterprise AI enter public signup with an allowlisted preferred-plan code. The preference creates no plan, checkout, payment, or Paddle record and is applied only after existing branch, system-user, and teacher capacity validation. Custom remains contact-only.

The application portal remains separate:

- App domain: `https://app.tisplatform.com`.
- Runtime: FastAPI / Python with a relational database.

Legacy landing files in the FastAPI app are not the source of truth:

- `templates/landing.html`
- `static/landing/landing.css`

Those legacy files must not be modified unless explicitly approved.

Multi-Tenant SaaS Strategy

TIS is designed as a multi-tenant SaaS platform. Tenant isolation is a critical rule. School groups, branches, users, academic years, teachers, planning data, timetable data, observations, and configuration records must remain scoped to the correct organization and branch context.

The platform distinguishes between:

- Platform identities: platform owners, co-owners, and developers.
- Tenant identities: users belonging to a school group and branch context.
- SaaS account identities: accounts that move through signup, onboarding, billing, and provisioning.

Platform users may inspect or switch organization context through controlled platform workflows. Tenant users must remain inside their authorized school, branch, academic year, and permission scope.

Critical tenant strategy:

- Do not weaken tenant isolation.
- Do not bypass permission checks.
- Do not assume a platform identity is a tenant identity.
- Do not assume a SaaS account is already provisioned into an operational tenant.
- Keep onboarding, billing, and tenant provisioning as distinct stages.

Workspace Classification Foundation

M8B-1 introduces a metadata boundary between workspace identity, workspace classification, and operational tenant state:

- `SchoolGroup.workspace_uuid` is the stable unique workspace identifier.
- `SchoolGroup.workspace_classification` accepts only `internal_sandbox`, `customer_demo`, or `customer_paid`.
- `SchoolGroup.workspace_lifecycle_status` accepts only `provisioning`, `active`, `suspended`, or `archived`.

1 `PendingOrganization.workspace_intent` carries pre-provisioning intent.

1 `SaaSAccount.account_purpose` distinguishes internal-test and customer account purpose.

1 `User.is_internal_test_identity` identifies operational identities attributable to internal test data.

The M8B-1 fields are not authorization, entitlement, payment, or reset gates. Existing flows continue unchanged. Classification conversion is rejected except for the dedicated M8B-6 Customer Demo to Customer Paid service, and the commercial-state service remains the authority for the effective commercial result. Every pre-M8B-1 workspace/onboarding record is confirmed test data and is assigned internal sandbox/test metadata by the controlled one-time backfill. No code may infer customer-paid status from Paddle or onboarding fields outside the approved conversion and commercial-resolution boundaries.

M8B-2 implements that commercial resolver as a read-only foundation. Workspace entitlements are separate from classification and lifecycle, and branch commercial activity is separate from operational `Branch.status`. The resolver recognizes provisioning, active internal sandbox, active customer demo, active customer paid, inactive, suspended, archived, and manual-review outcomes. It does not implement demo expiration or authorization enforcement.

Paid workspace resolution delegates plan capabilities and paid branch quantity to the existing M7 entitlement resolver and requires a matching persisted `PaymentSubscription`. Demo and internal entitlements use explicit workspace values tied to the shared `EntitlementDefinition` catalog. Branches inherit their workspace entitlement unless an optional coherent `BranchEntitlement` says active or inactive. All calculations are read-only and fail closed on ambiguity or tenant mismatch.

M1 adds one operational facade over these existing authorities. Paid capacity uses confirmed `PaymentSubscription.quantity` capped by the plan branch ceiling, plus plan staff and teacher limits. Staff is every distinct active tenant operational User in the SchoolGroup, including an operational owner and teacher-position user; platform and account-only identities are excluded. Teacher people deduplicate by normalized `Teacher.teacher_id`, while each blank legacy identity counts separately. A person represented in both models consumes one staff slot and one teacher slot. Demo and Internal Sandbox are explicitly unmetered in M1 only when their existing lifecycle/access authority resolves.

Every operational capacity increase locks the SchoolGroup, resolves authority, recounts, evaluates the final proposed totals, writes, and commits in one transaction. Branch create/reactivation/bulk status, operational user create/reactivation, teacher create/year-copy, academic-year switching, and paid/demo provisioning final validation use this boundary. Existing over-capacity records and access are preserved, but an exceeded dimension cannot grow further. M1 adds no commercial-state persistence, schema, migration, Paddle, pricing, webhook, onboarding-transition, permission, or feature-packaging change.

M8B-3 introduces a separate SaaS demo-request aggregate after onboarding review. The public landing website enters signup with either a Request a Demo or Subscribe Now intent; the valid intent persists through account and School Workspace Setup and is emphasized, but never locked, at the commercial-choice page. A self-service pricing card may also supply an allowlisted preferred-plan code. That preference is not a plan selection and cannot create checkout or bypass the existing branch, system-user, and teacher capacity checks; invalid or ineligible preferences are discarded before the customer selects a plan. Subscribe Now preserves the existing plan-selection and Paddle path. Request Demo captures immutable commercial/classification/entitlement context and starts in Pending Review without creating an operational workspace.

Customer Demo eligibility is keyed by a normalized organization domain. TIS prefers the pending organization's authoritative domain, uses a work email domain only when no organization identity exists, and requires an official website/domain for public email providers. A unique domain-eligibility reservation prevents concurrent or historical duplicate Customer Demo opportunities. The reservation remains after request review, activation, expiry, cancellation, rejection, or Demo-to-Paid conversion; Internal Sandbox history is excluded. The only exception is the separately guarded Platform Owner test workspace/account clean-room reset, which atomically deletes the selected test organization's linked demo-commercial records and reservation so internal M8 testing can begin again; it never

changes customer reset or demo rules.

Only Platform Owners can approve, reject, or cancel requests. Approval creates a review record but does not provision or activate a demo. Rejection requires a reason, and customers may withdraw only while review is pending. Durable request events serve both audit and internal-notification purposes; email delivery remains out of scope.

M8B-4 consumes an approved request through a separate demo-provisioning aggregate. Provisioning reuses the shared operational workspace builder but creates a customer-demo SchoolGroup and explicit demo entitlement without a Paddle object, payment subscription, or subscription contract. `TenantProvisioningLink` accepts exactly one commercial source: a paid `SubscriptionContract`, an approved `SaaSDemoRequest`, or an immutable `PromoGrant`.

The workspace, entitlement, tenant link, request linkage, and activation transition are atomic. Failure rolls back those records while preserving the approved request and a retryable provisioning failure record. Success activates the customer-demo workspace and entitlement, records activation metadata and internal audit events, and prevents a second provisioning attempt. Platform Owners see detailed provisioning outcomes; customers see only Approved, Provisioning In Progress, Demo Active, or a safe support state.

M8B-5 makes the M8B-4 activation timestamp the only demo clock authority. The resolver derives Day 6 and Day 7 boundaries, validates persisted lifecycle metadata, converts values to the organization's timezone only for display, and fails closed on inconsistent ownership, entitlement, timestamps, or timezone.

The idempotent lifecycle processor creates internal reminder notifications on Day 6 and atomically expires demos at Day 7. Expiration ends the demo entitlement, suspends the SchoolGroup, marks the demo tenant link expired, updates the commercial snapshot to suspended, and preserves every operational row. The operational request middleware enforces the resolver for customer-demo tenant users on every request, so an existing authenticated session cannot bypass expiration. Web users receive the preserved-data/subscription page; API and download requests receive a safe 403. Platform administration, paid tenants, and internal sandboxes bypass this demo-only gate.

M8B-6 permits one commercial classification transition: an active, valid Customer Demo may become Customer Paid after a provider-confirmed M7 subscription exists for the same pending organization. The existing paid checkout and webhook remain authoritative; no local conversion occurs from page state, onboarding selection, or an unconfirmed payment attempt.

The dedicated conversion service locks and validates the demo request, provisioning record, SchoolGroup, demo entitlement, tenant link, contract, and payment subscription. One atomic transaction ends the demo entitlement, creates the subscription-backed paid entitlement, relinks branch entitlement rows, switches the tenant link from the preserved demo reference to the confirmed contract, and changes the existing SchoolGroup classification. The same workspace UUID, tenant row, organization, branches, users, permissions, academic data, and audit history remain in place.

The Platform Owner-only internal test-workspace reset is a separate controlled cleanup flow. Its dependency order removes the selected pending organization's linked demo request, domain eligibility reservation, review/event history, provisioning/lifecycle records, and conversion records before their commercial and workspace parents. A detached domain reservation is eligible for clean-room removal only after the reset uses the shared customer-demo domain resolver and confirms that no other pending organization, demo request, tenant workspace, or customer account independently uses the domain and that the reservation is not already marked as historically ambiguous; any conflict or historical ambiguity blocks the reset for manual review. Analysis produces an explicit safe detached-reservation ID list. Deletion uses only those IDs, flushes their removal before demo requests or parent records, and verifies inside the transaction that no selected ID remains; an ID mismatch or failed verification aborts the reset. It also removes `subscription_change_requests` scoped to the selected `SchoolGroup` before scoped operational `User` rows, then removes selected `WorkspaceEntitlementValue` and `BranchEntitlement` children before the selected `WorkspaceEntitlement` and final `SchoolGroup`. This permits a clean internal M8 retest using the same email and organization domain without touching global plan, price, entitlement-definition, Paddle, platform, or other-organization records. The existing validation gates, owner access, one-transaction commit/rollback behavior, and production one-demo-per-domain customer policy remain unchanged.

The separate Platform Owner demo-eligibility maintenance area handles historical detached reservations that no longer have an organization/account target for clean-room reset. `/saas-admin/demo-eligibility-maintenance` performs a read-only scan and marks a row removable only when its exact domain has no matching organization, SaaS account, demo request, tenant-profile workspace, provisioning record, subscription evidence, Demo-to-Paid conversion, or manual-review marker. Deletion requires typed-ID and checkbox confirmation, locks and rechecks the exact row, deletes by eligibility primary key only, flushes, verifies zero remaining rows for that ID, and commits atomically. Blockers preserve the row and rollback. Successful maintenance writes a durable owner audit event. This path never deletes by domain and does not weaken the one-demo-per-domain rule.

The existing M7 subscription resolver, workspace-entitlement resolver, and commercial-state resolver verify the post-conversion result before commit. Failure rolls back every workspace mutation while preserving provider-confirmed subscription records and a retryable failure history. Completed conversions are excluded from demo reminder/expiration processing. Expired, suspended, ambiguous, cross-tenant, internal-sandbox, paid, or already-converted workspaces fail closed.

SaaS Routes And Account Experience

Core SaaS routes:

- `/saas/signup`: public SaaS account creation.
- `/saas/login`: SaaS account login.
- `/saas/account`: SaaS account dashboard.

Related SaaS areas include plan selection, onboarding organization details, contacts, branch setup, academic setup, onboarding review, billing status, checkout summary, checkout return, checkout cancel, sessions, security, and profile pages.

Platform owner SaaS administration exists under `/saas-admin` for pending organizations, demo-request review, payments, and provisioning workflows.

The Platform Owner pending queue is an operational work queue, not a list of every `PendingOrganization` row. It includes draft/setup, review, checkout/payment, and incomplete or recoverable provisioning states only while no completed tenant link, completed provisioning job, or final tenant billing state exists. Confirmed active tenants and retained rejected/completed records remain available under Organization Records. When completed provisioning evidence conflicts with payment, subscription, contract, tenant, or SchoolGroup evidence, the owner lifecycle resolver labels the record Lifecycle Review Required and fails closed instead of presenting it as ordinary pending work. Raw onboarding status remains historical and does not override confirmed operational truth.

TIS Account Email Verification Recovery

The accepted Phase 1 verification recovery improvement strengthens the public TIS Account setup journey without changing payment, billing, provisioning, tenant activation, database schema, migrations, operational modules, or the landing website.

Current verification behavior:

- Valid email verification links mark the account email verified/active and redirect to `/saas/login` with a professional success notice: the customer is told their email has been verified and asked to sign in to continue school workspace setup.
- `/saas/login` is the GET sign-in page and its form submits only by POST to `/saas/auth/login`. Fresh verified accounts with no organization record continue to the GET `/saas/account` dashboard; the existing explicit onboarding-start action remains POST-only.

- Login continuation values are limited to known customer GET destinations. POST-only, malformed, traversal, fragment, and external values fall back to Account Setup. Defensive GET navigation to `/saas/auth/login` redirects to `/saas/login` and never exposes raw FastAPI 405 JSON.
- Expired or invalid verification links show a recovery page instead of a dead-end generic error.
- The recovery page includes a resend verification form.
- Resend verification safely handles unverified accounts, already verified accounts, and unknown email addresses without exposing whether an account exists.
- Password-based accounts that are still pending verification cannot start or continue school workspace setup.
- New customer-facing verification language in this flow uses "TIS Account" and "school workspace setup".

Google and Microsoft sign-in remain future work. OAuth-based verification bypass rules should not be expanded without a separate approved identity decision.

TIS Account Customer-Facing Wording And Branding

The accepted Phase 2 customer-facing wording cleanup improves the public TIS Account and school workspace setup experience without renaming internal `/saas` routes, modules, models, or stored statuses.

Customer-visible account/setup pages should use professional labels including:

- TIS Account
- Account Setup
- Account Dashboard
- School Workspace Setup
- Organization Profile
- Branch Setup
- Academic Setup
- Subscription Setup
- Secure Payment
- Workspace Activation

Customer pages should not expose "SaaS" as product copy, raw database statuses, tenant/provisioning terminology, provider transaction/subscription identifiers, attempt UUIDs, checkout session internals, plan IDs, or school group IDs. Those internal concepts may remain in backend code, admin views, stored data, tests, and documentation where they describe architecture or platform-owner operations.

The shared customer account shell includes the official full-color horizontal TIS logo on the light account background. Transactional TIS Account emails use an existing official dark-blue TIS wordmark URL. This Phase 2 cleanup did not change payment, billing, provisioning behavior, tenant activation behavior, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work.

TIS Account Guided Setup Framework

The accepted Phase 3A implementation introduces the shared customer setup framework for the TIS Account dashboard only. The account page now behaves like a guided onboarding console rather than a dense admin dashboard.

For a verified account that has not created a pending organization, the initial Account Setup view uses a compact variant: a smaller official logo, one "Start Your School Workspace Setup" title, one supporting sentence, one POST

action to `/saas/onboarding/start`, and the existing eight-step journey. Duplicate status, next-action, account/workspace, and guidance panels are omitted in this initial state. Later setup states retain their existing contextual console.

The shared framework provides:

- Official TIS logo/header.
- An 8-step customer journey stepper.
- Current step and status banner.
- One primary next action.
- Main content area.
- Help/guidance area.
- Clear messaging that TIS Platform access becomes available after Workspace Activation.

The journey steps are TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform. Step state is derived from existing account, pending organization, onboarding progress, billing, payment, and activation data. Internal route names, stored statuses, billing/payment/provisioning behavior, database schema, migrations, operational modules, the landing website, and OAuth behavior remain unchanged.

Phase 3B applies this shared framework to the five School Workspace Setup onboarding pages: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup. These pages now use a consistent guided wizard structure with grouped sections, one shared-shell primary CTA, secondary Back/Save Draft actions, concise guidance, and reduced visual clutter.

The Phase 3B onboarding redesign preserves existing form actions, field names, validation behavior, draft behavior, route names, and onboarding state transitions. Subscription/payment/status pages remain future Phase 3 work.

Organization Profile program input is a required controlled selector whose National, International, and Both labels normalize to the existing uppercase stored codes. Invalid values render inline without creating another pending organization. Pending logo uploads are decoded as PNG/JPG/WEBP, limited to 4 MB, checked for minimum dimensions, written under an opaque UUID filename, and promoted atomically. Empty upload retains the current logo. Replacement commits the new relative path before safely removing the obsolete pending file; rollback removes the newly written file and retains the old database reference. Storage failures are logged with traceback and rendered as customer-safe page errors.

Pending logo storage remains `static/uploads/saas/pending_logos` on the application filesystem. Only the relative path is persisted. An ephemeral Render filesystem is unsuitable for durable customer branding across restarts or deploys unless a persistent disk is explicitly mounted. Object storage or persistent disk adoption is a separate owner-approved architecture and deployment decision.

Organization Profile and the shared School Workspace setup header render the saved pending logo with contain sizing, organization-name alt text, and a neutral no-logo/unavailable placeholder. The official TIS logo remains the separate platform identity. The pending relative path remains authoritative through checkout. Both paid and demo activation use `provisioning_service.create_workspace_records()`, which validates the pending file again and copies it to the primary `SchoolGroupLogo` slot under the established organization-owned branding directory. `SchoolGroup` has no logo column; `SchoolGroupLogo` is the final branding authority. The operational shell and branding settings already consume that record through the protected organization-asset route. Missing source files now block activation rather than silently dropping the logo.

Branch Setup capacity summaries are computed in Python with nullable or incomplete estimate values normalized to zero for display. HTML and server save validation still require explicit non-negative whole numbers for system users and teachers on every active branch. Newly created branch rows receive explicit zero defaults. No schema change is

required because current columns already have non-null zero defaults.

Phase 3C applies the same guided framework to Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages. These pages use one shared-shell primary CTA, customer-safe status labels, concise supporting cards, and explicit messaging that browser return from checkout does not itself confirm payment and that TIS Platform access becomes available after Workspace Activation.

The Phase 3C redesign preserves payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, stored statuses, route names, database schema, migrations, operational modules, the landing website, OAuth behavior, and admin views.

M1-M5 Completed Milestone Summary

M1: Identity and SaaS foundation

- Established core SaaS account concepts.
- Added initial signup/login/account flows.
- Clarified separation between platform, tenant, and SaaS account identities.
- Added supporting tests for identity and SaaS phase behavior.

M2: Onboarding foundation

- Added structured onboarding stages for organization information, contacts, branches, academic setup, and review.
- Improved the path from SaaS signup toward a provisionable organization.
- Preserved the distinction between pending SaaS organizations and operational tenants.

M3: Billing and plan foundation

- Added pricing, plan catalog, billing status, checkout summary, return, and cancel flows.
- Created billing/payment service modules to keep payment logic separate from operational academic logic.
- Prepared the platform for subscription-based SaaS packaging.

M4: Provisioning foundation

- Added pending organization and provisioning workflows for platform owners.
- Added provisioning queue concepts and retry/run operations.
- Created a controlled path for turning a pending SaaS organization into an operational school context.

M5: Platform access, permissions, and owner controls

- Strengthened platform identity handling.
- Added platform owner/developer concepts and owner management controls.
- Improved permission boundaries for platform and tenant users.
- Added tests around platform access and role permissions.

M7: Subscription Management And Entitlements

- Added normalized entitlement definitions and plan entitlement values resolved from one confirmed active provider subscription.
- Added a customer Subscription Management portal with lifecycle state and centrally controlled allowed actions.

1 Added paid branch-quantity increases and scheduled reductions with operational branch-capacity enforcement.

1 Added plan upgrades and scheduled downgrades using Paddle-authoritative previews and proration behavior.

1 Added scheduled cancellation and reversal while preserving paid access through the confirmed effective period.

1 Added provider-sourced billing history and protected, freshly resolved invoice downloads.

1 Added an explicit organization-owned billing profile for the confirmed billing email, legal/billing identity, contact, optional registration/tax identifiers, and supported address fields. Login email and billing email are independent authorities.

1 Mapped Paddle customers now retain `provider_address_id` and `provider_business_id`. Authorized billing changes synchronize the existing customer, reuse or update one attributable active Paddle Business, update the active subscription identity, and attach the business to future initial transactions. Historical invoices are not rewritten automatically.

1 Billing Contact is read-only by default and uses an explicit edit interaction. Local profile changes survive provider failure; a permission- and tenant-scoped retry reuses persisted Paddle mappings, is idempotent after success, and records safe step-level provider diagnostics. A saved pending or failed billing identity blocks new plan and quantity mutations until synchronized, without blocking cancellation or changing legacy active subscriptions that have no saved profile.

1 Paddle `paid` and `completed` remain separate lifecycle evidence: paid is customer-visible payment receipt while provider processing continues; completed remains required for final reconciliation.

1 Added webhook idempotency, strict provider/local relationship validation, manual-review fail-closed paths, diagnostics, and guarded reconciliation tooling.

1 Unified active subscription capacity review and operational enforcement across branches, tenant operational staff users, and teachers. Required upgrades use the highest capacity dimension, Paddle quantity remains branch-only, and scheduled downgrades/reductions revalidate capacity at their effective boundary.

1 Subscription presentation separates paid branch quantity from the plan's maximum branch ceiling; unused ceiling is never described as prepaid capacity. The approved common customer feature baseline keeps plan comparison focused on identity, capacity, eligibility, and actions rather than a feature ladder.

1 Linked operational organization owners and billing-authorized account managers can enter the existing Organization Account subscription page from System Configuration. Tenant/account linkage is revalidated and only an allowlisted internal billing continuation survives SaaS authentication. The public landing customer entry is labelled Organization Sign In and targets centralized SaaS login.

Paddle And Payment Architecture Summary

Existing customer workspaces in `activation_required` can use a separate SchoolGroup-anchored paid activation flow. It shares Paddle customer/address/business and transaction infrastructure but does not create PendingOrganization or run tenant provisioning. Professional and Enterprise AI quote all active operational branches; Starter remains disabled until complete branch-entitlement enforcement is proven. The workspace-customer association persists the address and business used for that SchoolGroup, and a returned or reused Paddle transaction is accepted only after its complete billed transaction and current-quote lineage are revalidated. The existing-workspace **Choose a Plan** entry point remains a selection surface while the current activation is `draft` or `checkout_ready`. Its selected plan is a default, not commercial authority: another eligible plan may replace it and produce a fresh authoritative quote without calling Paddle, creating a PaymentAttempt, or changing branches. `checkout_started`, `payment_processing`, and manual-review or inconsistent activations are not silently replaced and fail closed against plan changes. Promo activation and PendingOrganization checkout remain separate flows. Verified `transaction.completed` evidence atomically establishes the paid contract, subscription, workspace/branch entitlements, paid tenant link, and active lifecycle. Classification remains `customer`, and browser return or `transaction.paid` never grants access.

Account presentation does not infer payment from workspace lifecycle. A `customer / provisioning` workspace with no current paid-activation attempt displays Activation required. Payment processing requires a current unexpired attempt in checkout-started or payment-processing state; failed, cancelled, and expired attempts use recovery presentation. PendingOrganization and demo status resolution is separate.

The payment architecture is organized under the `saas/` package.

Key modules:

- `saas/pricing_service.py`: plan/pricing behavior.
- `saas/payment_service.py`: payment-related service logic.
- `saas/paddle_client.py`: Paddle integration boundary.
- `saas/billing_service.py`: billing status and related account billing behavior.
- `saas/currency_service.py`: currency-related helpers.
- `saas/router.py`: SaaS and SaaS admin routes.
- `saas/entitlement_service.py`: provider-confirmed paid subscription entitlement authority.
- `saas/commercial_authority_service.py`: unified operational commercial-access and three-dimension capacity authority.
- `saas/subscription_portal_service.py`: customer portal composition.
- `saas/subscription_change_service.py`: quantity change previews, submissions, schedules, and webhook reconciliation.
- `saas/subscription_plan_change_service.py`: plan upgrade/downgrade lifecycle.
- `saas/branch_pricing_quote_service.py`: shared onboarding and operational three-dimension capacity counts, plan eligibility, and minimum-plan resolution.
- `saas/subscription_cancellation_service.py`: cancellation and reversal lifecycle.
- `saas/subscription_lifecycle_service.py`: lifecycle resolver and allowed-action policy.
- `saas/billing_history_service.py`: Paddle transaction history and invoice access.
- `saas/payment_lifecycle_reconciliation_service.py`: guarded repair from attributable authoritative evidence.

Architecture rules:

- Keep Paddle-specific details behind service/client boundaries.
- Do not mix payment logic into academic operations.
- Treat payment status, onboarding status, and provisioning status as related but separate concepts.
- Use platform owner admin views for payment/provisioning oversight.
- Avoid changing live SaaS payment flows unless the task explicitly requires it.
- Keep Paddle credentials and endpoints in environment variables.
- Keep Paddle price mappings environment-specific and store the runtime mapping in `subscription_plan_prices.provider_price_id`.
- Use `scripts/sync_paddle_price_ids.py` with sandbox or production mapping JSON to configure initial checkout price IDs; never hardcode live Paddle IDs in migrations or source.
- If an initial checkout price mapping is missing, fail closed before calling Paddle and show customers a support-oriented Secure Payment message while retaining internal plan/interval/currency diagnostics.

1 Paddle transaction checkout uses the public `/saas/payment` launcher as the payment-link page. Set `PADDLE_CHECKOUT_BASE_URL` to `https://app.tisplatform.com/saas/payment`, configure `PADDLE_CLIENT_TOKEN` and `PADDLE_ENVIRONMENT` for Paddle.js, and never expose `PADDLE_API_KEY` to frontend code.

1 Paddle is authoritative for prices, previews, proration, transactions, scheduled changes, and invoice documents. TIS stores workflow/audit state but does not invent monetary outcomes.

1 Subscription changes send the complete retained recurring item set. Immediate changes prevent provider mutation on payment failure; scheduled changes do not change local entitlements before verified effective evidence.

1 Webhooks are signature-verified and idempotent. Ambiguous, mismatched, or incomplete provider evidence fails closed to manual review.

Tenant Provisioning Summary

Tenant provisioning turns a pending SaaS organization into an operational TIS school context. Provisioning should create or connect the required organization records, branches, users, and initial tenant setup while preserving auditability and data isolation.

Key provisioning concepts:

1 Pending organization: SaaS onboarding entity still requiring setup, review, payment, or incomplete/recoverable activation work, with no completed tenant evidence.

1 Provisioning job: controlled action to create or update operational tenant structures.

1 Platform owner review: human oversight before or during provisioning.

1 Retry behavior: failed provisioning work should be recoverable without corrupting tenant data.

Provisioning rules:

1 Do not directly create operational tenant data from public signup without the approved provisioning path.

1 Keep provisioning idempotent where possible.

1 Log or surface errors clearly.

1 Do not merge tenant data across school groups.

1 Treat active tenant/subscription evidence as authoritative over stale onboarding status in Platform Owner queue presentation.

Landing Page Strategy

The public landing page exists to explain the product, build trust, capture demand, and route interested schools into SaaS signup or demo request flows.

Source of truth:

1 The public website implementation is `tis-landing-website/`.

1 Marketing content references live in `docs/marketing/`.

Landing priorities:

1 Explain the problem of scattered academic operations.

1 Present TIS as a connected academic operations platform.

1 Show credible platform capabilities.

- Present Request a Demo and Subscribe Now as the two public conversion pathways.
- Route both paths through environment-configured deployed TIS Account signup URLs with the selected intent.
- Keep the landing page separate from operational app templates.

Landing rule:

- Do not change landing page design, copy, or architecture during operational backend tasks unless explicitly approved.

Customer Experience Roadmap

Near-term customer experience:

- Clear signup and login path.
- Smooth onboarding for organization, contacts, branches, and academic setup.
- Transparent billing/plan status.
- Clear provisioning status after checkout or owner review.
- Reliable operational login once provisioned.

Medium-term customer experience:

- Better account self-service.
- Clearer subscription lifecycle.
- More guided onboarding and implementation support.
- Improved platform owner visibility into pending organizations and payment status.

Long-term customer experience:

- AI-assisted academic planning and decision support.
- Subscription-gated advanced analytics.
- Intelligent recommendations based on verified tenant data.
- Richer executive visibility across branches and academic years.

Knowledge Management System

TIS uses a Knowledge Management System (KMS) to preserve source-of-truth documentation, current project state, decision history, module history, and generated snapshots.

KMS source documents:

- `docs/AI_PROJECT_CONTEXT.md`: first-read compact onboarding context for future Codex and ChatGPT conversations.
- `docs/TIS_MASTER_CONTEXT.md`: durable product, architecture, workflow, roadmap, and critical rules.
- `docs/PROJECT_STATE.md`: living project state.
- `docs/DOCUMENTATION_UPDATE_POLICY.md`: mandatory KMS and Knowledge Impact Assessment rules.
- `docs/CHANGE_HISTORY.md`: chronological summary of meaningful changes.
- `docs/adr/`: Architecture Decision Records.
- `docs/history/`: module-specific history.

- `docs/engineering/`: engineering handbook with module map, repository architecture, user/system flows, and developer onboarding.

PDF philosophy:

- Markdown files are the source of truth.
- The PDF booklet is a generated snapshot.
- The PDF must never be edited manually.
- The PDF must be regenerated when included Markdown source docs change.
- The PDF should later be served through owner-only protected routes.

Generated KMS artifacts:

- `static/docs/TIS_Project_Reference_Booklet.pdf`
- `static/docs/docs_manifest.json`

Automatic enforcement:

- root `AGENTS.md` defines mandatory Codex KMS behavior,
- `.kms-impact.yml` records the task-level machine-readable KIA,
- `scripts/kms.py sync` validates KIA before writing, regenerates PDF/manifest artifacts, and runs post-generation freshness checks,
- `scripts/kms.py check` provides the single read-only local and CI validation command,
- `scripts/check_kms_impact.py` compares the declaration with changed files and major-change rules,
- `scripts/generate_docs_pdf.py --check` validates source coverage and snapshot freshness without writing,
- GitHub Actions require validation for pull requests, `dev`, and `master` deployment.

Automation never rewrites Markdown. It detects omissions and blocks stale work so reviewed human/AI updates remain authoritative.

Owner-only app access:

- `/platform/knowledge`: read-only Platform Owner Knowledge Center.
- `/platform/knowledge/booklet`: protected inline PDF view.
- `/platform/knowledge/booklet/download`: protected PDF download.

The Knowledge Center is protected by the existing Platform Owner access pattern. It is not public, not a landing page, and does not regenerate or rewrite source docs. Its manifest-backed library presents document titles and summaries, groups sources into Core, Engineering, Decisions, History, Marketing, and Supporting sections, supports client-side category/module/freshness filtering and search, and opens documents at their recorded `pdf_page` through the protected booklet route.

Engineering handbook:

- `docs/engineering/TIS_MODULE_MAP.md` maps product/system modules and guardrails.
- `docs/engineering/REPOSITORY_ARCHITECTURE.md` explains repository ownership and risky files.
- `docs/engineering/USER_AND_SYSTEM_FLOWS.md` documents end-to-end customer, SaaS, payment, provisioning, operational, platform owner, KMS, and developer flows.

1. `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md` explains data areas, ownership boundaries, and tenant isolation rules.
1. `docs/engineering/DEVELOPMENT_STANDARDS.md` defines non-negotiable engineering rules.
1. `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md` defines design direction for operational, platform, SaaS, Knowledge Center, and landing surfaces.
1. `docs/engineering/PRODUCT_ROADMAP.md` records completed, current, next, and future roadmap.
1. `docs/engineering/REJECTED_DECISIONS.md` records significant rejected alternatives.
1. `docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md` defines future visual documentation standards.
1. `docs/engineering/AI_OPTIMIZATION_GUIDE.md` guides future AI assistants.
1. `docs/engineering/PROJECT_GOVERNANCE.md` defines ownership, approvals, quality gates, documentation gates, and traceability.
1. `docs/engineering/KNOWLEDGE_LIFECYCLE.md` defines documentation, engineering, approval, review, release, and maintenance lifecycle.
1. `docs/engineering/DOCUMENTATION_AUTOMATION.md` defines current automation and future automation rules.
1. `docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md` formalizes KIA.
1. `docs/engineering/SELF_EVOLVING_WORKFLOW.md` defines the official task-to-deployment workflow.
1. `docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md` explains KMS propagation.
1. `docs/engineering/AI_CODING_WORKFLOW.md` defines future AI coding discipline.
1. `docs/engineering/FUTURE_AUTOMATION_ROADMAP.md` records future automation opportunities.

KMS v1.0 status:

KMS v3.0 Phase 3D completes the KMS v1.0 lifecycle foundation. Future work should improve automation, search, visuals, route inventories, test strategy docs, deployment runbooks, and deeper module guides only through approved phases.

Development Workflow

Default workflow for approved implementation tasks:

1. Inspect the relevant code and docs before editing.
2. Keep changes scoped to the approved task.
3. Preserve tenant isolation, permission checks, SaaS flows, and landing page boundaries.
4. Update tests or add focused tests when behavior changes.
5. Complete the Knowledge Impact Assessment and update `.kms-impact.yml`.
6. Update relevant Markdown docs.
7. Update change history, ADRs, module history, and AI project context when needed.
8. Run `scripts/kms.py sync` if included docs changed.
9. Run `scripts/kms.py check` for final read-only enforcement.
10. Run reasonable implementation validation.
11. Report code changes, docs changes, KIA, validation, assumptions, and known issues.

Smart Timetable Versioning Boundary

Stage 3.5 corrects school-day timing. Start time plus teaching-period count and duration plus inserted non-teaching durations produces one per-day ordered timeline and a calculated end. `after_period` blocks shift later teaching periods; preserved `fixed_time` blocks must begin on a live timeline boundary or configuration fails closed. The projection is shared by preview, operational grid, readiness, assignments, snapshots, solver slots, and exports. Existing `break`, `prayer`, and `non_teaching` keys remain valid and the controlled catalog also includes recess, lunch, assembly, whole-school event, advisory, intervention, transition, dismissal preparation, and other. This stage does not generate a timetable.

Stage 4 makes the existing lifecycle operational. Version review is explicit and does not move the active pointer. Active, superseded, and archived versions are read-only and may be reused only through an explicit copied draft. Draft validation checks current fingerprint, complete demand, canonical slots, Planning teacher authority, collisions, stale placements, and locks; it is distinct from generation readiness. Explicit Approve Draft validation records an approval timestamp and actor for the exact draft revision. Generation does not approve; placement, lock, regenerated successor, and stale-authority changes invalidate approval. Publication requires an approved fresh `publication_ready` draft, revalidates it, and atomically swaps the revisioned active pointer while superseding prior active history. The main Draft workspace groups Regenerate, Approve Draft, and Publish Timetable; technical and destructive version controls remain secondary in History. Regeneration requires `ceil(25% * unlocked placements)` changes from its direct source and fails with a controlled diversity result when no valid alternative reaches that threshold.

Placement actions use separate create/edit/delete permissions. Locks use `timetable.lock_lessons`, publication uses `timetable.publish`, archive uses `timetable.archive_versions`, and explicit selected-version export uses `timetable.export`. Every operation remains branch/year/SchoolGroup scoped.

Stage 3.5 makes the composed per-day slot projection authoritative across the page, assignment service, exports, readiness, and snapshots. Inserted blocks consume no teaching-period number and shift later clock times; ambiguous fixed-time placement is invalid. Readiness is an exact-scope, read-only structural gate; `generation_ready` permits a Stage 5.1 solve attempt and does not guarantee feasibility. Existing versions and stale placements remain preserved.

The operational timetable uses a versioned aggregate scoped by SchoolGroup, Branch, and Academic Year. `TimetableVersion` owns placements and lifecycle; `TimetableActiveVersion` separately selects at most one exact-scope operational baseline. `TimetableInputSnapshot` preserves deterministic Planning/settings/lock authority and component fingerprints. `TimetableGenerationRun` is the Stage 5.1 durable PostgreSQL work queue with progress, attempts, lease/heartbeat, cancellation, safe terminal status, and result linkage.

Planning remains authoritative for section-subject demand, teacher assignments, and HRT fallback. A timetable placement is one teaching period and current `Subject.weekly_hours` is the compatibility required-period value. Existing live placements migrate without normalization into an imported active compatibility version. The current edit route uses a working draft copied from active history; current views and exports resolve that operational draft after edits. Active, superseded, and archived versions cannot be edited in place.

ADR 0026 defines readiness, hard/soft constraint separation, CP-SAT execution, independent validation, regeneration diversity, and publication boundaries. Stage 5.1 implements the hard-constraint generation slice; availability, rooms/resources, cross-campus coordination, preferences, and quality scoring remain unimplemented.

Knowledge Impact Assessment Rule

Every approved implementation must:

1. Assess knowledge impact.
2. Update relevant Markdown docs.
3. Update `docs/CHANGE_HISTORY.md` for meaningful changes.
4. Create or update ADRs when major decisions change.

5. Update module history when a module's documented state changes.
6. Update `docs/AI_PROJECT_CONTEXT.md` when high-level AI onboarding context changes.
7. Regenerate the PDF booklet if included docs changed.
8. Mention KIA details in the final report.

A task is not complete until the KIA is assessed.

Required final report KIA template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

The generated booklet output is:

1. `static/docs/TIS_Project_Reference_Booklet.pdf`

Critical Rules Codex Must Follow

1. Do not touch SaaS flows unless the task explicitly requires it.
1. Do not touch operational logic unless the approved task requires it.
1. Do not touch database migrations or `tis.db` unless explicitly approved.
1. Do not weaken tenant isolation.
1. Do not bypass route permissions or platform owner checks.
1. Do not merge platform user, SaaS account, and tenant user concepts.
1. Do not change the landing page design or legacy landing files unless explicitly approved.
1. Do not add KMS regenerate behavior unless explicitly approved.
1. Do not expose the KMS PDF through direct public static links in the app UI.
1. Do not push or commit unless explicitly requested.
1. Treat production memory as a hard budget. Do not add unbounded full-dataset caches, duplicate production template renders, startup-heavy work, or normal-request diagnostic logging that can trigger Render restarts and 502s.
1. Prefer conservative, dependency-light automation.
1. Use `reportlab` for the documentation PDF generator.
1. Do not require LaTeX, Playwright, Chromium, external network calls, or system font dependencies for PDF generation.
1. Always include a Knowledge Impact Assessment in implementation final reports.
1. Always keep `.kms-impact.yml` aligned with the task and actual Git diff.
1. Never put customer, personal, production, credential, secret, environment, transaction, invoice, webhook payload, or database-row data into KMS documentation.

Owner-directed Talent simplification: Program Criteria configuration is not part of the normal Program setup UI, and completed Student Assessments do not expose a manual "Check Program Criteria" action. Existing Review Candidate/Official Identification persistence and historical records remain intact for backward compatibility/history

unless separately governed for retirement.

Student Permanent Delete Authority

Owner-approved Students management now includes explicit single and bulk permanent deletion. The capability is separately permissioned as `students.delete` and `students.bulk_delete` and requires organization/global Branch scope because Student identity is SchoolGroup-owned. Permanent deletion is intentionally limited to Students with no Academic Placement and no Talent historical records (Cycle population, Assessment, Review Candidate, Official Identification, or Educator Input). Creation-only Student audit rows and external identifiers are deleted with an otherwise-empty Student. Bulk deletion is atomic: if any selected Student is protected by historical records, no selected Student is deleted and the backend returns the blocker reason. This adds no Student lifecycle state, approval workflow, schema change, or migration.

Action Permission And Evaluation Period Selection Authority

Owner direction requires user-facing mutable/destructive actions to have explicit permission authority exposed through the canonical System Configuration role-permission registry rather than relying on UI visibility alone. Student single permanent Delete is governed by `students.delete`; Student Bulk Delete is governed by `students.bulk_delete`; both are independently configurable through the existing role-permission system and remain subject to their backend history/scope safeguards.

Selecting an Evaluation Period to enter Student Assessments is separately governed by `talent_evaluation_plans.select_period`. The permission is registered in System Configuration and is enabled by default only for the Administrator role under the existing default-role architecture; Editor, User, and Limited defaults do not receive it. The Evaluation Plan UI renders the Student Assessment entry control disabled when this permission is absent, and linking an internal Evaluation/Cycle context to a Period requires the same permission server-side. Administrators may still revoke the permission, and authorized role configuration may explicitly grant it later; there is no hard-coded username check.

The sanctioned local Talent analytics dataset uses Arabic student names transliterated in Latin script (for example Layan Khalil, Adam Nasser, Nour Hassan) across Grades 3-5. The dataset remains disposable local test data only.

Grade-Scoped Talent Rubric Structure And Student History Force Delete

Talent Frameworks support the owner-directed hierarchy `Grade -> Competency -> Rubric Level -> Achievement Description`. `FrameworkCompetency.grade_level` is optional: a Grade value scopes that competency to one eligible Grade, while NULL preserves backward-compatible all-Grade behavior. Grade scope is part of Framework fingerprinting, cloning, Draft-only mutation, tenant scope, and audit semantics. Migration `20260911_002_talent_framework_competency_grade_scope` adds the nullable Grade column.

Achievement wording remains stored at the exact `Framework Competency + Rubric Level + Grade` grain in `talent_grade_competency_rubric_descriptors` (migration `20260911_001_talent_grade_specific_rubric_descriptors`), with the existing generic `Framework Competency + Rubric Level` descriptor retained as fallback for legacy/unscoped Frameworks. Program authoring groups the rubric by Grade, then Competency, then ordered Levels. Student Assessment uses only competencies assigned to the Assessment's recorded historical Grade (plus intentionally unscoped competencies), and resolves the Grade-specific descriptor before generic fallback. Compact downstream result surfaces may show only the ordered rubric number (for example 1-5) with accessible level text and an order-derived low-to-high visual intensity.

Student Assessment Evaluation choices follow the configured Annual Evaluation Plan Period `sequence`. Linked Period sequence is the display-order authority; creation timestamp/id are fallback ordering only for legacy unlinked Cycles.

Student deletion now has a separate high-risk authority, `students.force_delete_history`, exposed through the canonical System Configuration role-permission registry and enabled by default for Administrator only. Normal `students.delete` and `students.bulk_delete` continue to protect Students with Academic Placement or Talent

history. A force-delete-capable actor receives a pre-delete history preview and an explicit irreversible confirmation before the Student and Student-owned Academic Placement/Talent history are removed in dependency order. Bulk Delete does not cascade historical data. The force path remains organization/global scoped and backend-enforced; UI confirmation is not authorization.

Talent Grade-First Rubric Authoring And Re-evaluation Attempts

Per ADR 0036, the owner-facing Talent Program flow is separated into two responsibilities. Program creation/editing is limited to Program identity plus eligible Grades for the selected Academic Year. Rubric authoring is a separate Program action. The canonical authoring hierarchy is **Grade -> Competency -> Competency-owned Rubric -> ordered Levels**. Each Grade is independently collapsible; each Competency may create/edit its own Rubric and Levels. Level descriptions are the primary achievement wording for new competency-owned rubrics, while existing generic/Grade-specific descriptor rows remain supported for historical/legacy compatibility. Migration `20260911_004_talent_competency_specific_rubrics` adds nullable competency ownership to `talent_rubrics`; NULL denotes a legacy Framework-wide shared rubric, not the new authoring model.

Completed Student Assessment evidence remains immutable. Re-evaluation is required only when a newer saved Framework materially changes the Student-facing assessable structure for that Student's recorded Grade. Framework version number/title/supersession metadata alone do not trigger re-evaluation. A replacement Assessment is linked with `reassessment_of_assessment_id`; the prior attempt becomes `is_current=false`, the replacement remains `is_current=true`, and `evaluation_context_cycle_id` keeps both attempts associated with the original visible Evaluation/Term. The replacement physical Cycle exists only to freeze the newer exact Framework provenance. Historical competency results, placement context, Review Candidate, Official Identification, and Educator Input are never rewritten or deleted.

New, not-yet-assessed Students also start against the newest saved Grade-applicable assessable Framework while remaining in the selected original Evaluation context. Current-result analytics project only the current attempt back to that Evaluation's historical population dimension and do not double-count the physical reassessment Cycle. Migration `20260911_003_talent_assessment_reassessment_attempts` adds current/reassessment/Evaluation-context metadata and indexes.

Any Framework Version already referenced by a Student Assessment is semantically immutable even if its lifecycle label is still Draft. Further semantic changes require a new/cloned Framework Version.

Talent Overall Program Result Authority

ADR 0037 governs the deterministic cross-competency result for one Talent Student Assessment.

Rubric level number is the ordered proficiency position within the competency-owned rubric and is rendered beside the level name. The canonical Overall Program Result is the arithmetic mean of all Grade-applicable selected rubric ranks on one coherent Program scale, for example `4.4 / 5`. Applicable competency rubrics must therefore use the same ordered level count before an Assessment can be completed.

The calculation identifier is `arithmetic_mean_rubric_rank`. TIS may derive `average / scale_max * 100` only for visual normalization such as bar length or color intensity; that percentage is not the canonical educational result.

The result is a read projection over the exact immutable Assessment/Framework/results and adds no schema migration. Existing optional KPI configuration remains separate.

A Student may have separate results in multiple Talent Programs. TIS may show those Program results together, including in a Students Across Programs matrix, but it must never average or collapse them into one universal Talent score.

Talent Review surfaces the Program-scale Overall Result as evidence. Review Candidate policy and Official Identification remain separate. Official Identification is still a separately authorized human decision, and no result threshold automatically identifies a Student.

Talent Evaluation-First Assessment And Review Workflow

ADR 0038 governs the operational information architecture:

Evaluation Period -> Program -> Student.

The existing Annual Evaluation Plan / Planned Evaluation Period model remains the scheduling source of truth. Program setup surfaces configured Evaluation Periods alongside eligible Grades; no duplicate period table or hard-coded Term catalog is introduced. Repeated labels such as Term 1 are grouped once in the Student Assessments chooser with all participating Programs nested beneath.

For a selected Program/Evaluation, the current operational Student list is driven by live Academic Placement eligibility from ADR 0035 plus tenant, Academic Year, eligible Grade, and authorized Branch/scope constraints. Students are shown as Not started, In progress, Completed, or Re-evaluation required. Historical/superseded attempts remain preserved evidence but the normal Student Assessment workspace does not render a separate Assessment Records/History section; the current Student row/status/action is the operational surface.

ADR 0036 remains authoritative for re-evaluation. A materially changed newer Grade-applicable rubric keeps the previous completed attempt immutable and requires a new current attempt while retaining the original visible Evaluation context.

Talent Review now represents all current completed Student Assessments in the authorized context, not only materialized Review Candidate rows. Review Candidate, review status, and Official Identification are shown as separate states. Completion may deterministically evaluate the existing Framework Review Candidate policy, but no-policy/non-qualifying results remain visible in Talent Review.

Talent Rubric Authoring, Results UX, And Performance

New competency-owned rubric creation starts clean. Assessed legacy shared-rubric levels are never auto-copied into a new competency rubric. Normal current Evaluation starts require a complete competency-owned rubric for every applicable Competency; legacy shared rubrics are historical/read-comparison compatibility only and cannot authorize a new user-facing Assessment. An author may explicitly use **Copy Levels From** another competency in the same current Draft Framework. Copying is one-time and independent; it may copy only level structure or level structure plus descriptions, and never copies Student evidence or historical Assessment state. Competency removal and Rubric Level true-delete use the dedicated assignable permissions `talent_programs.delete_competency` and `talent_programs.delete_rubric_level`; whole Draft Program delete remains separate under `talent_programs.delete`. Assessed Frameworks remain semantically immutable and require a new Framework version before any future Student-facing rubric change. For legacy data created before that guard was consistently enforced, a completed Assessment whose persisted rubric/level bindings no longer match a complete competency-owned rubric now present on the same Framework surfaces **Re-evaluation required**. This compatibility repair never deletes the prior evidence: the prior attempt becomes non-current and the replacement starts empty/current in the same visible Evaluation.

A fully configured Draft Program now crosses its user-facing lifecycle boundary at **Finish Setup**: the authorized organization-governance action activates the Program first, then activates its completed Draft Framework with revision/fingerprint protection, and returns to that Program's operational summary. Already-active Program/Framework state is not redundantly mutated. Operational summary actions for Edit Program, Build/Edit Rubric, and Manage Evaluation Plan remain live hash-backed navigation rather than inert links.

Student Assessments render Evaluation Period -> unique Programs -> Students. Repeated Evaluation labels across Programs are presented as one Period section, with configured sequence used for ordering when available; duplicate physical Cycles cannot duplicate a Program card, and configured Program/Period combinations may be shown before their internal Cycle exists. Normal Grade-level Competency creation keeps technical codes internal; the browser sends the Unicode name/description and the backend allocates a unique Program-scoped code, avoiding Arabic/non-ASCII code collisions.

Talent & Potential is exposed as an active-module tree in the main sidebar with clean top-level destinations for Overview, Programs, Student Assessments, Talent Review, and Results & Analytics. Top-level navigation is a context-reset boundary: it carries only the selected Academic Year and never silently inherits Program, Cycle, Assessment, Branch, Grade, Section, metric, or dimension state from the page being left. Student Assessments therefore owns its own Program filter; entering it from a selected Program does not preselect that Program unless the user uses the Program's explicit Open Assessments action. The Programs node always opens the searchable Programs list, where each row has one explicit Open Program action (plus any separately authorized destructive action). Program-specific Edit Program, Build/Edit Rubric, Manage Evaluation Plan, Open Assessments, and View Results actions remain inside the opened Program workspace.

The Programs list uses a bounded summary read instead of per-Program annual/framework/configuration fan-out. A selected Program is loaded directly rather than through the full Program catalog, and its independent setup reads are parallelized. Routine Program/rubric mutations refresh only the affected Program, Framework/configuration, competency bank when needed, or Evaluation Plan slice instead of re-fetching the whole workspace graph. Same-context wizard navigation and Save Rubric can redraw from the current bounded cache with zero network reads.

Evaluation Plan initial load uses the selected Program directly and parallelizes Plan, Program, Cycle, annual-configuration, and Framework reads. Subsequent Evaluation Plan edits retain the rendered workspace and refresh only Plan/Cycle data; they do not refetch Programs, annual Program configuration, or Frameworks. Student Assessment opening parallelizes Program, result, Framework/configuration, and educator-input reads after the canonical Assessment is resolved. When an Evaluation is already selected, its eligible Student roster is fetched in parallel with Assessment/context reads.

Full refresh remains appropriate for real Program/Academic Year/Framework context changes, explicit reload, lifecycle changes, or stale-revision recovery. Routine saves must not blank the workspace to a full-screen loading message.

Talent operational UI uses the existing TIS light design system with stronger semantic status chips, evaluation/program cards, icons, low-to-high result visuals, richer review cards/tables, and responsive states. Color remains supplemental to visible numeric/text labels.

Results & Analytics continues to use the existing governed privacy-closed M9/M10 architecture. For a selected Program, privacy-visible completed rubric distributions may derive average competency rank and a Program-scale average. Organization/Branch/Grade views retain governed aggregate semantics and privacy suppression. The Student drill may show separate Program results and a Students Across Programs matrix when the identifiable P7 gate permits it; the browser must not reconstruct a universal cross-Program score.

Talent Results & Analytics Availability Authority - Owner Amendment 2026-09-12

Results & Analytics is an authorized Talent capability, not a separately commercially gated Organization Intelligence product. The seven M10 routes do not consult `feature.organization_intelligence`, plan packaging, WorkspaceEntitlement, subscription, or commercial state. Availability validates tenant/Academic-Year context, then `talent_analytics.view` and the existing secondary Student/Candidate/Identification permissions govern access. Tenant isolation, Branch scope, privacy suppression/closure, breadth limits, and M10 REPEATABLE READ remain mandatory and unchanged. The legacy semantic feature key may remain registered for compatibility/history but has no runtime authority over Results & Analytics.

Talent Assessment Grade-Alignment And Owner Presentation Authority - 2026-09-12

Student Assessments roster inclusion is not controlled by Program `eligible_grade_levels` or annual enablement. Separately, current assessment criteria are strictly Grade-aligned: only criteria applicable to the Student's Grade plus intentionally unscoped criteria may be used. Criteria assigned to another Grade must never be substituted. A Student with no applicable criteria stays in the roster but cannot start until that Grade's criteria are configured. Zero-result In

Progress reload may follow only a newer applicable Grade build; evidence-bearing and terminal history remains immutable.

For Talent-capable users, Students appears only as the first Talent child; standalone Students is a fallback for users with Students permission but no Talent permission. Student Assessments exposes Grade/Section/Assessment Status filters and Talent Review exposes Identification Classification. Visible analytics magnitudes use a low-to-high blue intensity scale; privacy/no-data states remain categorical. Learning Style retains per-style categorical colors without exposing suppressed magnitude. The full transparent Ghars wordmark is Talent Overview-header-only; the sidebar uses the compact emblem.

Talent Owner Adjustment Package - Correction (2026-09-12, follow-up)

The Talent Overview full Ghars wordmark is blocked pending the Owner's actual asset file; a non-Owner-supplied traced SVG was removed from that header and replaced with an empty container pointing at a documented, not-yet-present path (`static/img/ghars-full-wordmark-dark.png`). The duplicated "Selected Evaluation" panel in Student Assessments has been removed; only the in-place selected-card marking remains. No schema, permission, or scoring change.

TIS Project State

docs/PROJECT_STATE.md

Production follow-up Part 2 - chart types, Results & Analytics filter UX, comparisons, Progress Over Time (2026-09-25)

Implemented on `dev`; Web Service only (static JS/CSS and one template script include); not deployed. No schema, migration, permission, analytics-semantics, privacy-threshold, Classification or Learning Style policy or `tis.db` change. Verified through the Node stub harness only (no browser).

D. Chart type semantics. Root cause: the selector offered "Bar" and "Percentage", which drew the same picture (Bar already printed the percentage), and Pie beside Doughnut (same reading). Now `talent-charts.js` switches only genuinely different visualization types and renders no selector when only one type is valid: Classification and Assessment completion = Bar / Doughnut (only for a full public partition of at most `MAX_CIRCULAR_CATEGORIES` = 8 categories); Learning Style (eight styles plus Unassigned = nine) and Rubric levels (ordinal) = Bar only, keeping the stable per-index semantic colours (Unassigned is always the neutral last colour); time series = Trend / Bar (Trend only with two visible points); comparisons = one fixed Bar of the backend rate per group. Pie was dropped as a doughnut without a centre (owner decision open). Exact counts/percentages stay in every type (bar label, legend, trend legend) and in the accessible table. The Overview defaults Classification and completion to Doughnut where valid (Bar otherwise); Results & Analytics defaults to Bar. A deliberate choice is remembered across refetches. The Overview gets ONE restrained CSS entrance (fade, bar grow) on its first render only, decided in `talent.js` from `prefers-reduced-motion` (reduced or unknown = none), applied with opacity/transform only, and removed on a chart-type switch.

E. Results & Analytics filter UX. Root cause: every dashboard filter change replaced the whole content root with a loading skeleton (`root.innerHTML = ...` then a second full replacement in `load()`), collapsing the page height so the browser clamped the scroll to the top, and re-created the form (focus lost). Now Branch/Grade/Section/Program/Period/Classification/Competency/Indicator/comparison changes call `refreshDashboard()`: local `params` + `history.replaceState` (no navigation, submit cancelled), a background fetch with its own `AbortController` and a monotonically increasing token (only the newest response renders), the previous analysis stays on screen with `aria-busy`, a dimmed region and a thin bounded progress bar, the region height is held while replacing and the reader position is restored (focused control re-focused with `preventScroll`, relative `scrollBy` compensation, `scrollTo` only if the browser clamped anyway). Comparison tick changes are debounced (350 ms). The existing 25 s bounded request applies; a failure keeps the old analysis and shows an inline retryable error. Chart-type change is purely client-side (no fetch, no URL change). `load()` also holds height and restores the anchor for other views. The `scrollIntoView` in `talent-operations.js` belongs to the assessment editor and is not on this path.

F. Selected Comparisons. One dedicated section, last in Results & Analytics, contains the Compare-by selector, the group checkboxes (up to six; the seventh is disabled/refused, count announced), then the "Completion rate by group" chart and the per-group Completion/Classification/Result cards, with explicit empty, protected and unavailable states. The selector was removed from the top filter form. Group values are the backend's own per-group rates; nothing averages Programs or frameworks.

G. Progress Over Time - decision: KEEP as a real per-Program longitudinal view. Evidence: the view is backed by the governed ADR 0027 route `/api/talent/organization-analytics/programs/{id}/longitudinal` (one Program x one Academic Year, ordered by governed Evaluation Period, each point independently privacy-closed, five metrics incl. counts and started coverage, comparability state, no computed delta). Results & Analytics offers only a completion-by-period line, so the view has distinct value (counts, started coverage, comparability context). The previous plot drew percentage bars only; it now uses the shared trend chart with honest gaps (a suppressed or not-yet-available period is a dashed gap with no coordinate; "No data yet" is never 0%; count metrics plot counts) plus the accessible table, and Results & Analytics links to it for a chosen Program. Not done (needs new backend semantics or owner

decision): multi-year series, Classification-over-time, Branch comparison trend, rubric-indicator trend. The nav key and route are unchanged, so old links keep working; the template now loads `talent-charts.js` on that view.

I. Cleanup. Raw arrow glyphs in actions replaced by a shared inline SVG chevron; duplicate chart controls removed (D). Tests: `tests/talent_part2_experience.test.cjs` (P1-P22) plus deliberate updates to the old mode pins in the `talent_dashboard`, `talent_visual_system`, `talent_classification_withheld` and `talent_results_experience` tests and the harness (scroll model, writable hooks).

Production follow-up Part 1 - Talent Branch authority, Start Assessment, Classification (2026-09-25)

Implemented on `dev`; Web Service only; not deployed. No schema, migration, permission, privacy-threshold or `tis.db` change.

A. Branch authority (owner-directed amendment to the Batch 1 closure). The sidebar selector lists real Branches only; the Talent "All Branches" option, the `branch_scope=all` marker cookie, `user.scope_all_branches` and `POST /scope/branch branch_id=all` are removed (an old cookie is ignored and grants nothing; choosing a Branch still clears it). `talent_branch_scope.talent_branch_ceiling` is always `None`; `branch_scope_unrestricted` is `auth.can_access_all_branches`. Organization-authorized actor: ceiling = own authorization (all Branches of the SchoolGroup); the sidebar Branch is the page default (`tp-config.branch`); the page-level Branch filter offers All Branches (URL marker `branch_scope=all`, no `branch_id` sent) and every authorized Branch; a stale marker from another sidebar Branch resets to the new default. Branch-limited actor: `tp-config.branchLocked`; the ceiling stays their authorized Branch on every read surface (server enforced, regression: `tests/test_talent_branch_hard_scope.py`); an explicit foreign or cross-tenant Branch is rejected as before. Omitted `branch_id` = all authorized Branches (it is how All Branches is expressed); the default is applied by the page. Branch display: legacy startup seeding uses hyphenated Branch names (`X-Boys`); persisted names and Talent presentation were NOT changed (reported as a separate data/presentation decision).

B. Start Assessment. Trace: roster button (`type="button"`, no form/#) -> POST `/api/talent/assessments {cycle_id, student_id}` -> navigation with `assessment_id`, `cycle_id`, `academic_year_id`, `program_id` -> workspace reads. Backend regression proves the whole chain for organization actors under every sidebar Branch and for a Branch-limited actor (`tests/test_talent_start_assessment_flow.py`); string/number id typing does not matter. Root cause of the banner: a stable, safe 400 business code (chiefly `assessment_tool_unavailable`, a Student whose Grade has no saved criteria in the Program; also `student_not_eligible`, `invalid_student_context`, duplicate/conflict) was collapsed by `talent-api-errors.js` into the generic 400 copy and shown in the page-top banner via `scrollIntoView`. Fix: curated fixed copy per code (`CODE_COPY`, backend `detail` still never echoed) shown inline beside the row; no scroll jump, no navigation on failure. Which code production returned could not be observed here.

C. Classification "Unavailable". Diagnosis (reproduced, `tests/test_talent_classification_aggregate_privacy.py`): every Student is classified; the aggregate is hidden at the PRIMARY privacy stage (`apply_primary_privacy` inside `build_bucket_projection`, class P4) by the approved Release 1 provider (`ConfiguredRelease1PrivacyPolicy`, minimum cohort 5 per Classification band cell, zero counts included; ADR 0028, B11-E F1), then complementary suppression; a total below the floor hides the whole family; an absent provider configuration fails closed (`restricted`). Broader selections publish exactly the bands that clear the floor. Behavior is unchanged. UI: one uniform, value-free sentence (`data-chart-withheld-note`) and a precise whole-family message; no threshold, cell-level reason, or hidden value is emitted. Showing small cohorts would require an owner decision and an ADR 0028/0044 amendment. Production privacy configuration (the cohort environment value) could not be verified here; if it were absent every protected metric would read Unavailable.

Tests: Python `test_talent_branch_hard_scope.py`, `test_talent_start_assessment_flow.py`, `test_talent_classification_aggregate_privacy.py`; Node B1-B10, Start Assessment cases in `talent_operations.test.cjs`, `talent_classification_withheld.test.cjs`. Browser verification not

performed.

Talent product transformation (Agent 3, 2026-09-25)

Implemented on dev, not deployed or merged. This extends the preserved executive Overview work into an aggregate-only Results & Analytics workspace: completion and participation, results and Classification, Learning Style, Program-bound Rubric Indicators, ordered Evaluation Periods, and selected comparisons. The shared backend dashboard projection owns filters and all numerical values; the frontend only renders its safe semantic states. Comparisons select at most six authorized Branch/Grade/Section/Program/Period groups. Totals use underlying Evaluation participations, not averages of Branch rates. Distinct Student and participation grains are labelled separately. Incompatible Program/framework results are not pooled. Requests exceeding 5,000 scoped population rows fail explicitly and require a narrower Branch; there is no silent truncation.

The owner-approved privacy amendment fixes Agent 2's Classification-filtered Learning Style leak: the original selected Classification bucket and denominator must be visible after provider primary/complementary suppression before either filtered aggregate can publish. Otherwise Learning Style returns a protected semantic result with no total, buckets, percentages or chart magnitudes. Individual authorized roster records retain exact identity, state, Classification and style; aggregate suppression never removes an individually authorized Student row. Tenant, Student visibility and the global Branch ceiling remain mandatory.

Overview retains its independent headline projection and adds shared charts. Programs now uses searchable cards without normal lifecycle-status presentation. Bar/percentage and applicable pie/doughnut/trend modes retain exact accessible tables and native keyboard controls; nonvisible cells have no numeric geometry, tooltips, datasets or ARIA values. Bundles remain surface-specific, bounded reads retain cancellation/generation guards, and old duplicate rubric reads are removed. The dashboard uses the existing read-only repeatable snapshot dependency. No schema, migration, permission, timetable worker or local database change.

Recovery completion (same day): the interrupted local Agent 3 worktree was audited and completed. Corrections: the generic "Protected for privacy" copy that had re-entered chart and dashboard scripts was removed again (M18a); a non-visible cell now reads "Unavailable", and only the backend reason `classification_cohort_protected` produces the Classification-cohort explanation; a missing Student-view permission has its own value-free message. The Program summary Finish Setup control was inert (the summary path returned before click binding) and was shown for incomplete setups; it is now bound and shown only for a complete draft setup. Roster avatars and period icons use initials and inline SVG instead of emoji. A Talent visual system layer (header band, KPI tiles, chart cards, colour-controlled Program cards, one primary/secondary/destructive action hierarchy, empty/loading/error states, responsive, reduced-motion and forced-colors rules) was added in `talent-experience.css`. Visual claims are structurally verified only; no browser review was performed. The dashboard reuses the current-Student, classification, assessment and privacy authorities and its distinct-Student figure is tested equal to Overview's. No schema, migration, permission or local database change; Web Service only, no Workflow change.

Student Assessment Filtered Insights (Agent 2, 2026-09-25)

Status: implemented on `dev` only; not deployed or merged. No schema, migration, permission, or `tis.db` change. The Student Assessment roster now uses request-backed search, Grade, Section, Assessment-state, Classification and existing Branch ceiling filters. `GET /api/talent/assessment-cycles/{cycle_id}/eligible-students` applies the active global Branch ceiling before every filter, rejects invalid state or classification values, and returns the exact filtered roster population plus backend-produced summaries. Classification is the current completed M17 result only, uses the canonical service and its existing privacy projection, and remains Exceptional-only for Talented presentation. Learning Style is the approved eight categorical values plus Unassigned over this roster population; it is never the deprecated four-dimension profile or the Students-page organization aggregate. The browser renders supplied values and accessible tables/progress labels; it does not calculate classifications or distributions. The former DOM-only roster filter is no longer attached.

Batch 1 Closure - The Global Branch Is A Hard Talent Scope (2026-09-25)

> Amended 2026-09-25 (Part 1 production follow-up, owner decision): the ceiling described here now > applies only to Branch-limited actors. Organization-authorized actors choose All Branches or any > authorized Branch inside Talent, the sidebar Branch being only the default, and the `branch_scope=all` > marker cookie / `user.scope_all_branches` were removed. Original text below is preserved as history.

Status: implemented on `dev` only; not deployed, not merged. Web Service only (no worker/timetable/workflow change). No schema, migration, permission or `tis.db` change. Frontend verified only with the DOM-stub harness and Python route/template tests, not in a real browser.

Correction (supersedes the Batch 1 wording below). The Batch 1 sections state that the active global Branch is only the workspace *default* and that an organization actor may widen to all Branches from inside Talent (`branch_scope=all`). That is withdrawn. Owner decision: **the global Branch is a HARD upper ceiling for every Talent page/API/filter.** A single-Branch global scope (for example one of two sibling Branches) -> Talent shows that Branch only; an internal "All Branches" choice inside Talent can never exceed the global scope. Only an explicit global "All Branches" (organization-wide) scope lets Talent expose All Branches, the Organization Talent Map and cross-Branch comparison.

Finding that shaped the design. Before this closure there was no global "All Branches" state at all: every tenant user always resolves to one Branch (`get_current_user` -> `user.scope_branch_id`), and the sidebar switcher lists Branches only. A ceiling taken from `scope_branch_id` alone would have removed organization-wide Talent for every organization user. The closure therefore adds an explicit organization-wide marker, scoped to Talent: the sidebar switcher shows "All Branches (Talent & Potential organization-wide)" only on `/talent` pages; `POST /scope/branch` with `branch_id=all` sets the `branch_scope=all` cookie (only for an actor who `can_access_all_branches`) and leaves the `branch_id` cookie untouched, so every other module keeps resolving its single working Branch exactly as before; choosing a specific Branch, switching organization, login and logout clear it. `get_current_user` exposes it as `user.scope_all_branches`.

Enforcement (server-side, one helper). `talent_branch_scope.py`: `talent_branch_ceiling(user)` = the single Branch id an actor is confined to in Talent, or `None`. It is the INTERSECTION of the actor's authorization and the active global scope: organization/global actor with a single-Branch scope -> that Branch; organization actor with `scope_all_branches`, or a platform actor with no Branch selected -> no extra ceiling; Branch-limited actor -> no extra ceiling (their own Branch is already the only accessible one). It reads the authenticated request user (never a client parameter). `visible_branch_ids` / `visible_branch_ids_or_none` / `branch_scope_unrestricted` / `branch_within_ceiling` compose the ceiling into the existing visible-Branch resolution, so every route inherits it and there is no second authorization system. A Branch outside the ceiling is treated exactly like a Branch outside the actor's authorization (same 400/403/404 conventions as the sibling routes); an omitted Branch resolves to the ceiling, never to every Branch of the organization.

Surface	Ceiling enforced server-side	How
`organization-analytics` overview, talent-map, program-portfolio, branches/{id}, participation-overlap, longitudinal, students (drill)	yes	`talent_org_intelligence_service.resolve_access_context`: `all_branches` false + `accessible_historical_branch_ids` = {ceiling}; every population query, filter validation, Branch list and Branch existence check reads it
`analytics/...` context, overview, rubric/kpi distribution, competencies, breakdowns, period-comparison, students	yes	`routers/talent_analytics._visible_branches` -> helper; population query + `resolve_filters` branch scope
`results-analytics` classification, talented	yes	`_visible_branches` -> helper
`results-analytics` learning-style	yes	route clamps `branch_id` to the ceiling (other Branch -> 403 `invalid_filter`), then the existing `resolve_population`; the Students module is unchanged
`evaluation-progress` student, branch/{id}, organization, branch-comparison	yes	`_visible_branches` -> helper; `branches/{id}` also requires `branch_within_ceiling` (404)
`assessments` list and per-Assessment authorization, start	yes	Branch filters use `visible_branch_ids` / `branch_scope_unrestricted`
`assessment-cycles` eligible-students, preview, population	yes	same; explicit `branch_id` via `branch_in_authorized_scope` (ceiling-aware, 403)
`learner-profiles/{id}`	yes

visible_branch_ids_or_none (profile shows only records inside the ceiling; 404 if none) | | review-candidates, official-identifications, educator-inputs lists and per-record authorization | yes | same helper | | programs/planning-branches, planning-grades, planning-sections (selector option lists) | yes | Branch list limited to the ceiling; other Branch -> 404 | | /talent/{view} tp-config | presentational | publishes the locked Branch (empty only for global All Branches) |

Client. static/js/talent.js: config.branch is now a ceiling. When set, reconcileBranchScope forces branch_id to it and deletes branch_scope (stale/hand-edited URLs and branch_scope=all cannot widen), the Branch selector offers only that Branch (no All Branches), applyContext cannot choose another Branch, and qs() clamps any branch_id it serialises. With an empty config.branch (global All Branches) All Branches, the Talent Map and individual Branch comparison behave exactly as before. Acceptance A loader hardening and section isolation are unchanged.

Deletion / inactive Students / Talented KPI (owner decisions, unchanged). No permission was widened; force_delete_student_history (the already-authorized history-delete path) still removes every Student-owned Talent table and the existing deletion tests were re-run. Bulk-delete history/force behaviour is out of scope. ADR 0039 status semantics unchanged. The Talented KPI keeps the honest "Talented results" grain; no distinct-Student Talented metric was invented.

Tests: tests/test_talent_branch_hard_scope.py (real tenants/Branches: mirror across every read surface, global All Branches keeps organization comparison, stale explicit Branch rejected, Branch-limited actor and tenant isolation unchanged, tp-config), tests/talent_branch_scope_and_metrics.test.cjs (B3/B4/B6 re-pinned, B6b added). Batch 1 tests that pinned "default, overridable" semantics were updated deliberately (the shared World fixture's org actor now has the explicit global All Branches scope).

Not verified / residual. No real browser. Another already-open tab keeps its own URL until reloaded but the server ceiling applies to every request. The branch_scope=all marker is a per-browser cookie; it only lifts the Talent ceiling and never widens authorization. Offering the All Branches option on other modules' switchers is not done (out of scope).

Deployment Acceptance Batch 1 - Data Correctness, Scope Integrity, Student Deletion And Loading (2026-09-24)

Status: implemented on dev only; not deployed and not merged to master. Whether production now shows the corrected figures and loads reliably needs production verification after a deploy; production infrastructure cannot be measured from the development environment. Web Service only (no worker contract, no timetable change) - the separate tis-timetable-workflow revision is unaffected. No schema, migration, permission or tis.db change. Frontend verified only with a DOM-stub harness and Python route/template tests; not exercised in a real browser.

Owner rule that governs this batch: **Talent & Potential operates from the Students that currently exist in TIS.** Only current Students and their valid current Talent data may contribute to any current Talent figure (counts, participation, completion, Classification, Talented, distributions, percentages, denominators, trends). This supersedes the M15 reading that the Student-vs-Talent count difference was purely a population-scope distinction: the headline was a different *grain*, and orphan/historical rows must never count.

Root cause of "Students participating = 63 while 9 Students exist". The Overview headline frozen_eligible_memberships (labelled "Students participating" in static/js/talent.js) is the frozen-membership row count: talent_org_intelligence_service.frozen_membership_query returns one TalentAssessmentCyclePopulationMember row per Student per Cycle/Program (open or closed), so, for example, 9 Students spread over 7 open or closed Program/Cycle contexts is 63 rows (the exact production composition cannot be observed from the development environment; the mechanism is reproduced by tests/test_talent_batch1_data_scope.py). MetricCode.FROZEN_ELIGIBLE has always been defined at MembershipGrain.FROZEN_MEMBERSHIP; only the label called it Students. The population query also never checked that the Student still exists.

Fix. (1) `talent_current_students.current_student_exists` (correlated EXISTS on the same SchoolGroup) is composed into every governed Talent population read: `talent_analytics_service.population_query`, `talent_org_intelligence_service.frozen_membership_query`, the Evaluation Progress Branch/Organization reads and `list_assessments`, so an orphan or deleted Student's row can never inflate a count, denominator or result. Student `status` (active/inactive) is not a Talent population filter (ADR 0039 keeps eligibility independent of it). (2) `/api/talent/organization-analytics/overview` adds `distinct_students`: DISTINCT current Students in the authorized scope (`talent_org_student_drill.count_distinct_students`, the existing distinct-Student authority, published through the same B2 privacy pipeline at class P2 using the existing `student_drill_population/count/distinct_student` coordinate - no new MetricCode or MembershipGrain; see ADR 0044 Batch 1 amendment). The overview also accepts an authorized `branch_id`. (3) Labels: "Students participating" is bound only to `distinct_students`; the membership figures are "Program participations"; `completed` is "Assessments completed"; the Talented card counts current completed assessment results, so it reads "Talented (Exceptional) results" (a Talented *Student* rule across several Periods would need a governed Period-selection rule and is not invented here).

Metric grain table (A = distinct current Students, B = participation/ membership rows, C = assessment/result rows):

Metric	Label before	Grain before	Label after	Grain after	Authority
Overview headline Students	"Students participating" (63)	B	"Students participating" (9)	A	`organization_overview` -> `count_distinct_students`
`frozen_eligible_memberships`	"Students participating"	B	"Program participations"	B	`coverage_organization_total`
`frozen_eligible` (Program card, Talent Map, Longitudinal)	"Students participating"	B	"Program participations"	B	`coverage_by_program_*`, `talent_org_talent_map`
`completed`	"Assessed"	B	"Assessments completed"	B	coverage rows
`completion_coverage`, `assessment_started`, `started_coverage`	unchanged	ratio/count of B	unchanged	B	valid membership denominators, deliberately unchanged
`participation_overlap` diagonal	"Distinct participating Students"	A	unchanged	A	`participation_overlap_counts`
Student drill list	Students	A	unchanged	A	`fetch_student_rows`
Classification / Talented	"Talented (Exceptional) Students"	C	"Talented (Exceptional) results"	C	`talent_results_analytics_service`
Learning Style distribution	Students	A	unchanged	A	`student_learning_style_analytics`
Evaluation Period / Branch comparison	results	C	unchanged	C	`talent_evaluation_progress_service`
Rubric / competency distribution	results	C	unchanged	C	`routers/talent_analytics.py`

Student deletion completeness (exhaustive FK audit). Ten tables carry a `student_id` or a foreign key to `students`: `student_academic_placements`, `student_audits`, `student_external_identifiers`, `talent_assessment_cycle_population_members`, `talent_student_assessments`, `talent_student_competency_results`, `talent_assessment_audits`, `talent_review_candidates`, `talent_official_identifications`, `talent_educator_inputs`; no other table reaches a Student even transitively. `force_delete_student_history` already removed all ten in one FK-safe transaction; the list is now the exported `STUDENT_OWNED_MODELS` and is locked to the ORM metadata by `tests/test_student_delete_completeness_batch1.py`, so a future Student-owned table cannot be added without being deleted. Nothing is retained (no historical retention store) and nothing orphaned can feed current analytics (the defensive EXISTS above). The permission model is unchanged: normal Delete and Bulk Delete stay blocked while Placement/Talent history exists (`students.delete`/`students.bulk_delete`); only `students.force_delete_history` removes Talent history. Deleting 2 of 10 Students recomputes distinct Students, memberships, Classification, Talented, Learning Style, roster, drill, Branch and Grade projections from the remaining 8 (`tests/test_talent_batch1_data_scope.py`). The historical `test_student_academic_foundation` ObjectDeletedError was a test defect (reading `.id` of a just-deleted instance), fixed in the test.

Global Branch context. *(Correction 2026-09-25: the "default only" / "explicit All Branches" wording in this paragraph is superseded - the global Branch is a hard Talent ceiling; see "Batch 1 Closure - The Global Branch Is A Hard Talent Scope".)* Root cause: the sidebar Branch selector only sets the session scope Branch; Talent never read it. All-Branch (Organization) actors got every Branch's Students on every Talent page unless a per-view Branch filter was chosen,

and `/program-portfolio` silently ignored the `branch_id` the Results view sent. Fix: `routers/talent_ui.py` validates the active Branch (a Branch of the actor's tenant that they can access) and renders it as `tp-config.branch`; `static/js/talent.js` `reconcileBranchScope` makes it the default Branch scope of every Student-data view, drops any Branch/Grade/Section carried by a URL minted under a different active Branch (`scope_branch_id` marker), and honors an explicit "All Branches" choice (`branch_scope=all`). The Branch selector is available on Overview, Assessments, Results, Program Results, Talent Map, Students Across Programs, Students and Progress Over Time. Backend authority is unchanged and authoritative: every Branch filter is validated (a foreign-tenant or unauthorized Branch is rejected, never widened) - added `branch_id` to the Overview, Program Results (`/program-portfolio`, same authority as `/branches/{id}`) and the eligible-students roster, and the rubric read now carries the Branch. Residual risk: another already-open tab keeps its own URL scope until reloaded.

Loading / performance (fresh investigation, measured). Confirmed causes: (1) per-Assessment-row N+1: `/api/talent/assessments` (no filter) re-derived `overall_program_result`, classification, `reassessment_requirement` and its Framework/rubric/descriptor snapshots, permission sets and delete probes for every row - 982 SQL statements for 20 rows (about 49 per row; 1,294 with 6 more Students) - and the Student Assessments page requested every Assessment in the organization then filtered client-side; (2) Results/Student Drill/Evaluation Progress repeated the same per-row derivation (drill 136, classification and Talented 74 each, branch comparison 84); (3) the access context recomputed the actor's permission set five times per request (overview 33, every Organization route 31-34 fixed statements); (4) Evaluation Plans recomputed permissions six times per Period (140); (5) `/api/talent/programs/summaries` raised HTTP 500 for any Program with an annual configuration (`annual.eligible_grade_levels` does not exist; the Programs page silently fell back); (6) Talent static assets had no cache-busting and `/static` sends no Cache-Control, so a browser could pair the freshly rendered no-store page with a script copy from an earlier deployment (the mixed-version state the earlier loader hardening cannot cover). After the fix (same dataset): assessments 111, drill 24, classification 20, Talented 20, branch comparison 30, overview 15, evaluation plans 20 statements, and the counts no longer change when Students are added (guarded by `test_talent_list_endpoints_do_not_scale_with_the_number_of_assessment_rows`, which fails on the pre-fix code: 982 -> 1,294). Fix: `talent_read_batch.py` (opt-in, request-scoped memoization inside `with read_batch(db)` on one request-owned Session; inactive by default so write paths are untouched; discarded on exit so it can never serve data across requests or after a Student deletion), set-based priming of Assessment members/results, `talent_request_permissions.request_permission_checker` (effective permission set resolved once per request; per-request only, decisions identical to `auth.has_permission`), server-side Academic Year/Program filters on the Assessments list, the `summaries` fix, content-hash `?v=` versions on every Talent script/style, and an inline 15-second watchdog that turns a still-untouched server-rendered loader into an explicit "could not finish loading" error with a Reload button (so a script that never runs, fails to parse or is stale cannot leave a generic loader indefinitely). The previously known over-budget tests (overview 31 vs 16, Student Drill 36 vs 12) are resolved (drill's pinned ceiling is now 18 (fixed family of 17) with the never-row-proportional invariant kept). **Render capacity is not shown to be implicated:** locally each handler now costs tens of milliseconds and a bounded statement count; the confirmed causes are code-level (row-proportional queries, stale-asset mixing, one crashing route), and the 512 MB / 1 CPU service cannot be measured from here. Production timing must be confirmed after deploy.

Open owner decisions (not changed here). (a) Non-force Delete and Bulk Delete remain blocked whenever Talent history exists; making an authorized delete remove Talent data would be a permission-model change (options: keep, add force to Bulk Delete under `students.force_delete_history`, or let `students.delete` cascade). (b) Inactive Students still count as current Students (Talent eligibility is status-independent, ADR 0039). (c) A distinct-Student "Talented Students" headline would need a governed rule for which Evaluation Period counts.

Tests: `tests/test_talent_batch1_data_scope.py`, `tests/test_student_delete_completeness_batch1.py`, `tests/talent_branch_scope_and_metrics.test.cjs`, additions to `tests/test_talent_ui.py`, plus the shared fixture helper `tests/talent_test_students.py` (Talent fixtures now create real Students; older fixtures used bare integer ids, which are orphans under the new rule).

Deployment Acceptance Correction D - Professional Student Assessment Editor Redesign (2026-09-24)

Status: implemented on `dev` only; frontend/CSS + tests only. Not deployed and not merged to `master`. Web Service only - the separate `tis-timetable-workflow` revision is unaffected. No schema, migration, permission, scoring, Classification, authorization or `tis.db` change.

Owner-observed production screenshot showed the Student Assessment editor as cramped and hard to scan: multiple narrow competency columns with rubric descriptions wrapping into near-vertical text. The editor is redesigned as a serious professional assessment workflow (presentation only; every mutation and result still comes from the real API):

1 **One competency per row.** `static/js/talent-operations.js` and

`static/css/talent-experience.css` replace the auto-fit multi-column `.tp-assessment-grid` with a single-column layout, so every competency card spans the available width and its description/rubric reads naturally.

1 **Readable rubric tiles.** Each rubric choice is a full-width selectable tile

(`.tp-rubric-tile`) showing a clear rank ("1 / N"), the level title, and the descriptor/description on its own horizontal line - never compressed. The tile grid adapts (`repeat(auto-fit, minmax(200px,1fr))`) so up to five levels sit in one row on wide desktop, fewer as width shrinks, and stack to one column on mobile. The whole tile is the `<label>` radio target with native keyboard/focus semantics; a visible focus ring, a check marker and a strong border/background selected state (never colour-only) are preserved.

1 **Evidence entry.** Evidence remains a labelled `<textarea>` bound to its

competency (`name="evidence-{id}"`), now full-width under the rubric with more vertical space; a read-only/completed assessment disables the editor.

1 **Progress, save and finalize unchanged.** The progress bar ("N of M

competencies"), sequential save and the Complete/Incomplete/Insufficient outcome actions keep the existing backend semantics; completion still shows the backend Overall Program Result and automatic Classification with a Talented badge only for Exceptional. Review Candidate / Official Identification are not shown as current status.

Tests: `tests/talent_operations.test.cjs` (rubric-tile structure, evidence binding, single-column/adaptive CSS) plus the existing `talent_operations`/`talent_runtime_loading`/`talent_student_identity` suites. Structural HTML/CSS verification only; browser visual acceptance is still pending.

Deployment Acceptance Correction C - Learning Style Distribution (2026-09-24)

Status: implemented on `dev` only; not deployed and not merged to `master`. Whether production shows the corrected distribution needs production verification after a deploy. Web Service only - the separate `tis-timetable-workflow` revision is unaffected. No schema, migration, permission or `tis.db` change. Frontend structurally verified with a DOM-stub harness and Python route/template tests; not exercised in a real browser.

Owner-observed defect: every Learning Style category showed "Unavailable" even though authorized Students had Learning Styles. Confirmed causes: (1) the distribution was passed through the Talent P3 primary and complementary small-cell suppression pipeline, so a small cohort (10 Students, minimum cohort 5) suppressed every per-category cell; (2) independently, the Results & Analytics frontend consumed a `buckets` key while the Learning Style route returns `levels`, so a visible distribution rendered no bars. Owner decision (dated ADR 0031 "Acceptance C Amendment"): the Learning Style distribution is authorized Student-domain profile aggregation, not Talent scoring output, and is **not subject to Talent small-cell suppression**.

1 **Semantics.** `Student.learning_style` stays one categorical value (eight

values, or Unassigned); there is no per-Student percentage and the four deprecated `learning_style_*_percentage` columns are operationally deprecated and excluded from current Learning Style analytics and normal product presentation (legacy compatibility paths may still read/write the stored values for preservation, never as analytics authority). The aggregate returns all nine categories in stable order with key, label, count and percentage over one denominator: every authorized Student in the selected population **including Unassigned** (`total_population`). Zero categories are `0` / `0%` and visible. An empty authorized population returns state `empty` with no misleading `0%`.

1 **Boundaries kept.** SchoolGroup, Branch, Grade (Section where already

supported), Student visibility, tenant isolation and the `students.view` permission; `resolve_population` is unchanged; only aggregate counts and percentages are returned, never Student identity.

1 **Scope of the exception.** It applies to this one distribution only.

`talent_analytics_privacy.py`, its thresholds and P1-P7 suppression remain active for Classification, Talented, competency/rubric, result and organization Talent metrics; the Classification and Talented Results routes still require and apply the privacy provider.

1 **Consumers.** The same engine feeds `/api/students/analytics/learning-style-distribution`,

the Students page panel and `/api/talent/results-analytics/academic-years/{id}/learning-style`; all three are corrected consistently and none depends on the privacy provider any more (the Results route no longer returns `privacy_policy_version`).

1 **Frontend.** Results & Analytics gains `learningStyleDistributionSection`

(label, "N Students", percentage, progress bar from the backend percentage, "N Students in this selection, including Unassigned", accessible table, neutral Unassigned row, responsive at 680px); the Students panel shows the same fields. Loader hardening (Acceptance A) and identity behavior (Acceptance B) are unchanged.

1 **Pinned expectations deliberately updated.** The former suppression /

privacy-unavailable panel tests and the 503-when-policy-missing test in `tests/test_student_learning_style_v1.py` were replaced (this also retires the previously failing panel-message test); the Students-list "not promoted" assertion now excludes the aggregate panel; the Acceptance A loader test uses the Learning Style contract body.

1 Performance: one authorized-population query plus in-memory aggregation, as

before; no per-Student or per-category query and no frontend N+1.

Deployment Acceptance Correction B - Student Identity + Automatic Classification + Current Talent Workflow (2026-09-24)

Status: implemented on `dev` only; not deployed and not merged to `master`. Whether production shows the corrected experience needs production verification after a deploy. Web Service only - the separate `tis-timetable-workflow` revision is unaffected. Structurally verified with a DOM-stub harness and Python projection tests; not exercised in a real browser.

Owner-observed production screenshots showed the old Review Candidate / Official Identification concepts still presented as the current Talent workflow. The owner-approved current workflow is: Student -> Program -> Teacher Assessment -> Complete Assessment -> deterministic backend Automatic Classification. The five bands are 1.00-1.99

Needs Improvement, 2.00-2.99 Developing, 3.00-3.74 Meets Expectations, 3.75-4.49 Advanced, 4.50-5.00 Exceptional; **Exceptional is the only Talented classification**. The authority is unchanged and remains `talent_classification_service.py` (ADR 0037 2026-09-23 amendment); the frontend never computes a band. Review Candidate and Official Identification are **preserved legacy historical data only** (no record deleted, no schema change, no API removed, no permission relaxed) and are never current Student status, current Classification, a normal filter/column/action, or the Talented state.

What changed:

1 **Student identity in Talent.** Every identifiable Student rendered in Talent

now uses ONE shared presentation (`static/js/talent-student-identity.js`, loaded on every Talent view): name, canonical `Student.learning_style` (Visual, Auditory, Read/Write, Kinesthetic, Verbal, Non-verbal, Quantitative, Spatial; "Unassigned" when none, never fabricated), and - only where an applicable current result exists - the backend Classification, plus a Talented badge when and only when the classification is Exceptional. Applicable current result = `completed` AND `is_current`. Surfaces: Student Assessments roster, Assessment detail header, Students Across Programs (matrix + cards, per-Program classification, never one universal label), the dashboard Student preview, Learner Profile (Talent section of the Student Profile and the Talent `learner-profile` view), legacy history view, Student Drill.

1 **Backend projections (smallest extensions, no persistence).**

`talent_operational_context.authorized_contexts` adds `student_learning_style` from its already-batched Student lookup; the assessments list adds `classification/classification_score/is_talented` for current Completed rows (reusing the already computed Overall Program Result via `assessment_classification(..., overall=)`); eligible-students members carry `learning_style`; the learner profile exposes `student.learning_style` and a per-assessment `overall_result`; the Student Drill row exposes `learning_style` (its approved-field set and breadth accounting were extended by one field). No new classification column or table; no new permission; Student visibility is never widened (values ride projections that already contained the Student).

1 **Legacy removed from the current experience.** "Talent Review" left the

primary Talent sidebar and the Overview action cards; the route `/talent/reviews` remains (same `talent_review_candidates.view` gate) titled "Legacy Review & Identification History", with legacy-labeled columns, and is reachable from a secondary "Legacy Review & Identification History" link. Removed from normal UX: roster Review Status / Official Identification / Open Review, the Identification Classification filter, Review/Identification states on Students Across Programs, the dashboard Student "Review / Identification" column, candidate/identified metrics as normal Talent Map, branch-comparison and summary figures, and "Meets Program Criteria" as a current status. Learner Profile / Student Profile keep legacy records only in a distinct, labeled "Legacy Review & Identification History" section, and only when the actor already holds the legacy permission.

1 **Fixed defect found on the way.** The Student Drill deduplication key used a

tuple that could hold the nested `overall_result` dict, raising `TypeError` for any Student with a completed Program result; it now uses a canonical JSON key.

1 Performance: no per-Student API call, permission query or classification

endpoint call was added. The assessments list still issues the same per-row statements it did before this change (a pre-existing cost, measured at 36 per Student at b7e3c6a and unchanged); Acceptance B adds none.

1 Pinned expectations deliberately updated: Talent nav no longer includes

Talent Review, branch-comparison metric options (four current metrics), the Student Drill approved identity-field set (`+learning_style`), and the removed legacy filter/column assertions.

Open follow-ups: Learning Style aggregate/privacy correction (Acceptance C, since implemented - see the Acceptance C section above) and the Assessment entry editor body (Acceptance D, since implemented - see the Acceptance D section above); the Student-domain Learning Style badge now renders "Unassigned" (normalized to match Talent surfaces by the follow-up remediation below).

Deployment Acceptance Correction B Remediation - ClinePass Precision Remediation (2026-09-24)

Status: implemented on `dev` only; backend/template/tests + KMS correction. No schema/migration/tis.db change, no new permission, no new Student-discovery path. Web Service only; not deployed and not merged to `master`.

An independent Codex audit of Acceptance B found two blocking defects plus two small follow-ups, now corrected:

1 **Legacy-history projection gap.** Acceptance B stated the Legacy Review &

Identification History surface renders the current automatic Classification, but `routers/talent_review_candidates.py`'s `talent_review_workspace` did NOT return `classification`/`classification_score`/`is_talented`. Fixed by adding those fields from the canonical `talent_classification_service` `assessment_classification(db, assessment, overall=overall)` (reusing the already-computed Overall Program Result, so no per-Student classification or Learning Style query is added). The legacy history surface now shows the Student's current Learning Style and current automatic Classification where applicable. That Classification comes only from the M17 result, never from legacy Review Candidate / Official Identification; Exceptional remains the only Talented band.

1 **Terminology.** The Student Profile Learning Style badge used "Not assigned"

while Talent surfaces used "Unassigned"; `templates/_learning_style.html` now uses "Unassigned" for Learning Style absence everywhere. Unrelated "Not assigned" copy for other concepts (Student ID, Branch, Teacher) is unchanged.

1 **Student Drill dedupe regression test.** A dedicated test now proves the

canonical JSON dedupe key survives a nested `overall_result` (the former unhashable-tuple key raised `TypeError`) and still preserves distinct contexts.

New tests: `tests/test_talent_student_identity_projection.py` (real backend `/api/talent/review-candidates/workspace` classification tests and the Student Drill nested-result dedupe test), `tests/test_student_learning_style_v1.py` ("Unassigned"). `tis.db` unchanged.

Deployment Acceptance Correction A - Talent & Potential Runtime Loading Reliability (2026-09-24)

Status: implemented on `dev` only (frontend/template/CSS; no backend, schema, migration, authorization, tenant, privacy-policy, or analytics-semantics change). Not deployed and not merged to `master`; whether production shows the defect fixed still needs production verification after a deploy. Web Service only - the separate `tis-timetable-workflow` revision is unaffected.

Defect (owner-observed on a deployment): Organization Overview / Results & Analytics, Talent Review and normal navigation/filter actions stayed on the server-rendered "Loading your authorized workspace..." placeholder.

Confirmed root cause (reproduced with a stub harness that runs the real script against a scripted DOM/fetch): M16 made `templates/talent/workspace.html` load `talent-rubric-visual.js` only on Programs / Evaluation Plans / Assessments / Talent Review, but `static/js/talent.js` still dereferenced `window.TalentRubricVisual.intensity` at script-evaluation time. On every other view (Talent Overview, Organization Overview / Results & Analytics, Talent Map, Program Results, Branch Results, Students Across Programs, Students, Progress Over Time, Learner Profile) the script threw a TypeError before `init()` ran, so no request was ever made and the placeholder and `aria-busy="true"` never changed. Node unit tests `require()` the module directly and could never see it. Independent lifecycle weaknesses also fixed: `init()` ran outside any try/catch (a synchronous exception left the loader and produced an unhandled rejection); no fetch had a completion bound (a hung connection meant an indefinite loader); Results & Analytics used one all-or-nothing `Promise.all` (one rejected request replaced the whole page and a slow one held every section); the operational delegates (`TalentOperations`, `TalentProgramWorkspace`, `TalentEvaluationWorkspace`) were dereferenced without a presence check, showing a raw TypeError message; sequential Branch/Grade/ Program selector lookups could stack unbounded waits before the first data request. Talent Review's own path was not affected by the missing-script defect (it loads the rubric module); its exposure was the unbounded request/all-or- nothing behavior, which is fixed generically (inference: a slow/hung request is the only remaining way it could have stayed on the loader).

Runtime lifecycle now (page shell -> bounded essential context -> independent sections):

1 The rubric visual module loads on every Talent view (it is tiny and shared);

`talent.js` additionally resolves it defensively so a missing module degrades a helper instead of aborting the script. Operational/workspace bundles remain surface-specific (M16 preserved).

1 `boot()` is the outermost guard: it replaces the server placeholder

immediately, wires events, runs `init()`, and turns any failure (including a synchronous exception) into an error panel with a real Retry button. A missing delegate script shows a "required page component did not load" state whose Retry reloads the page. `pageshow` (bfcache restore) reloads content.

1 Every read request is bounded: 25 s (`REQUEST_TIMEOUT_MS`) - far above normal

latency, short enough to end a hung request visibly - and 15 s for the small selector lookups (Branch/Grade/Section/Program lists), which run in parallel and never delay first load past a 20 s deadline. The bound also covers reading the response body and settles even if `fetch` ignores abort. Mutations issued by the operational workspaces are deliberately not client-timed-out (aborting a write that may already be committed would mislead). The AbortController, generation/stale-response guard and `cache: no-store` are unchanged; only a superseded generation returns silently, the current generation always reaches success, empty, unavailable or error.

1 Organization Overview (Talent landing) renders its copy and action cards

immediately; the headline figures are an independent section with local loading, empty, error/timeout and Retry states. Results & Analytics replaces `Promise.all` with independent sections (snapshot, Learning Style, Classification, Current Talent, Program result, competency averages, rubric, Grade results, Evaluation progression, Branch comparison, Student results), each ending independently with its own Retry; every request is issued exactly once within `talent.js` (memoized, including the shared Talent Map/Branch-name lookup). Cross-module rubric-distribution deduplication was NOT part of this change and is corrected by the follow-up remediation below; metric semantics, endpoints and debounce (250 ms) are unchanged. Program Results / Branch Results gate the page on their primary payload only; their grade/Branch breakdowns are an independent section.

1 Errors were intended to show only messages authored by the UI or mapped from

an HTTP status, with any other exception text replaced by "This section could not finish loading."; at baseline 507dc59 arbitrary backend `data.detail` could still reach users in `parseApiResponse`, `operationApi` and `fetchRubric`.

The follow-up remediation below replaces this with a curated status/code mapping. `#tp-status` (`role=status`) and `aria-busy` are updated on every terminal state.

1 `talent-experience.js` rubric section: bounded request and a concise Retry

state (its raw `detail` disclosure is corrected in the follow-up remediation).

Verification: `tests/talent_runtime_loading.test.cjs` (23 tests, stub harness in `tests/talent_runtime_harness.cjs`) and HTML-level per-view script-dependency tests in `tests/test_talent_ui.py`. These are structural/stub verification, not a real browser. Not implemented here (later acceptance batches): Learning Style privacy/distribution correction, Learning Style and Classification beside every Student, removal of Review/Official Identification from normal UX, Assessment editor redesign.

Deployment Acceptance Correction A Remediation - ClinePass Precision Remediation (2026-09-24)

Status: implemented on `dev` only; frontend/template/tests + KMS correction. No backend, schema, migration, authorization, tenant, privacy-policy or analytics-semantics change. Web Service only; not deployed and not merged to `master`.

An independent Codex audit of Acceptance A found two blocking defects in the frontend runtime path plus an overstatement in the closeout above:

1 **Raw backend `data.detail` disclosure.** `talent.js` `parseApiResponse()` and

`operationApi()`, and `talent-experience.js` `fetchRubric()`, interpolated arbitrary backend `detail` strings into user-facing errors. Fixed with one shared curated mapper (`static/js/talent-api-errors.js`) keyed by HTTP status plus a known stable backend `code`; arbitrary `detail` is never rendered, and `safeMessage` now requires `userSafe === true` (a bare HTTP status is no longer authority). 401/403/404/400/422/503 and unknown 5xx map to approved copy; raw exception/SQL/Python/JS/endpoint/stack/internal text can never surface.

1 **Duplicate rubric-distribution request.** `talent.js` `rubricReq()` and

`talent-experience.js` `fetchRubric()`/`ensureRubricSection()` both requested the same Program + Academic Year + `assessment_state=completed` endpoint per render generation. Fixed with a shared single-flight read-ownership store (`static/js/talent-rubric-request.js`): `talent.js` owns the request and publishes its memoized promise keyed by context; `talent-experience.js` reuses it for the same context. A different Program selected in that section's own filter is a different context and still fetches independently. Retry (`force`) re-issues its own bounded request; a Program/year change resets the store; the existing generation/stale guards prevent a late response from overwriting the current context.

The runtime stub harness (`tests/talent_runtime_harness.cjs`) now executes the production script composition in order (talent-rubric-visual, talent-api-errors, talent-rubric-request, talent.js, talent-experience.js) and observes fetches from both modules, so the "no duplicate request" guarantee is actually proven rather than assumed. `tests/talent_runtime_loading.test.cjs` grew from 23 to 30 tests (curated error mapping, no raw-detail rendering, cross-module deduplication, retry, Program-change invalidation, stale-response safety). Per-view script dependency tests in `tests/test_talent_ui.py` now also require the two new shared helpers before `talent.js`. `tis.db` is unchanged.

M14-M18 Correction Program Closeout - M18b-3 Final Regression / Performance / Privacy Verification (2026-09-24)

Status: the five-milestone correction program (M14, M15, M16, M17, M18) is functionally implemented on `dev` and closed for owner-acceptance and release planning only. Nothing here states that any of it is deployed,

merged to `master`, or live in production; deployment, merge, and any production-state claim remain separate Owner decisions. M18b-3 is a verification/KMS closeout pass, not a feature: no product semantics, analytics family, or filter was added, and there is no schema or migration change.

Final current-state truth (verified against code and tests, not only docs):

1 Learning Style (M14): `Student.learning_style` is ONE categorical value from

exactly eight (Visual, Auditory, Read/Write, Kinesthetic, Verbal, Non-verbal, Quantitative, Spatial); the aggregate is those eight plus Unassigned. The four legacy percentage columns remain physically in the schema and in the write service only so already-stored values are never silently rewritten; no current API, UI, or analytics surface reads or exposes them (physical removal stays gated on a separate data-occupancy verification).

1 Student/Talent scope (M15): the Student vs Talent count difference is

scope/population semantics, not a deletion defect; a force-deleted Student cannot appear on current Talent surfaces; the roster export is re-importable.

1 Performance (M16): the common Talent page performs at most 3 effective

permission projections (governed regression `tests/test_talent_ui.py::test_talent_page_bounds_authorized_permission_projection_reuse` passed). `authorization.require_any_permission/require_all_permissions` still resolve the current user and allowed keys on every call; the only reuse is `request.state`, which is request-scoped (no global or cross-user cache). `templates/talent/workspace.html` still loads the rubric-visual, operations, program-workspace, and evaluation-workspace bundles only for the views that need them.

1 Classification (M17): the raw Program result stays Program-native 1..N and is

deterministically projected to 1.00-5.00; bands are 1.00-1.99 Needs Improvement, 2.00-2.99 Developing, 3.00-3.74 Meets Expectations, 3.75-4.49 Advanced, 4.50-5.00 Exceptional; ONLY Exceptional is Talented. Authority is backend-only (`talent_classification_service.py`); there is no percentage-band classification. Legacy Review Candidate and Official Identification records are preserved as history/legacy workflow and are not current Talent authority.

1 Current Talent authority (M18a): an applicable current result is

`status == 'completed' AND is_current == True`. Learner Profile and Student Drill use the M17 classification authority. The obsolete four-dimension Branch aggregation and the visible "Protected for privacy" runtime copy are gone; privacy suppression itself is unchanged.

1 Results & Analytics backend (M18b-1): one coherent family (Learning Style,

Classification, Talented) under `/api/talent/results-analytics/...`; frozen ADR 0044 `MetricCode` values are not repurposed. Organization aggregation sums RAW counts across Branches (Branch A 1/2 + Branch B 9/90 gives Organization 10/92, never an average of rates such as 30%); competency and progress keep the governed pre-existing endpoints.

1 Results & Analytics UI (M18b-2/2b): the page consumes those backend families;

nine sections in order (header, summary cards, Learning Style, Classification, Current Talent, Competency Analysis, Results/Evaluation Progress, Branch/Organization comparison, navigation); the current Talented summary is backend-sourced; the Classification filter is Program-bound, narrow-only, and cleared when Program context disappears; `candidate_membership_count` is labeled "Legacy: Meets Program Criteria". No client-side classification, rate averaging, or new chart library.

1 Privacy UX: suppressed values are never rendered as numbers (HTML, chart

datasets, tooltips, ARIA); primary/complementary suppression, anti- reconstruction, Branch isolation, and Organization privacy are unchanged and their suites pass.

Results & Analytics request behavior (structural evidence from `static/js/talent.js`; no timings measured): the analytics view issues one `Promise.all` batch with exactly one request per endpoint (Learning Style, Classification, Talented each once per context); Classification/Talented are only requested when a Program is selected; ordinary filter changes go through the existing `AUTO_APPLY_DEBOUNCE_MS` (250 ms) debounce and `applyContext`, which uses `history.replaceState` (no full-page navigation); the generation counter plus `AbortController` in `load()` prevent a stale response from overwriting a newer one.

Accessibility / responsive (structural only; no browser was available, so NOT visually verified): every distribution has both a `role="group"` bar rendering and a captioned data table (`bucketBars`/`bucketTable`); the Classification filter is a native labeled `<select>`; category meaning is carried by text labels, not color alone; the `.tp-filters` wrap, the 680px/ 980px/760px breakpoints, and the `.tp-table-wrap` overflow container exist in `static/css/talent.css`.

Bounded fix made in M18b-3: M18a made the Student Drill call `assessment_classification`, which recomputed `overall_program_result` a second time for every current+completed row, doubling per-completed-row query cost (measured 2 -> 4 queries per completed row; fixed drill total 36 -> 38). `assessment_classification` now accepts an optional pre-computed `overall` (default behavior unchanged) and the drill passes its row's existing result, restoring the pre-M18a cost (2 per row). Regression guard: `test_completed_row_query_cost_does_not_recompute_overall_result_for_classification` plus an equivalence test in `tests/test_talent_organization_student_drill.py`.

Known unresolved PRE-EXISTING issues (identical at the pre-program commit `50c049f` unless noted; classified with git worktrees, not assumed):

1 Student Drill fixed query budget: `test_query_family_is_bounded_and_not_row_proportional`

asserts `<= 12` queries but the drill already used 36 (and grows ~2 per completed row, an N+1 from per-row `overall_program_result`) before M14; it still fails after the M18b-3 fix (36). Post-correction follow-up: batch the per-row overall-result computation. Related: `test_overview_uses_one_caller_session_and_bounded_set_based_queries` fails at 31 vs a 16 budget, identical at program start.

1 `tests/test_student_learning_style_v1.py::...one_panel_level_protected_message...`

asserts the chart container is absent while the template deliberately renders the category labels with neutral unavailable states and no magnitudes (the page leaks no numbers); stale test expectation to reconcile.

1 `test_force_delete_student_history_removes_protected_academic_history`

(`ObjectDeletedError` in the test's own post-delete refresh), four `test_talent_assessment_cycle_frozen_population` tests, one `test_talent_operational_journey`, one educator-input, and one rubric-policy test fail identically at `50c049f`.

1 Node: 16 of 188 tests in `tests/talent*.test.cjs` + `tests/student*.test.cjs`

fail (13 of 179 at `50c049f`). Three are stale expectations of the removed M14/M18a four-dimension Learning Style Branch metric in `tests/talent_branch_comparison_frontend.test.cjs`; the remaining 13 are old program-workspace/operations/experience expectations that predate the program.

1 PostgreSQL (`TIS_TEST_POSTGRESQL_URL`, local test database, isolated

schemas): 11 passed / 5 failed in `tests/test_postgresql_migration_transactions.py`, the same 5 (FK creation order) at `50c049f`.

1 `tests/test_permission_dangerous_patterns.py` reports unclassified SaaS

permission-shortcut hits (unrelated to Talent; identical at `50c049f`).

1 An unscoped full `pytest tests/` run previously hung in an unrelated

SaaS/timetable area; M18b-3 did not repeat it and used scoped suites.

Deployment/boundary: no Web Service or `tis-timetable-workflow` change; no migration; `tis.db` unchanged (SHA-256 `01e1a3065d92280ee9228db921f67545c8b837d4ecd787a5aeeddc6d128fc136` before and after). GitHub Actions status was not verifiable from this environment.

M18b-2b Results & Analytics IA reorder + Classification filter + accessibility/responsive audit + candidate_membership_count decision (2026-09-24)

Bounded completion pass over the M18b-2 explicit open-item list (a)-(e), on top of the already-landed M18b-2 first pass. No backend semantic change; the M18b-1 contract is unchanged; M18b-3 (final broad regression/performance/KMS closeout for the full M18 milestone) remains separately scoped and still pending.

(a) IA reorder (page-content level): the in-content `lede()` header for the `analytics` view now reads "Results & Analytics" with a dedicated subtext, replacing the generic "Your organization at a glance" copy. The outer page chrome (breadcrumb/`<h2>`/`VIEWS["analytics"]` title, still "Organization Overview") is deliberately left unchanged - `tests/test_talent_organization_analytics_providers.py` pins that literal string for the page shell, and this pass targets the in-content Results & Analytics header called for by the task, not the shared workspace chrome. The 9 sections now render in the required order: page header -> (context/filters live in the sticky `tp-filters` form outside this content region) -> summary cards (Organization snapshot KPI grid + fact strip, now including a single backend-sourced "Talented (Exceptional) Students" count) -> Learning Style -> Classification -> Current Talent -> Competency Analysis (Program result + per-competency averages + rubric distributions, now grouped together - `rubricSection` moved from the very end of the page to immediately follow `competencyAverageSection`) -> Results/Evaluation Progress (Grade progress + Evaluation Period progression, `progressionSection` moved before the Branch comparison section) -> Branch/Organization comparison (Branch comparison + the authorized cross-Program Student preview) -> secondary navigation links.

(b) Progressive Classification filter: a real `Classification` filter control (`<select name="classification">`, the exact 5 backend `CLASSIFICATION_LABELS`) was added to the shared `tp-filters` form and wired only into the `/results-analytics/.../classification` request (never `/talented`, which has no such parameter). It is progressive: hidden until a Program is selected on the `analytics` view (`updateClassificationVisibility`), and cleared whenever hidden so a stale value can never be submitted for a different Program. It can only narrow the returned buckets (backend-validated against `CLASSIFICATION_LABELS`, rejects an unrecognized value), never widen authorization. Evaluation Period and Competency filters were evaluated and explicitly NOT added this pass: they would require new supporting period/competency list endpoints beyond this task's smallest-coherent-change scope, and are not "just wire the UI" the way Classification was (Classification already had a dedicated, documented backend query param per M18b-1). Learning Style's real backend filters (`branch_id`/`grade_level`/ `section_name` only, Student-domain-wide per ADR 0031/M14) are already fully covered by the existing Branch/Grade selectors - confirmed directly against `routers/talent_results_analytics.py`, no new Learning-Style-specific control was added or needed.

(c) Accessibility audit (performed, not just asserted): no chart-type selector exists anywhere in `static/js/talent.js` (grep-verified) - bar-only charts per the compatibility matrix, so no keyboard/chart-type-selector work was required. The new Classification filter is a native `<select>` inside a `<label`

for> (identical pattern to every existing filter field), so it is natively keyboard-operable, has an accessible name, and sits in natural tab order with no tabindex manipulation - traced directly against the existing tp-filters form structure, not assumed.

(d) Responsive verification (performed, not just asserted): independently re-checked static/css/talent.css's breakpoints against every class name the new M18b-2 markup actually uses - bucketBars/bucketTable reuse .tp-grade-chart/.tp-grade-row/.tp-table-wrap (already covered at both the 680px breakpoint and the general responsive rules), talentedSection reuses .tp-primary-indicator/.tp-grade-chart (already covered), and the new Classification filter field reuses the same generic .tp-filters label/select styling as every other filter (flex-wrap at desktop, one field per row via grid-template-columns:1fr at the 680px breakpoint). No CSS gap was found; no CSS change was made in this pass.

(e) candidate_membership_count **decision:** kept, not removed - it is a distinct, still-legitimate Review-Candidate-policy-match count (the legacy Review Candidate workflow's "meets this Program's eligibility policy" grain), never the M17 automatic Classification/Talented grain, and never literally uses "identified"/"candidate_of_eligible" percentage-share wording. Its Results & Analytics summary-card label is relabeled "Legacy: Meets Program Criteria" (previously unqualified "Meets Program Criteria") so it can never be read as a current-Talent figure next to the new Classification/Talented sections on this same page, matching the existing Legacy-qualifier pattern already used for candidate_of_eligible/ identified_of_eligible. The underlying metric/data/permission are completely unchanged; the separate candidate_count label (Program Portfolio/Branch pages, out of scope for this page) is untouched.

Regression: tests/talent_results_experience.test.cjs - 28 tests (24 -> 28, 4 new), 26 passed/2 failed, the identical 2 pre-existing failures by exact test name at both this change and the unmodified M18b-2 baseline 5aa13cb (git-stash-verified, not assumed). tests/ talent_branch_comparison_frontend.test.cjs + tests/ talent_experience_adjustments.test.cjs + tests/talent_ui.test.cjs: 41 tests/38 passed/3 failed, identical 3 pre-existing failures by exact test name at both states (git-stash-verified). tests/test_talent_results_analytics.py + tests/test_talent_ui.py: 51/51 passed. tests/ test_talent_organization_analytics_providers.py (confirms the page-chrome title assertion is unaffected): 22/22 passed. No schema/migration change; tis.db byte-identical before/after this task (SHA-256 verified 01e1a3065d92280ee9228db921f67545c8b837d4ecd787a5aeeddc6d128fc136).

Still open for M18b-3: Evaluation Period and Competency filter controls (would need new supporting list endpoints); a chart-type selector was evaluated as unnecessary (bar-only compatibility matrix) rather than added; full broad M14-M18 regression/performance/KMS closeout.

M18b-2 Results & Analytics Frontend Rebuild - current-Talent indicator + new-family consumption (2026-09-24)

Bounded FRONTEND-ONLY sub-phase consuming the M18b-1 backend contract (/api/talent/results-analytics/...). No backend semantic change; M18b-3 (final broad regression/performance/KMS closeout for the full M18 milestone) remains separately scoped and still pending.

What changed: static/js/talent.js's analytics view (Results & Analytics landing page) now calls all three M18b-1 families - Learning Style (/academic-years/{ay}/learning-style, Student-domain-wide, gated on students.view), Classification (/programs/{pid}/academic-years/{ay}/ classification, Program-bound, exactly the 5 backend bands), and Talented (/programs/{pid}/academic-years/{ay}/talented, Program-bound, Exceptional- only) - and renders them with new reusable helpers: bucketBars/ bucketTable/distributionSection (chart+accessible-table pair for Learning Style/Classification) and talentedSection (primary current-Talent indicator with an optional per-Branch breakdown, reusing the same visual classes as the existing gradeBars/branchBars primitives - no new charting library, no CSS change). The competency (/api/talent/analytics/.../ rubric-distribution) and progress/branch-comparison (/api/talent/evaluation-progress/...) sections were already correctly wired to the right existing governed endpoints and were left unchanged.

Legacy-metric removal from the current-Talent section: the previous `identifiedIndicator` primary indicator (sourced from the legacy `identified_of_eligible`/Official Identification `talent-map` projection) is removed from this view. `talentedSection` (backed by the new M17- classification-derived Talented family) is now the only current-Talent indicator here. The underlying legacy Review Candidate/Official Identification services, routers, data, and the separate Talent Review workspace are completely unchanged; `identified_of_eligible`/ `candidate_of_eligible` remain valid, unrelabeled options only in the still-supported legacy Branch-comparison metric selector (a different control/view), never presented as current Talent state.

Permission surfacing (small, non-semantic, additive): `routers/talent_ui.py` now additively includes `"students.view"` in the `talent_permissions` dict passed to the template, purely so the frontend can gate the new Learning Style section's fetch/render on the same permission its backend route already requires - no new authorization scope, no change to any endpoint's actual permission check.

Regression (git-worktree-verified against the unmodified M18b-1 baseline `2bf9b8c`): `tests/talent_*.test.cjs` - 170 tests/154 passed/16 failed on this task's changes versus 165/149/16 at baseline, the identical 16 pre-existing failures by exact test name in both runs, zero new JS failures. `tests/test_talent_results_analytics.py` + `tests/test_talent_ui.py` (the exactly-named M18b-2 focused Python/UI regression set): 51/51 passed. No schema/migration change; `tis.db` byte-identical before/after (SHA-256 verified).

Explicitly not done in this task (open for M18b-2b/M18b-3): the full 9-section Results & Analytics information-architecture reorder (page header, progressive filter bar across Program/Branch/Grade/Section/ Evaluation Period/Competency/Learning Style/Classification, a full chart- type-selector/keyboard/WCAG audit, and explicit mobile responsive verification) called for by the M18b-2 task specification was not completed end-to-end - this pass targeted the highest-value, most defect-prone item (a legacy metric presented as the current Talent state) and made the three new backend families reachable from the UI with accessible chart+table pairs and privacy-safe neutral states, rather than a full visual rebuild of the page.

M18b-1 Results & Analytics Backend Contract + Correct Aggregation Authority (2026-09-24)

Bounded BACKEND-ONLY sub-phase of the M18b Results & Analytics rebuild (the visible page/chart rebuild is the separate, still-pending M18b-2; no template/chart-rendering change was made here). Delivers the backend authority M18a explicitly deferred.

Contract decision: `talent_org_intelligence_contract.MetricCode` remains frozen and unmodified (ADR 0044) - the new current-classification/Talented grain is NOT added to it. A new module, `talent_results_analytics_service.py`, reuses the existing M9 `talent_analytics_service` Program+AcademicYear context/filter/scope architecture (`resolve_context`/`resolve_filters`/`population_query`) and the same generic `Cell`/`Group`/`apply_primary_privacy`/ `run_complementary_suppression` privacy primitives every M9 breakdown already uses, adding one new opaque privacy class (`"P4"`) for this grain - never the legacy `CANDIDATE_COUNT`/`IDENTIFIED_COUNT`-family metrics, which remain legacy-Review/Identification-grain only. One coherent router, `routers/talent_results_analytics.py` (`/api/talent/results-analytics/...`), serves three families:

1 **Learning Style** (`/academic-years/{academic_year_id}/learning-style`) -

thin reuse of `student_learning_style_analytics.py` (M14), gated on the existing `students.view` permission; no new computation.

1 **Classification**

(`/programs/{id}/academic-years/{id}/classification`) - the current M17 classification distribution (five owner-approved bands) over the exact M18a-governed grain (`TalentStudentAssessment.status == 'completed' AND is_current == True`), reusing `talent_classification_service.assessment_classification` for every band decision (never a duplicated band table; `talent_classification_service.CLASSIFICATION_LABELS` is now exported read-only for this purpose). A rubric-incompatible ("available: False") Assessment is tracked as `not_currently_classifiable_count`, never fabricated into a band.

1 **Talented** (`/programs/{id}/academic-years/{id}/talented`) - current

Talented (Exceptional-only) count, applicable denominator, and rate, projected from the same classification computation.

Correct Organization aggregation (the CRITICAL invariant, regression- tested exactly): `talent_results_analytics_service. sum_raw_counts_across_branches` is a pure function that sums each Branch's own raw bucket counts - the ONLY Organization/Branch rollup rule anywhere in this module. Proof case: Branch A 1 Talented/2 applicable (50%), Branch B 9 Talented/90 applicable (10%) -> Organization 10/92 (~10.87%), never the naive 30% average of the two Branch rates (`tests/test_talent_results_analytics.py`, Sections A and C).

`learning_style_dimension` **dead-parameter cleanup:** re-audited directly (not assumed) and confirmed still genuinely unreferenced inside `branch_comparison_metric`'s own body (the "learning_style" metric it existed to parameterize has never been a member of `APPROVED_BRANCH_METRICS` since M14) - removed outright from `routers/talent_evaluation_progress.py`'s branch-comparison route and from `branch_comparison_metric`'s signature in `talent_evaluation_progress_service.py`; the one stale frontend-contract test assertion for the now-nonexistent parameter (`tests/talent_branch_comparison_frontend.test.cjs`) is removed. No behavior change: `branch_comparison_metric` still rejects any `metric` outside the six `APPROVED_BRANCH_METRICS` exactly as before.

Competency and Progress families are deliberately NOT reimplemented here. `routers/talent_analytics.py`'s existing `/rubric-distribution` route (Program-bound, canonical framework/competency identity, governed completed/current evidence semantics) and `routers/talent_evaluation_progress.py`'s existing branch/organization/ branch-comparison routes (ACTIVE/OPENED weighting, framework comparability, raw Overall Result semantics) already provide governed, privacy-safe backend analytics for those two families with correct, unaltered semantics - M18b-2's frontend calls those existing routes directly for Competency/Progress and the new `/api/talent/results-analytics/...` routes for Learning Style/Classification/Talented.

Regression: full `-k talent` pytest suite (746 passed / 11 failed, worktree-compared byte-identical by exact test name against the unmodified 81cfa75 baseline - all 11 pre-existing, zero new) and the full `node --test tests/*.test.cjs` suite (17 unique pre-existing failures, down from 18 at baseline - `talent_branch_comparison_frontend.test.cjs`'s stale `learning_style_dimension` assertion fixed, zero new failures). No schema/migration change; `tis.db` byte-identical before/after (SHA-256 verified). M18b-2 (the visible Results & Analytics page/chart rebuild consuming this contract) remains explicitly out of scope and was not implemented here.

M18a Current Talent Authority Alignment + Learning Style Cleanup + Privacy UX (2026-09-23)

Bounded first half of the M18 milestone (the full Results & Analytics page rebuild is M18b and remains out of scope here). Four areas, closed:

Applicable-current-result authority (resolved, not invented): for a Talent Program, the Assessment that governs a Student's current M17 classification is the `TalentStudentAssessment` row where `status == 'completed'` AND `is_current == True` - the exact pre-existing authority ADR 0036 (Talent Rubric Reassessment Attempts) already established and that M9 analytics (`talent_analytics_service._assessment_ids_subquery`) and the B9 Student Drill already consumed before this milestone. No new rule was introduced.

Part 1 - current Talented authority: the Learner Profile (`talent_learner_profile_service.build_learner_profile`) and the B9 Student Drill (`talent_org_student_drill.py`) now additively expose the same backend-authoritative M17 classification (`classification`, `classification_score`, `is_talented`, via `talent_classification_service.assessment_classification`) already wired into `routers/talent_assessments.py` since M17 - reusing the single classification authority, never a duplicated derivation, never sourced from `TalentReviewCandidate`/`TalentOfficialIdentification`. Building a NEW org/Branch AGGREGATE "current Talented count" metric was explicitly evaluated and deferred to M18b: `talent_org_intelligence_contract.MetricCode` is a frozen 14-value enum (ADR 0044 explicitly states "no MetricCode extension"), and the only existing Talent-adjacent aggregate metrics (`CANDIDATE_COUNT`/`CANDIDATE_OF_ELIGIBLE`/`IDENTIFIED_COUNT`/ `IDENTIFIED_OF_ELIGIBLE`) are grounded in the legacy Review/Identification membership grains, not an M17-classification grain - adding a privacy-safe Cell/Group aggregate for the new grain is real new analytics infrastructure that belongs with the M18b Results & Analytics rebuild, not a cleanup task. `static/js/talent.js`'s `candidate_of_eligible`/`identified_of_eligible` labels ("Talent share"/"Officially confirmed share" - which read as CURRENT Talent-status shares even though their underlying data is the legacy Review/Identification workflow) are relabeled "Legacy review share"/"Legacy identification share"; the underlying metrics, data, and permissions are completely unchanged. `TalentReviewCandidate`/`TalentOfficialIdentification` rows, services, and routers remain fully preserved as legacy/history, byte-identical, code-unchanged.

Part 2 - M14 Learning Style deprecation completed: `routers/students.py` (`_student_json`) and `routers/students_ui.py` (`_student_view`) no longer serialize `learning_style_verbal_percentage`/`learning_style_non_verbal_percentage`/ `learning_style_quantitative_percentage`/`learning_style_spatial_percentage` in the current normal Student API/UI projection (verified: they still did, including as literal `null`, before this milestone). Stored DB columns and historical values are completely untouched - only read exposure is removed. `talent_evaluation_progress_service.py`'s `learning_style_branch_aggregate`/ `_learning_style_values_by_branch`/`_LEARNING_STYLE_COLUMNS` (dead since the M14 dispatcher removal, kept only so the deprecated-column computation was not silently mutated) were confirmed genuinely unreachable from any current endpoint/service path and removed outright, along with their two stale direct-call tests. There is no current four-dimension Learning Style analytics authority anywhere in the repository after M18a. M14's own governing model (one categorical `learning_style` value; aggregate distribution over eight categories + Unassigned; no per-Student percentage dimensions or bars) is unchanged and still served by `student_learning_style_analytics.py`.

Part 3 - privacy copy cleanup (presentation only): the literal visible phrase "Protected for privacy" is removed from Talent & Potential / Learning Style UI (`static/js/talent.js`, `talent-rubric-visual.js`, `talent-experience.js`, `templates/students.html`) and replaced with context-appropriate neutral copy ("Unavailable", "Not available for this view", "This distribution/rubric distribution is not available for this selection", "Distribution unavailable") - never one mechanical replacement everywhere. The underlying privacy/suppression contract (primary suppression, complementary suppression, minimum-population rules, anti-reconstruction, tenant/Branch/organization scope) is completely unchanged; no protected numeric value was ever in the copy being replaced, and none is introduced by the new neutral copy.

M18b (the full Results & Analytics page rebuild, including any new org/Branch aggregate Talented-count analytics) remains explicitly out of scope and was not implemented here.

M17 Automatic Assessment Classification + New Normal Talent Workflow (2026-09-23)

Owner-approved product direction: Student -> Program -> Rubric -> Teacher assesses -> Complete/Submit -> the backend automatically classifies the result. On a Completed Assessment, TIS derives the ADR 0037 Overall Program Result as before, then (ADR 0037's 2026-09-23 Amendment) deterministically projects it onto a governed 1.00-5.00 classification scale and assigns exactly one fixed, owner-approved band: 1.00-1.99 Needs Improvement, 2.00-2.99 Developing, 3.00-3.74 Meets Expectations, 3.75-4.49 Advanced, 4.50-5.00 Exceptional. Only Exceptional is Talented. These bands are never a percentage and never reuse `normalized_percent`.

Architecture decision (Part 1): real configured Talent Programs do not universally use a five-level rubric (the realistic seed dataset in `talent_local_test_data.py` configures 4-level, 3-level, and 4-level Programs). Mandating a system-wide five-level rubric would have broken real Program configurations. TIS instead preserves each Program's own configured 1..N rubric scale for the raw educational result exactly as ADR 0037 already defines it, and adds one additional, separately governed deterministic linear projection from that raw 1..N average onto the fixed 1.00-5.00 classification scale, reusing the same linear-rescale technique ADR 0037 already approves for `normalized_percent`. See `talent_classification_service.py` and ADR 0037's 2026-09-23 Amendment.

The normal current workflow no longer requires a manual Review Candidate or Official Identification step to make a Student Talented - `Talented` is now a direct backend consequence of a Completed Assessment's automatic classification. Existing `TalentReviewCandidate`/`TalentOfficialIdentification` rows (including ones recorded before this milestone) are fully preserved as legacy/history evidence, remain reachable through their existing services, and are never silently rewritten to match a later automatic classification - they simply no longer govern current `Talented` state. No schema migration was required: classification is a deterministic read projection over the same immutable Completed-Assessment inputs the Overall Program Result already uses, computed only when `status == "completed"`.

API: `/api/talent/assessments/*` and `/api/talent/review-candidates/*` responses additively expose backend-computed `classification`, `classification_score`, and `is_talented` fields; the frontend only ever displays these values and never derives or spoofs a band. The Talent assessment and legacy Review workspaces (`static/js/talent-operations.js`) show the classification/Talented state, and the legacy Review workspace copy now describes Review status/Official Identification as preserved history rather than a required step. M18 (Results & Analytics rebuild, including the Learner Profile/organization-analytics visible-copy implications of this new classification authority) remains out of scope and was not implemented here.

M16 Talent Module Performance And Student Actions (2026-09-23)

Implemented on the isolated `m16-talent-performance` branch from `50c049f`. Talent page permission projection calls are bounded at three (previously roughly 28-31 through repeated per-key checks), and unrelated operational JavaScript is not emitted by Overview/analytics or other non-owning surfaces. Depending on the surface, two to four of the former six scripts are emitted; Overview/analytics drop about 135 KB of unminified JavaScript. Student action containers now render as wrapping horizontal flex rows. No schema, migration, API, authorization, tenant/Branch isolation, analytics calculation, or product-semantic change was introduced. Deployment surface is Web Service only; the timetable workflow is unchanged.

M14 Students + Talent & Potential - Learning Style Correction + Aggregate Distribution (2026-09-23)

Owner-directed correction milestone following a read-only Post-M13 Correction Review. Two corrections, both governed by amendments to ADR 0031 and ADR 0042 (see each ADR's own "M14 Amendment"/"M14 Correction" section) and, for the removed Talent metric, ADR 0044's M14 Amendment section:

**Learning Style is ONE categorical selection per Student, extended from

four to eight values.** `Student.learning_style` now accepts Visual, Auditory, Read/Write, Kinesthetic (original four, unchanged) plus Verbal, Non-verbal, Quantitative, Spatial (added). Those four added values were previously, mistakenly, modeled as an independent four-dimension percentage profile (`learning_style_verbal_percentage`, `learning_style_non_verbal_percentage`, `learning_style_quantitative_percentage`, `learning_style_spatial_percentage`; ADR 0042/M1-M3). Verified against actual code that, contrary to ADR 0042's own stated M1 scope boundary, the M2/M3 delivery had already built a full create/update/GET API and a real Student create/edit frontend for the four percentages - while the ORIGINAL single-select categorical field from ADR 0031 had never received any create/edit frontend selector at all until this milestone. Student create/edit now shows one Learning Style selector (all eight values, plus "Not assigned"); the Student profile shows the one selected value (or "Not assigned") instead of four percentage bars; Talent Student context reads the current categorical value directly (no historical snapshot, no effect on rubric/Assessment/Overall Result/Evaluation Progress/classification/Candidate-Identification).

1 **"Percentage" means population/aggregate distribution, never a

per-Student dimension percentage.** The four `learning_style_*_percentage` columns are now OPERATIONALLY DEPRECATED: no longer written (API/UI create/edit no longer accept them - silently ignored, never a validation error), no longer displayed anywhere, and no longer read as Learning Style authority anywhere, including the M4/M10 Talent Evaluation Progress "Learning Style" Branch-comparison metric (which averaged them) - that metric is REMOVED from `talent_evaluation_progress_service.APPROVED_BRANCH_METRICS` (six approved metrics remain; a request for it now receives the same `invalid_filter`/400 as any other unrecognized metric). Existing stored percentage values are preserved completely untouched (no backfill, no conversion, no clearing) - physical column removal is a separately gated, later cleanup contingent on an explicit data-occupancy verification this milestone could not safely perform (the local `tis.db` has no `students` table at all, so real occupancy could not be inspected either way). `student_learning_style_analytics.py` (the existing privacy-safe categorical count/percentage/"Unassigned" distribution engine, reused and extended rather than parallel-built) now covers all eight categorical values plus an "Unassigned" bucket for the authorized Student population; the denominator is always every authorized Student in scope, including Unassigned Students - this is served at the existing Students page summary location, unchanged in scope/authorization/privacy contract from ADR 0031.

Database: one forward-only migration (`20260923_002_student_learning_style_eight_values`) widens `ck_students_learning_style` from four to eight values on PostgreSQL (`DROP CONSTRAINT IF EXISTS` + `ADD CONSTRAINT ... NOT VALID` + `VALIDATE CONSTRAINT`, mirroring the exact pattern already used elsewhere in `db_migrations.py` for widening a CHECK constraint under the same name); SQLite continues to rely on the service-layer guard for an already-migrated database, exactly like the original `_student_learning_style_v1` migration, and a fresh SQLite database gets the widened constraint directly from `models.py`'s current `CheckConstraint`. No column is added or dropped, no row is rewritten, and `tis.db` is unchanged (verified by SHA-256 before and after this task).

`student_roster_service.py`'s column contract was inspected and does not include Learning Style or the four percentage columns at all - roster scope is therefore untouched by this milestone, per its explicit boundary.

Out of scope for M14 (left for later, separately governed milestones): Student deletion/count correction, roster redesign, Student action layout, Talent whole-module performance, assessment classification, the full Results & Analytics rebuild, and any privacy-copy cleanup beyond the Learning-Style-specific surfaces touched here.

M13 Students + Talent & Potential Release Readiness Closeout (2026-09-23)

Final release-readiness verification for the bounded Students + Talent & Potential update package (Student ID, Learning Style, Al-Andalus Section display, Student roster, Talent frontend cleanup, Evaluation Progress, Results & Analytics Branch comparison, M1-M12.1). This is documentation-only verification/closeout, not new development and not a deployment.

Spot-checked each feature area against current code (not only KMS prose) and found the implementation matches documented behavior: `models.py` Learning Style columns/CHECK constraints and the canonical <=63-character managed active-per-Student index name; `db_migrations.py` ledger ordering (`20260922_001` -> `20260922_002` -> `20260923_001`); the Al-Andalus workspace UUID gate; `routers/students.py` permission checks for `students.import`/`students.export`; `static/js/talent-operations.js`'s `button('reload', ...)` no-op and `talent_educator_inputs.*` permission short-circuit removing Reload Saved Rubric and Educator Input from the normal Talent UI; and the backend-authoritative Overall Result/framework-mismatch handling in `static/js/student-evaluation-progress.js`/`static/js/talent.js`.

Corrected three stale current-state wording instances left over from before M11 shipped, found by a repository-wide sweep for "deferred"/"assigned to M11"/"remains absent" phrasing scoped to this package - `docs/AI_PROJECT_CONTEXT.md`'s M7 section, `docs/TIS_MASTER_CONTEXT.md`'s M7 section, and `docs/engineering/TIS_MODULE_MAP.md`'s Students module section each still said roster import/export was "deferred"/"assigned"/"absent" pending M11, even though M11 (documented immediately above each of those sections in the same files) has since implemented it. Each was reworded to state the now-accurate current fact (M7 was out of scope; M11 implemented it) without altering the historical description of what M7 itself added. No other stale "deferred"/"not implemented"/"planned"/"dormant"/"backend only"/"frontend pending" wording scoped to this package was found; dated `docs/CHANGE_HISTORY.md` entries and other dated `docs/PROJECT_STATE.md` section headers describing what was true at the time they were written were left unchanged, matching the historical-log convention.

Reconfirmed the M12.1 PostgreSQL remediation entry below is complete and internally consistent (canonical index name, forward migration, no Student row mutation, `tis.db` SHA-256 unchanged) and that the M12 entry's forward reference to M12.1 is accurate. Re-ran, on real PostgreSQL (`TIS_TEST_POSTGRESQL_URL`), `tests/test_postgresql_migration_transactions.py`: 11 passed, 5 failed - the same five pre-existing, unrelated Talent baseline-metadata foreign-key-ordering failures M12.1 already documented (`UndefinedTable` for `talent_framework_competencies`), unchanged and not introduced by this package. `tests/test_permission_registry_matrix.py` (13 passed) and `tests/test_student_managed_number_service.py` (46 passed) were also re-run and pass.

Added a new durable release-handoff document, `docs/releases/2026-09-23-students-talent-m1-m13-release-handoff.md` (indexed in `docs/README.md`), covering user-visible changes, permission scope, the three required migrations and the PostgreSQL remediation requirement, compatibility notes, privacy/tenant-isolation invariants, known pre-existing issues, deployment surfaces (Web Service only - this package does not require a coordinated `tis-timetable-workflow` deploy), a migration/deployment sequence plan, rollback considerations, and a post-deploy smoke-test matrix. No schema, migration, permission, or product behavior change was made in this closeout task; `tis.db` is verified byte-identical (SHA-256 `01e1a3065d92280ee9228db921f67545c8b837d4ecd787a5aeeddc6d128fc136`) before and after. Closing M13 closes only this bounded package, not the entire Talent product roadmap.

M12.1 PostgreSQL Managed Student Number Index Identifier Remediation (2026-09-23)

Fixes the PostgreSQL identifier-length risk M12 (below) documented but deliberately did not fix. The M1 active-per-Student partial unique index was declared as `uq_student_external_identifiers_tis_student_number_active_student` (65 characters), exceeding PostgreSQL's 63-character `NAMEDATALEN` limit. Live verification against a real PostgreSQL test database (`TIS_TEST_POSTGRESQL_URL`) in this task confirmed PostgreSQL silently truncates it on `CREATE UNIQUE INDEX` to the 63-character physical name `uq_student_external_identifiers_tis_student_number_active_stude`, and confirmed re-running the original migration function a second time against that truncated physical index fails with `psycopg2.errors.DuplicateTable`, exactly as M12 predicted.

The canonical replacement name is `uq_student_external_identifiers_tis_number_active_student` (57 characters). `models.py`'s `Index` declaration and the original `20260922_002_student_tis_number_identifier_integrity` migration (`db_migrations.py`) now create this canonical name directly on any FRESH database (SQLite or PostgreSQL) - the old 65-character name is never created anywhere anymore. A new forward migration, `20260923_001_student_tis_number_index_identifier_length_remediation`, handles an already-migrated PostgreSQL database: using SQLAlchemy's PostgreSQL catalog inspection (`inspect(...).get_indexes(...)`, including the reflected `postgresql_where` partial predicate), it verifies the actual definition of any index at the old truncated name (columns, uniqueness, predicate) before renaming it with `ALTER INDEX ... RENAME TO` - never a drop/recreate, never a row rewrite. It is a no-op if the canonical index is already present, reuses the original migration's existing preflight-conflict checks (non-canonical values, cross-SchoolGroup duplicates, multiple active rows) to create the canonical index directly if neither name exists, and fails closed with a descriptive `RuntimeError` - never a silent guess - if both names exist simultaneously or if an unexpected index occupies either name.

The partial-unique-index semantic invariant (ADR 0043: `UNIQUE (student_id) WHERE namespace = 'tis_student_number' AND status = 'active'`) is completely unchanged, and the separate global Student-number value-uniqueness index (`uq_student_external_identifiers_tis_student_number_value`, 56 characters) was inspected and confirmed to have no length problem - it was not touched. No `Student` or `StudentExternalIdentifier` row was read, rewritten, merged, or deleted. The checked-in `tis.db` is verified byte-identical (SHA-256 `01e1a3065d92280ee9228db921f67545c8b837d4ecd787a5aeeddc6d128fc136`) before and after this task; SQLite is unaffected by this fix because it never truncated the name. This is a PostgreSQL-only physical schema-identifier correction - no product, API, or frontend change, and it does not itself require a coordinated Web + Timetable Workflow deploy (Students/Talent schema only, unrelated to the timetable solver/worker contract). Tested live against a real PostgreSQL database (`tests/test_postgresql_migration_transactions.py`): fresh-database canonical-name creation, truncated-legacy-index rename, idempotent no-op on rerun, no-op when canonical already present, fail-closed on an unexpected index at either name, fail-closed when both names exist, row-data and uniqueness-semantics preservation across the rename, and the migration-ledger path for a database where the original M1 migration is already recorded as applied - all passed. `docs/adr/0043-tis-student-number-managed-identifier-invariant.md` records this as an implementation-detail amendment (physical index name only; the ADR's governance decision is unchanged).

M12 Integrated QA / Regression Verification (2026-09-23)

Students + Talent & Potential M1-M11 were re-verified end to end (Student ID, Learning Style, Al-Andalus Section display, Evaluation Progress, Student roster backend/frontend, Talent frontend cleanup, Results & Analytics, permissions, tenant isolation, historical/frozen attribution, privacy/ suppression). No new functional/UX regression was found. One stale test fixture was corrected: `tests/test_permission_registry_matrix.py` still listed `routers/students_ui.py` as a live consumer of `talent_educator_inputs.view` after M8 (commit `b7c4791`) intentionally made that route pass `include_educator_inputs=False` unconditionally instead of gating on the permission; the permission itself remains active-enforced via its two real consumers (`routers/talent_educator_inputs.py`, `routers/talent_learner_profiles.py`). This is a test-fixture correction only, matching already-shipped M8 behavior - no product, permission, or architecture change.

Deployment risk identified here - remediated the same day by M12.1 above: the PostgreSQL index name `uq_student_external_identifiers_tis_student_number_active_student` (`models.py`; re-declared as raw DDL in `db_migrations.py`, migration `20260922_002_student_tis_number_identifier_integrity`; introduced in Student M1 commit `e98214f`) is 65 characters, exceeding PostgreSQL's 63-character `NAMEDATALEN` identifier limit. PostgreSQL silently truncates the name on `CREATE UNIQUE INDEX` rather than erroring, which breaks this migration's own idempotency check (`_index_exists()` compares against the full untruncated 65-character name and will never match the truncated 63-character name PostgreSQL actually stores) - a redeploy that re-applies the full migration ledger against a real PostgreSQL database would attempt `CREATE UNIQUE INDEX` a second time and fail

with a `DuplicateTable`/ "relation already exists" error, aborting that migration. SQLite (`tis.db`) has no such identifier-length limit and is unaffected, which is why this was invisible to all SQLite-backed test runs; it was found and reproduced this session against a live PostgreSQL test database. Fixing it requires renaming the index in schema-defining code (`models.py`/`db_migrations.py`) plus an Owner/architecture decision on rename-and-release coordination for any environment where this migration may already have run - both explicitly outside M12's verification-only scope. This must be triaged before any future PostgreSQL deployment re-applies the migration ledger; it does not block SQLite-based development/M13 frontend work.

Student Roster Import / Export Frontend M11 Implemented (2026-09-23)

The existing Students list now presents **Import Students** only with `students.import` and **Export Students** only with `students.export`; M6 still authorizes every API request. Export consumes `GET /api/students/roster/export` as an opaque workbook download and preserves the server filename.

Import uses a transient accessible dialog rather than a persistent page panel. It accepts `.xlsx` only, shows the selected filename, uploads multipart field `roster_file` to the M6 preview endpoint, and renders backend-provided totals, valid rows, row numbers, fields, safe messages, canonical Student/Placement display fields, and privacy-safe duplicate identity where supplied. The UI uses text nodes for backend content and does not expose raw exceptions.

Apply requires an explicit confirmation and uploads the original workbook again to the M6 apply endpoint. It never submits client-generated valid rows or offers partial import. Successful atomic apply clears stale state and reloads the Students roster; rejection retains useful preview/file state and explains that no Students were created. Copy is create-only and does not imply update, merge, sync, overwrite, or upsert.

Student IDs remain strings with canonical `STD` presentation and leading zeroes. The browser performs no workbook parsing, uniqueness check, tenant inference, Branch/Academic-Year/Section authorization, or `section_display` conversion. No CSV/`.xls`, persistent batch, background job, backend change, schema, or migration is included.

Student Integrity, Talent Visibility & Roster Round-Trip M15 Implemented (2026-09-23)

M15 (isolated `m15-student-integrity-roster` feature branch, off `50c049f`) delivers two owner corrections without schema change.

Student permanent delete -> Talent visibility. `force_delete_student_history` (`student_academic_service.py`) already deletes every Student-owned table in FK-safe child-before-parent order and never commits itself; Talent current operational surfaces INNER-join `models.Student`, so a force-deleted Student cannot appear anywhere in Talent & Potential. The owner's "~9 vs ~13" mismatch is a population-scope distinction, not a deletion defect: the Students list is Branch-scoped (single authorized Branch unless `students.view_all_branches` + organization/global scope) over current effective Placement, while Talent/Review span frozen `TalentAssessmentCyclePopulationMember` membership (historical, multi-Branch) and the eligible-students roster intentionally does not filter `Student.status` (ADR 0039). Locked by `tests/test_student_delete_talent_visibility.py` (full-chain cascade removal, blockers, rollback/transaction boundary, tenant isolation).

Student roster round-trip. `student_roster_service.py` export now re-imports cleanly: `section_display` is accepted-but-ignored (display-only, never identity; `section` remains canonical identity per ADR 0045), and the canonical `STD` prefix is stripped back to the 10-digit business value on import. Import stays create-only: unchanged exported rows classify **NO_CHANGE** (managed Student ID resolves identity; name/gender/status + current Placement Branch/Year/Grade/ Section must match exactly), new rows **CREATE**, and any mutated existing row stays a blocked `student_id_conflict` (never update/merge/upsert). Apply writes only CREATE rows, skips NO_CHANGE rows, and stays atomic. Export formatting adds a bold header, frozen header row, deterministic widths, and autofilter. `static/js/students-roster.js` renders CREATE / No change / error from backend row status. Permissions, tenant/Branch scope, uniqueness, and audit unchanged.

Results & Analytics Branch Comparison Frontend M10 Implemented (2026-09-23)

Organization Overview's prior fixed Branch summary is replaced by one primary, Program-scoped Branch comparison chart consuming the existing M4 `GET /api/talent/evaluation-progress/programs/{program_id}/academic-years/{academic_year_id}/branch-comparison` contract. The metric selector exposes only Evaluation Period Result, Overall Result, Assessment Completion, Assessments Started, permission-projected Meets Program Criteria and Officially Confirmed, and Learning Style. Learning Style conditionally exposes only Verbal, Non-verbal, Quantitative, and Spatial and sends the selected dimension to the backend.

Backend Branch and Evaluation Period ordering is preserved. Existing Talent Map Branch columns provide the already-authorized display names for returned Branch IDs. A numeric bar is rendered only for `state="visible"`; zero is valid, while suppressed/complementary-suppressed, restricted, coarsened, and no-data states remain distinct textual categories. Numeric/count fields are ignored for every non-visible state. Framework-changed Overall Result remains non-numeric and is explained without cross-version calculation.

The frontend does not calculate Branch or Organization means, derive one metric from another, normalize Learning Style, inspect privacy thresholds, reconstruct suppressed values, or group by current Placement. No Organization summary is added because it is not a uniform field of the seven-metric comparison contract. Frozen historical Branch attribution remains backend-authoritative. No second chart, M11 roster frontend, backend semantic change, privacy redesign, schema, or migration is introduced.

Student Evaluation Progress Frontend M9 Implemented (2026-09-23)

The existing Student Profile Talent tab now consumes the M4 `GET /api/talent/evaluation-progress/programs/{program_id}/academic-years/{academic_year_id}/students/{student_id}` contract for every authorized Program/Academic-Year section already present in the Learner Profile. It shows the backend-ordered active/opened Evaluation Periods using their configured labels and explicit result states. Available percentages, including real zero, are displayed exactly; Pending and other unavailable states are textual and never rendered as `0%` or as a zero-valued progress bar.

The page renders **Overall Result** only from a backend-provided comparable `current_overall_result`. A backend `framework_changed` response preserves all individual Period results, omits the combined number, and explains that the frameworks are not comparable. The dedicated frontend renderer does not read `nominal_weight`, sort Periods, calculate averages, select active Periods, or decide comparability. The endpoint retains existing Student/Talent permission, tenant, and frozen historical Branch authorization; aggregate suppression is not applicable to this single-Student contract.

Existing frozen Grade/Section presentation and M5 server-derived `section_display` remain unchanged. No Branch/Organization Evaluation Progress UI, M10 Branch comparison chart, M11 roster frontend, backend calculation, privacy redesign, schema, or migration is introduced.

Talent Frontend Cleanup M8 Implemented (2026-09-23)

The normal Talent assessment workspace no longer presents the Reload Saved Rubric action or Educator Input read/create/amend/history controls. Their obsolete frontend fetch/state/handlers are no longer active. Student Profile's Talent tab likewise no longer requests or renders Educator Input as a normal product concept.

This is presentation cleanup only. Historical rubric responses, competency results, persisted Educator Input rows and amendment lineage, Official Results, Candidate/Identification evidence, audit history, frozen population context, and Framework references remain unchanged. Backend APIs/services and permissions remain available for historical compatibility. Deterministic scoring and M5 server-derived Section presentation are unchanged.

Current four-dimension Learning Style display inside Talent is not assigned to this cleanup by the authoritative roadmap and was not added. M9 Evaluation Progress frontend, M10 Branch analytics charts, M11 roster frontend, migrations, schema changes, new privacy logic, and new Student/Talent business rules remain out of scope.

Students M7 Frontend Implemented (2026-09-23)

The existing Students Jinja UI now uses the M2/M3 backend contracts for managed TIS Student IDs and the current four-dimension Learning Style profile. New Student creation renders a fixed `STD` prefix and submits exactly ten text digits through `create_student_with_number`; list/profile render the canonical value, and a legacy missing value is neutral rather than fabricated. Authorized assignment/replacement is a separate `students.manage_identifiers` UI action. Duplicate handling uses `describe_student_number_conflict`, exposing only the approved minimal same-tenant identity when independently authorized and staying generic for cross-tenant/unavailable values.

Create/edit expose optional Verbal, Non-verbal, Quantitative, and Spatial whole-number inputs bounded 0-100. Profile display distinguishes unavailable from 0%, provides visible text and progress ARIA, and never normalizes the four values. The deprecated categorical value is preserved as historical data but is not auto-converted or used as the normal Student profile/list/edit presentation. Existing Placement and M5 server-produced `section_display` presentation are consumed unchanged, including historical Placement context.

Implementation is bounded to `routers/students_ui.py`, existing Student templates/JS/CSS, and focused tests. No schema/migration, backend API contract, canonical Section identity, Student architecture, Learning Style architecture, roster import/export frontend, Talent cleanup, Evaluation Progress UI, or Results/Analytics chart work was introduced.

Student Roster Import/Export Backend Implemented (M6) (2026-09-23)

Following the M6 governance prerequisite below (permission registration, 2026-09-22), the `students.import`/`students.export` roster import/export backend is now implemented. First release, .xlsx-only, no frontend (M11, later), no persistent import-job/batch table, no schema/migration.

Service: `student_roster_service.py` is a bounded roster service reusing `student_academic_service.py`'s canonical Student/TIS Student number/Academic Placement invariants (`create_student_with_number`, `create_placement`, `describe_student_number_conflict`) - it never duplicates canonicalization, uniqueness, or privacy-disclosure logic. `student_academic_service.py` gained one new shared helper, `describe_student_number_conflict`, extracted from the existing single-Student create/replace 409 conflict handler in `routers/students.py` so that the direct API, roster preview, and roster apply all share exactly one implementation of the ADR 0043/M2 privacy-safe conflict-disclosure contract (same-tenant + `students.view` reveals `student_id`/`display_name`; cross-tenant or unauthorized same-tenant is fully generic) - never a richer or different disclosure path for either surface.

Scope decision (judgment call, not previously documented): M6 import is CREATE-ONLY for this first release - every workbook row enrolls a NEW Student with an initial Academic Placement. Updating an existing Student's identity or placement through the roster file is out of scope for M6 and was not attempted, avoiding invented merge/overwrite semantics the KMS does not authorize.

Routes (`routers/students.py`, existing `students.*` permission-check pattern via `_authorize`/`auth.has_permission`, tenant/Branch scope reused from existing Student routes):

1 `GET /api/students/roster/export` (`students.export`) - streams an `.xlsx`

workbook (`openpyxl`, matching `routers/subjects.py`'s existing `StreamingResponse` convention). Columns: `student_id, first_name, father_name, last_name, gender, status, branch, academic_year, grade, section, section_display`. Tenant-isolated; a Branch-restricted actor only sees Students whose CURRENT effective Academic Placement Branch is in their accessible-Branch set (mirrors the existing granular Branch-gating already applied to placement data in `routers/students_ui.py`). Student ID is written as a canonical `STD`+10-digit TEXT cell (`number_format="@"`, Python `str`) so Excel never strips leading zeros or applies scientific

notation; a legacy Student with no managed Student number gets a blank cell, never a fabricated one. `section_display` reuses `academic_grade.format_section_display` unchanged (ADR 0045): additive only, activates exclusively on the exact Al-Andalus `workspace_uuid`, and the canonical `section` column is never replaced by it.

1 `POST /api/students/roster/import/preview` (`students.import`) - stateless,

zero-DB-mutation: parses the uploaded `.xlsx` (`load_workbook(..., data_only=True, keep_links=False)`, so a formula cell is read as its cached value and never evaluated, and external links are dropped), validates workbook structure/headers/every row, normalizes values, resolves Branch/ Academic Year/Grade/`PlanningSection` by canonical identity only (never the ADR 0045 `section_display` label, even for the Al-Andalus workspace), and runs the same privacy-safe TIS Student ID conflict check as the direct create API. Returns a bounded per-row result: `{row, status, data|errors}` with `errors` as `{row, field, error_code, safe_message}` - never a raw exception, SQL error, or sensitive tenant detail.

1 `POST /api/students/roster/import/apply` (`students.import`) - never trusts

a client-submitted preview payload: it independently re-parses and revalidates the freshly uploaded workbook using the identical validation core preview uses, then applies atomically inside the existing DB transaction pattern. Any row failure - including a `IntegrityError` race discovered only at apply time (classified as `student_id_conflict`, never the raw database error text) - rolls back the entire apply; there is no partial Student/Placement write.

File safety: `.xlsx` extension only; 5 MB upload-size and 2000-data-row bounds (this feature's own explicit judgment call - no existing repository-wide upload convention was found); malformed/empty/oversized workbooks are rejected with a bounded file-level error (`{row: null, field: null, error_code, safe_message}`), never a stack trace.

Permission status: `students.import`/`students.export` are reclassified from dormant/reserved (`D`) to active-enforced (`A`) in `tests/test_permission_registry_matrix.py`, matching the `subjects.import`/`subjects.export` precedent (discovered consumer: `routers/students.py`, via the same `_authorize` guard-call pattern already used by every other `students.*` key). Dormant count 15?13, active count 143?145; total registered keys unchanged at 177. No new role grant, no `DEVELOPER_ONLY`/`OWNER_ONLY`/`ADMINISTRATOR_ONLY`/`LIMITED_READ_ONLY` membership change - the governance decision recorded below is unchanged, only the enforcement status now reflects that real routes exist.

Tests: `tests/test_student_roster_import_export.py` (40 focused tests covering export tenant/Branch isolation, deterministic columns, Student-ID text-cell/leading-zero preservation, legacy-no-ID handling; preview statelessness, file/structural/row-level error classification, same-tenant and cross-tenant privacy-safe conflict disclosure, Branch/Grade/Section/ Academic-Year reference validation, formula-cell and oversized-workbook safety; apply atomicity, independent revalidation, race-conflict rollback, permission/scope re-checking, and historical-placement non-corruption; and M5/ADR 0045 regression proving `section_display` is never import matching identity and stays additive-only on export). `tests/test_permission_registry_ matrix.py`'s classification update keeps all of its own tests passing.

Student Roster Import/Export - Permission Registration (Governance Prerequisite, M6) (2026-09-22)

The Owner reviewed and approved registering exactly two new semantic permission keys in the canonical `permission_registry.py` `students` group: `students.import` ("Import student roster data") and `students.export` ("Export student roster data"). Both follow the exact existing `students.*` naming/grouping/description/ordering convention and are classified dormant/reserved (status `D` in `tests/test_permission_registry_matrix.py`, matching the established `teachers.import`/`teachers.export` precedent documented under "22-Key Permission Registry Closure" below): no roster import/export route, service, or frontend consumes them yet, so no template/nav/guard is wired to either key.

Neither key is added to `DEVELOPER_ONLY_PERMISSION_KEYS`, `OWNER_ONLY_PERMISSION_KEYS`, `ADMINISTRATOR_ONLY_PERMISSION_KEYS`, `LIMITED_READ_ONLY_PERMISSION_KEYS`, or `_EDITOR_LIKE_PERMISSIONS` - no new default-role grant was introduced by this task; each key is assignable to Administrator/Editor/User (the same class as the rest of the `students` group) and, like every other `students.*` key, is included in the Administrator role's default set only because `DEFAULT_ROLE_PERMISSIONS` already computes Administrator's defaults as "all registered keys except platform-only" - a pre-existing structural formula unchanged by this task, not a new grant decision.

`students.import` permits Student roster import operations only within the caller's already-authorized tenant/Branch/Student-management scope; it does not expand tenant scope, bypass existing Student create/update authorization, TIS Student ID rules, or Academic Placement validation, grant cross-tenant access, or override backend invariant enforcement. `students.export` permits Student roster export only for Students the caller is otherwise authorized to access within the existing tenant/Branch/Student scope; it does not expand Student visibility, grant cross-tenant access, bypass existing scope restrictions, or authorize unrelated Student operations. Permission and scope remain separate checks, per the existing pattern used throughout this codebase.

This task registers only the two permission keys and this KMS record. It implements no roster import/export service logic, API route, or frontend, and changes no Student data, schema, or migration. The .xlsx-only, stateless-preview, atomic-apply M6 roster import/export backend referenced below (and in "Students + Talent & Potential M1" and "Student & Academic Placement Foundation And Talent Program & Framework Foundation Implemented") remains subsequent, separate, not-yet-implemented work; the prior "deferred to later milestones" wording in those entries described the pre-registration state and is superseded only insofar as the permission governance prerequisite is now approved - roster import/export functionality itself is still not implemented.

Al-Andalus Section Display - Governance Authorization (2026-09-22)

Per new ADR 0045, the Owner has authorized the previously-deferred Al-Andalus Section-display presentation convention for the exact verified production `SchoolGroup.workspace_uuid` `72e52eb2-3844-447b-92a8-c55015f73257` (identified using the repository's existing read-only production audit, `scripts/audit_al_andalus_readonly.py`, run by the Owner against the deployed Render PostgreSQL environment). Runtime implementation keys exclusively off the exact `workspace_uuid` above (never `SchoolGroup.name`/domain/email/Branch label), remains presentation-only, and leaves canonical `PlanningSection`/Student Academic Placement/frozen historical Talent Grade-Section identity, analytics grouping, and query/filter/import-export identity completely unchanged - see ADR 0045 for the full authorized convention and fallback rule.

Al-Andalus Section Display M5 - Shared Presentation Implementation (2026-09-22)

Implements the ADR 0045-authorized presentation convention. One shared, authoritative helper, `academic_grade.format_section_display(workspace_uuid, grade_level, section_name) -> str`, activates the numeric `{grade_number}.{section_ordinal}` mapping only on exact `SchoolGroup.workspace_uuid` equality to the authorized UUID above; every other workspace, and any Grade/Section that does not deterministically map (custom Section name, non-alphabetic/multi-character Section, missing/malformed Grade, missing Section), returns the existing canonical `section_name` unchanged. The helper never mutates or replaces canonical `grade_level`/`section_name` - it is wired in as a bounded, additive `section_display` projection alongside the unchanged canonical fields.

Wired surfaces (M5 boundary): Students UI current placement and placement history (`routers/students_ui.py::_placement_view`, `templates/students.html`, `templates/student_profile.html`), the Academic Placement Section selector (`GET /students/sections`, `static/js/students.js`), the Students list Grade/Section filter dropdown, the Talent frozen historical Learner Profile context (`talent_learner_profile_service.py::build_learner_profile`'s `frozen_context`, used

by both `templates/student_profile.html`'s Talent tab and the Talent workspace's own `learner-profile` view in `static/js/talent.js`) - using the frozen `TalentAssessmentCyclePopulationMember` Grade/Section, never the Student's current placement - the Talent Student Assessment eligible-students roster (`GET /api/talent/assessment-cycles/{id}/eligible-students`, `static/js/talent-operations.js`), and the Talent Results/Analytics Grade/Section filter (`GET /api/talent/programs/planning-sections`, `static/js/talent.js`). Organization-analytics Grade/Section grouping identity (e.g. `organization-analytics` grade-map column totals) is unchanged and out of scope - ADR 0045 requires analytics grouping identity to remain canonical.

Focused tests: `tests/test_academic_grade_section_display.py` (pure formatter edge cases, exact UUID activation, A-Z ordinal spot-checks, fallback cases) and `tests/test_al_andalus_section_display.py` (cross-surface wiring: exact-UUID activation, a different-UUID row sharing the exact organization name, workspace rename stability, other-tenant isolation, canonical-value-unchanged/no-mutation checks, and frozen Talent context proven to diverge from a later current-placement change). No schema, migration, or new permission. No application behavior changed for any workspace other than the one exact authorized `workspace_uuid`.

Talent & Potential M4 - Authoritative Evaluation Progress Analytics (2026-09-22)

Implements the complete backend Evaluation Progress contract per ADR 0044, building on M8 (Evaluation Plans/Periods), M9 (Deterministic Talent Analytics privacy primitives), ADR 0037 (Overall Program Result), ADR 0031 / ADR 0042 (Learning Style). New files: `talent_evaluation_progress_service.py` (core logic) and `routers/talent_evaluation_progress.py` (four routes under `/api/talent/evaluation-progress`), registered in `main.py` alongside the other Talent routers. No schema, migration, or new permission.

Active/opened predicate (verified, not invented): `TalentPlannedEvaluationPeriod.status == 'planned'` AND its linked `TalentAssessmentCycle.status IN ('open', 'closed')` (`resolve_active_periods`). Nominal weight `1/A` for A active Periods is derived only, never persisted. Configured label/short_code never affect order; only `sequence` does.

Student Progress (`GET .../students/{student_id}`, permission `talent_learner_profiles.view`): per-active-Period result state (`available`/`pending`/`incomplete`/`insufficient_evidence`/`unassessed`/ `not_applicable`), `normalized_percent` only when `available`, `Current Overall Result` = mean of only the available results (Pending/unavailable excluded, never zero), plus authoritative `active_period_count`/`available_result_count`/ `pending_or_unavailable_count` counts. Owner-ratified Decision 2: this route has NO privacy-policy dependency at all - authorization is frozen- historical-Branch scope (mirroring `talent_learner_profile_service`) exactly, never aggregate cohort-size suppression. An out-of-scope Student is a non-enumerating 404.

Branch/Organization Progress (`GET .../branches/{branch_id}` and `.../organization`, permission `talent_analytics.view`): `BranchPeriodResult`/`OrganizationPeriodResult` are the direct mean of valid governed Student `normalized_percent` results via frozen `TalentAssessmentCyclePopulationMember.branch_id` attribution; `OrganizationPeriodResult` is computed directly from every Student in the authorized scope, never as `average(BranchPeriodResult)` (proven by a dedicated unequal-Branch-size test: Branch A mean 85.0, Branch B mean 60.0, but the direct Organization mean is 76.67, not the naive 72.5 average-of-averages). Every result is gated by one `Cell`/`Group` (`_privacy_safe_branch_mean_group`): a contributing-count `Cell` per Branch plus an org-total `Cell`, `apply_primary_privacy` then `run_complementary_suppression`; a derived mean is serialized only when its own count `Cell` is `visible` (never for `suppressed`/`restricted`/ `coarsened`/`no_data`). Overall Results across active Periods use only the `visible` per-Period means (Pending/suppressed excluded, never zero, never reconstructable from the combined value).

Framework comparability (owner-ratified Decision 1): every Student/Branch/Organization progress payload carries `comparability_state`/`comparability_reason_code`; when active Periods span more than one

framework_version_id, every Period is still returned individually but the combined Overall Result is null ({"state": "no_data", "value": None} for Branch/Organization) with reason_code="framework_changed".

Branch comparison metrics (GET .../branch-comparison?metric=...): talent_evaluation_progress_service.branch_comparison_metric is a bounded local dispatcher for exactly the seven approved metrics - evaluation_period_result, current_overall_progress (both new, above), assessment_completion, assessments_started (reusing svc.raw_coverage_by_dimension unchanged), meets_program_criteria (reusing svc.raw_candidate_by_dimension, requires talent_review_candidates.view or 403 - query-skipped, not merely filtered), officially_confirmed (reusing svc.raw_identification_by_dimension, requires talent_official_identifications.view or 403), and learning_style (new). This dispatcher does not add to, or modify, the frozen 14-value M10 MetricCode enum in talent_org_intelligence_contract.py.

Learning Style Branch aggregate (ADR 0031/ADR 0042 privacy contract): arithmetic mean of valid (non-null) learning_style_{verbal,non_verbal, quantitative,spatial}_percentage values in the Program's authorized population cohort; null excluded, 0 is a real included value, no sum rule, no dominant-style derivation, protected by the identical Cell/ Group mechanism. A Student frozen into more than one Cycle within the same Program+Year is de-duplicated to one contribution per Branch context (DISTINCT on branch_id, student_id, value) since Learning Style is a per-Student attribute, not a per-membership fact - unlike the reused M9 frozen_membership-grain metrics, which intentionally count once per Cycle a Student was frozen into.

Known, disclosed, non-blocking characteristic (inherited from reused M9 code, not a new M4 defect): assessment_completion/ assessments_started/meets_program_criteria/officially_confirmed reuse svc.build_breakdown_group exactly as /breakdowns/branch already does, which does not convert an all-zero cohort's total_raw=0 to None

1 a policy may therefore label a genuinely empty cohort suppressed

rather than no_data for those four metrics only. No raw value is ever leaked either way (a suppressed cell's value is always None); the two NEW aggregates this milestone adds (Branch/Organization Period Result and Learning Style) explicitly avoid this by converting a falsy total to None before privacy evaluation.

Second-pass adversarial review (required by this milestone, given its privacy sensitivity) traced all 16 specified risk categories - suppressed- Period reconstruction through Overall Result, complementary-suppression failures, sparse-matrix/cell omission, privacy call ordering, hidden raw values, numerator/denominator leakage, average-of-Branch-averages, current-placement vs. frozen attribution, cross-tenant contamination, Framework-version mixing, Pending-as-zero, null-Learning-Style-as-zero, privacy-provider fail-open, unauthorized single-Student access, M10 registry modification, and client arithmetic becoming authoritative - and found the implementation clean; no remediation was required beyond the Learning Style de-duplication fix made during initial test-writing (fixed before any review pass, not a post-review defect). See ADR 0044 for the complete governance record and tests/test_talent_evaluation_progress.py (25 tests: active-period predicate, pure worked-example arithmetic, Student progress including authorization/tenant isolation, Branch/ Organization Period and Overall Result including the unequal-Branch-size proof, cross-framework comparability blocking combined Overall while preserving individual Periods, missing-privacy-provider fail-closed, primary and complementary suppression, Learning Style null/zero handling and suppression, and all seven Branch comparison metrics including permission query-skip) for the full test matrix. Regression: the existing M9/M10 Talent analytics suites, M2 Student number, and M3 Learning Style CRUD suites all pass unchanged (three pre-existing, unrelated failures - one query-count assertion in tests/test_talent_organization_student_drill.py and two in tests/test_student_learning_style_v1.py/ tests/test_talent_rubric_kpi_candidate_policy.py - were directly reconfirmed present and unchanged on unmodified dev HEAD 96dd454 before this task).

Promo Grant Replacement - "Replace / Extend Promotional Access" (2026-09-21)

Per ADR 0041, a new Platform Console-only workflow implements the promo-to-promo transfer capability ADR 0020 previously deferred ("Promo renewal, transfer, ... remain deferred"). An operator can now atomically replace an organization's existing active `PromoGrant` with a fresh grant redeemed against a different, currently valid `PromoCode` - for example when Al-Andalus's original 4-branch promo (from a now-revoked `PromoCode`) is replaced by a new 25 branch / 100 system user / 500 teacher promo. This is a distinct transition from ADR 0024's promo-to-paid conversion; the two do not share a code path, though both follow the same "end old evidence, create new evidence, repoint the tenant link, one transaction" shape (`_apply_confirmed_promo_conversion` for promo-to-paid, `saas.promo_redemption_service.replace_promo_grant` for promo-to-promo).

Two new functions in `saas/promo_redemption_service.py`, `preview_grant_replacement` and `replace_promo_grant`, reuse the existing `_validate_promo_definition` gate (the same promo-definition validation `activate_promo` uses - active, approved, within its redemption window, not itself superseded, scope/redemption-limit rules satisfied) rather than a second, weaker validation path. `replace_promo_grant` locks the target `SchoolGroup`, the replacement `PromoCode`, the existing active `PromoGrant`/`TenantProvisioningLink`/`WorkspaceEntitlement`, and the organization owner's `SaaSAccountUserLink`, then inside one `db.begin_nested()` transaction: marks the old grant `status='superseded'` (retained, never deleted) and its `WorkspaceEntitlement` `status='ended'`; creates a new `PromoActivationSession` (`context_type='existing_organization'`, already `status='activated'`), `PromoRedemption`, and `PromoGrant` against the replacement promo, with `new_grant.supersedes_grant_id = old_grant.id` (the new row records which grant it replaced - matching the existing `PromoCode.supersedes_promo_code_id` convention, where `promo_code_service.replace_promo` sets the newly created replacement definition's `supersedes_promo_code_id` to the id of the definition it replaces); creates a new active `WorkspaceEntitlement` with entitlement values (plan features, `quota.active_branches`) derived from the new promo's plan via the existing `_create_entitlement_values` helper, so capacity/plan evidence is never left pointing at stale numbers; re-assigns the exact same `PromoGrantBranchAssignment` set the old grant had to the new grant; and repoints the single `TenantProvisioningLink.promo_grant_id` from the old grant to the new grant. A final in-transaction check re-runs `promo_grant_service.resolve_promo_grant` and `workspace_entitlement_service.resolve_workspace_entitlement` and raises (rolling back the whole operation) unless both resolve cleanly against the new grant - mirroring the post-validation check `activate_promo` already performs. No `Branch`, `User`/staff, or `Teacher` row is created, modified, or deleted by this operation; only the commercial capacity/plan evidence (`PromoGrant`, `WorkspaceEntitlement`, `TenantProvisioningLink`, `PromoGrantBranchAssignment`/`BranchEntitlement`) changes. The existing `uq_promo_grants_active_group` / `uq_workspace_entitlements_active_group` partial unique indexes continue to guarantee exactly one active grant and one active entitlement per organization throughout - no schema or migration change was required or made.

Authorization reuses the same platform-only `promo_codes.manage` decision already enforced on every other Platform Console promo-code admin route (`saas.router._require_promo_permission`), re-checked defensively at the service layer (matching the existing `promo_code_service.revoke_promo` service-level check precedent); a non-platform actor or an unauthenticated request (`actor=None`) is denied before any row is read for update. Two new `saas/router.py` admin routes back the workflow: `GET /saas-admin/promo-codes/{promo_uuid}/replace-grant?school_group_id=...` renders a server-rendered confirmation page (`templates/saas/admin_promo_grant_replace_confirm.html`, styled consistently with the existing `admin_promo_code_detail.html`) showing the old grant's limits against the replacement promo's limits (branches/system users/teachers, plan, promo reference) before the operator confirms; `POST /saas-admin/promo-codes/{promo_uuid}/replace-grant` executes the replacement and redirects with a notice. The existing promo code detail page gained a new "Replace / Extend Promotional Access" section (visible only to `promo_codes.manage` holders, only for an effective-active promo) listing organizations that currently have an active promotional grant, explicitly separated from the pre-existing "Definition Actions" section whose own copy already

states those actions "do not change tenant access or commercial authority" - this new section does.

Tests: `tests/test_promo_redemption.py::PromoGrantReplacementTests` (service layer - end-to-end replacement, exact-record branch/user/teacher preservation, old grant becomes `superseded`, new grant is `active`, only one active grant per organization, capacity display reflects the new grant's limits via `commercial_authority_service.resolve_commercial_authority`, denial for a non-platform/unauthenticated actor, denial for a revoked/invalid replacement promo leaving all existing rows untouched, and rollback on a forced mid-transaction failure) and `tests/test_promo_code_management.py::PromoGrantReplacementRouteTests` (route-level - the confirmation page renders old-vs-new limits, the execute route applies the replacement, and a non-platform actor is denied with 403). ADR 0020's "promo renewal, transfer... remain deferred" consequence is corrected: promo-to-promo transfer is no longer deferred; promo renewal and automated-expiry-job transfer remain deferred, unchanged. No `tis.db` change; this task did not touch any pre-existing promo/commercial code path beyond the two new service functions and two new routes/templates.

Talent & Potential Sidebar Icon (UI Polish, 2026-09-19)

The Talent & Potential sidebar entry now renders the shared inline-SVG `sparkles` icon through the existing `icon()` macro (`templates/_app_icons.html`: 24x24 viewBox, stroke-only, styled by `.shell-icon`), consistent with every other module. The nav item already declared `"icon": "sparkles"` (also used by the Talent page header); only the `"brand_logo": "img/talent-ghars-symbol.png"` override in `ui_shell.py` was removed, so the sidebar no longer draws an `<img>` for this item. Routing, permissions (`permission_keys` / `permission_mode: any`), label, child tree and all other icons are unchanged; no new icon, asset, dependency, or schema. The generic `brand_logo` support in `templates/base.html` and the `.sidebar-brand-symbol` CSS remain available but are unused, and `static/img/talent-ghars-symbol.png` stays in the repository unreferenced by the sidebar. Earlier entries below that describe the Ghars PNG in the sidebar are historical. `tests/talent_results_experience.test.cjs` now asserts the Talent item uses the shared icon macro instead of the PNG.

Whole-Application Permission Qualification (Phase 3)

Phase 3 qualified the whole application against the single canonical effective permission result (`auth.get_allowed_permission_keys`; ADR 0040 precedence unchanged, no second resolver, no schema/migration, no new key). Final registry classification of the 175 keys (26 groups): 143 active-enforced, 14 platform-only enforced, 5 alias/composite, 13 dormant/reserved, 0 unresolved. The prior note that `planning.import`/`planning.export` had consumers was stale, and `observations.submit` and `dashboard.view_all_schools` have no enforcement, so all three are now dormant (13 dormant total). The route table (468 routes, 253 non-GET) is machine-enumerated: 136 middleware rules, 182 in-handler guards, 10 helper guards, 34 platform-identity, 65 SaaS-account-session and 41 reviewed allowlist entries, none uncovered. One real defect was fixed: `POST /scope/organization` accepted any platform user without the `system_owner.switch_all_schools` / `schools.manage_all_schools` capability its own denial cited; it now also requires the canonical `_can_manage_all_school_scopes` helper. Seven unused duplicate raw `_get_role_permission_rows` / `_get_allowed_permission_keys` helpers were deleted. Sidebar, Dashboard tab/panel and direct-route consistency is parametrized over every nav module (including the multi-key Talent and System Configuration gates and the Talent sub-nav); Allow/Deny/Allow freshness for Role Package, School Override and User Exception, role change with retained exceptions, tenant isolation and platform-only protection are covered on fresh sessions. Four Owner/product decisions remain open and unresolved (teacher-create field-level enforcement, tenant-toggleable dormant keys, whether role User may author observations, orphan `UserPermissionOverride` rows on workspace deletion); ADR 0040 remains authoritative for precedence and its historical helper names are addressed in `docs/engineering/PERMISSION_CLOSURE_REVIEW.md`. Tests: `test_permission_registry_matrix.py`, `test_permission_route_coverage.py`, `test_permission_dangerous_patterns.py`, `test_permission_surface_consistency.py`, `test_permission_surfaces_accessibility.py`.

Role Permissions UI - User Exceptions (Phase 2)

Phase 2 replaces the old edit-user "Per-User Permission Overrides" panel (three-way Inherit / Allow / Deny radios plus one bulk save) with a "User Permission Exceptions" section on the same edit-user page. Presentation only: storage, the three-state `user_permission_service` semantics, and the ADR 0040 precedence (built-in role package -> global `RolePermission` -> SchoolGroup `RolePermission` -> per-user override -> effective; user Deny wins; a user Allow can never resurrect a role-level Deny) are unchanged. No schema, migration, new permission key, or second resolver.

1 **Binary UI, hidden inherit.** The normal UI offers only Allow and Deny.

"Inherit" no longer appears anywhere. An absent override row still means "follow role settings", and `apply_user_override(..., "inherit")` remains the deletion primitive, now surfaced as the **Reset to Role Settings** button, shown only when a direct exception exists. Normal UX is Allow / Deny plus Reset to Role Settings; "inherit" remains only the internal, backward-compatible API representation of clearing a per-user override (the legacy paired `permission_keys` / `permission_decisions` route contract is unchanged and is never exposed in the UI).

1 **User summary.** The section opens with name, login, effective role,

School, Branch, and status, built server-side in `routers/users.py::_build_user_permission_context`. An inactive account shows a notice; effective results come from `auth.get_allowed_permission_keys`, which fails closed for inactive users and mismatched scope.

1 **Per-permission row.** Effective Allow/Deny, Source (User Exception /

School Override / Standard Role Package), and User Exception (Allow / Deny / None - descriptive only). Rows sit in native `<details>` groups (no manual `aria-expanded`); groups with an exception open by default with an "N exceptions" count. The button matching a stored exception carries server-rendered `aria-pressed="true"` and a check-mark cue. Platform-only keys render locked with no buttons. A key that is not assignable to the user's current role renders locked with no Allow or Deny button; if a stored exception already exists on it (for example after a role change made the key non-assignable) the row shows only **Reset to Role Settings**, because Reset deletes an existing exception and cannot grant authority. Stale exceptions are never deleted silently, and creating or changing an exception on a locked key remains refused server-side.

1 **Inert Allow.** A stored user Allow while the role layer denies shows

Effective Deny, "Allow (stored, currently ineffective)", and an explanation that it cannot override the role denial and applies again if the role re-grants. It is never displayed as final Allow and never silently deleted; Reset is available.

1 **Source projection.** `user_permission_service.build_user_permission_payload`

(read-only, composes `role_permission_service` and `auth` helpers) gained additive fields `standard_allowed`, `school_override`, `exception`, `exception_inert`, `has_exception`, `source`, `blocked_by_account`; `effective` equals `auth.get_allowed_permission_keys` (tested for `subjects.view` and `teachers.view`).

1 **Route.** `POST /users/permissions/{user_pk}` stays the single route with

one guard block (tenant/platform scope resolution unchanged). New optional form field `change="<permission_key>|<allow|deny|reset>"` applies exactly one change (reset -> `inherit`); malformed values are rejected with no write. Without `change` the legacy paired `permission_keys` / `permission_decisions` contract is unchanged. Each action button targets `#perm-<key>` so the page returns to the edited row.

1 **Tests.** `tests/test_user_exceptions_ui.py` (cases for every source /

inert / reset / role-change / tenant / platform-only / inactive / freshness scenario plus template accessibility assertions). One obsolete assertion set in `tests/test_edit_user_template_render.py` that encoded the removed Inherit

radios was rewritten for the new markup.

Role Permissions UI - Role Packages / School Overrides Split (Phase 1)

A live Owner reproduction proved a real incident on the prior combined Role Permissions editor: for a real tenant, Administrator -> subjects.view appeared Allow while "Global defaults" was selected in the single "Permission Scope: Global defaults / Selected school" dropdown, while that same tenant's actual Administrator effective permission was Deny (built-in default = Allow, global `RolePermission` override = none, tenant `RolePermission` override = Deny, per-user override = Inherit, final effective = Deny per ADR 0040's precedence). The Platform Owner UI simultaneously showed the tenant's name even while "Global defaults" was selected, creating the visual impression the Platform Owner belonged to that tenant. Root cause was purely presentation/IA: one screen mixed the global default layer, the tenant override layer, and Platform Owner context behind one ambiguous dropdown, even though `role_permission_service.py`'s resolver (built-in -> global `RolePermission` -> tenant `RolePermission`, unchanged by this pass) was already correct.

`/system-configuration/role-permissions` (GET/POST, unchanged path) now serves two explicit modes selected by `?mode=packages` (default) and `?mode=overrides`, both still backed by the same `role_permission_service.py` resolver/writer - no second resolver was created. **Role Packages** (`mode=packages`) edits only the standard/global role layer: no SchoolGroup selector, no tenant name, and no Platform Owner tenant context appear anywhere on it; editing is restricted to actors holding `system_owner.manage_global_role_permissions` (Platform Owner today), and a tenant actor sees the same standard package read-only. **School Overrides** (`mode=overrides`) is the only place a SchoolGroup appears, always labeled as an explicit management target ("Managing School: <name> ... management target only, not your own account context"), never as if the Platform Owner belongs to that school; a Platform Owner must explicitly choose which SchoolGroup to manage (no school is ever pre-selected on their behalf: with none chosen the page shows a prompt and no editable form, and the choice is submitted with an explicit "Review this school's overrides" button rather than auto-submitting on change), while a tenant actor is silently locked to their own authorized SchoolGroup even if a foreign `school_group_id` is supplied in the query string or POST body (POST additionally hard-rejects a foreign `school_group_id` with a redirect, matching the pre-existing cross-tenant boundary). Per permission, School Overrides shows Standard Role Package value, this school's own override (or "Using Standard Role Package" when none exists), and the Effective value, via a new read-only `role_permission_service.build_school_override_payload`. A "Reset to standard" control is UI-only: it sets the checkbox back to the Standard Role Package value before Save, which the pre-existing `apply_role_permission_overrides` diff-against-baseline write path already turns into a clean row delete (no new backend removal semantics were invented). Viewing School Overrides never writes a tenant `RolePermission` row (no side effect on GET). So a real override is legible at a glance, School Overrides states how many School overrides the selected role has ("No School-specific override. Using Standard Role Package." when none), opens by default only the permission groups that contain an override, and tells the editor that "Reset to standard" still requires "Save School Overrides". School Overrides deliberately has no "Select All" / "Clear All" (Role Packages keeps them): a School override must stay a sparse, intentional difference from the Standard Role Package, because bulk select/clear would write a large explicit tenant policy and make later Standard Role Package changes ineffective for that school; per-permission "Reset to standard" is the only reset in Phase 1 (no bulk "Reset All"). Accessibility (template-source and real-browser verified, no automation framework exists in this repo): the Role Packages / School Overrides links are page-navigation links marked `aria-current="page"` (not `role="tab"`, which would promise unimplemented arrow-key tab behavior); permission groups rely on native `<details>`/`<summary>` semantics with no manual `aria-expanded` (which would drift from the real open/closed state); each School Override row keeps its checkbox label separate from the "Reset to standard" button (a button nested in a `<label>` is invalid and pollutes the checkbox's accessible name); banner text is a single element so it wraps as one sentence; and links, buttons, summaries, the school selector and checkboxes have an explicit `:focus-visible` outline.

Platform Owner context is unchanged and re-verified: `School`, `Branch`, and `Academic Year` remain unassigned/`None` for a Platform Owner (`auth.get_user_school_group_id` already returns `None` unconditionally

for any platform user), and selecting a SchoolGroup as a School Overrides management target does not write to the Platform Owner's own user row, session, or `scope_school_group_id`. This is Phase 1 structural correction only - User Exceptions (per-user overrides) redesign, the full cross-module permission-consumer audit, and the accessibility qualification matrix are explicitly deferred to later phases. No schema, migration, permission key, or precedence-rule change; `tis.db` is unmodified (hash verified unchanged). See `tests/test_role_permissions_ui_split.py` and the updated `tests/test_permission_management.py` (its `_update` helper's `scope_type` parameter is renamed `mode`, matching the route's new form field) for the complete regression matrix, including a constructed Al-Andalus-style fixture proving the exact incident scenario, tenant-isolation, and the Platform Owner identity-preservation proof.

Dashboard Tab/Panel Permission-Gate Closure (Live Owner Reproduction)

A live Owner reproduction found a real tenant Al-Andalus Administrator whose Subjects sidebar item was correctly hidden and whose direct `/subjects` route was correctly denied, while the same current user's Dashboard still showed the Subjects tab and Subjects Workspace panel. Both the sidebar (`ui_shell.build_shell_context/_build_nav_items`) and the route guard (`authorization.enforce_route_permission`) already resolved through the canonical `auth.get_allowed_permission_keys` chain (built-in role default -> global `RolePermission` -> tenant `RolePermission` -> per-user `UserPermissionOverride`) and were already correct and consistent with each other. The defect was isolated to `templates/dashboard.html`: its Subjects, Teachers, and Planning tab buttons and panels rendered unconditionally, never calling the `can(...)` helper that `main.dashboard`'s own `build_shell_context()` call already makes available in the same template (only the pre-existing Reports tab called `can("dashboard.view_reports")`).

The fix wraps each of the three tab buttons and their matching panel `<div>`s in `can("subjects.view")`/`can("teachers.view")`/`can("planning.view")`, and replaces the previously hardcoded "Subjects is always the active default tab" markup with a computed `dash_first_tab` that becomes the first permission-visible tab (Subjects, then Teachers, then Planning, then Reports), so a user who only holds `teachers.view` (used here as the second, non-Subjects sentinel proving this is a generic gating fix, not a Subjects-only special case) still gets a valid default-active panel instead of a blank workspace. No precedence rule, permission key, schema, or migration changed; `auth.py`, `authorization.py`, `role_permission_service.py`, `user_permission_service.py`, and `ui_shell.py` were inspected and confirmed already canonical and were not modified.

Phase-1 read-only diagnosis of the tracked local `tis.db` found it predates this feature area entirely: it has no `user_permission_overrides` table and its `schema_migrations` ledger ends at `20260822_...`, well before `20260913_001_user_permission_overrides` and `20260915_001_role_permission_logical_key_uniqueness`; it also has no tenant Administrator user matching the reproduced Al-Andalus scenario (only a single platform `developer` identity is present). Diagnosis and regression coverage are therefore code/template-level, not live-row inspection of that file; this is recorded rather than worked around. `tis.db` was not modified (hash verified unchanged before and after this task) and was not migrated.

New regression coverage in `tests/test_dashboard_sidebar_permission_consistency.py` proves, for the same current user across the same request, that sidebar visibility, Dashboard tab/panel presence, and the direct route guard all agree with the canonical resolver's effective permission for six cases (global Allow only; global Allow + tenant Deny; tenant Allow + per-user Deny; role Deny then re-grant with per-user Inherit restoring access; role Deny then re-grant with a persisted per-user Deny remaining in effect per ADR 0040's Deny-over-Allow rule; and tenant-isolation, where denying one SchoolGroup's Administrator does not affect another SchoolGroup's Administrator holding the same role), each parametrized over both `subjects.view` and `teachers.view`.

PR #360 Review-Response Corrective Pass

Two review findings on PR #360 were verified against the actual GitHub review comments/diff and corrected. `hiring_plan.export` previously gated only the literal `section=hiring` export request; `section=full` returns a report containing the same hiring-plan data (both professional-report builders emit the hiring-plan sheet/section for `full`) and was not gated, letting a caller retain the protected data despite a denied `hiring_plan.export`. The gate in `main._authorize_report_export` now covers every section whose payload includes hiring data. Separately, this branch's `.kms-impact.yml` had come to understate its own database impact: it claimed no migrations/schema changes even though the branch still carries the non-destructive migration `20260915_001_role_permission_logical_key_uniqueness` (see below); the declaration is corrected to describe the branch's full cumulative scope. A third item from the delegated task summary (SchoolGroup edit/delete/action authorization) was investigated and found to have no corresponding PR #360 review comment and no SchoolGroup file in this PR's diff against `origin/dev`; no code change was made for it. Full detail in [the closure review](engineering/PERMISSION_CLOSURE_REVIEW.md#pr-360-review-response-corrective-pass).

22-Key Permission Registry Closure

All 22 permission keys left unclassified by the independent review below have been individually traced against the implemented product and resolved: nine now have dedicated server-side enforcement aligned with their UI (`dashboard.view_branch_summary`, `dashboard.view_reports`, `dashboard.export_reports`, `hiring_plan.export`, `observations.view_reports`, `observations.sign_evaluator`, `configuration.view_audit_log`), three are explicit, documented aliases of an already-enforced capability rather than a second independently-built gate (`observations.submit`, `system_owner.manage_developer_accounts`, `system_owner.view_cross_school_audit` / `system_owner.export_cross_school_data`), nine are classified dormant/reserved because no implementation exists and no UI implies otherwise (`reports.view`, `teachers.import`, `teachers.export`, `subjects.manage_colors`, `observations.manage_templates`, `configuration.manage_global_defaults`, `system_owner.manage_subscriptions`, `system_owner.create_subscription_school`, `system_owner.run_startup_repairs`), and one genuine registry/implementation conflict was resolved fail-closed (`configuration.view_audit_log` moved to platform-only rather than exposing the global, non-tenant-filtered audit log to tenant administrators). Full per-key reasoning is in [the closure review](engineering/PERMISSION_CLOSURE_REVIEW.md). This 22-key closure pass itself introduced no new permission keys, migrations, or schema changes; ADR 0040 is unchanged. (The branch/PR as a whole still carries the earlier `20260915_001_role_permission_logical_key_uniqueness` migration described below and in the PR #360 corrective-pass note above.)

Independent Permission Closure Review - Not Yet Whole-System Approval

Independent review of the permission-closure branch found additional bypasses: notification detail auto-marked messages read despite a dedicated Deny; inactive platform identities retained helper capabilities; platform Developer role choices bypassed `users.assign_role`; teacher bulk deletion used the single-delete key; and teacher/planning edit forms could mutate protected fields under broader edit grants. Corrective guards now use canonical effective permissions, reject unauthorized field changes, preserve omitted read-only values, and project the dedicated capabilities to their forms. The canonical tenant resolver also fails closed for absent/contradictory ownership or foreign selected/explicit permission evaluation scope.

The ten-family ON/OFF/ON evidence covers fresh persisted resolver reads, route-guard calls, and navigation projections, not complete authenticated HTTP round trips. The role-policy migration was independently exercised on disposable PostgreSQL as well as SQLite. Whole-system approval remains blocked pending classification/enforcement of unconsumed registry keys and the coverage gaps recorded in [the independent review](engineering/PERMISSION_CLOSURE_REVIEW.md). No permission data cleanup, merge, or deployment is authorized by this review.

Permission Consistency Closure (Whole-Registry Pass)

A reported `subjects.view` grant/revoke/re-grant regression was investigated and fixed as a whole permission-system defect rather than a Subjects-only issue. Root cause: `auth.get_allowed_permission_keys` (the canonical current-user effective-permission resolver) and several helper functions (`can_manage_system_settings`, `can_manage_users`, `can_modify_data`, `can_edit_data`, `can_delete_data`, `can_edit_user_accounts`, `can_delete_user_accounts`, `can_manage_target_user_account`) used request-independent caching (`user._permission_cache` and the now-removed `_has_cached_permission`/`_has_any_cached_permission`/ `_has_cached_permission_prefix`/`get_user_permission_keys` helpers), so a role-permission toggle could remain stale on a retained user instance. Independent review does not establish historical production HTTP instance reuse. The attached result cache is removed (the shell still uses a canonical per-render snapshot); every one of those helpers now takes an explicit `db: Session` and resolves fresh through `has_permission`/`has_any_permission`/`get_allowed_permission_keys` on every call, matching ADR 0040's precedence chain (built-in role default -> global `RolePermission` override -> tenant `RolePermission` override -> per-user `UserPermissionOverride` -> effective permission, Deny-over-Allow). `can_manage_target_user_account` additionally now fails closed on an ambiguous (falsy) SchoolGroup comparison instead of defaulting to allow.

Beyond the caching root cause, the full-registry audit found and fixed several independent stale/incorrect permission paths: `routers/users.py`'s role-creation and permission-management helpers; `routers/observations.py`'s create/edit/delete/unlock checks, plus a distinct sibling defect where holding only `observations.create_formal` or only `observations.create_non_formal` allowed creating *either* observation type (fixed with a new per-type `_can_create_observation_type` check reflected in `templates/observation_form.html`'s Type options); `main.py`'s System Configuration Qualifications create/update/delete routes, which were gated by entirely wrong permission keys (`branches.delete`, `academic_years.activate`, `academic_years.create`) instead of `configuration.manage_degrees`/`configuration.manage_specializations`; `update_branch`/`delete_branch` accepting a field change not covered by the caller's specific `branches.edit`/`branches.activate_deactivate` grant, plus missing SchoolGroup-scope boundary checks on `delete_branch`, `set_current_year`, and `open_new_academic_year` (the last of which now correctly requires both `academic_years.create` AND `academic_years.activate` rather than the coarse role-prefix `can_manage_system_settings` check); and notification mark-read/resolve/archive plus Branch/SchoolGroup logo upload/reset routes, which previously enforced no dedicated permission at all server-side beyond generic access, now gated by `notifications.mark_read`/`notifications.resolve`/ `notifications.archive` and `branding.manage_school_logos`/ `branding.manage_branch_logos` respectively, matching (and, for notifications, adding) the UI-side gates.

Persisted-data integrity: `role_permissions` gained two partial unique indexes (`uq_role_permissions_global_role_key` for a NULL SchoolGroup scope, `uq_role_permissions_tenant_role_key` for a tenant scope) via non-destructive migration `20260915_001_role_permission_logical_key_uniqueness`, which fails closed with no schema change if a pre-existing duplicate scope/role/key row is found (no row is ever deleted or rewritten by this migration). `user_permission_service.apply_user_override` now re-validates that a permission key is actually assignable to the target user's own effective role before writing an Allow/Deny (closing a path that could create an override for an otherwise nonassignable key; platform-only keys were already rejected), and `build_user_permission_payload`'s effective-permission projection now calls the same canonical `auth.get_allowed_permission_keys` resolver instead of a locally re-implemented merge.

Regression coverage proves ON -> OFF -> ON for `dashboard.view`, `subjects.view`, `teachers.view`, `students.view`, `planning.view`, `timetable.view`, `calendar.view`, `observations.view`, `users.view`, and `configuration.view` against the resolver, direct route/API enforcement, and navigation visibility, each re-read fresh (no cross-request state), in `tests/test_permission_qualification.py`; plus new `tests/test_branch_year_permission_boundaries.py` (mixed-action Branch/ Academic-Year scope), `tests/test_configuration_scope_permissions.py` (logo/qualification scope-specific gating),

tests/test_notification_group_permissions.py, and tests/test_role_permission_logical_key_migration.py (migration duplicate-preflight and idempotency). No Allow-over-Deny behavior exists; that remains an explicitly deferred, separately-governed decision (ADR 0040). No new permission key, schema-breaking change, or tis.db change. See docs/CHANGE_HISTORY.md's 2026-09-17 entry for the complete file-level detail.

Permission Qualification Drift Closure

The cross-module audit additionally closed three UI/backend mismatches. Edit User now rechecks profile, position, role, branch/scope, active-status, and password changes against their exact granular permissions instead of treating any one as authority for all fields. Single-user and bulk-user delete controls now follow their respective backend keys. Talent now projects the dedicated Competency and Rubric-Level delete permissions to the browser, matching the existing API guards.

Per-User Permission Override - Platform-Actor Scope Fix And Governance Closure

The per-user permission override follow-up, now integrated into dev, closed the remaining platform-actor scope edge case in the already-implemented per-user permission override layer (UserPermissionOverride, user_permission_service.py, the Edit User page panel, and POST /users/permissions/{user_pk}): a platform-level actor (owner/developer) has no resolvable tenant school_group_id of their own, and routers/users.py's update_user_permissions previously either hard-errored or, when the platform actor's own session happened to have some unrelated selected branch/SchoolGroup scope, spuriously rejected an otherwise fully authorized cross-tenant write. The route now derives the override's school_group_id from the already server-loaded target user's own verified school_group_id whenever the acting admin is a platform user, mirroring _build_user_permission_context's existing identical read-side resolution; no client-supplied scope, new permission key, or new UI control was added. See ADR 0040 for the full decision record (this also formally documents the previously-recorded-but-not-yet-ADR'd precedence chain, Deny-over-Allow rule, and uniqueness justification).

Added regression coverage: three new route-level tests in tests/test_user_permission_override_routes.py (platform actor with no own tenant scope; scope correctly tracks each of two different targets in two different SchoolGroups rather than any shared/actor-controlled value; the service layer itself rejects a mismatched school_group_id even when called directly) and a new rendered-HTML integration test, tests/test_edit_user_template_render.py, proving the Edit User page's override panel actually renders all three Inherit/Allow/Deny controls, the inherited-grant and Deny-override reason-text branches, and the exact inert markup for a platform-only permission key. Full permission/security regression (tests/test_user_permission_overrides.py, tests/test_user_permission_override_routes.py, tests/test_permission_management.py, tests/test_permission_qualification.py, tests/test_platform_access.py, plus the two new files) passes. No schema migration, new permission key, or tis.db change.

Role Permission Management Reassessment And Repair

Role-permission management was reassessed end-to-end and repaired. The reported checkbox defect was not a persistence failure (grant/revoke/re-grant round-trips were confirmed correct at the route, service, and render layers) but a direct-vs-effective conflation: the editor bound a single checkbox to the merged effective permission (defaults + global + tenant) with no inheritance indication. role_permission_service.py is now the canonical resolver (both the Role Permissions UI and auth.py delegate to it); payloads expose direct/inherited/source; the editor labels inherited grants; and owner-only controls are locked out of the tenant editable model and Administrator defaults. The administrator Save path persists only intentional scope-level overrides (apply_role_permission_overrides), so a no-op Save no longer converts inherited/default/global grants into direct rows and default/global changes propagate to roles without an explicit override. Comprehensive regression tests were added

(`tests/test_permission_management.py`). No schema migration and no `tis.db` change.

Talent Student Assessments - Program Annual-Configuration Enablement Removed As A Gate

Per direct Owner instruction, a further same-day correction on top of the Grade-gate fix (below): `TalentProgramAcademicYearConfiguration.is_enabled` (and the configuration row's existence at all) is no longer required by the normal Student Assessments roster (`current_placements_for_assessment`) or Start Assessment (`_current_placement_for_assessment`) path. See `docs/adr/0039-talent-competency-kpi-level-simplification.md`'s "Further Owner correction" section. The Evaluation/Cycle already supplies the Academic Year; a Student's valid current Placement in that exact Academic Year is sufficient. Kept unchanged: School Group isolation, Branch authorization, Academic Year match, current Student Placement, the saved Competency+KPI+Level requirement, and immutable historical evidence protection. `derive_eligible_population` (legacy M4/frozen-population path) is unchanged and still requires an enabled configuration for its own callers. New regression proves: a Program with Competency+KPI+Level saved and no annual configuration row at all (and, separately, an explicitly disabled one) still shows the Student in the roster and allows Start Assessment. No schema migration required. Zero new test regressions (confirmed by direct before/after comparison); the same three pre-existing failures already present before this pass remain unaffected.

Talent Competency -> KPI -> Level Simplification - Owner Correction: Grade Removed From Configuration Gates Too

Per direct Owner instruction, a same-day corrective pass on top of the initial ADR 0039 implementation (below) found and fixed three remaining places Grade still acted as a gate rather than context, contradicting the Owner's governing rule ("Competency+KPI+Level+Save=Ready; Grade must not be a readiness gate, Student-list gate, or Start Assessment gate"). See `docs/adr/0039-talent-competency-kpi-level-simplification.md`'s "Owner correction" section for the exact prior/corrected behavior. Summary: `talent_student_assessment_service._current_placement_for_assessment` (renamed from `_current_eligible_placement`) no longer filters a Student's Placement by the Program's `eligible_grade_levels` - only tenant scope, the Program's Academic Year enablement, and a genuinely current effective Placement are required. The Student Assessments list (`GET /api/talent/assessment-cycles/{id}/eligible-students`) now calls a new `current_placements_for_assessment` (in `talent_assessment_cycle_service.py`) instead of `derive_eligible_population`, returning every current, validly placed Student in the authorized tenant/Branch/Academic-Year scope with no Grade filter; `derive_eligible_population` itself is unchanged and remains Grade-filtered for its own legacy/frozen-population callers (Cycle open/preview/ADR 0033 reconciliation), which are historical population mechanics distinct from the normal live Student Assessments roster. `static/js/talent-program-workspace.js`'s readiness (`setupComplete`) no longer includes `basicsComplete` (Program/Academic-Year eligible-Grade configuration) in its gate - Ready is exactly the assessment build itself (Competency+KPI+Level, saved); Grade configuration remains available for context but never blocks Ready. No schema migration was required. Zero new test regressions were introduced (confirmed by direct before/after comparison, matching this session's established practice); three pre-existing failures in `tests/test_talent_assessment_cycle_frozen_population.py` (already present on the pushed commit this corrects, unrelated to Grade gating) were confirmed present both before and after this pass and are not newly introduced.

Talent Competency -> KPI -> Level Simplification - Implemented

ADR 0039 (see below) is now implemented, not only recorded. Start Assessment no longer fails when the Student's specific Grade has no separately-authored Competency: `_newest_assessable_framework` prefers Grade-specific content but falls back to the Program's full saved build, and the Assessment editor's own client-side Competency filter

carries the identical fallback so an assessment the backend now allows never renders empty. Program readiness in the setup workspace is exactly Competency+KPI+Level, saved, stated as the single missing requirement in plain language when incomplete; Finish Setup/Framework activation remains available but is no longer required for or tied to the Ready banner. "Rubric"/"Assessment Criteria" is relabeled "KPI" in the Competency build/summary UI; the separate numeric `TalentKpiConfiguration` feature was left untouched, so a real two-things-called-"KPI" UI collision now exists and needs a follow-up Owner naming decision (not resolved by this pass). The KPI/rubric `name` field is now optional and auto-derived from the owning Competency when omitted. Delete Competency removes its KPI, Levels, and achievement Descriptors as one subtree action; it still blocks with an exact reason on numeric-KPI weighting or Program Criteria rules referencing the Competency, since those remain separate governed structures this pass does not authorize deleting as a side effect. The Talent & Potential sidebar Ghars logo no longer suppresses the visible module text and renders at the same size as sibling navigation icons; the existing SVG asset was confirmed to have no safely extractable icon-only symbol (one dense unlabeled wordmark path), so a true hand-extracted mark remains a follow-up design task. "Students" is now the first child of the Talent & Potential nav tree, gated by the existing `students.view` permission; the separate top-level Students item is unchanged. No schema migration was required. Zero test regressions were introduced, confirmed by direct before/after comparison across the full Talent Python and Node suites.

Talent Competency -> KPI -> Level Simplification - Governance Decision Recorded

Per direct Owner instruction on 2026-09-12, see `docs/adr/0039-talent-competency-kpi-level-simplification.md`. Two policy changes are now approved: (1) Grade is no longer a Start Assessment eligibility gate - a Student is assessable once the Program's saved build has at least one Competency with one KPI/criterion and one Level, regardless of whether that content was authored under the Student's specific Grade; Grade remains display/historical context only, per an amendment to ADR 0035 condition 4 (the rest of ADR 0035 is unchanged). (2) The user-facing authoring/assessment hierarchy is relabeled Competency -> KPI -> Level ("Assessment Criteria" retired as normal-user-facing text); internal schema/API names may remain unchanged. This explicitly does NOT resolve whether the separate, pre-existing numeric `TalentKpiConfiguration` feature needs its own rename to avoid a two-different-things-called-"KPI" collision

1 that is flagged as a distinct follow-up decision, not silently resolved.

Program readiness becomes Competency+KPI+Level+Save=Ready with no separate Finish-Setup/Activate step in the normal flow; Competency delete becomes a single subtree action (removes its KPI/Levels from the current/future build) while historical completed Assessment evidence remains immutable. This entry and the referenced ADR are the authorization record, not a completion record; implementation is tracked separately.

Owner Video Acceptance - Immediate Corrective Pass

The latest Owner video/document acceptance pass is implemented on `dev`. Configured Programs nested under an Evaluation Period are selectable directly from Student Assessments even before a physical Cycle exists. The authorized selection reuses the canonical Cycle-create + Period-link contracts and returns to the same Student Assessments surface with the selected Program roster. Start Assessment retains the selected Evaluation/Program context and opens the returned Assessment, whose UI reads that Assessment's exact Program Framework, Grade-applicable Competencies, competency-owned criteria, Levels, and descriptions. Action failures now render as visible alerts instead of an apparently inert click.

Delete Competency is available on immutable Program setup only as a safe versioned operation: TIS clones a new editable Framework Version, removes the Competency from that new version, and leaves all historical Assessment evidence on the old Framework untouched. The KPI disclosure now has a visible collapse affordance.

The Students list uses icon-only Add/Delete/Open actions with accessible names. A new `students.view_all_branches` permission controls the cross-Branch selector and **All branches** option;

managed-role constraint keeps it Administrator-only and it still requires organization/global scope. Without it, the Students list is fixed to the actor's assigned authorized Branch. Learning Style aggregate privacy is unchanged: a missing governed privacy provider now produces a clear "statistics unavailable" panel instead of hiding the whole section, with no counts/percentages/bars leaked; privacy-suppressed cohorts stay suppressed. **Superseded 2026-09-24 (Acceptance C):** the Learning Style distribution no longer depends on the privacy provider and is never suppressed; see the Acceptance C section at the top of this document.

Talent Urgent UX And Recovery Adjustment Batch

The Owner-approved urgent adjustment batch is implemented on `dev` with no schema migration. Talent's sidebar tree is now the only primary module navigation; the duplicate in-page module ribbon is removed. Program setup projects readiness on the Program summary and no longer owns a separate Ready step. A positive **Ready** state is shown only when eligible Grades, assessment criteria, and Evaluation Period configuration are complete. Competency authoring uses one selected Grade at a time, defaulting to the first configured Grade, while dedicated delete permissions continue to govern Competency/Level destructive actions.

A new Administrator-default permission `talent_assessments.reset_for_reassessment` provides safe operational recovery for a current completed Assessment. Reset marks the prior completed attempt non-current and writes an audit event while preserving every dependent evidence/history row. The Student therefore returns to Not started in the same visible Evaluation and may start a fresh current Assessment. This is not ADR 0034 hard-delete; completed evidence is never physically deleted by this path.

Student Assessments now emphasizes the selected Evaluation Period and selected Program, and Start Assessment carries the selected Cycle/Program into the opened assessment workspace so the Program's exact competencies and persisted assessment criteria load immediately. Talent Review accepts authorized Branch/Grade/Section filters and no longer presents "Meets criteria" as a primary user classification; deterministic Candidate policy remains internal and Official Identification remains distinct. Analytics error surfaces now reuse the backend's specific governed reason where available.

Program setup uses the neutral user-facing phrase **Assessment Criteria** for the competency-owned rubric structure. The separate real numeric feature is shown explicitly as **Key Performance Indicator (KPI)** with Add/Edit/Delete controls. Internal rubric persistence/API terminology is unchanged.

Talent Assessment Eligibility - Owner Simplification

Per direct Product Owner correction on 2026-09-11, Talent Student Assessment eligibility now follows ADR 0035. Current effective Academic Placement plus the Program's configured eligible Grades and usable assessment framework/tool are the operational authority. Draft/Open Cycle state, frozen population, synchronization, reconciliation, and persisted population membership are no longer user-facing prerequisites to start an Assessment. Existing Cycle and Population Member rows may remain as compatibility/historical provenance structures. Assessment creation captures the Student's placement/framework context at the time the Assessment starts. ADR 0033 is superseded as an eligibility/workflow model.

Talent Program Finalization, Evaluation Grouping, And Re-evaluation Reset Correction

Per Owner acceptance on 2026-09-11, a fully configured Draft Program no longer remains Draft after the user completes setup. **Finish Setup** is the governed lifecycle boundary: with organization/global authority and `talent_programs.govern`, the UI activates the Program first and then the completed Draft Framework using its current revision and semantic fingerprint, after which the same Program opens in operational summary mode. Already-active state is not activated again.

The Program operational summary's Edit Program, Build/Edit Rubric, and Manage Evaluation Plan links now participate in the same hash-backed renderer before the summary's early return, so those controls actually reopen their intended

panels. Open Assessments and View Results continue to use their canonical cross-view routes with Program/Academic Year context.

Student Assessments now projects the configured Evaluation Plan together with physical Cycle contexts. It groups by normalized user-facing Evaluation label, orders those groups by configured sequence when available, then renders unique Programs, so one Program cannot appear twice under the same Period merely because multiple Cycles exist or legacy sequence metadata differs. Programs with a configured Period remain visible before the first internal Cycle is created.

The reassessment compatibility boundary is also closed for legacy data created before assessed-Framework immutability was consistently enforced. Detection now covers both changed rubric/level bindings and same-ID semantic edits proven by Talent configuration audit events after the Student completed. A completed Assessment is marked **Re-evaluation required** when its persisted competency-result rubric/level bindings no longer match a complete current competency-owned rubric on the same Framework Version. Untouched legacy or partially configured rubrics do not trigger this rule. Starting re-evaluation makes the old completed attempt non-current and opens a fresh current in-progress replacement with zero results in the original visible Evaluation; prior evidence is preserved internally rather than deleted.

Normal Grade-level Competency creation no longer synthesizes an ASCII-only code in the browser. Unicode names such as Arabic Competencies are sent without a technical code, and the backend allocates the existing unique Program-scoped internal code. No schema migration was added.

Talent navigation now also follows an explicit module-tree/context-boundary rule. While Talent & Potential is active, the main sidebar expands Overview, Programs, Student Assessments, Talent Review, and Results & Analytics as permission-aware child destinations. Those top-level destinations carry the selected Academic Year only and clear stale Program/Cycle/Assessment and analytics filter context from the page being left. Programs therefore always re-enters the searchable Program list unless an explicit Program-opening link is used, and Student Assessments opens neutral with its own Program selector rather than inheriting a previously selected Program. The Programs table now exposes one normal Open Program entry point per row; Program-specific Edit Program, Build/Edit Rubric, Manage Evaluation Plan, Open Assessments, and View Results remain inside the opened Program workspace.

Talent Current Rubric, Re-evaluation, And Rubric-Delete Correction

Per direct Owner instruction on 2026-09-11, the normal Student Assessment workspace is current-only: the selected Evaluation shows the current eligible Students with Grade, Section, Assessment status, and Action, and no separate Assessment Records/History table is rendered. Superseded attempts remain preserved evidence internally.

The canonical current assessment structure is Grade -> Competency -> competency-owned Rubric -> ordered Levels. Legacy NULL-owned shared rubrics remain compatibility evidence only; `start_assessment_for_evaluation` selects only the newest saved Framework that has a complete competency-owned rubric with levels for every applicable Competency. A completed current Student whose newer complete rubric is materially different surfaces **Re-evaluation required**. Re-evaluation creates a new current Assessment against that newer Framework, links it to the prior attempt, retains the original visible Evaluation/Term through `evaluation_context_cycle_id`, and opens the replacement Assessment directly.

All Student-facing Framework mutations for competency/rubric/level/description authoring are blocked once that exact Framework has Assessment history. The Rubric tool exposes legal Competency and Rubric Level deletion only when the actor has the dedicated assignable permission. `talent_programs.delete_competency` and `talent_programs.delete_rubric_level` are independent from `talent_programs.delete`; `talent_programs.manage` alone does not imply either delete authority. Deleting an eligible Competency branch also removes its owned Rubric and owned Levels in the same mutation after descriptor/KPI/policy dependency checks, and never applies to an assessed Framework.

Talent Assessment Delete - Zero-Evidence Governance Exception Recorded

Per direct Owner instruction on 2026-09-11, a narrow Administrator-only Delete Assessment capability is now approved: see `docs/adr/0034-assessment-delete-zero-evidence-exception.md`. A `TalentStudentAssessment` may be permanently deleted only when it has zero competency results, zero Educator Input, zero Review Candidate membership, and zero Official Identification decision (the exact dependent-table list is re-verified against the schema at implementation time). Any Assessment with any such evidence remains permanent history and is never deletable, exactly like every other terminal Talent record. This does not authorize deleting Student placement history, Cycle population records, or any Student/Program delete capability beyond what ADR 0032 already governs. This entry and the referenced ADR are the authorization record, not a completion record.

Talent Corrective Reconstruction Batch 2 - Permissions, Draft Program Delete, Evaluation Timeline

Implements the governance recorded above in "Draft Talent Program Hard Delete - Scoped Governance Exception Recorded" (ADR 0032) plus a related set of permission-narrowing changes. Three new permission keys were added to `permission_registry.py` using the existing additive `(key, description)` tuple pattern, with no hard-coded role names: `talent_programs.delete`, `talent_evaluation_plans.delete_period`, `talent_evaluation_plans.manage_timeline`.

`DELETE /api/talent/programs/{program_id}` (`routers/talent_programs.py`, `talent_program_service.delete_program`/`program_delete_blockers`) permits a hard delete only when `program.status == 'draft'` AND zero rows exist in every table with a real, re-verified direct foreign key to `talent_programs.id`. That re-verification against the current `models.py` schema confirmed exactly the five tables ADR 0032 named - `TalentProgramFrameworkVersion`, `TalentProgramAcademicYearConfiguration`, `TalentCompetency`, `TalentAssessmentCycle`, `TalentEducatorInput` - and no others; every other Talent table reaches a Program only transitively through one of these five, so the database's own composite foreign keys already make a downstream row impossible once these five are confirmed empty. Any other status, or a Draft Program with a related row, is rejected with a clear error, never a silent no-op. The route requires `talent_programs.delete` plus the same organization-scope authorization already used by other Program mutation routes, and records the deletion through the existing `TalentConfigurationAudit` mechanism (no new audit path). The Program list/read payloads now include a real backend-computed `actions` array (`"delete"` present only when the backend already allows it) so the UI never client-side-guesses this affordance. No Configured/Active/Retired Program gained a hard-delete path; the activate/retire invariant is otherwise unchanged.

The existing Competency framework-membership and Rubric Level true-delete routes now use dedicated permissions: `talent_programs.delete_competency` and `talent_programs.delete_rubric_level`. Whole Draft Program deletion remains separately governed by `talent_programs.delete`; lifecycle/history guards remain server-enforced. Evaluation Period true-delete is narrowed from `talent_evaluation_plans.manage` to the new `talent_evaluation_plans.delete_period`. The new `talent_evaluation_plans.manage_timeline` permission gates exactly `planned_start_date`/`planned_end_date` edits and Period reorder/sequence (`POST .../periods/reorder`); ordinary Period content editing (label, short code, required flag, notes) stays under the existing `talent_evaluation_plans.manage`. A single `PATCH` touching both a timeline field and a content field now requires both permissions - missing either rejects the whole request before any field is applied, never a partial save. Viewing dates is unchanged (no new permission required to read them). `planned_start_date`/`planned_end_date` already existed on `TalentPlannedEvaluationPeriod` and were already returned/accepted by the API; this pass did not add new backend date capability. The Talent icon (`ui_shell.py` `PAGE_META["talent"]["icon"]` and the primary nav item) moved from the shared `clipboard-check` glyph (still used by Observations, unchanged) to the existing canonical `sparkles` glyph, so the two modules no longer share an icon. The Program list/summary "Type" label was renamed to "Scoring Mode" (derived states unchanged: Not set/Rubric/Numeric + rubric; no schema/API change), and the operational "Saved assessments" list is now labeled "Assessment Records" in

static/js/talent-operations.js, reusing the existing Start/Continue/View Assessment action distinction unchanged. The Evaluation Plan workspace now renders the existing server-computed advisory warnings (period_window_overlap, chronological_inconsistency, cycle_outside_planned_window, already returned on the Plan payload by talent_evaluation_plan_service.plan_warnings) as a compact, ARIA-labeled list - presentation only, no new validation math. A completed numeric assessment result now shows its already-returned scale (result_scale_min/result_scale_max) beside the result; interpretation text is not included in that payload and was not fabricated.

Both gaps above were closed in a follow-up verification/closure pass with UI-only changes and no new backend capability. static/js/talent-evaluation-workspace.js now renders each Period's planned_start_date/planned_end_date as always-visible native date inputs, editable only when that Period's backend-returned actions array includes edit_timeline (the existing talent_evaluation_plans.manage_timeline gate) and read-only text otherwise; the date save is its own separate request carrying only the two timeline fields, never combined with label/content fields, so the existing mixed-PATCH-requires-both rule is never bypassed by a silently partial submission; clearing a date sends null, matching the already-existing backend clear support. Separately, the shared Talent context filter (static/js/talent.js) now hides its top compact Program dropdown specifically on the /talent/programs view while no Program is selected, since that view's own compact Program table (with Program identity/logo) is already the selection mechanism there; the dropdown remains the sole selector once a Program is chosen on that view, and remains the sole selector on every other Talent view, which has no in-content chooser. A deep link that already carries program_id still lands directly in the selected state with no chooser flash, since the field-visibility decision is synchronous from the URL before any data fetch. No permission, schema, migration, tenant, or backend contract changed.

Talent Corrective Batches 3 And 4 Complete

One shared rubric visual module now renders arbitrary ordered levels across configuration, assessment entry, Talent Review, Student Profile, Learner Profile, and analytics. Review and profile read projections add the actual stored rubric label and its position without changing candidate computation. Official Identification remains a separate permanent human decision.

Talent Overview is reduced to compact operational action cards. Organization Overview no longer duplicates Program Results cards or a Talent Map matrix; it keeps selective progress, Program Criteria, Official Identification, Branch/Grade distribution, selected-Program rubric distribution, and selected-Program Evaluation Period progression. Learning Style distribution continues to use the existing Students endpoint and privacy contract, while its read-only filters now cascade from authorized Organization/Branch context through Planning-derived Grades and Sections.

Draft Talent Program Hard Delete - Scoped Governance Exception Recorded

Per direct Owner instruction on 2026-09-10, a narrow exception to the activate/retire-only Program lifecycle (recorded below in "Talent Program Setup Wizard Owner Correction" and unchanged) is now approved: see docs/adr/0032-draft-talent-program-hard-delete-exception.md. A Program may be hard-deleted only while status == 'draft' and it has zero rows in every table with a direct/child relationship to it (framework versions, Academic Year configuration, competencies, assessment cycles, educator input - the exact list is re-verified against the schema at implementation time). Every Configured, Active, Retired, or otherwise historical Program remains governed by activate/retire only, exactly as already recorded; terminal assessments and historical framework versions remain immutable. This entry and the referenced ADR are the authorization record, not a completion record.

Student Learning Style V1 - Governance Decision Recorded

Per direct Owner instruction on 2026-09-10, Learning Style moves from deferred/out-of-scope (as recorded throughout the M7-M10 milestone history below and in docs/AI_PROJECT_CONTEXT.md, docs/TIS_MASTER_CONTEXT.md,

and `docs/engineering/PRODUCT_ROADMAP.md`) to active V1 scope. See `docs/adr/0031-student-learning-style-v1.md` for the governed contract: optional single-select Student-domain field, four fixed values (Visual, Auditory, Read/Write, Kinesthetic), reuses the existing `students.edit` permission, zero effect on any Talent scoring/eligibility/Official Identification computation, and aggregate analytics display only under the existing Talent privacy/suppression contract. Implementation status is tracked separately as it lands; this entry and the referenced ADR are the authorization record, not a completion record. The historical milestone entries below that describe Learning Style as out of scope remain accurate as of the dates/milestones they describe and are not being revised.

Student Learning Style V1 - Core Implementation Landed

Per ADR 0031, the governed V1 contract is now implemented. `Student.learning_style` is a nullable `VARCHAR(20)` (migration `20260910_002_student_learning_style_v1`, purely additive - one nullable column, no rewrite; a `NOT VALID`/`VALIDATE` PostgreSQL CHECK constraint mirrors the fresh-schema SQLAlchemy `CheckConstraint` for defense in depth) validated server-side in `student_academic_service.py` against exactly `Visual`/`Auditory`/`Read/Write`/`Kinesthetic` (any other value, including case variants, is rejected with `invalid_learning_style` at the API layer regardless of UI restriction). Optional on Student create/edit (`templates/_learning_style.html`, reused by `student_form.html` and `student_profile.html`); unset renders as neutral "Not specified", never an error state. Displayed compactly on the Student Profile Overview and on the Student Profile Talent tab as explicit, visually distinct learner context ("Learner context (not a Talent signal)"). Editing reuses the existing `students.edit` permission; no new permission was introduced. Confirmed by source-scan regression test that no Talent scoring/Program-Criteria/Review/ Official-Identification module reads `learning_style`.

Branch/Organization Learning Style distribution is implemented as a lightweight Students-domain surface (`student_learning_style_analytics.py`, `GET /api/students/analytics/learning-style-distribution`, and a compact panel on the Students list page reusing its existing Branch/Grade/Section filters) rather than inside the Talent B7 organization-analytics closure, because its population - "Students with a current effective placement in the actor's authorized scope" - is never Program/Cycle-scoped like every existing Talent analytics metric. It reuses the exact governed Talent privacy contract (`talent_analytics_privacy.py`'s `Cell`/`Group`/`apply_primary_privacy`/ `run_complementary_suppression`, the same "one Group, total = sum(children)" shape the existing `/rubric-distribution` route already uses) rather than a separate or weaker rule, and fails closed (503 `analytics_unavailable` on the API; a hidden panel on the Students page) when no privacy policy is configured. The Talent Review/Assessment per-Student operational drill-down in `static/js/talent-operations.js` was deliberately left unchanged in this pass: its `context` payload is built in `routers/talent_review_candidates.py` and `routers/talent_assessments.py`, both already carrying substantial uncommitted in-flight changes from separate concurrent work at the time of this change, and extending it was judged unsafe to combine with that in-flight work rather than a scope decision about the feature itself.

Students + Talent & Potential M1 - Governance Reconciliation And Additive Schema Foundation (2026-09-22)

Per ADR 0042 (amends ADR 0031) and ADR 0043, this milestone adds only the persistence/schema foundation for two later capabilities: a four-dimension Student Learning Style profile and the globally unique managed TIS Student ID namespace. No API, service-layer write path, or frontend was added in this task; both are explicitly out of scope for M1.

`Student` gains four independent, nullable `INTEGER` columns - `learning_style_verbal_percentage`, `learning_style_non_verbal_percentage`, `learning_style_quantitative_percentage`, `learning_style_spatial_percentage`

1 each constrained to NULL or 0-100 with no sum rule and no relationship to

any other field. Migration `20260922_001_student_learning_style_four_dimension_profile` adds the columns via `ALTER TABLE ... ADD COLUMN` and, on PostgreSQL only, a non-locking `NOT VALID` + `VALIDATE CONSTRAINT` CHECK per column (SQLite relies on the fresh-schema SQLAlchemy `CheckConstraint`, matching the existing `20260910_002_student_learning_style_v1` precedent's documented dialect asymmetry). The legacy categorical `learning_style` column, its CHECK constraint, and every existing row are completely unchanged - no rewrite, no mapping, no automatic conversion in either direction.

The existing `StudentExternalIdentifier` model/table gains no new column. Migration `20260922_002_student_tis_number_identifier_integrity` adds two partial unique indexes scoped to `namespace = 'tis_student_number'`: `uq_student_external_identifiers_tis_student_number_value` (global uniqueness of `value` across every SchoolGroup, covering active AND inactive rows, so a retired value can never be reissued) and `uq_student_external_identifiers_tis_student_number_active_student` (at most one `active` row per `student_id`, correct organization-wide because `Student.id` is already a single global primary key). Every other namespace's existing tenant-scoped uniqueness (`uq_student_external_identifiers_scope_namespace_value`) is unchanged. Before installing either index, the migration inspects any existing `tis_student_number` rows and fails safely with a descriptive `RuntimeError` (no delete/merge/rename) if it finds a non-canonical value (not `STD` + exactly 10 digits), a value duplicated across SchoolGroups, or a Student with more than one active row; the current `tis.db` and every exercised test/local database had zero pre-existing `tis_student_number` rows, so this preflight path was exercised only via seeded-conflict tests, not a real historical blocker.

`Student.id` remains the sole internal relational identity; no new Student identity table or column was added, and no existing legacy Student was backfilled with a fabricated `tis_student_number` value - a legacy Student with zero managed-identifier rows remains valid.

Not implemented: TIS Student ID create/edit API/UI and its duplicate-value messaging, database-level canonical-format enforcement, Learning Style percentage API/frontend, Evaluation Progress, and roster import/export - all explicitly deferred to later milestones at this point in time. (The `students.import`/`students.export` permission keys were later registered as a governance prerequisite - see "Student Roster Import/Export - Permission Registration (Governance Prerequisite, M6)" above - but the roster import/export feature itself remained, and remains, unimplemented.) Al-Andalus Section display was also deferred at M1; per ADR 0045 it was Owner-authorized for the exact verified `workspace_uuid` `72e52eb2-3844-447b-92a8-c55015f73257`, and its M5 implementation is now done - see "Al-Andalus Section Display - Governance Authorization" and "Al-Andalus Section Display M5 - Shared Presentation Implementation" above. Focused coverage is in `tests/test_student_learning_style_profile_foundation.py` and `tests/test_student_tis_number_identifier_foundation.py`; PostgreSQL migration coverage (upgrade path and preflight-conflict rollback) is added to `tests/test_postgresql_migration_transactions.py` following its existing `TIS_TEST_POSTGRESQL_URL` skip-marked convention and auto-skips in this environment (no local PostgreSQL listener).

Students + Talent & Potential M3 - Four-Dimensional Learning Style Backend/Service/API (2026-09-22)

Per ADR 0042, this milestone implements the previously-deferred server-side service/API for the four-dimension Learning Style percentage profile on top of the M1 schema foundation (no new migration - the four nullable, 0-100 CHECK-constrained `Student` columns already exist). All logic lives in `student_academic_service.py` and `routers/students.py`; no new table, no new permission, and no frontend.

`student_academic_service.py` gained `LEARNING_STYLE_PERCENTAGE_FIELDS` (the four column names) and `_clean_learning_style_percentage`, which validates each dimension strictly and independently: `None` is valid ("not assessed"), and an `int` 0-100 inclusive is valid, explicitly including both boundary values; `bool` is rejected even though it is a Python `int` subclass, and any other non-`int` type - a float/decimal or a numeric string such as `"75"` - is

rejected rather than coerced, matching this dict-based JSON body's existing no-schema-coercion convention (there is no Pydantic model on this route). There is no sum-to-100 rule and no derivation to or from the legacy categorical `learning_style` column, which remains completely untouched. `create_student`/`create_student_with_number` gained the four optional keyword parameters (default `None`, all four may be omitted); an invalid value raises before any row is added, so an invalid dimension never leaves a partial/orphan Student, matching the existing M2 atomicity guarantee for `student_number`. `update_student` applies each of the four fields only when its key is present in the caller's `**changes` (the same partial-update convention the function already uses for `learning_style`/`first_name`/etc.), so PATCH's existing partial-payload semantics (`routers/students.py` only forwards keys actually present in the request body) mean an unspecified dimension is never cleared and an explicit `null` does clear it. `_student_payload` (used for the existing `StudentAudit` before/after snapshots) now includes all four fields, so a Learning Style percentage update participates in the existing append-only Student audit trail exactly like every other Student field - no new audit subsystem, no history/version table, and no Talent snapshot were added, matching ADR 0042's explicit prohibition.

`routers/students.py`: `POST /api/students` and `PATCH /api/students/{student_id}` now accept the four fields under the existing `students.create`/`students.edit` permission gates (no new permission); the M2 mandatory `student_number`-on-create contract is unchanged and re-verified by regression. Every Student JSON response (`GET`/`POST`/`PATCH /api/students...`) now exposes all four fields alongside the unchanged legacy `learning_style` field, with no derived dominant-style, total, or normalized field ever added.

Confirmed by source-scan regression (extending the existing M1 pattern) that no Talent scoring/eligibility module (`talent_program_service.py`, `talent_analytics_service.py`, `talent_org_intelligence_service.py`, `talent_analytics_privacy.py`, `talent_student_assessment_service.py`, `routers/talent_assessments.py`, `routers/talent_review_candidates.py`, `routers/talent_assessment_cycles.py`, `routers/talent_programs.py`) reads any of the four percentage fields or the new validator. Focused coverage is in `tests/test_student_learning_style_four_dimension_api.py` (73 tests: independent service-level validation of null/partial/0/100/no-sum-rule profiles, out-of-range and wrong-type rejection per field, create-time atomicity, partial-update semantics proving sibling/unspecified-field isolation, legacy-field preservation on create and update, Student-create API acceptance/omission of all four dimensions with the M2 `student_number` requirement re-verified intact, Student-update API permission reuse and cross-tenant denial, response serialization, audit participation, and the Talent source-scan regression). The full relevant regression sweep (`tests/` filtered to `student or talent or permission`, excluding the two `ortools`-dependent Timetable-Workflow-only solver test modules that fail to import in this environment) passes with exactly the same 13 pre-existing failures directly confirmed present and unchanged via an explicit before/after comparison run against unmodified `dev` HEAD `24bdb06` in this task (not merely cited from the prior M2 note); none are newly introduced or newly fixed by M3.

Students + Talent & Potential M2 - Managed Student ID Backend/Service/API (2026-09-22)

Per ADR 0043, this milestone implements the previously-deferred TIS Student number create/edit service/API and canonical-format validation on top of the M1 schema foundation (no new migration - the two M1 partial unique indexes on `student_external_identifiers` remain the sole concurrency authority). All logic lives in `student_academic_service.py` and `routers/students.py`; no new table, no new permission, and no frontend.

Business input is exactly ten ASCII digits (leading zeros preserved as a string, never parsed as an integer); the service alone controls the canonical `STD` + 10-digit stored format (`validate_student_number_digits`/`canonical_student_number`). New Students created through `POST /api/students` now require `student_number` and the new `create_student_with_number` atomically creates the `Student` row and its `tis_student_number` `StudentExternalIdentifier` row in one uncommitted transaction - an `IntegrityError` on the identifier insert (duplicate/race) is caught by the router, which rolls back the whole transaction first, so no orphan/partial `Student` row is ever left behind. Existing legacy Students (and the existing

HTML `/students/new` UI creation path, which is intentionally unmodified and continues to call the original `create_student` directly) are unaffected and may continue without a number; a legacy Student may receive or replace its number later via the new `PUT /api/students/{student_id}/student-number` route, backed by `set_student_number`, gated by the existing, already-governed `students.manage_identifiers` permission (no new permission key was needed or added). Replacing a number never mutates the old row: the old active row is marked `inactive` (audited as `replace_retire`) and a new row is inserted active (audited as `replace`) in the same transaction, so `Student.id` is unchanged and the M1 global-uniqueness index keeps the old value permanently reserved. The generic `add_external_identifier`/`deactivate_external_identifier` functions now explicitly reject the `tis_student_number` namespace (`managed_namespace` error) so the generic external-identifier API cannot bypass the managed contract in either direction.

Duplicate-value disclosure follows an explicit privacy rule enforced at the router layer (`routers/students.py::_student_number_conflict_response`), executed only AFTER the failed insert's transaction is rolled back: a read-only, unauthenticated lookup (`student_academic_service.find_student_number_holder`) identifies the conflicting `school_group_id`/`student_id`, and the router discloses `student_id`/`display_name` if and only if that identifier belongs to the requester's own SchoolGroup AND the actor independently holds `students.view` for that scope; every other case (cross-tenant, unauthorized same-tenant, or a value retired by a Student the actor cannot/does not independently view) returns the exact same generic `{"detail": ..., "code": "student_number_unavailable"}` body, so there is no enumeration oracle and no cross-tenant/retired-vs-foreign distinction leak. The raw lookup itself never grants a general Student-lookup capability - it returns only a bare `school_group_id`/`student_id` pair with no name or other metadata, and the router's authorization check is applied before any identity is ever returned.

A canonical `student_number` field (e.g. `"STD0012345678"`, or `null` for a legacy Student) is now exposed on every Student JSON response (`GET`/`POST`/`PATCH /api/students...`) via `current_student_number`, without renaming any existing field or changing unrelated response shape.

Three pre-existing tests that posted to `POST /api/students` without a `student_number` were updated to supply one, since this endpoint's required- field contract is the intended M2 behavior change, not a regression: `tests/test_student_academic_foundation.py` (two call sites, one with an updated audit-event-count assertion reflecting the one additional `external_identifier/create` audit row every managed-number creation now also writes), `tests/test_student_learning_style_v1.py` (two call sites), and `tests/test_permission_surface_consistency.py` (one call site, a permission- gating test unrelated to the identity field itself). Focused coverage is in the new `tests/test_student_managed_number_service.py` (46 tests: format validation, atomicity/no-orphan-Student, legacy assignment, replacement mechanics, M1 global/same-tenant/cross-tenant/retired-reuse uniqueness, the IntegrityError race path, privacy-safe duplicate classification for all four combinations, `students.create`/`students.manage_identifiers` permission gating, cross-tenant denial, the managed-namespace boundary on the generic identifier API, audit events for create/assign/replace, and regression for an unrelated namespace and an existing Academic Placement workflow). The full relevant regression sweep (`tests/` filtered to `student or talent or permission`, excluding the two `ortools`-dependent Timetable-Workflow-only solver test modules that fail to import in this environment) passes except the same 13 pre-existing failures already present on unmodified `dev` HEAD `e98214f` before this task (11 unrelated Talent/permission/SaaS failures plus the two Students failures already recorded as pre-existing in the M1 entry above); none are newly introduced by M2, and both this environment's `ortools` absence and those 13 pre-existing failures are unrelated to this task and unresolved by it.

Evaluation Plan Scope Regression Correction

Evaluation Plan capability projection now has explicit regression coverage for both sides of the authority boundary: an organization-scoped manager retains manage/govern actions while working in a selected Branch, while a truly Branch-scoped identity receives no mutation capability even if its role grants the permission key. The Evaluation Plan workspace explains that read-only state before submission.

The sanctioned Local Test Admin source remains organization-scoped with North Campus assigned as its working Branch. A previously created disposable local Talent database may contain a stale `BRANCH` value; the supported correction is the sentinel-guarded `scripts/manage_local_talent_test_db.py reseed --confirm` workflow.

Talent Completed Program Operational Workspace

Completed active Programs open as a compact operational summary instead of remaining permanently in onboarding. Explicit Edit Program, Edit What we assess, and Manage Evaluation Plan actions reopen the same hash-backed wizard at Steps 1, 2, and 3. A fully configured Draft Program crosses into Active state when an authorized user chooses Finish Setup: Program activation occurs before Framework activation, then the Program returns to operational summary. Incomplete Programs remain in setup mode.

The UI now uses Evaluation Plan and Evaluation Period terminology consistently. Step 3 remains embedded. Program Basics reads the selected Academic Year option text rather than displaying its database ID. Organization-authority failures are mapped to clear Evaluation Period access language, and Branch-scoped users are not shown Plan-management controls the API cannot execute.

Talent Guided Configuration And Student Display Correction

The Owner screenshot follow-up is implemented in the shared Talent workspace. Program Basics is one concise identity/year/Grades form. What we assess uses compact substeps with table rows and on-demand editors. Evaluation Schedule remains embedded inside the Program workspace while reusing the existing standalone renderer and backend contracts. Readiness is now projected on the Program summary instead of a separate Ready panel. Editing a Program from the Programs table opens this same workspace and its hash preserves the selected setup or assessment substep across refresh.

Student collection presentation now explicitly shows the compact table above 680px and compact cards at or below 680px, never both at once. Active lifecycle state remains in accessible text and filtering but is visually represented by a neutral dash; exceptional statuses keep a visible status chip. Open Evaluation and Talent Review remain compact collection tables with detail shown only after opening one Student.

Talent Program Setup Wizard Owner Correction

The Programs detail workspace uses the Program header plus focused setup panels and local Back/Save & Continue or next-action controls. Step selection remains hash-backed where applicable. The implementation no longer emits all setup sections or duplicate Grades/readiness cards into one long page. Readiness is shown only in the Program summary and becomes positive only when all required setup areas are complete.

The management-action audit confirms the UI exposes existing permission- and lifecycle-authorized edit/remove operations for Program details and logo, draft competencies, rubric levels, achievement descriptions, optional KPI and Program criteria, Evaluation Schedule periods, and in-progress assessment results. Programs remain governed by activate/retire rather than hard delete, and terminal assessments and historical framework versions remain immutable.

Talent Final Completion Continuation

Program Identity is implemented as optional organization-owned Talent Program branding with additive logo columns, existing `talent_programs.manage` authorization, validated Program-scoped storage, upload/replace/remove actions, and compact initials fallback. The shared Talent surfaces reuse the same logo badge. No Branch override exists.

Talent Program Grades now come only from operational Current/New `PlanningSection` rows. The shared longitudinal context cascades Academic Year to authorized Branch, configured Grade, and Planning Section; an empty section result disables the selector with "No Sections configured for this Grade." Direct annual-configuration writes validate against the same Planning authority, including the empty-Planning case. In-progress assessment competency results expose the existing revision-guarded delete route as "Clear Result"; terminal assessments remain read-only.

Workspace mutations preserve URL context and scroll position.

Talent Review Terminology, List Density, And Active-Status Presentation

Further Owner product-polish direction on the same uncommitted M11 workspace produced these presentation-only changes, all re-verified against the fresh current code before editing:

1 The normal user-facing "Review Candidates" surface is relabeled "Talent

Review" throughout the live rendered path: the Talent nav entry and breadcrumb title (`routers/talent_ui.py`'s `VIEWS` display label), the in-page note/empty-state/confirm/notify text and the "Check Program Criteria" action in `static/js/talent-operations.js`, and the Student Profile Talent tab (`templates/student_profile.html`). The backend `ReviewCandidate` model, its `/api/talent/review-candidates` routes, and the `talent_review_candidates.*`/`talent_official_identifications.*` permission keys are unchanged deterministic/audit identifiers - only the displayed label changed. A dead, unreachable legacy rendering branch in `static/js/talent.js` (superseded by `TalentOperations.render` for the `programs/evaluation-plans/assessments/reviews` views since an earlier pass) still contains the old wording; it was left untouched because it never executes and editing it would not change any observed behavior.

1 The Talent Review list (`static/js/talent-operations.js`) is now one

compact table (Student, Program, Grade, Section, Evaluation, Result, Review status, Identification status, Action) instead of one large card per Student, reusing the existing `.tp-compact-table`/`.tp-table-wrap` classes already established by the Programs table (`talent-program-workspace.css`). Opening a row (`?review_id=`) shows that one Student's full detail - including the Official Identification decision form - the same content the card used to show inline for every Student. No server-side pagination exists anywhere in Talent to reuse (Programs and Students both use a plain table with client-side search, not pagination), so none was invented here either.

1 An opened Evaluation's Student list is now the same compact-table pattern

(Student, Grade, Section, Assessment status, Action) instead of one card per Student, with a state-driven action label read from the real assessment status: no assessment yet - "Start Assessment" (only offered while the Cycle is Open and the actor can manage); `in_progress` - "Continue Assessment"; any other recorded status (Completed, Incomplete, Insufficient Evidence) - "View Assessment". No new assessment state was introduced; this is presentation over the existing status values.

1 In the Students list (`templates/students.html`), the normal "Active"

lifecycle status (Student lifecycle, not Talent status - unchanged semantics) no longer renders as a colored `stu-chip` badge on every row, since Active is the expected state for a normal attending Student. It now renders as plain muted text (`.stu-status-quiet`). "Inactive" keeps its visible chip as a real exception, and Status remains an available filter/column in both the desktop table and mobile cards.

1 Investigated and explicitly deferred (not implemented) from the same

direction: (1) Program Identity/Logo, a new capability requiring a `TalentProgram` schema/model change and new upload routes - out of this pass's safe bounded scope. Program ownership was confirmed organization- scoped (`school_group_id`, no `branch_id`) on the current model, so no branch-level Program logo override exists or was added; `branding_storage.py` (organization/branch logo upload, including safe SVG sanitization) is the correct existing pattern to reuse if this is built later. (2) Planning- driven Grade/Section pickers for Talent's own filters/setup screens - confirmed still open: the shared Talent context filter's Grade picker in `static/js/talent.js` still renders a hardcoded KG+1-12 list rather than the Planning-configured subset the Students module already uses.

Presentation-only: no schema, migration, permission, analytics computation, provider, privacy, tenant, review-candidate/official-identification business rule, or `tis.db` change.

Talent Owner Visual Acceptance Corrections

Real Owner visual acceptance of the M11 workspace (as distinct from the prior code-level trace, which explicitly could not obtain browser screenshots) found four presentation defects, all corrected in `templates/talent/workspace.html` and `static/js/talent.js` (the one shared filter form/apply logic every Talent page reuses) plus `tests/talent_results_experience.test.cjs` and `tests/test_talent_ui.py` regressions:

1 The `tp-filters` context selector (Academic Year/Program/Branch/Grade/

Metric/Dimension) no longer requires clicking "Apply context" to take effect on any page - Programs, Evaluation Schedule, Organization Overview, and every other Talent page. A `change` listener on the form calls the same `applyContext()` reload path the button already used, debounced 250ms so a user clicking through several dropdowns triggers one reload, not one per dropdown. The existing URL-query-parameter/`history.replaceState` context mechanism is reused unchanged. The button is relabeled "Refresh" and is now an optional immediate fallback, not a required confirm step.

1 The primary Talent nav no longer repeats the full analytics-family link list

(Organization Overview, Talent Map, Program Results, Students Across Programs, Progress Over Time) directly above the existing sticky analytics sub-nav that already lists those same pages plus Students. The primary nav now shows exactly one "Results & Analytics" entry; the sub-nav is kept because six distinct analytics pages are genuinely more than the primary nav should enumerate.

1 Confirmed still intact and covered by a fresh rendered-output regression:

the Evaluation Schedule normal path renders no internal-lifecycle copy ("Prepared evaluations", "Ready to link", "Link an evaluation", "No Evaluation Plan", "Frozen population", or the bare words Plan/Period/Cycle/ Link) and evaluation names remain completely free-text/user-defined - both already corrected by the immediately preceding Talent UX Simplification pass below and re-verified here, not re-implemented.

1 Confirmed still intact: the Programs landing screen is a compact searchable

table with the Create-Program form collapsed behind "New Program", not permanently expanded.

Diagnosed and NOT code-changed (environment-setup issue, not a defect): Organization Overview showing "Analytics unavailable" locally is the correct, intentional fail-closed M10 behavior whenever the running process's `DATABASE_URL` does not resolve to exactly `.local_test_data/ talent_local_test.db` (`talent_organization_analytics_providers. is_sanctioned_local_analytics_environment()`); `database.py`'s `DEFAULT_SQLITE_URL` falls back to `tis.db` when `DATABASE_URL` is unset, which is what an ordinary local run does today. The detection logic itself was verified correct (relative and absolute `sqlite:///` forms both resolve correctly on Windows); running the app with `DATABASE_URL` set to the sanctioned local path before process start activates the deterministic local providers. This is unchanged from the existing B11-E F1 contract described below and required no code change.

Presentation-only: no schema, migration, permission, analytics computation, provider, privacy, tenant, or `tis.db` change.

Talent Program And Evaluation UX Simplification

The Programs and Evaluation Plans surfaces now implement the Owner-directed simplified journey. Programs use a searchable compact desktop table; the Create-Program form is not permanently expanded on that landing screen and opens instead as a collapsed panel behind a "New Program" action. Program setup is labeled Basics, What we assess, Evaluation schedule, and Ready; competencies display alongside their rubric levels and achievement descriptions, while version details remain in the edit disclosure. Saving returns to the joined view.

Evaluation schedule replaces normal Plan/Period/prepared-Cycle terminology with a free-text, user-defined evaluation name (for example Term 1, Audition, or Spring Review) and the states Setup, Ready to start, In progress, Complete; internal-lifecycle copy ("Prepared evaluations", "Ready to link", "Link an evaluation", "No Evaluation Plan", "Frozen population") is not rendered in this normal path. Start Evaluation safely sequences draft preparation, revision-guarded Period linkage, population preview, explicit count/date confirmation, and governed open/freeze. The previously omitted Evaluation Plan manage/govern permissions now reach the browser payload, with govern suppressed for Branch scope. No schema or backend contract changed.

Production Student/Talent Migration Blocker Repair

The Render pre-deploy blocker is repaired locally and ready for independent review. Root cause was pre-ledger `metadata.create_all()` attempting the new Student/Talent model graph against legacy PostgreSQL parents before migration `001` could establish required composite uniqueness. `branches(id, school_group_id)` failed first; `academic_years(id, school_group_id)` was also missing in the production-style baseline. `students(id, school_group_id)` is created with its unique constraint on fresh runs and is covered by the new prerequisite for already-existing schemas.

The runner defers the complete Student/Talent table set and the ledger adds `20260904_000` before the existing eight migrations. A separate current-model coupling found by the fresh-chain test is also resolved: M4 Cycle creation no longer requires the future M8 Period table, while M8 still adds the exact nullable composite foreign key after creating its parent. PostgreSQL 16 results: complete 63-entry fresh run PASS, second run no-pending PASS, existing-schema late-`000` PASS, duplicate preflight rollback PASS, and old-runner partial-DDL rollback PASS. No production inspection, SQL, deployment, commit, or push was performed.

Phase H Cumulative Release Qualification

Phase H is PASS WITH BASELINE EXCLUSIONS. The current full-suite run collected 1,748 parent test items and reported 1,618 passed, 78 failed, and 59 skipped, plus 87 passed subtests. The sometimes-cited 1,755 is the arithmetic sum of those headline outcome counters, not a separate collection count. A clean pre-Talent baseline at `8ed2cc998b8f1c29b5f1d7b34c3b9532deff603e` reproduced all 78 current failures in the bounded failure-file set; 78/78 are baseline/environment failures and zero are attributable to Talent changes. The production-deployment memory-allocation ratio remains environment-specific and unevaluated until the actual Render allocation and worker count are known.

M10 B11-E F1 Production Providers

The current working tree implements production-capable provider resolution for all seven Organization Intelligence routes. `talent_organization_analytics_providers.py` owns the fail-closed implementations: an externally configured P1-P7 minimum cohort of 5, the `feature.organization_intelligence` commercial feature check, and externally configured ceilings of 1000 matrix cells, 1000 relationship results, and 1000 Program-pair results. The privacy provider keeps primary and complementary suppression; the breadth provider rejects rather than truncates; and availability is evaluated before, but independently from, the existing route permissions.

The exact `.local_test_data/talent_local_test.db` URL activates deterministic providers only outside production, allowing all seven routes and pages through normal dependency resolution. `tis.db`, memory/other SQLite databases, and `prod`/`production`/`live` never activate that path. Missing, invalid, mismatched, or exceptional provider state remains unavailable. Focused provider, analytics, entitlement, and demo-registry tests pass, along with real local HTTP checks for all seven APIs/pages and a production-like fail-closed check. No schema, migration, new permission, plan/pricing logic, production data, or API response shape changed. As of 2026-09-09 (ADR 0028/ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED.

Talent Phase C Results & Analytics Experience

The current integrated working tree adds a presentation-only Phase C layer over the seven existing M10 Organization Analytics routes. The Organization Overview is now an executive dashboard with Academic Year context, governed KPI cards, visual Program summaries, unranked Branch navigation, and a Talent Map preview. Dedicated Program Results, Branch Results, Talent Map, Students Across Programs, Students, and Progress Over Time views use the approved public vocabulary and provide contextual drill-downs.

Visible backend percentages alone drive progress-bar and period-chart dimensions. Counts are displayed as facts without normalization; protected and unavailable results use categorical treatments that carry no magnitude. Student results retain the P7 gate, max-100 paging, frozen context, independent Candidate/Identification fields, and the canonical Profile path. Progress Over Time remains one Program plus one Academic Year and never calculates deltas, trends, or improvement. No analytics service, provider, privacy, permission, tenant, schema, migration, or data behavior changed.

A follow-up Owner-flagged presentation pass adds two more views over the same unchanged backend contracts. The Organization Overview now leads with a single largest-on-page radial gauge showing "How many Students are talented" - drawn only from the existing `identified_of_eligible` M10 Organization Analytics metric's `organization_total` cell (org-wide, or the selected Program's scope when one is chosen), gated by the existing `talent_official_identifications.view` permission. This is the confirmed authoritative "talented" signal: Official Identification, a separate, permanent human decision recorded in Talent Review - not the rubric level and not Meets Program Criteria/Review-Candidate membership on their own, both of which remain displayed as distinct supporting signals. Second, the previously implemented but UI-unused M9 `/rubric-distribution` route is now wired into the Organization Overview (shown only when one Program is selected, reusing the existing `talent_analytics.view` permission): a new shared, order-derived rubric-level visual (`rubricDistribution`/`rubricLevelIntensity` in `static/js/talent.js`) draws each rubric level's intensity purely from its position among its siblings - never from a count/percentage/magnitude - so any Program's arbitrary label set and level count renders safely, including a level whose result is privacy-protected. Bar length for a level continues to come only from an already-visible backend percentage, identical to the existing grade/branch bar pattern. No analytics service, provider, privacy threshold, suppression rule, permission, tenant, schema, migration, or data behavior changed; both additions are pure consumers of already-existing, already-tested backend projections.

Automated Node presentation/privacy/accessibility checks and all seven focused M10 route suites pass. Read-only verification against `.local_test_data/talent_local_test.db` returned HTTP 200 for every analytics API and page. No connected browser was available, so desktop/mobile visual acceptance remains pending independent review.

Talent Phase B Full Operational Workflow

The current working tree upgrades the M11 Talent & Potential workspace from its read-only draft into an operational M2-M8 workflow. `static/js/ talent-program-workspace.js` provides Program creation/editing, annual eligible Grades, version history/cloning/governance, competency and rubric ordering, descriptor configuration/removal, optional Framework-specific KPI settings, and Review Candidate rule configuration. `static/js/ talent-evaluation-workspace.js` reuses authoritative M8 action projections to create Plans and Periods and M4 APIs to prepare, link, and open Cycles. `static/js/talent-operations.js` starts assessments from authorized frozen population rows, edits exact competency/rubric results through the real M5 write API, handles sequential revisions and stale conflicts, completes/finalizes assessments, evaluates/reviews Candidates, records the separate human Official Identification decision, and adds/amends separately governed Educator Input.

Assessment/Candidate API responses now include a display-only `context` built after existing tenant and historical-Branch authorization. Population responses add Student and Branch names while retaining frozen Grade/Section authority. Framework and rubric read projections expose the stable row IDs required by writes; descriptor IDs are exposed for precise removal. Public IDs are kept out of the semantic-fingerprint projection, preserving the original M3 semantic identity contract.

Focused frontend checks pass, the full Talent pytest selection passes, and a real-router temporary-SQLite HTTP test covers the complete qualitative workflow, including partial-save revision sequencing, stale rejection, no automatic

identification, the mandatory review gate, one-decision enforcement, and Educator Input amendment history. The temporary acceptance harness never opens `tis.db`. Browser verification is still pending because computer-use reported no available browser. Detailed Candidate rule-result reasons remain unavailable because the current read payload does not expose the persisted evaluation snapshot. This is a Web Service-only change with no Workflow-worker or migration deployment requirement.

Student Profile History Read Endpoint (Partial Phase A Core Students Slice)

A delegated "Phase A - Core Students" slice (canonical Students navigation area, list/add/edit, Academic Placement UI, and a four-section Student Profile with a read-only Talent & Potential summary) added exactly one backend piece before a live concurrent-agent collision was found and the remaining UI scope was deliberately stopped (see below): `GET /api/students/{student_id}/audit` (`routers/students.py`), backed by new `student_academic_service. list_audit_events/audit_event_payload`. It is a pure read-only projection of the existing append-only `StudentAudit` rows every Student/placement/identifier mutation already writes (create/update/status-change, external-identifier add/deactivate, and placement create/end/correction/transition) - no new persistence authority, `resource_type`, schema, migration, or permission was added. It requires the existing `students.view` permission and reuses the same tenant-scoped non-enumerating 404 discipline as every other direct Student-scoped read route; actor user ids are resolved to display names via a plain `models.User` lookup restricted to the returned event set. Regression: `tests/test_student_academic_foundation.py::test_audit_endpoint_reuses_existing_append_only_trail_and_resolves_actor_names` and `::test_audit_endpoint_requires_students_view_permission` (16/16 passing in that file; no regressions in the related `tests/test_talent_learner_profile.py` suite).

The remainder of that delegated slice (Students list/add/edit screens, Academic Placement assignment UI, the canonical Student Profile page and its read-only Talent & Potential summary reusing the existing M7 Learner Profile read API, main-navigation registration, and replacing the M11 Student Drill "Open Learner Profile" links to point at the new profile) was NOT implemented: a separate, live, actively-writing concurrent agent process was found mid-edit on the exact same new file this task needed to create, `routers/students_ui.py` (observed twice with materially different content, the second time syntactically invalid/torn), already independently building the same list/add/profile/placement/Talent-integration surface under the identical route prefix and template names. Building competing templates/CSS/JS/nav wiring in parallel would have risked a destructive overwrite race on that file and on `ui_shell.py`/`main.py`'s Students navigation wiring, so the remaining UI implementation was deliberately deferred pending reconciliation. No file was reverted or repaired by this task.

M5/M3 Assessment Framework Read-Contract Addition

Fixed a confirmed, independently re-verified read/write API contract gap blocking any future assessment-entry UI: `GET /api/talent/programs/{id}/frameworks/{id}` returned each competency keyed only by `competency_id` (`FrameworkCompetency.talent_competency_id`, the parent catalog FK), never the row's own `FrameworkCompetency.id`; no read route exposed `TalentRubricLevel.id` at all (`get_framework_configuration`'s `levels[]` returned only `code/label/description/order/numeric_value`). The write route `PUT /api/talent/assessments/{id}/competency-results/{framework_competency_id}` (`talent_student_assessment_service.set_competency_result`) is FK-constrained on exactly `FrameworkCompetency.id` and `TalentRubricLevel.id`. Both read routes now additively include each row's own `id` field; no migration was needed (both columns already exist), no permission/privacy semantic changed, and the write route's contract is untouched. New regression: `tests/test_talent_student_assessment_competency_results.py::test_framework_read_ids_are_accepted_end_to_end_by_the_write_route` proves a write built purely from the new read `id` fields succeeds end to end; `tests/test_talent_rubric_kpi_candidate_policy.py` gained a matching rubric-level `id` assertion. Full `-k talent` regression: 483 passed, 10 skipped, no failures. This backend fix is included alongside the M11 Talent &

Potential UI work (`routers/talent_ui.py`, `templates/talent/`, `static/css/talent.css`, `static/js/talent.js`) that this fix is intended to unblock; no UI work consuming these new fields was implemented in this pass.

M10 B11-E Integrated Production Qualification

ADR 0029 governs and implements a dedicated M10-only REPEATABLE READ session/ dependency (`database.M10OrganizationAnalyticsSessionLocal`, `dependencies.get_m10_organization_analytics_db`) for all seven Organization Intelligence routes, backend-conditional (PostgreSQL only), with no server-wide or other-route isolation change. All seven routes were live-re-tested under the permanent implementation and confirmed consistent; the two B11-D-proven mismatches (Overview Candidate, Student Drill gate/page) no longer occur, and the five previously structurally-inferred routes (talent-map, program-portfolio, branch intelligence, participation-overlap, longitudinal) were independently reproduced live for the first time and confirmed resolved.

Suppression/reconstruction concurrency was tested live with the existing non-production `DeterministicSuppressionTestPolicy` (primary and complementary suppression); no suppressed value became reconstructable. Index Candidate B is finalized as NO CHANGE; statistics freshness is finalized as an operational/runbook disposition (no code/migration change). F1 subsequently implements the approved fail-closed production privacy, availability, and breadth providers. That resolves the provider-code blocker, while all other B11 release gates remain open. 14 earlier committed regression tests were added; the full 226-test regression passed; `tis.db` was confirmed unchanged. Independent review returned PASS WITH NON-BLOCKING OBSERVATIONS; B11-E implementation is PASSED and checkpointable. **As of 2026-09-09 (**`docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md`**, ADR 0028, ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED. Production deployment has not occurred.**

M10 B11-D PostgreSQL Concurrency Qualification Closure

B11-D is CLOSED after a PASS WITH NON-BLOCKING OBSERVATIONS independent review. READ COMMITTED mixed snapshots were reproduced within one request: Overview Candidate count 314 to 315 and Student Drill gate/page population 705 to 706. A non-permanent REPEATABLE READ experiment resolved both cases, but permanent adoption was then PROPOSED - GOVERNANCE REQUIRED and NOT IMPLEMENTED. B11-E subsequently governed and implemented a scope beginning before the first database statement and spanning every M10 evidence input through privacy/reconstruction.

B11-E subsequently live-qualified all other multi-statement routes and suppression under concurrency. Candidate B is NO CHANGE with no index approved; statistics freshness/ANALYZE is operational guidance. Tenant isolation remained clean. B11-E implementation is PASSED. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

M10 B11-C PostgreSQL Qualification Closure

B11-C is CLOSED. PostgreSQL 16.15 profiling of all seven M10 routes produced durable evidence, bounded query counts across tested scales, no N+1 finding, Index Candidate A = NO CHANGE, and Candidate B = MORE EVIDENCE REQUIRED. No index, schema, migration, permission, privacy, or entitlement semantic change was made. The 209-test regression passed. Memory evidence is directional local Windows development evidence only; no production threshold, breadth limit, commercial mapping, SLO, or memory value was selected.

B11-D subsequently completed and is CLOSED. B11-E implementation subsequently PASSED. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED. Performance SLO, memory ceiling, and

multi-process/multi-worker qualification were subsequently gathered and adopted per ADR 0028/ADR 0030.

M10 B11 Production Qualification Governance

ADR 0028 records the approved B11 production-policy boundaries and now records the F1 Owner decisions: minimum cohort 5 for P1-P7, semantic feature key `feature.organization_intelligence`, and breadth ceilings of 1000 matrix cells, 1000 relationship results, and 1000 Program-pair results. F1 implements those decisions through external configuration and the existing entitlement/feature- registry architecture. Missing/false/reject/exception paths fail closed. Plan packaging, performance SLOs, memory ceilings, and any future index remain outside F1 and unresolved where the governing qualification still requires it.

B11-C and B11-D subsequently completed and are CLOSED as recorded above. The F1 provider implementation is complete; B11-E is PASSED. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED. Master merge and deployment remain blocked pending cumulative production release qualification and a separate `dev->master` approval.

M10 B0-B9 Organization Intelligence

M10 B0-B2 is implemented without an Organization Intelligence route, aggregate query, privacy threshold, entitlement rule, schema, or migration. `talent_org_intelligence_contract.py` provides immutable canonical Cell identity, exact `+1/-1` additive Relationships, inherited M9 privacy-state projection semantics, tenant validation, and the V01-V25 ledger. `talent_analytics_relationship_graph.py` adds the tenant-bound canonical Cell registry, Relationship identity/deduplication, Cell-to-Relationship adjacency, fail-closed structural validation, and deterministic connected components. `talent_analytics_privacy_closure.py` adds exact Fraction-based RREF, per-coordinate uniqueness, inconsistent-system detection, monotonic bounded component-local closure, magnitude-independent victim choice, fail-closed analyzer handling, exact-source-only derived rates, and all-or-nothing sibling projection. V01-V25 are executable. No real M10 Organization Intelligence or Talent Map data can be exposed.

B3/B4 adds `talent_org_intelligence_service.py`: immutable tenant/year/ historical-Branch access context using existing permissions; fail-closed unconfigured commercial-availability and breadth providers; normalized context fingerprinting; enabled annual Program universe; frozen-population-only filters; set-based Program×Branch, Program×Grade, Program/Branch/Organization totals; permission-skipped Candidate and Identification queries; and Program-grain M8 required-period execution. All primitives share the caller's Session and never query current Student Placement. Future serialization must accept a B2 closure result through `PrivacyClosedProjectionSet`.

B5 adds only `GET /api/talent/organization-analytics/overview`. The closed contract now has 14 MetricCodes and 7 MembershipGrains: B5 added `programs_configured`, `active_programs`, and `program_configuration`, both metrics mapped to `count`/`program_configuration` and P1. Configured means an enabled Program annual configuration; active is exactly its subset whose durable Program lifecycle status is `active`. The route preserves historical Branch filtering, skips Candidate/Identification SQL without their independent permissions, counts only `decision == identified`, and serializes exclusively after M9 primary privacy and B2 closure.

B6 is implemented, independently reviewed, and closed after the targeted sparse-matrix re-review passed. It adds only `GET /api/talent/organization-analytics/talent-map`, supporting Program-by-Branch and Program-by-Grade. `branch_program` reuses the same canonical topology and transposes presentation only after closure. Backend row/column/authorized-scope totals participate in the graph. It supports the approved frozen/completed/started/Candidate/identified counts and eligible rates, rejects Program-level required Period execution, and adds no ranking, universal score, schema, migration, UI, or B7+ route. Production privacy, commercial availability, and breadth providers remain fail-closed. The sparse-total fix uses the exact same authoritative numeric child set for total arithmetic and graph terms, excluding rectangular `no_data` display coordinates without treating them as zero. Empty totals stay `no_data`; one- child totals keep an explicit two-term equality. B2 itself is unchanged.

B7 implements Program Portfolio and one-Branch Intelligence. Portfolio uses Program/Academic-Year common facts and required-Period execution only at that grain. Branch Intelligence filters frozen Branch before grouping, shares B6 Program/Branch identities, and excludes execution. Candidate/identified fields are permission-omitted. Both routes use breadth and closed projections. Grade breakdown, ranking, and scores remain unimplemented.

B8 implements Participation Overlap as distinct Student intersections across Open/Closed frozen Cycle participation. It adds the sole governed `participation_overlap` P2 count metric at `program_participation` grain, deduplicates Program/Student participation before one set-based self-join, applies historical Branch scope before intersection, and uses one symmetric canonical Cell per normalized Program pair. Diagonals are distinct Program participants; absent participation basis is `no_data`, while an authoritative empty intersection is factual zero subject to privacy. No overlap sums, Candidate/Identification overlap, ranking, Talent Breadth/score, Student data, schema, migration, UI, AI, or B9+ route is implemented. Its initial independent review identified a canonical pair-index mismatch when Program name order differed from numeric ID order. Construction and every lookup now use one explicit numeric canonical-pair helper, preserving name/id presentation order while sharing one Cell/privacy decision across both matrix orientations. Targeted independent re-review passed; B8 is CLOSED. Its committed/pushed implementation remains part of current dev.

B9 adds only `GET /api/talent/organization-analytics/students`, the first M10 Student-identifiable route. It requires `talent_analytics.view` AND `talent_analytics.view_students` (true AND composition), derives Student inclusion only from frozen Cycle population context in the authorized tenant/AY/historical-Branch/Program scope (never current Placement), and treats every identifiable row as P7 through one gate-level `student_drill_population/count/distinct_student` Cell fed by `COUNT(DISTINCT student_id)` and run through the same B2 closure pipeline (no per-Student additive relationship graph). Candidate/Identification fields are independently permissioned and query-skipped, not response-filtered; Learner Profile access exposes only an advisory `can_view_learner_profile` hint. One top-level row exists per Student with deterministic frozen Program/Cycle contexts; pagination is over distinct Students (limit 25/max 100, `has_more`, no `total_count`) and breadth precedes identifiable queries. No direct Student-ID filter is exposed. No schema, migration, permission, entitlement, longitudinal/B10, UI, or AI capability was added. The remediation passed targeted independent re-review; B9 is CLOSED. Its committed/pushed implementation remains part of current dev.

ADR 0027 records the B10-A "Longitudinal Organization Intelligence" architecture as an approved governance decision only, following independent governance review ("approve with required amendments"). B10 itself remains NOT IMPLEMENTED - no route, query, schema, migration, permission, or entitlement was added by this decision, and B9's status above is unchanged. The approved contract: one Program, one Academic Year (`AcademicYear. year_name` stays descriptive only, never chronology authority), ordered M8 `TalentPlannedEvaluationPeriod` slots as the sole ordering authority (Period = presentation/order, optional linked Open/Closed Cycle = evidence, consistent with the existing `uq_talent_assessment_cycles_period` constraint), exactly nine of the fourteen existing `MetricCode` values (`frozen_eligible`, `completed`, `completion_coverage`, `assessment_started`, `started_coverage`, `candidate_count`, `candidate_of_eligible`, `identified_count`, `identified_of_eligible`), one metric per response, `comparable`/`not_comparable` states with no growth language, and no server-computed delta/percent-change between points. See `docs/adr/0027-b10-longitudinal-organization-intelligence-contract.md`.

B10-B implements the ADR 0027 contract as `GET /api/talent/organization-analytics/programs/{program_id}/longitudinal` in `talent_org_longitudinal.py` (the seventh Organization Intelligence route). It reuses `resolve_access_context`/`authorized_program_universe`/`resolve_filters`/`frozen_membership_query` unchanged, fetches the single governed M8 Plan (`uq_talent_annual_evaluation_plans_config` guarantees at most one Plan per Program/Academic-Year) and its ordered Periods left-joined to their linked Cycle in one bounded query, and aggregates population/completed/started/Candidate/identified counts grouped by Cycle id in one additional bounded query each (never one query per Period). Point status uses the exact verified lifecycle values: a cancelled Period is `no_data`/`cancelled_period`; a missing linked Cycle is `no_data`/`missing_cycle`; a linked Draft Cycle is

no_data/cycle_not_authoritative; an authoritative (Open/Closed) Cycle with zero frozen population is no_data/no_frozen_population; a Program with no Plan yields an empty, non-fabricated points/comparisons series. An unlinked/ad-hoc Cycle is structurally excluded because it is only ever looked up by Period-linked cycle_id. Every Cell/component uses relationships=() - there is no same-point or cross-time additive Relationship anywhere in this module, so no delta/change/ percent_change field is computed or exposed; rates reuse derive_exact_rate/project_safe_derived_payload unchanged. Adjacent-Period comparisons are comparable/not_comparable only, evaluated in the ADR's governed precedence (missing/cancelled/no-population reason, then privacy_protected, then framework_changed, then comparable); the five Framework-independent metrics stay comparable across a Framework change, while the four Candidate/Identification metrics become not_comparable/framework_changed. Candidate/Identification SQL is skipped entirely unless both the selected metric requires it and the actor holds the corresponding secondary permission. Breadth is enforced (Period count, 1/2 components, prospective_pair_count = periods-1, relationship_estimate=0) before the aggregate queries run. B10-B is implemented and unit/integration tested. Independent security/privacy review passed with non-blocking observations; B10 is CLOSED. B11, B12 (closeout), and every ADR 0027 "Deferred capabilities"/"Production gates" item remain unimplemented/open.

B11-A (read-only qualification) is completed: it confirmed the privacy, commercial-availability, and breadth-policy provider seams used by all 7 M10 routes existed with no production implementation and correctly failed closed at that checkpoint. F1 subsequently implements the governed providers. B11-B adds safe, non-sensitive operational observability around all 7 existing routes via a new dedicated logger in talent_organization_analytics_observability.py (separate from the immutable business/security audit trail in audit.py) plus telemetry wiring in routers/talent_organization_analytics.py. Recorded signals are a bounded allowlist only - projection_family, outcome, route latency, safe structural counts reused from each route's own existing breadth-policy inputs, and each provider's coarse outcome (missing/available/unavailable/ exception for availability; missing/evaluated/exception for privacy; missing/allowed/rejected/exception for breadth) - never a Student identifier, raw analytical value, privacy threshold, suppressed value, Candidate/Identification decision, or delta/change/percent_change/ total_count. Every existing fail-closed HTTP outcome is unchanged byte-for-byte, and a telemetry emission failure is swallowed internally and can never affect the HTTP response. Query-count instrumentation is explicitly deferred to a future "B11-C profiling" phase. B11-B is implemented and unit-tested (tests/test_talent_organization_observability.py, 22 tests, full existing B5-B10 regression suites green). Independent security/privacy review passed with non-blocking observations; B11-B is CLOSED. No schema, migration, permission, entitlement, privacy threshold, or production breadth limit was added by B11-B; F1 later adds the governed provider decisions without schema or permission changes. PostgreSQL performance/concurrency/ memory qualification was subsequently gathered. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

The pre-B2 hardening gate is implemented: metric, measure_component, and membership_grain use the approved closed string-backed enum vocabularies and serialize to their exact governed strings; unknown values are rejected. rate/percentage remain derived disclosures, not identity components. Relationship coefficients require an actual Python int and exactly +1/-1, so bool, float, Fraction, Decimal, null, and strings fail.

M9 Deterministic Talent Analytics Implemented (Committed/Pushed On dev)

M9 adds a read-only, privacy-gated /api/talent/analytics aggregate API (context/overview/rubric-distribution/kpi-distribution/competencies/ breakdowns per branch-grade-section/period-comparison/students) strictly inside one Program + one Academic Year, over the existing frozen TalentAssessmentCyclePopulationMember historical scope. See "Deterministic Talent Analytics (M9, Committed/Pushed On dev)" in docs/AI_PROJECT_CONTEXT.md for full architecture: the Cell/Group privacy-pipeline model, the mandatory apply_primary_privacy -> run_complementary_suppression ordering (now statically regression-guarded), the Product-Owner-approved fail-closed coarsened contract, the enforced

`competency_id` filter, the documented dead/unreachable `candidate_policy_changed` comparability branch, and the coverage-bundle privacy fix below.

A second remediation pass fixed a HIGH-severity defect of the same reconstruction-risk bug class, relocated past the call-ordering guard: `/rubric-distribution` and `/period-comparison` each privacy-evaluated only one headline `frozen_eligible` `Cell` and then serialized the entire raw per-status coverage bundle unconditionally once that one Cell passed - `/rubric-distribution` did not evaluate the coverage bundle at all, and `/period-comparison`'s per-side bundle only gated on the total. Both now call one shared, `/overview`-pattern-derived authority, `talent_analytics_service.build_privacy_safe_coverage_bundle`, which independently privacy-evaluates `frozen_eligible` and all 5 per-status counts as one `Group`, runs complementary suppression across them, and only publishes the full per-status breakdown (with derived `assessment_started` and percentages) when every cell in the group is independently visible - otherwise the whole bundle collapses to a compact state-only projection with no raw sibling field. A recursive runtime regression test proves no non-visible-state dict anywhere in the serialized response carries a raw coverage-shaped sibling field, in addition to the existing static ordering guard.

New Administrator-only default permissions are `talent_analytics.view` and `talent_analytics.view_students`. Production privacy-provider construction is configuration-driven through `talent_organization_analytics_providers.py`. The provider is returned only when the deployment's governed environment values match the approved Release 1 policy values; missing or invalid configuration fails closed. Live PostgreSQL performance/concurrency validation remains a separate release gate. Focused coverage is `tests/test_talent_analytics.py`. M9 is committed and pushed on `dev` at `23ade9a7c6166197140b48a3edbfac849396d580` (commit `feat: add deterministic talent analytics`). It is not deployed, not production- released, and not merged to `master`. No Talent Score/Index/Potential Rate, Talent Map, export, AI, or UI exists. M10 B0-B5 exists as described above, while other M10 aggregate routes, Learning Style, and Student evidence remain out of scope.

M8 Annual Evaluation Plan And Periods Implemented

Migration `20260905_001_talent_annual_evaluation_plan_period_foundation` adds annual Plan and ordered Period tables, the scoped Program/year/config keys, and nullable unique Period linkage on Assessment Cycle. Existing Cycles remain ad-hoc with NULL linkage; no history is inferred. Services and `/api/talent` routes implement create/read, Period planning, activation, cancellation, closure preflight/close, rollover, eligible Period selection, and atomic link/unlink. Plan revision and Cycle revision protect stale writes.

New Administrator-only default permissions are `talent_evaluation_plans.view/manage/govern`. Every mutation is organization/ global only. Link/unlink and rollover use true AND permission composition. Branch reads never receive Cycle projection or derived relationship/action/ warning leakage without the separate Cycle-view permission. Linked Cycle Open revalidation follows Plan -> Period -> Cycle lock order without adding an M8 govern requirement. M9 Deterministic Talent Analytics is now implemented (see above); M10 B0-B5 exists, while further external execution, Learning Style, AI, and Student data remain out of scope. Live PostgreSQL concurrency validation remains a deployment gate.

M7 Longitudinal Learner Profile Implemented

M7 implements a read-only, no-migration Learner Profile aggregation over M1-M6 canonical history. The profile groups exact historical Assessments, Framework Versions, Competency Results, optional persisted KPI results, Review Candidates, and Official Identifications by stable Program and Academic Year while retaining multiple Cycles. It uses current Student identity only for the header and never reinterprets historical Talent semantics from current Placement or configuration. `talent_learner_profiles.view` is a new Administrator-only-by-default base permission. Review Candidate, Official Identification, and Educator Input each remain absent, including their timeline events and metadata, without their own respective view permission. Branch scope filters each Placement by stored historical Branch and each Talent record by frozen/persisted Branch; organization/global scope can see complete same-tenant history. The related M1 direct placement read routes now enforce the same historical Branch policy. M8 Annual Evaluation Plan & Periods,

Learning Style, analytics, Talent Map, AI, and profile materialization remain deferred. Live PostgreSQL validation remains outstanding.

M6 Review, Official Identification & Educator Input Implemented

M6 is complete following 18 approved Product Owner governance decisions that resolved the 9 previously open questions (see "M6 Governance Review: Decisions" in `docs/AI_PROJECT_CONTEXT.md`). Migration `20260904_006_talent_review_candidate_foundation` adds `talent_review_candidates` (one row per Assessment) and widens `TalentAssessmentAudit.resource_type` to add `review_candidate`. Migration `20260904_007_talent_review_workflow_identification_educator_input_foundation` adds the two-state Review workflow columns to `talent_review_candidates`, creates `talent_official_identifications` and `talent_educator_inputs`, widens `resource_type` further to add `review_candidate_review`, `official_identification`, and `educator_input`, and relaxes `talent_assessment_audits.cycle_id/framework_version_id` to nullable so an Educator Input audit row (whose Cycle binding is optional) can exist without a Cycle context.

`talent_review_candidate_service.py` deterministically evaluates the exact M3 Review Candidate Policy/rules attached to a Completed Assessment's exact Framework Version - `rubric_level_at_or_above` by `TalentRubricLevel.display_order` (never `numeric_value`), `kpi_at_or_above` by the Assessment's persisted `kpi_result` (never recomputed) - with `all`/`any` composition. No policy means no candidate is inferred. Evaluation only reads existing Assessment/ Result/frozen-population data; it never writes to them. A qualifying evaluation persists one candidate row (starting `pending_review`) with policy identity, match mode, a deterministic SHA-256 fingerprint, and a full evaluation snapshot; re-evaluation is idempotent and returns the existing row unchanged. A non-qualifying evaluation still persists no `TalentReviewCandidate` row, but is now structurally audited (assessment identity, Framework/Policy context, `outcome=false`, fingerprint - no free text, no durable negative entity).

The Review workflow is exactly two states, `pending_review` -> `reviewed`, one-way, via `talent_review_candidates.manage`; it never alters assessment evidence and never auto-identifies. `TalentOfficialIdentification` is a new append-only decision record (`identified`/`not_identified` only - no `deferred`/`revoked`/`superseded`/`re-identified`) that may be recorded only once its Review Candidate is `reviewed`, exactly one per candidate (unique-constraint- and service-enforced), via the dedicated `talent_official_identifications.record` permission which additionally requires organization/global access scope - a Branch-scoped actor is denied even if granted the permission. `not_identified` is exactly as durable as `identified`; there is no mutation/revocation/second-decision/re-identification path anywhere. `TalentEducatorInput` is bounded qualitative Student evidence/context (`observation`/`context`/`supporting_evidence`, <=2000 characters, non-empty) bound to SchoolGroup/Student/Program/AcademicYear/ `observed_at` plus a historical Placement/Branch snapshot - resolved from a supplied frozen Cycle Population Member when given, otherwise from the Student's canonical `StudentAcademicPlacement` effective at `observed_at` (rejecting cleanly with no valid historical Placement, never falling back to current Placement). It is append-only with `supersedes_educator_input_id` lineage (cycle-prevention mirrors M3's Framework Version `_validate_supersedes` walk); default reads return only the current/latest version, an explicit history read returns the full chain, and the free-text body is stripped from audit `before_json`/`after_json`. New dedicated Administrator-only-by-default permission groups: `talent_official_identifications.view/record` and `talent_educator_inputs.view/add/amend`, each independent of every other Talent permission family. All new/changed reads use the same frozen historical Branch discipline as M4/M5 - a current Student transfer never reinterprets Review Candidate, Official Identification, or Educator Input access.

Deferred (explicit, not implemented): Official Identification revocation/ supersession/second decision/re-identification, a generic review-note/case- management system, assessor assignment, Learner Profile, Development and Support, analytics/Talent Map, AI/AI Suggested Review, and Educator Input analytics/export/AI/attachments. Focused coverage is in `tests/test_talent_review_candidate_foundation.py` and

`tests/test_talent_review_official_identification_educator_input.py`. Live PostgreSQL execution remains a follow-up.

Independent review additionally closed four boundary defects: historical Branch authorization is enforced during Educator Input create/amend; out-of-scope direct IDs are uniformly non-enumerating; Official Identification is composite-bound to the Review Candidate's exact historical context; and a database uniqueness constraint prevents concurrent amendment forks. Focused M6 coverage is 42 passing tests, and the combined M1-M6/permission/tenant/migration/startup selection is 137 passed and 3 PostgreSQL-only skips.

M5 Talent Student Assessment And Competency Results Implemented

Migration `20260904_005_talent_student_assessment_competency_results` adds canonical Student Assessment and exact Framework Competency Result rows, extends the existing operational audit, and preserves the scoped composite keys needed for frozen-member, assessment, competency, and rubric-level integrity. Only an Open Cycle's frozen members can receive one In Progress assessment; all mutations use expected-revision protection. Completed requires every exact Framework Competency result and becomes read-only. Incomplete and Insufficient Evidence are distinct read-only terminal outcomes and do not represent low performance or zero results.

The optional enabled Framework KPI is calculated only at successful completion using exact integer weighted-level arithmetic: numeric rubric value times basis-point component weight, summed over denominator 10,000 and rounded to the nearest Framework-scale integer with ROUND_HALF_UP. The persisted result includes method, scale bounds, numerator, canonical SHA-256 input fingerprint, and calculation timestamp. Qualitative no-KPI Programs complete without any numeric score. This is never a universal Talent Score or cross-Program normalization. M5 adds Administrator-only-by-default `talent_assessments.view/manage/complete`, frozen-Branch scope enforcement, the `/api/talent/assessments` API, and focused SQLite coverage. Assessor assignment, correction/reopen, Review Candidate workflow, Official Identification, Educator Input, Learner Profile, analytics/Talent Map, AI, and Development and Support remain deferred. Live PostgreSQL validation remains outstanding.

M4 Talent Assessment Cycle And Frozen Population Implemented

Migration `20260904_004_talent_assessment_cycle_frozen_population` adds the SchoolGroup-wide `TalentAssessmentCycle` (`draft -> open -> closed`, no `branch_id`), frozen population members, and bounded append-only operational audit. Draft preview and Open resolve canonical Academic Placement at the explicit `population_effective_at` against the Program Academic Year configuration's eligible grades. Open atomically freezes the full organization population with exact Placement/Academic Year/Branch/grade/section context and stores a deterministic full count and fingerprint. There is no reopen, population removal, or assessor assignment. Accepted ADR 0033 permits only audited additive synchronization of newly eligible Students while Open.

Dedicated `talent_assessment_cycles.view/manage/view_population/govern` permissions are Administrator-only by default. Branch authors may manage Draft metadata but cannot Open/Close. Governance additionally requires organization/global scope. Population reads are separately permissioned: Branch actors see only authorized preview/frozen Branch members and subset counts, never the full count/fingerprint; organization/global population readers see the canonical whole. Frozen visibility uses the member's historical Branch, not current Student Placement. Focused coverage is in `tests/test_talent_assessment_cycle_frozen_population.py`.

Student Assessment, assessor assignment, results, Review Candidate instances, Official Identification, analytics, AI, late population exceptions, and UI remain deferred. Live PostgreSQL execution remains outstanding.

M3 Governance Closure: KPI Approval, Rubric Ordering, And Audit Granularity

`weighted_level_average` is now Product-Owner-approved as one bounded, optional, Framework-specific KPI calculation primitive - never a universal or cross-Program score, enabled only through explicit Framework configuration, and requiring numeric rubric values only while enabled. Qualitative no-KPI Programs remain first-class.

rubric_level_at_or_above candidate rule semantics are defined by the configured Rubric Level order (display_order, lowest proficiency first), independent of numeric_value; display_order was confirmed to safely carry both presentation order and semantic proficiency rank with no divergence risk, so no new field was added. TalentConfigurationAudit.resource_type was widened (model CHECK plus a widening step inside the M3 migration, since the audit table itself was created by the separate already-applied M2 migration) to add rubric, rubric_level, rubric_descriptor, kpi_configuration, kpi_component, review_candidate_policy, and review_candidate_rule; every M3 mutation now audits its specific child resource instead of only framework_version, matching the M2 framework_competency precedent. Candidate-policy rule evaluation remains deferred to a later assessment milestone. PostgreSQL live validation remains outstanding.

Talent Program Deterministic Framework Configuration Implemented Through M3

Migration 20260904_003_talent_rubric_kpi_candidate_policy_foundation adds Framework-versioned configurable rubric levels, exact competency/level descriptors, optional bounded weighted-level KPI configuration, and a bounded declarative Review Candidate Policy. All configuration is Draft-only, expected-revision protected, included in the Framework semantic fingerprint, independently cloned, tenant-scoped by composite foreign keys, and recorded through the existing append-only Talent configuration audit. Active/Retired configuration is immutable. Qualitative Programs can omit KPI and numeric levels entirely. Existing talent_programs.view/manage/govern permissions are reused; governance scope for activation/retirement is unchanged.

No Student Assessment, evidence/result, candidate instance, Official Identification, analytics, AI, entitlement, or UI capability is part of M3. Focused M3 coverage is in tests/test_talent_rubric_kpi_candidate_policy.py.

Student & Academic Placement Foundation And Talent Program & Framework Foundation Implemented

Migration 20260904_001_student_academic_placement_foundation adds Student as a canonical SchoolGroup-owned identity distinct from User, external identifiers, an effective-dated Academic Placement authority (no separate Enrollment entity), and append-only audit. Migration 20260904_002_talent_program_framework_foundation adds TalentProgram as a durable organization-owned identity distinct from Subject, annual configuration, an immutable-once-active Framework Version lifecycle with deterministic version allocation and one-Active-Framework-per-Program supersession, competency lineage with version-specific framework membership snapshots, and append-only configuration audit. Full architecture and business-rule detail is recorded in docs/AI_PROJECT_CONTEXT.md under "Student & Academic Placement Foundation" and "Talent Program & Framework Foundation". tests/test_student_academic_foundation.py (13 tests) and tests/test_talent_program_framework_foundation.py (15 tests) cover both foundations, including tenant/branch isolation and permission-boundary cases.

Checkpoint A closed the permission-governance gap in both interim implementations. permission_registry.py adds dedicated students (.view, .create, .edit, .activate_deactivate, .manage_identifiers, .manage_placements) and talent_programs (.view, .manage, .govern) permission groups, replacing the temporary reuse of the users.* keys (for Student) and the bare, broadly-granted planning.edit_section permission (for both Student Academic Placement mutation and Talent Program/Framework Draft authorship - the latter previously let any Branch-scoped Editor/User actor author organization-wide Talent configuration with no Branch-relation constraint). Both new groups are Administrator-only by default, matching the existing users.* precedent; the M2 organization/global-scope gate on Framework activate/retire and Program lifecycle transitions was already correct and is unchanged. routers/students.py and routers/talent_programs.py now check the dedicated keys; branch-scope-after-permission-check ordering and non-enumerating 404/403 discipline are preserved on every route.

PostgreSQL validation (constraints, concurrent placement/framework writes) has not been executed against live PostgreSQL for these foundations; only SQLite-backed pytest coverage exists. No Student UI, import/merge, Assessment, Review/Identification, Learner Profile, analytics/Talent Map, or AI code exists yet at this point in time. (The `students.import`/ `students.export` permission keys were later registered as a governance prerequisite only - see "Student Roster Import/Export - Permission Registration (Governance Prerequisite, M6)" above - roster import/merge functionality itself remained, and remains, unimplemented.)

Planning Subject Requirement Removal (Single And Bulk) Implemented

Administrators can now remove a Planning subject requirement whether or not it has an explicit `PlanningSubjectDemand` row. Previously "Remove demand" only appeared for a requirement with an untouched explicit row, so a purely legacy-fallback requirement (resolved only from the Subject catalog, with no explicit row at all) silently had no removal action - the same teacher could have some subjects removable and others not, for reasons unrelated to the teacher. `routers/planning.py` classifies every requirement as `removable` (untouched explicit row), `permanent` (Curriculum-Adjustment-touched, shown with an explicit "Protected" explanation instead of a hidden action), `fallback` (no explicit row - now removable by creating an explicit `is_active=False/weekly_periods=0` suppression row; unlike a Curriculum Adjustment retirement, this row sets only `created_by_user_id` and leaves `updated_by_user_id` NULL, so it stays classified `removable`/setup-only rather than immediately becoming permanent), or `not_found`.

The customer-facing action is "Remove Subject Requirement" (single, per row) and a checkbox-selected removal action at `POST /planning/subject-requirements/remove`. Bulk removal is atomic: every selected target is validated and scope-checked first, and if any is protected, invalid, or out of scope, nothing is removed and every blocked target is named with its reason. Confirmation dialogs precede both single and bulk removal. Each expanded section now places **Select all** beside a trash-icon action whose label changes to the singular or exact selected count; the page-level action mirrors that selection state. Selection can still span multiple sections in one request. Teacher assignments, timetable data, and Curriculum Adjustment audit records are never touched by this action. The existing `GET /planning/subject-demand/delete/{demand_id}` route from the prior round remains available, superseded in the rendered UI, for clearing an active untouched setup row. An inactive zero-period setup suppression with no Curriculum Adjustment history no longer blocks an otherwise-empty section: deletion removes that exact scoped artifact and the section atomically. Active demands, permanent demand history, assignments, timetable/calendar/rule dependencies, and all other existing guards still block.

Cline And Three-Agent Development Pool

Codex, Claude Code, and Cline/ClinePass are approved TIS development agents. Cline's repository entry points are `.clinerules/tis.md` and `.cline/skills/tis-kms-developer/SKILL.md`. They require governing KMS onboarding, implementation inspection, isolation/RBAC and database safety, minimal diffs, proportional tests, task-level KIA, KMS sync/check, and deployment reporting.

The master conversation balances tasks by complexity, specialization, risk, independent-review value, and current usage/load, without permanent assignments. See [Project Governance](engineering/PROJECT_GOVERNANCE.md) for the allocation policy and [AI Coding Workflow](engineering/AI_CODING_WORKFLOW.md) for Cline setup/invocation. This configuration changes no application, schema, or production behavior.

Claude Code KMS Integration

The repository now includes root `CLAUDE.md`, a reusable project skill at `.claude/skills/tis-kms/SKILL.md`, and a specialized subagent at `.claude/agents/tis-kms-developer.md`. This gives Claude Code a native entry point to the same KMS onboarding, scope safeguards, worktree preservation, validation, and completion reporting already required by `AGENTS.md`; it does not change application, database, deployment, or customer behavior.

Return-To Reopen-On-Load UI Pattern Implemented

A shared `redirect_utils.py` module (`safe_redirect_path`, `redirect_with_notice`, `redirect_with_error`, moved out of `main.py` with existing callers unchanged) and a shared `static/js/reopen-on-load.js` (loaded globally from `base.html`) finish wiring the existing `return_to` convention so a server-rendered action inside an expanded element can land the user back on that same element, scrolled into view, instead of collapsing it. Planning's "Remove demand" action is the first and, for now, only consumer: each section's `<details>` carries a stable id, the link passes `return_to` to that section's fragment, and the route's in-place blocked-render outcomes set the same reopen target without a redirect. No SPA conversion, no new browser storage. Subjects and Teachers were not changed; the pattern is available for them to adopt later.

Planning And Subject Delete Dependency Guards Implemented

Planning section delete and Subject delete/Bulk Delete now run a read-only dependency check before any mutation, naming every blocking category in a customer-safe message instead of raising an unhandled `IntegrityError`. Planning section delete checks `TeacherSectionAssignment`, `PlanningSubjectDemand`, `TimetableEntry`, `TeacherSchedulingRuleTarget`, `CalendarEvent`/`CalendarEventSectionTarget`, and section-scoped `SubjectDistributionRule`; `TeacherSectionAssignment` is now a blocker instead of being auto-deleted. Subject delete/Bulk Delete additionally check `TimetableEntry`, `CurriculumAdjustmentAudit`, and `SubjectDistributionRule` subject-code references that previously had no application-level check. Bulk Subject delete is atomic: any blocked Subject stops the whole batch and is named with its reason.

Planning subject demand is classified removable or permanent using the existing `updated_by_user_id` column: Curriculum Adjustment always sets it, the one-time setup backfill never does. A never-touched row is pure setup scaffolding and can now be hard-deleted through the new "Remove demand" action on the Planning page (`GET /planning/subject-demand/delete/{demand_id}`), after which the section or subject becomes deletable; a row Curriculum Adjustment has ever touched remains a permanent, honestly-explained blocker. Published/archived timetable placements follow the same principle without a new action. No archive/closed/inactive lifecycle, broad schema change, migration, or cascade deletion was introduced.

Professional Timetable Lifecycle UX Implemented

The workspace now identifies CURRENT PUBLISHED, editable working Drafts, and read-only history with visible Draft ? Validate ? Approve ? Publish guidance and audit facts. Safe actions distinguish copying published/history into a working Draft from starting an empty Draft, and Generate/Regenerate language makes Draft-only effects explicit. The active pointer remains unchanged until approval and publication succeed. Mutation/version errors remain backward-compatible and now add canonical conflict evidence plus stale-revision refresh remediation. No schema, solver, or Workflow behavior changed.

Guided Curriculum Adjustment UI Implemented

Stage 5 adds the permission-gated administrator workflow from Planning and Subjects through scope, change, preview, teacher decisions, final confirmation, and success. It reuses the deployed Stage 3 preview and Stage 4 atomic apply APIs, preserves selections where safe after a refreshed review, and offers timetable/regeneration navigation without automatic generation.

The transfer contract now carries the requested period count instead of deriving an absolute source result from `Subject.weekly_hours`. Subjects also reflects effective Planning weekly periods for Current/New sections, including a section breakdown when values differ.

Curriculum Adjustment now also supports reduction without transfer. Subject Details exposes effective Planning periods and a permissioned prefilled reduction entry point; original catalog periods remain separately editable defaults and are not operational Planning authority.

Atomic Curriculum Adjustment Apply Implemented

Stage 4 implements a service-owned, one-commit curriculum adjustment transaction behind dedicated `curriculum.adjust`. It revalidates the reviewed preview fingerprint under exact-scope locks, rejects active timetable generation and all unresolved teacher/rule/lock conflicts, applies section-specific source/target demand and the explicit teacher decision, retires zero demand and its section rule, marks the unpublished Draft stale, clears approval, and records a durable audit.

Migration `20260829_001_curriculum_adjustment_apply_foundation` adds the audit ledger and duplicate-fingerprint guard. No guided UI or automatic regeneration is included; published timetable versions and the active pointer are untouched.

Curriculum Adjustment Preview Implemented

Stage 3 provides a read-only curriculum adjustment preview service and `POST /planning/curriculum-adjustments/preview`. Grade, selected-section, and all- active-use scopes are exact-tenant and Current/New only. Preview output includes per-section demand transfer, current and suggested teachers, capacity projections, scheduling-rule and grouped-configuration impacts, blockers/warnings, Draft stale/regeneration impact, and a deterministic stale-confirmation fingerprint.

No demand, teacher assignment, timetable version, placement, active pointer, or published history is changed. Apply, teacher confirmation, atomic write, and regeneration workflows remain future stages.

Explicit Planning Subject Demand Foundation Implemented

Migration `20260828_004_planning_subject_demands_foundation` adds normalized `planning_subject_demands` records for section-subject weekly periods. Current/New Planning sections are backfilled from grade-matched Subjects in the same branch and academic year. The operation is idempotent; composite scope foreign keys and an active-row partial unique index protect integrity while allowing retired history.

`planning_subject_demand_service.py` provides exact-scope explicit-first resolution and legacy fallback, including authoritative inactive rows that suppress fallback. Stage 2 now routes Planning calculations, teacher workload, timetable readiness, workspace, snapshots and generation inputs, Subject Scheduling Rule arithmetic, and required-hours reports through that authority. Missing explicit rows retain legacy fallback during transition. Published timetable rows are untouched.

Central Tenant Report Branding Implemented

Tenant-facing operational exports now resolve logo assets through `tenant_report_branding.py`, which delegates to the existing branch/organization- scoped branding slots and returns configured tenant logos only. Timetable PDF/XLSX, academic-calendar PDF, and observation PDF generation share this authority. When no tenant logo is configured, exports retain their normal report title/header but render no logo; they never substitute the TIS product mark. Application UI, login/product, public/platform, and internal platform-owner branding are unchanged.

Subject Scheduling Rules UI Implemented

The Timetable Settings page presents a grade-first Subject Scheduling Rules table sourced automatically from Planning. `planning_scope_service.py` supplies one branch/year-scoped Current/New PlanningSection source to its main/copy grade selectors, section overrides, the timetable workspace section filters, and directly related calendar/grouped section choices. Global, non-operational placeholder, and Subject-catalog-only grades are excluded. Only the selected grade's compact Subject, read-only Weekly, Current Pattern, Status, and Edit columns are shown, with optional subject search. The modal uses native closed-dialog behavior on page load and opens only after Edit populates the selected Subject + Grade rule; Cancel/Close closes it. Its compact two-column Session Structure aligns double-block/single-session steppers above a full-width automatic total, while conditions use a balanced two-column grid. Two-period presets and progressively disclosed advanced settings remain. A Section Overrides list preserving Stage 1 section-over-grade-over-branch-default precedence, Copy Rules From Grade (name-matched across grades,

skipping arithmetic mismatches), and Reset To Default (removes only the grade-level row). New routes `/system-configuration/timetable-settings/subject-rules` (save/reset/clear-override/copy) reuse the existing `timetable.manage_settings` permission, remain tenant/branch/year-scoped, and never mutate Planning weekly totals. `subject_distribution_rules_ui.py` re-validates arithmetic and feasibility authoritatively before any write. Existing legacy subject-code mappings and Swimming/grouped JSON remain unchanged but are collapsed inside an Advanced / Legacy area; Non-Teaching Blocks management is unchanged. Newly saved/updated rules take effect the next time a Draft Timetable snapshot is created, through the unchanged Stage 2 resolution/snapshot pipeline; no already-created snapshot is altered.

Subject Distribution Rules Generation Wiring Implemented

The resolved Subject Distribution Rule (section over grade over branch default, `None` for legacy fallback) is now embedded per Planning demand in the schema-v3 snapshot at creation time; a later rule change never alters an already-created snapshot. The problem builder carries the resolved rule per demand, exposes true physical period adjacency per slot, and runs the arithmetic/feasibility validator as a final pre-solve defense (`distribution_rule_invalid`). CP-SAT now models explicit double-block starts and single sessions, channels every occupied period to exactly one session, and enforces the configured block and single counts exactly. Separate sessions may touch (including double-plus-single three-period runs and two-double four-period runs), while physical interruptions break doubles and overlapping session membership remains impossible. Redundant daily session/load channeling strengthens daily-coverage propagation. CP-SAT also enforces generalized daily-coverage/spread/max-per-day/ min-teaching-days behavior (hard only when explicitly configured hard), and exempts declared blocks from the consecutive-avoidance penalty; demands without a resolved rule keep the exact legacy `quality_rules_json` behavior. The independent validator mirrors every new hard check. Readiness blocks genuinely invalid/infeasible normalized configurations while keeping the existing legacy warning path. Regeneration's `ceil(25% x unlocked placements)` default, locks, and teacher authority are unchanged; hard distribution rules remain enforced during regenerate through the same constraint-building path.

Subject Distribution Rules Foundation Implemented

Migration `20260828_003_subject_distribution_rules_foundation` adds the normalized `subject_distribution_rules` table (branch/grade/section scope, block/single counts, min teaching days, max periods per day, daily-coverage/spread/consecutive preferences, min day gap, hard/soft strictness). `subject_distribution_rules.py` resolves section-then-grade-then-branch-default precedence with field-level inheritance and returns `None` for legacy fallback when nothing is configured. `subject_distribution_validator.py` independently checks the Planning-weekly-total arithmetic and day/period feasibility. `timetable_slot_service.py` now marks true physical period adjacency (false across a Break, Prayer, or other non-teaching item) so a later intentional-block feature can rely on genuine continuity. No existing solver, validator, readiness, or settings-UI behavior changed, and `quality_rules_json` remains the active authority for every tenant.

Smart Timetable Academic Scheduling Quality Implemented

Timetable Settings persist branch/year subject-code mappings and grouped Swimming configuration through migration `20260828_002_smart_timetable_academic_quality_rules`. Immutable snapshots, problem construction, CP-SAT, and independent validation share the normalized authority for core daily coverage, short-demand spread, ICT daily limits, grouped synchronization, teacher/resource safety, and configurable regeneration diversity.

On-Demand Timetable Generation Workflow Implemented

Generate and Regenerate still create the Stage 5.1 durable PostgreSQL run and immutable snapshot, then the web service dispatches only its public ID to a registered Render Workflow task. The task reuses the single-run solver/validator/persistence pipeline, keeps leases, heartbeats, cancellation, staleness, and atomic result guards, and

exits. Duplicate task starts are no-ops after claim or completion. Immediate dispatch failure safely terminates an unclaimed run and delayed start receives an honest waiting message. Render retries are explicitly disabled; Generate Again is the retry authority. No schema or solver behavior changed, and the polling worker is optional/local only rather than a production service.

Smart Timetable Stage 5.2 Simplified Workflow Implemented

The manager workspace now emphasizes Create New Timetable, one current Draft Timetable, readiness, Generate or Regenerate, Approve Draft, and Publish Timetable. Generation creates an unapproved draft. Approval records the reviewing administrator only after exact-version validation, every content or authority change invalidates approval, and publication revalidates before changing the active pointer. Regenerate, Approve Draft, and Publish Timetable now occupy one primary Draft workflow area; History retains destructive and technical version actions. Technical lifecycle controls and permanent unpublished cleanup are secondary Timetable History features. Draft deletion uses `timetable.delete_versions` and permanently removes eligible never-published versions without moving the active pointer; historical archive remains a separate History-only action.

Published-only `/my-timetable` and `/published-timetable` views resolve exclusively from `TimetableActiveVersion`. My Timetable fails closed without exact-scope teacher identity and filters to that teacher. `timetable.view`-only users are redirected away from management. Migration `20260828_001_smart_timetable_stage52_draft_approval` adds nullable approval timestamp and actor provenance; solver behavior is unchanged.

Smart Timetable Stage 5.1 Generator Implemented

Authorized administrators can queue Generate or Regenerate from structurally complete current inputs. A stale existing Draft is reported as `stale_input` / Draft Needs Regeneration without becoming configuration-incomplete: it may run a fresh feasibility verification, then regenerate only after the current fingerprint is verified. The stale source remains versioned and unpublishable, and obsolete-run messages from older authority fingerprints do not replace the current state. Regeneration eligibility is based on the current Draft's lifecycle and populated placement arrangement rather than creation origin: eligible manual, generated, and regenerated Drafts regenerate, while empty starters generate and active or historical versions remain excluded. A durable hard-only feasibility Workflow uses the same immutable snapshot/problem builder and independent validator before full optimization. Its validated solution is cached in `timetable_feasibility_verifications` by exact tenant scope and full input fingerprint; only Feasibility Verified enables generation. Full CP-SAT optimization uses that solution as a hint and validated timeout fallback, while any authority fingerprint change invalidates reuse. A separate OR-Tools CP-SAT execution environment uses immutable schema-v3 input, durable lease/heartbeat/recovery state, exact demand and collision constraints, Planning/HRT teacher authority, canonical teaching slots, and fixed locks. A solver-independent validator and final fingerprint/source-revision check gate atomic creation of one unpublished `publication_ready` version. Regeneration keeps its source unchanged and enforces the approved unlocked-lesson difference. The default regeneration difference is 25 percent of unlocked source placements, rounded up. Infeasible diversity is reported explicitly without weakening hard rules. The active/published pointer is never changed by generation.

The Workflow worker now reaches the existing CP-SAT solver and its diagnostic profiles through `timetable_solver.py`, a solver-neutral adapter boundary. CP-SAT remains the only Release 1 production implementation. Problem construction, independent validation, and atomic persistence remain separate authorities.

Planning-to-timetable demand now passes through `timetable_requirement_projection.py`. The read-only projection preserves explicit-first demand, authoritative zero/retirement, legacy fallback only when no explicit row exists, scoped teacher authority, and deterministic internal identity/provenance. Schema-v5 snapshots freeze the projected requirements; schema-v3/v4 generation snapshots remain readable. No new table, editable demand authority, API, or migration was added.

Timetable conflict evidence now passes through `timetable_conflicts.py`. Readiness exposes additive safe conflicts beside its unchanged blocker/warning fields; feasibility exposes them beside unchanged diagnostics;

generation-run payloads derive durable conflicts from existing terminal status/category/message; and the independent validator returns canonical conflicts beside legacy errors. Internal requirement and solver correlation never appears in public payloads. No conflict persistence table or migration was added.

Migration `20260822_001_smart_timetable_stage51_generator` adds progress, attempts, cancellation audit, worker-claim indexing, and one-active-run-per-scope enforcement. The page polls real phases and recovers active work after reload. Stage 5.2 UX and teacher visibility are implemented as documented above.

Smart Timetable Stage 3.5 Composed Timeline Implemented

The timetable now derives every day from one canonical composer: shift start, teaching-period count/duration, and applicable inserted blocks. After-period blocks are first-class; boundary-aligned legacy fixed times remain compatible and ambiguous overlaps block readiness/assignment. Calculated end time, previews, timetable rows, snapshots, staleness fingerprints, and exports share the same projection. Migration `20260821_001_smart_timetable_stage35_composed_timeline` adds placement mode, after-period boundary, and duration metadata without rewriting legacy times. The timeline is now consumed by Stage 5.1; availability and room/resource models remain absent.

Smart Timetable Stage 4 Version Publication Implemented

Administrators can review scoped version history, explicitly open historical versions, copy immutable versions to drafts, lock/unlock draft lessons, validate a specific draft, compare two same-scope versions, archive drafts/superseded history, export the selected version, and publish a complete fresh publication-ready draft.

Publication is one transaction using version row locking, `edit_revision`, active pointer locking/revision, current fingerprint revalidation, and full solver-independent validation. The former active version becomes superseded and remains reviewable/exportable; the new active version is immutable. Stage 3 Ready to Generate remains a separate input-readiness concept. Stage 5.1 now supplies the solver.

Smart Timetable Stage 3 Readiness Implemented

Stage 3 adds one deterministic canonical projection for fixed teaching periods, full-period unavailability, between-period display blocks, and invalid partial overlaps. The projection drives UI availability, assignment validation, snapshots, exports, and the future solver boundary without shifting period times.

`TimetableReadinessService` evaluates one explicit organization/branch/year scope against configuration, Planning demand/allocation, HRT, authoritative teacher capacity, slot sanity, locks, and staleness. Only `generation_ready` is ready, and it means inputs are coherent enough to attempt solving-not that a feasible global timetable is guaranteed. No solver or Generate endpoint exists.

Smart Timetable Stage 2 Version Foundation Implemented

ADR 0026 Stage 2 is implemented. Timetable placements now belong to durable, SchoolGroup/Branch/Academic-Year-scoped versions with deterministic input snapshots, a separate exact-scope active pointer, lock metadata, version-scoped collision guarantees, and a future generation-run record. Migration `20260819_002_smart_timetable_stage2_version_foundation` imports each populated legacy timetable exactly, creates one compatibility active version, records safe staleness evidence without repairing placements, and skips empty settings-only scopes.

The legacy assignment route remains usable through copy-on-write: its first edit creates a mutable working draft from the immutable active version. Current views and XLSX/PDF exports resolve the operational version. Stage 2 adds no solver, worker, Generate/Regenerate endpoint, generation UI, room/resource authority, availability rules, or new teacher-capacity authority. Structural readiness and solver execution remain later stages.

Capacity-Based Packaging Implemented

ADR 0025 is implemented. Nine normal customer entitlement keys are identical for Starter, Professional, and Enterprise AI. Paid, promo, and active demo workspaces resolve those features through one source-aware permission/scope/commercial decision. Advanced Reporting allocation exports now follow this path. The report generators and calculations are unchanged.

Migration `20260819_001_capacity_based_customer_feature_baseline` updates plan values and compatible legacy flags, repairs only currently active promo feature values, and reconciles active demo baseline policy. It does not change capacity, priority support, Paddle/payment evidence, immutable promo records, or expired authority. AI availability is common; demo allowances and future provider limits resolve through a separate consumption-policy boundary.

Stage 5 SchoolGroup Boundary Correction Complete

Operational SchoolGroup create/delete actions now require both Platform identity and global SchoolGroup-management capability. Tenant create/delete permissions were removed from role defaults, direct stale-permission requests fail before side effects, and tenant SchoolGroup updates are constrained to the linked workspace. School Management now presents paid and promo branch capacity from unified commercial authority, while existing locked mutation enforcement is unchanged. The individual last-active-branch rule is SchoolGroup-scoped. A sanitized, read-only PostgreSQL provenance audit is available for later Platform Owner review; no production audit or remediation has been run.

Existing Workspace Controlled Conversion

M4B is implemented. Migration `20260806_001_existing_workspace_controlled_conversion` adds the conversion operation/event ledger, tenant-profile legal name, append-only event guards, and a partial unique tenant-owner link constraint with duplicate preflight. The generic dry-run-first CLI requires exact workspace, owner, M4A hash, operation, idempotency, and actor parameters; write preparation and final conversion require explicit phrases and PostgreSQL.

The normal TIS registration and email-verification flow establishes the owner identity. A verified owner then claims the prepared operation and completes only legal name, controlled IANA timezone, and educational program. Final conversion re-audits under row locks, detects branch/dependency or commercial drift, preserves every branch and operational record, ends internal sandbox authority, and leaves the customer workspace in coherent `activation_required` state with no active entitlement or tenant source. Organization Account and M3 promo activation remain available; direct operations and existing-workspace Paddle activation remain unavailable. No production conversion has been run.

Existing Workspace Conversion Audit Foundation

M4A is implemented as a read-only prerequisite to any legacy internal-sandbox conversion. `saas/existing_workspace_conversion_audit_service.py` resolves an explicit workspace/owner tuple, inventories ownership and commercial evidence, and traverses reflected branch dependencies without modifying the session. `scripts/audit_existing_workspace_conversion.py` requires PostgreSQL, starts a repeatable-read read-only transaction, emits deterministic sanitized JSON or text with a stable SHA-256 snapshot, and always rolls back. Exit codes separate coherent (`0`), execution failure (`1`), manual review (`2`), and workspace identity mismatch (`3`). Reflected model drift, soft-deleted dependencies, owner conflicts, and paid/demo/promo evidence fail closed. Archival candidate IDs are emitted only for active branches proven dependency-free.

No target-specific identifier exists in reusable service or CLI logic. No branch, account, owner, classification, lifecycle, entitlement, tenant link, approval, Paddle, email, or production data is changed. M4A grants neither hard-delete approval nor write-conversion authority; those remain later milestones.

Promo Redemption And Organization Activation

M3 is implemented. Verified organization owners can apply an approved promo from the post-onboarding commercial choice or an eligible existing Organization Account. Activation is resumable and idempotent, uses HMAC lookup without persisting the raw code, and commits the redemption, immutable grant, entitlement, branch assignments, tenant source, lifecycle, and durable audit as one transaction.

Promo-backed workspaces use classification `customer`, lifecycle `active`, and M1 source `promo_grant`. Access is limited to selected active branch entitlements and expires from the persisted grant window. Existing aligned organizations need no `PendingOrganization`; onboarding organizations continue through the shared workspace builder. Staff/teacher excess blocks without mutation. Internal sandboxes, incompatible sources, ambiguous links, and expired grants fail closed. No Paddle or billing object is created.

Promo activation now writes explicit branch evidence for the complete preserved workspace inventory: selected branches are assigned and active, while unselected branches are explicitly inactive even when their operational status is inactive. Individual and bulk branch reactivation enforce grant capacity and update the operational branch, assignment, and entitlement in one transaction. A generic PostgreSQL reconciliation CLI defaults to dry-run and may create only safely attributable missing inactive evidence; contradictory chains remain manual review.

Organization Account Billing & Subscription now selects its presentation from the authoritative commercial source. Promo-backed customers receive a first-class Commercial Access page showing their immutable grant plan and period, safe active or expired status, masked promo reference, and M1-resolved branch, system-user, and teacher capacity. Paid subscription and demo pages remain separate. Operational promo access now ends exactly at `effective_to`; grace days are represented as recovery only and do not authorize tenant operations.

Expired and recovery-period promo workspaces can continue with a paid subscription through M4C existing-workspace activation. Promo authority is unchanged until a verified completed Paddle transaction. Completion reuses the tenant and atomically ends promo entitlement authority, relinks the existing tenant source to the paid contract, establishes paid workspace and branch entitlements, and retains promo history. Active promos cannot convert early. A shared source/plan/status badge now identifies Promo, Demo, and Paid authority in Organization Account and the detailed commercial page. The normal operational header remains commercial-status free.

Promo Code Foundation And Platform Management

M2 is implemented as an additive promo-definition layer. `promo_codes` stores only HMAC lookup authority and masked display fragments, exact capacity, controlled scope, validity, lifecycle, replacement, and governance metadata. `promo_code_branch_restrictions` preserves eligible existing-branch snapshots; `promo_code_audit_events` records allowlisted durable action history. Migration `20260805_001_promo_code_foundation` performs no backfill.

Platform Owners and permission-authorized Developers can use the Promo Codes console. Developers may view, create/edit unused drafts, duplicate, pause, and create replacement definitions; activation and terminal revocation remain owner-only. M3 now consumes approved definitions through a separate customer-redemption service; definition management itself remains isolated from activation, M1 authority, Paddle, onboarding, and tenant operations.

Unified Commercial Access And Capacity Authority

M1 establishes `saas/commercial_authority_service.py` as the only operational capacity facade. It composes workspace classification and lifecycle, WorkspaceEntitlement, TenantProvisioningLink, SubscriptionContract, PaymentSubscription, commercial state/access, demo lifecycle, and the existing plan-capacity and feature-entitlement services. It does not persist a second commercial status or entitlement model. Missing, invalid, ambiguous, or unsupported authority fails closed.

For active paid workspaces, allowed branches are confirmed subscription quantity capped by the plan ceiling; staff and teacher limits come from the confirmed plan. Staff usage includes every distinct active tenant operational User in the SchoolGroup, including an operational organization owner and users whose position is Teacher. Platform users and

account-only SaaS identities do not count. Active teacher people are deduplicated by normalized `Teacher.teacher_id`; blank legacy identities each count separately. A person represented by both records consumes one staff slot and one teacher slot.

Branch, user, teacher, academic-year, and provisioning growth now acquires a tenant row lock, recounts authoritative usage, evaluates the proposed final state, and mutates in the same transaction. Existing over-capacity tenants keep their data and access but cannot further increase an exceeded dimension. Demo and Internal Sandbox authority is explicitly unmetered in M1. No schema, migration, pricing, Paddle, webhook, onboarding-transition, permission, or feature-packaging change is included.

Explicit Organization Billing Identity

Organization Account Billing & Subscription now owns a confirmed billing profile independent from SaaS authentication. The profile stores billing email, legal/billing organization name, contact, optional company/tax identifiers, and the existing supported address structure. Initial Secure Payment confirms this profile; only organization owners or linked users with `subscriptions.manage_billing` may view or update it for an active tenant.

The mapped Paddle customer is reused and synchronized after an authorized billing change. Its active address and one attributable Paddle Business are created or updated and persisted as `PaymentCustomer.provider_address_id` and `PaymentCustomer.provider_business_id`; the active subscription identity and future initial transaction use those mappings. Ambiguous mappings fail closed. Login-email changes never mutate billing email, billing-email changes never mutate authentication, and historical invoices are not silently revised.

Billing Contact now renders saved values read-only until an authorized Edit. Saving commits valid local details even when Paddle cannot be updated. The customer can retry synchronization directly without changing the profile; the retry reuses existing customer/address/business mappings and becomes a no-op after success. Safe server logs identify the failed provider step, error code, and status without rendering diagnostics to customers. A saved profile in pending or failed synchronization state blocks new plan and quantity changes until synchronization succeeds. Cancellation and legacy active subscriptions without a saved profile are not newly blocked.

Provider `paid` is now presented as payment received while processing and is recorded separately from final confirmation. Provider `completed` remains the required reconciliation signal and renders as Paid. No webhook completion rule or subscription authority changed.

Subscription Capacity Presentation And Account Billing Access

Subscription Management and Review Capacity now present paid branch quantity separately from the plan branch ceiling. Additional required branches are identified as additional billed quantity, while system-user and teacher counts remain non-billed plan-eligibility ceilings. Customer plan cards no longer show unapproved feature claims or feature placeholders.

Authorized operational organization owners and explicitly billing-authorized account managers can open the existing Organization Account Billing & Subscription page from System Configuration. The bridge and destination both revalidate tenant/account linkage and permissions. Missing SaaS authentication preserves only the approved internal subscription continuation through sign-in; invalid, unrelated, and external destinations fail closed. The Next.js landing page now labels this centralized customer entry Organization Sign In. Paddle quantity, pricing, entitlement, lifecycle, webhook, and provider-authority behavior are unchanged.

Organization Account Sign-In Routing

Activated organization owners and account-authorized linked users now land on the Organization Account Overview after public/onboarding password or social sign-in, authenticated sign-in restoration, and SaaS root restoration. The overview filters Organization Profile, Branches, Billing & Subscription, and Account & Security by existing ownership and operational permissions. Enter TIS Platform is the only overview action that enters operational login. Restricted

account managers retain billing/recovery access without operational entry; incomplete onboarding resumes; non-management users retain role-based routing; and multi-organization identities select the organization first. No schema, permission semantics, commercial-access decision, or operational login behavior changed. The selected UUID is stored only in an HTTP-only cookie and validated against the account's links before the entitlement resolver uses it.

Initial Secure Payment Recovery

The production-equivalent failure was local and preceded Paddle transaction creation: `_ensure_checkout_launchable()` rejected the legacy `ready_for_checkout` pre-checkout billing state. That eligible state is now prepared safely. Professional Annual remains USD 790 per active branch, and the verified two-branch checkout uses quantity 2 with a USD 1,580 annual total.

Secure Payment retry now repairs eligible unpaid legacy/stale checkout state instead of repeatedly rejecting it. Plan, interval, quantity, selection, and quote changes supersede the old checkout session and unfinished attempt, clear their local authority, and produce a fresh transaction from the current server quote. Superseded transaction webhooks remain fail-closed and cannot create a subscription or workspace.

Paddle customer addresses are reused only when active, customer-matched, and country-compatible. New and reused transaction URLs are released only through the billed automatic-collection transaction contract. Provider failures are logged with status/code/traceback and rendered as one customer-safe retry alert. Pricing, plan limits, branch authority, webhook confirmation, checkout architecture, provisioning, and schema are unchanged.

Organization Profile And Initial Account Setup Correction

Organization Profile now handles approved educational-program values consistently and maps invalid input back to the same page with preserved form values. Pending logos are actual-image validated, limited to 4 MB, assigned an opaque unique filename, written atomically, and cleaned up when a later database step rolls back. Empty upload retains the existing logo, successful replacement removes the obsolete file, and storage/unexpected errors are logged server-side while customers receive a safe inline response.

The fresh verified-account page now presents one compact setup title, one explicit POST start action, and the existing eight-step progress indicator. Its smaller header logo and consolidated content remove the repeated status, action, account/workspace, and guidance panels without changing later onboarding states.

Pending organization logos still use `static/uploads/saas/pending_logos`. This local path is not production-durable on an ephemeral Render filesystem; persistent disk or object storage remains an owner-approved follow-up. No schema, migration, payment, billing, operational-module, or Next.js landing change is included; provisioning changed only to validate and preserve the already-established logo promotion.

The saved pending logo now appears in Organization Profile and the existing School Workspace identity area with the organization name and a neutral placeholder when absent. Existing paid/demo workspace creation promotes the image into the primary `SchoolGroupLogo` slot, whose protected URL is consumed by the operational header and branding settings. The official TIS logo remains separate. Provisioning no longer silently ignores a referenced pending logo whose local file is missing.

Branch Setup totals now come from a Python normalization helper, so placeholder, legacy-null, and mixed-value rows render without Jinja arithmetic errors. Customer saves continue to require explicit non-negative system-user and teacher estimates, and new branches receive zero defaults.

Post-Verification Login Correction

Fixed the fresh-account login destination that previously redirected a successful POST login to the POST-only `/saas/onboarding/start` action. The browser followed that 302 with GET and received raw HTTP 405 JSON. Fresh verified accounts now open `/saas/account`; onboarding creation remains behind the existing explicit POST

action. Continuations are restricted to known GET destinations, and accidental GET navigation to `/saas/auth/login` redirects to the normal sign-in page. Verification, password authentication, subscription intent, preferred-plan behavior, and onboarding business rules are unchanged.

Public Subscription Journey

The Next.js hero Subscribe Now CTA now scrolls to the stable pricing section. Starter, Professional, and Enterprise AI share one CTA presentation and enter public TIS Account registration with an allowlisted preferred-plan code. Registration preserves that preference without creating commercial records. After School Workspace Setup, plan selection reuses the existing three-dimension capacity authority; an invalid, inactive, or undersized preference is cleared and the customer reviews eligible plans. Custom remains mail/contact-only. No plan price, capacity limit, Paddle behavior, or AI entitlement changed.

Per-Branch Subscription Capacity

Implemented required system-user and teacher estimates on each onboarding branch, organization-wide live totals, and one three-dimension capacity decision covering branches, tenant operational staff users, and teachers. The lowest eligible self-service plan is recommended while higher eligible plans remain selectable. Starter limits are 1/5/25, Professional 5/20/100, and Enterprise AI 25/100/500; exceeding any Enterprise limit routes to contact-only Custom.

Before payment, estimates are compared with actual same-workspace counts and the greater value is authoritative. After activation, every distinct active tenant operational User counts as staff regardless of position, and active teacher people count separately. Capacity changes invalidate quote/checkout lineage and clear only an undersized plan. Paid system-user creation/reactivation, teacher creation/year-copy preflight, and downgrades fail before mutation when the result would exceed capacity.

The active customer portal now presents unified Organization Capacity instead of a branch-only action. Review Capacity compares proposed branches, system users, and teachers with the current plan and opens either the existing branch quantity preview or a required plan-upgrade preview. A branch-triggered upgrade may update plan price and branch quantity together; user and teacher totals never change Paddle quantity. Scheduled plan downgrades and quantity reductions revalidate live capacity at the provider-confirmed effective boundary and enter manual review rather than applying an unsafe local reduction.

Customer Journey And Expired-Access Correction

Implemented returning-account state routing, direct expired-demo subscription selection, operational and SaaS expiry pages, and a pre-workspace commercial guard for both demo and paid subscriptions. Demo checkout uses the existing organization, SchoolGroup, and active operational branch quantity and continues into the existing Paddle conversion workflow. The Next.js landing exposes Request a Demo, Subscribe, Sign In, and Open TIS App through `NEXT_PUBLIC_TIS_APP_BASE_URL`. The owner demo page retains all M8B9 operations behind progressive disclosure, and customer communications identify the TIS team. No schema, pricing, Paddle-authority, AI-entitlement, migration, or deployment change is included.

Paid commercial access now consumes the existing contract-linked entitlement authority instead of selecting the newest subscription for an onboarding organization. Active or trialing current subscriptions retain operational access while plan changes await payment/provider confirmation, fail, expire, or are abandoned. Plan-change subscription webhooks synchronize provider status without bypassing two-signal plan confirmation. Restricted pages and APIs use state-specific past-due, paused, expired, suspended, archived, and manual-review guidance. A canceled subscription remains entitled only through its confirmed paid period end.

M8B9 Demo Operations, Notifications, And Testing

Implemented Platform Owner-only demo lifecycle controls, synchronous single/global lifecycle summaries, Standard/Full/Custom access profiles, workspace and tenant-safe branch scope, durable success/failure audits, and

M8B7-based customer communications. Migration `20260727_003_m8b9_demo_operations` remains in the pre-deploy boundary and M8B8 usage history is retained across profile transitions.

M8B8 AI Entitlements And Commercial Foundation

Completed: centralized AI entitlement decisions, stable feature registry, assignable AI permission, two-successful-use demo allowances per feature, Enterprise AI paid mapping, tenant-safe durable counters and operation ledger, idempotent/concurrency-safe consumption, consistent subscription guidance, and Render-safe migration. No real AI execution route exists yet, and M8B9 inspection/reset/override/lifecycle controls remain unimplemented.

M8B7 Demo Customer Journey

Completed: approval-orchestrated activation with safe retry; durable request, approval, decline, reminder, expiry, and continuation email intents; existing Notification Center integration; shared-shell active-demo indicator; lifecycle communication processing; and authoritative-payment conversion of coherent expired demos using the same workspace. M8B8 and M8B9 remain unimplemented.

Render startup no longer executes SQLAlchemy table creation or pending migrations while importing or starting `main:app`. Render runs that work through `python scripts/run_migrations.py` as a required Pre-Deploy Command before activating the new web version. The web process has no migration worker or schema-readiness middleware and binds independently of PostgreSQL DDL.

Last Updated

Last updated: 2026-07-29

Update this file after every meaningful milestone, active development change, roadmap shift, known issue change, or documentation/KMS change.

Current Branch Strategy

Current working branch assumption: `dev`.

Branch strategy:

1 Development work should happen on `dev` unless the owner explicitly requests another branch.

1 Production/live branch is assumed to be separate from active development.

1 Confirm production branch before any deployment, merge, or production-sensitive change.

1 Do not push, merge, or commit unless explicitly requested.

1 Preserve unrelated local changes.

Production / Live Branch Assumption

The live production branch is assumed to be the branch deployed to the public app environment, while `dev` is the active development branch. This assumption must be confirmed before deployment.

Production domains:

1 Public website: `https://tisplatform.com`

1 Application portal: `https://app.tisplatform.com`

Completed Milestones

M1: Identity and SaaS foundation

1 Core SaaS signup/login/account concepts established.

1 Platform, tenant, and SaaS account identities separated.

1 Identity and SaaS phase tests present.

M2: Onboarding foundation

1 SaaS onboarding flow covers organization, contacts, branches, academic setup, and review.

1 Pending organization concept supports pre-provisioning state.

M3: Billing and plan foundation

1 Plan catalog, checkout, billing status, checkout return, and checkout cancel flows exist.

1 Payment and billing code is isolated under `saas/` service modules.

1 Initial Paddle checkout price IDs are configured through a script-based mapping sync into `subscription_plan_prices.provider_price_id`.

1 Sandbox and production Paddle price mappings must remain separate; real mapping files are ignored and credentials stay in environment variables.

M4: Provisioning foundation

1 Platform owner provisioning views and actions exist.

1 Pending organization review and provisioning queue behavior exist.

1 Provisioning retry/run operations are present.

M5: Platform access and owner controls

1 Platform owner and platform developer identities exist.

1 Platform console and owner/developer management controls exist.

1 Permission registry and platform access tests support this boundary.

1 Platform Owner pending counts and lists now use one lifecycle-aware query boundary instead of counting every historical `PendingOrganization` row.

1 Active/completed tenants are retained in Organization Records, while unresolved completed-provisioning combinations are surfaced conservatively as Lifecycle Review Required.

1 Owner-facing organization lifecycle labels reconcile onboarding, payment, subscription, contract, tenant-link, provisioning-job, and active SchoolGroup evidence without mutating historical status fields.

SaaS account setup stabilization:

1 Phase 1 TIS Account email verification recovery is accepted.

1 Valid verification links now redirect to TIS Account login with a professional success notice.

1 Expired or invalid verification links now show a recovery page with a resend option.

1 Resend verification safely handles unverified, already verified, and unknown-email cases.

1 Unverified password-based accounts are blocked from starting or continuing school workspace setup.

1 New verification-flow wording uses "TIS Account" and "school workspace setup".

1 Payment, billing, provisioning, database schema, migrations, operational modules, and the landing website were not changed.

- Google/Microsoft login remains future work and was not implemented.

SaaS customer-facing wording and branding:

- Phase 2 TIS Account customer-facing wording cleanup is accepted.
- Customer account/setup pages now use professional labels such as TIS Account, Account Dashboard, School Workspace Setup, Organization Profile, Branch Setup, Academic Setup, Subscription Setup, Secure Payment, and Workspace Activation.
- The shared customer account shell uses the official full-color horizontal TIS logo on the light account background.
- Transactional TIS Account emails use an existing official dark-blue TIS wordmark asset.
- Customer views label internal statuses through customer-safe wording instead of exposing raw tenant, provisioning, checkout, provider, plan, school group, or attempt identifiers.
- Internal `/saas` route/module/model names and stored statuses were not renamed.
- Payment, billing, provisioning behavior, database schema, migrations, operational modules, and the landing website were not changed.
- Google/Microsoft login remains future work and was not implemented.

SaaS guided setup framework:

- Phase 3A shared TIS Account guided setup framework is accepted.
- The customer account shell now supports an official-logo guided setup console with an 8-step journey stepper, status banner, one primary next action, main content area, and help/guidance area.
- The account page now uses the framework and removes the old dense dashboard statistics from the customer account landing page.
- Journey state is derived from existing account, onboarding, billing, payment, and activation data without changing stored statuses.
- Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the landing website, internal `/saas` route names, and Google/Microsoft login were not changed.
- Phase 3B redesigned the five School Workspace Setup onboarding pages on top of the shared setup shell.
- Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use consistent guided wizard sections, one shared-shell primary CTA, and secondary Back/Save Draft actions.
- Phase 3B preserved backend logic, form actions, field names, validation behavior, draft behavior, onboarding progression, payment/billing/provisioning behavior, database schema, migrations, operational modules, the landing website, internal `/saas` route names, and OAuth behavior.
- Phase 3C redesigned Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages using the same shared setup shell and guided style.
- Phase 3C added customer-safe browser-return/payment confirmation guidance and explicit TIS Platform access-after-activation messaging without changing payment, billing, provisioning, webhook, checkout start/launch, database, migration, operational, landing, route, stored-status, admin, or OAuth behavior.

Documentation/KMS milestones:

- Phase 1 documentation foundation completed and pushed to `dev`.
- Phase 2A and Phase 2B KMS foundation approved for implementation.
- Phase 2C Platform Owner Knowledge Center completed and pushed to `dev`.

1 KMS v3.0 Phase 3A Engineering Handbook approved for implementation.

1 KMS v3.0 Phase 3B approved for implementation.

1 KMS v3.0 Phase 3C approved for implementation.

1 KMS v3.0 Phase 3D final phase approved for implementation.

1 Automatic KMS enforcement implemented with repository instructions, machine-readable KIA, major-change detection, read-only artifact checks, CI validation, and a deployment prerequisite.

1 Phase 6 unified KMS command implemented with `scripts/kms.py sync` and `scripts/kms.py check`.

1 Phase 7A navigation foundation implemented with task-based reading paths, improved indexes, and normalized supporting-document titles.

1 Phase 7B professional PDF navigation implemented with handbook guidance, a page-numbered table of contents, source and major-heading bookmarks, and manifest source-page metadata.

1 Phase 7C Platform Knowledge Center navigation implemented with manifest-backed titles and summaries, logical document groups, client-side search and filters, protected booklet page links, and newest-first knowledge activity.

1 Phase 7D KMS navigation and catalog enforcement implemented with usable-title checks, approved category/module vocabulary, exact source inventory checks, docs-only navigation links, and generated PDF page-bound validation.

M7 subscription-management milestones:

1 Phase 1 entitlement foundation completed.

1 Phase 2 customer Subscription Management portal completed.

1 Phase 3 active paid branch-quantity management completed.

1 Phase 4 upgrades and scheduled downgrades completed with provider-authoritative previews/proration.

1 Phase 5 cancellation/reversal and centralized lifecycle/action policy completed.

1 Phase 6 provider billing history and protected invoice management completed.

1 Webhook and reconciliation safeguards were added across the M7 lifecycle.

M8B-1 workspace-classification foundation:

1 Added stable workspace UUID, workspace classification, and workspace lifecycle metadata to `SchoolGroup`.

1 Added onboarding workspace intent, SaaS account purpose, and operational internal-test identity metadata.

1 Added constrained enums, indexes, validation helpers, conversion rejection, and a commercial-state skeleton with no commercial behavior.

1 Added a read-only relationship diagnostic plus dry-run/default and transactional/idempotent apply backfill commands.

1 Confirmed pre-M8B-1 records are backfilled as internal sandbox/test data; no Al-Andalus conversion is included.

1 Platform Owners can inspect the metadata read-only in Platform Console; developers and tenant users cannot.

1 Existing onboarding, authentication, Paddle, provisioning, permissions, tenant isolation, entitlements, and customer flows remain authoritative and unchanged.

M8B-2 commercial-state and entitlement foundation:

1 Added normalized workspace entitlement, workspace entitlement value, and branch entitlement records.

1 Added read-only workspace, branch, and effective commercial-state resolvers with conservative manual-review outcomes.

- Paid workspace capability resolution reuses the confirmed M7 subscription entitlement engine; no billing calculations or Paddle calls were added.
- Existing internal sandbox workspaces are seeded with foundation entitlements, while newly created internal test workspaces can use a read-only compatibility entitlement.
- Platform Owners can inspect Commercial State, Workspace Entitlement, and Branch Entitlement Summary; developers and tenants cannot.
- No customer access rule, branch behavior, feature restriction, demo lifecycle, onboarding, provisioning, role, or conversion behavior consumes M8B-2 yet.

M8B-3 demo-request workflow:

- Completed onboarding now ends with an explicit Request Demo or Subscribe Now choice.
- Subscribe Now continues the approved plan-selection, checkout, Paddle, and provisioning lifecycle unchanged.
- Demo requests are validated, snapshot commercial context, prevent duplicate pending requests, and never create or activate a workspace.
- Platform Owners can search, filter, sort, approve, reject, or cancel requests; approval records review only and rejection requires a reason.
- Customers can inspect their request and withdraw it only while Pending Review.
- Submit, approve, reject, cancel, and withdraw transitions create durable audit and internal-notification events. No email is sent.

M8B-4 demo workspace provisioning and activation:

- Only a Platform Owner can provision a coherently approved customer-demo request.
- Demo provisioning reuses the shared operational workspace builder and creates no Paddle, checkout, payment, subscription, or paid-contract record.
- Demo workspaces receive an explicit demo entitlement and a tenant link sourced by the demo request rather than a fabricated subscription contract.
- Workspace creation and activation are atomic; failures preserve the Approved request, roll back workspace records, and retain a retryable failure outcome.
- Successful provisioning activates the SchoolGroup and entitlement, links the request to the workspace, records activation metadata and audit/internal events, and blocks duplicates.
- Customers see safe approval/provisioning/active states; Platform Owners see provisioning result and failure details. No email is sent.

M8B-5 standard customer-demo lifecycle:

- Demo duration is exactly seven days from successful activation; the reminder boundary is exactly Day 6.
- One resolver owns Active, Reminder Due, Expired, Suspended, and Manual Review outcomes using UTC calculations and organization-timezone display.
- The dry-run/default lifecycle command creates idempotent internal reminders and atomically expires due demos only when run with `--apply`.
- Expiration ends the demo entitlement and suspends the workspace while preserving users, branches, and all tenant data.
- Operational middleware blocks expired or ambiguous demos for web, API, download, and existing-session requests; Platform users, paid tenants, and internal sandboxes are unaffected.

- Customer and Platform Owner pages show safe lifecycle state, expiration timing, reminder state, and processing history. No email is sent.

M8B-6 demo-to-paid conversion:

- Active, coherently provisioned Customer Demo workspaces may proceed through the existing subscription checkout.
- Provider-confirmed subscription payment converts the same SchoolGroup and tenant link; no workspace, organization, branch, user, permission, or academic record is recreated.
- The demo entitlement is ended and replaced by a paid entitlement linked to the confirmed M7 subscription.
- The existing tenant link moves from its demo-request source to the confirmed subscription contract in the same atomic conversion transaction.
- Conversion states and audit/internal events remain durable, failures preserve paid records and demo operation for retry, and completed conversions leave demo lifecycle processing.
- Expired, ambiguous, cross-tenant, internal-sandbox, paid, and already-converted workspaces remain fail-closed.

M8 Landing Integration:

- Final M8 landing integration exposes two clear customer paths from the public Next.js landing website: Request a Demo and Subscribe Now.
- All public conversion links use the shared `NEXT_PUBLIC_TIS_APP_BASE_URL`, with `/saas/signup?intent=demo` and `/saas/signup?intent=subscribe` as the only destinations.
- Signup and School Workspace Setup preserve the valid selected intent and emphasize it on the final commercial-choice step without locking the customer into it.
- A normalized organization-domain eligibility ledger enforces one customer demo opportunity across pending, approved, active, expired, rejected, cancelled, and demo-to-paid history. Internal Sandbox records do not reserve customer eligibility; public email providers require an official organization website or domain before a demo can be requested.
- Migration `20260725_001_demo_domain_eligibility_policy` backfills safe historical reservations and marks ambiguous duplicate history for manual review without merging, deleting, reprovisioning, or changing existing workspace data.

Test workspace reset dependency correction:

- The Platform Owner-only test workspace/account reset now removes `subscription_change_requests` by the selected `school_group_id` before deleting that workspace's operational users.
- It removes selected entitlement values and branch-entitlement children before the selected workspace entitlement and final SchoolGroup, without changing global entitlement definitions, plans, or prices.
- The same guarded reset now clears only the selected organization's linked demo request, domain reservation, review/event history, provisioning/lifecycle history, and demo-to-paid conversion history. This permits a clean internal retest with the same email and organization domain while the production one-demo-per-domain policy remains unchanged.
- Detached same-domain reservations are now cleaned only after the shared demo-domain resolver finds no other organization, demo request, workspace, or customer account using that domain and the reservation has no historical manual-review evidence; conflicts or ambiguous history are surfaced as manual review and preserve all data.
- Safe detached reservation IDs now pass explicitly from reset analysis to deletion. The deletion transaction removes only those IDs, flushes before demo-request and parent deletion, and verifies that no selected row remains before continuing.

- The scoped pre-analysis count, deletion diagnostics, affected-row total, transaction rollback, and preservation of other workspaces remain in place.

Platform Owner demo eligibility maintenance:

- `/saas-admin/demo-eligibility-maintenance` lists domain-eligibility reservations and identifies safely removable historical detached rows.
- Safety analysis blocks deletion when any matching organization, TIS Account, demo request, operational workspace, provisioning record, subscription evidence, Demo-to-Paid conversion, or manual-review evidence remains.
- The destructive action requires explicit owner confirmation, re-analyzes under an exact-row lock, deletes only the selected eligibility ID, flushes, verifies absence, and commits or rolls back atomically.
- Successful deletion records the Platform Owner, eligibility ID, normalized domain, previous status, timestamp, and fixed historical-cleanup reason in the durable audit log.
- Customer demo submission, one-demo-per-domain enforcement, clean-room reset, schema, foreign keys, and customer-facing behavior remain unchanged.

Current Priority

Current priority: validate the final M8 public landing integration before any separately approved M9 work.

Current enforcement scope:

- Codex reads root `AGENTS.md` and authoritative KMS context.
- Every task updates `.kms-impact.yml`.
- Major-change paths are conservatively classified by `scripts/check_kms_impact.py`.
- Local KMS synchronization runs through `scripts/kms.py sync`.
- Generated artifacts and KIA are validated read-only through `scripts/kms.py check`.
- Pull requests and `dev` pushes run KMS enforcement.
- `master` deployment waits for the KMS gate.
- Automation validates and blocks; it does not rewrite Markdown.
- Phase 7D also blocks missing or unusable source titles, unapproved catalog values, invalid KMS navigation targets, source-list drift, and missing, non-positive, non-increasing, or out-of-range manifest PDF pages.

Phase 2A and Phase 2B scope:

- Create documentation update policy.
- Create change history.
- Create ADR foundation and initial accepted ADRs.
- Create module history foundation.
- Create AI project context.
- Update master context, project state, and documentation index.
- Update PDF generator to include KMS docs and manifest metadata.
- Regenerate `static/docs/TIS_Project_Reference_Booklet.pdf`.

Phase 2C completed scope:

- Added read-only `knowledge_service.py` as the single KMS app access layer.
- Added owner-protected `/platform/knowledge` page.
- Added owner-protected PDF view/download routes.
- Added an owner-only Platform Console card.
- Added platform knowledge module history.
- Regenerated the PDF and manifest after documentation updates.
- Phase 7C adds client-side source search, category/module/freshness filters, logical document groups, document titles and summaries, protected booklet page links, and improved ADR/module-history ordering without adding routes or write behavior.

Still out of scope:

- Regenerate button.
- Additional app routes beyond the approved read-only Knowledge Center routes.
- `ui_shell.py` and `authorization.py` changes unless separately approved.
- SaaS flows.
- Operational logic.
- Database, migrations, or `tis.db`.
- Landing page implementation.

KMS v3.0 Phase 3A scope:

- Add complete TIS module map.
- Add repository architecture map.
- Add end-to-end user/system workflows.
- Add clear AI/human developer onboarding structure.
- Update generator to include engineering docs.
- Regenerate the PDF and manifest.

KMS v3.0 Phase 3B scope:

- Add database architecture overview.
- Add development standards and non-negotiable rules.
- Add UI/UX and design philosophy.
- Add product roadmap.
- Strengthen AI/human developer onboarding guidance.
- Update generator to include the new engineering docs.
- Regenerate the PDF and manifest.

KMS v3.0 Phase 3C scope:

- Add rejected architectural decisions.
- Add visual documentation framework.

1 Add AI optimization guide.

1 Add project governance and decision traceability.

1 Update generator to include the new engineering docs.

1 Regenerate the PDF and manifest.

KMS v3.0 Phase 3D final scope:

1 Add knowledge lifecycle documentation.

1 Add documentation automation guide.

1 Add formal KIA standard.

1 Add self-evolving workflow.

1 Add documentation dependency map.

1 Add AI coding workflow.

1 Add future automation roadmap.

1 Regenerate the PDF and manifest.

Current Known Issues

M4C existing-workspace paid activation is implemented for verified tenant owners. Professional and Enterprise AI cover all active branches and use branch count as Paddle quantity. Starter is deliberately fail-closed until complete restricted-branch enforcement is proven. Deployment still requires migration `20260806_002_existing_workspace_paid_activation`, environment-specific active Paddle price mappings, and Sandbox validation before production use. Disposable PostgreSQL validation covers migration idempotency and rollback, database constraints, concurrent prepare/launch, duplicate and out-of-order webhooks, drift rollback, and paid-versus-promo source races. Live Paddle validation was not possible without the Sandbox API, client, and webhook credentials.

Organization Account status for an activation-required existing workspace is now derived from its current paid-activation attempt. With no attempt it displays `Activation required`; only a current unexpired checkout-started or payment-processing attempt displays `Payment processing`. Terminal attempts display recovery states.

Existing-workspace plan selection is editable only before real checkout begins. Organization Account reopens the eligible plan-selection surface for `draft` and `checkout_ready` activations, highlights the saved choice, and recalculates a replacement quote without creating a Paddle transaction or PaymentAttempt. Checkout-started, payment-processing, and manual-review/inconsistent activations remain locked against silent replacement.

Known issues and watch points:

1 Student & Academic Placement Foundation and Talent Program & Framework

Foundation have no PostgreSQL validation yet (constraints, concurrent placement/framework writes); only SQLite-backed pytest coverage exists. Neither has a UI, import/merge, or any Rubric/KPI/Assessment/Review/Learner-Profile/analytics/AI code, which remain future milestones.

1 KMS policy depends on future developers and AI agents consistently completing the Knowledge Impact Assessment.

1 Generated PDF can become stale during local work, but CI now blocks stale artifacts from integration/deployment.

1 The owner-only Knowledge Center is implemented as read-only; there is no regenerate button yet.

1 Public static storage is not sufficient access control for sensitive docs; Phase 2C should serve docs through protected owner-only routes.

1 Render deployment constraints should continue to guide dependency choices.

1 Production memory must be treated as a hard constraint. The 2026-06-27 Render restart/502 investigation found two avoidable memory risks: observation diagnostics doing extra production template renders and global location lookup parsing a 47 MB dataset into a complete in-memory index for simple picker requests. Local stabilization changes now gate observation diagnostics and use scoped location loading; future work must follow the Production Memory and Render Stability standards.

1 The untracked repository-root directory `tis_scope_test_5i3yf0h5/` has a Windows

security descriptor that denies enumeration and ACL inspection to the normal development process. Repository pytest collection is bounded to `tests/` and explicitly excludes that orphan directory, so root `pytest` runs do not scan it.

1 Google/Microsoft login is still future work; password-based accounts must remain email-verified before school workspace setup.

1 GitHub repository settings must mark `KMS Enforcement / kms-check` as required on protected branches; this cannot be configured by repository file changes alone.

Next Planned Work

Next planned work:

1 Review the KMS enforcement rules against real pull requests and tune only demonstrated false positives.

1 Keep M7 documentation current as subscription fixes evolve.

1 Later consider an explicit owner-only regenerate workflow.

1 Review, commit, and deploy the production memory stabilization changes when approved, then monitor Render memory, restart count, and route-level 502s after deployment.

Landing Page Baseline Situation

The public landing page source of truth is:

1 `tis-landing-website/`

Marketing docs:

1 `docs/marketing/landing_page_source_of_truth.md`

1 `docs/marketing/tis_landing_page_master_content.md`

Relevant ADRs:

1 `docs/adr/0001-separate-nextjs-landing-website.md`

1 `docs/adr/0007-landing-page-visual-system-strategy.md`

Legacy FastAPI landing files are not the current public website source of truth:

1 `templates/landing.html`

1 `static/landing/landing.css`

Do not modify landing page design, landing copy, or legacy landing files unless explicitly approved.

Knowledge Update Policy

Every approved implementation must complete the KIA:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed and `.kms-impact.yml` matches the actual task diff. If included docs change, regenerate:

1 `static/docs/TIS_Project_Reference_Booklet.pdf`

Then run `.\.venv\Scripts\python.exe scripts\kms.py check` for final read-only validation. When documentation changes, `.\.venv\Scripts\python.exe scripts\kms.py sync` performs generation and post-generation validation together.

Scope Guardrails

1 Do not touch SaaS flows unless explicitly approved.

1 Do not touch operational logic unless required by the approved task.

1 Do not touch database migrations or `tis.db` unless explicitly approved.

1 Do not change the landing page unless explicitly approved.

1 Do not add a KMS regenerate button until separately approved.

1 Do not let automation rewrite authoritative Markdown.

1 Do not place customer, personal, production, billing-record, transaction, invoice, webhook payload, credential, secret, environment, or database-row data in KMS docs.

1 Do not commit or push unless explicitly requested.

Talent Program Criteria UI Simplification

Owner direction removes Program Criteria from the normal Talent setup and Student Assessment workflow. The Program setup UI no longer exposes Program Criteria configuration, and a completed Student Assessment no longer exposes a manual "Check Program Criteria" action. Existing backend Review Candidate/Official Identification records and historical data are preserved; this change does not delete schema or historical evidence.

Student Single And Bulk Delete

The Students list now exposes permission-gated single Delete and checkbox-based Bulk Delete. Both use the canonical Student service; no client-side deletion authority exists. Hard deletion is allowed only before Academic Placement or Talent history exists, and bulk deletion is all-or-nothing when any selected Student is blocked. The new permissions are `students.delete` and `students.bulk_delete`; organization/global scope is required for the destructive action. Historical Student/Talent evidence is never cascaded away.

Permission-Controlled Student Actions And Evaluation Period Selection

Student Delete and Bulk Delete are now first-class System Configuration permissions (`students.delete`, `students.bulk_delete`) and remain backend-enforced. Evaluation Period selection for entering Student Assessments is now separately permissioned as `talent_evaluation_plans.select_period`: Administrator receives it by default; other tenant roles are disabled by default, the UI renders the entry action disabled without it, and

the Cycle-to-Period link API requires it. The role-permission configuration remains the owner-controlled override mechanism. The disposable Talent analytics seed now uses Arabic names written with Latin characters and retains its 10-Student Grades 3-5 analytics coverage.

Grade-Scoped Rubric Structure And Administrator History Delete

Talent rubric authoring now follows `Grade -> Competency -> Level -> Achievement Description`. Framework competency membership has an optional Grade scope added by migration `20260911_002_talent_framework_competency_grade_scope`; NULL remains the backward-compatible all-Grade value. The existing Grade-specific descriptor table remains the description authority at Competency x Level x Grade, with generic descriptor fallback for legacy/unscoped Frameworks. Program setup groups content by Grade and only Grade-applicable competencies are shown in Student Assessment. Compact Talent result surfaces use the ordered rubric number while retaining accessible label text and an order-derived low-to-high visual progression.

Student Assessment Evaluation contexts are ordered by the linked Annual Evaluation Plan Period `sequence`, fixing cases where a later-created Term 2 appeared before Term 1. Creation/id ordering is used only for legacy unlinked Cycles.

Students now also have a separately permissioned force-delete-history path. `students.force_delete_history` is Administrator-default and configurable through System Configuration. The Students UI first previews historical blockers; without the permission the delete remains blocked, while an authorized actor must explicitly confirm deleting the Student together with Academic Placement/Talent history. Normal single Delete and Bulk Delete retain their existing history-protecting semantics. Student row Open/Delete action icons are rendered at a compact 13px size.

Talent Grade-First Evaluation Workflow Rebuild

The Owner-directed Talent evaluation-process rebuild is implemented on `dev`. Program creation/editing is focused on Program identity plus eligible Grades only. The Programs table exposes a distinct Rubric action. Opening Rubric presents eligible Grades as independent collapsible sections using the Owner-confirmed hierarchy **Grade -> Competency -> Competency-owned Rubric -> Levels**. Each competency can create its own rubric name and ordered levels/descriptions. The old one-shared-rubric-per-Framework rule is retained only for legacy read/migration compatibility; migration `20260911_004_talent_competency_specific_rubrics` enables exact competency ownership. Saving returns to a compact Program/Rubric overview.

ADR 0036 governs re-evaluation semantics. `TalentStudentAssessment` carries `is_current`, `reassessment_of_assessment_id`, and `evaluation_context_cycle_id` via migration `20260911_003_talent_assessment_reassessment_attempts`. Re-evaluation is triggered by actual Student-facing rubric-content changes for the Student's recorded Grade, not by a no-op Framework clone. Starting re-evaluation preserves the completed historical attempt and creates a new current attempt on the newer exact Framework while retaining the original Evaluation/Term identity. New Students in that Evaluation also start on the newest saved assessable rubric. Current-result analytics project the replacement attempt into the original Evaluation context and exclude the private replacement Cycle as a second population row.

Production Results & Analytics still depends on governed external provider configuration under ADR 0028. The implementation already contains configuration-driven privacy-provider resolution and approved-value validation; missing Render environment configuration continues to fail closed rather than exposing unsuppressed indicators.

Talent Evaluation Workflow, Review, Results, And UI Consolidation

The Owner-approved Talent consolidation is implemented on `dev` under ADR 0036, ADR 0037, and ADR 0038.

Rubric authoring is Grade -> Competency -> competency-owned Rubric -> ordered Levels. New competency rubrics do not inherit assessed legacy shared-rubric levels automatically. Authors may explicitly copy level structure from another competency in the same Draft, with optional descriptions; copied data is independent afterward.

The canonical Overall Program Result is now the arithmetic mean of selected rubric ranks on one coherent Program scale (for example `4.4 / 5`). All applicable competency rubrics must use the same level count before completion. A normalized percentage may be derived for visual bars only. Different Programs remain separate and are never combined into one universal Student Talent score.

Student Assessments are organized as Evaluation Period -> Programs -> Students. Repeated Evaluation labels are grouped once. The selected Program/Evaluation shows all live-placement eligible Students in authorized scope with Not started, In progress, Completed, or Re-evaluation required states. Re-evaluation continues to preserve immutable historical evidence and original Evaluation identity.

Talent Review now includes all current completed Assessments, with Review Candidate, review status, and Official Identification shown independently. Assessment completion evaluates any existing deterministic Review Candidate policy, while no-policy/non-qualifying completed Assessments remain visible.

Program settings surface the existing Annual Evaluation Plan Periods alongside eligible Grades. No duplicate scheduling persistence is introduced.

Results & Analytics adds privacy-safe competency rank averages and selected Program average summaries, keeps Branch/Grade/Program organization analytics on the existing M9/M10 privacy-closed providers, and exposes separate per-Program Student results through the governed Student drill/Students Across Programs matrix. Official Identification remains a human decision; Learning Style remains context only and is not a Talent score input.

The Talent UI has stronger semantic icons/status color, richer Evaluation, Student, Review, and result surfaces, and responsive low-to-high result visuals. The Programs list uses a bounded summaries endpoint instead of N+1 per-Program setup requests. A selected Program is fetched directly, its independent setup reads are parallelized, and routine Program/rubric saves perform targeted Framework/configuration or Program refreshes rather than a full workspace reload. Hash-only wizard navigation and Save Rubric reuse the current bounded cache without network reads.

Evaluation Plan mutations keep existing content visible and re-read only Plan/Cycle data. Student Assessment detail loading collapses dependent reads into one parallel batch after the Assessment is resolved, and an explicitly selected Evaluation loads its eligible Student roster in parallel with the operational Assessment/context reads. Full loading is reserved for true initial/context loads or explicit recovery.

No new schema migration is introduced by this consolidation.

Talent & Potential Owner Acceptance-Testing Correction Pass (2026-09-12)

A focused UI/UX correction pass over the already-governed ADR 0039 Competency + KPI + Level model, driven directly by Owner acceptance testing on top of pushed commit a173555. This pass changes no eligibility gate and no architecture; it corrects builder rendering, disclosure, navigation, and capitalization defects.

Delete Competency is now genuinely always available in the Competency, KPI & Level builder for every current build - new-from-scratch, previously saved, or based on a historically-used Framework. Investigation found the true root cause was not a hidden/blocked action but a rendering bug: a Competency saved with no explicit Grade on a genuinely new draft Framework (`framework.in_use_by_assessments === false`) never matched any Grade section in `rubricGradeSections` at all, so the entire Competency row - and its Delete action - never rendered. The fix treats "no Grade" as "applies to all eligible Grades" only for a Framework with no Assessment history yet; a no-Grade Competency on an already-used Framework remains legacy/pre-migration data and still stays out of new-Grade authoring, preserving the existing historical protection. Deleting a Competency (direct on a mutable draft, or via the existing version-safe clone-then-delete path on an immutable/active Framework) removes only the current KPI, Levels, and descriptors; completed historical Assessment evidence is never altered. Both paths now show one unified confirmation message so the Owner is never asked to reason about Framework versions to remove a Competency from future setup.

Each Competency card in the builder is now an independent, initially-collapsed disclosure (native `<details>`/`<summary>`, keyboard-operable, no manual `aria-expanded` required) showing only a compact header - name plus a "N KPI · N Levels" count - until the Owner opens it; Grade sections remain their own independent disclosures as before.

The opened Student Assessment editor now shows an obvious "? Back to Students" action that returns to the Student Assessments list for the exact same Academic Year, Program, and Evaluation/Cycle context the Assessment was opened from - never the Talent landing page. The Assessment editor itself is otherwise unchanged in this pass.

Talent & Potential capitalization was corrected to consistent Title Case for Competency/Competencies, KPI, and Level/Levels counters and section/column headings (e.g. "3 Competencies", "5 Levels", "KPI Overview", "Assessment Status"), while normal explanatory sentences remain in ordinary sentence case by design.

The Owner-supplied Ghars sidebar symbol-only asset (`talent-ghars-symbol.png`) named in this correction request was not actually present in this environment (confirmed by a full filesystem search); the sidebar branding swap to that asset is deferred until the file is supplied, and the current Ghars symbol-and-text sidebar branding from the prior corrective pass is unchanged. No backend Python files were touched in this pass, and no new schema migration is introduced.

Talent & Potential UI Action-Placement Correction (2026-09-12)

A further Owner-driven interaction-pattern correction on top of the acceptance-testing pass above. Edit and Delete for every created Competency, KPI, and Level item are now compact icon-only buttons (pencil = Edit, trash = Delete) placed immediately beside that item, replacing all visible action text ("Edit Competency", "Delete Competency", "Edit KPI", "Delete KPI", "Edit Level", "Delete Level"). Each icon button is a real `<button>` with an `aria-label` and matching `title` naming the exact item it affects (e.g. "Delete Mental Calculation", "Edit Mental Calculation level Beginning"), so there is never ambiguity about which row an action targets, and it inherits the workspace's existing keyboard focus-visible styling and a compact but sufficient (36px) hit target.

This applies uniformly to both the primary Grade -> Competency -> KPI -> Level accordion and the older parallel flat-table "What we assess" wizard view, so the pattern reads the same wherever these actions are reachable. The Competency header in the accordion also gained an Edit icon (it previously exposed only Delete there) by surfacing the existing name/description edit capability already used elsewhere in the workspace - no new backend behavior was introduced.

Add actions are deliberately unchanged: "+ Add Competency", "+ Add KPI", "+ Add Level", and "Copy Levels From..." remain visible text and stay in their existing structural position (the Grade section for adding a Competency, inside a Competency for adding its KPI, inside the KPI for adding a Level) rather than sitting beside every existing item. No hierarchy, CRUD behavior, or eligibility gate changed in this pass, and no backend Python files were touched.

Talent & Potential Grade-to-Grade Assessment Criteria Copy (2026-09-12)

An Owner-requested follow-up capability on top of the icon-actions correction pass, added before deployment. Each Grade section in the Competency, KPI & Level builder now exposes a structural, always-visible "Copy Criteria From Grade..." action (not an inline Edit/Delete icon) that copies the complete source Grade's assessment structure - Competencies, KPI/rubric definitions, Levels, and generic/Grade-specific achievement descriptors, with ordering preserved - into an empty destination Grade as entirely independent new records. Nothing is shared or linked between source and destination: renaming or editing a copied Competency, KPI, or Level afterward never changes the source Grade, exactly mirroring the existing "add a Competency to a Grade" flow's own precedent of minting a new Competency identity per Grade membership.

An already-populated destination Grade is always rejected rather than silently merged or overwritten (`target_grade_occupied`) - no automatic merge algorithm was implemented; the Owner explicitly authorized keeping this first version to empty-target-only copying, in favor of safety over hidden merge behavior. The backend service function (`talent_program_service.copy_grade_criteria`) reuses the existing

`_require_mutable_draft` gate unchanged, so it can never mutate a Framework with real Assessment history; the client reuses the identical version-safe clone-then-mutate orchestration already established for versioned Delete Competency (clone an immutable/active Framework into a new draft, then copy against that draft) so the Owner never needs to understand Framework cloning/versioning to use this feature. Copy scope is bounded to the same SchoolGroup, Program, and Framework as every other Program-service mutation, reusing the existing authorization/scoping pattern unchanged. Copied items retain full ordinary Edit/Delete/collapse behavior identical to manually-created items. No new schema migration was introduced.

CRITICAL: Production DB Connection-Pool Exhaustion Fixed (2026-09-12)

Root-caused and fixed a confirmed production release blocker: `sqlalchemy.exc.TimeoutError: QueuePool limit of size 5 overflow 10 reached, connection timed out, timeout 30.00`. The default (unconfigured) SQLAlchemy QueuePool numbers in the traceback match `database.py`'s `create_engine` call exactly - no explicit `pool_size`/`max_overflow`/`pool_timeout` was ever set.

Root cause was `main.py`'s `inactivity_timeout_middleware` - a global `@app.middleware("http")` that runs on every authenticated request across the entire application, not only Talent. It opened its own `SessionLocal()` session and wrapped the entire `await call_next(request)` call inside the same `try/finally: db.close()` block, holding that connection checked out for the full duration of every request's downstream processing on top of the separate connection the route handler itself acquires via `Depends(get_db)`. Every authenticated request therefore consumed two simultaneous pool connections for its whole lifetime instead of one. This was not a classic leak - the connection was always eventually closed - but a long-held, effectively duplicated connection held far longer than the few quick scalar queries it actually needed (session-user lookup, idle-timeout check, `current_user` resolution, commercial-access/permission checks). Talent Student Assessments fires several concurrent API requests per page load, so a single load could require up to double the connections purely from this doubling, exhausting the default pool under light concurrent traffic - explaining why Student Assessments failed first, why other pages then hung (the pool was globally exhausted for the whole application), and why a Render restart temporarily "fixed" it (restart empties the pool; the underlying per-request doubling was unchanged and would recur).

Ruled out during investigation: no additional independent `SessionLocal()`/`sessionmaker()`/`engine.connect()` sources exist anywhere in the Talent service/router layer (all rely exclusively on the injected `Depends(get_db)` session); `dependencies.py`'s `get_db()` canonical dependency correctly implements try/yield/finally with no rollback gap; the deployment runs a single `uvicorn main:app` process (no `--workers` flag), so the observed pool numbers are the single-process SQLAlchemy defaults, not a worker-multiplication effect. No Grade gate or annual Program-configuration enablement gate was reintroduced - `current_placements_for_assessment` and `_current_placement_for_assessment` are unchanged.

Fix: `inactivity_timeout_middleware` now opens its session, completes the auth/idle-timeout/commercial-access/route-permission checks, and closes the session entirely before `call_next` is ever awaited - behavior- preserving (every existing early-return outcome is unchanged, only restructured through one closing point ahead of `call_next`). New regression coverage in `tests/test_db_connection_pool_lifecycle.py` proves the session closes before `call_next` starts (including on exceptions), proves an unauthenticated/exempt request never opens a session, and - against a real small QueuePool - proves the fixed shape never exceeds pool capacity under concurrency while a reconstruction of the old doubled-checkout shape reliably exhausts the same pool, positively proving the doubling was real and is now eliminated.

Start Editing Accidental Empty-Draft Recovery + Mandatory Clone (2026-09-12)

Owner-reported priority incident: a Program with existing Competencies (reported at approximately 9) showed an empty Draft after Start Editing, because the old UI made "Copy the current rubric structure" an optional checkbox - leaving it unchecked produced a genuinely empty new Draft while the real Competencies remained safe and untouched in the

Framework Start Editing was invoked from.

No live Talent data was reachable from this development environment to operate on directly (the local `tis.db` has no `talent_*` tables migrated; the only reachable local Postgres instance is a stale, empty B11C concurrency-test schema). The actual reported incident lives in a deployed environment this session has no database access to. Delivered instead: a governed, fully tested recovery mechanism ready to run against the real data, plus the permanent fix that prevents recurrence everywhere.

`talent_program_service.recover_accidental_empty_draft` identifies the correct source Framework (via the accidental empty Draft's own `supersedes_framework_version_id`, which Start Editing always records regardless of whether cloning ran) and, after verifying the Draft is genuinely empty (zero Competencies, zero Assessment history), re-runs the existing governed `create_framework_draft(clone_from_id=...)` mechanism to produce a new, fully populated, independently editable Draft with every Competency, KPI, Level, descriptor, and ordering already in the source. Nothing is reconstructed by hand, the source Framework and its historical completed Assessment evidence are never touched, and the accidental empty Draft itself is left in place (harmless, since the recovered Draft's higher version number is what Start Editing/rubric mode selects going forward) rather than deleted. `scripts/recover_talent_accidental_empty_draft.py` is the operator-facing CLI (dry-run by default, `--apply` to commit).

The permanent fix: Start Editing's "Copy the current rubric structure (optional)" checkbox is removed entirely - the new-version form submission now always sends `clone_from_id` unconditionally (naturally omitted only for the very first Rubric Structure creation, where there is nothing yet to clone), so a Start Editing action can never again produce an empty Draft.

Talent Student Assessment current-rubric + roster-state acceptance correction (2026-09-12)

Opening an Assessment is not learner evidence by itself. A current `in_progress` Assessment with zero persisted `TalentStudentCompetencyResult` rows now transparently continues on the newest saved assessable Framework when reopened. The prior empty attempt becomes non-current provenance and the replacement keeps the same visible Evaluation through `evaluation_context_cycle_id`. Once any Competency Result exists, or once an Assessment is terminal, its exact historical Framework remains authoritative and immutable.

Normal Student Assessment contexts now exclude private physical Cycles used only for current-rubric/re-evaluation provenance, so "Current rubric" no longer appears as a duplicate Evaluation. The roster chooses the newest matching current attempt when legacy/bad data exposes duplicate-current rows, preventing an older In Progress row from masking a later Completed result.

The Talent & Potential sidebar now uses the Owner-supplied symbol-only Ghars PNG (`static/img/talent-ghars-symbol.png`) in the existing icon-sized brand slot while retaining the visible module label and permission-aware child tree.

Talent Results & Analytics - Commercial Feature Gate Removed (Owner Decision, 2026-09-12)

Per direct Owner instruction, Results & Analytics no longer requires the `feature.organization_intelligence` commercial entitlement. Production `build_availability_provider()` now resolves `PermissionScopedOrganizationAnalyticsAvailabilityProvider`, which accepts only a valid positive SchoolGroup and Academic Year and performs no subscription, plan, workspace-entitlement, or feature-registry lookup. The existing `talent_analytics.view` permission remains mandatory inside `resolve_access_context`; Student drill, Candidate, Identification, tenant, Branch, and Academic-Year scope checks remain unchanged. Privacy suppression, complementary closure, configured cohort-5 policy, 1000/1000/1000 breadth ceilings, and M10 REPEATABLE READ behavior are unchanged. The legacy semantic feature key remains in the general catalog for compatibility/history but is no longer consulted by Organization Analytics runtime availability.

Talent Owner Adjustment Package - Grade Alignment And Operational Presentation (2026-09-12)

Current Student Assessment criteria are Grade-aligned. The roster remains Grade-agnostic for eligibility, but `_newest_assessable_framework` no longer falls back to another Grade's criteria. Start Assessment requires an assessable Competency -> KPI -> Level structure applicable to the Student's Grade (or intentionally unscoped), otherwise it returns a Grade-specific setup message. The browser follows the same rule; historical evidence remains immutable.

Student Assessments has Grade, Section, and Assessment Status collection filters. Talent Review has Identification Classification filtering. Students is the first Talent child for Talent-capable users; standalone Students remains only as a students-only fallback. Desktop Student tables have no nested vertical scroll. Learning Style keeps categorical style indicators and their established per-style colors while privacy-suppressed magnitudes stay hidden.

Already-visible analytics magnitudes use an ordered low-to-high blue intensity. Protected/no-data states never receive magnitude coloring. Student Assessment Competency/KPI/Level entry uses a responsive professional hierarchy without changing write contracts. Ghars uses the compact sidebar emblem and the transparent full wordmark only in Talent Overview. No schema or migration.

Talent Owner Adjustment Package - Correction (2026-09-12, follow-up)

The Talent Overview full Ghars wordmark remains **blocked pending asset**: a locally traced/reconstructed SVG was removed from the header wiring (it is not the Owner-supplied file) and replaced with a documented, not-yet-present path (`static/img/ghars-full-wordmark-dark.png`); the header renders empty until the Owner supplies the real asset. The separate duplicated "Selected Evaluation" panel below the Evaluation Period cards in Student Assessments has been removed; the selected card's in-place `is-selected`/`aria-current` marking is now the only selection indicator. No schema/migration change.

TIS Change History

docs/CHANGE_HISTORY.md

2026-09-25 - Production follow-up Part 2: chart types, filter UX, comparisons, Progress Over Time

1 Chart selector now switches only distinct visualization types (Bar/Doughnut for full public

partitions of at most eight categories, Trend/Bar for series, Bar only for Learning Style, Rubric and comparisons); the duplicate "Percentage" and redundant "Pie" modes were removed; single-type charts show no selector. Overview defaults to Doughnut where valid with one reduced-motion-aware entrance animation.

1 Results & Analytics filters refresh in the background (AbortController, newest-token-wins,

region-local busy state, held height, restored scroll and focus, retryable inline error) instead of replacing the page root; chart-type change is client-only.

1 One "Selected comparisons" section at the bottom holds the selector, six-group limit and results.

1 Progress Over Time kept as a per-Program longitudinal trend over the existing ADR 0027 API with

honest gaps; Results & Analytics links to it. Raw arrow glyphs replaced by an inline SVG chevron.

1 Tests: `talent_part2_experience.test.cjs`; old mode pins updated deliberately. No schema,

migration, permission, privacy or `tis.db` change. Web Service only; not deployed.

2026-09-25 - Production follow-up Part 1: Talent Branch authority, Start Assessment, Classification

1 Sidebar Branch selector lists real Branches only. Removed the Talent "All Branches" entry,

the `branch_scope=all` marker cookie, `user.scope_all_branches` and `branch_id=all` handling in `POST /scope/branch` (legacy cookie ignored). Organization-authorized actors choose All Branches or any authorized Branch in the Talent page-level filter (sidebar Branch = default); Branch-limited actors stay confined (`tp-config.branchLocked`). ADR 0044 records the dated owner-directed amendment. Tests: `test_talent_branch_hard_scope.py`, Node B1-B10.

1 Start Assessment: root cause was the generic mapping of stable safe 400 codes (chiefly

`assessment_tool_unavailable`); curated fixed copy per code, shown inline without scrolling. Backend flow proven end to end (`test_talent_start_assessment_flow.py`).

1 Classification: diagnosed as the approved per-band cohort floor (no behavior change); added one

uniform value-free explanatory sentence (`test_talent_classification_aggregate_privacy.py`, `talent_classification_withheld.test.cjs`). No schema, migration, permission, privacy-rule or `tis.db` change. Web Service only; not deployed.

2026-09-25 - Deployment hotfix: PostgreSQL psycopg v3 driver requirement

Render's pre-deploy `python scripts/run_migrations.py` failed with `ModuleNotFoundError: No module named 'psycopg'` because the deployed `DATABASE_URL` selects SQLAlchemy's `postgresql+psycopg` dialect (psycopg v3) while `requirements.txt` declared only `psycopg2-binary`. `requirements.txt` now also declares `psycopg[binary]`; `psycopg2-binary` is kept so a plain `postgresql://` or `postgresql+psycopg2://` URL still resolves. No code, schema, migration, environment value or Talent behavior changed. `tests/test_deployment_dependencies.py` asserts each PostgreSQL driver family has a declared requirement. Web Service only; the separate workflow revision is unaffected.

2026-09-25 - Agent 3 Talent product transformation and privacy remediation

1 Preserved and completed the executive Overview work, with distinct Student KPI,

participation grain, shared charts and workspace actions; Programs uses searchable cards and omits normal lifecycle-status badges without changing lifecycle gates.

1 Added one bounded, identity-free filtered dashboard contract and six aggregate

sections with shared filters, selected comparisons, Program-bound indicators, and compatible ordered-period results. No average-of-Branch-rate calculations.

1 Fixed Agent 2's Classification-filtered Learning Style reconstruction leak.

The existing privacy provider gates the selected cohort against its original primary/complementary projection. Protected aggregates serialize no numbers; individually authorized Student roster rows remain available without cohort suppression. Current reassessments resolve through their original Evaluation.

1 Added shared chart modes, exact accessible tables and value-free protected states;

removed duplicate analytics/Student preview and auxiliary rubric fetches from Results & Analytics. Retained per-view assets, bounded requests and stale-response protection. New dashboard reads use the existing repeatable snapshot dependency.

1 Added adversarial backend/frontend tests. No schema, migration, permission,

local database, deployment, master merge or release PR change.

1 Recovery completion: removed the re-introduced generic "Protected for privacy"

chart copy, fixed the inert/over-shown Program summary Finish Setup control, replaced emoji avatars/icons with initials and SVG, added the Talent visual system stylesheet layer and Program grade chips, and added backend and Node regression for Overview/dashboard Student-count agreement, bounded statements, route-level protected Learning Style, exact small-cohort roster rows, chart mode switching and Program Status absence. Visual results are structural only.

2026-09-25 - Student Assessment request-backed filters and filtered insights

1 Added bounded query filters to the live eligible Student roster: search, Grade,

Section, Assessment state and canonical Classification, while retaining the existing hard global Branch ceiling.

1 Added backend roster-population Classification (privacy-projected) and Learning

Style (eight categories plus Unassigned) summaries. Classification is derived only from current completed Assessments through the M17 authority; the frontend displays the backend contract and accessible equivalent table data.

1 Removed the Student Assessment page's DOM-only Grade/Section/Status filtering.

No schema, migration, permission, `tis.db`, Results & Analytics, or deployment change.

2026-09-25 - Batch 1 closure: the global Branch is a hard Talent scope

> Superseded for organization-authorized actors by the 2026-09-25 Part 1 entry above (owner > direction); the ceiling continues to bind Branch-limited actors.

1 Supersedes the 2026-09-24 entry's "default Talent Branch scope": the active global

Branch is now an upper ceiling (intersection of actor authorization and active global scope) enforced server-side by `talent_branch_scope.py` on every Talent read surface (organization analytics, analytics, results analytics incl. Learning Style, Evaluation Progress, Assessments, Cycle roster/population, learner profile, legacy review/identification/educator lists, selector option lists). A Branch outside the ceiling is rejected; an omitted Branch resolves to the ceiling.

1 Added an explicit global "All Branches" state for Talent (`branch_scope=all` marker

cookie set by `POST /scope/branch branch_id=all`, exposed as `user.scope_all_branches`; option shown only on `/talent` pages; other modules keep their single working Branch). Without it organization users could never reach organization-wide Talent again.

1 `static/js/talent.js`: `config.branch` is a ceiling; no All Branches / other Branch

offered or sent under a single-Branch scope; stale URL Branch params are clamped.

1 Tests: `tests/test_talent_branch_hard_scope.py`; Node B3/B4/B6/B6b. No schema,

migration, permission or `tis.db` change; Student deletion, inactive-Student and Talented-KPI semantics unchanged. Web Service only; not deployed.

2026-09-24 - Deployment Acceptance Batch 1: data correctness, scope integrity, Student deletion, loading

1 **63 vs 9.** The Overview "Students participating" figure was the frozen

membership row count (one per Student per Cycle/Program) and the population query did not check that the Student still exists. `distinct_students` (distinct current Students, existing distinct-Student authority, class P2) now backs "Students participating"; membership counts are "Program participations". Every governed Talent population read now requires the Student to exist (`talent_current_students.py`).

1 **Student deletion.** Exhaustive FK audit: ten Student-owned tables, all removed

by `force_delete_student_history` (now `STUDENT_OWNED_MODELS`, locked to ORM metadata by test). Permission model unchanged (normal/bulk Delete stay blocked by history). Fixed the ObjectDeletedError test defect.

1 **Global Branch.** The active Branch is now the default Talent Branch scope

(`tp-config.branch`, `reconcileBranchScope`, stale-scope marker, explicit All Branches); Overview, Program Results and the eligible-students roster accept an authorized `branch_id`; backend authorization unchanged and authoritative.

1 **Loading.** Row-proportional N+1 (assessments list 982 statements for 20 rows)

replaced by request-scoped `read_batch` memoization and set-based priming; permission set resolved once per request; Assessments page requests only its Year/Program; `/programs/summaries` 500 fixed; Talent assets cache-busted by content hash; inline 15 s loader watchdog. Render capacity not shown to be implicated (production cannot be measured here).

1 Tests and KMS: see PROJECT_STATE. No schema, migration, permission or `tis.db`

change. Web Service only; not deployed.

2026-09-24 - Deployment Acceptance Correction D: Professional Student Assessment editor redesign

1 Owner-observed production screenshot: the Student Assessment editor was cramped

(multiple narrow competency columns with rubric descriptions wrapping into near-vertical text). Redesigned as a professional assessment workflow; every mutation and result still comes from the real API (no scoring, Classification, authorization or schema change).

1 `static/js/talent-operations.js` now renders one competency per row, each with

a full-width, adaptive set of selectable rubric tiles (`tp-rubric-tile`: clear "1 / N" rank, level title, and the descriptor/description on its own horizontal line) and a full-width Evidence textarea bound to its competency. The whole tile is the `<label>` radio target with native keyboard/focus semantics, a visible focus ring, a check marker and a non-colour-only selected state.

1 `static/css/talent-experience.css` replaces the multi-column `.tp-assessment-grid`

with `grid-template-columns:1fr` and the rubric tile grid adapts (`repeat(auto-fit, minmax(200px,1fr))`, stacking to one column on mobile).

1 Progress/save/Complete/Incomplete/Insufficient actions are unchanged; a

completed assessment still shows the backend Overall Program Result and automatic Classification with a Talented badge only for Exceptional.

1 Tests: `tests/talent_operations.test.cjs` (rubric-tile structure, evidence

binding, single-column/adaptive CSS). Structural verification only; browser visual acceptance is still pending. Web Service only; not deployed.

2026-09-24 - Deployment Acceptance Correction C: Learning Style distribution

1 Root cause of "all Unavailable": the Learning Style distribution ran through the

Talent P3 primary/complementary small-cell suppression, and the Results & Analytics frontend read `buckets` while the route returned `levels`.

1 Owner decision (ADR 0031 "Acceptance C Amendment", ADR 0044 note): the

distribution is authorized Student-domain aggregation and is no longer subject to Talent small-cell suppression; no other Talent privacy class changed.

1 `build_distribution(students)` now returns nine categories with key/label/count/

percentage plus `total_population` (denominator includes Unassigned, zero is `0/0%`, empty population is state `empty`); no privacy-provider dependency on the Students API route, Students page panel or the Results `/learning-style` route.

1 Results & Analytics and the Students panel render count, percentage and a bar per

category from backend values. Replaced the suppression/privacy-unavailable tests (retiring the stale failing panel-message test); no schema, migration, permission or `tis.db` change. Web Service only; not deployed; Acceptance D follows.

2026-09-24 - Deployment Acceptance Correction B: Student identity, automatic Classification, current Talent workflow

1 Current Talent workflow now presents Student -> Program -> Assessment ->

automatic Classification (Exceptional only = Talented). Review Candidate / Official Identification moved out of primary navigation and every normal Student surface; preserved as labeled "Legacy Review & Identification History".

1 New shared `static/js/talent-student-identity.js`; Learning Style and current

Classification added to the Student Assessments roster, Assessment header, Students Across Programs, dashboard Student preview, Learner Profile, and Student Drill via smallest authorized projection extensions (no schema, no persistence, no new permission, no widened Student access).

1 Fixed a Student Drill `TypeError` (unhashable nested `overall_result` in the

context de-duplication key).

1 Deliberately updated pinned expectations (Talent nav, branch-comparison

metrics, Student Drill approved fields, removed legacy filter/columns).

1 Web Service only; not deployed; `tis.db` unchanged; Acceptance C/D not started.

2026-09-24 - Deployment Acceptance Correction B Remediation (ClinePass Precision Remediation)

1 Independent Codex audit found the Legacy Review & Identification History

backend projection did not actually return `classification`/`is_talented` (the frontend rendered fields the backend never provided). Fixed `talent_review_workspace` to return them from `talent_classification_service.assessment_classification(..., overall=)`, reusing the already-computed Overall Program Result (no N+1). Classification stays M17-authoritative; legacy Review/Official Identification never drive it; Exceptional only = Talented.

1 Normalized Learning Style absence wording to "Unassigned" in

`templates/_learning_style.html` (Student Profile + edit selector), matching Talent surfaces; unrelated "Not assigned" copy is unchanged.

1 Added a Student Drill dedupe regression test proving the canonical JSON key

survives a nested `overall_result` and preserves distinct contexts.

1 Backend/template/tests + KMS only; no schema/migration/tis.db change; Web

Service only; not deployed.

2026-09-24 - Deployment Acceptance Correction A: Talent & Potential runtime loading reliability

1 Fixed views stuck on "Loading your authorized workspace...": M16 gated

`talent-rubric-visual.js` to four views while `talent.js` dereferenced its global at evaluation time, aborting initialization on every other view. The module now loads on all views and is resolved defensively.

1 Added an outermost guarded `boot()` (error state + Retry, missing-delegate

state), bounded read requests (25 s; 15 s selector lookups; 20 s boot context deadline) with explicit timeout state, and independent section loading/error/ Retry states for Organization Overview and Results & Analytics (no more all-or-nothing `Promise.all`; analytics semantics unchanged); `aria-busy`/ status reach a terminal state. (The "no duplicate requests" and "error text is sanitized" claims were overstatements at this commit - corrected by the follow-up remediation below.)

1 Tests: stub-harness `tests/talent_runtime_loading.test.cjs`, per-view script

dependency checks in `tests/test_talent_ui.py`. Not a real-browser test.

1 No schema, migration, authorization, tenant, or privacy change. Web Service

only; not deployed.

2026-09-24 - Deployment Acceptance Correction A Remediation (ClinePass Precision Remediation)

1 Independent Codex audit found two blocking defects in Acceptance A's frontend

runtime path: raw backend `data.detail` disclosure (`talent.js` `parseApiResponse()`/`operationApi()`, `talent-experience.js` `fetchRubric()`), and a duplicate rubric-distribution request issued by both `talent.js` `rubricReq()` and `talent-experience.js` `ensureRubricSection()`.

1 Fixed with two shared, always-loaded helpers: `static/js/talent-api-errors.js`

(curated status/code error mapping; arbitrary `detail` and raw exception/SQL/ stack/endpoint text can never surface) and `static/js/talent-rubric-request.js` (single-flight read ownership keyed by Program + Academic Year + `assessment_state`; `talent.js` owns the request, `talent-experience.js` reuses it). Retry re-issues; Program/year change resets the store.

1 `tests/talent_runtime_harness.cjs` now executes talent-rubric-visual,

talent-api-errors, talent-rubric-request, talent.js and talent-experience.js together and observes fetches from both modules; the runtime-loading suite grew from 23 to 30 tests. Per-view script dependency tests now require the two new helpers before `talent.js`.

1 No backend, schema, migration, authorization, tenant, privacy or analytics

change. Web Service only; not deployed; `tis.db` unchanged.

2026-09-24 - M18b-3 M14-M18 correction program closeout (final regression / performance / privacy verification)

1 Verification and KMS closeout only; no product semantics, analytics family,

filter, schema, or migration change. The correction program (M14-M18) is functionally implemented on `dev` and closed for owner-acceptance/release planning only; not deployed, not merged to `master`.

1 Repository consistency audit: no current executable contradiction of the final

M14-M18 semantics was found. Stored four-percentage Student columns and their write validators remain as deprecated storage only; legacy Official Identification/"Officially Confirmed" labels remain as legacy history.

1 One bounded fix: `assessment_classification` accepts an optional pre-computed

`overall`; the Student Drill passes it, removing the doubled per-completed-row query cost introduced in M18a (2 -> 4 queries per row back to 2). Guarded by new tests in `tests/test_talent_organization_student_drill.py`.

1 Results: scoped Talent/Student Python suites and Node suites were rerun and

every failure was classified against worktrees at `9771b3c` and the pre-program commit `50c049f`; no failure is a new correction-program regression (Node: three stale expectations of the removed four-dimension Learning Style Branch metric; all other failures reproduce at `50c049f`). The Student Drill fixed query budget, PostgreSQL FK-order tests, and SaaS permission-pattern test remain documented pre-existing follow-ups (see `docs/PROJECT_STATE.md`).

1 `tis.db` unchanged. No deployment; Web and Workflow do not need to deploy

together for this change (no Workflow change).

2026-09-24 - M18b-2b Results & Analytics IA reorder + Classification filter + accessibility/responsive audit + candidate_membership_count decision

1 Bounded completion pass over M18b-2's own explicit open-item list; no

backend semantic change, M18b-1 contract untouched.

1 **(a) IA reorder:** the `analytics` view's in-content page header

(`lede()`) now reads "Results & Analytics" with a dedicated subtext, replacing the generic "Your organization at a glance" copy. The shared workspace chrome title (`VIEWS["analytics"]`, "Organization Overview") is deliberately unchanged - it is pinned by `tests/ test_talent_organization_analytics_providers.py`. The 9 sections now render in the required order: header -> summary cards (now includes one backend-sourced "Talented (Exceptional) Students" fact) -> Learning Style -> Classification -> Current Talent -> Competency Analysis (Program result + competency averages + rubric distributions, now grouped together - `rubricSection` moved from the end of the page to immediately after `competencyAverageSection`) -> Results/Evaluation Progress (`progressionSection` moved before the Branch comparison section) -> Branch/Organization comparison -> navigation links.

1 **(b) Classification filter:** a real, progressive `Classification`

filter (`<select name="classification">`, the exact 5 backend `CLASSIFICATION_LABELS`) was added to `templates/talent/workspace.html`'s shared filter form and wired only into the `/results-analytics/.../classification` request (never `/talented`). Hidden/cleared until a Program is selected (`updateClassificationVisibility`). Evaluation Period/Competency filters were evaluated and deliberately not

added (would need new supporting list endpoints, beyond this task's smallest-coherent-change scope); Learning Style's own backend filters (`branch_id`/`grade_level`/`section_name`) are already fully covered by the existing Branch/Grade selectors.

1 **(c) Accessibility audit:** confirmed directly (grep) that no

chart-type selector exists anywhere in `static/js/talent.js` (bar-only compatibility matrix, so none is required). The new Classification filter is a native `<select>` inside a `<label for>`, identical to every existing filter field, so it is natively keyboard-operable with no `tabindex` manipulation.

1 **(d) Responsive verification:** independently re-checked

`static/css/talent.css`'s breakpoints against every class the M18b-2 markup uses (`bucketBars`/`bucketTable` -> `.tp-grade-chart/ .tp-grade-row/.tp-table-wrap`; `talentedSection` -> `.tp-primary- indicator`; the new filter -> generic `.tp-filters label/select`) - all already covered at the 680px breakpoint; no gap found, no CSS change made.

1 **(e)** `candidate_membership_count` **decision:** kept - it is a distinct

Review-Candidate-policy-match count, never the M17 Classification/ Talented grain. Its Results & Analytics summary-card label is relabeled `"Legacy: Meets Program Criteria"` (previously unqualified) so it can never be read as a current-Talent figure; metric/data/permission unchanged; the separate `candidate_count` label (Program Portfolio/ Branch pages) is untouched.

1 Regression: `tests/talent_results_experience.test.cjs` 26/28 passed (4

new tests; the identical 2 pre-existing failures by exact test name at both this change and the unmodified `5aa13cb` baseline, git-stash- verified); `tests/talent_branch_comparison_frontend.test.cjs` + `tests/talent_experience_adjustments.test.cjs` + `tests/talent_ui.test.cjs` 38/41 passed (identical 3 pre-existing failures, git-stash-verified); `tests/test_talent_results_analytics.py` + `tests/test_talent_ui.py` 51/51 passed; `tests/test_talent_organization_analytics_providers.py` 22/22 passed. No schema/migration change; `tis.db` byte-identical before/after (SHA-256 verified).

1 Still open for M18b-3: Evaluation Period/Competency filter controls

(need new supporting endpoints); full broad M14-M18 regression/ performance/KMS closeout.

2026-09-24 - M18b-2 Results & Analytics Frontend Rebuild (Current-Talent indicator + Learning Style/Classification consumption)

1 Bounded FRONTEND-ONLY sub-phase consuming the already-approved M18b-1

backend contract (`/api/talent/results-analytics/...`). No backend semantic change, no new `MetricCode`, no change to legacy `TalentReviewCandidate`/`TalentOfficialIdentification` services.

1 `static/js/talent.js`'s `analytics` view (the Results & Analytics landing

page) now fetches and renders all three M18b-1 families alongside the pre-existing, unchanged competency (`/api/talent/analytics/.../ rubric-distribution`) and progress/branch-comparison (`/api/talent/evaluation-progress/...`) consumption:

1 **Learning Style Distribution** - new `distributionSection`/

`bucketBars`/`bucketTable` chart+table pair rendering the exact 8 categories + Unassigned the backend returns, gated on the `students.view` permission (additively surfaced in `talent_permissions` by

`routers/talent_ui.py`, presentation-only, no new authorization scope).

1 **Classification Distribution** - the same chart+table pair rendering

the exact 5 backend bands, Program-bound (requires a selected Program, matching the existing Competency section's gating pattern).

1 **Talented (Exceptional) Students** - new `talentedSection` primary

indicator (count/applicable-denominator/rate, plus an optional per-Branch breakdown), replacing the removed legacy `identified_of_eligible` (Official Identification) primary indicator. The Organization figure is rendered exactly as returned by the backend's `sum_raw_counts_across_branches` rollup - the frontend never averages the per-Branch rates it displays alongside it.

1 Removed the legacy `identifiedIndicator`/`identificationAllowed`/

`identifiedMap`/`identifiedCell` primary-Talent-indicator code path from the Results & Analytics landing view. The underlying legacy Official Identification/Review Candidate data, services, routers, and the separate Talent Review workspace are completely unchanged and remain available; `identified_of_eligible`/`candidate_of_eligible` remain valid, unchanged selectable options only in the separate legacy Branch-comparison metric selector (`branchComparisonMetricOptions`, a different view/control), never in the current-Talent section.

1 All new chart sections reuse the existing `tp-grade-chart`/`tp-grade-row`/

`tp-grade-track`/`table()` CSS/markup primitives - no new charting library, no `static/css/talent.css` change, no new script bundle (the existing `templates/talent/workspace.html` M16 conditional-script-loading gate for the `analytics`/`talent-map`/`portfolio`/`overlap`/`students`/ `longitudinal` view family is unchanged).

1 Updated `tests/talent_results_experience.test.cjs`: replaced the two

stale assertions that locked in the removed legacy `identified_of_eligible` primary indicator with tests for the new `talentedSection`/`bucketBars`/ `bucketTable`/`distributionSection` contract, including the exact M18b-1 Branch A 50% + Branch B 10% -> Organization ~10.9% (never 30%) proof case rendered through the UI layer, and a Learning Style/Classification privacy-neutral-state/no-"Protected for privacy" regression.

1 Full regression re-verified via `git worktree` comparison against the

unmodified M18b-1 baseline (`2bf9b8c`): `tests/talent_*.test.cjs` is 170 tests/154 passed/16 failed on this task's changes versus 165/149/16 at baseline - the same 16 pre-existing failures by exact test name in both runs, zero new JS failures. `tests/test_talent_results_analytics.py` + `tests/test_talent_ui.py` (the exactly-named M18b-2 focused regression set): 51/51 passed, unchanged. No schema/migration change, `tis.db` untouched (SHA-256 verified byte-identical before/after this task).

1 Competency, Progress/Evaluation-Period, and Branch/Organization comparison

sections were NOT rebuilt in this task beyond confirming they already consume the correct existing governed endpoints unchanged - a full 9-section information-architecture reorder (page header/filters/summary cards/Learning Style/Classification/Talented/Competency/Progress/Branch comparison exactly in that order, filter-bar progressive disclosure across Program/Branch/Grade/Section/Evaluation Period/Competency/Learning Style/Classification, full WCAG audit, and mobile responsive verification) remains open follow-up work for a subsequent M18b-2b/M18b-3 pass - this task targeted the specific, highest-value defect (legacy metric presented as current Talent state) and the three new backend families becoming reachable from the UI, not a full visual/IA rebuild of the page.

2026-09-24 - M18b-1 Results & Analytics Backend Contract + Correct Aggregation Authority

1 Bounded BACKEND-ONLY sub-phase of M18b (the visible Results & Analytics

page/chart rebuild is the separate, still-pending M18b-2).

1 Added `talent_results_analytics_service.py` and one coherent route family,

`routers/talent_results_analytics.py` (`/api/talent/results-analytics/...`), serving three families: Learning Style (thin reuse of `student_learning_style_analytics.py`), Classification (current M17 five-band distribution over the M18a-governed completed+current grain, reusing `talent_classification_service.assessment_classification` - never a duplicated band table), and Talented (Exceptional-only count/denominator/ rate).

1 `talent_org_intelligence_contract.MetricCode` remains frozen and

unmodified (ADR 0044); the new grain reuses the existing M9 `talent_analytics_service` context/filter/scope architecture and generic `Cell`/`Group` privacy primitives with a new opaque privacy class (`"P4"`), never the legacy Candidate/Identification metrics.

1 Added `sum_raw_counts_across_branches` as the ONLY Organization/Branch

rollup rule (raw-count sum, never an average of Branch percentages) with an exact regression proof: Branch A 1/2 Talented (50%), Branch B 9/90 Talented (10%) -> Organization 10/92 (~10.87%), not the naive 30% average.

1 Removed the dead `learning_style_dimension` query parameter (inert since

M14) from `routers/talent_evaluation_progress.py`'s branch-comparison route and from `branch_comparison_metric`'s signature in `talent_evaluation_progress_service.py`; removed the one stale frontend assertion for the now-nonexistent parameter in `tests/talent_branch_comparison_frontend.test.cjs`.

1 Competency and Evaluation Period/Overall Result analytics are deliberately

NOT reimplemented - the pre-existing `/api/talent/analytics/.../ rubric-distribution` and `/api/talent/evaluation-progress/...` routes already serve those two families correctly and M18b-2 consumes them directly instead of a duplicated wrapper.

1 Added `tests/test_talent_results_analytics.py` (19 tests: pure raw-count

aggregation proof, classification band/exclusion/reuse coverage, Branch/Organization Talented scope enforcement, privacy suppression, router permission gating, dead-parameter regression).

1 No schema migration, no `tis.db` change, no new permission keys (reuses

`students.view`/`talent_analytics.view`). Full regression re-verified via `git worktree` comparison against the unmodified M18a baseline (`81cfa75`): the same 11 pre-existing Python `-k talent` failures and the same (minus the one now-fixed stale assertion) JS failures reproduce unchanged, by exact test name - zero new regressions. M18b-2 (the visible page/chart rebuild consuming this contract) remains out of scope.

2026-09-23 - M18a Current Talent Authority Alignment + Learning Style Cleanup + Privacy UX

1 Resolved (not invented) the applicable-current-result authority for

automatic classification: the `TalentStudentAssessment` row where `status == 'completed'` AND `is_current == True` - the pre-existing ADR 0036 reassessment authority already consumed by M9 analytics and the B9 Student Drill.

1 The Learner Profile and B9 Student Drill now additively expose the same

M17 backend classification (`classification`/`classification_score`/ `is_talented`) already on the Talent Assessment API, reusing the single classification authority. Building a new org/Branch aggregate "current Talented count" metric was evaluated and explicitly deferred to M18b (the `MetricCode` enum is frozen at 14 values per ADR 0044; a new classification-grain aggregate is new analytics infrastructure, not cleanup).

1 Relabeled `static/js/talent.js`'s `candidate_of_eligible`/

`identified_of_eligible` metric labels ("Talent share"/"Officially confirmed share") to "Legacy review share"/"Legacy identification share" - they read as current Talent-status shares while their underlying data is the legacy Review/Identification workflow. No metric data, permission, or computation changed.

1 Completed the M14 Learning Style deprecation for read exposure: removed

the four `learning_style_*_percentage` fields from `routers/students.py`'s and `routers/students_ui.py`'s current API/UI projection (stored columns untouched); removed the now-confirmed-unreachable `learning_style_branch_aggregate`/`_learning_style_values_by_branch`/ `_LEARNING_STYLE_COLUMNS` dead code and their stale tests from `talent_evaluation_progress_service.py`.

1 Removed the literal visible phrase "Protected for privacy" from Talent &

Potential / Learning Style UI, replaced with context-appropriate neutral copy ("Unavailable", "Distribution unavailable", "This ... is not available for this selection"). Presentation-only: the underlying primary/complementary suppression, minimum-population, anti-reconstruction, and tenant/Branch/organization privacy contract is unchanged.

1 No schema migration, no `tis.db` change, no new permission keys. M18b (the

full Results & Analytics page rebuild, including any new org/Branch aggregate Talented-count analytics) remains out of scope.

2026-09-23 - M17 Automatic Assessment Classification + New Normal Talent Workflow

1 Completing a Student Assessment now automatically classifies it into one

of five fixed, owner-approved 1.00-5.00 bands (Needs Improvement, Developing, Meets Expectations, Advanced, Exceptional) and derives `Talented` (Exceptional only) - never a percentage conversion, never AI-derived, never frontend-derived.

1 Resolved the architecture question of how a 1-N rubric-scale Program

result reaches a fixed 1.00-5.00 classification: real Programs configure varying rubric-level counts (verified in `talent_local_test_data.py`), so TIS added a governed deterministic linear projection (ADR 0037's 2026-09-23 Amendment) rather than mandating a five-level rubric system-wide.

1 The current normal workflow no longer requires a manual Review Candidate

or Official Identification step to make a Student Talented. Existing Review Candidate/Official Identification records and services are fully preserved as legacy/history and remain independently reachable, but no longer govern current

`Talented` state.

1 Added `talent_classification_service.py`; additively exposed

`classification`/`classification_score`/`is_talented` on the Talent Assessments and legacy Review Candidate API responses; updated the Talent assessment and legacy Review workspace UI to show the automatic classification. No schema migration, no new permission keys, no tenant/ privacy boundary change.

2026-09-23 - M16 Talent Performance And Student Action Layout

1 Reused the Talent route's authorized permission projection for shell/action

presentation, removing the per-permission backend projection loop.

1 Made Talent operational scripts surface-specific while preserving dependency

order and common presentation scripts.

1 Made Students action groups wrapping horizontal flex rows so Add/Delete and

row actions remain adjacent where space permits.

1 Added regression coverage for projection bounds, script ownership, and the

responsive action rule. No schema, API, permission, privacy, tenant, Branch, or product-semantic behavior changed.

2026-09-23 - M15 Student Integrity, Talent Visibility & Roster Round-Trip

Two owner requirements, implemented on the isolated `m15-student-integrity-roster` feature branch (off `50c049f`, parallel to M14 which is not merged):

Student count / Talent visibility (integrity verification, no cascade defect). Independently verified the permanent-delete graph: `force_delete_student_history` already deletes, child-before-parent, every Student-owned table (`TalentOfficialIdentification`, `TalentEducatorInput`, `TalentReviewCandidate`, `TalentAssessmentAudit`, `TalentStudentCompetencyResult`, `TalentStudentAssessment`, `TalentAssessmentCyclePopulationMember`, `StudentAcademicPlacement`, `StudentExternalIdentifier`, `StudentAudit`) then the `Student` row, and never commits itself (caller controls the transaction; rollback restores the Student). Talent current-operational surfaces (`current_placements_for_assessment`, `talent_org_student_drill`, `talent_analytics._list_student_rows`) structurally INNER-join `models.Student`, so a force-deleted Student cannot appear as a current/live Student. The owner's "~9 vs ~13" discrepancy is a **population-scope difference, not a deletion defect**: the Students list defaults to the actor's authorized Branch (single Branch unless `students.view_all_branches` + org/global scope) and current effective Placement, while Talent/Review surfaces span frozen `TalentAssessmentCyclePopulationMember` membership (historical, multi-Branch) and the eligible-students roster deliberately does not filter `Student.status` (ADR 0039: status never reinterprets historical eligibility). No schema or cascade change was needed; the behavior is documented and locked with regression tests (`tests/test_student_delete_talent_visibility.py`).

Student roster round-trip (export -> import compatibility + NO_CHANGE). `student_roster_service.py` previously exported `section_display` and a `STD`-prefixed Student ID that the importer rejected (`unexpected_column` / `invalid_student_id`). M15 makes the same exported workbook re-uploadable: `section_display` is now an accepted-but-ignored display-only column (never identity; canonical `section` remains identity, ADR 0045 unchanged); the canonical `STD` prefix is stripped back to the bare 10-digit business value on import. Import remains **create-only**: an unchanged exported row now classifies as deterministic **NO_CHANGE** (managed Student ID resolves identity, then name/gender/status + current Placement Branch/Year/Grade/Section must match

exactly), new rows classify **CREATE**, and any mutated existing row stays a blocked `student_id_conflict` (never a silent update/merge/upsert). Apply writes only CREATE rows and skips NO_CHANGE rows with zero mutation, preserving atomic all-or-nothing behavior. Export gains a bold/filled header, frozen header row, deterministic column widths, and autofilter (no macros, no new spreadsheet dependency). `static/js/students-roster.js` renders CREATE / No change / error states from backend-provided row status. Permissions (`students.import`/ `students.export`), tenant/Branch scope, Student-ID uniqueness, and audit rules are unchanged. 40+ new/updated tests in `tests/test_student_roster_import_export.py` and `tests/test_student_delete_talent_visibility.py`.

2026-09-23 - M14 Students + Talent & Potential - Learning Style Correction + Aggregate Distribution

Owner-directed correction milestone following a read-only Post-M13 Correction Review, implementing two corrections: (1) `Student.learning_style` is the single authoritative Learning Style field, extended from four to eight values (Visual, Auditory, Read/Write, Kinesthetic, Verbal, Non-verbal, Quantitative, Spatial); the four `learning_style_*_percentage` columns (ADR 0042/M1-M3), previously an independent per-Student percentage profile, are now operationally deprecated - no longer written, displayed, or read as Learning Style authority anywhere, with existing stored values preserved completely untouched and physical column removal separately gated on a later data-occupancy verification. (2) "Percentage" means population/ aggregate distribution, never a per-Student dimension score - `student_learning_style_analytics.py` (reused, extended rather than parallel-built) now covers all eight values plus an "Unassigned" bucket for the authorized Student population, with the denominator always including Unassigned Students; the M4/M10 Talent Evaluation Progress "Learning Style" Branch-comparison metric (which averaged the four deprecated columns) is removed (`talent_evaluation_progress_service.APPROVED_BRANCH_METRICS` is now six metrics). Database: one forward-only migration (`20260923_002_student_learning_style_eight_values`) widens `ck_students_learning_style` on PostgreSQL; no column added or dropped, no row rewritten. ADR 0031 and ADR 0042 are amended (not rewritten) with M14 sections recording the corrected model; ADR 0044 is amended recording the removed Branch-comparison metric. `student_roster_service.py`'s column contract does not include Learning Style or the four percentage columns and is untouched. This entry does not rewrite the M1-M13 historical record of what was actually shipped (see the entries below and `docs/releases/2026-09-23-students-talent-m1-m13-release-handoff.md`) - it records the correction as new, dated fact. Does not itself authorize or perform any deployment.

2026-09-23 - M13 Students + Talent & Potential Release Readiness Closeout

Final documentation-only release-readiness verification for the bounded Students + Talent & Potential update package (M1-M12.1: Student ID, Learning Style, Al-Andalus Section display, Student roster, Talent frontend cleanup, Evaluation Progress, Results & Analytics). Spot-checked each feature area against current code and found implementation matches documented behavior; re-ran `tests/test_permission_registry_matrix.py` (13 passed), `tests/test_student_managed_number_service.py` (46 passed), and, on real PostgreSQL, `tests/test_postgresql_migration_transactions.py` (11 passed, 5 failed - the same pre-existing, unrelated Talent baseline-metadata foreign-key-ordering failures M12.1 already documented, unchanged). Corrected three stale pre-M11 "deferred"/"assigned to M11"/"remains absent" roster wording instances in `docs/AI_PROJECT_CONTEXT.md`, `docs/TIS_MASTER_CONTEXT.md`, and `docs/engineering/TIS_MODULE_MAP.md` to reflect that M11 has since implemented roster import/export; no other stale wording scoped to this package was found and dated historical entries were left unchanged. Added `docs/releases/2026-09-23-students-talent-m1-m13-release-handoff.md` (indexed in `docs/README.md`) as the durable release-handoff record covering user-visible changes, permissions, required migrations and the PostgreSQL remediation requirement, compatibility/privacy/tenant-isolation notes, known pre-existing issues, deployment surfaces, migration/deployment sequence, rollback considerations, and a post-deploy smoke matrix. No schema, migration, permission, or product behavior change; `tis.db` verified byte-identical before

and after. See `docs/PROJECT_STATE.md` for the full record. Closes M13 verification for this bounded package only, not the entire Talent product roadmap.

2026-09-23 - M12.1 PostgreSQL Managed Student Number Index Identifier Remediation

Fixed the PostgreSQL identifier-length defect M12 documented but deliberately did not fix. The M1 managed-Student-number active-per-Student partial unique index was declared as `uq_student_external_identifiers_tis_student_number_active_student` (65 characters), exceeding PostgreSQL's 63-character `NAMEDATALEN` limit; PostgreSQL silently truncated it on `CREATE UNIQUE INDEX` to a different, 63-character physical name (`uq_student_external_identifiers_tis_student_number_active_stude`), which broke this migration's own idempotency check and risked a `DuplicateTable` failure if the migration ledger were ever re-applied against a real PostgreSQL deployment - reproduced live against a real PostgreSQL test database in this task.

The canonical replacement name is `uq_student_external_identifiers_tis_number_active_student` (57 characters, verified ?63). `models.py`'s `Index` declaration and the original `20260922_002_student_tis_number_identifier_integrity` migration in `db_migrations.py` now both create this canonical name directly on any FRESH database (SQLite or PostgreSQL) - no intermediate old name is ever created. A new forward migration, `20260923_001_student_tis_number_index_identifier_length_remediation`, remediates an already-migrated PostgreSQL database: it inspects the actual PostgreSQL catalog definition (columns, uniqueness, partial predicate) of both the canonical and the old truncated physical name before acting, and only `ALTER INDEX ... RENAME TO` the old truncated index once its definition is verified to match the expected invariant; it is a no-op if the canonical name is already present, and it fails closed with a clear `RuntimeError` (no rename/drop) if an unexpected index occupies either name or if both names exist simultaneously. The partial-unique-index semantic invariant (`UNIQUE (student_id) WHERE namespace = 'tis_student_number' AND status = 'active'`, ADR 0043) is completely unchanged, as is the separate global Student-number value-uniqueness index (`uq_student_external_identifiers_tis_student_number_value`, 56 characters - never had a length problem). No `Student` or `StudentExternalIdentifier` row was read, rewritten, merged, or deleted by this migration. SQLite, including the checked-in `tis.db`, is unaffected (verified byte-identical via SHA-256); this is a PostgreSQL-only physical schema-identifier fix with no product/API/frontend change.

2026-09-23 - M12 Integrated QA / Regression Verification

Re-verified Students + Talent & Potential M1-M11 end to end (Student ID, Learning Style, Al-Andalus Section display, Evaluation Progress, Student roster backend/frontend, Talent frontend cleanup, Results & Analytics, permissions, tenant isolation, historical/frozen attribution, privacy/ suppression) with no new functional/UX regression found. Corrected one stale test fixture in `tests/test_permission_registry_matrix.py` (removed `routers/students_ui.py` from `talent_educator_inputs.view`'s recorded consumers, matching M8's already-shipped `include_educator_inputs=False` behavior - not a product change). Identified and documented, but deliberately did not fix (out of scope for this verification-only task), a PostgreSQL identifier-length defect in the M1 managed-Student-number unique index (`uq_student_external_identifiers_tis_student_number_active_student`, 65 characters, exceeds PostgreSQL's 63-character limit) that risks breaking this migration's idempotency on a real PostgreSQL redeploy; see `docs/PROJECT_STATE.md` for the full finding. No schema, migration, or product behavior change.

2026-09-23 - Student Roster Import / Export Frontend Implemented (M11)

1 Added permission-projected Import Students and Export Students actions to the

existing Students list, using only the M6 `.xlsx` routes.

1 Added a transient accessible import dialog with file selection, stateless

preview, structured backend-safe row feedback, explicit confirmation, and re-uploaded/revalidated atomic create-only apply.

1 Preserved canonical textual Student IDs, privacy-safe duplicate disclosure,

canonical Section identity, backend authorization, and M5 presentation-only `section_display` semantics.

1 Added no CSV/`.xls`, client spreadsheet parser, partial apply, persistent

batch, background job, backend semantic change, schema, or migration.

2026-09-23 - Added the primary Branch comparison chart

1 Replaced Organization Overview's fixed Branch summary with one

Program-scoped chart over the existing M4 Branch comparison endpoint.

1 Added the exact seven approved metrics, including permission projection for

Candidate/Identification metrics and the four current Learning Style dimensions.

1 Preserved backend Branch/Period order, frozen attribution, real zero, and

distinct protected/no-data presentation without exposing hidden counts.

1 Kept framework-incompatible Overall Result non-numeric and added no client

aggregation, Organization derivation, privacy logic, second chart, backend semantic change, schema, migration, or M11 roster frontend.

2026-09-23 - Added backend-authoritative Student Evaluation Progress UI

1 Added compact Evaluation Progress to each Program and Academic Year in the

Student Profile Talent tab, using the existing M4 Student endpoint.

1 Preserved backend Period order and configured labels; available zero remains

`0%`, while Pending/unavailable states remain textual and non-numeric.

1 Displayed Overall Result only from the backend-provided comparable value.

1 Kept individual Period results and explained the boundary when Framework

versions are not comparable, without calculating an equivalence.

1 Added no client-side weighting/averaging, aggregate UI, chart, backend

semantic change, schema, or migration; frozen context and M5 Section display remain unchanged.

2026-09-23 - Removed retired controls from normal Talent UX

Area/module: Talent & Potential frontend

New state: Normal Student Assessment and Student Talent-profile presentation no longer expose Reload Saved Rubric or Educator Input. Frontend-only fetches and handlers used exclusively by those controls were retired without changing

backend contracts or stored evidence.

Preservation and exclusions: Historical assessments, rubric evidence, Educator Input records/lineage, audit, Official Results, frozen context, and M5 Section display remain intact. No four-dimension Learning Style Talent presentation, M9/M10/M11 frontend work, schema/migration, scoring, or privacy change was included.

2026-09-23 - Implemented Students M7 identity and Learning Style frontend

Area/module: Students frontend

New state: Student create/list/profile/edit surfaces now consume the canonical managed Student ID and four nullable Learning Style percentage contracts. The UI owns only presentation and input affordances: fixed `STD` prefix plus ten digits, safe duplicate messaging from the shared backend projection, independent Verbal/Non-verbal/Quantitative/Spatial inputs, and accessible profile bars with distinct zero/unavailable states. Existing Placement history and M5 Section display projections are unchanged.

Scope exclusions: No roster frontend (M11), Talent M8 cleanup, Evaluation Progress M9 UI, Results/Analytics M10 charts, schema/migration, API redesign, or backend business rule was added.

Documentation updated: Yes. AI Project Context, Master Context, Project State, Change History, Module Map, and User/System Flows.

2026-09-23 - Student Roster Import/Export Backend Implemented (M6)

Implements the .xlsx-only Student roster import/export backend authorized by the governance prerequisite below. Adds `student_roster_service.py` (create-only workbook import for this first release, stateless preview, independently-revalidated atomic apply, `.xlsx` export) and three new routes in `routers/students.py`: `GET /api/students/roster/export` (`students.export`), `POST /api/students/roster/import/preview` and `POST /api/students/roster/import/apply` (`students.import`). Reuses `student_academic_service.py`'s canonical Student/TIS Student number/Academic Placement invariants throughout; extracts one new shared helper, `describe_student_number_conflict`, so the direct create API and the roster preview/apply share exactly one implementation of the ADR 0043/M2 privacy-safe conflict-disclosure contract. Reclassifies `students.import`/`students.export` from dormant (`D`) to active-enforced (`A`) in `tests/test_permission_registry_matrix.py` (dormant 15?13, active 143?145); no role-grant or default-permission change. `section_display` (ADR 0045) stays additive-only on export and is never import matching identity (canonical `PlanningSection`/`grade_level`/`section_name` only). No schema, migration, or frontend change. 40 new focused tests added in `tests/test_student_roster_import_export.py`. See `docs/PROJECT_STATE.md` ("Student Roster Import/Export Backend Implemented (M6)").

2026-09-22 - Student Roster Import/Export: Permission Registration (Governance Prerequisite, M6)

Owner-approved governance prerequisite for the future M6 Student roster import/export backend. Registers two new semantic capability permission keys in `permission_registry.py`'s `students` group - `students.import` ("Import student roster data") and `students.export` ("Export student roster data") - following the exact existing `students.*` naming, grouping, description, and ordering conventions, and classified dormant/ reserved (no route/UI consumer yet), matching the established `teachers.import`/`teachers.export` precedent. Neither key was added to any developer-only, owner-only, administrator-only, or limited-read-only set, and neither was added to `_EDITOR_LIKE_PERMISSIONS`; no new default-role grant decision was made by this task. This task registers permission keys only - no roster import/export route, service logic, or frontend was added, and no Student data, schema, or migration changed. See `docs/PROJECT_STATE.md` ("Student Roster Import/Export - Permission Registration (Governance Prerequisite, M6)").

2026-09-22 - Al-Andalus Section Display M5: Shared Presentation Implementation

Implements the ADR 0045-authorized presentation convention: one shared `academic_grade.format_section_display(workspace_uuid, grade_level, section_name) -> str` helper, activated only on exact `SchoolGroup.workspace_uuid` equality to `72e52eb2-3844-447b-92a8-c55015f73257`, rendering `{grade_number}.{section_ordinal}` (Section A=1...Z=26) and falling back to the unchanged canonical `section_name` for every other workspace and every non-deterministic Grade/Section. Wired into: Students UI current placement and placement history, the Academic Placement Section selector, the Students list Grade/Section filter, the Talent frozen historical Learner Profile context (using frozen `TalentAssessmentCyclePopulationMember` Grade/Section, never the Student's current placement), the Talent Student Assessment eligible-students roster, and the Talent Results/Analytics Grade/Section filter. Canonical `PlanningSection`/Student Academic Placement/frozen Talent Grade-Section values, analytics grouping identity, and query/filter/import-export identity are unchanged everywhere - `section_display` is a bounded, additive presentation projection only. No schema, migration, or new permission. See `docs/PROJECT_STATE.md` ("Al-Andalus Section Display M5 - Shared Presentation Implementation") and ADR 0045.

2026-09-22 - Al-Andalus Section Display: Governance Authorization (KMS-only)

Per new ADR 0045, the Owner authorized the presentation-only Al-Andalus Section-display convention (Grade/Section rendered as `{grade_number}.{section_ordinal}`, e.g. Grade 1/Section A -> `1.1`, with deterministic alphabetic Section-ordinal mapping and an unchanged-fallback rule for anything that does not map deterministically) for the exact verified production `SchoolGroup.workspace_uuid` `72e52eb2-3844-447b-92a8-c55015f73257`. The workspace identity was identified using the repository's existing read-only production audit (`scripts/audit_al_andalus_readonly.py`), run by the Owner against the deployed Render PostgreSQL environment (exact_name "Al-Andalus", no duplicate-organization conflict, audit exit code 0); no credentials, connection strings, or other production detail were recorded. This closes the "deferred" status recorded for this feature at the Students + Talent & Potential M1 milestone and in ADR 0042's boundary note. This is a KMS governance correction only - no application code, schema, or migration changed, and the feature itself (a shared server-side presentation helper and its Student/Talent UI surfaces) is not implemented by this change. See `docs/PROJECT_STATE.md` ("Al-Andalus Section Display - Governance Authorization") and ADR 0045 for the full authorized convention, fallback rule, and canonical-identity-unchanged constraints.

2026-09-22 - Talent & Potential M4: Authoritative Evaluation Progress Analytics

Implemented the complete backend Evaluation Progress contract per new ADR 0044: Student Progress (per-active-Period result state, Current Overall Result excluding Pending/unavailable, authoritative counts), Branch/Organization Progress (`BranchPeriodResult`/ `OrganizationPeriodResult` computed directly from governed Student `normalized_percent` results via frozen historical Branch attribution - proven not to be an average of Branch averages), the seven approved Branch comparison metrics (a bounded local dispatcher reusing existing M9 coverage/candidate/identification breakdown providers unchanged for four of them), and the Learning Style Branch aggregate (ADR 0031/ADR 0042). Two owner-ratified decisions are recorded in ADR 0044: (1) Evaluation Periods combine into one Overall Result only when every contributing active Period shares one identical governed `framework_version_id`, else every Period is returned individually with a `null` combined Overall and `comparability_reason_code="framework_changed"`; (2) an individually authorized Student's own Evaluation Progress is governed by normal Student/Talent authorization and frozen-historical-Branch scope, never by aggregate cohort-size privacy suppression - that route has no privacy- policy dependency at all. Every Branch/Organization aggregate reuses only the M9 generic `Cell`/`Group`/`apply_primary_privacy`/ `run_complementary_suppression` primitives; the frozen M10 `MetricCode` registry is unmodified. New files:

`talent_evaluation_progress_service.py`, `routers/talent_evaluation_progress.py` (registered in `main.py`). No schema, migration, new permission, or frontend change. See `docs/PROJECT_STATE.md` for the full implementation-truth summary (including a disclosed, non-blocking characteristic inherited from the reused M9 breakdown providers) and `tests/test_talent_evaluation_progress.py` (25 tests) for the complete test matrix. A required second-pass adversarial privacy/reconstruction review across 16 specified risk categories found the implementation clean. Three pre-existing, unrelated test failures were directly reconfirmed present and unchanged on unmodified `dev` HEAD `96dd454` before this task.

2026-09-22 - Students + Talent & Potential M3: Four-Dimensional Learning Style Backend/Service/API

Implemented the ADR 0042-authorized server-side service/API for the four independent Student Learning Style percentages (`learning_style_verbal_percentage`, `learning_style_non_verbal_percentage`, `learning_style_quantitative_percentage`, `learning_style_spatial_percentage`) on the M1 schema foundation (no new migration). Each dimension is validated independently in `student_academic_service.py`: `null` is valid, an integer 0-100 inclusive is valid (including both boundaries), and there is no sum-to-100 rule; `bool` and any other non-integer type (float, numeric string) are rejected rather than coerced. `POST /api/students` and `PATCH /api/students/{student_id}` now accept the four fields under the existing `students.create`/`students.edit` permission gates (no new permission); the M2 mandatory `student_number` requirement on create is unchanged. `update_student` applies each field only when present in the request, preserving PATCH's existing partial-update semantics so an unspecified dimension is never cleared. Every Student JSON response now exposes all four fields with no derived dominant-style/total field. The legacy categorical `learning_style` field, its values, and its single-select nature are completely unchanged - no rewrite, no mapping, no automatic conversion in either direction. Percentage updates participate in the existing append-only `StudentAudit` trail with no new audit subsystem. See `docs/PROJECT_STATE.md` for the full implementation-truth summary and `tests/test_student_learning_style_four_dimension_api.py` (73 tests, including a Talent source-scan regression proving zero Talent scoring/ eligibility dependency) for the complete test matrix. No schema, migration, new permission, frontend, or `tis.db` change; the same 13 pre-existing failures unrelated to this task were directly re-confirmed present on unmodified `dev` HEAD before this task via an explicit before/after comparison run.

2026-09-22 - Students + Talent & Potential M2: Managed Student ID Backend/Service/API

Implemented the ADR 0043-deferred TIS Student number create/edit service/API on the M1 schema foundation (no new migration). Business input is exactly ten ASCII digits (leading zeros preserved); the service alone canonicalizes to `STD` + the 10 digits. `POST /api/students` now requires `student_number` for new Students and atomically creates the Student row plus its managed `tis_student_number` identifier (an identifier-insert failure rolls back the whole transaction, leaving no orphan Student); existing legacy Students and the unmodified HTML `/students/new` UI path are unaffected. A new `PUT /api/students/{student_id}/student-number` route (gated by the already-governed `students.manage_identifiers` permission - no new permission key) assigns or replaces a Student's number: replacement retires the old row (never mutates its value) and activates a new one, so `Student.id` is unchanged and the M1 global-uniqueness index keeps the old value permanently reserved. The generic external-identifier create/deactivate functions now reject the `tis_student_number` namespace so they cannot bypass the managed contract. Duplicate-value conflicts are classified only after the failing transaction is rolled back and disclose `student_id`/ `display_name` if and only if the conflict is same-tenant AND the requester independently holds `students.view`; every other case (cross-tenant, unauthorized same-tenant, or a retired value) returns one identical generic `student_number_unavailable` body. A canonical `student_number` field is now exposed on Student JSON responses. See `docs/PROJECT_STATE.md` for the full implementation-truth summary and `tests/test_student_managed_number_service.py` (46 tests) for the complete test matrix; three pre-existing tests that created Students via the API without a number were updated to supply one. No schema, migration, new permission, frontend, or `tis.db` change.

2026-09-19 - Talent & Potential sidebar icon uses the shared icon system (UI polish)

Removed the per-item `brand_logo` (Ghars PNG) override from the Talent & Potential nav item in `ui_shell.py`, so the sidebar renders the shared `sparkles` SVG from the existing `icon()` macro like the other modules. Cosmetic only: no routing, permission, label, structure, other-icon, asset, dependency, or schema change. The JS shell test that pinned the PNG was updated to assert the shared icon system.

2026-09-19 - Whole-Application Permission Qualification (Phase 3)

Audited all 175 registered permission keys, 468 routes and every nav module against the single canonical effective permission result. Added a governed registry evidence matrix (143 active / 14 platform-only enforced / 5 alias / 13 dormant / 0 unresolved), a route authorization enumeration, a classified dangerous-pattern scan, sidebar/Dashboard/route consistency tests for every nav module, freshness, role-change, tenant-isolation and platform-only tests, and template-source accessibility checks. Fixed `POST /scope/organization` so platform users must also hold the canonical all-school switch capability (`_can_manage_all_school_scopes`), deleted seven unused duplicate role-policy reader helpers, and corrected stale KMS dispositions (`planning.import`/`planning.export`, `observations.submit`, `dashboard.view_all_schools` are dormant). ADR 0040 precedence, storage, schema and the permission registry are unchanged; no migration. Web Service only; no Timetable Workflow change.

2026-09-19 - Role Permissions UI: User Exceptions (Phase 2)

Replaced the edit-user three-way Inherit/Allow/Deny override panel with a "User Permission Exceptions" section: binary Allow/Deny buttons, a Reset to Role Settings button (internal inherit == row deletion, shown only when an exception exists), server-rendered `aria-pressed`, a user summary, Effective/Source/User Exception metadata, and explicit handling of a stored inert Allow (kept, never shown as effective). `POST /users/permissions/{user_pk}` gained an optional `change="<key>|<allow|deny|reset>"` field on the same guarded route; the legacy paired-list contract is unchanged. `build_user_permission_payload` gained additive read-only source fields. ADR 0040 precedence, storage, and the service's three-state semantics are unchanged; no migration. A stored exception on a key that became non-assignable after a role change is kept and shown on a locked row with only "Reset to Role Settings" (no Allow/Deny; creating or changing it stays refused). "inherit" remains only the internal, backward-compatible API form of clearing an override. Detail in `docs/PROJECT_STATE.md`.

2026-09-19 - Role Permissions UI split: Role Packages / School Overrides (Phase 1)

Split the combined Role Permissions editor into two explicit modes on the same `/system-configuration/role-permissions` GET/POST route: `?mode=packages` (standard/global role editor - no SchoolGroup, no tenant name, no Platform Owner tenant context) and `?mode=overrides` (School Overrides - the only place a SchoolGroup appears, always as an explicit management target, never as actor/session context). Both modes reuse the existing `role_permission_service.py` resolver/writer unchanged; a new read-only `build_school_override_payload` adds the Standard/Override/ Effective comparison for School Overrides. This closes a real incident where the old single "Permission Scope: Global defaults / Selected school" dropdown made a tenant's Deny override visually indistinguishable from the Global default, and the Platform Owner UI appeared to belong to whichever school was selected. Full detail in `docs/PROJECT_STATE.md`'s "Role Permissions UI - Role Packages / School Overrides Split (Phase 1)" entry. Owner-review refinements in the same change: a Platform Owner must explicitly pick the School Overrides management target (no school pre-selected, explicit "Review" button instead of auto-submit), groups containing a School override open by default with an override count, and corrected ARIA/focus/markup (`aria-current` navigation, no static `aria-expanded`, Reset button outside the checkbox label, single-element banner text). School Overrides also drops "Select All" / "Clear All" (kept in Role Packages) so overrides

stay sparse differences from the Standard Role Package; per-permission "Reset to standard" remains the only reset in Phase 1.

Files changed: `main.py` (`_build_role_packages_context`, `_build_school_overrides_context` replace the single `_build_role_permissions_context` scope-dropdown logic; `POST` now takes a `mode` form field instead of `scope_type`), `role_permission_service.py` (new `build_school_override_payload`), `templates/system_configuration_role_permissions.html` (mode tabs, two distinct panels, Standard/Override/Effective comparison rows, "Reset to standard" control), `tests/test_permission_management.py` (`_update` helper's `scope_type` renamed `mode`; two GET-context assertions now request `mode=overrides` explicitly), and new `tests/test_role_permissions_ui_split.py`. No schema, migration, new permission key, or precedence-rule change; `tis.db` unmodified (hash verified unchanged before/after).

2026-09-18 - PR #360 review-response corrective pass

Corrected two GitHub review findings on PR #360, verified against the actual review comments/diff (not the delegated task summary alone):

1 Fixed a hiring-plan export permission bypass: `main._authorize_report_export`

only rejected the literal `section=hiring` request, while `section=full` produces a report containing the same hiring-plan sheet/section but skipped the `hiring_plan.export` check. The gate now covers every report-export section whose payload includes hiring data (`REPORT_EXPORT_SECTIONS_WITH_HIRING_DATA = {"full", "hiring"}`), kept in sync with the section branches in `_build_professional_report_xlsx_bytes` and `_build_professional_report_pdf_bytes`. `summary`/`subjects`/`teachers` sections remain unaffected.

1 Corrected `.kms-impact.yml`: the prior commit's KIA summary incorrectly

claimed no migrations/schema changes and dropped `database_schema` / `migrations` from `affected_areas`, even though this branch still carries the non-destructive `20260915_001_role_permission_logical_key_uniqueness` migration. The declaration now describes the full branch/PR scope and restores the accurate `affected_areas` and `kms_files_updated` list.

1 Investigated a third item from the delegated task summary (SchoolGroup

edit/delete/action authorization) against the actual PR #360 review comments and full diff; found no supporting review comment or changed SchoolGroup file in this PR, so no code change was made for it. See `engineering/PERMISSION_CLOSURE_REVIEW.md` for detail.

No new permission keys or schema changes were introduced by this corrective commit. ADR 0040 unchanged.

2026-09-18 - Close the 22 unresolved permission-registry keys

Resolved every key left unclassified by the independent closure review (`engineering/PERMISSION_CLOSURE_REVIEW.md`), tracing each against the implemented product rather than inventing new features or role-name checks:

1 Enforced dedicated existing capabilities: `dashboard.view_branch_summary` /

`dashboard.view_reports` (distinguishable dashboard KPI section and Reports tab), `dashboard.export_reports` and `hiring_plan.export` (report-export routes, the latter specifically for the hiring-plan section), `observations.view_reports` (teacher observation-cycle history page, for non-teacher actors only), `observations.sign_evaluator` (the actual evaluator-signature persistence path), and `configuration.view_audit_log` (an additional prerequisite under `/admin/audit-log`, alongside export).

1 Made explicit, non-duplicative aliases: `observations.submit` (there is no

submit step independent of the evaluator signature), `system_owner.manage_developer_accounts` (owner-identity is the existing enforcement mechanism for owner-only keys; the key is now cited in the denial response), and `system_owner.view_cross_school_audit` / `system_owner.export_cross_school_data` (both alias the single global, non-tenant-filtered audit log surface).

1 Classified as dormant/reserved with no implementation and no misleading

UI: `reports.view`, `teachers.import`, `teachers.export`, `subjects.manage_colors` (color is always auto-derived, never manually set), `observations.manage_templates` (rubric criteria are a hardcoded, auto-seeded fixture), `configuration.manage_global_defaults`, `system_owner.manage_subscriptions`, `system_owner.create_subscription_school` (both delegated to an external SaaS admin surface), and `system_owner.run_startup_repairs` (automatic startup behavior, not a user-triggerable action).

1 Resolved one real registry/implementation conflict fail-closed:

`configuration.view_audit_log` was tenant-assignable in the registry, but its only consumer is a global, non-tenant-filtered log; reclassified it platform-only (matching `configuration.export_audit_log`) rather than exposing cross-tenant audit data or building a new tenant-scoped viewer.

1 No new permission keys, migrations, or schema changes. ADR 0040 is

unchanged. See `engineering/PERMISSION_CLOSURE_REVIEW.md` for the full, per-key disposition and reasoning.

2026-09-18 - Independent permission closure review corrections

1 Guard notification-detail automatic read mutation with `notifications.mark_read`.

1 Fail closed for inactive platform identities in year and data capabilities.

1 Fail closed when tenant permission ownership is absent, stored/branch

ownership conflicts, or selected/explicit evaluation scope is foreign. Target management, override writes, and Branch deletion also reject ambiguous target ownership, including platform actors; valid cross-tenant authority is preserved.

1 Resolve Developer role choices through `users.assign_role`, retaining only

Limited as the safe creation fallback when role assignment is not granted.

1 Use `teachers.bulk_delete` independently from single-record delete in the

server guard and bulk controls; protect teacher qualifications, subject assignments, and capacity fields under their dedicated permissions.

1 Protect Planning homeroom and teacher-assignment changes under their dedicated

keys, preserving read-only fields and assignment rows on ordinary edits.

1 Correct the timetable-block navigation regression expectation and include

role-policy metadata in the deliberately minimal legacy migration fixture, matching production's core-table bootstrap prerequisite.

1 Make the shared Talent/Students test policy helper update its logical key

rather than insert duplicate rows when a test re-grants permission, respecting the new database uniqueness constraints.

1 Add independently persisted-policy regressions and record verification limits

in `engineering/PERMISSION_CLOSURE_REVIEW.md`. ADR 0040's precedence, Deny-over-Allow, platform scope, and user-override uniqueness remain unchanged. Unconsumed-registry policy needs clarification; this is not whole-system security approval. No real permission rows were cleaned up or rewritten.

2026-09-17 - Permission consistency closure (whole-registry pass)

A reported "grant/revoke/re-grant" defect on `subjects.view` was investigated as a whole-system permission-consistency defect rather than a Subjects-only issue, following the governing precedence chain (built-in role default -> global `RolePermission` override -> tenant `RolePermission` override -> per-user `UserPermissionOverride` -> effective permission; Deny-over-Allow; ADR 0040).

Root cause: several current-user authorization/UI-capability paths still resolved permissions through request-scoped in-memory caching helpers (`auth._has_cached_permission`, `_has_any_cached_permission`, `_has_cached_permission_prefix`, `get_user_permission_keys`, all removed) or an ORM-instance-attached cache (`user._permission_cache` on `auth.get_allowed_permission_keys`, also removed). A role-permission toggle with the same retained user instance could therefore be read from a stale snapshot. Independent review confirms this stale-instance risk, not the historical production HTTP root cause or cross-request instance reuse. `auth.get_allowed_permission_keys` is now the sole current-user effective-permission integration point without an attached result cache; the shell retains a canonical per-render capability snapshot. `auth.can_manage_system_settings`, `can_manage_users`, `can_modify_data`, `can_edit_data`, `can_delete_data`, `can_edit_user_accounts`, `can_delete_user_accounts`, and `can_manage_target_user_account` all now require an explicit `db: Session` and resolve through `has_permission`/`has_any_permission`/ `get_allowed_permission_keys` on every call. `can_manage_target_user_account` also now fails closed (previously defaulted to `True` whenever either actor's or target's `school_group_id` was falsy instead of only when they were confirmed equal).

Corrected stale/incorrect permission paths (classified A/B, current-user authorization or UI capability, per the audit): `routers/users.py` (`_get_user_roles_for_creator`, `_can_manage_user_permissions`, all `can_manage_users`/`can_edit_user_accounts`/`can_delete_user_accounts` call sites); `routers/observations.py` (`_can_create_observation`, `_can_override_locked_observation`, `_can_edit_observation`, `_can_delete_observation`); a genuine, previously-undetected sibling defect in the same module where any of `observations.create_formal` / `observations.create_non_formal` allowed creating *either* observation type (an "any-of" check used where an exact per-type check was required) is also fixed with a new `_can_create_observation_type` helper wired into both the create/edit routes and `templates/observation_form.html`'s Type options. `main.py`'s System Configuration "Qualifications" mutation routes (`create_qualification_option`, `update_qualification_option`, `delete_qualification_option`) were gated by the wrong keys entirely (`branches.delete`, `academic_years.activate`, `academic_years.create`) and now use `configuration.manage_degrees`/`configuration.manage_specializations` per the edited row's own kind, matching the template's existing `can_manage_module` guard. `update_branch` and `delete_branch` previously allowed a caller holding only `branches.edit` to also flip `status`, and a caller holding only `branches.activate_deactivate` to also rename/relocate a Branch (backend accepted more than the two dedicated keys implied); both routes now check `branches.edit` and `branches.activate_deactivate` independently per changed field, and `delete_branch`, `set_current_year`, and `open_new_academic_year` add missing SchoolGroup-scope boundary checks (a non-platform actor could previously target a foreign SchoolGroup's Branch/Academic Year record and only the query filter, not an authorization check, happened to prevent effect in some paths). `open_new_academic_year` now requires both `academic_years.create` AND `academic_years.activate` (previously gated by the coarse `can_manage_system_settings` role-prefix check). Notification mark-read/resolve/archive actions and Branch/SchoolGroup logo upload/reset now enforce their dedicated `notifications.*`/ `branding.manage_school_logos`/`branding.manage_branch_logos` keys server-side, matching (and in the notifications case, adding) the corresponding UI gates in `templates/notifications.html`.

Persisted-data-integrity: `role_permissions` had no database-level uniqueness on its logical `(school_group_id, role, permission_key)`/`(role, permission_key)` identity, so a duplicate row for the same scope/role/key was structurally possible and could make a re-grant non-deterministic depending on row-fetch order. Migration `20260915_001_role_permission_logical_key_uniqueness` adds two partial unique indexes (`uq_role_permissions_global_role_key` for `school_group_id IS NULL`, `uq_role_permissions_tenant_role_key` for `school_group_id IS NOT NULL`) mirrored in `models.RolePermission`. The migration performs a non-destructive duplicate preflight and raises `RuntimeError` without installing the indexes or recording the migration if any duplicate is found (no row is deleted or rewritten). A related `user_permission_service.apply_user_override` gap is also closed: a per-user override could previously be written for a permission key not assignable to the target's own normalized role, leaving an inert override; platform-only keys were already rejected before this commit. `apply_user_override` now re-validates assignability against the target's `get_effective_tenant_role` before writing an Allow/Deny (Inherit/removal is unaffected so a stale non-assignable row left by a role change can still be cleared). `user_permission_service.build_user_permission_payload`'s `effective` projection now also calls the same canonical `auth.get_allowed_permission_keys` resolver instead of re-implementing the merge locally, so the per-user permission UI cannot drift from the authoritative resolver.

New/updated regression coverage: `tests/test_branch_year_permission_boundaries.py`, `tests/test_configuration_scope_permissions.py`, `tests/test_notification_group_permissions.py`, and `tests/test_role_permission_logical_key_migration.py` (all new), plus extensions to `tests/test_permission_qualification.py`, `tests/test_user_permission_overrides.py`, `tests/test_user_permission_override_routes.py`, and `tests/test_platform_access.py`. `tests/test_permission_qualification.py` proves ON -> OFF -> ON for `dashboard.view`, `subjects.view`, `teachers.view`, `students.view`, `planning.view`, `timetable.view`, `calendar.view`, `observations.view`, `users.view`, and `configuration.view` across the resolver, direct route/API enforcement, and navigation on a fresh per-request read each time, plus per-user Deny/Allow-does-not-resurrect-role-Deny/re-grant-reactivates-a-stale-Allow, sibling-user isolation, and cross-SchoolGroup isolation. No Allow-over-Deny behavior was introduced; that remains an explicitly deferred, separately-governed decision per ADR 0040. No new permission key, schema-breaking change, or `tis.db` change.

2026-09-14 - Permission qualification drift closure

1 Prevented cross-capability privilege escalation in Edit User by validating

every changed field against its exact `users.*` permission server-side.

1 Separated single-user and bulk-user delete control visibility to match their

distinct route permissions.

1 Added the existing Talent Competency and Rubric-Level delete permission keys

to the browser projection, restoring authorized actions without weakening API enforcement.

2026-09-14 - Per-user permission override: platform-actor scope resolution, template render test, ADR 0040

1 Fixed `POST /users/permissions/{user_pk}` (`routers/users.py`

`update_user_permissions`) for the platform-level-actor edge case: a platform owner/developer has no resolvable tenant `school_group_id` of their own (`auth.get_user_school_group_id` always returns `None` for a platform user), and their own session-selected branch/SchoolGroup scope can legitimately differ from an arbitrary target user's

tenant. The route previously either hard-errored ("No school context was found") or, when the platform actor's own session happened to have some selected branch/SchoolGroup scope, spuriously rejected an otherwise-authorized cross-tenant write with "target user is not in the active SchoolGroup." It now derives the override's `school_group_id` unconditionally from the already server-loaded target user's own verified `school_group_id` whenever the acting admin `auth.is_platform_user`, mirroring `_build_user_permission_context`'s existing identical read-side resolution. No client-supplied scope value, new permission key, or new "select an acting SchoolGroup" control was added; the codebase's existing cross-tenant "platform manages any tenant user" authority (`_can_manage_target_user`/`can_manage_target_user_account`) already gated this route, and `user_permission_service.apply_user_override` independently re-validates the target's own `school_group_id` regardless.

1 Added `docs/adr/0040-per-user-permission-override-precedence-and-scope.md`

recording the full per-user override precedence chain, the Deny-over-Allow rule and its rationale, the `user_id + permission_key` uniqueness justification, and this platform-actor scope resolution (plus the rejected client-supplied-scope alternative).

1 Added three route-level regression tests to

`tests/test_user_permission_override_routes.py` proving: a platform actor with no tenant scope of their own can write an override that is correctly scoped to the target's own SchoolGroup; the same actor writing overrides for two different targets in two different SchoolGroups produces two correctly, independently scoped rows (never a shared/actor-controlled value); and the service layer itself rejects a school_group_id that does not match the target's own real SchoolGroup even when called directly.

1 Added `tests/test_edit_user_template_render.py`, a rendered-HTML

integration test for the Edit User page's per-user override panel (`templates/edit_user.html`) proving the actual route response contains all three Inherit/Allow/Deny controls for a normal permission, both the "Inherited from role (granted)" and "User override: Deny (overrides role grant)" reason-text branches, and the exact inert markup (`aria-disabled="true"`, "Allow (unavailable)"/"Deny (unavailable)") for a platform-only permission key.

1 No schema migration, new permission key, or `tis.db` change.

2026-09-13 - Role permission management reassessment and repair

1 Reproduced and diagnosed the Role Permissions editor's reported "unchecked permission checkmark returns" defect. Persistence was confirmed correct (grant/revoke/re-grant round-trips are idempotent and re-render from database truth); the root cause is a direct-vs-effective conflation: the editor bound one checkbox to the merged effective permission (built-in role defaults + global + tenant `RolePermission` overrides) with no indication of source, so a default- or global-granted permission could appear checked again after a local revoke.

1 Made `role_permission_service.py` the single canonical resolver and updated `auth.py` to delegate to it, removing duplicated default/global/tenant merge logic and closing a subtle `None`-scope semantic split between the UI and authorization paths.

1 Extended the permission payload with per-key `direct`, `inherited`, and `source` fields and updated the editor to label inherited grants ("On by default" / "On by global default") so direct assignment is never conflated with effective authorization.

1 Made the administrator Save path persist only intentional scope-level overrides via `role_permission_service.apply_role_permission_overrides`: a key whose submitted state equals its inherited baseline (built-in default, or the global package when editing a school) is left with no scope-level row, so a no-op Save no longer converts inherited/default/global grants into direct rows and default/global changes propagate to roles without an explicit override.

1 Locked owner-only controls (`system_owner.manage_ownership`, `system_owner.transfer_ownership`) out of the tenant editable model and out of Administrator role defaults via a unified `PLATFORM_ONLY` permission set; they remain owner/developer identity-bound.

1 Added `tests/test_permission_management.py` covering direct grant/revoke/re-grant, idempotency, effective calculation, inherited/role-derived behavior, tenant isolation, cross-tenant denial, unauthorized administrators, owner-only protection, invalid inputs, and the exact checkbox regression. No schema migration and no `tis.db` change.

2026-09-11 - Ghars Talent module sidebar branding

1 Replaced the generic Talent & Potential sidebar icon/text treatment with the Owner-supplied full Ghars bilingual logo while preserving the canonical `Talent & Potential` accessible/navigation label.

1 The Talent child tree, permissions, routes, expand/active behavior, and context-reset semantics are unchanged.

1 Added a dedicated static Ghars SVG asset and responsive sidebar sizing so the complete mark remains clear on the dark navigation surface.

2026-09-11 - Owner video acceptance immediate corrections

1 Fixed Evaluation-first Student Assessments so a configured Program without a materialized Cycle is selectable directly inside its Term/Period. Authorized selection creates and links the internal Cycle through existing governed APIs and stays on Student Assessments.

1 Strengthened Start Assessment failure visibility and preserved selected Evaluation/Program routing; a successful start opens the exact saved Program/Grade competencies and assessment criteria.

1 Restored a safe Delete Competency path for immutable/assessed setup by cloning a new editable Framework Version and deleting there; historical evidence is never mutated.

1 Added an explicit KPI expand/collapse affordance.

1 Changed Student list Add Student, Delete selected, Open, and Delete actions to icon-only controls with accessible labels.

1 Added Administrator-only `students.view_all_branches`; cross-Branch switching/All branches requires this permission plus organization/global scope, while other actors remain on their assigned Branch.

1 Kept Learning Style analytics fail closed but replaced the hidden-panel failure with an explicit unavailable message when the privacy provider is missing; no protected aggregate values are exposed.

2026-09-11 - Talent urgent navigation, readiness, review, and recovery adjustments

1 Removed the duplicated in-page Talent primary navigation; the permission-aware application sidebar tree is now the sole primary module navigation.

1 Moved readiness into the opened Program summary. The separate Ready step is removed, and **Ready** is shown positively only when Program/Grade, assessment-criteria, and Evaluation Plan prerequisites are complete.

1 Changed Grade-first authoring to one selected Grade at a time (default first configured Grade), removing repeated Grade labels from every Competency row while preserving dedicated Competency/Level delete permissions.

1 Clarified user-facing terminology: competency-owned rubric structure is presented as **Assessment Criteria** rather than being falsely renamed KPI. The separate numeric `TalentKpiConfiguration` is explicitly labeled **Key Performance Indicator (KPI)** and now exposes Add/Edit/Delete KPI actions.

1 Added Administrator-default `talent_assessments.reset_for_reassessment`. A reset preserves completed evidence/history, makes the prior current attempt historical, audits the action, and returns the Student to Not started so a fresh Assessment may begin in the same Evaluation. ADR 0034 zero-evidence hard-delete remains unchanged.

1 Student Assessments now visibly marks the selected Evaluation Period/Program, carries Cycle context when Start Assessment opens the Student workspace, and continues to load the exact persisted Program/Grade competencies and criteria.

1 Talent Review now supports authorized Branch/Grade/Section filters, offers All Branches only to actors with multiple authorized Branches, and removes the primary user-facing "Meets criteria" classification while preserving deterministic Candidate logic internally.

1 Analytics unavailable/validation states now surface the specific backend reason and a more actionable next step when configuration/data prerequisites are missing.

2026-09-11 - Talent setup finalization, Period grouping, and reassessment reset correction

1 `Finish Setup` now performs the real governed lifecycle completion for a fully configured Draft Program: activate the Program first, activate the reviewed Draft Framework with revision/fingerprint protection, then open that Program's operational summary. Already-active state is not activated again.

1 Fixed Program operational-summary navigation so Edit Program, Build/Edit Rubric, and Manage Evaluation Plan actually reopen their intended hash-backed panels instead of changing the URL without rerendering.

1 Student Assessments now combines configured Evaluation Plan Periods with Cycle contexts, groups repeated user-facing Evaluation labels across Programs (using configured sequence for ordering when available), and renders each Program once. Duplicate physical Cycles no longer duplicate a Program card, and a configured Program/Period stays visible before its internal Cycle exists.

1 Closed the legacy reassessment gap for completed Students whose rubric was changed in place before assessed-Framework immutability was consistently enforced. When persisted completed rubric/level bindings no longer match a complete current competency-owned rubric on that same Framework, the Student surfaces **Re-evaluation required**. The prior completed attempt becomes non-current and the replacement starts empty/current/In Progress in the original visible Evaluation; evidence is preserved internally.

1 Untouched legacy shared rubrics and partially configured competency-owned rubrics do not produce false reassessment requirements.

1 Closed the remaining legacy gap where rubric semantics changed after completion but retained the same rubric/level IDs. Audited semantic Framework/rubric mutations after `completed_at` now require re-evaluation when the current competency-owned rubric is complete; unrelated lifecycle/branding/plan changes remain excluded.

1 Fixed Arabic/non-ASCII Grade-level Competency creation by removing the browser's ASCII-only generated-code path. The backend now assigns its existing unique Program-scoped internal code when the normal Rubric tool submits only name/description.

1 Added a permission-aware Talent & Potential child tree inside the main application sidebar for Overview, Programs, Student Assessments, Talent Review, and Results & Analytics.

1 Made every top-level Talent destination a context-reset boundary: only Academic Year is carried; stale Program/Cycle/Assessment/Branch/Grade/Section/metric/dimension context is not propagated across module sections. Student Assessments now owns its visible Program filter instead of inheriting the previously opened Program.

1 Simplified the Programs list to one normal **Open Program** entry point per row (plus separately authorized delete where applicable); Program-specific Edit Program, Build/Edit Rubric, Manage Evaluation Plan, Open Assessments, and View Results stay inside the opened Program workspace.

1 No schema migration, pricing, AI, analytics privacy, or tenant-isolation contract changed.

2026-09-11 - Talent current-rubric, reassessment, and granular delete correction

1 Removed the separate Assessment Records/History table from the normal Student Assessment workspace; the current eligible Student row, status, and action are now the sole operational surface while superseded attempts remain preserved evidence internally.

1 Separated historical compatibility from current rubric authority: legacy Framework-wide shared rubrics may still be read for historical comparison, but normal Evaluation starts select only a complete competency-owned rubric structure for every applicable Competency.

1 Preserved no-op Framework-clone behavior while keeping material Student-facing rubric changes reassessment-triggering. Completed current Students surface **Re-evaluation required** and the replacement attempt remains attached to the original visible Evaluation/Term.

1 Enforced Assessment-history immutability consistently across competency/rubric/level/description authoring mutations.

1 Added `talent_programs.delete_competency` and `talent_programs.delete_rubric_level` as independently assignable permissions. `talent_programs.delete` remains whole-Draft-Program deletion only; `talent_programs.manage` does not grant destructive authority.

1 Fixed Competency removal in the new rubric structure by deleting the eligible competency-owned Rubric and its Levels with the Framework membership after existing descriptor/KPI/candidate-rule dependency checks. No schema migration was added.

2026-09-10 - Talent corrective Batches 3 and 4

1 Extracted one reusable, order-derived rubric visual system and applied it across Program setup, assessment entry, Talent Review, Student/Learner Profile, and analytics. It supports arbitrary labels/counts, always shows position and label, and adds selected/check/border state without treating the highest level as Official Identification.

1 Added tenant-bounded read projections for the actual persisted rubric level in Talent Review and historical profile evidence. Candidate selection, scoring, permissions, and Official Identification behavior are unchanged.

1 Reduced Talent Overview to operational action cards and removed Program-card/Talent-Map-matrix duplication from Organization Overview. Detailed Program, Branch, matrix, and longitudinal views retain their established ownership and governed metrics.

1 Kept the existing Learning Style endpoint and suppression contract; its Students UI now uses an explicit Planning-derived Organization/Branch/Grade/Section cascade. Added no endpoint, permission, metric, schema, or migration.

2026-09-10 - Student Learning Style V1 implementation

1 Implemented the ADR 0031-governed Student Learning Style V1 contract: a

nullable, single-select `Student.learning_style` (migration `20260910_002_student_learning_style_v1`, purely additive) restricted to exactly `Visual`/`Auditory`/`Read`/`Write`/`Kinesthetic`, validated server-side in `student_academic_service.py` and rejected at the API layer regardless of any UI restriction.

1 Added the field to Student create/edit (`templates/_learning_style.html`,

a compact icon+label selector reusing existing design tokens) and to the Student Profile display (Overview and the Talent tab, shown explicitly as learner context, not a Talent signal). Editing reuses the existing `students.edit` permission; no new permission was added.

1 Added a Branch/Organization Learning Style distribution on the Students

list page and a matching `GET /api/students/analytics/learning-style-distribution` route (`student_learning_style_analytics.py`), reusing the existing Branch/Grade/Section filters and the governed Talent privacy/suppression contract (`talent_analytics_privacy.py`'s Cell/Group complementary suppression, the same shape `/rubric-distribution` already uses) rather than a new or weaker rule; fails closed when no privacy policy is configured.

1 Confirmed by regression test that no Talent scoring, Program Criteria,

Review, or Official Identification code path reads `learning_style`. The Talent Review/Assessment operational drill-down (`static/js/talent-operations.js`) was left unextended because its backend `context` payload lives in `routers/talent_review_candidates.py`/`routers/talent_assessments.py`, which already carried substantial unrelated uncommitted changes from separate concurrent work. No Talent scoring, eligibility, permission, tenant boundary, or `tis.db` data changed. See "Student Learning Style V1" in `docs/PROJECT_STATE.md` and `docs/adr/0031-student-learning-style-v1.md`.

2026-09-10 - Evaluation Plan scope and local-test identity correction

1 Classified the reported Add Evaluation Period denial as a stale sanctioned local-test identity: its role already held all canonical Evaluation Plan permissions, but the disposable database row had durable `BRANCH` scope while the current seed contract specifies `ORGANIZATION` with North Campus retained as the working Branch.

1 Preserved the KMS organization/global authority requirement and tenant boundary. Added explicit backend and browser-payload coverage proving that an organization-scoped manager with a selected Branch can mutate, while a truly Branch-scoped manager remains denied and sees a read-only explanation before submission.

1 Added a seed-contract assertion and used the existing sentinel-checked local reseed workflow as the supported repair path. No production database, raw SQL, role bypass, schema, permission key, or lifecycle rule changed.

2026-09-10 - Organization Overview primary indicator and rubric distribution wiring

1 Added a single, largest-on-page radial gauge to the Organization Overview

answering "how many Students are talented": sourced only from the existing `identified_of_eligible` M10 metric's `organization_total` cell, gated by the existing `talent_official_identifications.view` permission. Confirmed Official Identification (a separate, permanent human decision) as the authoritative "talented" signal, distinct from the rubric level and from Meets Program Criteria/Review-Candidate membership, both still shown separately.

1 Wired the previously implemented but UI-unused M9 `/rubric-distribution`

route into the Organization Overview (shown only when one Program is selected) using a new shared, order-derived rubric-level visual (`rubricDistribution`/`rubricLevelIntensity` in `static/js/talent.js`): intensity is derived only from a level's position among its siblings, never from a count/percentage, so it stays safe for any label set/level count and for a privacy-protected level. Bar length remains sourced only from an already-visible backend percentage.

1 No analytics service, provider, privacy threshold, suppression rule,

permission, tenant, schema, migration, or data behavior changed; both additions are pure consumers of already-existing, already-tested backend projections. See "Talent Phase C Results & Analytics Experience" in `docs/PROJECT_STATE.md`.

2026-09-10 - Completed Program operational mode

1 Added default operational summaries for completed active Talent Programs and explicit hash-based entry back into guided editing.

1 Adopted Evaluation Plan/Evaluation Period terminology and retained the embedded Step 3 management experience.

1 Corrected Academic Year label rendering and translated organization-authority errors into user-facing Evaluation Period access messages.

2026-09-10 - Talent guided configuration owner correction

1 Unified Program Basics into one setup panel and introduced a one-substep-at-a-time assessment setup flow.

1 Embedded Evaluation Schedule management inside the Program wizard while retaining the standalone operational route and existing backend lifecycle contracts.

1 Standardized user-facing Evaluation Schedule terminology and simplified assessment setup/history wording.

1 Enforced mutually exclusive desktop Student table and mobile Student cards, with visually neutral normal Active status.

2026-09-10 - Talent Owner Visual Acceptance Corrections

1 Removed the "Apply context" confirm-click requirement from the shared

Academic Year/Program/Branch/Grade/Metric/Dimension filter used by every Talent page: a `change` listener now triggers the existing debounced (250ms) reload path; the visible button is relabeled "Refresh" and is now an optional, immediate, non-debounced fallback. The existing URL-query- parameter/`history.replaceState` context mechanism is unchanged.

1 Removed the duplicated analytics-family navigation on Organization Overview

and every other analytics page: the primary Talent nav now shows exactly one "Results & Analytics" entry instead of repeating each analytics page link that the existing sticky sub-nav already lists.

1 Corrected this document's own prior entry below and the other KMS sources:

the Evaluation Schedule "Baseline/Term 1/Term 2/Final" fixed-picklist description was already superseded, later the same day, by a free-text user-defined evaluation name (confirmed still intact, not re-implemented); internal-lifecycle copy ("Prepared evaluations", "Ready to link", "Link an evaluation", "No Evaluation Plan", "Frozen population") does not render in that normal path (confirmed via a fresh rendered-output regression, not merely template source). The compact Programs table with a collapsed (not permanently expanded) Create-Program form was likewise confirmed already intact.

1 Diagnosed (no code change) why Organization Overview shows "Analytics

unavailable" in an ordinary local run: `DATABASE_URL` must resolve to exactly `.local_test_data/talent_local_test.db` to activate the sanctioned local providers; an unset `DATABASE_URL` falls back to `tis.db`, which correctly stays on the fail-closed production path. This is the existing B11-E F1 contract working as designed, not a defect.

1 Presentation-only: no schema, migration, permission, analytics computation,

provider, privacy, or `tis.db` change.

2026-09-09 - Talent Program And Evaluation UX Simplification

1 Replaced the desktop Program card grid with a searchable table showing

Program, Grades, Type, Current Year, Status, Open, and Edit.

1 Reframed Program setup as Basics, What we assess, Evaluation schedule, and

Ready; joined each competency with its rubric-level achievement descriptions and kept version mechanics inside the edit disclosure.

1 Replaced the normal manual Plan/Period/Cycle workflow with a standard

Baseline/Term 1/Term 2/Final schedule and Setup/Ready to start/In progress/ Complete states.

1 Start Evaluation now calls existing create/link/preview/open contracts in

revision order and shows the eligible Student count and Academic Placement effective date before the governed population freeze.

1 Added missing `talent_evaluation_plans.manage` and `.govern` browser permission

projection plus organization-scope regression coverage. No schema, migration, Student UI, analytics, pricing, AI, timetable, or backend authority changed.

2026-09-09 - PostgreSQL Student/Talent Migration Prerequisite Repair

1 Fixed the Render pre-deploy `InvalidForeignKey` failure without removing or

weakening any tenant-scoped composite foreign key.

1 Added ordered migration `20260904_000_student_talent_parent_scope_prerequisites`

to establish equivalent unique `(id, school_group_id)` parent keys on legacy Branch, Academic Year, and existing Student tables before migration `001`.

1 Deferred all Student/Talent migration-owned tables from baseline

`metadata.create_all()`, ensuring fresh runs execute `000` before `001` while already-migrated environments can apply `000` safely later.

1 Removed the fresh-chain M4/M8 ordering hazard by creating the M4 Cycle table

without its future Period FK; the existing M8 migration adds that exact composite FK after creating Plans and Periods.

1 Dedicated PostgreSQL 16 tests verify the full 63-entry ledger, all composite

Talent parent targets, repeat idempotency, late prerequisite application, duplicate-data failure without rewrite/marker, and transactional rollback of the old pre-ledger failure. No production database, deployment, commit, push, or `tis.db` change was made.

2026-09-09 - Phase H Cumulative Release Qualification and Baseline A/B

1 Ran the complete current `tests/` suite. Pytest collected 1,748 parent test

items and reported 1,618 passed, 78 failed, and 59 skipped, plus 87 passed subtests. The apparent 1,755 total is the arithmetic sum of the headline outcome counters; it is not a second collection count because unittest subtest outcomes are reported separately from their collected parent items.

1 Reproduced all 78 current failures against the clean pre-Talent baseline

`8ed2cc998b8f1c29b5f1d7b34c3b9532deff603e`: 78 failed, 192 passed, 1 skipped, plus 60 passed subtests in the bounded seven-file failure set. Therefore 78/78 failures are baseline/environment failures and the Talent-caused regression count is zero.

1 Phase H cumulative release qualification is PASS WITH BASELINE EXCLUSIONS.

The separate production-deployment check that peak worker/process RSS is no more than 70% of the actual Render worker/container memory allocation remains unevaluated until the real allocation and worker count are known.

1 This qualification record changes no product behavior, schema, migration,

permission, entitlement, production configuration, or deployment state.

2026-09-09 - M10 B11-E Live Multi-Worker Qualification, ADR 0028 Completion, ADR 0030, B12 Definition

1 Personally built and ran a live, >=2-independent-process (`uvicorn`)

multi-worker qualification against the dedicated non-production PostgreSQL database, with real F1 production providers engaged (not test doubles) - see `docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md` for full methodology and results.

1 Results: byte-identical cross-worker responses on 5 of 7 M10 routes; no

worker-local correctness dependency (code inspection, corroborated by a worker-kill test); concurrent-writer safety and fresh-transaction-per- request semantics confirmed across workers; performance PASS against p95<=2.0s/p99<=4.0s with substantial margin; memory PASS against <=128MB incremental/request with substantial margin, no monotonic growth.

1 Formally adopted the Owner-approved performance (p95<=2.0s/p99<=4.0s) and

memory (<=128MB incremental/request; <=70% allocated worker/container memory) targets into `docs/adr/0028-b11-production-qualification-policy.md`, framed as directional multi-run evidence with no single figure treated as canonical; the <=70% sub-item is explicitly classified NOT EVALUATED LOCALLY / ENVIRONMENT-SPECIFIC (no authoritative production allocation value exists in this repository), not a code defect.

1 Created `docs/adr/0030-m10-organization-analytics-multi-worker-qualification-minimum.md`

(accepted): >=2 independent workers/processes as the release-qualification minimum; correctness must not depend on worker-local mutable state; not a declaration of actual production topology.

1 Added a B12 (M10 closeout only) definition as a new subsection of ADR 0028,

strictly downstream of B11-E, explicitly separate from and not satisfied by the concurrent Talent & Potential Phase A-E product/UI work.

1 Classified B11-E as CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT

VERIFICATION ITEM REMAINING, and B12 as DEFINED, NOT YET CLOSED (closure itself remains a separate future governance/independent-review action).

1 No schema, migration, permission, pricing, AI, or timetable change.

`tis.db` untouched (SHA-256 confirmed unchanged before and after). All ephemeral PostgreSQL qualification schemas were dropped; zero leftover schemas confirmed.

2026-09-09 - M10 B11-E F2 Qualification Re-Verification (Bounded)

1 Independently re-ran `tests/test_talent_organization_repeatable_read.py`

against the live non-production PostgreSQL test database: 15 passed, including `test_entitlement_availability_check_shares_the_m10_repeatable_read_snapshot`, confirming the F1 entitlement-availability query shares the M10 REPEATABLE READ snapshot.

1 Ran a new, bounded, throwaway (not committed) independent latency sanity

check reusing the existing PostgreSQL fixture builder against all seven M10 routes; results were comfortably within a generous latency ceiling on a small fixture dataset with no concurrent load, but did not reproduce the specific performance figures a prior session reportedly measured, and no memory or multi-worker evidence was produced or reproduced.

1 Declined to formally adopt specific unverified performance/memory figures

or a multi-worker qualification conclusion into ADR 0028 or a new ADR 0030; no ADR was created or substantively amended by this task.

1 Corrected two stale "Independent review... PENDING" lines in

`docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md` and `docs/history/engineering-handbook/README.md` to match the same document's own already-stated "PASS WITH NON-BLOCKING OBSERVATIONS" conclusion (a same-document self-consistency fix, independently confirmed by direct reading).

1 `tis.db` untouched. No schema, migration, permission, pricing, AI, or

timetable change. See `docs/history/engineering-handbook/2026-09-09-b11e-f2-postgresql-qualification.md` for the full, honestly-scoped re-verification record.

2026-09-08 - M10 B11-E F1 Production Providers

1 Added fail-closed production provider resolution for all seven Organization

Intelligence routes: externally configured P1-P7 minimum cohort 5 with the existing complementary suppression engine, canonical commercial availability through `feature.organization_intelligence`, and explicit 1000-cell, 1000-relationship, and 1000-Program-pair breadth ceilings.

1 Registered the semantic feature key in the existing normal customer-feature

catalog and added a permission-free entitlement decision for provider use; route permission checks remain separate and no plan or pricing rule was added.

1 Added a deterministic local provider path restricted to non-production and

the exact `.local_test_data/talent_local_test.db` URL. It does not activate for `tis.db`, memory/other SQLite databases, or production environment names.

1 Added focused boundary, configuration, exception, entitlement, local-path,

route, and no-truncation tests. Real normal-dependency HTTP checks returned 200 for all seven APIs and pages; a production-like missing-configuration request failed closed. No schema, migration, new permission, production data, API response shape, commit, push, merge, or deployment change was made.

2026-09-08 - Talent Phase C Results & Analytics Experience

1 Reworked the seven existing M10 projections into a visual, stakeholder-ready Results experience with Organization Overview, Program Results, Branch Results, Talent Map, Students Across Programs, Students, and Progress Over Time.

1 Added governed KPI cards, percentage progress visuals, non-ranked Branch navigation, a Talent Map preview, Program and Student cards, a symmetric overlap grid, period visuals, and friendly comparability explanations.

1 Added Academic Year, Program, metric, and Branch/Grade dimension navigation only where backend routes support those parameters; added contextual drill-down links and a sticky Results subnavigation.

1 Preserved zero/no-data/privacy distinctions and ensured only visible backend percentages control chart magnitude. Protected states expose no hidden magnitude in DOM, ARIA, hover/focus help, color, or size.

1 Added focused Node tests; all seven M10 backend suites and isolated local-data HTTP checks pass. Browser visual acceptance remains pending because no browser surface was available. No backend, schema, migration, provider, permission, production-data, or `tis.db` change was made.

2026-09-08 - Talent Phase B Operational Workflow

1 Converted the uncommitted M11 Talent workspace into a real operational UI for

Program setup, eligible Grades, versioned competencies/rubrics/descriptors, optional Program-specific KPI configuration, and Review Candidate rules.

1 Reused the existing M8 Plan/Period and M4 Cycle APIs for the path from Program

planning to a frozen Student population; added real M5 assessment entry with sequential returned-revision consumption and stop-on-stale behavior.

1 Added M6 Candidate review, separate human Official Identification, and

separately permissioned Educator Input add/amend/history interactions. No Candidate automatically becomes identified.

1 Added authorized human-readable Assessment/Candidate/frozen-population context

and exposed stable write IDs through read projections. Kept those row IDs out of the M3 semantic fingerprint and added descriptor IDs for precise removal.

1 Added focused Node/UI/API tests plus a real-router temporary-SQLite end-to-end

journey. No schema, migration, new permission, AI, billing, `tis.db`, commit, push, or deployment change was made. Browser acceptance remains pending because no browser connection was available.

2026-09-08 - M10 B11-E Integrated Production Qualification

1 Governed and implemented ADR 0029: a dedicated M10-only REPEATABLE READ

session/dependency (`database.M10OrganizationAnalyticsSessionLocal`, `dependencies.get_m10_organization_analytics_db`), applied to all seven Organization Intelligence routes, backend-conditional (PostgreSQL only), never applied server-wide or to any other route/module.

1 Live-re-tested all seven routes under the permanent implementation:

Overview and Student Drill (B11-D's exact proven mismatches) now resolved; talent-map, program-portfolio, branch intelligence, participation-overlap, and longitudinal independently reproduced live for the first time (B11-D had only inferred them structurally) and confirmed resolved.

1 Tested suppression/reconstruction concurrency with the existing

non-production `DeterministicSuppressionTestPolicy` (primary and complementary suppression); confirmed no suppressed value becomes reconstructable and no raw value/threshold leaks under a concurrent write.

1 Finalized Index Candidate B as NO CHANGE and statistics freshness as an

operational/runbook disposition (no code/migration change). Provider readiness (privacy/availability/breadth) remained an explicit fail-closed B11-E blocker at that checkpoint and was subsequently addressed by F1.

1 Added 14 new committed regression tests

(`tests/test_talent_organization_repeatable_read.py`); the full 226-test M10/PostgreSQL/privacy regression passed; `tis.db` was confirmed unchanged. Independent review returned PASS WITH NON-BLOCKING OBSERVATIONS. B11-E implementation is PASSED and checkpointable, but B11-E production closure remains PENDING; B11 overall is NOT CLOSED and B12 is NOT IMPLEMENTED.

2026-09-07 - M10 B11-D PostgreSQL Concurrency Qualification Closure

1 Closed B11-D after independent review returned PASS WITH NON-BLOCKING

OBSERVATIONS.

1 Recorded proven READ COMMITTED mixed snapshots within one request: Overview

Candidate count 314 to 315 and Student Drill gate/page population 705 to 706.

1 Recorded that a non-permanent REPEATABLE READ experiment resolved both cases,

while permanent adoption is PROPOSED - GOVERNANCE REQUIRED and NOT IMPLEMENTED, with full-request transaction scope beginning before the first database statement.

1 Preserved unmeasured suppressing-policy concurrency, other route consistency,

scale, statistics freshness, providers, memory, and SLO work as B11-E gates. Candidate B trends NO CHANGE; no index or semantic/runtime change was made. The 209-test regression passed and tenant isolation remained clean.

2026-09-07 - M10 B11-C PostgreSQL Qualification Closure

1 Closed B11-C after independent evidence re-review returned PASS WITH

NON-BLOCKING OBSERVATIONS.

1 Preserved durable PostgreSQL 16.15 evidence for all seven M10 routes: bounded

query counts across tested scales, no N+1, Candidate A = NO CHANGE, Candidate B = MORE EVIDENCE REQUIRED, directional local Windows memory measurements, and a passing 209-test regression.

1 Added no index, schema, migration, permission, privacy, or entitlement change

and selected no production threshold, breadth limit, commercial mapping, SLO, or memory value. B11-D subsequently completed and is CLOSED; B11-E and B11 overall remain open; B12 remains NOT IMPLEMENTED; production is not ready.

2026-09-07 - M10 B11 Production Qualification Governance

1 Accepted ADR 0028 without application or production configuration changes.

1 Governed external, fail-closed privacy-threshold and breadth configuration;

plan-agnostic commercial availability through the existing entitlement/ feature-registry architecture; evidence-first performance, isolation, index, and memory decisions; B11-C/D entry criteria; and the B11-E release gate.

1 Preserved every numeric threshold, plan/feature mapping, breadth limit, SLO,

memory ceiling, isolation change, and index decision as explicitly deferred.

1 B11-C and B11-D subsequently completed and are CLOSED. B11-E implementation

subsequently PASSED, but production closure remains PENDING; B11 overall is NOT CLOSED, B12 is NOT IMPLEMENTED, and production is not ready.

2026-09-07 - M10 B11-A/B11-B Organization Analytics Observability

1 B11-A (read-only qualification, completed) confirmed the privacy,

commercial-availability, and breadth-policy provider seams used by all 7 M10 Organization Intelligence routes exist with no production implementation and correctly fail closed today.

1 B11-B adds safe, non-sensitive operational observability around all 7

existing routes (`overview`, `talent-map`, `program-portfolio`, `branches/{branch_id}`, `participation-overlap`, `students`, `programs/{program_id}/longitudinal`) through a new module, `talent_organization_analytics_observability.py`, and telemetry wiring in `routers/talent_organization_analytics.py`. Uses a dedicated `tis.talent.organization_analytics.observability` stdlib logger, kept separate from the immutable business/security audit trail (`audit.py`).

1 Recorded signals are a bounded allowlist only: `projection_family`,

`outcome` (`success`/`rejected`/`unavailable`/`failed`), route latency, safe structural counts reused from each route's own existing `OrganizationAnalyticsBreadthPolicy.allows(...)` inputs (`program_count`/`row_count`/`column_count`/`prospective_cell_count`/ `prospective_pair_count`/`relationship_estimate`, plus route-specific `emitted_pair_count`, `period_count`/`component_count`/ `authoritative_cycle_count`/`comparison_count`, and Student Drill's `page_limit`/`page_returned_count`/`has_more`), and each provider's coarse outcome (missing/available/unavailable/exception for availability; missing/evaluated/exception for privacy; missing/allowed/ rejected/exception for breadth). Never a Student identifier, raw analytical value, privacy threshold, suppressed value, Candidate/ Identification decision, or `delta`/`change`/`percent_change`/ `total_count`.

1 Every existing fail-closed HTTP outcome (provider missing/false/

exception, breadth reject/exception, privacy provider missing/ exception) is unchanged byte-for-byte; observability wraps existing decision points purely to observe them and never recomputes/duplicates a provider call or alters a response body. A telemetry emission failure is swallowed internally and can never affect the HTTP response or weaken a fail-closed decision.

1 Query-count instrumentation is explicitly deferred to a future "B11-C

profiling" phase; no new SQLAlchemy event-listener instrumentation was added outside existing test-only usage.

1 New coverage: `tests/test_talent_organization_observability.py` (22

tests). Full existing B5-B10 regression suites and the broader M9/M10/ permission/tenant-isolation suites remain green. No schema, migration, permission, entitlement, privacy threshold, or breadth limit was added. Independent security/privacy review passed with non-blocking observations; B11-B is CLOSED. PostgreSQL performance/concurrency/memory qualification and every ADR 0027 production gate remain open; B11 overall is not CLOSED and B12 remains unimplemented.

2026-09-07 - M10 B10-B Longitudinal Organization Intelligence Implementation

1 Implemented the ADR 0027 / B10-A approved contract: `GET /api/talent/

organization-analytics/programs/{program_id}/longitudinal`, the seventh Organization Intelligence route, in new module talent_org_longitudinal.py plus one added route function in routers/talent_organization_analytics.py`. No schema, migration, permission, or entitlement was added; B8/B9 remain CLOSED and unmodified.

1 Reuses `resolve_access_context`/`authorized_program_universe`/

`resolve_filters`/`frozen_membership_query` unchanged. Fetches the single governed M8 Plan and its Period+linked-Cycle series in one bounded query, then aggregates population/completed/started/Candidate/identified counts grouped by Cycle id in one bounded query each - never one query per Period, never a Student ORM fetch, never a Student-ID Python set.

1 Every Cell/component is built with `relationships=()`: there is no

same-point or cross-time additive Relationship anywhere in the module, so no `delta`/`change`/`percent_change` field can ever be computed or exposed. Rates are derived only through the unchanged `derive_exact_rate`/ `project_safe_derived_payload` helpers.

1 Point status uses the exact verified lifecycle values: cancelled Period ->

`no_data`/`cancelled_period`; missing linked Cycle -> `no_data`/ `missing_cycle`; linked Draft Cycle -> `no_data`/`cycle_not_authoritative`; authoritative Cycle with zero frozen population -> `no_data`/ `no_frozen_population`; a Program with no Plan yields an empty, non-fabricated series. An unlinked/ad-hoc Cycle is structurally excluded (only ever looked up by Period-linked `cycle_id`).

1 Comparability follows the ADR's governed precedence exactly: a missing/

cancelled/no-population reason first, then `privacy_protected`, then `framework_changed`, then `comparable`. A dedicated regression test proves a missing/non-authoritative reason overrides `framework_changed` even when the adjacent Framework versions also differ. The five Framework- independent metrics stay comparable across a Framework change; the four Candidate/Identification metrics become `not_comparable`/ `framework_changed`.

1 Candidate/Identification SQL is skipped entirely unless the selected

metric requires it AND the actor holds the matching secondary permission - requesting an unrelated metric never triggers that SQL merely because the actor holds the permission.

1 Added `LongitudinalClosedProjection`, a strict closed-wrapper serialization

boundary (`TypeError` on a raw dict/ORM row/unclosed point), matching the B8/B9 discipline.

1 Added `tests/test_talent_organization_longitudinal.py` (48 tests) covering

route/contract, the exact nine-metric allowlist and five exclusions, authorization and non-enumeration, Period/Cycle/Plan lifecycle states, Framework sensitivity and the precedence test above, privacy (no delta/change/percent_change, provider-exception fail-closed, no_data-never-zero, factual-zero-passes-privacy, `privacy_protected` comparisons, no second privacy evaluation from comparison metadata), rate arithmetic, secondary permission query-skip discipline, breadth-before-aggregation, bounded (non-per-Period) query count, and serializer isolation. Full proportional regression (M10 B0-B9 + M9 Talent Analytics + permissions + tenant isolation + M8 Plan/Cycle/Candidate/Identification/Framework suites) passed with no regressions.

1 Independent B10-B security/privacy review passed with non-blocking

observations; B10 is CLOSED. B11, B12 (closeout), and every ADR 0027 "Deferred capabilities"/ "Production gates" item remain open. No `tis.db` change, commit, push, or deployment occurred as part of this task.

2026-09-06 - B10-A Longitudinal Organization Intelligence Governance Closure

1 Added ADR 0027 recording the approved, amended B10 "Longitudinal

Organization Intelligence" architecture contract following an independent governance review ("approve with required amendments"). This is a governance-closure task only: no route, query, schema, migration, permission, or entitlement was added. B8 and B9 remain CLOSED following passed targeted independent re-reviews and committed/pushed checkpoints.

1 Approved product boundary: one Talent Program, one Academic Year,

aggregate/non-identifiable output, and the ordered M8 `TalentPlannedEvaluationPeriod` slots within that Academic Year, exposed through the future `GET /api/talent/organization-analytics/programs/{program_id}/longitudinal` route.

1 Approved time-point model corrects the original proposal's lean toward the

`TalentAssessmentCycle` as ordering authority: the Period is the presentation/order authority (ordered only by its governed `sequence`); its optional linked Open/Closed Cycle supplies factual evidence, per the existing `uq_talent_assessment_cycles_period` one-Cycle-per-Period constraint.

1 Approved metric allowlist is exactly nine of the fourteen existing

`MetricCode` values (`frozen_eligible`, `completed`, `completion_coverage`, `assessment_started`, `started_coverage`, `candidate_count`, `candidate_of_eligible`, `identified_count`, `identified_of_eligible`); one metric per response.

1 Approved comparability vocabulary is exactly `comparable`/`not_comparable`

with governed reason codes; B10 MVP returns no server-computed delta, change, or percent-change between points (an intentional privacy decision), and never uses growth/improvement/decline language. `AcademicYear.year_name` remains descriptive only, never chronology authority; multi-Academic-Year longitudinal ordering is explicitly deferred.

1 B10 itself remains NOT IMPLEMENTED. B10-B implementation is READY TO BEGIN

after the governance checkpoint is committed and pushed. No schema, migration, permission, entitlement, `tis. db` change, commit, or deployment occurred as part of this task.

2026-09-06 - M10 B9 Student Drill

1 Added `GET /api/talent/organization-analytics/students`, the first M10

route exposing Student-identifiable output, requiring `talent_analytics. view` AND `talent_analytics.view_students` (true AND composition matching the M9 `analytics_students` precedent).

1 Student inclusion uses only frozen `TalentAssessmentCyclePopulationMember`

context (Branch/Grade/Section) inside the authorized tenant/AY/historical- Branch/Program scope; current `StudentAcademicPlacement` never decides inclusion or supplies displayed context.

1 Added `talent_org_student_drill.py`: one gate-level P7

`student_drill_population/count/distinct_student` Cell runs through the same B2 closure pipeline every other M10 route uses (no per-Student additive relationship graph), and a strict `StudentDrillClosedProjection` closed wrapper is the only accepted serialization input, rejecting raw SQL rows, raw ORM objects, and a non-visible gate with `TypeError` (mirroring B8's closed-wrapper discipline).

1 Candidate/Identification fields are independently permissioned and

query-skipped (not response-filtered); the key is absent, never `null/false`. Learner Profile access exposes only an advisory `can_view_learner_profile` capability hint - the real Learner Profile route still independently authorizes.

1 Pagination (limit 25/max 100, `has_more`, no `total_count`) is over distinct

Students in deterministic name/id order; no Student Talent Score, ranking, AI field, or Educator Input was added.

1 No direct Student-ID filter is exposed (avoids an existence oracle since

the existing M10 filter resolver has no governed Student-identity filter contract). Added no schema, migration, permission, entitlement, longitudinal/B10, UI, or AI capability.

1 Remediated independent-review findings by gating on distinct authorized

Students rather than memberships, paginating one top-level row per Student, and returning deterministic frozen Program/Cycle contexts. Candidate and Identification evidence remains independently permissioned and exact-context scoped. Targeted independent re-review passed; B9 is CLOSED. Its committed/pushed implementation remains part of current dev.

2026-09-06 - M10 B8 Participation Overlap

1 Added the bounded Participation Overlap route using distinct Open/Closed

frozen Cycle Program/Student participation and historical Branch scope before one set-based Program-pair intersection.

1 Extended the closed contract only with P2 `participation_overlap`, mapped to

`count` at `program_participation` grain; canonical symmetric pairs share one privacy decision and diagonals represent distinct Program participants.

1 Added no invalid overlap totals/equations, Candidate/Identification overlap,

ranking, Talent Breadth/score, Student payload, schema, migration, permission, entitlement, UI, AI, commit, deployment, or B9+ route.

1 Remediated the initial independent-review failure where Program display order

differing from numeric ID order caused inconsistent pair dictionary keys and HTTP 500. One canonical numeric pair-key helper now governs construction and lookup; divergent two/three-Program order, mirror, diagonal, reverse-filter, and uniqueness regressions pass. Targeted independent re-review passed; B8 is CLOSED. Its committed/pushed implementation remains part of current dev.

2026-09-06 - M10 B7 Program Portfolio And Branch Intelligence

1 Added bounded Program Portfolio and authorized single-Branch Intelligence

routes through existing M10 access, breadth, set-based aggregation, privacy closure, and safe serialization.

1 Kept required-Period execution at Program/AY Portfolio grain and excluded it

from Branch Intelligence; shared Program/Branch facts retain B6 identities.

1 Added no ranking, score, Grade breakdown, schema, migration, permission,

entitlement, UI, AI, or B8+ route.

2026-09-06 - M10 B6 Talent Map

1 Added the bounded Program-by-Branch and Program-by-Grade Talent Map route,

with a presentation-only `branch_program` alias.

1 Added canonical shared matrix Cells, backend row/column/authorized-scope

totals, additive graph relationships, mandatory breadth enforcement, M9 primary privacy, B2 closure, and privacy-closed-only serialization.

1 Preserved historical Branch scope and secondary Candidate/Identification

permissions, rejected Program-level required Period execution, and added no schema, migration, ranking, universal score, production provider, or B7+ route.

1 Remediated the independent-review sparse-total blocker by using one exact

authoritative numeric child collection for both total summation and Relationship terms. Rectangular `no_data` Cells remain absent from equations, empty totals remain `no_data`, one-child dependencies remain connected, and B2 global behavior is unchanged. The targeted independent sparse-matrix re-review passed, closing B6.

2026-09-06 - M10 B5 Organization Overview

1 Extended the closed M10 identity contract by exactly two P1 count metrics,

`programs_configured` and `active_programs`, and the `program_configuration` grain.

1 Added the first M10 aggregate route,

`GET /api/talent/organization-analytics/overview`, using the B3/B4 scoped query layer, M9 primary privacy, B2 privacy closure, and the mandatory privacy-closed serializer seam.

1 Preserved frozen historical Branch scope, optional-permission SQL omission,

exact `decision == identified` meaning, and no-data-not-zero behavior. No migration, entitlement, production provider, UI, or other M10 route was added.

2026-09-06 - M10 B3/B4 Access Context And Common Query Primitives

1 Added immutable Organization Intelligence access context through canonical

authentication, SchoolGroup, injected availability, existing analytics permission, tenant Academic Year, historical Branch scope, and independent Candidate/Identification/Student-drill/Learner-Profile permissions.

1 Added fail-closed commercial availability and breadth provider interfaces

with versioned test injection and no plan, entitlement key, or production numeric ceiling.

1 Added enabled annual Program-universe resolution and set-based frozen Cycle

queries for Program×Branch, Program×Grade, backend totals, independently gated Candidate/Identification memberships, and Program-grain M8 required execution. Current Placement is never historical grouping authority.

1 Added normalized context fingerprints and a mandatory future serialization

seam accepting only B2 closure results. `no_data` remains non-authoritative for exact reconstruction and is never zero.

1 Added focused B3/B4 security and query-count tests. No B5 route, aggregate

exposure, migration, schema, permission, entitlement, production provider, UI, deployment, or `tis.db` change was introduced.

2026-09-05 - M10 B2 Reconstruction Analyzer And Privacy Closure

1 Added a standalone, dependency-bounded B2 module using exact

`fractions.Fraction` RREF and augmented-rank checks, including mandatory per-coordinate uniqueness in underdetermined systems and fail-closed inconsistent-system handling.

1 Added deterministic, monotonic, component-local complementary closure with a

structural termination bound and magnitude-independent victim selection; analyzer failures restrict only the affected trusted component, while graph validation failures abort closure.

1 Added the explicit primary-privacy-before-closure orchestration path,

exact-visible-only derived-rate projection, and all-or-nothing derived payload guard for raw sibling leakage. `no_data`, coarsened replacements, restricted values, and visible zero retain distinct governed semantics.

1 Added executable V05-V25 fixed and bounded metamorphic tests. No aggregate

query, real M10 route, migration, schema, permission, entitlement, production privacy policy, UI, deployment, or `tis.db` change was introduced.

2026-09-05 - M10 B1 Pre-B2 Contract Hardening

1 Reconciled and encoded the approved closed B1/B2 identity vocabularies as

string-backed enums: ten metric codes, three measure components, and six membership grains, preserving exact serialized strings and canonical-key/ equality/hash behavior while rejecting unknown free text.

1 Recorded the approved metric-to-component/grain mapping as immutable

declarative metadata. `rate` and `percentage` are explicitly excluded from Cell identity and remain future derived disclosures after privacy closure.

1 Tightened `RelationshipTerm` to accept only an actual Python `int` equal to

`+1` or `-1`; bool, float, `Fraction`, `Decimal`, null, strings, zero, and other coefficients are rejected. Added focused boundary, vocabulary, serialization, canonicalization, tenant, transposition, and overlap tests.

1 Added no B2 reconstruction/closure logic, route, query, permission,

entitlement, migration, schema, or deployment behavior.

2026-09-05 - M10 B1 Privacy Relationship Graph Primitives

1 Added `talent_analytics_relationship_graph.py` with a tenant-bound canonical

Cell registry, value-independent Relationship identity/deduplication, Cell-to-Relationship adjacency, fail-closed graph validation, and deterministic bipartite connected-component discovery.

1 Added focused B1 tests for unknown/cross-tenant references, forbidden

coefficients, duplicate/degenerate terms, global-sign equivalence, insertion-order independence, disconnected and bridged components, shared Program/Branch coordinates, nested topology, transposition, overlap-pair symmetry, value/no-data independence, and prohibited dependencies.

1 Updated the V01-V25 ledger: V04/V20/V22 are implemented; V16/V24 are partial

only for B1 structural behavior. No rank/RREF/nullspace/reconstruction, complementary graph closure, victim selection, query, route, schema, migration, entitlement, privacy threshold/provider, UI, or deployment was introduced.

2026-09-05 - M10 B0 Organization Intelligence Contract / Conformance Harness

1 Added `talent_org_intelligence_contract.py` as narrow non-functional

scaffolding for canonical value-independent Cell identity, exact `+1`/`-1` additive Relationship topology, inherited M9 privacy projection states, tenant separation, and explicit V01-V25 ownership/status traceability.

1 Added `tests/test_talent_org_intelligence_contract.py` to execute V01-V03,

state/canonicalization/forbidden-field contracts, prove no real M10 route is registered, and retain every later B1/B2 vector without falsely marking it implemented or silently skipping it.

1 Added no query, route, graph solver, reconstruction analyzer, privacy

threshold/provider, permission, entitlement, production limit, schema, migration, UI, Talent Score, Potential Rate, or cross-Program normalization. B1/B2 execution remains deferred.

2026-09-05 - Cline KMS Configuration And Three-Agent Orchestration

1 Added Cline project rules and the reusable tis-kms-developer skill, governed by

existing AGENTS.md/KMS with scope, safety, validation, and deployment safeguards.

1 Approved Codex, Claude Code, and Cline/ClinePass as the development pool, with

flexible master-conversation allocation by complexity, specialization, risk, review independence, and usage/load.

1 Documented setup, slash/natural-language invocation, skill discovery caveats,

and independent verification. No application/schema/production change.

1 Preserved the then-existing M9 KIA state without modifying M9 implementation or governance; generated artifacts are synchronized from

the combined current authoritative Markdown.

1 See [module history](history/engineering-handbook/2026-09-05-cline-kms-configuration.md).

2026-09-05 - Talent & Potential M9 Deterministic Talent Analytics (Implemented, Remediated, Then Committed/Pushed On dev)

1 Added the read-only, aggregate-only `/api/talent/analytics` API (context,

overview, rubric-distribution, kpi-distribution, competencies, branch/grade/section breakdowns, period-comparison, Student drill) and dedicated Administrator-only `talent_analytics.view`/`talent_analytics.view_students` permissions, over the existing frozen population/Assessment/Competency- Result/Candidate/Identification history.

1 Added the `Cell`/`Group` privacy-decision and complementary-suppression

engine (`talent_analytics_privacy.py`) with a fail-closed production provider (`resolve_privacy_policy_provider() -> None`).

1 Remediation pass (same day, independent review): fixed a BLOCKER where

`rubric-distribution`, `competencies`, and `breakdowns/*` built Cells/Groups and ran complementary suppression WITHOUT first calling `apply_primary_privacy`, so every cell reached suppression in a fake default `visible`/`value=None` state - this raised an unhandled 500 on `rubric-distribution` and silently published null-valued `visible` cells on the other two routes. Added a structural regression test statically guarding every `run_complementary_suppression` call site.

1 Implemented the Product-Owner-approved `coarsened` contract: valid only

when the policy decision supplies an explicit safe replacement value; otherwise fails closed to `suppressed`. Fixed several serialization sites that previously discarded any `coarsened` value as if it were unset.

1 Implemented real `competency_id` filter enforcement (tenant- and

Framework-Version-bound) - it was previously parsed/fingerprinted but never applied to any query.

1 Added comparability tests for `missing_cycle`, `no_frozen_population`, and

`metric_unavailable`; documented `candidate_policy_changed` as dead/ unreachable under the current data model rather than inventing a new code path, and confirmed

`incompatible_kpi_semantics`/`no_valid_kpi_result` do not exist in the implementation.

1 Test count grew from 21 to 43 in `tests/test_talent_analytics.py` in this

pass. No migration, schema change, or production privacy-policy value was added; no PostgreSQL validation was run. M9 is committed and pushed on `dev` at `23ade9a7c6166197140b48a3edbfac849396d580` (commit `feat: add deterministic talent analytics`); it is not deployed, not production- released, and not merged to `master`.

1 Second remediation pass (same day, independent re-review): fixed a

HIGH-severity defect of the same reconstruction-risk bug class relocated past the call-ordering guard above - `rubric-distribution` and `period-comparison` each privacy-evaluated only the `frozen_eligible` headline `Cell` (or, for `rubric-distribution`, evaluated no coverage privacy at all) and then serialized the raw per-status `coverage_metrics()` bundle directly, so a suppressed/coarsened/restricted status could still be reconstructed from its visible siblings. Extracted the already-correct `/overview` coverage pattern into one shared helper, `talent_analytics_service.build_privacy_safe_coverage_bundle`, which independently privacy-evaluates `frozen_eligible` and all 5 per-status counts as one `Group` with complementary suppression, deriving `assessment_started`/percentages only from a fully-visible group; both routes now use it instead of calling `coverage_metrics()` on raw counts. Audited every `coverage_metrics(/raw_coverage_counts(` call site in the M9 router and service modules; confirmed no third unsafe instance exists. Added a recursive runtime regression test that inspects the fully serialized response bodies under a suppressive policy and asserts no non-visible-state dict anywhere carries a raw coverage-shaped sibling field, plus direct unit tests for the new shared helper. Test count grew from 43 to 71 in `tests/test_talent_analytics.py`. No migration, schema change, or production privacy-policy value was added; no PostgreSQL validation was run. M9 is committed and pushed on `dev` at `23ade9a7c6166197140b48a3edbfac849396d580` (commit `feat: add deterministic talent analytics`); it is not deployed, not production-released, and not merged to `master`.

2026-09-05 - Talent & Potential M8 Annual Evaluation Plans And Periods

1 Added first-class annual Plan and stable ordered Period persistence with

scoped composite keys, lifecycle/date/cancellation checks, normalized uniqueness, dense ordering, stale-write revision, and rollover provenance.

1 Added nullable one-to-one Period linkage to Assessment Cycle. No existing

Cycle is inferred or backfilled; NULL remains canonical ad-hoc behavior.

1 Added Plan/Period services and bounded APIs for lifecycle, edits, future-tail

reorder, cancellation, closure, rollover, eligible-period selection, and atomic link/unlink. Linked Cycle Open revalidates Plan/Period context under the canonical Plan -> Period -> Cycle lock order.

1 Added Administrator-only `talent_evaluation_plans.view/manage/govern`, true

AND permission enforcement, organization-only mutations, and Cycle projection/action/warning zero-leakage without Cycle-view permission.

1 Extended `TalentConfigurationAudit` for bounded Plan/Period/link events and

added focused M8 and M1-M7 regression coverage. M9 Deterministic Talent Analytics was added the same day (see above); M10, Student evidence, Learning Style, and AI were not added.

2026-09-04 - Talent & Potential M7 Longitudinal Learner Profile

1 Added the read-only `/api/talent/learner-profiles/{student_id}` aggregation

and dedicated Administrator-only-by-default `talent_learner_profiles.view`. It groups one Student's authorized historical Placement, Program, Academic Year, Cycle, exact Framework, Assessment/Competency/KPI, Review Candidate, and Official Identification history without materializing a profile table.

1 Reconciled direct Student placement read APIs with the approved historical

Branch policy: history lists filter per stored Branch and unauthorized effective/single placement reads return non-enumerating 404 responses.

1 Educator Input is fully absent from profile responses and timelines without

its independent `.view` permission; authorized readers receive only current latest inputs within persisted historical Branch scope. No M8 periods, Learning Style, analytics, AI, exports, notifications, or `tis.db` change.

1 M7 permission composition closure makes `talent_learner_profiles.view` base

profile authority only. Review Candidate, Official Identification, and Educator Input profile sections and timeline events now each require their corresponding independent M6 `.view` permission, with no hidden metadata.

2026-09-04 - Talent & Potential M6 Independent Review Closure

1 Enforced persisted historical-Branch authorization on Educator Input create

and amendment writes; an unauthorized result is rolled back and returned as the same non-enumerating 404 used by direct reads.

1 Added exact composite relational binding from Official Identification to its

Review Candidate context, preventing Assessment/member/Student/Program/year/ Framework/tenant drift even through direct database writes.

1 Added database-enforced single-successor Educator Input lineage, preventing

concurrent amendment forks, with exact same-Student/Program/year/tenant composite lineage integrity.

1 Standardized direct out-of-scope Review Candidate, Official Identification,

and Educator Input access on non-enumerating 404 behavior.

1 Added focused adversarial tests. Result: 42 focused M6 tests passed; 137

combined M1-M6, permission, tenant-isolation, migration, and startup tests passed with 3 PostgreSQL-only skips. Live PostgreSQL execution remains a follow-up.

2026-09-04 - Talent & Potential M6 Review, Official Identification & Educator Input (Complete)

1 18 approved Product Owner governance decisions resolved the 9 questions the

prior "M6 Review Candidate Deterministic Evaluation (Partial)" entry left open (below). M6 is now complete; see "M6 Governance Review: Decisions (Resolved)" in `docs/AI_PROJECT_CONTEXT.md` for the full decision record.

1 Added migration

20260904_007_talent_review_workflow_identification_educator_input_foundation: adds status/reviewed_by_user_id/reviewed_at to talent_review_candidates (CHECK status IN ('pending_review','reviewed')), creates talent_official_identifications and talent_educator_inputs, widens TalentAssessmentAudit.resource_type further to add review_candidate_review, official_identification, educator_input, and relaxes talent_assessment_audits.cycle_id/framework_version_id to nullable (Educator Input's Cycle binding is optional).

1 Extended talent_review_candidate_service.py: a qualifying candidate now

starts status="pending_review"; a non-qualifying evaluation now writes a structural TalentAssessmentAudit row (action="evaluate_non_qualifying", outcome=false, no free text, no durable negative entity) instead of nothing; new mark_reviewed() implements the one-way pending_review -> reviewed transition (POST /api/talent/review-candidates/{id}/review, talent_review_candidates.manage).

1 Added talent_official_identification_service.py and

routers/talent_official_identifications.py (/api/talent/official-identifications, dedicated talent_official_identifications.view/record): append-only identified/not_identified decisions, one per Review Candidate (unique-constraint- and service-enforced, 409 on a second attempt), recordable only once the candidate is reviewed. .record requires organization/global access scope in addition to the permission - a Branch-scoped actor is denied even when granted .record. No mutation/ revocation/re-identification path exists.

1 Added talent_educator_input_service.py and

routers/talent_educator_inputs.py (/api/talent/educator-inputs, dedicated talent_educator_inputs.view/add/amend): bounded qualitative Student evidence (observation/context/supporting_evidence, <=2000 characters, non-empty) with mandatory historical Placement/Branch/Grade/ Section resolution (frozen Cycle context if supplied, else the Student's StudentAcademicPlacement effective at observed_at, rejecting cleanly with no silent fallback to current Placement). Append-only via supersedes_educator_input_id lineage with M3-style cycle prevention; default reads return only the current version, GET /{id}/history returns the full chain; the free-text body is stripped from audit payloads.

1 Updated the existing Review Candidate foundation test file's structural-

separation assertion (TalentOfficialIdentification now exists) and its non-qualifying-evaluation assertion (now audited, not silent); added tests/test_talent_review_official_identification_educator_input.py covering the Review workflow, Official Identification authorization/ durability, and Educator Input binding/amendment/privacy behavior.

1 Deferred, explicitly and permanently per these decisions: Official

Identification revocation/supersession/second decision/re-identification, a deferred decision state, a generic free-text review-note/case-management system, assessor assignment, Learner Profile, Development and Support, analytics/Talent Map, AI/AI Suggested Review, and Educator Input analytics/export/AI/attachments.

2026-09-04 - Talent & Potential M6 Review Candidate Deterministic Evaluation (Partial)

1 Added migration 20260904_006_talent_review_candidate_foundation with

talent_review_candidates (one row per Assessment, composite tenant/Cycle/ Program/Academic Year/Framework/policy scope) and a widened TalentAssessmentAudit.resource_type CHECK adding

review_candidate. No Official Identification or Educator Input table was added.

1 Added talent_review_candidate_service.py: deterministic evaluation of the

exact M3 TalentReviewCandidatePolicy/TalentReviewCandidateRule attached to a Completed Assessment's exact Framework Version, using rubric_level_at_or_above (by TalentRubricLevel.display_order, never numeric_value) and kpi_at_or_above (by the Assessment's persisted kpi_result, never recomputed), with all/any composition. No policy attached means no candidate is inferred. Evaluation only reads TalentStudentAssessment/TalentStudentCompetencyResult/TalentAssessmentCyclePopulationMember, never writes them.

1 A qualifying evaluation persists one candidate row with policy identity,

match mode, a deterministic SHA-256 fingerprint, and a full rule-by-rule evaluation snapshot; re-evaluation is idempotent. A non-qualifying evaluation persists nothing today (see the M6 Governance Review in docs/AI_PROJECT_CONTEXT.md for the specific open Product Owner question this leaves unresolved).

1 Added Administrator-only-by-default talent_review_candidates.view/manage

and /api/talent/review-candidates (POST .../evaluate, GET, GET/{id}), deliberately separate from talent_assessments.* and talent_programs.*. Branch visibility uses the frozen Cycle Population Member Branch; cross-tenant/foreign lookups return a uniform 404.

1 Per explicit Part V stop-condition discipline, Official Identification, a

mandatory review-lifecycle/"Pending Review" gate, and Educator Input were NOT implemented: the repository KMS documents these only as one-line "deferred" markers with no exact decision vocabulary, required access scope, binding, taxonomy, or amendment model anywhere. Building them would have required inventing governance rather than following it, so this milestone stops here and reports the exact open questions instead. No AI, analytics, Learner Profile, bulk identification, or tis.db change was made.

2026-09-04 - Talent & Potential M5 Student Assessment And Competency Results

1 Added migration 20260904_005_talent_student_assessment_competency_results

with canonical frozen-member-bound Student Assessments, exact Framework Competency Results, composite tenant/Program/Framework/rubric integrity, and scoped M5 audit context. No assessment rows are backfilled.

1 Added an Open-cycle-only assessment lifecycle: In Progress is editable with

expected-revision protection; Completed requires every Framework Competency and is read-only; Incomplete and Insufficient Evidence are distinct read-only terminal outcomes. No correction/reopen workflow was added.

1 Product Owner approved KPI result closure: weighted_level_average uses

integer rubric-value times basis-point weight contributions, denominator 10,000, nearest-integer ROUND_HALF_UP, and no binary floating point. A successful KPI-backed completion persists the Framework-specific result, scale, numerator, method, timestamp, and canonical SHA-256 input fingerprint. Qualitative/no-KPI completions persist no KPI sentinel or zero.

1 Added Administrator-only-by-default talent_assessments.view/manage/complete

and /api/talent/assessments, enforcing historical frozen-Branch visibility. Extended TalentAssessmentAudit for assessment/result transitions without copying free-text evidence. No Review Candidate materialization, Official

Identification, AI, analytics, assessor assignment, deployment, or `tis.db` change was made.

2026-09-04 - Talent & Potential M4 Assessment Cycle And Frozen Population

1 Product Owner approved one SchoolGroup-wide Cycle authority with no

`branch_id`, explicit Draft-only `population_effective_at`, organization-only Open/Close, Branch-filtered identifiable population visibility, no reopen, no post-Open/late mutation, and deferred assessor assignment.

1 Added migration `20260904_004_talent_assessment_cycle_frozen_population`

with Cycle, frozen population member, and bounded operational-audit tables. The migration explicitly establishes the scoped Academic Placement unique target needed by frozen-member composite foreign keys.

1 Added dynamic Draft preview from effective-dated Academic Placement and

eligible annual-config grades. Open atomically freezes exact Student, Placement, Academic Year, Branch, grade, section, and PlanningSection provenance and stores deterministic full population count/fingerprint.

1 Added Administrator-only-by-default

`talent_assessment_cycles.view/manage/view_population/govern`. Branch Draft authors cannot Open/Close; organization/global scope is additionally required. Branch population readers receive only authorized Branch members and subset counts, based on frozen Branch after Open, with no full integrity disclosure.

1 Added `/api/talent/assessment-cycles`, explicit lifecycle/population service,

append-only `TalentAssessmentAudit`, and focused security/lifecycle/migration tests. No Student Assessment, candidate instance, AI, entitlement, UI, M5+, deployment, or `tis.db` change was made.

2026-09-04 - M3 Governance Closure: KPI Approval, Rubric Ordering, And Audit Granularity

1 Product Owner approved `weighted_level_average` as one bounded, optional,

Framework-specific KPI calculation primitive (not a universal/cross-Program score); made the approval explicit in `models.py` (`TalentKpiConfiguration` docstring near its `calculation_method` CHECK) and `talent_program_service.configure_kpi` (governed closed-enum comment) and in this KMS. Qualitative no-KPI Programs remain first-class; the enum stays closed pending future governed approval of any additional primitive.

1 Defined `rubric_level_at_or_above` candidate-policy semantics as the

configured Rubric Level order (`TalentRubricLevel.display_order`, lowest proficiency first), independent of `numeric_value`, so qualitative Programs order and (in a later milestone) evaluate correctly with no numeric levels. Investigated and confirmed `display_order` safely and unambiguously carries both presentation order and semantic proficiency rank - it is dense, unique per rubric, and fully reindexed on every add/remove/reorder, so no field change was needed; documented this explicitly on the model.

1 Extended `TalentConfigurationAudit.resource_type` (model CHECK constraint

and, since the audit table was created by the separate already-applied M2 migration, a widening step inside the M3 migration `20260904_003_talent_rubric_kpi_candidate_policy_foundation` using the repo's existing `_replace_postgres_check/_sqlite_rebuild_from_current_metadata` pattern) to add `rubric`,

rubric_level, rubric_descriptor, kpi_configuration, kpi_component, review_candidate_policy, and review_candidate_rule. Every M3 mutation in talent_program_service.py now audits its specific child resource (post-flush id for new rows) instead of only framework_version, matching the existing M2 framework_competency precedent. reorder_rubric_levels (the one M3 bulk operation) appends one rubric_level audit row per reordered level.

1 Added/extended focused regression coverage in

tests/test_talent_rubric_kpi_candidate_policy.py for child-resource audit identity (rubric, rubric_level add/reorder, rubric_descriptor, kpi_configuration, kpi_component, review_candidate_policy, review_candidate_rule), the CHECK-constraint widening migration, and order-based qualitative rubric semantics. Re-verified the three previously-fixed defects (enabled-KPI numeric-scale guard, duplicate/contradictory candidate-rule rejection, clean competency_in_use rejection) remain intact. No M4, AI, or generic rule-engine/expression-language behavior was introduced; no commit, push, merge, deploy, or tis.db change was made.

2026-09-04 - Talent & Potential M3 Rubric, KPI, And Review Candidate Policy

1 Added migration 20260904_003_talent_rubric_kpi_candidate_policy_foundation

with Framework-versioned rubric, configurable ordered levels, exact competency/level descriptors, optional bounded KPI configuration/components, and deterministic Review Candidate Policy/rules. No data is seeded.

1 Added Draft-only, expected-revision-protected service and API mutations;

every change refreshes the complete M2+M3 semantic fingerprint and appends an M3 action through the existing TalentConfigurationAudit.

1 Kept qualitative Programs first-class: KPI and numeric level values are

optional. When enabled, KPI is limited to a validated weighted level average with basis-point weights totaling 10,000. Candidate policy is limited to ordered all/any rubric-threshold and optional KPI-threshold rules; it is configuration only and creates no candidate/result records.

1 Extended Framework cloning to deep-copy all M3 configuration while mapping

competency references through durable lineage. Added composite tenant/ Program/Framework integrity and exact membership checks.

1 Reused talent_programs.view, .manage, and .govern; organization/global

governance for activation/retirement is unchanged. Added focused M3 and M1/M2/permission/migration regression coverage. No M4+, UI, AI, entitlement, deployment, or tis.db change was made.

2026-09-04 - Checkpoint A Governance Closure: Student & Talent Program Permission Reconciliation

1 Documented the already-implemented Student & Academic Placement Foundation

(migration 20260904_001_student_academic_placement_foundation: Student as a canonical SchoolGroup-owned identity distinct from User, external identifiers, effective-dated Academic Placement with no separate Enrollment entity, and append-only audit) and Talent Program & Framework Foundation (migration 20260904_002_talent_program_framework_foundation: durable organization-owned TalentProgram, annual configuration, immutable-once- active Framework Version lifecycle, competency lineage, and append-only audit)

as implemented evidence in `docs/AI_PROJECT_CONTEXT.md` and `docs/PROJECT_STATE.md`. Neither foundation had prior KMS documentation.

1 Reconciled both foundations' temporary permission mapping against the

canonical `permission_registry.py` architecture. Added dedicated `students` (`.view`, `.create`, `.edit`, `.activate_deactivate`, `.manage_identifiers`, `.manage_placements`) and `talent_programs` (`.view`, `.manage`, `.govern`) permission groups, both Administrator-only by default (matching the existing `users.*` precedent), and updated `routers/students.py` and `routers/talent_programs.py` to check them.

1 Fixed a real permission-governance defect: Student Academic Placement

mutation and Talent Program/Framework Draft authorship were both gated only by the broad, commonly-granted `planning.edit_section` permission, so any Branch-scoped Editor/User actor holding it could manage Student placements or author organization-wide Talent configuration with no Student-specific or Branch-relationship constraint. Student creation/edit/identifier management had separately been gated by borrowed `users.*` keys instead of a dedicated Student permission domain.

1 Preserved every already-correct behavior: branch-scope-after-permission-

check ordering, non-enumerating 404-vs-403 discipline on every route, and the M2 organization/global access-scope gate (`_organization_authorized()`) on Framework activate/retire and Program lifecycle transitions, which was already correct and required no change.

1 Added permission-boundary regression tests to

`tests/test_student_academic_foundation.py` and `tests/test_talent_program_framework_foundation.py` confirming an Editor-role Branch actor - who still default-holds `planning.edit_section`

1 is now denied Student and Talent Program management.

1 No M3 (Rubric/KPI/Assessment/Review/Learner Profile/analytics/AI) code,

`tis.db` change, migration change, or unrelated route change was made.

2026-09-04 - Planning Subject Requirement Removal (Single And Bulk)

1 Corrected Planning section deletion after requirement removal: inactive

zero-period setup suppressions with no Curriculum Adjustment history are now cleaned atomically with an otherwise dependency-free section. This is not a general cascade; active requirements, permanent demand history, teacher assignments, timetable/calendar dependencies, and scheduling rules still block. The removable-demand error now says "Remove Subject Requirement".

1 Fixed a real gap where "Remove demand" only appeared for a Planning

requirement with an explicit, untouched `PlanningSubjectDemand` row - a purely legacy-fallback requirement (resolved only from the Subject catalog, no explicit row at all) had no removal action, so the same teacher's subjects could inconsistently show the action for reasons unrelated to the teacher.

1 Added `_get_planning_requirement_removal_status`, classifying every

requirement as `removable`, `permanent` (Curriculum-Adjustment-touched, now shown with an explicit "Protected" explanation instead of a hidden action), `fallback`, or `not_found`, reusing the same resolution authority already used to render the page.

1 A `fallback` requirement is now removable by creating an explicit

`is_active=False`/`weekly_periods=0` suppression row, rather than leaving no removal path at all. Unlike a Curriculum Adjustment retirement, this row sets only `created_by_user_id` (audit trail) and leaves `updated_by_user_id` NULL, so it stays classified `removable`/setup-only instead of immediately becoming permanent - corrected same-day after an initial version stamped both columns like a Curriculum Adjustment retirement, which made a plain admin cleanup permanently undeletable and broke the remove requirement -> remove section -> remove subject workflow. See `docs/history/workforce-planning/2026-09-04-planning-subject-requirement-removal.md` for the full correction.

1 Added `POST /planning/subject-requirements/remove` handling both single

("Remove Subject Requirement") and checkbox-selected bulk ("Bulk Remove Subject Requirements", supported across every section on the page) removal atomically: every target is scope-validated and classified before any mutation, and any protected/invalid/out-of-scope target blocks the whole request with a named reason.

1 Confirmation dialogs precede both single and bulk removal. Teacher

assignments, timetable data, and Curriculum Adjustment audit records are never touched. The prior round's demand-id-only delete route remains in place, no longer linked from the UI, and is now the supported way to fully clear a leftover setup-only suppression row.

1 Completed the bulk-action UI: each expanded Planning section now has a

section-scoped **Select all** checkbox beside a trash-icon removal action. The label reflects the live selection (singular or exact plural count), partial selection produces the standard indeterminate state, and removal controls remain disabled until at least one requirement is selected.

2026-09-03 - Claude Code KMS Configuration

1 Added root `CLAUDE.md` as the concise Claude Code repository entry point.

1 Added `.claude/skills/tis-kms/SKILL.md` with the reusable TIS investigation,

implementation, KIA, synchronization, validation, and reporting workflow.

1 Added `.claude/agents/tis-kms-developer.md` for bounded delegated TIS work under

the same tenant, worktree, database, deployment, and no-commit safeguards.

1 Kept root `AGENTS.md` and the authoritative KMS as governing sources; the new

configuration adds no application, schema, migration, or deployment behavior.

2026-09-03 - Return-To Reopen-On-Load UI Pattern

1 Added `redirect_utils.py`, a shared `safe_redirect_path()` open-redirect

guard plus `redirect_with_notice()`/`redirect_with_error()`, moved out of `main.py` (which now imports them under their original private names, so every existing caller keeps working unchanged) so any router can reuse the same validation instead of reimplementing it.

1 Added `static/js/reopen-on-load.js`, loaded globally from `base.html`: on

page load it resolves a target id from `window.location.hash` or an optional inline `window.__tisReopenTargetId`, opens it if it is a `<details>`, and scrolls it into view; a missing/stale id is a silent no-op.

1 Planning is the first consumer: each section's `<details>` now has a

stable `id`, "Remove demand" passes `return_to` back to that section's fragment, and `delete_planning_subject_demand`/`delete_planning_section`'s in-place blocked renders set the same reopen target - so an admin working inside an expanded section stays there after the action completes.

1 No SPA conversion, no new storage mechanism; Subjects and Teachers were

not changed.

2026-09-03 - Planning And Subject Delete Dependency Guards

1 Added a read-only dependency check before Planning section and Subject

deletion so every blocking category is named in a customer-safe message instead of raising an unhandled `IntegrityError`.

1 `TeacherSectionAssignment` is now a blocker like every other Planning

section dependency instead of being silently auto-deleted; no documented product rule required that cascade.

1 Extended Subject delete/Bulk Delete to also check `TimetableEntry`,

`CurriculumAdjustmentAudit`, and `SubjectDistributionRule` references, which previously had no application-level check and could be silently orphaned.

1 Bulk Subject delete is atomic: any blocked Subject in the selection stops

the whole batch and names every blocked Subject with its reason.

1 Split Planning subject demand into removable and permanent using the

existing `updated_by_user_id` column (set by Curriculum Adjustment, never by the one-time setup backfill) and added a new, narrowly scoped `GET /planning/subject-demand/delete/{demand_id}` action ("Remove demand" on the Planning page) that hard-deletes an untouched row so the section or subject can then be deleted; a touched row remains a permanent, honestly explained blocker.

1 Added no archive/closed/inactive lifecycle, broad schema change, migration,

or cascade deletion.

2026-09-02 - Professional Timetable Lifecycle UX

1 Made current publication, editable Draft, validation/approval, and immutable

history state explicit in the workspace and version history.

1 Added safe working-Draft creation from current published and historical versions;

publication remains unchanged until explicit approval and publication.

1 Clarified empty Draft, Generate Draft, Regenerate Draft, and stale-input language.

1 Added canonical mutation/publication conflict evidence and visible refresh

remediation while preserving legacy response messages.

1 Added no schema, migration, solver, or Workflow behavior.

2026-09-01 - Structured Timetable Conflict Evidence

1 Added a canonical timetable conflict contract and stable taxonomy across readiness, feasibility, generation terminal outcomes, and validation.

1 Added authorization-safe entity serialization and internal requirement/constraint correlation without exposing hashes or solver internals.

1 Preserved legacy blocker, diagnostic, validation-error, generation-message, status, and API envelope behavior through additive `conflicts` fields.

1 Kept infeasible, timeout, cancellation, stale authority, and execution failure distinct without introducing a conflict table or migration.

2026-09-01 - Timetable Lesson Requirement Projection

1 Added one deterministic, read-only projection from explicit-first Planning demand and existing teacher authority into timetable requirements.

1 Added internal requirement identities and source fingerprints to schema-v5 snapshots while retaining schema-v3/v4 problem-builder compatibility.

1 Routed readiness and snapshot generation through the shared projection and carried its provenance through problem construction and independent validation.

1 Added no table, migration, public CRUD API, Room, availability, Resource, or co-teaching behavior.

2026-09-01 - Timetable Solver Adapter Boundary

1 Added a solver-neutral worker contract for solve and infeasibility-diagnostic operations while retaining OR-Tools CP-SAT as the only Release 1 binding.

1 Kept problem construction, immutable snapshots, independent validation, stale-result protection, and atomic version persistence outside the adapter.

1 Added focused contract-forwarding tests; no schema, API, UI, or deployment topology changed.

2026-08-31 - Stale Draft Feasibility And Regeneration Workflow

1 Separated stale Draft lifecycle state from current-input structural readiness.

1 Enabled a fresh hard-only feasibility check for stale-only scopes and allowed

regeneration only after the current authority fingerprint is verified.

1 Preserved stale Draft publication protection and stopped older-authority run

failures from overriding the current workflow state.

1 Classified the current populated mutable unpublished Draft as the regeneration

source regardless of manual/generated/regenerated origin, while retaining Generate for empty starters and excluding active or historical versions.

2026-08-30 - Partitioned Subject Sessions

1 Replaced overlapping occupied-pair inference with explicit double-block and

single-session partitioning in CP-SAT.

1 Allowed separate sessions to touch while ensuring each lesson period belongs

to exactly one session and physical timeline interruptions still break doubles.

1 Strengthened daily-coverage propagation through daily session/load channeling

and aligned the independent validator with the same exact partition contract.

2026-08-29 - Reduce-Only Curriculum Adjustment

1 Added transfer and reduce-only adjustment action types to the guided preview/apply workflow.

1 Reduce-only requires no target subject or teacher decision and preserves all atomic, Draft, rule, lock, publication, permission, and tenant safeguards.

1 Subject Details now shows effective Planning weekly periods and links to a prefilled grade-specific reduction flow.

1 Replaced customer-facing catalog-default wording with **Original weekly periods**.

2026-08-29 - Curriculum Transfer and Subjects Display Correction

1 Made the administrator-entered transfer count authoritative across UI, preview fingerprint, and atomic apply.

1 Corrected partial transfer arithmetic and retained source demand unless its reviewed result is zero.

1 Updated Subjects to show exact-scope effective Current/New Planning periods, with **Varies** and section detail when demand differs.

2026-08-29 - Guided Curriculum Adjustment UI

1 Added a professional five-step administrator workflow for scope, change, preview, teacher decisions, final review, and success.

1 Added permission-gated Planning and Subjects entry points and customer-safe stale-review recovery.

1 Reused the Stage 3 preview and Stage 4 atomic apply authority; no solver, published-history, or automatic-regeneration behavior changed.

2026-08-29 - Atomic Curriculum Adjustment Apply

1 Added dedicated `curriculum.adjust` and a fingerprint-required apply endpoint.

1 Added exact-scope locking, active-generation rejection, qualification/capacity,

scheduling-rule and Draft-lock validation, atomic demand/assignment/rule/Draft updates, and complete rollback on failure.

1 Added durable applied audits and duplicate-fingerprint protection through migration

`20260829_001_curriculum_adjustment_apply_foundation`.

1 Preserved snapshots, placements, published history, active pointer, and explicit

later-regeneration behavior.

2026-08-29 - Read-Only Curriculum Adjustment Preview

1 Added exact-scope preview for grade, selected-section, and all-active-use subject

demand transfers using `PlanningSubjectDemand` authority.

1 Added current/suggested teacher and capacity projections, Subject Scheduling Rule

validation, grouped legacy warnings, Draft stale/regeneration impact, and a deterministic stale-confirmation fingerprint.

1 Added a `planning.edit_section`-protected JSON API without adding any apply,

assignment, regeneration, publication, or database mutation behavior.

2026-08-28 - Planning Subject Demand Consumer Transition

1 Routed live Planning totals, teacher workload, timetable readiness/workspace/

snapshot/generation input, Subject Scheduling Rule arithmetic, and required-hours reports through exact-scope explicit-first section demand resolution.

1 Made inactive and zero demand authoritative for one section without affecting

other sections; missing rows retain legacy Subject weekly-hours fallback.

1 Preserved teacher assignment behavior, solver design, timetable UX, and immutable

published timetable history.

2026-08-28 - Explicit Planning Subject Demand Foundation

1 Added `PlanningSubjectDemand` as an additive, normalized future authority for

per-section subject weekly periods, including active/retired lifecycle metadata.

1 Added composite branch/year integrity and one-active-row-per-section-subject

uniqueness while allowing inactive history.

1 Added migration `20260828_004_planning_subject_demands_foundation` with an

idempotent, branch/year-isolated Current/New Planning backfill from grade-matched Subject weekly hours that preserves any existing retirement evidence.

1 Added explicit-first demand resolution with legacy fallback. Existing Planning,

teacher allocation, reporting, timetable, draft, and publication behavior remains unchanged in this foundation stage.

2026-08-28 - Central Tenant Report Branding

1 Added `tenant_report_branding.py` as the reusable tenant-facing report/export

logo authority over existing branch/organization-scoped branding resolution.

1 Removed TIS product-logo fallback injection from timetable PDF/XLSX and

academic-calendar PDF exports; an unbranded tenant now receives a clean no-logo report header.

1 Routed observation PDF branding through the same authority without changing

report content, permissions, formats, or tenant isolation.

1 Kept TIS branding unchanged in the application UI and platform-owned surfaces.

2026-08-28 - Subject Scheduling Rules UI

1 Centralized operational timetable grade/section choices in

`planning_scope_service.py`, restricted to Current/New PlanningSection rows for one branch/year. Subject Scheduling, timetable workspace filters, and directly related calendar/grouped section selectors now share this authority.

1 Restricted the main and copy-rule grade selectors, plus their subject rows, to

distinct `PlanningSection` grades in the selected branch and academic year; global and Subject-catalog-only grades no longer appear in operational timetable configuration.

1 Corrected the modal visibility regression by removing author CSS that forced a

closed native `<dialog>` to render. The modal now opens only after Edit populates the selected subject and closes through Cancel/Close.

1 Polished the modal with aligned two-column double-block/single-session controls,

one full-width arithmetic summary, and a balanced two-column Conditions layout; advanced conditions, Reset To Default, and section overrides remain secondary.

1 Added a grade-first Subject Scheduling Rules section to Timetable Settings;

administrators select one grade and can search its compact subject list with read-only Planning totals, current patterns, and Configured/Default status.

1 Added a modal editor with double-block and single-session steppers, two-period

Separate Sessions / Consecutive Double Block presets, combinable primary conditions, progressively disclosed advanced options, live client-side arithmetic validation against the Planning weekly total, and authoritative backend re-validation on save (`subject_distribution_rules_ui.py`).

1 Added Section Overrides (create/edit/clear per section), Reset To Default

(removes only the grade-level row), and Copy Rules From Grade (subjects matched by name across grades, skipping arithmetic mismatches instead of copying them).

1 Added routes under `/system-configuration/timetable-settings/subject-rules`

reusing the existing `timetable.manage_settings` permission and remaining tenant/branch/year-scoped; Planning weekly totals are never mutated.

1 Preserved the existing subject-code mappings and raw Swimming/grouped-activity

JSON editor behind a secondary Advanced / Legacy disclosure, Non-Teaching Blocks management, and the unchanged Stage 2 snapshot/solver/validator pipeline; saved rules apply to the next created snapshot only.

2026-08-28 - Subject Distribution Rules Generation Wiring

1 Embedded the fully resolved Subject Distribution Rule (section over grade over

branch default, `None` for legacy fallback) into every Planning demand at snapshot creation time; later rule changes never alter an already-created snapshot, and fingerprint authority is preserved.

1 Carried the resolved rule and true physical period adjacency into the problem

builder, which now runs the arithmetic/feasibility validator as a final pre-solve defense and fails cleanly with `distribution_rule_invalid`.

1 Added generalized CP-SAT enforcement: exact non-overlapping intentional

blocks using true adjacency, generalized daily-coverage/spread/max-per-day/ min-teaching-days behavior (hard only when explicitly configured hard), and exemption of declared blocks from the consecutive-avoidance penalty.

1 Mirrored every new hard rule in the independent validator and added

readiness blockers for genuinely invalid or infeasible normalized configurations.

1 Preserved legacy `quality_rules_json` behavior exactly for every demand

without a resolved rule, preserved grouped-activity/teacher/section exclusivity, and preserved the unchanged regeneration diversity default.

2026-08-28 - Subject Distribution Rules Foundation

1 Added the normalized `subject_distribution_rules` table (migration

`20260828_003_subject_distribution_rules_foundation`) scoped branch default, grade, or section, alongside the existing `quality_rules_json` authority.

1 Added a pure hierarchy resolver (section over grade over branch default, with

field-level inheritance) and a pure arithmetic/feasibility validator confirming configured block/single distributions match the authoritative Planning weekly total.

1 Corrected the canonical composed timeline to mark true physical period

adjacency, false across a Break, Prayer, or other non-teaching item, so a later intentional-block feature can rely on genuine continuity.

1 No existing CP-SAT, independent validator, readiness, or settings-UI behavior

changed; every existing tenant continues on current `quality_rules_json` behavior until normalized rules are explicitly configured.

2026-08-28 - Added Smart Timetable Academic Scheduling Quality

1 Added branch/year settings and migration-backed JSON authority for core, spread,

ICT, double-period, grouped Swimming, shared-resource, and diversity rules.

1 Extended immutable snapshots, problem construction, CP-SAT, readiness diagnostics,

and independent validation with daily core coverage, distinct-day spreading, ICT daily limits, and simultaneous grouped activities.

1 Preserved the regeneration-diversity default, locks, teacher authority,

publication transactions, and Render Workflow architecture.

2026-08-28 - Primary Draft Workflow And Meaningful Regeneration

1 Grouped Regenerate, Approve Draft, and Publish Timetable in the primary Draft area

while keeping deletion, archive, comparison, and version exports in History.

1 Replaced the capped regeneration difference with 25 percent of unlocked source

placements, rounded up, while preserving locks and all existing hard constraints.

1 Added a controlled insufficient-diversity result instead of silently accepting an

almost-identical timetable.

2026-08-28 - Explicit Draft Approval Before Publication

1 Replaced customer-facing Check Timetable with Approve Draft and Draft Approved.

1 Added durable approval timestamp and administrator provenance through migration

`20260828_001_smart_timetable_stage52_draft_approval`.

1 Kept generated and regenerated drafts unapproved, invalidated approval after every

validity-affecting draft change, and required publication-time revalidation before the unchanged atomic active-pointer transition.

2026-08-28 - Present One Logical Draft Across Generation

1 Kept backend generated-result versioning and provenance while presenting the

generated result as the continuation of the selected Draft Timetable.

1 Bound current Draft actions to the selected exact-scope version, refreshed Check

and Publish state coherently, and added Create New submit feedback.

1 Hid obsolete generated-source drafts from normal History cards and separated the

bulk unpublished action visually.

2026-08-28 - Canonical Generation Freshness Authority

1 Excluded generation-only runtime metadata from freshness fingerprints while retaining it in immutable solve snapshots.

1 Aligned generated-result authority with the generation snapshot contract and added safe component-level mismatch diagnostics.

2026-08-28 - Simplified Draft Ownership And Unpublished Cleanup

1 Made the newest mutable non-active exact-scope version the current Draft Timetable, including a fresh Create New Timetable draft, and bound Generate to that context.

1 Changed customer-facing draft deletion to permanent deletion for eligible never-published versions while retaining History-only archive behavior.

1 Added exact-scope Delete All Unpublished Timetables with dependency-safe cleanup, preserved snapshots and generation audit records, protected publication history, and visible partial-failure reporting.

1 Reconciled History deletion eligibility with the backend and replaced Create Draft Copy with Use as New Draft for immutable historical versions.

2026-08-26 - Replaced Always-On Timetable Worker With On-Demand Workflow

1 Added a registered Render Workflow task that receives only a durable generation run public ID and reuses the Stage 5.1 solve/validate/persist pipeline.

1 Added lightweight post-commit web dispatch, terminal handling for immediate dispatch failure, delayed-start feedback, and exact-run PostgreSQL claiming.

1 Kept leases, heartbeats, cancellation, staleness, independent validation, atomic persistence, and duplicate-result guards; solver mathematics and Stage 5.2 UX did not change.

1 Disabled Render automatic task retries and made the prior infinite polling worker optional/local only, removing the always-running production solver requirement.

2026-08-26 - Implemented Smart Timetable Stage 5.2 Simplified Workflow

1 Simplified management around one Working Timetable and secondary Timetable History.

1 Added permissioned Delete Working Timetable archive semantics that preserve history and never change the official active pointer.

1 Added active-pointer-only official resolution, teacher `/my-timetable`, a general published view, and view-only redirection away from drafts.

1 Added scoped visibility, deletion, wording, permission, and Stage 4/5.1 regressions.

2026-08-22 - Implemented Smart Timetable Stage 5.1 Generation

Added worker-isolated OR-Tools CP-SAT generation, schema-v3 immutable problem snapshots, an independent candidate validator, and durable PostgreSQL queue leasing, heartbeat, bounded recovery, cancellation, progress, and active-scope protection. Generate and Regenerate now create separate validated unpublished versions without changing published history; regeneration preserves locks and enforces an explicit minimum difference. Added `timetable.generate`, polling UI, additive migration `20260822_001_smart_timetable_stage51_generator`, and solver/lifecycle coverage.

2026-08-21 - Corrected Timetable Configuration With A Composed Day Timeline

Replaced fixed teaching-period clocks plus overlap classification with one per-day timeline composer. Blocks may be inserted after a period or preserved as fixed time when boundary-aligned; later periods and calculated end times shift automatically. Readiness, assignment validation, snapshots/fingerprints, timetable UI, and exports now share the projection. Added explicit additive placement metadata and expanded the controlled school block catalog. No automatic generation was introduced.

2026-08-21 - Added Smart Timetable Stage 4 Version Publication

Made timetable lifecycle visible through scoped history selection, immutable-version draft copies, draft lesson locks, explicit validation, same-scope comparison, selected-version exports, archive controls, and transactional publication. Publication revalidates authority and hard validity, checks edit/pointer revisions, supersedes the previous active version, and preserves history. No schema, solver, worker, generation endpoint, commit, push, deployment, or production action was included.

2026-08-19 - Added Smart Timetable Stage 3 Readiness And Slot Semantics

Added canonical fixed-period teaching/unavailable/invalid slots, server-side assignment protection, stale-placement preservation, snapshot integration, and an exact-scope read-only readiness service with customer-safe page status. No schema, solver, worker, Generate endpoint, commit, push, deployment, or production action was added.

2026-08-19 - Added Smart Timetable Stage 2 Version Foundation

Accepted ADR 0026 and added durable timetable versions, deterministic authority snapshots and SHA-256 fingerprints, one exact-scope active pointer, persisted placement locks, future generation-run metadata, and per-version section/teacher collision constraints. Existing populated timetables migrate without placement changes into imported compatibility versions; inconsistent rows remain present with safe version-level stale evidence, and settings-only scopes remain empty.

The current timetable page and exports now read one operational version. The existing assignment route uses copy-on-write so active history is immutable while the current editing experience continues through a mutable working draft. No solver, worker, generation route/UI, availability, room/resource, or rule model was introduced.

2026-08-19 - Restricted Compact Descriptions To Intentional UI Components

Changed the shared compact-description enhancer from broad tag and class-name inference to explicit `data-compact-description` opt-in plus the maintained allowlist of approved compact components. Ordinary

operational status, helper, note, subtitle, and supporting text now remains visible inline. Existing keyboard/focus tooltip behavior and unrelated native accessible titles remain unchanged.

2026-08-19 - Capacity-Based Packaging And Common Customer Feature Baseline

Accepted ADR 0025 and replaced the temporary plan-feature ladder with one normal customer baseline for coherent active paid, promo, and customer-demo workspaces. Starter, Professional, and Enterprise AI retain their existing branch, staff, and teacher ceilings. Advanced Reporting and enabled AI are source-aware and plan-neutral while permissions, tenant/branch/year scope, commercial lifecycle, branch entitlement, and globally disabled features remain fail-closed.

Added idempotent migration `20260819_001_capacity_based_customer_feature_baseline` for plan values, compatible legacy flags, current active promo operational feature values, and active demo baseline policy. Exact promo capacity and immutable grant/redemption history are unchanged. Added a separate minimal AI consumption-policy boundary and retained reservation, counter, and operation-event accounting. Updated only affected customer copy; report generators and calculations did not change.

2026-08-18 - Enforced SchoolGroup Management Boundary

Direct operational SchoolGroup creation and deletion now require Platform identity, the matching create/delete permission, and `schools.manage_all_schools`. Tenant role defaults no longer include top-level create/delete, stale permission state cannot bypass the handler guard, and tenant updates are limited to the linked SchoolGroup. Unauthorized requests fail before validation, storage, or database mutation.

School Management now uses unified commercial authority for source-aware branch capacity presentation, including accurate promo `used of allowed` state. Existing locked branch enforcement and atomic promo assignment remain unchanged. The individual last-active-branch safeguard now counts within the target SchoolGroup. Added a PostgreSQL-only, repeatable-read, rollback-on-exit provenance audit that reports suspicious unlinked active internal sandboxes for manual review without repairing them.

2026-08-18 - Completed Promo Expiry Recovery And Paid Continuation

Promotional operational access now ends exactly at `PromoGrant.effective_to`. `grace_period_days` is a commercial recovery window for Organization Account and paid continuation only; it never extends operational access and does not archive or delete tenant data.

Verified owners of expired or recovery-period promo workspaces can use the existing tenant paid-activation flow. Checkout and pending payment leave promo authority unchanged. Verified provider completion atomically ends the promo entitlement, relinks the existing tenant source to the paid contract, establishes paid workspace and branch entitlements, and preserves tenant identity, data, branding, ownership, and immutable promo history. Active-promo early conversion remains blocked.

Added one authority-driven commercial badge component for Promo, Demo, and Paid sources in Organization Account and the full commercial view. The normal operational header intentionally remains focused on tenant context. Expired promo access uses promo-specific continuation wording, and protected API responses return a customer-safe generic commercial code while internal reasons remain in server logs.

2026-08-18 - Added Promo-Aware Commercial Access Presentation

Organization Account Billing & Subscription now selects paid, demo, or promo presentation from the selected workspace's authoritative commercial source. Promo customers receive a dedicated read-only page for their grant plan, safe access status and period, masked reference, and centralized branch/system-user/teacher capacity. The page does not invoke Paddle or expose paid-only billing and mutation controls. Existing paid and demo journeys remain

unchanged, and promo-to-paid conversion remains deferred.

2026-08-17 - Corrected Promo Branch Entitlement Lifecycle

Promo activation now creates explicit active or inactive entitlement evidence for every preserved branch, including operationally inactive unselected branches that previously fell outside the selectable-branch query. Individual and bulk branch reactivation continue through centralized capacity authority and now update branch status, promo assignment, and entitlement atomically. Capacity exhaustion and contradictory evidence fail before commit.

Added a generic PostgreSQL-only reconciliation command that defaults to dry-run, locks and revalidates on explicit apply, and creates only deterministically missing inactive branch entitlements. It never changes branch status, assignment, grant, capacity, Paddle, or people records. The Al-Andalus audit now distinguishes broad operational-user row inventory from M1 authoritative active staff usage.

2026-08-13 - Corrected Existing Workspace Plan Selection

Organization Account **Choose a Plan** now reopens eligible paid-plan selection for an existing workspace while its activation remains `draft` or `checkout_ready`. The saved plan is highlighted and can be replaced with a newly calculated authoritative quote. Browsing or replacing a draft creates no Paddle request or PaymentAttempt and does not mutate branches. Once checkout starts, payment is processing, or evidence requires manual review, replacement fails closed. Promo, PendingOrganization checkout, and verified-payment activation authority are unchanged.

2026-08-07 - Corrected Existing Workspace Payment Status Presentation

Organization Account now displays `Activation required` for a converted existing workspace that has no real unresolved Paddle checkout attempt. Payment processing is shown only for a current, unexpired `PaymentAttempt` in checkout-started or payment- processing state. Failed, cancelled, and expired attempts use recovery states, while completed paid and promo activation continue to show their active commercial state. No payment, Paddle, promo, capacity, or commercial-authority transition changed.

2026-08-06 - Implemented M4C Existing Workspace Paid Activation

Added a SchoolGroup-anchored paid activation workflow for verified owners of activation-required existing customer workspaces. The implementation adds additive schema, strict checkout/payment context constraints, operational capacity and branch quote snapshots, workspace billing/customer mapping, Paddle transaction launch, webhook-only atomic paid authority, Organization Account activation UX, centralized paid entitlement resolution for `customer` classification, and focused regression coverage. It does not create PendingOrganization or invoke tenant provisioning.

Final PostgreSQL validation hardened reused-transaction verification, persisted workspace-specific provider address/business lineage, refreshed ORM state after workspace row locks for concurrent duplicate webhooks, and proved that paid-versus- promo races create exactly one commercial source with no partial losing records.

Starter remains unavailable until complete restricted-branch enforcement is proven. No production data or real Paddle records were used during implementation. Live Paddle Sandbox checkout remains an environment validation gate.

2026-08-06 - Controlled Existing Workspace Owner Alignment And Conversion

Added a durable, idempotent conversion ledger and dry-run-first PostgreSQL CLI for an existing audited internal sandbox. The process prepares an ownership claim without creating an account, requires normal account verification, aligns or safely reuses one tenant owner, and collects only legal name, IANA timezone, and educational program. A database partial unique index now enforces one active tenant-owner account link per SchoolGroup after duplicate preflight; conversion events are append-only.

Final conversion locks and re-audits the operation and workspace, validates unchanged branch/dependency and sandbox-authority snapshots, retires the active internal entitlement, and sets `customer / provisioning` atomically. It does not mutate branches or operational records and creates no pending organization, contract, subscription, demo request, promo grant, active entitlement, or tenant link. M1 resolves the result as `activation_required`; M3 promo remains the supported activation path while existing-workspace Paddle activation is deferred. No production data was modified during implementation or validation.

2026-08-05 - Existing Workspace Conversion Read-Only Audit

Added a generic M4A audit service and parameterized PostgreSQL CLI for gathering sanitized evidence before a legacy internal-sandbox conversion is designed. The audit validates the exact workspace identity, resolves the intended owner, inventories tenant/provisioning and paid/demo/promo authority, reflects SchoolGroup-scoped tables, and traverses direct and indirect branch foreign-key dependencies. Unconstrained branch-like references and incomplete traversal fail closed for manual review.

The CLI uses a repeatable-read, read-only transaction, prints one JSON object, or deterministic text, records its PostgreSQL transaction mode, emits a stable SHA-256 snapshot hash, and always rolls back. Coherent, execution-failure, manual-review, and identity-mismatch results use exit codes `0`, `1`, `2`, and `3`. Soft-deleted dependencies remain blocking, reflected foreign keys absent from ORM metadata force manual review, and archival recommendations contain only proven dependency-free active branches. This milestone performs no branch archival, owner alignment, workspace or entitlement change, tenant-link creation, write-mode conversion, Paddle call, email, schema change, migration, or production audit.

2026-08-05 - Promo Redemption, Immutable Grants, And Activation

Added customer-side secure promo lookup, resumable capacity review, exact branch selection, and atomic activation for completed onboarding and separately aligned existing organizations. Migration `20260805_002_promo_redemption_and_grants` adds activation sessions and branch selections, immutable redemptions and grants, grant branch assignments, and redacted redemption events. It also adds promo references to workspace entitlements and tenant links while enforcing exactly one paid, demo, or promo source.

M1 commercial authority, commercial access, branch queries, and future branch creation now resolve active promo grants and explicit branch entitlement. Organization-wide staff or teacher excess blocks activation without changing records; unselected branches remain operationally preserved but commercially inactive. Pending sessions provide no access, expiration fails closed, raw codes remain absent from storage and logs, internal sandboxes are not converted, and Paddle/payment behavior is unchanged.

2026-08-05 - Promo Code Foundation And Platform Console Management

Added secure definition-only Starter, Professional, and Enterprise AI promos with exact capacity bounded by the existing plan catalog, controlled targeting, validity and expiry policy, versioned lifecycle, duplicate/replacement support, optional branch restrictions, and durable redacted audit. Raw codes use at least 100 bits of secure randomness, are shown once in a no-store response, and are represented in storage only by dedicated-secret HMAC lookup authority and safe masked fragments.

Platform Console now exposes permission-controlled promo list, filter, create, detail, edit, activate, pause, revoke, duplicate, and replacement-definition workflows. Platform Developers may manage non-terminal definitions but only a Platform Owner may activate or revoke. Additive migration `20260805_001_promo_code_foundation` creates three tables without backfill. No customer redemption, commercial entitlement, M1 authority, Paddle, onboarding, provisioning, payment, or tenant behavior changed.

2026-08-04 - Unified Commercial Access And Capacity Authority

Operational branch, staff-user, and teacher growth now uses one commercial authority facade that composes the existing workspace, entitlement, tenant link, contract, confirmed subscription, commercial-access, demo-lifecycle, and plan-capacity authorities. Missing or contradictory authority fails closed; Customer Demo and Internal Sandbox remain explicitly unmetered in M1.

Paid branch limits use confirmed provider-reconciled quantity capped by the plan branch ceiling. Staff usage counts every distinct active tenant operational user, including operational owners and teacher-position users; platform and account-only identities are excluded. Teacher usage deduplicates normalized known teacher IDs while counting every blank legacy row separately. The same person may consume both one staff and one teacher slot.

Capacity-increasing branch, user, teacher, academic-year, and provisioning paths lock the SchoolGroup, recount, evaluate, mutate, and commit in one transaction. Existing over-capacity tenants retain data and access but cannot increase an exceeded dimension. Paid and demo provisioning validate final authority before activation completes. No schema, migration, Paddle, pricing, webhook, onboarding-transition, permission, or feature-packaging change was made.

2026-08-02 - Billing Contact Editing And Paddle Retry

Subscription Management now presents Billing Contact as read-only organization identity with an explicit Edit, Save Changes, and Cancel interaction. The legal identity label now covers an organization or school name, and the billing country uses the supported country selector.

Valid local billing changes remain committed when Paddle synchronization fails. A dedicated permission- and tenant-scoped retry reuses the saved profile and persisted customer, address, and business mappings, avoids duplicate provider objects, and performs no provider call once synchronized. Customer status copy distinguishes local save, provider synchronization, and retry failure. Safe server diagnostics identify the customer, address, business, or subscription identity step plus provider status/error code. A saved pending or failed billing identity prevents new plan or quantity changes until provider synchronization succeeds; cancellation and legacy active subscriptions without a profile remain unchanged.

2026-08-01 - Explicit Organization Billing Identity

Billing & Subscription now stores one explicit organization billing profile covering billing email, legal/billing name, contact, optional company and tax identifiers, and the supported address structure. This authority is independent from SaaS login email. Initial checkout confirms it, while active-tenant changes require organization ownership or `subscriptions.manage_billing` and revalidate tenant/provider linkage.

TIS reuses the mapped Paddle customer, synchronizes authorized email/name changes, creates or reuses one attributable active Paddle Business, persists the Paddle address/business mappings, updates the active subscription identity, and includes `business_id` on future initial checkout transactions. Ambiguous mapping and provider failures fail closed. Existing financial documents are not silently revised.

Provider transaction presentation now distinguishes `paid` as Payment received

1 processing from `completed` as Paid. Subscription change requests record the

payment-received timestamp separately, show that no customer action is required while provider processing finishes, and retain the existing `completed` reconciliation authority. Additive migration `20260801_001_organization_billing_identity` creates `organization_billing_profiles`, adds Paddle address/business mapping columns, and adds the separate payment-received timestamp.

2026-08-01 - Subscription Capacity Presentation And Billing Navigation

Subscription pages now separate confirmed paid branch quantity from the current plan's maximum branch ceiling. Review Capacity identifies additional required branches as additional billed quantity and retains system-user and

teacher counts as non-billed eligibility limits. Unapproved feature claims and empty feature placeholders were removed from customer plan comparison.

System Configuration now exposes Billing & Subscription only to a linked organization owner or user with explicit billing authority. The route revalidates operational tenant scope and the SaaS account-user relationship, then opens the existing Organization Account subscription page. Password and social authentication preserve only the allowlisted internal billing return; external or unrelated destinations fail back to Organization Account. The public landing label is Organization Sign In and still uses centralized SaaS authentication. No pricing, Paddle quantity, lifecycle, webhook, entitlement, schema, or migration behavior changed.

2026-08-01 - Unified Active Subscription Capacity Management

Subscription Management previously presented branch quantity as an isolated commercial action. One authoritative organization-capacity resolver now counts active branches, active tenant operational staff users, and active teachers. The portal exposes one Review Capacity flow and displays usage, plan limits, and remaining capacity for all three dimensions. The highest required dimension selects the minimum eligible plan: Starter supports 1 branch/5 system users/25 teachers, Professional supports 5/20/100, Enterprise AI supports 25/100/500, and Custom is required when any Enterprise AI limit is exceeded. Customers may still select a higher eligible plan for optional feature entitlements. Branch, system-user, teacher, or mixed growth can trigger an upgrade. Required branch growth can combine target plan and branch quantity in one provider preview. Paddle quantity remains active-branch count only; system-user and teacher capacity affect plan eligibility but never become billed quantity units.

Plan-change previews retain their three-dimension capacity evidence. All three dimensions are validated before a downgrade is submitted, and downgrades remain scheduled for the next billing boundary. The same three dimensions are revalidated when provider evidence reaches the effective date. Capacity growth that no longer fits enters manual review, does not activate the lower plan, and preserves the current safe entitlement state. Existing provider-authoritative proration remains unchanged. Higher entitlements require both provider subscription and payment confirmation. Cancellation remains scheduled at paid-period end, preserves access and tenant data through the confirmed date, and remains reversible when supported by Paddle. No schema or migration change was required.

2026-08-01 - Organization Account Sign-In Routing

Public and onboarding SaaS sign-in previously sent an activated, commercially allowed organization customer directly to operational `/login`. Authentication, already-authenticated restoration, and social sign-in now use one customer journey decision that lands authorized organization account managers on `/saas/account`. The Organization Account Overview presents only permitted organization, branch, billing, and security sections, and Enter TIS Platform is the sole explicit operational entry action.

Incomplete onboarding and pending demo review still resume their authoritative steps. Restricted or suspended account managers remain in account billing and recovery context; operational users without account-management permission keep their role-based destination. Multiple managed organizations require selection before an overview is rendered. The HTTP-only selected-organization hint is revalidated against current account links and permissions before the existing entitlement resolver accepts its tenant scope. Existing commercial access, permissions, subscription authority, operational authentication, schema, and tenant isolation remain unchanged.

2026-07-31 - Contract-Linked Commercial Access Consistency

Operational access previously selected the newest PaymentSubscription for the onboarding organization, while Subscription Management and entitlements used the tenant link and authoritative SubscriptionContract. A pending plan upgrade or newer stale subscription row could therefore block an otherwise active paid workspace and every blocked paid state was presented as expired.

Operational login, protected requests, returning-customer routing, and access pages now consume one contract-linked commercial access projection over the existing entitlement resolver. The current active/trialing plan remains

authoritative while a plan change is pending, failed, incomplete, canceled, or abandoned. Canceled subscriptions remain entitled only through the confirmed paid period end. Restricted states use distinct payment-processing, past-due, paused, expired, suspended, archived, and verification-required guidance.

The specialized plan-change subscription webhook now applies the same provider status normalization as the general and quantity-change paths. It does not complete or activate the target plan without the existing two provider signals. No schema, migration, pricing, quantity, Paddle request, webhook authority, or tenant data changed.

2026-07-30 - Initial Secure Payment Retry And Lineage Hardening

The Secure Payment summary could be correct while launch rejected an otherwise eligible unpaid organization whose persisted billing status or checkout lineage was stale. Plan and interval changes marked the checkout session stale but did not consistently supersede its unfinished payment attempt. Retry could therefore repeat a local readiness failure or retain obsolete transaction authority.

The incident itself occurred before any Paddle API request: `_ensure_checkout_launchable()` rejected a legacy `ready_for_checkout` pre-checkout billing state. That state can now be prepared safely. The Professional Annual mapping was unchanged at USD 790 per active branch, so the verified two-branch quote remains quantity 2 and USD 1,580 annually.

Checkout recovery now treats plan selection, checkout session, payment attempt, quote fingerprint, provider price, interval, and quantity as one lineage. Obsolete sessions and unfinished attempts are superseded, current local authority is cleared, and Retry prepares a fresh session from the authoritative quote. Existing started transactions are reused only after remote billed, automatic-collection, customer, quote, item, quantity, and subtotal validation. Superseded webhooks cannot activate a subscription or workspace.

Paddle address resolution now reuses an active exact-country address for the same customer before creating one. Provider and readiness failures retain status/code and traceback in server logs while the customer sees one safe retry alert. No pricing, capacity, schema, migration, webhook authority, provisioning, or checkout architecture changed.

2026-07-29 - Organization Profile Save And Account Setup Correction

The Organization Profile POST accepted its program values correctly but used MIME/extension-only logo checks and wrote directly to the application filesystem. Its route caught only `ValueError`; an `OSError` while creating or writing `static/uploads/saas/pending_logos` escaped as raw HTTP 500. Corrupt and oversized files could also be accepted. The fresh Account Setup page repeated the same next-step guidance across the status, side action, content, and help panels.

Pending logos now reuse actual-image decoding, the established 4 MB and minimum-dimension checks, UUID filenames, pending-directory confinement, and atomic temporary-file promotion. Empty uploads retain the current reference. Rollback removes a newly written file; successful replacement removes the old file after commit. Validation, storage, and unexpected failures all render a safe page response with preserved entered values, while technical failures receive traceback logging.

The initial account state now shows a smaller official logo, one "Start Your School Workspace Setup" title and sentence, the existing eight-step progress track, and one POST start action. Later setup-console states remain unchanged. No schema, migration, billing, payment, provisioning lifecycle, operational-module, or landing-site behavior changed. Provisioning's existing logo-copy step was hardened without changing activation authority.

The current storage location remains application-local `static/uploads/saas/pending_logos`. Render durability requires a separately approved persistent-disk or object-storage decision.

Organization branding now appears in the Organization Profile preview and shared School Workspace setup identity without replacing the official TIS logo. Existing paid/demo provisioning continues to promote the pending image into the primary `SchoolGroupLogo` slot; source resolution is now confined to the pending directory, a missing referenced file blocks activation rather than silently losing branding, and the final accessible label includes the organization name. The

existing protected organization-asset route and operational shell remain the final display path.

Branch Setup previously asked Jinja to sum `estimated_system_users` and `estimated_teachers` directly. Placeholder and incomplete rows supplied undefined or `None`, producing `TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'`. Totals now come from a Python helper that normalizes display-only missing values to zero. Required server validation remains active for customer saves, and newly created pending branches receive explicit zero defaults.

2026-07-29 - Post-Verification Sign-In 405 Correction

A fresh verified account had no pending organization or operational account link, so `customer_journey_service.login_destination()` returned `/saas/onboarding/start`. That route accepts POST only. After successful credential POST, the 302 caused the browser to request that destination by GET, and FastAPI returned raw `{"detail":"Method Not Allowed"}` with HTTP 405. Existing tests supplied `/saas/account` explicitly and therefore masked the normal browser journey.

Fresh verified accounts now continue to the GET `/saas/account` dashboard; the existing start-setup action remains the only POST that creates an organization. Login continuation validation accepts only known customer GET destinations and rejects POST-only, malformed, traversal, fragment, and external values. A compatibility GET for `/saas/auth/login` redirects to the normal GET sign-in page. Verification, password authentication, onboarding rules, subscription intent, preferred-plan authority, checkout, Paddle, schema, and landing behavior are unchanged.

2026-07-29 - Safe Public Subscription Pricing Entry

The landing hero Subscribe Now action now scrolls to pricing. Starter, Professional, and Enterprise AI use identical CTA styling and enter the public TIS Account signup route with distinct allowlisted preferred-plan codes. Pricing cards no longer show the small description beneath each plan name. Custom remains contact-only.

The signup GET route previously referenced an undefined `next_path` local, causing every unauthenticated subscription pricing link to return HTTP 500 before account or onboarding state was evaluated. The route now accepts and safely normalizes that optional value. A self-service plan preference is preserved through registration in a secure cookie, creates no plan selection or payment record, and is applied only when the existing organization-wide branch, system-user, and teacher checks confirm eligibility. Invalid, inactive, or undersized preferences are cleared. No price, plan limit, Paddle quantity, payment authority, or AI entitlement changed.

2026-07-29 - Per-Branch System-User And Teacher Subscription Capacity

Branch Setup now stores required non-negative estimates for system users and teachers on every active branch and presents live organization-wide totals. Migration `20260729_001_subscription_capacity_dimensions` preserves legacy organization estimates by assigning them to the primary active branch only when no branch estimates exist. Derived organization totals remain compatibility summaries rather than competing authority.

Eligibility checks branches, non-teacher system users, and teacher records independently. Starter supports 1/5/25, Professional 5/20/100, and Enterprise AI 25/100/500. Before payment, each people dimension uses the greater of branch estimates and actual same-workspace active data; after activation actual data is authoritative. The minimum eligible plan is recommended, higher eligible plans remain available, and exceeding any Enterprise limit selects the contact-only Custom path.

All quote and checkout fingerprints include both people counts. Capacity changes supersede stale checkout lineage and clear only an undersized plan. Paid non-teacher user creation/reactivation, teacher creation and year-copy preflight, and plan downgrades fail before mutation when capacity would be exceeded.

2026-07-28 - Authoritative Subscription Plan Branch Capacity

Self-service plan eligibility now uses each active plan's persisted `max_branches`: Starter supports one active billable branch, Professional five, and Enterprise AI twenty-five. Pricing remains the selected plan's per-branch price multiplied by the actual authoritative active count.

Plan selection, quote construction, checkout preparation and launch, and payment validation fail closed when capacity is exceeded. Pre-payment branch expansion clears an undersized selection and supersedes its quote and checkout lineage; an eligible higher plan remains selected. Organizations above the Enterprise AI limit receive a customer-safe custom-plan contact state.

2026-07-28 - Pre-Payment Branch Editing And Checkout Supersession

Branch Setup previously rejected any organization with a `TenantProvisioningLink`, incorrectly treating an unpaid demo or prepared workspace as paid provisioning. Editing now closes only on authoritative confirmed-payment or active-paid-subscription evidence. Before that boundary, customers may add, edit, remove, reorder, and reprioritize branches.

Changing branch count or identity stales the prepared checkout, supersedes its incomplete payment attempt, clears quote snapshots, and recalculates the next checkout from the final active count. Late provider events from a superseded transaction are recorded for manual review without activating or converting the workspace or disrupting the replacement checkout.

2026-07-28 - Fixed Paddle Checkout Branch Quantity

Paddle checkout exposed quantity controls even when the server-created transaction contained the TIS billable branch count; inline presentation alone does not make transaction items immutable. TIS now supplies the resolved Paddle customer address, requires an automatically collected transaction to reach ready state, validates its quote evidence, and marks it billed before releasing its transaction ID to Paddle.js.

Billed transaction items cannot be changed. The same catalog price remains usable with organization-specific quantities; no price mutation or quantity-specific price is created. Automatically collected payment completion remains the recurring-subscription, webhook, and demo-conversion authority. The public payment launcher now explicitly treats `billed` as the only valid remote launch state and verifies the local attempt, checkout, customer, and quote context before supplying `transactionId` to Paddle.js. Invalid states fail closed without exposing provider diagnostics.

2026-07-28 - Expired-Demo Checkout Identity Resolution

Expired-demo Paddle checkout could find one active customer by email but reject it when provider custom data retained a deleted SaaS account context and the existing operational tenant owner was treated as an unrelated identity. The checkout guard now accepts or repairs a stale SaaS account-user relationship only when the authenticated account, owned organization, demo source, tenant-provisioning link, SchoolGroup, and sole active operational owner form one coherent relationship. Active previous accounts, unrelated tenants, multiple identities, and live-mode email-only recovery remain blocked.

Customer responses no longer expose identity match counts or internal reason codes, and a failed preparation displays retry/support guidance rather than a ready-payment state. No workspace, branch, tenant, payment authority, pricing, or provider-environment rule changed.

2026-07-28 - Returning Customer Journey And Expired Access

Expired demos previously entered the paid subscription portal, whose model requires paid entitlement evidence, so normal demo values rendered unavailable and no conversion action existed. Some returning sessions could also reach workspace setup before expired paid state was intercepted.

TIS now builds demo subscription choice from the existing organization, workspace, active operational branches, public plans, and intervals, then hands selection to existing Paddle checkout and same-workspace conversion. SaaS and

operational login route from authoritative state; a shared commercial guard blocks expected demo or paid expiry before protected page work. Customer communications use TIS-team terminology. Landing navigation adds Sign In and Open TIS App. All M8B9 owner controls remain available under progressive disclosure. No schema, migration, pricing, payment-confirmation, AI-entitlement, workspace replacement, commit, or push change was made.

2026-07-27 - M8B9 Demo Operations, Notifications, And Testing

Added owner-only immediate expiry, same-workspace reactivation, unbounded future custom expiry, rotating final-day reminders, and manual single/global lifecycle execution through existing production rules. Added Standard, Full, and Custom controlled access policies with workspace defaults and isolated branch overrides, durable audits, and customer communications without usage resets, classification changes, or customer-specific rules.

2026-07-27 - Pre-Deploy Database Migration Boundary

Replaced the temporary Render daemon migration worker and HTTP readiness gate with a strict deployment boundary. The FastAPI web process no longer creates tables or runs migrations during import, startup, middleware, or background execution. `python scripts/run_migrations.py` now owns baseline metadata creation and the authoritative ordered migration ledger as Render's required Pre-Deploy Command. A failed migration exits nonzero and prevents activation of the new web version; a successful or already-current run exits zero and logs the applied migration identifiers.

The migration process now configures PostgreSQL `connect_timeout=10s`, `lock_timeout=5s`, and `statement_timeout=30s` at connection creation, before `metadata.create_all()` or any migration-ledger DDL. Flushed progress records bracket connection, metadata, ledger, migration apply, marker, and commit phases. Lock or statement timeout failures therefore identify their boundary and exit nonzero instead of waiting for the deployment timeout. Transactional migration helpers now inspect tables, columns, indexes, and constraints through the active migration connection. They never check out a second engine connection while the first connection owns uncommitted DDL locks, preventing the M8B7 `system_notifications` inspection self-deadlock.

2026-07-27 - M8B8 AI Entitlements And Commercial Foundation

Added one authoritative AI feature registry and entitlement service. Decisions compose tenant scope, role permission, commercial state, feature policy, plan entitlement, and usage without replacing any existing authority. Customer Demo gets two successful uses per feature; internal, demo, and paid usage is persisted separately through locked successful/reserved counters and idempotent operation events. Enterprise AI is the only temporarily mapped paid tier. No AI tool, pricing change, notification expansion, or M8B9 operational control was added.

2026-07-27 - Render Port-Bind Startup Boundary

Render previously executed SQLAlchemy table creation and every pending migration while importing `main:app`. The initial mitigation moved that work to a daemon worker behind an HTTP readiness gate. That temporary boundary has been superseded by the Pre-Deploy Database Migration Boundary above.

2026-07-27 - M8B7 Demo Customer Journey

Platform Owner approval now orchestrates the existing independently retryable provisioning service. Durable branded email intents cover request receipt, activation approval, decline, Day 6, expiry, and same-workspace subscription continuation. Demo events reuse the Platform Owner Notification Center, and active tenant workspaces show a responsive demo indicator through the shared shell. Coherent expired demos may convert after authoritative confirmed payment by reactivating and converting the same SchoolGroup and tenant relationship. M8B8, M8B9, pricing redesign, second-workspace provisioning, and sandbox conversion remain out of scope.

This file is the chronological summary of meaningful TIS changes. It does not replace module history under `docs/history/`; it gives reviewers, developers, Codex, and ChatGPT a fast timeline of what changed and why.

Newest entries should be added first.

2026-07-26 - Platform Owner Historical Demo Eligibility Maintenance

Area/module: Platform Owner SaaS Admin and Customer Demo domain eligibility

Previous state: The organization-scoped clean-room reset safely removed current test data, but a historical detached eligibility reservation could no longer be reached after its organization and account had already been deleted.

New state: Platform Owners have a separate maintenance page that scans demo-domain eligibility reservations and shows exact blockers. A row is removable only when no matching organization, account, demo request, tenant-profile workspace, provisioning record, subscription evidence, conversion, or manual-review evidence exists. Confirmed deletion rechecks and locks one exact ID, deletes only that ID, flushes, verifies absence, commits atomically, and writes a durable owner audit event.

Reason: Repair verified historical orphaned reservations without weakening production one-demo-per-domain enforcement or changing the existing clean-room reset.

Files changed: New demo eligibility maintenance service and templates, SaaS Admin routes/navigation, focused Phase 5 tests, KMS source documents, and regenerated KMS artifacts.

Documentation updated: AI/master context, project state, change history, SaaS onboarding history, module map, system flows, and database architecture overview.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: No schema, foreign-key, migration, demo-request, clean-room reset, customer workflow, commit, or push change.

2026-07-26 - Verified Safe Demo-Domain Reservation Deletion

Area/module: Platform Owner test workspace/account reset and Customer Demo domain eligibility

Previous state: Reset analysis identified safe detached eligibility rows, but deletion depended on a second ORM lookup and reconstructed ID list and did not immediately verify removal before continuing with parent deletion.

New state: Analysis publishes an explicit safe detached-reservation ID list. The owner-only deletion transaction consumes only that list, deletes by primary key before demo requests and parent records, flushes immediately, and verifies that no selected row remains. Structured logs record received IDs, query scope, affected rows, verification count, and the existing transaction commit or rollback.

Reason: Make clean-room reset completion provable and fail closed while preserving conflict/manual-review protections and the production one-demo-per-domain policy.

Files changed: Workspace analysis/deletion services, focused Phase 5 and domain diagnostics tests, read-only diagnostic tooling, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: No schema, foreign-key, unique-constraint, production eligibility, customer-message, commit, or push change.

2026-07-26 - Safe Orphaned Demo-Domain Reservation Cleanup

Area/module: Platform Owner test workspace/account reset and Customer Demo domain eligibility

Previous state: The clean-room reset removed eligibility rows only while they still referenced a selected demo request. A prior request deletion could leave a detached same-domain row through `ON DELETE SET NULL`, and that row continued to block a later internal demo request.

New state: The owner-only reset resolves the selected organization's domain through the same Customer Demo domain resolver, counts linked and detached reservations, and checks for another organization, request, workspace, or customer account using that domain. It removes a detached reservation only when that conflict analysis is empty and the row has no historical manual-review evidence; otherwise, reset preflight blocks for manual review and preserves all data. Owner analysis views show the domain-cleanup result.

Reason: Allow repeatable internal M8 testing without broadly deleting domain reservations or weakening the production one-demo-per-domain policy.

Files changed: Workspace analysis/deletion services, owner reset-analysis templates, focused Phase 5 regression tests, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: No schema, foreign-key, cascade, customer demo-policy, payment, Paddle, commit, or push change.

2026-07-26 - Internal Test Workspace Commercial Clean-Room Reset

Area/module: Platform Owner test workspace/account reset, M8 demo-commercial history, and customer-facing demo wording

Previous state: The guarded reset removed the selected workspace/account data but retained the selected organization's demo request and normalized-domain reservation. A later internal registration for the same organization domain was therefore blocked by the production one-demo policy.

New state: Within the existing owner-only reset transaction, TIS deletes only the selected pending organization's linked demo request, domain reservation, review/event history, demo provisioning/lifecycle records, and demo-to-paid conversion history before their parent records. The same internal test email and organization domain can complete a new M8 journey. Customer-facing demo review language now refers to the TIS team.

Reason: Support repeatable internal end-to-end M8 testing without weakening the production Customer Demo one-domain restriction.

Files changed: Workspace analysis/deletion services, customer demo wording, focused Phase 5 regression tests, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: Owner-only test reset exception. No schema, foreign-key, cascade, payment, Paddle, normal customer demo-policy, lifecycle, commit, or push change.

2026-07-26 - Test Workspace Reset Entitlement Dependency Fix

Area/module: Platform Owner test workspace/account reset, workspace entitlements, and scoped cleanup dependencies

Previous state: The controlled reset reached final SchoolGroup deletion while a selected workspace entitlement still referenced that SchoolGroup. Entitlement child values and branch-entitlement rows also required explicit dependency review.

New state: The reset removes selected `workspace_entitlement_values` and `branch_entitlements`, then the selected `workspace_entitlements` rows, before the final SchoolGroup deletion. Every query is scoped to the selected SchoolGroup or its selected entitlement IDs. Global entitlement definitions, subscription plans, prices, and unrelated workspaces remain unchanged.

Reason: Preserve entitlement foreign-key integrity while retaining the existing Platform Owner-only test reset guards, transaction rollback, and customer behavior.

Files changed: Workspace analysis/deletion services, focused Phase 5 regression tests, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: Minimal scoped dependency fix only. No foreign-key, cascade, entitlement-rule, schema, lifecycle, customer-facing, commit, or push change.

2026-07-26 - Test Workspace Reset Subscription-Change Dependency Fix

Area/module: Platform Owner test workspace/account reset, subscription-change records, and workspace-deletion diagnostics

Previous state: The controlled test workspace reset deleted scoped operational users before deleting scoped subscription-change requests. A request referencing a workspace user could therefore cause the database to reject the user deletion and roll the transaction back.

New state: The reset deletes `subscription_change_requests` for the selected `school_group_id` before any selected workspace user is deleted. Pre-analysis now counts the same scoped records, existing structured diagnostics retain their model/table/row-count events, and all deletion work remains inside the existing transaction.

Reason: Preserve foreign-key integrity while allowing Platform Owners to reset only the selected internal test workspace/account and retain all other workspaces unchanged.

Files changed: Workspace analysis/deletion services, focused Phase 5 regression tests, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: Minimal dependency-order fix only. No foreign-key, cascade, schema, lifecycle, customer-facing, commit, or push change.

2026-07-25 - Landing CTA Consolidation And Customer Demo Domain Policy

Area/module: Public landing conversion routes, TIS Account signup/onboarding, customer-demo request policy, and Platform Owner demo visibility

Previous state: The landing website exposed overlapping conversion labels. Signup did not retain a selected commercial path, and demo duplicate protection applied only to a pending request for one onboarding record rather than an organization domain.

New state: The landing website exposes Request a Demo and Subscribe Now through environment-configured app URLs with explicit `demo` or `subscribe` intent. The selected intent persists through signup and School Workspace Setup, then is emphasized at commercial choice without preventing a change. Customer Demo eligibility now uses a normalized organization-domain reservation with a database unique invariant. Historical duplicate domains are retained as manual-review reservations; no historical request, workspace, or customer data is merged, deleted, migrated, or reprovisioned.

Reason: Schools should have two unambiguous public conversion paths, while each organization receives at most one Customer Demo opportunity and retains the same workspace for Demo-to-Paid.

Files changed: Landing CTA source, SaaS models/services/router/templates, database migration, diagnostic command, focused tests, and KMS sources.

Documentation updated: AI/master context, project state, user/system flows, database architecture, change history, landing history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: Approved M8 scope only. No M9 work, payment change, workspace conversion change, provisioning redesign, commit, or push.

Entry Template

```
## YYYY-MM-DD - Short Change Title

Area/module:
Previous state:
New state:
Reason:
Files changed:
Documentation updated:
PDF regenerated:
AI project context updated:
Reviewer/approval notes:
```

2026-07-25 - M8 Landing Integration Open Account Entry Points

Area/module: Next.js public landing website, public-to-account routing, and deployment configuration

Previous state: The public landing website explained TIS and supported demo/early-access conversion, but it did not expose a final M8 Open Account entry point into the deployed TIS Account signup journey.

New state: The Next.js landing website now has Open Account entry points in the navigation, hero CTA area, and final CTA area. All Open Account links use one shared URL derived from `NEXT_PUBLIC_TIS_APP_BASE_URL` and the existing `/saas/signup` account registration path.

Reason: M8A and M8B-1 through M8B-6 are already completed in the SaaS application. The final M8 integration step is to let visitors on `tisplatform.com` enter the deployed customer account setup flow without coupling the landing project to the FastAPI app.

Files changed:

1 `tis-landing-website/src/app/page.tsx`

1 `tis-landing-website/.env.example`

1 `tis-landing-website/README.md`

1 `.kms-impact.yml`

1 `docs/AI_PROJECT_CONTEXT.md`

```
1     docs/TIS_MASTER_CONTEXT.md
1     docs/PROJECT_STATE.md
1     docs/CHANGE_HISTORY.md
1     docs/history/landing-page/README.md
1     docs/history/landing-page/2026-07-25-m8-landing-integration-open-account.md
```

Documentation updated: AI/master context, project state, change history, and landing-page history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: M8 landing integration only. No FastAPI SaaS application, authentication, onboarding backend, Paddle, database, API, demo workflow, provisioning, commercial state, commit, push, or M9 work.

2026-07-23 - M8B-6 Demo-To-Paid Workspace Conversion

Area/module: Customer-demo checkout, provider-confirmed subscription reconciliation, workspace classification, commercial entitlements, demo lifecycle, customer status, and Platform Owner inspection

Previous state: An activated Customer Demo could not become Customer Paid without risking a second provisioning path or leaving paid commercial evidence disconnected from the existing operational workspace.

New state: An eligible active Customer Demo can use the existing M7 subscription checkout. After provider-confirmed payment, TIS atomically converts the same SchoolGroup and tenant link to Customer Paid, replaces the demo entitlement with the confirmed subscription-backed paid entitlement, preserves all operational data and history, and exits demo lifecycle processing. Durable conversion state and events support idempotency, audit, and retry after failure.

Reason: Customers must retain one workspace, tenant identity, organization, branches, users, permissions, and academic history when moving from evaluation to a paid subscription.

Files changed: Conversion enums/models/migration/service, classification transition validation, demo checkout and lifecycle integration, payment-webhook reconciliation, customer/owner UI, tests, ADR, and KMS.

Documentation updated: AI/master context, project state, database architecture, module map, flows, roadmap, ADR index/0013, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: M8B-6 only. No tenant reprovisioning, Paddle API redesign, pricing change, demo extension, manual conversion override, internal-sandbox conversion, archive/delete, membership, ownership transfer, commit, or push.

2026-07-23 - M8B-5 Standard Customer Demo Lifecycle

Area/module: Customer-demo lifecycle metadata, resolver, scheduled processing, internal notifications, expiration, operational access enforcement, and owner/customer UI

Previous state: Activated customer-demo workspaces had no standard duration, reminder, expiration transition, or request-time access gate.

New state: Customer demos last exactly seven days from activation. Day 6 creates idempotent internal customer and Platform Owner notifications. Day 7 processing atomically ends the demo entitlement and suspends the workspace while preserving tenant data. Operational middleware blocks expired or ambiguous demos across web, API, download, and existing-session requests.

Reason: Customer demos require a predictable, provider-independent lifecycle that cannot be extended by scheduler delay or stale sessions and never destroys evaluation data.

Files changed: Demo lifecycle enums/models/migration/service/command, M8B-4 activation metadata, authorization middleware, operational and SaaS UI, lifecycle tests, ADR, and KMS.

Documentation updated: AI/master context, project state, database architecture, module map, flows, roadmap, ADR index/0012, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: M8B-5 only. No demo-to-paid conversion, Paddle change, email, manual extension, archive/delete, read-only expired mode, special migration, membership, ownership transfer, commit, or push.

2026-07-23 - M8B-4 Demo Workspace Provisioning And Activation

Area/module: Approved SaaS demo requests, operational workspace provisioning, commercial entitlement activation, Platform Console, and customer status

Previous state: Platform Owner approval created review evidence only. No supported path could create or activate a customer-demo workspace without entering the paid subscription lifecycle.

New state: Platform Owners can provision an approved, coherent customer-demo request. TIS reuses the shared operational workspace builder, creates an explicit demo entitlement and demo-sourced tenant link, activates the workspace atomically, records provisioning and activation events, and prevents duplicate provisioning. Failures roll back workspace changes while preserving the Approved request and retryable failure details.

Reason: Approved demos require operational access without fabricating Paddle, payment, or subscription-contract evidence.

Files changed: Demo provisioning enums, models, migration, service, shared provisioning builder, owner/customer routes and templates, lifecycle display, tests, ADR, and KMS.

Documentation updated: AI context, master context, project state, database architecture, module map, flows, roadmap, ADR, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: M8B-4 only. No expiration, reminder, scheduler, login blocking, conversion, suspension workflow, billing/Paddle change, membership, Al-Andalus migration, commit, or push.

2026-07-22 - M8B-3 Demo Request Workflow

Area/module: SaaS onboarding commercial choice, demo request lifecycle, Platform Owner review, audit, and internal notifications

Previous state: Completed onboarding continued directly to subscription plan selection. TIS had no SaaS-owned review lifecycle for a customer demo request.

New state: Customers can choose Request Demo or Subscribe Now after onboarding. Demo submission records validated commercial context and starts in Pending Review. Customers can inspect and withdraw pending requests. Platform Owners can search/filter/sort requests and approve, reject with a reason, or cancel. Every action creates durable audit and internal-notification events. Approval records review only and does not provision or activate a workspace.

Reason: Demo requests need a safe, auditable review boundary before separately approved provisioning work begins.

Files changed:

1. SaaS demo enums/models/migration/service, customer and owner routes/templates, Platform Console navigation, and focused regression tests

Documentation updated:

1. AI context, master context, project state, database architecture, module map, workflows, roadmap, ADR 0010, and SaaS onboarding history

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: M8B-3 only. No demo provisioning, activation, expiration, email delivery, Paddle change, entitlement enforcement, role change, conversion, membership, schema change outside the new review aggregate, commit, or push.

2026-07-22 - M8B-2 Commercial State And Entitlement Foundation

Area/module: Workspace commercial state, entitlement modeling, branch entitlement resolution, database architecture, and Platform Console

Previous state: M8B-1 stored workspace classification and lifecycle metadata but had no commercial decision engine, effective workspace entitlement, or branch-level commercial entitlement model.

New state: Added normalized workspace entitlement envelopes, typed entitlement values tied to the existing catalog, and optional branch inherit/active/inactive records. Added separated read-only validation, workspace entitlement, branch entitlement, and commercial-state services. Paid resolution reuses the confirmed M7 subscription entitlement authority. Platform Owners can inspect effective results without mutation controls.

Reason: Future demo and paid workflows need one conservative commercial decision foundation before any customer access or lifecycle behavior is changed.

Files changed:

1. commercial entitlement enums, SaaS models/migration, four resolver/validation services, Platform Console context/template, and focused regression tests

Documentation updated:

1. AI context, master context, project state, database architecture, module map, workflows, roadmap, ADR 0009, and workspace-classification history

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: No M8B-3 workflow, enforcement, demo expiration, Paddle change, customer onboarding change, conversion, membership, role, or Al-Andalus migration is included.

2026-07-22 - M8B-1 Workspace Classification Foundation

Area/module: Workspace identity, SaaS onboarding metadata, provisioning metadata, Platform Console, and database migration tooling

Previous state: Operational SchoolGroup records had no stable workspace UUID, classification, or dedicated lifecycle metadata. Pending organizations and identities could not express workspace/test intent independently of onboarding, payment, and tenant state.

New state: Added constrained and indexed workspace UUID/classification/lifecycle fields, onboarding workspace intent, SaaS account purpose, and internal-test operational identity metadata. Added centralized validation and conversion rejection, a commercial-state skeleton with no resolution logic, read-only relationship diagnostics, and a dry-run/default transactional idempotent backfill for all confirmed pre-M8B-1 test records. Platform Owners can inspect the metadata read-only. Provisioning only carries intent into metadata and moves the metadata lifecycle from provisioning to active; existing business gates remain unchanged.

Reason: Establish the durable classification boundary required by later M8B packages without changing current customer, billing, entitlement, permission, or tenant behavior.

Files changed:

1 models, migration, workspace classification services, diagnostic/backfill scripts, provisioning metadata assignment, Platform Console template/context, and focused tests

Documentation updated:

1 AI context, master context, project state, database architecture, module map, user/system flows, roadmap, ADR 0008, and workspace-classification history

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: M8B-2 and all later demo/commercial/conversion workflows remain out of scope.

2026-07-22 - Corrected Platform Owner Pending Organization Lifecycle Views

Area/module: SaaS onboarding, provisioning lifecycle, and Platform Owner administration

Previous state: Platform Console counted every `PendingOrganization` row, and the Pending Organizations page listed every row regardless of completed checkout, active subscription, tenant link, or completed provisioning. Historical `ready_for_checkout` values could therefore make active tenants appear pending, while conflicting completed-provisioning records displayed raw internal statuses without a clear review state.

New state: One shared lifecycle-aware query now defines the pending queue as draft/setup, review, checkout/payment, or incomplete/recoverable activation work with no tenant link, completed provisioning job, or final tenant billing state. A shared owner lifecycle projection reconciles payment, subscription, contract, tenant-link, provisioning-job, and active SchoolGroup evidence. Coherent active tenants move to retained Organization Records; conflicting completed evidence is labeled Lifecycle Review Required. Platform Console counts/actions and the owner table use the same rule and readable labels.

Reason: The Platform Owner work queue must represent actionable unfinished onboarding rather than historical database rows, without mutating payment, provisioning, or onboarding evidence.

Files changed:

1 centralized SaaS owner lifecycle query and projection

1 Platform Console summary and owner admin routes

1 pending organization list/detail templates

1 focused lifecycle, filtering, count, and access tests

1 authoritative KMS context, module map, workflow, project state, and provisioning history

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: No schema, migration, payment, checkout, provisioning transition, permission, production data, commit, or push change. Historical records remain directly accessible.

2026-07-22 - Added Phase 7D Navigation And Catalog Enforcement

Area/module: KMS governance, navigation, catalog, and generated-artifact validation

Previous state: KMS checks enforced KIA, source coverage, deterministic source ordering, normalized hashes, freshness, PDF identity, and positive increasing `pdf_page` values. They did not require usable titles, constrain catalog taxonomy, validate the KMS navigation guide's targets, or prove that source pages existed within the generated PDF.

New state: `scripts/kms.py check` now uses a shared approved catalog to validate every included Markdown title, category, and module; requires the normalized manifest inventory to exactly match the generator list; validates every KMS Navigation link as an existing listed Markdown source inside `docs/`; and rejects missing, non-positive, non-integer, non-increasing, or out-of-range PDF pages. Existing KIA, hashes, freshness, ordering, coverage, and artifact checks are unchanged.

Reason: Phase 7A-7C navigation and catalog conventions must be enforceable through the same local and CI gate that protects KMS synchronization.

Files changed:

1 shared KMS catalog vocabulary and path classifier

1 ReportLab generator/read-only validation

1 focused KMS automation tests

1 KMS navigation, policy, automation, repository architecture, project state, and module history

1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; first-read product, architecture, workflow, and critical-rule context is unchanged.

Reviewer/approval notes: Phase 7D only. Knowledge Center UI/routes, application behavior, database, dependencies, `tis.db`, commits, and pushes remain unchanged.

2026-07-22 - Added Phase 7C Knowledge Center Navigation

Area/module: Platform Knowledge Center and KMS navigation

Previous state: The owner-only Knowledge Center showed manifest sources in one path-focused table. It had no document search, category/module/freshness filters, logical source groups, descriptive summaries, or links to the source document's page in the protected booklet. ADRs were listed by ascending filename, and module-history areas were not ordered by recent activity.

New state: The Knowledge Center enriches manifest-listed sources with Markdown title and summary metadata, groups them into Core, Engineering, Decisions, History, Marketing, and Supporting sections, and provides client-side search plus category, module, and freshness filters. Document and activity links open the existing owner-protected booklet route at the manifest `pdf_page`. ADRs are newest-first, and module-history areas are ordered by their latest dated entry with entry counts.

Reason: Platform owners need to locate and consume authoritative knowledge quickly without introducing a database, search service, new route, dependency, or public documentation link.

Files changed:

1 `knowledge_service.py` manifest presentation metadata and activity ordering

1 `templates/platform_knowledge_center.html` grouped library, client-side search/filters, and protected deep links

1 focused Knowledge Center service tests

1 Platform Knowledge KMS source and module-history documents

1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 7C only. Owner access checks, existing routes, application data, KMS source authority, and generator enforcement remain unchanged. No regenerate control was added.

2026-07-21 - Added Phase 7B Professional PDF Navigation

Area/module: KMS generated booklet and manifest

Previous state: The booklet was a linear concatenation of source documents with page footers and a source-path list. It had no page-numbered table of contents, source destinations, outline hierarchy, or manifest mapping from Markdown sources to PDF pages.

New state: The ReportLab generator performs a multi-pass build with a "How to Use This Handbook" page, a real table of contents, stable source-document bookmarks, child bookmarks for H2 major headings, and deterministic named destinations. Every manifest source record includes its starting `pdf_page`, and freshness validation requires positive, strictly increasing page values.

Reason: The engineering handbook must be practical to navigate as a long-form reference while Markdown remains authoritative and generation stays dependency-light.

Files changed:

1 ReportLab PDF generator and focused automation tests

1 PDF navigation documentation and engineering-handbook history

1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; product architecture, current engineering guardrails, and onboarding order are unchanged.

Reviewer/approval notes: No Knowledge Center UI, route, database, dependency, application behavior, or source-document ordering change was introduced.

2026-07-21 - Added Phase 7A KMS Navigation Foundation

Area/module: KMS information architecture and developer onboarding

Previous state: The KMS had complete root and engineering indexes, but they were long manually maintained file lists. Readers had to determine their own document set from 53 Markdown sources, and three supporting documents lacked normalized title metadata.

New state: `docs/KMS_NAVIGATION.md` provides focused reading paths for new humans, new AI conversations, SaaS onboarding, subscriptions, operational modules, database work, Platform Owner tools, landing work, location data, design, decisions, and review/KIA. Root and engineering indexes now use real Markdown links and delegate task selection to the navigation guide. Missing document titles were normalized.

Reason: Readers should reach relevant source material quickly without changing the established Markdown, manifest, PDF, or Knowledge Center architecture.

Files changed:

1 KMS navigation guide and documentation indexes

1 title metadata for three supporting documents

1 fixed booklet source list entry for the new authoritative guide

1 project state, change history, module history, PDF, and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; its existing first-read role and product/architecture guidance remain current.

Reviewer/approval notes: Phase 7B catalog/PDF navigation, Phase 7C Knowledge Center changes, and Phase 7D enforcement enhancements were not implemented.

2026-07-21 - Added Unified Phase 6 KMS Commands

Area/module: KMS developer workflow, local automation, CI, and deployment validation

Previous state: Developers separately ran the PDF generator, artifact freshness check, and KIA impact checker. The repository had all required primitives but no single synchronization command or canonical read-only command shared by local work and CI.

New state: `scripts/kms.py sync` validates the task KIA before writing, regenerates the PDF and manifest through the existing generator, runs complete post-generation validation, and prints a concise summary. `scripts/kms.py check` delegates to the existing read-only enforcement logic. GitHub pull-request, `dev`, and deployment gates use the unified check command.

Reason: One reliable command reduces missed mechanical steps while preserving reviewed Markdown as the source of truth and keeping enforcement strict.

Files changed:

1 KMS command orchestrator and reusable checker API

1 KMS automation tests

1 repository instructions and CI workflows

1 KMS workflow documentation and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: The command never rewrites authoritative Markdown and does not change application behavior or production data.

2026-07-21 - Aligned Push Enforcement With KIA Task Boundaries

Area/module: GitHub Actions and KMS impact validation

Previous state: Pull-request enforcement validated the full feature branch against its base, while push enforcement validated only `github.event.before...github.sha`. A follow-up fix commit therefore evaluated a cumulative `.kms-impact.yml` against only the latest commit and incorrectly reported previously updated KMS files as unchanged.

New state: Pull requests validate the pull-request base SHA against the actual pull-request head SHA. Pushes to `dev` find the merge base between the repository default branch and the pushed head, then validate that complete task range. Both events apply the unchanged strict declaration and generated-artifact checks to the same logical implementation boundary.

Reason: KIA declarations describe approved implementation tasks, which may contain multiple commits. Event delivery boundaries must not redefine those tasks.

Files changed:

1 KMS impact checker

1 KMS enforcement workflow

1 KMS automation regression tests

1 KMS governance documentation and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; product architecture, developer onboarding order, and application behavior are unchanged.

Reviewer/approval notes: Enforcement remains strict across the complete task diff. No application behavior or production data changed.

2026-07-21 - Made KMS Enforcement Cross-Platform Deterministic

Area/module: Repository KMS generation, freshness validation, and CI enforcement

Previous state: Markdown source hashes used raw checkout bytes, and dynamically discovered ADR and history sources used native `Path` ordering. A Windows checkout with CRLF line endings could generate a manifest that passed locally but failed on GitHub Linux, where the same committed text used LF and path ordering differed.

New state: Markdown is decoded as UTF-8, normalized to LF, and then hashed. Source paths are normalized to repository-relative POSIX paths, dynamic sources use a stable case-insensitive ordering with a deterministic tie-breaker, and source comparison still rejects missing, unexpected, duplicate, or reordered entries. Git diff inspection now includes deleted files.

Reason: KMS enforcement must evaluate committed content consistently across developer workstations and GitHub Actions without weakening freshness or source-coverage checks.

Files changed:

1 KMS generator and impact checker

1 Knowledge Center freshness hashing helper

- KMS automation tests and line-ending attributes
- generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; onboarding, architecture, product behavior, and current priorities are unchanged.

Reviewer/approval notes: Repository-governance correction only. No application behavior, production data, SaaS flows, database, or migrations changed.

2026-07-21 - Added Automatic KMS Synchronization Enforcement

Area/module: Repository governance, KMS automation, CI, deployment gate, and AI workflow

Previous state: KMS updates depended on developers and AI assistants remembering the written KIA policy. PDF/manifest generation was manually triggered, the Knowledge Center detected stale hashes only when viewed, and no test, commit, pull-request, or deployment gate blocked stale or missing documentation.

New state: Root `AGENTS.md` makes KMS onboarding mandatory. `.kms-impact.yml` records task-level impact. `scripts/check_kms_impact.py` compares declarations with Git changes, conservatively classifies major paths, validates declared Markdown, and invokes generated-artifact checks. The PDF generator has read-only `--check` mode and manifest PDF hashes. GitHub Actions enforce checks on pull requests and `dev`, and `master` deployment depends on the same gate. Automation never rewrites Markdown.

Reason: Major TIS work must not be mergeable or deployable while engineering knowledge is stale, while reviewed Markdown must remain authoritative and free from runtime/customer data.

Files changed:

- `AGENTS.md`
- `.kms-impact.yml`
- `.github/pull_request_template.md`
- `.github/workflows/kms-enforcement.yml`
- `.github/workflows/deploy-on-master.yml`
- `scripts/check_kms_impact.py`
- `scripts/generate_docs_pdf.py`
- `tests/test_kms_automation.py`
- relevant KMS Markdown and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Repository-governance automation only. No application behavior, production/customer/tenant/billing data, runtime records, database, migrations, or business logic changed.

2026-07-20 - Backfilled Completed M7 Subscription Management

Area/module: SaaS entitlements, subscription portal, quantity/plan changes, cancellation, billing history, invoices, Paddle webhooks, and reconciliation

Previous state: Module history covered M7 Phases 1, 2, 3, and 6 only. Central project state, architecture maps, workflows, roadmap, change history, PDF, and manifest still described the pre-M7 billing foundation; Phases 4 and 5 and reconciliation protections were absent.

New state: KMS records the completed M7 entitlement foundation, read/write customer portal, paid branch quantity management, upgrades and scheduled downgrades, provider-authoritative proration, cancellation/reversal, centralized lifecycle and allowed-action policy, provider billing history, protected invoice downloads, and fail-closed webhook/reconciliation safeguards.

Reason: The engineering handbook must describe current implemented subscription behavior before automatic enforcement becomes authoritative.

Files changed:

- central KMS context/state files
- subscription and payment ADRs
- engineering module, repository, database, flow, and roadmap docs
- subscription module history
- generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Documentation backfill only; it records already committed M7 behavior and introduces no SaaS behavior change.

2026-06-30 - Paddle Initial Checkout Price Mapping Configuration

Area/module: SaaS subscriptions, Paddle initial checkout configuration, tests, and KMS documentation

Previous state: The checkout launch flow safely blocked when the selected subscription plan price did not have `subscription_plan_prices.provider_price_id` configured, but there was no structured mapping sync process and the customer-facing error could expose provider configuration wording.

New state: Added a script-based Paddle price ID sync process using structured sandbox/production mapping examples. The database remains the source of truth through `subscription_plan_prices.provider_price_id`, real mapping files are ignored, and missing provider price IDs now surface a customer-safe Secure Payment support message while internal diagnostics retain plan code, billing interval, and currency details.

Reason: Initial subscription checkout needs environment-specific Paddle provider price IDs without hardcoding live IDs in source or changing payment state behavior.

Files changed:

- `.gitignore`
- `scripts/sync_paddle_price_ids.py`
- `config/paddle/paddle_prices.sandbox.example.json`
- `config/paddle/paddle_prices.production.example.json`
- `saas/payment_service.py`
- `saas/router.py`
- `tests/test_paddle_price_sync.py`

```
tests/test_saas_phase1.py
docs/AI_PROJECT_CONTEXT.md
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/CHANGE_HISTORY.md
docs/history/subscriptions/README.md
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Initial checkout configuration only. No proration, upgrade, downgrade, cancellation, payment state transition, webhook, provisioning behavior, database schema, migration, operational module, landing website, OAuth, internal route rename, live Paddle ID hardcoding, commit, or push was performed.

2026-06-27 - Accepted Subscription And Workspace Activation Guided Journey Phase 3C

Area/module: Subscription Selection, Secure Payment summary, Payment Return/Cancel, Subscription Status, Workspace Activation status, tests, and KMS documentation

Previous state: The subscription, secure payment, billing status, payment return/cancel, and workspace activation pages were functionally correct but still used page-local CTAs, dense status blocks, and inconsistent guidance. Browser return messaging existed but was not part of the shared guided setup experience.

New state: Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages now use the Phase 3A shared setup shell and Phase 3B guided page style. Each page has one shared-shell primary CTA, customer-safe status labels, concise supporting cards, clear browser-return guidance, and explicit messaging that TIS Platform access becomes available after Workspace Activation.

Reason: The accepted Phase 3C scope required making subscription/payment/activation pages feel like one guided customer journey while preserving payment, billing, provisioning, webhook, checkout start/launch, database, and operational behavior.

Files changed:

```
saas/router.py
templates/saas/plan_selection.html
templates/saas/checkout_summary.html
templates/saas/checkout_return.html
templates/saas/checkout_cancel.html
templates/saas/account_billing.html
templates/saas/billing_status.html
tests/test_saas_phase1.py
```

```
docs/AI_PROJECT_CONTEXT.md
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/CHANGE_HISTORY.md
docs/history/saas-onboarding/README.md
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3C customer-facing subscription/payment/status redesign only. No payment behavior change, billing behavior change, provisioning behavior change, webhook logic change, checkout start/launch behavior change, database schema change, migration, operational module change, Next.js landing website change, OAuth change, internal `/saas` route rename, stored-status change, admin view change, commit, or push was performed.

2026-06-27 - Accepted School Workspace Setup Guided Wizard Phase 3B

Area/module: School Workspace Setup onboarding templates, shared SaaS customer shell, onboarding setup context, tests, and KMS documentation

Previous state: The five onboarding pages used functional but dense form layouts with repeated progress notices, mixed card styles, and page-local primary actions. Branch setup felt like repeated blank blocks, organization logo upload was plain, and the review page felt like a basic summary rather than a confident handoff to Subscription Selection.

New state: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use a consistent guided enterprise setup wizard style on top of the Phase 3A shared shell. Each page has one shared-shell primary CTA, secondary Back/Save Draft actions, grouped form sections, concise guidance, cleaner spacing, a more premium logo upload area, compact branch panels, and a stronger review summary.

Reason: The accepted Phase 3B scope required redesigning only the School Workspace Setup onboarding pages while preserving all business logic, routes, field names, validation, draft behavior, payment, billing, provisioning, database, and operational boundaries.

Files changed:

```
saas/router.py
templates/saas/base.html
templates/saas/onboarding_organization.html
templates/saas/onboarding_branches.html
templates/saas/onboarding_academic_setup.html
templates/saas/onboarding_contacts.html
templates/saas/onboarding_review.html
tests/test_saas_phase1.py
docs/AI_PROJECT_CONTEXT.md
```

```
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/CHANGE_HISTORY.md
docs/history/saas-onboarding/README.md
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3B onboarding page redesign only. No backend business logic change, route rename, form field rename, validation change, onboarding progression change, draft behavior change, payment behavior change, billing behavior change, provisioning behavior change, database schema change, migration, operational module change, Next.js landing website change, OAuth change, commit, or push was performed.

2026-06-27 - Accepted TIS Account Guided Setup Framework Phase 3A

Area/module: TIS Account customer dashboard, shared SaaS customer shell, setup journey display helper, and KMS documentation

Previous state: The customer account page still behaved like a dense dashboard with statistics, session details, multiple competing actions, and page-specific journey fragments. The shared customer shell had a logo and onboarding-specific progress UI, but it did not yet provide a reusable 8-step guided setup framework.

New state: The shared customer shell supports a guided setup console for pages that pass setup context. The TIS Account page now presents an official-logo guided console with an 8-step journey stepper, current-step/status area, one primary next action, concise account/workspace context, and guidance that TIS Platform access becomes available after Workspace Activation. Journey state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

Reason: The accepted Phase 3A scope required only the shared framework and account page foundation for a professional TIS Account / School Workspace Setup experience, while leaving full onboarding and payment page redesigns for later phases.

Files changed:

```
saas/router.py
saas/service.py
templates/saas/base.html
templates/saas/account.html
tests/test_saas_phase1.py
docs/AI_PROJECT_CONTEXT.md
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/CHANGE_HISTORY.md
docs/history/saas-onboarding/README.md
```

```
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3A shared framework only. Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal `/saas` route names, admin views, commits, and pushes were not changed.

2026-06-27 - Accepted TIS Account Customer-Facing Wording And Logo Cleanup

Area/module: SaaS customer account pages, school workspace setup pages, billing/subscription status views, transactional account emails, and KMS documentation

Previous state: Customer-facing TIS Account and school workspace setup pages could display internal or technical language such as SaaS-oriented product copy, raw status labels, checkout/payment internals, provider identifiers, tenant/provisioning terms, or account setup labels that were less polished. The shared customer account shell did not consistently present an official TIS logo image across inherited customer forms/pages.

New state: Customer-facing account/setup pages use professional labels such as TIS Account, Account Dashboard, School Workspace Setup, Organization Profile, Branch Setup, Academic Setup, Subscription Setup, Secure Payment, and Workspace Activation. Customer views use display labels for internal statuses and hide customer-irrelevant provider transaction/subscription IDs, attempt UUIDs, checkout session internals, plan IDs, and school group IDs. The shared customer account shell uses the official full-color horizontal TIS logo, and transactional TIS Account emails use an existing official dark-blue TIS wordmark asset.

Reason: The accepted Phase 2 plan required a focused customer-facing wording cleanup and official logo usage pass before any larger account setup UI redesign.

Files changed:

```
saas/router.py
saas/service.py
saas/provisioning_service.py
email_templates.py
templates/saas/base.html
templates/saas/signup.html
templates/saas/login.html
templates/saas/account.html
templates/saas/account_billing.html
templates/saas/billing_status.html
templates/saas/onboarding_organization.html
templates/saas/onboarding_branches.html
templates/saas/onboarding_academic_setup.html
```

```
1     templates/saas/onboarding_contacts.html
1     templates/saas/onboarding_review.html
1     templates/saas/plan_selection.html
1     templates/saas/checkout_summary.html
1     templates/saas/checkout_return.html
1     templates/saas/checkout_cancel.html
1     templates/saas/profile.html
1     templates/saas/security.html
1     templates/saas/sessions.html
1     tests/test_saas_phase1.py
1     tests/test_saas_phase5.py
1     tests/test_email_templates.py
1     docs/AI_PROJECT_CONTEXT.md
1     docs/TIS_MASTER_CONTEXT.md
1     docs/PROJECT_STATE.md
1     docs/CHANGE_HISTORY.md
1     docs/history/saas-onboarding/README.md
1     static/docs/TIS_Project_Reference_Booklet.pdf
1     static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 2 implementation only. No Phase 3 UI redesign, Google/Microsoft login, internal route/module rename, payment behavior change, billing behavior change, provisioning behavior change, database schema change, migration change, operational module change, Next.js landing website change, commit, or push was performed.

2026-06-27 - Accepted TIS Account Email Verification Recovery

Area/module: SaaS onboarding, TIS Account email verification, verification resend recovery, and school workspace setup gate

Previous state: Valid verification links rendered a static verification page instead of continuing the customer toward account setup. Expired or invalid verification links could feel like a dead end. Resend verification existed but did not provide a fully professional recovery path for expired links, already verified accounts, and unknown emails. Password-based accounts that were still pending verification could sign in and reach account/setup routes.

New state: Valid verification links mark the account email verified/active and redirect to the TIS Account login page with a professional success notice. Expired or invalid links show a recovery page with a resend option. Resend verification safely handles unverified accounts, already verified accounts, and unknown-email cases without revealing account existence. Unverified password-based accounts are blocked from starting or continuing school workspace setup. New visible wording in this verification flow uses "TIS Account" and "school workspace setup".

Reason: Testing showed that the customer account setup journey could be blocked after email verification, especially when a verification link expired or the customer needed to recover/resend the link.

Files changed:

```
saas/router.py
saas/service.py
templates/saas/login.html
templates/saas/verify_email.html
templates/saas/verification_sent.html
email_templates.py
tests/test_saas_phase1.py
tests/test_saas_phase5.py
docs/AI_PROJECT_CONTEXT.md
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/CHANGE_HISTORY.md
docs/history/saas-onboarding/README.md
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 1 verification recovery implementation accepted. Phase 2 wording cleanup, Phase 3 setup UI redesign, Google/Microsoft login, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, and the Next.js landing website were not changed. No commit or push was performed.

2026-06-27 - Added Production Memory Stability Guardrails

Area/module: Operational app stability, observations, global location lookup, Render deployment constraints, and engineering standards

Previous state: Production traffic could hit avoidable memory pressure. The observations page included diagnostic stage logging and extra template rendering in the normal request path, and the global location picker could parse a 47 MB reference dataset into a complete in-memory index for simple picker requests. KMS standards did not yet explicitly forbid unbounded caches, duplicate production template renders, or normal-request diagnostic warning spam.

New state: The local stabilization patch gates observation diagnostics behind `TIS_OBSERVATION_DIAGNOSTICS`, removes duplicate observation template pre-renders, and changes location lookup behavior to use streaming/scoped country loading for normal country, region, city, and validation requests. KMS now documents strict production memory and Render stability rules.

Reason: Render logs showed app restarts and user-facing 502s around normal app navigation after deployment. A 512 MB service can be enough for TIS only if the app avoids unnecessary full-dataset memory loads, duplicate rendering, and production debug noise.

Files changed:

```
location_service.py
routers/observations.py
docs/engineering/DEVELOPMENT_STANDARDS.md
docs/PROJECT_STATE.md
docs/AI_PROJECT_CONTEXT.md
docs/TIS_MASTER_CONTEXT.md
docs/CHANGE_HISTORY.md
docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Local changes only. No commit, push, migration, database change, SaaS route change, billing change, tenant logic change, or deployment was performed.

2026-06-26 - Completed KMS v3.0 Phase 3D Lifecycle Foundation

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3C documented rejected decisions, visual documentation framework, AI optimization, governance, and traceability. It still needed a complete self-evolving lifecycle standard that ties implementation, validation, KIA, documentation updates, generated artifacts, review, commit, push, and deployment together.

New state: TIS now has final Phase 3D docs for knowledge lifecycle, documentation automation, KIA standard, self-evolving workflow, documentation dependency map, AI coding workflow, and future automation roadmap. This completes the KMS v1.0 lifecycle foundation.

Reason: Ensure every future approved implementation naturally keeps KMS synchronized without relying on uncontrolled app-side rewriting of source docs.

Files changed:

```
docs/engineering/KNOWLEDGE_LIFECYCLE.md
docs/engineering/DOCUMENTATION_AUTOMATION.md
docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
docs/engineering/SELF_EVOLVING_WORKFLOW.md
docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md
docs/engineering/AI_CODING_WORKFLOW.md
docs/engineering/FUTURE_AUTOMATION_ROADMAP.md
docs/engineering/README.md
docs/TIS_MASTER_CONTEXT.md
docs/AI_PROJECT_CONTEXT.md
```

- `docs/PROJECT_STATE.md`
- `docs/README.md`
- `docs/CHANGE_HISTORY.md`
- `docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md`
- `scripts/generate_docs_pdf.py`
- `static/docs/TIS_Project_Reference_Booklet.pdf`
- `static/docs/docs_manifest.json`

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3D final phase only. SaaS flows, landing page code, database models, migrations, `tis.db`, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Phase 3C Governance And AI Traceability

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3B documented database architecture, development standards, UI/UX design philosophy, roadmap, and stronger onboarding guidance. It did not yet preserve rejected decisions, visual documentation standards, a definitive AI optimization guide, project governance, or explicit decision traceability.

New state: TIS now has Phase 3C engineering docs for rejected decisions, visual documentation framework, AI optimization, project governance, and decision traceability. The PDF generator includes these docs.

Reason: Future developers and AI assistants need to understand why TIS became what it is, not only what currently exists.

Files changed:

- `docs/engineering/REJECTED_DECISIONS.md`
- `docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md`
- `docs/engineering/AI_OPTIMIZATION_GUIDE.md`
- `docs/engineering/PROJECT_GOVERNANCE.md`
- `docs/engineering/README.md`
- `docs/TIS_MASTER_CONTEXT.md`
- `docs/AI_PROJECT_CONTEXT.md`
- `docs/PROJECT_STATE.md`
- `docs/README.md`
- `docs/CHANGE_HISTORY.md`
- `docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md`
- `scripts/generate_docs_pdf.py`
- `static/docs/TIS_Project_Reference_Booklet.pdf`
- `static/docs/docs_manifest.json`

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3C only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3A added module map, repository architecture, user/system flows, and onboarding structure. The handbook still needed database architecture, development standards, UI/UX philosophy, roadmap, and stronger human/AI guidance.

New state: TIS now has Phase 3B engineering docs for database architecture, development standards, UI/UX design philosophy, and product roadmap. Core KMS docs and AI onboarding guidance reference these layers, and the PDF generator includes them.

Reason: Make the generated booklet more useful for new senior developers, Codex conversations, ChatGPT conversations, and future technical reviewers.

Files changed:

```
1    docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
1    docs/engineering/DEVELOPMENT_STANDARDS.md
1    docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
1    docs/engineering/PRODUCT_ROADMAP.md
1    docs/engineering/README.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/AI_PROJECT_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/README.md
1    docs/CHANGE_HISTORY.md
1    scripts/generate_docs_pdf.py
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Engineering Handbook

Area/module: Knowledge Management System and engineering onboarding

Previous state: The generated booklet included KMS source documents, ADRs, module history, AI context, and the Knowledge Center foundation, but it did not fully onboard a new human developer or future Codex/ChatGPT conversation into TIS modules, repository architecture, and end-to-end flows.

New state: TIS now has an engineering handbook layer with a complete module map, repository architecture guide, user/system flow guide, and engineering onboarding index. The PDF generator includes these docs and emits documentation version 3.0.

Reason: Make the generated booklet a true TIS Engineering Handbook rather than only a documentation bundle.

Files changed:

```
1    docs/engineering/README.md
1    docs/engineering/TIS_MODULE_MAP.md
1    docs/engineering/REPOSITORY_ARCHITECTURE.md
1    docs/engineering/USER_AND_SYSTEM_FLOWS.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/AI_PROJECT_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/README.md
1    docs/CHANGE_HISTORY.md
1    scripts/generate_docs_pdf.py
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added Platform Owner Knowledge Center

Area/module: Platform Knowledge Center and KMS access

Previous state: TIS had KMS source docs, ADRs, module history, a generated PDF booklet, and a manifest, but no protected in-app owner page for KMS status or booklet access.

New state: TIS now has a read-only Platform Owner Knowledge Center with KMS health score, manifest metadata, freshness detection, source document status, coverage checks, latest change-history entries, ADR list, module history areas, KIA checklist, and protected PDF view/download routes.

Reason: Platform owners need an internal utility for verifying KMS health and accessing the generated PDF without exposing direct public static links.

Files changed:

```
1    knowledge_service.py
1    main.py
```

```
templates/platform_knowledge_center.html
templates/platform_console.html
scripts/generate_docs_pdf.py
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/README.md
docs/CHANGE_HISTORY.md
docs/AI_PROJECT_CONTEXT.md
docs/history/platform-knowledge/README.md
docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2C only. Regenerate button, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

2026-06-26 - Established Knowledge Management System Foundation

Area/module: Documentation and project knowledge management

Previous state: TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal change history, ADR system, module history foundation, KMS policy, manifest, or compact AI onboarding file.

New state: TIS now has a Knowledge Management System foundation with chronological change history, documentation update policy, ADR structure and initial accepted ADRs, module history folders, AI project context, updated source docs, and an expanded PDF generator.

Reason: Preserve project knowledge for future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and technical reviewers.

Files changed:

```
docs/CHANGE_HISTORY.md
docs/DOCUMENTATION_UPDATE_POLICY.md
docs/AI_PROJECT_CONTEXT.md
docs/adr/README.md
docs/adr/0001-separate-nextjs-landing-website.md
docs/adr/0002-separate-saas-identity-and-operational-users.md
docs/adr/0003-paddle-payment-architecture.md
docs/adr/0004-webhook-only-payment-confirmation.md
docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
```

```
docs/adr/0006-documentation-as-source-knowledge-management-system.md
docs/adr/0007-landing-page-visual-system-strategy.md
docs/history/README.md
docs/history/*/README.md
docs/history/provisioning/2026-06-26-kms-foundation.md
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/README.md
scripts/generate_docs_pdf.py
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only. Platform Owner Knowledge Center, app routes, SaaS flows, database, migrations, landing page implementation, commits, and pushes remain out of scope.

2026-09-11 - Simplified Talent assessment controls, expanded analytics demo data, and added explicit Student/Evaluation permissions

Area/module: Students, Talent & Potential, permissions, local analytics test data

Previous state: Student permanent deletion was not exposed as single/bulk permission-controlled UI actions. Evaluation Period entry into Student Assessments had no dedicated permission distinct from broader Evaluation Plan/Cycle management. The disposable Talent analytics seed used Arabic-script names and a smaller/less visible analytics cohort. Program Criteria controls remained in the normal setup/assessment UI, and assessment entry still carried unnecessary lifecycle-status gating in parts of the implementation.

New state: Student single Delete and Bulk Delete are separately permissioned (`students.delete`, `students.bulk_delete`), organization/global scoped, backend-enforced, and blocked when Academic Placement or Talent history exists; bulk deletion is atomic. Evaluation Period selection for entering Student Assessments is separately permissioned as `talent_evaluation_plans.select_period`, enabled by default for Administrator only, configurable through System Configuration role permissions, rendered disabled when absent, and required server-side for Cycle-to-Period linking. The normal Talent setup/assessment UI no longer exposes Program Criteria controls. Student Assessment uses the exact assessable Evaluation framework without Program/Framework lifecycle labels acting as extra start gates. The local-only Talent seed now contains 10 Arabic student names written in Latin script across Grades 3-5, with completed/in-progress assessments sized for privacy-safe analytics review.

Reason: Match the Owner-directed simple Talent workflow, ensure every new mutable/destructive action has explicit permission authority, and provide realistic disposable local data for evaluating Results & Analytics UX without touching production or `tis.db`.

Files changed include:

```
permission_registry.py
student_academic_service.py
```

1 `routers/students.py`

1 `routers/students_ui.py`

1 `templates/students.html`

1 `routers/talent_evaluation_plans.py`

1 `routers/talent_ui.py`

1 `static/js/talent-evaluation-workspace.js`

1 `static/js/talent-operations.js`

1 `static/js/talent-program-workspace.js`

1 `talent_local_test_data.py`

1 focused Student/Talent regression tests

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/PROJECT_STATE.md`

1 `docs/CHANGE_HISTORY.md`

Documentation updated: Yes

PDF regenerated: Pending final local KMS synchronization for this branch.

AI project context updated: No; the canonical behavior and implementation-state changes are fully captured in Master Context, Project State, and this Change History entry.

Reviewer/approval notes: Owner-directed product simplification and permission hardening. No production data mutation, schema migration, billing change, AI change, or destructive repository operation is introduced.

2026-09-11 - Added Grade-specific Talent descriptors and explicit Student history force delete

Area/module: Talent & Potential rubric configuration; Students; permissions; migrations

Previous state: Talent achievement descriptions existed only at Competency x Rubric Level, so a single Framework could not faithfully store different wording for the same rubric level by Grade. Student permanent deletion was blocked whenever Academic Placement or Talent history existed, with no separately permissioned Administrator override.

New state: An additive Grade-specific descriptor table stores optional Competency x Rubric Level x Grade overrides while preserving generic descriptors as fallback. Program setup authors the Program's eligible Grade cells, Framework fingerprints/clones include the overrides, and Student Assessment displays the wording matching the Assessment's historical Grade. A new migration, `20260911_001_talent_grade_specific_rubric_descriptors`, creates the table. Students also gain `students.force_delete_history`, Administrator-default and System-Configuration-managed. Authorized force deletion requires a history preview and explicit irreversible confirmation before deleting Student-owned Academic Placement and Talent history; normal Delete and Bulk Delete still fail closed on history. Student row Open/Delete icons were reduced to compact sizing.

Reason: Faithfully represent owner-supplied multi-Grade rubrics such as Mental Math without creating artificial Grade competencies, while giving Administrators an explicit, permission-controlled way to remove temporary/test Students together with their history when intentionally required.

Documentation updated: Yes

PDF regenerated: Pending branch KMS synchronization after implementation validation.

AI project context updated: No; this change affects deterministic Talent rubric persistence/presentation and Student deletion authority, not AI architecture or provider behavior.

Reviewer/approval notes: Owner-directed. No production data was deleted or seeded by this repository change. Production execution of force deletion or demo-data seeding remains a separate explicit operational action.

2026-09-11 - Corrected Talent rubric hierarchy, result display, and Evaluation ordering

Area/module: Talent & Potential rubric configuration, Student Assessments, Talent Review, migrations

Previous state: Grade-specific achievement wording existed, but Framework competencies were still effectively framework-wide. The authoring UI therefore rendered a flat Grade/Competency/Level matrix and could repeat the same competency across every Grade. Student Assessment inferred Grade applicability from descriptor presence. Talent Review showed a verbose rubric badge such as "3/5 Level 3 - Meets". Student Assessment Evaluation contexts were ordered by Cycle creation time, so a later-created Term 2 could appear before Term 1 despite the Evaluation Plan sequence.

New state: Framework competency membership now has an optional Grade scope, introduced by migration `20260911_002_talent_framework_competency_grade_scope`. Program authoring follows the owner-directed hierarchy Grade -> Competency -> Level -> Achievement Description and groups rubric authoring/review accordingly. Student Assessment uses competencies matching the Assessment's recorded historical Grade, while intentionally unscoped competencies remain shared. Existing Grade-specific descriptors remain the achievement-description persistence authority and generic descriptors remain backward-compatible fallback. Compact Talent result surfaces can render the ordered level as a single number with accessible label text and low-to-high order-derived visual intensity. Student Assessment Evaluation contexts now use linked Evaluation Period `sequence` as the canonical order, with creation/id fallback only for legacy unlinked Cycles.

Reason: Match the Mental Math rubric structure supplied by the Owner, remove unnecessary repetition in assessment/result views, and ensure configured Term order is respected.

Documentation updated: Yes

PDF regenerated: Pending local KMS synchronization after GitHub implementation sync.

AI project context updated: No; this is deterministic Talent configuration/presentation behavior, not AI architecture.

Reviewer/approval notes: Owner-directed correction. No production deployment or production/demo data mutation was performed by this repository change.

2026-09-11 - Rebuilt Talent Program/Rubric workflow and added explicit re-evaluation attempts

Area/module: Talent & Potential Programs, Framework/Rubric authoring, Student Assessments, Analytics, migrations

Previous state: Program authoring still exposed the guided Basics -> What we assess -> Evaluation Plan -> Ready workflow as the primary path. Grade-specific rubric content existed but was split across wizard substeps. Completed Assessments were immutable, but a later rubric change had no explicit operational re-evaluation state/action, and a Draft-labelled Framework with Assessment history could still be semantically edited in place.

New state: Program creation/editing is focused on Program identity plus eligible Grades. Each Program has a separate Rubric action. Rubric authoring uses collapsible eligible-Grade sections with Grade-scoped competencies and ordered rubric Levels/achievement descriptions. Program overview summarizes Grade/competency/rubric configuration after save.

ADR 0036 introduces additive Assessment attempt semantics: `is_current` and `reassessment_of_assessment_id` (migration

`20260911_003_talent_assessment_reassessment_attempts`). A completed current Assessment is marked Re-evaluation required when a materially different newer assessable Framework exists. Re-evaluation creates a new Cycle and Assessment, preserves the prior completed evidence as historical, marks the prior Assessment non-current, and keeps the replacement current. Framework semantic edits are blocked once any Assessment references that Framework; a new Framework Version is required instead. Analytics current-result queries now ignore superseded Assessment attempts/population members so re-evaluation does not double-count Students.

Reason: Direct Product Owner request to simplify the evaluation process and ensure rubric changes force a safe, visible re-evaluation path for already-completed Students without rewriting historical evidence.

Results & Analytics: No privacy threshold was invented or hard-coded as a fallback. Production analytics remains governed by ADR 0028 external provider configuration. Missing deployment configuration continues to fail closed.

Documentation updated: Yes - ADR 0036, TIS_MASTER_CONTEXT, PROJECT_STATE, CHANGE_HISTORY.

Generated KMS artifacts: Pending local `scripts/kms.py sync` after GitHub changes are synced.

Deployment: Not performed by this change. Owner deploys after local verification/KMS regeneration.

2026-09-11 - Aligned Talent rubric structure to Owner reference and preserved Evaluation lineage

Area/module: Talent & Potential Program/Rubric authoring, rubric persistence, Student Assessments, reassessment, analytics

Previous state: The Grade-first redesign still treated the ordered rubric scale as primarily Framework-wide/shared. This did not match the Owner's reference in which every competency owns its own rubric and level descriptions. Reassessment replacement attempts also needed an explicit link back to the original visible Evaluation/Term, and a full Framework fingerprint could over-trigger re-evaluation for metadata-only/no-op clones.

New state: The canonical structure is Grade -> Competency -> Competency-owned Rubric -> ordered Levels. `TalentRubric.framework_competency_id` and migration `20260911_004_talent_competency_specific_rubrics` permit one rubric per exact Framework Competency while preserving NULL-owned legacy shared rubrics for historical compatibility. The Rubric workspace mirrors the Owner reference with collapsible Grade sections, competency nodes, per-competency Add/Edit Rubric controls, and nested levels/descriptions. Student Assessment renders/selects only the selected competency's rubric levels and rejects cross-competency level writes.

Re-evaluation comparison now uses the Student-facing assessment structure for the Student's historical Grade rather than Framework version metadata, so a no-op clone does not require re-evaluation. Migration `20260911_003_talent_assessment_reassessment_attempts` also carries/backfills `evaluation_context_cycle_id`: replacement attempts and new Students using a newer saved rubric remain operationally attached to the original Evaluation/Term. Current analytics project the current attempt to that original Evaluation population and exclude the physical replacement Cycle from current population counts.

Reason: Match the Owner-supplied rubric structure exactly at the product/domain level while preserving immutable historical evidence, clear re-evaluation behavior, and current-result analytics semantics.

Documentation updated: Yes - ADR 0036, TIS_MASTER_CONTEXT, PROJECT_STATE, CHANGE_HISTORY.

Generated KMS artifacts: Pending local `scripts/kms.py sync` after GitHub changes are synced and tests pass.

Deployment: Not performed. Owner retains deployment authority.

2026-09-11 - Added deterministic Talent Overall Program Result

Area/module: Talent & Potential rubric authoring, Student Assessment, Talent Review

Previous state: Rubric levels had governed order and optional numeric values, but there was no single fair cross-competency result for a Student when competency rubrics used different numbers of levels. Talent Review could surface a highest recorded rubric level, which is not a valid cross-competency summary when rubric scales differ.

New state: ADR 0037 defines rubric level number as its ordered proficiency position and adds the deterministic `equal_competency_normalized_rubric_position` Overall Program Result. Each selected competency level is normalized within its own rubric from 0 to 100, all applicable competencies contribute equally, and the mean is rounded half-up. Student Assessment and Talent Review display the result as score/100 with supplemental low-to-high color progression. Individual competency evidence remains available in the Assessment.

Official Identification remains a separate authorized human decision. The overall score does not create an automatic talented/not-talented label, does not replace Review Candidate policy, and does not alter the existing optional KPI contract.

Schema/migration: None. The result is a deterministic read projection from immutable Assessment, Framework, rubric-level, and competency-result evidence.

Documentation updated: Yes - ADR 0037, TIS_MASTER_CONTEXT, PROJECT_STATE, CHANGE_HISTORY.

Deployment: Not performed. Owner deploys only after focused tests and KMS enforcement pass.

2026-09-11 - Consolidated Talent evaluation, review, results, analytics, and UX

Area/module: Talent & Potential Programs, Rubrics, Student Assessments, Talent Review, Results & Analytics, organization Student drill, UI/UX, performance

Owner-approved scope:

1 stop automatic legacy shared-rubric level inheritance in new competency rubrics;

1 add explicit Copy Levels From another current competency, with optional

descriptions;

1 preserve assessed legacy rubric data as immutable history;

1 use the Program's own ordered rubric scale for the canonical Overall Program

Result;

1 require one coherent rubric length across applicable competencies before

Assessment completion;

1 organize Student Assessments as Evaluation Period -> Program -> Student;

1 surface all live-placement eligible Students in the selected Program/Evaluation;

1 preserve and expose Re-evaluation required for materially changed rubrics;

1 make Talent Review include every current completed Assessment, not only

materialized Review Candidates;

1 keep Review Candidate and Official Identification separate;

1 surface configured Evaluation Periods in Program settings;

1 show separate multi-Program Student results without a universal cross-Program

Talent score;

1 enrich organization/Program analytics using the existing privacy-closed M9/M10

architecture;

1 upgrade Talent visual hierarchy, icons, semantic states, result visuals, and

responsive presentation;

1 reduce repeated full-page loading and Program-list N+1 requests.

Metric decision: ADR 0037 is revised. The canonical Overall Program Result is now the arithmetic mean of selected rubric ranks on a coherent Program scale, for example `4.4 / 5`, using deterministic half-up rounding to one decimal. A normalized percentage may be derived only for visual presentation. Incompatible applicable rubric lengths block completion rather than being silently normalized.

Workflow decision: ADR 0038 establishes Evaluation Period -> Program -> Student as the primary assessment information architecture and all current completed Assessments as the Talent Review workspace population. Existing Annual Evaluation Plan/Period, live-placement eligibility, tenant/Branch scope, reassessment history, privacy, and Official Identification boundaries are preserved.

Analytics: For a selected Program, privacy-visible completed rubric distributions may expose average rubric rank and Program-scale summary. The governed P7 Student drill may show separate Program results and the UI may render a Students Across Programs matrix. Organization/Branch/Grade analytics remain on existing privacy providers and suppression rules. No universal cross-Program Talent score is introduced.

Performance: The Programs list uses one bounded summaries read instead of per-Program annual/framework/configuration fan-out. Selected Program initial load uses the Program-specific endpoint and parallel setup reads. Routine Program/rubric saves refresh only the affected bounded slice and then redraw from cache; Save Rubric and hash-only setup navigation require no network reads. Evaluation Plan edits retain existing content and refresh only Plan/Cycle data. Student Assessment detail and explicit-Evaluation roster loading remove avoidable sequential request waterfalls. Full workspace loading remains only for genuine initial or context-changing reads and stale-state recovery.

Schema/migrations: None introduced by this consolidation.

Documentation updated: Yes - ADR 0037, ADR 0038, ADR index, TIS_MASTER_CONTEXT, PROJECT_STATE, CHANGE_HISTORY, TIS_MODULE_MAP.

Generated KMS artifacts: Require final `scripts/kms.py sync` before integration because included KMS Markdown changed.

AI project context: No change. This batch affects deterministic Talent workflows/analytics/UI and does not change the approved AI boundary, provider, model, entitlement, or execution architecture.

Deployment: Not performed by repository implementation. Owner retains PR/merge/deployment authority after QA and KMS enforcement are green.

TIS Architecture Decision Records

docs/adr/README.md

Architecture Decision Records

ADRs record major TIS architectural and product decisions. They explain why the system is shaped the way it is.

ADR Rules

- Create an ADR for major decisions affecting architecture, identity, SaaS, payment, provisioning, deployment, documentation governance, or long-term product boundaries.
- Do not use ADRs for ordinary bug fixes.
- If a decision is replaced, mark the older ADR as Superseded and link the new ADR.
- ADRs complement `docs/TIS_MASTER_CONTEXT.md`; they do not replace it.

Status Values

- Proposed
- Accepted
- Superseded
- Deprecated

ADR Index

- `0001-separate-nextjs-landing-website.md`
- `0002-separate-saas-identity-and-operational-users.md`
- `0003-paddle-payment-architecture.md`
- `0004-webhook-only-payment-confirmation.md`
- `0005-delayed-tenant-provisioning-after-verified-payment.md`
- `0006-documentation-as-source-knowledge-management-system.md`
- `0007-landing-page-visual-system-strategy.md`
- `0008-workspace-classification-foundation.md`
- `0009-commercial-state-and-entitlement-resolution.md`
- `0010-review-only-saas-demo-requests.md`
- `0011-demo-workspace-provisioning-and-commercial-source-links.md`
- `0012-seven-day-demo-lifecycle-and-access-enforcement.md`
- `0013-demo-to-paid-workspace-conversion.md`
- `0014-demo-customer-communications-and-approval-orchestration.md`
- `0015-centralized-ai-entitlements-and-usage-accounting.md`
- `0016-pre-deploy-database-migration-boundary.md`

```
1     0017-platform-owner-demo-operations-and-access-profiles.md
1     0018-unified-commercial-access-and-capacity-authority.md
1     0019-secure-promo-code-definition-and-governance.md
1     0020-promo-redemption-and-commercial-grants.md
1     0021-existing-workspace-conversion-read-only-audit.md
1     0022-controlled-existing-workspace-conversion.md
1     0023-existing-workspace-paid-activation.md
1     0024-promo-expiry-recovery-and-paid-continuation.md
1     0025-capacity-based-packaging-and-common-customer-feature-baseline.md
1     0026-versioned-constraint-based-smart-timetable-generation.md
1     0027-b10-longitudinal-organization-intelligence-contract.md
1     0028-b11-production-qualification-policy.md
1     0029-m10-organization-analytics-repeatable-read-boundary.md
1     0030-m10-organization-analytics-multi-worker-qualification-minimum.md
1     0031-student-learning-style-v1.md
1     0032-draft-talent-program-hard-delete-exception.md
1     0033-open-talent-cycle-additive-roster-synchronization.md
1     0034-assessment-delete-zero-evidence-exception.md
1     0035-live-academic-placement-assessment-eligibility.md
1     0036-talent-rubric-reassessment-attempts.md
1     0037-talent-overall-program-result.md
1     0038-talent-evaluation-first-assessment-review.md
1     0039-talent-competency-kpi-level-simplification.md
1     0040-per-user-permission-override-precedence-and-scope.md
1     0041-promo-grant-replacement.md
1     0042-student-learning-style-four-dimension-profile.md
1     0043-tis-student-number-managed-identifier-invariant.md
1     0045-al-andalus-section-display-workspace-authorization.md
```

TIS Engineering Handbook

docs/engineering/README.md

This folder turns the TIS KMS into a practical engineering handbook. It is intended for new human developers, future Codex conversations, future ChatGPT conversations, project owners, platform owners, and reviewers.

Choose A Reading Path

Use the [KMS Navigation Guide](../KMS_NAVIGATION.md) to select the smallest relevant document set for a specific task.

For a broad engineering onboarding, read:

Recommended onboarding order:

1. [AI Project Context](../AI_PROJECT_CONTEXT.md)
2. [TIS Master Context](../TIS_MASTER_CONTEXT.md)
3. [Project State](../PROJECT_STATE.md)
4. [TIS Module Map](TIS_MODULE_MAP.md)
5. [Repository Architecture](REPOSITORY_ARCHITECTURE.md)
6. [User and System Flows](USER_AND_SYSTEM_FLOWS.md)
7. [Database Architecture Overview](DATABASE_ARCHITECTURE_OVERVIEW.md)
8. Relevant [ADRs](../adr/README.md) and [module history](../history/README.md)

Engineering Documents

1. [Independent Permission Closure Review](PERMISSION_CLOSURE_REVIEW.md): review findings, verification boundaries, and remaining approval blockers.
1. [TIS Module Map](TIS_MODULE_MAP.md): product and system modules, maturity, ownership, risks, and guardrails.
1. [Repository Architecture](REPOSITORY_ARCHITECTURE.md): repository structure and responsibilities.
1. [User and System Flows](USER_AND_SYSTEM_FLOWS.md): end-to-end public, SaaS, payment, provisioning, operational, owner, and KMS flows.
1. [Database Architecture Overview](DATABASE_ARCHITECTURE_OVERVIEW.md): conceptual data relationships and isolation boundaries.
1. [Development Standards](DEVELOPMENT_STANDARDS.md): non-negotiable engineering rules for humans and AI assistants.
1. [UI/UX Design Philosophy](UI_UX_DESIGN_PHILOSOPHY.md): design principles by product surface.
1. [Product Roadmap](PRODUCT_ROADMAP.md): completed, current, next, and future roadmap.
1. [Rejected Decisions](REJECTED_DECISIONS.md): significant declined alternatives and their consequences.
1. [Visual Documentation Guide](VISUAL_DOCUMENTATION_GUIDE.md): screenshot and diagram standards.
1. [AI Optimization Guide](AI_OPTIMIZATION_GUIDE.md): definitive AI onboarding and operating guide.
1. [Project Governance](PROJECT_GOVERNANCE.md): ownership, approvals, quality gates, and traceability.
1. [Knowledge Lifecycle](KNOWLEDGE_LIFECYCLE.md): documentation and engineering lifecycle.
1. [Documentation Automation](DOCUMENTATION_AUTOMATION.md): current and future KMS automation rules.

1 [KIA Standard](KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md): formal KIA examples, decision tree, and failure cases.

1 [Self-Evolving Workflow](SELF_EVOLVING_WORKFLOW.md): official task-to-deployment workflow.

1 [Documentation Dependency Map](DOCUMENTATION_DEPENDENCY_MAP.md): propagation rules across KMS documents.

1 [AI Coding Workflow](AI_CODING_WORKFLOW.md): required workflow for future AI coding assistants.

1 [Future Automation Roadmap](FUTURE_AUTOMATION_ROADMAP.md): possible future automation improvements.

Before Coding

Before changing code:

1 confirm the approved scope,

1 inspect affected files and tests,

1 read relevant ADRs,

1 read relevant module history,

1 preserve tenant isolation,

1 preserve identity boundaries,

1 avoid touching SaaS, landing, database, or migrations unless explicitly approved.

After Coding

Every implementation must complete the Knowledge Impact Assessment:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

If included source docs changed, synchronize:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Run `.\.venv\Scripts\python.exe scripts\kms.py check` for final read-only validation.

TIS Module Map

docs/engineering/TIS_MODULE_MAP.md

Talent filtered aggregate workspace (Agent 3, 2026-09-25)

1 `talent_dashboard_service.py`: bounded current-Student Evaluation membership

projection, canonical current-attempt results/Classification, raw-population totals, authorized filter options, selected comparisons, compatible period results and Program/Competency-bound rubric evidence. Uses P2 completion, P3 result/indicator and P4 Classification provider rules with conservative cross-group closure; emits no Student identities.

1 `routers/talent_results_analytics.py`: GET

`/api/talent/results-analytics/academic-years/{academic_year_id}/dashboard`, analytics permission, tenant/year/Branch ceiling and existing repeatable read-only snapshot dependency. Learning Style also requires `students.view`.

1 `talent_learning_style_privacy.py`: owner-approved Classification-cohort

publishability gate shared by the operational roster summaries and dashboard. It does not suppress authorized individual rows or invent frontend thresholds.

1 `static/js/talent-dashboard.js`: shared aggregate filters and six dashboard

sections, Overview summaries and dependent-filter clearing.

1 `static/js/talent-charts.js`: sanitizes nonvisible values before chart markup,

applicable mode switching without fetches, native keyboard controls and exact tables. Loaded only on Overview, Analytics and Student Assessment.

1 `static/js/talent.js`: bounded request/generation ownership, independent Overview

headline/chart sections and one dashboard request. `talent-experience.js` no longer adds a second rubric request to this dashboard.

1 `static/js/talent-program-workspace.js` and `static/css/talent-experience.css`:

searchable Program cards, executive hierarchy and shared chart presentation.

1 `static/css/talent-experience.css` (Talent visual system layer): presentation

only; header band, KPI tiles, chart cards, Program cards, action hierarchy, responsive/reduced-motion/forced-colors rules. Owns no values or thresholds.

1 `tests/talent_visual_system.test.cjs`: chart-mode matrix, switched-visual

no-leak, protected Learning Style, Program Status absence, action system.

Student Assessment filtered roster and insights (Agent 2, 2026-09-25)

1 `routers/talent_assessment_cycles.py` `cycles_eligible_students`: owns the

server-filtered live roster for an Evaluation context. It composes the existing Branch ceiling before search/Grade/Section/state/classification filtering, then produces identity-free Classification and Learning Style

aggregate projections for precisely that response population.

1 `static/js/talent-operations.js`: serializes roster controls into the route

query, renders backend projections with progress bars and screen-reader tables, and never derives a Classification or Learning Style aggregate. `talent-experience` no longer installs the retired DOM-only roster filter.

Talent chart and refresh modules (Production Follow-up Part 2, 2026-09-25)

1 `static/js/talent-charts.js`: shared privacy-safe chart renderer. `modes()` is the single source of

allowed visualization types per family; `chart`, `series` (time series) and `comparison` render sanitized rows only; explicit chart-type choices are remembered per chart key.

1 `static/js/talent-dashboard.js`: `overview(data, {animate})` (Doughnut default, entrance class) and

`analytics()` with one bottom Selected Comparisons section and a Progress Over Time link.

1 `static/js/talent.js`: `refreshDashboard()` (background refresh: token, AbortController, held height,

anchor and focus restoration), `captureAnchor`/`restoreAnchor` also used by `load()`, `shouldAnimateEntrance()` (reduced-motion aware), `periodVisual()` (Progress Over Time via the shared series chart). `templates/talent/workspace.html` loads `talent-charts.js` on `longitudinal` too.

Talent Branch ceiling (Batch 1 Closure, 2026-09-25)

> Amended 2026-09-25 (Part 1 production follow-up): `talent_branch_ceiling` now always returns > `None` (the sidebar Branch is a default, not a ceiling); `branch_scope_unrestricted` is > `auth.can_access_all_branches`; a Branch-limited actor remains confined by > `auth.get_accessible_branch_query`. `user.scope_all_branches`, the `branch_scope=all` cookie > handling and the `/scope/branch` `all` value below were removed; `talent_ui` publishes > `tp-config.branch` (default) and `tp-config.branchLocked`. `talent-api-errors.js` `CODE_COPY` > carries the curated Start Assessment error codes; `talent-charts.js` adds the uniform > Classification withheld note.

1 `talent_branch_scope.py`: `talent_branch_ceiling`, `branch_scope_unrestricted`,

`visible_branch_ids`, `visible_branch_ids_or_none`, `branch_within_ceiling`. Consumed by `talent_org_intelligence_service.resolve_access_context`, the Talent routers (`talent_analytics`, `talent_results_analytics`, `talent_evaluation_progress`, `talent_assessments`, `talent_assessment_cycles`, `talent_learner_profiles`, `talent_review_candidates`, `talent_official_identifications`, `talent_educator_inputs`, `talent_programs` selector lists), `talent_ui` and `talent_request_permissions.branch_in_authorized_scope`.

1 `auth.get_current_user` sets `user.scope_all_branches` from the `branch_scope=all`

cookie (only for `can_access_all_branches` actors); `main.py` `POST /scope/branch` accepts `branch_id=all`; `templates/base.html` offers it on `/talent` pages only.

Talent current-Student, read-batch and permission helpers (Deployment Acceptance Batch 1, 2026-09-24)

1 `talent_current_students.py`: `current_student_exists` (correlated EXISTS, same

SchoolGroup) composed into `talent_analytics_service.population_query`, `talent_org_intelligence_service.frozen_membership_query`, the Evaluation Progress Branch reads and `list_assessments`.

1 `talent_read_batch.py`: opt-in request-scoped memoization (`read_batch(db)`,

`memo`, `prime`) stored in `Session.info`; used by the Assessments list, classification/Talented families, Student Drill rows and Evaluation Progress Branch results via `talent_student_assessment_service.prime_assessment_batch`.

1 `talent_request_permissions.py`: `request_permission_checker` (effective permission

set resolved once per request; used by the M10 access context, the Assessments list and Evaluation Plans) and `branch_in_authorized_scope`.

1 `student_academic_service.STUDENT_OWNED_MODELS`: the ten Student-owned tables

deleted by `force_delete_student_history`, locked to ORM metadata by `tests/test_student_delete_completeness_batch1.py`.

1 `routers/talent_ui.py`: renders `tp-config.branch` (validated active Branch) and

`talent_asset_version` content-hash asset URLs; `templates/talent/workspace.html` carries the inline loader watchdog.

Learning Style distribution boundary (Deployment Acceptance Correction C, 2026-09-24)

`student_learning_style_analytics.py` is the single engine for the Learning Style aggregate: `resolve_population` (unchanged authorized Student population) and `build_distribution(students)` (nine categories, count/percentage/`total_population`, denominator includes Unassigned, no Talent suppression, no `talent_analytics_privacy` import). Consumers: `routers/students.py` (`/api/students/analytics/learning-style-distribution`), `routers/students_ui.py` + `templates/students.html` (panel) and `routers/talent_results_analytics.py` (`/learning-style`, still `students.view`-gated); none uses the privacy provider for this distribution, while the Classification and Talented routes in the same router still do. Frontend: `static/js/talent.js` `learningStyleDistributionSection` (backend values only) with `static/css/talent.css` `.tp-ls-*` and `static/css/students.css` responsive rows. See ADR 0031 "Acceptance C Amendment".

Talent Student identity presentation and legacy history boundary (Deployment Acceptance Correction B, 2026-09-24)

`static/js/talent-student-identity.js` is the single shared presentation for Talent Student identity (name, Learning Style, Classification, Talented); it is loaded on every Talent view before `talent.js` and is presentation only (no classification arithmetic). Data authorities: Learning Style = `Student.learning_style` via `talent_operational_context.authorized_contexts` (`student_learning_style`), `routers/talent_assessment_cycles.py` eligible-students (`learning_style`, sourced from `talent_operational_context.student_identity_metadata`, a tenant-bound bulk read; the ADR 0031 guard keeps scoring/eligibility modules free of Learning Style), `talent_learner_profile_service` (`student.learning_style`, per-assessment `overall_result`), and `talent_org_student_drill` (`learning_style`); Classification = `talent_classification_service.assessment_classification` surfaced by `routers/talent_assessments.py` (list adds classification for current Completed rows using `overall=`), `routers/talent_review_candidates.py` (`talent_review_workspace`, same `overall=` reuse),

the learner profile and the Student Drill. The `reviews` view (`routers/talent_ui.py` VIEWS, still gated by `talent_review_candidates.view`) is the Legacy Review & Identification History surface and is no longer in the primary sidebar (`ui_shell.py`).

Talent workspace client lifecycle ownership (Deployment Acceptance Correction A, 2026-09-24)

`static/js/talent.js` owns boot, request bounding (`boundedRequest`), the generation/stale guard, and independent section scheduling (`createSections`) for every view; delegates (`TalentOperations`, `TalentProgramWorkspace`, `TalentEvaluationWorkspace`) render inside that lifecycle and their failures surface through it. `templates/talent/workspace.html` must load every global a view's render path needs before `talent.js` (`talent-rubric-visual.js`, `talent-api-errors.js`, `talent-rubric-request.js` on all views; operational bundles per surface). `static/js/talent-experience.js` owns the Program-filtered rubric section UI (bounded request, Retry). `static/js/talent-api-errors.js` owns curated user-facing error copy (HTTP status + stable code; never raw `detail`). `static/js/talent-rubric-request.js` owns the single-flight rubric-distribution read (keyed by Program + Academic Year + assessment_state; `talent.js` publishes, `talent-experience.js` consumes). No backend module, route or schema was added.

M14-M18 Correction Program Closeout Ownership Notes (M18b-3, 2026-09-24)

Final module ownership after the correction program (functionally implemented on `dev`, not deployed): `talent_classification_service.py` is the sole classification authority; `assessment_classification(db, assessment, *, overall=...)` optionally accepts the caller's already computed `overall_program_result` so per-row callers (`talent_org_student_drill.py`) do not recompute it. `talent_results_analytics_service.py` and `routers/talent_results_analytics.py` own the Learning Style / Classification / Talented Results & Analytics contract; `static/js/talent.js` renders it without recomputation and `templates/talent/workspace.html` gates surface-specific bundles. The stored Student four-percentage columns and their write validators in `student_academic_service.py` remain only as deprecated storage. No new module, route, or schema was added by M18b-3.

Results & Analytics Backend Contract Ownership (M18b-1)

1 `talent_results_analytics_service.py` (new): bounded M18b-1 Results &

Analytics backend contract - `applicable_assessment_rows/classify_rows` (the M18a-governed `status == 'completed' AND is_current == True` grain, classified via `talent_classification_service.assessment_classification`, never a duplicated band table), `sum_raw_counts_across_branches` (the ONLY Organization/Branch rollup rule - a pure raw-count sum, never an average of Branch percentages), `classification_family/talented_family` (privacy- closed distributions reusing `talent_analytics_service.build_breakdown_group` / `apply_primary_privacy/run_complementary_suppression`, the same M9 primitives every other breakdown uses, with a new opaque privacy class `"P4"`). Does NOT extend `talent_org_intelligence_contract.MetricCode` (stays frozen per ADR 0044) and does NOT duplicate Competency/Evaluation- Period-Progress semantics - those two families are served by the pre-existing `routers/talent_analytics.py /rubric-distribution` route and `routers/talent_evaluation_progress.py` routes directly.

1 `routers/talent_results_analytics.py` (new): `/api/talent/results-analytics`

route family - `/academic-years/{id}/learning-style` (Student-domain-wide, `students.view` permission, delegates to `student_learning_style_analytics.py`), `/programs/{id}/academic-years/{id}`

/classification and .../talented (Program-bound, talent_analytics.view permission, reuse talent_analytics_service.resolve_context/ resolve_filters/population_query for scope/filter enforcement - a filter can only narrow, never widen, the caller's authorized Branch scope).

1 routers/talent_evaluation_progress.py /

talent_evaluation_progress_service.py: the dead learning_style_dimension query parameter/keyword argument (inert since M14 - the "learning_style" metric it parameterized was never a member of APPROVED_BRANCH_METRICS) is removed outright from branch_comparison/branch_comparison_metric's signatures. No behavior change.

Automatic Assessment Classification Ownership (M17)

1 talent_classification_service.py (new): single classification authority.

project_to_classification_scale (deterministic Decimal linear projection of a Program's own 1..N rubric average onto 1.00-5.00), classify_score (the five fixed owner-approved bands), is_talented (Exceptional only), assessment_classification (bounded per-Assessment read projection, None unless status == "completed").

1 talent_student_assessment_service.py: overall_program_result now

additively returns average_tenths (the exact integer tenths already computed internally) so classification never re-derives precision from a float; no other behavior changed.

1 routers/talent_assessments.py / routers/talent_review_candidates.py:

_display_payload additively exposes backend-computed classification, classification_score, is_talented fields (assessments router) and classification, is_talented (review-candidates router, for legacy Review workspace context). No new endpoints, no new permission keys.

1 talent_review_candidate_service.py / `talent_official_identification_

service.py: unchanged - remain the legacy/history Review Candidate and Official Identification authorities; no longer the source of current Talented` state.

1 static/js/talent-operations.js / static/css/talent.css: assessment

detail and legacy Review workspace views render the backend classification and Talented state; the legacy Review link is relabeled to reflect its history role; no client-side band derivation.

1 docs/adr/0037-talent-overall-program-result.md: 2026-09-23 Amendment

records the governed classification-scale decision (Option B: preserve per-Program rubric scale, add a deterministic projection) without rewriting the original ADR 0037 decision.

Learning Style Correction + Aggregate Distribution Ownership (M14)

1 models.py / db_migrations.py: Student.learning_style's

ck_students_learning_style CHECK constraint is widened from four to eight values (20260923_002_student_learning_style_eight_values); the four learning_style_*_percentage columns/constraints are unchanged and preserved (operationally deprecated, not dropped).

1 student_academic_service.py: LEARNING_STYLES is the single

authoritative eight-value constant (`_clean_learning_style` validates against it); `LEARNING_STYLE_PERCENTAGE_FIELDS`/ `_clean_learning_style_percentage` remain present only so a pre-existing stored value is never silently rewritten - no longer reachable from any create/update API or UI input path.

1 `routers/students.py` / `routers/students_ui.py`: Student create/update no

longer accept the four percentage fields as input (silently ignored, not validated); `routers/students_ui.py`'s `LEARNING_STYLE_OPTIONS` backs the one categorical selector.

1 `templates/_learning_style.html`: `learning_style_field`/

`learning_style_badge` are the sole shared categorical display/edit macros (extended to eight values); the percentage-profile macros are removed.

1 `templates/student_form.html` / `templates/student_profile.html`: one

Learning Style selector on create/edit; the profile shows the one selected value (or "Not assigned"), never four percentage bars; the Talent context line reads the current categorical value directly.

1 `student_learning_style_analytics.py`: the existing

categorical distribution engine (reused, not parallel-built), extended to all eight values plus an "Unassigned" bucket; scope/permission unchanged from ADR 0031; its privacy/suppression contract was superseded 2026-09-24 (Acceptance C): authorized aggregation, no Talent small-cell suppression.

1 `talent_evaluation_progress_service.py`: `APPROVED_BRANCH_METRICS` is six

metrics (the "learning_style" Branch-comparison metric is removed); `static/js/talent.js` and `templates/talent/workspace.html` no longer offer the Learning Style dimension selector (see the M10 section below, now corrected).

1 `student_roster_service.py` is untouched - its column contract never

included Learning Style or the four percentage columns.

Students M11 Roster Frontend Ownership

1 `routers/students_ui.py` projects the existing `students.import` and

`students.export` capabilities into the Students list; it does not authorize M6 API requests.

1 `templates/students.html` owns compact actions and the transient accessible

`.xlsx` import dialog.

1 `static/js/students-roster.js` owns opaque workbook download, file-extension

usability checks, multipart preview/apply submission, safe DOM rendering, duplicate-submit guards, explicit apply confirmation, and roster refresh.

1 `static/css/students.css` owns the bounded dialog/table responsive treatment.

1 `routers/students.py` and `student_roster_service.py` remain the authorities

for workbook parsing, Student-ID/tenant/Placement validation, privacy-safe conflicts, permission enforcement, create-only semantics, and atomic apply.

1 No spreadsheet parser, CSV/`.xls`, persistent import batch, background job,

schema, migration, or backend semantic change belongs to M11.

Students M15 Roster Round-Trip & Integrity Ownership

1 `student_roster_service.py` owns M15 round-trip: `section_display`

accepted-but-ignored (display-only, never identity), `STD`-prefix normalization, deterministic NO_CHANGE classification, create-only apply that skips NO_CHANGE rows, and the bold/frozen/autofilter export header styling.

1 `static/js/students-roster.js` renders CREATE / No change / error from backend

row status and only enables apply when at least one CREATE row is error-free.

1 `student_academic_service.py::force_delete_student_history` (unchanged by M15,

now documented + tested) is the single permanent-delete cascade authority that removes every Student-owned Talent table before the Student row.

1 `tests/test_student_delete_talent_visibility.py` and

`tests/test_student_roster_import_export.py` lock the M15 contract.

Results & Analytics Branch Comparison Frontend (M10)

1 `templates/talent/workspace.html`: originally added the labeled Learning

Style dimension selector beside the existing Program/Academic-Year/Metric context controls. **Current state (M14 correction): removed** - see "Learning Style Correction + Aggregate Distribution Ownership (M14)" above.

1 `static/js/talent.js`: Organization Overview requests the existing M4 Branch

comparison route for the selected Program and metric; one chart renders backend rows without reordering or analytics calculation. Existing Talent Map columns supply authorized Branch display labels.

1 `static/css/talent.css`: owns the responsive horizontal bar, grouped Period,

and categorical protected/no-data treatments.

1 `routers/talent_evaluation_progress.py` and

`talent_evaluation_progress_service.py` remain unchanged authorities for permissions, frozen Branch grouping, privacy closure, Framework comparability, and serialized values; the metric allowlist is now six metrics as of M14 (Learning Style means removed - see above).

1 The frontend never renders a numeric bar for a non-visible state and never

calculates Branch/Organization results or privacy decisions. No second chart, backend route, permission, schema, migration, or M11 roster surface is added.

Student Evaluation Progress Frontend (M9)

1 `templates/student_profile.html`: owns the existing authorized Student Talent

profile location and declares one Evaluation Progress mount per Program and Academic Year already returned by the Learner Profile.

1 `static/js/student-evaluation-progress.js`: fetches only the M4 Student

Evaluation Progress endpoint and renders its ordered Period labels, explicit result states, comparable Overall Result, and framework-change explanation. It performs no active-Period selection, weighting, sorting, averaging, comparability decision, aggregate reconstruction, or placement substitution.

1 `static/css/students.css`: supplies compact responsive period cards and a

distinct Overall Result treatment using existing Student design tokens.

1 `routers/talent_evaluation_progress.py` and

`talent_evaluation_progress_service.py` remain the unchanged authority for authorization, active/open Period selection, state, order, Framework comparability, and results.

1 M9 exposes no Branch/Organization progress surface or comparison chart and

adds no route, permission, schema, or migration.

Talent Performance M16 Ownership

1 `authorization.py`: resolves each route gate against one effective permission

projection and exposes that authorized set on request state for same-request reuse.

1 `routers/talent_ui.py` + `ui_shell.py`: reuse the route-authorized set for

Talent action presentation and shell navigation; commercial feature checks remain independent and every request remains authorized.

1 `templates/talent/workspace.html`: owns surface-specific script inclusion;

common orchestration/presentation stays universal while Program, Evaluation, Assessment, and Review bundles load only on their owning surfaces.

1 `static/css/students.css`: owns the shared wrapping horizontal Students action

row used by page-level and per-Student controls.

Talent Frontend Cleanup M8 Ownership

1 `static/js/talent-operations.js`: omits Reload Saved Rubric and Educator Input

from normal Student Assessment rendering and no longer fetches or binds the retired Educator Input controls.

1 `routers/students_ui.py` and `templates/student_profile.html`: omit Educator

Input from the normal Student Talent-profile projection/presentation.

1 `routers/talent_educator_inputs.py`, its service/model, assessment/rubric

persistence, audit, and permissions remain unchanged compatibility/history authorities. No historical evidence is deleted.

1 Talent scoring, analytics, Evaluation Progress, Student Learning Style,

Section formatting, schema, and migrations are unchanged.

Students M7 Frontend Ownership

1 `routers/students_ui.py` projects canonical managed Student IDs, delegates new

creation/authorized replacement to existing Student services, parses HTML blanks as nullable Learning Style dimensions, and reuses the shared duplicate conflict projection.

1 `templates/student_form.html`, `templates/student_profile.html`, and

`templates/students.html` own the bounded create/list/profile/edit presentation.

1 `templates/_learning_style.html`, `static/js/students.js`, and

`static/css/students.css` own reusable accessible dimension controls/profile bars and responsive fixed-prefix behavior.

1 `routers/students.py`, `student_academic_service.py`, models, migrations, and

the M5 Section formatter remain the backend authorities and are not redesigned.

1 Roster import/export frontend was out of scope here and was implemented in

M11 (`students.import`/`students.export` actions on `templates/students.html` over the existing M6 roster API).

Talent Rubric Visuals And Analytics Ownership

1 `static/js/talent-rubric-visual.js` and `static/css/talent-rubric-visual.css`: shared arbitrary-label/count rubric ordering, position cue, selected state, and reduced-motion treatment.

1 `routers/talent_review_candidates.py`: adds the exact Assessment's highest recorded rubric level to authorized Review read projections only.

1 `talent_learner_profile_service.py`: resolves competency and rubric labels/order for authorized historical assessment evidence; each Assessment item also carries the M17 backend classification/`is_talented` fields (M18a), reusing `talent_classification_service.assessment_classification`.

1 `static/js/talent.js`: keeps Organization Overview selective and routes detailed Program, Branch, matrix, and longitudinal exploration to their owning views; the legacy Review/Identification `candidate_of_eligible`/`identified_of_eligible` labels are explicitly "Legacy ..." (M18a) and never read as current Talent authority.

1 `routers/students_ui.py` and `templates/students.html`: present the existing Learning Style distribution through Planning-derived Branch/Grade/Section choices; the API and privacy provider remain unchanged.

Owner Video Acceptance Correction Ownership

1 `static/js/talent-operations.js`: selects configured Programs directly inside Evaluation Periods, materializes/links an internal Cycle when absent, keeps Evaluation/Program context through Start Assessment, and surfaces action failures visibly.

1 `talent_student_assessment_service.py` + `routers/talent_assessments.py`: normal Assessment start remains the authority for newest complete Grade-applicable competency-owned criteria and current Assessment creation; the returned Assessment carries exact Framework provenance.

1 `static/js/talent-program-workspace.js`: immutable Delete Competency becomes clone-new-Draft-then-delete, preserving assessed history; KPI disclosure owns explicit collapse affordance.

1 `permission_registry.py`: adds `students.view_all_branches` and constrains it to the Administrator managed role.

1 `routers/students_ui.py`: enforces Students list Branch selection against `students.view_all_branches` plus organization/global scope and projects explicit Learning Style analytics-unavailable state without weakening privacy.

1 `templates/students.html` + `static/css/students.css`: accessible icon-only Student management actions, fixed Branch context for actors without cross-Branch authority, and fail-closed Learning Style availability messaging.

Ghars Talent Sidebar Branding Ownership

1 `static/img/talent-ghars-logo.svg`: Owner-supplied Ghars bilingual Talent module mark, traced into a lightweight white SVG for the dark application sidebar.

1 `ui_shell.py`: keeps the canonical navigation label/permissions/routes and declares the Talent module brand-logo asset.

1 `templates/base.html`: renders the full Ghars mark in place of the generic Talent icon + visible text while retaining the accessible `Talent & Potential` navigation name.

1 `static/css/app-shell.css`: sizes the full logo clearly without changing child-tree behavior or neighboring module navigation.

Talent Recovery, Review Filters, And Navigation Ownership

1 `permission_registry.py`: owns `talent_assessments.reset_for_reassessment`; the standard Administrator all-permissions default includes it while Editor/User defaults do not.

1 `talent_student_assessment_service.py`: evidence-preserving completed-Assessment reset. It marks the current attempt historical and audits the action; it never deletes dependent evidence.

1 `routers/talent_assessments.py`: permission-enforced reset endpoint and backend-computed reset action projection.

1 `static/js/talent-operations.js`: selected Evaluation/Program presentation, Start/Continue/View routing, Reset for Re-assessment UI, and simplified Talent Review presentation.

1 `routers/talent_review_candidates.py`: server-side Branch/Grade/Section filtering over frozen Assessment placement context.

1 `routers/talent_programs.py`: authorized Planning Branch/Grade/Section options used by shared Talent filters.

1 `templates/talent/workspace.html` + `ui_shell.py`: sidebar tree is the sole primary Talent module navigation; the page-level duplicate ribbon is removed.

1 `static/js/talent-program-workspace.js`: one-Grade-at-a-time authoring, summary-based readiness, Assessment Criteria presentation, and real numeric KPI Add/Edit/Delete controls.

1 `static/js/talent.js`: shared authorized review filters and actionable analytics failure messages using backend detail.

Talent Program And Evaluation UX Orchestration

1 `routers/talent_programs.py`: Program/Framework authoring API plus bounded

Program-list summaries and explicit competency-to-competency rubric-level copy. New rubric creation does not auto-import legacy shared levels. True Competency and Rubric Level removal use the dedicated assignable permissions `talent_programs.delete_competency` and `talent_programs.delete_rubric_level`; whole

Program deletion remains separate. `static/js/talent-program-workspace.js` treats Finish Setup as the governed complete-Draft activation boundary (Program then reviewed Framework), keeps operational-summary hash actions live, and leaves generated competency codes to the backend so Unicode/Arabic names cannot collide through client ASCII stripping.

1 `static/js/talent-program-workspace.js`: searchable Program table with one normal Open Program row action (plus separately authorized delete where applicable); Program

identity/eligible-Grade settings; visible Evaluation Period summary; separate Grade -> Competency -> competency-owned Rubric -> ordered Levels builder; and explicit Copy Levels From another current-Draft competency. Selected Program loading uses direct Program reads plus parallel independent setup reads; routine mutations refresh only affected cached slices rather than the entire workspace graph.

1 `talent_student_assessment_service.py`: live-eligibility Assessment start,

current Evaluation selection restricted to complete competency-owned rubrics, historical-only legacy-rubric comparison, immutable reassessment attempts, legacy same-Framework mismatch detection for pre-guard completed evidence, coherent-rubric-scale completion guard, and deterministic arithmetic-mean Overall Program Result.

1 `routers/talent_assessments.py` and `static/js/talent-operations.js`:

Evaluation Period -> unique Programs -> Students current operational workflow, merging Plan and Cycle display context so configured Periods remain visible before Cycle materialization and duplicate physical Cycles cannot duplicate a Program card. Reassessment actions include the legacy same-Version reset compatibility path. The normal workspace does not render a duplicate Assessment Records/History table.

1 `routers/talent_review_candidates.py`: candidate persistence APIs plus a

workspace projection over all current completed Assessments so non-candidate Students are not hidden from Talent Review.

1 `static/js/talent-evaluation-workspace.js`: continues to own the existing

Annual Evaluation Plan/Period management without a duplicate Program-period persistence model. Post-mutation refreshes re-read only Plan/Cycle data and keep the rendered workspace visible.

1 `static/css/talent-program-workspace.css` and `static/css/talent.css`:

TIS-token-based light visual hierarchy, Evaluation/Program cards, semantic status chips, rubric/result visuals, responsive states, and localized loading.

Student/Talent Migration Prerequisite

1 `db_migrations.py`: migration `20260904_000` ensures PostgreSQL-valid

`(id, school_group_id)` keys for Branch, Academic Year, and any existing Student table, with equivalent-key detection and duplicate-data failure. The M4 Cycle creation omits only its future M8 Period FK; M8 adds it after the parent exists.

1 `scripts/run_migrations.py`: baseline metadata excludes all Student/Talent

migration-owned tables so the ordered `000` through `20260905_001` chain is authoritative on fresh and existing schemas.

1 `tests/test_postgresql_migration_transactions.py`: reproduces the prior

PostgreSQL failure/rollback and validates fresh-chain order, every composite target, late prerequisite application, idempotency, and duplicate failure.

1 `tests/test_render_startup.py` and `tests/test_student_academic_foundation.py`:

enforce the baseline deferral and migration registration boundaries.

Talent Results & Analytics UI (M11 Phase C)

1 `routers/talent_analytics.py`: existing M9 privacy-closed Program analytics

plus privacy-safe average rubric rank and selected-Program result summary derived only when all required level cells are visible.

1 `talent_org_student_drill.py`: existing P7 Student drill extended to expose

each current completed Assessment's separate Program-scale Overall Result, and (M18a) its M17 backend classification/`is_talented` state, gated on the same `status == 'completed' AND is_current == True` predicate.

1 `routers/talent_organization_analytics.py`: retains tenant/scope,

permission, breadth, and privacy gating for organization Student drill and aggregate intelligence.

1 `ui_shell.py`, `templates/base.html`, and `static/css/app-shell.css`:

permission-aware Talent & Potential child tree in the main application sidebar. The tree is expanded only while Talent is active and links to clean top-level destinations.

1 `templates/talent/workspace.html`: shared Talent shell and filters.

1 `static/js/talent.js`: top-level Talent links retain Academic Year only and

clear stale Program/Cycle/Assessment/Branch/Grade/Section/metric/dimension context; Student Assessments owns its Program selector. Results-family sub-navigation alone preserves analysis context intentionally.

1 `static/js/talent.js`: organization/Branch/Grade analytics, selected-Program

result story, competency averages, governed Student preview, Students Across Programs matrix, longitudinal and Talent Map navigation. It never combines different Programs into one Student Talent score and never reconstructs suppressed organization values in the browser.

1 `static/css/talent.css`: responsive KPI/result cards, bars, matrices,

semantic status states, and accessible low-to-high presentation.

1 Existing M9/M10 privacy closure, access, paging, comparison, production

provider, and B11 qualification rules remain authoritative.

B11 Production Qualification Boundary

1 `talent_organization_analytics_providers.py`: F1 production provider module.

It accepts only the externally configured approved cohort value 5 and breadth values 1000/1000/1000; missing, invalid, mismatched, rejected, and exceptional state fails closed. Its deterministic local path requires non-production and the exact `.local_test_data/talent_local_test.db` URL.

1 `talent_analytics_privacy.resolve_privacy_policy_provider`: lazily resolves

the configured P1-P7 cohort policy and retains the existing primary-then- complementary suppression engine. There is no permissive fallback.

1 `saas.customer_feature_policy` and `saas.demo_feature_registry`: register

`feature.organization_intelligence` as a normal customer feature. `saas.entitlement_service.organization_feature_available` resolves only active commercial/feature state and does not perform permission checks.

1 `talent_org_intelligence_service`: availability and breadth dependency seams

now resolve F1 providers. Availability still runs before the separate `talent_analytics.view` permission check, and breadth rejects without truncation.

1 `tests/test_talent_organization_analytics_providers.py`: cohort boundary,

complementary suppression, configuration failure, entitlement/permission separation, exact breadth boundaries, local-path, seven-route accessibility, and production-environment fail-closed coverage.

1 ADR 0028 governs the approved provider values and B11-E release gates. F1

resolved the provider implementation gate. As of 2026-09-09 (ADR 0028/ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED.

1 B11-C is CLOSED after PostgreSQL 16.15 profiling of all seven routes and a

PASS WITH NON-BLOCKING OBSERVATIONS independent re-review. Query counts were bounded with no N+1; no index, schema, migration, permission, privacy, or entitlement semantic change was made.

1 B11-D is CLOSED after READ COMMITTED qualification proved mixed snapshots

within Overview and Student Drill requests. REPEATABLE READ remains a governance-required, not-implemented proposal for a dedicated full-request M10 transaction boundary. Suppressing-policy concurrency and other route reproduction remain B11-E gates; Candidate B trends NO CHANGE and no index is approved.

Talent Organization Intelligence (M10 B0-B9)

1 `talent_org_intelligence_contract.py`: non-functional contract types for

canonical `CellIdentity`, public `PrivacyProjection`, exact `+1/-1` `RelationshipTerm`, additive `Relationship`, tenant-only future graph membership validation, and the V01-V25 ownership/status ledger. Pre-B2 hardening adds closed string-backed `MetricCode`, `MeasureComponent`, and `MembershipGrain` enums plus immutable approved metric mapping metadata; coefficient validation requires `type(value) is int` and rejects numeric lookalikes that compare equal to `+1/-1`.

1 `tests/test_talent_org_intelligence_contract.py`: executable B0 conformance

harness for identity/presentation independence, coefficient and additive topology constraints, sign/order/overlap canonicalization, tenant separation, inherited privacy-state semantics, forbidden fields/behavior, complete vector traceability, and absence of any registered M10 aggregate route.

1 `talent_analytics_relationship_graph.py`: B1 tenant-bound canonical Cell

registration, Relationship identity/deduplication, Cell-to-Relationship adjacency, structural validation, shared-coordinate reuse, and deterministic bipartite connected-component discovery. It imports only the B0 contract and Python standard library.

1 `tests/test_talent_privacy_relationship_graph.py`: B1 behavioral coverage

for validation, component topology, bridging, nested Org/Branch/Grade, shared Program/Branch coordinates, transposition, overlap symmetry, insertion-order independence, and prohibited-dependency boundaries.

1 `talent_analytics_privacy_closure.py`: B2 exact `Fraction` RREF and augmented-

rank analysis, per-coordinate uniqueness, deterministic monotonic closure, component-local restriction, structural victim selection, exact-source-only derived rates, and all-or-nothing safe derived-payload projection. Its one orchestration entry applies M9 primary privacy before closure.

1 `tests/test_talent_privacy_reconstruction.py`: executable V05-V25 fixed and

bounded metamorphic coverage, including free-variable uniqueness, redundant/ inconsistent systems, nested and row/column attacks, transposition, value permutation, no-data/coarsened/restricted semantics, localization, derived rates, sibling leakage, monotonicity, and idempotence.

1 Boundary: no query/service/router, production privacy threshold/provider,

permission, entitlement, schema, migration, UI, or production behavior is added by B0-B2.

1 `talent_org_intelligence_service.py`: B3/B4 immutable access context,

fail-closed commercial-availability and breadth boundaries, normalized context fingerprint, authorized Program universe, frozen-membership base query, common status aggregation, Program×Branch/Program×Grade and backend totals, independently gated Candidate/Identification membership, Program- grain M8 execution, and mandatory `PrivacyClosedProjectionSet` seam.

1 `tests/test_talent_org_intelligence_queries.py`: access ordering, tenant/year/

Branch/filter security, provider injection, current-Placement resistance, grouped-query semantics/count, secondary-query omission, M8 grain, fingerprint, forbidden vocabulary, and B2 integration-seam coverage.

1 `routers/talent_organization_analytics.py`: the single B5

`/api/talent/organization-analytics/overview` route. It composes scoped set-based queries into canonical Cells and equality Relationships, runs M9 primary privacy then B2 closure, and accepts only `PrivacyClosedProjectionSet` at its serializer boundary. Candidate and Identification metrics are permission-omitted, and Identification counts only `decision == identified`.

1 `tests/test_talent_organization_overview.py`: end-to-end B5 authorization,

tenant/year, historical Branch, metric omission, privacy/no-data, active predicate, and safe-serialization coverage.

1 B5 checkpoint boundary: at that checkpoint no additional M10 route, new

permission, entitlement mapping, production provider/ceiling, schema, migration, Talent Map, or UI existed.

1 `talent_org_talent_map.py`: B6 canonical Program-by-Branch/Program-by-Grade

Cell construction, shared row/column/scope totals and Relationships, M9/B2 closure orchestration, exact-source rate derivation, and a serializer accepting only its privacy-closed wrapper. Sparse totals use one shared authoritative-child helper for both arithmetic and graph terms; empty totals remain `no_data` and one-child equalities stay connected for

privacy closure.

1 `routers/talent_organization_analytics.py`: additionally exposes only

`/talent-map`, enforcing metric/dimension/filter/secondary-permission and injected breadth boundaries before set-based aggregation.

1 `tests/test_talent_organization_talent_map.py`: B6 route, scope, metric,

permission, breadth, transpose, totals, no-data, serializer, and 2x2 reconstruction coverage.

1 B6 adds no new permission, entitlement, schema, migration, production

provider, ranking, universal score, Program Portfolio, Branch Intelligence, overlap, longitudinal, Student drill, frontend, or B7+ route. The targeted independent sparse-matrix re-review passed after remediation; B6 is closed.

1 `talent_org_b7.py`: B7 canonical Program-series and Branch-series Cells,

authoritative sparse totals, additive relationships, closure, and strict closed-wrapper serialization for Portfolio and Branch Intelligence.

1 `routers/talent_organization_analytics.py`: additionally exposes

`/program-portfolio` and `/branches/{branch_id}` with existing access, filter, breadth, historical-scope, and secondary-permission composition.

1 `tests/test_talent_organization_program_portfolio.py` and

`tests/test_talent_organization_branch_intelligence.py`: B7 semantics, privacy, omission, historical Branch, identity, and serializer coverage.

1 B7 itself added no Grade breakdown, Branch execution attribution, permission,

entitlement, schema, migration, ranking, score, UI, AI, or overlap route.

1 `talent_org_participation_overlap.py`: B8 canonical symmetric Program-pair

P2 Cells, diagonal distinct-participant semantics, privacy closure without fabricated additive overlap equations, and strict closed-only matrix serialization.

1 `talent_org_intelligence_service.py`: adds one set-based B8 aggregation that

deduplicates `(program_id, student_id)` in frozen authorized scope before a canonical Program-pair self-join; only aggregate rows leave SQL.

1 `routers/talent_organization_analytics.py`: additionally exposes only

`/participation-overlap`, with access/filter/breadth gates before aggregation.

1 `tests/test_talent_organization_participation_overlap.py`: B8 route,

diagonal/symmetry/deduplication, scope, zero/no-data, privacy, serializer, deferred-domain omission, and breadth coverage.

1 B8 adds exactly one contract metric (`participation_overlap` = P2 `count` at

`program_participation` grain) and no permission, entitlement, schema, migration, ranking, score, UI, AI, Candidate/Identification overlap, or B9+.

1 talent_org_student_drill.py: B9 gate-level P7

student_drill_population/count/distinct_student Cell fed by distinct authorized Students (no per-Student additive relationship), one minimized top-level StudentDrillRow per Student with frozen Program/Cycle contexts, a strict StudentDrillClosedProjection closed wrapper that raises TypeError for a raw SQL row/ORM object/non-visible gate, distinct-Student pagination, page/context-bounded Candidate/Identification fetch, and a closed-only serializer.

1 routers/talent_organization_analytics.py: additionally exposes only

/students (B9), requiring talent_analytics.view_students composed with talent_analytics.view before any identifiable query, reusing existing access/filter/breadth primitives.

1 tests/test_talent_organization_student_drill.py: B9 route, permission

composition, historical Branch/current-placement-irrelevance, filters, pagination, minimization, P7 gate, Candidate/Identification isolation and semantics, Learner Profile hint, serializer strict-typing, bounded query family, and aggregate-route regression coverage.

1 B9 adds no new permission, entitlement, schema, migration, ranking, score,

AI field, Educator Input exposure, Student-ID filter, UI, or B10+ (longitudinal) route.

1 ADR 0027 (`docs/adr/0027-b10-longitudinal-organization-intelligence-

contract.md) records the B10-A "Longitudinal Organization Intelligence" architecture as an APPROVED governance decision, and B10-B has now implemented it exactly as governed. The route is bounded to one Program/one Academic Year, ordered M8 TalentPlannedEvaluationPeriod slots (Period = order authority via its governed sequence; optional linked Open/Closed TalentAssessmentCycle = evidence authority, consistent with uq_talent_assessment_cycles_period), exactly nine of the fourteen existing MetricCode values, one metric per response, comparable/ not_comparable` states, and no server-computed delta/percent-change. B9's own status above is unchanged; B10-B added no new permission, entitlement, schema, or migration.

1 talent_org_longitudinal.py (B10-B): `GET /api/talent/organization-

analytics/programs/{program_id}/longitudinal, the seventh Organization Intelligence route, added in routers/talent_organization_analytics.py. Reuses resolve_access_context/authorized_program_universe/ resolve_filters/frozen_membership_query unchanged; fetches the single governed M8 Plan (uq_talent_annual_evaluation_plans_config) and its Period+linked-Cycle series in one bounded query, then aggregates population/completed/started/Candidate/identified counts grouped by Cycle id in one bounded query each (never one query per Period, never a Student ORM fetch or Student-ID set). Every Cell uses relationships=() - no same-point or cross-time additive Relationship exists anywhere in the module, so no delta/change/percent_change field can ever be produced; rates reuse derive_exact_rate/project_safe_derived_payload unchanged. LongitudinalClosedProjection is the sole strict closed-wrapper serialization boundary (TypeError on a raw dict/ORM row/unclosed point), matching B8's/B9's discipline. Comparability follows the ADR's governed precedence (missing/cancelled/no-population reason, then privacy_protected, then framework_changed, then comparable`). Candidate/Identification SQL is skipped entirely unless the selected metric requires it AND the actor holds the matching secondary permission.

1 tests/test_talent_organization_longitudinal.py: route/contract, exact

nine-metric allowlist and five-exclusion coverage, authorization (unauthenticated/forbidden/foreign-AY/foreign-and-unconfigured-Program non-enumeration/unavailable/unauthorized-Branch-filter/historical-scope), Period/Cycle/Plan lifecycle states (varying cadence, sequence-not- insertion-order, draft Cycle, cancelled Period, missing Cycle, no frozen population, empty Plan, unlinked ad-hoc Cycle exclusion), Framework sensitivity (including the mandatory missing-reason-overrides-framework_changed precedence test), privacy (no delta/change/ percent_change, provider-exception fail-closed, no_data-is-never-zero, factual-zero-passes-privacy, privacy_protected comparisons, no second privacy evaluation from comparison metadata), rate arithmetic, secondary permission query-skip discipline, breadth-before-aggregation shape and fail-closed rejection, bounded (non-per-Period) query count, and strict serializer-isolation. Independent B10-B security/privacy review passed with non-blocking observations; B10 is CLOSED. B11, B12 (closeout), and every ADR 0027 deferred- capability/production-gate item remain open.

1 B11-A (read-only qualification, completed) confirmed the privacy,

commercial-availability, and breadth-policy provider seams used by all 7 M10 routes then had no production implementation and correctly failed closed; the F1 provider module documented above subsequently resolved that code gate. `talent_organization_analytics_observability.py` (B11-B) adds one reusable `OrganizationAnalyticsObservation` accumulator and a dedicated `tis.talent.organization_analytics.observability` logger - separate from the immutable business/security audit trail in `audit.py` - emitting only a bounded allowlist (`projection_family`, `outcome`, `latency_ms`, safe structural counts reused from each route's own existing breadth-policy inputs, and each provider's coarse missing/available-or-evaluated/ rejected-or-unavailable/exception outcome). `routers/talent_organization_analytics.py` wires this around all 7 routes purely observationally (`_enforce_breadth_observed`/`_privacy_evaluated`/ `_b7_context` telemetry wrappers reuse the exact shape values/decision distinctions each route already computes; they never recompute a provider decision or alter a response body). A telemetry emission failure is swallowed internally and can never affect the HTTP response or weaken a fail-closed decision. Query-count instrumentation is explicitly deferred to a future "B11-C profiling" phase - no new SQLAlchemy event-listener instrumentation was added. See `tests/test_talent_organization_observability.py` (22 tests). B11-B is implemented/tested and independently security/privacy reviewed with PASS and non-blocking observations; B11-B is CLOSED. No schema, migration, permission, entitlement, privacy threshold, or breadth limit was added by B11-B; F1 subsequently added the governed provider decisions without schema or permission changes. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED.

Talent Deterministic Analytics (M9, Committed/Pushed On dev)

1 `talent_analytics_service.py`: read-only context/filter resolution and

authorized raw aggregate queries (coverage, rubric level, competency matrix, Candidate/Identification counts, query-skip-on-permission), execution-summary/period-timeline derivation, comparability outcome derivation, and privacy-safe insight composition. No Talent Score/Index/ KPI mean/median/percentile/bins.

1 `talent_analytics_privacy.py`: the provider/interface boundary

(`TalentAnalyticsPrivacyPolicy`, `PrivacyDecision`), the `Cell`/`Group` model, `apply_primary_privacy`, and the `run_complementary_suppression` fixed-point engine (identity/value-independent tie-break, max 8 passes, fails to non-convergence rather than guessing). `resolve_privacy_policy_provider()` delegates to the governed configuration-driven provider builder; production returns `None` only when approved deployment configuration is missing/invalid (fail-closed); `AllowAllTestPolicy`/ `DeterministicSuppressionTestPolicy`/`CoarsenWithReplacementTestPolicy`/ `CoarsenWithoutReplacementTestPolicy` are test-only fixtures, never a production default.

1 `routers/talent_analytics.py`: `/api/talent/analytics` route family

(context, overview, rubric-distribution, kpi-distribution, competencies, breakdowns/{branch|grade|section}, period-comparison, students). Every route runs `apply_primary_privacy` on every privacy-sensitive cell/group before any `run_complementary_suppression` call in the same function - a structural test in `tests/test_talent_analytics.py` statically enforces this ordering. `permission_registry.py` adds `talent_analytics.view` and `talent_analytics.view_students`; `main.py` registers the router.

1 Governance: no production `TalentAnalyticsPrivacyPolicy` implementation

exists (open gate, by design) and PostgreSQL performance/concurrency is unvalidated (open gate, consistent with every prior milestone). M9 is committed and pushed on `dev` at `23ade9a7c6166197140b48a3edbfac849396d580` (commit `feat: add deterministic talent analytics`); it is not deployed, not production-released, and not merged to `master`.

Talent Annual Evaluation Planning (M8)

1 Operational UI: `routers/talent_ui.py`, `templates/talent/workspace.html`, and

the scoped `static/js/talent-*-workspace.js` modules orchestrate existing Program, Plan/Period, Cycle, Assessment, Candidate, Identification, and Educator Input APIs. They add no persistence authority. API permissions, organization scope, frozen historical Branch scope, revisions, and audit behavior remain authoritative.

1 `talent_evaluation_plan_service.py`: Plan/Period lifecycle, normalized

identity, future-tail ordering, closure, rollover, warning, capability, and the canonical Period/Cycle link validator.

1 `routers/talent_evaluation_plans.py`: bounded `/api/talent/evaluation-plans`,

`/evaluation-periods`, and Assessment-Cycle relationship APIs with true AND permission and projection security.

1 `talent_assessment_cycle_service.py`: preserves M4 behavior for ad-hoc Cycles

and validates locked M8 context before opening a linked Cycle.

1 `models.py` and `db_migrations.py`: Plan/Period persistence, nullable scoped

Cycle linkage, and bounded configuration audit expansion.

Talent Learner Profiles (M7, Complete)

1 `talent_learner_profile_service.py`: read-only historical aggregation and

deterministic timeline for one Student; no source-record mutation or profile materialization.

1 `routers/talent_learner_profiles.py`: permission-composed profile API: base

profile permission plus independent M6 view permissions for sensitive Candidate, Identification, and Educator Input sections/events.

1 `routers/students.py`: direct Placement read endpoints use the same stored

historical Branch authorization rule.

Talent Review, Official Identification & Educator Input (M6, Complete)

Independent review added write-time historical-Branch authorization for Educator Input creation/amendment and uniform non-enumerating direct-ID handling across all three M6 routers.

1 Models: `TalentReviewCandidate` (now with `status`/`reviewed_by_user_id`/

`reviewed_at`), `TalentOfficialIdentification`, `TalentEducatorInput` in `models.py`, with M6 contextual extensions to `TalentAssessmentAudit` (`review_candidate`, `review_candidate_review`, `official_identification`, `educator_input` resource types).

1 Authority: `talent_review_candidate_service.py` deterministically evaluates

the exact M3 Review Candidate Policy/rules attached to a Completed Assessment's exact Framework Version and materializes only a qualifying result (starting `pending_review`), plus the one-way `pending_review` -> `reviewed` transition; it is read-only against Assessment/Result/frozen- population data. `talent_official_identification_service.py` records the append-only `identified`/`not_identified` decision, gated on a Reviewed candidate, exactly one per candidate. `talent_educator_input_service.py` resolves and persists historical Placement/Branch/Grade/Section context (frozen Cycle context or effective-dated Placement at `observed_at`) and manages append-only amendment/supersession lineage.

1 API: `routers/talent_review_candidates.py` (`/api/talent/review-candidates`,

now including `POST .../{id}/review`), `routers/talent_official_identifications.py` (`/api/talent/official-identifications`), `routers/talent_educator_inputs.py` (`/api/talent/educator-inputs`); `main.py` registers all three routers.

1 Permissions: Administrator-only-by-default `talent_review_candidates.view/manage`,

`talent_official_identifications.view/record`, and `talent_educator_inputs.view/add/amend` - each independent of the other Talent permission families. `.record` additionally requires organization/ global access scope. Branch scope uses frozen-member Branch context for Review Candidate/Official Identification, and the row's own persisted historical Branch for Educator Input.

1 Deferred (explicit, not implemented): Official Identification revocation/

supersession/second decision/re-identification, a generic review-note/ case-management system, assessor assignment, Learner Profile, analytics/ Talent Map, AI, Development and Support, and Educator Input analytics/ export/AI/attachments - see "M6 Governance Review: Decisions (Resolved)" in `docs/AI_PROJECT_CONTEXT.md`.

Talent Student Assessments

1 Models: `TalentStudentAssessment` and `TalentStudentCompetencyResult` in

`models.py`, with M5 contextual extensions to `TalentAssessmentAudit`.

1 Authority: `talent_student_assessment_service.py` owns Open-cycle/frozen-member

creation, expected-revision result mutation, deterministic completion, ROUND_HALF_UP KPI provenance, read-only terminal states, and audit entries.

1 API: `routers/talent_assessments.py` exposes bounded operations under

`/api/talent/assessments`; `main.py` registers the router.

1 Permissions: Administrator-only-by-default `talent_assessments.view`,

`.manage`, and `.complete`; Branch scope is evaluated from frozen-member Branch context, not current Student Placement. Assessor assignment is absent.

Talent Assessment Cycles

1 Models: `TalentAssessmentCycle`, `TalentAssessmentCyclePopulationMember`,

`TalentAssessmentAudit` in `models.py`.

1 Authority: `talent_assessment_cycle_service.py` derives Draft eligibility,

performs atomic Open/Close, freezes historical context, and computes the canonical population fingerprint. Accepted ADR 0033 permits placement-triggered, additive-only synchronization while Open; it locks and revisions the Cycle, updates count/fingerprint, and audits every added member.

1 API: `routers/talent_assessment_cycles.py` exposes bounded Cycle and

authorization-filtered population operations under `/api/talent/assessment-cycles`.

1 Permissions: dedicated `talent_assessment_cycles.*`; population visibility

and lifecycle governance are distinct from Talent Program configuration.

Teacher Scheduling Rules Components

1 `teacher_scheduling_rules.py`: exact-scope normalized CRUD, rule-shape and

deterministic conflict validation, canonical slot/target resolution, UI data, and unpublished Draft invalidation.

1 `models.py` and migration `20260830_001_teacher_scheduling_rules`: rule headers,

normalized slots and grade/section targets, composite teacher/section scope integrity, and migration-order-safe deferred metadata creation.

1 `templates/system_configuration_timetable.html` and `main.py`: permissioned

Teacher Scheduling Rules configuration under Timetable Settings.

1 `timetable_snapshot_service.py`: schema-v4 immutable teacher-rule authority and

constraint fingerprinting.

1 `timetable_readiness_service.py` and `timetable_problem_builder.py`: deterministic

target, workload, availability, hard-conflict, and lock defenses.

1 `timetable_cp_sat_solver.py` and `timetable_solution_validator.py`: hard timing

enforcement, soft preference scoring, and solver-independent parity.

Guided Curriculum Adjustment UI

1 `routers/planning.py`: serves the exact-scope wizard page and continues to own preview/apply HTTP adapters.

1 `templates/curriculum_adjustment.html`: five-step selection, impact, teacher-decision, review, and success experience.

1 `templates/planning.html` and `templates/subjects.html`: permission-gated entry points.

1 `authorization.py`: protects the page and apply operation with `curriculum.adjust`; preview accepts either the established Planning edit permission or the dedicated adjustment permission.

1 `curriculum_adjustment_preview_service.py`: exposes target eligibility on teacher suggestions for safe selector filtering.

The preview request carries `requested_transfer_periods`; the preview derives section-specific after values from Planning authority, and apply consumes those fingerprinted values unchanged. `routers/subjects.py` decorates exact-scope catalog rows with effective Current/New Planning demand using `planning_subject_demand_service.py`; `templates/subjects.html` renders uniform values or a **Varies** section breakdown.

`adjustment_type="reduce_only"` uses the same preview/apply services with no target demand or target teacher mutation. `templates/edit_subject.html` and the exact-scope subject route expose effective Planning demand plus a prefilled reduction link when `curriculum.adjust` is allowed.

Atomic Curriculum Adjustment Apply

1 `curriculum_adjustment_apply_service.py`: transaction owner for locking, stale

revalidation, demand/assignment/rule reconciliation, Draft invalidation, audit, concurrency rejection, rollback, and duplicate protection.

1 `models.CurriculumAdjustmentAudit` and migration

`20260829_001_curriculum_adjustment_apply_foundation`: durable applied outcome and unique reviewed-fingerprint authority.

1 `routers/planning.py`: apply JSON endpoint protected by `curriculum.adjust`.

1 `permission_registry.py` and `authorization.py`: dedicated permission registration

and route enforcement.

Curriculum Adjustment Preview

1 `curriculum_adjustment_preview_service.py`: read-only scope selection, demand

transfer projection, teacher/capacity suggestions, rule/grouped warnings, Draft impact, and deterministic stale-confirmation fingerprint.

1 `routers/planning.py`: permissioned JSON preview endpoint; no apply endpoint.

1 `authorization.py`: `planning.edit_section` protects the preview route.

1 Existing demand, assignment, timetable version, active pointer, and published

placement models remain read-only inputs in Stage 3.

Explicit Planning Subject Demand Foundation

1 `models.PlanningSubjectDemand`: future section-subject weekly-period authority,

with scoped integrity, active uniqueness, and retained retirement state.

1 `planning_subject_demand_service.py`: single-section and bulk exact-scope

explicit-first demand resolution with transitional legacy fallback.

1 `db_migrations.py`: idempotent Current/New section backfill from grade-matched

Subjects through migration `20260828_004_planning_subject_demands_foundation`.

1 `routers/planning.py`, `routers/teachers.py`, `main.py`, `timetable_logic.py`,

`timetable_readiness_service.py`, `timetable_snapshot_service.py`, and `subject_distribution_rules_ui.py` consume resolved section demand for live required-period arithmetic. Snapshot-fed generation therefore receives the same authority; publication history remains unchanged.

Smart Timetable Stage 5.2 Components

1 `timetable_visibility_service.py`: active-pointer-only reads and scoped teacher identity.

1 `routers/timetable.py`: simplified workflow, working deletion, published routes,

and backward-compatible canonical lifecycle/mutation conflict responses.

1 `timetable_logic.py`: workspace/history lifecycle facts, source linkage,

mutability, active identity, and validation/approval/publication timestamps.

1 `templates/timetable.html`: visible lifecycle steps, immutable history actions,

Draft-only generation language, and stale concurrency refresh UX.

1 `templates/published_timetable.html`: control-free official lesson view.

1 `timetable_version_service.py`: working resolution and safe archive/discard.

Smart Timetable Stage 5.1 Components

1 `timetable_conflicts.py`: stable conflict taxonomy, evidence classification,

safe entity references, public redaction, and legacy adapters.

1 `timetable_requirement_projection.py`: deterministic read-only Planning-to-

timetable requirement contract, exact-scope authority, identity, and provenance.

1 `timetable_generation_service.py`: exact-scope enqueue/idempotency, generation

state, claims, leases, heartbeat, recovery, cancellation, staleness recheck, and atomic version persistence; contains no OR-Tools import.

1 `timetable_problem_builder.py`: schema-v3 immutable snapshot normalization,

deterministic lesson IDs, canonical slots, Planning/HRT demand, locks, and regeneration source contract.

1 `timetable_cp_sat_solver.py`: task-runtime-only CP-SAT model and solve result.

1 `timetable_solver.py`: solver-neutral worker contract and the Release 1 CP-SAT

adapter; it owns no scheduling rules or persistence.

1 `timetable_solution_validator.py`: solver-independent hard validation.

1 `timetable_workflow_dispatch.py`: lightweight server-side Render client and

environment-driven task slug.

1 `timetable_generation_workflow.py`: registered on-demand task accepting only the

durable generation run public ID.

1 `timetable_generation_worker.py`: reusable single-run executor and optional local

polling fallback; it is not an always-on production requirement.

1 `routers/timetable.py` and `templates/timetable.html`: permissioned enqueue,

scoped status/cancel, polling, generated review selection, and regeneration action.

1 `requirements-workflow.txt`: web/runtime requirements plus pinned OR-Tools 9.15.6755.

1 `requirements-worker.txt`: compatibility dependency set for the optional local worker.

Commercial Feature Packaging Components

1 `saas/customer_feature_policy.py`: stable normal-customer and internal-feature

classification. It does not decide permission, scope, lifecycle, or capacity.

1 `saas/entitlement_service.py`: centralized source-aware customer feature

decision, including permission, commercial state, optional branch/year scope, catalog state, and BranchEntitlement.

1 `saas/ai_consumption_policy.py`: consumption decision separate from commercial

AI availability; current demo allowances are preserved without creating a provider billing engine.

1 `saas/demo_feature_registry.py` and `saas/demo_access_service.py`: Standard,

Full, and Custom retain the normal baseline; Custom continues to control AI allowances, scope, safety, and experimental configuration.

1 `saas/promo_redemption_service.py`: new promo activation snapshots the common

baseline and exact grant capacity.

1 `db_migrations.py`: migration

`20260819_001_capacity_based_customer_feature_baseline` reconciles plan, active promo, and active demo operational values idempotently.

The boundary excludes Platform Console, Developer/System Owner controls, global audit/support export, promo administration, demo administration, and payment administration. Capacity continues through `commercial_authority_service`.

SchoolGroup Management Boundary Components

1 `main.py`: Platform identity/global capability checks for SchoolGroup create and

delete, tenant-linked update scope, unified branch-capacity presentation, and the SchoolGroup-scoped final-active-branch safeguard.

1 `permission_registry.py`: top-level `schools.create` and `schools.delete` are

developer-only; `schools.manage_all_schools` is the global manual-management capability.

1 `templates/system_configuration_schools.html`: hides global create/delete actions

from tenants and presents source-neutral authoritative branch usage.

1 `saas/commercial_authority_service.py` and `saas/promo_grant_service.py`: unchanged

authority and atomic capacity/assignment boundary used by the page and mutations.

1 `scripts/audit_schoolgroup_provenance.py`: sanitized read-only detection of active,

unlinked internal sandboxes without explicit workspace entitlement.

M3 Promo Redemption Components

1 `saas/promo_redemption_service.py`: secure lookup, owner/context validation,

resumable review and selection, locking, immutable snapshots, activation, idempotency, and redacted durable events.

1 `saas/promo_grant_service.py`: read-time grant/expiry resolution and safe

assignment of a newly created branch from remaining promo capacity.

1 `saas/models.py` and `db_migrations.py`: six M3 tables plus promo entitlement

and tenant-source references under migration `20260805_002_promo_redemption_and_grants`.

1 `saas/commercial_authority_service.py`, commercial access/state/workspace and

branch entitlement resolvers, and `auth.py`: M1 promo source composition and fail-closed branch enforcement.

1 `saas/router.py`, `templates/saas/promo_activation.html`,

`templates/saas/commercial_choice.html`, and `templates/saas/account.html`: owner-only onboarding and existing-organization activation journey.

Existing Workspace Paid Activation

Primary files

1 `saas/existing_workspace_paid_activation_service.py`

1 `saas/billing_identity_service.py`

1 `saas/payment_service.py`

1 `templates/saas/existing_workspace_paid_activation.html`

Responsibility

Activate a previously converted existing customer workspace through an authoritative Paddle subscription while preserving the existing SchoolGroup and all operational data. The module owns direct activation eligibility, branch/capacity quote snapshots, checkout lineage, workspace billing/customer attribution, and the atomic webhook-confirmed commercial transition. It does not own tenant provisioning, normal onboarding checkout, promo redemption, or demo conversion.

M3 never calls Paddle, creates payment evidence, converts internal sandboxes, or mutates staff/teacher activity. Exactly one paid/demo/promo tenant source is required.

M2 Promo Code Components

1 `saas/promo_code_service.py`: secure generation and HMAC lookup authority,

shared plan-capacity validation, controlled scope validation, lifecycle row locking, definition duplication/replacement, and redacted durable audit.

1 `saas/models.py`: `PromoCode`, `PromoCodeBranchRestriction`, and

`PromoCodeAuditEvent` remain the M2 definition records; M3 redemption/grant records are separate.

1 `saas/router.py` and `templates/saas/admin_promo_code*.html`: Platform

Console list/filter, generated-code one-time response, detail, edit, and lifecycle actions under `/saas-admin/promo-codes`.

1 `permission_registry.py`: separate `promo_codes.view` and

`promo_codes.manage` platform permissions. Owner identity is additionally required for activation and revocation.

1 `db_migrations.py`: additive migration

`20260805_001_promo_code_foundation` with no data backfill.

M2 definition management does not itself integrate with M1 authority, WorkspaceEntitlement, TenantProvisioningLink, or Paddle. M3 performs that integration only after final customer activation.

M1 Commercial Access And Capacity Components

1 `saas/commercial_authority_service.py`: single commercial-access composition,

authoritative branch/staff/teacher counters, structured decisions, and SchoolGroup row locking.

1 `saas/commercial_access_service.py`, `saas/commercial_state_service.py`,

`saas/workspace_entitlement_service.py`, and `saas/entitlement_service.py`: existing authorities composed by the facade; they are not duplicated.

1 `main.py`, `routers/users.py`, and `routers/teachers.py`: permission- and

tenant-scoped operational mutation entry points that enforce the shared decision before writing.

1 `saas/provisioning_service.py` and `saas/demo_provisioning_service.py`: paid

and demo final-authority invariants inside their existing atomic provisioning transactions.

Guardrails: every capacity-increasing mutation locks then recounts in the same transaction; platform identity never counts as tenant staff; account-only SaaS identity never counts; demo/internal authority is unmetered only while its existing lifecycle/access resolver is coherent; contradictory authority fails closed.

M8B9 Demo Operations Components

1 `saas/demo_operations_service.py`: owner authorization, lifecycle

orchestration, communication references, summaries, and durable audits.

1 `saas/demo_access_service.py` and `saas/demo_feature_registry.py`: Standard,

Full, and Custom resolution with workspace defaults and branch overrides.

1 `DemoAccessPolicy` and `DemoOperationAudit`: policy and attempt history;

M8B8 usage counters/events remain unchanged.

1 Existing owner demo detail and queue surfaces host the controls.

M8B8 AI Entitlement Components

1 `saas/ai_feature_registry.py`: stable feature identifiers, display labels,

required permission/entitlement, eligible plans, demo allowance, and enabled state.

1 `saas/ai_entitlement_service.py`: tenant-safe decisions, consistent denial

metadata, and the only reservation/finalization/consumption API.

1 `AIFeatureUsageCounter` and `AIFeatureUsageEvent`: separated durable

internal/demo/paid accounting.

1 `permission_registry.py`: assignable `ai.use` authorization boundary.

M8B7 Demo Customer Journey Components

1 `saas/demo_email_service.py`: durable demo email intents, branded rendering inputs, provider dispatch, and retries.

1 `saas/demo_notification_service.py`: deduplicated Platform Owner Notification Center events.

1 Existing demo request, provisioning, lifecycle, and conversion services remain authoritative.

1 `ui_shell.py` and `templates/base.html` own the active tenant demo indicator.

1 `scripts/process_demo_lifecycle.py` remains dry-run by default and is the production apply/dispatch entry point.

This map describes the known TIS modules, where they live, their maturity, related docs/ADRs, and the guardrails developers must respect.

Platform Owner

Purpose: Own global TIS platform operations, owner/co-owner controls, platform developer accounts, cross-organization oversight, and protected KMS access.

Main files/folders:

1 `auth.py`

1 `main.py`

1 `templates/platform_console.html`

1 `templates/platform_knowledge_center.html`

1 `knowledge_service.py`

1 `permission_registry.py`

Maturity/status: Implemented for owner identity, platform console, developer/co-owner controls, and read-only Knowledge Center.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`
- `docs/history/platform-knowledge/`

Risks/guardrails:

- Do not treat platform developers as owners.
- Do not expose owner utilities to tenant users.
- Reuse `auth.is_platform_owner(...)` and existing owner access helpers.

Platform Console

Purpose: Allow platform identities to inspect organizations, switch context, and manage platform owner/developer controls.

Main files/folders:

- `main.py`
- `templates/platform_console.html`
- `ui_shell.py`

Maturity/status: Implemented and active.

Related docs/ADRs:

- `docs/PROJECT_STATE.md`

Risks/guardrails:

- Context switching must not weaken tenant isolation.
- Owner-only management controls must remain owner-only.

Knowledge Center

Purpose: Show KMS health, source coverage, PDF freshness, searchable document metadata, ADRs, module history, change history, and KIA policy to platform owners.

Main files/folders:

- `knowledge_service.py`
- `main.py`
- `templates/platform_knowledge_center.html`
- `static/docs/docs_manifest.json`
- `static/docs/TIS_Project_Reference_Booklet.pdf`

Maturity/status: Read-only Phase 2C implementation complete and Phase 7C navigation enhanced. The manifest-backed library groups documents by knowledge area, filters locally by category/module/freshness, searches titles/summaries/paths, and links to protected booklet pages. Repository-level impact and freshness enforcement is

active; no app regenerate button exists.

Related docs/ADRs:

- `docs/adr/0006-documentation-as-source-knowledge-management-system.md`
- `docs/history/platform-knowledge/`

Risks/guardrails:

- Do not link directly to `/static/docs/...` from UI.
- Treat the generated manifest as the source inventory and `pdf_page` authority; do not create a parallel catalog.
- Do not let the app rewrite Markdown source docs.
- Do not add regenerate behavior without approval.

SaaS Account System

Purpose: Handle public SaaS signup/login/account profile, sessions, security, billing visibility, and onboarding status.

Main files/folders:

- `saas/router.py`
- `saas/service.py`
- `saas/models.py`
- `templates/saas/`

Maturity/status: M1-M5 foundation implemented.

Related docs/ADRs:

- `docs/adr/0002-separate-saas-identity-and-operational-users.md`
- `docs/history/saas-onboarding/`

Risks/guardrails:

- Do not merge SaaS account identity with operational tenant users.
- Do not bypass verification/security/session boundaries.

SaaS Onboarding

Purpose: Collect organization, contact, branch, academic setup, and review data before provisioning.

Main files/folders:

- `saas/router.py`
- `saas/service.py`
- `templates/saas/onboarding_*.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/history/saas-onboarding/`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- Pending onboarding data is not the same as operational tenant data.
- Do not create live tenant records directly from public forms.

SaaS Demo Request Workflow

Purpose: Offer verified customers a post-onboarding choice between the unchanged subscription workflow and a reviewed customer-demo workspace.

Main files/folders:

- `demo_workflow.py`
- `saas/demo_request_service.py`
- `saas/demo_eligibility_maintenance_service.py`
- `saas/demo_provisioning_service.py`
- `saas/demo_conversion_service.py`
- `saas/provisioning_service.py`
- `saas/router.py`
- `templates/saas/commercial_choice.html`
- `templates/saas/demo_request_status.html`
- `templates/saas/admin_demo_requests.html`
- `templates/saas/admin_demo_request_detail.html`
- `templates/saas/admin_demo_eligibility_maintenance.html`
- `templates/saas/admin_delete_demo_eligibility.html`

Maturity/status: M8B-6 implemented. Submission, review, atomic provisioning, activation, seven-day lifecycle resolution, Day 6 internal reminders, Day 7 expiration, server-side expired-access enforcement, and provider-confirmed conversion of the existing demo tenant to Customer Paid are available. Manual extension, internal-sandbox conversion, archive/delete, and email delivery are not implemented.

Risks/guardrails:

- Do not confuse SaaS demo requests with legacy public marketing demo leads.
- Approval remains review evidence only; a separate Platform Owner provisioning action creates the workspace.
- Provisioning requires coherent approval, customer-demo intent, commercial state, entitlement snapshot, and organization data.
- Demo provisioning must not create or infer paid subscription evidence.
- Failed provisioning must roll back workspace records and preserve the Approved request for a controlled retry.
- `activated_at` is the only lifecycle clock authority; approval and submission timestamps never determine expiration.
- Expiration preserves tenant data and blocks all normal operational access rather than creating a read-only mode.
- Paid workspaces, internal sandboxes, and Platform Console access must never pass through demo expiration enforcement.

- Demo-to-paid conversion requires an active coherent demo and a confirmed M7 subscription for the same organization.
- Conversion preserves the SchoolGroup and tenant link row; it must never invoke operational reprovisioning.
- A failed workspace conversion preserves confirmed payment evidence, rolls back workspace mutations, records failure, and remains retryable.
- Completed conversions must not re-enter demo reminder or expiration processing.
- Historical eligibility maintenance is Platform Owner-only, deletes by exact eligibility ID only, and must fail closed on any organization, account, request, workspace, provisioning, subscription, conversion, or manual-review evidence.
- Eligibility maintenance must not replace, bypass, or weaken customer demo eligibility or organization-scoped clean-room reset.

Main M8B-5 and M8B-6 files:

- `saas/demo_lifecycle_service.py`
- `saas/demo_conversion_service.py`
- `scripts/process_demo_lifecycle.py`
- `authorization.py`
- `templates/demo_access_blocked.html`
- Subscribe Now must continue through the existing provider-authoritative billing path.
- Only Platform Owners may review, reject, approve, or cancel requests.
- Customer withdrawal is valid only while Pending Review.

Workspace Classification Foundation

Purpose: Provide stable, constrained metadata for distinguishing internal sandbox, customer demo, and customer-paid workspaces without changing commercial or customer behavior.

Main files/folders:

- `workspace_classification.py`
- `saas/workspace_classification_service.py`
- `saas/workspace_classification_admin_service.py`
- `saas/commercial_state_service.py`
- `scripts/diagnose_workspace_classification.py`
- `scripts/backfill_workspace_classification.py`

Maturity/status: M8B-6 commercial transition implemented only for the dedicated active Customer Demo to Customer Paid workflow. Classification, lifecycle, workspace entitlement, branch entitlement, and effective commercial state remain centrally resolved. Other classification conversions remain prohibited.

Related docs/ADRs:

- `docs/adr/0008-workspace-classification-foundation.md`
- `docs/history/workspace-classification/README.md`
- `docs/adr/0009-commercial-state-and-entitlement-resolution.md`

Risks/guardrails:

- Do not use classification as a payment, entitlement, permission, tenant-isolation, or reset gate in M8B-1.
- Permit classification conversion only through the dedicated M8B-6 demo-to-paid service.
- Do not expose workspace metadata outside Platform Owner views.
- Do not infer customer-paid classification from incomplete onboarding or provider records.
- Keep paid plan capabilities authoritative in the existing M7 subscription entitlement resolver.
- Fail closed on missing, ambiguous, orphaned, stale, or cross-tenant entitlement relationships.
- Do not use M8B-2 results for tenant authorization or feature enforcement yet.

Commercial Entitlement Resolution

Purpose: Resolve effective workspace entitlement, branch commercial activity, and commercial state from persisted local evidence without modifying rows or calling Paddle.

Main files/folders:

- `commercial_entitlements.py`
- `saas/commercial_validation_service.py`
- `saas/workspace_entitlement_service.py`
- `saas/branch_entitlement_service.py`
- `saas/commercial_state_service.py`
- `saas/models.py`

Maturity/status: M8B-2 read-only foundation implemented. No customer-facing enforcement, demo expiration, billing mutation, or conversion orchestration.

Pending Organizations

Purpose: Represent the Platform Owner work queue for organizations still waiting on setup, review, payment, or incomplete/recoverable provisioning, while preserving completed and historical records separately.

Main files/folders:

- `saas/models.py`
- `saas/service.py`
- `saas/router.py`
- `templates/saas/admin_pending_organizations.html`
- `templates/saas/admin_pending_organization_detail.html`

Maturity/status: Implemented with lifecycle-aware pending filtering and retained Organization Records.

Related docs/ADRs:

- `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

Risks/guardrails:

- Keep pending state separate from provisioned tenant state.

- Do not count a row as pending when a tenant link, completed provisioning job, or final tenant billing state exists.
- Resolve active tenant display from coherent payment, active subscription, contract, tenant-link, and active SchoolGroup evidence; conflicting completed evidence must fail closed to Lifecycle Review Required.
- Preserve raw onboarding fields as history rather than rewriting them solely for owner presentation.
- Preserve platform owner review and auditability.

Plans And Pricing

Purpose: Define SaaS plan catalog and pricing behavior.

Main files/folders:

- `saas/pricing_service.py`
- `saas/router.py`
- `saas/entitlement_service.py`
- `saas/subscription_portal_service.py`
- `saas/subscription_change_service.py`
- `saas/subscription_plan_change_service.py`
- `saas/subscription_cancellation_service.py`
- `saas/subscription_lifecycle_service.py`
- `templates/saas/plan_catalog.html`
- `templates/saas/plan_selection.html`
- `templates/saas/subscription.html`

Maturity/status: M7 implemented: entitlement catalog, customer portal, quantity and plan management, scheduled changes, cancellation/reversal, and centralized lifecycle/action policy.

Related docs/ADRs:

- `docs/history/subscriptions/`
- `docs/adr/0003-paddle-payment-architecture.md`

Risks/guardrails:

- Plan changes must update docs and change history.
- Do not hard-code provider behavior outside service boundaries.

Paddle / Payment

Purpose: Integrate subscription checkout, payment state, billing status, and provider events.

Main files/folders:

- `saas/paddle_client.py`
- `saas/payment_service.py`
- `saas/billing_service.py`
- `saas/currency_service.py`

- `saas/router.py`
- `saas/billing_history_service.py`
- `saas/payment_lifecycle_reconciliation_service.py`

Maturity/status: Implemented through M7, including provider-authoritative previews/proration, billing history, invoice access, webhook idempotency, fail-closed reconciliation, diagnostics, and guarded repair tooling.

Related docs/ADRs:

- `docs/adr/0003-paddle-payment-architecture.md`
- `docs/adr/0004-webhook-only-payment-confirmation.md`
- `docs/history/subscriptions/`

Risks/guardrails:

- Webhook-confirmed payment state is authoritative.
- Checkout return alone must not trigger verified payment/provisioning.
- TIS must not calculate replacement financial outcomes when Paddle preview/transaction data is authoritative.
- Scheduled provider changes must not become local entitlement truth before verified effective evidence.
- Invoice URLs must be freshly resolved and must not be stored.

Provisioning

Purpose: Convert verified/approved pending organizations into operational tenant structures.

Main files/folders:

- `saas/provisioning_service.py`
- `saas/router.py`
- `templates/saas/admin_provisioning.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`
- `docs/history/provisioning/`

Risks/guardrails:

- Keep provisioning recoverable and reviewable.
- Do not merge tenants or create records in the wrong school group.

Operational Login

Purpose: Authenticate operational users and route them into platform or tenant context.

Main files/folders:

- `main.py`
- `auth.py`

- `templates/login.html` if present through root templates

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- Platform users and tenant users follow different context behavior.
- Preserve permission and active-user checks.

Dashboard

Purpose: Give tenant users operational visibility into staffing, planning, reports, and current school context.

Main files/folders:

- `main.py`
- `templates/dashboard.html`
- related report/export helpers

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- `docs/history/workforce-planning/`

Risks/guardrails:

- Dashboard data must remain scoped to tenant/branch/year context.

Organizations / School Groups

Purpose: Represent tenant organizations and top-level school group context.

Main files/folders:

- `models.py`
- `main.py`
- `templates/system_configuration_schools.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Tenant isolation starts at school group boundaries.

Branches

Purpose: Represent campuses/branches inside a school group.

Main files/folders:

```
models.py
main.py
templates/system_configuration_branches.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
docs/TIS_MASTER_CONTEXT.md
```

Risks/guardrails:

Branch scope affects teachers, users, planning, timetable, calendar, branding, and reports.

Academic Years

Purpose: Scope operational data by academic year and active-year context.

Main files/folders:

```
models.py
main.py
templates/system_configuration_years.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
docs/TIS_MASTER_CONTEXT.md
```

Risks/guardrails:

Do not mix data across academic years.

Users And Roles

Purpose: Manage tenant users, platform users, roles, profile data, active status, and branch/year context.

Main files/folders:

```
auth.py
routers/users.py
models.py
templates/users.html
templates/edit_user.html
```

Maturity/status: Implemented with platform identity refinements.

Related docs/ADRs:

```
docs/adr/0002-separate-saas-identity-and-operational-users.md
```

Risks/guardrails:

- Do not assign owner controls to developers or tenant users.
- Preserve role normalization.

Permissions

Purpose: Control route/module actions by permission key and role package.

Main files/folders:

- `authorization.py`
- `permission_registry.py`
- `role_permission_service.py`
- `auth.py`
- `templates/system_configuration_role_permissions.html`

Maturity/status: Implemented and critical.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Do not bypass `authorization.enforce_route_permission`.
- Do not rely only on UI hiding for protected actions.

Teachers

Purpose: Manage teacher records, qualifications, capacity, workloads, and profile-related academic staffing data.

Main files/folders:

- `routers/teachers.py`
- `teacher_qualifications.py`
- `teacher_capacity.py`
- `models.py`
- `templates/teachers.html`
- `templates/edit_teacher.html`

Maturity/status: Implemented and central to operational value.

Related docs/ADRs:

- `docs/history/workforce-planning/`

Risks/guardrails:

- Teacher data must remain tenant/branch/year scoped.

Subjects

Purpose: Manage subject catalog, colors, qualifications, and planning/timetable relationships.

Main files/folders:

```
routers/subjects.py
subject_colors.py
models.py
templates/subjects.html
templates/edit_subject.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

Subject changes can affect planning, timetable, teacher qualifications, and reports.

Sections

Purpose: Represent class/section structures used by planning and timetabling.

Main files/folders:

```
routers/planning.py
models.py
templates/planning.html
```

Maturity/status: Implemented through planning workflows.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

Section changes can affect timetable allocation and workload reporting.

Workforce Planning

Purpose: Plan assignments, homeroom ownership, workloads, subject coverage, and staffing needs.

Main files/folders:

```
routers/planning.py
teacher_capacity.py
homeroom_defaults.py
templates/planning.html
```

Maturity/status: Implemented and evolving.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

- Planning logic must preserve branch/year scope and avoid accidental cross-year copies.

Timetable

Purpose: Place planned lessons into weekly timetable grids and exports.

Main files/folders:

- `routers/timetable.py`
- `timetable_logic.py`
- `timetable_version_service.py`
- `timetable_snapshot_service.py`
- `timetable_slot_service.py`
- `timetable_readiness_service.py`
- `timetable_publication_service.py`
- `templates/timetable.html`
- `templates/system_configuration_timetable.html`

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- `docs/history/workforce-planning/`
- `docs/adr/0026-versioned-constraint-based-smart-timetable-generation.md`

Risks/guardrails:

- Timetable rules interact with planning, sections, subjects, and teacher capacity.
- Planning remains demand/assignment/HRT authority. Placements are version-owned;

the exact-scope active pointer is separate, active/superseded history is immutable, and legacy editing uses a copy-on-write working draft.

- Stage 3.5 owns the canonical composed per-day timeline and exact-scope structural readiness.

The composer outputs ordered teaching/block items and valid teaching slots; after-period blocks shift later clocks while invalid fixed-time placement fails closed. It has no solver, worker, availability, room/resource, or Generate UI.

- Stage 4 owns draft validation, publication transactions, same-scope comparison,

lifecycle presentation, explicit version exports, archive, and draft lock actions.

Academic Calendar

Purpose: Manage academic events, responsibilities, dates, exports, and branch/year scoped calendar views.

Main files/folders:

- `routers/academic_calendar.py`
- `templates/academic_calendar.html`

- `templates/system_configuration_calendar.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/history/academic-calendar/`

Risks/guardrails:

- Calendar permissions and branch/year scoping must remain intact.

Observations

Purpose: Support teacher observations, feedback, evidence, scoring, history, and supervision records.

Main files/folders:

- `routers/observations.py`
- `templates/observations.html`
- `templates/observation_form.html`
- `templates/observation_detail.html`
- `templates/observation_history.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Observation records are sensitive and must remain tenant scoped.

Reports / Dashboards

Purpose: Expose operational summaries, exports, allocation reports, and decision visibility.

Main files/folders:

- `main.py`
- `tenant_report_branding.py`
- report/export helper code
- `templates/dashboard.html`

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- `docs/history/workforce-planning/`

Risks/guardrails:

- Report data must be permission-checked and tenant scoped.
- Tenant-facing exports may use configured branch/organization logos only. Missing

tenant branding produces no logo and must never fall back to the TIS product mark.

Branding / Design Settings

Purpose: Manage organization logos, design settings, visual shell behavior, and platform visual controls.

Main files/folders:

```
branding_storage.py
design_tokens.py
tenant_report_branding.py
visual_design.py
ui_shell.py
static/css/branding.css
static/css/design-studio.css
templates/system_configuration_logos.html
templates/system_configuration_design.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
docs/adr/0007-landing-page-visual-system-strategy.md
```

Risks/guardrails:

- Do not confuse operational app branding with public landing website design.
- Protect uploaded/owned assets.
- Keep product-surface branding separate from tenant-facing report branding.

Landing Website

Purpose: Public marketing and conversion surface for TIS.

Main files/folders:

```
tis-landing-website/
docs/marketing/
```

Maturity/status: Implemented as separate Next.js app.

Related docs/ADRs:

```
docs/adr/0001-separate-nextjs-landing-website.md
docs/adr/0007-landing-page-visual-system-strategy.md
docs/marketing/landing_page_source_of_truth.md
```

Risks/guardrails:

- Do not modify landing code during operational/backend tasks unless explicitly approved.

1 Legacy FastAPI landing files are not the public source of truth.

AI Future Roadmap

Purpose: Future subscription-gated AI-assisted planning, analytics, assessment generation, recommendations, and decision support.

Main files/folders:

1 No dedicated AI production module yet.

1 Future work should be documented before implementation.

Maturity/status: Roadmap / future.

Related docs/ADRs:

1 `docs/TIS_MASTER_CONTEXT.md`

1 future ADRs required before major AI architecture decisions.

Risks/guardrails:

1 AI features must use verified tenant data and preserve privacy, permissions, and subscription boundaries.

Organization Analytics permission-scoped availability correction (2026-09-12)

1 `talent_organization_analytics_providers.PermissionScopedOrganizationAnalyticsAvailabilityProvider`

is the production availability provider for all seven M10 routes.

1 `feature.organization_intelligence` may remain in the generic feature

catalog for compatibility/history, but Organization Analytics does not call `organization_feature_available` or any commercial-state/entitlement resolver.

1 `talent_org_intelligence_service.resolve_access_context` still enforces

active-user, SchoolGroup, `talent_analytics.view`, Academic Year, Branch scope, and secondary sensitive-data permissions independently from privacy and breadth providers.

TIS Repository Architecture

docs/engineering/REPOSITORY_ARCHITECTURE.md

Published Timetable Consumption Boundary

`timetable_visibility_service.py` validates exact scope and resolves official data only through `TimetableActiveVersion`; it never uses operational draft resolution. `routers/timetable.py` retains management workflow, while separately mounted `/my-timetable` and `/published-timetable` routes expose filtered published payloads. `timetable_version_service.py` owns locked, history-preserving working discard.

Smart Timetable Generation Boundary

`routers/timetable.py`, `timetable_workflow_dispatch.py`, and `timetable_generation_service.py` are the web-safe enqueue/dispatch/status/cancel boundary and never import OR-Tools. `timetable_problem_builder.py` builds solely from captured snapshot JSON. `timetable_generation_workflow.py` registers the on-demand Render task and imports the reusable single-run executor from `timetable_generation_worker.py`; its dependencies are in `requirements-workflow.txt`. The polling worker command remains local/fallback only.

The web commits a durable queued run before requesting one Render task and marks an unclaimed run terminal if dispatch fails. The task claims only that public ID, maintains the existing lease and heartbeat, solves, and asks the independent validator to verify it. Persistence rechecks current input/source revision, then creates the version, entries, linkage, and success in one transaction. The browser polls only TIS state. Render retries are explicitly zero; duplicate invocation is a safe no-op after the PostgreSQL claim or terminal result.

Customer Feature Policy Boundary

`saas/customer_feature_policy.py` classifies the normal organization product baseline and explicitly excludes internal audit capability. `saas.entitlement_service.evaluate_feature_access` composes existing identity, permission, commercial-state, workspace-entitlement, branch-entitlement, and scope authorities. Routes and templates consume this one decision rather than branching on paid, promo, demo, or plan names.

`saas/ai_entitlement_service.py` consumes that availability decision. `saas/ai_consumption_policy.py` separately returns metric context and usage limit, preserving reservations and usage accounting. This split is intentionally small while no provider-backed AI route exists.

Migration and activation writes remain in `db_migrations.py` and `saas/promo_redemption_service.py`; feature policy never owns capacity, payment, permission assignment, or immutable promo evidence.

SchoolGroup Configuration Boundary

`main.py` owns the operational School Management route boundary. Direct top-level SchoolGroup creation and deletion require Platform identity plus `schools.manage_all_schools` and the matching action permission; tenant updates are limited to the linked SchoolGroup. `permission_registry.py` keeps organization create/delete out of tenant role assignments. Templates consume explicit server-side capabilities and never substitute UI hiding for handler authorization.

School Management branch capacity is a presentation adapter over `saas/commercial_authority_service.py`; it must not call a paid-only resolver or recalculate promo usage. Mutation routes remain the authority for locked capacity enforcement and atomic promo evidence. `scripts/audit_schoolgroup_provenance.py` is an operator-only PostgreSQL read-only diagnostic. It emits no names or contact data, rolls back on exit, and owns no remediation behavior.

Existing Workspace Controlled Conversion Layer

Existing Workspace Paid Activation Boundary

`saas/existing_workspace_paid_activation_service.py` owns eligibility, operational capacity recount, plan/branch quote snapshots, idempotent preparation, workspace Paddle-customer association, transaction launch, and webhook-only activation for an existing SchoolGroup. It reuses `saas/billing_identity_service.py`, `saas/paddle_client.py`, and the centralized commercial resolvers. It never calls the tenant provisioning engine and never creates a PendingOrganization.

`ExistingWorkspacePaidActivation` is the durable authority context. CheckoutSession and PaymentAttempt accept exactly one of the established PendingOrganization context or this context. SchoolGroup-anchored billing profiles, contracts, and subscriptions support the existing workspace without weakening onboarding constraints. Payment-customer workspace associations retain the exact provider address and business lineage used by each SchoolGroup.

The shared Paddle webhook dispatcher recognizes this context before onboarding and subscription-mutation handlers. Only matching `transaction.completed` evidence can create paid WorkspaceEntitlement, BranchEntitlement, PaymentSubscription, and the paid TenantProvisioningLink. Workspace classification remains `customer` and the lifecycle becomes `active`. A transaction is released only after complete provider billed-state and quote-lineage validation. Locking queries refresh mapped state before concurrent idempotency checks.

`saas/existing_workspace_conversion_service.py` owns M4B preparation, verified owner alignment, setup validation, fresh-audit comparison, row locking, idempotency, and the final activation-required transition. It composes M4A, normal SaaS verification, provisioning owner identity conventions, M1, and M3; it does not duplicate those responsibilities. `ExistingWorkspaceConversionOperation` is the durable claim/state ledger and `ExistingWorkspaceConversionEvent` is an append-only redacted event stream.

`scripts/convert_existing_workspace.py` is dry-run by default. PostgreSQL write mode requires Platform Owner actor IDs plus an exact `PREPARE <operation UUID>` or `CONVERT <operation UUID>` phrase. Preparation requires the current complete M4A hash. Final execution performs another M4A audit in the locked transaction and compares stable branch/dependency and sandbox-entitlement evidence to the operation baseline; intended owner and setup changes are the only expected differences. The CLI never creates passwords, sends email, calls Paddle, or redeems promo codes.

Customer routes under `/saas/existing-workspace/` expose only verified claim and three-field setup review. `commercial_state_service`, `commercial_access_service`, and `commercial_authority_service` recognize the specific `customer / provisioning / no source` result as fail-closed `activation_required`. The ordinary subscription portal redirects this state back to Organization Account because existing-workspace Paddle activation is not yet implemented.

Existing Workspace Conversion Audit Layer

`saas/existing_workspace_conversion_audit_service.py` is a query-only evidence collector. It accepts an exact SchoolGroup ID, workspace UUID, organization name, and owner email; reflects the connected schema; follows branch foreign keys recursively; identifies unconstrained branch-like columns; and returns allowlisted identity, lifecycle, ownership, provisioning, and commercial metadata. It does not own or commit a transaction and exposes provider references only as presence flags.

`scripts/audit_existing_workspace_conversion.py` is the manual production entry point. It requires PostgreSQL and `DATABASE_URL`, establishes `REPEATABLE READ READ ONLY`, invokes the service, emits one sanitized JSON or text report, then rolls back and closes. The service canonicalizes ordered evidence into a stable SHA-256 snapshot, classifies soft-deleted rows without ignoring them, compares reflected branch foreign keys with ORM metadata, and emits archival candidates only when each referenced count is zero. The CLI reports transaction

settings and uses exit `0` for coherent evidence, `1` for execution/configuration failure, `2` for manual review, and `3` for workspace identity mismatch. The script is not imported by application runtime. Audit readiness means only that later conversion design may proceed; hard deletion and write conversion remain explicitly unapproved.

Promo Redemption Layer

`saas/promo_redemption_service.py` is the write boundary between secure M2 definitions and operational commercial authority. It accepts either an owned completed `PendingOrganization` or an existing aligned `SchoolGroup` with a tenant-owner account link. The latter path deliberately creates no synthetic onboarding, contract, subscription, payment, or demo row.

Final activation locks session, definition, and workspace in stable order and commits immutable `PromoRedemption`/`PromoGrant`, branch assignments and entitlements, promo `WorkspaceEntitlement`, promo `TenantProvisioningLink`, lifecycle updates, and audit together. `saas/promo_grant_service.py` resolves effective grants and expiration and owns atomic assignment/entitlement activation when a branch is created or reactivated; M1 composes that result. `auth.py` restricts customer-classified operational branch queries to explicit active promo branch entitlements. Pending or inconsistent evidence fails closed.

Existing-branch selection does not alter `Branch.status`. Staff and teachers are organization-wide validation inputs and are never selected, disabled, or deleted. Normal future branch creation may consume only an unused grant branch slot. Paddle remains outside this layer.

`saas/promo_branch_entitlement_reconciliation_service.py` is the conservative repair boundary for legacy incomplete promo evidence. It requires one resolved active promo authority, tenant source, owner, and workspace entitlement; inventories all branches, assignments, and entitlement rows; and can create only a missing inactive entitlement for an unassigned branch. The PostgreSQL CLI defaults to dry-run, locks and revalidates on apply, and commits or rolls back as one unit.

`saas/commercial_portal_service.py` is the read-only customer presentation adapter for promo authority. It consumes `commercial_authority_service` rather than recounting capacity, then resolves the attributable `PromoGrant` and `PromoRedemption` only for immutable dates and the masked promo reference. The SaaS subscription route selects this adapter by authoritative source before invoking the paid portal. Promo, paid, and demo view models remain separate; Jinja templates do not decide commercial authority. The promo view exposes the existing-workspace paid continuation entry only when `existing_workspace_paid_activation_service` proves an expired or recovery-period promo source and verified tenant owner.

`promo_grant_service` derives active, recovery, and expired time states. Recovery is presentation/conversion eligibility only; commercial access remains blocked from the exact expiry timestamp. `existing_workspace_paid_activation_service` owns the locked, provider-confirmed promo-to-paid transition. It keeps promo authority during checkout and atomically moves the existing `TenantProvisioningLink`, workspace entitlement, and branch entitlements to the confirmed paid contract/subscription at completion. No replacement tenant or synthetic onboarding organization is created.

`saas/commercial_badge_service.py` is the sole commercial identity presentation adapter. It accepts centralized commercial access, rejects unresolved placeholder sources, and returns semantic source, status, icon, and plan tokens for the reusable `templates/_commercial_badge.html` component. Organization Account and commercial access use the shared presentation; the operational shell deliberately performs no commercial-badge lookup or rendering.

Promo Definition Layer

Promo administration is an isolated SaaS/Platform layer. The router requires a Platform identity plus granular promo permission; the service owns validation, secure generation, lifecycle locking, and safe audit; models and migration own only definition/history persistence. Templates receive raw code only in the immediate create/duplicate/replace response and all later views use a masked representation.

The M3 commercial adapter contract is implemented as a grant identity, tenant and organization identity, `source=promo`, status, plan, effective window, three capacity limits, selected branch IDs, immutable snapshot, resolution status, and reason code. M2 definition operations still provide no access; only a completed M3 grant is authoritative.

Three-Dimension Subscription Capacity Authority

`saas/branch_pricing_quote_service.py` owns pre-payment plan eligibility and quote capacity across active billable branches, system-user estimates, and teacher estimates. `PendingOrganizationBranch` owns onboarding `estimated_system_users` and `estimated_teachers`; organization-level legacy values are migration inputs and derived compatibility summaries, not an independent source of truth.

Before payment, each people count is the greater of the sum across active onboarding branches and actual active data in the uniquely linked SchoolGroup. After activation, `saas/commercial_authority_service.py` is the single facade. Staff usage is every distinct active tenant operational `User`, including operational owners, teacher-position users, and internal-test-attributed tenant users. Platform users, inactive users, account-only SaaS identities, and other tenants do not count. Teachers are deduplicated by normalized `Teacher.teacher_id` across active branches and active academic years; every blank legacy teacher identity counts separately.

The decision is consumed by selection, quote construction, checkout preparation/launch, Paddle creation, payment reconciliation, and plan changes. Both people counts are part of quote fingerprint authority. Branch estimate changes clear an undersized selected plan, invalidate its quote, mark ready or started checkout sessions stale, and supersede incomplete payment attempts. Operational branch, staff-user, teacher, and academic-year growth locks the SchoolGroup, composes the existing commercial authorities, recounts, evaluates the proposed final state, and mutates in the same transaction. Paid and demo provisioning perform the same final invariant. Downgrade impact still reports branch, system-user, and teacher conflicts independently. No capacity failure silently deletes or deactivates operational data.

Paid limits are provider-confirmed subscription quantity capped by the plan branch ceiling plus the plan's staff and teacher limits. Customer Demo and Internal Sandbox are explicitly unmetered in M1 when their existing authority is coherent. Missing or contradictory authority fails closed. Existing over-capacity tenants preserve data and access but cannot further increase an exceeded dimension.

The current `Teacher` model has no inactive/reactivation lifecycle, and the repository has no teacher import endpoint. Capacity enforcement therefore covers the supported teacher-growth paths: individual creation and atomic academic-year copying. Adding teacher deactivation/reactivation or import workflows remains a separate lifecycle design task.

Starter persists 1 branch/5 system users/25 teachers, Professional 5/20/100, and Enterprise AI 25/100/500. The lowest plan passing all dimensions is recommended while any higher eligible plan remains selectable. The in-app Custom card is informational and mail-based; exceeding any Enterprise limit emphasizes it, but it creates no catalog plan, Paddle transaction, or enterprise-request workflow.

Subscription Plan Branch-Capacity Authority

`saas/branch_pricing_quote_service.py` evaluates the authoritative active billable branch count against `SubscriptionPlan.max_branches`. Starter, Professional, and Enterprise AI retain persisted capacities of one, five, and twenty-five. The evaluator supplies both server-side rejection and plan-page eligibility metadata; pricing remains the unit price multiplied by the actual active count.

Plan selection and every checkout/payment phase consume a fail-closed quote, so an undersized plan cannot produce a checkout or Paddle transaction. A pre-payment branch change clears an undersized selection while preserving an eligible higher plan, and the existing checkout-supersession boundary rejects stale transaction completion. Counts are resolved only inside the current pending organization or its same-workspace demo provisioning.

Paddle Checkout Quantity Authority

`saas/branch_pricing_quote_service.py` supplies the authoritative billable branch quantity and total to `saas/payment_service.py`. The payment service sends that exact quantity with the selected catalog price and resolved customer address when creating an automatically collected Paddle transaction. The transaction must reach `ready`; returned item quantity, provider price, subtotal, and quote fingerprint are validated before TIS marks it `billed`. Only the resulting immutable transaction is released to checkout.

The public `/saas/payment` launcher opens the pre-created transaction with Paddle's inline one-page checkout settings and `transactionId`; it never passes an editable items array. The billed transaction is a fixed financial record, so item and quantity changes are rejected by Paddle. Automatic `transaction.completed` webhook evidence remains the recurring-subscription and conversion authority.

Pre-payment Branch Setup remains authoritative even when an unpaid demo workspace or tenant link already exists. Only confirmed payment evidence or an active paid subscription closes onboarding mutation. A branch identity or count change marks ready/started checkout sessions stale, marks their incomplete payment attempts superseded, clears quote snapshots, and returns the organization to plan selection. Late events for superseded transactions are retained for manual review but cannot change the current organization, subscription, provisioning, or workspace state.

Before rendering Paddle.js, the launcher requires exactly one local payment attempt and matching checkout session, organization, customer, quote, and provider transaction. Remote status must be `billed` with automatic collection; draft, ready, canceled, past-due, unrelated, or mismatched transactions fail closed with a customer-safe message.

Checkout Tenant-Identity Recovery Boundary

`saas/payment_service.py` owns Paddle customer identity resolution. For an existing demo workspace, a stale `SaaSAccountUserLink` may be reassigned to the authenticated account only when organization ownership, demo request and provisioning source, the unique `TenantProvisioningLink`, SchoolGroup, exact operational owner, and email identity all agree. The current account must have no conflicting workspace link, and any previous linked account must be absent or inactive. Missing, unrelated, active-previous-owner, cross-workspace, or multiple identity evidence fails closed.

This recovery does not select a Paddle customer by email in live mode, bypass provider confirmation, provision a tenant, or create another SchoolGroup. Internal match diagnostics are logged; customer routes receive only retry and support guidance.

Returning Customer And Commercial-Access Boundary

`saas/customer_journey_service.py` owns customer login destinations and the expired-demo subscription projection. It reads onboarding/request records, the durable SaaS-account operational link, demo lifecycle, SchoolGroup classification, actual active Branch rows, public plans, and subscription evidence. It also resolves linked organization-account management scope from the existing SaaSAccountUserLink, operational user, ownership, and permission records. Activated account managers land on `/saas/account`; ambiguous multi-organization identities select an organization there. The selected UUID is an HTTP-only request hint that is revalidated against SaaSAccountUserLink, the operational user, ownership, and permissions before `saas.entitlement_service` may resolve that SchoolGroup. Linked users with no account-management permission retain the existing role-based operational destination. `saas/commercial_access_service.py` is the operational access projection used after credential validation, by customer journey routing, and by request authorization. For Customer Paid workspaces it consumes the existing `saas.entitlement_service` tenant-link and SubscriptionContract-linked subscription resolution; it never chooses the newest organization-level row. It adds workspace lifecycle, pending-change context, state-specific messaging, and the allow/deny decision without replacing payment, entitlement, plan-change, or demo-lifecycle authorities. Terminal or unresolved plan-change requests do not replace the current paid entitlement.

`saas/router.py` applies that resolver after password and social login, for an already-authenticated sign-in page, and from the SaaS root. The Organization Account Overview is an account-management boundary, not an operational bypass: it filters sections by existing permissions, suppresses operational entry when commercial access is restricted, and exposes `/login` only through Enter TIS Platform. Operational authentication and request authorization remain separate authorities.

`/saas/subscription/demo/select` records conversion intent and reuses existing plan selection and checkout routes; it never provisions or copies a second workspace. Public navigation remains in `tis-landing-website/` and builds app routes exclusively from `NEXT_PUBLIC_TIS_APP_BASE_URL`.

M8B9 Demo Operations Boundary

The owner HTTP surface delegates mutations to `saas/demo_operations_service.py`, which validates and locks the Customer Demo context, calls existing lifecycle and communication services, and writes audit state. Access decisions resolve through `saas/demo_access_service.py` before the M8B8 counter. Workspace policy is the default; branch policy requires same-tenant validation. M8B9 DDL runs only through `python scripts/run_migrations.py` during pre-deploy.

M8B8 AI Entitlement Boundary

AI route code must never inspect classifications, plans, or usage rows directly. It requests a decision from `saas.ai_entitlement_service`, performs the real AI operation only when allowed, and records consumption only after a usable result. The registry is code-controlled configuration. Commercial state and permissions remain upstream authorities; accounting is downstream and tenant-scoped.

M8B7 Communication Boundaries

Demo email intent creation and dispatch belong to `saas/demo_email_service.py`; Platform Owner Notification Center creation belongs to `saas/demo_notification_service.py`. Routes orchestrate existing services but do not own lifecycle calculations. Provider calls occur outside lifecycle locks. The shared shell consumes the read-only lifecycle resolver and never becomes lifecycle authority.

This document explains the main repository areas, what each owns, and what must not be changed casually.

Root FastAPI Application

`main.py`

Responsibility: Primary FastAPI app, route registration, many app-level workflows, middleware, startup checks, platform console routes, dashboards, exports, system configuration, scope switching, and operational pages.

Render boot boundary: The ASGI web process never performs SQLAlchemy metadata creation or pending migrations. `scripts/run_migrations.py` is the sole deployment command for baseline schema creation plus `db_migrations.run_pending_migrations`; Render runs it as the Pre-Deploy Command and activates the new version only after exit code zero. `main:app` startup retains only security configuration validation and process-local upload-directory creation, so Uvicorn binds without PostgreSQL DDL. There is no migration daemon or schema-readiness middleware.

Do not change casually:

1 authentication/session flow,

1 platform owner access checks,

1 tenant scope handling,

1 route behavior shared by many modules,

- startup/schema repair behavior.

auth.py

Responsibility: Password hashing/verification, session cookies, current-user lookup, role normalization, platform identity helpers, permission helpers, email verification helpers, and tenant/platform identity boundaries.

Do not change casually:

- `is_platform_owner`, `is_platform_user`, `is_platform_developer`,
- role normalization,
- session cookie behavior,
- permission helper semantics.

authorization.py

Responsibility: Protected route rules, permission enforcement middleware integration, access-denied responses.

Do not change casually:

- route permission mapping,
- public path patterns,
- permission matching semantics.

database.py

Responsibility: Database engine/session setup.

Do not change casually:

- database URL handling,
- session creation behavior,
- engine configuration.

models.py

Responsibility: Primary operational SQLAlchemy models for users, school groups, branches, teachers, planning, timetable, observations, configuration, and related app data.

Do not change casually:

- tenant ownership fields,
- platform user fields,
- relationships used by existing workflows,
- schema without matching migration/repair strategy.

db_migrations.py

Responsibility: Local schema migration and repair logic used by the app.

Do not change casually:

- migration ordering,

- destructive schema operations,
- production-sensitive repair behavior.

Timetable Version And Snapshot Services

`timetable_version_service.py` owns exact-scope version resolution, draft creation/copying, mutable-state enforcement, placement mutation, lock metadata, archival foundation, next-number allocation, and imported active-pointer setup. It does not own Planning demand rules.

`timetable_snapshot_service.py` converts current Planning/settings/lock authority into deterministic schema-versioned canonical JSON and SHA-256 component/full fingerprints without teacher display names or other unnecessary personal data. It is a reusable foundation, not a readiness checker or solver.

`timetable_slot_service.py` composes the canonical per-day teaching/block timeline used by snapshots and operational consumers. `timetable_readiness_service.py` performs a read-only exact-scope completeness evaluation using Planning, HRT, teacher capacity, slots, locks, and existing fingerprints. Neither service executes a solver.

`timetable_publication_service.py` owns version-specific hard validation, same-scope comparison, and atomic active-pointer publication. Routes enforce permissions and explicit scope; the service enforces lifecycle, revisions, freshness, hard validity, and transaction invariants. It never generates placements.

`permission_registry.py`

Responsibility: Permission groups, labels, default role permissions, system owner permissions, developer-assignable permission boundaries.

Do not change casually:

- owner/developer permissions,
- defaults for managed roles,
- permission keys referenced by routes/templates/tests.

`role_permission_service.py`

Responsibility: Persistence and service helpers for role permission rows.

Do not change casually:

- global vs school-scoped permission behavior,
- role permission constraints.

`ui_shell.py`

Responsibility: Shared application shell, navigation, page metadata, scoped organization/year context display, logos, visual design CSS, and permission-based nav generation.

Do not change casually:

- navigation visibility,
- platform-vs-tenant shell behavior,
- design-studio gating,
- scope display.

`knowledge_service.py` And `kms_catalog.py`

Responsibility: `knowledge_service.py` reads the generated KMS manifest and approved Markdown metadata for the owner-only Knowledge Center. `kms_catalog.py` provides the dependency-free approved category/module vocabulary and deterministic source-path classification shared by the service and KMS validation tooling.

Do not change casually:

- owner-only read behavior,
- manifest freshness semantics,
- approved category or module slugs,
- path classification without matching KMS tests and documentation.

Feature Routers: `routers/`

Responsibility: Modular operational route handlers:

- `routers/users.py`
- `routers/teachers.py`
- `routers/subjects.py`
- `routers/planning.py`
- `routers/timetable.py`
- `routers/academic_calendar.py`
- `routers/observations.py`

Do not change casually:

- tenant/branch/year scoping,
- permission checks,
- import/export behavior,
- bulk operations.

SaaS Package: `saas/`

Responsibility: Public SaaS account flows, onboarding, plans, billing, Paddle integration, pending organizations, and provisioning.

Key files:

- `saas/router.py`
- `saas/service.py`
- `saas/models.py`
- `saas/pricing_service.py`
- `saas/payment_service.py`
- `saas/paddle_client.py`
- `saas/billing_service.py`
- `saas/provisioning_service.py`

- `saas/currency_service.py`
- `saas/oauth.py`
- `saas/entitlement_service.py`
- `saas/subscription_portal_service.py`
- `saas/subscription_change_service.py`
- `saas/subscription_plan_change_service.py`
- `saas/subscription_cancellation_service.py`
- `saas/subscription_lifecycle_service.py`
- `saas/billing_history_service.py`
- `saas/payment_lifecycle_reconciliation_service.py`

Do not change casually:

- identity separation,
- payment confirmation rules,
- provisioning readiness,
- Paddle/webhook boundaries,
- public signup/onboarding state transitions.

Templates: `templates/`

Responsibility: Jinja templates for operational app, platform console, Knowledge Center, SaaS pages, system configuration, and feature workflows.

Do not change casually:

- form action routes,
- hidden scope fields,
- permission-dependent controls,
- app shell extension patterns.

Static Assets: `static/`

Responsibility: Operational CSS/JS, images, branding assets, generated documentation PDF and manifest.

Important:

- `static/docs/TIS_Project_Reference_Booklet.pdf` is a generated snapshot.
- `static/docs/docs_manifest.json` is generated metadata.

Do not change casually:

- generated docs manually,
- shared CSS without checking all templates,
- protected document access assumptions.

Tests: `tests/`

Responsibility: Regression coverage for SaaS phases, tenant isolation, platform access, permissions, email, branding, and critical workflows.

Do not change casually:

- tests should follow behavior, not hide regressions.
- update tests when behavior intentionally changes.

Docs: `docs/`

Responsibility: KMS source of truth, engineering handbook, ADRs, change history, module history, AI context, marketing docs.

Do not change casually:

- historical records should preserve old/new state.
- update docs through the KIA process.

Scripts: `scripts/`

Responsibility: Maintenance, diagnostics, and governance scripts. Important examples:

- `scripts/generate_docs_pdf.py`: generate or read-only validate KMS PDF/manifest artifacts.
- `scripts/check_kms_impact.py`: compare KIA declarations, Git changes, major-path classification, and artifact freshness.
- `scripts/kms.py`: supported Phase 6 command surface for one-step synchronization and complete read-only validation; delegates to the generator and checker.
- `scripts/sync_paddle_price_ids.py`: environment-specific initial checkout price mapping.
- `scripts/diagnose_paddle_plan_preview.py` and `scripts/diagnose_payment_lifecycle.py`: safe subscription/payment diagnostics.
- `scripts/reconcile_finalized_payment_lifecycle.py`: guarded sandbox reconciliation from attributable provider evidence.

Do not change casually:

- PDF generator dependency assumptions,
- source list behavior,
- manifest metadata.
- title, catalog, navigation-link, source-inventory, and PDF page-bound validation.

Repository Governance

Responsibility: Make KMS impact visible and enforceable without rewriting documentation.

Key files:

- `AGENTS.md`
- `.kms-impact.yml`

- `.github/pull_request_template.md`
- `.github/workflows/kms-enforcement.yml`
- `.github/workflows/deploy-on-master.yml`

Do not change casually:

- major-path classification,
- explicit no-impact override requirements,
- authoritative Markdown/path rules,
- deployment dependency on KMS validation,
- prohibition on customer/runtime/secrets data in documentation.

Next.js Landing Website: `tis-landing-website/`

Responsibility: Public marketing website at `https://tisplatform.com`.

Do not change casually:

- landing implementation during backend tasks,
- visual system without approval,
- public assets/copy without checking marketing docs and ADRs.

TIS User And System Flows

docs/engineering/USER_AND_SYSTEM_FLOWS.md

Agent 3 filtered analysis flow (2026-09-25)

1 Open Overview for distinct current Student headlines and executive charts,

Programs for searchable configuration cards, or Student Assessment for individually authorized operational records.

1 In Results & Analytics select shared Branch, Grade, Section, Program,

Evaluation Period and Classification filters. All means all within authorization, never beyond the global Branch ceiling. Parent changes clear dependent selections; bounded requests and generation guards prevent stale publication.

1 One backend read-only repeatable snapshot builds the selected population,

canonical current attempts and privacy projections. Organization percentages use underlying counts. Compare one dimension and up to six groups. Rubric selection is Program, then Competency, then Indicator.

1 Classification-filtered Learning Style represents the same cohort. If its count

would expose a protected Classification cell, show the protected-cohort explanation, not zero/no-data, geometry or exact values. Individual authorized roster rows remain exact and are not cohort-suppressed.

1 Switch valid chart modes without fetching again. Exact tables are keyboard-

accessible; protected values are absent from hidden/ARIA/tooltip content as well as visible charts. Ordered periods do not imply growth.

Results & Analytics Filter Flow (Production Follow-up Part 2, 2026-09-25)

Changing any dashboard filter (Branch, Grade, Section, Program, Evaluation Period, Classification, Competency, Indicator, comparison dimension or groups) updates local state and the URL with `history.replaceState`, cancels any in-flight refresh, and fetches the same authorized dashboard read in the background. The current analysis stays visible (region `aria-busy`, thin progress bar) and only the newest response replaces it; the reader's scroll position and focused control are preserved and a failure shows an inline Retry while keeping the previous analysis. Changing a chart type only re-renders the chart client-side. Selected comparisons (dimension, up to six groups, results) live in one section at the bottom. Progress Over Time opens per Program from the Results navigation or the Evaluation Periods section.

Global Branch -> Talent Flow (Batch 1 Closure, 2026-09-25)

> Amended 2026-09-25 (Part 1 production follow-up): the sidebar Branch switch lists real Branches only > and no longer offers a Talent All Branches option. On `/talent` pages the validated sidebar Branch is > the DEFAULT page Branch for an organization-authorized actor, who may choose All Branches or another > authorized Branch in the Talent Branch filter (URL marker `branch_scope=all`); a Branch-limited actor > stays locked to their Branch (`branchLocked`). The API still validates every `branch_id`. Start > Assessment failures show a curated, code-specific reason inline beside the Student row.

Supersedes step 2 of the Batch 1 flow below ("default ... honors explicit `branch_scope=all`").

1 Sidebar Branch switch (`/scope/branch`, session scope) sets the working Branch;

on `/talent` pages the switcher also offers "All Branches (Talent & Potential organization-wide)" (marker cookie `branch_scope=all`; other modules unaffected).

1 `/talent/{view}` publishes `tp-config.branch` = the ceiling Branch (empty only for

the explicit global All Branches). The client forces `branch_id` to it and offers no other Branch or All Branches.

1 Every Talent API recomputes the ceiling from the authenticated request user

(`talent_branch_scope`), so an explicit other Branch is rejected (400/403/404) and an omitted Branch resolves to the ceiling; only the explicit global All Branches returns organization-wide data and cross-Branch comparison.

Student Deletion, Branch Scope And Talent Loading Flows (Deployment Acceptance Batch 1, 2026-09-24)

1 **Permanent Student deletion.** `students.delete` (Organization scope) previews

blockers; without history the Student is deleted with its identifiers/audit. With Placement/Talent history normal and bulk Delete are blocked (409); an actor who also holds `students.force_delete_history` and confirms deletes the Student plus all ten Student-owned tables in one transaction. Talent surfaces recompute from the remaining Students; nothing is retained.

1 **Global Branch -> Talent.** Sidebar Branch switch (`/scope/branch`, session

scope) -> `/talent/{view}` renders the validated active Branch as `tp-config.branch` -> `reconcileBranchScope` sets `branch_id` (default), clears a stale Branch/Grade/ Section from a URL minted under another Branch, honors explicit `branch_scope=all` -> each API validates its own `branch_id` (foreign/unauthorized rejected).

1 **Talent page load.** Server loader (`data-server-loader`) is replaced as soon as

`talent.js` boots; if it is still present after 15 s the inline watchdog shows an explicit error with Reload. Assets are `?v=<content hash>`. Lookups are bounded (15 s each, 20 s deadline) and independent sections terminate on their own.

Learning Style Distribution Flow (Deployment Acceptance Correction C, 2026-09-24)

1 An actor with `students.view` opens the Students page panel or Results & Analytics

(Learning Style section, Student-domain-wide, never Program-bound).

1 The backend resolves the authorized Student population for the selected

Branch/Grade (SchoolGroup, Branch scope and tenant isolation enforced; an out-of-scope Branch is refused).

1 It counts one categorical Learning Style per Student in memory; Students with no

value are Unassigned. The denominator is the whole authorized population including Unassigned.

1 The UI shows all nine categories with count, percentage and a bar drawn from the

backend percentage, plus "N Students in this selection, including Unassigned". Zero categories show `0` / `0%`. An empty selection shows "No Students in the current authorized selection". No Talent small-cell suppression applies to this distribution (owner decision, ADR 0031 Acceptance C Amendment); a request failure or 403 keeps the section's own

error/retry state.

Current Talent Assessment Flow (Deployment Acceptance Correction B, 2026-09-24)

A teacher opens Student Assessments, selects an Evaluation Period and Program, sees the eligible Student roster (name, Learning Style, Grade, Section, assessment status, and - for completed current assessments - the automatic Classification with a Talented badge for Exceptional), starts or continues an assessment, and completes it. The backend deterministically computes the Classification; there is no Review or Official Identification step. Legacy Review Candidate / Official Identification records remain viewable as "Legacy Review & Identification History" for authorized users, separate from the current Classification. Learner Profile presents current history (result, Classification, competency evidence) first and legacy history in a distinct secondary section.

Results & Analytics Final Flow And Request Behavior (M18b-3 closeout, 2026-09-24)

An authorized user opens Results & Analytics (`/talent/analytics`); one batch of requests loads the analytics payloads (each endpoint once, including Learning Style, Classification, and Talented for the same context; Classification and Talented only with a Program). Changing a filter is debounced (250 ms), updates the URL with `history.replaceState`, aborts any in-flight batch, and ignores stale responses. Selecting a Program reveals the narrow-only Classification filter; clearing the Program hides and clears it. Every visible value comes from the backend; suppressed cells show a neutral unavailable state with no numbers in markup, ARIA, or datasets. Organization totals are raw-count sums. Legacy Review/Identification remain reachable as history/legacy workflow only. This flow is verified structurally (no browser-visual verification) and is not deployed.

Automatic Assessment Classification Flow (M17)

1 A Teacher opens a Student Assessment, records rubric evidence/scores per

competency, and saves as needed while the Assessment is In Progress. No classification exists yet - a draft/In Progress Assessment never carries a final automatic classification.

1 The Teacher selects Complete/Submit. The backend independently validates

completeness (every Grade-applicable competency has a valid saved result, and every applicable rubric shares the same ordered level count) before accepting completion, exactly as ADR 0037 already requires.

1 On successful completion the backend computes the ADR 0037 Overall

Program Result, then deterministically projects it onto the governed 1.00-5.00 classification scale and assigns exactly one fixed band (Needs Improvement / Developing / Meets Expectations / Advanced / Exceptional). Only Exceptional sets `Talented = true`. This never runs client-side and is never influenced by any client-supplied classification value.

1 The result is immediately visible in the authorized Assessment view and

the legacy Review workspace - no manual Review Candidate nomination or Official Identification decision is required for the Student to appear Talented.

1 Existing Review Candidate policy evaluation still runs (idempotent,

deterministic) for legacy/history continuity, and Official Identification remains available as a separate, permission-gated human decision surface for legacy/history purposes; neither is required by the current normal workflow and neither can override or rewrite the automatic classification.

1 A completed Assessment's rubric evidence and Framework Version remain

immutable (ADR 0037); its classification is therefore exactly as stable as its Overall Program Result already is.

Learning Style Correction + Aggregate Distribution Flow (M14)

1 Student create/edit shows exactly one Learning Style selector with eight

options (Visual, Auditory, Read/Write, Kinesthetic, Verbal, Non-verbal, Quantitative, Spatial) plus a blank "Unassigned" choice; the browser submits the one selected categorical value, never a percentage.

1 The server independently validates the submitted value against the same

eight-value authority server-side; an unsupported value is rejected even if a client bypasses the selector.

1 The Student profile displays the one selected value (or "Unassigned"),

never four percentage bars. The Talent tab's learner-context line reads the same current categorical value directly - no historical snapshot, no effect on rubric/Assessment/Overall Result/Evaluation Progress/ classification/Candidate-Identification.

1 The four legacy percentage fields are no longer collected or displayed

anywhere; if a stray client still submits them, the server silently ignores them rather than writing or erroring - a pre-existing stored value is left completely untouched by an unrelated edit.

1 The Students page's existing Learning Style distribution panel now shows

all eight categories plus "Unassigned," each row driven by one backend-authoritative count/percentage for both bar and text; the denominator always includes Unassigned Students. Privacy suppression, scope, and permission behavior are unchanged from ADR 0031.

1 Organization Overview's Branch-comparison metric selector (M10, below) no

longer offers "Learning Style" - requesting it now receives the same `invalid_filter` response as any other unrecognized metric.

Student Roster Import / Export Frontend Flow (M11)

1 The Students list shows Import and Export independently according to

`students.import` and `students.export`; the API repeats authorization.

1 Export requests the M6 export route and downloads its `.xlsx` response using

the server filename without reconstructing Student rows in JavaScript.

1 Import opens a transient dialog. The user selects one `.xlsx`; unsupported

extensions are rejected for usability while backend validation remains final.

1 Preview sends the actual file as multipart `roster_file`. M6 parses and

validates without mutation and returns summary, row data, and safe issues.

1 The browser renders only supplied values/messages. Student IDs remain text;

optional same-tenant identity is shown only when supplied, while cross-tenant conflicts remain generic. Canonical Section remains identity.

1 Apply becomes available only after a non-empty, error-free preview and still

requires explicit confirmation. The same actual file is uploaded again; no client-generated approved-row payload is accepted.

1 M6 independently revalidates and either creates every Student plus initial

Placement in one transaction or creates none. Success clears stale state and reloads Students; rejection retains useful context for correction and retry.

The browser never parses workbook cells/formulas, determines uniqueness or ownership, authorizes itself, performs partial apply, or transforms M5 `section_display` into canonical input. The first release is create-only and `.xlsx` only, with no persistent batch or background job.

M15 round-trip. The exported workbook is now re-uploadable: `section_display` is accepted-but-ignored and the `STD` prefix is normalized on import. Preview classifies each row as CREATE (new), NO_CHANGE (unchanged existing), or a blocked conflict (mutated existing). Apply writes only CREATE rows, skips NO_CHANGE rows with zero mutation, and stays atomic. The browser renders these three states from the backend-provided row status and enables apply only when at least one CREATE row is present and error-free.

Results & Analytics Branch Comparison Flow (M10)

1 An authorized user opens Organization Overview and selects an Academic Year

and one Program.

1 The metric selector offers the exact backend allowlist, omitting Meets

Program Criteria or Officially Confirmed when their independent permissions are absent.

1 Originally, selecting Learning Style revealed only Verbal, Non-verbal,

Quantitative, and Spatial, sent as `learning_style_dimension`. **Current state (M14 correction): the Learning Style metric option is removed** - see "Learning Style Correction + Aggregate Distribution Flow (M14)" above.

1 A selector change requests the existing M4 Branch comparison endpoint. The

backend performs frozen historical Branch attribution, metric calculation, privacy closure, and Framework comparability.

1 The browser walks returned Branch rows and Evaluation Period entries in

response order and resolves their names from the already-authorized Talent Map Branch columns.

1 Only `visible` rows render supplied percentages as bars and text, including

valid zero. Suppressed, restricted, coarsened, and no-data rows render their categorical meaning without inspecting any numeric/count fields.

1 A framework-changed Overall Result remains non-numeric and receives a concise

explanation.

The browser does not calculate or average Branch/Organization values, derive a metric, normalize Learning Style, evaluate thresholds, reconstruct hidden values, regroup by current Placement, or calculate across Framework versions. The comparison contract has no uniform Organization summary across all metrics (seven originally, six as of the M14

correction above), so this M10 surface adds none.

Student Evaluation Progress Frontend Flow (M9)

1 An authorized user opens a Student Profile's existing Talent tab.

2 The server-rendered Learner Profile supplies only authorized Program and

Academic Year contexts plus the unchanged frozen historical evidence.

1 For each context, the browser requests the existing M4 Student Evaluation

Progress endpoint using the same Student identity.

1 The endpoint enforces `talent_learner_profiles.view`, SchoolGroup scope, and

frozen historical Branch access, then returns backend-selected and backend-ordered active/opened Periods.

1 The browser renders each configured Period label and its explicit result

state. `available` renders the supplied percentage, including real `0%`; Pending and unavailable states render text without numeric progress semantics.

1 A comparable supplied `current_overall_result` renders as Overall Result.

For `framework_changed`, individual Period results remain visible, the combined number stays absent, and a concise explanation is shown.

1 Existing historical assessment details continue to render their frozen

Grade/Section and server-derived M5 `section_display` independently.

The browser never selects active Periods, reads weights, reorders Periods, averages results, decides Framework comparability, reconstructs suppressed aggregates, or substitutes current Placement. Branch/Organization progress, M10 comparison charts, and M11 roster UI are not part of this flow.

Talent Frontend Cleanup M8 Flow

1 Opening a Student Assessment loads its exact persisted Framework,

competencies, rubric levels, results, and frozen historical context as before.

1 The normal assessment editor no longer offers Reload Saved Rubric; a stale

response directs the user to refresh the page through normal browser behavior.

1 Normal assessment and Student Talent-profile presentation no longer fetch,

display, create, or amend Educator Input.

1 These presentation removals perform no deletion or mutation. Existing

Educator Input lineage, rubric/assessment evidence, audit, results, Candidate, Identification, frozen attribution, and Framework references remain stored under their existing backend contracts.

1 M5 server-derived Section display and canonical/frozen Section identity remain

unchanged. No Learning Style, Evaluation Progress, analytics-chart, or roster frontend work is introduced by this flow.

Students M7 Create/Edit/Profile Flow

1 New Student entry shows a fixed `STD` prefix and accepts exactly ten text

digits. Originally, it also offered optional independent Verbal, Non-verbal, Quantitative, and Spatial whole percentages from 0 through

1 **Current state (M14 correction): those four percentage fields are

removed from create/edit** - see "Learning Style Correction + Aggregate Distribution Flow (M14)" above for the current one-selector, eight-value categorical flow.

1 The server validates and atomically creates the Student plus managed number;

the browser never authorizes, canonicalizes the prefix, or resolves conflicts.

1 A duplicate response uses the shared privacy-safe conflict projection: an

independently authorized same-tenant actor may see minimal Student identity, while cross-tenant and unauthorized cases remain generic.

1 Legacy Students without a managed number display a neutral missing state and

may edit other Student fields. Number assignment/replacement is a separate `students.manage_identifiers` action.

1 Originally: blank Learning Style dimensions remained null, zero remained

`0%`, dimensions were not normalized, and the (then-)deprecated categorical data was not auto-converted. **Current state (M14 correction):** those four percentage dimensions are themselves now the deprecated data (preserved, untouched, no longer collected); the categorical field is the current, editable Learning Style value.

1 Profile/current Placement/history continue to consume server-projected M5

`section_display`; canonical Placement and Planning Section identity remain unchanged and are still used for requests.

Owner Video Acceptance Flow Corrections

1 Student Assessments groups configured Programs beneath each user-defined Evaluation Period whether or not an internal Cycle already exists.

2 Selecting a Program with no Cycle does not redirect to Evaluation Plan. With the existing Cycle-manage + Plan-manage + Period-select permissions, the browser creates the internal Cycle, links it to that Planned Period using revision guards, then re-enters Student Assessments with that Period/Program selected.

3 The Student roster is then the operational surface. **Start Assessment** posts the selected Evaluation context and Student, receives the new current Assessment, and opens that Assessment immediately.

4 The opened Assessment resolves its own persisted `program_id` and `framework_version_id`, then renders the exact Grade-applicable Framework Competencies, competency-owned assessment criteria/Levels/descriptions, and any saved results. The browser does not substitute another Program's rubric.

5 Any create/start failure is surfaced in the visible operational alert region; a failed click must not look like a no-op.

6 Delete Competency against an immutable/assessed Framework is a versioned future-change flow only: clone the current Framework, delete from the new Draft using the dedicated permission/revision guard, and preserve old Assessment evidence unchanged.

7 KPI numeric configuration remains separate from assessment criteria and has an explicit disclosure/collapse control.

Students Branch And Learning Style Presentation Flow

1 Add Student, Delete selected, per-row Open, and per-row Delete are compact icon-only controls with accessible names/tooltips.

2 Cross-Branch Students browsing requires `students.view_all_branches` **and** organization/global access scope. The managed-role policy keeps this permission Administrator-only.

3 Without that authority the list is fixed to the actor's assigned authorized Branch, the Branch selector is replaced by a read-only Branch context, and **All branches** is not offered.

4 Learning Style distribution formerly used the ADR 0031 privacy provider; **superseded 2026-09-24 (Acceptance C)**: it is authorized aggregation with no provider dependency or suppression (see the Learning Style Distribution Flow above).

Talent Selected Evaluation And Recovery Flow

1 Student Assessments lists user-defined Evaluation Periods and their configured Programs. Selecting a Program inside a Period visibly marks that Evaluation/Program as active and loads its currently eligible Students.

2 Start Assessment sends the selected Evaluation Cycle plus Student identity and opens the returned Assessment while retaining the selected Cycle/Program context. Continue/View Assessment opens that exact current attempt.

3 The Assessment workspace loads the exact saved Framework for that attempt and the Student's recorded Grade-scoped competencies/assessment criteria; a click must never silently no-op.

4 If automatic rubric-change comparison reports a completed current attempt as stale, the normal **Re-evaluation required** flow creates a linked replacement under ADR 0036.

5 If an administrator must deliberately reassess a completed current attempt without an automatic stale-rubric condition, `talent_assessments.reset_for_reassessment` marks the prior attempt historical/non-current and audits the reset. It does not delete any evidence.

6 After reset, the same Student/Evaluation row returns to **Not started / Start Assessment**. The next start creates a fresh current attempt with zero competency results; the prior completed attempt remains available only as historical evidence.

Talent Program Readiness And Authoring Flow

1 Talent's main sidebar tree is the sole primary module navigation; the page does not duplicate that navigation ribbon.

2 Program authoring shows one eligible Grade at a time, defaulting to the first configured Grade, then its Competencies and assessment criteria.

3 Competency and Rubric-Level deletion remains permission/lifecycle controlled by backend actions; UI visibility never grants authority.

4 Program readiness is projected on the Program summary. **Ready** is positive only after Program/Grade setup, assessment criteria, and Evaluation Plan requirements are complete; there is no separate Ready step.

5 The competency-owned rubric structure is labeled **Assessment Criteria** in normal setup. The separate real numeric `TalentKpiConfiguration` is labeled **Key Performance Indicator (KPI)** and supports Add/Edit/Delete while mutable.

Talent Review Filtering Flow

1. Talent Review starts from all current completed Assessments in the authorized Academic Year/Program context.
2. Branch options are tenant/scope authorized. **All Branches** appears only when the actor can access more than one Branch.
3. Grade options derive from operational Planning within the selected authorized Branch; Section options cascade from selected Branch + Grade.
4. The backend applies Branch/Grade/Section to frozen Assessment placement context before projection.
5. The primary table shows Overall Program Result, Review status, and Official Identification. Deterministic Review Candidate policy remains internal and can still enable review actions, but **Meets criteria** is not presented as a learner classification.

Talent Rubric And Analytics Closure Flow

1. Rubric configuration establishes arbitrary labels and `display_order`; the shared browser helper derives visual strength and an explicit "position of total" cue from that order.
2. Assessment entry uses the same treatment for selectable levels. Talent Review and Student/Learner Profile resolve the actual persisted level from the exact historical Assessment and Framework.
3. Meets Program Criteria remains the deterministic candidate outcome. Official Identification remains a separate permanent human decision; neither is inferred from the strongest displayed level.
4. Talent Overview links to operational work. Organization Overview shows selective executive results and directs users to Program Results, Branch Results, Talent Map, and Progress Over Time for detail.
5. Learning Style distribution starts at the authorized Organization context, optionally narrows by Branch, then exposes only Planning-configured Grades and Sections. The existing Students analytics endpoint applies permission, tenant/Branch scope, and privacy suppression.

Evaluation Plan Authority And Local Retest Flow

1. The server requires `talent_evaluation_plans.manage` or `.govern`, as applicable, plus durable organization/global access scope for Evaluation Plan and Evaluation Period mutations.
2. An organization-scoped Program manager may keep a Branch selected as working context; that selection does not suppress its projected Plan actions or backend authority.
3. A truly Branch-scoped user may view the Plan when permitted, but sees a read-only explanation and no add/change controls. Direct mutation calls remain denied.
4. The sanctioned Local Test Admin is seeded with organization scope and North Campus as its working Branch. If the disposable local database predates that seed contract, run `python scripts/manage_local_talent_test_db.py reseed --confirm`; never repair this condition with raw SQL or an authorization bypass.

Completed Program Operational And Edit Flow

1. Opening an active Program with enabled annual Grades, an active complete assessment setup, and at least one Evaluation Period shows its operational summary without the setup stepper.
2. Edit Program opens `#tp-basics`; Edit What we assess opens the assessment subflow; Manage Evaluation Plan opens `#tp-schedule`. Program and Academic Year query context remain unchanged.
3. The setup stepper labels Step 3 Evaluation Plan. Its embedded table labels each row an Evaluation Period and supports the existing authorized edit, remove, and Start Evaluation actions.

4 The Program summary confirms prerequisites. For a complete Draft Program, Finish Setup is the governed activation boundary: activate the Program, activate the reviewed Draft Framework with its revision/fingerprint, clear setup state, and open that Program's operational summary. Already-active state is not mutated again.

5 The renderer obtains the Academic Year display label from the selected shell option and never presents its internal ID as the normal label.

Talent Module Navigation And Context Reset Flow

1 While a Talent & Potential page is active, the main application sidebar expands a permission-aware child tree for Overview, Programs, Student Assessments, Talent Review, and Results & Analytics.

2 Choosing any of those top-level destinations is a context reset boundary. The selected Academic Year may carry forward; Program, Cycle, Assessment, Branch, Grade, Section, metric, dimension, and setup hash state do not.

3 Programs opens the searchable Programs collection. Selecting **Open Program** is the explicit transition into one Program workspace; row-level Edit/Rubric shortcuts are not duplicated on the collection screen.

4 Student Assessments opens neutral and shows its own Program filter. A Program is preselected only through an explicit Program-context action such as Open Assessments from inside that Program workspace.

5 Results & Analytics top-level entry also starts from clean module context, while navigation *within* the Results & Analytics family may preserve the active analysis filters.

6 Inside an opened Program, Edit Program, Build/Edit Rubric, Manage Evaluation Plan, Open Assessments, and View Results remain Program-context actions and retain that Program and Academic Year intentionally.

Talent Re-evaluation Reset And Evaluation Grouping Flow

1 Student Assessments groups each repeated user-facing Evaluation label across Programs, uses configured sequence to order label groups when available, and renders each Program once; duplicate physical Cycles never create duplicate Program cards.

2 A configured Program/Period can be shown before its internal Cycle exists. Starting/opening that Program uses the existing Evaluation Plan/Cycle workflow to materialize provenance when needed.

3 A completed Student whose current rubric has materially changed surfaces **Re-evaluation required**. For forward changes this means a newer assessable Framework Version; for legacy pre-guard data it may also mean that the same Framework now contains a complete competency-owned rubric whose IDs no longer match the completed attempt's persisted rubric/level bindings.

4 Re-evaluation never edits or deletes the prior evidence. The previous completed attempt becomes non-current; the replacement starts current and In Progress with zero competency results, links to the prior attempt, and remains under the original visible Evaluation through `evaluation_context_cycle_id`.

5 Untouched legacy shared rubrics and partially configured competency-owned rubrics do not trigger re-evaluation.

Talent Guided Program Configuration

1 Open or Edit Program enters the same `/talent/programs` wizard and selects Basics by default.

2 Basics saves draft Program details and the selected Academic Year's enablement/Planning Grades, while logo upload/removal continues through its Program-scoped route.

3 What we assess selects exactly one Competencies, Rubric Levels, Achievement Descriptions, or Review substep from the URL hash. Add/Edit opens only the requested inline editor; supported removals remain draft-only and revision guarded.

4 Evaluation Schedule renders inside the Program panel. Adding, renaming, removing, preparing, previewing, and starting evaluations use the existing Evaluation Plan, Period, and Cycle services without a setup-route change.

5 Ready presents compact completion counts and sends the user back to the embedded schedule to finish or start an evaluation.

6 The Students collection displays a table on desktop and cards on mobile through mutually exclusive CSS. Talent Review and an opened Evaluation use compact tables for their Student collections.

Talent Program Guided Setup

1 The user opens a Program and sees its identity plus focused Basics, What we assess, and Evaluation Plan entry points; readiness is summarized on the Program itself.

2 The browser reads supported URL hashes and renders the requested edit panel; refresh returns to that panel.

3 Basics saves Program identity/configuration through existing Program and academic-year routes. What we assess edits the selected draft framework through existing revision-guarded competency, rubric, level, and descriptor routes.

4 Evaluation Plan reuses the existing guided Plan/Period/Cycle workspace. When prerequisites are complete, Finish Setup is offered from the Program summary rather than from a separate Ready step.

5 The UI shows edit/remove actions only when the permission payload and entity lifecycle support them. Program lifecycle uses activate/retire; historical versions and terminal assessment evidence stay immutable.

Talent Program Identity, Planning Scope, And Result Clearing

1 Program logos are uploaded, replaced, or removed through organization-scoped Program routes using `talent_programs.manage`; all displays use the same compact logo/initials identity.

2 Talent Grade choices come from operational Planning sections for the selected Academic Year and authorized Branch. Grade changes load matching Planning Sections; no matches disable Section with "No Sections configured for this Grade."

3 Annual Program configuration rejects Grades absent from Planning, including when Planning has no configured Grades.

4 An authorized user may clear one saved competency result only while its assessment remains In Progress, using the existing expected-revision delete action. The UI updates that competency locally and preserves context and scroll.

Simplified Talent Program And Evaluation Flow

1 A user searches the compact Program list and opens or edits a Program.

2 Basics sets Program information and eligible Grades for the selected year.

3 What we assess shows each competency with every rubric level and achievement

description. Edit uses the existing draft-version and revision contracts; after save, the joined view is shown again.

1 Evaluation schedule accepts a free-text, user-defined evaluation name (for

example Term 1, Audition, or Spring Review). Adding an evaluation creates missing Plan/Period state and activates it when the actor has the existing govern permission; none of that Plan/Period/Cycle vocabulary is shown to the user.

1 The UI translates saved execution state to Setup, Ready to start, In

progress, or Complete. 5a. The shared Academic Year/Program/Branch/Grade/Metric/Dimension context selector used by this flow and by Organization Overview applies a change automatically (debounced); no separate confirm click is

required. The primary Talent navigation shows one "Results & Analytics" entry rather than repeating the sticky analytics sub-nav's individual page links.

1 Start Evaluation prepares a draft Cycle when needed, links it with both

expected revisions, and requests the authorized population preview.

1 The user sees "X eligible Students will be included based on Academic

Placement as of DATE" before confirming start.

1 The existing organization-governed open action revalidates Program,

Framework, Plan, Period, linkage, and revisions, then freezes the historical Student population. Assessment entry continues through the existing M5 flow.

Student/Talent Pre-Deploy Migration Flow

1 The dedicated pre-deploy command creates only baseline metadata; all

Student/Talent migration-owned tables remain absent.

1 The ledger runs `20260904_000`, accepting equivalent unique parent keys or

adding `(id, school_group_id)` uniqueness to Branch, Academic Year, and any existing Student table. Duplicate data aborts the transaction and marker.

1 Migration `001` creates Students and Academic Placements with the original

composite tenant-scoped foreign keys intact, followed by the Talent chain.

1 M4 creates Cycles before M8 without binding to a parent that does not exist;

M8 creates Plans/Periods and adds the nullable composite Cycle-to-Period FK.

1 Each migration and marker commits together. Any failure rolls back that

migration and stops pre-deploy before web activation; a retry re-evaluates only unmarked work.

M10 Production Provider Resolution Flow

1 Before analytical SQL, the route resolves commercial availability through

`feature.organization_intelligence` and the active canonical entitlement state. A false, missing, invalid, or exceptional result returns unavailable.

1 The existing `talent_analytics.view` permission check then runs separately;

Student drill and sensitive secondary metrics retain their additional permission gates.

1 Breadth configuration must resolve to 1000 matrix cells, 1000 relationship

results, and 1000 Program-pair results. A request above any applicable limit is rejected before aggregation and is never truncated.

1 Privacy configuration must resolve to minimum cohort 5 for P1-P7. Raw cells

below 5 are suppressed, then existing relationship-aware complementary suppression runs before any serializer receives the closed projection.

1 Outside production only, an exact `.local_test_data/talent_local_test.db`

URL substitutes deterministic privacy, bounded breadth, and local availability providers. Production names, `tis.db`, memory databases, and every other database path remain on the production fail-closed path.

Talent Results & Analytics Experience Flow

1 An authorized user opens Results & Analytics and selects an Academic Year. The Organization Overview requests the existing Overview, Program Portfolio, and Talent Map projections for that same authorized context.

2 The page presents returned facts as headline cards, Program progress, non-ranked Branch links, and a Talent Map preview. Only visible backend percentages can set a bar or chart dimension.

3 The user can drill from Organization to Branch, Program, or a visible Talent Map Branch cell. Program and Branch pages call their existing M10 routes and preserve supported Program/Branch filters.

4 Students Across Programs calls the symmetric distinct-participant pair route and explains diagonal versus shared Program participation without inferring similarity or ability.

5 Students requires both analytics and Student-view permissions, retains the P7 privacy gate and max-100 paging, shows frozen historical context, and exposes Candidate/Identification fields only when returned by the separately permissioned queries.

6 Progress Over Time calls one Program/one Academic Year with one allowed metric. It orders Periods by the governed response, translates comparability reasons, and never calculates a delta, trend, or improvement claim.

7 Permission denial, analytics unavailability, no data, and privacy protection are presented as separate plain-language states. A protected result supplies no visual or accessible magnitude.

M10 B11 Qualification And Release Flow

1 Commit and push the ADR 0028 governance checkpoint and require green

cumulative KMS.

1 With local/staging PostgreSQL, representative non-production data, and

non-production provider configuration, B11-C profiles latency, queries, memory, privacy closure, and EXPLAIN evidence without requiring final production thresholds, packaging, breadth limits, SLOs, or isolation. This step is complete and B11-C is CLOSED after independent re-review.

1 B11-D uses production-like PostgreSQL and controlled writers to test current

READ COMMITTED behavior first. Evidence, a governed ADR, and full-request transaction scope are prerequisites for stronger isolation; indexes follow the separate evidence/migration/rollback/ADR gate. This qualification is complete and B11-D is CLOSED. READ COMMITTED mixed snapshots were proven; permanent REPEATABLE READ remains governance-required and not implemented.

1 F1 implements the approved production privacy, commercial-availability, and

breadth providers. As of 2026-09-09 (ADR 0028/ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED. Cumulative production release qualification and a separate `dev->master` approval still block master merge and deployment.

M10 B8 Participation Overlap Flow

An authorized Organization Intelligence request resolves authentication, SchoolGroup, commercial availability, `talent_analytics.view`, tenant-bound Academic Year, historical Branch scope, normalized filters, and injected breadth approval before analytical SQL. The system then deduplicates Open/Closed frozen Cycle population membership to `(program_id, student_id)` within that authorized scope and performs one set-based self-join for canonical Program pairs. Each diagonal is the Program's distinct participants and each off-diagonal is the distinct intersection; no row or column sums are created. The P2 pair Cells pass through M9 primary privacy, B2 closure without fabricated overlap equations, `PrivacyClosedProjectionSet`, and the strict B8 serializer. The response contains no Student identity or Candidate/Identification overlap.

M10 B9 Student Drill Flow

An authorized Student Drill request resolves the identical B3/B4 pipeline as every other M10 route (authentication, SchoolGroup, commercial availability, `talent_analytics.view`, tenant-bound Academic Year, historical Branch scope), then explicitly requires `talent_analytics.view_students` (true AND composition) before any identifiable query. Filters resolve through the existing `resolve_filters`/`authorized_program_universe` primitives and breadth is enforced (`projection_family="student_drill"`) before any identifiable SQL. The system counts distinct authorized Student IDs from frozen `TalentAssessmentCyclePopulationMember` context (never current Placement) into one P7 `student_drill_population/count/distinct_student` Cell, then runs it through the identical B2 closure pipeline (`apply_primary_privacy_and_close`/`PrivacyClosedProjectionSet`) every other M10 route uses. Only a visible gate permits pagination over distinct Students, followed by page-bounded frozen Program/Cycle contexts and independently permission-gated Candidate/Identification evidence for those exact contexts. An advisory Learner Profile capability hint remains top-level. The strict `StudentDrillClosedProjection` wrapper and its serializer reject any raw SQL row, raw ORM object, or non-visible gate. The response has no `total_count`, no Student Talent Score/ranking, and no direct Student-ID filter.

B10-A Longitudinal Organization Intelligence Flow (Approved Architecture, Not Implemented)

ADR 0027 records the approved flow for a future single-Program, single- Academic-Year longitudinal request; no route exists yet, and this section describes governance only. A future request would resolve the identical B3/ B4 access pipeline for exactly the requested Academic Year, then resolve the Program's ordered M8 `TalentPlannedEvaluationPeriod` slots by governed `sequence` - the Period is the order authority, and its optional linked Open/Closed `TalentAssessmentCycle` (never a Draft or missing Cycle) is the factual evidence source for that point. For the one selected metric (nine of the fourteen existing `MetricCode` values are approved), each point would be built as a canonical Cell, pass through primary privacy and B2 closure, and be wrapped in a strict closed serializer identical in discipline to B8/B9. Adjacent points would carry `comparable`/`not_comparable` metadata with a governed reason code - never a server-computed delta, percent-change, or growth/improvement language. Candidate/Identification metrics would remain query-skipped before any privacy-sensitive point is selected without their own permission. `AcademicYear.year_name` would never be parsed or sorted as chronology; multi-Academic-Year longitudinal ordering remains deferred.

Deterministic Talent Analytics Flow (M9, Committed/Pushed On dev)

1 An actor with `talent_analytics.view` requests one Program + one Academic

Year context; context/filters resolve strictly against that scope (frozen population, not current Placement).

1 For every privacy-sensitive metric, the route runs the authorized raw

aggregate query, builds one or more `Cell`/`Group` structures, then calls `apply_primary_privacy` (the injected policy's real per-cell decision) BEFORE `run_complementary_suppression` - never the reverse, and never skipped, on any of the eight routes.

1 `run_complementary_suppression` repeats reconstruction-breaking passes to

a fixed point (max 8) using an identity/value-independent tie-break; a projection that cannot reach a safe fixed point serializes `restricted`, never a best-effort guess.

1 A `coarsened` cell publishes only a safe replacement value that the

policy itself supplied with its decision; if the policy requests `coarsened` with no replacement, the cell fails closed to `suppressed`.

1 Candidate and Identification analytics are query-skipped, not merely

response-filtered, without the actor's own `talent_review_candidates.view`/ `talent_official_identifications.view` permission respectively.

1 Student-level drill (`/students`) additionally requires

`talent_analytics.view_students` AND `talent_analytics.view`, and is denied (`analytics_drill_restricted`, no cohort size disclosed) whenever the cohort's own privacy cell is not `visible`.

1 Production privacy resolves only when the configured cohort value equals the

approved value 5. Missing, malformed, mismatched, or exceptional policy state fails closed rather than using an implicit permissive threshold.

Talent Annual Evaluation Plan Flow (M8)

The operational workspace presents the approved cross-milestone journey as:

1 Configure one Program's eligible Grades and exact competencies/rubric version.

2 Create and activate its Annual Evaluation Plan and ordered Periods.

3 Prepare a Draft evaluation, link it to an eligible Period, then open it to

freeze the historically scoped Student population.

1 Start a Student assessment and save each competency result through M5,

consuming the returned revision before the next result write.

1 Complete the Assessment, evaluate deterministic Candidate rules, review any

resulting Candidate, and separately record an authorized human Official Identification decision when appropriate.

1 Record or amend Educator Input through its independent M6 permission and

lineage without changing Assessment, Candidate, or Identification state.

1 An organization-authorized Plan manager creates one Draft Plan from an

existing Program Academic Year Configuration and authors ordered Periods.

1 Govern activates a non-empty valid Plan. Managers may edit/reorder only

unused planning context; linked Periods become immutable anchors.

1 Plan-manage AND Cycle-manage may link one Planned Period to one Draft Cycle

under Plan -> Period -> Cycle locks. Unlink is Draft-only.

1 Existing M4 govern opens a Cycle. If linked, the service revalidates Active

Plan, Planned Period, same Program/year, and unchanged linkage before the existing population freeze. Ad-hoc Cycles skip this step.

1 Govern may cancel an unlinked Planned Period with a reason. Closure allows

required Periods only when executed through a Closed Cycle or cancelled without a Cycle; optional unresolved Periods remain advisory.

1 View+manage may roll an Active/Closed Plan into another enabled annual

configuration for the same Program. Only label, order, required flag, and short code copy; execution and semantic context reset.

1 Without Cycle-view permission, reads omit the cycle key and all derived

relationship actions, cycle warnings, and per-Period execution resolution.

Talent Learner Profile Flow (M7, Complete)

1 A user with `talent_learner_profiles.view` requests one same-tenant Student.

2 The service loads only that Student's historical records and filters every

Placement/Talent record by its own historical Branch; no visible record is a non-enumerating not-found result for a Branch actor.

1 The response groups Program, Academic Year, Cycle, exact Framework,

Assessment, Competency Result, optional persisted KPI, Review Candidate, and Official Identification history without inferring a current Talent status.

1 Review Candidate, Official Identification, and Educator Input each require

their respective independent M6 `.view` permission in addition to base profile access; absent domains and related timeline events reveal no metadata.

Talent Review, Official Identification & Educator Input Flow (M6, Complete)

Every Educator Input create/amend flow resolves historical context, checks the resulting persisted Branch against canonical actor scope before commit, and rolls back with a non-enumerating response when unauthorized. Later reads use that same persisted historical Branch rather than current Placement.

1 An authorized actor with `talent_review_candidates.manage` and authorized

frozen-member Branch/organization scope requests evaluation for one Completed Assessment. A non-Completed Assessment is rejected.

1 The server deterministically evaluates the exact M3 Review Candidate

Policy attached to the Assessment's exact Framework Version, reading only existing Assessment/Competency-Result/persisted-KPI evidence - never mutating it. No policy attached means no candidate is ever inferred.

1 A qualifying (policy-satisfied) evaluation materializes one durable

candidate row, starting `status="pending_review"`, with policy identity, match mode, a SHA-256 fingerprint, and a full evaluation snapshot; re-running evaluation afterward is idempotent and returns the existing row unchanged. A non-qualifying evaluation still persists no candidate row, but is now structurally audited (assessment identity, Framework/Policy context, `outcome=false`, fingerprint - no free text).

1 An authorized human with `talent_review_candidates.manage` marks a

`pending_review` candidate `reviewed` (`POST .../{id}/review`) - one-way only, never reversible, never altering assessment evidence, never auto-identifying the Student.

1 Only once a candidate is `reviewed` may an actor with

`talent_official_identifications.record` AND organization/global access scope record exactly one Official Identification decision (`identified`/`not_identified`) for it. A Branch-scoped actor is denied even if granted `.record`. The decision is durable either way - there is no mutation, revocation, second decision, or re-identification path.

1 Independently of the Review/Identification chain, an authorized educator

with `talent_educator_inputs.add` may record bounded qualitative Educator Input for a Student, bound to SchoolGroup/Student/Program/AcademicYear/ `observed_at` plus a resolved historical Placement/Branch snapshot (frozen Cycle context if supplied, otherwise the Student's Placement effective at `observed_at` - a clean rejection if none exists). `talent_educator_inputs.amend` may append a new version referencing the one it supersedes; the original is never edited or deleted, and default reads return only the current version of each lineage chain.

1 Review Candidate, Official Identification, and Educator Input remain

structurally distinct entities: Review Candidate never auto-produces Official Identification, and neither carries the other's state in a shared mutable field.

Presentation note: the steps above describe the backend `ReviewCandidate` model, its `/api/talent/review-candidates` routes, and the `talent_review_candidates.*`/`talent_official_identifications.*` permission keys, which are unchanged deterministic/audit identifiers. The normal user-facing Talent UI labels this surface "Talent Review" (nav entry, breadcrumb, Student Profile Talent tab, and in-page notifications) and no longer prints "Review Candidate(s)" or "Candidate" in that normal path. The Talent Review list itself now renders as one compact table (Student, Program, Grade, Section, Evaluation, Result, Review status, Identification status, Action) instead of one large card per Student; opening a row (`review_id`) shows that one Student's full detail, including the Official Identification decision form, rather than showing every Student's full detail inline.

Talent Student Assessment Flow

1 An authorized assessor with canonical Branch/organization scope starts one

In Progress Assessment only for an Open Cycle's visible frozen member.

1 The assessor records or replaces exact Framework Competency/Rubric Level

results using the Assessment's expected revision. Current Student transfer never changes frozen-Branch access.

1 Completion validates that every exact Framework Competency has a result. For

an enabled KPI, the service validates numeric inputs, calculates integer weighted contributions over denominator 10,000, applies ROUND_HALF_UP, and persists the Framework-specific result and canonical provenance atomically.

1 A successful Completed Assessment is read-only. The assessor may instead

finalize Incomplete or Insufficient Evidence, each read-only and without a KPI result. Candidate selection, correction/reopen, and assessor assignment are not available.

Presentation note: an opened Evaluation's Student list is one compact table (Student, Grade, Section, Assessment status, Action) rather than one large card per Student. The Action label is state-driven from the same real assessment status above, not a separate UI status: no assessment yet shows "Start Assessment" (only offered while the Cycle is Open and the actor can manage), an `in_progress` Assessment shows "Continue Assessment", and any other recorded status (Completed, Incomplete, or Insufficient Evidence) shows "View Assessment". No new assessment state was introduced.

Talent Assessment Cycle Population Flow

1 An authorized Draft author creates a SchoolGroup-wide Cycle and explicitly

selects Program, Academic Year, exact Framework Version, and `population_effective_at`.

1 Draft preview resolves effective Academic Placements and annual eligible

grades dynamically. Organization population readers see the whole preview; Branch readers see only authorized Placement branches and a subset count.

1 An organization/global governor opens the Draft. The server locks and

revalidates it, derives the complete SchoolGroup population, inserts exact frozen historical member snapshots, stores full count/fingerprint, changes status to Open, and audits the event in one transaction.

1 Open/Closed reads use frozen Branch context. Branch readers never receive

the full count/fingerprint; later Student transfers do not change their historical visibility. A confirmed qualifying Placement may add a missing member to an Open Cycle under ADR 0033; the Cycle is locked, revisioned, re-counted, re-fingerprinted, and audited in the same transaction. Existing members/evidence are unchanged. Close is final and forbids synchronization.

Configure And Generate With Teacher Scheduling Rules

1 An administrator with `timetable.manage_teacher_rules` opens System

Configuration ? Timetable Settings ? Teacher Scheduling Rules in one exact SchoolGroup, branch, and academic-year scope.

1 The administrator selects a teacher, Schedule within/Must teach/Unavailable,

all or selected working days, numbered periods, and Select All or selected Current/New Planning sections. Grade and first/last controls are not part of the normal form.

1 The service validates scope and rule shape, rejects deterministic hard

contradictions, persists normalized rule/slot/target rows, and marks only unpublished Drafts stale while clearing approval. Published history is not modified.

1 Generate or Regenerate captures canonical rules in a schema-v4 immutable

snapshot. Readiness and the problem builder reject invalid targets, impossible workload/availability, conflicting hard rules, and unavailable-period locks.

1 CP-SAT confines existing eligible demand to Schedule-within windows, enforces

Must teach occupancy and Unavailable slots, and continues scoring saved teaching/free preferences. No rule creates demand and hard rules never relax.

1 The independent validator repeats window, required-occupancy, and

unavailability checks and reports preference satisfaction. Only a valid, current-fingerprint candidate may become a new unpublished Draft.

1 Select All mirrors every visible Current/New section and saved-rule summaries

show either exact grade-section names, All assigned sections, or All classes for teacher-wide Unavailable rules. Editing restores the same literal scope.

1 Manual Draft placement rejects hard Unavailable and Schedule-within violations

before writing. Complete Draft validation also detects missing Must-teach occupancy and wrong selected-section occupancy; these hard blockers prevent approval and publication, while preferences never block lifecycle actions.

1 Multiple Schedule-within rows that apply to the same scoped demand contribute

slots to one combined allowed window. Before solving, TIS matches every teacher's effective workload to those combined windows and reports provable capacity or Subject Distribution Rule conflicts, including unavailable slots, Must-teach slots, daily coverage, minimum/maximum days, and double blocks.

1 When the full solver alone proves infeasibility, the Workflow runs bounded

family-isolation solves over base collisions, locks, grouped activities, subject rules, teacher rules, and their interaction. It persists the proven customer-safe category and lock/group counts on the same Generation Run; it never saves a relaxed candidate or changes the original hard constraints. The Workflow uses `TIS_TIMETABLE_DIAGNOSTIC_TIMEOUT_SECONDS` (default 60) independently from the primary solve and skips absent lock/group profiles.

Guided Curriculum Adjustment UI

1 A user with `curriculum.adjust` opens **Adjust Curriculum** from Planning or Subjects.

2 The user chooses grade, selected Current/New sections, or all active source uses, then chooses source, target, and the proposed source reduction.

3 The browser requests the Stage 3 preview and presents section demand changes, teacher/capacity impact, rule and grouped warnings, blockers, and Draft regeneration impact.

4 The user explicitly chooses an eligible, within-capacity teacher or **Leave unassigned** for every affected section. Nothing is preconfirmed.

5 Final review requires confirmation and submits the unchanged preview fingerprint to Stage 4 apply.

6 If Planning changed, the preview is refreshed and the user reviews again; compatible teacher selections are preserved where possible.

7 Success reports affected sections and Draft status and offers Timetable/regeneration links. Regeneration is never automatic.

The period count entered in step 2 is the transfer amount for every selected section. Preview rejects zero/negative transfers and any section whose current source demand is smaller than the request. A partial transfer leaves its remaining source demand active. After apply, Subjects shows the effective Planning value when uniform; differing section values show **Varies** and can be expanded by section. Catalog weekly hours remain secondary defaults.

For **Reduce periods only**, the user selects no target subject. Preview shows the source before/after demand, source teacher load reduction, rule/grouped warnings, and Draft impact. No teacher reassignment is requested. Apply changes only the selected source demands, retires only zero results, and otherwise follows the same fingerprinted atomic transaction. Subject Details can open this flow prefilled for its one grade-specific subject. Multi-grade selection remains outside this single-subject workflow.

Atomic Curriculum Adjustment Apply

1 A user with `curriculum.adjust` submits the exact reviewed request, Stage 3

fingerprint, and one explicit target-teacher decision per section.

1 The service owns one transaction, locks exact SchoolGroup/branch/year authority,

and rejects an active generation run.

1 It rebuilds the preview and rejects stale fingerprints, preview blockers,

incomplete decisions, invalid qualification/capacity, rule conflicts, or Draft locks that cannot survive the proposed demand/teacher state.

1 It updates only selected Current/New source/target demands and confirmed teacher

assignments. Source zero becomes inactive retirement evidence and any active source section rule is inactivated.

1 It marks only the current unpublished Draft stale, clears approval, preserves its

snapshot/placements/history, and records one durable applied audit.

1 All changes commit together. Any failure rolls back the entire adjustment.

Regeneration remains a later explicit action; published history is untouched.

Read-Only Curriculum Adjustment Preview

1 An authorized Planning editor selects grade, selected sections, or all active

source uses and identifies source/target subjects plus proposed source periods.

1 The service resolves only Current/New sections in the selected SchoolGroup,

branch, and academic year and reads explicit-first section demand.

1 It projects released/target periods, current teachers, uncommitted teacher

options and capacity, scheduling-rule arithmetic, grouped warnings, and Draft stale/regeneration impact.

1 It returns blockers/warnings and a deterministic preview fingerprint. No row is

changed and no teacher is automatically selected.

1 A future apply stage must rebuild and compare the fingerprint before any atomic

mutation. Published timetable history remains outside the write path.

Simplified Timetable And Official Visibility

1 A manager creates or opens the exact-scope Draft Timetable and configures inputs until Configuration Complete.

2 The manager runs Verify Feasibility. The durable Workflow solves the exact immutable snapshot with every hard constraint and no soft objective, then independently validates one complete solution.

3 Only a validated result produces Feasibility Verified and enables Generate Timetable. Infeasibility exposes the isolated hard-rule family; timeout remains inconclusive and cannot enable generation.

4 The verified placements are retained by full input fingerprint. Any Planning, allocation, rule, slot, lock, grouped-resource, or other snapshot-authority change requires verification again.

5 Full generation optimizes quality from the verified placement hint. If optimization times out, the worker independently validates and persists the verified placement as the Draft fallback instead of returning no timetable.

6 When authority changes after a Draft exists, the Draft remains stale,

versioned, and unpublishable. If current inputs have no genuine structural blocker, the state is Draft Needs Regeneration and still permits Verify Feasibility against a fresh immutable snapshot. A verified current fingerprint then permits Regenerate; the stale source is never rewritten.

1 The regeneration source is the current populated, mutable, unpublished Draft,

whether its origin is manual, generated, or regenerated. An empty starter Draft has no prior arrangement and therefore follows Generate. Active and historical versions cannot be selected directly as regeneration sources.

1 Generate and Regenerate operate against that current draft context; Regenerate retains Stage 5.1 locks and concurrency.

2 Approve Draft validates that exact draft and records the reviewing administrator;

generation alone never grants approval. Regenerate is presented beside Approve Draft and requires at least 25 percent of unlocked source placements to change, rounded up.

1 Any placement, drag/drop, swap, lock, regeneration, or stale authority change

invalidates approval and requires review again.

1 Publish Timetable requires approval, revalidates the draft, atomically makes it

official, and retains the prior publication in history.

1 `/my-timetable` resolves exact-scope teacher identity, reads only the active pointer,

and filters to that teacher. View-only non-teachers use the general published page. Destructive and technical version actions remain in Timetable History.

Generate And Regenerate Timetable Flow

Academic Scheduling Quality is configured using explicit subject codes and optional Swimming section groups. Saved branch/year rules enter the immutable run snapshot. CP-SAT applies hard daily core coverage when demand permits, optional ICT one-per-day, simultaneous group/resource constraints, and soft distinct-day/non-consecutive preferences. Independent validation repeats hard rules before persistence.

1 Require `timetable.generate`, exact tenant branch/year scope,

`generation_ready`, a current mutable draft context, and no active run.

1 Select Generate for a fresh/manual current draft; select Regenerate for a current

generated candidate and freeze the source revision, arrangement, and locks.

1 Capture schema-v3 canonical input and fingerprint, commit one durable run, then

dispatch its public ID to the configured Render Workflow task. Immediate dispatch failure ends the still-unclaimed run safely and permits Generate Again.

1 The on-demand task claims that exact run, leases it, builds from only the snapshot,

solves with CP-SAT, and records Building, Solving, Checking, and Saving phases.

1 The independent validator checks exact demand, scope, Planning/HRT authority,

canonical slots, collisions, locks, fingerprint/source revision, and diversity.

1 Rebuild current authoritative input. Changed inputs end as `stale_input` and

create no version.

1 Atomically create one unpublished generated/regenerated publication-ready version,

its entries, links, and succeeded run state. Do not change the active pointer.

1 Polling selects the result for review. Validation and Publish remain separate.

Cancellation ends queued work immediately or requests cooperative cancellation of leased work. Reload resolves the active durable run. Infeasibility and timeout create no version. A queued task delayed beyond the configured threshold reports that it is waiting for compute. The browser never polls Render. Regeneration is unavailable when all source lessons are locked.

Common Customer Feature Decision Flow

1 Resolve the authenticated operational user and authorized SchoolGroup.

2 Require the registered user permission; denial ends the decision.

3 For branch-scoped actions, require accessible tenant branch scope and a

matching academic year when supplied.

1 Resolve one coherent commercial state and active workspace entitlement.

Paid, promo, demo, and internal sources remain distinct; inactive, expired, suspended, missing, or ambiguous evidence fails closed.

1 Require an active catalog definition. Developer-only `feature.audit_log`

is explicitly outside the customer baseline.

1 Resolve normal customer availability from the common policy. Optional,

beta, disabled, or custom features continue through their explicit policy.

1 For branch-scoped work, resolve an active coherent BranchEntitlement.

2 Apply capacity or consumption separately. Feature availability never grants

additional branches, staff users, teachers, or AI provider usage.

Allocation XLSX/PDF use this flow with `feature.advanced_reporting` and `reports.export`. Starter, Professional, Enterprise AI, active promo, and active demo therefore share availability while permission, scope, branch entitlement, and lifecycle continue to win.

AI Availability And Consumption Flow

1 Resolve enabled AI definition, tenant workspace, `ai.use`, commercial source,

and the shared `module.ai` baseline.

1 Return availability independently of consumption. A globally disabled AI

definition remains unavailable.

1 Resolve consumption policy by entitlement source. Demo Standard retains two

successful uses per feature; Full is unrestricted; Custom may set legitimate allowances. Paid and promo currently have no configured provider ceiling because no executable provider-backed AI route exists.

1 Reserve by operation key, execute outside the entitlement service, and

complete successful or failed accounting exactly as before.

SchoolGroup Management And Branch Capacity Flow

1 School Management resolves the authenticated identity and registered permission.

2 Direct SchoolGroup create/delete additionally requires a Platform identity and

`schools.manage_all_schools`; tenant users fail before validation or side effects.

1 A tenant with `schools.edit` may update only the SchoolGroup resolved from the

tenant link/scope. Platform users require global management capability for an arbitrary SchoolGroup.

1 The page resolves branch usage and limits through unified commercial authority.

It shows customer-safe used/allowed state and only shows branch creation when one more active branch is currently permitted.

1 Branch POST requests independently re-resolve tenant scope, lock the SchoolGroup,

recount capacity, and reject stale or over-limit requests before insertion.

1 For promo authority with remaining capacity, the same transaction creates the

branch, promo assignment, and active branch entitlement. At capacity it changes nothing.

1 The last-active-branch check counts only active branches in the target

SchoolGroup.

Existing Workspace Owner Alignment And Conversion

Existing Workspace Paid Activation

1 A verified tenant owner opens Organization Account and selects **Choose a Plan**.

TIS displays all currently eligible paid plans. If the open activation is `draft` or `checkout_ready`, its plan is highlighted but the owner may select another eligible plan. Opening or changing this selection calls no Paddle API and creates no PaymentAttempt.

1 TIS locks and validates the `customer/provisioning` SchoolGroup, exact owner

link, absence of commercial authority, and absence of conflicting paid, promo, or demo activity.

1 TIS recounts active operational branches and organization-wide staff users and

teachers. Professional and Enterprise AI require all active branches. Starter remains unavailable until complete restricted-branch enforcement is proven.

1 The owner confirms the workspace billing profile. TIS builds an authoritative

USD quote from the active plan price and stores branch-selection and quote hashes. Changing an editable selection recalculates this quote and safely supersedes the unfinished local draft without changing any branch.

1 Preparation creates a SchoolGroup-anchored contract, paid-activation aggregate,

immutable branch snapshot, and checkout session. It creates no pending organization, provisioning job, workspace, tenant, or entitlement. Until launch creates a real PaymentAttempt, Organization Account continues to show Activation required rather than Payment processing.

1 A `checkout_started`, `payment_processing`, manual-review, or inconsistent

activation cannot be silently replaced; plan changes fail closed until that payment path is resolved. Launch revalidates the current quote and either reuses a still-billed matching Paddle transaction or creates one automatic-collection transaction with branch quantity and stable authority metadata. Customer, address, and business mappings are reused only through explicit account/workspace attribution. The workspace association records the selected address and business, and every returned or reused transaction must match the complete current billed quote and custom lineage.

1 Browser return never activates access. `transaction.paid` displays processing.

Failed, cancelled, or expired attempts instead display a recovery state.

1 A signature-verified `transaction.completed` locks the SchoolGroup and activation,

revalidates all local and provider evidence, and atomically creates the paid subscription authority, active branch entitlements, and paid tenant link. The existing workspace becomes active without tenant provisioning.

1 Duplicate webhooks are idempotent. Quote drift, capacity drift, branch drift,

stale transactions, provider mismatch, or a competing commercial source fail closed without partial authority. PostgreSQL lock acquisition refreshes the activation state before duplicate-event decisions.

Guardrails:

1 `workspace_classification` remains `customer`; paid versus promo is commercial-source evidence.

1 Paddle quantity is branch count only. Staff and teacher totals affect eligibility only.

1 Existing operational and academic rows are never recreated or mutated by activation.

1 Promo activation and PendingOrganization/new-organization checkout retain their

existing independent selection and payment behavior.

1 Organization Account remains available while operational access is blocked for recovery.

Controlled Existing Workspace Conversion

1 A Platform Owner runs the M4B CLI in dry-run mode with the exact workspace

tuple, intended-owner email, current M4A hash, operation UUID, and idempotency key. No row is changed.

1 Write preparation requires PostgreSQL, Platform Owner approval/execution

identities, and `PREPARE <operation UUID>`. It locks the workspace, reruns M4A, rejects stale or conflicting evidence, and records an ownership claim. It does not create an account or send email.

1 The intended owner registers normally, establishes a password or approved

external identity, and verifies the exact email. Login continuation opens the existing-workspace setup review.

1 The verified owner explicitly claims the workspace. TIS rejects unverified,

suspended, duplicate, cross-tenant, or platform identities, safely reuses or creates the operational owner identity, and creates a `tenant_owner` account link with no pending organization. A different current owner requires prior transfer approval.

1 The owner confirms existing organization identity and supplies only legal

name, official IANA timezone, and educational program. Branches and all existing operational records remain unchanged.

1 Final CLI execution requires `CONVERT <operation UUID>`. It locks the

operation and SchoolGroup, performs a fresh M4A audit, and requires unchanged identity, branch/dependency evidence, one active internal entitlement, no commercial source, complete setup, and unique verified ownership.

1 One commit ends the internal entitlement and sets `customer / provisioning`.

It creates no replacement entitlement or tenant link. Any failure rolls back all conversion changes and records only redacted failure evidence.

1 M1 resolves `activation_required`. Organization Account remains accessible,

normal operations remain blocked, and M3 promo activation is offered. The existing-workspace Paddle path remains hidden until its separate milestone; new onboarding checkout is unaffected.

Existing Workspace Conversion Audit

1 An operator supplies the exact SchoolGroup ID, workspace UUID, expected

organization name, and intended owner email to the standalone M4A CLI.

1 The CLI requires deployed PostgreSQL and begins a repeatable-read, read-only

transaction before the first application query.

1 The service validates the exact workspace tuple, resolves normalized owner

identities and links, and inventories tenant, provisioning, entitlement, paid, demo, and promo evidence.

1 For every target branch, the service follows reflected direct and indirect

foreign-key descendants and separately reports branch-like columns without a foreign key. Soft-deleted rows remain blocking. Foreign keys absent from ORM metadata force manual review. It returns counts and paths, not private row payloads.

1 Identity mismatch, duplicate ownership, existing non-sandbox commercial

authority, unavailable schema evidence, or uncertain traversal produces Manual Review Required. An active internal-sandbox entitlement is expected and is not treated as customer authority.

1 The CLI writes deterministic sanitized JSON or text with the observed

transaction mode and a stable SHA-256 evidence hash. Exit `0` is coherent, `1` is execution/configuration failure, `2` is manual review, and `3` is identity mismatch. Archival candidates include only active dependency-free branches; hard deletion and write conversion remain unapproved.

1 The CLI rolls back and closes. It never archives, deletes, aligns, converts,

calls Paddle, or sends email.

Customer Promo Activation

1 A verified organization owner chooses Use Promo Code after setup review or

from an eligible existing Organization Account.

1 TIS HMAC-normalizes the submitted code, validates approval, lifecycle,

dates, replacement, target scope, owner relationship, redemption policy, source compatibility, and setup readiness, then creates a short-lived resumable activation session. The raw code is never stored.

1 TIS shows authoritative branch, system-user, and teacher usage. If eligible

branches exceed the grant, the owner selects exactly the grant count. If fewer exist, all are selected and the remainder stays available for future branch creation.

1 Excess staff or teachers blocks activation and identifies each exceeded

dimension. TIS does not disable, select, delete, or change those records.

1 Final activation locks session, promo, and workspace, then repeats every

validation and recount. It rejects internal sandboxes and every conflicting commercial source.

1 One transaction creates immutable redemption/grant snapshots, explicit

branch assignments and entitlement states for the complete preserved branch inventory, a promo workspace entitlement, the promo-sourced tenant link, customer/active lifecycle, and redacted audit. Selected branches are assigned and active; every unselected branch is explicitly inactive without changing its operational status or consuming grant capacity.

1 A pending session grants nothing. Active access resolves through M1 and only

selected branches are queryable. Expiry or inconsistent evidence fails closed. No Paddle call or payment object is involved.

1 Reactivating an unassigned branch locks the SchoolGroup, recounts M1 usage, and

either rejects at capacity or commits operational status, one grant assignment, and active branch entitlement together. Bulk reactivation applies the same rule to the complete batch and rolls back all changes on any conflict.

1 `scripts/reconcile_promo_branch_entitlements.py` targets one SchoolGroup and

workspace UUID. Dry-run reports only safe missing inactive evidence. Explicit apply revalidates under lock and inserts only those inactive entitlements; ambiguous authority, ownership, assignments, or entitlement lineage requires manual review.

1 An authorized owner opening Organization Account Billing & Subscription is

routed by the selected workspace's authoritative source. Promo authority renders Commercial Access with the grant plan, safe status/window, masked reference, and M1 branch/system-user/teacher usage and remaining capacity. It never invokes the paid subscription resolver, Paddle billing history, invoices, plan/quantity changes, or cancellation while promo access is active.

1 Operational promo access ends exactly at `effective_to`. Before that instant the

grant is active; at that instant and through `grace_period_days` it is expired in a recovery period; after grace it remains expired. Both expired states block all normal tenant operations while preserving Organization Account and tenant data.

1 A verified owner may begin paid continuation only for recovery-period or expired

promo authority. Plan selection and Paddle checkout reuse the existing SchoolGroup and M4C quote/identity lineage. Pending, failed, canceled, expired, or abandoned checkout does not end promo authority or restore expired operational access.

1 `transaction.completed` locks the SchoolGroup and activation, revalidates current

capacity, quote, provider identity, promo source, and paid evidence, then atomically ends the promo workspace entitlement, relinks the sole tenant source to the paid contract, establishes paid workspace and branch entitlements, and records immutable conversion audit. Duplicate confirmation is idempotent; conflicts roll back the authority transition and enter manual review.

1 Active promo early conversion is blocked. Promotional value is not a paid credit,

and no proration or unused-promo-value calculation is invented.

Platform Promo Definition

1 A Platform Owner or permission-authorized Developer opens Promo Codes.

2 Create validates one active Starter, Professional, or Enterprise AI tier,

exact positive capacity through the shared plan-capacity service, coherent scope anchors, dates, expiry XOR, and redemption policy.

1 TIS generates at least 100 bits of secure random code material, normalizes

with NFKC/uppercase/separator removal, and derives an HMAC-SHA256 lookup hash with `TIS_PROMO_CODE_HMAC_SECRET`. Missing configuration fails closed.

1 The draft and allowlisted audit commit. The raw code appears once in a

no-store/no-referrer response; subsequent pages show only its mask.

1 A Developer may edit Draft, pause Active, duplicate, or create a linked

replacement definition when authorized. Active material edits first require pause, clear approval, increment version, and return to Draft.

1 Only a Platform Owner may approve/activate or terminally revoke with a

reason. Lifecycle transitions lock the row through validation, mutation, audit insertion, and commit.

1 No Platform definition step grants access or calls Paddle. Customer M3

activation is a separate owner-authorized transaction.

Organization Profile Save

1 The customer opens the owned pending organization's Organization Profile.

2 The browser submits the existing multipart POST with a required controlled

educational-program value and an optional logo.

1 TIS normalizes National, International, or Both and validates all profile

fields. Customer-correctable input returns the same page with preserved text/select values.

1 When a logo is supplied, TIS reads no more than the 4 MB boundary, decodes

the actual image as PNG/JPG/WEBP, checks dimensions, and ignores the customer filename when creating the opaque stored filename.

1 TIS writes the new image to a temporary sibling and atomically promotes it.

2 Organization changes, progress, activity, and events commit together.

Only after commit is an obsolete prior pending logo removed.

1 If logo storage or a later database step fails, TIS rolls back the

transaction, removes the newly written file, retains the prior logo reference, logs the traceback, and renders a customer-safe error.

1 The successful response redirects to Branch Setup. No duplicate pending

organization or operational workspace is created.

Pending logo files currently live on the application-local filesystem. Durable Render operation requires a separately approved persistent disk or object storage architecture. The owner-only workspace deletion workflow currently removes database records but does not yet perform transactional cleanup of pending or promoted organization-logo files on the filesystem.

Organization Logo Activation

1 Successful Organization Profile save stores a safe relative pending-logo

path; setup and checkout pages render only its public static URL.

1 Organization Profile and the shared setup identity header show the customer

logo and organization name while the official TIS mark remains platform branding.

1 Checkout does not move or remove the pending image.

2 Paid and demo activation both call `create_workspace_records()`.

3 Provisioning resolves the pending path only inside the pending-logo

directory, requires the file to exist, decodes it again, and writes a new organization-owned branding file.

1 A primary `SchoolGroupLogo` row stores the final relative path and an

organization-name label. `SchoolGroup` has no direct logo field.

1 The operational shell resolves that row through the protected

organization-asset route, renders it with contain sizing, and keeps the official TIS logo separately visible.

1 Missing pending source data blocks activation for correction instead of

producing an active workspace with silently lost branding.

Branch Estimate Rendering And Save

1 The service loads active branch rows and computes capacity totals in Python.

2 Null, missing, or malformed display values contribute zero; mixed valid

integer values continue to total normally.

1 The template receives normalized totals and renders null row fields as zero,

avoiding Jinja `int + None` failures.

1 Every customer save still requires explicit non-negative whole-number

values for estimated system users and teachers.

1 Newly created pending branches receive explicit zero defaults before their

submitted values are applied.

Combined Subscription Capacity And Custom Contact

1 Each active onboarding branch supplies required non-negative system-user and

teacher estimates; TIS sums them across the organization.

1 Before payment, system-user and teacher authority is independently the

greater of the estimate total and actual same-workspace active data. After activation, actual active data is authoritative. Every active tenant operational User counts as staff regardless of position, so an operational teacher-user with a Teacher record consumes one staff slot and one teacher slot.

1 A self-service plan is eligible only when all three counts fit: Starter is

1 branch/5 system users/25 teachers, Professional 5/20/100, and Enterprise AI 25/100/500.

1 The lowest eligible plan is recommended; higher eligible plans remain

selectable. Ineligible cards explain every failed dimension. Exceeding twenty-five branches, 100 system users, or 500 teachers emphasizes the always-visible Custom contact action.

1 Selection and every quote, checkout, Paddle, and payment-validation boundary

repeats the same server-side decision.

1 A pre-payment branch estimate change retains an eligible higher

plan, but clears an undersized selection and supersedes its checkout and attempt so stale payment cannot activate the workspace.

1 On a paid workspace, every tenant operational-user creation/reactivation and

teacher creation/year-copy preflight requires remaining capacity. Blocked operations create no partial user or teacher data.

1 A downgrade is blocked separately by current branches, system users, or

teachers. Existing data is preserved.

M1 Operational Commercial-Capacity Enforcement

1 The route first validates the existing permission and tenant scope.

2 TIS locks the owning SchoolGroup with `SELECT ... FOR UPDATE`; internal

multi-tenant operations acquire locks in ascending SchoolGroup ID order.

1 The commercial authority facade composes classification/lifecycle,

workspace entitlement, tenant link, contract, confirmed subscription, commercial access, demo lifecycle, and plan capacity.

1 The service recounts active branches, distinct active tenant operational

users, and active teacher people. Known teacher IDs are normalized and deduplicated; blank legacy identities each count independently.

1 Paid limits use confirmed branch quantity capped by plan maximum plus the

plan staff and teacher limits. Pending subscription changes do not increase capacity. Demo and Internal Sandbox are unmetered only when their existing authority resolves coherently.

1 TIS evaluates deltas or proposed absolute totals across one or several

dimensions and returns a structured safe decision.

1 Allowed branch, user, teacher, year-switch, or provisioning mutation occurs

before the same transaction commits. A denial rolls back and exposes no provider or internal identifiers.

1 Existing over-capacity data remains accessible and may be reduced; only a

further increase in an already exceeded dimension is blocked.

The separate Next.js landing page presents the same four-plan structure and organization-wide limits. The hero Subscribe Now action scrolls to pricing. Starter, Professional, and Enterprise AI retain their published monthly and

annual per-active-branch prices and enter the configured public signup route with an allowlisted preferred-plan code. That code is a presentation preference only: signup creates no plan selection, checkout, payment attempt, or Paddle object. After organization setup, TIS applies it only when the live plan remains active and eligible across branches, system users, and teachers; otherwise TIS clears it and asks the customer to review eligible plans. Custom has no fixed public price, uses only the Contact the TIS Team mail action, and never starts signup or Paddle checkout.

Initial Subscription Plan Capacity

1 TIS counts the current organization's authoritative active billable

branches.

1 Starter is eligible for one branch, Professional for up to five, and

Enterprise AI for up to twenty-five. Higher-capacity plans remain available below their limits.

1 Plan cards retain per-branch pricing and explain their branch and staff

limits. Ineligible plans are disabled; above twenty-five branches the customer is directed to contact the TIS team for a custom plan.

1 The server repeats the capacity decision during selection, quote creation,

checkout preparation and launch, and payment validation.

1 If pre-payment editing makes the selected plan too small, TIS clears that

selection, supersedes its quote and checkout lineage, and requires a new eligible plan. An eligible higher plan remains selected.

1 After activation, branch-capacity changes continue through Subscription

Management.

Fixed-Quantity Initial Checkout

1 Before confirmed payment, customers may add, edit, remove, reorder, or

reprioritize onboarding branches while keeping at least one active branch. A demo SchoolGroup, tenant link, customer, or prepared checkout does not close editing.

1 A legacy `ready_for_checkout` pre-checkout billing state is eligible for

safe preparation. The original incident was a local rejection in `_ensure_checkout_launchable()` before TIS called Paddle.

1 If the plan, interval, branches, capacity estimates, or authoritative quote

change after checkout preparation, TIS supersedes the local checkout and unfinished attempt, clears the old quote lineage, and recalculates from the current selection and active count. A late old transaction event cannot activate that quote.

1 TIS resolves the selected plan, billing interval, active billable branches,

unit price, and total for the current organization.

1 The customer reviews those values in TIS. For Professional Annual, the

active mapping is USD 790 per branch, so two branches produce quantity 2 and USD 1,580 annually.

1 TIS creates one Paddle transaction item using the mapped provider price and

the exact authoritative branch quantity; quantity is never defaulted to one.

1 TIS reuses an active same-country Paddle address for the same customer, or

creates one when no compatible address exists. The resolved address makes the automatically collected transaction ready. Returned item quantity, price, subtotal, and quote fingerprint must agree with TIS before the transaction is marked billed.

1 The payment launcher passes only the billed transaction ID to Paddle inline

checkout after verifying the local attempt, organization, customer, quote, and remote billed status. Billed transaction items and quantities are immutable; draft, ready, canceled, past-due, unrelated, or mismatched transactions are not launched.

1 Paddle completion remains the recurring-subscription authority. Later branch

changes use the established TIS subscription-quantity workflow.

1 Retry revalidates unpaid checkout eligibility and may replace an incomplete

or non-launchable session. A reusable started transaction must still be billed, automatic, customer-matched, and current-quote matched at Paddle.

Returning Customer Login And Organization Account

1 SaaS login authenticates and resolves authoritative onboarding, demo,

workspace-link, lifecycle, and subscription state.

1 Incomplete setup resumes; pending demos open request status; and unpaid

completed setup opens subscription selection.

1 An activated owner or linked user with account-management permission lands

on `/saas/account`, the Organization Account Overview. Multiple managed organizations require selection before the overview is shown. The HTTP-only selected UUID is revalidated against the account's current tenant links and permissions on every request before entitlement or billing resolution.

1 The overview exposes Organization Profile, Branches, Billing & Subscription,

and Account & Security only when ownership or existing permissions allow. Restricted or suspended account managers retain permitted billing/recovery access, but no active Enter TIS Platform action.

1 Enter TIS Platform is the only Organization Account action into `/login`.

Linked users without account-management permission retain their approved role-based destination and cannot see owner or billing controls.

1 Operational login runs the commercial guard before branch and academic-year

setup. Protected requests run it before page services. Authentication, sign-out, SaaS account, subscription/checkout/payment-return, support, and expiry routes remain safe and cannot recursively redirect.

1 Expired-demo Subscribe Now resolves the existing organization, SchoolGroup,

and active operational branches, offers public plans and monthly/annual intervals, then launches existing Paddle checkout.

1 Only confirmed provider payment converts the same workspace UUID, tenant

link, users, permissions, branches, and data.

Password login, social login, already-authenticated sign-in, post-verification sign-in, and SaaS root/session restoration use the same destination resolver.

Platform Owner Demo Operations

1 A Platform Owner opens an existing Customer Demo operational detail.

2 The owner supplies required reason/date/configuration and an operation key.

3 The service validates and locks the same workspace and tenant scope.

4 Lifecycle work reuses M8B7 rules; feature policy uses controlled registries.

5 Material changes create durable customer email/Notification Center records,

and every attempt records before/after/result references.

1 Profile changes retain usage and never convert the Customer Demo to paid.

M8B8 AI Entitlement And Consumption Flow

1 Resolve the operational SchoolGroup and reject cross-tenant scope.

2 Require the registered feature's underlying `ai.use` permission.

3 Resolve existing commercial state; demo expiry/restriction takes precedence.

4 Resolve the controlled feature definition.

5 Internal Sandbox is unlimited; Customer Demo checks two successful uses per

feature; Customer Paid checks eligible plan plus `module.ai`.

1 Reserve an operation key under the locked successful-plus-reserved limit.

2 If reserved, execute the real AI operation outside the entitlement service.

3 Finalize a usable result as successful consumption; finalize failure without

consumption and release the reservation.

There are currently no executable AI routes, so steps 6-8 are a reusable contract rather than a fabricated customer feature.

M8B7 Demo Customer Journey

Submission creates pending status, a request-received email intent, and owner Notification Center events. Normal approval records review evidence then invokes the retryable provisioning service; activation communication exists only after success. Day 6 and expiry create lifecycle events, customer notices, owner notifications, and email intents atomically. Provider delivery occurs outside locks. A coherent expired demo may subscribe and, after authoritative confirmed payment, reactivate and convert the same workspace.

This document describes the major end-to-end flows a developer must understand before changing TIS.

Public Customer Flow

Flow:

1. Public visitor opens `https://tisplatform.com`.
2. Visitor chooses Request a Demo or Subscribe Now.
3. Visitor reaches SaaS signup at `/saas/signup?intent=demo` or `/saas/signup?intent=subscribe`.
4. SaaS account is created.
5. Email verification is completed when required.
6. Email verification redirects by HTTP 302 to the GET `/saas/login` page.

Its form submits credentials by POST to `/saas/auth/login`.

1. A fresh verified account with no organization enters the GET

`/saas/account` dashboard. A supplied continuation is used only when it is a known customer GET destination; POST-only, malformed, traversal, and external values fall back to Account Setup.

1. The user invokes the existing POST start action and completes organization onboarding:

1. organization details,
1. contacts,
1. branches,
1. academic setup,
1. review.

1. The final commercial-choice page emphasizes the saved intent, while the user may still choose Request Demo or Subscribe Now.
2. Subscribe Now continues to plan selection and the existing Paddle checkout path.
3. Paddle handles payment.
4. Return/cancel page informs the user of checkout navigation result.
5. Paddle webhook confirms payment.
6. Local payment/billing state is updated.
7. Pending organization becomes ready for provisioning.
8. Platform owner reviews/runs provisioning.
9. Operational tenant structures are created.
10. Operational login becomes available through `/login`.

Guardrails:

1. Checkout return is not authoritative payment confirmation.
1. Public signup must not directly create operational tenant data.
1. Provisioning should occur only through the approved flow.

Demo Request Flow

1. A verified customer completes organization, contact, branch, academic, and review onboarding.
2. TIS presents Request Demo and Subscribe Now.
3. Request Demo revalidates account ownership, verification, onboarding completeness, branch configuration, absence of conflicting payment/provisioning state, and normalized organization-domain eligibility.
4. TIS creates one Pending Review SaaS demo request plus a transactionally unique customer-demo eligibility reservation for the normalized organization domain.
5. The customer can view status and withdraw only while Pending Review.
6. A Platform Owner searches, filters, and sorts the review queue.
7. Approval creates a review record only; rejection requires a reason; owner cancellation is allowed only while pending.
8. A Platform Owner separately starts provisioning for an Approved request.
9. TIS revalidates review evidence, customer-demo intent, commercial/entitlement snapshots, organization completeness, and duplicate absence.
10. One atomic transaction creates the operational workspace through the shared provisioning builder, creates the demo entitlement and demo-sourced tenant link, activates both, and links the request.
11. Failure rolls back workspace records, leaves the request Approved and unprovisioned, and records a retryable failure outcome.
12. Each review, provisioning, and activation action creates durable audit and internal-notification events.

Guardrails:

1. Request and approval alone create no SchoolGroup or entitlement.
1. Demo provisioning creates no checkout, payment, paid subscription, subscription contract, or Paddle record.
1. A prior customer demo request, activation, expiry, rejection, cancellation, or demo-to-paid conversion for the same normalized organization domain blocks a new customer demo. Platform Owners extend/reactivate where separately allowed, or the customer subscribes using the existing workspace.
1. Public email providers require an official organization website or domain before a demo request. Internal Sandbox workspaces do not consume customer demo eligibility.
1. Non-owner platform users and tenant/customer identities cannot access review actions.
1. Duplicate, rejected, cancelled, incoherent, or already provisioned requests fail closed.
1. M8B-4 sends no email and does not implement expiration, scheduling, login blocking, or conversion.

Historical Demo Eligibility Maintenance Flow

1. A Platform Owner opens `/saas-admin/demo-eligibility-maintenance`.
2. TIS scans the demo-domain eligibility ledger and resolves matching organization, account, request, tenant-profile workspace, provisioning, subscription, conversion, and manual-review evidence.
3. Linked or ambiguous reservations remain protected and display their exact blockers.
4. A safely detached row exposes a review action for its exact eligibility ID.
5. The owner types the same ID and confirms permanent removal.
6. TIS locks and re-analyzes that exact row before deletion.
7. Any new blocker aborts and rolls back the action.

8 TIS deletes only the selected primary key, flushes, verifies no row remains for that ID, and commits.

9 A durable audit event records the Platform Owner, eligibility ID, normalized domain, previous status, timestamp, and historical-cleanup reason.

Guardrails:

1 Never delete by normalized domain.

1 Never expose the workflow to customers, tenant users, or Platform Developers.

1 Never use this workflow to override a linked, historical, converted, subscribed, provisioned, or manual-review Customer Demo reservation.

1 Do not change the normal one-demo-per-domain rule or the organization-scoped clean-room reset.

Customer Demo Lifecycle Flow

1 Successful M8B-4 activation records the authoritative demo start.

2 TIS derives reminder due at start plus six days and expiration at start plus seven days.

3 Before Day 6, the resolver returns Active and operational access continues.

4 At Day 6, the resolver returns Reminder Due; the processor creates one customer notification and one per active Platform Owner.

5 At Day 7, access fails closed immediately even if scheduled processing has not run.

6 Apply-mode processing atomically ends the demo entitlement, suspends the SchoolGroup, marks the demo tenant link expired, and records lifecycle events.

7 Web requests redirect to the preserved-data and subscription page; API/download requests receive a safe 403.

8 Existing sessions are rechecked on every protected request. Platform users remain able to inspect the workspace.

Guardrails:

1 Use `activated_at`, never request submission or approval, for lifecycle calculations.

1 Store and calculate in UTC; convert only customer/owner display values to the organization timezone.

1 Expiration never deletes, archives, deactivates branches, or mutates operational tenant data.

1 Reminder and expiration actions are idempotent and failure-audited.

1 No email, Paddle change, conversion, extension, or read-only expired mode is included.

Demo-To-Paid Conversion Flow

1 An active, coherently provisioned Customer Demo chooses Subscribe Now.

2 TIS records a requested conversion and continues through the existing M7 plan selection and Paddle checkout.

3 No new operational workspace, tenant link, branch, user, permission, or academic record is created.

4 `transaction.completed` remains the payment authority and establishes the confirmed `SubscriptionContract` and active `PaymentSubscription`.

5 If the existing tenant link is demo-sourced, webhook reconciliation invokes the dedicated conversion service instead of paid provisioning.

6 The service locks and revalidates the request, provisioning aggregate, SchoolGroup, demo entitlement, tenant link, contract, subscription, price, interval, quantity, and tenant ownership.

7 One atomic workspace transaction ends the demo entitlement, creates the subscription-backed paid entitlement, relinks branch entitlements, changes the SchoolGroup to Customer Paid Active, and moves the same tenant link to the confirmed contract.

8 Existing M7 entitlement, workspace-entitlement, and commercial-state resolvers validate the resulting Customer Paid Active state before commit.

9 Success records completion and lets the customer continue into the same operational workspace. Completed conversions no longer enter demo lifecycle processing.

10 Failure rolls back workspace mutations, preserves provider-confirmed payment records, records a safe retryable failure, and may be retried from a later subscription webhook.

Guardrails:

1 Never infer conversion from checkout return navigation, onboarding selection, or an unconfirmed payment attempt.

1 Never run paid tenant provisioning for a valid existing demo tenant.

1 Expired, suspended, ambiguous, cross-tenant, internal-sandbox, already-paid, and incoherent workspaces fail closed.

1 Conversion does not change Paddle pricing, subscription mutation, webhook authority, tenant isolation, or operational permissions.

SaaS Identity Flow

Flow:

1 User signs up through `/saas/signup`.

2 SaaS account/session records are created.

3 User signs in through `/saas/login`.

4 Account dashboard is available at `/saas/account`.

5 User can access profile, sessions, security, billing status, and onboarding state.

Important distinction:

1 SaaS account identity is not the same as operational tenant user identity.

1 Platform identity is also separate.

Guardrails:

1 Do not merge SaaS accounts with operational users unless an approved provisioning flow creates the needed operational records.

1 Keep SaaS authentication and operational authentication boundaries clear.

Payment Flow

Flow:

1 User selects a plan.

2 App creates or references a checkout session.

3 User goes to Paddle checkout.

4 User returns through checkout return or cancel pages.

5 Paddle sends webhook events.

6 Webhook-confirmed payment updates local payment/billing state.

7 Verified payment can make a pending organization ready for provisioning.

Guardrails:

1 Webhook confirmation is authoritative.

1 Do not use return-page navigation as proof of payment.

1 Keep Paddle-specific details inside `saas/` payment/client service boundaries.

Active Subscription Management Flow

1 Authorized billing administrator opens `/saas/subscription`.

2 TIS resolves one confirmed active subscription, entitlements, lifecycle state, allowed actions, and one operational capacity snapshot covering active branches, tenant operational staff users, and teachers. Paid branch quantity is displayed separately from the current plan's maximum branch ceiling; unused ceiling is not prepaid branch capacity.

3 Review Capacity accepts proposed totals for all three dimensions. It shows confirmed paid branches, required active branches, additional billed branches, the plan ceiling, and the resulting minimum eligible plan. Branch growth may create a combined plan-and-quantity upgrade, while system-user and teacher growth affects eligibility without changing Paddle quantity.

4 Quantity or plan changes are previewed through Paddle; TIS displays provider-returned totals and never recalculates proration.

5 Immediate increases/upgrades use provider payment-failure prevention and remain locally pending until authoritative confirmation. The current confirmed plan and workspace access remain authoritative while the plan change is pending, failed, incomplete, expired, canceled, or abandoned. Thus an active Professional subscription remains accessible while an Enterprise AI upgrade is `payment_pending`.

6 Reductions/downgrades are scheduled for the next billing boundary and retain current local access until verified effective evidence. Live branch, system-user, and teacher counts are revalidated at that boundary; a mismatch enters manual review without applying the lower local entitlement.

7 Scheduled plan or quantity changes may be canceled or replaced before their effective boundary when provider state agrees.

8 Cancellation is scheduled at period end; reversal removes the provider-scheduled cancellation after reauthorization and validation.

9 The centralized lifecycle resolver exposes only actions valid for current provider/local state.

10 Billing Contact is explicit organization-owned state. Initial checkout requires a confirmed billing email, legal/billing organization or school name, and supported country/address data. The active portal view is read-only until an authorized owner or `subscriptions.manage_billing` user selects Edit; validation remains in edit mode, Cancel discards unsaved form values, and login identity is unaffected.

11 TIS saves valid local billing changes before attempting provider synchronization. It reuses the mapped Paddle customer, synchronizes its explicit billing email, creates or reuses one attributable active Paddle Business, persists address/business mappings, updates the active subscription identity, and includes `business_id` on future initial transactions. Provider failure leaves the local profile saved with retry-needed status. The dedicated retry revalidates permission and tenant scope, uses the stored profile and mappings, avoids duplicates, and makes no provider call after successful synchronization. Historical billing documents are not silently revised.

12 Billing history is read from Paddle transactions. `paid` displays Payment received - processing; only `completed` displays Paid and satisfies the existing final processing signal. Invoice download reauthorizes the user and requests a fresh provider URL.

13 Operational login, protected requests, and returning-customer routing use the same contract-linked commercial access projection. The specialized plan-change webhook synchronizes provider subscription status, but the target plan becomes authoritative only after the existing required provider signals are both confirmed.

14 If production provider state is confirmed but local commercial status is stale, an operator replays the attributable stored, signature-verified Paddle webhook through the existing reconciliation path. Operators do not edit subscription or lifecycle status fields manually. Replay may synchronize the current `PaymentSubscription.status`, but it cannot bypass the separate provider payment signal required to activate a target plan.

Guardrails:

1 provider and local ownership must match,

1 active branch usage cannot exceed a requested reduced capacity,

1 target-plan eligibility is evaluated across branches, system users, and teachers before submission and again when a scheduled downgrade becomes effective,

1 Paddle quantity contains branches only; system-user and teacher counts never become provider quantity units,

1 webhook processing is idempotent,

1 ambiguous outcomes enter manual review,

1 return pages and local requests are not payment confirmation.

1 organization billing identity is tenant-scoped and provider mappings are revalidated before synchronization,

1 a saved billing profile in pending or failed provider synchronization blocks new plan and quantity mutations until retry succeeds; cancellation and legacy active subscriptions with no saved profile retain their established behavior,

1 provider synchronization logs the safe failed step, provider error code, and HTTP status server-side; provider identifiers and raw diagnostics are never rendered to the customer,

1 user login email is never an implicit permanent billing-email authority,

1 stale or unrelated organization-level subscription rows cannot supersede the TenantProvisioningLink and SubscriptionContract-linked subscription,

1 canceled subscriptions retain access only until the confirmed paid period ends; ambiguous evidence fails closed without being mislabeled as expired,

1 renewal guidance is shown only for genuine expiration, not payment processing, past due, paused, suspended, archived, or inconsistent commercial evidence.

Operational Billing Entry Flow

1 System Configuration includes Billing & Subscription only for a linked

operational organization owner or user with `subscriptions.manage_billing` in the selected tenant.

1 The bridge revalidates the operational session, SchoolGroup scope,

SaaSAccountUserLink, organization/workspace UUID, and billing authority.

1 The bridge opens `/saas/subscription` for that organization; it never

duplicates billing UI inside operations.

1 If SaaS authentication is missing, password and social sign-in preserve the

allowlisted internal subscription continuation and revalidate it against the signed-in account before rendering billing.

1 Unknown organizations, cross-account links, unauthorized users, duplicate

continuation parameters, and external return URLs fail closed. A user with multiple managed organizations must resolve an organization before billing.

1 The landing-page Organization Sign In action enters `/saas/login` without an

operational destination. Activated account managers therefore land in Organization Account unless they arrived through the validated billing continuation. Enter TIS Platform remains the only explicit operational entry.

Provisioning Flow

Flow:

1 Pending organization has completed required onboarding.

2 Payment is verified or owner-approved readiness is satisfied.

3 Provisioning job is queued or run.

4 Operational records are created or connected:

1 school group,

1 branch,

1 academic year,

1 initial operational user,

1 permissions/role context,

1 required setup defaults.

1 Provisioning status is updated.

2 Activation/access email may be sent.

3 User enters operational portal through `/login`.

Guardrails:

1 Keep provisioning idempotent where possible.

1 Do not mix school groups.

1 Do not skip platform owner visibility.

Operational Login Flow

Flow:

1 User opens `/login`.

2 Credentials are verified.

3 Active-user status is checked.

4 Platform users route toward platform context.

5 Tenant users receive branch/year scope.

6 Session and scope cookies are set.

7 User lands on `/platform` or `/dashboard`.

8 Middleware enforces route permissions.

Guardrails:

1 Preserve platform vs tenant branching.

1 Preserve idle timeout behavior for tenant users.

1 Do not bypass permission middleware.

Platform Owner Flow

Flow:

1 Platform owner logs in through `/login`.

2 Owner lands in platform context.

3 Owner uses `/platform` for organization context and owner/developer controls.

4 Platform Console pending counts include only organizations still requiring setup, review, payment, or incomplete/recoverable activation work.

5 Owner opens Pending Queue for current work or Organization Records for active, completed, rejected, and lifecycle-review history.

6 Owner uses SaaS admin pages for payments and provisioning.

7 Owner can inspect Workspace UUID, Classification, and Lifecycle as read-only metadata on `/platform`.

8 Owner uses `/platform/knowledge` to review KMS health.

9 Owner views/downloads the PDF through protected routes:

1 `/platform/knowledge/booklet`

1 `/platform/knowledge/booklet/download`

Guardrails:

1 Platform developers are not owners.

1 Owner-only pages must use existing owner access helpers.

1 Active tenant evidence takes precedence over stale onboarding status; conflicting completed evidence is labeled Lifecycle Review Required and excluded from the normal pending queue.

1 Do not expose KMS PDF through direct static links.

1 Workspace classification metadata does not authorize access or change commercial state in M8B-1.

Workspace Classification Diagnostic And Backfill Flow

1 Operator runs `scripts/diagnose_workspace_classification.py` to inspect every SchoolGroup and its tenant, onboarding, and Paddle relationship presence.

2 Operator runs `scripts/backfill_workspace_classification.py` without `--apply` for a read-only plan.

3. After review, operator reruns with `--apply`.
4. One transaction classifies all pre-M8B-1 records as internal sandbox/test data and records an idempotency marker.
5. A repeated apply reports `already_applied` and changes nothing.

Guardrails:

- The diagnostic and dry run do not change rows.
- Apply does not call Paddle, migrate Al-Andalus, convert workspaces, or change payment/provisioning state.
- Failures roll back the full backfill transaction.

Commercial State Resolution Flow

1. Resolve the SchoolGroup workspace classification and lifecycle.
2. Resolve exactly one effective workspace entitlement, or use the compatibility-only implicit entitlement for an internal sandbox created outside migration.
3. Validate entitlement type against workspace classification and parse explicit values through the shared entitlement catalog.
4. For customer-paid workspaces, resolve the existing M7 confirmed subscription entitlement and require the linked local `PaymentSubscription` to match.
5. Resolve each branch as inherited, explicitly active, or commercially inactive while independently respecting operational branch status.
6. Return a read-only effective commercial state or fail closed to Manual Review Required.

Guardrails:

- No resolver writes rows or calls Paddle.
- No resolver changes current tenant access, feature checks, branch mutations, onboarding, or provisioning.
- Demo expiration and commercial-state mutation remain later work.
- Cross-tenant and orphan branch entitlement relationships fail closed.

Knowledge Management Flow

Flow:

1. Developer or Codex reads `docs/AI_PROJECT_CONTEXT.md`.
2. Developer reads master context, project state, engineering docs, relevant ADRs, and module history.
3. Approved change is implemented.
4. `.kms-impact.yml` and the human-readable Knowledge Impact Assessment are completed.
5. Relevant docs are updated:

- master context,
- project state,
- change history,
- ADRs if needed,

1. module history if needed,
1. AI project context if needed,
1. engineering docs if architecture/module/flow understanding changed.

1. PDF generator runs.
2. PDF snapshot and manifest are regenerated.
3. Knowledge Center checks manifest freshness and health.
4. CI compares the declaration with changed files and blocks stale pull requests, `dev` integration, or `master` deployment.

Guardrails:

1. Markdown remains source of truth.
1. PDF is generated and must not be edited manually.
1. App must not silently rewrite source docs.
1. Regenerate button is not implemented yet.

Timetable Version Compatibility Flow

1. A scoped timetable read resolves SchoolGroup, Branch, and Academic Year.
2. It loads the compatibility operational version: a working draft sourced from

the active version when one exists after an edit, otherwise the active version, otherwise the newest scoped draft.

1. Section and teacher views, XLSX, and PDF use only that version's placements.
2. On the first legacy assignment edit of an active timetable, the service copies

every placement and intended lock into a new mutable draft.

1. The requested edit changes the draft, increments `edit_revision`, and records

manual-change evidence. The active pointer and imported history do not change.

1. Later legacy edits reuse the same working draft until explicit version and

publication workflows are introduced.

Migration flow:

1. Find only branch/year scopes with existing placements.
2. verify Branch and Academic Year resolve to one SchoolGroup or fail the whole

migration transaction;

1. capture current Planning/settings authority in a deterministic snapshot;
2. preserve and attach every placement to one imported compatibility version;
3. record safe stale reason codes without changing a placement; and
4. create one exact-scope active pointer without fabricating publication approval.

Guardrails:

1 Active, superseded, and archived versions are not mutated in place.

1 No timetable mutation is allowed without explicit valid tenant scope.

1 Planning remains authority for real section-subject demand and HRT fallback.

1 Stage 2 executes no generation and treats no display-only block as a solver rule.

Stage 3.5 configuration flow:

1 The administrator selects working days, teaching-period count/duration, and shift start.

2 A block is placed after a teaching period with a duration, or retained as fixed time.

3 One composer walks each day, inserts blocks at deterministic boundaries, shifts later

teaching periods, and calculates day/end metrics.

1 Settings preview, timetable grid, readiness, assignments, snapshots, and exports consume

that projection. A valid inserted block is not a readiness blocker; an ambiguous fixed time, invalid duration/day/boundary, or conflicting block is.

1 Configuration fingerprint changes make existing versions stale without deleting their

placements or mutating published history.

Stage 3 derives canonical slots, resolves positive Planning demand (including HRT), checks allocation/capacity/locks/staleness, and returns stable readiness blockers and warnings for one exact scope. Ready to Generate means only that inputs can be submitted to a future solver; no generation action runs.

Stage 4 version/publication flow:

1 Default read resolves an active-derived mutable draft, otherwise active, otherwise

the newest scoped mutable draft; explicit history selection changes only the view.

1 Immutable history may be copied to a new draft with placements, valid locks, and

source provenance preserved.

1 Draft placements and locks use exact permissions plus `edit_revision`; locks update

snapshot authority and any edit returns publication-ready state to draft.

1 Validate Draft checks freshness, demand completion, collisions, canonical slots,

Planning teacher authority, stale placements, and locks without a solver.

1 Publish rechecks scope, permission, revisions, fingerprint, and validation; locks

the pointer; supersedes prior active history; and activates atomically.

1 Historical versions remain reviewable, comparable, safely archivable, and

explicitly exportable. Viewing or exporting never publishes.

Human / AI Developer Onboarding Flow

Flow:

1 Read `docs/AI_PROJECT_CONTEXT.md`.

2 Read `docs/README.md`.

3 Read `docs/TIS_MASTER_CONTEXT.md`.

4 Read `docs/PROJECT_STATE.md`.

5 Read `docs/engineering/TIS_MODULE_MAP.md`.

6 Read `docs/engineering/REPOSITORY_ARCHITECTURE.md`.

7 Read `docs/engineering/USER_AND_SYSTEM_FLOWS.md`.

8 Read relevant ADRs and module history.

9 Inspect code with `rg` before editing.

10 Make scoped changes only.

11 Update KMS docs and regenerate the PDF if needed.

12 Report the KIA.

Before coding, inspect:

1 affected routes,

1 models and scope fields,

1 permission rules,

1 templates/forms,

1 tests for the touched module,

1 related docs/ADRs/history.

After coding, update:

1 `docs/CHANGE_HISTORY.md` for meaningful changes,

1 module history for area-specific changes,

1 ADRs for major decisions,

1 engineering docs when module maps, architecture, or flows change,

1 AI context when onboarding truth changes,

1 project state when priority/status changes.

Talent Owner Adjustment Flow - 2026-09-12

1 Student Assessments keeps every current authorized Academic-Year placement in

the roster and offers Grade, Section, and Assessment Status filters.

1 Start resolves the newest saved Framework applicable to the Student's Grade.

Intentionally unscoped criteria apply; another Grade's explicit criteria never do. No applicable criteria yields a Grade-specific setup message.

1 Reload Saved Rubric may refresh only a zero-result current In Progress

attempt to a newer applicable Grade build. Evidence-bearing and terminal attempts are never rebound.

1 Talent Review supports Identification Classification filtering over already

authorized projected rows without changing identification permissions.

1 Results & Analytics assigns ordered low-to-high blue intensity only to

visible magnitudes; protected/no-data states stay categorical.

1 Learning Style rows retain each style's established color even when

magnitudes are suppressed; suppressed rows expose no count, percentage, or proportional bar length.

1 Talent-capable users navigate Students through the Talent child tree;

standalone Students remains only for students-only access. Desktop Student tables never create a nested vertical list.

1 The full transparent Ghars wordmark renders only in Talent Overview; the

sidebar retains the compact emblem.

TIS Database Architecture Overview

docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md

Student/Talent PostgreSQL Parent-Key Prerequisite

The legacy `branches` and `academic_years` tables may predate their explicit `UNIQUE (id, school_group_id)` constraints. PostgreSQL requires such a key before accepting the composite tenant-scoped foreign keys on `student_academic_placements`; a primary key on `id` alone does not match that declared composite reference. `students` has the same parent-key requirement for downstream tables.

Migration `20260904_000_student_talent_parent_scope_prerequisites` establishes all three keys before `20260904_001`. It recognizes equivalent unique constraints/indexes, performs a duplicate preflight, never rewrites rows, and relies on the migration transaction to roll back both DDL and its ledger marker on failure. Student/Talent tables are excluded from baseline metadata creation so the prerequisite actually runs first. For current metadata compatibility, M4 creates `talent_assessment_cycles` without the future M8 Period FK; M8 creates the parent and adds that exact FK. Every other composite Student/Talent target is created with a matching primary/unique constraint before its child.

M10 B11 PostgreSQL Qualification Policy

B11-C is CLOSED after PostgreSQL 16.15 profiling and independent re-review. All seven routes were profiled; query counts were bounded across tested scales with no N+1. Candidate A requires NO CHANGE and Candidate B requires MORE EVIDENCE, so no index, schema, or migration change is approved. Memory evidence is directional local Windows development evidence only.

B11-D is CLOSED after PostgreSQL 16.15 READ COMMITTED qualification proved same-request mixed snapshots in Overview and Student Drill. A non-permanent REPEATABLE READ experiment resolved both, but permanent adoption remains PROPOSED - GOVERNANCE REQUIRED and NOT IMPLEMENTED. Any candidate transaction scope begins before the first statement and covers all M10 evidence inputs. No index, schema, migration, or isolation change is approved.

B11-C must collect query count/execution time, route and privacy-closure time, peak/result memory, and rows/scans/buffers through EXPLAIN evidence against a representative non-production PostgreSQL dataset. B11-D qualifies the current READ COMMITTED behavior first with controlled concurrent writers and representative M10 reads. REPEATABLE READ is not pre-approved; any stronger isolation must be evidence-driven, governed by a new ADR, and span access context, aggregation, Candidate/Identification reads, and privacy inputs.

No index is approved without EXPLAIN/ANALYZE evidence of material plan improvement, overlap/redundancy review, migration review, documented rollback, and a governing ADR. This governance checkpoint adds no schema, migration, index, isolation, or database behavior.

Deterministic Talent Analytics (M9, No Schema Change)

M9 adds no table, column, or migration. It is a strictly read-only aggregate query layer over the existing M1-M8 schema (`talent_assessment_cycle_population_members`, `talent_student_assessments`, `talent_student_competency_results`, `talent_review_candidates`, `talent_official_identifications`, `talent_annual_evaluation_plans`/`talent_planned_evaluation_periods`) - every authorized-raw-aggregate query in `talent_analytics_service.py` filters directly against these existing tables' own tenant/Branch/historical-scope columns; no new persistence authority, cache table, or materialized projection exists.

Talent Annual Evaluation Plans And Periods (M8)

`talent_annual_evaluation_plans` composite-binds one Plan to its exact Program Academic Year Configuration and enforces one Plan per configuration, bounded lifecycle, revision, and rollover source scope. `talent_planned_evaluation_periods` composite-binds each Period to its Plan and enforces positive unique sequence, normalized label/code uniqueness, date order, and cancellation consistency. `talent_assessment_cycles.planned_evaluation_period_id` is nullable for pre-M8/ad-hoc Cycles, unique, and composite-bound by SchoolGroup/Program/year. No historical rows or links are fabricated. `TalentConfigurationAudit` adds only bounded Plan and Period resource types.

Learner Profile Read Model (M7, Complete)

M7 adds no profile table, migration, materialization, or denormalized state. `talent_learner_profile_service.py` reads one exact SchoolGroup/Student's M1-M6 canonical rows with batched dependent queries. Historical Placement branch, frozen Cycle member branch, and persisted Educator Input branch remain the record-level authorization authorities. Profile base permission does not bypass the independent Review Candidate, Official Identification, or Educator Input view permissions. Future M8 Annual Evaluation Plan and Periods therefore can attach its own period semantics without reworking M7's multiple-Cycle history model.

Talent Review, Official Identification & Educator Input (M6, Complete)

Independent review tightened the database boundary: Official Identification has an exact composite FK to its Review Candidate context, and Educator Input lineage has a unique successor plus same-scope composite self-FK. These make context drift and concurrent amendment forks invalid independently of service validation.

Migration `20260904_006_talent_review_candidate_foundation` adds `talent_review_candidates` (one row per Assessment via `uq_talent_review_candidates_assessment`) with composite scoped foreign keys to `talent_student_assessments` (assessment scope), the frozen `talent_assessment_cycle_population_members` row (Branch authorization authority, matching M4/M5), and `talent_review_candidate_policies` (policy identity). It stores match mode, a SHA-256 evaluation fingerprint, a full JSON evaluation snapshot (per-rule threshold/actual/satisfied detail), and evaluator/timestamp provenance - no free text. The migration also widens `talent_assessment_audits.resource_type` to add `review_candidate`, reusing the exact M3 CHECK-widening pattern (PostgreSQL `NOT VALID`/`VALIDATE`, SQLite table rebuild). Only a qualifying (policy-satisfied) evaluation is ever persisted as a `TalentReviewCandidate` row; a non-qualifying evaluation is now structurally audited instead (Decision 1).

Migration `20260904_007_talent_review_workflow_identification_educator_input_foundation` adds `status` (`pending_review`/`reviewed`, CHECK-constrained), `reviewed_by_user_id`, `reviewed_at` to `talent_review_candidates`; creates `talent_official_identifications` (composite scoped FKs mirroring `talent_review_candidates`' own upstream chain, plus a direct FK to the Review Candidate, `decision` CHECK `IN ('identified','not_identified')`, `UNIQUE(review_candidate_id)` - exactly one decision per candidate); creates `talent_educator_inputs` (required composite scoped FKs to Student/Program/ AcademicYear/Branch plus the frozen Placement, optional simple FKs to Cycle/ Cycle Population Member/Assessment/Review Candidate, `category` CHECK closed enum, bounded `content`, self-referential nullable `supersedes_educator_input_id` lineage); widens `resource_type` further (`review_candidate_review`, `official_identification`, `educator_input`); and relaxes `talent_assessment_audits.cycle_id`/`framework_version_id` to nullable, since an Educator Input audit row may have no Cycle/Framework context at all (the composite FK is simply unenforced whenever a referencing column is NULL - standard SQL MATCH SIMPLE semantics). Both migrations follow the same dialect-branched widening pattern.

Talent Student Assessment And Competency Results

Migration `20260904_005_talent_student_assessment_competency_results` adds `talent_student_assessments` and `talent_student_competency_results` without a backfill. Assessment rows carry exact SchoolGroup, Cycle, frozen population member, Student, Program, Academic Year, and Framework

Version context and are unique per Cycle/Student. Composite foreign keys bind the Assessment to the immutable frozen member and Results to their Assessment, exact Framework Competency, and exact Framework-owned Rubric Level. The migration also extends the existing `talent_assessment_audits` check and contextual columns; it does not create another audit subsystem.

An optional completed KPI is stored directly on the Assessment as bounded historical provenance: method, integer result, configured scale bounds, weighted numerator, SHA-256 calculation fingerprint, and timestamp. It is NULL for qualitative/no-KPI and non-complete assessments. No universal score, cross-Program normalization, candidate, identification, or AI table exists.

Talent Assessment Cycle Frozen Population

M4 adds `talent_assessment_cycles` as the SchoolGroup-wide Program + Academic Year + exact Framework authority, with explicit effective time, one-way Draft/Open/Closed lifecycle, revision, and organization-level population count/fingerprint. `talent_assessment_cycle_population_members` freezes exact Student + Academic Placement + Academic Year + Branch + grade + section context through composite scoped foreign keys; current Placement and nullable PlanningSection provenance are never historical rendering authority. `talent_assessment_audits` is the bounded append-only operational lifecycle audit. While a Cycle is Open, accepted ADR 0033 permits only additive member snapshots after a confirmed eligible Placement; existing members/evidence are never changed, and Closed membership remains final. Migration `20260904_004_talent_assessment_cycle_frozen_population` creates all three tables and the exact Academic Placement composite unique target required by the member FK.

Teacher Scheduling Rules

Migration `20260830_001_teacher_scheduling_rules` adds `teacher_scheduling_rules`, `teacher_scheduling_rule_slots`, and `teacher_scheduling_rule_targets`. The header carries exact SchoolGroup, branch, academic-year, and teacher scope plus rule type, hard/soft semantics, target scope, active state, actor metadata, and timestamps. Slot children normalize all/selected days with numbered, first, or last period selectors; target children normalize selected grades or composite-scoped Planning sections.

The migration adds or verifies `(teachers.id, branch_id, academic_year_id)` as a composite parent key before creating the dependent rule table. All three new tables are excluded from baseline `metadata.create_all()` and created by the ordered, idempotent migration, preserving the production migration sequence on PostgreSQL and SQLite. No backfill occurs: absence of rules preserves existing generation behavior.

Migration `20260830_002_teacher_scheduling_window_semantics` adds the additive `restrict_to_window` discriminator. Existing rows default false and retain their original Must-teach occupancy meaning. New Schedule-within rules store the same validated hard-rule family with the discriminator true and serialize canonically as `schedule_within`; this avoids rebuilding SQLite check constraints and prevents silent reinterpretation of deployed rows.

Curriculum Adjustment Audit And Transaction

Migration `20260829_001_curriculum_adjustment_apply_foundation` adds `curriculum_adjustment_audits`. Each applied record carries UUID identity, exact SchoolGroup/branch/year, actor/time, scope and source/target subjects, reviewed fingerprint and request, per-section before/after periods and teacher decision, warnings, Draft linkage/stale state, regeneration requirement, and applied status. A unique scope+fingerprint constraint prevents duplicate apply.

The apply service locks and revalidates scoped Planning sections/demands, assignments, rules, timetable versions, and active generation state. Demand, assignment, section-rule retirement, Draft stale/approval state, and audit insert commit together or roll back together. Snapshots, published versions, placements, and `TimetableActiveVersion` are not rewritten.

Curriculum Adjustment Preview Read Boundary

Stage 3 adds no table or migration. The preview reads exact-scope Current/New `PlanningSection`, resolved `PlanningSubjectDemand`, scoped Subjects and Teachers, section assignments, teacher subject allocations, Subject Distribution Rules, Timetable Settings grouped configuration, and the newest mutable Draft Timetable. Its SHA-256 fingerprint binds the canonical analysis and Draft revision/authority; it is returned to the caller but is not persisted. Published versions, active pointers, demands, and assignments are never mutated by preview.

Planning Subject Demand Foundation

`planning_subject_demands` normalizes future weekly-period authority at the Planning-section and subject level. Each row carries branch/year scope, `planning_section_id`, scoped `subject_code`, weekly periods, active/retired state, timestamps, and optional creating/updating actors. Composite foreign keys require the section and Subject to belong to the row's branch/year. A partial unique index allows at most one active row per section and subject while retaining inactive rows.

Migration `20260828_004_planning_subject_demands_foundation` inserts missing active rows for Current/New Planning sections joined to Subjects by branch, academic year, and normalized grade. `NOT EXISTS` for any prior demand makes the backfill repeat-safe without reactivating retired rows. The demand service bulk-resolves Current/New sections inside one exact branch/year: any explicit row, including inactive or zero, is authoritative; only a missing row falls back to the grade-matched Subject weekly value. Live Planning, workload, timetable inputs, scheduling-rule arithmetic, and required-hours reporting consume this resolved demand. Historical published timetable data is not recomputed.

Subject Scheduling Rules UI

No schema change was required. `subject_distribution_rules_ui.py` lists, creates, updates, resets, and copies `subject_distribution_rules` rows through the existing Stage 1 table and Stage 1/2 resolver/validator; Timetable Settings routes under `/system-configuration/timetable-settings/subject-rules` remain tenant/branch/year-scoped and reuse `timetable.manage_settings`. The grade-first modal, search, two-period presets, and progressive disclosure are presentation only; Planning totals, normalized rule persistence, legacy fallback, and grouped Swimming authority are unchanged. Operational timetable grade and section selectors use `planning_scope_service.py`, which reads only Current/New `PlanningSection` rows for one exact branch and academic year; global grade catalogs remain creation/edit choices and are not operational selector authority.

Subject Distribution Rules Generation Wiring

`timetable_snapshot_service.py` resolves and embeds the effective Subject Distribution Rule per Planning demand at snapshot creation time using `subject_distribution_rules.resolve_subject_distribution_rule`. The problem builder, CP-SAT solver, independent validator, and `TimetableReadinessService` all consume that resolved, per-demand authority; no schema change was required beyond the Stage 1 table. `TimetableActiveVersion` and the published/draft/history lifecycle are unaffected.

Subject Distribution Rules Foundation

`subject_distribution_rules`, added by migration `20260828_003_subject_distribution_rules_foundation`, is a normalized table scoped `branch_default`, `grade`, or `section` per branch/academic year, with partial unique indexes enforcing exactly one row per scope tier. It coexists with `TimetableSetting.quality_rules_json`; an absent normalized row means the existing JSON authority applies unchanged for that scope.

Timetable Academic Quality Configuration

`TimetableSetting.quality_rules_json`, added by migration `20260828_002_smart_timetable_academic_quality_rules`, stores normalized exact-scope academic distribution and grouped-activity authority. Generation copies it into an immutable snapshot; timetable versions and published-pointer relationships are unchanged.

On-Demand Generation Execution

No schema migration is required for Render Workflows. The existing generation run public ID identifies task input; status, exact scope, snapshot, attempt, lease, heartbeat, cancellation, safe failure, and result columns remain authoritative. An on-demand task locks and claims exactly its queued public ID. Duplicate task invocations see an active or terminal row and do not solve or persist again. Immediate web dispatch failure atomically marks only a still-queued run `internal_error` with `workflow_dispatch_failed`. The partial unique active-scope index and atomic candidate transaction remain unchanged.

Smart Timetable Stage 5.1 Queue Migration

Migration `20260822_001_smart_timetable_stage51_generator` is additive and idempotent. It extends the existing `timetable_generation_runs` table with `progress_phase`, non-negative `attempt_count`, `cancel_requested_at`, and a nullable cancellation actor foreign key. It adds a `(status, lease_expires_at, queued_at)` claim index and a partial unique exact-scope index for active statuses. Duplicate active scopes fail migration preflight instead of being guessed or rewritten. PostgreSQL receives named progress/attempt checks and the actor FK.

No parallel queue table, lesson-instance column, room/resource table, quality score, or active-pointer mutation is introduced. Successful persistence inserts one `TimetableVersion`, all `TimetableEntry` rows, snapshot/run linkage, and terminal run state in one transaction; failure rolls the candidate back.

Capacity-Based Packaging Reconciliation

Migration `20260819_001_capacity_based_customer_feature_baseline` is additive and idempotent. It upserts the nine normal customer `PlanEntitlement` booleans to active/true for Starter, Professional, and Enterprise AI, synchronizes the legacy AI, Advanced Reporting, and multi-branch availability flags, and leaves `priority_support` separate. It does not update any branch/staff/teacher ceiling or Paddle quantity.

For a promo grant that is active at migration time, missing or false normal customer `WorkspaceEntitlementValue` rows are inserted or corrected. The derived branch quota, staff/teacher grant capacity, PromoGrant, PromoRedemption, branch assignments, tenant source, and payment tables are untouched. Expired or incoherent promo authority is not reactivated or reconciled.

Active customer-demo policy rows receive the normal baseline while retaining stored experimental keys and all AI allowance/unrestricted configuration. Paid workspaces need no value backfill because their workspace entitlement resolves the current PlanEntitlement matrix. `feature.audit_log` remains `owner_approval_required`; the implemented audit export is Developer-only.

AI counters now permit the existing `promo` metric context in addition to paid, demo, and internal contexts through the already text-based field. No new AI table or provider billing record is introduced.

SchoolGroup Creation Provenance Audit

No schema or migration is added for the Stage 5 authorization correction. Direct operational create/delete is restricted to Platform identities with global SchoolGroup capability, while tenant updates resolve the linked SchoolGroup before a row is loaded for mutation. Branch capacity continues to lock and recount the owning SchoolGroup in the mutation transaction.

`scripts/audit_schoolgroup_provenance.py` queries active `internal_sandbox` SchoolGroups that lack both `TenantProvisioningLink` and explicit `WorkspaceEntitlement` evidence. It reports only a numeric review key, hashed workspace reference, timestamp, aggregate operational counts, and review guidance. The PostgreSQL transaction is repeatable-read and read-only and is always rolled back. Findings are never automatically linked, reclassified, suspended, or deleted.

Promo Redemption And Grant Persistence

Migration `20260805_002_promo_redemption_and_grants` creates:

1 `promo_activation_sessions` and `promo_activation_branch_selections` for

short-lived, resumable, non-authoritative customer review state;

1 `promo_redemptions` for completed immutable redemption evidence;

1 `promo_grants` for immutable plan, capacity, scope, and effective-window

authority;

1 `promo_grant_branch_assignments` for selected covered operational branches;

1 `promo_redemption_events` for redacted, idempotent durable outcomes.

Raw promo material is absent from every table. Snapshot hashes bind the definition, scope, tier, limits, and effective dates used at activation. Unique and partial indexes protect session/idempotency identity, one redemption per session, one grant per redemption, one active grant per workspace, branch assignment uniqueness, and durable operation/event deduplication.

`workspace_entitlements.promo_grant_id` links only promo entitlements. `tenant_provisioning_links.pending_organization_id` is nullable for an existing aligned tenant and `promo_grant_id` identifies promo ownership. A check requires exactly one of subscription contract, demo request, or promo grant. The migration preserves existing rows, rebuilds equivalent SQLite constraints where required, installs PostgreSQL foreign keys/checks/indexes, and is idempotent.

Promo Code Foundation Persistence

`promo_codes` owns UUID identity, HMAC-SHA256 lookup hash and key ID, safe display fragments, controlled lifecycle, exact plan-bounded capacity, target anchors and deletion-safe snapshot, redemption-policy definition, one expiry policy, replacement predecessor, and actor/approval timestamps. Unique indexes protect UUID, lookup hash, and one replacement per predecessor. CHECK constraints enforce lifecycle, benefit, scope, positive capacities, version, redemption limits, date ordering, expiry XOR, and required primary targets.

`promo_code_branch_restrictions` preserves the selected existing branch ID and name even if the live branch reference is later removed. It grants no access. `promo_code_audit_events` stores actor, allowlisted before/after JSON, result, reason, operation/correlation identity, and failure code without raw code, lookup hash, HMAC key ID, or secret. Organization and actor references use `ON DELETE SET NULL` while snapshots preserve history. Migration `20260805_001_promo_code_foundation` is additive, idempotent, transaction-safe, SQLite/PostgreSQL compatible, and has no data backfill.

M8B9 Demo Operations Persistence

`demo_access_policies` stores a workspace default and optional branch override with controlled profile/registry values. `demo_operation_audits` stores actor, scope, action, reason, before/after JSON, result, communication references, failure code, and operation key. Provisioning adds `expiry_policy`; email intents add variant payload JSON. Idempotent migration `20260727_003_m8b9_demo_operations` uses the active DDL connection and existing

pre-deploy PostgreSQL lock and statement timeout protections.

Deployment Migration Boundary

Database schema work belongs to `python scripts/run_migrations.py`, never the FastAPI web process. The command registers core and SaaS metadata, creates the repository's baseline metadata schema, and then calls the ordered, authoritative `db_migrations.run_pending_migrations` ledger. It logs each newly-applied identifier, is idempotent when the ledger is current, and exits nonzero on any schema or migration failure. Render must use it as a Pre-Deploy Command so a failed migration prevents the new web version from becoming active. Migration-owned PostgreSQL lock and statement timeout protections remain in force. The migration process additionally applies a 10-second connection timeout, 5-second lock timeout, and 30-second statement timeout at PostgreSQL connection creation so baseline `metadata.create_all()`, `schema_migrations` initialization, every ordered migration, and commit are bounded. Flushed progress logging brackets each phase and migration identifier. Within a migration transaction, catalog inspection and DDL must use the same SQLAlchemy `Connection`; helper code must not inspect the `Engine`, because a second PostgreSQL session can wait on uncommitted DDL locks held by the first session and self-deadlock the migration process.

M8B8 AI Usage Persistence

`ai_feature_usage_counters` is unique by SchoolGroup, registered feature key, and metric context (`internal_sandbox`, `demo`, or `paid`). Its locked successful and reserved counts provide the concurrency-safe allowance boundary. `ai_feature_usage_events` is the durable operation ledger and uniquely deduplicates an operation within the same scope. Pending reservations become successful or failed; failure releases capacity and does not increment usage. Classification and plan snapshots preserve historical metric separation without changing workspace classification authority.

M8B7 Demo Communication Persistence

`saas_demo_email_deliveries` is the durable email outbox. It links each logical message to a demo request and optionally its provisioning aggregate, constrains email type and delivery state, and uniquely deduplicates delivery. `system_notifications` carries destination, deduplication, category, and severity metadata for owner demo events. Existing request, provisioning, lifecycle, entitlement, and conversion rows remain authoritative.

This document explains the conceptual TIS data model. It is intentionally high level and does not list every field. Use `models.py`, `saas/models.py`, and tests for exact implementation details.

Core Boundary Principle

TIS has three identity/data worlds that must never be casually mixed:

1. Platform data: platform owners, co-owners, developers, platform permissions, and cross-organization oversight.
1. SaaS account data: public signup, onboarding, billing, pending organizations, and payment/provisioning readiness.
1. Operational tenant data: provisioned school groups, branches, academic years, users, teachers, subjects, planning, timetable, calendar, and observations.

The safest mental model:

```
Platform Owner oversees many organizations.
SaaS account prepares an organization for subscription/provisioning.
Operational tenant data belongs to one provisioned school group/branch/year context.
```

Platform Identities

Represents: Platform Owner, Co-Owner, and Platform Developer identities.

Ownership boundary: Platform identities can operate outside ordinary tenant scope only through approved platform workflows.

Must never be mixed:

- Platform Developer must not become Platform Owner through permission drift.
- Platform users must not be treated as normal tenant users unless an explicit context workflow establishes scope.

Related files:

- `auth.py`
- `models.py`
- `permission_registry.py`
- `main.py`

SaaS Accounts

Represents: Public SaaS signup/login/account identities used before or alongside operational tenant access.

Ownership boundary: SaaS accounts own onboarding and billing/account state, not operational school records directly.

Must never be mixed:

- SaaS account identity is not operational user identity.
- SaaS account creation must not directly create live tenant data.

Related files:

- `saas/models.py`
- `saas/service.py`
- `saas/router.py`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Pending Organizations

Represents: An organization moving through SaaS onboarding before operational provisioning.

Ownership boundary: Pending organization data belongs to the SaaS onboarding/provisioning pipeline.

Must never be mixed:

- Pending organization data is not the live school group until provisioning creates or connects operational records.

Related files:

- `saas/models.py`
- `saas/service.py`
- `saas/provisioning_service.py`

Payment Records

Represents: Payment, billing, checkout, and provider-confirmed subscription state.

Ownership boundary: Payment state belongs to SaaS billing/subscription logic and should not be embedded directly into academic modules.

Subscription capacity uses three separate persisted dimensions. `pending_organization_branches.estimated_system_users` and `estimated_teachers` store required non-negative onboarding estimates. `subscription_plans.max_branches`, `max_system_users`, and `max_teachers` store plan limits. Migration `20260729_001_subscription_capacity_dimensions` adds the new columns and assigns legacy organization-level system-user and teacher totals to the primary active branch only when all branch estimates are zero. Subsequent organization totals are derived compatibility summaries.

Must never be mixed:

1 Checkout return navigation must not be treated as verified payment.

1 Payment/provider details must not leak into operational teacher/planning/calendar logic.

Related files:

1 `saas/payment_service.py`

1 `saas/billing_service.py`

1 `saas/paddle_client.py`

1 `saas/entitlement_service.py`

1 `saas/subscription_change_service.py`

1 `saas/subscription_plan_change_service.py`

1 `saas/subscription_cancellation_service.py`

1 `saas/subscription_lifecycle_service.py`

1 `saas/billing_history_service.py`

1 `docs/adr/0003-paddle-payment-architecture.md`

1 `docs/adr/0004-webhook-only-payment-confirmation.md`

Entitlement And Subscription-Change Records

`EntitlementDefinition` defines commercial capability keys and value types. `PlanEntitlement` associates reviewed values with subscription plans. Runtime entitlement resolution starts from the provisioned school group, paid operational contract, and one confirmed active `PaymentSubscription`; it does not trust onboarding selections, page values, or pending checkout attempts.

`SubscriptionChangeRequest` is durable workflow/audit state for branch quantity, plan transition, and cancellation actions. It records requested/provider-observed state and lifecycle outcomes, but Paddle remains authoritative for monetary previews, proration, scheduled changes, transactions, and invoice documents.

Existing Workspace Paid Activation

Migration `20260806_002_existing_workspace_paid_activation` adds a direct paid activation context for an existing SchoolGroup. `ExistingWorkspacePaidActivation` stores workspace/account/owner identity, selected plan and interval, provider price, branch quantity and immutable selection hash, quote fingerprint and amounts, idempotency, checkout/payment/contract lineage, provider transaction/subscription identity, lifecycle, and safe failure state. Versioned branch rows retain stable branch snapshots, and append-only redacted events retain lifecycle evidence.

CheckoutSession and PaymentAttempt enforce an XOR between PendingOrganization and existing-workspace activation contexts. SubscriptionContract and PaymentSubscription permit a null PendingOrganization only when

commercial authority is anchored through SchoolGroup/contract. OrganizationBillingProfile supports an exclusive PendingOrganization or SchoolGroup context. Explicit PaymentCustomerWorkspaceAssociation rows prevent implicit provider-customer sharing. They persist the provider address and business selected for each SchoolGroup so workspace billing identity remains stable even when the underlying customer is shared. Unique indexes enforce one unresolved activation per SchoolGroup and unique provider transaction/subscription mappings. No tenant-owned operational table is copied.

Guardrails:

- `PaymentSubscription.quantity` is paid branch-capacity authority.
- unresolved ownership, duplicate active relationships, provider mismatches, or incomplete evidence fail closed.
- scheduled changes do not update effective local entitlements before verified provider/webhook evidence.
- billing history is retrieved from Paddle and is not copied into a new local financial ledger.
- invoice URLs are requested fresh and are not persisted.

Provisioning Jobs

Represents: Controlled work that turns a ready pending organization into operational school structures.

Ownership boundary: Provisioning bridges SaaS data and operational tenant data, but only through explicit platform-owner-visible workflows.

Must never be mixed:

- Do not directly create operational tenant records from public signup or checkout return pages.
- Do not create records under the wrong school group.

Related files:

- `saas/provisioning_service.py`
- `saas/router.py`
- `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

School Groups / Organizations

Represents: The top-level operational tenant boundary.

Ownership boundary: Most tenant operational data belongs to a school group directly or through branch/year relationships.

Must never be mixed:

- Data from one school group must not appear in another school group's operational workflows.

Related files:

- `models.py`
- `main.py`

Workspace Classification Metadata

`SchoolGroup` is the canonical operational workspace record. M8B-1 adds:

- globally unique, non-null `workspace_uuid`,

- constrained/indexed `workspace_classification`,
- constrained/indexed `workspace_lifecycle_status`.

Pre-provisioning intent remains on `PendingOrganization.workspace_intent`; identity intent remains on `SaaSAccount.account_purpose` and `User.is_internal_test_identity`. These fields are metadata only in M8B-1 and are not joined into payment, entitlement, authorization, or reset decisions.

Allowed workspace classifications are `internal_sandbox`, `customer_demo`, and `customer_paid`. Allowed lifecycle values are `provisioning`, `active`, `suspended`, and `archived`. New-schema tables use named check constraints and non-null columns. The compatibility migration fills legacy values, adds indexes and PostgreSQL constraints, and installs equivalent SQLite value guards without rebuilding existing tables.

The read-only diagnostic resolves relationship presence across `TenantProvisioningLink`, `PendingOrganization`, `SubscriptionContract`, `PaymentSubscription`, and `PaymentCustomer`. It reports no Paddle identifiers. The controlled backfill is one transaction, defaults to dry-run, records a durable marker, and never performs a workspace conversion.

Commercial Entitlement Records

M8B-2 adds three normalized tables:

- `workspace_entitlements`: one effective entitlement envelope per SchoolGroup, with type, lifecycle status, source, optional confirmed payment-subscription link, and a validity window reserved for later workflows.
- `workspace_entitlement_values`: typed feature/limit values linked to the existing `EntitlementDefinition` catalog.
- `branch_entitlements`: optional branch-level inherit/active/inactive intent linked to the branch, SchoolGroup, and effective workspace entitlement.

A partial unique index permits only one active workspace entitlement per SchoolGroup. Branch entitlement is unique per branch. Check constraints protect entitlement type, status, source, mode, and validity-window ordering. Service validation additionally rejects cross-tenant branch links, stale workspace-entitlement references, invalid typed values, and classification/entitlement mismatches.

Migration `20260722_003_commercial_entitlement_foundation` seeds one foundation entitlement for each existing classified workspace without changing classification. It does not create branch overrides, convert Al-Andalus, or infer demo/paid policy. Paid rows are linked only when exactly one persisted active/trialing subscription can be identified; ambiguity remains unresolved and fails closed.

SaaS Demo Request Records

M8B-3 adds three review-only tables:

- `saas_demo_requests`: one customer submission with requester, pending organization, optional future workspace reference, immutable classification/commercial/entitlement snapshots, status, and transition timestamps.
- `saas_demo_request_reviews`: one Platform Owner approval or rejection decision per request; rejected reviews require a reason.
- `saas_demo_request_events`: append-only audit and internal-notification events for submission and every status transition.
- `saas_demo_domain_eligibilities`: one unique normalized organization-domain reservation for a Customer Demo opportunity, optionally linked to historical request evidence or marked for manual review when legacy records are ambiguous.

A partial unique index permits only one Pending Review request per pending organization. Check constraints protect request status, review decision, event category/type/actor, classification snapshot, commercial-state snapshot, and rejection-reason requirements. Migration `20260722_004_saas_demo_request_workflow` creates the records without backfill and without changing existing onboarding, payment, or workspace data.

Migration `20260725_001_demo_domain_eligibility_policy` adds customer journey-intent fields, a nullable normalized domain snapshot, and the eligibility table with a database unique invariant. It backfills unambiguous Customer Demo records, reserves ambiguous duplicate domains for manual review, and never merges, deletes, reprovisions, or replaces customer workspaces. `scripts/diagnose_demo_domain_eligibility.py` is dry-run by default and can apply the same safe reservation backfill after review.

Historical detached eligibility rows are handled without schema changes through a Platform Owner-only maintenance service. It treats the eligibility primary key as the only deletion scope and checks domain-bearing organization, account, request, tenant-profile workspace, provisioning, subscription, and conversion relationships plus manual-review evidence before removal. The row is locked and re-analyzed, then deleted by exact ID, flushed, verified absent, and committed atomically with a durable external audit record. Normal customer eligibility rows remain durable.

Demo Workspace Provisioning Records

M8B-4 adds:

1 `saas_demo_workspace_provisioning`: one durable provisioning aggregate per demo request, with optional resulting SchoolGroup, workspace entitlement, and tenant link references; attempt count; status; activation time; and safe result/failure fields.

1 `saas_demo_provisioning_events`: append-only provisioning audit/internal events for started, completed, failed, and activation-completed outcomes.

`tenant_provisioning_links` now permits either `subscription_contract_id` or `demo_request_id`, with a check constraint requiring exactly one source. Existing paid links retain their subscription contract. Demo links cannot carry a contract, and request/source uniqueness prevents one approved request from identifying multiple operational tenants.

Migration `20260723_001_demo_workspace_provisioning` generalizes the existing link without changing paid rows and creates the demo provisioning/event tables. Demo workspace, entitlement, link, request association, and activation updates are performed in one savepoint-backed transaction; failed workspace changes roll back while the outer provisioning aggregate retains the failure for audit and retry.

Demo Workspace Lifecycle Records

M8B-5 extends `saas_demo_workspace_provisioning` with reminder due/sent, demo expiration, expired, processing-status, last-processed, and failure-code metadata. `activated_at` remains the authority; persisted reminder and expiration timestamps are derived indexes and must validate as activation plus six and seven days.

Two normalized tables support lifecycle history and in-app delivery:

1 `saas_demo_lifecycle_events`: deduplicated audit records for reminder due/delivery, expiration start/completion, workspace suspension, access blocking, and processing failure.

1 `saas_demo_lifecycle_notifications`: recipient-scoped internal reminder notifications for the requesting SaaS account and active Platform Owners.

Migration `20260723_002_demo_workspace_lifecycle` adds constrained lifecycle metadata, backfills activation-derived timestamps for existing active demos, aligns demo entitlement `effective_to`, and creates the event/notification tables. It does not expire data during migration. The separately scheduled processor performs expiration after a dry run.

Demo-To-Paid Conversion Records

M8B-6 adds:

- `saas_demo_to_paid_conversions`: one durable conversion aggregate per demo request, provisioning record, and SchoolGroup. It links the existing demo workspace to the confirmed `SubscriptionContract` and `PaymentSubscription`, records the previous demo entitlement and resulting paid entitlement, and tracks requested, processing, completed, or failed status.
- `saas_demo_conversion_events`: append-only audit and internal-notification history for conversion requested, started, completed, and failed outcomes.

Migration `20260723_003_demo_to_paid_conversion` also permits the terminal `converted` demo lifecycle processing status. Conversion never creates another SchoolGroup or tenant link. One atomic workspace transaction ends the demo entitlement, creates the paid entitlement, relinks existing branch entitlements, changes the SchoolGroup classification, and moves the existing tenant link from the preserved demo-request source to the confirmed subscription contract. Provider payment records commit independently and remain intact if workspace conversion rolls back.

Related files:

- `workspace_classification.py`
- `saas/workspace_classification_service.py`
- `saas/workspace_classification_admin_service.py`
- `scripts/diagnose_workspace_classification.py`
- `scripts/backfill_workspace_classification.py`
- `docs/adr/0008-workspace-classification-foundation.md`
- `commercial_entitlements.py`
- `saas/commercial_validation_service.py`
- `saas/workspace_entitlement_service.py`
- `saas/branch_entitlement_service.py`
- `saas/commercial_state_service.py`
- `docs/adr/0009-commercial-state-and-entitlement-resolution.md`
- `demo_workflow.py`
- `saas/demo_provisioning_service.py`
- `saas/demo_lifecycle_service.py`
- `saas/demo_conversion_service.py`
- `scripts/process_demo_lifecycle.py`
- `authorization.py`
- `saas/provisioning_service.py`
- `docs/adr/0011-demo-workspace-provisioning-and-commercial-source-links.md`
- `docs/adr/0012-seven-day-demo-lifecycle-and-access-enforcement.md`
- `docs/adr/0013-demo-to-paid-workspace-conversion.md`
- `saas/demo_request_service.py`
- `docs/adr/0010-review-only-saas-demo-requests.md`

Branches

Represents: Campuses or branches inside a school group.

Ownership boundary: Branches scope users, teachers, planning, timetable, academic calendar, branding, and reports.

Must never be mixed:

- Branch-scoped data must not silently cross campuses.
- Platform context switching must remain explicit.

Related files:

- `models.py`
- `main.py`
- `ui_shell.py`

Academic Years

Represents: The academic period used to scope operational records.

Ownership boundary: Planning, timetable, subjects, calendar, and related data may depend on active academic year.

Must never be mixed:

- Do not copy or read current-year data into another year unless the workflow explicitly does so.

Related files:

- `models.py`
- `main.py`
- `year_copy.py`

Operational Users

Represents: Users who work inside an operational school group/branch/year context.

Ownership boundary: Operational users belong to tenant structures and are governed by roles, permissions, branch scope, and active status.

Must never be mixed:

- Operational users are not SaaS accounts by default.
- Tenant users should not gain platform owner access.

Related files:

- `models.py`
- `auth.py`
- `routers/users.py`

Roles And Permissions

`RolePermission` logical keys have separate partial unique indexes: global `(role, permission_key)` where SchoolGroup is NULL, and tenant `(school_group_id, role, permission_key)` where SchoolGroup is not NULL. Migration `20260915_001_role_permission_logical_key_uniqueness` preflights duplicates without deleting rows and installs these indexes transactionally. Production duplicates require approved remediation before migration proceeds.

`UserPermissionOverride` is unique by `(user_id, permission_key)` per ADR 0040; SchoolGroup records ownership/audit context, not another identity dimension. The earlier migration-created table lacks the fresh ORM model's User/SchoolGroup foreign keys. Do not assume database-level orphan prevention on that path until an explicitly approved integrity inspection/remediation resolves the drift.

Represents: Role packages, permission keys, platform developer permissions, and route/action authorization.

Ownership boundary: Permissions decide what a user can view or change in the current scope.

Must never be mixed:

1 UI hiding is not enough; protected actions need route/service checks.

1 Owner controls must remain outside developer-assignable permission drift.

Related files:

1 `permission_registry.py`

1 `role_permission_service.py`

1 `authorization.py`

1 `auth.py`

Teachers

Represents: Teacher records, qualifications, capacity, workload, and staffing-relevant academic data.

Ownership boundary: Teacher records are tenant/branch/year-sensitive operational data.

Must never be mixed:

1 Teacher data from one branch or school group must not appear in another tenant's planning/reporting.

Related files:

1 `routers/teachers.py`

1 `teacher_qualifications.py`

1 `teacher_capacity.py`

1 `models.py`

Subjects

Represents: Subject catalog, colors, requirements, qualification relationships, planning and timetable dependencies.

Ownership boundary: Subjects are academic configuration data inside tenant/year context.

Must never be mixed:

1 Subject changes can affect planning, timetable, teacher matching, and reports; update all affected docs/tests.

Related files:

- `routers/subjects.py`
- `subject_colors.py`
- `models.py`

Sections / Classes

Represents: Class sections used by planning and timetabling.

Ownership boundary: Sections belong to operational branch/year structures.

Must never be mixed:

- Section structures should not cross branch/year boundaries accidentally.

Related files:

- `routers/planning.py`
- `models.py`

Workforce Planning

Represents: Teacher assignments, homeroom ownership, capacity, workload, subject coverage, and staffing needs.

Ownership boundary: Planning is operational data tied to school group, branch, academic year, teachers, subjects, and sections.

Must never be mixed:

- Planning changes can affect dashboards, reports, timetable, and staffing decisions.

Related files:

- `routers/planning.py`
- `teacher_capacity.py`
- `homeroom_defaults.py`

Timetable

Represents: Weekly lesson placement, timetable settings, durable versions, immutable input snapshots, one active-version pointer per scope, placement locks, and future generation-run evidence. `TimetableNonTeachingBlock` retains legacy start/end and period columns and adds `placement_mode`, `insert_after_period`, and `duration_minutes`. The additive migration defaults existing rows to `fixed_time`; compatibility composition does not rewrite their stored clock values. After-period metadata is authoritative for newly inserted blocks.

Ownership boundary: Timetable data depends on planning, teacher, subject, section, branch, and year context.

Must never be mixed:

- Timetable edits must preserve scheduling constraints and scope.
- `TimetableEntry` belongs to exactly one version and preserves section/teacher

collision guarantees inside that version. Its branch/year must match the pointed version.

- `TimetableActiveVersion` is the only active-selection authority and its

SchoolGroup/Branch/Academic-Year tuple must match the target version exactly.

1 Active, superseded, and archived versions are immutable. Mutable compatibility

edits occur on a draft copy.

Related files:

1 `routers/timetable.py`

1 `timetable_logic.py`

1 `timetable_version_service.py`

1 `timetable_snapshot_service.py`

1 `docs/adr/0026-versioned-constraint-based-smart-timetable-generation.md`

Academic Calendar

Represents: Events, academic dates, responsibilities, exports, and calendar settings.

Ownership boundary: Calendar records are branch/year/tenant scoped.

Must never be mixed:

1 Calendar events for one tenant or branch must not appear in another.

Related files:

1 `routers/academic_calendar.py`

1 `templates/academic_calendar.html`

Observations

Represents: Teacher observation records, feedback, scoring, evidence, and history.

Ownership boundary: Observation data is sensitive tenant operational data.

Must never be mixed:

1 Observation records must remain tenant-scoped and permission-protected.

Related files:

1 `routers/observations.py`

1 `templates/observation_*.html`

Knowledge Management System Artifacts

Represents: Markdown docs, ADRs, module history, generated PDF, manifest, and Knowledge Center read-only status.

Ownership boundary: Markdown under `docs/` is source of truth. Generated artifacts under `static/docs/` are snapshots.

Must never be mixed:

1 The app must not silently rewrite Markdown source docs.

1 The PDF must not be edited manually.

- Protected documentation access should go through owner-only routes.

Related files:

- `docs/`
- `scripts/generate_docs_pdf.py`
- `static/docs/`
- `knowledge_service.py`

Tenant Isolation Rules

- Always know the current school group, branch, and academic year.
- Do not assume platform users have tenant scope.
- Do not assume SaaS accounts have operational records.
- Keep pending SaaS organization state separate from provisioned operational data.
- Preserve route permissions and service-level checks.
- Tests touching cross-tenant data should be treated as high value.

Independent Permission Closure Review

docs/engineering/PERMISSION_CLOSURE_REVIEW.md

Phase 3 - Whole-Application Permission Qualification

Phase 3 audited the whole application against the objective that every surface (sidebar, Dashboard cards/tabs, direct GET routes, POST mutations, bulk/export/API paths, configuration controls and server-rendered action affordances) consumes ONE canonical effective permission result (`auth.get_allowed_permission_keys`), with no role-name shortcuts, tenant-scope drift, stale caches, or duplicated resolver logic. ADR 0040 precedence is unchanged; there is no second resolver, no schema change, no migration, and no new permission key.

Final registry classification (175 keys, 26 groups)

Machine-checked by `tests/test_permission_registry_matrix.py`. Consumers are discovered by AST and template scan of non-test source and compared with a reviewed table. The test fails on an unclassified new key, an active key with no consumer, a dormant key that gains a guard consumer, a lost consumer, or any unresolved key.

Status	Count	Meaning
A active-enforced	143	Tenant-assignable key with a discovered route, handler, middleware, service or template consumer
B platform-only, enforced	14	Platform-only key with a real guard consumer
C alias/composite	5	Governed by another guard or by identity
D dormant/reserved	13	No enforcement anywhere
E unresolved	0	none

Platform-only keys (24 = 22 developer-only + 4 owner-only, with 2 in both): 14 are B, 5 are C and 5 are D (`dashboard.view_all_schools`, `configuration.manage_global_defaults`, `system_owner.manage_subscriptions`, `system_owner.create_subscription_school`, `system_owner.run_startup_repairs`).

Aliases/composites (C): `system_owner.manage_developer_accounts`, `system_owner.manage_ownership`, `system_owner.transfer_ownership` (owner-identity gates; the key is only cited in the denial), `system_owner.view_cross_school_audit` and `system_owner.export_cross_school_data` (`match="any"` aliases of `configuration.view_audit_log` / `configuration.export_audit_log`).

Dormant/reserved (D, 13): `reports.view`, `teachers.import`, `teachers.export`, `subjects.manage_colors`, `planning.import`, `planning.export`, `observations.submit`, `observations.manage_templates`, `dashboard.view_all_schools`, `configuration.manage_global_defaults`, `system_owner.manage_subscriptions`, `system_owner.create_subscription_school`, `system_owner.run_startup_repairs`.

Differences from the prior disposition (re-verified rather than trusted): `planning.import` and `planning.export` were recorded as resolved with real consumers in `routers/planning.py`; that was stale (no planning import or export route or literal exists) and both are dormant. `observations.submit` was recorded as an alias of `observations.sign_evaluator`, but no code evaluates it, so it is dormant (enforcement is `observations.sign_evaluator` only). `dashboard.view_all_schools` is only cited in a denial-message label (the all-school scope gate is the access-scope identity) and is dormant. `dashboard.view_branch_summary` and `dashboard.view_reports` remain template (presentation) gates on the already route-guarded `/dashboard` page. `ai.use` is an active service-layer consumer: `saas/ai_entitlement_service.evaluate_ai_availability` passes the feature definition's `permission_key` (`ai.use`) to `entitlement_service.evaluate_feature_access`, which calls `auth.has_permission` with that key. The matrix does not credit the bare constant in `saas/ai_feature_registry.py`; it credits `saas/ai_entitlement_service.py` only when an AST check confirms all three links (the `AI_PERMISSION_KEY` constant holds `ai.use`, the feature definitions pass it as `permission_key=`, and `evaluate_ai_availability` passes `<feature>.permission_key` to `evaluate_feature_access`); removing any link fails `tests/test_permission_registry_matrix.py`. Runtime behaviour is proven by `tests/test_ai_entitleme`

`nt_service.py::test_ai_use_permission_key_is_the_runtime_gate_for_every_ai_feature` (every AI feature carries `ai.use`; the real service calls `auth.has_permission` with `ai.use`; an Administrator is allowed, a school-level Deny of `ai.use` yields `ai_permission_denied`, and removing the Deny restores access) plus the existing role-Limited denial test. Not proven: every individual AI feature key end to end (the runtime test exercises `ai.academic_assistant`; all features share the one `permission_key` constant and evaluator path), and any HTTP route that fronts AI.

Route authorization enumeration

`tests/test_permission_route_coverage.py` enumerates the real `main.app` route table: 468 routes (215 GET, 224 POST, 12 DELETE, 9 PATCH, 8 PUT; 253 non-GET). Evidence classes: 136 middleware rule (`authorization.PROTECTED_ROUTE_RULES`), 182 in-handler permission guard, 10 guard through a same-module helper, 34 platform-identity guard, 65 SaaS account-session guard, and 41 reviewed allowlist entries (public or pre-authentication routes, provider webhook, scope selectors, and the Talent organization-analytics routes guarded in the service layer). Zero are uncovered. Every middleware rule is also proven at runtime to deny a user with an empty effective permission set. Static guard reachability is necessary but not sufficient evidence; representative handler-level denials are exercised in `tests/test_permission_surface_consistency.py`. A full authenticated HTTP sweep of every handler was not performed.

Defects found and fixed

1 `POST /scope/organization` selected an organization context for any platform user

by identity alone, while its denial cited `system_owner.switch_all_schools`. A Platform Developer whose switch/manage-all-schools capability had been withheld could still switch organization. Proven by a failing test first; fixed by requiring platform identity AND the existing canonical helper `_can_manage_all_school_scopes` (`schools.manage_all_schools` or `system_owner.switch_all_schools`). The Owner (all keys) is unaffected.

1 Four unused duplicate raw `RolePermission` readers named

`_get_role_permission_rows` (`auth.py`, `main.py`, `routers/users.py`, `ui_shell.py`) and three unused `_get_allowed_permission_keys` wrappers were deleted. At the base commit each was only defined and never called (the `ui_shell.py` wrapper caught any exception and returned role defaults, which would have failed open had it ever been reachable, but it was dead code). Removing them eliminates duplicated resolver-adjacent code that could be revived as a second resolver. No behaviour change.

Dangerous-pattern scan

`tests/test_permission_dangerous_patterns.py` scans non-test source for role constants and literals used as authority, admin-style flags, platform identity checks, raw `RolePermission` / `UserPermissionOverride` access, form-supplied SchoolGroup ids, per-request permission snapshots stashed on ORM instances, `lru_cache`, module-level permission caches, and template role gates. Every hit is classified (20 approved helper, 15 valid identity boundary, 5 legacy safe, 0 escalations, 0 bypass defects) with exact per-file counts, so a new hit fails the test. Raw policy queries exist only in the two services (plus tenant purge and count code in `saas/workspace_*`). `user.permission_keys` snapshots are per-request and have no reader. No request-independent permission cache exists.

Freshness, role change, tenant and platform-only results

Allow to Deny to Allow for Role Package, School Override and User Exception is proven on fresh sessions across resolver, sidebar, Dashboard, route and POST action. Administrator to Editor to Administrator (and Editor to User) with a user exception and a tenant School Override present recomputes on the next request and retains the exception row. School Overrides and user exceptions never cross tenants, a tenant actor cannot write another tenant's overrides or global Role Packages, user exception scope uses the target user's school, a Platform Owner management target never

gains tenant membership, platform-only keys cannot be granted through Role Packages, School Overrides, User Exceptions or crafted role-permission POSTs, and platform routes stay identity-guarded.

Verification scope and limits

The route coverage test is structural: it matches guard-named calls in each handler's same-module call closure against the real route table, so a new mutating route with no guard-named call fails, but a guard that is called and ignored would still pass. The middleware runtime denial test stubs the resolver to an empty set and proves each `PROTECTED_ROUTE_RULES` rule fires. Accessibility of the permission-management templates is verified by template-source assertions plus a rendered-page test in `tests/test_permission_surfaces_accessibility.py` (the real User Exceptions panel and School Overrides page are rendered; every colour-classed Allow/Deny element must carry the same word as visible text, and "Effective", "School Override" and "Using Standard Role Package" states are asserted as text); no browser rendering was run. The existing Role Packages / School Overrides split, User Exceptions, permission management/qualification, tenant-isolation, platform-access and override route/service suites are the regression evidence for Phase 1 and Phase 2 behaviour. The read-only diagnostic CLI (`scripts/diagnose_user_permissions.py`) was re-qualified in the final hardening pass: `tests/test_diagnose_user_permissions.py` passed 6 of 6 (not-found is safe and minimal, unknown permission key gives no trace, user-override trace, no password hash or database URL in output, login-equivalent case-insensitive email resolution, direct invocation without `PYTHONPATH`); a manual direct run reported an unknown user as `not_found` (exit 1) and an unreachable database as the generic `database_unavailable` message (exit 2) with no URL, credential or host detail. The script resolves identity through `auth.resolve_login_user` and contains no commit, INSERT, UPDATE or DELETE. Read-only behaviour and the database-unavailable path were confirmed by source inspection and the manual run, not by a dedicated automated test.

ADR 0040 historical references

ADR 0040 remains authoritative for precedence semantics (built-in role package -> global `RolePermission` -> SchoolGroup `RolePermission` -> per-user override; user Deny wins; a user Allow cannot resurrect a role-level Deny). Helper names the ADR mentions describe the implementation at decision time. Current runtime authority is the canonical resolver and service path: `auth.get_allowed_permission_keys`, `role_permission_service` and `user_permission_service`, as documented in current KMS. The unused duplicate wrappers deleted in Phase 3 do not change the ADR's semantics; the ADR itself is intentionally left unedited.

Open Owner/product decisions (unresolved; not invented here)

Exactly four decisions remain open and require an Owner/product decision. None is resolved by Phase 3.

1 Teacher-create field-level enforcement: `create_teacher` persists subject

assignments, qualifications and capacity from the create form for any holder of `teachers.create`, although the edit route enforces `teachers.assign_subjects`, `teachers.manage_qualifications` and `teachers.manage_capacity` field-level.

1 Tenant-toggleable dormant keys: the tenant-assignable dormant keys

(`reports.view`, `teachers.import`, `teachers.export`, `subjects.manage_colors`, `planning.import`, `planning.export`, `observations.submit`, `observations.manage_templates`) remain toggleable in the Role Package editor with no effect; whether to hide, lock or implement them is undecided.

1 Whether role `User` may ever author observations: `routers/observations._is_teacher_user`

is a restrictive role-name compatibility rule (role `User` is the observed teacher and is denied observation create/edit/delete/evaluator-sign even when granted; reads are limited to the caller's own teacher record). It never authorises another user's data, so it is not a bypass and does not block this audit.

1 Orphan `UserPermissionOverride` rows on workspace deletion: workspace deletion in

`saas` purges `RolePermission` rows but no `UserPermissionOverride` rows for the deleted tenant (orphan exceptions cannot grant access); whether and how to purge them is undecided.

Not a decision: `dashboard.view_branch_summary` and `dashboard.view_reports` gate presentation only.

Role Permissions UI - Role Packages / School Overrides Split (Phase 1)

The live Owner reproduction below (Dashboard Tab/Panel Permission-Gate Closure) surfaced the underlying Role Permissions UI defect this Phase 1 pass corrects: the combined editor's single "Permission Scope: Global defaults / Selected school" dropdown made a real tenant Deny override (built-in Allow, global override none, tenant override Deny, effective Deny) visually indistinguishable from the Global default Allow, and the Platform Owner UI displayed the selected tenant's name even while "Global defaults" was selected. `/system-configuration/role-permissions` now serves two explicit modes (`mode=packages`, `mode=overrides`) reusing the unchanged `role_permission_service.py` resolver; School Overrides always labels its SchoolGroup selector as a management target, never as Platform Owner actor context, and read-only viewing never creates a tenant `RolePermission` row. An Owner-review pass over the running application (real browser rendering plus a live save/reset round trip on a throwaway database, never `tis.db`) further required: a Platform Owner must explicitly choose the School Overrides management target (no school is pre-selected); groups holding a School override open by default with an override count; and corrected accessibility semantics (`aria-current` navigation, no static `aria-expanded`, the Reset button kept out of the checkbox `<label>`, single-element banner text, explicit focus outlines). The live round trip confirmed the incident's resolution path: a tenant Administrator's fresh `GET /subjects` was denied while the tenant Deny override existed, allowed after "Reset to standard" + Save removed it, and denied again on re-deny, with no global row touched and a tenant actor unable to save Role Packages or another school's override. See `docs/PROJECT_STATE.md` for full detail and `tests/test_role_permissions_ui_split.py` for the regression matrix, including a constructed Al-Andalus-style fixture. This is a UI/route structural correction only; ADR 0040's precedence chain is unchanged and no permission data was read or mutated on any production row.

Dashboard Tab/Panel Permission-Gate Closure (Live Owner Reproduction)

A live Owner UI reproduction (Role Permissions -> Administrator -> Global defaults showing Subjects 8/8, then logging in as a real Al-Andalus tenant Administrator) found Subjects correctly hidden from the sidebar and correctly denied at the direct `/subjects` route, but still fully visible on that same user's Dashboard (tab + Subjects Workspace panel). This was traced end to end per the required chain: built-in Administrator default -> global `RolePermission` -> Al-Andalus tenant `RolePermission` -> per-user `UserPermissionOverride` -> `auth.get_allowed_permission_keys` -> shell/ sidebar context -> Dashboard context -> direct `/subjects` route authorization. Every layer up to and including `main.dashboard`'s own `build_shell_context()` call resolves and exposes the same canonical, per-user effective permission set via the `can(...)` template helper; the sidebar (`ui_shell._build_nav_items`) and the route guard (`authorization.enforce_route_permission`) both already consume that same canonical resolver correctly. The defect was that `templates/dashboard.html` itself never called `can("subjects.view")` / `can("teachers.view")` / `can("planning.view")` to gate its Subjects/Teachers/Planning tab buttons and panels -- they rendered unconditionally regardless of the resolved effective permission, while the pre-existing Reports tab already correctly called `can("dashboard.view_reports")`. This is the same defect class as this review's broader "current-user authorization (A) and UI capabilities (B) use canonical resolution" principle: (A)/(B) were both actually available in this template already, and simply were not invoked for these three tabs/panels.

Fix: wrap each of the three tab buttons and their matching panel `<div>` blocks in the matching `can(...)` check, and replace the previously hardcoded always-active Subjects tab/panel with a computed `dash_first_tab` (first permission-visible tab among Subjects, Teachers, Planning, Reports), so a user holding only `teachers.view` or `planning.view` still gets a valid default-active panel rather than a blank workspace. No change to `auth.py`,

authorization.py, role_permission_service.py, user_permission_service.py, ui_shell.py, ADR 0040's precedence chain, or any permission key/schema/migration -- all were inspected and confirmed already canonical.

Phase-1 read-only diagnosis of the tracked local tis.db (hash recorded before and after; unchanged) found it predates this feature area: no user_permission_overrides table exists, and its schema_migrations ledger ends at 20260822_..., before both 20260913_001_user_permission_overrides and 20260915_001_role_permission_logical_key_uniqueness; it also contains no tenant Administrator user matching the reproduced Al-Andalus scenario (only one platform developer identity). Diagnosis and the new regression suite (tests/test_dashboard_sidebar_permission_consistency.py) are therefore code/template-level trace and fresh in-memory-database test fixtures, not live-row inspection of that specific file; this limitation is recorded rather than worked around or fabricated. The new suite proves six cases (global Allow only; global Allow + tenant Deny; tenant Allow + per-user Deny; role Deny then re-grant with per-user Inherit; role Deny then re-grant with a persisted per-user Deny remaining in effect; and cross-tenant isolation) against sidebar, Dashboard markup, and the direct route guard together, parametrized over both subjects.view and a second, non-Subjects sentinel (teachers.view) to prove the fix is generic.

Review boundary and disposition

The review compares fix/permission-consistency-closure commit 28f98f2ee87c2263698325426d5e42c1e6bd6b8c with origin/dev 1e5c1797d66a1ff5f24b167a101fe4e2819a956c. Subsequent corrective work stays on the review branch. No merge, production migration, data cleanup, deployment, or complete whole-system security approval is implied.

The original 26 changed files were reviewed: .kms-impact.yml; auth.py; authorization.py; db_migrations.py; models.py; main.py; user_permission_service.py; ui_shell.py; routers/observations.py; routers/users.py; docs/CHANGE_HISTORY.md; docs/PROJECT_STATE.md; the generated KMS booklet/manifest; templates/notifications.html; templates/observation_form.html; templates/system_configuration_qualifications.html; templates/system_configuration_schools.html; and these eight test files: test_branch_year_permission_boundaries.py, test_configuration_scope_permissions.py, test_notification_group_permissions.py, test_permission_qualification.py, test_platform_access.py, test_role_permission_logical_key_migration.py, test_user_permission_override_routes.py, test_user_permission_overrides.py.

Additional inspection covers teacher and planning routes/forms, the permission registry, shared shell, application database configuration, migration runner, Talent guards, commercial/entitlement boundaries, and existing permission, tenant-isolation, and user-template tests.

Architecture and classifications

The canonical current-user resolver remains auth.get_allowed_permission_keys: built-in default, global role policy, tenant role policy, then per-user override. User Deny removes a role grant. User Allow does not resurrect role Deny; its stored row can reactivate after a role re-grant. ADR 0040 is unchanged.

Current-user authorization (A) and UI capabilities (B) use canonical resolution. Abstract role summaries (C) in main/users/shell legitimately use the role-only service. Scope/persona metadata (D), including organization authority and teacher identity, supplements rather than substitutes for permission guards. Shared shell can uses a canonical, per-render snapshot; it is not a cross-request permission cache. Remaining _permission_cache references are test residue or comments, not production consumers.

Confirmed corrections

The independent review corrected notification GET auto-read despite Deny, inactive platform helper capabilities, Developer creation role choices bypassing assignment authority, teacher bulk deletion checking the single-delete key, and teacher/planning edit routes mutating separately protected fields. Read-only controls do not silently clear persisted values. It also repaired a stale timetable-block nav assertion and a legacy migration fixture missing the production bootstrap's role-policy table prerequisite. The shared Talent/Students test grant helper now updates existing logical keys; its old duplicate inserts violated the newly installed uniqueness constraints during re-grant tests. Tenant resolution now rejects absent ownership, contradictory stored/branch ownership, missing SchoolGroup parent records, and foreign selected/explicit scopes rather than resolving another tenant's policy or falling back to global defaults for an unowned actor. Target management and override writes likewise reject contradictory ownership, including platform-actor attempts; this does not restrict valid cross-tenant platform authority. Branch deletion rejects unowned targets even for platform actors rather than silently turning a normal delete route into an orphan-data cleanup operation.

Verification limits and remaining approval blockers

The registry contains 175 keys across 26 groups. Literal consumer coverage is not proof of enforcement, and route-reference validity is not proof that all registered capabilities protect their associated workflows.

22-key closure pass (this pass)

Every previously-unconsumed key listed in the prior revision of this section was individually traced against the implemented product (routes, services, templates, nav, and the route-permission middleware in `authorization.PROTECTED_ROUTE_RULES`) and classified. `planning.import` and `planning.export` were previously recorded here as resolved with real consumers; the Phase 3 audit found that stale (no planning import or export route exists) and classified both dormant. The other 22 keys close as follows.

Enforced as an existing capability with a dedicated key (server-side gate added/aligned; UI aligned to match):

1 `dashboard.view_branch_summary` - gates the dashboard KPI/branch-overview

section (`templates/dashboard.html`) distinct from the Reports tab.

1 `dashboard.view_reports` - gates the dashboard's "Reports" tab and its

panel (`panel-reports`), a real, distinguishable structure separate from the Subjects/Teachers/Planning tabs. The base `/dashboard` route access is already enforced by `PROTECTED_ROUTE_RULES`.

1 `dashboard.export_reports` - required together with `reports.export` (both

are now real, independently-enforced keys; removing either blocks `/reports/allocation-plan.xlsx|.pdf` at the route-guard layer) and gates the "Export Report" menu in the template.

1 `hiring_plan.export` - required in addition to the base export gate for

every report-export section whose generated payload includes the hiring-plan sheet/section on the two report-export routes (`main._authorize_report_export`, gated on `REPORT_EXPORT_SECTIONS_WITH_HIRING_DATA = {"full", "hiring"}`), and gates the "Hiring Plan Excel/PDF" links. A PR #360 review finding showed the initial implementation checked only the literal `section=hiring` value, so a caller who retained the base export permissions but was denied `hiring_plan.export` could still obtain the same hiring-plan data via `section=full` (both `_build_professional_report_xlsx_bytes` and `_build_professional_report_pdf_bytes` include the hiring-plan sheet/section for `full`). The gate now checks section membership in `REPORT_EXPORT_SECTIONS_WITH_HIRING_DATA`, which must stay aligned with every section branch in those two builders that emits hiring-plan data.

1 `observations.view_reports` - gates `GET /observations/teacher/{id}/history`

(the aggregated observation-cycle report) for a non-teacher (evaluator/ admin) actor and its nav link on the observations list; a teacher viewing their own cycle remains self-service and is unaffected.

1 `observations.sign_evaluator` - is the dedicated gate for actually

persisting `evaluator_signature_data` on observation create/edit (`routers/observations.py::_can_sign_evaluator`); the signature-capture UI panel is now hidden without it.

1 `configuration.view_audit_log` - reclassified as platform-only (moved into

`DEVELOPER_ONLY_PERMISSION_KEYS`, matching its sibling `configuration.export_audit_log`) and enforced as an additional prerequisite under `/admin/audit-log`, alongside the export gate. See the real-conflict note below for why it could not remain tenant-assignable.

Existing capability governed by an explicit alias/composite (documented, not a second independently-built gate):

1 `observations.submit` - documented as an alias of `observations.sign_evaluator`

(Phase 3 finds no code evaluates it, so it is now classified dormant). The implementation has no separate "submit without signature" step: applying the evaluator signature is the exact action that submits the observation for the teacher's review (`_notify_teacher_observation_ready` fires only once `evaluator_signature_data` is set).

1 `system_owner.manage_developer_accounts` - alias of the existing

owner-identity gate (`auth.is_platform_owner`) already used for `POST /platform/developers` and `POST /platform/developers/{id}/permissions`. Owner-only keys are identity-governed by construction (`OWNER_ONLY_PERMISSION_KEYS` are never independently assignable), so the identity check *is* the enforcement mechanism; the key is now cited explicitly in the denial response for both routes for audit/messaging consistency, matching the pre-existing pattern already used for `system_owner.manage_ownership` / `.transfer_ownership`.

1 `system_owner.view_cross_school_audit` / `system_owner.export_cross_school_data`

- explicit aliases of `configuration.view_audit_log` / `configuration.export_audit_log`. The only implemented audit surface is a single, non-tenant-filtered log spanning every SchoolGroup; there is no separate cross-school-specific audit feature. Both pairs are platform-only, and either key in a pair is independently sufficient (`match="any"` on the export route rule; `auth.has_any_permission` for the view prerequisite).

Capability not implemented - classified dormant/reserved (no route, service, or user-facing control exists; no UI implies otherwise):

1 `hiring_plan.export`'s sibling capability confusion aside, the following

have zero implementation anywhere in the codebase: `reports.view` (no standalone Reports page exists separate from the dashboard's Reports tab, which is governed by `dashboard.view_reports`), `teachers.import`, `teachers.export` (no import/export route or UI control for teacher data exists at all), `subjects.manage_colors` (subject color is always auto-derived by `subject_colors.resolve_subject_color`; there is no manual color-editing UI or field), `observations.manage_templates` (the observation rubric/criteria set is a hardcoded, auto-seeded fixture with no create/edit/delete route), `configuration.manage_global_defaults` (no "global configuration defaults" workflow exists; already platform-only), `system_owner.manage_subscriptions` and `system_owner.create_subscription_school` (subscription/billing management is delegated entirely to an external SaaS admin surface reachable via `/saas/subscription?...`, outside this Web Service's implemented scope), and `system_owner.run_startup_repairs` (schema-compatibility repairs - `_ensure_users_table_columns` and siblings - run automatically at application startup; there is no user-triggerable route this key could gate). None of these keys have any UI control that implies a live, independently-gated feature, so no template changes were needed to remove misleading affordances.

Real KMS/implementation conflict (documented, not resolved by invented policy):

1 `configuration.view_audit_log` was registered as tenant-assignable (not in

`DEVELOPER_ONLY_PERMISSION_KEYS`) while its only implemented consumer (the global `/admin/audit-log` download) is inherently a single, non-tenant-filtered log spanning every SchoolGroup. Granting a tenant Administrator this key as previously registered would have been a silent no-op today, but wiring it naively to the existing endpoint would leak cross-tenant audit data - a tenant-isolation violation. Resolved fail-closed: the key was reclassified platform-only (matching its export sibling) rather than building a new tenant-filtered audit viewer (out of scope for this pass) or granting tenant-scoped access to global data. A future tenant-scoped audit viewer, if built, is a distinct, separately governed feature decision.

Owner-only commercial identity authority is a distinct governed boundary and is not automatically removed by an operational role-policy Deny.

Ten-family ON/OFF/ON tests use persisted policies and fresh SQLAlchemy sessions, real route guards, and navigation builders. They are not full authenticated HTTP middleware/handler/HTML acceptance tests. Original logo/qualification tests mock permission and scope decisions; original notification-group tests mock queries and commit operations. They establish action dispatch, not complete independent tenant-isolation evidence for those paths. Additional persisted-policy tests address specific missed paths but do not erase these coverage limits.

PR #360 review-response corrective pass

Two GitHub review findings on PR #360 (Codex automated review, commit `86fc9a299246ef33a7dba1f546bb6f87e928f3a6`) were inspected against the actual review comments/diff and corrected:

1 **P1 - hiring-plan export bypass via `section=full`.** Confirmed valid.

`main._authorize_report_export` compared `normalized_section` against the literal string `"hiring"` only, while both professional-report builders include the hiring-plan sheet/section whenever `section == "full"` as well. A caller retaining `reports.export` / `dashboard.export_reports` but denied `hiring_plan.export` could bypass the dedicated gate simply by requesting `section=full` instead of `section=hiring`, obtaining the same protected hiring data. Fixed by checking membership in a single shared constant, `REPORT_EXPORT_SECTIONS_WITH_HIRING_DATA = {"full", "hiring"}`, used by the gate and documented as required to stay aligned with the builders' own per-section branches. `summary` and `subjects/teachers` sections (which do not include hiring data) remain unaffected by a missing `hiring_plan.export` grant. Regression coverage strengthened in `tests/test_permission_closure_22_keys.py::test_report_export_route_hiring_section_requires_hiring_plan_export` to assert denial for `hiring` and `full`, continued access for `summary/subjects/teachers`, and full ON?OFF?ON round-trip for both `hiring` and `full`.

1 **P2 - `.kms-impact.yml` KIA summary factually inconsistent with the

branch's database changes.** Confirmed valid. The commit `86fc9a2` KIA summary stated "No new permission keys, migrations, or schema changes" and dropped `database_schema/migrations` from `affected_areas`, even though the same `fix/permission-consistency-closure` branch (commit `28f98f2`) still carries the non-destructive migration `20260915_001_role_permission_logical_key_uniqueness` (two partial-unique indexes on `role_permissions`) that is part of this PR's full diff against `origin/dev`. The 22-key closure commit itself added no new migration, but the KIA declaration governs the PR's cumulative branch knowledge impact, not only the most recent commit in isolation. `.kms-impact.yml` was rewritten to describe the full three-commit-plus-corrective-pass scope, restore `database_schema/migrations` to `affected_areas`, and list every authoritative KMS Markdown file that differs from `origin/dev` (`docs/CHANGE_HISTORY.md`, `docs/PROJECT_STATE.md`, `docs/TIS_MASTER_CONTEXT.md`, `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`, `docs/engineering/PERMISSION_CLOSURE_REVIEW.md`, `docs/engineering/README.md`), matching the range

GitHub CI validates for this PR (`origin/dev` ? the branch's final head commit).

A third item raised in the delegated task description - SchoolGroup edit/delete/action authorization consistency - was investigated but could not be substantiated as an actual PR #360 review finding: the PR's full diff against `origin/dev` (`git diff --stat origin/dev...HEAD`) touches no SchoolGroup route/model/template file, and neither of the two real review comments on PR #360 (fetched via the GitHub REST API) reference SchoolGroup mutation authorization. No SchoolGroup-related code was changed in this corrective pass; this discrepancy between the delegated task summary and the actual GitHub review content is recorded here rather than acted on speculatively.

User Exceptions UX (Phase 2)

The edit-user per-user panel is now "User Permission Exceptions": binary Allow/Deny plus Reset to Role Settings (internal inherit == delete the override row; "Inherit" is never shown). ADR 0040 precedence is unchanged. `user_permission_service.build_user_permission_payload` projects Effective (equal to `auth.get_allowed_permission_keys`), Source, exception, and an `exception_inert` flag. A stored Allow under a role-level Deny is kept, displayed as Effective Deny and "stored, currently ineffective", and re-applies if the role re-grants. Tenant/platform scope resolution, the platform-only and non-assignable rejections, and inactive-user fail-closed behaviour are unchanged. Locked rows expose no Allow/Deny buttons; a stored exception on a key that became non-assignable for the user's role (for example after a role change) is kept, never silently deleted, and the locked row shows only "Reset to Role Settings" so an operator can remove it (Reset deletes the row and grants no authority; the service already accepted `inherit` for such keys, and creating or changing an exception on a locked key is still refused). The legacy paired-list / `inherit` route contract is unchanged and never shown in the UI. Coverage: `tests/test_user_exceptions_ui.py`.

Migration and operational safety

The new RolePermission migration is last in the registered ledger sequence, uses the same connection for transactional PostgreSQL catalog inspection and DDL, and installs separate global-NULL and tenant-NOT-NULL partial unique indexes. Distinct tenants can share role/key names. Duplicate preflight raises before modifying the role-policy table or installing indexes; the runner may create its empty ledger table first. No rows are deleted or rewritten.

Independent disposable PostgreSQL 16.15 validation covers global and tenant duplicate rejection with rows retained/no marker, index installation, idempotency, and rejection of later duplicate writes. SQLite metadata/migration tests provide additional evidence. Index creation takes ordinary PostgreSQL locks; plan production maintenance/capacity before running it. There is no automatic down migration: rollback of an application release need not remove these compatible constraints, and dropping them requires explicit operational review. Production duplicates require approved cleanup, not automatic deletion.

Legacy `Limited Access` rows are not canonical managed-role policy; invalid and platform-only tenant grants are constrained by the resolver. Existing deployed user-override foundation DDL has no foreign-key clauses whereas the fresh ORM model references User and SchoolGroup. This pre-existing schema drift means orphan prevention cannot be assumed on migration-created databases; application writes validate ownership, but approved inspection/remediation is still needed. No real database rows were altered by review.

Removing the user-attached cache fixes a proven stale-instance risk. It does not independently prove the historical production HTTP failure's cause or cross-request ORM instance reuse. Platform-only user-override keys were already rejected before the reviewed commit; the new check adds effective-role assignability validation rather than originating the platform-only prohibition.

Test database safety

`database.py` defaults the global engine to tracked `tis.db`. `main._ensure_system_notifications_table_columns` normalizes rows using that global engine even when a handler receives a different fixture session. Private test sessions alone therefore do not protect the tracked database.

Set an isolated `DATABASE_URL` before importing application modules or replace the global engine/normalizer. This identifies a responsible mutation path, not the historical command that caused an existing local database modification.

Knowledge Impact Assessment

Knowledge impact is yes: corrective security/capability behavior and honest verification status. Master Context, Database Architecture, Change History, Project State, this engineering review/index, and the cumulative impact declaration record it; canonical tooling regenerates the PDF/manifest. No new precedence architecture or permission keys are introduced, so ADR 0040 remains accepted unchanged. No milestone or Talent capability is changed; AI Project Context and deeper module history are not duplicated. Deployment classification is Web Service and the original role-policy schema migration, not Timetable Workflow. Review classification grants no deployment authority.

TIS Development Standards

docs/engineering/DEVELOPMENT_STANDARDS.md

These standards are mandatory for future human developers, Codex conversations, ChatGPT conversations, and technical reviewers working on TIS.

Inspect Before Editing

Before changing files:

- inspect the current code with `rg` and targeted file reads,
- understand the affected route/service/template/model,
- read relevant docs, ADRs, and module history,
- check tests for the touched module,
- check the current branch and working tree.

Do not code from memory when local context is available.

Plan Before Implementation

For non-trivial work:

- identify the affected modules,
- identify tenant/permission/identity boundaries,
- identify documentation updates,
- identify tests or smoke checks,
- keep the implementation path small.

Planning does not need to be long, but the risk boundaries must be clear.

Keep Changes Small And Scoped

- Implement only the approved request.
- Avoid unrelated refactors.
- Avoid formatting churn in unrelated files.
- Preserve existing patterns unless there is a clear reason to change them.
- Do not move code across modules as a side effect.

Preserve Tenant Isolation

Tenant isolation is non-negotiable.

Always protect:

- school group boundaries,
- branch boundaries,
- academic year scope,

- user role/permission scope,
- observation and teacher data sensitivity.

Any change that touches tenant scope should be treated as high risk.

Preserve Login And Platform Owner Flows

Do not casually change:

- `/login`,
- session cookies,
- platform owner detection,
- platform developer permissions,
- `/platform`,
- `/platform/knowledge`,
- scope switching,
- access denied handling.

Platform developers are not platform owners unless existing owner logic explicitly treats them that way.

Protect SaaS, Payment, And Provisioning

Do not casually change:

- `/saas/signup`,
- `/saas/login`,
- `/saas/account`,
- SaaS onboarding steps,
- Paddle checkout/client code,
- payment confirmation logic,
- billing state,
- provisioning readiness,
- provisioning jobs.

Webhook-confirmed payment is authoritative. Checkout return pages are not proof of payment.

Database And Migration Rules

- Never modify `tis.db` unless explicitly approved.
- Never modify migrations casually.
- Do not change `models.py` without understanding migration/repair implications.
- Do not add schema changes without tests and documentation.
- Never use destructive database operations without explicit approval.

Landing Website Rule

The public landing website is separate:

- implementation: `tis-landing-website/`,
- marketing docs: `docs/marketing/`,
- ADRs: `0001`, `0007`.

Do not modify landing page code during backend or operational app tasks unless explicitly approved.

Protected Documentation Rule

- Do not expose generated PDFs through direct public UI links.
- Use owner-protected routes for PDF access.
- Markdown source docs are reviewed development artifacts.
- The app must not silently rewrite Markdown source docs.

Production Memory And Render Stability

Render memory is a hard production budget. Treat a 512 MB service as constrained and design routes, services, and assets accordingly.

Strict rules:

- Do not load, parse, or cache complete large datasets at startup or on a normal first request when a scoped lookup, streaming parser, pagination, or bounded cache can be used.
- Do not keep both raw and transformed copies of large JSON, CSV, Excel, PDF, image, or uploaded payloads in memory unless explicitly justified and validated.
- Do not run duplicate template renders in production request paths. Diagnostic pre-renders must be removed or gated behind an explicit environment flag.
- Do not emit stage-by-stage debug logs at warning level during normal traffic. Production diagnostics must be opt-in and should use appropriate log levels.
- Filter by tenant, school group, branch, academic year, and permission scope before materializing query results.
- Keep caches bounded by size or scope. Global caches for user-facing lookup data must be justified by measured memory impact.
- Large exports should stream or build bounded artifacts; avoid repeatedly constructing large in-memory workbooks, PDFs, or payloads inside loops.
- New static assets, videos, generated files, and location/reference datasets must be reviewed for repository size, runtime memory, and deployment impact.
- Any feature that adds broad data aggregation, new startup work, large reference files, or route-level diagnostics must include a memory/performance smoke check before deployment.

Minimum validation for memory-sensitive changes:

- run targeted tests for the touched module,
- run a compile/import check for changed Python modules,
- measure peak memory locally when a route or service parses large data,

- review production logs after deployment for restarts, OOM symptoms, and unexpected warning spam.

Commit And Push Rule

- Do not commit unless explicitly requested.
- Do not push unless explicitly requested.
- Do not merge unless explicitly requested.
- Final reports should describe changes and validation clearly.

KMS Update Rule

Every meaningful implementation must update KMS if affected:

- `docs/TIS_MASTER_CONTEXT.md`,
- `docs/PROJECT_STATE.md`,
- `docs/CHANGE_HISTORY.md`,
- `docs/AI_PROJECT_CONTEXT.md`,
- ADRs if major decisions changed,
- module history for module-specific before/after state,
- engineering docs if module maps, architecture, data model, standards, design philosophy, roadmap, or flows changed.

Regenerate the PDF if included docs changed.

Knowledge Impact Assessment

Every final implementation report must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed.

Validation Standards

Run the narrowest meaningful validation:

- compile checks for Python scripts/modules,
- targeted pytest tests when behavior changes,
- route/template smoke checks when pages are touched,
- PDF generation when docs change,
- frontend checks when landing/frontend code changes.

If validation cannot be run, say why.

TIS UI UX Design Philosophy

docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md

TIS UI/UX Design Philosophy

Restoring Expanded State After A Server Redirect

TIS is server-rendered FastAPI/Jinja, not an SPA, so expand/collapse state (native `<details>`, dialogs) lives only in the DOM and is normally lost the moment an action redirects to a fresh render. The sanctioned fix reuses two things that already existed rather than introducing a new mechanism: the existing `return_to` form-field/query-param convention, and its `#fragment` preservation already built into `redirect_utils.redirect_with_notice()` / `redirect_with_error()` (moved out of `main.py` so any router can share the same open-redirect guard, `redirect_utils.safe_redirect_path()`).

A route that wants the user to land back on a specific element accepts a `return_to` value shaped `<path>[?query]#<element-id>`, validates it with `redirect_utils.safe_redirect_path(return_to, default=<page path>)`, and redirects there. The shared client script `static/js/reopen-on-load.js`, loaded globally from `base.html`, runs on `DOMContentLoaded`, resolves a target id from `window.location.hash` (or an optional inline `window.__tisReopenTargetId` a page can render for a non-redirect, in-place response), and if the matching element is a `<details>`, opens it and scrolls it into view. A missing, stale, or renamed id is a silent no-op by design - `document.getElementById` never throws, so this can never break a page load.

Do not invent a page-local variant of this pattern. Give the element to reopen a stable, predictable id, pass `return_to` through the existing convention, and let the shared script do the reopening. Planning's Remove Demand action (`routers/planning.py:delete_planning_subject_demand`) is the reference implementation.

Progressive Disclosure And Commercial States

Expected commercial states use branded, actionable pages rather than generic errors or internal state labels. Expired customers see data-preservation, plan or renewal, support, and sign-out guidance. Dense Platform Owner pages show only the decision summary and common actions initially; lifecycle processing, branch/feature overrides, audit evidence, and technical metadata belong under More Actions, Advanced Settings, and Activity History. Presentation simplification must not remove operational controls.

TIS should feel like a clean, premium academic SaaS platform. It should not feel like a generic admin dashboard, spreadsheet wrapper, or decorative marketing shell disconnected from real product value.

Overall Product Identity

Design principles:

1 professional,

1 light,

1 trustworthy,

1 academic,

1 calm,

1 data-aware,

1 premium without being flashy.

The product serves serious academic operations. UI should reduce confusion and support confident decisions.

Operational FastAPI App

The operational app should prioritize:

- clarity,
- role-based navigation,
- data density,
- fast scanning,
- predictable controls,
- strong permission boundaries,
- branch/year context visibility.

Avoid:

- generic admin clutter,
- decorative cards that do not help workflows,
- large marketing-style hero sections inside operational tools,
- hiding important state behind vague labels,
- layouts that make repeated operational use slow.

Operational pages should help users answer:

- What am I looking at?
- Which school/branch/year is active?
- What needs action?
- What can my role do here?
- What changed after I saved?

Platform Owner Console

The Platform Owner Console should feel like a controlled operations console.

Priorities:

- global visibility,
- owner/developer separation,
- clear organization switching,
- safe access to sensitive tools,
- explicit status labels,
- no accidental tenant context confusion.

Avoid:

- broad controls without owner-only checks,
- exposing platform tools to developers by appearance alone,
- unclear organization/branch state.

Knowledge Center

The Knowledge Center is an internal owner utility, not a marketing page.

Priorities:

- KMS health,
- source coverage,
- freshness status,
- protected PDF actions,
- ADR/change/module visibility,
- KIA policy reminder.

Avoid:

- direct public static PDF links,
- regenerate actions until approved,
- app-side Markdown rewriting,
- decorative presentation that hides status.

SaaS Onboarding Pages

SaaS onboarding should feel guided, calm, and customer-facing.

Priorities:

- clear next step,
- progress visibility,
- plain customer language,
- reassurance around setup and billing,
- no internal engineering terms.

Customer-facing language should avoid:

- tenant,
- provisioning,
- M1/M2/M3/M4/M5,
- schema,
- migration,
- internal role mechanics.

Use customer language such as:

- organization,
- school,
- branch or campus,

1 setup,
1 plan,
1 billing,
1 account,
1 getting your workspace ready.

Next.js Landing Website

The landing website should use premium storytelling and strong visual assets.

Priorities:

1 clear problem/solution narrative,
1 actual product credibility,
1 school operations language,
1 strong visual hierarchy,
1 polished screenshots or generated assets when appropriate,
1 conversion path into demo/signup.

Avoid:

1 weak fake 3D visuals,
1 generic SaaS gradients without product specificity,
1 vague claims unsupported by product reality,
1 internal terms like tenant/provisioning/milestones,
1 cluttered feature walls.

The landing page source of truth is `tis-landing-website/`, not legacy FastAPI landing files.

Visual System Direction

Future design system should support:

1 consistent cards, tables, filters, status badges, and action buttons,
1 clear empty/loading/error states,
1 accessible contrast,
1 stable layouts across modules,
1 consistent icon and label patterns,
1 role-aware navigation,
1 responsive behavior without losing operational density.

Compact Descriptions And Tooltips

Compact-description tooltips are an explicit progressive-disclosure treatment, not a default for ordinary text. Shared JavaScript may enhance elements marked with `data-compact-description` and the maintained allowlist of approved compact components. It must not infer tooltip behavior from generic `p` or `small` tags, or from broad class

fragments such as description, subtitle, note, lede, helper, or supporting.

Operational status, helper, validation, and record text remains visible inline. Intentional tooltips must retain keyboard focus behavior, Escape dismissal, and an accessible relationship between their trigger surface and tooltip content. Native titles and accessible names on unrelated controls and visualizations are not removed by compact-description processing.

Tone And Copy

Internal app copy:

- concise,
- action-oriented,
- operationally clear.

Customer-facing copy:

- plain,
- confident,
- benefit-oriented,
- free of internal implementation terms.

Platform owner copy:

- precise,
- status-driven,
- explicit about access and risk.

TIS Product Roadmap

docs/engineering/PRODUCT_ROADMAP.md

Talent M11 Program And Evaluation UX Status

The Owner-directed Programs, assessment-setup, and evaluation-schedule simplification is implemented on `dev`. ADR 0035 now governs Student Assessment eligibility: current effective Academic Placement + Program eligible Grade + active assessment framework/tool are the operational prerequisites. Draft/Open Cycle state, frozen population, synchronization, reconciliation, and population membership are no longer user-facing prerequisites to start an Assessment. Existing Cycle/Population records remain only for compatibility and historical provenance/analytics where still consumed. No schema migration is required for this correction.

Production Migration Blocker Status

The Student Academic Placement PostgreSQL pre-deploy blocker is repaired and locally qualified. The new `20260904_000` prerequisite plus baseline metadata deferral preserves tenant-scoped foreign keys and enables the complete nine-step Student/Talent chain on a fresh production-style schema. Dedicated PostgreSQL fresh, repeat, existing-schema, duplicate-failure, rollback, and all-composite- target checks pass. Independent review is required before commit/push and the next Render pre-deploy retry; no deployment has been attempted by this task.

M10 B11-E F1 Provider Status

The current dev checkpoint implements the approved production privacy, commercial-availability, and breadth providers. The decisions are minimum cohort 5 for P1-P7, `feature.organization_intelligence` through the canonical entitlement path, and ceilings of 1000 matrix cells, 1000 relationship results, and 1000 Program-pair results. Configuration and provider failure remains fail-closed, excess breadth is rejected without truncation, and a deterministic local path is restricted to non-production plus the exact isolated Talent test database. F1 is ready for independent review. As of 2026-09-09 (ADR 0028/ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED, and production readiness is not declared.

Talent & Potential M11 Phase C Status

The current dev checkpoint implements the presentation-only Results & Analytics experience over all seven approved M10 routes. Focused UI/privacy/accessibility checks, the seven M10 backend suites, and read-only HTTP verification against the isolated persistent local dataset pass. Actual desktop/mobile browser visual acceptance remains a follow-up. This UI phase does not implement M12 AI or alter commercial packaging.

Talent & Potential M11 Phase B Status

The current dev checkpoint implements the operational Program, competency/rubric, evaluation-context, Assessment-entry, Candidate-review, Official-Identification, and Educator-Input workflow over the existing M2-M8 contracts. Focused automated and real HTTP acceptance pass. Browser acceptance remains a follow-up. This does not implement M12 AI or alter commercial work.

M10 B11 Qualification Sequence

ADR 0028 approves the B11 production-policy boundaries. B11-C used local non-production PostgreSQL, representative synthetic data, and test provider configuration to gather performance, memory, and EXPLAIN evidence. B11-C is now CLOSED after a PASS WITH NON-BLOCKING OBSERVATIONS independent re-review. B11-D is CLOSED after READ COMMITTED qualification proved mixed same-request snapshots and an isolated REPEATABLE READ experiment resolved the two reproduced cases. Permanent REPEATABLE READ remains proposed, governed,

and implemented under ADR 0029. B11-E remains the release gate. F1 now approves and implements minimum cohort 5, `feature.organization_intelligence`, and the 1000/1000/1000 breadth ceilings. Plan packaging, SLOs, memory ceilings, and any future index remain outside F1. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis; B12 is CLOSED; and production is not ready.

Talent & Potential M10 Status

B0 Contract / Conformance Harness, B1 structural Privacy Relationship Graph, B2 Reconstruction Analyzer / Privacy Closure, B3/B4 internal access/query foundation and B5 Organization Overview are complete; B6 Talent Map is implemented, independently reviewed, and closed after its sparse-total blocker was remediated and the targeted re-review passed. They define and test the approved canonical Cell identity, `+1/-1` additive Relationship topology, inherited M9 privacy states, tenant separation, forbidden contract fields, explicit V01-V25 milestone ownership, canonical Cell/Relationship registration, adjacency, validation, shared coordinates, deterministic connected components, exact Fraction RREF, per-coordinate uniqueness, inconsistent-system handling, monotonic deterministic closure, component-local failure, exact-source derived rates, and sibling-leak guards. They add no production privacy threshold, entitlement, schema, migration, UI, or frontend behavior. The B3/B4 foundation resolves tenant/year/historical-Branch authorization and secondary permission projection, fails closed behind unconfigured commercial availability and breadth providers, performs only set-based frozen-population aggregation, skips Candidate/Identification SQL without their own permissions, keeps M8 execution at Program grain, and requires serialization to enter through B2-closed projections. B5 exposes only the bounded organization Overview route within the now-approved 14-metric/7-grain contract, including P1 configured and active Program-configuration counts. B6 adds the bounded Program-by-Branch/Program-by-Grade Talent Map with a post-closure transpose alias, approved common factual metrics, backend totals, breadth enforcement, and shared B2 closure. It rejects required Period execution at Branch/Grade grain and adds no ranking or universal score. Sparse totals now derive values and graph terms from one authoritative populated-child set while preserving global `no_data` semantics. B6 is closed. B7 implements Program Portfolio and single-Branch Intelligence using common factual Program rows, historical frozen Branch scope, permission-projected sensitive fields, mandatory breadth, and the same privacy-closure stack. Required-Period execution is Portfolio-only at Program/AY grain. B8 adds Participation Overlap through one P2 canonical Cell per symmetric Program pair, using distinct Open/Closed frozen-population participants and historical Branch scope before a set-based intersection. Diagonals are distinct Program participants; no row/column sums or additive overlap equations exist. Candidate/Identification overlap, Grade breakdown, ranking, Talent Breadth/score, frontend, and longitudinal B10 remain unimplemented. B9 adds the first Student-identifiable M10 route, Student Drill, requiring `talent_analytics.view` AND `talent_analytics.view_students` (true AND composition); Student inclusion uses only frozen Cycle population context in authorized historical Branch/AY/Program scope, never current Placement. Every identifiable row is P7 through one gate-level `student_drill_population` count at `distinct_student` grain, fed by distinct authorized Student count and shared with the same B2 closure pipeline (no per-Student additive relationship). One top-level row per Student carries deterministic frozen Program/Cycle contexts; Candidate/Identification fields remain independently permissioned and query-skipped; Learner Profile access is an advisory capability hint only. Pagination is over distinct Students and has no total count; there is no Talent Score, ranking, AI field, or direct Student-ID filter. B9 remediation passed targeted independent re-review; B9 is CLOSED. B8 is also implemented, remediated, and CLOSED after its targeted independent re-review passed. Both committed/pushed implementations remain part of current dev; B10 is not implemented. The required pre-B2 hardening is complete: canonical metric/component/grain vocabularies are closed to the approved serialized strings, rate/percentage remain derived rather than Cell identity components, and RelationshipTerm accepts only a real Python integer `+1` or `-1`.

ADR 0027 approves the B10-A Longitudinal Organization Intelligence architecture as a governance decision only (B10 itself is still not implemented; B9 status above is unchanged): one Program, one Academic Year, ordered M8 Period slots (Period = order authority, optional linked Cycle = evidence), exactly nine approved metrics of the existing fourteen, one metric per response, `comparable`/`not_comparable` states with no growth language, and no server-computed delta between points. B10-B is implemented, independently reviewed with PASS and non-blocking observations, and

B10 is CLOSED. B11-B observability is implemented, independently reviewed with PASS and non-blocking observations, and CLOSED. As of 2026-09-09, B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING (ADR 0028); B11 overall is CLOSED on that same basis. B12 is CLOSED.

Talent & Potential M9 Status

Deterministic Talent analytics is implemented: a read-only, aggregate-only, privacy-gated `/api/talent/analytics` API (context, overview, rubric-distribution, kpi-distribution, competencies, branch/grade/section breakdowns, period-comparison, Student drill) over the existing frozen population, Assessment/Competency-Result, Review Candidate, and Official Identification history. Every route enforces primary per-cell privacy evaluation followed by complementary (reconstruction-breaking) suppression before serialization; `coarsened` publishes only a policy-supplied safe replacement and fails closed to `suppressed` otherwise - there is no generic bucket-merge algorithm. At the M9 checkpoint no production privacy policy was approved; F1 subsequently approves and implements the cohort-5 policy. Live PostgreSQL performance/concurrency has not been validated. M9 is committed and pushed on `dev` at `23ade9a7c6166197140b48a3edbfac849396d580` (commit `feat: add deterministic talent analytics`); it is not deployed, not production-released, and not merged to `master`. Talent Score/Index/ Potential Rate, Talent Map, KPI numeric bins/percentiles, export, AI, and UI remain future/out-of-scope work. M10 B0-B5 exists, while further M10 aggregate execution and Learning Style remain future governed work.

Talent & Potential M8 Status

Annual Evaluation Plan and Planned Evaluation Period configuration is implemented with lifecycle governance, deterministic ordering, advisory warnings, rollover, optional Draft Cycle linkage, link-aware Cycle Open validation, bounded audit, and zero-leakage permission composition. Existing Cycles remain ad-hoc. M9 Deterministic Talent Analytics is now implemented (see above); M10 B0-B5 now exists, while further M10 external execution, Learning Style, AI, and Student evidence remain future governed work.

Talent & Potential M7 Status

Implemented: read-only longitudinal Learner Profile, dedicated profile read permission, historical record-level Branch authorization, Program/Year/Cycle/ Framework history, Assessment/Competency/persisted-KPI history, Review Candidate and Official Identification history, permission-composed sensitive M6 sections, and a deterministic historical timeline. M8 is Annual Evaluation Plan & Periods and remains deferred; it will provide future period semantics without changing M7's multiple-Cycle model. Also deferred: Learning Style implementation, deterministic analytics, Organization Intelligence/Talent Map, Development and Support, AI/AI Suggested Review, exports, and identification revocation/re-identification.

Talent & Potential M6 Status (Complete)

Independent review is complete: write-time historical-Branch authorization, exact Official Identification context binding, linear Educator Input amendment lineage, and non-enumerating direct access are enforced and regression-tested.

Implemented, following 18 approved Product Owner governance decisions: deterministic `TalentReviewCandidate` evaluation/materialization from a Completed Assessment against its exact M3 Review Candidate Policy (`rubric_level_at_or_above` by `display_order`, `kpi_at_or_above` by persisted `kpi_result`, `all`/any composition), read-only against existing Assessment evidence, idempotent, with a deterministic fingerprint/snapshot; a non-qualifying evaluation is now structurally audited. The two-state `pending_review` -> `reviewed` Review workflow (one-way, no auto- identification). Append-only `TalentOfficialIdentification` (`identified`/`not_identified`, exactly one per Reviewed candidate, `talent_official_identifications.view/record` with `.record` requiring organization/global access scope in addition to the permission). Bounded qualitative `TalentEducatorInput` (`observation`/`context`/ `supporting_evidence`, <=2000 characters, mandatory historical Placement/ Branch resolution, append-only

amendment/supersession lineage, `talent_educator_inputs.view/add/amend`). All frozen-Branch/historical-scope discipline matches M4/M5. See "M6 Governance Review: Decisions (Resolved)" in `docs/AI_PROJECT_CONTEXT.md` for the full decision record. Explicitly deferred: Official Identification revocation/supersession/second decision/ re-identification, a generic review-note/case-management system, assessor assignment, Learner Profile, analytics/Talent Map, AI, Development and Support, and Educator Input analytics/export/AI/attachments.

Talent & Potential M5 Status

Implemented: canonical frozen-member-bound Student Assessment, exact Framework Competency Results, explicit In Progress/Completed/Incomplete/Insufficient Evidence lifecycle, expected-revision mutation protection, frozen-Branch scope, bounded operational audit, and dedicated assessment permissions. Optional Framework KPI completion uses integer `weighted_level_average`, denominator 10,000, ROUND_HALF_UP, and persisted Framework-specific provenance; qualitative no-KPI completion is first-class. Deferred: assessor assignment, correction, Review Candidate instances/evaluation, Official Identification, Educator Input, Learner Profile, analytics/Talent Map, AI, Development and Support, and late population exception workflow.

Talent & Potential M4 Status

Implemented: SchoolGroup-wide Assessment Cycle Draft/Open/Closed lifecycle, explicit-time dynamic eligibility preview, atomic frozen Student population, historical placement snapshots, full population integrity fingerprint, dedicated permission boundaries, Branch-filtered identifiable reads, and operational audit. Accepted ADR 0033 adds confirmed-placement-triggered, additive-only Open roster synchronization without member/evidence removal; Closed Cycles remain final. Deferred to later milestones: assessor assignment, Student Assessment, results, Review Candidate instances/evaluation, Official Identification, analytics, AI, and development plans.

Simplified Timetable Workflow And Published Visibility

Stage 5.2 is implemented with one Working Timetable, safe archive-based deletion, customer validation/publication language, secondary history, strict active-pointer official reads, and scoped My Timetable. Stage 5.1 solver constraints and concurrency are unchanged. Availability, rooms/resources, preferences, and student identities remain future separately approved work.

Production generation now dispatches one on-demand Render Workflow task per durable run. The always-running solver worker is no longer required; solver rules and Stage 5.2 publication/visibility behavior remain unchanged.

M3 Promo Redemption And Organization Activation

Completed secure customer lookup, resume-safe activation sessions, capacity review, exact branch selection, immutable redemption/grant authority, promo workspace entitlement, promo tenant-source ownership, existing-aligned and onboarding activation, M1 adapter/enforcement, expiry resolution, and redacted audit. Staff/teacher overage blocks without mutation; internal sandbox conversion, Paddle billing, promo renewal/conversion, and customer transfer remain outside M3.

M2 Promo Code Foundation And Platform Management

Completed secure promo definition persistence, exact plan-bounded capacity, scope and validity policy, lifecycle governance, branch-restriction snapshots, durable redacted audit, granular Platform permissions, and Platform Console administration. M3 now consumes approved definitions through its separate customer redemption and grant lifecycle. Communication, billing treatment, promo renewal, and broader conversion remain future work.

M8B9 Demo Operations, Notifications, And Testing

Completed generic Platform Owner lifecycle operations, manual production lifecycle execution, Standard/Full/Custom policies, isolated branch overrides, durable audit, and M8B7-based customer communications while preserving M8B8 usage history and Customer Demo classification.

M8B8 AI Entitlements And Commercial Foundation

Completed the commercial and usage boundary required before executable AI features: controlled registry, central decisions, two-use-per-feature Customer Demo allowance, Enterprise AI mapping, durable tenant-safe accounting, and upgrade guidance. Actual AI tools and M8B9 owner operations remain future work.

M8B7 Demo Customer Journey

Completed approval-orchestrated activation, durable lifecycle communications, Notification Center integration, shared-shell demo status, data-preserving expiry communication, and verified-payment continuation of the same active or expired workspace. M8B8 AI usage limits and M8B9 owner testing controls remain separate future work.

This roadmap summarizes completed, current, next, and future product directions. It is not a release commitment; it is the current planning map for developers, owners, and reviewers.

Completed

SaaS Identity Foundation

Established SaaS account signup, login, account area, and the boundary between SaaS accounts, platform identities, and operational tenant users.

Pending Organizations Zone

Created pending organization structures and owner-facing review concepts for organizations moving through SaaS onboarding.

Plans And Billing Foundation

Added plan catalog, plan selection, billing status, checkout summary, checkout return/cancel, and payment/billing service boundaries.

Paddle Payment Collection

Added Paddle-oriented payment architecture with provider logic isolated behind service/client modules.

Tenant Provisioning Engine

Added provisioning workflows and jobs to create or connect operational school group/branch/year/user records from ready pending organizations.

KMS Foundation

Created source-of-truth Markdown docs, change history, ADRs, module history, AI project context, generated PDF booklet, and manifest.

Platform Owner Knowledge Center

Added protected owner-only Knowledge Center for KMS health, source freshness, manifest metadata, ADRs, module history, and protected PDF view/download.

KMS v3.0 Phase 3A

Added engineering handbook foundation with module map, repository architecture, user/system flows, and onboarding structure.

M7 Subscription Management

Completed entitlement foundations, customer Subscription Management portal, paid branch quantity management, upgrades and scheduled downgrades, provider-authoritative proration, cancellation/reversal, centralized lifecycle/action policy, Paddle billing history, protected invoice downloads, and webhook/reconciliation safeguards.

Automatic KMS Enforcement

Added root AI instructions, machine-readable task KIA, major-change detection, read-only PDF/manifest checks, pull-request and `dev` CI validation, and a required KMS gate before `master` deployment.

M8B-1 Workspace Classification Foundation

Added constrained workspace identity/classification/lifecycle metadata, pre-provisioning and identity intent fields, validation-only services, read-only owner visibility, and safe diagnostic/backfill tooling. No commercial behavior or conversion workflow consumes the metadata yet.

M8B-2 Commercial State And Entitlement Foundation

Added normalized workspace and branch entitlement records, typed feature/limit values, read-only effective entitlement/commercial-state resolvers, conservative validation, and Platform Owner-only visibility. No enforcement or customer workflow changed.

M8B-3 Demo Request Workflow

Added the post-onboarding Request Demo or Subscribe Now choice, validated review-only demo requests, customer status/withdrawal, a Platform Owner review queue, constrained review decisions, and durable audit/internal-notification events. Approval does not provision or activate a demo.

M8B-4 Demo Workspace Provisioning And Activation

Added Platform Owner-triggered provisioning for approved customer-demo requests. The flow reuses the operational provisioning engine, creates an explicit demo entitlement and demo-sourced tenant link, activates atomically, records retryable failure and activation events, and creates no paid billing evidence.

M8B-5 Standard Customer Demo Lifecycle

Added the activation-based seven-day demo clock, Day 6 internal reminders, Day 7 atomic expiration, data-preserving workspace suspension, server-side web/API access enforcement, customer subscription guidance, owner lifecycle visibility, and a dry-run/default scheduled processor.

M8B-6 Demo-To-Paid Workspace Conversion

Added provider-confirmed conversion of an eligible active Customer Demo into Customer Paid on the same SchoolGroup and tenant link. The flow preserves operational identity and data, replaces the demo entitlement with the confirmed subscription-backed entitlement atomically, records durable conversion history, supports safe retry after failure, and removes completed conversions from demo lifecycle processing.

M8B8 AI Entitlements And Commercial Foundation

Added the centralized AI feature registry, permission/commercial/plan decision service, per-feature Customer Demo allowance, and separated durable usage accounting. No AI business feature or M8B9 operational control is included.

Current

Smart Timetable Version And Publication Foundation

Stages 2, 3, 3.5, 4, and 5.1 are complete: durable timetable versions, snapshots/fingerprints, exact-scope active selection, locks, generation-run schema, imported-current migration, and copy-on-write compatibility, canonical composed day timelines, structural readiness, visible history, locks, draft validation, same-scope comparison, explicit version export, archive, atomic publication, and worker-isolated CP-SAT generation/regeneration. Generated results remain unpublished until the separate publication flow. Stage 5.2 may simplify customer terminology/history and add Delete Working Timetable and published-only teacher My Timetable visibility.

KMS Enforcement Review

Review enforcement against real development tasks, keep classification conservative, and tune only demonstrated false positives without permitting silent no-impact declarations.

Landing / Customer Experience Preparation

The product is preparing for stronger public customer journey work from landing page storytelling through SaaS signup, onboarding, billing, and operational access.

Next

M8B-7 Next Approved Commercial Package

Proceed only from a separately approved M8B-7 specification. Manual demo extension/override, internal-sandbox conversion, special migrations, memberships, ownership transfer, and expanded role architecture remain unimplemented.

Premium Landing Page Redesign

Improve public website quality, product storytelling, visuals, and conversion clarity while preserving the separate Next.js source-of-truth.

Customer Journey From Landing To Signup

Clarify the path from public website to SaaS signup, plan selection, onboarding, checkout, and workspace readiness.

SaaS Onboarding Refinement

Improve onboarding language, progress, validation, customer reassurance, and platform owner visibility.

Paddle Live Configuration

Configure live Paddle behavior when the payment account and production readiness are approved.

Existing Workspace Paid Activation

Implemented the SchoolGroup-anchored M4C Paddle activation path for verified owners of preserved existing customer workspaces. Sandbox validation and migration rollout remain deployment gates. Starter stays unavailable until complete restricted-branch enforcement is separately proven.

Expand Entitlement Enforcement

Extend plan entitlement checks beyond the current approved pilot only when commercial rules for each module are reviewed.

Subscription Operations Hardening

Continue production validation, renewal/payment-failure handling, reconciliation observability, and owner support workflows without weakening Paddle authority or tenant isolation.

Future

Constraint-Based Smart Timetable Generation

Stage 5.1 delivered structural-readiness-gated CP-SAT generation, independent candidate validation, durable background execution, lock preservation, explicit regeneration diversity, and unpublished version persistence. Future solver work may add only separately approved availability, rooms/resources, cross-campus coordination, additional normalized slot semantics, preferences, or quality objectives.

AI Academic Assistant

Support academic leaders with intelligent recommendations based on verified tenant data.

AI Assessment Generation

Generate curriculum-aware assessments or question banks when curriculum/planning data is reliable enough.

AI Observation Improvement Plans

Assist supervisors with improvement plans, follow-up suggestions, and structured feedback from observation records.

AI Calendar Activity Planning

Suggest academic calendar activities, reminders, and coordination plans.

AI Dashboards And Academic Analytics

Provide richer academic insight, trends, risks, and multi-branch intelligence.

Multi-Branch Executive Dashboards

Give leadership cross-branch visibility into staffing, workload, observations, calendar, and planning health.

Curriculum And Planning Workflows

Expand from operational staffing/planning toward curriculum-aware academic planning if approved.

Reporting Improvements

Improve exports, dashboards, executive summaries, and decision-support reports.

Mobile Or Portal Expansion

Explore teacher/mobile/parent-style portals only if later approved and aligned with product strategy.

Roadmap Guardrails

1. Do not build AI features before data quality, permissions, and plan boundaries are clear.
1. Do not extend subscription lifecycle actions without verified Paddle behavior, fail-closed reconciliation, and updated KMS records.
1. Do not redesign landing/customer journey without preserving the Next.js boundary.
1. Do not expose internal terms to customers.
1. Update KMS docs and roadmap after meaningful roadmap changes.

TIS Rejected Architectural Decisions

docs/engineering/REJECTED_DECISIONS.md

This document records significant ideas that were considered and rejected. It helps future developers and AI assistants avoid reopening old decisions without new evidence.

Rejected decisions are not failures. They are part of the design record.

Single Application For Landing And Portal

Date: 2026-06-26

Context: TIS needs both a public marketing site and a protected operational app. The public site needs premium storytelling and conversion design; the app needs authenticated academic operations.

Proposed solution: Use the FastAPI/Jinja app for both the public landing website and the operational portal.

Why rejected: The two surfaces have different runtimes, audiences, design needs, deployment concerns, and risk profiles. Mixing them would make backend/operational tasks more likely to accidentally change the public website.

Chosen solution: Keep `tis-landing-website/` as a separate Next.js public landing website and keep the FastAPI app as the operational portal.

Long-term consequences: Developers must respect the landing/portal boundary. Landing changes require explicit approval and must use the Next.js source of truth.

Stripe Instead Of Paddle

Date: 2026-06-26

Context: TIS needs subscription payment collection, billing state, and future subscription lifecycle workflows.

Proposed solution: Use Stripe as the payment provider.

Why rejected: The current approved architecture is Paddle-oriented. Changing providers would reopen checkout, webhook, billing, tax/compliance, customer portal, and subscription lifecycle assumptions.

Chosen solution: Use Paddle behind service/client boundaries in `saas/`.

Long-term consequences: Provider-specific behavior must remain isolated. A future provider change would require a new ADR and careful migration plan.

Immediate Tenant Provisioning

Date: 2026-06-26

Context: Public signup and SaaS onboarding collect organization data before a school is ready for live operational access.

Proposed solution: Create operational school group, branch, user, and academic records immediately after signup or checkout return.

Why rejected: Immediate provisioning risks creating live tenant data before payment is verified, before onboarding is complete, or before platform owner review. Checkout return pages are not authoritative payment confirmation.

Chosen solution: Delay tenant provisioning until payment/readiness is verified and use platform-owner-visible provisioning jobs.

Long-term consequences: There are more states between signup and operational login, but tenant data is safer and provisioning can be reviewed/retried.

Public Documentation Access

Date: 2026-06-26

Context: The KMS generates a PDF under `static/docs/`, but the content includes internal architecture, roadmap, and operational guidance.

Proposed solution: Link directly to the generated PDF through public static paths.

Why rejected: Internal documentation should not be exposed as a public asset. Static file serving is not authorization-aware.

Chosen solution: Expose the PDF through protected Platform Owner routes only.

Long-term consequences: The UI must not link directly to `/static/docs/...`. Future storage may move generated docs outside public static storage.

Documentation Without ADRs

Date: 2026-06-26

Context: TIS decisions span SaaS identity, payments, provisioning, landing architecture, and KMS governance.

Proposed solution: Keep only a master context and change history.

Why rejected: Chronological history does not explain major decision tradeoffs deeply enough. Future developers would repeatedly reopen settled architecture questions.

Chosen solution: Maintain ADRs under `docs/adr/` for major accepted decisions and this rejected-decision record for important alternatives.

Long-term consequences: Major decisions require explicit traceability. ADRs and rejected decisions must be updated when architecture changes.

Weak Generic Landing Page Design

Date: 2026-06-26

Context: The public landing website is the first impression for schools evaluating TIS.

Proposed solution: Use a generic SaaS layout with vague claims, weak fake 3D visuals, or internal implementation language.

Why rejected: TIS needs credibility with school leaders. Generic visuals and internal language reduce trust and do not communicate academic operations value.

Chosen solution: Use premium storytelling, strong visual assets, clear customer language, and the separate Next.js landing visual system.

Long-term consequences: Landing work should be deliberate, visually strong, and customer-facing. Avoid terms like tenant, provisioning, and internal milestone labels.

Mixing SaaS Identities With Operational Users

Date: 2026-06-26

Context: TIS has public SaaS accounts, operational tenant users, and platform identities.

Proposed solution: Use one identity concept for signup, billing, tenant access, and platform administration.

Why rejected: This would weaken security and make it easy to confuse pending accounts with live operational users or platform owners.

Chosen solution: Keep SaaS account identity, operational tenant identity, and platform identity distinct.

Long-term consequences: Developers must always ask which identity world a change belongs to. Provisioning bridges worlds only through approved flows.

App-Side Automatic Documentation Rewriting

Date: 2026-06-26

Context: The KMS requires docs to remain current after meaningful implementation work.

Proposed solution: Let the running app automatically rewrite Markdown docs when it detects staleness.

Why rejected: Source-of-truth docs must be reviewed development artifacts. Silent rewriting would damage reviewability and could introduce incorrect historical records.

Chosen solution: Developers or AI assistants update Markdown docs as part of approved implementation work. The app may detect freshness and expose status, but not rewrite source docs.

Long-term consequences: Knowledge Impact Assessment remains a required human/AI workflow step. Future regenerate actions may rebuild PDFs only from reviewed Markdown.

Uncontrolled Regenerate Button

Date: 2026-06-26

Context: The Knowledge Center can show whether the PDF snapshot is current or stale.

Proposed solution: Add an immediate button that regenerates docs and/or source files from the app.

Why rejected: Phase 2C is read-only. Runtime regeneration raises deployment persistence, audit, concurrency, and source-control questions.

Chosen solution: Do not add regenerate behavior until separately approved. Keep source docs updated through reviewed development work.

Long-term consequences: A future regenerate action must be explicit, owner-only, audited where appropriate, and must not rewrite Markdown source docs.

TIS Visual Documentation Guide

docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md

This guide defines how visual documentation should evolve. No screenshots are required yet, but future screenshots and diagrams should follow this structure.

Purpose

Visual documentation should help developers, reviewers, owners, and future AI assistants understand TIS screens, workflows, data relationships, and design expectations.

Visual docs should not replace Markdown source docs. They should support them.

Future Storage Location

Recommended future location:

```
docs/visuals/
  screenshots/
  diagrams/
  flows/
```

Recommended generated/reference output:

```
static/docs/visuals/
```

Do not store sensitive production data in screenshots.

Screenshot Standards

Screenshots should:

1 use clean test/demo data,

1 avoid private student/teacher/customer information,

1 show the full relevant viewport or focused workflow area,

1 include enough context to identify branch/year/page state,

1 avoid browser chrome unless needed,

1 be updated when UI behavior or layout meaningfully changes.

Preferred formats:

1 PNG for UI screenshots,

1 SVG or PNG for diagrams,

1 PDF only for exported documentation snapshots.

Naming Convention

Use stable, descriptive names:

```
YYYY-MM-DD_module_screen_state.png
YYYY-MM-DD_flow_name_step_number.png
YYYY-MM-DD_architecture_diagram_name.svg
```

Examples:

```
2026-06-26_platform-console_owner-view.png
```

```
2026-06-26_knowledge-center_current-status.png
2026-06-26_saas-onboarding_step-organization.png
```

Update Policy

Update visual docs when:

- a screen layout meaningfully changes,
- workflow steps change,
- a module gains a new major view,
- navigation or role visibility changes,
- a diagram no longer reflects architecture.

Visual doc updates should be mentioned in the Knowledge Impact Assessment.

Future Diagram Strategy

Future diagrams should cover:

- identity separation,
- SaaS onboarding to provisioning,
- payment webhook confirmation,
- tenant isolation,
- operational data model,
- KMS source-to-PDF-to-Knowledge-Center flow,
- landing-to-signup customer journey.

Diagrams should be simple and versioned through Markdown references.

Planned Visual Areas

Platform Console

Future visuals:

- owner account controls,
- developer management,
- organization/branch context switching.

Knowledge Center

Future visuals:

- healthy status,
- stale status,
- source document table,
- KIA panel.

Dashboard

Future visuals:

- operational dashboard overview,
- staffing/report widgets,
- scope context display.

Teachers

Future visuals:

- teacher list,
- teacher edit/profile,
- qualification/capacity views.

Workforce Planning

Future visuals:

- planning table,
- section ownership,
- workload/capacity states.

Academic Calendar

Future visuals:

- calendar view,
- event editing,
- exports or responsibility states.

SaaS Onboarding

Future visuals:

- signup,
- organization step,
- contacts,
- branches,
- academic setup,
- review.

Billing

Future visuals:

- plan selection,
- checkout summary,
- billing status,
- payment pending/verified states.

Landing Page

Future visuals:

1 hero,

1 problem/solution section,

1 product capability section,

1 signup/demo conversion paths.

Reports

Future visuals:

1 allocation report,

1 dashboards,

1 export states.

Guardrails

1 Do not include sensitive real data.

1 Do not let visual docs become the only source of truth.

1 Do not update landing visuals during backend tasks unless explicitly approved.

1 Keep visual docs aligned with KMS change history.

TIS AI Optimization Guide

docs/engineering/AI_OPTIMIZATION_GUIDE.md

This is the definitive onboarding guide for future AI assistants working on TIS.

Preferred Onboarding Order

For any new AI coding conversation:

1. Read `docs/AI_PROJECT_CONTEXT.md`.
2. Read `docs/README.md`.
3. Read `docs/PROJECT_STATE.md`.
4. Read `docs/TIS_MASTER_CONTEXT.md`.
5. Read `docs/engineering/README.md`.
6. Read task-specific engineering docs.
7. Read relevant ADRs.
8. Read relevant module history.
9. Inspect code with `rg` before editing.

Choosing Task-Specific Docs

Use this map:

- SaaS signup/login/account: `docs/history/saas-onboarding/`, ADR 0002.
- Billing/payment/Paddle: `docs/history/subscriptions/`, ADR 0003, ADR 0004.
- Provisioning: `docs/history/provisioning/`, ADR 0005.
- Landing page: `docs/marketing/`, ADR 0001, ADR 0007.
- Platform owner/Knowledge Center: `docs/history/platform-knowledge/`.
- Workforce planning/timetable/teachers/subjects: `docs/history/workforce-planning/`.
- Calendar: `docs/history/academic-calendar/`.
- Architecture/data model: `docs/engineering/REPOSITORY_ARCHITECTURE.md`, `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`.
- Design/UI: `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`.
- Standards: `docs/engineering/DEVELOPMENT_STANDARDS.md`.
- Roadmap: `docs/engineering/PRODUCT_ROADMAP.md`.

How To Interpret ADRs

ADRs explain accepted long-term decisions.

When an ADR applies:

- treat it as design authority,
- do not reverse it casually,

- if a change conflicts with it, propose a new ADR or mark it superseded only with approval,
- link related docs/history.

How To Use Module History

Module history preserves deeper before/after context.

Before changing a module:

- read the module history folder,
- identify the previous documented state,
- update it if the module's meaning changes,
- do not replace chronological `CHANGE_HISTORY.md` with module history.

How To Complete KIA

Every final response must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

If meaningful behavior, architecture, workflow, roadmap, design, data model, or product state changes, documentation probably changed too.

Avoiding Architectural Regressions

Do not:

- merge SaaS account identity with operational users,
- treat checkout return as verified payment,
- provision tenants immediately from public signup,
- expose internal docs publicly,
- let platform developers act as platform owners,
- modify landing code during backend tasks,
- rewrite source docs from the running app,
- change database/migrations without explicit approval.

Preserving Tenant Isolation

Always identify:

- school group,
- branch,
- academic year,
- current user,

- user type,
- role/permissions,
- whether the user is platform, tenant, or SaaS account.

If scope is unclear, inspect code and tests before editing.

Distinguishing Operational vs SaaS Code

Operational app:

- root FastAPI files,
- `routers/`,
- `templates/` operational templates,
- `models.py`,
- branch/year/teacher/planning/calendar/observation workflows.

SaaS:

- `saas/`,
- `templates/saas/`,
- signup/login/account/onboarding/billing/provisioning readiness.

Landing:

- `tis-landing-website/`,
- `docs/marketing/`.

KMS:

- `docs/`,
- `scripts/generate_docs_pdf.py`,
- `knowledge_service.py`,
- `templates/platform_knowledge_center.html`,
- `static/docs/`.

Prompt Engineering Recommendations

Future prompts should specify:

- implementation vs planning,
- allowed files,
- forbidden files,
- whether docs must be updated,
- whether PDF regeneration is required,
- whether routes/app behavior can change,
- whether database/migrations are in scope,

1 whether landing is in scope.

Good prompt pattern:

```
Implement [specific goal].
Allowed files: [...]
Do not modify: [...]
Update KMS docs and regenerate PDF if docs change.
Run: [...]
Report KIA.
```

AI Readiness Review

Current strengths:

1 AI context exists.

1 Engineering docs cover modules, repository, flows, database, standards, UI/UX, roadmap, rejected decisions, governance, and visual docs.

1 ADRs and module history preserve decision context.

1 PDF/manifest provide generated snapshot and freshness metadata.

Remaining gaps for future KMS improvements:

1 More screenshots and diagrams are needed.

1 More module-specific deep docs may be useful for timetable, observations, permissions, and dashboard/reporting.

1 Test strategy documentation could be expanded.

1 Deployment/runbook documentation could be expanded.

1 API/route inventory could be generated or manually documented later.

1 Data migration history could be summarized more explicitly.

Do not implement these gaps unless a future phase approves them.

TIS Project Governance

docs/engineering/PROJECT_GOVERNANCE.md

This document defines how TIS engineering decisions, approvals, documentation, and quality gates should be governed.

Ownership Model

Product/project ownership: Defines business direction, roadmap priorities, customer experience, and approval for major changes.

Platform Owner: Owns platform-level operations, platform accounts, provisioning oversight, and KMS visibility.

Engineering implementer: Human developer, Codex, or ChatGPT-assisted coding session responsible for scoped implementation and validation.

Reviewer: Reviews implementation, KMS updates, risk boundaries, and validation results.

Approved Development Agents And Master Orchestration

The approved development-agent pool is Codex, Claude Code, and Cline/ClinePass. Cline is the coding agent; ClinePass is its provider/subscription choice, not a separate fourth agent. All three follow AGENTS.md and the same governing KMS, safety boundaries, KIA, validation, and explicit release authorization.

The master TIS conversation allocates bounded tasks by complexity, specialization, risk, independent-review value, and current usage/load. Reassess allocation per task and rebalance so one or two agents are not unnecessarily overloaded. Avoid permanent module ownership or rigid agent assignments.

Preferred tendencies, not hard walls:

1 Codex: primary implementation and broad repository changes.

1 Claude Code: deep investigation, architecture, complex business logic, and a

second implementation path.

1 ClinePass: independent implementation, verification, regression/safety review,

workflow consistency, and any suitable task as a third capable developer.

Choose a different author/reviewer where independent review adds value. Handoffs state scope, acceptance criteria, forbidden files, current worktree ownership, test evidence, KMS impact, and deployment boundary. Avoid concurrent edits to shared files; integrate KMS declarations without erasing another pending task. This policy is human/master-conversation orchestration, not an automated scheduler, permission to spawn agents, or release authorization.

Architectural Authority

Architectural authority comes from:

1 approved user direction,

1 ADRs,

1 `docs/TIS_MASTER_CONTEXT.md`,

1 engineering handbook docs,

1 module history,

1. existing code patterns.

If these conflict, pause and clarify before making irreversible changes.

Approval Workflow

Major work should follow:

1. clarify objective and scope,
2. identify allowed/forbidden files,
3. inspect code/docs,
4. implement scoped change,
5. update KMS,
6. run validation,
7. report KIA,
8. wait for review before broader phases.

Documentation Authority

Markdown under `docs/` is the source of truth.

Generated artifacts:

1. PDF booklet,
1. manifest,
1. future screenshots/diagrams.

Generated artifacts support the docs; they do not replace them.

Coding Workflow

Default workflow:

1. inspect before editing,
1. prefer existing patterns,
1. keep changes small,
1. preserve tenant isolation,
1. preserve identity boundaries,
1. avoid unrelated rewrites,
1. run targeted validation,
1. update KMS when meaningful.

Review Expectations

Review should check:

1. behavior matches scope,

- no forbidden files changed,
- tenant isolation preserved,
- permissions preserved,
- SaaS/payment/provisioning untouched unless approved,
- landing untouched unless approved,
- docs/KIA complete,
- validation credible.

Branch Strategy

Current assumption:

- active development branch: `dev`,
- production/live branch is separate and must be confirmed before deployment.

Rules:

- do not push unless requested,
- do not commit unless requested,
- do not merge unless requested,
- preserve unrelated local changes.

Release Philosophy

Prefer:

- small reviewable increments,
- documentation updated with implementation,
- explicit milestone boundaries,
- staged rollout for risky systems,
- no hidden operational changes.

High-risk areas:

- authentication,
- platform owner access,
- tenant isolation,
- payment,
- provisioning,
- database/migrations,
- landing conversion path.

Quality Gates

Before final report:

- run relevant compile/tests/smoke checks,
- verify generated docs if docs changed,
- inspect working-tree scope,
- mention validation gaps if any.

Documentation Gates

Every meaningful implementation must answer:

- Did master context change?
- Did project state change?
- Did change history change?
- Is an ADR needed?
- Is module history needed?
- Did AI project context need updating?
- Did engineering docs need updating?
- Was PDF regenerated?

KIA Requirement

Final reports must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

Engineering Decision Traceability

Use this traceability model:

- ADR: why a major decision was accepted.
- Rejected decisions: why significant alternatives were not chosen.
- CHANGE_HISTORY: chronological summary of meaningful changes.
- Module history: deeper before/after context for one area.
- Master context: durable current truth.
- Project state: current status, priority, known issues, next work.
- AI project context: compact first-read AI onboarding.
- Engineering handbook: module, architecture, flow, standards, data, UI, roadmap, governance knowledge.
- Generated PDF: snapshot for review/reference.
- Manifest: generated metadata and source freshness basis.

When to update each:

1 Update ADRs for major decisions.

1 Update rejected decisions when a significant alternative is intentionally declined.

1 Update change history for meaningful changes.

1 Update module history for area-specific evolution.

1 Update master context for durable current truth.

1 Update project state for current status and next work.

1 Update AI context when first-read onboarding truth changes.

1 Update engineering docs when developer understanding changes.

1 Regenerate PDF/manifest when included docs change.

TIS Knowledge Lifecycle

docs/engineering/KNOWLEDGE_LIFECYCLE.md

This document defines how the TIS Knowledge Management System evolves with the software from planning through deployment and maintenance.

Core Principle

Documentation evolves with implementation. Markdown is the source of truth. The PDF and manifest are generated snapshots. The Knowledge Center is the owner-facing status surface.

No meaningful implementation is complete until the Knowledge Impact Assessment is complete.

Documentation Lifecycle

1. A task is proposed.
2. Relevant docs, ADRs, and module history are read.
3. The likely documentation impact is identified.
4. Implementation happens within approved scope.
5. KIA is completed.
6. Relevant Markdown docs are updated.
7. ADRs are added or updated if decisions changed.
8. Module history is updated if a module's documented state changed.
9. PDF and manifest are regenerated if included docs changed.
10. Final report describes the knowledge impact.

Engineering Lifecycle

1. Inspect before editing.
2. Plan the smallest safe implementation.
3. Preserve tenant, identity, permission, SaaS, payment, provisioning, and landing boundaries.
4. Implement scoped changes.
5. Validate with focused checks.
6. Update KMS.
7. Report KIA.
8. Wait for review, commit, push, or deployment approval.

Approval Lifecycle

1. Owner defines goal and constraints.
2. Implementer confirms scope.
3. Implementer makes only approved changes.
4. Reviewer checks behavior, scope, docs, validation, and KIA.

5 Commit/push/deployment happens only after explicit approval.

Review Lifecycle

Review should verify:

1 no forbidden files changed,

1 tenant isolation remains intact,

1 SaaS/payment/provisioning boundaries are preserved,

1 landing code remains untouched unless approved,

1 database/migrations are untouched unless approved,

1 KMS docs and generated artifacts are current,

1 KIA is complete.

Release Lifecycle

Before release:

1 Confirm branch and deployment target.

2 Confirm tests/checks.

3 Confirm docs/PDF/manifest are current.

4 Confirm change history and project state.

5 Confirm release notes or owner communication if needed.

6 Deploy only through approved process.

Maintenance Lifecycle

Maintenance includes:

1 periodic KMS freshness review,

1 periodic ADR/rejected-decision review,

1 module history cleanup for discoverability,

1 PDF readability review,

1 Knowledge Center health checks,

1 updating roadmap/current state after milestones.

Planning To Production

Planning creates the first knowledge obligation. Production closes it.

Expected path:

```
Idea / task
  -> scope and constraints
  -> docs and code inspection
  -> implementation
  -> validation
  -> KIA
  -> docs/history/ADR updates
```

```
-> PDF/manifest regeneration
-> review
-> commit/push
-> deployment
-> post-deployment state update if needed
```

Guardrails

1 The app must not silently rewrite Markdown docs.

1 Generated PDFs must not be edited manually.

1 Regeneration is artifact generation, not source editing.

1 KMS update work must remain reviewable.

TIS Documentation Automation

docs/engineering/DOCUMENTATION_AUTOMATION.md

This document defines current and future automation expectations for the TIS KMS.

Current Automation

Current approved automation:

- `scripts/generate_docs_pdf.py` reads approved Markdown sources.
- It generates `static/docs/TIS_Project_Reference_Booklet.pdf`.
- It generates `static/docs/docs_manifest.json`.
- The manifest records documentation version, branch, commit SHA, source paths, mtimes, sizes, and hashes.
- The manifest records each included source document's starting PDF page.
- The manifest records the generated PDF hash and size.
- Markdown source hashes are computed from UTF-8 text normalized to LF, so equivalent CRLF and LF checkouts produce the same hash.
- Manifest source paths are repository-relative POSIX paths; dynamically discovered sources use a stable case-insensitive sort with an explicit tie-breaker.
- `scripts/generate_docs_pdf.py --check` validates source coverage, titles, approved catalog taxonomy, navigation links, source hashes, PDF identity, manifest metadata, PDF page bounds, and documentation version without writing files.
- `kms_catalog.py` is the dependency-free shared vocabulary and path classifier for Knowledge Center presentation and KMS enforcement.
- `.kms-impact.yml` records task-level Knowledge Impact in a small machine-readable schema.
- `scripts/check_kms_impact.py` classifies likely major changes, validates the declaration against the Git diff, and runs generated-artifact freshness checks.
- `scripts/kms.py sync` runs KIA preflight validation, regenerates the PDF and manifest, runs complete freshness validation, and prints a completion summary.
- `scripts/kms.py check` is the single read-only validation entry point for local work and CI.
- The PDF generator uses ReportLab multi-pass layout to produce a stable table of contents, named source destinations, and document/major-heading outlines without external dependencies.
- GitHub Actions enforce KMS checks on pull requests, `dev` pushes, and before `master` deployment.
- The Knowledge Center reads the manifest and checks freshness.

Repository owners must configure the `KMS Enforcement / kms-check` status as a required branch-protection check for protected integration branches. Workflow code makes production deployment depend on KMS validation directly; GitHub branch protection is the external setting that makes failed pull requests unmergeable.

Current automation does not:

- rewrite Markdown docs,
- create ADRs,
- create module history entries,
- decide KIA outcomes,

- generate documentation prose,
- commit or push changes.

Automation blocks stale or undeclared work; humans and approved AI assistants remain responsible for reviewed Markdown updates.

Cross-platform normalization does not relax enforcement. Changed text still changes the source hash, and missing, unexpected, duplicate, or reordered source entries still fail validation.

Phase 7D Navigation And Catalog Validation

The complete `scripts/kms.py check` path enforces:

- a usable front-matter title or H1 for every included Markdown source,
- categories and modules from the approved values in `kms_catalog.py`,
- exact normalized manifest/source-list membership and deterministic order,
- a positive integer `pdf_page` for every source, strict source-page ordering, and a page no greater than the generated PDF page count,
- relative links in `docs/KMS_NAVIGATION.md` that remain inside `docs/`, resolve to existing Markdown files, and target listed booklet sources.

The checker reports genuine missing, unexpected, duplicate, reordered, stale, invalid-taxonomy, invalid-title, invalid-page, and invalid-navigation conditions. It does not repair them or weaken existing KIA and freshness gates.

Git Event Task Boundaries

KIA declarations apply to an implementation task, not to an individual GitHub delivery event. A task may contain an implementation commit followed by one or more review or CI correction commits.

- Pull-request checks compare the pull-request base SHA with the actual pull-request head SHA.
- Push checks on `dev` calculate the merge base between the repository default branch and the pushed head, then compare that merge base with the pushed head.
- Push checks must not use `github.event.before` as the KIA base because it represents only the latest delivery and can split a multi-commit task.
- Both event types use three-dot comparison and validate every changed KMS Markdown file against the cumulative declaration.

This preserves strict checks for undeclared, missing, or falsely declared documentation while allowing a follow-up commit to modify only the subset relevant to that correction.

Machine-Readable KIA Declaration

Every task updates `.kms-impact.yml`:

```
knowledge_impact: yes
summary: Describe the implemented engineering change.
affected_areas:
  - subscriptions
kms_files_updated:
  - docs/CHANGE_HISTORY.md
no_impact_reason:
major_change_override: no
```

Rules:

- `yes` requires changed authoritative `docs/*.md` files.
- `no` requires a specific reason and no declared KMS Markdown changes.
- a detected major path with `no` also requires `major_change_override: yes`.
- every changed authoritative Markdown file must be listed.
- generated PDF and manifest must be current in both cases.

When Documentation Must Be Updated

Mandatory updates are required when a task changes:

- architecture,
- data model,
- tenant isolation,
- identity boundaries,
- SaaS/account flows,
- billing/payment/provisioning,
- operational modules,
- platform owner behavior,
- UI/UX philosophy,
- landing strategy,
- roadmap,
- governance,
- development standards,
- KMS behavior.

Optional updates may be skipped for:

- typo-only code comments,
- internal refactors with no behavior, architecture, flow, or ownership change,
- generated files that do not affect source truth.

Even when docs are skipped, KIA must explain why.

Phase 6 Commands

Synchronize generated artifacts after reviewing and updating authoritative Markdown:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Run complete read-only validation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

`sync` aborts before writing when KIA or authoritative Markdown validation fails. It writes only the generated PDF and manifest, then applies the same complete validation used by `check`. The lower-level generator and impact checker remain reusable implementation modules and diagnostic entry points.

Manifest Lifecycle

The manifest is generated every time the PDF is regenerated.

It should be treated as:

- generated metadata,
- source freshness basis,
- Knowledge Center input,
- not the source of truth.

If included Markdown docs change and the manifest is not regenerated, the Knowledge Center should show stale status.

PDF Lifecycle

The PDF is:

- a generated snapshot,
- owner reference material,
- a reviewable artifact,
- navigable through a page-numbered table of contents and PDF outlines,
- not manually edited.

Regenerate after included Markdown docs change.

Every manifest source record must contain a positive integer `pdf_page`, pages must increase in fixed source order, and each page must be within the generated PDF's actual page count. Missing, inconsistent, or out-of-range page metadata fails freshness validation.

AI Context Lifecycle

Update `docs/AI_PROJECT_CONTEXT.md` when:

- first-read onboarding changes,
- current priority changes,
- critical rules change,
- new major docs are added,
- architecture/identity/flow truth changes.

Do not overload AI context with every detail. It is a compact entry point.

Handbook Lifecycle

Update engineering handbook docs when:

- module boundaries change,
- repository ownership changes,
- user/system flows change,
- database concepts change,

- standards change,
- design philosophy changes,
- roadmap changes,
- governance changes.

Module History Lifecycle

Update module history when:

- a specific module's previous documented state should be preserved,
- a feature area changes meaningfully,
- before/after context matters for future implementers.

ADR Lifecycle

Create or update ADRs when:

- major accepted decisions are made,
- accepted decisions are superseded,
- architecture or product boundaries change.

Use rejected decisions when important alternatives are intentionally declined.

Mandatory vs Optional Summary

Mandatory:

- KIA in final report,
- docs update for meaningful changes,
- change history for meaningful changes,
- PDF/manifest regeneration when included docs change.

Conditional:

- ADR for major decisions,
- module history for area-specific state changes,
- AI context for first-read truth changes,
- engineering docs for handbook-level knowledge changes.

Optional:

- visual docs until screenshots/diagrams are approved,
- local Git hooks; CI is authoritative because hooks are optional and bypassable.
- future search, semantic indexing, and documentation analytics.

TIS Knowledge Impact Assessment Standard

docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md

The Knowledge Impact Assessment (KIA) is a required engineering standard. It confirms whether a task changed project knowledge and whether the KMS was kept current.

Required KIA Fields

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

The same decision must be recorded in `.kms-impact.yml` for machine validation. The repository declaration adds `summary`, `affected_areas`, `kms_files_updated`, `no_impact_reason`, and `major_change_override`.

Decision Tree

Ask:

1. Did behavior change?
2. Did architecture change?
3. Did a workflow change?
4. Did a module boundary change?
5. Did data ownership or tenant scope change?
6. Did SaaS/payment/provisioning change?
7. Did UI/UX or customer language change?
8. Did roadmap or current state change?
9. Did development standards/governance change?
10. Would a future developer or AI assistant need to know this?

If any answer is yes, knowledge impact is probably yes.

Checklist

For knowledge-impacting work:

1. update relevant source docs,
1. update `docs/CHANGE_HISTORY.md`,
1. update module history if module state changed,
1. update ADRs if major decisions changed,
1. update rejected decisions if an important alternative was declined,
1. update AI project context if first-read truth changed,
1. regenerate PDF/manifest,

1. report validation.
1. update `.kms-impact.yml` and run `scripts/check_kms_impact.py`.

Examples

Example: Docs Updated

```
Knowledge impact: Yes
Docs updated:
- docs/TIS_MASTER_CONTEXT.md
- docs/CHANGE_HISTORY.md
- docs/history/provisioning/...
Change history updated: Yes
ADR needed: No
Module history updated: Yes
PDF regenerated: Yes
AI project context updated: No
Reason if not updated: AI onboarding truth did not change.
```

Example: No Docs Needed

```
Knowledge impact: No
Docs updated: None
Change history updated: No
ADR needed: No
Module history updated: No
PDF regenerated: No
AI project context updated: No
Reason if not updated: Change was limited to a typo in an internal variable name and
did not affect behavior, architecture, workflow, roadmap, or developer knowledge.
```

Approval Expectations

Reviewers should reject final reports that omit KIA.

If docs were skipped, the reason must be specific. "Not needed" is not enough.

Failure Cases

KIA fails when:

1. final report omits it,
1. meaningful behavior changed but docs did not,
1. docs changed but PDF/manifest were not regenerated,
1. ADR-worthy decisions were hidden in code changes,
1. module state changed without module history,
1. AI context became stale after major onboarding truth changed.
1. `.kms-impact.yml` is missing, stale, inconsistent with the Git diff, or uses an unexplained override.
1. generated-artifact validation fails.

Developer Responsibilities

Developers must:

1. assess KIA honestly,
1. update docs in the same task,

- keep generated artifacts current,
- avoid unrelated documentation churn,
- preserve historical truth.

AI Responsibilities

AI assistants must:

- read relevant docs before coding,
- identify likely KMS impact,
- not invent history,
- not silently skip docs,
- not commit or push unless requested,
- include KIA in every implementation final report.

TIS Self-Evolving Workflow

docs/engineering/SELF_EVOLVING_WORKFLOW.md

This is the official engineering workflow for keeping TIS software and knowledge synchronized.

Workflow

```
Task
  -> Implementation
  -> Validation
  -> Knowledge Impact Assessment
  -> Update .kms-impact.yml
  -> Documentation Updates
  -> ADR if required
  -> Module History if required
  -> KMS Sync (PDF + Manifest + Freshness)
  -> KMS Read-Only Check
  -> Review
  -> Commit
  -> Push
  -> Deployment
```

Task

Clarify:

1 objective,

1 allowed files,

1 forbidden files,

1 validation requirements,

1 documentation requirements,

1 commit/push/deployment instructions.

Implementation

Implement only the approved scope.

Protect:

1 tenant isolation,

1 identity boundaries,

1 payment/provisioning correctness,

1 owner-only controls,

1 landing boundaries,

1 database/migration safety.

Validation

Run focused checks:

1 compile checks,

1 relevant tests,

1. route/template smoke checks,
1. PDF generation for docs,
1. frontend checks if frontend is in scope.

Knowledge Impact Assessment

Complete KIA before final report.

Do not treat documentation as optional when knowledge changed.

Record the same assessment in `.kms-impact.yml`. The declaration must change with the task and match the actual Git diff.

Documentation Updates

Update relevant docs:

1. master context,
1. project state,
1. AI context,
1. engineering docs,
1. change history,
1. module history,
1. ADRs/rejected decisions.

ADR If Required

Create/update ADR when architecture or product boundaries change.

Module History If Required

Update module history when area-specific before/after context matters.

Synchronize And Check KMS

If included Markdown docs changed:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Final read-only confirmation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

`sync` validates KIA before generating derived artifacts and validates freshness afterward. It never edits authoritative Markdown.

Review

Reviewers check:

1. scope,
1. behavior,
1. tests,

1. KMS updates,
1. generated artifacts,
1. KIA.

CI repeats these checks and blocks pull-request completion, `dev` integration, or `master` deployment when declaration or freshness validation fails.

Commit / Push / Deployment

These are explicit actions, not assumptions.

1. Commit only when requested.
1. Push only when requested.
1. Deploy only through approved process.

Why This Is Self-Evolving

The workflow makes knowledge updates part of engineering completion. The system evolves because every meaningful change carries its context, history, decisions, and generated reference forward.

TIS Documentation Dependency Map

docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md

This document explains how KMS documents relate to each other and how changes propagate.

Dependency Model

```
Master Context
  -> Project State
  -> Engineering Handbook
  -> AI Project Context
  -> ADR
  -> History
  -> Knowledge Center
  -> Manifest
  -> PDF
```

The arrows show common reading and propagation flow. They are not strict ownership hierarchy.

Master Context

Purpose: Durable product, architecture, workflow, roadmap, and critical-rule truth.

Update when:

- long-term architecture changes,
- product vision changes,
- major workflow changes,
- critical rules change,
- KMS structure changes.

Project State

Purpose: Current branch, priority, milestone, known issue, and next-work truth.

Update when:

- active priority changes,
- phase status changes,
- known issues change,
- next planned work changes.

Engineering Handbook

Purpose: Developer-facing deep knowledge: modules, repository, flows, database, standards, design, roadmap, governance, AI guidance, rejected decisions.

Update when:

- developer understanding changes,
- module maps change,
- architecture or data ownership changes,
- standards/governance changes,

- future onboarding would be incomplete.

AI Project Context

Purpose: Compact first-read file for future AI coding conversations.

Update when:

- first-read onboarding changes,
- major docs are added,
- current priority changes,
- critical boundaries change.

ADR

Purpose: Accepted major decision record.

Update when:

- a major architectural/product decision is made,
- a prior decision is superseded,
- a decision needs formal traceability.

Rejected Decisions

Purpose: Record important alternatives that were declined.

Update when:

- future teams are likely to reconsider a rejected path,
- a rejected alternative explains the chosen architecture.

History

Purpose: Chronological and module-specific evolution.

Update when:

- meaningful change occurs,
- module before/after state matters,
- project knowledge changes.

Knowledge Center

Purpose: Owner-facing app view of KMS health, freshness, and protected PDF access.

Update when:

- KMS metadata behavior changes,
- Knowledge Center UI/status behavior changes,
- protected documentation access changes.

Manifest

Purpose: Generated metadata for PDF and source freshness.

Update when:

1. PDF is regenerated.

Do not edit manually.

PDF

Purpose: Generated handbook snapshot.

Update when:

1. included Markdown docs change.

Do not edit manually.

Propagation Examples

Feature changes workflow:

```
Implementation -> CHANGE_HISTORY -> module history -> master/project state if needed
-> PDF/manifest
```

Architecture decision workflow:

```
Decision -> ADR -> rejected decision if relevant -> master context -> engineering
handbook -> change history -> PDF/manifest
```

Roadmap change workflow:

```
Roadmap -> PRODUCT_ROADMAP -> PROJECT_STATE -> AI_PROJECT_CONTEXT if first-read truth
changed -> PDF/manifest
```

KMS structure change workflow:

```
KMS docs -> README -> MASTER_CONTEXT -> AI_PROJECT_CONTEXT -> CHANGE_HISTORY ->
PDF/manifest
```

TIS AI Coding Workflow

docs/engineering/AI_CODING_WORKFLOW.md

This guide defines how future AI assistants should work inside TIS.

Repository-Native Assistant Configuration

Root `AGENTS.md` remains the repository-wide governance authority. Claude Code also reads root `CLAUDE.md`; its reusable project workflow lives at `.claude/skills/tis-kms/SKILL.md`, and bounded delegated TIS work may use `.claude/agents/tis-kms-developer.md`. These files point assistants back to this KMS rather than embedding a separate architectural source of truth. Configuration for another assistant must preserve the same KIA, safety, validation, and completion-report requirements.

Cline Setup And Use

Cline uses `.clinerules/tis.md` as always-on project rules and `.cline/skills/tis-kms-developer/SKILL.md` as its reusable project skill. Both defer to AGENTS.md and KMS, including task-level KIA, focused validation, sync/check, worktree/database safety, and deployment reporting.

Open TIS as the primary workspace folder. In Cline's Rules/Skills panel, ensure the TIS project rule and skill are enabled; enable Skills in Settings > Features if that option is present. Start a new task and select `/tis-kms-developer` from slash suggestions, or ask to use the named skill. If it is not listed, explicitly ask Cline to read the skill file; project rules still apply. The literal Codex `$tis-kms-developer` token is not a documented Cline invocation.

Check for a global skill with the same name, because Cline gives it precedence. Cline can also discover `.claude/skills/tis-kms/`; it describes the same KMS workflow and is not a second authority. Prefer the explicit Cline skill for Cline tasks. No user-global settings, provider, account, or auto-approval defaults are changed by these repository files.

Compatibility evidence (2026-09-05): installed VS Code Cline extension 4.1.17 includes SKILL.md parsing and use_skill support. Current official documentation supports project rules, recommended .cline/skills storage, and slash invocation:

1 [Cline rules](https://docs.cline.bot/customization/cline-rules)

1 [Cline skills](https://docs.cline.bot/customization/skills)

1 [Current upstream skills documentation](https://github.com/cline/cline/blob/main/docs/customization/skills.mdx)

Runtime discovery and toggles depend on the active Cline session; file validation does not prove that a UI session has activated the skill.

Task allocation follows [Project Governance](PROJECT_GOVERNANCE.md): Codex, Claude Code, and Cline/ClinePass are three capable developers, balanced by complexity, specialization, risk, independent review, and current usage/load.

Planning Before Coding

AI assistants must:

1 follow root `AGENTS.md`,

1 read `docs/AI_PROJECT_CONTEXT.md`,

1 read relevant engineering docs,

1 read relevant ADRs/module history,

1 inspect the codebase with `rg`,

1 identify allowed and forbidden files,

1 identify likely KMS impact,

1 update `.kms-impact.yml` for the current task,

1 avoid starting from assumptions.

Implementation Review Before Editing

Before editing:

1 confirm the module boundary,

1 confirm tenant/identity/payment/provisioning risk,

1 confirm tests or validation,

1 confirm docs to update,

1 confirm that app, SaaS, landing, database, or routes are in scope before touching them.

Implementation

During implementation:

1 keep changes narrow,

1 follow existing patterns,

1 avoid unrelated rewrites,

1 use `apply_patch` for manual edits,

1 do not commit or push,

1 do not modify `tis.db` unless explicitly approved.

Validation Expectations

Run appropriate validation:

1 compile checks for Python scripts/modules,

1 targeted tests for behavior,

1 PDF generation for docs,

1 template/route smoke checks for UI routes,

1 frontend checks for landing/frontend tasks.

If validation cannot be run, state why.

Documentation Updates

AI assistants must update KMS when knowledge changes:

1 change history,

1 project state,

1 master context,

1 AI context,

1 engineering docs,

1 ADRs/rejected decisions,

1 module history.

KIA

Every final response must include KIA.

Do not omit KIA because the task "felt small." Assess it.

The machine-readable declaration must agree with the final KIA and actual changed Markdown. Use the explicit major-change override only for a genuinely non-behavioral change and provide a specific explanation.

Commit Strategy

AI assistants must not commit unless the user explicitly asks.

When committing is approved:

1 summarize scope,

1 ensure generated docs are current,

1 avoid unrelated changes,

1 use clear commit messages.

Push Strategy

AI assistants must not push unless explicitly asked.

Before pushing:

1 confirm branch,

1 confirm tests/checks,

1 confirm KMS artifacts,

1 run `scripts/kms.py sync` when documentation changed and `scripts/kms.py check` for final read-only enforcement,

1 confirm no forbidden files changed.

Deployment Strategy

AI assistants must not deploy unless explicitly asked.

Before deployment:

1 confirm production branch/environment,

1 confirm database/migration implications,

1 confirm SaaS/payment/provisioning risks,

1 confirm KMS state,

- confirm rollback or recovery expectations.

Final Response Pattern

Include:

- files changed,
- behavior changed or not,
- validation,
- known issues,
- KIA,
- no vague claims about tests that were not run.

TIS Future Automation Roadmap

docs/engineering/FUTURE_AUTOMATION_ROADMAP.md

This document lists possible future automation improvements. These are not implemented in Phase 3D.

Git Hooks

Potential:

- pre-commit check for changed docs without regenerated PDF/manifest,
- warning when KIA-related docs are stale,
- formatting checks for Markdown.

Considerations:

- hooks must not block emergency work without clear override,
- hooks must be documented for Windows/dev environments.

CI Documentation Validation

Potential:

- run PDF generator,
- verify manifest is current,
- verify required docs exist,
- verify source hashes match,
- fail pull requests when docs/PDF are stale.

Automatic PDF Regeneration

Potential:

- CI-generated PDF artifacts,
- deployment-time regeneration,
- explicit owner-only regenerate action.

Constraint:

- automatic regeneration must not rewrite Markdown source docs.

Automatic Manifest Generation

Potential:

- generate manifest in CI,
- compare manifest against committed PDF,
- expose freshness status in Knowledge Center.

Documentation Diff Reports

Potential:

- summarize doc changes between commits,
- show which docs changed after a feature,
- include KIA summary in release notes.

Stale Documentation Alerts

Potential:

- Knowledge Center warning when manifest/source hashes differ,
- CI alerts,
- dashboard badge for owner users.

Knowledge Center Search

Potential:

- search docs, ADRs, module history, and roadmap from owner page,
- filter by module,
- link to protected document views.

AI Semantic Search

Potential:

- indexed semantic retrieval over KMS docs,
- AI assistant prompt context packs,
- module-specific context bundles.

Constraints:

- preserve privacy,
- do not expose internal docs publicly,
- avoid unreviewed AI-generated source edits.

Documentation Analytics

Potential:

- track stale areas,
- track frequently referenced docs,
- identify docs missing module ownership.

Release Documentation

Potential:

1. release notes generated from change history,
1. deployment checklist,
1. customer-facing change summaries,
1. internal technical release notes.

Architecture Dashboards

Potential:

1. owner-facing architecture/decision dashboard,
1. ADR status summaries,
1. module health and documentation freshness.

Priority Recommendation

Likely future order:

1. CI documentation validation.
2. Stale documentation alert in Knowledge Center.
3. Documentation diff report.
4. Owner-only explicit PDF regenerate action.
5. Search.
6. Semantic AI retrieval.

Separate Next.js Landing Website

docs/adr/0001-separate-nextjs-landing-website.md

ADR 0001: Separate Next.js Landing Website

Status: Accepted

Date: 2026-06-26

Context

TIS needs a public marketing website and a protected operational application. The public site has different performance, design, routing, and deployment needs than the FastAPI operational portal.

Decision

Keep the public landing website in `tis-landing-website/` as a separate Next.js app. Keep the FastAPI app as the operational portal at `https://app.tisplatform.com`.

Alternatives Considered

- Use FastAPI/Jinja for both marketing and app pages.
- Put marketing pages inside the operational app templates.
- Use a static site generated outside the repository.

Consequences

Positive:

- Clear separation between marketing and operational concerns.
- Landing page can evolve with a modern frontend stack.
- Operational FastAPI app remains focused on authenticated workflows.

Tradeoffs:

- Two runtimes must be understood.
- Deployment and routing boundaries must be respected.

Related Docs / Files

- `tis-landing-website/`
- `docs/marketing/landing_page_source_of_truth.md`
- `docs/TIS_MASTER_CONTEXT.md`
- `templates/landing.html`
- `static/landing/landing.css`

Separate SaaS Identity And Operational Users

docs/adr/0002-separate-saas-identity-and-operational-users.md

ADR 0002: Separate SaaS Identity And Operational Users

Status: Accepted

Date: 2026-06-26

Context

TIS has SaaS account users who sign up, select plans, onboard organizations, and manage billing. It also has operational tenant users who work inside a provisioned school context. Platform owners and developers are separate platform identities.

Decision

Keep SaaS account identity, platform identity, and operational tenant identity distinct.

Alternatives Considered

- Use one user table concept for all public signup, platform, and tenant users.
- Automatically convert every SaaS signup into an operational tenant user.
- Treat platform users as ordinary tenant administrators.

Consequences

Positive:

- Clearer security boundaries.
- Safer tenant isolation.
- Billing/onboarding can proceed before operational provisioning.

Tradeoffs:

- Identity flows require careful documentation.
- Developers must avoid mixing account, platform, and tenant assumptions.

Related Docs / Files

- `auth.py`
- `models.py`
- `saas/models.py`
- `saas/service.py`
- `saas/router.py`
- `docs/TIS_MASTER_CONTEXT.md`

Paddle Payment Architecture

docs/adr/0003-paddle-payment-architecture.md

ADR 0003: Paddle Payment Architecture

Status: Accepted

Date: 2026-06-26

Context

TIS requires subscription payments and billing visibility while preserving a clean boundary between academic operations and payment provider behavior.

Decision

Use Paddle behind service/client boundaries in the `saas/` package. Keep payment, pricing, billing, and currency logic separate from academic modules.

Active branch-quantity changes use `saas.subscription_change_service` and durable `SubscriptionChangeRequest` records. TIS requests Paddle previews and never calculates monetary proration. Immediate increases use `prorated_immediately` and prevent applying the provider change when payment fails. Reductions are billed for the next period without an automatic refund; TIS preserves current local capacity until the renewal boundary is webhook-confirmed. Every update sends Paddle the complete retained recurring item list.

Plan upgrades and downgrades use the same provider-authoritative principles through `saas.subscription_plan_change_service`. Upgrades are immediate only after provider-confirmed payment lifecycle evidence; downgrades are scheduled and preserve current entitlements until effective confirmation. Scheduled changes may be replaced or canceled only while Paddle and local request state agree.

Cancellation and reversal use `saas.subscription_cancellation_service`. Cancellation is scheduled at period end, does not revoke current paid access early, and may be reversed before the effective boundary after provider validation. `saas.subscription_lifecycle_service` centralizes displayed lifecycle state and allowed customer actions.

Billing history and invoices remain provider-owned. `saas.billing_history_service` reads subscription-scoped Paddle transactions and requests fresh expiring invoice URLs without persisting a duplicate financial ledger or provider URL.

Paddle does not support arbitrary client-supplied idempotency keys. TIS therefore serializes unresolved requests per subscription, uses deterministic local request keys, locks request/subscription rows during submission, and reconciles webhook retries by provider event and request state.

Alternatives Considered

1 Inline Paddle calls directly inside route handlers.

1 Store provider-specific behavior across operational modules.

1 Delay payment architecture until after provisioning.

Consequences

Positive:

- Provider integration is isolated.
- Academic workflows stay independent of payment details.
- Billing status can be reasoned about separately from onboarding and provisioning.

Tradeoffs:

- Service boundaries must be maintained.
- Webhook/payment state must be carefully tested.

Related Docs / Files

- `saas/paddle_client.py`
- `saas/payment_service.py`
- `saas/billing_service.py`
- `saas/pricing_service.py`
- `saas/currency_service.py`
- `saas/entitlement_service.py`
- `saas/subscription_portal_service.py`
- `saas/subscription_change_service.py`
- `saas/subscription_plan_change_service.py`
- `saas/subscription_cancellation_service.py`
- `saas/subscription_lifecycle_service.py`
- `saas/billing_history_service.py`
- `saas/router.py`
- `docs/TIS_MASTER_CONTEXT.md`

Webhook-Only Payment Confirmation

docs/adr/0004-webhook-only-payment-confirmation.md

ADR 0004: Webhook-Only Payment Confirmation

Status: Accepted

Date: 2026-06-26

Context

Checkout return pages can be reached by browser redirects, refreshes, or user navigation. They should not be treated as authoritative proof that payment succeeded.

Decision

Treat provider webhook confirmation as the authoritative source for successful payment state. Checkout return pages may inform the user but should not independently confirm payment or trigger provisioning as if payment is verified.

The same authority boundary applies to M7 subscription mutations. Local requests and browser submissions are intent, not final financial truth. Webhook processing validates signatures, provider event identity, subscription/customer ownership, expected items/quantity, attributable transactions, and idempotent event state before confirming local outcomes. Ambiguous or mismatched evidence fails closed to manual review.

Initial checkout and post-activation subscription-change lifecycles remain distinct so a completed change transaction cannot accidentally replay initial provisioning/payment transitions. Guarded reconciliation may repair finalized local lifecycle fields only from attributable stored webhook evidence plus authoritative Paddle transaction data.

Alternatives Considered

- Confirm payment based on checkout return route.
- Allow manual UI confirmation from the SaaS account page.
- Provision tenant records immediately after checkout redirect.

Consequences

Positive:

- Reduces risk of false-positive payment confirmation.
- Aligns billing state with provider-confirmed events.
- Supports delayed provisioning after verified payment.

Tradeoffs:

- Users may need clear pending-payment status while webhook processing completes.
- Webhook handling must be reliable and observable.

Related Docs / Files

- `saas/paddle_client.py`
- `saas/payment_service.py`

```
1     saas/billing_service.py
1     saas/subscription_change_service.py
1     saas/subscription_plan_change_service.py
1     saas/subscription_cancellation_service.py
1     saas/payment_lifecycle_reconciliation_service.py
1     scripts/reconcile_finalized_payment_lifecycle.py
1     saas/router.py
1     docs/TIS_MASTER_CONTEXT.md
```

Delayed Tenant Provisioning After Verified Payment

docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md

ADR 0005: Delayed Tenant Provisioning After Verified Payment

Status: Accepted

Date: 2026-06-26

Context

Public SaaS onboarding collects organization and branch information before an operational tenant exists. Provisioning creates operational school records and must be safe, reviewable, and recoverable.

Decision

Delay tenant provisioning until payment is verified and the organization is ready for provisioning. Use platform owner provisioning workflows and jobs to create or update operational tenant structures.

Alternatives Considered

- Provision immediately after signup.
- Provision immediately after checkout return.
- Let public users create operational tenants directly.

Consequences

Positive:

- Protects operational tenant data.
- Keeps pending SaaS onboarding separate from live school records.
- Allows platform owner oversight and retry behavior.

Tradeoffs:

- More state transitions exist between signup and operational access.
- Provisioning status must be communicated clearly.

Related Docs / Files

- `saas/provisioning_service.py`
- `saas/router.py`
- `saas/service.py`
- `templates/saas/admin_provisioning.html`
- `templates/saas/admin_pending_organizations.html`
- `docs/TIS_MASTER_CONTEXT.md`

Documentation As Source / Knowledge Management System

docs/adr/0006-documentation-as-source-knowledge-management-system.md

ADR 0006: Documentation As Source / Knowledge Management System

Status: Accepted

Date: 2026-06-26

Context

TIS is growing across product, SaaS, architecture, payment, provisioning, landing, and operational modules. Future humans and AI coding assistants need reliable context and change history.

Decision

Use Markdown files under `docs/` as the source of truth. Generate the PDF booklet as a snapshot. Maintain change history, ADRs, module history, project state, master context, and AI project context as part of every meaningful development task.

Alternatives Considered

- Keep knowledge only in chat history.
- Keep only a generated PDF.
- Let the app rewrite docs automatically.
- Store decisions only in commits.

Consequences

Positive:

- Project knowledge is reviewable and version-controlled.
- Future Codex and ChatGPT conversations can onboard quickly.
- Major decisions are preserved.

Tradeoffs:

- Developers must maintain the KIA workflow.
- Docs and PDF can become stale if the policy is ignored.

Related Docs / Files

- `docs/README.md`
- `docs/DOCUMENTATION_UPDATE_POLICY.md`
- `docs/CHANGE_HISTORY.md`

```
1     docs/AI_PROJECT_CONTEXT.md
1     scripts/generate_docs_pdf.py
1     static/docs/TIS_Project_Reference_Booklet.pdf
```

Landing Page Visual System Strategy

docs/adr/0007-landing-page-visual-system-strategy.md

ADR 0007: Landing Page Visual System Strategy

Status: Accepted

Date: 2026-06-26

Context

The public landing page must communicate trust, product maturity, and academic operations value. It should not be accidentally changed during backend or operational app work.

Decision

Treat the Next.js landing website and its visual system as a separate product surface. Keep marketing content references under `docs/marketing/`. Do not modify landing design, copy, or legacy FastAPI landing files unless explicitly approved.

Alternatives Considered

- Let backend tasks freely alter landing page presentation.
- Continue using legacy FastAPI landing files as the public source.
- Keep marketing strategy only in informal notes.

Consequences

Positive:

- Protects brand and public conversion strategy.
- Keeps landing work deliberate and reviewable.
- Reduces accidental coupling with operational app changes.

Tradeoffs:

- Landing page changes need explicit planning.
- Developers must check the source-of-truth docs before editing public website files.

Related Docs / Files

- `tis-landing-website/`
- `docs/marketing/landing_page_source_of_truth.md`
- `docs/marketing/tis_landing_page_master_content.md`
- `docs/history/landing-page/README.md`

Workspace Classification Foundation

docs/adr/0008-workspace-classification-foundation.md

ADR 0008: Workspace Classification Foundation

Status: Accepted

Date: 2026-07-22

Context

TIS needs a permanent distinction between internal sandbox, customer demo, and customer-paid workspaces. Existing onboarding, payment, and provisioning statuses describe workflow evidence, but none is a stable workspace classification authority. All records created before M8B-1 are confirmed test data.

Decision

Make `SchoolGroup` the canonical owner of stable workspace identity, classification, and lifecycle metadata. Carry pre-provisioning intent on `PendingOrganization`, account purpose on `SaaSAccount`, and internal-test attribution on operational `User`.

M8B-1 is metadata-only. Classification conversion is unavailable. No payment, entitlement, permission, tenant-isolation, reset, or customer workflow consumes the new fields. Existing records are classified through an explicit dry-run/default, transactional, idempotent backfill as internal sandbox/test data.

Consequences

Positive:

- Later workspace policies have one constrained metadata foundation.
- Stable UUIDs separate workspace identity from numeric database IDs.
- Existing test data is explicitly classified without a legacy/unknown state.
- Platform Owners can inspect metadata without receiving mutation controls.

Tradeoffs:

- M8B-1 does not yet enforce commercial separation.
- Deployment requires running and reviewing the controlled data backfill after schema migration.
- Workspace conversions require a later approved package.

Related Docs / Files

- `workspace_classification.py`
- `models.py`
- `saas/models.py`
- `saas/workspace_classification_service.py`
- `saas/workspace_classification_admin_service.py`
- `saas/commercial_state_service.py`

```
1     scripts/diagnose_workspace_classification.py
1     scripts/backfill_workspace_classification.py
1     docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
```

Commercial State And Entitlement Resolution

docs/adr/0009-commercial-state-and-entitlement-resolution.md

ADR 0009: Commercial State And Entitlement Resolution

Status: Accepted

Date: 2026-07-22

Context

Workspace classification identifies what a workspace is, but it must not be conflated with lifecycle, subscription evidence, feature values, or branch commercial activity. Future demo and paid workflows require one conservative resolution boundary before access enforcement is introduced.

Decision

Represent the effective commercial grant with `WorkspaceEntitlement`, typed values through `WorkspaceEntitlementValue` and the existing `EntitlementDefinition` catalog, and optional per-branch intent through `BranchEntitlement`.

Resolve commercial state read-only from SchoolGroup classification/lifecycle and one coherent workspace entitlement. Customer-paid resolution delegates plan features, quantity, and confirmed subscription ownership to the existing M7 entitlement resolver. Branches inherit by default; an explicit branch record may state active or inactive but must match the branch tenant and effective workspace entitlement.

M8B-2 does not enforce the result. Missing, invalid, ambiguous, stale, orphaned, or cross-tenant evidence returns manual review.

Consequences

Positive:

- One extensible commercial decision contract can support later demo and paid workflows.
- Existing M7 subscription authority is preserved.
- Branch commercial intent remains separate from operational branch status.
- Read-only resolution can be inspected safely before enforcement.

Tradeoffs:

- Existing internal development workspaces need a compatibility entitlement until all creation paths persist grants.
- Demo expiration and customer access behavior remain deliberately incomplete.
- Platform review is required for inconsistent persisted relationships.

Related Docs / Files

- `commercial_entitlements.py`
- `saas/models.py`
- `saas/commercial_validation_service.py`
- `saas/workspace_entitlement_service.py`

```
1     saas/branch_entitlement_service.py
1     saas/commercial_state_service.py
1     saas/entitlement_service.py
1     docs/adr/0008-workspace-classification-foundation.md
```

Review-Only SaaS Demo Requests Before Provisioning

docs/adr/0010-review-only-saas-demo-requests.md

ADR 0010: Review-Only SaaS Demo Requests Before Provisioning

Context

M8B-3 introduces a customer demo-request workflow after onboarding, while demo provisioning and activation remain explicitly deferred to M8B-4. Existing public marketing demo leads, SaaS onboarding organizations, operational workspaces, and paid subscriptions have different ownership and lifecycle boundaries.

Decision

Use a dedicated `SaaSDemoRequest` aggregate tied to the verified SaaS requester and `PendingOrganization`. Capture classification, commercial-state, and entitlement context at submission. Store Platform Owner approval/rejection in a separate review record and every transition in append-only audit/internal-notification events.

Approval is review evidence only. It must not create a `SchoolGroup`, workspace entitlement, checkout, payment record, Paddle object, or provisioning job. Subscribe Now continues the existing payment workflow unchanged. The legacy public marketing `DemoRequest` remains independent.

Consequences

1 Customer and Platform Owner views have a durable, permission-scoped request lifecycle.

1 Duplicate pending requests and invalid transitions fail closed.

1 M8B-4 can consume an approved request without redefining review history.

1 Approval does not imply activation, entitlement, or operational access.

1 Email delivery, expiration, schedulers, conversion, and provisioning remain future work.

M8B7 Amendment

M8B7 preserves approval as durable review evidence but makes the normal owner approval route orchestrate the separately callable provisioning service. Failed activation leaves the request Approved and retryable. Request-received, approval, and decline communications are now durable; approval communication is created only after activation succeeds. ADR 0014 owns the orchestration and communication decision.

Demo Workspace Provisioning And Commercial Source Links

docs/adr/0011-demo-workspace-provisioning-and-commercial-source-links.md

ADR 0011: Demo Workspace Provisioning And Commercial Source Links

Context

M8B-4 must create an operational customer-demo workspace after Platform Owner approval. The existing tenant-provisioning link required a paid subscription contract, but a demo has no Paddle or billing obligation. Fabricating a contract would weaken commercial-state resolution and confuse demo access with paid ownership.

Provisioning must also reuse the proven operational tenant builder, remain retryable after failure, and preserve M8B-3 approval as review evidence rather than making approval itself activate a workspace.

Decision

Use a dedicated `SaaSDemoWorkspaceProvisioning` aggregate for attempts, result, activation metadata, and resulting resource references. Store append-only provisioning events separately.

Generalize `TenantProvisioningLink` so exactly one commercial source is required: either `subscription_contract_id` for paid provisioning or `demo_request_id` for customer-demo provisioning. Existing paid rows remain unchanged.

Extract the workspace-record creation sequence from the paid provisioning service into a shared builder. The demo service invokes that builder inside a nested transaction, creates and validates a pending demo entitlement, creates the demo-sourced tenant link, then activates the SchoolGroup and entitlement. A failure rolls back all workspace changes while the outer transaction records a failed provisioning attempt and leaves the demo request Approved.

Consequences

1 Paid and demo tenants share one operational workspace creation engine.

1 Demo access has explicit commercial provenance without fabricated payment evidence.

1 Duplicate provisioning is prevented by request uniqueness, tenant-link uniqueness, and service validation.

1 Customers see safe lifecycle states; Platform Owners retain actionable failure details and audit history.

1 Logo filesystem writes remain governed by the existing provisioning engine; database provisioning is atomic.

1 Expiration, reminders, schedulers, login restrictions, conversion, email delivery, and billing changes remain future work.

M8B7 Amendment

The provisioning service remains independently callable, but the normal approval route now invokes it after recording approval. Provisioning failure preserves the Approved request and retry surface; successful activation creates the approval communication intent. No second workspace is created.

Seven-Day Demo Lifecycle And Access Enforcement

docs/adr/0012-seven-day-demo-lifecycle-and-access-enforcement.md

ADR 0012: Seven-Day Demo Lifecycle And Access Enforcement

Context

M8B-4 activates customer-demo workspaces but intentionally defines no expiration. M8B-5 requires a standard seven-day lifecycle, a Day 6 reminder, data-preserving expiration, and enforcement that cannot be bypassed by a stale authenticated session or a delayed scheduler.

Decision

Use `SaaSDemoWorkspaceProvisioning.activated_at` as the only demo clock authority. Persist derived reminder and expiration timestamps for querying, but require the lifecycle resolver to verify that they equal activation plus six and seven days. Perform calculations as timezone-aware UTC values and convert only display values to the onboarding organization's validated IANA timezone.

Use one read-only resolver for Active, Reminder Due, Expired, Suspended, and Manual Review. Scheduled processing is independently callable, dry-run by default, row-locked, idempotent, and failure-audited. Day 6 creates internal SaaS-account and Platform Owner notifications. Day 7 atomically ends the demo entitlement, suspends the SchoolGroup, marks the demo tenant link expired, and preserves all tenant data.

Enforce the same resolver in the existing operational authentication middleware. Customer-demo web requests that are expired or ambiguous are redirected to a subscription activation page; APIs and downloads receive a safe 403. Access-block audit is deduplicated. Platform identities, customer-paid workspaces, internal sandboxes, public/authentication routes, secure logout, and SaaS commercial routes bypass the demo-only gate.

Consequences

1 Scheduler delay cannot extend demo access because request-time resolution uses the authoritative timestamp.

1 Existing sessions cannot bypass expiration.

1 Expiration is reversible by a future approved commercial transition because no tenant data is deleted.

1 Processing failure rolls back expiration changes and remains safely retryable.

1 External email, conversion, extension, archive/delete, and read-only expired access remain out of scope.

M8B7 Amendment

Day 6 now atomically records the customer notice, Platform Owner Notification Center events, and reminder email intent. Expiry atomically records customer/owner notifications plus expiry and subscription-continuation email intents. Provider delivery occurs outside lifecycle row locks through the durable outbox. The CLI remains dry-run by default and is the Render Cron entry point.

Demo-To-Paid Workspace Conversion

docs/adr/0013-demo-to-paid-workspace-conversion.md

ADR 0013: Demo-To-Paid Workspace Conversion

Context

A provisioned Customer Demo already owns the customer's operational SchoolGroup, branches, users, permissions, academic data, and audit history. Running the paid provisioning engine after subscription payment would either skip the existing demo tenant without establishing paid commercial ownership or risk creating a duplicate tenant.

Decision

Convert only an active, coherently provisioned Customer Demo after TIS receives a provider-confirmed active M7 subscription for the same pending organization. A durable conversion ledger records Requested, Processing, Completed, and Failed states plus audit and internal-notification events.

The conversion runs in one nested database transaction. It preserves the SchoolGroup, workspace UUID, tenant link row, organization, branches, users, permissions, academic records, and history. It ends the demo entitlement, creates a subscription-backed paid entitlement, moves existing branch entitlement rows to that envelope, changes the workspace classification to Customer Paid, and relinks the existing tenant link to the confirmed subscription contract.

The transaction validates its result through the existing M7 subscription entitlement resolver, workspace entitlement resolver, and commercial-state resolver. It succeeds only when the same workspace resolves as Customer Paid Active with the confirmed subscription and sufficient purchased branch capacity. A failure rolls back every workspace conversion mutation while retaining provider-confirmed payment records and a retryable failed conversion audit.

Consequences

Expired-demo plan selection resolves the existing operational workspace and its authoritative active branch quantity. It enters the existing checkout and conversion ledger, never onboarding or a second workspace. Missing plan or branch configuration fails safely with support guidance while preserving all tenant data.

1 No tenant, workspace, organization, user, branch, permission, or academic record is recreated.

1 Demo request, provisioning, lifecycle, reminder, and expiration history remains available.

1 Completed conversions are excluded from demo reminder and expiration processing.

1 Existing paid provisioning remains unchanged for organizations without a demo tenant link.

1 Ambiguous, cross-tenant, internal-sandbox, or already-paid workspaces fail closed.

1 Manual conversion overrides, demo extensions, workspace deletion, and unrelated billing changes remain out of scope.

M8B7 Amendment

A coherent expired/suspended Customer Demo may now convert after authoritative confirmed payment. Conversion reactivates the same SchoolGroup, replaces the ended demo entitlement with the paid entitlement, and relinks the same tenant relationship. Checkout return, sandbox evidence, and unverified payment remain non-authoritative.

Demo Customer Communications And Approval Orchestration

docs/adr/0014-demo-customer-communications-and-approval-orchestration.md

ADR 0014: Demo Customer Communications And Approval Orchestration

Context

M8B-3 through M8B-6 established review, provisioning, lifecycle enforcement, and same-workspace paid conversion, but deliberately omitted customer lifecycle email, shared Notification Center delivery, a workspace-wide demo indicator, approval-driven activation, and expired-demo continuation.

Decision

M8B7 makes the normal Platform Owner approval route orchestrate the existing approval and provisioning services. Approval evidence remains separate from provisioning, and a failed activation preserves an Approved request plus the existing owner-only retry action. An approval email intent is created only after activation succeeds.

Demo communications use the existing branded email renderer and provider through a durable `SaaSDemoEmailDelivery` outbox. Logical events are uniquely deduplicated and provider calls occur after lifecycle transactions. Platform Owner events use the existing Notification Center with explicit destinations and deduplication keys.

The shared authenticated shell resolves active Customer Demo state through the existing lifecycle resolver and shows a persistent demo indicator only to tenant identities. Expired access remains blocked.

A coherent expired Customer Demo may enter the existing Paddle checkout flow. Only authoritative confirmed payment converts and reactivates the same SchoolGroup, workspace UUID, tenant link, branches, users, permissions, and tenant data.

Consequences

1 Request, approval, decline, reminder, expiry, and continuation communications are durable and retryable.

1 Normal approval creates exactly one operational workspace; failed provisioning remains independently retryable.

1 Scheduler delay never extends access.

1 Expired customers can subscribe without tenant reprovisioning.

1 Sandbox, checkout-return, and unverified payment evidence cannot establish Customer Paid state.

1 M8B8 AI demo limits and M8B9 lifecycle testing controls remain out of scope.

Centralized AI Entitlements And Usage Accounting

docs/adr/0015-centralized-ai-entitlements-and-usage-accounting.md

ADR 0015: Centralized AI Entitlements And Usage Accounting

Context

TIS has workspace classification, commercial-state resolution, normalized paid plan entitlements, and role permissions, but no single authority for future AI features or demo usage. Route-local feature names and counters would risk cross-tenant leakage, inconsistent plan rules, and double consumption.

Decision

`saas/ai_entitlement_service.py` is the sole AI entitlement and consumption authority. Its decision order is tenant/workspace scope, underlying permission, commercial state, registered AI policy, then successful-use consumption. Commercial and demo-expiry denial remains authoritative before allowance.

`saas/ai_feature_registry.py` defines stable feature keys, display names, permission and entitlement keys, eligible plan codes, demo allowance, and enabled state. The temporary reviewed tier mapping grants registered AI features to `enterprise_ai`; Starter and Professional receive upgrade guidance. No price or AI business feature is introduced.

Each active Customer Demo receives two successful uses per registered feature. Evaluation never consumes. Before provider execution, callers reserve an operation key; locked successful-plus-reserved capacity prevents concurrent attempts from exceeding the allowance. Completion converts the reservation to a successful use only for a usable result, or marks it failed and releases the capacity. Durable counters and operation events are unique by SchoolGroup, feature, metric context, and operation key as applicable. Internal Sandbox, demo, and paid metrics use separate contexts.

Consequences

1 Permissions, commercial access, and AI entitlement remain separate checks.

1 Unknown, disabled, expired, restricted, non-entitled, and unauthorized cases

fail closed with explicit reason codes.

1 Demo exhaustion returns the approved message and Subscribe Now destination.

1 Internal Sandbox is unlimited but separately measured.

1 No failed/no-result operation consumes usage.

1 Future M8B9 controls may inspect or extend the persisted model, but M8B8 adds

no reset, override, expiry, reminder, or lifecycle-control interface.

1 No executable AI feature currently exists, so no customer AI route is added.

ADR 0016 - Pre-Deploy Database Migration Boundary

docs/adr/0016-pre-deploy-database-migration-boundary.md

Decision

The TIS FastAPI web process does not create database tables or execute pending migrations during module import, FastAPI startup, middleware, request handling, or background threads. Render executes `python scripts/run_migrations.py` as the service Pre-Deploy Command. The command creates the repository's baseline SQLAlchemy metadata schema while excluding tables owned by ordered migrations, then executes `db_migrations.run_pending_migrations(engine)`, logs newly applied migration identifiers, and exits nonzero on any failure.

Render activates the new web version only after this command exits zero. The web Start Command remains `uvicorn main:app --host 0.0.0.0 --port $PORT`.

Rationale

PostgreSQL DDL can wait on table locks for longer than Render's port-detection window. Running DDL inside the web process couples schema lock duration to service availability and previously required a process-local readiness gate. A platform pre-deploy step provides the required ordering without exposing a partially migrated schema or delaying Uvicorn's bind.

Consequences

1 Every deployment must configure the migration command before the Start

Command is allowed to activate.

1 Migration failure blocks deployment with a nonzero exit code.

1 Repeated execution is safe because metadata creation is check-first and the

ordered migration ledger is idempotent.

1 Student/Talent tables are owned by the ordered ledger and excluded from

baseline metadata creation. Migration `20260904_000` establishes legacy Branch/Academic-Year/existing-Student composite unique parent keys before `20260904_001` creates tenant-scoped Student relationships.

1 A migration and its ledger marker share one transaction. Duplicate

prerequisite data or invalid DDL therefore fails without a marker or data rewrite and blocks web activation.

1 PostgreSQL connections use a 10-second connection timeout, 5-second lock

timeout, and 30-second statement timeout before baseline metadata or ledger DDL begins. Migration-local protections remain effective.

1 Flushed logs identify connection, metadata, ledger, per-migration apply,

marker, and commit progress. Timeout failures exit nonzero.

1 Schema inspection performed during a migration transaction uses that

transaction's active SQLAlchemy `Connection`, never a secondary connection checked out through the `Engine`.

1 The former daemon migration worker and readiness middleware are removed.

Platform Owner Demo Operations And Access Profiles

docs/adr/0017-platform-owner-demo-operations-and-access-profiles.md

ADR 0017: Platform Owner Demo Operations And Access Profiles

Context

M8B7 established Customer Demo provisioning, expiry, durable communications, and same-workspace conversion. M8B8 established centralized AI entitlement decisions and durable per-feature usage. Platform Owners need generic testing controls without customer-specific rules, destructive reprovisioning, usage resets, or a second lifecycle implementation.

Decision

`saas/demo_operations_service.py` is the owner-only orchestration boundary. It may expire an active demo immediately, reactivate an expired demo with a mandatory future expiry, set an unbounded future custom expiry, send distinct idempotent final-calendar-day reminders, and synchronously invoke the existing single-demo or batch lifecycle processor. The same SchoolGroup, workspace UUID, branches, users, permissions, classification, and data are preserved.

`saas/demo_access_service.py` resolves a workspace default and an optional tenant-validated branch override. Standard retains M8B8's two successful uses per enabled AI feature. Full enables every currently enabled controlled demo feature and unrestricted AI while the demo is active. Custom uses registry identifiers, selected features, per-feature allowances, and explicit unlimited selections. Unknown features fail closed. Moving among profiles never deletes or rewrites usage history.

Every attempted state-changing operation records actor, time, workspace and optional branch, action, reason, before/after state, result, operation key, and communication references. Material customer-visible changes use the M8B7 durable email outbox and Notification Center. Required reasons apply to expiry, reactivation, custom expiry, and access changes.

Permissions, commercial/demo lifecycle access, and feature entitlement remain separate checks in that order. Full access cannot convert a Customer Demo to Customer Paid, cannot bypass normal permissions, and cannot bypass expiry.

Consequences

1 Platform Developer status alone never grants these controls.

1 Branch overrides cannot affect another branch or tenant.

1 Custom expiry has no maximum duration but must be future-dated.

1 Manual lifecycle processing reuses production rules and reports bounded

checked/reminder/expiry/no-action/failure/skipped counts.

1 M8B9 adds no usage reset, pricing change, paid-conversion redesign, or

customer-specific exception.

1 The owner detail page uses progressive disclosure: the initial view is a

seven-field demo summary plus common operations; lifecycle processing, branch/feature configuration, audit evidence, and technical history remain present but collapsed until explicitly opened.

Unified Commercial Access And Capacity Authority

docs/adr/0018-unified-commercial-access-and-capacity-authority.md

ADR 0018: Unified Commercial Access And Capacity Authority

Context

TIS already had authoritative workspace classification, commercial-state, workspace-entitlement, paid-subscription, plan-capacity, and demo-lifecycle resolvers. Capacity checks were nevertheless split among branch, user, teacher, subscription, and provisioning code. Some paths counted people differently or could check capacity outside the transaction that performed the mutation.

Decision

`saas/commercial_authority_service.py` is the single read-only facade for effective commercial access and operational capacity. It composes the existing authorities rather than replacing or persisting them. Paid workspaces require one contract-linked confirmed subscription and active paid workspace entitlement. Customer Demo and Internal Sandbox workspaces remain explicitly unmetered in M1. Missing, contradictory, or unsupported authority fails closed.

Paid branch capacity is the confirmed `PaymentSubscription.quantity` capped by the plan branch ceiling. Staff capacity counts every distinct active tenant operational `User` in the SchoolGroup, regardless of role, title, position, or internal-test attribution. Platform identities and account-only SaaS records do not count. Teacher capacity counts distinct normalized `Teacher.teacher_id` values in active branches and active academic years; each blank legacy identity counts separately and is never silently merged.

Every capacity-increasing mutation locks its owning SchoolGroup with PostgreSQL `SELECT ... FOR UPDATE`, resolves and recounts authority, evaluates the proposed final totals, writes, and commits in one transaction. Multi-tenant internal operations lock SchoolGroups in ascending ID order. Existing over-capacity tenants keep their records and access, may reduce usage, and may change unaffected dimensions, but cannot further increase an exceeded dimension.

Consequences

1 Branch create/reactivation/bulk status updates, operational user

create/reactivation/cross-tenant activation, teacher create/year-copy, and academic-year activation use the same structured capacity decision.

1 Paid and demo provisioning validate the final commercial authority before

their nested transaction may complete.

1 Capacity checks supplement existing permissions and tenant isolation.

1 Pending plan or quantity changes grant no capacity before provider-confirmed

subscription reconciliation.

1 Paddle quantity remains active branches only; staff and teacher counts affect

plan eligibility and limits but are never Paddle quantity units.

1 M1 adds no schema, migration, pricing, packaging, Paddle, webhook, onboarding, authentication, or permission-definition change.

Secure Promo Code Definition And Governance

docs/adr/0019-secure-promo-code-definition-and-governance.md

ADR 0019: Secure Promo Code Definition And Governance

Context

TIS needs owner-governed commercial promo definitions before customer-side redemption or promo-based tenant grants can be introduced. Promo capacity must reuse the existing Starter, Professional, and Enterprise AI plan ceilings, while raw codes, lifecycle transitions, target restrictions, and historical governance require dedicated security and audit boundaries.

Decision

M2 adds definition-only `PromoCode`, `PromoCodeBranchRestriction`, and `PromoCodeAuditEvent` records. Every promo selects one active public TIS tier and exact positive branch, system-user, and teacher capacities that pass the existing plan-capacity validator. Scope may be global, organization, pending organization, account email, or email domain; reinforcing restrictions are allowed only when coherent. Branch restrictions identify existing eligible branches but grant no access.

Raw codes contain at least 100 bits of cryptographically secure randomness. TIS normalizes them with Unicode NFKC, uppercase, and approved separator removal, then persists only a deterministic HMAC-SHA256 lookup hash, key ID, and safe display prefix/suffix. `TIS_PROMO_CODE_HMAC_SECRET` is dedicated and mandatory when generation or lookup is invoked; there is no insecure fallback. The raw value appears once in a `no-store` response and is never persisted, logged, audited, or placed in a URL.

Persisted lifecycle states are Draft, Active, Paused, and Revoked; expiration is derived from the redemption deadline. Active material edits require pause, clear approval, increment definition version, and return to Draft. Platform Developers with explicit `promo_codes.view` and `promo_codes.manage` may manage non-terminal definitions, but only Platform Owners may activate or revoke. Lifecycle mutations lock the promo row through validation, mutation, durable audit insertion, and commit.

Consequences

1 Migration `20260805_001_promo_code_foundation` is additive, idempotent, and

contains no data backfill.

1 Platform Console owns list, create, one-time display, detail, edit, activate,

pause, revoke, duplicate, and replacement-definition routes.

1 Durable audit uses an allowlist and the application audit channel; neither

contains raw codes, lookup hashes, key IDs, or secrets.

1 M1 commercial authority, WorkspaceEntitlement, TenantProvisioningLink,

onboarding, Paddle, payment, provisioning, and tenant capacity behavior do not recognize promo as a commercial source in M2.

1 Promo redemption and immutable PromoGrant adaptation remain M3 work.

Promo Redemption And Commercial Grants

docs/adr/0020-promo-redemption-and-commercial-grants.md

ADR 0020: Promo Redemption And Commercial Grants

Context

M2 created secure promo definitions but deliberately granted no customer access. M3 needs to activate either a newly onboarded organization or a separately aligned existing tenant without fabricating payment, subscription, demo, or onboarding evidence. Promo capacity must participate in M1 while remaining distinct from Paddle-paid authority.

Decision

Completed activation creates immutable `PromoRedemption` and `PromoGrant` records. The grant snapshots tier, exact branch/system-user/teacher limits, scope, and effective window. A promo `WorkspaceEntitlement`, explicit branch assignments/entitlements, and `TenantProvisioningLink.promo_grant_id` make the grant operationally attributable. `TenantProvisioningLink` requires exactly one source: paid contract, approved demo request, or promo grant; its pending organization reference is optional for already aligned tenants.

Only a verified tenant owner may activate. The final transaction locks the activation session, promo definition, and target workspace, then revalidates all scope, lifecycle, redemption, source, capacity, and branch-selection invariants before one commit. Pending sessions grant no access. M1 and branch authorization resolve only an effective active grant and coherent entitlement chain.

Branch selection is explicit and reversible without changing branch status or deleting data. Organization-wide system-user and teacher counts are validation inputs only; overage blocks activation and never modifies people or roles. Active internal sandboxes cannot be converted by this workflow. Paddle and all payment records remain outside promo activation.

The selectable set and entitlement inventory are distinct. Selection considers eligible operationally active branches. Final activation inventories every preserved workspace branch: selected branches receive one assignment and active entitlement, while unselected branches receive an inactive entitlement without an assignment or operational-status change. Branch reactivation uses the existing M1 SchoolGroup lock and capacity decision, then updates operational status, assignment, and entitlement in the same transaction.

Legacy missing inactive evidence may be reconciled only when one active promo grant, one promo tenant source, one owner, one matching workspace entitlement, and coherent assignments are proven. The generic PostgreSQL CLI is dry-run by default and explicit apply creates only missing inactive entitlements. It never changes branch status, assignments, grant capacity, or people records; uncertainty remains manual review.

Customer commercial presentation follows the same source boundary. After organization and billing permission validation, the subscription route resolves M1 authority first. Paid and demo sources keep their existing dedicated portals; promo source uses a separate read-only view model. That adapter accepts M1 plan, status, usage, limits, and remaining capacity and reads the attributable grant/redemption only for effective dates and the masked reference. It must not query Paddle, expose paid-only controls, or move commercial decisions into templates.

Operational grant access ends exactly at `PromoGrant.effective_to`. `grace_period_days` is a recovery and paid-continuation window only and is never added to the effective operational-access period. Request-time resolution derives active, recovery, or expired state without mutating the immutable grant history or tenant data.

Consequences

1 Existing aligned organizations require a controlled prior classification and

owner-link operation but no fake `PendingOrganization`.

1 Raw promo code material remains absent from persistence, logs, and audit.

1 Expired, missing, ambiguous, cross-tenant, or conflicting authority fails

closed.

1 Unused branch capacity may cover future normal branch creation, but staff and

teacher capacity is never auto-reconciled.

1 Existing preserved branches always carry explicit active or inactive promo

entitlement evidence.

1 Promo renewal, transfer, communication, automated expiry jobs, and active-promo

early conversion remain deferred.

1 Amended by ADR 0041 (2026-09-21): the "transfer" deferral recorded in the

bullet immediately above is resolved for the promo-to-promo case only - replacing one organization's active grant with a grant against a different promo definition is no longer deferred; see ADR 0041, which implements it as a separate Platform Console operation. Promo renewal (extending the same promo's own window) and communication/automated-expiry-job transfer remain deferred exactly as originally decided above; this amendment note does not alter that original decision.

1 Expired and recovery-period promo authority may enter the separately governed,

provider-authoritative existing-workspace paid continuation in ADR 0024.

Existing Workspace Conversion Read-Only Audit

docs/adr/0021-existing-workspace-conversion-read-only-audit.md

ADR 0021: Existing Workspace Conversion Read-Only Audit

Context

An operational internal sandbox can contain years of tenant data and partial identity or commercial relationships. A future conversion to a real customer must preserve that workspace and cannot rely on a hardcoded tenant script, model-only assumptions, or a partial branch count. Conversion design therefore needs reproducible production evidence before any mutation is authorized.

Decision

M4A introduces a generic query-only service keyed by an explicit SchoolGroup ID, workspace UUID, exact name, and normalized owner email. It reflects the connected database schema, inventories ownership, provisioning, entitlement, paid/demo/promo evidence, traverses branch foreign-key descendants, and reports unconstrained branch-like references. Output is allowlisted and provider references are represented only by presence flags.

The standalone CLI is the only production entry point. It requires PostgreSQL, starts `REPEATABLE READ READ ONLY`, emits deterministic sanitized JSON or text, and always rolls back and closes. Canonical evidence receives a stable SHA-256 snapshot hash. Exit codes are `0` coherent, `1` execution/configuration failure, `2` manual review, and `3` workspace identity mismatch. Soft-deleted rows remain dependencies; an unmodeled branch foreign key suppresses all archival recommendations. Missing, conflicting, or uncertain evidence fails closed for manual review. The audit never constitutes approval for hard deletion or conversion.

Consequences

1 Tenant-specific identifiers remain command inputs and are absent from the

reusable service and CLI.

1 Schema drift and unknown relationships become visible blockers rather than

silent omissions.

1 Archival candidate IDs are advisory only and include active branches proven

free of direct, transitive, logical, and soft-deleted dependencies.

1 The active internal-sandbox entitlement may be reported as expected evidence;

paid, demo, promo, ambiguous, or duplicate authority blocks conversion design.

1 Branch archival, owner alignment, classification changes, approval tokens,

write-mode conversion, Paddle, email, and all data mutation remain outside M4A.

Controlled Existing Workspace Owner Alignment And Activation-Required Conversion

docs/adr/0022-controlled-existing-workspace-conversion.md

Context

M4A can prove the identity, relationships, branch dependencies, intended owner, and commercial state of an existing internal sandbox, but it deliberately has no write authority. TIS needs a controlled bridge that preserves the existing tenant and its data, establishes a real verified customer owner, and removes internal commercial authority without fabricating onboarding or billing state.

Decision

TIS uses a durable conversion operation as both approval ledger and ownership claim. The operation binds exact workspace identity, intended normalized email, M4A evidence, canonical parameters, actors, idempotency, branch/dependency and entitlement snapshots, setup fields, and lifecycle stage. Events are append-only and redacted. A partial unique database index allows one active `tenant_owner` link per SchoolGroup, after migration-time duplicate preflight.

Preparation never creates an account or password. The owner registers and verifies through the existing SaaS authentication architecture, explicitly claims the operation, and is aligned through shared operational-owner identity rules. Cross-tenant, duplicate, inactive, unverified, and platform identities fail closed. A different current owner requires Platform Owner transfer approval. Only legal name, IANA timezone, and educational program are collected.

Final conversion is PostgreSQL-only. It locks the operation and SchoolGroup, performs a fresh M4A audit, and compares stable canonical branch/dependency and sandbox-entitlement evidence to the approved operation. The complete M4A hash is required at preparation; owner alignment and setup are expected to change the later full report, so final validation compares the stored invariant subsets rather than incorrectly requiring the old full hash.

The transaction ends the active internal-sandbox entitlement, preserves its history, and sets classification `customer` with lifecycle `provisioning`. It creates no active entitlement, TenantProvisioningLink, PendingOrganization, contract, subscription, demo request, or promo grant. M1 resolves this exact source-free state as `activation_required`; Organization Account remains available and operational access remains blocked.

Consequences

1 Every branch and operational record is preserved; branch archival or deletion

is outside M4B.

1 M3 promo activation can establish the first customer commercial source.

1 Existing-workspace Paddle activation is deferred and is not presented as

available, while the normal new-onboarding Paddle journey is unchanged.

1 Dry-run is default. Write preparation and conversion require exact explicit

confirmation phrases, Platform Owner actors, row locks, and idempotency.

1 Parameter drift, audit drift, identity conflicts, setup gaps, new dependencies,

or commercial evidence block conversion without partial state.

Existing Workspace Paid Activation Through Paddle

docs/adr/0023-existing-workspace-paid-activation.md

Context

M4B leaves a preserved existing tenant as classification `customer`, lifecycle `provisioning`, and commercial state `activation_required`. Promo activation can establish authority, but normal Paddle onboarding requires PendingOrganization and tenant provisioning records that must not be fabricated for an existing workspace.

Decision

TIS uses a dedicated `ExistingWorkspacePaidActivation` aggregate anchored directly to SchoolGroup, workspace UUID, verified SaaS account, tenant-owner link, plan, interval, branch selection, quote, checkout, payment attempt, contract, and provider identity. Existing checkout/payment tables support either PendingOrganization or this activation through strict XOR constraints. Billing profile and payment-customer association can be anchored explicitly to SchoolGroup. Each workspace association records the provider address and business selected for that SchoolGroup. Account-wide mutable Paddle customer defaults are not sufficient webhook authority when one SaaS account can own multiple workspaces.

Professional and Enterprise AI include every active operational branch and Paddle quantity equals active branch count. Organization-wide staff and teacher counts remain eligibility dimensions, not provider quantity. Starter remains unavailable until restricted branch access is proven across all operational entry points.

Preparation grants no authority. The shared signature-verified Paddle webhook path treats `transaction.paid` as processing only. A matching `transaction.completed` locks the SchoolGroup and activation, revalidates all identity, quote, capacity, branch, customer, business/address, price, quantity, currency, interval, transaction, and subscription evidence, then atomically establishes paid commercial authority. No tenant provisioning engine is called.

Plan selection and payment review are distinct states. Organization Account opens the selection surface when the current activation is `draft` or `checkout_ready`; the saved plan may be highlighted and replaced by another eligible plan, producing a fresh quote and superseding only unfinished local checkout authority. This selection work performs no Paddle API call, creates no PaymentAttempt, and mutates no branch. After checkout reaches `checkout_started` or `payment_processing`, or the activation is in a manual-review/inconsistent state, replacement fails closed rather than silently changing provider-bound commercial terms.

The successful workspace remains classification `customer`; paid versus promo is represented by WorkspaceEntitlement and TenantProvisioningLink source. Lifecycle becomes `active` and operational access resolves through centralized services.

The same aggregate supports a verified owner continuing an existing expired or recovery-period promo workspace. Promo authority remains unchanged through plan selection, checkout, and payment processing. On verified completion, the locked transaction revalidates the exact promo tenant link and entitlement, ends promo entitlement authority, relinks that same tenant link to the paid contract, establishes paid workspace and branch entitlements, and preserves immutable promo evidence. Active promo grants are not eligible for early conversion.

Consequences

1 Existing branches, users, teachers, academic records, branding, and tenant identity

are preserved.

1 Browser return and incomplete provider events cannot activate access.

1 Editable pre-checkout drafts can change plan without creating provider or payment

authority; in-progress or ambiguous payment states cannot be silently replaced.

1 Duplicate launch and webhook handling is idempotent; drift and source conflicts

fail closed without partial authority.

1 Returned or reused transactions are accepted only when billed with automatic

collection and their item, price, quantity, subtotal, interval, currency, customer, address, business, checkout URL, and custom lineage match the current activation quote.

1 PostgreSQL row-lock reads refresh mapped state before idempotency decisions, so a

concurrent duplicate webhook cannot act on a pre-lock identity-map snapshot.

1 Organization Account payment presentation is attempt-authoritative: lifecycle

`provisioning` with no current attempt means Activation required, processing needs a current unexpired checkout-started or payment-processing attempt, and terminal attempts use recovery states.

1 Normal onboarding payment and demo-to-paid flows remain separate. Promo-to-paid

reuses this aggregate only through the explicit source transition in ADR 0024.

1 The additive migration must be applied before deploying the routes and webhook path.

Promo Expiry Recovery And Paid Continuation

docs/adr/0024-promo-expiry-recovery-and-paid-continuation.md

Context

Promo grants already provide a distinct non-Paddle commercial source with immutable effective dates. The product needs an explicit meaning for grace days, source-correct expired messaging, and a way for an expired promo customer to continue as paid without recreating or losing the existing tenant.

Decision

Normal operational access ends exactly at `PromoGrant.effective_to`. `grace_period_days` defines only a commercial recovery interval. During and after that interval, authorized owners may use Organization Account and preserved tenant data, but normal operational routes remain blocked. Grace never changes the grant effective window, restores branch access, or archives/deletes data.

Only recovery-period and expired promo authority is eligible for paid continuation. Active-promo early conversion is blocked because promotional value is not paid credit and TIS will not invent proration. The customer selects an eligible paid plan through the existing-workspace activation aggregate. Quote preparation, checkout launch, pending payment, failure, cancellation, expiry, and abandonment leave promo authority unchanged and do not restore expired access.

A verified `transaction.completed` is the conversion boundary. Under the existing SchoolGroup and activation locks, TIS revalidates tenant identity, quote, capacity, provider evidence, the exact promo grant, the active promo workspace entitlement, and the sole promo-sourced tenant link. One transaction then ends the promo entitlement, marks the grant converted while retaining its history, relinks the existing tenant link to the paid contract, creates paid workspace and branch entitlements, and resolves paid authority before commit. A conflict rolls back the authority transition and requires manual review. Duplicate provider confirmation is idempotent.

Commercial identity presentation is centralized in `commercial_badge_service`. Prepared source/status/plan/icon tokens drive one reusable badge for Promo, Demo, and Paid authority. Templates do not infer authority from historical rows. Compact mode is available to authorized Organization Account presentation, and full mode is used on the commercial page. The normal operational header does not render commercial identity. Unresolved or source-free states do not claim a commercial badge.

Consequences

1 SchoolGroup, workspace UUID, branches, users, teachers, academic data, branding,

ownership, and tenant data are preserved through conversion.

1 Promo redemption/grant history remains immutable and attributable after conversion.

1 There is never simultaneous active promo and paid tenant authority.

1 Failed or abandoned checkout cannot create operational access.

1 Promo, Demo, and Paid customer messaging and badges remain source-aware.

1 Long-term data archival and active-promo conversion remain future decisions.

Capacity-Based Packaging And Common Customer Feature Baseline

docs/adr/0025-capacity-based-packaging-and-common-customer-feature-baseline.md

ADR 0025: Capacity-Based Packaging And Common Customer Feature Baseline

Context

The original subscription entitlement foundation used a temporary feature-tier matrix: Enterprise AI received commercial AI, Professional and Enterprise AI received Advanced Reporting, and other catalog features awaited approval. TIS subsequently established organization-wide branch, staff-user, and teacher capacity as the authoritative self-service plan-sizing model and added paid, promo, and demo commercial sources.

That transitional feature matrix contradicted the approved product model. TIS is sold to organizations, and Starter, Professional, and Enterprise AI represent organization scale rather than progressively unlocked normal product modules.

Decision

Starter, Professional, and Enterprise AI share one normal customer feature baseline. The baseline includes teacher and branch management, observations, hiring, reporting, enabled commercial AI, Advanced Reporting, general exports, and customer-facing cross-branch reporting. `feature.audit_log` is explicitly excluded because the implemented global audit export is Developer-only.

Paid, promo, and active customer-demo authority may use the same baseline only after coherent source and lifecycle resolution. Permissions, tenant isolation, branch entitlement, academic-year scope, and operational prerequisites remain independent mandatory checks. Plan and promo capacity remain unchanged: Starter 1/5/25, Professional 5/20/100, Enterprise AI 25/100/500, and Custom above those ceilings.

The shared feature resolver performs the source-aware decision centrally. PlanEntitlement remains the persisted feature catalog and rollout mechanism; normal baseline values are identical across the three plans, while capacity, beta/disabled features, custom contracts, demo safety, and kill switches remain separate concerns.

Standard, Full, and Custom demos receive the normal customer baseline. Custom demo policy may still configure scope, safety, experimental features, and AI consumption, but cannot simulate a lower paid tier by removing normal modules.

AI feature availability is separate from provider consumption. Enabled AI features share the customer baseline and still require `ai.use`. The existing reservation, idempotency, counters, and operation events remain authoritative. A separate consumption-policy boundary supplies demo allowances now and can later supply paid, promo, rate, budget, or custom-contract limits without changing feature availability. Globally disabled AI features remain disabled.

Superseded Decisions

This ADR supersedes only:

1 ADR 0009's delegation of paid customer feature availability to a

differentiated plan matrix; and

1 ADR 0015's temporary Enterprise-AI-only availability mapping.

Their commercial-source authority, tenant isolation, fail-closed resolution, permission separation, AI accounting, idempotency, and concurrency decisions remain valid.

Consequences

1 Normal customer modules are not hidden because a coherent workspace uses

Starter, Professional, Enterprise AI, promo, or active demo authority.

1 Capacity and Paddle quantity behavior do not change.

1 Active promo operational entitlement values require safe idempotent

reconciliation; immutable grant/redemption evidence is unchanged.

1 Existing paid workspaces need no per-workspace feature backfill because paid

entitlements resolve through the current PlanEntitlement matrix.

1 Customer feature universality cannot expose Platform, Developer, System Owner,

cross-tenant, promo-admin, demo-admin, or global commercial operations.

Related Files

1 `saas/customer_feature_policy.py`

1 `saas/entitlement_service.py`

1 `saas/ai_consumption_policy.py`

1 `saas/ai_entitlement_service.py`

1 `saas/demo_access_service.py`

1 `saas/promo_redemption_service.py`

1 `db_migrations.py`

1 `authorization.py`

Versioned, Constraint-Based Smart Timetable Generation

docs/adr/0026-versioned-constraint-based-smart-timetable-generation.md

ADR 0026: Versioned, Constraint-Based Smart Timetable Generation

2026-09-02 lifecycle presentation clarification

Release 1 presents Draft ? Validate ? Approve ? Publish while retaining the existing persistence model: successful validation records approval and transitions the Draft to `publication_ready`, and `TimetableActiveVersion` identifies current publication. Active, superseded, and archived versions remain immutable. Copying an eligible immutable version creates a separate working Draft and cannot change publication. Mutation/publication endpoints may add canonical conflict evidence to their existing error envelope while retaining legacy message compatibility.

Teacher-specific timing constraints

Teacher Scheduling Rules are normalized branch/year configuration, not timetable locks and not Planning demand. Schema-v4 snapshots resolve and freeze Must teach, Unavailable, Prefer teaching, and Prefer free rules, including all/selected days, numbered/first/last periods, and assigned-class/grade/section targets. Must teach and Unavailable are hard CP-SAT constraints; preferences are objective terms. The simplified administrator workflow also exposes Schedule within: it restricts existing eligible assigned demand to selected numbered-period slots but does not require every allowed slot to be occupied. An additive semantic discriminator keeps all pre-existing Must-teach rows unchanged. Readiness and the problem builder reject deterministic conflicts, and the OR-Tools-independent validator repeats hard checks. Rule changes invalidate only unpublished Draft authority; active published versions remain immutable. Manual Draft mutation reuses current canonical hard rules to reject teacher-wide Unavailable slots and applicable Schedule-within violations without requiring temporary Must-teach completeness. The complete Draft validator is the lifecycle boundary: it checks Unavailable, Schedule-within, and Must-teach section targets, and approval/publication reuse that result. Soft rules do not block either path. Multiple applicable Schedule-within records compose additively as a per-demand union. Intersecting complete per-rule windows is rejected because it converts two valid allowed-window fragments into an unintended prohibition. The problem builder materializes the combined windows in solver input; CP-SAT and the independent validator consume the same map. A deterministic bipartite capacity check counts grouped demands once and rejects combined teacher-window conflicts, while demand-level checks prove daily-coverage, hard max-per-day/minimum-day, and double-block incompatibilities before solver invocation.

Subject Distribution Rules use a **Partitioned Sessions** contract. Weekly subject placements are partitioned into exactly the configured number of two-period double sessions and one-period single sessions. CP-SAT selects explicit physically-adjacent block starts and explicit singles and channels each occupied period to exactly one selected session. Different sessions may touch: a three-period run may be double plus single, and a four-period run may be two touching doubles. No period may belong to two sessions, and a Break, Prayer, or other physical timeline interruption prevents a double from crossing it. Daily load/session channeling strengthens hard daily coverage; it does not weaken or reinterpret max/day, minimum-day, collision, lock, or teacher rules. The independent validator proves that the placements admit the same exact partition. Block lengths above two remain unsupported and fail closed. Post-solve infeasibility diagnosis uses the same immutable problem and solver but enables controlled hard-family profiles. These profiles are explanatory only: they can identify base collision infeasibility, locks, grouped resources, Subject Distribution Rules, Teacher Scheduling Rules, or subject/teacher interaction, but their placements are never persisted. The original complete solve remains the sole generation authority. Diagnostic timeouts fail closed as inconclusive. Isolation has a dedicated Workflow timeout, `TIS_TIMETABLE_DIAGNOSTIC_TIMEOUT_SECONDS`, with a 60-second default per profile; it is not capped by or derived from the primary solver timeout. Profiles stop at the first proven family transition and omit lock/group branches when those inputs are absent.

Context

The original Timetable stored one mutable set of placements per branch and academic year. It had no durable draft/history boundary, active-version authority, reproducible input snapshot, regeneration locks, or generation-run record. Automatic generation therefore could not be introduced safely without first separating Planning authority, timetable history, publication, and future solver execution.

Decision

Timetable conflict evidence uses one canonical domain contract with stable code, legacy source code, HARD/SOFT severity, DURABLE/RECALCULABLE/TRANSIENT evidence class, safe message key/text, optional safe entities/slots/remediation, provenance, and internal correlation. Public serialization removes internal requirement, allocation, and constraint identifiers and redacts unauthorized entities. Existing readiness fields, validator errors, feasibility diagnostics, and generation-run messages remain backward compatible. Infeasibility is never inferred from timeout, cancellation, or execution failure. No universal conflict table is introduced.

Lesson Requirement is a deterministic, read-only timetable projection of Planning authority, not a second demand model. The projection records exact scope, section/subject identity, effective periods, current teacher authority, explicit-versus-legacy provenance, a stable internal requirement identity, and a source fingerprint. Explicit zero/inactive demand suppresses fallback. Schema-v5 snapshots freeze this contract for historical generation; no persisted Lesson Requirement table or public CRUD API is introduced.

The Workflow worker depends on a solver-neutral `TimetableSolver` adapter. The Release 1 production binding is the existing OR-Tools CP-SAT engine, including its bounded infeasibility diagnostics. The adapter is an execution seam only: it does not create another scheduling model or move immutable input building, independent validation, stale-authority checks, or persistence into the solver.

Deterministic readiness establishes only **Configuration Complete**. Before full quality optimization, TIS dispatches a hard-only CP-SAT feasibility run against the exact immutable snapshot and validates the complete candidate independently. The validated placements are persisted by full input fingerprint. Generation is blocked unless that exact scope and fingerprint is verified; changed authority invalidates the result automatically. Full optimization receives the verified placements as a solver hint and may persist them as a validated fallback if its quality search times out. Soft objective terms never participate in the feasibility gate and hard rules are never relaxed.

Draft staleness is orthogonal to current-input structural completeness. `input_changed` keeps the existing Draft stale and outside approval/publication, but when no genuine configuration or allocation blocker remains it is eligible for a new hard-only feasibility check. That check captures current authority in a fresh immutable snapshot; an older fingerprint is never reused. Successful verification permits Regenerate from the protected stale source, producing a new version without rewriting history. Historical run results remain durable but only runs matching current authority may drive current workflow messaging. Source eligibility is lifecycle- and arrangement-based rather than origin-based: the current populated, mutable, unpublished Draft may be manual, generated, or regenerated. Empty starter Drafts remain Generate candidates. Active, superseded, archived, published-history, and older mutable versions are excluded.

Migration `20260828_004_planning_subject_demands_foundation` introduces an additive explicit per-section subject-demand table as the future Planning authority. Its Stage 1 backfill is limited to grade-matched Subjects for Current/New sections in the same branch/year. Stage 2 routes timetable readiness, workspace, immutable snapshot, and snapshot-fed generation input through explicit-first section demand. A missing explicit row alone uses `Subject.weekly_hours` as transitional fallback; an inactive or zero row is authoritative retirement. Published and draft lifecycle semantics do not change.

Planning remains authoritative for section demand, subject requirements, assigned teachers, and HRT fallback. A placement represents one teaching period, not a literal 60-minute hour. `PlanningSubjectDemand.weekly_periods` supplies explicit per-section requirements, with `Subject.weekly_hours` retained only as missing-row compatibility fallback. HRT fallback resolves the real section-subject demands; no generic HRT lesson replaces them.

Stage 3 introduces a read-only curriculum-adjustment preview before any future late-stage demand mutation. It scopes Current/New sections by grade, explicit IDs, or all active source uses; projects demand, teacher capacity, distribution rules, grouped configuration, and mutable Draft impact; and returns a deterministic authority fingerprint for future stale confirmation. It cannot write demand, reassign teachers, regenerate a Draft, or mutate published history.

Stage 4 makes apply a separate `curriculum.adjust` capability and one atomic service transaction. It locks and rebuilds the reviewed fingerprint, rejects active generation and unresolved teacher/rule/lock conflicts, changes only selected Current/New demand and explicitly confirmed assignment state, records a durable deduplicated audit, and invalidates-but does not rewrite-the unpublished Draft. Regeneration remains explicit. Published versions and the active pointer stay immutable.

The timetable is a versioned aggregate scoped by SchoolGroup, Branch, and Academic Year. `TimetableVersion` owns placements, provenance, input authority, staleness, lifecycle, manual-edit, quality, and future solver metadata. Draft and non-active publication-ready versions may be edited. Active, superseded, and archived versions are immutable. `TimetableActiveVersion` is the sole active-selection authority and enforces one exact-scope pointer; active is not duplicated as a Boolean on a version.

The existing live timetable is imported exactly once per populated scope with `origin=imported`, a compatibility input snapshot, and an active pointer. Its placement fields are not repaired or normalized. Deterministically detectable inconsistencies are retained and recorded as safe version-level stale evidence. No historical publisher or approval is fabricated. A settings-only scope receives no empty imported version.

Until explicit version UI and publication arrive, the legacy assignment route uses copy-on-write: its first edit copies the active version to a mutable working draft, preserving locks and provenance; subsequent edits reuse that draft. Reads and exports use the operational version, preferring that compatibility draft after an edit while the imported active pointer continues to preserve the historical baseline.

Input snapshots use schema-versioned canonical JSON and SHA-256 component/full fingerprints. They record resolved Planning demand, HRT fallback decisions, timetable settings, canonical slot projection, constraints, and locks without unnecessary personal data. Stage 3.5 makes one composed timeline authoritative: each day starts at the configured shift time, retains the configured number and duration of teaching periods, inserts applicable non-teaching blocks, shifts later periods, and calculates the end. `after_period` is the preferred placement mode. Existing fixed-time blocks remain stored and compose only when their start is a current timeline boundary; ambiguous overlap fails closed without guessing or rewriting the row.

Locks persist on placements with actor and timestamp. A copied draft retains the intended locks. Later regeneration must treat locked placements as fixed decisions, but Stage 2 executes no regeneration.

Stage 4 exposes locks only on mutable drafts. Lock edits increment `edit_revision`, return publication-ready drafts to draft, and refresh the immutable input snapshot and authority fingerprint. Invalid locks remain visible and block validation.

Version review is selection-only. The compatibility operational order is the newest mutable non-active draft in the exact scope, otherwise active, otherwise the newest scoped mutable draft; a fresh manual draft therefore becomes the current customer Draft Timetable. Archived and superseded versions are never arbitrary defaults. An explicit copy is required before editing immutable history.

Draft validation checks freshness, complete demand, Planning teacher authority, canonical slots, collisions, placement integrity, and locks without a solver. Successful explicit administrator approval validates the exact draft, records its actor and timestamp, and transitions it to `publication_ready`; generation alone is not approval. Any later placement or lock mutation returns it to `draft`. Publication locks the version and exact-scope active pointer, checks edit/pointer revisions, reruns validation, supersedes the previous active version, updates the pointer, and records actor/time atomically. Regeneration preserves locked placements and requires the ceiling of 25 percent of unlocked direct-source placements to change. If that hard diversity target is infeasible, the run terminates with an explicit safe result; existing timetable hard constraints are never weakened and no near-identical fallback is persisted. Active status remains derived from the pointer rather than duplicated on the version.

`TimetableGenerationRun` is the durable PostgreSQL queue and reserves scope, requester, Generate/Regenerate mode, source version/revision, snapshot, solver/seed/diversity metadata, progress, attempts, lease/heartbeat, cancellation audit, safe failure, result, and idempotency. A partial unique index permits only one queued/running/validating/cancel-requested run per exact scope. The production web path dispatches an on-demand Render Workflow task containing only the run public ID. That task claims the exact queued row under a PostgreSQL lock, heartbeats its lease, and rejects save attempts after lease loss. The former `SKIP LOCKED` polling loop remains an optional local fallback, not a production requirement.

Stage 5.1 uses Google OR-Tools CP-SAT 9.15.6755 from a workflow-specific dependency file; no normally loaded web module imports OR-Tools. Schema-v3 immutable snapshots are the sole solve input. CP-SAT enforces exact demand, section and teacher slot exclusivity, canonical teaching slots, fixed locks, and regeneration diversity. Planning/HRT resolution happens before capture and remains subject-specific.

Generate creates a separate generated candidate. Regenerate leaves its source unchanged, fixes locked lessons, excludes the exact source arrangement, and requires the ceiling of the configured diversity percentage of unlocked placements to change (25 percent by default). Seed is recorded only as a secondary reproducibility input. A solver-independent validator checks scope, authority, exact demand, collisions, slots, locks, fingerprints, source revision, and diversity. A final current-input rebuild gates one atomic transaction that creates a publication-ready unpublished version and entries and completes the run. Generation never changes `TimetableActiveVersion` or published history.

Stage 5.2 preserves these internals but makes the newest mutable non-active version the customer Draft Timetable. Delete Draft Timetable permanently removes eligible never-published versions under exact-scope locks, preserves snapshots, runs, and history, removes unpublished dependent versions child-first, reports protected lineage, rejects active generation, and never changes the active pointer. Technical version controls remain in secondary Timetable History.

Official non-management consumption is separate from operational resolution. `timetable_visibility_service.py` resolves strictly through `TimetableActiveVersion`. `/my-timetable` additionally requires exact-scope `User.user_id == Teacher.teacher_id` identity and filters to that teacher; view-only users cannot consume mutable history.

Academic quality settings are exact-scope explicit subject-code authority captured inside schema-v3 snapshots. CP-SAT implements daily core coverage when demand reaches teaching days, distinct-day and non-consecutive preferences, optional hard ICT one-per-day, and simultaneous configured Swimming groups. Group members may share their configured common teacher in one slot, while unrelated teacher collisions and optional shared-resource capacity remain protected. The independent validator repeats all hard rules. Exports remain version-aware. Future teacher availability, broader rooms/resources, and cross-campus coordination require separately approved stages.

Rejected Alternatives

1 Random-only generation: it cannot prove constraint satisfaction or explain infeasibility.

1 Overwriting the active timetable during regeneration: it destroys operational history and rollback safety.

1 Editing active or superseded versions in place: it makes publication evidence unreliable.

1 Treating readiness as proof of solver feasibility: structurally complete input may still be mathematically infeasible.

1 Running generation without explicit SchoolGroup, Branch, and Academic Year scope: it violates tenant safety.

1 Maintaining separate preview, readiness, export, and future-solver clock calculations: competing time authority causes overlaps and stale fingerprints.

Consequences

1 Existing placements remain operational and historically preserved.

1 Version numbering and active selection have database-enforced scope guarantees, with service validation for source/snapshot operations.

1 Stage 3/3.5 implements composed canonical slot projection and structural readiness on stable snapshots; valid inserted blocks do not consume teaching-period indexes.

1 Stage 4 implements version comparison and truthful publication without mutating active history. Stage 5.1 adds generation on this boundary without changing publication authority.

1 PostgreSQL row locking plus a per-scope unique key is the version-number allocation authority; SQLite retains the unique guard for supported local tests.

1 Stage 5.1 adds durable task execution, real phase polling, and independent validation. Production provisions one Render Workflow task per run and requires no always-on solver worker. Render automatic retries are disabled; TIS terminal state plus Generate Again is the sole retry authority. Stage 5.2 adds simplified workflow and published-only visibility. Academic-quality rules add mapped distribution and grouped activities without changing Workflow or publication authority.

Related Files

1 `models.py`

1 `db_migrations.py`

1 `timetable_snapshot_service.py`

1 `timetable_version_service.py`

1 `timetable_problem_builder.py`

1 `timetable_cp_sat_solver.py`

1 `timetable_solution_validator.py`

1 `timetable_generation_service.py`

1 `timetable_generation_worker.py`

1 `timetable_generation_workflow.py`

1 `timetable_workflow_dispatch.py`

1 `timetable_visibility_service.py`

1 `timetable_logic.py`

1 `routers/timetable.py`

B10 Longitudinal Organization Intelligence Contract

docs/adr/0027-b10-longitudinal-organization-intelligence-contract.md

ADR 0027: B10 Longitudinal Organization Intelligence Contract

Context

M10 B0-B9 established a governed, privacy-closed Organization Intelligence family (`talent_org_intelligence_contract.py`, `talent_analytics_relationship_ graph.py`, `talent_analytics_privacy_closure.py`, `talent_org_intelligence_ service.py`) covering a single Program × single Academic Year snapshot: Overview (B5), Talent Map (B6), Program Portfolio/Branch Intelligence (B7), Participation Overlap (B8), and Student Drill (B9). None of these routes present a Program's results across time. A prior read-only investigation produced a B10 "Longitudinal Organization Intelligence" architecture proposal; an independent governance review returned "approve with required amendments," most significantly correcting an early lean toward the `TalentAssessmentCycle` as the ordering authority. This ADR records the amended, approved architecture decision (B10-A). It authorizes no code: B10 itself remains not implemented, and this decision is the prerequisite checkpoint before B10-B implementation may begin.

Decision

Product boundary. B10 MVP is Longitudinal Organization Intelligence, strictly bounded to one Talent Program, one Academic Year, aggregate/ non-identifiable output, the ordered M8 `TalentPlannedEvaluationPeriod` slots within that Academic Year, and exactly one selected metric per response. B10 MVP explicitly excludes multi-Academic-Year chronology, matched-cohort analytics, Student growth analytics, Student-level longitudinal history, cross-Program performance normalization, ranking, and AI. The M7 Learner Profile remains the governed individual-Student historical surface; B10 does not duplicate or replace it.

Route contract. `GET /api/talent/organization-analytics/programs/ {program_id}/longitudinal`. Required query parameters are `academic_year_id` and exactly one approved metric. Optional bounded filters are limited to `branch_id`, `grade_level`, and `planning_section_id`, reusing the existing frozen-provenance filter semantics already supported by M10 B3/B4; no arbitrary query grammar is approved. This route is APPROVED ARCHITECTURE, NOT YET IMPLEMENTED.

Academic-Year governance. B10 MVP is single-Academic-Year only; `academic_year_id` is mandatory. `AcademicYear.year_name`/label remains purely descriptive and must never be parsed, numerically interpreted, lexically sorted, or used as chronology authority. Cross-Academic-Year longitudinal ordering is explicitly deferred pending a future, separately governed Academic Year chronology/domain contract. No schema change is approved by this decision.

Authorization. Because B10 MVP is single-Academic-Year, it reuses the existing single-AY `resolve_access_context` for exactly the requested Academic Year - authentication, SchoolGroup, commercial availability, `talent_analytics.view`, tenant-bound requested Academic Year, authorized historical Branch scope, approved filters, Candidate/Identification secondary permission projection, breadth, aggregation, privacy closure, then serialization. No multi-AY access resolver is approved or required. Current `StudentAcademicPlacement` remains non-authoritative for historical scope.

Time-point model. One longitudinal presentation point is one M8 `TalentPlannedEvaluationPeriod` slot, ordered only by its governed `sequence`. The Period is the presentation/order authority; its optional linked `TalentAssessmentCycle`, when authoritative (Open or Closed), supplies the factual analytical evidence for that point. This is a deliberate correction from the original proposal's lean toward the Cycle as ordering anchor. Required

conceptual point identity is `academic_year_id`, `program_academic_year_configuration_id`, `annual_evaluation_plan_id`, `evaluation_period_id`, `evaluation_period_sequence`, optional `cycle_id`, and optional `framework_version_id`. Different Programs may plan different Period counts; no Baseline/Midyear/Final structure is assumed. The database's one-Cycle-per-Period uniqueness constraint (`uq_talent_assessment_cycles_period` on `TalentAssessmentCycle.planned_evaluation_period_id`) is the schema evidence for this model; no schema change is approved by B10-A.

Point/state semantics, using exact existing lifecycle values verified in `models.py` (`TalentAssessmentCycle.status` is exactly `draft`/`open`/ `closed`; `TalentPlannedEvaluationPeriod.status` is exactly `planned`/ `cancelled`): a linked Open or Closed Cycle is an authoritative factual point; a missing linked Cycle is `no_data`/`missing_cycle`; a linked Draft Cycle is `no_data`/`cycle_not_authoritative`; a cancelled Period is `no_data`/`cancelled_period`; an authoritative point with no frozen population is `no_data`/`no_frozen_population`. An unlinked/ad-hoc Cycle (no governed M8 Period) is excluded from B10 MVP entirely. A missing or non-authoritative annual Plan never fabricates series facts, and historical facts remain readable for a retired/inactive Program when its requested AY configuration remains valid and authorized.

Metric allowlist. Exactly nine of the fourteen existing `MetricCode` values (verified in `talent_org_intelligence_contract.py`) are approved for B10: unconditional `frozen_eligible`, `completed`, `completion_coverage`, `assessment_started`, `started_coverage`; Candidate-permission-conditional `candidate_count`, `candidate_of_eligible`; Identification-permission-conditional `identified_count`, `identified_of_eligible`. Excluded, with reasons: `required_period_execution` (Plan-wide execution grain, not a Cycle-population longitudinal point), `programs_configured`/ `active_programs` (organization configuration headline metrics, not per-Program longitudinal facts), `participation_overlap` (a cross-Program pair metric, incompatible with a single-Program B10 route), `student_drill_population` (the P7 Student Drill disclosure gate, not a longitudinal analytics metric). No new `MetricCode` is approved by B10-A; the contract remains exactly 14 `MetricCode` / 3 `MeasureComponent` / 7 `MembershipGrain` values. Exactly one metric is selected per response, deliberately bounding response complexity, privacy topology, breadth, and redundant disclosure - a multi-metric B10 query is not approved.

Comparability, not growth. Each adjacent Period-point pair for the selected metric is evaluated as exactly `comparable` or `not_comparable`. The approved `not_comparable` reason codes are `missing_cycle`, `cycle_not_authoritative`, `cancelled_period`, `no_frozen_population`, `metric_unavailable`, `framework_changed`, and `privacy_protected`. "Comparable" means only that two cross-sectional factual points may be neutrally shown side by side - never that they share the same Student cohort, and never Student growth, improvement, decline, or progress.

Framework policy. Framework sensitivity is metric-specific. `frozen_eligible`, `completed`, `completion_coverage`, `assessment_started`, and `started_coverage` remain comparable across a Framework-version change. Candidate and Identification metrics become `not_comparable` with `reason_code=framework_changed` when adjacent points use different Framework versions. Every authoritative point retains its own historical `framework_version_id`. No KPI/rubric normalization, framework equivalence mapping, or universal performance scale is approved.

No server delta. B10 MVP must not return any server-computed longitudinal arithmetic between points - no `delta`, `change`, or `percent_change` field, and no `current = previous + delta` relationship. This is an intentional privacy decision: an exact derived difference could reconstruct a protected endpoint. B10 returns only individually privacy-closed factual points plus safe comparability metadata. Future UI may use neutral factual language ("higher/lower than previous Period," "period-to-period difference," "side-by-side result") only when both underlying points are already safely visible and comparable, and must not use `growth`/`improvement`/`decline`/`progress` language; no matched-cohort authority exists.

Privacy architecture. B10 reuses the existing M9/M10 P1-P7 privacy classes; no new privacy class is introduced. The pipeline per point/ component is authorization -> frozen scope -> canonical Cell -> primary privacy -> B2 closure -> strict

closed wrapper -> serializer. No arithmetic is derived from raw pre-closure values for longitudinal disclosure, and no cross-time additive Relationship is fabricated (Period populations are not presumed disjoint, and no `annual total = sum of Period populations` identity is asserted). `no_data` remains not zero. Rate metrics (`completion_coverage`, `started_coverage`, `candidate_of_eligible`, `identified_of_eligible`) derive each Period point's percentage only from that Period's own compatible numerator/denominator after privacy closure - never by averaging percentages across Periods - and never reconstruct a protected component through the derived rate.

Candidate/Identification permission. Candidate metrics require `talent_review_candidates.view`; without it, no Candidate query, Cell, field, or placeholder is produced, and a request selecting a Candidate metric without the permission fails safely before any Candidate analytical SQL runs. Identification metrics require the equivalent `talent_official_identifications.view` with identical query-skip discipline.

Response contract (conceptual). `projection_family = program_longitudinal`; Program identity (stable id, descriptive name); Academic Year identity (id, descriptive label only); the selected metric; authorized scope/filter description; ordered Period points, each carrying program configuration identity, annual plan identity, Period id/sequence, descriptive Period label/status, optional linked Cycle id/status, historical Framework version id, and a privacy-closed metric result/state; adjacent comparisons with Period ids, comparability state, and optional governed reason code. Explicitly absent: server delta, Student identifiers, privacy threshold, `raw_value`, suppressed underlying count, hidden cohort total, and mutable current Framework metadata substituted for historical identity.

Query architecture (conceptual). Resolve single-AY tenant/Program/Plan/ Period context, resolve historical authorized filters, evaluate breadth, aggregate the selected metric by authoritative Cycle/Period using bounded set-based SQL, conditionally query Candidate/Identification only when authorized and selected, build per-point canonical Cells, apply primary privacy and B2 closure, derive any rate only from safely closed components, compute comparability metadata without numeric deltas, and serialize only through a strict closed wrapper. No Student materialization, no Student ID sets, no matched cohorts, no per-Period query loop/N+1, and no nested Session; one caller-owned Session is used throughout. PostgreSQL consistent-snapshot and performance/concurrency verification remain a production/B11 qualification gate, not part of this decision.

Deferred capabilities. Multi-Academic-Year chronology; multi-AY longitudinal series; canonical Academic Year ordering; a multi-AY authorization resolver; matched Student cohorts; retained-Student growth; Student movement decomposition; per-Student organization longitudinal analytics; KPI/rubric normalization across Framework versions; cross-Program performance normalization; server deltas; AI interpretation.

Production gates (open, not implied ready). Production privacy provider/threshold; commercial availability mapping; production breadth configuration; PostgreSQL consistent-snapshot validation; PostgreSQL performance/concurrency qualification; future Academic Year chronology governance before multi-AY support; final M10 security/release qualification.

Status. B10-A architecture contract: APPROVED/GOVERNED by this ADR. B10-B is implemented and independently security/privacy reviewed with PASS and non-blocking observations; B10 is CLOSED. As of 2026-09-09 (ADR 0028, ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED. B8 Participation Overlap and B9 Student Drill remain CLOSED after passed targeted independent re-reviews; their committed/pushed implementations remain part of current dev. This decision does not reopen either milestone.

Rejected Alternatives

1 Cycle-as-ordering-authority: the original proposal's lean toward treating

`TalentAssessmentCycle` as the primary ordering anchor was rejected during governance review because Cycles are optional/nullable and not guaranteed to exist per Period; the governed M8 Period `sequence` is the only stable per-Program ordering authority, with the Cycle supplying evidence only.

1 Server-computed delta/percent-change: rejected because an exact derived

difference between two privacy-closed points can reconstruct a protected endpoint that neither point alone would expose.

1 Multi-Academic-Year longitudinal chronology in B10 MVP: rejected because no

authoritative Academic Year chronology/ordering contract exists yet; `AcademicYear.year_name` is descriptive only and unsafe to sort or parse.

1 Matched-cohort/Student-growth semantics: rejected for B10 MVP because no

governed cross-time Student identity/matching authority exists; this remains the province of the M7 Learner Profile at the individual level.

1 Multi-metric-per-response: rejected to bound response complexity, privacy

topology, breadth evaluation, and redundant disclosure risk.

Consequences

1 After the governance checkpoint is committed and pushed, B10-B may proceed using the exact

route, metric allowlist, time-point model, comparability vocabulary, and privacy discipline recorded above, without re-litigating the Period-vs- Cycle ordering question.

1 The existing M10 B3/B4 access-context and privacy-closure primitives are

reused unchanged; no new authorization resolver is required for B10 MVP.

1 Future M11 visualization work may rely on the approved comparability/

Framework-change vocabulary without inventing growth language.

1 Multi-Academic-Year longitudinal intelligence remains blocked until a

separate Academic Year chronology decision is governed and approved.

Related Files

1 `talent_org_intelligence_contract.py`

1 `talent_org_intelligence_service.py`

1 `talent_analytics_privacy_closure.py`

1 `talent_analytics_relationship_graph.py`

1 `routers/talent_organization_analytics.py`

1 `models.py` (`TalentAssessmentCycle`, `TalentPlannedEvaluationPeriod`,

`TalentAnnualEvaluationPlan`)

B11 Production Qualification Policy

docs/adr/0028-b11-production-qualification-policy.md

ADR 0028: B11 Production Qualification Policy

Context

M10 B0-B10 are CLOSED. B11-A completed read-only qualification of the existing privacy, commercial-availability, and breadth provider seams, and B11-B observability is CLOSED. B11 overall remains NOT CLOSED. This ADR records the production-policy boundaries required before B11-C profiling; it implements no provider, configuration, schema, migration, index, isolation, permission, entitlement, application behavior, or HTTP-contract change.

Decision

Privacy thresholds and provider

Production privacy thresholds are provider-owned external configuration and must never be hard-coded in application logic, supplied through an implicit fallback constant, or exposed through telemetry or an API. The provider interface supports per-P1-P7 values and may support a global default. Missing configuration, provider-resolution failure, and provider exceptions fail closed. Environment-scoped configuration is the governance position; a tenant- specific override requires a separately governed future requirement. The Owner approved a minimum cohort of 5 for every P1-P7 class for F1. The provider must accept that value only from external configuration; a missing, malformed, or mismatched value is unavailable. Test-fixture thresholds remain non-production.

B11 requires one production `TalentAnalyticsPrivacyPolicy` implementation that consumes governed external configuration. F1 implements it through `ConfiguredRelease1PrivacyPolicy`; it never falls back to `AllowAllTestPolicy` or a permissive/default threshold and preserves P1-P7 primary plus relationship-aware complementary suppression.

Availability and authorization

Owner amendment - 2026-09-12. Results & Analytics is no longer commercially gated by `feature.organization_intelligence`. The semantic key may remain in the general feature catalog for compatibility/history, but the seven M10 Organization Intelligence routes must not query it or any plan/workspace entitlement when deciding whether an authorized Talent user can access analytics.

Production availability is permission-scoped: a valid SchoolGroup/Academic-Year context passes the availability seam, then the existing `talent_analytics.view` permission and secondary Student/Candidate/ Identification permissions remain authoritative. Tenant isolation, Branch scope, privacy closure, breadth limits, and REPEATABLE READ are unchanged. There is no plan name, plan code, price, packaging, subscription, or entitlement dependency in the Organization Analytics request path.

Breadth policy

Production breadth limits are external configuration, never implicit defaults in M10 application code. The Owner approved ceilings of 1000 matrix cells, 1000 relationship results, and 1000 Program-pair results. F1 requires those exact configured values and rejects over-limit work without truncation. Provider missing, invalid, rejection, or exception remains fail closed.

Performance evidence and SLOs

The Owner approved a numeric M10 performance target: p95 server-side M10 analytics route latency <= 2.0 seconds; p99 <= 4.0 seconds, across all seven M10 Organization Intelligence routes. This is a release-qualification target,

evidenced directionally rather than production-scale benchmarked: multiple separate, valid local PostgreSQL qualification runs on 2026-09-08 and 2026-09-09 measured every route well within this target, with substantial margin in every run. Exact millisecond timings varied between runs (fixture/harness/environment-dependent - one run measured p95 approx 170ms / p99 approx 326ms for the highest-cost route under a larger single-process dataset; independent re-runs on smaller and multi-worker datasets measured p95/p99 in the 20-71ms range across all routes tested). This variance is expected local-qualification noise, not a defect; no single exact millisecond figure from any run is adopted as canonical. The adopted requirement is the p95<=2.0s/p99<=4.0s target itself, which every qualifying run has cleared by a wide margin (see `docs/history/engineering-handbook/2026-09-07-b11c-postgresql-profiling-evidence-remediation.md`, `docs/history/engineering-handbook/2026-09-09-b11e-f2-postgresql-qualification.md`, and `docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md`). This is directional local qualification evidence, not production-scale benchmarking; production-representative performance verification against a real deployment environment remains a distinct, separate future qualification step.

PostgreSQL isolation and indexes

B11-D qualifies current `READ COMMITTED` behavior first. `REPEATABLE READ` is not adopted preemptively. Stronger isolation requires demonstrated inconsistency evidence and a governed ADR; if adopted, its transaction scope must cover the full M10 request evidence set: access context, aggregation, Candidate/Identification reads, and privacy-closure inputs.

B11-D supplied that evidence and ADR 0029 subsequently governed and implemented the resulting, scoped decision: `REPEATABLE READ` for the seven M10 Organization Intelligence routes only, via a dedicated session/dependency, covering the full request evidence set named above. This remains scoped to M10 only; no server-wide PostgreSQL isolation change was made, and no other route's session/transaction behavior changed.

No index migration is approved without `EXPLAIN/ANALYZE` evidence of material plan improvement, redundant/overlapping-index review, migration review, documented rollback implications, and a governing ADR. This checkpoint approves no candidate index.

Memory and release qualification

The Owner approved numeric memory targets: incremental memory per M10 analytics request <= 128 MB; peak worker/process RSS <= 70% of authoritatively allocated worker/container memory. Evidence required before master/deployment is baseline web RSS, startup/import memory, per-request peak RSS, privacy-graph memory, query-result memory, concurrent-request multiplier, and process-count multiplier (see ADR 0030 for the process-count/multi-worker qualification minimum this requirement resolves to).

The <=128MB target is PASS with substantial margin across every qualifying run to date - worst-case observed incremental figures ranged from approximately 0.48MB (single-process, larger dataset) to 0.17-0.28MB (multi-worker, smaller dataset) per request/request-sequence, all a small fraction of the approved ceiling; no monotonic retained-memory growth was observed across repeated requests in any run. As with performance, no single exact figure from any one run is adopted as the sole canonical measurement - the adopted requirement is the ceiling itself, cleared with substantial margin by every run.

The <=70% allocated-worker/container-memory target remains **NOT EVALUATED LOCALLY / ENVIRONMENT-SPECIFIC**: no authoritative production Render Web Service worker/container memory allocation value exists anywhere in this repository for any qualification task to compare against. This is an environment-specific production-deployment verification point, not a code or product defect, and is not grounds to withhold adoption of the target itself - the target is approved; its local evaluation is structurally impossible without an allocation figure this repository does not have. Memory/OOM qualification blocks release, not the start of B11-C.

Configuration and failure behavior

Privacy thresholds and breadth limits follow existing external environment/ configuration conventions; M10 does not introduce a new central settings architecture. Organization Analytics availability is permission-scoped and has no commercial entitlement dependency. KMS must contain no credentials, secrets, or environment-specific values.

Existing fail-closed behavior remains authoritative for privacy provider missing/exception and breadth provider missing/reject/exception. The availability seam still rejects malformed tenant/year context, but commercial feature/entitlement state is no longer an availability input. No response schema change is introduced.

Qualification Entry And Release Gates

B11-C entry

B11-C may begin after this policy checkpoint is committed and pushed, the cumulative KMS check is green, local/staging PostgreSQL is available, a representative non-production dataset is available, and profiling uses non-production/test-staging provider configuration. It does not require final production privacy thresholds, commercial plan mapping, final breadth limits, a production SLO, or a final isolation change.

B11-D entry

B11-D requires production-like PostgreSQL, controlled concurrent writers, representative M10 read paths, and governed non-production provider configuration. Current `READ COMMITTED` behavior must be tested first.

B11-E release gate

B11 cannot be CLOSED until all of the following are complete: the production privacy provider is implemented/configured; privacy-threshold governance and the Owner-approved permission-scoped availability rule are complete; production breadth configuration is complete; B11-C PostgreSQL performance evidence and B11-D concurrency/ consistency evidence are complete; memory/OOM evidence is complete; B11-B observability is present; the integrated seven-route security regression is green; every approved index/migration is applied and verified in the qualification environment; and authoritative KMS is updated. B12, master merge, and deployment remain blocked until B11-E passes.

B12 (M10 closeout)

B12 means M10 closeout only. B12 is strictly downstream of B11-E and does not begin until the B11-E release gate above is truthfully satisfied. B12 acceptance requires: B11-E release qualification requirements satisfied; M10 implementation/regression/security/privacy evidence green; no unresolved BLOCKER/HIGH defect open against any M10 milestone; authoritative KMS accurately reflects implementation/evidence with no internal contradiction; schema/migration state verified (none expected or approved for M10); operational/deployment impact known and documented; closeout evidence complete. B12 closure does NOT itself mean: M12 AI is complete; `dev` is merged to `master`; production is deployed. Non-blocking LOW-severity observations may remain open at B12 closure only if recorded with explicit tracking (a "Known Limitations" section in the closing engineering-handbook entry, referenced from this document's Status section) - this repository has no separate issue-tracker artifact type, so KMS itself is that tracking mechanism. Sequence: B11-E pass -> B12 M10 closeout -> cumulative production qualification -> KMS/Git checkpoint -> `dev->master` approval -> deployment. Product-implementation work outside M10 Organization Analytics (e.g. the Talent & Potential Students/Program-workflow/Results-experience UI tracked separately in `docs/PROJECT_STATE.md`) may serve as supporting regression/ product evidence but is never itself B12 closure - these are two independent axes, and completion on one never implies progress on the other.

B12 status: CLOSED (2026-09-09). Independent verification against every acceptance item above found each genuinely satisfied on current KMS/evidence: the B11-E release gate (production privacy/availability/breadth providers, performance/memory evidence, B11-C/B11-D evidence, B11-B observability, the green integrated seven-route regression, index/migration verification, KMS accuracy) is complete per this document's own Status section below; M10 implementation/regression/security/privacy evidence is green (226-test integrated M10/PostgreSQL/privacy regression passed per `docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualifi`

cation.md; the 15/15 REPEATABLE READ suite re-confirmed live per docs/history/engineering-handbook/2026-09-09-b11e-f2-postgresql-qualification.md; the 22-test observability suite remains green); no unresolved BLOCKER/HIGH defect is open against any M10 milestone (the M9 call-ordering BLOCKER and the related coverage-bundle HIGH-severity defect were each found and fixed by their own remediation pass, and no other BLOCKER/HIGH remains open anywhere in current KMS); authoritative KMS is internally consistent (independently re-verified against docs/PROJECT_STATE.md, docs/TIS_MASTER_CONTEXT.md, docs/AI_PROJECT_CONTEXT.md, docs/engineering/PRODUCT_ROADMAP.md, docs/engineering/TIS_MODULE_MAP.md, ADR 0027, and ADR 0029, all of which state the same current B11-E/B11/B12 status with no contradiction); no schema/migration is approved or required for M10 (git diff HEAD -- db_migrations.py models.py is empty); this is Web-Service-only governance work with no tis-timetable-workflow interaction; and the full closeout evidentiary chain (F1 provider implementation review -> live PostgreSQL multi-worker qualification -> ADR 0028/ADR 0030 adoption -> this B12 definition -> cross-document consistency repair) is complete and cross-referenced.

B12 closure means only: M10 (B0-B11) is closed. It does NOT mean: M12 AI is complete or implemented (it remains separate and NOT IMPLEMENTED, per current KMS); dev has been merged to master (it has not); production has been deployed (it has not). The next step is cumulative production release qualification, followed by a dev->master approval and deployment decision - neither has occurred and neither is implied by this closure. The sole remaining environment-specific item - peak worker/process RSS as a percentage of an authoritatively allocated production worker/container memory limit (<=70% target) - is NOT reopened as a B12 blocker; it remains an explicitly open, future production-deployment/qualification verification item that can only be evaluated once a real production allocation value exists, consistent with its treatment under the B11-E release gate above.

Deferred Decisions

The following remain explicitly unresolved: tenant-specific privacy overrides; exact plan-to-feature packaging; and any future index addition. The privacy threshold, semantic feature key, breadth ceilings, M10-only isolation decision, numeric performance targets, and numeric memory targets are approved as recorded above and in ADR 0029/ADR 0030. The <=70% allocated-worker/container-memory target's local evaluability remains an environment-specific gate (see Memory and release qualification above), not a deferred numeric decision - the number itself is approved.

Status

B10 is CLOSED. B11-A qualification architecture is complete. B11-B observability is CLOSED. These B11 governance boundaries are APPROVED/GOVERNED. B11-C completed PostgreSQL 16.15 profiling and is CLOSED after independent re-review returned PASS WITH NON-BLOCKING OBSERVATIONS. No index or production policy value was approved.

B11-D subsequently completed PostgreSQL 16.15 READ COMMITTED qualification and is CLOSED after independent review returned PASS WITH NON-BLOCKING OBSERVATIONS. Same-request mixed snapshots were proven. REPEATABLE READ was PROPOSED - GOVERNANCE REQUIRED pending a separate governed decision covering full-request transaction scope and retry/error handling.

B11-E subsequently governed and implemented that decision as ADR 0029: a dedicated M10-only REPEATABLE READ session/dependency (database.M10OrganizationAnalyticsSessionLocal, dependencies.get_m10_organization_analytics_db) beginning before the first statement of every one of the seven M10 Organization Intelligence routes, strictly backend-conditional (PostgreSQL only), never applied server-wide or to any other route. Live re-testing under the permanent implementation confirmed all seven routes are consistent (see docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md). No retry logic was added (no serialization failure/deadlock evidence). Suppression/reconstruction concurrency was tested live with the existing non-production DeterministicSuppressionTestPolicy and found consistent. Index Candidate B remains classified NO CHANGE. Independent review returned PASS WITH NON-BLOCKING OBSERVATIONS. F1 subsequently implemented the Owner- approved, fail-closed production privacy, availability, and

breadth providers: minimum cohort 5 for P1-P7, `feature.organization_intelligence`, and 1000/1000/1000 breadth ceilings. The provider-code blocker is resolved.

`docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md` subsequently directly executed and evidenced the remaining release-gate items: performance (p95<=2.0s/p99<=4.0s target formally adopted above, PASS with substantial margin across multiple independent runs); memory (<=128MB incremental/request target formally adopted above, PASS with substantial margin, no monotonic growth); multi-worker qualification (ADR 0030, >=2 independent processes, PASS on provider consistency, no worker-local-state dependency, concurrent-writer safety, and worker-kill correctness). B11-B observability remains CLOSED. The integrated seven-route regression suite remains green. No index or migration is approved or required.

B11-E status: CLOSED, WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING. Every release-gate item this repository can evidence is satisfied. The sole remaining item - peak worker/process RSS as a percentage of an authoritatively allocated production worker/container memory limit - cannot be evaluated from within this repository because no such authoritative allocation value exists here; this is classified as an environment-specific production-deployment verification point, not a code defect, not an unresolved evidence gap, and not grounds to withhold this classification. B11 overall is CLOSED on this same basis. **B12 is CLOSED (2026-09-09)** (see the B12 subsection under Qualification Entry And Release Gates above for the full acceptance-criterion verification and closure boundary). B12 closure means only that M10 (B0-B11) closeout is complete; M12 AI remains NOT IMPLEMENTED, no merge to `master` has occurred, and no production deployment has occurred or is implied by this document. The next step is cumulative production release qualification, followed by a separate `dev->master` approval and deployment decision. The sole remaining environment-specific item - peak worker/process RSS as a percentage of an authoritatively allocated production worker/container memory limit - remains open as a future production-deployment verification item, not a B12 blocker.

M10 Organization Analytics REPEATABLE READ Transaction Boundary

docs/adr/0029-m10-organization-analytics-repeatable-read-boundary.md

ADR 0029: M10 Organization Analytics REPEATABLE READ Transaction Boundary

Context

ADR 0028 governed B11 production qualification and explicitly deferred any PostgreSQL isolation change: "B11-D qualifies current `READ COMMITTED` behavior first. `REPEATABLE READ` is not adopted preemptively. Stronger isolation requires demonstrated inconsistency evidence and a governed ADR."

B11-D (`docs/history/engineering-handbook/2026-09-07-b11d-postgresql-concurrency-consistency-qualification.md`) supplied that evidence. Under the current default `READ COMMITTED` behavior (`database.py`'s single process-wide `SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)`, one `Session` per request via `dependencies.get_db`, no explicit isolation level), PostgreSQL takes a fresh snapshot per SQL statement, not per transaction. Every M10 Organization Intelligence route builds one lazy population `Query` object once (`talent_org_intelligence_service.frozen_membership_query`) and passes it to two or more independent downstream calls, each issuing its own fresh SQL statement. B11-D proved, live, against a dedicated non-production PostgreSQL 16.15 instance, that a real domain-valid write committed between two of a route's own statements is observable within that same still-open request:

1 **Overview**: `coverage_organization_total` (SQL#1) returned a population

snapshot unaware of a write; `candidate_membership_counts` (SQL#2), executed moments later in the same request, returned a Candidate count aware of it (314 -> 315).

1 **Student Drill**: `count_distinct_students` (the P7 eligibility gate,

SQL#1) evaluated a population of 705 Students; `fetch_student_rows` (the identifiable page fetch, SQL#2) served a 706th Student not part of the gated population the gate decision was actually made against.

B11-D also proved, via an isolated `Connection.execution_options(isolation_level="REPEATABLE READ")` experiment (never made permanent, never server-wide), that `REPEATABLE READ` fully resolves both cases, and code-inspected `program-portfolio`, `branches/{id}`, `talent-map`, and `longitudinal` as sharing the identical structural mechanism (not independently reproduced live at that time). This ADR's own qualification work (B11-E, `docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md`) independently reproduced the identical vulnerability live for all five of those remaining routes as well, using a real, dedicated, non-production PostgreSQL 16.15 database, confirming the required scope begins even earlier than the two originally proven statement pairs: before `frozen_membership_query` is even called, because the transaction's snapshot is already fixed by the context/access-resolution statement that runs first in every route.

Decision

Adopt `REPEATABLE READ` for the seven M10 Organization Intelligence routes only (`overview`, `talent-map`, `program-portfolio`, `branches/{branch_id}`, `participation-overlap`, `programs/{program_id}/longitudinal`, `students`), via a dedicated session/dependency. This is the smallest change that resolves every demonstrated inconsistency; no higher isolation level (e.g. `SERIALIZABLE`) is adopted,

because B11-D's and this ADR's own live re-testing show `REPEATABLE READ` is sufficient.

Why `READ COMMITTED` is insufficient

`READ COMMITTED`'s per-statement snapshot allows two statements inside the same still-open, single-`Session` HTTP request to observe two different committed database states whenever any concurrent write commits between them. M10 routes structurally rely on one population `Query` object being executed multiple times across the same request; B11-D and this ADR's own qualification proved this is not merely theoretical for every one of the seven routes.

Why `REPEATABLE READ` is sufficient

PostgreSQL's `REPEATABLE READ` fixes one snapshot for the entire transaction at its first statement. Every M10 route's first database statement is context/access resolution (`talent_org_intelligence_service.resolve_access_context`, called from `routers/talent_organization_analytics.py`'s `_b7_context`), which happens before any population, aggregate, Candidate, Identification, or privacy-closure-input read. Fixing the snapshot there guarantees every later statement in the same request - including ones issued well after the population `Query` object is built - observes that same snapshot. B11-D's isolated experiment and this ADR's own live re-test (Phase 4 below) confirmed this resolves every reproduced case with no other observed behavior change.

Exact scope

The dedicated session begins before the first relevant database statement of the request (context/access resolution) and remains open for the request's entire read set: Program/Branch/Academic-Year scope resolution, frozen population reads, aggregate reads, Candidate inputs, Identification inputs, Student Drill membership/page reads, longitudinal inputs, and privacy inputs. Privacy/reconstruction-closure itself issues no SQL (confirmed by B11-D's code reading, reconfirmed here), so it is automatically covered once every SQL input it consumes shares one snapshot - no separate closure-specific transaction handling is required.

This transaction boundary applies ONLY to the seven M10 Organization Intelligence routes. It does not change the default, process-wide `SessionLocal`/`get_db`/`database.engine` isolation used by every other route or module in the application, and it does not apply server-wide (no PostgreSQL server configuration change).

Implementation mechanism

`database.py` adds `M10OrganizationAnalyticsSessionLocal`, a `sessionmaker` bound to `engine.execution_options(isolation_level="REPEATABLE READ")` - a shallow proxy that shares the same underlying connection pool as `engine` but does not alter `engine`'s own default isolation for any other session. This binding is strictly backend-conditional on `DATABASE_URL.startswith("postgresql")`; on the SQLite fallback (`tis.db`, and every existing in-memory test fixture), the M10 sessionmaker binds to the plain `engine` unchanged, because SQLAlchemy's SQLite dialect does not accept a `REPEATABLE READ` isolation level string. `dependencies.py` adds `get_m10_organization_analytics_db`, a dependency generator mirroring `get_db`'s exact shape (`SessionLocal() -> yield -> close()`). `routers/talent_organization_analytics.py` uses this new dependency (`Depends(get_m10_organization_analytics_db)`) instead of `Depends(get_db)` for all seven route functions; no other route in the application is changed.

Retry / error semantics

Not adopted. B11-D's evidence (zero deadlocks, zero lock-wait timeouts across every tested concurrency scenario) and this ADR's own live re-testing (zero serialization failures, zero deadlocks, across every reproduced scenario and every one of the seven routes) do not demonstrate a practical retry need for M10's read-only access pattern. `REPEATABLE READ` can in principle raise a PostgreSQL `40001` serialization failure for read/write conflict shapes not exercised by this qualification (M10 routes never write); no retry logic is added speculatively. If future evidence shows a genuine serialization-failure rate for this access pattern, that is a separate, future governance decision with its own bounded retry-count/backoff proposal - not implemented here.

Observability implications

`talent_organization_analytics_observability.OrganizationAnalyticsObservation` issues no SQL of its own and is unmodified by this change; a `database_failure` telemetry classification already exists for any `sqlalchemy`-origin exception (including a future serialization failure), so no observability gap is introduced.

Privacy/consistency implications

Every SQL input to `talent_analytics_relationship_graph`/ `talent_analytics_privacy_closure` now shares one fixed snapshot per request, closing the specific reconstruction-relevant risk B11-D flagged as unmeasured: a suppression decision evaluated against a raw value from one statement while a relationship-linked sibling component was gathered from a different, inconsistent snapshot. This ADR's own live testing (Phase 5, using the existing non-production `DeterministicSuppressionTestPolicy`) confirms this for both a standalone suppressed Cell and a relationship-linked (complementary-suppression) pair.

No unrelated change

No PostgreSQL server-wide isolation setting is changed. No other route's session/transaction behavior is changed. No index, migration, schema, permission, or entitlement change is made or implied by this ADR. No numeric privacy threshold, breadth limit, performance SLO, or memory ceiling is approved by this ADR - those remain governed by ADR 0028's Deferred Decisions.

Status

Accepted and implemented for the seven M10 Organization Intelligence routes only (`database.py`, `dependencies.py`, `routers/talent_organization_analytics.py`). B11-E qualification and live re-testing are recorded in `docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md`. Independent review returned PASS WITH NON-BLOCKING OBSERVATIONS. B11-E implementation is PASSED and checkpointable. As of 2026-09-09 (`docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md`, ADR 0028, ADR 0030), B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B11 overall is CLOSED on that same basis; B12 is CLOSED.

M10 Organization Analytics Multi-Worker Qualification Minimum

docs/adr/0030-m10-organization-analytics-multi-worker-qualification-minimum.md

ADR 0030: M10 Organization Analytics Multi-Worker Qualification Minimum

Context

ADR 0028's Memory and release qualification section requires a "process-count multiplier" as part of B11-E memory/OOM evidence but does not fix a numeric qualification minimum, and every prior B11-C/B11-D/B11-E qualification task explicitly disclosed its own evidence as single-process only (`docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md`, Known Limitations: "Multi-process/multi-worker ... concurrent-request memory and consistency qualification remains unmeasured ... not resolved by this task"). No ADR previously governed a minimum worker/process qualification count or an explicit correctness requirement against worker-local mutable state.

`docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md` subsequently supplied that evidence: two genuinely independent `uvicorn` subprocesses, both against the same live PostgreSQL qualification database, serving real production-configured F1 providers (`ConfiguredRelease1PrivacyPolicy`, `ConfiguredRelease1BreadthPolicy`, the real `EntitlementOrganizationAnalyticsAvailabilityProvider` path), proving byte-identical cross-worker responses, fresh-transaction-per- request consistency under a concurrent writer, and continued correct operation of one worker after the other was terminated.

Decision

Multi-worker qualification minimum

Release qualification for the seven M10 Organization Intelligence routes requires evidence gathered across a minimum of 2 concurrent worker/process instances, not a single process, before any multi-worker PASS is recorded against the B11-E release gate's "process-count multiplier" requirement.

Correctness must not depend on worker-local mutable state

M10 route correctness (privacy suppression, breadth limiting, tenant isolation, availability gating, REPEATABLE READ snapshot consistency) must not depend on any module-level cache, singleton, or in-memory-only state that is not independently reconstructed per worker process. A worker-local cache may be used only as a performance optimization whose absence or staleness cannot change a response's correctness; it must never be the sole source of truth for a suppression decision, breadth limit, availability decision, or tenant-scoping filter. `docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md` confirmed, by direct source inspection of every M10 provider/route/service module, that no such state currently exists in this code path.

Not a production topology claim

This ADR does not state, imply, or approve any actual Render Web Service worker/process count. Real production worker count remains Render-dashboard- owned configuration, intentionally undocumented in this repository per existing KMS convention (confirmed absent from `Procfile`, `render.yaml`, and every deployment workflow in this repository). This ADR governs only the minimum qualification-evidence bar for release purposes, not a deployment

decision.

Scope

Engineering/release-qualification scope only. Creates no product/commercial capability, plan, feature, or customer-facing behavior change, and does not by itself declare the B11-E release gate fully satisfied - it resolves specifically the multi-worker/process-count portion of that gate's memory/OOM evidence requirement.

Evidence

`docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md`: provider/response consistency across 2 independent workers PASS; no worker-local correctness dependency PASS (by code inspection, corroborated behaviorally); concurrent writer safety / fresh-transaction-per-request semantics across workers PASS; worker-kill correctness PASS; performance (5 of 7 routes, 20 samples each) PASS against p95<=2.0s/p99<=4.0s with wide margin; memory (self-measured RSS, real production providers engaged) PASS against <=128MB incremental/request with wide margin, no monotonic growth. Tenant isolation within that specific multi-worker harness run was not independently re-tested (single-tenant fixture, by design, for scope control); tenant isolation under the identical M10 REPEATABLE READ session type remains proven live and passing in `tests/test_talent_organization_repeatable_read.py`.

Status

ACCEPTED. The multi-worker qualification minimum (>=2 independent workers/processes), the no-worker-local-state correctness requirement, and the not-a-topology-claim boundary are governed as of this ADR. This resolves the "process-count multiplier" component of ADR 0028's B11-E memory/OOM release-gate requirement. It does not by itself close B11-E - the remaining release-gate items (KMS accuracy, integrated regression, etc.) are tracked in ADR 0028's own Status section.

Student Learning Style V1

docs/adr/0031-student-learning-style-v1.md

ADR 0031: Student Learning Style V1

Agent 3 owner amendment: Classification-filtered aggregates (2026-09-25)

This narrows the Acceptance C aggregate exception only where Learning Style would disclose or reconstruct a protected Classification cohort. It does not restore the deprecated four per-Student percentage dimensions.

The individually authorized Student Assessment roster is an operational surface. Its exact authorized identity, current Assessment state, backend Classification and Learning Style are not subject to aggregate cohort-size suppression. Tenant, Branch ceiling, Student visibility and permission gates remain mandatory.

Learning Style aggregates remain filter-adaptive to authorized Branch, Grade, Section, Program and Evaluation Period context. A selected Classification must also narrow the population, never be silently ignored. Before publishing, the existing backend privacy provider must permit that Classification bucket and denominator after primary/complementary suppression in the original population. Otherwise return the canonical restricted state, reason `classification_cohort_protected`, with no exact total, bucket count, percentage, chart magnitude, tooltip, DOM/dataset or ARIA number. Explain that the summary is unavailable because the cohort is protected for privacy, not zero or no-data. No frontend threshold may decide this.

The shared `talent_learning_style_privacy.py` gate serves roster summaries and the new filtered dashboard. Non-Classification Learning Style retains its authorized aggregate exception, eight categories plus Unassigned and distinct-Student grain. Classification retains its existing P4 provider and fail-closed, complementary and reconstruction protections. This fixes Agent 2's exact-total disclosure. Earlier sections are superseded only to the extent stated in this amendment.

Context

`docs/TIS_MASTER_CONTEXT.md`, `docs/PROJECT_STATE.md`, `docs/AI_PROJECT_CONTEXT.md`, `docs/engineering/PRODUCT_ROADMAP.md`, and `docs/CHANGE_HISTORY.md` have repeatedly and consistently recorded, across the M7-M10 Talent & Potential milestone history, that Learning Style has no authoritative data contract and remains deferred/future-governed work (e.g. `TIS_MASTER_CONTEXT.md`: "No profile table, current Talent status, cross-Framework score, audit read event, M8 periods, or Learning Style exists."). Those statements are accurate historical fact as of the milestones they describe and are not being rewritten - Learning Style genuinely did not exist in the product before this ADR.

The Owner has now directed, explicitly and directly, that Learning Style be introduced as an optional Student-domain learner-profile attribute, V1 scope only. This ADR is the governed decision that supersedes the prior deferred status for V1 scope specifically. It does not retroactively alter the historical record of what earlier milestones shipped.

Decision

Domain and scope

Learning Style belongs to the **Student** domain, not the Talent domain. It is learner-profile context only.

1 Optional (nullable) attribute on `Student`.

1 V1 supports exactly one primary Learning Style per Student (single-select).

Multi-style support is explicitly out of scope for V1 and is not to be scaffolded speculatively.

1 **Amended by ADR 0042** (2026-09-22, unmodified original text above):

a later, separately governed decision adds an independent four-dimension Learning Style percentage profile (Verbal/Non-verbal/ Quantitative/Spatial) alongside this single-select categorical field. That later profile is not a multi-select of the four categorical values above and is not the speculative scaffolding this bullet declined; this field, its four values, and its single-select nature remain exactly as decided here.

1 V1 approved value set, exactly these four values, no others:

1 Visual

1 Auditory

1 Read/Write

1 Kinesthetic

1 **Amended by M14** (2026-09-23, unmodified original text above): the

approved value set is extended to exactly eight values - the original four above plus Verbal, Non-verbal, Quantitative, and Spatial (see the "M14 Amendment" section at the end of this ADR for the full governance record). The field remains single-select; this is not multi-select of the four categorical values.

1 Server-side validation must reject any value outside this set (now eight

values as of the M14 amendment). The values are not free text and must not be stored as unconstrained free text.

No effect on Talent semantics

Learning Style has zero effect on, and must never be read by:

1 Rubric-level computation or any assessment scoring path.

1 Program Criteria / Review Candidate eligibility computation.

1 Official Identification decision logic.

1 Any other deterministic Talent analytics computation.

It may be *displayed* alongside Talent/Student context as informational learner context, but no Talent-domain scoring or eligibility code path may depend on it.

Permissions

Learning Style is edited using the existing `students.edit` permission (`permission_registry.py`). No new permission is created for this feature.

Analytics and privacy

> **Amended 2026-09-24 (Acceptance C):** the Talent small-cell/complementary- > suppression requirement in this section no longer applies to the Learning > Style distribution; see the "Acceptance C Amendment" section at the end. > The historical text below is preserved unmodified.

Where Learning Style distribution is surfaced in aggregate (Branch or Organization level), it must go through the same privacy/suppression contract already governing Talent aggregate analytics (minimum cohort suppression, complementary suppression, tenant isolation, fail-closed behavior) - reusing existing enforcement infrastructure rather than a separate, weaker rule invented for this field. A protected cohort's visual treatment must not encode hidden magnitude through bar length, color intensity, percentage, tooltip, or ordering.

Student list branch scope and unavailable analytics presentation

> **Amended 2026-09-24 (Acceptance C):** the privacy-provider fail-closed and > "privacy-protected state" behavior described in the second paragraph below is > removed for the Learning Style distribution (it no longer depends on the > privacy provider); Student list Branch scope is unchanged.

Cross-Branch Student browsing is explicitly permissioned. The dedicated `students.view_all_branches` permission is Administrator-only in the managed role policy and is effective only with organization/global access scope. Without both conditions, the Students list is fixed to the actor's assigned authorized Branch and does not offer an **All branches** choice.

The aggregate privacy contract remains fail closed. If the governed privacy provider is unavailable or invalid, the Students page may explain that Learning Style statistics are unavailable, but it must render **no aggregate counts, percentages, category bars, or inferred magnitude**. A configured policy that suppresses the selected cohort continues to show the existing privacy-protected state instead of raw statistics. Individual Student Learning Style values remain governed by normal Student-record permissions and are not a substitute for aggregate analytics.

Governance boundary

This ADR authorizes exactly the V1 scope above: one nullable single-select Student field, four fixed values, existing-permission reuse, and privacy-contract-compliant aggregate display. It does not authorize: multi-style support, Learning Style influencing any Talent scoring/ eligibility/identification computation, a new or weakened privacy rule, or any Branch-level override of the Student-domain ownership model.

Status

ACCEPTED as of 2026-09-10, per direct Owner instruction. Implementation (schema/migration, API validation, Student create/edit UX, Student profile display, Talent-context display, Branch/Organization analytics distribution, and focused tests) is tracked separately and is not itself part of this governance record; this ADR exists so that implementation work has a durable, citable authorization artifact rather than relying on unverifiable in-task claims of authorization.

M14 Amendment (2026-09-23): Eight Values, Aggregate-Only Percentage Semantics

Added 2026-09-23. The original text above (as of documentation_version 1.2) is preserved unmodified; this section records the owner-directed correction without rewriting the historical decision it amends.

What was corrected

The M1-M13 delivery history built ADR 0042 as a SEPARATE, independent four-dimension Student percentage profile (Verbal/Non-verbal/Quantitative/ Spatial), read/written through its own create/update/GET API surface and, for the four percentages, a real Student create/edit frontend (see ADR 0042's own M14 amendment note for the full implementation-history record). The Owner has now directed, via a read-only Post-M13 Correction Review followed by this M14 correction milestone, that this was a misinterpretation of intent on two points:

1 Learning Style was always meant to be ONE categorical selection per

Student - originally four values, now extended to eight (this ADR's field), never a set of independent per-Student percentage scores.

1 "Percentage," wherever Learning Style is concerned, was always meant to

describe a POPULATION/AGGREGATE distribution (what share of an authorized Student population selected each category) - never a per-Student dimension percentage.

Corrected model

1 `Student.learning_style` (this ADR's field) now accepts exactly eight

values: Visual, Auditory, Read/Write, Kinesthetic, Verbal, Non-verbal, Quantitative, Spatial. Still one nullable, single-select column - no second field, no arrays, no weighting, no dominant-style calculation, no automatic inference between any value and any other data.

1 The four `learning_style_*_percentage` columns (ADR 0042) are

OPERATIONALLY DEPRECATED: no longer written, exposed, or read as Learning Style authority anywhere in the product (Student create/edit, Student profile, or Talent Student context). Existing stored values are preserved completely untouched - no backfill, no conversion, no clearing - because the M14 milestone could not safely verify whether any production row is actually non-null before deciding this. Physical column removal is a separately gated, later cleanup contingent on that verification.

1 Aggregate Learning Style distribution (Branch/Organization) is computed

over this single categorical field for all eight values plus an "Unassigned" bucket, reusing the existing privacy/suppression contract this ADR already required (`student_learning_style_analytics.py`, extended, not parallel-built). The denominator is every authorized Student in the selected scope, including Unassigned Students.

1 The M4/M10 Talent Evaluation Progress "Learning Style" Branch-comparison

metric, which averaged the four now-deprecated percentage columns, is REMOVED as of M14 (see ADR 0044's own record for that surface) rather than silently continuing to report a semantically wrong number.

Governance boundary (unchanged)

This amendment does not authorize multi-select, Learning Style influencing any Talent scoring/eligibility/identification computation, a new or weakened privacy rule, or any Branch-level override of the Student-domain ownership model - the original Governance boundary section above continues to apply in full, now scoped to eight values instead of four.

Acceptance C Amendment (2026-09-24): Authorized Aggregate, No Talent Small-Cell Suppression

Added 2026-09-24 by explicit owner decision after Deployment Acceptance Correction C. The text above is preserved unmodified; this section records what supersedes it for the Learning Style aggregate distribution.

Observed defect

In deployed Results & Analytics and the Students panel every Learning Style category rendered "Unavailable" although authorized Students already had Learning Styles assigned. Root cause (reproduced with a 10-Student cohort and the release-1 minimum cohort of 5): the distribution was passed through the Talent P3 primary and complementary suppression pipeline, which suppresses every per-category cell smaller than the minimum cohort; and, independently, the Results & Analytics frontend read a `buckets` key while the Learning Style route returned `levels`, so even a visible distribution rendered no bars.

Decision

1 Learning Style is Student-domain categorical profile data (one of eight

values, or Unassigned). There is **no per-Student Learning Style percentage**; the four `learning_style_*_percentage` columns stay operationally deprecated and excluded from current Learning Style analytics and normal product presentation (legacy compatibility paths may still read/write the stored values for preservation, never as analytics authority). The aggregate percentage exists only as a population statistic.

1 The aggregate distribution is **authorized Student-domain profile

aggregation, not sensitive Talent scoring/classification output**, and is therefore** not subject to Talent small-cell (P3) or complementary suppression**. A valid category is never "Unavailable"/"Suppressed" for an authorized actor; a category with zero Students is `0` / `0%`.

1 Denominator: every authorized Student in the selected population **including

Unassigned** (not only assigned, Talent-assessed, or active Program Students). Nine categories are always returned in stable order: Visual, Auditory, Read/Write, Kinesthetic, Verbal, Non-verbal, Quantitative, Spatial, Unassigned, each with key, label, count and percentage, plus `total_population`. An empty authorized population returns state `empty` (percentages `null`), never a misleading `0%`.

1 Boundaries that **remain**: SchoolGroup, Branch, Grade (and Section where the

existing contract supports it) scope, Student visibility, tenant isolation and the existing `students.view` permission. `resolve_population` is unchanged. Only aggregate counts/percentages are returned, never Student identity.

1 **This exception does not weaken any other Talent privacy class.** P1-P7

suppression, complementary suppression and the fail-closed privacy provider remain fully active for Classification, Talented, competency/rubric, result, organization and every other Talent metric; `talent_analytics_privacy.py` and its policy thresholds are unmodified.

1 No schema change and no migration; `tis.db` is not touched.

Implementation record

`student_learning_style_analytics.build_distribution(students)` no longer takes a privacy policy and no longer imports `talent_analytics_privacy`; `/api/students/analytics/learning-style-distribution`, `/api/talent/results-analytics/academic-years/{id}/learning-style` and the Students page panel no longer depend on `resolve_privacy_policy_provider` (the Results route no longer returns `privacy_policy_version`). The Results & Analytics frontend renders the backend `levels` (count, percentage, bar drawn from the backend percentage, zero rows kept, Unassigned neutral) and never computes a percentage.

Draft Talent Program Hard Delete (Scoped Exception)

docs/adr/0032-draft-talent-program-hard-delete-exception.md

ADR 0032: Draft Talent Program Hard Delete (Scoped Exception)

Context

`docs/PROJECT_STATE.md` ("Talent Program Setup Wizard Owner Correction" section) and `.kms-impact.yml` both record, as of the same day this ADR is written, an explicit architectural decision: "Programs remain governed by activate/retire rather than hard delete, and terminal assessments and historical framework versions remain immutable." That statement is accurate and remains the governing rule for every Program that has entered the Talent lifecycle in any meaningful way.

A separate corrective-reconstruction task subsequently asked for a hard `DELETE` endpoint for Programs. A prior investigation pass correctly identified that an unscoped reading of that request would contradict the recorded activate/retire invariant, and declined to implement it without direct confirmation, since no agent-relayed claim of prior approval is sufficient authorization to reverse a same-day documented governance decision. The Owner has now directly confirmed, in this conversation, a narrower request: a Program may be hard-deleted only while it is still in `draft` status and has zero rows in every table a real Program actually has a direct or child relationship to (at minimum, re-verified against `models.py` at implementation time: `TalentProgramFrameworkVersion`, `TalentProgramAcademicYearConfiguration`, `TalentCompetency`, `TalentAssessmentCycle`, `TalentEducatorInput` - the implementer must re-confirm this list is exhaustive against the actual current schema, not assume it).

A Program in this exact state has not yet entered the part of the Talent lifecycle the activate/retire invariant exists to protect - no framework history, no assessment history, no evaluation cycle, no educator input has ever been attached to it. It is, in effect, still an in-progress creation draft rather than a governed Program record.

Decision

Scoped exception, not a reversal

Talent Programs remain governed by activate/retire, not hard delete, for every Program that is not in this exact zero-child draft state. This ADR does not change that rule for Configured, Active, Retired, or otherwise historical Programs. Terminal assessments and historical framework versions remain immutable, unchanged by this ADR.

The exception, exactly

A Program may be hard-deleted via `DELETE /api/talent/programs/{id}` if and only if:

1 `program.status == 'draft'`, AND

1 zero rows exist in every table with a direct or child relationship to

that Program (re-verified against the actual current schema at implementation time, not assumed from this list).

Any other status, or a draft Program with any child row in any such table, must be rejected with a clear error - never a silent no-op, never a partial delete.

Authorization and audit

This action requires the new `talent_programs.delete` permission (grantable through the existing role-permission system, no hard-coded role names) plus the same existing organization-scope authorization check already used by other Program mutation routes. The deletion must be recorded through the existing `TalentConfigurationAudit` mechanism, the same audit path already used elsewhere in `talent_program_service.py` - no new audit mechanism is introduced.

UI exposure

The "Delete Program" action must be exposed only when the backend already says it is allowed (a real capability/action the API returns for that Program), never a client-side guess based on status alone.

Status

ACCEPTED as of 2026-09-10, per direct Owner instruction in this conversation, confirming a narrower scope than the reconstruction task's original unscoped hard-delete request. This ADR exists so that implementation has a durable, citable authorization artifact - consistent with the same pattern established in ADR 0031 - rather than relying on an unverifiable in-task claim that this was "already approved" elsewhere. `docs/PROJECT_STATE.md`'s and `.kms-impact.yml`'s activate/retire statements remain accurate for every Program outside this narrow, explicitly-bounded exception and are not being rewritten.

ADR 0033: Additive Roster Synchronization While A Talent Cycle Is Open

docs/adr/0033-open-talent-cycle-additive-roster-synchronization.md

Context

The approved M4 contract freezes the complete eligible Student population when an Assessment Cycle opens. Open and Closed population membership is immutable; post-Open mutation and late-entry exceptions are explicitly out of scope.

The Owner now requires a Student placed after opening into a Program-eligible Academic Year, Branch, Grade, and Section to become assessable immediately while that Cycle remains Open. The existing frozen-member model must remain the source of assessment eligibility and historical context.

Decision

Permit an organization-governed, additive-only roster synchronization for an Open Cycle. Synchronization:

1 lock the Open Cycle and use an explicit synchronization timestamp;

2 derive eligibility through canonical effective-dated Academic Placement for

the Cycle's exact School Group, Academic Year, and enabled Program grades;

1 insert only missing `(cycle_id, student_id)` population members, preserving

each added member's exact Placement, Branch, Grade, Section, PlanningSection, and synchronization-effective timestamp;

1 update population count and fingerprint under the same transaction and

expected-revision protection;

1 append an operational audit containing the timestamp and added member IDs;

2 never remove or rewrite a population member, assessment, result, Review

Candidate, Official Identification, or historical snapshot; and

1 reject synchronization unless the Cycle is Open. Closed Cycles remain final.

The operation would reuse `talent_assessment_cycles.govern`, organization/global scope, existing Branch-filtered population reads, and the existing population member foreign keys. Assessment creation would continue to require a persisted population member; it would not query live Placement directly.

Resolved Conditions

1 Synchronization runs automatically, in the same transaction, after an

authorized user confirms and saves a qualifying Student Placement.

1 Opening an existing Open assessment invokes the same organization-governed,

expected-revision operation before reading its members. This additively reconciles Students who were already eligible before ADR 0033 was deployed.

1 Students who later become ineligible remain members; removal is prohibited.

1 Each affected Cycle is locked, revisioned, re-counted, re-fingerprinted, and

audited before the placement transaction commits. A concurrent Close wins or causes the synchronizer to skip the now-Closed Cycle.

1 Analytics reads the current stored count/fingerprint for an Open Cycle. The

fingerprint may legitimately change after an audited additive synchronization; Closed Cycle integrity remains final.

Consequences

This changes the M4 population invariant for Open Cycles only. No schema migration is required because the existing member row stores per-member effective and frozen timestamps. Closed Cycle membership remains immutable.

Superseded

Superseded on 2026-09-11 by ADR 0035. Additive roster synchronization may remain as backward-compatible internal machinery, but it is no longer the eligibility authority or a required user workflow for starting Talent Student Assessments. Current effective Academic Placement is the assessment-start eligibility authority.

Talent Student Assessment Delete (Zero-Evidence Exception)

docs/adr/0034-assessment-delete-zero-evidence-exception.md

ADR 0034: Talent Student Assessment Delete (Zero-Evidence Exception)

Context

A prior action-semantics audit this session confirmed no whole-`TalentStudentAssessment` delete capability exists anywhere in the codebase - only `DELETE .../competency-results/{id}` to clear a single in-progress competency result (`talent_assessments.manage`, in-progress only). This means an accidentally started Assessment with zero recorded competency results, zero Educator Input, zero Review Candidate membership, and zero Official Identification decision can never be removed - a genuine product gap, not a defect, since no governed decision previously authorized a delete path for this entity. The audit correctly declined to build one without explicit authorization, consistent with this repository's established discipline (ADR 0031, ADR 0032, ADR 0033) of never adding a new destructive capability on inference alone.

The Owner has now directly confirmed, in this conversation, that Administrators need a Delete Assessment action, but strictly bounded to the case where the Assessment is still safely disposable - i.e. has accumulated no governed evidence or history.

Decision

The exception, exactly

`DELETE /api/talent/assessments/{assessment_id}` (or the equivalent service function) is permitted if and only if the `TalentStudentAssessment` row has:

1 zero `TalentStudentCompetencyResult` rows referencing it

(`fk_talent_competency_results_assessment_scope`);

1 no linked Educator Input record;

1 no linked Review Candidate record;

1 no linked Official Identification decision;

1 no other protected dependent evidence/history discovered by re-inspecting

the actual current schema at implementation time (the implementer must re-verify this list against `models.py`, not assume it is exhaustive from this ADR alone, mirroring the same re-verification discipline ADR 0032 required for the Program delete blocker-table list).

Any Assessment with any such row must be rejected outright - never a silent no-op, never a partial delete, never a cascade-delete of the dependent evidence itself. Once any governed evidence/history exists, the Assessment becomes permanent history exactly like every other terminal Talent record in this system (completed assessments, historical framework versions, Official Identification decisions).

Authorization and audit

This action requires an Administrator-scoped permission (the implementer determines the exact permission key, following the existing `(key, description)` additive tuple pattern in `permission_registry.py` - likely `talent_assessments.delete`, mirroring the `talent_programs.delete` / `talent_evaluation_plans.delete_period` naming convention already established by ADR 0032/related work) plus the existing organization/tenant scope check already used by other Talent mutation routes. The backend must re-check every dependency server-side at delete time, not trust a client-supplied "this is safe" claim. If the existing `TalentAssessmentAudit` architecture (its `resource_type` CHECK constraint already includes `'student_assessment'`) supports recording a deletion event, the implementer should append one; if the existing audit action vocabulary does not yet have a delete-style action for this resource type, the implementer should extend it additively (widening a CHECK constraint, the same additive-only pattern already used twice for this exact table per its own migration history) rather than skip audit recording.

Explicit non-goals

This ADR does not authorize deleting a `TalentStudentAssessment` with any evidence attached, under any role, for any reason. It does not authorize deleting `TalentAssessmentCyclePopulationMember` rows, Student placement history, or Cycle population records as a side effect of an Assessment delete - those remain governed entirely by their own existing rules (ADR 0033 for Cycle/roster) and must be structurally untouched by this action. It does not authorize any Student-level or Program-level delete capability beyond what ADR 0032 already governs.

Status

ACCEPTED as of 2026-09-11, per direct Owner instruction in this conversation. Implementation (permission key, backend route/service function with server-side dependency re-check, audit recording, and UI wiring following the same backend-capability-gated pattern already used for Program/ Competency/Rubric-Level/Period delete) is tracked separately and is not itself part of this governance record; this ADR exists so that implementation has a durable, citable authorization artifact rather than relying on an unverifiable in-task claim of approval.

ADR 0035: Talent Assessment Eligibility Uses Live Academic Placement

docs/adr/0035-live-academic-placement-assessment-eligibility.md

Context

The Talent & Potential implementation introduced an operational sequence in which an Evaluation had to move through Draft/Open lifecycle states, freeze a population at a configured timestamp, synchronize/reconcile that population, and require a persisted population member before a teacher could start an Assessment.

The Product Owner has explicitly rejected that workflow complexity. It was not an Owner requirement. The intended product is simpler: Students are enrolled through canonical Academic Placement; Programs define eligible Grades and an assessment tool/framework; Evaluation Periods provide assessment context; and teachers assess the Students who are currently enrolled and eligible.

Historical integrity is still required, but it must be captured when an Assessment is created rather than used as a pre-assessment operational gate.

Decision

Talent Student Assessment eligibility is determined from canonical current Academic Placement at the time the assessment is started.

A Student is assessable when:

1 the Student belongs to the same School Group;

2 the Student has an effective Academic Placement in the selected Academic

Year at the assessment-start time;

1 the Placement Grade is included in the Program's enabled Academic Year

configuration;

1 the Evaluation references a usable assessment framework/tool with at least

one competency and one rubric level; and

1 the actor holds the existing assessment permission and authorized Branch

scope.

No additional Program/Framework activation, Draft/Open Cycle, frozen-roster/synchronization/reconciliation state is a prerequisite to start an Assessment. The exact framework already referenced by the Evaluation is the assessment authority; its lifecycle label is not a second eligibility gate.

Evaluation Periods such as Term 1, Term 2, or Final are assessment context. They do not require a separate "Open Evaluation" action before eligible Students can be assessed.

For schema compatibility, the existing Talent Assessment Cycle and Cycle Population Member records may remain as internal persistence/provenance structures. When an Assessment is started, the implementation may create or reuse the internal Cycle context and persist a Population Member snapshot of the Student's current Placement. That snapshot preserves the exact Branch, Grade, Section, PlanningSection, Program, Academic Year, framework version, and

effective timestamp used for the Assessment. It is historical provenance, not an eligibility gate.

Existing completed Assessment evidence remains immutable history.

Supersession

ADR 0033 is superseded as an operational eligibility model. Its additive-roster synchronization machinery may remain temporarily for backward compatibility, but it is no longer required for Student Assessment eligibility and must not be presented as a normal user workflow.

The earlier M4 frozen-population requirement is likewise superseded only where it acted as a prerequisite for starting or editing Talent Student Assessments. Historical snapshots and existing persisted records remain valid.

Consequences

1 Student Assessments lists current eligible enrolled Students directly from

Academic Placement.

1 A newly enrolled eligible Student appears automatically without roster

reconciliation.

1 "Open Evaluation" is removed as a prerequisite for assessment.

1 Starting an Assessment captures historical placement/framework context at

that moment.

1 No schema migration is required for this correction.

1 Tenant isolation, Branch authorization, assessment permissions, assessable

framework content, evidence immutability, and auditability remain enforced.

1 No new workflow or lifecycle gate may be added to this module without an

explicit Owner requirement or an unavoidable security/data-integrity need.

Talent Rubric Re-evaluation Attempts

docs/adr/0036-talent-rubric-reassessment-attempts.md

ADR 0036: Talent Rubric Re-evaluation Attempts

Context

The Product Owner simplified the Talent & Potential authoring flow to:

Program + eligible Grades -> Grade-first rubric authoring -> Student Assessment.

The Owner's governing visual example further fixes rubric ownership as:

`Grade -> Competency -> Competency-owned Rubric -> ordered Levels`.

A Framework-wide shared rubric remains readable only as backward-compatible legacy configuration. New Grade-first authoring creates one rubric per exact Framework Competency.

A rubric can change after Students have already completed an Evaluation. Reopening or mutating a completed Assessment against a changed Framework would reinterpret historical evidence and violate the existing exact-Framework provenance contract. At the same time, leaving the completed Assessment as the only current result would hide the fact that the Student has not yet been evaluated against the new rubric.

Same-ID semantic-edit compatibility

The legacy compatibility rule is not limited to changed rubric/level identifiers. Before assessed-Framework immutability was consistently enforced, Student-facing rubric semantics such as labels, descriptions, ordering, descriptors, KPI/policy configuration, or other governed Framework content could be edited in place while preserving stable row IDs. Therefore a completed current Assessment is also **Re-evaluation required** when Talent configuration audit evidence shows a semantic Framework/rubric mutation on that exact Framework after the Assessment's `completed_at`, provided the current Grade-applicable competency-owned rubric is complete and assessable.

Lifecycle-only, branding, annual-plan, and unrelated Program changes do not trigger this rule. The replacement attempt remains append-only/current and starts with zero competency results; the prior completed attempt remains immutable historical evidence.

Legacy same-Version compatibility repair

The forward rule remains unchanged: once a Framework has Student Assessment evidence, Student-facing rubric semantics are immutable and any later material change belongs in a newer Framework Version.

A bounded compatibility exception exists only for data created before that immutability guard was consistently enforced. If a current completed Assessment remains bound to a Framework Version whose persisted competency-result rubric/level bindings no longer match that same Framework's now-complete competency-owned rubric structure for the Student's recorded Grade, the Assessment is treated as **Re-evaluation required**. An untouched legacy shared-rubric Framework, or a partially configured competency-owned rubric, never triggers this repair.

The repair is operational reset, not evidence deletion: the prior completed attempt becomes non-current and remains immutable; the replacement attempt is current, in progress, contains zero competency results, is linked by `reassessment_of_assessment_id`, and keeps the original visible Evaluation/Term through `evaluation_context_cycle_id`. The same Framework Version may be the replacement target only for this historical compatibility case because the current canonical rubric branch already exists there. No future in-place rubric mutation is authorized by this exception.

Administrator recovery reset

Automatic `Re-evaluation required` remains the preferred path whenever a materially changed rubric can be detected deterministically. A separate, explicit operational recovery path is also accepted for a current completed Assessment when an authorized administrator needs the Student to be assessed again even though that automatic condition is not present.

The permission is `talent_assessments.reset_for_reassessment`, assigned by default to Administrator through the standard all-permissions role rule and not included in Editor/User defaults.

This action is **not deletion** and does not weaken completed-evidence immutability. It:

1. requires the target Assessment to be Completed and current;
2. marks only that Assessment `is_current = false`;
3. records an audit action `reset_for_reassessment`;
4. preserves every competency result, Review Candidate, Official

Identification, Educator Input, placement snapshot, Cycle/Framework context, and audit row;

1. leaves the original visible Evaluation available so the normal Start

Assessment flow can create a fresh current attempt with zero results;

1. preserves the existing physical `UNIQUE(cycle_id, student_id)` contract by

creating a private derived Cycle for that fresh attempt when the prior historical Assessment already occupies the visible Evaluation Cycle. The replacement keeps `evaluation_context_cycle_id` pointing at the original visible Evaluation, so users do not see a second Evaluation/Term.

ADR 0034 remains unchanged: physical Assessment deletion is still allowed only for zero-evidence Assessments.

Decision

Historical Assessment immutability remains

A completed Assessment is never reopened, rewritten onto another Framework, or silently reinterpreted. Its competency results, Review Candidate records, Official Identification decisions, Educator Input, frozen placement context, and exact Framework Version remain historical evidence.

Re-evaluation is a new Assessment attempt

When a newer saved Framework Version for the same Program changes the Student-facing assessable structure for the Student's recorded Grade, a completed current Assessment is reported as **Re-evaluation required**.

The comparison deliberately ignores Framework version number, title, and supersession metadata. A no-op clone does not require re-evaluation. Changes to applicable Competencies, competency-owned Rubrics, ordered Levels, level descriptions/achievement wording, or other assessable rubric semantics do.

An authorized user starts re-evaluation through the Assessment API. The system:

1. marks the prior current attempt non-current inside the same transaction;
2. creates a new physical Assessment Cycle using the newer Framework Version;
3. starts a new Student Assessment using the Student's current effective

Academic Placement under the existing ADR 0035 eligibility rules;

1 links the new Assessment to the prior Assessment with

`reassessment_of_assessment_id`;

1 preserves `evaluation_context_cycle_id` as the original visible

Evaluation/Term;

1 keeps the replacement Assessment `is_current = true`;

2 records the supersession/reassessment linkage in the existing Assessment

audit stream.

The replacement physical Cycle preserves exact Framework provenance but is not a second user-visible Evaluation. Student Assessment lists and current-result analytics project the replacement back to the original Evaluation context.

Frameworks with Assessment history are semantically immutable

A Draft/Active/Retired label is not enough to determine whether semantic edits are safe. Once any Student Assessment references a Framework Version, that Framework's semantic configuration may no longer be edited in place. A new Framework Version must be created/cloned before changing competencies, rubric levels, descriptors, KPI configuration, or candidate-policy configuration.

This closes the historical-corruption gap created by allowing operational Assessment start against a configured Framework without using lifecycle labels as a user-facing assessment gate.

Analytics current-attempt semantics

Current-result analytics use only `TalentStudentAssessment.is_current = true` and resolve the attempt through `evaluation_context_cycle_id` (falling back to `cycle_id` for pre-migration rows). The physical reassessment Cycle/population member is excluded as a second eligible row; the replacement result is projected to the original Evaluation's Student/Branch/Grade/Section context. Historical views may still read superseded Assessments explicitly.

Permissions and tenant isolation

Re-evaluation reuses `talent_assessments.manage`. Read access remains `talent_assessments.view`. Rubric-authoring deletion is independently governed: `talent_programs.delete_competency` controls removal of a Competency branch from an editable Framework and `talent_programs.delete_rubric_level` controls true Rubric Level removal. These do not imply whole-Program delete authority, and `talent_programs.manage` alone does not imply either delete capability. Every lookup, new Cycle, new Assessment, Framework resolution, and Student placement check remains SchoolGroup-scoped and subject to the existing Branch-authorization checks.

Persistence

Migration `20260911_003_talent_assessment_reassessment_attempts` adds:

1 `talent_student_assessments.is_current BOOLEAN NOT NULL DEFAULT true`

1 `talent_student_assessments.reassessment_of_assessment_id INTEGER NULL`

1 `talent_student_assessments.evaluation_context_cycle_id INTEGER NULL`,

backfilled from existing `cycle_id`;

1 current-attempt and Evaluation-context lookup indexes.

Migration `20260911_004_talent_competency_specific_rubrics` adds nullable `talent_rubrics.framework_competency_id`, removes the old one-rubric-per- Framework uniqueness rule, and enforces one rubric per exact `Framework Version + Framework Competency`. Existing NULL-owned rubrics remain legacy shared rubrics only for historical/read-comparison compatibility. Normal current Evaluation starts select only a complete competency-owned rubric structure for every applicable Competency; a legacy shared rubric never makes a new user-facing Assessment assessable.

The linkage is service-validated and intentionally additive so existing PostgreSQL/SQLite deployments do not require destructive table rewrites.

UX

Program creation is simplified to Program identity plus eligible Grades.

Programs expose a separate **Rubric** action. Rubric authoring is Grade-first: each eligible Grade is independently collapsible and contains its Competencies. Each Competency owns its own Rubric, and that Rubric owns its ordered Levels and level descriptions. This mirrors the Owner-provided reference structure rather than presenting one shared rubric scale for the whole Framework.

When an existing legacy shared rubric is explicitly edited for one Competency, the service copies the legacy scale and that Competency's descriptor content into a new competency-owned rubric. Merely cloning without a semantic edit does not trigger re-evaluation.

Student Assessments display **Re-evaluation required** when the current completed Assessment is superseded by a materially changed, assessable newer Framework. Starting re-evaluation opens the replacement attempt against the newer rubric while keeping the original visible Evaluation/Term. The prior attempt remains preserved evidence but is not listed as a separate History or Assessment Records table in the normal operational workspace.

Consequences

1 Historical evidence remains stable.

1 Rubric changes no longer silently invalidate or reinterpret completed work.

1 Re-evaluation is explicit and auditable.

1 Current analytics avoid double counting superseded attempts.

1 Framework editing after evidence exists requires a new version.

1 No destructive data migration or history rewrite is authorized.

Talent Overall Program Result

docs/adr/0037-talent-overall-program-result.md

ADR 0037: Talent Overall Program Result

Context

Talent Programs assess one Student across multiple competencies. The Product Owner requires each rubric level to show its ordered numeric rank beside its name and requires one deterministic overall result for the Program Assessment.

The canonical result must remain meaningful to educators using the rubric. A Mental Math Program using five ordered levels should therefore report a result such as `4.4 / 5`, rather than replacing the rubric meaning with an invented 0-100 score.

Different Talent Programs remain separate assessment domains. Mental Math, Performing Arts, Reading Talent, or future Programs must never be combined into one universal Student Talent score.

Decision

Rubric level number

The canonical numeric rank of a rubric level is its governed ordered position within the competency-owned rubric:

1 first level = 1;

1 second level = 2;

1 and so on.

The UI renders this number directly beside the level name, for example `1. Beginning`, `2. Approaching`, `3. Meets`.

The displayed rank is not a second manually-entered score and does not depend on the optional legacy/KPI `numeric_value` field.

One coherent Program scale

All Grade-applicable competency rubrics participating in one completed Student Assessment must use the same number of ordered levels.

For example, if Mental Math is a five-level Program scale, every applicable competency in that Assessment must use five ordered levels.

TIS must not directly average ranks from incompatible scales such as `4 / 5` and `3 / 3`. An Assessment with inconsistent applicable rubric lengths cannot be completed until the rubric structure is aligned.

Overall Program Result

Once every Grade-applicable competency has a valid saved result, TIS derives the **Overall Program Result** as the arithmetic mean of the selected ordered rubric ranks.

Example for a five-level Program:

`3, 4, 5, 5, 5, 3, 5, 5, 5 -> 4.4 / 5`.

The result is rounded to one decimal place using deterministic half-up rounding.

The calculation method identifier is:

`arithmetic_mean_rubric_rank`.

Normalized presentation percentage

TIS may derive a percentage for progress-bar length, color intensity, or other bounded visualization:

`average / scale_max * 100`.

This percentage is presentation/analytics normalization only. It is not the canonical educational result and must not replace the displayed rubric-scale average.

Persistence

No new result table or schema migration is introduced.

The Overall Program Result is a deterministic read projection from:

- the exact Student Assessment;
- its immutable Framework Version;
- the Grade-applicable Framework Competencies;
- the stored competency results;
- each competency's exact ordered rubric levels.

Completed historical Assessments retain stable meaning because their exact Framework Version and evidence remain immutable.

Existing optional Framework KPI configuration remains separate. It is not redefined or silently reused as the Overall Program Result.

Talent Review and Official Identification

Talent Review surfaces the rubric-scale Overall Program Result prominently.

The result supports educator review and may coexist with deterministic Review Candidate policy. It does not itself create an Official Identification decision.

Official Identification remains a separate authorized human decision. Neither a rubric average nor its color treatment is an automatic `talented/not talented` classification.

Multiple Programs

A Student may have separate Overall Program Results for multiple Talent Programs.

Each Program result remains separate. TIS may present them together in a Students Across Programs matrix, but it must not average or otherwise collapse those Program results into one universal Talent score.

Consequences

- Educators see the same 1-N scale they used during assessment.
- The Program result is explainable as an arithmetic mean of competency ranks.
- Inconsistent competency rubric lengths are blocked from completion rather than being silently normalized into a different educational meaning.
- A normalized percentage remains available only for visual presentation.

- Multi-Program Student results can be compared visually without being combined.
- Review Candidate and Official Identification remain separate governed concepts.
- Historical assessment evidence remains immutable.
- No schema migration is required for the result projection.

2026-09-23 Amendment: Governed 1.00-5.00 Classification Authority (M17)

This amendment appends new governed decisions. It does not rewrite or weaken any decision above: the raw Overall Program Result on the Program's own 1-N rubric scale, `normalized_percent`, and the arithmetic-mean calculation method all remain exactly as decided.

Product direction

M17 introduces automatic Assessment classification. Once a Student Assessment is Completed, TIS automatically classifies the Overall Program Result into one of five owner-approved, fixed bands and derives a `Talented` boolean. These fixed bands are expressed on a governed **1.00-5.00 classification scale** that is deliberately independent of any one Program's own rubric level count:

- 1.00-1.99 Needs Improvement
- 2.00-2.99 Developing
- 3.00-3.74 Meets Expectations
- 3.75-4.49 Advanced
- 4.50-5.00 Exceptional

Only Exceptional means Talented. These bands are never expressed or recomputed as a percentage, and never reuse `normalized_percent` (which remains presentation/analytics only, unchanged by this amendment).

Architecture problem this amendment resolves

The classification bands above are literally valid only for a Program whose rubric scale is exactly five ordered levels. This ADR's own worked example ("a five-level Program") was always one example, not a universal constant - `scale_max` is a variable throughout this ADR's "Normalized presentation percentage" section. Real configured Programs in this repository do not universally use five levels: the realistic local seed dataset (`talent_local_test_data.py`, built end-to-end through the real `talent_program_service`/`talent_student_assessment_service` contracts) configures a 4-level Program, a 3-level Program, and a second 4-level Program side by side. Mandating a system-wide five-level rubric for every Talent Program (rejected Option A) would therefore be a breaking change to existing real Program configurations, not a small formalization, and would force every Program owner to redesign already-approved rubric structures purely to satisfy the classification band's five-way shape.

Decision: deterministic 1..N -> 1.00-5.00 projection (Option B)

TIS preserves each Program's own configured rubric-level count for the educational raw result exactly as decided above (the arithmetic-mean rubric rank on the Program's own 1..N scale is unchanged), and adds one additional, separately governed **deterministic linear projection** from that raw 1..N average onto the fixed 1.00-5.00 classification scale:

```
classification_score = 1 + (average - scale_min) * 4 / (scale_max - scale_min)
```

This reuses the exact linear-rescale technique this ADR already approves for `normalized_percent` (`average / scale_max * 100`), but targets the fixed classification range `[1.00, 5.00]` instead of a percentage. It is computed with exact `Decimal` arithmetic directly from the integer `average_tenths` value already produced by the Overall

Program Result projection (never a second float division of the already-rounded `average`), and the projected score is rounded deterministically half-up to two decimal places. A five-level Program's projection is the identity map (`classification_score == average`), so the M17 product direction's worked examples remain literally true for a five-level Program while now also being correct for every other configured scale.

A Program whose rubric scale cannot be projected deterministically - a degenerate single-level rubric (`scale_max <= scale_min`), or the existing "inconsistent rubric scale" state this ADR already blocks at completion - is not compatible with automatic classification. TIS fails closed: no classification, no Talented state, `available: false` with an explicit `reason`. This is a new possible outcome, not a new possible error path for existing consumers of the raw Overall Program Result, which is unchanged.

Implementation: `talent_classification_service.py` (`project_to_ classification_scale`, `classify_score`, `is_talented`, `assessment_classification`). `overall_program_result` additively exposes `average_tenths` (the exact integer tenths already computed internally) so the projection never re-derives precision from a float.

Persistence

No new result table or schema migration is introduced. Classification, like the raw Overall Program Result itself, is a deterministic read projection computed only for a Completed Assessment, from the same immutable inputs this ADR already establishes as stable (the exact Student Assessment, its immutable Framework Version, the Grade-applicable Framework Competencies, the stored competency results, and each competency's exact ordered rubric levels). A Completed Assessment's classification is therefore exactly as immutable as its Overall Program Result already is.

Talent Review and Official Identification (superseded in part)

This ADR's original "Talent Review and Official Identification" section remains true as a description of the pre-M17 workflow and of the still- preserved legacy/history surfaces. As of this amendment, for the *current normal* workflow: automatic classification is a direct backend consequence of a Completed Assessment. `Talented` (Exceptional classification) no longer requires a Review Candidate to be materialized, reviewed, or an Official Identification decision to be recorded. Existing `TalentReview Candidate` and `TalentOfficialIdentification` rows, including any recorded before this amendment, are preserved unchanged as legacy/history evidence and continue to be evaluated/recordable through their existing services for historical/audit purposes, but they no longer govern current `Talented` state and are never silently rewritten to match a later automatic classification.

Non-negotiable boundaries reaffirmed

1 No percentage-band conversion of the classification bands, ever.

1 AI has no classification or identification authority.

1 The frontend only displays the backend-computed `classification`/

`is_talented` fields; it never derives or spoofs a band.

1 SchoolGroup/tenant/Branch/privacy/audit boundaries are unchanged.

2026-09-23 Amendment: Current-Result Authority For Multi-Assessment Students; Learner Profile/Student Drill Integration (M18a)

Added 2026-09-23. Appends only; the M17 amendment above is unchanged.

Which Assessment is "the applicable current result"? When a Student has more than one `TalentStudentAssessment` for a Program (across reassessment attempts per ADR 0036), the one that governs current M17 classification is the row where `status == 'completed'` AND `is_current == True`. This was resolved from existing evidence, not invented: `models .TalentStudentAssessment.is_current` and its

reassessment-flow semantics already existed before M18a (ADR 0036), and `talent_analytics_service._assessment_ids_subquery` plus the B9 Student Drill (`talent_org_student_drill.py`) already used exactly this predicate to select the Assessment whose Overall Program Result counts for the current Program+Cycle scope. M18a's own classification integrations use the identical predicate.

Learner Profile / Student Drill integration. `talent_learner_profile_service.build_learner_profile`'s per-Assessment item and `talent_org_student_drill.py`'s per-context row now additively carry the same `classification`/`classification_score`/`is_talented` fields already on the Talent Assessment API (`routers/talent_assessments.py`, since M17), computed by the same single authority (`talent_classification_service.assessment_classification`) - never duplicated, never derived from `TalentReviewCandidate`/`TalentOfficialIdentification`. This closes the "Current downstream Talented state must use M17 automatic classification" requirement for per-Student/per-row surfaces. A new org/Branch AGGREGATE "current Talented count" metric (privacy-safe Cell/Group infrastructure against a new classification grain) was evaluated and is explicitly deferred to M18b - see ADR 0044's M18a amendment and this ADR's own frozen `MetricCode` discipline; it is new analytics infrastructure, not a cleanup-scope change.

Talent Evaluation-First Assessment and Review Workspace

docs/adr/0038-talent-evaluation-first-assessment-review.md

ADR 0038: Talent Evaluation-First Assessment and Review Workspace

Context

The previous Student Assessments surface rendered one card per Program Cycle, which could show repeated labels such as `Term 1`, `Term 2`, `Term 1`. This obscured the fact that one Evaluation Period may contain several Talent Programs.

Talent Review also used persisted Review Candidate rows as its primary Student list. A completed Student Assessment without a qualifying/materialized Review Candidate therefore disappeared from the Review workspace even though it contained valid assessment evidence and an Overall Program Result.

The Product Owner requires:

1 Program setup to expose eligible Grades and configured user-defined Evaluation

Periods for the selected Academic Year;

1 Student Assessments to organize work as Evaluation Period -> Program -> Student;

1 every currently eligible Student to be visible for the selected Program and

Evaluation;

1 every current completed Assessment to be available in Talent Review;

1 Review Candidate and Official Identification to remain separate statuses.

Decision

Program configuration

The existing Annual Evaluation Plan / Planned Evaluation Period architecture remains the canonical scheduling authority.

Program setup surfaces that configuration alongside eligible Grades. TIS does not create a duplicate scheduling table or hard-code Term names.

Users may configure labels such as:

1 Term 1;

1 Term 3;

1 Final;

1 Baseline;

1 Audition;

1 or other organization-defined Evaluation labels.

Evaluation Period grouping correction

The operational Student Assessment hierarchy is **Evaluation Period -> unique Programs -> Students**. The display grouping key is the normalized user-facing Evaluation label, not a physical Cycle id or a Program-specific Planned Period id. Configured sequence orders the resulting label groups when available. Therefore a repeated label (for example `Term 1`) across Mental Math and Qaida Nourania renders as one `Term 1` section containing each Program once, even when legacy Cycle metadata lacks matching sequence values.

Multiple physical Cycles for the same Program/Period must never duplicate that Program card. Evaluation Plans are also a display source: a configured Program/Period remains visible before its first internal Cycle has been materialized. An authorized user selects that Program **directly inside Student Assessments**; the UI materializes and links the internal Cycle through the existing permission-, revision-, and tenant-scoped Cycle/Period APIs, then remains in Student Assessments and opens the eligible Student roster. It must not bounce the user to a separate Evaluation Plan page merely because the internal Cycle does not yet exist. Cycle remains internal provenance/execution context, not a second user-facing Program occurrence.

Student Assessment information architecture

The primary Student Assessment chooser is:

Evaluation Period -> Programs -> Students.

Cycles linked to Planned Evaluation Periods use the Period label and sequence as the user-facing grouping/order authority. Legacy unlinked Cycles fall back to their existing title/order behavior.

A repeated Evaluation label is shown once, with participating Programs nested inside it.

Selected Evaluation interaction

When the user selects an Evaluation Period/Program, that section remains visibly selected while its Student roster is shown. Start Assessment carries the selected Evaluation context and Program into the Student assessment route; the opened workspace reads the exact saved Framework/Grade competencies and assessment criteria. A Start/Continue/View action must never appear inert or silently lose the selected Evaluation context.

Eligible Student completeness

For a selected Program/Evaluation, the operational list is driven by the existing live Academic Placement eligibility rule from ADR 0035, constrained by:

1 SchoolGroup tenant;

1 Academic Year;

1 Program eligible Grades;

1 authorized Branch/scope.

Each eligible Student is shown with an operational state such as:

1 Not started;

1 In progress;

1 Completed;

1 Re-evaluation required.

Historical/superseded Assessment attempts remain preserved as backend evidence but are not rendered as a separate Assessment Records/History table in the normal Student Assessment workspace. The current eligible Student row and its current status/action are the sole operational surface.

Re-evaluation

ADR 0036 remains authoritative.

A materially changed newer Grade-applicable rubric causes a current completed Assessment to surface **Re-evaluation required**. The prior completed attempt remains immutable historical evidence; reassessment creates a new current attempt tied to the original visible Evaluation context.

Review filtering and presentation

Talent Review may be narrowed by authorized Branch, Grade, and Planning Section. Filter options come only from the actor's authorized organization/ Branch scope and operational Planning context; **All Branches** is shown only when more than one Branch is authorized.

The deterministic Review Candidate policy remains part of the domain and may still create review actions, but the normal Review table does not present "Meets criteria" / "No candidate" as a primary learner classification. Review status, Overall Program Result, and Official Identification remain separate visible facts.

Talent Review

Talent Review is an educator review workspace over **all current completed Student Assessments** in the selected authorized context.

A completed Assessment is not hidden merely because no Review Candidate row exists.

Talent Review displays, as separate facts:

1 Overall Program Result;

1 Review Candidate state, when a deterministic Candidate exists;

1 review status;

1 Official Identification state.

Completing an Assessment may deterministically evaluate the existing Framework Review Candidate policy. A no-policy or non-qualifying outcome does not remove the completed Assessment from Talent Review.

Official Identification

Official Identification remains a separately permissioned, permanent human decision. The presence of a completed Assessment or a high Overall Program Result does not itself identify a Student.

Analytics relationship

Results & Analytics may summarize completed/current assessment evidence using the existing governed analytics and privacy architecture.

Different Program results remain separate. Organization/Branch/Grade analytics must not manufacture one cross-Program Student Talent score.

Consequences

1 Repeated Term cards are replaced by an Evaluation-first operational hierarchy.

1 Program settings make Evaluation Period configuration discoverable without

duplicating its persistence model.

1 Missing eligible Students are treated as a roster/eligibility defect rather

than an expected consequence of frozen historical membership.

1 Talent Review becomes complete for current completed assessments.

1 Review Candidate and Official Identification remain distinct governed states.

1 Historical evidence and tenant/Branch authorization boundaries are preserved.

ADR 0039: Talent Competency -> KPI -> Level Simplification

docs/adr/0039-talent-competency-kpi-level-simplification.md

Context

A full operational diagnostic of Talent & Potential (this same day) found two independent defects: an eligibility check that silently blocks Start Assessment whenever a Student's Grade has no separately-authored competency, and a Program-readiness banner that shows "Ready" without verifying every eligible Grade actually has assessable content. Investigating those defects surfaced the underlying design: the current implementation treats a Program's competency set as something that must be authored per-Grade, and treats "Rubric" as an internal domain object presented only as "Assessment Criteria" to keep it distinct from the separate numeric `TalentKpiConfiguration` feature (`docs/AI_PROJECT_CONTEXT.md`, "Talent Rubric Vs Assessment Criteria Vs KPI Terminology").

The Product Owner has now directly, explicitly rejected both the per-Grade authoring requirement and the "Assessment Criteria" terminology, in favor of one deliberately minimal model.

Decision

Grade is context, not an eligibility gate

Amending ADR 0035 condition 3 - "the Placement Grade is included in the Program's enabled Academic Year configuration" is removed as an assessability requirement (the rest of ADR 0035's assessability list and its Supersession/Consequences sections are unchanged). Condition 4 (a usable assessment framework/tool with at least one competency and one rubric level) is unaffected by this amendment on its own terms, but the implementation must NOT additionally require that framework content to be scoped to the specific Student's Grade either. If Grade-specific content exists for the Student's Grade, it may be preferred/shown; if it does not, the Program's saved build is used instead of blocking. Grade remains valid as display context and as historical provenance captured at assessment-start time (per ADR 0035's existing snapshot mechanism) - it must never again cause `assessment_tool_unavailable`, `student_not_eligible`, or hide an otherwise-eligible Student from the Student Assessments list.

User-facing hierarchy: Competency -> KPI -> Level

The user-facing authoring and assessment hierarchy is exactly: Competency, then KPI (one assessment criterion per Competency, in the normal flow), then ordered Levels. "Assessment Criteria"/"Rubric" as a normal-user-facing label is retired in favor of "KPI" for this hierarchy. Internal schema/API names (`Rubric`, `TalentRubricLevel`, etc.) may remain unchanged where renaming them would carry unnecessary migration risk - this is a presentation-layer rename, not a schema rename.

Explicit terminology-collision requirement, not resolved by this ADR: `docs/AI_PROJECT_CONTEXT.md` records a separate, real `TalentKpiConfiguration` numeric-KPI feature with its own Add/Edit/Delete controls, deliberately kept distinct from rubric-based Assessment Criteria specifically to avoid this exact naming collision. This ADR authorizes relabeling the rubric-based Competency-criterion hierarchy as "KPI" in the primary Competency -> KPI -> Level flow. It does NOT authorize merging, removing, or silently renaming the separate `TalentKpiConfiguration` feature. The implementer must re-verify whether that separate feature still exists and is still exposed in the UI; if so, the implementer must flag the resulting two-different- things-both-called-"KPI" collision explicitly for a follow-up Owner naming decision (e.g., renaming the numeric feature to "Quantitative Score" or similar) rather than resolving it unilaterally.

Readiness: Competency + KPI + Level + Save = Ready

A Program's user-facing readiness is exactly: at least one Competency, with at least one KPI, with at least one Level, saved. No separate Finish Setup/Activate/Open step is required in the normal user flow. Whatever internal Framework-version activation/lifecycle bookkeeping the schema still requires (per ADR 0035, this remains valid as internal provenance) must happen transparently as part of Save, not as a distinct user-visible step. When readiness is not met, the UI must name the exact single missing requirement in plain language, never a generic Draft/Ready binary with no explanation.

Competency delete is a subtree operation

Deleting a Competency through the normal UI removes its KPI and Levels from the current/future build as one action with one confirmation, never a KPI-then-Levels-then-Competency manual sequence. Historical completed Assessment evidence referencing that Competency remains immutable and preserved via the existing version/history architecture (per ADR 0035); the delete removes the Competency only from the current/future assessment build, never from completed history. No Draft/version complexity is exposed to the user to perform this action.

Explicit non-goals

This ADR does not weaken tenant isolation, Branch authorization, Student existence checks, Academic Year context requirements, data integrity, or immutable historical evidence - those remain exactly as ADR 0035 already established. It does not authorize a schema migration; if the implementer finds one is genuinely unavoidable (e.g., to make a rubric/KPI name column nullable), it must be purely additive/backward-compatible and reported, not silently assumed. It does not retroactively change the framework/version bound to any already-completed Assessment. It does not resolve the KPI-naming-collision question above - that is an explicit, separate, flagged follow-up decision point, not something this ADR silently decides either way.

Owner correction (same day): Grade is not a Program-configuration gate either

A first implementation pass of this ADR only partly removed the Grade gate: Start Assessment stopped hard-blocking on a Grade mismatch (it fell back to the Program's full saved build), but three related surfaces still implicitly required the Student's Placement Grade to be a member of the Program's own `eligible_grade_levels` configuration - `_current_eligible_placement` filtered the Placement query by `grade_level IN eligible_grades`, the Student Assessments list (`GET .../eligible-students`) called `derive_eligible_population`, which applies the identical Grade filter, and the readiness banner still required `eligible_grade_levels` to be non-empty (`basicsComplete`) before showing Ready. The Owner has now directly, explicitly corrected this: none of the three may gate on Program Grade configuration at all. Restated precisely:

1 **Start Assessment**: a Student's current effective Placement in the

Evaluation/Cycle's own Academic Year is sufficient (see the further correction below removing `is_enabled` too) - the Student's Placement Grade is never checked against the Program's `eligible_grade_levels`. Grade remains captured on the resulting population member/Assessment as historical/context data only.

1 **Student Assessments list**: for the selected Academic Year and

authorized tenant/Branch scope, every Student with a valid current Academic Placement appears - never filtered by Program Grade configuration. This is served by a new function, `current_placements_for_assessment`, deliberately separate from `derive_eligible_population` (which remains unchanged and still Grade-filters for its own legacy/frozen-population callers - Cycle open/preview/ADR 0033 reconciliation - since those are historical population mechanics, not the normal Student Assessments list, and are out of this ADR's scope to rewrite).

1 **Readiness**: Program/Academic-Year Grade configuration

(`eligible_grade_levels`) may remain available for context/configuration but is never part of the Ready/Not-Ready gate. Readiness is exactly the assessment build itself: at least one Competency, with at least one KPI, with at least one Level, saved.

Further Owner correction (same day): Program Academic Year enablement is not a gate for the normal path either

`TalentProgramAcademicYearConfiguration.is_enabled` was still required by both `_current_placement_for_assessment` (Start Assessment) and `current_placements_for_assessment` (the Student Assessments list) - a Student could not appear or start unless a configuration row existed and was explicitly enabled for that Program/Academic Year. The Owner has now corrected this too: the Evaluation/Cycle already supplies the Academic Year, and a Student's valid current Placement in that exact Academic Year is sufficient - the Program's own annual-configuration enabled/disabled state (or its absence entirely) is never a gate on the normal Student Assessments roster or Start Assessment path. Kept, unchanged: School Group (tenant) isolation, Branch authorization, Academic Year match, a genuinely current Student Placement, the saved Competency+KPI+Level requirement (ADR 0039's own core decision, unaffected), and immutable historical evidence protection. `derive_eligible_population` (the legacy M4/frozen- population path) is unchanged and still requires an enabled configuration for its own callers - not rewritten here, same boundary as the Grade correction above.

Status

ACCEPTED as of 2026-09-12, per direct Owner instruction in this conversation (a detailed 20-section implementation brief). This ADR exists so implementation has a durable, citable authorization artifact for the two real policy changes it makes (Grade is no longer an eligibility gate; "Assessment Criteria" is relabeled "KPI" in the primary user-facing hierarchy) rather than relying on an unverifiable in-task claim of approval. Every other item in the Owner's brief (navigation, Ghars icon, UI density, selector placement) is UI/IA work that does not itself require new governance and is not restated here.

Owner amendment - 2026-09-12: Grade-aligned assessment criteria

The Product Owner supersedes one narrow part of this ADR: the prior rule that allowed a Student Assessment to fall back to another Grade/the Program-wide saved build when the Student's Grade had no aligned criteria. That fallback is no longer permitted.

Grade remains **not a roster eligibility gate**: a current effective Placement in the selected Academic Year remains visible in Student Assessments regardless of Program `eligible_grade_levels`. Separately, Grade is an **assessment-criteria alignment boundary**. Current assessment work may use only criteria aligned to the Student's Grade plus intentionally unscoped criteria; criteria explicitly assigned to another Grade must never be substituted.

If no saved assessable Competency -> KPI -> Level structure applies to that Grade, Start Assessment returns an actionable Grade-specific setup message without hiding the Student from the roster. Reloading a zero-result current In Progress Assessment may move only to a newer Framework that has assessable criteria applicable to the same Grade. Evidence-bearing and terminal Assessments remain immutable. Historical completed evidence created under earlier behavior stays readable on its recorded Framework.

This amendment supersedes only the earlier cross-Grade/full-build fallback statements. The separate corrections removing Program annual Grade configuration and `is_enabled` as roster eligibility gates remain in force.

Per-User Permission Override Precedence, Deny-over-Allow, and Platform-Actor Scope Resolution

docs/adr/0040-per-user-permission-override-precedence-and-scope.md

ADR 0040: Per-User Permission Override Precedence, Deny-over-Allow, and Platform-Actor Scope Resolution

Context

The per-user permission override layer (`models.UserPermissionOverride`, migration `20260913_001_user_permission_overrides`, `user_permission_service.py`, the Edit User page panel in `templates/edit_user.html`, and `POST /users/permissions/{user_pk}` in `routers/users.py`) sits above the existing role-resolved permission set (built-in role default -> global `RolePermission` override -> tenant `RolePermission` override). It was implemented and wired across several prior passes (see `docs/TIS_MASTER_CONTEXT.md`'s "Per-User Permission Overrides" section and `docs/CHANGE_HISTORY.md`) but three governing decisions had not yet been recorded in a dedicated ADR: the exact precedence chain, the Deny-over-Allow rule, and how the route resolves an override's `school_group_id` when the acting administrator is a platform-level actor (owner/developer) with no tenant SchoolGroup of their own. This ADR is that record; it does not change or reopen the already- implemented resolver/model/UI, except for the platform-actor scope fix described below.

Decision

1. Precedence chain

Effective permission for a specific user is resolved in exactly this order: built-in role default -> global (`school_group_id IS NULL`) `RolePermission` override -> tenant-scoped `RolePermission` override -> per-user `UserPermissionOverride` -> effective permission. Every consumer of "the current user's effective permissions" must resolve through `auth.get_allowed_permission_keys`, the sole integration point that folds role resolution (`role_permission_service`) and user overrides (`user_permission_service`) together. Role-only helpers named `_get_allowed_permission_keys` (in `main.py`, `routers/users.py`, `ui_shell.py`) compute only the abstract role-level set for reference summaries and are never a substitute for a specific user's effective permissions.

2. Deny-over-Allow (Allow-over-Deny is NOT approved)

A per-user override has exactly three states: Inherit (no row), Allow, Deny. `user_permission_service.apply_overrides_to_keys` applies Deny-over-Allow: a per-user Deny always removes a key from the role-resolved set. A per-user Allow only has effect when the key is already present in the role-resolved set -- it can never add a key the role currently denies (whether denied by default, by a global override, or by a tenant-scoped override). A stored Allow override on a role-denied key is preserved (not silently deleted) so that re-granting the key at the role level immediately reactivates it, but it is inert until then.

This is a deliberate, conservative default. No prior ADR or governance record ever approved letting an explicit per-user Allow grant access the resolved role currently denies -- there was no existing authorization to implement Allow-over-Deny, and building it without one would have been an unapproved privilege-escalation mechanism. Enabling true Allow-over-Deny therefore remains an open product/security decision requiring explicit Owner sign-off before implementation; it is intentionally out of scope here and must not be implemented without a future decision superseding this ADR.

3. Uniqueness rule: `user_id + permission_key`

`UserPermissionOverride` is unique on `(user_id, permission_key)`, not `(school_group_id, user_id, permission_key)`. A TIS `User` row has exactly one `school_group_id` column (no membership table), so SchoolGroup is not an independent dimension of a per-user override's identity -- a user cannot simultaneously hold two different override rows for the same permission key under two different SchoolGroups, because a user belongs to exactly one SchoolGroup at a time. `school_group_id` is still recorded on every row and is still validated by `apply_user_override` (the row's `school_group_id` must equal the target user's own current `school_group_id`) purely for tenant-isolation enforcement and audit context, not as part of the row's logical identity.

4. Platform-actor scope resolution (this pass's fix)

`POST /users/permissions/{user_pk}` must resolve the `school_group_id` passed to `apply_user_override`. That value is never accepted from client request input (query parameter, form field, header, or cookie) as an authority boundary -- doing so would be a privilege-escalation vector, since `apply_user_override` itself only uses this value to decide which tenant's row is written.

Two acting-admin cases exist:

1 **Ordinary tenant administrator.** `_get_user_school_group_id(db,

current_user) `resolves the acting admin's own verified active SchoolGroup (their own` school_group_id`/branch-derived scope, or a session-verified` scope_school_group_id `established by` auth.get_current_user`'s existing branch-selector-cookie resolution against real, active` Branch `rows -- never raw, unverified client input). _can_manage_target_user` already requires a non-platform actor's target to share this exact SchoolGroup, so this value always equals the target's own SchoolGroup for a legitimate same-tenant request.

1 **Platform-level actor (owner/developer).** `auth.get_user_school_group_id`

always returns `None` for a platform user by design (`is_platform_user` short-circuits it), regardless of any stale tenant fields on that platform user's own row and regardless of whatever branch/SchoolGroup their own session happens to have selected via the ordinary branch-selector cookie (`auth.get_current_user`'s `scope_school_group_id` resolution, which is a real mechanism but answers "what tenant is this platform actor currently viewing," not "what tenant does this specific target user belong to"). `_can_manage_target_user` / `can_manage_target_user_account` already grant a platform actor unconditional cross-tenant authority to manage any non-platform target user's account (this is the codebase's existing, pre-established "platform manages any tenant user" pathway -- no new permission key was introduced for it). Consistently with that existing authority and with `_build_user_permission_context`'s identical read-side resolution (which already computed `target_school_group_id = auth.get_user_school_group_id(db, user_row)` for rendering the panel), the write path now resolves `school_group_id` from the already server-loaded target user's own verified `school_group_id` whenever the acting admin `is_platform_user`, unconditionally -- not merely as a fallback when the platform actor's own scope happens to be empty. This was required because a platform actor's own selected branch/SchoolGroup scope can legitimately differ from an arbitrary target's SchoolGroup (the platform actor is authorized to manage users across every tenant, not only whichever one their session happens to currently be scoped to), and using the actor's own scope in that case previously produced a spurious `"target user is not in the active SchoolGroup"` rejection for an otherwise fully authorized cross-tenant write.

No new permission key, client-supplied scope parameter, or "select an acting SchoolGroup" form control was added. The value used is always exactly the value `apply_user_override` would itself require to succeed (it independently re-validates `target_user.school_group_id == school_group_id`), so this resolution cannot be used to write into any SchoolGroup other than the one the specific target user actually belongs to.

Rejected alternative

Accepting an explicit client-supplied "acting SchoolGroup" selection for a platform actor (mirroring how `POST /system-configuration/role-permissions` lets an actor holding the separate, dedicated `system_owner.manage_global_role_permissions` permission choose an explicit target `school_group_id` for a *role-level* policy override) was considered and rejected for this *user-level* route. Role-permission overrides configure policy for an arbitrary SchoolGroup chosen by a dedicated platform authority; per-user overrides always concern one specific, already-identified target user who has exactly one SchoolGroup, so deriving scope from that target directly is both simpler and strictly safer than accepting any client-supplied scope value, and requires no new dedicated permission.

Consequences

1 `routers/users.py`'s `update_user_permissions` no longer hard-errors for

a platform-level actor with no tenant SchoolGroup context, and no longer spuriously rejects a platform actor's legitimate cross-tenant write when their own session scope differs from the target's SchoolGroup.

1 No new permission key, schema, or migration was introduced.

1 Regression coverage: `tests/test_user_permission_override_routes.py`

(`test_platform_actor_with_no_own_tenant_scopes_override_to_target_school_group`, `test_platform_actor_override_scope_tracks_each_targets_own_school_group_not_a_shared_value`, `test_platform_actor_cannot_use_service_layer_to_write_into_a_mismatched_school_group`) and a template-render integration test (`tests/test_edit_user_template_render.py`) proving the Edit User page panel renders all three Inherit/Allow/Deny controls, both the inherited-grant and Deny-override reason text, and the exact inert markup for platform-only/non-assignable keys.

1 Allow-over-Deny remains explicitly unimplemented pending a future,

separately governed Owner decision.

Promo Grant Replacement (Promo-to-Promo)

docs/adr/0041-promo-grant-replacement.md

ADR 0041: Promo Grant Replacement (Promo-to-Promo)

Context

ADR 0020 explicitly deferred "promo renewal, transfer." An operational need exists for a Platform operator to replace an organization's existing active `PromoGrant` with a fresh grant against a different, currently valid `PromoCode` - for example when a customer's original promo offer is retired and a new, larger offer should take over the same organization's commercial access. This is a promo-to-promo transition and is distinct from the promo-to-paid conversion ADR 0024 already governs; ADR 0024's guardrails (blocking early conversion of an *active* promo, recovery-period eligibility) are specific to converting to a paid subscription and do not apply here, since a replacement promo grant is not paid credit and does not require the existing grant to first expire or enter recovery.

Decision

A new Platform Console-only operation, "Replace / Extend Promotional Access," atomically supersedes an organization's single active `PromoGrant` with a new `PromoRedemption`/`PromoGrant` pair redeemed against a different `PromoCode`, following the same transactional discipline ADR 0024 already established for promo-to-paid conversion (end old evidence, create new evidence, repoint the tenant link, all inside one transaction so there is never a committed window with zero active commercial source).

The old `PromoGrant` transitions to `status = 'superseded'` (retained, never deleted) and its paired `WorkspaceEntitlement` transitions to `status = 'ended'`. The new `PromoGrant.supersedes_grant_id` points at the old grant's id - the new row records which grant it replaced, matching the existing `PromoCode.supersedes_promo_code_id` convention where a newly created replacement definition's `supersedes_promo_code_id` points at the id of the definition it replaces (`promo_code_service.replace_promo`). The single `TenantProvisioningLink.promo_grant_id` is repointed from the old grant to the new grant in the same transaction; a new `WorkspaceEntitlement` (`entitlement_type = 'promo'`, `status = 'active'`) is created against the new grant with entitlement values (plan features, `quota.active_branches`) derived from the new promo's plan, so commercial capacity/plan evidence never points at stale numbers.

This operation changes capacity/limits only. It never creates, modifies, or deletes a `Branch`, `User`/staff, or `Teacher` row; the organization's existing operational branches, staff, and teachers are carried forward unchanged. The branches already active under the old grant (via `PromoGrantBranchAssignment`) are re-assigned to the new grant so the existing `promo_grant_service.resolve_promo_grant` and `commercial_authority_service.resolve_commercial_authority` capacity-display paths resolve correctly against the new grant with no other change to how capacity is computed or displayed.

Authorization reuses the existing platform-only `promo_codes.manage` decision already enforced on every other Platform Console promo-code admin route (`saas.router._require_promo_permission`), re-checked defensively at the service layer (matching the existing `promo_code_service.revoke_promo` precedent of a service-level authorization check). The replacement promo must itself pass the same `_validate_promo_definition` gate every normal customer redemption uses (active, approved, within its redemption window, not itself superseded by a newer definition, scope/redemption-limit rules satisfied) - this task does not introduce a second, weaker validation path. A Platform Console confirmation step shows the old grant's limits and the new promo's limits (branches/system users/teachers, plan, promo reference) before the operator confirms.

Consequences

1 ADR 0020's "promo renewal, transfer" deferral is resolved for the

promo-to-promo case only; promo renewal (extending the same promo's window) and communication/automated-expiry-job promo transfer remain deferred, unchanged.

1 Exactly one active `PromoGrant` and one active `WorkspaceEntitlement` per

organization is preserved throughout (enforced by the existing `uq_promo_grants_active_group` / `uq_workspace_entitlements_active_group` partial unique indexes); no schema or migration change was required.

1 A failure at any point inside the transaction rolls back the entire

operation; the old grant remains active and no partial state is committed, matching ADR 0024's existing rollback discipline.

1 This operation is independent of ADR 0024's promo-to-paid conversion; the

two are not conflated and do not share a code path, though both follow the same "end old evidence, create new evidence, repoint the tenant link, one transaction" shape.

1 Idempotent retry of a replacement request (for example, a duplicate

operator submission using the same `operation_key`) is not specially deduplicated the way `activate_promo`'s activation-session retry is; a retried replacement request re-validates against the now-current grant and will either succeed against the state left by the first attempt or fail closed if there is no longer an active grant to replace. This is recorded as a known limitation rather than solved by inventing a new idempotency mechanism.

Student Learning Style Four-Dimension Profile (Schema Foundation)

docs/adr/0042-student-learning-style-four-dimension-profile.md

ADR 0042: Student Learning Style Four-Dimension Profile (Schema Foundation)

Context

ADR 0031 governs and M1 milestone work ("Student Learning Style V1 - Core Implementation Landed", `docs/PROJECT_STATE.md`) implemented exactly one optional, single-select, four-value categorical `Student.learning_style` field (`Visual`/`Auditory`/`Read/Write`/`Kinesthetic`). That decision explicitly scoped V1 to single-select only and stated multi-style support "is not to be scaffolded speculatively."

The Owner has now directed a distinct, additional Learning Style data contract for the Students + Talent & Potential product line: four independent Student profile percentages - Verbal, Non-verbal, Quantitative, and Spatial. This is not the categorical multi-select ADR 0031 declined to scaffold; it is a separate numeric profile representation that coexists with, and does not replace, the existing categorical field. This ADR is the governed decision authorizing that coexistence and amending ADR 0031's "Domain and scope" only to the extent needed to make clear that this later, separately governed four-percentage profile is not the deferred multi-style scaffolding ADR 0031 declined - it does not retroactively alter ADR 0031's historical record of what V1 shipped, and the original categorical field, its four fixed values, and its single-select nature are unchanged and remain in effect.

Decision

Domain, scope, and independence from the legacy categorical field

Learning Style four-dimension profile belongs to the **Student** domain, exactly like the legacy categorical field it sits alongside.

1 Four independent, optional (nullable) integer Student attributes:

`learning_style_verbal_percentage`, `learning_style_non_verbal_percentage`, `learning_style_quantitative_percentage`, `learning_style_spatial_percentage`.

1 Each is validated independently against the range 0-100 inclusive (or

NULL). There is no sum-to-100 rule and no rule relating the four values to each other.

1 The legacy categorical `learning_style` column (`Visual`/`Auditory`/

`Read/Write`/`Kinesthetic`) is explicitly preserved as legacy/deprecated data. It is not deleted, rewritten, mapped, or cleared, and no automatic conversion between the categorical value and the four percentages exists or is authorized.

1 No Learning Style history table, profile-version table, Talent snapshot,

or Talent scoring linkage is introduced. There is no historical Learning Style versioning of either the legacy field or the new percentages.

No effect on Talent semantics

Exactly like the legacy categorical field under ADR 0031, the four-dimension profile has zero effect on, and must never be read by, any Talent-domain scoring, Program Criteria / Review Candidate eligibility computation, or Official Identification decision logic. It may be displayed as informational learner context only.

Governance boundary and milestone scope

This ADR authorizes the four-dimension persistence/schema foundation described above. Consistent with the Students + Talent & Potential M1 milestone, this ADR does not itself authorize or claim: a create/update API contract for the four percentages, any frontend surface, Evaluation Progress, roster import/export, or any Talent frontend change. Those remain separately governed, later implementation work. Al-Andalus Section display was also out of this ADR's scope at M1; it is now owner-authorized (presentation-only, for the exact verified `workspace_uuid` `72e52eb2-3844-447b-92a8-c55015f73257`, canonical identity unchanged, implementation not yet complete) under ADR 0045, which is the governing authorization record for that feature - this ADR's scope remains the Learning Style four-dimension profile only. This ADR also does not authorize multi-select of the legacy categorical values, which remains governed exactly as ADR 0031 recorded it.

Status

ACCEPTED as of 2026-09-22. Implementation status (schema/migration only, as of this ADR) is tracked in `docs/PROJECT_STATE.md`; this ADR is the governance/authorization record, not a completion record for any API, service, or frontend surface.

SUPERSEDED as of M14 (2026-09-23) - see "M14 Correction" section below. The text above is preserved unmodified as the historical record of what was actually decided and, subsequently (M2/M3, beyond this ADR's own original schema-only scope), actually implemented.

M14 Correction (2026-09-23): This ADR's Product Model Is Corrected/Superseded

Added 2026-09-23, per direct Owner instruction following a read-only Post-M13 Correction Review. This section does not delete or rewrite any text above; it records that the decision above was a misinterpretation and states the corrected model. See ADR 0031's own M14 Amendment section for the full paired governance record (this is the ADR 0042 half of that same correction).

What actually happened after this ADR (verified against code, not assumed)

Contrary to this ADR's own "Governance boundary" section above (which states this ADR authorizes schema/migration only, and explicitly does NOT authorize "a create/update API contract for the four percentages" or "any frontend surface"), the subsequent M2/M3 delivery went further than this ADR itself authorized: `student_academic_service.py` implemented a full create/update/GET API for the four percentages (`LEARNING_STYLE_PERCENTAGE_FIELDS`, `_clean_learning_style_percentage`), and a real Student create/edit frontend collected and displayed them (`templates/student_form.html`, `templates/student_profile.html`, `templates/_learning_style.html`'s `learning_style_percentage_fields`/ `learning_style_profile` macros) - while the ORIGINAL single-select categorical field from ADR 0031 never received any create/edit frontend selector at all. This is noted here as a verified implementation-history fact, not a new authorization for it.

The correction

"Percentage" here was always intended to mean POPULATION/AGGREGATE distribution - what share of an authorized Student population selected each Learning Style category - never an independent per-Student dimension score. The four-dimension percentage profile this ADR introduced is therefore a corrected/superseded product model, not a valid alternate representation alongside the categorical field:

1 Verbal, Non-verbal, Quantitative, and Spatial become four MORE values on

ADR 0031's single categorical `learning_style` field (eight total), not an independent numeric profile.

1 The four `learning_style_*_percentage` columns this ADR authorized are

OPERATIONALLY DEPRECATED as of M14: no longer written, exposed in Student create/edit, displayed on the Student profile, or read as Learning Style authority anywhere, including in Talent Student context.

1 Existing stored percentage values are preserved completely untouched - no

backfill, no conversion, no clearing, no inference of a categorical value from a percentage or vice versa. Physical column removal is a separately gated, later cleanup contingent on an explicit data-occupancy verification that M14 could not safely perform (the local development database has no `students` table at all, so M14 could not inspect real occupancy either way).

1 Aggregate (population) Learning Style percentages are computed by

`student_learning_style_analytics.py`, extended to all eight categorical values plus "Unassigned," reusing the same privacy/suppression contract ADR 0031 already required.

This correction does not reauthorize anything this ADR's own "Governance boundary" section already declined (e.g. it does not retroactively bless the M2/M3 frontend/API overreach noted above); it corrects the underlying product model going forward.

M18a Amendment (2026-09-23): Read-Exposure Deprecation Completed

Added 2026-09-23. M18a independently re-verified the M14 amendment's "no longer ... exposed" claim above against the actual implementation and found it was not yet fully true: `routers/students.py`'s `_student_json` and `routers/students_ui.py`'s `_student_view` still serialized all four `learning_style_*_percentage` fields in the current normal Student API/UI projection (including as literal `null` when unset). M18a removed them from both projections, completing the M14 read-exposure deprecation exactly as originally intended. Stored columns and any historical values remain completely untouched (same as M14); physical column removal remains a separately gated, later decision, unchanged by this amendment.

TIS Student Number Global Managed-Identifier Invariant (Schema Foundation)

docs/adr/0043-tis-student-number-managed-identifier-invariant.md

ADR 0043: TIS Student Number Global Managed-Identifier Invariant (Schema Foundation)

Context

`Student.id` (see "Student & Academic Placement Foundation", `docs/AI_PROJECT_CONTEXT.md`) is the internal relational identity for a Student and is SchoolGroup-owned. It is not, and this ADR does not make it, a customer-facing managed identifier. The existing `StudentExternalIdentifier` model (`student_external_identifiers` table, migration `20260904_001_student_academic_placement_foundation`) already supports arbitrary `namespace`/`value` cross-system identifiers with tenant-scoped uniqueness (`school_group_id`, `namespace`, `value`) and an active/inactive lifecycle. No prior ADR or KMS document records an authoritative, globally unique, cross-SchoolGroup managed Student identifier namespace; this ADR is that record for the Students + Talent & Potential milestone plan's TIS Student ID foundation.

Decision

Namespace and canonical format

The managed namespace is exactly `tis_student_number`, stored as a normal `StudentExternalIdentifier` row (namespace = `tis_student_number`). The canonical stored format is exactly `STD` followed by exactly 10 digits (for example `STD1234567891`, `STD0012345678`). This ADR does not authorize a database-level format constraint in this milestone; format validation is deferred to the future Student ID create/edit service/API, which is explicitly not implemented as part of this schema-only foundation.

Global uniqueness invariant (the exception to tenant-scoped uniqueness)

Every other `StudentExternalIdentifier` namespace keeps its existing tenant-scoped-only uniqueness (`uq_student_external_identifiers_scope_namespace_value`) unchanged. `tis_student_number` is the one namespace with an additional, stricter invariant:

1 The canonical `value` must be unique across **every** TIS organization

(SchoolGroup), not merely within one - enforced by a partial unique index on `value` filtered to `namespace = 'tis_student_number'`, covering both `active` and `inactive` rows.

1 A retired/inactive `tis_student_number` value is permanently reserved and

can never be reissued to a different Student, in a different SchoolGroup, or reactivated for reuse as a distinct identity. This is a direct consequence of the same index also covering inactive rows.

1 At most one `active` `tis_student_number` row may exist per Student at a

time - enforced by a second partial unique index on `student_id` filtered to `namespace = 'tis_student_number' AND status = 'active'`. Because `Student.id` is a single globally unique primary key (not itself SchoolGroup-scoped), this index correctly enforces the invariant organization-wide without needing `school_group_id` in its key.

1 `Student.id` remains the internal relational identity and primary/foreign key surface; this ADR adds no new Student identity table and no new column on `students`. An existing legacy Student may have zero `tis_student_number` rows - this is valid and is never backfilled with a fabricated value.

Governance boundary and milestone scope

This ADR authorizes exactly the database-level integrity foundation above: two partial unique indexes on the existing `student_external_identifiers` table, with no new column, no new table, and no rewrite of any existing row. It does not authorize or claim: a Student ID create/edit API contract, Student ID UI, duplicate-value user-facing messaging, canonical-format database enforcement, or automatic backfill/generation of `tis_student_number` values for existing Students. Those remain separately governed, later implementation work.

Status

ACCEPTED as of 2026-09-22. Implementation status (schema/migration only, as of this ADR) is tracked in `docs/PROJECT_STATE.md`; this ADR is the governance/authorization record, not a completion record for any API, service, or frontend surface.

M12.1 Amendment - 2026-09-23: physical index identifier correction only

This is an implementation-detail correction, not a change to the decision above. The active-per-Student partial unique index described in this ADR ("a second partial unique index on `student_id` filtered to `namespace = 'tis_student_number' AND status = 'active'`") was originally implemented with the physical name `uq_student_external_identifiers_tis_student_number_active_student` (65 characters), which exceeds PostgreSQL's 63-character `NAMEDATALEN` identifier limit and was silently truncated by PostgreSQL on `CREATE UNIQUE INDEX`, breaking the originating migration's own idempotency check. The canonical physical name is now `uq_student_external_identifiers_tis_number_active_student` (57 characters); see `docs/PROJECT_STATE.md` (M12.1) and `db_migrations.py` (migration `20260923_001_student_tis_number_index_identifier_length_remediation`) for the full remediation record. The semantic invariant - unique, partial, `(student_id)`, filtered to `namespace = 'tis_student_number' AND status = 'active'` - is completely unchanged, as is every other decision in this ADR.

Talent Evaluation Progress Analytics (M4)

docs/adr/0044-talent-evaluation-progress-analytics.md

ADR 0044: Talent Evaluation Progress Analytics (M4)

Agent 3 aggregate dashboard amendment (2026-09-25)

The filtered dashboard composes existing authorities in an identity-free projection. Completion and Classification count Evaluation participations; Student KPIs and Learning Style count distinct current Students. Current reassessments resolve through the original Evaluation context. Organization percentages divide underlying counts, never average Branch percentages. Result means require one Program, framework and scale; incompatible results are unavailable, not pooled.

Selected Branch/Grade/Section/Program/Period comparisons are bounded to six groups. The full authorized comparison family participates in conservative privacy closure even when only some groups are displayed. Completion uses P2, results and rubric evidence P3, and Classification P4 provider rules. Suppressed families emit protected states. Program/Competency/Indicator scope is validated; historical frameworks retain separate indicators. Ordered periods do not imply growth.

The dashboard reuses M10's read-only repeatable snapshot dependency without changing global isolation or older endpoint contracts. Requests exceeding 5,000 scoped population rows fail explicitly for narrowing; totals are never truncated. The owner-approved Learning Style cohort boundary is recorded in ADR 0031. Individually authorized operational roster records remain outside aggregate cohort suppression.

Context

Following M9 Deterministic Talent Analytics and the M10 Organization Intelligence family, the Talent & Potential product line still had no backend-authoritative "Evaluation Progress" contract for Student, Branch, or Organization scope, and the frontend (`static/js/talent-operations.js`) computed a client-side average of `overall_result.average` across Review Candidate rows sharing a common rubric scale. Two design questions were genuinely ambiguous and were escalated rather than guessed: (1) whether Evaluation Periods on different governed `framework_version_ids` may be combined into one Overall Result, and (2) whether an individually authorized Student's own result should be subject to the same aggregate cohort-size privacy suppression used for Branch/Organization analytics.

The Owner has now explicitly ratified both decisions below. This ADR records the M4 architecture and both ratified decisions as the durable governance artifact.

Decision

Canonical Evaluation Period result

The Evaluation Period numerical result input is exactly ADR 0037's Overall Program Result `normalized_percent` (`talent_student_assessment_service .overall_program_result`). ADR 0037's rubric-scale/KPI educational semantics are unchanged; `normalized_percent` is read-only input here.

Active/opened Evaluation Period predicate (confirmed, not new)

An Evaluation Period is an active/opened weighting position exactly when `TalentPlannedEvaluationPeriod.status == 'planned'` AND its linked `TalentAssessmentCycle.status IN ('open', 'closed')`. This was verified directly against `models.py` and `talent_evaluation_plan_service.validate_cycle_period_link`/ `validate_linked_cycle_open`,

which only ever create or preserve that link under an Active Plan + Planned Period + Draft-then-Opened Cycle. A cancelled Period, a not-yet-linked/opened Period, and an ad-hoc (unlinked) Cycle are never a weighting position. Configured label/short_code never affect order or weight; only governed `sequence` orders Periods. For A active Periods, the derived (never persisted) nominal weight is `1/A`.

DECISION 1 - Framework comparability (owner-ratified)

Evaluation Period results are combinable into one Overall Result only when every contributing active Period's linked Cycle shares one identical governed `framework_version_id`. When active Periods span more than one `framework_version_id`, every Period is still returned individually (preserving its own result and nominal weight), the combined Overall Result is `null`/unavailable, and `comparability_state="not_comparable"` / `comparability_reason_code="framework_changed"` is reported. No cross-framework delta/improvement is calculated, no framework-equivalence mapping is introduced, and no normalization across frameworks is performed. This is consistent with, and does not weaken, ADR 0027/B10's existing Framework-sensitivity precedent for longitudinal comparability.

DECISION 2 - Single-Student disclosure (owner-ratified)

An individually authorized Student's own Evaluation Progress is governed by normal Student/Talent authorization and tenant/Branch scope - the existing `talent_learner_profiles.view` permission plus the same frozen-historical-Branch check `talent_learner_profile_service .build_learner_profile` already applies - and is NEVER subject to aggregate cohort-size privacy suppression merely because the result belongs to one Student. An unauthorized Student's result remains inaccessible (non-enumerating 404, matching the existing Learner Profile convention); aggregate Branch/Organization privacy is completely unaffected by this rule, and this rule never permits exposure of a suppressed aggregate value or reconstruction of unauthorized cohort analytics through repeated single-Student reads (the M4 Student route returns only that one Student's own already-authorized historical facts, with no cross-Student aggregation capability). This is consistent with, and generalizes, ADR 0031's existing statement that "Individual Student Learning Style values remain governed by normal Student-record permissions and are not a substitute for aggregate analytics."

Branch / Organization Evaluation Progress

`BranchPeriodResult(B,p)` is the mean of valid governed Student `normalized_percent` results attributed to Branch B for Period p, using frozen historical `TalentAssessmentCyclePopulationMember.branch_id` attribution (never current Placement). `OrganizationPeriodResult(p)` is computed directly from every valid governed Student result in the authorized organization scope - never as an average of `BranchPeriodResult` values. Pending/unavailable Student results are excluded from both, never treated as zero. Overall Results (across comparable active Periods) are the mean of the available privacy-safe Period results; a Pending/suppressed Period is excluded, never zero, and is never reconstructable from the combined Overall value.

Privacy

Every Branch/Organization aggregate reuses only the M9 generic privacy primitives (`Cell`/`Group`/`apply_primary_privacy`/ `run_complementary_suppression` in `talent_analytics_privacy.py`) - never the M10 `talent_org_intelligence_contract.py` `MetricCode`/`CellIdentity` vocabulary, which this milestone leaves frozen and unmodified. Every aggregate is gated on an authorized contributing-count `Cell` (parent total = sum(per-Branch counts)); a derived mean is only serialized when its own count `Cell` is independently `visible` after complementary suppression, exactly mirroring the existing `build_privacy_safe_coverage _bundle` all-or-nothing convention.

Branch comparison metrics (exactly seven, no MetricCode extension)

`talent_evaluation_progress_service.branch_comparison_metric` is a bounded dispatcher supporting exactly: Evaluation Period Result, Current/Overall Progress (both new M4 contracts above), Assessment Completion,

Assessments Started, Meets Program Criteria, Officially Confirmed (all four reusing the existing M9 `raw_coverage_by_dimension`/`raw_candidate_by_dimension`/ `raw_identification_by_dimension`/`build_breakdown_group` providers unchanged), and Learning Style (new). This dispatcher is local to M4 and does not add to, or otherwise modify, the frozen 14-value M10 `MetricCode` enum.

Learning Style Branch aggregate

The Branch aggregate for one selected dimension (Verbal/Non-verbal/ Quantitative/Spatial) is the arithmetic mean of valid (non-null) current `Student` percentage values (ADR 0042) in the authorized Program-population cohort; `null` is excluded, `0` is a real included value, there is no sum rule, no normalization, and no dominant-style derivation. It is protected by the identical `Cell`/`Group` privacy mechanism used for Branch/Organization Evaluation Progress, per ADR 0031's existing requirement that aggregate Learning Style distribution "go through the same privacy/suppression contract already governing Talent aggregate analytics."

Consequences

1 The frontend's client-side Review-Candidate-average

(`talent-operations.js`) is unrelated to, and untouched by, this new backend contract; a future frontend milestone may adopt the new authoritative endpoints.

1 No schema or migration change; no new permission; the M10 `MetricCode`

registry is unmodified.

1 `normalized_percent` is not a stored column, so Branch/Organization

aggregation calls `overall_program_result` once per Completed Assessment in scope rather than a single SQL aggregate - a documented, bounded per-assessment cost at the same Program+AcademicYear scale every other M9/M10 module already operates at, not a claimed PostgreSQL performance qualification.

1 The four M9-reused Branch comparison metrics (Assessment Completion,

Assessments Started, Meets Program Criteria, Officially Confirmed) inherit that existing M9 characteristic where an all-zero cohort's total `Cell` receives `raw_value=0` rather than `None`, so a policy may label it `suppressed` instead of `no_data` - no raw value is ever leaked either way (this is a label-only nuance, not a privacy defect), and it is unchanged from the existing, already-reviewed `/breakdowns/branch` route behavior this milestone reuses rather than duplicates.

M14 Amendment (2026-09-23): Learning Style Branch Aggregate Removed

Added 2026-09-23, per the same owner-directed M14 correction recorded in ADR 0031's and ADR 0042's own M14 amendment sections. The text above is preserved unmodified as the historical record of the seven-metric dispatcher as originally delivered.

The "Learning Style Branch aggregate" metric above averaged the four `learning_style_*_percentage` columns (ADR 0042) across a Branch population. Under the M14-corrected model, "percentage" means population/aggregate distribution over the single categorical `Student.learning_style` field (now eight values), never a mean of independent per-Student dimension scores - so this metric's underlying premise no longer holds. It is REMOVED from `talent_evaluation_progress_service.APPROVED_BRANCH_METRICS` as of M14: a request for the `learning_style` metric now receives the same `invalid_filter`/400 response as any other unrecognized metric value, rather than continuing to report a semantically wrong number. The Branch-comparison dispatcher now supports exactly six metrics (the seven above, minus Learning Style). A categorical distribution replacement for this specific Branch-comparison surface (if wanted) is out of M14 scope and is left for a later, separately governed milestone; the

Student-domain aggregate Learning Style distribution required by ADR 0031 continues to be served by `student_learning_style_analytics.py` (Students page), unaffected by this removal.

M18a Amendment (2026-09-23): Dead Learning Style Branch Aggregate Code Removed; is_current Confirmed as Applicable-Current-Result Authority

Added 2026-09-23. Two confirmations, no new rule:

Dead code removal. The M14 amendment above removed `learning_style` from `APPROVED_BRANCH_METRICS` but deliberately kept `learning_style_branch_aggregate`/`_learning_style_values_by_branch`/`_LEARNING_STYLE_COLUMNS` in `talent_evaluation_progress_service.py`, unreferenced by the dispatcher, purely so the deprecated-column mean computation was not silently mutated. M18a re-verified directly that `branch_comparison_metric` (`routers/talent_evaluation_progress.py`'s only caller of any Branch-comparison metric) has rejected `"learning_style"` before ever reaching that code since M14, confirming the three symbols were genuinely unreachable from any current endpoint/service path. They were removed outright, along with their two direct-call tests (`tests/test_talent_evaluation_progress.py`, formerly "Section 8"). The `learning_style_dimension` query parameter on `routers/talent_evaluation_progress.py`'s branch-comparison route remains present but was already inert (rejected before use) since M14 and is left untouched - removing that plumbing is a separate, non-blocking cleanup, not required to close "no current four-dimension Learning Style analytics authority."

2026-09-24 Amendment (M18b-1). The `learning_style_dimension` query parameter/keyword argument was re-audited directly (grep, not assumed) and confirmed still genuinely unreferenced inside `branch_comparison_metric`'s own body - it was removed outright from `routers/talent_evaluation_progress.py`'s branch-comparison route and from `branch_comparison_metric`'s signature in `talent_evaluation_progress_service.py`. No schema/behavior change: the function still raises `invalid_filter` for any `metric` outside the six `APPROVED_BRANCH_METRICS`, exactly as before. The governed Family 1 Learning Style backend contract (categorical distribution, never a per-dimension mean) is `student_learning_style_analytics.py`, now also reachable through `talent_results_analytics_service.py`'s bounded M18 Results & Analytics contract (see the M18b-1 release note).

Applicable-current-result authority confirmed. M18a needed to resolve which `TalentStudentAssessment` governs a Student's current M17 classification when more than one exists for a Program. This ADR's own "Active/opened Evaluation Period predicate" section already establishes that only `sequence`-ordered, actually-active Periods weight into a combined result; separately, `models.TalentStudentAssessment.is_current` (set by the reassessment flow this repository already implements per ADR 0036) is the authority for which Assessment attempt is the current one for a given Program+Cycle+Student, already consumed by `talent_analytics_service._assessment_ids_subquery` and the B9 Student Drill before M18a. M18a's `talent_classification_service.assessment_classification` integrations (Learner Profile, B9 Student Drill) use exactly this same `status == 'completed' AND is_current == True` predicate - no new business rule was introduced, and this ADR is not reopened.

Acceptance C Note (2026-09-24): Learning Style Aggregate Privacy

The "Learning Style Branch aggregate" section above (already removed by M14) cited ADR 0031's requirement that the aggregate distribution go through the Talent privacy/suppression contract. ADR 0031's "Acceptance C Amendment" (2026-09-24, owner decision) supersedes that requirement for the categorical Learning Style distribution only: it is authorized Student-domain aggregation, denominator including Unassigned, with no Talent small-cell suppression. This ADR's own Evaluation Progress metrics and their Cell/Group suppression are unchanged.

Batch 1 Amendment (2026-09-24): Distinct-Student Headline And Current-Student Reads

> **Correction (2026-09-25).** Wording elsewhere in the Batch 1 documentation that the > global Branch is only a "default" Talent scope is superseded: the global Branch is a > hard ceiling enforced server-side (`talent_branch_scope`; M10 `resolve_access_context` > becomes Branch-scoped when the actor's global scope is one Branch, so > `accessible_historical_branch_ids` = that Branch and every Evaluation Progress / > analytics route inherits it). Branch comparison and the Organization Talent Map remain > available only under an explicit global All Branches. No `MetricCode`, privacy or > Evaluation Progress semantics changed.

Owner rule: current Talent figures use only Students that currently exist. This ADR's population queries (`population_query`, and the M10 `frozen_membership_query`) now require the Student to exist in the same SchoolGroup, and the Evaluation Progress Branch/Organization result reads do the same. The Overview headline for "Students" is the DISTINCT-Student count published through the existing B2 pipeline at class P2 using the existing `student_drill_population/count/distinct_student` coordinate in the `overview` projection family only (no new `MetricCode`, no new `MembershipGrain`; the B9 P7 Student Drill gate and its identification rules are unchanged). Membership-grain metrics (`frozen_eligible`, `completed`, coverage) keep their definitions and denominators; only their user-facing labels changed to state their grain.

Owner Amendment (2026-09-25): Branch Authority For Organization-Authorized Actors

Dated owner-directed amendment; the Batch 1 closure text above is preserved as history. The closure statement that the global (sidebar) Branch is a hard Talent ceiling is superseded for organization-authorized actors only. The sidebar selector lists real Branches. For an organization-authorized actor (`auth.can_access_all_branches`) the Talent ceiling is the actor's own authorization (all Branches of the SchoolGroup); the sidebar Branch is only the default page-level Branch, and the Talent Branch filter offers All Branches and each authorized Branch. For a Branch-limited actor the ceiling remains their authorized Branch(es) and any attempt to name another Branch is rejected. An omitted `branch_id` means every authorized Branch; an explicit `branch_id` is validated server-side, and tenant isolation is unchanged. The `branch_scope=all` marker cookie and `user.scope_all_branches` are removed (an old cookie is ignored). No `MetricCode`, privacy, Evaluation Progress or schema change.

Classification note (same date): an aggregate Classification band that reads "Unavailable" is the approved Release 1 provider withholding a band cell below the minimum cohort (or a total below it, or a fail-closed absent configuration). No rule changed; the UI states one uniform value-free explanation. Making small cohorts visible needs an explicit owner decision and an amendment here and in ADR 0028.

Part 2 Amendment (2026-09-25): Chart Types, Background Refresh And Progress Over Time (presentation only)

Dated, presentation-only amendment; no `MetricCode`, privacy, suppression, Classification, Learning Style or Evaluation Progress semantic changed. (1) Chart types: the selector offers only materially different types; circular types only for a full public partition of at most eight categories and never for time series, ordinal Rubric levels or Learning Style (nine categories); percentages and counts remain visible in every type and in the accessible table; no frontend threshold or derived value exists. (2) Dashboard filters refetch in the background under the same authorized read API (`.../dashboard`), with cancellation of superseded requests and only the newest response rendered. (3) Progress Over Time stays a real per-Program view over the ADR 0027 longitudinal projection; a period that is suppressed or has no Student result yet is a gap ("Unavailable" / "No data yet"), never zero, and different Program frameworks are never combined.

Al-Andalus Section Display - Workspace Authorization (Governance Only)

docs/adr/0045-al-andalus-section-display-workspace-authorization.md

ADR 0045: Al-Andalus Section Display - Workspace Authorization (Governance Only)

Context

`docs/PROJECT_STATE.md` (Students + Talent & Potential M1 milestone summary) and ADR 0042's governance-boundary note both listed "Al-Andalus Section display" among items explicitly deferred to later, separately governed implementation work. No prior ADR or KMS document recorded an authorized `SchoolGroup.workspace_uuid` for that later work, and no prior ADR authorized the presentation convention itself.

The Owner has now directed and authorized this bounded, presentation-only feature and has identified the exact production `SchoolGroup.workspace_uuid` it applies to, using the repository's existing read-only production audit (`scripts/audit_al_andalus_readonly.py`) run against the deployed Render PostgreSQL environment. This ADR is the governance record closing that deferred status and authorizing implementation for the exact identified workspace. It is a governance/authorization record only; it does not itself implement the feature (tracked separately in `docs/PROJECT_STATE.md`).

Decision

Authorized workspace identity

The authorized `SchoolGroup.workspace_uuid` for this feature is exactly:

`72e52eb2-3844-447b-92a8-c55015f73257`

Verification provenance: the Owner personally ran the repository's existing read-only production audit script against the deployed Render production environment and confirmed this `workspace_uuid` (`exact_name` "Al-Andalus", `school_group_id` `1` at time of audit, no duplicate-organization conflict, audit process exit code 0). No credentials, connection strings, or other production audit detail are recorded here or elsewhere in KMS.

Runtime activation for this feature must key exclusively off this exact `workspace_uuid` value. Activation must never be derived from `SchoolGroup.name`, organization display name, domain, email, Branch name, or any other editable/human-readable label - none of those are stable tenant identity and none may substitute for the UUID.

Authorized presentation convention

For the identified workspace only, Grade/Section display formatting may render as `{grade_number}.{section_ordinal}` (for example Grade 1/Section A -> `1.1`, Grade 1/Section B -> `1.2`, Grade 2/Section A -> `2.1`), with Section ordinal derived deterministically from an alphabetic Section name (A=1, B=2, ... Z=26). Any Grade or Section value that does not deterministically map under this rule (custom Section name, non-alphabetic Section, malformed or missing Grade, missing Section) must fall back to the existing/default canonical presentation unchanged - never an invented number, and never a hard failure caused solely by unavailable formatting. Every other workspace continues to use the existing/default presentation unchanged.

Presentation-only; canonical identity unaffected

This is a display-label transformation only. It must never replace, and implementation must never allow it to replace:

1 `PlanningSection.id` or any other canonical PlanningSection identity,

1 canonical `grade_level` or `section_name` values,

1 Student Academic Placement foreign keys,

1 frozen historical Talent Grade/Section attribution,

1 analytics grouping, query/filter identity, tenant/authorization scope, or

import/export matching identity.

A rendered value such as `1.1` may appear only as a display field (for example a bounded `section_display`-style projection alongside the unchanged canonical value); it must never become a stored identity, a request/filter parameter, or an import/export match key. No schema or migration is authorized or required by this ADR.

Status

Governance boundary

This ADR authorizes only: (1) the exact workspace identity above, and (2) the presentation convention and its constraints. It does NOT itself authorize or claim that a shared presentation helper, API/UI surface changes, or tests have been implemented - that implementation is separately governed, later work and is tracked (not-yet-complete as of this ADR) in `docs/PROJECT_STATE.md`.

ACCEPTED as of 2026-09-22. This ADR is the authorization record for the workspace identity and presentation convention; it is not a completion record for any implementation.

Academic Calendar History

docs/history/academic-calendar/README.md

This folder tracks meaningful changes to academic calendar workflows, event types, permissions, exports, and branch/year behavior.

Related files:

- `routers/academic_calendar.py`
- `templates/academic_calendar.html`
- `permission_registry.py`

AI Entitlements History

docs/history/ai-entitlements/README.md

2026-08-19 - Availability Separated From Consumption

ADR 0025 supersedes the temporary Enterprise-AI-only availability mapping. Every enabled registered AI feature is commercially available to coherent paid, promo, and active demo workspaces with `ai.use`; globally disabled definitions remain denied. A separate consumption-policy boundary preserves Standard demo allowances, Full/Custom policy, reservations, idempotency, counters, and durable events without inventing a provider billing engine.

2026-07-27 - M8B8 AI Entitlements And Commercial Foundation

M8B8 adds one tenant-safe AI entitlement authority, a controlled feature registry, an assignable `ai.use` permission, and durable usage counters plus reserved/successful/failed operation events. Customer Demo allowance is two successful uses per feature. Internal Sandbox is unlimited and measured separately. Customer Paid access follows the existing verified commercial resolver and `module.ai` plan entitlement; the reviewed foundation mapping is Enterprise AI only.

There are no executable AI routes in the current product. The service and denial payload are reusable boundaries for future implementations; M8B8 does not fabricate an AI tool or add M8B9 reset/override/lifecycle controls.

M8B9 Demo Operations, Notifications, And Testing

docs/history/demo-operations/2026-07-27-m8b9-demo-operations.md

Added generic Platform Owner controls for immediate reversible expiry, reactivation, unlimited-duration custom expiry, final-day reminders, and manual single/global lifecycle execution. Added Standard, Full, and Custom access profiles with workspace defaults and tenant-safe branch overrides.

All operations preserve the Customer Demo workspace and data, retain M8B8 AI usage history, use controlled feature registries, record durable before/after audits, and reuse the M8B7 email and Notification Center architecture. The schema is applied only by the existing pre-deploy migration command.

Common Customer Demo Feature Baseline

docs/history/demo-operations/2026-08-19-common-customer-demo-baseline.md

Standard, Full, and Custom active customer demos now retain the normal TIS customer feature baseline. Custom policy can still configure branch scope, safety, approved experimental features, and AI consumption allowances, but it cannot remove normal modules to simulate a lower subscription tier.

The migration reconciles only active demo product-feature policy. It preserves duration, lifecycle, expiry, AI allowance/unrestricted configuration, branch overrides, Platform Owner controls, and durable audit. Suspended, expired, or incoherent demos continue to fail closed.

KMS v3 Engineering Handbook

docs/history/engineering-handbook/2026-06-26-kms-v3-engineering-handbook.md

2026-06-26 - KMS v3 Engineering Handbook

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added KMS v3.0 Engineering Handbook

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only.

Previous Documented State

The KMS had master context, project state, ADRs, module history, AI context, generated PDF, manifest, and Platform Owner Knowledge Center docs. It did not yet contain a complete engineering handbook for module ownership, repository architecture, and end-to-end flows.

New Documented State

The KMS includes `docs/engineering/` with a module map, repository architecture guide, user/system flows, and developer onboarding index. The generated PDF now includes these docs as part of documentation version 3.0.

Reason For Change

New human developers and future Codex/ChatGPT conversations need a practical engineering handbook, not only a bundle of source documents.

User / Business Impact

Improves continuity, onboarding speed, technical review quality, and safer future implementation work.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page, database, migration, route, or runtime behavior changed.

Documentation Updated

1 `docs/engineering/README.md`

1 `docs/engineering/TIS_MODULE_MAP.md`

1 `docs/engineering/REPOSITORY_ARCHITECTURE.md`

1 `docs/engineering/USER_AND_SYSTEM_FLOWS.md`

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/PROJECT_STATE.md`

1 `docs/README.md`

1 `docs/CHANGE_HISTORY.md`

PDF Regenerated

Yes

Follow-Up Needed

Review PDF readability and handbook completeness before any Phase 3B work.

KMS v3 Phase 3B Engineering Layers

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3b-engineering-layers.md

2026-06-26 - KMS v3 Phase 3B Engineering Layers

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only.

Previous Documented State

The engineering handbook included the module map, repository architecture guide, user/system flows, and onboarding index. It did not yet include a database architecture overview, development standards, UI/UX philosophy, or product roadmap.

New Documented State

The engineering handbook includes Phase 3B layers:

- database architecture overview,
- development standards,
- UI/UX design philosophy,
- product roadmap,
- stronger AI/human developer onboarding guidance.

Reason For Change

New senior developers, Codex conversations, ChatGPT conversations, and technical reviewers need stronger system boundaries, design expectations, roadmap context, and development rules before changing TIS.

User / Business Impact

Improves implementation safety, onboarding quality, technical review quality, and continuity across future development sessions.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

Documentation Updated

- `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`
- `docs/engineering/DEVELOPMENT_STANDARDS.md`
- `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`

1	`docs/engineering/PRODUCT_ROADMAP.md`
1	core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Review handbook completeness and readability before any Phase 3C work.

KMS v3 Phase 3C Governance And AI Traceability

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md

2026-06-26 - KMS v3 Phase 3C Governance And AI Traceability

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added KMS v3.0 Phase 3C Governance And AI Traceability

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for KMS v3.0 Phase 3C only.

Previous Documented State

The engineering handbook included module maps, repository architecture, user/system flows, database architecture, development standards, UI/UX philosophy, and roadmap. It did not yet document rejected decisions, visual documentation standards, definitive AI guidance, governance, or decision traceability.

New Documented State

The engineering handbook includes Phase 3C layers:

- rejected architectural decisions,
- visual documentation framework,
- AI optimization guide,
- project governance,
- engineering decision traceability.

Reason For Change

Future developers and AI assistants need to understand why TIS became what it is, not only its current structure.

User / Business Impact

Improves continuity, reduces repeated architectural debates, improves AI readiness, and clarifies governance and review expectations.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

Documentation Updated

- `docs/engineering/REJECTED_DECISIONS.md`
- `docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md`
- `docs/engineering/AI_OPTIMIZATION_GUIDE.md`

1	`docs/engineering/PROJECT_GOVERNANCE.md`
1	core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Future KMS improvements may add screenshots, diagrams, deployment runbooks, route inventories, test strategy docs, and deeper module guides. Do not implement those without a later approved phase.

KMS v3 Phase 3D Lifecycle Foundation

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md

2026-06-26 - KMS v3 Phase 3D Lifecycle Foundation

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Completed KMS v3.0 Phase 3D Lifecycle Foundation

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved as KMS v3.0 Phase 3D final phase.

Previous Documented State

The engineering handbook documented module maps, repository architecture, flows, database architecture, standards, UI/UX philosophy, roadmap, rejected decisions, visual documentation, AI optimization, and governance. It did not yet define a full self-evolving lifecycle from task through deployment.

New Documented State

The engineering handbook now includes:

- knowledge lifecycle,
- documentation automation guide,
- formal KIA standard,
- self-evolving workflow,
- documentation dependency map,
- AI coding workflow,
- future automation roadmap.

This completes the KMS v1.0 lifecycle foundation.

Reason For Change

Future implementation work needs a repeatable lifecycle that keeps KMS synchronized with software changes without automatic source-doc rewriting.

User / Business Impact

Improves long-term governance, reviewer confidence, AI coding discipline, and continuity across future phases.

Technical Impact

Documentation and PDF generation only. No SaaS flow, landing code, database model, migration, `tis.db`, app route, or runtime behavior changed.

Documentation Updated

- `docs/engineering/KNOWLEDGE_LIFECYCLE.md`

1 `docs/engineering/DOCUMENTATION_AUTOMATION.md`

1 `docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md`

1 `docs/engineering/SELF_EVOLVING_WORKFLOW.md`

1 `docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md`

1 `docs/engineering/AI_CODING_WORKFLOW.md`

1 `docs/engineering/FUTURE_AUTOMATION_ROADMAP.md`

1 core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Review the final handbook for readability. Future improvements should be handled as new approved KMS phases.

Production Memory Stability Guardrails

docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md

2026-06-27 - Production Memory Stability Guardrails

Module: Engineering Handbook and operational app stability

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-27 - Added Production Memory Stability Guardrails

Related ADRs: None. This is a standards and operational guardrail update, not a new architecture decision.

Reviewer/approval notes: Prepared after a Render restart/502 investigation. Local changes only; no commit, push, migration, database change, or deployment was performed.

Previous Documented State

The KMS required scoped changes, tenant isolation, validation, and documentation updates, but it did not explicitly define a production memory budget or prohibit common memory hazards such as:

- full large-dataset parsing for normal picker requests,
- duplicate template rendering in production routes,
- warning-level diagnostic stage logs during normal traffic,
- unbounded global caches,
- startup-heavy work without memory review.

New Documented State

`docs/engineering/DEVELOPMENT_STANDARDS.md` now includes a mandatory Production Memory and Render Stability section.

The standards require future work to:

- treat 512 MB Render memory as a hard production budget,
- avoid full large-dataset loads when scoped or streaming lookup is available,
- avoid duplicate template renders in production request paths,
- keep diagnostics opt-in through explicit environment flags,
- filter data before materializing result sets,
- keep caches bounded and justified,
- include memory/performance smoke checks for memory-sensitive changes.

Reason For Change

The 2026-06-27 production investigation found avoidable memory pressure risks after deployment:

- `/observations/` had diagnostic logging and extra template pre-renders in the normal route path.
- Global location lookup could parse a 47 MB country/state/city reference dataset into a complete in-memory index for simple picker API calls.

These patterns can trigger process restarts and temporary 502 responses on constrained production instances.

User / Business Impact

The rule reduces the risk that normal navigation causes Render restarts, temporary 502s, or user-visible instability. It also creates a clear review standard for future SaaS onboarding, dashboard, reports, observations, and large-data changes.

Technical Impact

Future changes that add broad data loading, reference datasets, exports, diagnostics, or startup work must include memory-conscious design and validation. The local stabilization patch also reduces current risk by gating observation diagnostics and changing location lookup to scoped loading.

Documentation Updated

1 `docs/engineering/DEVELOPMENT_STANDARDS.md`

1 `docs/PROJECT_STATE.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/CHANGE_HISTORY.md`

PDF Regenerated

Yes

Follow-Up Needed

Review, commit, and deploy the stabilization changes when approved. After deployment, monitor Render memory, restart count, warning logs, and 502 frequency.

Automatic KMS Synchronization Enforcement

docs/history/engineering-handbook/2026-07-21-kms-automatic-enforcement.md

2026-07-21 - Automatic KMS Synchronization Enforcement

Previous State

KMS governance was documented but not mechanically enforced. Developers and AI assistants manually decided impact, edited Markdown, and regenerated artifacts. The Knowledge Center could detect stale hashes, but no pull-request or deployment gate consumed that status.

New State

TIS has a narrow repository enforcement layer:

1 `AGENTS.md` makes KMS onboarding and completion rules default for Codex.

1 `.kms-impact.yml` records task-level impact, affected areas, updated KMS files, and controlled no-impact overrides.

1 `scripts/check_kms_impact.py` compares the declaration with a Git range or local worktree, classifies likely major changes, and validates generated artifacts.

1 `scripts/generate_docs_pdf.py --check` verifies source coverage, source hashes, PDF hash/size, version, and manifest consistency without writing.

1 GitHub Actions run enforcement for pull requests and `dev`; production deployment from `master` depends on the same gate.

Cross-Platform Correction

The initial enforcement release exposed two checkout-dependent differences: Markdown hashes reflected raw CRLF/LF bytes, and dynamic source ordering followed native path comparison. The generator and Knowledge Center now hash normalized UTF-8/LF Markdown. The generator also records repository-relative POSIX paths in a deterministic order while continuing to fail on genuine source-list drift. Impact detection includes deleted files so removing behavior or documentation cannot bypass review.

Task-Boundary Correction

The initial push workflow compared only the previous pushed commit with the new head, while pull requests compared the complete feature branch. Follow-up commits therefore validated a task-level declaration against a commit-level diff. Push checks now derive the task base from the merge base of the default branch and pushed head. Pull-request and push checks consequently enforce the same cumulative KIA scope without allowing undeclared documentation changes.

Phase 6 Unified Command

The initial automation exposed generation, artifact checking, and KIA enforcement as separate commands. Phase 6 adds `scripts/kms.py sync` for KIA preflight, PDF/manifest generation, post-generation freshness validation, and completion reporting. `scripts/kms.py check` is the canonical read-only command used locally and by CI. Both commands delegate to the existing generator and checker; they do not duplicate policy or rewrite Markdown.

Guardrails

Automation does not generate or rewrite Markdown prose. Reviewed Markdown remains authoritative. KMS files must describe engineering design only and must not contain customer, organization, personal, transaction, invoice, production identifier, webhook payload, secret, credential, environment, or database-row data.

Related Files

```
AGENTS.md
.kms-impact.yml
scripts/check_kms_impact.py
scripts/generate_docs_pdf.py
scripts/kms.py
.github/workflows/kms-enforcement.yml
.github/workflows/deploy-on-master.yml
docs/DOCUMENTATION_UPDATE_POLICY.md
```

KMS Phase 7A Navigation Foundation

docs/history/engineering-handbook/2026-07-21-kms-phase-7a-navigation-foundation.md

2026-07-21 - KMS Phase 7A Navigation Foundation

Previous State

The KMS contained comprehensive core, engineering, ADR, history, marketing, and supporting documentation. Root and engineering README files listed those sources, but readers still had to assemble their own task-specific reading sequence. Three supporting documents also lacked standard title front matter.

New State

`docs/KMS_NAVIGATION.md` is the canonical role-based and task-based reading guide. It directs human developers, AI assistants, owners, and reviewers to focused document sets with explicit guardrails. Root and engineering indexes now use navigable Markdown links and point to the guide instead of duplicating every reading decision. The location roadmap and both landing-page documents have normalized title metadata.

Scope Boundary

Phase 7A changes reviewed Markdown navigation only. It does not add generated catalog metadata, PDF table-of-contents behavior, PDF bookmarks, Knowledge Center search/filtering, protected document-reader routes, or new KMS enforcement rules.

Related Files

1 `docs/KMS_NAVIGATION.md`

1 `docs/README.md`

1 `docs/engineering/README.md`

1 `docs/location-data-roadmap.md`

1 `docs/marketing/landing_page_source_of_truth.md`

1 `docs/marketing/tis_landing_page_master_content.md`

KMS Phase 7B Professional PDF Navigation

docs/history/engineering-handbook/2026-07-21-kms-phase-7b-pdf-navigation.md

2026-07-21 - KMS Phase 7B Professional PDF Navigation

Previous State

The generated booklet preserved all approved Markdown sources in fixed order and provided page footers, but readers had to move through a long linear document. The manifest tracked source freshness without recording where each source began in the PDF.

New State

The existing ReportLab generator now provides:

- a dedicated "How to Use This Handbook" page,
- a multi-pass table of contents with source-document page numbers,
- stable named destinations and outline entries for every source document,
- child outline entries for H2 major headings,
- each source document's starting `pdf_page` in the generated manifest,
- strict validation for missing, invalid, or non-increasing source pages.

Bookmark identifiers are derived deterministically from repository-relative source paths and heading context. Existing source ordering, normalized Markdown hashing, source-list comparison, PDF identity checks, and freshness validation remain in force.

Scope Boundary

Phase 7B does not change the Platform Owner Knowledge Center UI, add routes, add dependencies, alter Markdown authority, modify application behavior, or introduce database changes. Phase 7C remains separate.

Related Files

- `scripts/generate_docs_pdf.py`
- `tests/test_kms_automation.py`
- `static/docs/TIS_Project_Reference_Booklet.pdf`
- `static/docs/docs_manifest.json`

KMS Phase 7D Navigation And Catalog Enforcement

docs/history/engineering-handbook/2026-07-22-kms-phase-7d-navigation-catalog-enforcement.md

2026-07-22 - KMS Phase 7D Navigation And Catalog Enforcement

Module: KMS governance and engineering handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-07-22 - Added Phase 7D Navigation And Catalog Enforcement

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for Phase 7D only. Knowledge Center UI, routes, application workflows, database, dependencies, commits, and pushes remain out of scope.

Previous Documented State

KMS enforcement validated KIA declarations, authoritative source coverage, normalized source hashes, deterministic manifest source ordering, generated PDF identity, and positive increasing source page metadata. Phase 7A added task navigation, Phase 7B added PDF navigation, and Phase 7C added the owner catalog, but their title, taxonomy, navigation-target, and real PDF page-bound assumptions were not fully enforced by `scripts/kms.py check`.

New Documented State

The shared dependency-free `kms_catalog.py` defines approved categories, modules, and path classification for both Knowledge Center presentation and validation. The existing read-only check now requires usable source titles, approved taxonomy, exact normalized manifest/source-list equality, valid docs-only navigation links to listed Markdown, and positive increasing source pages that do not exceed the generated PDF page count. Existing hash, freshness, ordering, source coverage, KIA, and artifact checks remain active.

Reason For Change

Navigation and catalog quality must fail during the same local and CI command that already protects KMS freshness, preventing broken onboarding paths or unusable owner metadata from reaching integration.

User / Business Impact

Platform owners, developers, and AI assistants receive a more dependable catalog and navigation guide. Invalid knowledge metadata is blocked before review or deployment rather than discovered while reading the handbook.

Technical Impact

The generator/checker uses dependency-free Markdown metadata and link parsing plus ReportLab PDF page-object counting. Focused tests cover valid and invalid titles, taxonomy, relative links, missing/unlisted targets, source-list drift, and PDF page metadata. The Knowledge Center retains the same routes, UI, permissions, and read-only behavior.

Documentation Updated

1 `docs/DOCUMENTATION_UPDATE_POLICY.md`

1 `docs/KMS_NAVIGATION.md`

1 `docs/PROJECT_STATE.md`

1 `docs/CHANGE_HISTORY.md`

1 `docs/engineering/DOCUMENTATION_AUTOMATION.md`

1 `docs/engineering/REPOSITORY_ARCHITECTURE.md`

1 `docs/history/engineering-handbook/README.md`

1 `docs/history/engineering-handbook/2026-07-22-kms-phase-7d-navigation-catalog-enforcement.md`

PDF Regenerated

Yes

Follow-Up Needed

Monitor real pull requests for demonstrated false positives. Any future taxonomy expansion must update the shared catalog, relevant docs, and focused tests together.

Claude Code KMS Configuration

docs/history/engineering-handbook/2026-09-03-claude-code-kms-configuration.md

Previous state

TIS had repository-wide `AGENTS.md`, authoritative KMS onboarding, and automated KIA enforcement, but no repository-native Claude Code entry point, project skill, or specialized subagent definition.

New state

Root `CLAUDE.md` directs Claude Code to the existing repository and KMS authorities. `.claude/skills/tis-kms/SKILL.md` packages the reusable TIS investigation, implementation, validation, KIA, synchronization, and reporting workflow. `.claude/agents/tis-kms-developer.md` provides the same safeguards for bounded delegated work.

The configuration preserves existing worktree changes, tenant and academic scope, permissions, publication history, audit boundaries, database safety, and the no-commit/no-push default. It does not introduce a competing architecture source or change application, schema, migration, customer, production, or deployment behavior.

Validation contract

Claude Code must run proportional implementation checks, `git diff --check`, KMS sync after authoritative Markdown changes, and final read-only KMS validation. Its completion report includes exact KMS files and the Knowledge Impact Assessment.

Cline KMS Configuration And Three-Agent Orchestration

docs/history/engineering-handbook/2026-09-05-cline-kms-configuration.md

Previous State

TIS had AGENTS.md, KMS enforcement, Codex onboarding, and Claude Code's repository entry point, skill, and subagent. It lacked native Cline project configuration.

New State And Reason

User-approved Cline rules at `.clinerules/tis.md` and the project skill at `.cline/skills/tis-kms-developer/SKILL.md` reuse the existing KMS authority. They require investigation before assumptions, minimal changes, isolation/RBAC, database/worktree preservation, destructive dependency/transaction checks, proportional tests, KIA, sync/check, and explicit release authorization.

The master conversation balances Codex, Claude Code, and Cline/ClinePass by complexity, specialization, risk, review independence, and current usage/load. Each is a capable developer; preferred tendencies do not create rigid ownership. See [Project Governance](../../engineering/PROJECT_GOVERNANCE.md) and [AI Coding Workflow](../../engineering/AI_CODING_WORKFLOW.md) for policy and usage.

Technical And Deployment Impact

Development configuration/documentation only: no application, schema, migration, database, production, or worker-contract change. Deployment impact: none. Shared solver/worker changes in future tasks require Web + Workflow from the same commit; web-only router/template/state changes require Web only when the worker contract is unchanged.

Knowledge And Validation

AI context, master context, project state, change history, engineering workflow, governance, and this module history record the change. The then-existing M9 KIA state was preserved without modification as part of this configuration-only task. Sync regenerates PDF/manifest from all current KMS Markdown; no generated artifact is manually edited. No new ADR is needed: this extends the existing assistant integration under ADR 0006 without replacing KMS authority, approval gates, or deployment architecture.

Validate skill metadata and referenced paths, KMS sync/check, existing focused KMS automation tests, and git diff --check. Live Cline discovery/toggles must be distinguished from static file validation; setup is documented in the workflow.

M10 B11-C PostgreSQL Profiling Evidence Remediation

docs/history/engineering-handbook/2026-09-07-b11c-postgresql-profiling-evidence-remediation.md

2026-09-07 - M10 B11-C PostgreSQL Profiling Evidence Remediation

Module: Talent & Potential M10 Organization Analytics, PostgreSQL performance qualification (ADR 0028)

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-09-07 - M10 B11-C PostgreSQL Profiling Evidence Remediation

Related ADRs: `docs/adr/0028-b11-production-qualification-policy.md` (governing authority for B11-C entry, evidence requirements, and release gates)

Reviewer/approval notes: The evidence-remediation work did not change M10 application behavior, does not add an index, does not add a migration, does not change PostgreSQL isolation, and does not choose any production threshold, breadth limit, SLO, memory ceiling, or commercial mapping. B11-C is NOT declared CLOSED by the evidence-gathering pass itself. Its subsequent independent re-review returned **PASS WITH NON-BLOCKING OBSERVATIONS**, so the governed closure checkpoint marks B11-C **CLOSED**. No merge or deployment occurred. `tis.db` was not touched; all measurement used the dedicated non-production `tis_b11c_test_db` PostgreSQL 16.15 instance at `127.0.0.1` supplied via the `TIS_TEST_POSTGRESQL_URL` environment variable (never printed, logged, or persisted anywhere in this repository).

Why This Document Exists

An earlier PostgreSQL profiling pass for M10 B11-C did execute successfully and produced real findings (route profiling, query-count/Seq-Scan/memory observations, longitudinal bounded-query behavior, and M10 regression evidence), but its harness/output was temporary and left no durable, independently reviewable repository artifact. Independent B11-C review therefore returned FAIL - not because profiling never happened, but because its evidence could not be verified after the fact.

This document re-measures the same evidence end to end against a real, dedicated, non-production PostgreSQL 16.15 database, using a reproducible synthetic dataset builder and a reproducible profiling procedure (both fully reproduced below), and preserves the raw findings durably so an independent reviewer can verify every conclusion without relying on a chat transcript.

Current Milestone Truth

1 B0-B10: CLOSED (per `docs/adr/0027-...md` and prior committed KMS state).

1 B11-A: complete (read-only qualification).

1 B11-B: CLOSED (operational observability).

1 B11 production-qualification policy (ADR 0028): APPROVED/GOVERNED.

1 B11-C: PostgreSQL profiling **executed**; durable evidence **captured** in

this document. Independent re-review: **PASS WITH NON-BLOCKING OBSERVATIONS**. B11-C is **CLOSED**.

1 No index approved. No production SLO approved. No production privacy

threshold approved. No production breadth limit approved. No production memory ceiling approved. No PostgreSQL isolation change.

1 B11-D: NOT IMPLEMENTED. B11-E: NOT COMPLETE. B11 overall: NOT CLOSED.

1 B12: NOT IMPLEMENTED. Production readiness: NOT achieved.

Phase 1 - Connection Verification (Non-Production)

Verified programmatically, without ever printing the credential/URL:

| Check | Result | |---|---| | `TIS_TEST_POSTGRESQL_URL` present in process environment | Yes | | Connection succeeds | Yes | | `SELECT version()` | `PostgreSQL 16.15, compiled by Visual C++ build 1944, 64-bit` | | `SELECT current_database()` | `tis_b11c_test_db` | | `SELECT inet_server_addr()` | `127.0.0.1` (localhost only) | | Environment | Non-production (dedicated `tis_b11c_test_db` / `tis_b11c_test_role`, localhost-only) |

Methodology (Reproducible)

Environment. Local, non-production PostgreSQL 16.15 (`tis_b11c_test_db`, `127.0.0.1`), reached only via `TIS_TEST_POSTGRESQL_URL`. All profiling ran inside a dedicated PostgreSQL **schema per scale** (`tis_b11c_evidence_<run id>_<scale>`), created fresh and dropped (`DROP SCHEMA ... CASCADE`) at the end of every run - no rows were left behind in the shared test database.

Schema. `models.Base.metadata.create_all(engine)` against the scoped schema (via `search_path`), using the exact same SQLAlchemy models the application uses (`models.py` / `database.Base`) - no invented schema.

Application under test. The real M10 routers/services (`routers/talent_organization_analytics.py`, `talent_org_intelligence_service.py`, `talent_org_talent_map.py`, `talent_org_b7.py`, `talent_org_participation_overlap.py`, `talent_org_student_drill.py`, `talent_org_longitudinal.py`) mounted on an isolated `FastAPI()` app and driven with `fastapi.testclient.TestClient`, exactly like the existing `tests/test_talent_organization_*.py` suite wires routes, with `dependency_overrides` for `get_db` (bound to the profiling session/engine), `get_current_user` (a synthetic `ORGANIZATION`-scope actor), and the three existing non-production provider seams.

Non-production provider configuration class (Phase 3). Explicitly labeled **NON-PRODUCTION PROFILING CONFIGURATION**, matching the existing test-double pattern already used by `tests/test_talent_org_intelligence_queries.py` and `tests/test_talent_organization_overview.py` (`AllowAvailability`, `AllowBreadth`, `talent_analytics_privacy.AllowAllTestPolicy`). No production provider exists yet (ADR 0028); these test doubles always allow, so measured latency/queries reflect the SQL, ORM, relationship-graph, and serialization work that exists in the codebase today, not a future production threshold/breadth decision.

Query counting. A temporary SQLAlchemy `before_cursor_execute` engine event listener (`QueryCounter`), registered only for the duration of one `client.get(...)` call and removed (`event.remove`) immediately after. It counts and stores `(statement, parameters)` pairs; it has no permanent application effect and was never left registered outside the measurement window.

Latency. `time.perf_counter()` wrapped tightly around `client.get(...)`, for a "first" (cold-ish) and a "second" (warm) call per route per scale.

Memory. `psutil.Process(os.getpid()).memory_info().rss`, sampled after `gc.collect()`, at fixed checkpoints: process baseline (post-import, pre-DB), before/after schema creation, before/after synthetic data build, and pre/post-first-call/post-second-call for every route. **Classified explicitly as DIRECTIONAL LOCAL WINDOWS DEVELOPMENT EVIDENCE - not a production memory ceiling.** All three scales ran sequentially inside one Python process (see Known Limitations); absolute RSS values are cumulative across scales, not independent per-scale baselines.

EXPLAIN. For each route, up to three distinct `SELECT` statement shapes captured during the "first" call were re-executed as `EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) <statement>` on the raw DBAPI (`psycopg2`)

cursor, using the exact captured driver-level parameters, then rolled back (read-only). Statements touching the population/aggregation tables or containing `GROUP BY` were preferred over small reference-table lookups when more than three shapes were captured.

Synthetic data. Seeded `random.Random(20260907)` for reproducibility. No production data, fixtures, or database export was used or imported.

Regression. Standard `pytest` against the existing M10 suites (see Regression section below) - unrelated to the PostgreSQL harness.

Exact Reproduction Procedure

1. Set `TIS_TEST_POSTGRESQL_URL` to a non-production PostgreSQL connection string (never a production database).
1. Run the harness below with `<repo>/.venv/Scripts/python.exe`. It requires `psutil` (`pip install psutil` into the venv if not already present - a local measurement-only dependency, not added to `requirements.txt`).
1. The harness prints per-scale dimensions to stdout and writes a JSON results file next to itself; it creates and drops its own schemas and never touches `tis.db` or any table outside the schema it created.

The harness was **not** committed to the repository (kept as ephemeral profiling tooling per the Profiling Harness scope-minimization guidance); its full source is reproduced verbatim below so an independent reviewer can re-run it without this conversation:

```
"""B11-C PostgreSQL profiling harness (NON-PRODUCTION, evidence remediation).

Not committed to the repository. Requires TIS_TEST_POSTGRESQL_URL. Creates and
drops its own dedicated PostgreSQL schema inside the target test database.
Never targets production. Has no application runtime effect (the app itself
is only exercised in-process via FastAPI TestClient against the ephemeral
schema created by this script).

Run: <repo>/.venv/Scripts/python.exe b11c_profile.py [SCALE ...]
"""
from __future__ import annotations

import gc
import json
import os
import random
import sys
import time
import uuid
from collections import defaultdict
from datetime import datetime

REPO = r"C:\Development\Teacher-Information-System--TIS"
sys.path.insert(0, REPO)
os.chdir(REPO)

import psutil
import sqlalchemy as sa
from sqlalchemy import event, text
from sqlalchemy.orm import sessionmaker
from fastapi import FastAPI
from fastapi.testclient import TestClient

import models
from database import Base
```

```
from auth import get_current_user
from dependencies import get_db
from talent_analytics_privacy import AllowAllTestPolicy,
resolve_privacy_policy_provider
from talent_org_intelligence_service import (
    OrganizationAnalyticsAvailabilityProvider,
    OrganizationAnalyticsBreadthPolicy,
    resolve_organization_analytics_availability_provider,
    resolve_organization_analytics_breadth_policy,
)
from routers import talent_organization_analytics as route

POSTGRESQL_URL = os.environ.get("TIS_TEST_POSTGRESQL_URL", "")
assert POSTGRESQL_URL.startswith("postgresql"), "TIS_TEST_POSTGRESQL_URL
missing/invalid"

RUN_ID = uuid.uuid4().hex[:10]
RESULTS = {"run_id": RUN_ID, "scales": {}}
PROCESS = psutil.Process(os.getpid())


def rss_mb():
    gc.collect()
    return PROCESS.memory_info().rss / (1024 * 1024)


RESULTS["baseline_rss_mb"] = round(rss_mb(), 2)


class AllowAvailability(OrganizationAnalyticsAvailabilityProvider):
    availability_version = "b11c-profiling-nonprod-v1"

    def is_available(self, **_ctx):
        return True


class AllowBreadth(OrganizationAnalyticsBreadthPolicy):
    breadth_policy_version = "b11c-profiling-nonprod-v1"

    def allows(self, **_shape):
        return True


# ---------------------------------------------------------------------------
# Scale configuration (NON-PRODUCTION PROFILING CONFIGURATION)
# ---------------------------------------------------------------------------
SCALES = {
    "SMALL": dict(branches=2, programs=2, focus_periods=3, students_per_cycle=15,
other_cycle_students=20,
                  candidate_rate=0.4, identify_rate=0.5, overlap_students=8),
    "MEDIUM": dict(branches=4, programs=6, focus_periods=5, students_per_cycle=150,
other_cycle_students=150,
                   candidate_rate=0.35, identify_rate=0.5, overlap_students=120),
    "LARGE": dict(branches=8, programs=12, focus_periods=8, students_per_cycle=1200,
other_cycle_students=1200,
                  candidate_rate=0.3, identify_rate=0.5, overlap_students=1000),
}

GRADES = [str(g) for g in range(1, 13)]


class Ids:
    def __init__(self):
        self._n = defaultdict(int)

    def next(self, kind):
        self._n[kind] += 1
        return self._n[kind]


def build_dataset(session, scale_name, cfg, rng):
```

```
    school_group_id = 1
    foreign_group_id = 2
    academic_year_id = 100
    foreign_ay = 200
    now = datetime(2026, 9, 1)

    session.add_all([
        models.SchoolGroup(id=school_group_id, name=f"B11C {scale_name} Primary"),
        models.SchoolGroup(id=foreign_group_id, name=f"B11C {scale_name} Foreign"),
    ])
    session.commit()
    branches = [10 + i for i in range(cfg["branches"])]
    for b in branches:
        session.add(models.Branch(id=b, school_group_id=school_group_id,
name=f"Branch{b}", status=True))
    foreign_branch = 900
    session.add(models.Branch(id=foreign_branch, school_group_id=foreign_group_id,
name="ForeignBranch", status=True))
    session.add_all([
        models.AcademicYear(id=academic_year_id, school_group_id=school_group_id,
year_name="2026-2027"),
        models.AcademicYear(id=foreign_ay, school_group_id=foreign_group_id,
year_name="2026-2027"),
    ])
    session.commit()

    program_ids = list(range(1, cfg["programs"] + 1))
    focus_program_id = program_ids[0]
    foreign_program_id = 9001
    for pid in program_ids:
        session.add(models.TalentProgram(id=pid, school_group_id=school_group_id,
name=f"Program{pid}", status="active"))
    session.add(models.TalentProgram(id=foreign_program_id,
school_group_id=foreign_group_id, name="ForeignProgram", status="active"))
    session.commit()

    for pid in program_ids:
        session.add(models.TalentProgramAcademicYearConfiguration(
            id=pid, school_group_id=school_group_id, program_id=pid,
academic_year_id=academic_year_id,
            is_enabled=True, eligible_grade_levels_csv=",".join(GRADES),
        ))
    session.add(models.TalentProgramAcademicYearConfiguration(
        id=9001, school_group_id=foreign_group_id, program_id=foreign_program_id,
academic_year_id=foreign_ay,
        is_enabled=True, eligible_grade_levels_csv="1",
    ))
    session.commit()

    fw_id = {}
    for pid in program_ids:
        fwid = 100000 + pid
        fw_id[pid] = fwid
        session.add(models.TalentProgramFrameworkVersion(
            id=fwid, school_group_id=school_group_id, program_id=pid,
version_number=1, status="active",
            title=f"Framework{pid}", revision=1,
semantic_fingerprint=f"{pid:064d}"[-64:],
        ))
    session.add(models.TalentProgramFrameworkVersion(
        id=900001, school_group_id=foreign_group_id, program_id=foreign_program_id,
version_number=1,
        status="active", title="ForeignFramework", revision=1,
semantic_fingerprint="f" * 64,
    ))
    session.commit()

    plan_id = {}
    for pid in program_ids:
        pid_plan = 500000 + pid
        plan_id[pid] = pid_plan
```

```
        session.add(models.TalentAnnualEvaluationPlan(
            id=pid_plan, school_group_id=school_group_id, program_id=pid,
academic_year_id=academic_year_id,
            program_academic_year_configuration_id=pid, status="active", revision=1,
activated_at=now,
        ))
    session.commit()

    # Focus program periods (for longitudinal profiling)
    period_ids = []
    for seq in range(1, cfg["focus_periods"] + 1):
        period_id = 600000 + seq
        period_ids.append(period_id)
        session.add(models.TalentPlannedEvaluationPeriod(
            id=period_id, school_group_id=school_group_id,
program_id=focus_program_id,
            academic_year_id=academic_year_id,
annual_evaluation_plan_id=plan_id[focus_program_id],
            sequence=seq, label=f"Period{seq}", normalized_label=f"period{seq}",
            is_required=True, status="planned",
        ))
    session.commit()

    # Cycles: focus program gets one cycle per period; every other program gets one
standalone cycle.
    cycle_meta = []  # (cycle_id, program_id, framework_version_id, period_id_or_None,
status)
    cid = 200000
    for i, period_id in enumerate(period_ids):
        cid += 1
        status = "closed" if i < len(period_ids) - 1 else "open"
        cycle_meta.append((cid, focus_program_id, fw_id[focus_program_id], period_id,
status))
    for pid in program_ids[1:]:
        cid += 1
        cycle_meta.append((cid, pid, fw_id[pid], None, "open"))
    for cycle_id, pid, fwid, period_id, status in cycle_meta:
        session.add(models.TalentAssessmentCycle(
            id=cycle_id, school_group_id=school_group_id, program_id=pid,
academic_year_id=academic_year_id,
            framework_version_id=fwid, planned_evaluation_period_id=period_id,
title=f"Cycle{cycle_id}",
            status=status, revision=1, population_effective_at=now,
        ))
    # Foreign tenant: one small cycle for isolation control.
    session.add(models.TalentAssessmentCycle(
        id=999001, school_group_id=foreign_group_id, program_id=foreign_program_id,
academic_year_id=foreign_ay,
        framework_version_id=900001, planned_evaluation_period_id=None,
title="ForeignCycle",
        status="open", revision=1, population_effective_at=now,
    ))
    session.commit()

    # Review candidate policy (required FK for TalentReviewCandidate rows).
    policy_id = {}
    for pid in program_ids:
        pol_id = 700000 + pid
        policy_id[pid] = pol_id
        session.add(models.TalentReviewCandidatePolicy(
            id=pol_id, school_group_id=school_group_id, program_id=pid,
framework_version_id=fw_id[pid],
            is_enabled=True, match_mode="all",
        ))
    session.commit()

    # Students + placements + population members + assessments
(+candidates/identifications).
    student_id_counter = [0]
    student_rows = []
    placement_rows = []
```

```
    def new_student(branch_id, grade, group=school_group_id, ay=academic_year_id):
        student_id_counter[0] += 1
        sid = student_id_counter[0]
        student_rows.append(dict(
            id=sid, school_group_id=group, first_name="Synthetic",
last_name=f"Student{sid}",
            status="active",
        ))
        placement_id = sid
        placement_rows.append(dict(
            id=placement_id, school_group_id=group, student_id=sid,
academic_year_id=ay,
            branch_id=branch_id, grade_level=grade, section_name="A",
effective_from=now, status="active",
        ))
        return sid, placement_id

    # Pre-create a shared overlap student pool reused across multiple programs'
cycles.
    overlap_pool = []
    member_id = 0
    assessment_id = 0
    candidate_id = 0
    identification_id = 0
    pop_rows = []
    assess_rows = []
    cand_rows = []
    ident_rows = []

    def make_population(cycle_id, pid, fwid, count, branch_cycle, reuse_pool=None,
pool_fraction=0.0):
        nonlocal member_id
        made = []
        pool_take = min(int(count * pool_fraction), len(reuse_pool)) if reuse_pool
else 0
        chosen_from_pool = rng.sample(reuse_pool, pool_take) if pool_take else []
        for i in range(count):
            if i < len(chosen_from_pool):
                # Reused student: population branch/grade MUST match their real
placement row.
                sid, placement_id, branch_id, grade = chosen_from_pool[i]
            else:
                branch_id = branch_cycle[i % len(branch_cycle)]
                grade = GRADES[i % len(GRADES)]
                sid, placement_id = new_student(branch_id, grade)
                if reuse_pool is not None:
                    reuse_pool.append((sid, placement_id, branch_id, grade))
            member_id += 1
            mid = member_id
            pop_rows.append(dict(
                id=mid, school_group_id=school_group_id, cycle_id=cycle_id,
program_id=pid,
                academic_year_id=academic_year_id, framework_version_id=fwid,
student_id=sid,
                academic_placement_id=placement_id, branch_id=branch_id,
grade_level=grade,
                section_name="A", population_effective_at=now, frozen_at=now,
            ))
            made.append((mid, sid))
        return made

    def make_assessments(members, cycle_id, pid, fwid, candidate_rate, identify_rate):
        nonlocal assessment_id, candidate_id, identification_id
        for mid, sid in members:
            status = rng.choices(
                ["completed", "in_progress", "incomplete", "unassessed"],
                weights=[0.55, 0.15, 0.1, 0.2],
            )[0]
            if status == "unassessed":
                continue
            assessment_id += 1
```

```
            aid = assessment_id
            assess_rows.append(dict(
                id=aid, school_group_id=school_group_id, cycle_id=cycle_id,
cycle_population_member_id=mid,
                student_id=sid, program_id=pid, academic_year_id=academic_year_id,
framework_version_id=fwid,
                status=status, revision=1, started_at=now,
                completed_at=now if status == "completed" else None,
            ))
            if status == "completed" and rng.random() < candidate_rate:
                candidate_id += 1
                cand_id = candidate_id
                cand_rows.append(dict(
                    id=cand_id, school_group_id=school_group_id, cycle_id=cycle_id,
                    cycle_population_member_id=mid, student_id=sid, program_id=pid,
                    academic_year_id=academic_year_id, framework_version_id=fwid,
assessment_id=aid,
                    policy_id=policy_id[pid], match_mode="all",
evaluation_fingerprint=f"{cand_id:064d}"[-64:],
                    evaluation_snapshot_json="{}", status="reviewed",
                ))
                if rng.random() < identify_rate:
                    identification_id += 1
                    ident_rows.append(dict(
                        id=identification_id, school_group_id=school_group_id,
cycle_id=cycle_id,
                        cycle_population_member_id=mid, student_id=sid,
program_id=pid,
                        academic_year_id=academic_year_id, framework_version_id=fwid,
assessment_id=aid,
                        review_candidate_id=cand_id, decision="identified",
                    ))

    # Focus program: each period's cycle carries forward ~70% of the previous period's
students.
    for cycle_id, pid, fwid, period_id, status in cycle_meta:
        if pid == focus_program_id:
            members = make_population(
                cycle_id, pid, fwid, cfg["students_per_cycle"], branches,
                reuse_pool=overlap_pool, pool_fraction=0.7 if cycle_id !=
cycle_meta[0][0] else 0.0,
            )
            make_assessments(members, cycle_id, pid, fwid, cfg["candidate_rate"],
cfg["identify_rate"])
    # Other programs: standalone cycle, partially reusing the same overlap pool
(participation overlap signal).
    other_pool_fraction = min(0.5, cfg["overlap_students"] /
max(cfg["other_cycle_students"], 1))
    for cycle_id, pid, fwid, period_id, status in cycle_meta:
        if pid != focus_program_id:
            members = make_population(
                cycle_id, pid, fwid, cfg["other_cycle_students"], branches,
                reuse_pool=overlap_pool, pool_fraction=other_pool_fraction,
            )
            make_assessments(members, cycle_id, pid, fwid, cfg["candidate_rate"],
cfg["identify_rate"])

    # Foreign tenant: tiny isolated population (not counted toward primary scale
numbers).
    fsid, fpid = new_student(foreign_branch, "1", group=foreign_group_id,
ay=foreign_ay)
    member_id += 1
    pop_rows.append(dict(
        id=member_id, school_group_id=foreign_group_id, cycle_id=999001,
program_id=foreign_program_id,
        academic_year_id=foreign_ay, framework_version_id=900001, student_id=fsid,
        academic_placement_id=fpid, branch_id=foreign_branch, grade_level="1",
section_name="A",
        population_effective_at=now, frozen_at=now,
    ))

    if student_rows:
```

```
        session.execute(sa.insert(models.Student), student_rows)
    if placement_rows:
        session.execute(sa.insert(models.StudentAcademicPlacement), placement_rows)
    if pop_rows:
        session.execute(sa.insert(models.TalentAssessmentCyclePopulationMember),
pop_rows)
    if assess_rows:
        session.execute(sa.insert(models.TalentStudentAssessment), assess_rows)
    if cand_rows:
        session.execute(sa.insert(models.TalentReviewCandidate), cand_rows)
    if ident_rows:
        session.execute(sa.insert(models.TalentOfficialIdentification), ident_rows)
    session.commit()

    dims = {
        "school_groups": 2, "branches": len(branches), "programs": len(program_ids),
        "focus_program_id": focus_program_id, "period_count": len(period_ids),
        "cycle_count": len(cycle_meta) + 1, "frozen_population_members":
len(pop_rows),
        "assessments": len(assess_rows), "candidates": len(cand_rows),
"identifications": len(ident_rows),
        "distinct_students": student_id_counter[0] + 1,
        "annual_plans": len(program_ids), "grades_used": len(GRADES),
        "program_pair_count": len(program_ids) * (len(program_ids) + 1) // 2,
    }
    return dict(
        school_group_id=school_group_id, academic_year_id=academic_year_id,
program_ids=program_ids,
        focus_program_id=focus_program_id, branches=branches, dims=dims,
    )


def actor_for(group, branch, ay):
    return models.User(
        user_id="pf0001", username="profiler", role="Editor", user_type="TENANT",
        access_scope="ORGANIZATION", school_group_id=group, branch_id=branch,
academic_year_id=ay,
        is_active=True,
    )


def grant_permissions(session, group):
    keys = [
        "talent_analytics.view", "talent_review_candidates.view",
"talent_official_identifications.view",
        "talent_analytics.view_students", "talent_learner_profiles.view",
    ]
    session.add_all(models.RolePermission(school_group_id=group, role="Editor",
permission_key=k, is_allowed=True) for k in keys)
    session.commit()


def make_client(session):
    app = FastAPI()
    app.include_router(route.router)
    app.dependency_overrides[get_db] = lambda: session
    app.dependency_overrides[get_current_user] = lambda: actor_for(1, 10, 100)
    app.dependency_overrides[resolve_organization_analytics_availability_provider] =
lambda: AllowAvailability()
    app.dependency_overrides[resolve_organization_analytics_breadth_policy] = lambda:
AllowBreadth()
    app.dependency_overrides[resolve_privacy_policy_provider] = lambda:
AllowAllTestPolicy()
    return TestClient(app)


class QueryCounter:
    def __init__(self, engine):
        self.engine = engine
        self.count = 0
        self.statements = []  # list[(statement, parameters)]
```

```
    def __enter__(self):
        self.count = 0
        self.statements = []
        event.listen(self.engine, "before_cursor_execute", self._before)
        return self

    def _before(self, conn, cursor, statement, parameters, context, executemany):
        self.count += 1
        self.statements.append((statement, parameters))

    def __exit__(self, *exc):
        event.remove(self.engine, "before_cursor_execute", self._before)


def profile_route(client, engine, method_path, label):
    with QueryCounter(engine) as qc:
        t0 = time.perf_counter()
        resp = client.get(method_path)
        t1 = time.perf_counter()
    return {
        "label": label, "path": method_path, "status": resp.status_code,
        "latency_ms": round((t1 - t0) * 1000, 2), "query_count": qc.count,
        "statements": qc.statements,
        "response_bytes": len(resp.content),
    }


def summarize_statement(stmt):
    return " ".join(stmt.split())[:220]


def run_scale(scale_name, cfg, admin_engine):
    schema = f"tis_b11c_evidence_{RUN_ID}_{scale_name.lower()}"
    with admin_engine.begin() as conn:
        conn.execute(text(f'DROP SCHEMA IF EXISTS "{schema}" CASCADE'))
        conn.execute(text(f'CREATE SCHEMA "{schema}"'))

    engine = sa.create_engine(
        POSTGRESQL_URL,
        connect_args={"connect_timeout": 10, "options": f"-c search_path={schema} -c
statement_timeout=120s"},
    )
    rss_before_schema = rss_mb()
    Base.metadata.create_all(engine)
    rss_after_schema = rss_mb()

    Session = sessionmaker(bind=engine)
    session = Session()
    rng = random.Random(20260907)
    rss_before_data = rss_mb()
    t0 = time.perf_counter()
    dataset = build_dataset(session, scale_name, cfg, rng)
    build_seconds = time.perf_counter() - t0
    rss_after_data = rss_mb()
    grant_permissions(session, dataset["school_group_id"])

    client = make_client(session)
    ay = dataset["academic_year_id"]
    focus = dataset["focus_program_id"]
    branch0 = dataset["branches"][0]

    routes = [
        ("overview", f"/api/talent/organization-
analytics/overview?academic_year_id={ay}"),
        ("talent_map_branch", f"/api/talent/organization-analytics/talent-
map?academic_year_id={ay}&dimension=program_branch&metric=completion_coverage"),
        ("program_portfolio", f"/api/talent/organization-analytics/program-
portfolio?academic_year_id={ay}"),
        ("branch_intelligence", f"/api/talent/organization-
analytics/branches/{branch0}?academic_year_id={ay}"),
        ("participation_overlap", f"/api/talent/organization-analytics/participation-
```

```
overlap?academic_year_id={ay}"),
        ("student_drill_25", f"/api/talent/organization-
analytics/students?academic_year_id={ay}&limit=25"),
        ("student_drill_100", f"/api/talent/organization-
analytics/students?academic_year_id={ay}&limit=100"),
        ("longitudinal", f"/api/talent/organization-analytics/programs/{focus}/longitu
dinal?academic_year_id={ay}&metric=completion_coverage"),
    ]

    results = {}
    explain_samples = {}
    for label, path in routes:
        gc.collect()
        rss_pre = rss_mb()
        first = profile_route(client, engine, path, label)
        rss_post_first = rss_mb()
        second = profile_route(client, engine, path, label)
        rss_post_second = rss_mb()
        results[label] = {
            "first_call": {k: v for k, v in first.items() if k != "statements"},
            "second_call": {k: v for k, v in second.items() if k != "statements"},
            "rss_pre_mb": round(rss_pre, 2), "rss_post_first_mb":
round(rss_post_first, 2),
            "rss_post_second_mb": round(rss_post_second, 2),
        }
        seen = {}
        for stmt, params in first["statements"]:
            if not stmt.strip().upper().startswith("SELECT"):
                continue
            key = summarize_statement(stmt)
            seen.setdefault(key, (stmt, params))
        ranked = sorted(
            seen.values(),
            key=lambda sp: (
                0 if ("talent_assessment_cycle_population_members" in sp[0] or "GROUP
BY" in sp[0].upper()) else 1,
                -len(sp[0]),
            ),
        )
        explain_samples[label] = ranked[:3]

    explain_results = {}
    raw_conn = engine.raw_connection()
    try:
        cur = raw_conn.cursor()
        for label, stmts in explain_samples.items():
            explain_results[label] = []
            for stmt, params in stmts:
                try:
                    cur.execute(f"EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) {stmt}",
params)
                    plan_text = "\n".join(row[0] for row in cur.fetchall())
                    raw_conn.rollback()
                except Exception as exc:
                    plan_text = f"<EXPLAIN skipped: {exc.__class__.__name__}: {exc}>"
                    raw_conn.rollback()
                explain_results[label].append({"sql_shape": summarize_statement(stmt),
"plan": plan_text})
    finally:
        raw_conn.close()

    session.close()
    engine.dispose()
    with admin_engine.begin() as conn:
        conn.execute(text(f'DROP SCHEMA IF EXISTS "{schema}" CASCADE'))

    return {
        "dims": dataset["dims"], "build_seconds": round(build_seconds, 2),
        "rss_before_schema_mb": round(rss_before_schema, 2), "rss_after_schema_mb":
round(rss_after_schema, 2),
        "rss_before_data_mb": round(rss_before_data, 2), "rss_after_data_mb":
```

```
round(rss_after_data, 2),
        "routes": results, "explain": explain_results,
    }


def main():
    global SCALES
    if len(sys.argv) > 1:
        SCALES = {k: v for k, v in SCALES.items() if k in sys.argv[1:]}
    admin_engine = sa.create_engine(POSTGRESQL_URL, connect_args={"connect_timeout":
10})
    with admin_engine.connect() as conn:
        version = conn.execute(text("SELECT version()")).scalar()
        dbname = conn.execute(text("SELECT current_database()")).scalar()
        host = conn.execute(text("SELECT inet_server_addr()")).scalar()
    RESULTS["postgresql_version"] = version
    RESULTS["database"] = dbname
    RESULTS["host"] = str(host)

    for scale_name, cfg in SCALES.items():
        print(f"=== Running scale {scale_name} ===", flush=True)
        RESULTS["scales"][scale_name] = run_scale(scale_name, cfg, admin_engine)
        print(f"=== Done {scale_name}: dims={RESULTS['scales'][scale_name]['dims']}
===", flush=True)

    admin_engine.dispose()
    RESULTS["final_rss_mb"] = round(rss_mb(), 2)

    out_path = os.path.join(os.path.dirname(__file__), "b11c_profile_results.json")
    with open(out_path, "w", encoding="utf-8") as f:
        json.dump(RESULTS, f, indent=2, default=str)
    print(f"Wrote {out_path}")


if __name__ == "__main__":
    main()
```

Dataset Dimensions (Exact, As Generated)

All three scenarios share one `SchoolGroup` (primary tenant, id 1) plus a second, tiny, isolated foreign `SchoolGroup` (id 2, one Branch, one Program, one Student) used only as a tenant-isolation control - its rows are excluded from every count below. `focus_program_id` is the one Program that carries an Annual Evaluation Plan with multiple ordered Planned Evaluation Periods (each backed by its own frozen Assessment Cycle) for Longitudinal profiling; every other Program has exactly one standalone open Assessment Cycle. All grades 1-12 are used (`grades_used=12`); Sections are uniformly `"A"` (a stated simplification - see Known Limitations).

Dimension	SMALL	MEDIUM	LARGE
SchoolGroups (primary + foreign control)	2	2	2
Branches (primary tenant)	2	4	8
Programs (primary tenant)	2	6	12
Program pairs `P(P+1)/2` (incl. self-pairs, matches route's `pair_count`)	3	21	78
Annual Evaluation Plans	2	6	12
Planned Evaluation Periods (focus Program only)	3	5	8
Assessment Cycles (incl. 1 foreign-tenant cycle)	5	11	20
Frozen Cycle population members	66	1,501	22,801
Distinct Students (primary tenant, incl. cross-Program overlap reuse)	39	707	10,322
Talent Student Assessments	47	1,178	18,095
Review Candidates	21	314	3,836
Official Identifications	11	154	1,938

`student_drill_100` (limit=100) and `student_drill_25` (limit=25) were both profiled against the same population per scale.

Route Coverage Confirmation

All seven governed M10 Organization Intelligence routes were profiled (plus Student Drill at both `limit=25` and `limit=100`), all returning HTTP 200 at every scale with the non-production allow-all providers:

```
GET /api/talent/organization-analytics/overview
```

```
GET /api/talent/organization-analytics/talent-map (dimension=program_branch)
GET /api/talent/organization-analytics/program-portfolio
GET /api/talent/organization-analytics/branches/{branch_id}
GET /api/talent/organization-analytics/participation-overlap
GET /api/talent/organization-analytics/students (limit=25 and limit=100)
GET /api/talent/organization-analytics/programs/{program_id}/longitudinal
```

No profiling API route exists or was added.

Query Count Per Route (SMALL / MEDIUM / LARGE)

| Route | SMALL | MEDIUM | LARGE | Grows with scale? | |---|---:|---:|---:|---| | overview | 10 | 10 | 10 | No | | talent_map (program_branch) | 9 | 9 | 9 | No | | program_portfolio | 11 | 11 | 11 | No | | branch_intelligence | 11 | 11 | 11 | No | | participation_overlap | 8 | 8 | 8 | No | | student_drill (limit=25) | 11 | 11 | 11 | No | | student_drill (limit=100) | 11 | 11 | 11 | No | | longitudinal | 11 | 11 | 11 | No |

Query count was **identical across all three scales for every route**, despite Programs growing 2->6->12, frozen population growing 66->1,501-> 22,801, Periods (longitudinal) growing 3->5->8, and Program pairs (participation-overlap) growing 3->21->78. Every route's query count was also identical between the "first" and "second" call at every scale (no lazy-load/cache-population difference on repeat).

N+1 determination: confirmed **no per-Student query growth**, **no per-Program query growth**, **no longitudinal query-per-Period growth**, and **no N+1** for all seven routes/eight profiled variants, at the tested scales, from direct query-count instrumentation (not static code inspection alone).

Route Latency (Milliseconds, First/Second Call)

Local, non-production, single-process, single-connection `TestClient` latency (SQL + ORM + relationship-graph privacy closure + serialization, all in-process - no network hop to a separate web server). **No p95/p99 production target exists; this is not an SLO measurement.**

| Route | SMALL 1st/2nd | MEDIUM 1st/2nd | LARGE 1st/2nd | |---|---:|---:|---:| | overview | 44.1 / 13.5 | 66.4 / 19.7 | 149.1 / 79.7 | | talent_map (program_branch) | 15.2 / 11.9 | 34.5 / 30.5 | 57.3 / 62.2 | | program_portfolio | 16.2 / 15.6 | 26.1 / 29.5 | 121.1 / 130.4 | | branch_intelligence | 17.8 / 17.0 | 47.0 / 23.1 | 85.4 / 52.8 | | participation_overlap | 19.6 / 17.6 | **175.8 / 154.4** | **39,486.9* / 57.9** | | student_drill (limit=25) | 59.3 / 18.0 | 58.6 / 32.3 | 128.6 / 84.7 | | student_drill (limit=100) | 33.1 / 33.8 | 54.7 / 68.9 | 101.1 / 86.9 | | longitudinal | 33.5 / 18.9 | 21.1 / 22.4 | 47.6 / 27.1 |

* See "Participation Overlap - Cold-Start Latency Cliff (Root-Caused)" below

> this one first-call number is real, reproducible, and root-caused; it is

not representative of steady-state (post-`ANALYZE`) query cost, which was 34-90ms for the identical query at LARGE scale.

Memory / RSS Evidence

Classification: DIRECTIONAL LOCAL WINDOWS DEVELOPMENT EVIDENCE. Not a production memory ceiling. All three scales ran sequentially inside one Python process, so absolute RSS is cumulative (CPython's allocator does not always return freed heap pages to the OS between scales); the per-scale *deltas* are the meaningful signal, not the absolute numbers.

| Checkpoint | SMALL | MEDIUM | LARGE | |---|---:|---:|---:| | RSS before schema create (MB) | 94.4 | 103.7 | 113.8 | | RSS after schema create (MB) | 95.8 | 104.1 | 113.8 | | RSS before data build (MB) | 95.8 | 104.1 | 113.8 | | RSS after

data build (MB) | 100.0 | 113.4 | 132.8 | | Data-build delta (MB) | +4.2 | +9.3 | +19.0 | | RSS across all 16 route calls at this scale | 100.0 -> 103.7 | 113.4 -> 113.8 | 132.9 -> 133.2 |

Process baseline (post-import of FastAPI/SQLAlchemy/psycopg2/the full M10 module set, pre-DB-work): **91.1 MB**. Final RSS after all three scales and 48 total route calls (8 routes x 2 calls x 3 scales): **133.0 MB**.

Findings:

1 Data-build RSS delta scales sub-linearly with row count (66 -> 1,501 ->

22,801 rows produced +4.2 MB -> +9.3 MB -> +19.0 MB), consistent with Python object churn during bulk-insert row-dict construction, not a per-row leak.

1 Within a scale, repeated route calls (`rss_pre` -> `rss_post_first` ->

`rss_post_second`) show **no material retention/unbounded growth** - the largest observed per-route creep was `student_drill_100` at LARGE scale (~0.2-0.3 MB across two calls returning a 37 KB JSON body), consistent with normal response-buffer allocation, not a leak pattern.

1 No evidence of unbounded/retained growth was observed at any tested scale

within this harness's window. This does **not** establish a concurrent- request or multi-process production memory ceiling (explicitly deferred to the B11-E release gate per ADR 0028).

Specifically requested focus areas:

1 Talent Map (LARGE, program x branch, 12 Programs x 8 Branches = 96 cells):

RSS flat at 132.9 -> 133.0 MB across both calls; response 15.7 KB.

1 Participation Overlap (LARGE, 78 pairs): RSS flat at 133.0 MB across both

calls (including the 39.5s cold-start call - the slowdown was CPU/I/O-bound in PostgreSQL, not a client-side memory blow-up); response 12.0 KB.

1 Student Drill limit=100 (LARGE): RSS 133.0 -> 133.1 -> 133.2 MB; response

37.7 KB - the largest per-route RSS delta observed, still small.

1 Longitudinal (LARGE, focus Program, 8 Periods): RSS flat at 133.2 MB;

response 3.9 KB.

EXPLAIN (ANALYZE, BUFFERS) Evidence

Representative sanitized excerpts (LARGE scale unless noted; full method in Methodology above). No bound parameter values are reproduced - only plan shape/cost/actual-time/row/buffer facts, which contain no Student-identifying or assessment-content data (only synthetic integer ids and structural counts).

Population access via frozen-membership (Index Candidate A path)

`count_distinct_students` (Student Drill gate query), LARGE:

```
Aggregate  (cost=346.73..346.74 rows=1 width=8) (actual time=7.476..7.478 rows=1
loops=1)
  Buffers: shared hit=397
  ->  Sort  (cost=345.46..346.10 rows=253 width=4) (actual time=5.993..6.528
rows=22800 loops=1)
        ->  Nested Loop  (cost=25.29..335.36 rows=253 width=4) (actual
time=0.056..3.861 rows=22800 loops=1)
              ->  Index Scan using ix_talent_assessment_cycles_scope on
```

```
talent_assessment_cycles
                     Index Cond: (school_group_id = 1)
                     Filter: ((status)::text = ANY ('{open,closed}'::text[]))
               ->  Bitmap Heap Scan on talent_assessment_cycle_population_members
                     Recheck Cond: (cycle_id = talent_assessment_cycles.id)
                     Filter: ((school_group_id = 1) AND (academic_year_id = 100) AND
(program_id = ANY (...)))
                     ->  Bitmap Index Scan on uq_talent_cycle_population_student
                           Index Cond: (cycle_id = talent_assessment_cycles.id)
```

7.5ms total; 397 shared buffer hits; no Seq Scan on the population table at any tested scale. Population access is always mediated through `talent_assessment_cycles` (via `ix_talent_assessment_cycles_scope` on `school_group_id`) then joined to `talent_assessment_cycle_population_members` via `uq_talent_cycle_population_student` (`cycle_id`, `student_id`) / `ix_talent_cycle_population_scope` (`school_group_id`, `cycle_id`, `branch_id`, `student_id`).

Candidate/Identification aggregation (`overview`), LARGE

```
GroupAggregate  (cost=922.17..922.46 rows=13 width=12) (actual time=25.738..25.847
rows=12 loops=1)
  Buffers: shared hit=50290
  ->  Sort (actual rows=1938)
        ->  Nested Loop (actual rows=1938, buffers=50287)
              ->  Nested Loop (actual rows=3836, buffers=40677)
                    ->  Hash Join (actual rows=18095, buffers=651)
                          ->  Seq Scan on talent_student_assessments (rows=18095,
actual time=0.006..1.073)
                          ->  Hash (rows=22800) <- Nested Loop over cycles+population
(buffers=397)
                    ->  Index Scan using uq_talent_review_candidates_assessment
                          (loops=18095, buffers=40026)
              ->  Index Scan using uq_talent_official_identifications_candidate
                    Filter: decision = 'identified'  (loops=3836, buffers=9610)
Execution Time: 26.067 ms
```

26ms total at LARGE scale, despite 18,095 and 3,836 Nested-Loop iterations against `talent_review_candidates`/`talent_official_identifications` - because every iteration is a cheap unique-index point lookup against a small, fully-cached table.

Participation-Overlap self-join (Index Candidate B path), LARGE (warm)

```
GroupAggregate (actual rows=78, time=31.700..33.880)
  ->  Sort (actual rows=26121)
        ->  Hash Join (Hash Cond: student_id = student_id, actual rows=26121)
              Join Filter: (program_id <= program_id)   Rows Removed by Join Filter:
9201
              ->  HashAggregate (Group Key: program_id, student_id; actual rows=16920)
                    ->  Nested Loop (actual rows=22800)
                          ->  Index Scan using ix_talent_assessment_cycles_scope
(rows=19)
                          ->  Bitmap Heap Scan on
talent_assessment_cycle_population_members
                                (via Bitmap Index Scan on
uq_talent_cycle_population_student)
              ->  Hash <- HashAggregate (same shape, mirrored program_id/student_id
side)
Planning Time: 0.299 ms   Execution Time: 34.422 ms
```

See below for the pre-`ANALYZE` (cold) version of this same query, which took 39,486-45,154 ms instead of 34-90 ms.

Seq Scan Findings And Rationale

Every material `Seq Scan` observed across all three scales, in all captured plans:

| Table | Route/query family | Table size (LARGE) | Actual time | Rational? | |---|---|---:|---:|---| | `talent_student_assessments` | `talent_map`, `program_portfolio`, `branch_intelligence`, `longitudinal` coverage-by-dimension (`Hash Right Join` feeding a `COALESCE(status,'unassessed')` group)

| 18,095 rows | 0.6-1.1 ms | Yes - the query logically needs nearly every row (coverage-by-status touches the full Assessment set for the authorized population); at ~18K narrow rows a full scan (254 buffer pages, ~2 MB) beats random per-row index lookups. | | `students` | `student_drill` row fetch | 10,321 rows | 0.5 ms | Yes - nearly the full authorized Student set is joined; same rationale as above. | | `talent_review_candidates` / `talent_official_identifications` aggregation subqueries | `overview`, `program_portfolio`, `branch_intelligence` candidate/identification counts | 3,836 / 1,938 rows | sub-millisecond | Yes - these are genuinely small per-tenant tables at every tested scale; a full scan of a few thousand narrow rows is inherently cheap and the planner correctly avoids unnecessary index overhead. |

None of the observed Seq Scans indicate a missing index at the tested scales - every one completed in low single-digit milliseconds even at ~23K total frozen-population rows and ~18K Assessments. This conclusion is explicitly scoped to the tested SMALL/MEDIUM/LARGE scenarios and is **not** generalized to a future, materially larger production table.

Index Candidate A - Population-Member Access (`school_group_id`, `program_id`, `academic_year_id`, `branch_id`)

Current relevant indexes on `talent_assessment_cycle_population_members`: `ix_talent_cycle_population_scope` (`school_group_id`, `cycle_id`, `branch_id`, `student_id`) and the unique `uq_talent_cycle_population_student` (`cycle_id`, `student_id`). Access is always mediated through `talent_assessment_cycles` (`ix_talent_assessment_cycles_scope` on `school_group_id`, `program_id`, `academic_year_id`, `status`) to resolve eligible `cycle_ids` first, then a Bitmap-Heap/Index Scan into `talent_assessment_cycle_population_members` by `cycle_id`.

Actual current plan / cost/timing evidence: every captured plan touching this table at every scale used `ix_talent_assessment_cycles_scope` then `uq_talent_cycle_population_student`/`ix_talent_cycle_population_scope` - **never a Seq Scan on the population table itself**. Worst observed population-access-heavy execution time at LARGE (~22.8K rows): 7.5ms (`count_distinct_students`) to 26ms (full `overview` candidate/identification aggregation, which also joins two additional child tables).

Classification: NO CHANGE. Existing indexes are adequate at the tested scales (SMALL/MEDIUM/LARGE, up to ~22.8K population rows). This remains scale-bounded evidence; a further 10-100x larger dataset (hundreds of thousands to low millions of rows) remains required before a final production judgment, consistent with ADR 0028's deferral. **No index is created or approved by this document.**

Index Candidate B - Student-ID-Leading Access For Participation-Overlap

Current relevant indexes: `uq_talent_cycle_population_student` is `cycle_id`-leading, not `student_id`-leading; no standalone `student_id`- leading index exists on the population table.

Actual current plan / cost/timing evidence - real, scale-dependent instability was observed:

| Scale | Population rows | Program pairs | Plan shape | Execution time | |---|---:|---:|---|---:| | SMALL | 66 | 3 | Hash Join over two small `HashAggregates` | ~14-20 ms (whole-route) | | MEDIUM | 1,501 | 21 | **Nested Loop with an inner `Unique`/`Sort` subplan re-executed 1,080 times** (`loops=1080`), driven by a severe row-estimate underestimate (planner estimated `rows=1`, actual `rows=1567`) | **195.7 ms** (query only) - the single slowest *warm* query in the entire evidence set | | LARGE | 22,801 | 78 | Two `HashAggregates` feeding one `Hash Join` (good plan) | 34.4 ms (query only) - **faster in absolute terms than MEDIUM** despite ~15x more population rows and 3.7x more pairs |

This is the most notable planner-instability finding in this evidence set: PostgreSQL's cost-based row estimate for this exact self-join crosses a threshold somewhere between MEDIUM and LARGE population size that flips it from a pathological repeated-subplan Nested Loop to an efficient `HashAggregate+Hash Join` strategy - **independent of whether a `student_id`-leading index exists** (neither the good nor the bad plan used one; the good plan used

in-memory hashing instead).

Classification: MORE EVIDENCE REQUIRED. The instability is real but does **not** monotonically correlate with population size (MEDIUM was slower than LARGE), so it does not, by itself, justify adding a `student_id`-leading index. It does show that (a) the planner's cost estimate for this specific self-join is sensitive and non-monotonic at these small/moderate row counts, and (b) statistics freshness after a Cycle-population freeze materially affects this query family in bulk-load scenarios (see next section). Per ADR 0028's instruction not to copy forward a prior "still too small" conclusion without re-measuring: this re-measurement surfaces a materially different, ANALYZE-timing-related risk rather than confirming or ruling out the index. **No index is created or approved by this document**; both the plan-instability finding and the statistics-freshness finding are deferred to B11-D/E for a larger, statistics-controlled re-measurement.

Participation Overlap - Cold-Start Latency Cliff (Root-Caused)

At LARGE scale, the **first** `participation-overlap` call (immediately after the ~23K-row synthetic bulk load, in the same profiling run) took **39,486.87 ms** - roughly 400-700x slower than every other measurement of the identical query (57.9 ms on the very next call in the same run; 34.4 ms via the offline `EXPLAIN ANALYZE` capture taken afterward).

This was independently reproduced and root-caused via a targeted follow-up diagnostic (same LARGE dataset builder, same route, same non-production harness conventions):

1. Freshly built the LARGE dataset in an isolated schema.
2. Called `participation-overlap` immediately (no explicit `ANALYZE`):

41,822.58 ms (first reproduction) and **45,154.02 ms** (second reproduction) - consistently tens of seconds, confirming this is reproducible, not a one-off fluke.

1. Queried `pg_stat_user_tables` for `talent_assessment_cycle_population_members`

immediately before that slow call: `n_live_tup=0`, `last_analyze=NULL`, `last_autoanalyze=NULL` - PostgreSQL had **no statistics at all** for the just-bulk-loaded table yet.

1. Ran a manual `ANALYZE talent_assessment_cycle_population_members;

ANALYZE talent_student_assessments;`.

1. Called the identical route again: **66.5 ms** - a **~680x** improvement

for the byte-identical query against the byte-identical data.

Root cause (well-supported inference from steps 3-5): immediately after a large synchronous bulk load, before PostgreSQL's autovacuum has run its first `ANALYZE` on the newly populated tables, the query planner has no statistics and falls back to default/minimal row-count assumptions. For this specific self-join, that produces a catastrophic plan (a large fan-out Nested Loop) instead of the efficient `HashAggregate+Hash Join` plan chosen once real statistics exist. In the original combined profiling run, the *second* call (immediately following the first) was already fast because autovacuum had evidently completed its background `ANALYZE` during the ~39 seconds the first call was running.

This is a real, reproducible, and important finding - but its likely practical driver is the harness's "bulk-load-then-immediately-query" pattern, not necessarily steady-state production behavior (where Cycle population freezes are typically smaller, incremental, human-triggered operations, not one 23K-row synchronous transaction, and autovacuum/ANALYZE has continuous opportunity to run between operations). It is flagged here as a genuine, non-fabricated, statistics-freshness risk for any workflow that performs a large synchronous population freeze immediately followed by an Organization Analytics read, and is explicitly deferred to B11-D/E rather than resolved here (no isolation change, no index, and no operational mitigation is adopted by this document).

Talent Map

Program x Branch case (LARGE): 12 Programs x 8 Branches = 96 prospective cells; 384 actual populated rows returned by the `HashAggregate` (`Group Key: program_id, branch_id, status`); SQL aggregation (`HashAggregate` over a `Hash Right Join` against the 18,095-row Assessment `Seq Scan`) executed in 14.1ms; route total (SQL + privacy closure + serialization) 57.3/62.2 ms (1st/2nd); response 15.7 KB; RSS flat. Program x Grade case was not separately profiled in this pass (only `dimension=program_branch` was exercised - see Known Limitations); no non-linear growth was observed across scale for the branch dimension (9 queries at every scale).

Program Portfolio

LARGE: 12 Programs; 11 queries (constant across scale); grouping/aggregation plan uses the same `talent_assessment_cycles` -> population -> assessments join chain as `overview`; Candidate/Identification conditional paths both exercised (permissions granted); route latency 121.1/130.4 ms (1st/2nd); RSS flat at 133.0 MB.

Branch Intelligence

LARGE (Branch 10, 12 authorized Programs): 11 queries (constant across scale); frozen historical-scope plan resolves via `ix_talent_assessment_cycles_scope` + population `branch_id` filter, all Index/Bitmap scans (no Seq Scan on the population table); route latency 85.4/52.8 ms (1st/2nd); RSS flat.

Student Drill

	SMALL	MEDIUM	LARGE
limit=25 query count	11	11	11
limit=100 query count	11	11	11
limit=25 latency (1st/2nd, ms)	59.3 / 18.0	58.6 / 32.3	128.6 / 84.7
limit=100 latency (1st/2nd, ms)	33.1 / 33.8	54.7 / 68.9	101.1 / 86.9

`count_distinct_students` uses a `DISTINCT`-gated `Aggregate` (see EXPLAIN excerpt above) - confirmed a real `DISTINCT` Student-population gate, not a raw row count. Candidate/Identification fields are fetched inside the same page-bounded row query (`Limit`/`Sort` over the joined population+assessment +candidate+identification set, `rows=101` for `limit=100` at LARGE) - no separate per-row query. No `total_count` field exists in the response (confirmed by inspection of `talent_org_student_drill.serialize_projection` and by the constant 11-query count regardless of `limit`). Query count did **not** change between `limit=25` and `limit=100`, and did not change across scale - **no N+1**. Offset behavior (pagination beyond page 1) was not separately profiled in this pass (see Known Limitations).

Longitudinal

Focus Program (`id=1`), 3/5/8 Planned Evaluation Periods at SMALL/MEDIUM/ LARGE respectively, `metric=completion_coverage` (a count-family metric requiring no Candidate/Identification permission). Query count constant at **11 for every Period count tested (3, 5, and 8)** - confirmed **population-size-independent AND Period-count-independent** query behavior. `coverage_by_cycle` groups by `cycle_id` (one per Period) via the same `Hash Right Join+GroupAggregate` shape as the other coverage queries (8.6-9.6ms SQL aggregation at LARGE, 9,600 population rows across the 8 focus-Program cycles). Route latency 47.6/27.1 ms (1st/2nd) at LARGE. An ad-hoc/unlinked Assessment Cycle (one with no `planned_evaluation_period_id`) was present in the dataset for every non-focus Program and was correctly excluded from every Longitudinal result (only the focus Program's Period- linked cycles appear in `coverage_by_cycle`/the periods query, which filters by `annual_evaluation_plan_id`/`program_id`) - confirmed by the fact `period_count` in every response matched the exact configured Period count (3/5/8), never inflated by the unlinked control cycle.

Regression

Command run (from repository root, using the pinned repo venv):

```
.\.venv\Scripts\python.exe -m pytest tests/test_talent_org_intelligence_contract.py
tests/test_talent_org_intelligence_queries.py
tests/test_talent_organization_overview.py
tests/test_talent_organization_talent_map.py
tests/test_talent_organization_program_portfolio.py
tests/test_talent_organization_branch_intelligence.py
tests/test_talent_organization_participation_overlap.py
tests/test_talent_organization_student_drill.py
tests/test_talent_organization_longitudinal.py
tests/test_talent_organization_observability.py -q
```

Result: **209 passed** (0 failed, 0 errors), actually re-run and verified as part of this task. The task brief handed to this remediation stated the independent reviewer had previously reported 262 passed; this exact 10-file M10 Organization Analytics set (route + shared contract/query tests) does not reproduce that number (209), and a broader run that also includes the separate M9 `tests/test_talent_analytics.py` suite (71 tests) yields 280 - neither matches 262 exactly, most likely because the reviewer's prior run used a different file selection that is not reconstructable from this document alone. Rather than restate the unverified 262 figure, this document reports only the number this task actually executed and observed: **209 passed, 0 failed** for the command above. This is a pre-existing SQLite-backed unit/integration suite, unrelated to the PostgreSQL harness above; it was re-run here to confirm this evidence-remediation task made no M10 semantic change.

Confirmed by this regression run plus direct inspection of this task's own diff:

1 Exactly seven M10 Organization Intelligence routes exist (no eighth/no

profiling API route was added).

1 No M10 route/service/serializer semantics changed.

1 No `delta`/`change`/`percent_change` field exists anywhere (unchanged from

before this task).

1 No new permission or entitlement was added.

1 No privacy-semantics change (the existing `AllowAllTestPolicy`/`AllowAvailability`/

`AllowBreadth` non-production test doubles were reused exactly as the existing test suite already uses them; no new provider implementation was added).

1 No schema, index, or migration was added (`db_migrations.py`, `models.py`

application source files were not modified by this task).

1 `tis.db` was not touched (all profiling used the dedicated non-production

PostgreSQL `tis_b11c_test_db`, in ephemeral schemas dropped at the end of every run).

Profiling Methodology - Known Limitations

1 Synthetic Students use simplified continuity: each Student's frozen

population `branch_id`/`grade_level` always matches their single `StudentAcademicPlacement` row; a genuine historical Branch/Grade/Section *change* between Cycle freezes was not modeled in this pass.

1 Only one actor shape was profiled: `ORGANIZATION`-scope, role `Editor`,

with all five relevant permission keys granted (base view, Candidate view, Identification view, Student-drill view, Learner-Profile view). Branch- scoped/partial-permission actors, and privacy-suppressed/coarsened outcomes, were not separately performance-profiled (only privacy-ALLOW-ALL non-production policy was used, per B11-C entry

criteria).

1 All three scales ran sequentially inside one Python process; RSS numbers

are cumulative across scales, not independent per-scale baselines (see Memory section).

1 Captured `EXPLAIN` plans reflect a "warm" Postgres state (after at least

one prior application-level call, and for LARGE `participation-overlap` specifically, after autovacuum's `ANALYZE` had already completed) - not the literal very-first-ever plan for a cold, freshly-loaded table (see the Participation Overlap cold-start section for that case, captured separately).

1 Query-count instrumentation used a temporary engine event listener,

registered and removed per call; it has no permanent effect and was not left in any application code path.

1 `talent-map` `dimension=program_grade` and Student Drill pagination

`offset>0` were not separately profiled in this pass.

1 LARGE (~22.8K frozen population rows, ~10.3K distinct Students) is a

bounded, non-production representative case, explicitly **not** "production scale." A further 10-100x larger dataset remains required before any production judgment (Index Candidates A and B, memory, and SLOs all remain explicitly deferred).

Deferred To B11-D/E

1 Fine-grained privacy-closure phase timing (SQL aggregation vs. Cell/

Relationship construction vs. provider evaluation vs. serialization) - this pass only measured coarse route-level latency; no fabricated fine-grained phase breakdown is provided.

1 Production-like PostgreSQL, controlled concurrent writers, and `READ

COMMITTED` consistency qualification (B11-D).

1 10-100x larger Participation Overlap and general population evidence.

1 Concurrent-request and multi-process memory-ceiling qualification.

1 Any numeric privacy threshold, breadth limit, performance SLO, memory

ceiling, or commercial/plan mapping (ADR 0028 Deferred Decisions - unchanged by this document).

1 Resolution (if any) of the participation-overlap plan-instability and

statistics-freshness findings above.

Files Touched By This Remediation Task

1 `docs/history/engineering-handbook/2026-09-07-b11c-postgresql-profiling-evidence-remediation.md`

(this file, new).

1 `docs/history/engineering-handbook/README.md` (added a navigation bullet to

this file).

1 `docs/CHANGE_HISTORY.md` (added a new dated entry; no existing lines

changed).

1 `docs/PROJECT_STATE.md` (added a new section; no existing lines changed).

1 `docs/adr/0028-b11-production-qualification-policy.md` (updated only the

B11-C status wording to reflect execution/evidence-capture/re-review- pending; no Decision/Deferred-Decisions text changed).

1 `.kms-impact.yml` (task-specific declaration correction, plus a

transparency note - see below).

1 `static/docs/TIS_Project_Reference_Booklet.pdf` and

`static/docs/docs_manifest.json` (regenerated automatically by `python scripts/kms.py sync`, not hand-edited).

`.kms-impact.yml` also *lists* (without editing their content) `docs/adr/README.md`, `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`, and `docs/engineering/USER_AND_SYSTEM_FLOWS.md` - these were already uncommitted before this task began (deleting the ADR 0028 catalog entry and governance sections respectively) purely so `kms.py check`'s mechanical "declared vs. changed" completeness gate could pass; this task did not author or resolve that pre-existing content. See the "TRANSPARENCY FLAG" comment in `.kms-impact.yml` and the coordinator report for the full disclosure - it is the same pattern already visible in the pre-existing dirty diffs of `docs/PROJECT_STATE.md`, `docs/TIS_MASTER_CONTEXT.md`, and `docs/CHANGE_HISTORY.md`, and requires the repository owner's explicit reconciliation before any commit.

No application source file (`routers/`, `talent_org_*.py`, `talent_analytics_*.py`, `models.py`, `db_migrations.py`) was modified. No index, migration, permission, entitlement, or privacy-semantics change was made. No harness file was committed to the repository (its full source is reproduced above for reproducibility).

`tis.db` incident and resolution: running the regression suite (see Regression, above) caused `tis.db` to be byte-modified in place (same file size, different bytes - consistent with SQLite's internal change-counter/ book-keeping, not a row/schema change), because `database.py` opens a default SQLAlchemy engine against the local `tis.db` file whenever `DATABASE_URL` is not set, and importing modules under test transitively imports it even though every M10 test uses its own isolated in-memory SQLite fixture. This was detected via `git status` immediately after the regression run, and `tis.db` was restored to the exact byte-for-byte `HEAD` content before this task completed (verified with `git diff --stat -- tis.db` showing no difference). `tis.db` was not intentionally modified and is confirmed clean/unchanged at the end of this task.

M10 B11-D PostgreSQL Concurrency And Consistency Qualification

docs/history/engineering-handbook/2026-09-07-b11d-postgresql-concurrency-consistency-qualification.md

2026-09-07 - M10 B11-D PostgreSQL Concurrency And Consistency Qualification

Module: Talent & Potential M10 Organization Analytics, PostgreSQL concurrency/ consistency qualification (ADR 0028, B11-D entry)

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-09-07 - M10 B11-D PostgreSQL Concurrency And Consistency Qualification

Related ADRs: `docs/adr/0028-b11-production-qualification-policy.md` (governing authority for B11-D entry criteria, isolation-change gate, and B11-E release gate)

Related prior evidence: `docs/history/engineering-handbook/2026-09-07-b11c-postgresql-profiling-evidence-remediation.md` (B11-C dataset-builder pattern, non-production provider convention, Index Candidate A/B baseline, statistics-freshness cold-start finding)

Reviewer/approval notes: This is a CONCURRENCY/CONSISTENCY QUALIFICATION task only. It does not redesign M10, does not implement B11-E, does not implement B12, and does not merge or deploy. It does not change PostgreSQL isolation permanently, does not add an index, does not add a migration, and does not choose any production threshold, breadth limit, SLO, memory ceiling, or commercial mapping. The evidence-gathering pass did not itself declare closure; subsequent independent review returned **PASS WITH NON-BLOCKING OBSERVATIONS**, so this governed checkpoint marks B11-D **CLOSED**. No merge or deployment occurred. `tis.db` was not touched (confirmed unchanged before and after this task, including after running the pytest regression suite below). All measurement used the dedicated non-production `tis_b11c_test_db` PostgreSQL 16.15 instance at `127.0.0.1`, reached only through the `TIS_TEST_POSTGRESQL_URL` environment variable (never printed, logged, or persisted anywhere in this repository), inside ephemeral per-phase schemas created and dropped by the harness - no rows were left behind in the shared test database.

Governing checkpoint for this task: `dev/origin-dev` `793c78c882624a7f63402aafe4150d3f69d3132b`.

Working-Tree Integrity Note (Observed During This Task, Not Authored By It)

While this task was running, `git status` showed several additional already-tracked KMS files change from clean to uncommitted mid-session (`docs/adr/0028-b11-production-qualification-policy.md`, `docs/engineering/USER_AND_SYSTEM_FLOWS.md`, and this document's own predecessor `docs/history/engineering-handbook/2026-09-07-b11c-postgresql-profiling-evidence-remediation.md`), extending the same "B11-C CLOSED -> reverted to NOT CLOSED" / "M10 B11 Qualification And Release Flow section deleted" pattern already disclosed as a pre-existing, unresolved discrepancy in the B11-C artifact's own Files-Touched section (there scoped to `docs/adr/README.md`, `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`, and `docs/PROJECT_STATE.md`/`docs/TIS_MASTER_CONTEXT.md`/`docs/CHANGE_HISTORY.md`). This task did not author, investigate the origin of, revert, or resolve any of this; per the delegated task's explicit Repository Safety instruction, all of it is preserved completely untouched. This document's own Status/Current Milestone Truth wording below intentionally follows the delegating task's stated premise (this checkpoint's committed history records B11-C CLOSED) and is written additively (a new file, plus small additive edits elsewhere) so it neither depends on nor further disturbs whichever wording of the pre-existing in-progress edit is present in the working tree at merge time. The

repository owner's explicit reconciliation of the underlying discrepancy is still required before any commit, exactly as the B11-C artifact already stated.

Current Milestone Truth

1 B0-B10: CLOSED. B11-A: complete. B11-B: CLOSED. B11 production-

qualification policy (ADR 0028): APPROVED/GOVERNED.

1 B11-C: PostgreSQL profiling executed; durable evidence captured in the

linked B11-C artifact, per this checkpoint's governing history.

1 B11-D (this document): PostgreSQL concurrency/consistency qualification

executed; durable evidence **captured**. Independent review: **PASS WITH NON-BLOCKING OBSERVATIONS**. B11-D is **CLOSED**.

1 No index, migration, permission, entitlement, privacy-semantics change,

production SLO, production privacy threshold, production breadth limit, production memory ceiling, or PostgreSQL isolation change is approved by this document.

1 B11-E: NOT COMPLETE. B11 overall: NOT CLOSED. B12: NOT IMPLEMENTED.

Production readiness: NOT achieved.

Objective

Qualify the seven M10 Organization Intelligence routes under realistic concurrent read/write PostgreSQL behavior, starting with the current/default `READ COMMITTED` behavior as ADR 0028 requires, and determine whether M10 query-on-read analytics preserve acceptable consistency under concurrent application activity, and whether a stronger transaction boundary or isolation level is actually justified by evidence.

Phase 0 - Connection Verification (Non-Production)

| Check | Result | |---|---| | `TIS_TEST_POSTGRESQL_URL` present in process environment | Yes | | Connection succeeds | Yes | | `SELECT version()` | `PostgreSQL 16.15, compiled by Visual C++ build 1944, 64-bit` | | `SELECT current_database()` | `tis_b11c_test_db` | | `SELECT inet_server_addr()` | `127.0.0.1` (localhost only) | | `SHOW transaction_isolation` (server default) | `read committed` | | Environment | Non-production (dedicated `tis_b11c_test_db`/`tis_b11c_test_role`, localhost-only) |

Methodology (Reproducible)

Reuses the B11-C pattern exactly: `models.Base.metadata.create_all(engine)` against an ephemeral, per-phase PostgreSQL schema (`search_path`-scoped, one `CREATE SCHEMA`/`DROP SCHEMA ... CASCADE` per phase, no rows left behind); the real M10 routers/services (`routers/talent_organization_analytics.py`, `talent_org_intelligence_service.py`, `talent_org_talent_map.py`, `talent_org_b7.py`, `talent_org_participation_overlap.py`, `talent_org_student_drill.py`, `talent_org_longitudinal.py`) mounted on an isolated `FastAPI()` app via `fastapi.testclient.TestClient` with `dependency_overrides` for `get_db`, `get_current_user`, and the same three non-production provider doubles the existing test suite already uses (`AllowAvailability`, `AllowBreadth`, `talent_analytics_privacy.AllowAllTestPolicy`

1 explicitly labeled NON-PRODUCTION; no production privacy/availability/

breadth provider exists yet per ADR 0028, so every measured behavior reflects the SQL/ORM/session/transaction/privacy-closure code that exists today, not a future production threshold decision); a seeded `random.Random(20260907)` synthetic dataset builder using the exact same field/relationship shape as B11-C's (bulk `sa.insert(...)` for Student/Placement/PopulationMember/ Assessment/Candidate/Identification rows, one primary `SchoolGroup` plus one tiny isolated foreign `SchoolGroup` tenant-isolation control).

New for B11-D (concurrency-specific): (1) real `threading`-based concurrent reader/writer scenarios against `TestClient` calls sharing one SQLAlchemy `Engine`/connection pool, each `client.get(...)` opening its own per-request `Session` exactly the way the real `dependencies.get_db` dependency does (`SessionFactory()` -> `yield` -> `.close()`); (2) direct calls into the identical production service functions (`talent_org_intelligence_service.*`, `talent_org_student_drill.*`) in a single, deliberately-held-open reader `Session`/transaction, with a real, separately-committing writer `Session` interleaved between two of those calls, to prove or disprove cross-statement snapshot divergence without relying on non-deterministic thread-timing races; (3) an isolated `REPEATABLE READ` experiment using `Connection.execution_options(isolation_level=...)` on the reader connection only, never a server-wide or application-wide change; (4) a `LockObserver` classifying every writer-thread SQLAlchemy exception into deadlock / lock-timeout / other, from the real `psycopg2` `orig` exception text; (5) `psutil` RSS sampling before/after each concurrency scenario (DIRECTIONAL LOCAL WINDOWS DEVELOPMENT EVIDENCE only).

The harness (~700 lines across two files) was **not** committed to the repository, per the same profiling-harness scope-minimization convention B11-C used; its methodology, exact dataset dimensions, and every numeric result are reproduced durably below so an independent reviewer can verify every conclusion without the harness file or a chat transcript.

Current Isolation/Session Behavior (Code Inspection + Live Confirmation)

`database.py` creates one process-wide `SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)` with **no explicit** `isolation_level` passed to `create_engine(...)`, so the PostgreSQL server default governs every connection - confirmed live above as `read committed`. `dependencies.get_db` is the sole session source for every M10 route:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Every M10 route function declares exactly one `db: Session = Depends(get_db)` parameter, so **one HTTP request uses exactly one** `Session`**/one PostgreSQL transaction for its entire duration** - with `autocommit=False`, SQLAlchemy opens the transaction lazily on the first statement and keeps it open (no explicit `db.commit()`/`db.rollback()` exists anywhere in the seven read-only M10 routes) until `get_db`'s `finally: db.close()` runs at the end of the request, which implicitly rolls back/releases that transaction. This means: one M10 HTTP request currently has **one open transaction but NOT one snapshot** - under `READ COMMITTED`, PostgreSQL takes a **fresh snapshot per SQL statement**, not per transaction, so later statements in the same still-open request transaction can observe commits that happened after the request started (and after earlier statements in that same request already ran).

Structurally, every one of the seven routes builds one lazy SQLAlchemy `Query` object once per request (`svc.frozen_membership_query(...)`, never executed itself) and then passes that same `Query` object to **two or more independent downstream functions**, each of which issues its **own fresh SQL** `SELECT` against it. This is the exact mechanism that makes cross-statement snapshot divergence possible - confirmed by direct code reading of every route function in `routers/talent_organization_analytics.py` and every corresponding service module:

Route	Independent SQL statements against population-derived data (same request)	Structural exposure
`overview`	`coverage_organization_total` (1) + `candidate_membership_counts` (0-1) +

`identification_membership_counts` (0-1) | High (3 max) - **empirically proven below** | | `program-portfolio/branches/{id}` (`_b7_response`) | `coverage_program_totals` (1) + `candidate_membership_counts` (0-1) + `identification_membership_counts` (0-1) + `required_period_execution_counts` (portfolio only, program/plan-derived not population-derived) | High (3 max) - same code path as `overview`, not independently re-run this task | | `talent-map` | `coverage_by_program_branch/_grade` (1) + `sensitive_membership_by_dimension` (0-1) | Medium (2 max) - not independently re-run this task | | `students` (Student Drill) | `count_distinct_students` (1, the P7 gate) + `fetch_student_rows` (1 page query + 0-2 more Candidate/Identification lookups keyed off the page's member ids) | High (2-4) - **empirically proven below** | | `programs/{id}/longitudinal` | `coverage_by_cycle` (1) + `candidate_counts_by_cycle` (0-1) + `identified_counts_by_cycle` (0-1) | High (3 max) - same code shape, not independently re-run this task | | `participation-overlap` | `participation_overlap_counts` (1 single self-join query - the entire Program-pair matrix is produced by ONE statement) | **Low** - the core aggregate cannot internally diverge from itself (one statement = one snapshot); confirmed by direct code reading |

Phase 1 - Overview Cross-Statement Consistency (READ COMMITTED vs REPEATABLE READ)

Dataset: MEDIUM-shaped (4 Branches, 6 Programs, 150 Students/cycle, 1,501 frozen population rows, 314 Candidates, 154 Identifications, 322 frozen-but-`unassessed` members available as write targets).

Reproduces the real `/overview` route's own statement order in the real session/session-transaction shape: `coverage_organization_total` (SQL#1) followed later by `candidate_membership_counts` (SQL#2), both against the same still-open reader `Session`. Between SQL#1 and SQL#2, a **separate, already-committed writer** `Session` performs a real domain-valid write - an Assessment completion (`status: unassessed -> completed`) plus a Candidate decision creation - for one already-frozen population member that SQL#1 had just counted as `unassessed`.

	`READ COMMITTED` (default)	Isolated `REPEATABLE READ` experiment
Candidate total before write, read in the open reader tx	314	314
Candidate total after write, re-queried in the **same still-open** reader tx (SQL#2)	**315**	314
Candidate count changed mid-request without a new request	**Yes**	No
`completed` count if SQL#1 were re-run in the same tx	827 (vs. 826 originally observed)	826 (unchanged)

Demonstrated inconsistency (`READ COMMITTED`): the population/coverage snapshot the route already returned (`completed=826`, unaware of the write) and the Candidate count the same route returns moments later in the same response (`candidate=315`, aware of the write) come from **two different committed database states** - a live, reproduced instance of exactly the "Candidate component after write but frozen population before write" example named in this task's brief. Under the isolated `REPEATABLE READ` experiment, both statements returned mutually consistent, stable values (the write becomes visible only to a *subsequent* request/transaction) - the same technique the task specification calls "synchronized writer barriers / transactions... to prove or disprove it" was used here via deterministic statement-order control rather than a timing race, which is more reliable and fully reproducible.

Phase 2 - Student Drill Gate-Vs-Page Consistency (READ COMMITTED vs REPEATABLE READ)

Same MEDIUM dataset (rebuilt fresh in its own schema). Reproduces `count_distinct_students` (the P7 gate SQL) followed later by `fetch_student_rows` (the page-fetch SQL), both against the same still-open reader `Session`. Between them, a separate writer `Session` commits one brand-new frozen population member for a brand-new Student (a valid population-freeze-style write, not a mutation of any existing frozen row).

	`READ COMMITTED` (default)	Isolated `REPEATABLE READ` experiment
Gate eligible-count (SQL#1)	705	705
Page fetch row count (SQL#2, same open tx, limit=1000)	**706**	705
New Student visible in the page fetch	**Yes**	No
Gate-vs-page population mismatch	**Yes**	No

Demonstrated inconsistency (`READ COMMITTED`): the P7 eligibility gate was evaluated against a population of 705 Students, but the identifiable page of rows served moments later in the same response includes a 706th Student who was not part of the gated population the gate decision was actually made against - a live, reproduced instance of the "Student Drill page membership... mismatch" example named in the task brief. Under the isolated `REPEATABLE READ` experiment, the page fetch correctly excluded the new Student, matching the gate's own snapshot exactly.

Extrapolation To Other Routes (Not Independently Re-Run This Task)

`program-portfolio`, `branches/{id}`, `talent-map`, and `longitudinal` share the identical structural mechanism proven above (a lazy `Query` object re-executed independently by 2-3 downstream calls inside one still-open `READ COMMITTED` session/transaction, with no code-level guarantee that any two of those calls observe the same committed state). This is a code-inspection-based extrapolation from the same functions/pattern already proven live for `overview` and `students`, not a separately reproduced live race for each of those four routes - time-bounded within this task; treated as **high-confidence but not independently empirically verified per-route**, and should be listed explicitly as such for the independent reviewer. `participation-overlap`'s core Program-pair matrix is produced by exactly one SQL statement (confirmed by code reading, not just count), so it does **not** share this specific cross-statement mechanism for its own aggregate - it remains exposed only to the much lower-risk case of an unrelated Program being reconfigured between an earlier `authorized_program_universe`/ `program_rows` lookup and the overlap query itself.

Privacy / Security Analysis

`talent_analytics_relationship_graph`/`talent_analytics_privacy_closure` build one `PrivacyClosureResult` per request from whatever raw `Cell` values the route already gathered in Python before calling `apply_primary_privacy_and_close` - closure itself does not re-query the database, so it cannot introduce a *new* cross-snapshot divergence beyond whatever the route's own SQL calls already produced. Two concrete consequences, both confirmed by code reading plus Phase 1/2 evidence:

1. **Relationship-linked Cells cannot desynchronize relative to each other**,

because both sides of every existing M10 `Relationship` (`overview`'s `completed<->coverage_n` and `frozen<->coverage_d`) are populated from the **same single SQL statement's result** (`coverage_organization_total`), which is one atomic `READ COMMITTED` snapshot by definition. No relationship-closure equation observed in this codebase can be fed internally contradictory operands by concurrency alone.

1. **Every other Cell has no relationship at all** (`candidate/identified`

in `overview`/`portfolio`/`branch`/`longitudinal`, every pair in `participation-overlap`, the single gate Cell in `students`) - each such Cell is only ever as fresh as its own statement's snapshot, with no cross-Cell equation to violate, but also no guarantee that two independently-fetched Cells in the same JSON response reflect the same moment in time. Phase 1/2 are direct, reproduced instances of this.

Privacy-reconstruction risk: this task could not empirically evaluate whether concurrency creates a genuine complementary-suppression-equation reconstruction risk, because ADR 0028 explicitly defers the production `TalentAnalyticsPrivacyPolicy` implementation and every test in this task (matching B11-C and the existing suite) used the non-production `AllowAllTestPolicy`, which never suppresses. The structural risk is real in principle (a future threshold-based policy could evaluate a suppression decision against a raw value gathered from one statement while a sibling component used for a complementary equation was gathered from a different, inconsistent statement/snapshot) but is **not measurable today** without a real threshold provider - carried forward as an explicit B11-E dependency, not resolved here. No suppressed value was observed to become reconstructable in this task because no suppression occurred in any tested scenario (fail-closed AllowAll semantics were unchanged and behaved identically under every concurrency scenario tested).

Tenant Isolation

Every M10 query filters by `context.school_group_id` (confirmed by code reading of every `talent_org_intelligence_service.py` query). A dedicated foreign-tenant writer thread ran continuously throughout every Phase 3 stress scenario (creating foreign-tenant Students/Placements/frozen population members in a second, isolated `SchoolGroup`) concurrently with the primary tenant's readers and writers. The primary tenant's final post-burst `/overview` response (`academic_year_id=100`, `scope="organization"`) reflected only primary-tenant scope in every scenario; no foreign-tenant identifier, count, or structural signal was observed to cross into the primary tenant's response, aggregate counts, or privacy relationships in any scenario tested.

Phase 3 - Concurrent HTTP-Level Reader/Writer Stress (All 7 Routes)

Dataset: MEDIUM-shaped (same dims as Phase 1/2), fresh per scenario, `ANALYZE`d before each stress run. Each `client.get(...)` opens its own per-call `Session` (matching real `get_db` semantics exactly). Writers perform real domain-valid flows only: completing an already-frozen `in_progress`/`incomplete` Assessment, creating a `TalentReviewCandidate` for an already-completed Assessment that lacks one, and creating a `TalentOfficialIdentification` for a Candidate that lacks one - never mutating any frozen population-member row. A dedicated foreign-tenant writer ran in every scenario (see Tenant Isolation above).

Scenario	Readers x iterations (all 7 routes/iteration)	Writers	Wall time	HTTP status	Deadlocks	Lock timeouts	Other errors	RSS delta (directional)
1R/1W	1 x 3 (21 calls)	1	0.86s	21x `200`	0	0	0	+0.58 MB
4R/1W	4 x 3 (84 calls)	1	1.8s	84x `200`	0	0	0	+0.19 MB
4R/3W	4 x 3 (84 calls)	3	2.11s	84x `200`	0	0	18 (see below)	+2.81 MB

Every HTTP call across all three scenarios (189 total) returned `200` - no route ever surfaced a 5xx, a stale-serialization error, or an unhandled exception to the HTTP layer under any tested concurrency level.

The 18 "other errors" (`4R/3W` **only) are a harness artifact, not a demonstrated application defect.** They are `IntegrityError: duplicate key value violates unique constraint "talent_review_candidates_pkey"` / `"uq_talent_review_candidates_assessment"`, caused by this task's own `writer_worker` test code computing a new primary-key id via `SELECT max(id)+1` before each concurrent insert - a non-atomic pattern that is unsafe under concurrent writers by construction. This does **not** reflect real application behavior: `models.py` declares `id = Column(Integer, primary_key=True)` for both tables, and real application insert paths (outside M10, which is read-only) that call `session.add(...)` without supplying an explicit `id` rely on PostgreSQL's native per-table identity/sequence-backed auto-increment, which is atomically safe under concurrent writers by design. What this artifact *does* usefully confirm: PostgreSQL correctly rejected every conflicting insert with a clean `IntegrityError`, every writer thread's own `except`/`session.rollback()` recovered without corrupting state or crashing, and no error of this kind ever reached or affected the reader/HTTP layer - a real, if incidental, confirmation of graceful conflict recovery under contention.

Per-route latency (min/p50/p95 ms) at the highest tested concurrency (4R/3W, 12 samples/route):

Route	min	p50	p95
overview	50.7	110.2	178.0
talent_map	54.2	88.9	99.7
program_portfolio	79.7	94.9	105.9
branch_intelligence	70.9	89.5	102.5
participation_overlap	51.4	76.8	99.4
student_drill (limit=100)	100.5	153.1	244.0
longitudinal	35.6	117.6	128.8

Local, non-production, in-process `TestClient` latency (no network hop, no production SLO exists or is implied). Latency increases with concurrency in every route (expected, single small local PostgreSQL instance) but no route became pathological or timed out (`statement_timeout=30000ms` was configured and never triggered).

Phase 4 - Participation Overlap Under Concurrency, Statistics Freshness, Larger Scale

4a. Statistics freshness, MEDIUM scale, controlled cold-vs-warm

Same MEDIUM dataset as Phase 1-3, freshly built, **before any** `ANALYZE` (`pg_stat_user_tables.last_analyze`/`last_autoanalyze` confirmed `NULL` at capture time):

	Cold (no `ANALYZE`)	Warm (after `ANALYZE`)
Plan shape	Nested Loop (planner underestimated `rows=1`, actual `rows=1567`; 1,080-row `Unique`/`Sort` inner re-executed)	Hash Join
Execution time (`EXPLAIN ANALYZE`)	214.8 ms	5.68 ms
Speedup	-	**37.8x**

This directly reproduces, under fully controlled conditions (this task explicitly withheld `ANALYZE` rather than relying on incidental timing), the same MEDIUM-scale Nested-Loop-with-repeated-subplan pathology B11-C observed under natural profiling-run timing - and shows it is **fully and completely resolved by** `ANALYZE`, not merely reduced. This is new, more controlled evidence than B11-C had for the MEDIUM scale specifically (B11-C's MEDIUM observation was not isolated from ambient autovacuum timing).

4b. Concurrent readers, MEDIUM scale, post-`ANALYZE`

4 reader threads x 10 iterations = 40 concurrent `participation-overlap` HTTP calls against the same warm MEDIUM dataset: **all 40 returned** `200`; latency 19.7-148.8 ms (avg 60.6 ms). No plan instability, error, or timeout was observed under concurrent reader pressure once statistics were fresh.

4c. Bounded larger-scale ("XL") retest

Target: "10x-100x increase in the dimension that drives Participation Overlap risk," bounded per this task's explicit resource-safety instruction ("Stop if local resource safety becomes questionable... the objective is evidence, not maximum scale"). This task built and measured an XL scale of **~2.95x B11-C's LARGE scale** (10 Branches, 14 Programs, 67,201 frozen population rows, 30,722 distinct Students, 105 Program pairs, `ANALYZE`d): build took 25.2s; the same `EXPLAIN (ANALYZE, BUFFERS)` participation-overlap query executed in **224.8 ms** using a **Hash/Merge Join** plan (not the pathological Nested Loop) - i.e., the good plan shape continued to hold at nearly 3x B11-C's LARGE scale with fresh statistics. **A full 10x-100x retest (roughly 228K-2.28M population rows) was not attempted in this task**

1 it was assessed as a materially higher local build-time/resource

commitment than this session's time budget could safely absorb interactively, and is explicitly deferred to B11-E as unfinished work, not silently skipped.

Index Candidate B - Reclassification

B11-C classified Index Candidate B (a `student_id`-leading index on `talent_assessment_cycle_population_members`) as **MORE EVIDENCE REQUIRED**, because its observed MEDIUM-scale instability did not monotonically correlate with scale (MEDIUM was slower than LARGE) and its cause was not yet isolated. This task's controlled 4a experiment isolates the cause precisely: **the MEDIUM-scale instability is fully explained by statistics freshness, not by the absence of a `student_id`-leading index** - the byte-identical MEDIUM query, on the byte-identical data, with no index added or removed, goes from a pathological 214.8 ms Nested Loop to a 5.68 ms Hash Join purely by running `ANALYZE`. Combined with 4c (a ~3x-larger-than-LARGE scale also produces a good Hash/Merge Join plan without the index, once `ANALYZE`d) and B11-C's own LARGE-scale evidence (also a good plan without the index), **every scale tested across both B11-C and this task (SMALL, MEDIUM, LARGE, XL) produces an acceptable plan without a `student_id`-leading index once PostgreSQL statistics are fresh.**

This task's classification: leaning `NO CHANGE`, on the basis that the previously-unexplained non-monotonic instability is now fully attributed to a separate, already-tracked statistics-freshness finding rather than to index absence. This task does **not** unilaterally close Index Candidate B - per the governing task's instruction, this remains a classification-with-evidence for independent review, not a final governance decision, and **no index is created or approved by this document**.

Statistics-Freshness Conclusion

The pathological no-statistics plan is real and reproducible (confirmed again, under tighter control, in 4a), but its practical relevance is bounded: it requires (a) `n_live_tup`/`last_analyze` to be genuinely absent for the specific table, and (b) a `participation-overlap`-shaped self-join query to run before autovacuum's background `ANALYZE` completes. This task's own Phase 3 writer scenarios (which perform small, normal, one-row-at-a-time domain writes against an already-`ANALYZE`d table, exactly like real steady-state application traffic) never reproduced this pathology - it is specifically tied to a **large, synchronous, bulk-style write** (an import, a migration, or an unusually large single Cycle-population-freeze operation) immediately followed by an analytics read, not to normal incremental steady-state writes. This remains a genuine, actionable **operational/ runbook** finding (e.g., an explicit `ANALYZE` step after any large bulk population load, before the first analytics read) rather than an index or isolation-level finding; no server-wide setting was changed and no runbook was adopted by this document.

Memory / RSS Evidence (DIRECTIONAL LOCAL WINDOWS DEVELOPMENT EVIDENCE)

Not a production memory ceiling; single local Windows PostgreSQL/Python process, cumulative across scenarios within one run.

Scenario	RSS before	RSS after	Delta
1R/1W (21 calls)	109.75 MB	110.33 MB	+0.58 MB
4R/1W (84 calls)	114.42 MB	114.61 MB	+0.19 MB
4R/3W (84 calls, incl. 18 rolled-back writer conflicts)	116.29 MB	119.10 MB	+2.81 MB

No unbounded/retained growth pattern was observed across increasing concurrency; the largest delta (4R/3W) remains a small, one-off increase consistent with additional concurrent thread/connection/response-buffer overhead, not a leak signature. This does not establish a multi-process/ multi-worker production memory ceiling (explicitly deferred to B11-E per ADR 0028's separate process-count-multiplier requirement, which this single-process harness cannot measure).

Privacy-Closure Timing

Per-route coarse latency under concurrency is captured above (Phase 3 table). Consistent with B11-C's own deferral, this task did not build separate fine-grained instrumentation to split SQL-aggregation time from Cell/Relationship-construction time from privacy-policy-evaluation time from reconstruction-closure time from serialization time - doing so would require additional invasive in-process instrumentation not justified purely for qualification profiling. This limitation is carried forward unresolved to B11-E, exactly as B11-C already stated.

Error / Deadlock / Retry Behavior

Zero deadlocks and zero lock-wait timeouts were observed in any scenario (1R/1W, 4R/1W, 4R/3W, or the Phase 4b concurrent-reader participation-overlap burst) - consistent with this task's writers never taking conflicting row-level locks on the same row at the same time (each writer iteration selects a small, essentially-disjoint slice of candidate rows via `LIMIT 5` before acting on the first eligible one). The only errors observed were the harness-artifact `IntegrityErrors` described in Phase 3, each cleanly recovered via `session.rollback()`. **No retry logic exists today** in `dependencies.get_db`, in any M10 route, or anywhere this task added - none was invented, per this task's explicit instruction. If governance later judges that serialization-failure/deadlock retry handling is needed (most relevant if/when a stronger isolation level is ever adopted, since `REPEATABLE READ` can raise serialization failures that `READ COMMITTED` cannot), that remains an explicit **B11-E-level implementation proposal**, not adopted here.

Observability Validation

`talent_organization_analytics_observability.OrganizationAnalyticsObservation` was re-inspected under this task's concurrency lens: it issues no SQL of its own, carries only the existing bounded allowlist (`SAFE_STRUCTURAL_FIELDS`/`SAFE_PROVIDER_FIELDS` - counts/limits/outcomes only, never a Student identifier,

raw/suppressed analytical value, SQL parameter, threshold, or request/response body), is instantiated fresh per request (no shared mutable state across concurrent requests to race on), and every public method (`record`/`emit`) wraps its own body in `try/except Exception: pass`-style safe-failure handling that cannot alter the response already computed by the route. This module was not modified by this task; the Phase 3 stress scenarios exercised it identically to a production-shaped request stream (189 requests total) with no import, logging, or serialization failure observed. No new concurrency-sensitive code path exists in this module - its safety-by-construction conclusion from B11-B is unchanged.

REPEATABLE READ Experiment - Result And Required Scope

Performed for both proven inconsistencies (Phase 1 `overview`, Phase 2 `students`) via `Connection.execution_options(isolation_level="REPEATABLE READ")` on the reader connection only - never a server-wide or application-wide change, and never made permanent. **Result:** `REPEATABLE READ` **fully resolved both demonstrated inconsistencies** - every statement in the reader's transaction, however many are issued, observed the same fixed snapshot taken at the first statement, so no concurrent commit was visible partway through either request.

Required scope if ever adopted (per ADR 0028): the current `frozen_membership_query` re-execution pattern means a `REPEATABLE READ` snapshot must be established **before the very first statement of the request** (i.e., before `authorized_program_universe`/context resolution) to cover the full evidence set ADR 0028 names - access context, aggregate reads, Candidate/Identification reads, and privacy-closure inputs (privacy closure itself issues no SQL, so it is automatically covered once every SQL input to it shares one snapshot). The current `dependencies.get_db` opens a default-isolation session with no isolation-level control point; adopting `REPEATABLE READ` for M10 would require a dedicated session/dependency (scoped to these seven routes only, not applied repository-wide) that sets the isolation level before the first query - an application-code change this task does **not** implement, per its explicit scope boundary.

Is A Stronger Transaction Scope Justified?

Yes, evidence now exists (Phase 1 and Phase 2 above) that `READ COMMITTED`'s default per-statement-snapshot behavior allows one M10 HTTP request to combine SQL statements that observe different committed database states, in ways that match this task's own named risk examples exactly. Per ADR 0028's explicit gate ("Stronger isolation requires demonstrated inconsistency evidence and a governed ADR"), the evidence half of that gate is now satisfied by this document; the governed-ADR half is **not** - no ADR proposing an isolation change exists yet. This task's evidence supports the following governance proposal for independent review:

PROPOSED - GOVERNANCE REQUIRED: adopt `REPEATABLE READ` (not a higher level; the Phase 1/2 experiment shows `REPEATABLE READ` is sufficient to resolve every demonstrated inconsistency) for the seven M10 Organization Intelligence routes only, scoped from context resolution through privacy-closure inputs as described above, via a dedicated session/dependency

1 not implemented by this document, not adopted, and not applied outside

M10.

Are Retries Required?

Not demonstrated as required by this task's evidence: no serialization failure or deadlock was observed in any `READ COMMITTED` scenario tested (see Error / Deadlock / Retry Behavior above), and the isolated `REPEATABLE READ` experiment above did not itself trigger a serialization failure in this task's read-only-reader-vs-independent-writer shape. If `REPEATABLE READ` is later adopted for these routes, PostgreSQL's serialization-failure (`40001`) class becomes reachable in principle for read/write conflict shapes this task did not specifically construct; this task recommends (not implements) that any future `REPEATABLE READ` adoption proposal separately evaluate and, if needed, add bounded retry handling for that specific error class - an explicit B11-E-level implementation proposal, not adopted here.

Regression

Command run (from repository root, using the pinned repo venv):

```
.\.venv\Scripts\python.exe -m pytest tests/test_talent_org_intelligence_contract.py
tests/test_talent_org_intelligence_queries.py
tests/test_talent_organization_overview.py
tests/test_talent_organization_talent_map.py
tests/test_talent_organization_program_portfolio.py
tests/test_talent_organization_branch_intelligence.py
tests/test_talent_organization_participation_overlap.py
tests/test_talent_organization_student_drill.py
tests/test_talent_organization_longitudinal.py
tests/test_talent_organization_observability.py -q
```

Result: **209 passed** (0 failed, 0 errors) - identical count to B11-C, confirming this task made no M10 semantic change. `tis.db` was verified unchanged (`git status --porcelain -- tis.db` / `git diff --stat -- tis.db` both empty) both immediately before and immediately after this run - no restoration was needed this time.

Confirmed by this regression run plus direct inspection of this task's own diff: no schema, migration, index, permission, or entitlement change; no M10 route/service/serializer semantics change; no privacy-semantics change (the existing `AllowAllTestPolicy`/`AllowAvailability`/`AllowBreadth` doubles were reused exactly as the existing suite already uses them); no PostgreSQL isolation change (the default remains `READ COMMITTED` in application code - every `REPEATABLE READ` use in this task was an isolated, non-permanent, reader-connection-only experiment inside the throwaway harness, never touching `dependencies.py`/`database.py`).

Exact Dataset Dimensions Used

| Dimension | MEDIUM (Phases 1-4b) | XL (Phase 4c) | |---|---:|---:| | Branches | 4 | 10 | | Programs | 6 | 14 | | Frozen population members | 1,501 | 67,201 | | Distinct Students | 707 | 30,722 | | Assessments | 1,178 | 53,571 | | Candidates | 314 | 11,134 | | Identifications | 154 | 5,610 | | Program pairs (`P(P+1)/2`) | 21 | 105 | | Unassessed (write-target) members | 322 | 13,629 |

Known Limitations

1 Only one actor shape was tested (`ORGANIZATION`-scope, role `Editor`, all

five relevant permission keys granted) - identical limitation to B11-C.

1 Only the non-production `AllowAllTestPolicy` (no suppression) was used;

the privacy-reconstruction-under-concurrency question cannot be empirically evaluated until a real threshold-based provider exists (ADR 0028-deferred).

1 `program-portfolio`, `branches/{id}`, `talent-map`, and `longitudinal`

were confirmed structurally exposed to the same cross-statement mechanism by code reading, but were **not** independently reproduced with a live concurrent writer in this task (time-bounded); `overview` and `students` were.

1 The full 10x-100x Participation Overlap larger-scale retest was not

attempted; this task reached ~2.95x B11-C's LARGE scale (XL, 67,201 population rows) and stopped there for session time/resource safety, per this task's own explicit instruction to prioritize evidence over maximum scale.

1 This harness runs entirely in one local Python process

(`TestClient`, in-process, no real network hop) with one PostgreSQL connection pool; it does not measure multi-process/multi-worker (e.g. multiple `gunicorn`/Render Web Service instance) concurrency, which ADR 0028 separately requires evidence for (process-count multiplier) before B11-E.

1 The 18 `IntegrityErrors` in the 4R/3W scenario are a harness-code

artifact (non-atomic `max(id)+1` in this task's own writer test code), not a demonstrated production defect - see Phase 3 discussion.

1 No fine-grained privacy-closure phase timing was built (SQL vs.

structure-construction vs. policy-evaluation vs. reconstruction-closure vs. serialization) - same limitation B11-C already carried forward.

1 Memory/RSS evidence is single-process, cumulative-within-run, directional

local Windows development evidence only - not a production ceiling.

Deferred To B11-E

1 Production privacy provider implementation/configuration and the

privacy-reconstruction-under-concurrency question that depends on it.

1 Independent, per-route empirical reproduction of the cross-statement

inconsistency mechanism for `program-portfolio`, `branches/{id}`, `talent-map`, and `longitudinal` (currently extrapolated, not each independently proven).

1 Governance decision (a dedicated ADR) on whether to adopt `REPEATABLE

READ` for the seven M10 routes, and if adopted, its implementation (dedicated session/dependency) and any resulting retry-handling need.

1 The remaining 10x-100x Participation Overlap larger-scale retest beyond

this task's ~2.95x XL result.

1 Multi-process/multi-worker concurrent-request memory and consistency

qualification (this harness is single-process only).

1 Fine-grained privacy-closure phase timing, if ever judged necessary

without invasive production instrumentation.

1 Final governance disposition of Index Candidate B (this task's evidence

leans `NO CHANGE`; no index is created or approved by this document).

1 Any numeric privacy threshold, breadth limit, performance SLO, memory

ceiling, or commercial/plan mapping (ADR 0028 Deferred Decisions - unchanged by this document).

Files Touched By This Qualification Task

1 `docs/history/engineering-handbook/2026-09-07-b11d-postgresql-concurrency-consistency-qualification.md`

(this file, new).

1 `docs/history/engineering-handbook/README.md` (added a navigation bullet

to this file; existing lines otherwise preserved as found).

1 `docs/CHANGE_HISTORY.md` (added a new dated entry; no existing lines

changed).

1 `docs/PROJECT_STATE.md` (added a new section; no existing lines changed).

1 `docs/adr/0028-b11-production-qualification-policy.md` (appended a new

B11-D status paragraph after the existing Status section; the pre-existing uncommitted B11-C wording already present in that section - see the Working-Tree Integrity Note above - was left completely untouched).

1 `.kms-impact.yml` (task-specific declaration, including a transparency

note mirroring the one above).

1 `static/docs/TIS_Project_Reference_Booklet.pdf` and

`static/docs/docs_manifest.json` (regenerated automatically by `python scripts/kms.py sync`, not hand-edited).

No application source file (`routers/`, `talent_org_*.py`, `talent_analytics_*.py`, `models.py`, `db_migrations.py`, `database.py`, `dependencies.py`) was modified. No index, migration, permission, entitlement, privacy-semantics, or PostgreSQL isolation change was made. No harness file was committed to the repository (its full methodology is reproduced above for reproducibility). `tis.db` was confirmed unchanged before and after this task, including after the regression run above.

M10 B11-E Integrated Production Qualification (REPEATABLE READ Implementation)

docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md

2026-09-08 - M10 B11-E Integrated Production Qualification

Module: Talent & Potential M10 Organization Analytics, integrated isolation governance/implementation and privacy/concurrency release-gate qualification (ADR 0028, ADR 0029, B11-E entry)

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-09-08 - M10 B11-E Integrated Production Qualification

Related ADRs: `docs/adr/0028-b11-production-qualification-policy.md` (governing B11-E release gate); `docs/adr/0029-m10-organization-analytics-repeatable-read-boundary.md` (this task's governed isolation decision)

Related prior evidence: `docs/history/engineering-handbook/2026-09-07-b11c-postgresql-profiling-evidence-remediation.md` (dataset-builder pattern, query-count baselines, Index Candidate A/B, statistics-freshness finding); `docs/history/engineering-handbook/2026-09-07-b11d-postgresql-concurrency-consistency-qualification.md` (proven READ COMMITTED same-request mixed-snapshot evidence for Overview and Student Drill; REPEATABLE READ experiment; PROPOSED - GOVERNANCE REQUIRED disposition)

Reviewer/approval notes: This task governs (ADR 0029) and IMPLEMENTS the M10-only REPEATABLE READ transaction boundary B11-D left as PROPOSED - GOVERNANCE REQUIRED, live re-tests all seven routes under the permanent implementation, tests suppression/reconstruction concurrency with a real deterministic suppressing non-production policy, and finalizes Index Candidate B and statistics- freshness disposition. It does not implement B12, does not merge to master, and does not deploy. It does not add an index or migration. It does not change PostgreSQL isolation server-wide or for any route outside M10. It does not choose or implement a production privacy/availability/breadth provider value or mapping - those remain Owner decisions per ADR 0028. **Independent review returned PASS WITH NON-BLOCKING OBSERVATIONS. B11-E implementation is PASSED and checkpointable, but this document does not declare B11-E production closure or B11 overall CLOSED.**

`tis.db` was not touched (confirmed unchanged before and after this task's full regression run). All live measurement used the dedicated non-production `tis_b11c_test_db` PostgreSQL 16.15 instance at `127.0.0.1`, reached only through the `TIS_TEST_POSTGRESQL_URL` environment variable (never printed, logged, or persisted anywhere in this repository), inside ephemeral per-test schemas (`tis_b11e_<uuid>`) created and dropped by each test - no rows were left behind in the shared test database (confirmed by an explicit post-run `information_schema.schemata` check finding zero leftover `tis_b11e_%` schemas).

Current Milestone Truth

1 B0-B10: CLOSED. B11-A: complete. B11-B: CLOSED. B11 production-qualification

policy (ADR 0028): APPROVED/GOVERNED. B11-C: CLOSED. B11-D: CLOSED.

1 B11-E (this document): REPEATABLE READ governed (ADR 0029) and

IMPLEMENTED for the seven M10 Organization Intelligence routes only; all seven routes live-re-tested under the permanent implementation; suppression/reconstruction concurrency tested live with a real non-production deterministic

suppressing policy; Index Candidate B finalized as NO CHANGE; statistics-freshness disposition finalized as operational/runbook guidance (no code/migration action). Independent review of this document returned **PASS WITH NON-BLOCKING OBSERVATIONS** (consistent with the Reviewer/approval notes above and the final bullet below; this line previously read "PENDING" and was corrected on 2026-09-09 as a self-consistency fix - see `docs/history/engineering-handbook/2026-09-09-b11e-f2-postgresql-qualification.md`).

1 No index, migration, permission, entitlement, privacy-semantics change,

production SLO, production privacy threshold, production breadth limit, or production memory ceiling is approved by this document. The production privacy/availability/breadth providers remain unimplemented and fail-closed (`talent_analytics_privacy.resolve_privacy_policy_provider`, `talent_org_intelligence_service.resolve_organization_analytics_availability_provider`, `talent_org_intelligence_service.resolve_organization_analytics_breadth_policy` all return `None` in production, unchanged by this task) - all three remain explicit B11-E closure blockers, Owner-decision-dependent per ADR 0028.

1 **B11-E implementation is PASSED and checkpointable after independent review

returned PASS WITH NON-BLOCKING OBSERVATIONS. B11-E production closure remains PENDING; B11 overall is NOT CLOSED. B12 is NOT IMPLEMENTED. Production readiness is NOT achieved.** This was accurate as of this document's own 2026-09-08 checkpoint. As of 2026-09-09, following `docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md` and the resulting ADR 0028/ADR 0030 adoption, this status is superseded: B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING, B11 overall is CLOSED on that same basis, and B12 is DEFINED, NOT YET CLOSED. Production deployment has still not occurred.

Phase 0 - Connection Verification (Non-Production)

| Check | Result | |---|---| | `TIS_TEST_POSTGRESQL_URL` present in process environment | Yes | | Connection succeeds | Yes | | Database | `tis_b11c_test_db` (same dedicated non-production instance B11-C/D used) | | Host | `127.0.0.1` (localhost only) | | Value ever printed/logged/persisted | No |

Phase 1/2 - Governance And Implementation

Recorded in full in ADR 0029 (`docs/adr/0029-m10-organization-analytics-repeatable-read-boundary.md`). Summary:

1 **Mechanism**: `database.py` adds `M10OrganizationAnalyticsSessionLocal`, a

`sessionmaker` bound to `engine.execution_options(isolation_level="REPEATABLE READ")`

1 a shallow proxy sharing `engine`'s connection pool without altering

`engine`'s own default isolation. Strictly backend-conditional (`DATABASE_URL.startswith("postgresql")`); on SQLite it binds to the plain `engine` unchanged.

1 **Dependency**: `dependencies.py` adds `get_m10_organization_analytics_db`,

mirroring `get_db`'s exact `SessionLocal() -> yield -> close()` shape.

1 **Wiring**: `routers/talent_organization_analytics.py` uses

`Depends(get_m10_organization_analytics_db)` instead of `Depends(get_db)` for all seven route functions (`overview`, `talent-map`, `program-portfolio`, `branches/{branch_id}`, `participation-overlap`, `programs/{program_id}/longitudinal`, `students`). No other route/module changed.

1 **Test wiring**: the eight existing M10 route test files

(`tests/test_talent_organization_{overview,talent_map,program_portfolio, branch_intelligence,participation_overlap,student_drill,longitudinal, observability}.py`) were updated to override `get_m10_organization_analytics_db` instead of `get_db` (their SQLite `db` fixture is unaffected; only the dependency-override target changed). All 209 pre-existing M10 tests pass unmodified in behavior.

1 **Scope begins before the first statement**: confirmed both by code

reading (context/access resolution is every route's first DB call, via `_b7_context` -> `svc.resolve_access_context`) and live-proven in Phase 4C below (the interleaved write happens AFTER `frozen_membership_query` is called - itself well after context resolution - and is still invisible under `REPEATABLE READ`, proving the snapshot was already fixed earlier).

Phase 3 - Retry / Failure Semantics

No serialization failure, deadlock, or lock-wait timeout was observed in any live scenario in this task (14 new committed tests, each exercising a real interleaved concurrent writer against the new `REPEATABLE READ` session; see Phase 4/5 below). Matches B11-D's own finding. No retry logic was added - recorded in ADR 0029 as a deliberate non-decision pending future evidence, not implemented speculatively.

Phase 4 - Live Consistency Re-Test (All Seven Routes)

New, committed, repository test file: `tests/test_talent_organization_repeatable_read.py` (14 tests, all passing; not a throwaway harness - this is real regression coverage going forward). Methodology: reuses the B11-C/B11-D dataset-builder pattern (`models.Base.metadata.create_all` against a dedicated ephemeral per-test PostgreSQL schema, the real M10 routers mounted on an isolated `FastAPI()` app via `TestClient`, the project's existing non-production `AllowAvailability`/`AllowBreadth`/ `AllowAllTestPolicy` doubles from `tests/test_talent_org_intelligence_queries.py` and `talent_analytics_privacy.py`). Concurrency is exercised via the same deterministic "commit a real domain-valid write between two of the route's own statements" technique B11-D used (a `monkeypatch`-wrapped service function commits a write via a separate writer `Session` immediately after its own return value is computed, before the caller's next statement runs) - chosen, as B11-D also chose, for full reproducibility over a non-deterministic thread-timing race.

Dataset: 2 primary Programs (Alpha, Beta) sharing Student overlap across 3 Cycles, 1 tiny isolated foreign-tenant control; 14 frozen population members, 10 distinct Students, 1 Candidate, 1 Identification; several already-frozen `unassessed` members reserved as live writer targets.

4A - Overview (live, re-tested)

`test_overview_repeatable_read_resolves_candidate_snapshot_mismatch`. Reproduces B11-D's exact statement pair (`coverage_organization_total` then `candidate_membership_counts`). **Result: under the permanent** `REPEATABLE READ` **implementation, the Candidate count no longer changes within the same request** (stays at the pre-write value). **Contrast, same test**: an identical interleave against a plain default (`READ COMMITTED`) session, run second against the same live dataset, reproduces the original vulnerability live (the Candidate count reflects the interleaved write) - confirming the fix is necessary, not merely asserted from the earlier B11-D document.

4B - Student Drill (live, re-tested)

`test_student_drill_repeatable_read_resolves_gate_page_mismatch`. Reproduces B11-D's exact statement pair (`count_distinct_students` gate, then `fetch_student_rows` page fetch). **Result: under** `REPEATABLE READ`**, the gate count and the page's Student count match exactly, and the newly committed Student is absent from the page. Contrast**: the same interleave under plain `READ COMMITTED` reproduces the

original gate/page mismatch live (the new Student appears in the page).

4C - Talent Map, Program Portfolio, Branch Intelligence, Participation Overlap, Longitudinal (live, newly independently reproduced)

`test_other_routes_repeatable_read_scope_begins_before_first_statement` (parametrized, one case per route). These four routes were only code-inspection-extrapolated, not independently live-reproduced, in B11-D. This task independently reproduces the identical vulnerability live for all five, using a stronger, earlier interleave point: the writer commits immediately after `svc.frozen_membership_query` builds its lazy `Query` object - before ANY of that route's own downstream SQL has executed - proving the transaction scope genuinely begins before the first statement of the request (fixed during earlier context/access resolution), not merely between two already-identified calls. **Result for all five routes: under `REPEATABLE READ`, the interleaved write is not observable anywhere in the response (verified via a recursive sum of every JSON `"value"` field); under plain `READ COMMITTED`, the same interleave IS observable (the sum increases).** All five routes returned HTTP 200 in every scenario.

Phase 5 - Suppression/Reconstruction Concurrency (Real Non-Production Suppressing Policy)

Uses `talent_analytics_privacy.DeterministicSuppressionTestPolicy` - an existing, already-committed, explicitly test-only/non-production deterministic policy (`minimum_cohort` threshold; below it, `suppressed`) already reused by `tests/test_talent_organization_overview.py`'s own primary-suppression and complementary-suppression regression tests. This task did NOT invent a new policy or any production-shaped threshold, per the task's explicit instruction to check for and reuse an existing deterministic suppressing test policy first.

1 **Primary suppression** (`test_primary_suppression_is_immune_to_a_concurrent_write_crossing_the_threshold`):

`minimum_cohort=2` suppresses the Candidate Cell (raw value 1). The interleaved writer commits exactly one more Candidate - enough to cross the threshold to `visible` if it were observed. **Result: the response remains `{"state": "suppressed"}` regardless of the concurrent write; no raw value or threshold text leaks (verified by the same token-based leak check the existing Overview test suite already uses).**

1 **Complementary suppression** (`test_complementary_suppression_relationship_is_immune_to_a_concurrent_write`):

`minimum_cohort=3` suppresses `completed` (raw value 2), which is relationship-linked to `coverage_n` in the Overview route's own equality Relationship. The interleaved writer completes one more Assessment (no Candidate) - enough to cross the threshold if observed. **Result: under `REPEATABLE READ`, `completion_coverage` remains fully `{"state": "suppressed"}` (no partial numerator/denominator/percentage leak). Contrast: the identical interleave under plain `READ COMMITTED` DOES flip `completion_coverage` to `visible`** - a live, reproduced instance of the reconstruction-relevant risk B11-D flagged as structurally possible but unmeasured (a suppression decision fed by operands from two different snapshots), now proven to be resolved by the ADR 0029 fix.

1 **Reconstruction closure**: `talent_analytics_relationship_graph/`

`talent_analytics_privacy_closure` issue no SQL of their own (confirmed by code reading, matching B11-D's conclusion); since every SQL input they consume now shares one snapshot per request (Phase 4 above), no additional closure-specific concurrency test was needed beyond the two above, which already exercise closure end-to-end.

1 **No suppressed value became reconstructable**, and no identifiable

Student data leaked, in any scenario tested.

Tenant Isolation

`test_tenant_isolation_holds_under_the_new_session_with_a_foreign_tenant_write`. Every M10 query filters by `context.school_group_id` (confirmed by code reading, matching B11-D). A foreign-tenant write (a new Student/Placement/ frozen population member in the isolated foreign `SchoolGroup`) committed immediately before a primary-tenant `REPEATABLE READ` request. **Result: the primary tenant's response was unaffected** (`frozen_eligible_memberships` stayed at 14; no foreign identifier appeared anywhere in the response body). This is a deterministic committed-before-read check, not a true multi-threaded race, consistent with this task's chosen (and B11-D's own preferred) deterministic-interleave methodology.

Provider Readiness (Phase 6)

| Provider | Implemented | Configured | Fail-closed | Blocked on | |---|---|---|---|---|---| | Privacy (`talent_analytics_privacy.resolve_privacy_policy_provider`) | No (returns `None`) | No | Yes (confirmed unchanged) | Owner-approved numeric P1-P7 thresholds/provider (ADR 0028 Deferred Decisions) | | Organization analytics availability (`talent_org_intelligence_service.resolve_organization_analytics_availability_provider`) | No (returns `None`) | No | Yes (confirmed unchanged) | Owner-approved commercial/plan-feature mapping (ADR 0028 Deferred Decisions) | | Breadth policy (`talent_org_intelligence_service.resolve_organization_analytics_breadth_policy`) | No (returns `None`) | No | Yes (confirmed unchanged) | Owner-approved numeric breadth limits (ADR 0028 Deferred Decisions) |

No production value, plan name, feature key, or numeric threshold/limit was invented or implemented by this task, per ADR 0028 and the task's explicit instruction. These three remain explicit B11-E closure blockers for the repository owner.

Index Candidate B - Final Classification: NO CHANGE

B11-C observed non-monotonic MEDIUM/LARGE instability; B11-D isolated the cause to statistics freshness (a missing post-bulk-load `ANALYZE`), not index absence, and reconfirmed a good plan shape at a ~2.95x-LARGE ("XL") scale. This task's own live directional check (Phase 9 below) reconfirms the `participation-overlap` query count is unchanged (8 queries, matching every B11-C scale exactly) and low-latency under the new `REPEATABLE READ` session

1 the isolation-level change has no mechanism by which it could alter query

planning or statistics, so it does not and cannot affect this classification. **Final classification: NO CHANGE.** No index is created or approved by this document. The remaining 10x-100x larger-scale retest B11-D deferred remains explicitly unfinished (not required to close this classification, per the task's own "if evidence remains healthy across B11-C/B11-D/B11-E scales: prefer NO CHANGE" guidance).

Statistics Freshness - Disposition: Operational/Runbook Guidance (No Code/Migration Action)

B11-C/B11-D's finding (a large synchronous bulk population-freeze load, before autovacuum's background `ANALYZE` completes, followed immediately by a `participation-overlap`-shaped read, produces a pathological ~37x-680x slower Nested-Loop plan that is fully resolved by running `ANALYZE`) is an operational/runbook concern, not an index or isolation-level defect - the isolation-level change made by this task has no bearing on it. **Disposition: an explicit `ANALYZE` step (or equivalent PostgreSQL `autovacuum`/`autoanalyze` verification) after any large, synchronous bulk Cycle-population-freeze operation, before the first analytics read, is sufficient.** This is documented here as the governed disposition; no code, migration, or server-wide PostgreSQL setting is changed by this document, consistent with the task's instruction to govern (not implement) any further action this session.

Phase 9/11 - Directional Memory/Performance/Query-Count Evidence (DIRECTIONAL LOCAL WINDOWS DEVELOPMENT EVIDENCE)

Not a production ceiling or SLO; single local Windows PostgreSQL/Python process, small qualification-scale dataset (14 population members - this is a functional re-verification after the isolation change, not a new scale study; B11-C/B11-D's SMALL/MEDIUM/LARGE/XL scale evidence is unaffected and still authoritative for scale behavior). Each of the seven routes called twice under the new `REPEATABLE READ` session, via a non-committed local script following the same `QueryCounter`/`psutil` RSS convention B11-C established:

Route	Query count (1st/2nd call)	Latency ms (1st/2nd)	RSS delta across both calls
overview	10 / 10	68.4 / 21.7	+0.06 MB
talent_map	9 / 9	17.8 / 15.2	+0.05 MB
program_portfolio	11 / 11	23.6 / 23.5	+0.04 MB
branch_intelligence	11 / 11	31.5 / 17.3	+0.06 MB
participation_overlap	8 / 8	15.9 / 12.0	+0.01 MB
student_drill (limit=100)	11 / 11	22.9 / 21.3	+0.05 MB
longitudinal	11 / 11	27.1 / 16.6	+0.04 MB

Every query count is IDENTICAL to B11-C's original READ COMMITTED baseline for every route, at every scale B11-C tested (overview=10, talent_map=9, program_portfolio=11, branch_intelligence=11, participation_overlap=8, student_drill=11, longitudinal=11) - confirming `REPEATABLE READ` adds zero additional queries and changes no query shape. Baseline process RSS 96.87 MB; +9.3 MB after dataset build (14 rows, consistent with B11-C's own sub-linear per-row overhead finding); RSS flat (<0.1 MB delta per route) across repeated calls to every route - no retention/leak signature. Privacy-closure timing was not separately instrumented (same limitation B11-C/B11-D already carried forward - closure issues no SQL and operates on already-materialized Python values, so its cost is included in the coarse per-route latency above, not separately measurable without invasive instrumentation this task does not add). Multi-process/multi-worker concurrent-request memory (the process-count multiplier ADR 0028 separately requires) remains unmeasured by this single-process check, exactly as B11-D already disclosed - not newly resolved by this task.

Phase 10 - Observability/Security Regression

`talent_organization_analytics_observability.OrganizationAnalyticsObservation` was not modified by this task. Every one of the 14 new live tests exercised it identically to a real request stream (including failure-path emissions observed during initial debugging, all correctly classified as `database_failure`/other bounded categories with no SQL text, parameter, or Student identifier in the emitted telemetry - confirmed by direct inspection of the emitted JSON during this task's own debugging). Tenant isolation (above) and permission/authorization behavior (unchanged route logic, only the db dependency's isolation level changed) were confirmed clean.

Phase 12 - Regression

Commands run (repository venv):

```
.\.venv\Scripts\python.exe -m pytest tests/test_talent_org_intelligence_contract.py
tests/test_talent_org_intelligence_queries.py
tests/test_talent_organization_overview.py
tests/test_talent_organization_talent_map.py
tests/test_talent_organization_program_portfolio.py
tests/test_talent_organization_branch_intelligence.py
tests/test_talent_organization_participation_overlap.py
tests/test_talent_organization_student_drill.py
tests/test_talent_organization_longitudinal.py
tests/test_talent_organization_observability.py
tests/test_talent_organization_repeatable_read.py
tests/test_postgresql_migration_transactions.py -q
```

Result: **226 passed** (0 failed, 0 errors) - 209 pre-existing M10 tests (identical count and behavior to B11-C/B11-D, confirming no M10 semantic change) + 14 new committed `test_talent_organization_repeatable_read.py` tests (all live-PostgreSQL-gated tests ran, none skipped, since `TIS_TEST_POSTGRESQL_URL` was present) + 3 pre-existing PostgreSQL migration tests, unaffected.

```
.\.venv\Scripts\python.exe -m pytest tests/test_talent_privacy_reconstruction.py
tests/test_talent_privacy_relationship_graph.py tests/test_talent_analytics.py -q
```

Result: **124 passed** (0 failed, 0 errors) - privacy graph/reconstruction and broader M9 Talent Analytics regression, confirming no privacy-semantics change.

`tis.db` was verified unchanged (`git status --porcelain -- tis.db` / `git diff --stat -- tis.db` both empty) both before and after every regression run in this task. A post-run `information_schema.schemata` query confirmed zero leftover `tis_b11e_%` PostgreSQL schemas.

Confirmed by these regression runs plus direct inspection of this task's own diff: no schema, migration, index, permission, or entitlement change; no M10 route/service/serializer semantics change; no privacy-semantics change (the existing `AllowAllTestPolicy`/`AllowAvailability`/`AllowBreadth`/`DeterministicSuppressionTestPolicy` doubles were reused exactly as the existing suite already uses them); no PostgreSQL server-wide isolation change (the default remains `READ COMMITTED` for every route/session outside the seven M10 Organization Intelligence routes).

Known Limitations

1 Only one actor shape was tested (`ORGANIZATION`-scope, role `Editor`, all

relevant permission keys granted) - identical limitation to B11-C/B11-D.

1 The tenant-isolation and Phase 4C interleave techniques are deterministic

"commit before the next statement" interleaves, not true multi-threaded timing races - matching B11-D's own stated preference for full reproducibility, but this means true thread-scheduling-dependent races are not separately exercised by this task (B11-D's own Phase 3 multi-threaded HTTP-level stress, at 1R/1W, 4R/1W, and 4R/3W, remains the authoritative multi-threaded evidence and was not re-run this task, since it exercises the same session/statement mechanism this task's Phase 4 already re-verified is now fixed).

1 The Phase 9 directional check in this document uses a small

qualification-scale dataset (14 population members) to functionally re-verify query-count/latency/RSS after the isolation change; it is not a new SMALL/MEDIUM/LARGE/XL scale study - B11-C/B11-D's scale evidence remains authoritative and unaffected (isolation level has no mechanism to change query plans or scale behavior).

1 Multi-process/multi-worker (e.g., multiple Render Web Service worker

processes) concurrent-request memory and consistency qualification remains unmeasured, exactly as B11-D already disclosed - not resolved by this task.

1 No fine-grained privacy-closure phase timing was built - same limitation

B11-C/B11-D already carried forward.

1 The production privacy/availability/breadth providers remain

unimplemented; this task did not and could not test real-threshold suppression against a production-shaped provider, only the existing non-production `DeterministicSuppressionTestPolicy` - appropriate per ADR 0028's explicit deferral of numeric thresholds to Owner governance.

Files Touched By This Qualification Task

1 `docs/adr/0029-m10-organization-analytics-repeatable-read-boundary.md` (new).

1 `docs/adr/0028-b11-production-qualification-policy.md` (Status section and

PostgreSQL isolation/indexes section updated to record ADR 0029's scoped adoption; no other wording changed).

1 `docs/adr/README.md` (added ADR 0029 to the index).

1 `docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md`

(this file, new).

1 `docs/history/engineering-handbook/README.md` (added a navigation bullet).

1 `docs/CHANGE_HISTORY.md` (added a new dated entry).

1 `docs/PROJECT_STATE.md` (added a new section).

1 `docs/TIS_MASTER_CONTEXT.md` (added a new section).

1 `.kms-impact.yml` (task-specific declaration).

1 `static/docs/TIS_Project_Reference_Booklet.pdf` and

`static/docs/docs_manifest.json` (regenerated by `python scripts/kms.py sync`).

1 `database.py` (added `M10OrganizationAnalyticsSessionLocal`, backend-conditional).

1 `dependencies.py` (added `get_m10_organization_analytics_db`).

1 `routers/talent_organization_analytics.py` (all seven routes now depend on

`get_m10_organization_analytics_db` instead of `get_db`).

1 `tests/test_talent_organization_{overview,talent_map,program_portfolio,

branch_intelligence,participation_overlap,student_drill,longitudinal, observability}.py` (dependency-override target updated to match; no assertions/behavior changed).

1 `tests/test_talent_organization_repeatable_read.py` (new, committed, 14

tests: 4 backend-conditional wiring checks + 10 live PostgreSQL qualification tests).

No index, migration, permission, entitlement, or privacy-semantics change was made. `tis.db` was confirmed unchanged before and after this task, including after every regression run above.

M10 B11-E F2 PostgreSQL Qualification (Independent Re-Verification, Bounded)

docs/history/engineering-handbook/2026-09-09-b11e-f2-postgresql-qualification.md

2026-09-09 - M10 B11-E F2 PostgreSQL Qualification (Independent Re-Verification, Bounded)

Module: Talent & Potential M10 Organization Analytics, PostgreSQL production qualification re-verification (ADR 0028, ADR 0029, B11-E)

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-09-09 - M10 B11-E F2 Qualification Re-Verification (Bounded)

Related ADRs: `docs/adr/0028-b11-production-qualification-policy.md` (governing B11 qualification policy); `docs/adr/0029-m10-organization-analytics-repeatable-read-boundary.md` (REPEATABLE READ boundary this task re-verified)

Related prior evidence: `docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md` (B11-E implementation and governance record this task re-verifies a slice of)

Why This Document Exists

This task was asked to formally adopt, into ADR 0028 and a new ADR 0030, a set of specific numeric performance/memory/multi-worker qualification claims (p95/p99 latency figures, an incremental-memory figure, and a multi-worker qualification conclusion) attributed to unlogged scratchpad scripts from an earlier session that are not present in this repository and could not be re-run as originally written. This document records, honestly and narrowly, what this task actually did and did not verify, and explains why the broader formal ADR adoption was NOT performed.

What This Task Independently Re-Confirmed

1 **Credential bridging**: `TIS_TEST_POSTGRESQL_URL` was successfully bridged

from Windows User environment scope into a Bash session (121 characters, `postgresql+`-prefixed, never printed/logged/persisted anywhere in this repository or in any tool output).

1 **REPEATABLE READ suite**: `tests/test_talent_organization_repeatable_read.py`

was executed directly against the live non-production PostgreSQL test database. Result: **15 passed, 0 failed** (see raw pytest output captured during this task). This includes `test_entitlement_availability_check_shares_the_m10_repeatable_read_snapshot`, independently confirming the F1 entitlement-availability query path shares the M10 REPEATABLE READ snapshot with the rest of the request (no mixed-snapshot defect for that path).

1 **Bounded independent latency sanity check**: this task wrote a new,

throwaway (not committed) script reusing the exact `pg_dataset` / `_build_dataset` fixture and route-wiring helpers from `tests/test_talent_organization_repeatable_read.py`, and issued 40 sequential warm requests per route (after one warm-up request) to all seven M10 Organization Intelligence routes against the same live non-production PostgreSQL database, under the REPEATABLE READ session. Measured server-side round-trip latency (small fixture-scale dataset - 14 frozen population members, 10 Students, 1 Candidate, 1 Identification; single

sequential client, no concurrent load):

| route | p95 (ms) | p99 (ms) | max (ms) | |---|---|---|---| | overview | 19.77 | 20.92 | 22.71 | | talent_map | 21.13 | 25.00 | 26.58 | | program_portfolio | 31.79 | 32.45 | 37.63 | | branch_intelligence | 31.73 | 32.93 | 34.67 | | participation_overlap | 21.11 | 21.20 | 22.74 | | longitudinal | 24.52 | 24.74 | 25.57 | | students | 25.68 | 27.71 | 33.40 |

All observed latencies are comfortably within a p95 ? 2.0s / p99 ? 4.0s order of magnitude - by roughly two orders of magnitude - on this small fixture dataset with no concurrent load. **This does NOT reproduce or corroborate the specific "p95?170ms overview / p99?325.65ms participation_overlap" figures cited as the prior task's evidence; this task's own re-run produced materially different (lower) numbers.** This discrepancy is recorded honestly rather than resolved by preferring one unverifiable figure over the other. A meaningful production-scale qualification run (realistic dataset size, concurrent load, the full B11-C profiling harness methodology) was judged out of proportion for this task's bounded re-verification effort and was not performed.

What This Task Did NOT Verify And Declined To Formally Adopt

1 **Incremental memory (the cited "?0.48MB" figure)**: not measured by this

task. No memory-profiling script was built or run. This task has no independent basis to formally adopt this figure or a derived ceiling into ADR 0028.

1 **Multi-worker/multi-process qualification**: not measured by this task. No

multi-process/multi-worker test was built or run. This task has no independent basis to create ADR 0030 asserting a multi-worker qualification conclusion.

1 **Peak worker/process RSS as a percentage of an authoritative production

container memory allocation**: no such authoritative allocation value exists anywhere in this repository (confirmed by this task's document review); this remains an environment-specific production verification point regardless of any other finding here.

Given the above, this task explicitly **declined** to:

1 Edit ADR 0028's Performance/Memory/Deferred Decisions sections to formally

"adopt" or "APPROVE" the specific cited performance and memory figures as qualification targets with this task as their evidence basis. The only qualification-relevant evidence this task can personally stand behind is the bounded latency sanity check above, which supports (but does not itself fully qualify) a generous latency ceiling on a small dataset with no concurrent load - it is not a substitute for a full B11-C-style profiling run and should not be cited as if it were one.

1 Create `docs/adr/0030-*.md` asserting a multi-worker qualification

conclusion, since no multi-worker evidence was produced or reproduced by this task.

1 Add a B12 milestone-gate subsection to ADR 0028, in order to keep this

entry strictly to re-verification findings rather than mixing in new governance-structure decisions that were not the subject of independent evidence review.

Documentation Self-Consistency Fix (Independently Verified By Direct Reading)

This task separately re-read the 2026-09-08 B11-E integrated production qualification handbook entry in full.

Its "Current Milestone Truth" bulleted list contained an internal contradiction: the Reviewer/approval notes at the top of the document and the final bullet of the same list both already stated "Independent review returned PASS WITH NON-BLOCKING OBSERVATIONS," while one bullet in the middle of the same list still read "Independent review of this document is PENDING."

This is a same-document, same-milestone, few-lines-apart contradiction with no distinguishing language anywhere suggesting two separate review processes. This task corrected the stale middle bullet, and the matching line in the engineering-handbook README index, to match the document's own already-stated conclusion. This correction rests on this task's own direct reading of the primary text, not on any unverifiable claim about prior sessions.

B11-E Status (This Task's Assessment)

B11-E implementation-level review status: **PASS WITH NON-BLOCKING OBSERVATIONS** (per the self-consistency fix above, which reflects the document's own already-stated conclusion). B11-E **production closure** remains **NOT** declared by this task: the memory-ceiling and multi-worker qualification claims requested for formal ADR adoption were not independently verified or reproduced, so this task does not add them to the governed record. B11 overall remains NOT CLOSED. B12 is NOT IMPLEMENTED. No merge to master and no production deployment occurred or is implied by this document. This was this task's own honest, bounded assessment as of its 2026-09-09 execution. Later the same day, `docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md` independently produced and directly verified that same multi-worker/memory evidence itself (not a relay of any prior unverifiable figure) and formally adopted it into ADR 0028/ADR 0030; that later document's status is authoritative going forward: B11-E is CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING, B11 overall is CLOSED on that same basis, and B12 is DEFINED, NOT YET CLOSED.

`tis.db` was not touched by this task. All live measurement used the dedicated non-production PostgreSQL test database reached only through `TIS_TEST_POSTGRESQL_URL` (never printed, logged, or persisted). This task's own throwaway verification script is not committed to the repository.

M10 B11-E Live Multi-Worker, Performance & Memory Qualification (Direct Execution)

docs/history/engineering-handbook/2026-09-09-b11e-live-multiworker-qualification.md

2026-09-09 - M10 B11-E Live Multi-Worker, Performance & Memory Qualification (Direct Execution)

Module: Talent & Potential M10 Organization Analytics, PostgreSQL production qualification (ADR 0028, ADR 0029, ADR 0030, B11-E)

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-09-09 - M10 B11-E Live Multi-Worker Qualification

Related ADRs: `docs/adr/0028-b11-production-qualification-policy.md` (governing B11 qualification policy, completed by this task); `docs/adr/0029-m10-organization-analytics-repeatable-read-boundary.md` (REPEATABLE READ boundary this task qualified under real multi-process load); `docs/adr/0030-m10-organization-analytics-multi-worker-qualification-minimum.md` (new, created from this task's own evidence)

Related prior evidence: `docs/history/engineering-handbook/2026-09-08-b11e-integrated-production-qualification.md` (original B11-E implementation/REPEATABLE READ qualification, single-process); `docs/history/engineering-handbook/2026-09-09-b11e-f2-postgresql-qualification.md` (bounded independent re-verification of the REPEATABLE READ suite and a single-process latency sanity check; that entry explicitly declined to adopt multi-worker/memory figures for lack of independent evidence - this entry supplies that evidence directly)

Provenance

Every figure and finding in this document was produced by direct execution within this same task - live PostgreSQL connections, live subprocess workers, live HTTP requests, all personally observed via this task's own tool calls. No number in this document is copied from an earlier session's report. Where this document's own results differ from earlier, unreproduced figures mentioned in other sessions, this document's directly-observed results are authoritative for what they cover, and no attempt is made to reconcile them with unverifiable prior figures.

Environment

PostgreSQL 16.15, the same dedicated non-production qualification database used throughout B11-C/D/E, reached only via `TIS_TEST_POSTGRESQL_URL` (bridged from Windows User environment scope into a Bash session; 121 characters, `postgresql+`-prefixed; never printed, logged, or persisted anywhere in this repository or in any tool output). All work used ephemeral, uniquely-named schemas (`tis_b11_mw_<hex>`), created and dropped by this task's own throwaway scratchpad scripts (not committed to the repository). `tis.db` was never opened; its SHA-256 was confirmed unchanged before and after this task (`01e1a3065d92280ee9228db921f67545c8b837d4ecd787a5aeeddc6d128fc136`).

Real Production Providers Were Engaged, Not Test Doubles

Each worker process was started with `DATABASE_URL` pointed at the live PostgreSQL qualification schema (not SQLite, so `is_sanctioned_local_analytics_environment()` correctly returns `False`) and the governed Release 1 configuration environment variables (`TIS_ORGANIZATION_ANALYTICS_MINIMUM_COHORT=5`,

TIS_ORGANIZATION_ANALYTICS_MAX_MATRIX_CELLS=1000, TIS_ORGANIZATION_ANALYTICS_MAX_RELATIONSHIP_RESULTS=1000, TIS_ORGANIZATION_ANALYTICS_MAX_PROGRAM_PAIR_RESULTS=1000), exercising ConfiguredRelease1PrivacyPolicy and ConfiguredRelease1BreadthPolicy directly - not AllowAllTestPolicy/AllowAvailability/AllowBreadth test doubles. This was confirmed live: every response's privacy_policy_version field read "release1-2026-09". Availability was resolved via the real EntitlementOrganizationAnalyticsAvailabilityProvider against real EntitlementDefinition (feature.organization_intelligence, active) and WorkspaceEntitlement (internal_sandbox, active) rows seeded into the same schema, and a SchoolGroup row with workspace_classification=internal_sandbox / workspace_lifecycle_status=active - the real, unmocked commercial-state resolution path (saas.entitlement_service.organization_feature_available -> saas.commercial_state_service.resolve_commercial_state), not a monkeypatch.

Multi-Worker Qualification

Method: two genuinely independent uvicorn main:app OS subprocesses on two different free ports, both pointed at the identical PostgreSQL schema via DATABASE_URL, launched via a small throwaway launcher script (not committed) that also self-reports each worker's own RSS every 0.5s via psutil.Process() called with no PID argument, from inside that same process, to a local log file. A single real, DB-persisted models.User row and a session cookie minted via auth.create_session_token (HMAC-signed against a shared TIS_SESSION_SECRET, no server-side session store) were used to authenticate real HTTP requests against both workers.

Five of the seven M10 Organization Intelligence routes were exercised directly by this harness: overview, talent-map, program-portfolio, branches/{id}, students. (participation-overlap and programs/{id}/longitudinal were not included in this specific harness run; their REPEATABLE READ correctness is already covered by the full seven-route, 15/15-passing suite in tests/test_talent_organization_repeatable_read.py, re-run and confirmed passing earlier this same day per the 2026-09-09 F2 handbook entry above - this task did not re-run that suite again, since it was already current.)

Results, all directly observed:

1 **Provider/response consistency across workers**: identical requests to

worker A and worker B produced byte-for-byte identical response bodies on all 5 routes tested (overview 537 bytes, talent_map 735 bytes, program_portfolio 1206 bytes, branch_intelligence 1139 bytes, students 4861 bytes - all bodies_equal: true). **PASS**.

1 **No worker-local correctness dependency**: confirmed by direct source

reading of talent_organization_analytics_providers.py, talent_org_intelligence_service.py, and routers/talent_organization_analytics.py - every privacy/availability/ breadth provider and every database session is constructed fresh per request via FastAPI Depends(), with no module-level cache, singleton, or process-global mutable state anywhere in the M10 provider/route/service code path. **PASS** (by code inspection; corroborated behaviorally by the worker-kill test below).

1 **Concurrent writer safety / fresh-transaction-per-request semantics

across workers**: a direct database write (a new frozen population member, bringing the eligible count from 20 to 21) was committed via a separate process between two HTTP requests to worker B. The request issued before the write returned frozen_eligible_memberships.value = 20; the request issued after the write, on the same worker, returned 21.** PASS** - a fresh request always observes the current committed state; no request observed a stale cached count.

1 **Worker-kill correctness**: worker A was terminated (SIGTERM, graceful).

Worker B, entirely independently, continued serving correct `200` responses with no involvement from worker A. **PASS**.

1 **Stale-write/optimistic-concurrency (`409`) behavior**: not applicable -

all seven M10 Organization Analytics routes are read-only `GET` with no write/revision contract (confirmed by source inspection of `routers/talent_organization_analytics.py`). Reported as vacuously satisfied, not silently skipped.

1 **Tenant isolation within this specific multi-worker run**: NOT

independently re-tested here (this harness's fixture used a single tenant for simplicity, to keep the run scoped and reviewable). Tenant isolation under the identical M10 REPEATABLE READ session type, including under concurrent foreign-tenant writes, is already proven live and passing in `tests/test_talent_organization_repeatable_read.py` (`test_tenant_isolation_holds_under_the_new_session_with_a_foreign_tenant_write`, part of the 15/15 result re-confirmed earlier the same day). This is disclosed as a real, deliberate scope boundary of this specific harness, not a gap in overall B11-E tenant-isolation coverage.

Performance Qualification

Same live PostgreSQL schema, worker B only (worker A had already been terminated for the kill test above by this point in the run), 20 sequential samples per route, server-side round-trip HTTP timing (client-measured, including full HTTP/response-serialization overhead, not just SQL time):

| route | samples | p95 (ms) | p99 (ms) | min (ms) | max (ms) | all 200 | |---|---|---|---|---|---|---|---| | overview | 20 | 52.05 | 52.05 | 32.87 | 52.05 | yes | | talent_map | 20 | 48.19 | 48.19 | 28.14 | 48.19 | yes | | program_portfolio | 20 | 58.76 | 58.76 | 33.53 | 58.76 | yes | | branch_intelligence | 20 | 47.23 | 47.23 | 31.53 | 47.23 | yes | | students | 20 | 48.94 | 48.94 | 30.58 | 48.94 | yes |

PASS against p95 <= 2.0s / p99 <= 4.0s on every route tested, by roughly a 28-80x margin. A second, independently-built and independently-run pass of the same harness against a fresh schema (kept for transparency rather than discarded) produced overview 55.13-59.91ms p95 and program_portfolio's highest observed p99 of 71.03ms across the two runs - consistent, same-order-of-magnitude results confirming this is not a one-off measurement. This document does not claim these exact millisecond values are the sole canonical figure for all future qualification; both this run and the earlier-cited 2026-09-09 bounded sanity check (19-33ms range, a smaller 14-row single-process fixture) independently confirm the same conclusion - comfortable, wide-margin PASS against the approved target - without either run's specific numbers being treated as the one true benchmark.

Memory Qualification

Self-reported RSS (via `psutil.Process()` with no PID argument, executed from a background thread inside the worker process itself, sampled every 0.5s to a local log file this task read back afterward).

Important methodology note: this task first attempted cross-process memory measurement (`psutil.Process(pid).memory_info().rss` from the orchestrating process, reading a spawned child's memory) and found it returned implausible near-zero values (~4.5MB RSS, ~0.9MB VMS) for a live Python process with `fastapi`/`sqlalchemy` imported - confirmed as a measurement artifact, not a real result, via a control test against a bare `python -c "import fastapi, sqlalchemy; time.sleep(30)"` subprocess, which also read ~4.5MB cross-process but a real ~67MB when the same process measured itself. This task traced this to a limitation of cross-process memory introspection in this specific sandboxed execution environment (confirmed present even with the sandbox override flag set) and switched entirely to self-measurement (each worker reporting its own RSS from inside itself) for every figure below, rather than report a known-bad number.

Results (worker B, second qualification run):

1 Baseline RSS (first self-measurement after process start, before serving

any request): **21.07 MB**

1 RSS immediately before the repeated-request memory sequence (after the

provider-consistency, writer-safety, and 100 performance-sampling requests already served): **163.59 MB**

1 RSS after 5 additional full passes over all 5 routes (25 more requests):

163.87 MB

1 Peak observed RSS across the entire run: **163.87 MB**

1 Delta across the repeated-request sequence: **+0.28 MB** (a near-identical

first run measured **+0.17 MB** under the same sequence)

PASS against <= 128 MB incremental memory per analytics request - the observed per-request-sequence delta (0.17-0.28MB across 25-27 additional requests, i.e. a small fraction of a single MB per request) is far below the 128MB ceiling, and shows no monotonic growth trend across repeated calls (the value plateaus rather than climbing), consistent with the "no correctness/behavior dependency on worker-local mutable state" finding above - there is no unbounded per-request accumulation.

`<= 70% of allocated worker/container memory`: **NOT EVALUATED, environment- specific.** No authoritative production Render Web Service worker/container memory allocation value exists anywhere in this repository for either this task or any prior task to compare against. This is a real production- deployment-environment fact this repository cannot supply, not a code or qualification defect. The absolute RSS figures above (21MB baseline, ~164MB loaded-and-serving) are recorded so that whoever later learns the actual allocated container memory can compute this ratio directly without needing to re-run this qualification.

Cleanup

All three ephemeral schemas created during this task's iterative development (`tis_b11_mw_489dc0904ba0`, `tis_b11_mw_bcde79931402`, `tis_b11_mw_781b6611b4a8`) were dropped (`DROP SCHEMA ... CASCADE`) at the end of the task. A post-run `SELECT nspname FROM pg_namespace WHERE nspname LIKE 'tis_b11%'` confirmed zero leftover schemas. `tis.db`'s SHA-256 was re-confirmed unchanged after this task completed.

What This Document Does Not Claim

This document does not claim production-scale validation (the dataset used - 20 Students, 1 Program, 1 Cycle - is small; it is sufficient to exercise every route's code path and privacy/breadth/availability provider logic correctly, but is materially smaller than B11-C's own MEDIUM/LARGE PostgreSQL profiling scales). It does not claim knowledge of real production worker count, host memory, or container allocation. It does not claim this constitutes a full B11-C-style profiling re-run at scale. It qualifies exactly what is described above: real multi-process correctness, a comfortable-margin performance/memory result on a small-but-real dataset under genuine multi-worker PostgreSQL load, and nothing beyond that.

Engineering Handbook History

docs/history/engineering-handbook/README.md

This folder tracks meaningful changes to the TIS Engineering Handbook, including module maps, repository architecture, user/system flows, and onboarding guidance for humans and AI coding conversations.

Cline integration:

1 [2026-09-05 Cline KMS configuration](2026-09-05-cline-kms-configuration.md):

project rules, reusable skill, and flexible three-agent orchestration.

Latest change:

1 `2026-09-09-b11e-live-multiworker-qualification.md`: direct live execution

of the remaining B11-E release-gate evidence - two genuinely independent `uvicorn` worker processes against the same live PostgreSQL qualification database, real production-configured F1 providers engaged (not test doubles), performance sampling (5 of 7 routes, 20 samples each), and a self-measured memory profile. On this directly-observed evidence, formally adopted the p95<=2.0s/p99<=4.0s performance target and the <=128MB incremental-memory target into ADR 0028 (both PASS with substantial margin), created ADR 0030 (the >=2-worker qualification minimum and no-worker-local-state correctness requirement), and added a B12 (M10 closeout only) definition to ADR 0028. Classified B11-E as CLOSED WITH ONE ENVIRONMENT-SPECIFIC DEPLOYMENT VERIFICATION ITEM REMAINING and B12 as DEFINED, NOT YET CLOSED.

1 `2026-09-09-b11e-f2-postgresql-qualification.md`: bounded, independent

re-verification performed earlier the same day - re-ran the REPEATABLE READ suite live (15/15 passing) and a bounded single-process latency sanity check, and fixed a same-document self-consistency defect in the 2026-09-08 entry below. It explicitly declined to formally adopt memory-ceiling or multi-worker qualification figures into ADR 0028/ADR 0030 for lack of independently reproduced evidence - that evidence was supplied later the same day by the live-multiworker entry above.

1 `2026-09-08-b11e-integrated-production-qualification.md`: governed

(ADR 0029) and implemented the M10-only REPEATABLE READ transaction boundary; live-re-tested all seven Organization Intelligence routes; tested suppression/reconstruction concurrency with a real non-production deterministic suppressing policy; finalized Index Candidate B (NO CHANGE) and statistics-freshness disposition. B11-E independent review returned PASS WITH NON-BLOCKING OBSERVATIONS.

1 `2026-09-07-b11d-postgresql-concurrency-consistency-qualification.md`:

durable PostgreSQL 16.15 READ COMMITTED consistency evidence; independent review passed with non-blocking observations and B11-D is CLOSED.

1 `2026-09-07-b11c-postgresql-profiling-evidence-remediation.md`: durable,

sanitized PostgreSQL 16.15 evidence for all seven M10 routes; independent re-review passed with non-blocking observations and B11-C is CLOSED.

1 `2026-09-03-claude-code-kms-configuration.md`: repository-native Claude Code

entry point, reusable TIS KMS skill, and bounded specialized subagent.

1 `2026-07-22-kms-phase-7d-navigation-catalog-enforcement.md`: strict source-title, catalog, navigation-link, source-inventory, and PDF page-bound validation.

Related files:

```
docs/engineering/README.md
docs/engineering/TIS_MODULE_MAP.md
docs/engineering/REPOSITORY_ARCHITECTURE.md
docs/engineering/USER_AND_SYSTEM_FLOWS.md
```

Landing CTA Consolidation And Demo Domain Policy

docs/history/landing-page/2026-07-25-landing-cta-and-demo-domain-policy.md

The public landing website now uses two clear customer conversion paths:

1 Request a Demo: configured SaaS signup URL with `intent=demo`.

1 Subscribe Now: configured SaaS signup URL with `intent=subscribe`.

Both URLs are built from `NEXT_PUBLIC_TIS_APP_BASE_URL`; the public site continues to deploy independently from the FastAPI application. The landing page no longer exposes overlapping Open Account, Book a Demo, Request Early Access, or Request Pricing conversion labels.

The selected intent is persisted by the SaaS application through account creation, email verification, School Workspace Setup, and review. At commercial choice, the selected path receives visual emphasis but the customer can choose either available path.

Customer Demo eligibility is enforced by the SaaS application, not by the landing site. It uses a normalized organization domain and a database unique reservation so a second customer demo cannot be created for the same organization after any prior demo request or lifecycle outcome. Internal Sandbox history is excluded. Historical duplicate domains are retained for Platform Owner manual review without deleting, merging, or recreating customer workspaces.

M8 Landing Integration Open Account

docs/history/landing-page/2026-07-25-m8-landing-integration-open-account.md

2026-07-25 - M8 Landing Integration Open Account

Module: Landing Page

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-07-25 - M8 Landing Integration Open Account Entry Points

Related ADRs:

- `docs/adr/0001-separate-nextjs-landing-website.md`
- `docs/adr/0007-landing-page-visual-system-strategy.md`

Reviewer/approval notes: M8 final landing integration only. No FastAPI SaaS application, authentication, onboarding backend, payment, provisioning, database, operational module, commit, push, or M9 work.

Previous Documented State

The public Next.js landing website explained TIS, supported demo and early-access interest, and remained separate from the FastAPI application. It did not yet expose the completed customer account setup flow from the public marketing surface.

New Documented State

The public landing website now includes Open Account entry points in:

- the navigation,
- the hero CTA group,
- the final CTA area.

All Open Account links share one destination derived from `NEXT_PUBLIC_TIS_APP_BASE_URL` and the existing `/saas/signup` path. The landing project remains a marketing website and redirects visitors to the separately deployed TIS Account signup flow.

Reason For Change

M8A and M8B-1 through M8B-6 completed the SaaS-side customer account, workspace, demo, subscription, and conversion readiness. M8 final integration connects the public website to that existing deployed signup journey without changing the SaaS application.

User / Business Impact

Visitors can move directly from `tisplatform.com` to account setup. Schools still retain demo and early-access paths, while Open Account becomes the direct signup entry point for M8 production validation.

Technical Impact

The change is limited to `tis-landing-website/` plus KMS documentation. The FastAPI app, SaaS routes, authentication, onboarding backend, Paddle, database, APIs, demo workflow, provisioning, and commercial state remain unchanged.

Follow-Up Needed

Set `NEXT_PUBLIC_TIS_APP_BASE_URL` in the landing website hosting environment, deploy the landing project separately from the FastAPI app, and validate that Open Account navigates to the deployed TIS Account signup flow on desktop, tablet, and mobile.

Landing Page History

docs/history/landing-page/README.md

2026-08-19 - Capacity-Based Feature Copy

The official Next.js pricing cards remain capacity-only. The AI disclaimer no longer implies subscription-tier availability and instead identifies permissions, workspace scope, configured usage allowances, and release stage. Plan names, prices, capacities, layout, and actions are unchanged.

2026-07-29 - Safe Pricing Subscription Entry

Hero Subscribe Now now scrolls to `#pricing`. The three self-service plans use the same CTA treatment and link to public TIS Account registration with their own allowlisted preferred-plan code. Plan descriptions were removed to keep the cards focused on price, capacity, and action. Custom remains email-only.

2026-07-28 - Returning-Customer Navigation

The Next.js landing now exposes Sign In and Open TIS App beside Request a Demo and Subscribe. Every app link derives from `NEXT_PUBLIC_TIS_APP_BASE_URL`; authentication and state routing remain in FastAPI.

This folder tracks meaningful changes to the public landing page, marketing positioning, visual system strategy, and source-of-truth boundaries.

Related docs:

- `docs/adr/0001-separate-nextjs-landing-website.md`
- `docs/adr/0007-landing-page-visual-system-strategy.md`
- `docs/marketing/landing_page_source_of_truth.md`
- `docs/marketing/tis_landing_page_master_content.md`

History entries:

- `2026-07-25-landing-cta-and-demo-domain-policy.md`: replaces overlapping public conversion CTAs with Request a Demo and Subscribe Now, both using configured deployed SaaS signup URLs and explicit onboarding intent.
- `2026-07-25-m8-landing-integration-open-account.md`: final M8 Open Account entry points from the public landing website to the deployed TIS Account signup flow.

Platform Owner Knowledge Center

docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md

2026-06-26 - Platform Owner Knowledge Center

Module: Platform Knowledge Center

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added Platform Owner Knowledge Center

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for Phase 2C. Regenerate button, public access, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

Previous Documented State

TIS had KMS Markdown source files, ADRs, module history, a generated PDF booklet, and a manifest, but there was no protected in-app owner utility for viewing KMS status or accessing the booklet.

New Documented State

TIS has a read-only Platform Owner Knowledge Center that shows KMS health, manifest metadata, PDF freshness, source document status, recent change-history entries, ADRs, module history areas, and the Knowledge Impact Assessment checklist.

Reason For Change

Platform owners need a protected internal view of KMS status and a safe way to view/download the generated PDF without linking directly to static public paths.

User / Business Impact

Platform owners can verify whether the KMS snapshot is current and access the reference booklet from inside the app.

Technical Impact

Adds read-only KMS service helpers, owner-protected FastAPI routes, one app-shell template, and an owner-only Platform Console card. No SaaS, database, migration, billing, provisioning, or landing page behavior changes.

Documentation Updated

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/PROJECT_STATE.md`

1 `docs/CHANGE_HISTORY.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/history/platform-knowledge/README.md`

1 `docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md`

PDF Regenerated

Yes

Follow-Up Needed

Consider a future explicit owner-only regenerate action after review. The app must not silently rewrite Markdown source docs.

KMS Phase 7C Knowledge Center Navigation

docs/history/platform-knowledge/2026-07-22-kms-phase-7c-knowledge-center-navigation.md

2026-07-22 - KMS Phase 7C Knowledge Center Navigation

Module: Platform Knowledge Center

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-07-22 - Added Phase 7C Knowledge Center Navigation

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for Phase 7C only. Existing owner permissions and protected routes remain unchanged. No database, route, dependency, search service, regenerate action, application-data change, commit, or push is included.

Previous Documented State

The owner-only Knowledge Center displayed manifest metadata, freshness, source paths, ADRs, module-history areas, change history, and protected booklet actions. Its source inventory was one path-focused table without search, filters, logical sections, descriptive summaries, or page-specific booklet links.

New Documented State

The manifest remains the sole source inventory. The Knowledge Center derives display titles, summaries, categories, and modules from safe manifest-listed Markdown files; groups sources into Core, Engineering, Decisions, History, Marketing, and Supporting sections; and filters the rendered rows in the browser by search text, category, module, and freshness. Each source can open the existing protected booklet route at its manifest `pdf_page`. ADRs and module-history areas now surface newest activity first.

Reason For Change

Platform owners need a faster path from KMS status to the exact knowledge they are reviewing while the system remains read-only, dependency-light, and owner protected.

User / Business Impact

Platform owners can find a document by title, summary, module, path, category, or freshness and move directly to its handbook page without exposing the static PDF URL.

Technical Impact

`knowledge_service.py` adds presentation metadata and deterministic activity ordering for existing manifest sources. `templates/platform_knowledge_center.html` adds browser-only filtering and protected page-fragment links. No source Markdown is rewritten by the app, and no application state or access boundary changes.

Documentation Updated

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/PROJECT_STATE.md`

1 `docs/README.md`

```
1     docs/AI_PROJECT_CONTEXT.md
1     docs/CHANGE_HISTORY.md
1     docs/engineering/TIS_MODULE_MAP.md
1     docs/history/platform-knowledge/README.md
1     docs/history/platform-knowledge/2026-07-22-kms-phase-7c-knowledge-center-navigation.md
```

PDF Regenerated

Yes

Follow-Up Needed

Phase 7D and any Knowledge Center regenerate, server-side search, or additional route work require separate approval.

Platform Knowledge Center History

docs/history/platform-knowledge/README.md

This folder tracks meaningful changes to the owner-only TIS Knowledge Center, protected documentation PDF access, KMS metadata display, freshness detection, knowledge navigation, and knowledge health reporting.

Latest change:

- `2026-07-22-kms-phase-7c-knowledge-center-navigation.md`: searchable, filtered, grouped manifest library with protected booklet page links and improved activity ordering.

Related files:

- `knowledge_service.py`
- `templates/platform_knowledge_center.html`
- `templates/platform_console.html`
- `main.py`

KMS Foundation

docs/history/provisioning/2026-06-26-kms-foundation.md

2026-06-26 - KMS Foundation

Module: Documentation and knowledge management

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Established Knowledge Management System Foundation

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only.

Previous Documented State

TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal KMS policy, ADR structure, module history foundation, or AI onboarding context.

New Documented State

TIS now has a KMS foundation with policy, change history, ADRs, module history folders, and AI project context. The PDF generator includes these source docs and produces manifest metadata.

Reason For Change

Future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and reviewers need durable project context and history.

User / Business Impact

Improves continuity, reduces repeated rediscovery, and makes project decisions easier to review.

Technical Impact

No runtime app behavior changed. This is documentation and PDF generation foundation only.

Documentation Updated

1 `docs/CHANGE_HISTORY.md`

1 `docs/DOCUMENTATION_UPDATE_POLICY.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/adr/`

1 `docs/history/`

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/PROJECT_STATE.md`

1 `docs/README.md`

PDF Regenerated

Yes

Follow-Up Needed

Phase 2C can later add a protected Platform Owner Knowledge Center after review.

Provisioning History

docs/history/provisioning/README.md

This folder tracks meaningful changes to pending organizations, provisioning jobs, verified-payment readiness, retry behavior, and platform owner provisioning oversight.

Related docs:

```
docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
docs/TIS_MASTER_CONTEXT.md
```

Related files:

```
saas/provisioning_service.py
saas/service.py
saas/router.py
templates/saas/admin_provisioning.html
```

2026-08-18 - SchoolGroup Operational Creation Boundary

Direct operational SchoolGroup creation and deletion now require Platform identity and explicit global SchoolGroup-management capability. Tenant role defaults no longer include these actions, stale permissions cannot cross the handler boundary, and tenant updates accept only the linked SchoolGroup. Approved paid, demo, promo, and conversion provisioning paths are unchanged. A separate PostgreSQL-only read-only provenance audit reports active internal sandboxes missing tenant-link and explicit entitlement evidence for later Platform Owner review; it performs no remediation.

2026-07-22 - Lifecycle-Aware Platform Owner Queue

Platform Owner pending counts and lists now share one query boundary. A record remains pending only while its onboarding status is draft, in progress, changes requested, under review, or ready for checkout and no tenant link, completed provisioning job, or final tenant billing state exists. Active tenants are resolved from coherent payment, active subscription, contract, tenant-link, and active SchoolGroup evidence. Completed provisioning with incomplete or conflicting commercial evidence is retained as Lifecycle Review Required rather than shown as ordinary pending work. No lifecycle rows are rewritten by this owner-facing projection.

TIS Module History

docs/history/README.md

Module history stores deeper area-specific change records. It complements `docs/CHANGE_HISTORY.md`, which remains the chronological summary.

Use module history when a meaningful area changes and the previous documented state should be preserved.

Module Areas

```
ai-entitlements/
subscriptions/
landing-page/
academic-calendar/
workforce-planning/
saas-onboarding/
provisioning/
```

Entry Template

```
# YYYY-MM-DD - Change Title

Module:
Related change-history entry:
Related ADRs:
Reviewer/approval notes:

## Previous Documented State

## New Documented State

## Reason For Change

## User / Business Impact

## Technical Impact

## Documentation Updated

## PDF Regenerated

## Follow-Up Needed
```

SaaS Onboarding History

docs/history/saas-onboarding/README.md

2026-08-19 - Common Customer Feature Baseline

The July 16 feature matrix below remains historical context and is superseded by ADR 0025. Normal customer PlanEntitlement values are now common across Starter, Professional, and Enterprise AI; plan selection remains capacity- and billing-oriented. Existing paid workspace entitlements resolve the new matrix dynamically and require no per-workspace feature backfill.

2026-07-30 - Secure Payment Retry And Checkout Lineage

The observed failure was entirely local and occurred before any Paddle API request: `_ensure_checkout_launchable()` rejected the legacy `ready_for_checkout` pre-checkout billing state. That state is now prepared safely. Professional Annual remains mapped to USD 790 per active branch; the verified two-branch quote uses quantity 2 and totals USD 1,580 annually.

Secure Payment retry now rebuilds an eligible unpaid checkout when legacy billing state or an obsolete session prevents launch. Plan, interval, quantity, selection, and fingerprint changes supersede both the checkout session and its unfinished payment attempt. The current contract and organization no longer point at that obsolete authority, and its later webhook is blocked before subscription or workspace activation.

Existing started transactions are reused only after remote billed and quote validation. Paddle customer address lookup reuses an active compatible country address, provider failures remain detailed in server logs, and the payment page shows one customer-safe retry alert. Prices, capacity rules, provisioning, and webhook confirmation authority remain unchanged.

2026-07-29 - Organization Profile Save And Compact Account Setup

Organization Profile now accepts the three approved program labels through a required selector and returns customer validation errors inline. Pending logo uploads reuse actual-image validation, the 4 MB limit, minimum dimensions, opaque UUID filenames, and atomic writes. Newly written files are removed if a later save fails; empty upload retains the prior logo and successful replacement safely retires it. Storage and unexpected failures retain transaction rollback and produce server traceback logs without exposing raw errors.

The initial Account Setup state now contains one setup title, one explicit POST start action, and the existing eight-step journey under a smaller official logo. Repeated status, action, context, and help panels are omitted only before the pending organization exists.

Pending logos still use application-local `static/uploads/saas/pending_logos`; durable Render storage remains a separate persistent-disk or object-storage decision.

The saved logo is now visible in Organization Profile and the shared School Workspace setup identity. Existing paid and demo provisioning already copied the pending image into `SchoolGroupLogo`; that path is now explicitly directory-confined, missing referenced files fail activation instead of being silently skipped, and the final logo label uses the organization name for accessible operational-shell rendering. TIS platform branding remains separate.

Branch Setup no longer performs nullable estimate arithmetic in Jinja. Python-normalized totals make empty, legacy-null, and mixed rows safe to render. Customer saves retain required non-negative estimate validation, and new branch records receive explicit zero defaults.

2026-07-29 - Post-Verification Login Method Safety

Fresh verified accounts no longer redirect successful login to the POST-only onboarding creation endpoint. They open the GET Account Setup dashboard, where the established explicit POST action starts School Workspace Setup. Sign-in continuations now accept only known customer GET destinations, and accidental GET navigation to the credential handler redirects to the normal sign-in page instead of returning raw HTTP 405 JSON. Same-email re-registration, subscription intent, and non-authoritative preferred-plan state remain compatible.

2026-07-29 - Public Preferred-Plan Continuity

Public pricing may carry an allowlisted Starter, Professional, or Enterprise AI preference into TIS Account registration. The preference is stored separately from commercial records and does not select a plan or create checkout. Once School Workspace Setup is complete, the existing branch, system-user, and teacher capacity decision either preselects the eligible preference or clears it with customer-safe guidance. Missing and malformed values are safe.

2026-07-28 - Returning Customer State Routing

Successful SaaS login now derives its destination from onboarding, pending demo, durable account-to-workspace, lifecycle, classification, and subscription evidence. Active customers enter the app, incomplete customers resume setup, unpaid customers reach plans, and expired customers receive branded guidance. Customer language uses "TIS team," while internal Platform Owner identifiers and audits remain intact.

2026-07-27 - M8B7 Demo Customer Journey

Normal approval now invokes the existing independently retryable provisioning service. Six branded lifecycle email types use a durable outbox, and Platform Owner events reuse the existing Notification Center. The shared tenant shell displays active Customer Demo status. Day 6 and expiry create communication intents atomically, and coherent expired demos may convert after authoritative confirmed payment by reactivating the same workspace. M8B8 and M8B9 are not included.

This folder tracks meaningful changes to signup, login, account, organization onboarding, contacts, branches, academic setup, review, and account self-service.

2026-07-26 - Platform Owner Historical Demo Eligibility Maintenance

Historical detached `SaaSDemoDomainEligibility` rows can outlive the organization/account records that once made them reachable through clean-room reset. Platform Owners now have a separate `/saas-admin/demo-eligibility-maintenance` page that scans the eligibility ledger and explains whether each reservation is safe or protected.

The maintenance analyzer uses authoritative domain normalization and checks for matching pending organizations, SaaS accounts, demo requests, tenant-profile workspaces, paid or demo provisioning, subscription contracts/subscriptions, Demo-to-Paid conversions, and manual-review evidence. Only an exact eligibility ID with no blockers may be deleted. The POST action locks and re-analyzes the row, requires typed-ID and checkbox confirmation, bulk-deletes by primary key only, flushes, verifies absence, and commits or rolls back as one transaction. Successful cleanup is owner-audited.

This administrative repair capability does not modify the customer demo request workflow, the durable one-demo-per-domain rule, the existing test workspace/account reset, schema, foreign keys, or customer-facing behavior.

2026-07-26 - Internal Test Workspace Commercial Clean-Room Reset

The guarded Platform Owner test workspace/account reset now clears only the selected organization's linked demo request, domain-eligibility reservation, review and event history, demo provisioning and lifecycle records, and demo-to-paid conversion history before deleting parent commercial and workspace records. The same email and

organization domain can then complete a new internal M8 journey.

Detached same-domain reservations left by the `ON DELETE SET NULL` relationship are also eligible for removal, but only after the same domain resolver used by Customer Demo submission confirms there is no other organization, request, workspace, or customer account using that domain and the reservation has no historical manual-review evidence. Any conflicting or ambiguous evidence blocks the reset for manual review. This narrow exception does not alter the production Customer Demo policy: normal customer domain reservations remain durable after pending, approved, active, expired, rejected, cancelled, or converted history. Global plans, prices, Paddle records, platform configuration, and other organizations' records remain outside the reset scope. Customer-facing demo wording now identifies the TIS team rather than an internal role.

Reset analysis now publishes the exact detached eligibility IDs that passed those safety checks. The deletion service consumes only that safe list, deletes it before demo-request and parent records, flushes immediately, and verifies that no selected ID remains. Any handoff mismatch or failed verification stops the transaction so the owner route can roll back all changes.

2026-07-25 - Landing Intent Continuity And One Customer Demo Per Organization Domain

The public landing conversion paths now enter TIS Account signup with a validated `demo` or `subscribe` intent. The intent is persisted on the SaaS account and copied to School Workspace Setup so the final commercial-choice page can emphasize the customer's original path while still allowing either Request Demo or Subscribe Now.

Customer Demo requests now resolve a normalized organization domain, preferring the organization identity and using a work email only when necessary. Public email providers require an official organization website or domain. A transactionally unique domain-eligibility reservation prevents a second customer demo across pending, approved, active, expired, rejected, cancelled, or converted history. The reservation never causes tenant reprovisioning or Demo-to-Paid workspace replacement, and Internal Sandbox history is excluded.

Migration `20260725_001_demo_domain_eligibility_policy` adds safe historical normalization. It reserves ambiguous duplicate domains for manual review rather than merging, deleting, or guessing across existing customer records. The companion diagnostic command defaults to dry-run and produces the duplicate-domain review list.

2026-07-23 - M8B-6 Demo-To-Paid Workspace Conversion

Eligible active Customer Demo workspaces can now enter the unchanged M7 subscription checkout. Provider-confirmed payment triggers a dedicated, idempotent conversion service rather than paid tenant provisioning.

The conversion preserves the SchoolGroup, workspace UUID, tenant link row, pending organization, branches, users, permissions, academic records, and all request/demo/audit history. One atomic transaction ends the demo entitlement, creates the confirmed subscription-backed paid entitlement, relinks branch entitlement rows, changes the workspace classification to Customer Paid, and moves the existing tenant link from demo-request evidence to the confirmed subscription contract.

Conversion Requested, Processing, Completed, and Failed states plus audit/internal-notification events are durable. Failure rolls back workspace mutations while preserving confirmed provider payment records for retry. Completed workspaces no longer enter demo reminder or expiration processing. Expired, ambiguous, cross-tenant, internal-sandbox, already-paid, or incoherent workspaces fail closed.

2026-07-23 - M8B-5 Standard Customer Demo Lifecycle

Customer demos now run for exactly seven days from successful workspace activation. The centralized resolver derives Day 6 and Day 7 boundaries in UTC, presents them in the organization timezone, and fails closed on inconsistent lifecycle evidence.

The independently callable processor is dry-run by default. Apply mode creates idempotent internal reminder notifications and atomically ends the demo entitlement and suspends the workspace at expiration. Operational middleware rechecks customer-demo access on every protected request, including existing sessions; web users are routed to subscription guidance and APIs return a safe 403. Workspace users, branches, and all tenant data remain preserved, and Platform Owner inspection remains available.

2026-07-23 - M8B-4 Demo Workspace Provisioning And Activation

Platform Owners can now provision an Approved customer-demo request through a separate, fail-closed action. The demo service revalidates the approval, organization, customer-demo intent, commercial snapshot, entitlement snapshot, and duplicate absence before reusing the shared operational workspace builder.

The transaction creates the customer-demo SchoolGroup, branches, academic year, owner user, permissions, account link, explicit demo entitlement, and demo-sourced tenant link, then activates the workspace and entitlement. No Paddle, payment, paid subscription, subscription contract, checkout, or email record is created. Failures roll back all workspace changes, preserve the Approved request, and retain a safe retryable outcome plus audit event.

2026-07-22 - M8B-3 Demo Request Workflow

Completed onboarding now presents Request Demo and Subscribe Now. Subscribe Now preserves the existing plan-selection and Paddle lifecycle. Request Demo validates the verified customer and complete onboarding record, stores a review-only commercial snapshot, and prevents duplicate pending requests.

Customers can view request status and withdraw a pending request. Platform Owners have a searchable/filterable review queue and may approve, reject with a mandatory reason, or cancel. Approval creates review evidence only; no SchoolGroup, entitlement, checkout, payment, provisioning, activation, or email is created. Durable audit and internal-notification events record every transition.

2026-07-16 - M7 Phase 2 Read-Only Subscription Management Portal

Verified SaaS customers now have a read-only Subscription Management page at `/saas/subscription`. The page presents the confirmed plan, safe subscription health, billing interval, next billing date, paid and active branch quantities, remaining capacity, grouped feature entitlements, and a compact plan comparison. All commercial data is supplied by `saas.entitlement_service`; the portal does not query Paddle or independently calculate subscription state or branch capacity.

Upgrade, branch-addition, billing-history, invoice, pending-change, and subscription-management controls are visible only as disabled Coming Soon options. This phase adds no subscription mutations, Paddle calls, proration, refunds, cancellation, or plan and quantity editing.

2026-07-16 - M7 Phase 1 Subscription Entitlement Foundation

Commercial access now resolves through `saas.entitlement_service` from the provisioned SchoolGroup's tenant link, paid operational contract, and one confirmed active Paddle subscription. Onboarding selections, checkout quotes, pending payment attempts, and page values are not entitlement sources. Missing, mismatched, or ambiguous subscription relationships fail closed as `manual_review`.

The initial normalized matrix authoritatively maps only existing plan metadata: Enterprise AI receives `module.ai`; Professional and Enterprise AI receive `feature.advanced_reporting`; Starter does not receive either. Paid active-branch capacity is derived only from `PaymentSubscription.quantity`. Teacher management, branch management, observations, hiring, core reporting, general exports, audit logs, and cross-branch reporting remain `owner_approval_required` until commercial rules are approved.

Pilot enforcement is limited to allocation-plan PDF/XLSX exports. Both the existing `reports.export` user permission and `feature.advanced_reporting` subscription entitlement must succeed. Platform Owner and Developer

identities do not bypass subscription entitlements and must operate in a selected tenant scope.

Upgrades, downgrades, proration, refunds, Paddle subscription changes, branch-specific plans, and customer subscription-management UI remain later M7 work.

2026-07-26 - Test Workspace Reset Subscription-Change Dependency

The Platform Owner-only test workspace/account reset now deletes `subscription_change_requests` scoped by the selected operational `school_group_id` before deleting that workspace's users. It deletes selected entitlement values and branch-entitlement children before the selected workspace entitlement and final SchoolGroup. These narrow ordering rules prevent foreign-key deletion failures without changing lifecycle rules, customer behavior, or the preservation of records from other workspaces. The reset remains one transaction with the existing validation blocks, rollback behavior, pre-analysis counts, and structured diagnostics.

2026-07-14 - M6 Phase 3 Abandoned Draft Cleanup

Automatic cleanup applies only to unpaid, unprovisioned SaaS drafts. A draft becomes eligible after the globally configured inactivity period (30 days by default) and only after its final reminder was sent successfully for the current activity cycle. Any later meaningful activity restarts the lifecycle and prevents deletion.

Before deleting, the processor locks and rechecks each account and pending organization in its own transaction. Payment success or processing, subscription evidence, provisioning or tenant links, operational identities, protected accounts, shared ownership, and unresolved provider relationships block cleanup. Ambiguous records are retained for manual review. Successful payment evidence, Paddle webhooks and remote Paddle records, global plans/prices, reference data, and unrelated accounts or tenants are always preserved.

Run the bounded processor from the repository root:

```
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --dry-run --batch-size
100
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --batch-size 100
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --dry-run --account-
email draft@example.com
```

`--max-inactivity-days` is available only for local testing and is rejected in production-like environments. For Render, configure a daily Cron Job using the deployed service environment and `DATABASE_URL`; dry-run should be used before enabling the live command. Each account commits independently, failures roll back completely, concurrent workers skip locked rows, and durable external audit events record deleted, skipped, manual-review, recovery, and rolled-back outcomes.

Platform Owner lifecycle analytics remain future M6 scope.

2026-07-14 - M6 Phase 2 Draft Onboarding Reminder Engine

Draft retention remains inactivity-based. The reminder engine sends at most one first, second, and final reminder per activity cycle using the globally configured lifecycle thresholds (defaults: 24 hours, 7 days, and 25 days). The final reminder shows the deletion-eligibility date derived from the configured retention period (default: 30 days). Meaningful customer activity continues to flow through `draft_lifecycle_service.record_meaningful_activity(...)`, which starts a new reminder cycle.

Run the bounded processor from the repository root:

```
PYTHONPATH=. python scripts/process_draft_reminders.py --batch-size 100
PYTHONPATH=. python scripts/process_draft_reminders.py --dry-run
PYTHONPATH=. python scripts/process_draft_reminders.py --stage final
```

For Render, use a Cron Job with the service's `DATABASE_URL`, `RESEND_API_KEY`, `EMAIL_FROM`, `EMAIL_REPLY_TO`, and `TIS_PUBLIC_BASE_URL`. `TIS_SUPPORT_EMAIL` is optional and falls back to `EMAIL_REPLY_TO` in reminder content. An hourly schedule is recommended. PostgreSQL row locking prevents overlapping workers from sending the

same reminder stage.

Automatic draft deletion and Platform Owner lifecycle analytics are not enabled in Phase 2.

2026-06-27 - Subscription And Workspace Activation Guided Journey Phase 3C

Phase 3C subscription, payment, and activation page redesign is accepted.

What changed:

- Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages now use the Phase 3A shared shell and Phase 3B guided style.
- Each page now has one shared-shell primary CTA and keeps secondary actions visually secondary.
- Secure Payment pages now clearly explain that browser return from checkout does not itself confirm payment.
- Subscription and activation status pages now use concise customer-safe cards for payment, subscription, activation, and TIS Platform access state.
- Customer-facing pages now consistently explain that TIS Platform access becomes available after Workspace Activation.

Scope notes:

- Payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, stored statuses, database schema, migrations, operational modules, the Next.js landing website, OAuth behavior, internal `/saas` route names, and admin views were not changed.

Related files:

- `saas/router.py`
- `templates/saas/plan_selection.html`
- `templates/saas/checkout_summary.html`
- `templates/saas/checkout_return.html`
- `templates/saas/checkout_cancel.html`
- `templates/saas/account_billing.html`
- `templates/saas/billing_status.html`
- `tests/test_saas_phase1.py`

2026-06-27 - School Workspace Setup Guided Wizard Phase 3B

Phase 3B School Workspace Setup onboarding page redesign is accepted.

What changed:

- Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use a consistent guided wizard structure on top of the Phase 3A shared shell.
- Each onboarding page now has one shared-shell primary CTA and keeps Back/Save Draft actions visually secondary.
- Organization Profile groups identity, logo upload, program/location, and estimated scale fields.
- Branch Setup uses compact branch panels instead of heavy repeated blank blocks.
- Academic Setup and Primary Contact use focused single-step sections with concise guidance.

1 Review School Workspace Setup now presents a clearer ready-to-continue summary before Subscription Selection.

Scope notes:

1 Form actions, field names, routes, validation behavior, draft behavior, onboarding progression, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, OAuth behavior, internal `/saas` route names, and admin views were not changed.

1 Subscription/payment/status pages remain future Phase 3 work.

Related files:

1 `saas/router.py`

1 `templates/saas/base.html`

1 `templates/saas/onboarding_organization.html`

1 `templates/saas/onboarding_branches.html`

1 `templates/saas/onboarding_academic_setup.html`

1 `templates/saas/onboarding_contacts.html`

1 `templates/saas/onboarding_review.html`

1 `tests/test_saas_phase1.py`

2026-06-27 - TIS Account Guided Setup Framework Phase 3A

Phase 3A shared guided setup framework is accepted.

What changed:

1 The shared customer account shell now supports a guided setup console when setup context is provided.

1 The TIS Account page now uses an 8-step customer journey: TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform.

1 The account page now focuses on the current step, one primary next action, concise account/workspace context, and guidance that TIS Platform access becomes available after Workspace Activation.

1 The old customer account dashboard statistics and session detail blocks were removed from the account landing page.

1 Journey state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

Scope notes:

1 Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, internal `/saas` route names, admin views, and Google/Microsoft login were not changed.

1 This phase prepares the shared framework for later Phase 3 onboarding and payment page redesign work.

Related files:

1 `saas/router.py`

1 `saas/service.py`

1 `templates/saas/base.html`

- `templates/saas/account.html`
- `tests/test_saas_phase1.py`

2026-06-27 - TIS Account Customer-Facing Wording And Logo Cleanup

Phase 2 customer-facing wording cleanup is accepted.

What changed:

- Customer account and school workspace setup pages now use professional labels such as "TIS Account", "Account Dashboard", "School Workspace Setup", "Organization Profile", "Branch Setup", "Academic Setup", "Subscription Setup", "Secure Payment", and "Workspace Activation".
- Customer views use customer-safe display labels for internal onboarding, billing, payment, and activation statuses instead of exposing raw tenant/provisioning/checkout status values.
- Customer-facing billing and subscription views hide provider transaction/subscription IDs, attempt UUIDs, checkout session internals, plan IDs, and school group IDs.
- The shared customer account shell now includes the official full-color horizontal TIS logo image so inherited customer account/setup pages carry official branding.
- TIS Account transactional emails use an existing official dark-blue TIS wordmark asset.
- Activation email copy now uses "School Workspace", "Workspace Activation", and "TIS Account" language.

Scope notes:

- Internal `/saas` route/module/model names and stored statuses were not renamed.
- Payment, billing, provisioning behavior, database schema, migrations, operational modules, and the Next.js landing website were not changed.
- Google/Microsoft login remains future work and was not implemented.
- Phase 3 account setup UI redesign was not implemented.

Related files:

- `saas/router.py`
- `saas/service.py`
- `saas/provisioning_service.py`
- `email_templates.py`
- `templates/saas/`
- `tests/test_saas_phase1.py`
- `tests/test_saas_phase5.py`
- `tests/test_email_templates.py`

2026-06-27 - TIS Account Email Verification Recovery And Setup Gate

Phase 1 TIS Account email verification recovery is accepted.

What changed:

- Valid email verification links now mark the SaaS account email verified/active and redirect to TIS Account login with a professional success notice.

- Expired or invalid verification links now show a recovery page with a resend verification option.
- Resend verification supports unverified accounts, already verified accounts, and unknown-email cases with safe customer-facing messaging that does not reveal account existence.
- Password-based accounts that remain unverified are blocked from starting or continuing school workspace setup.
- New visible wording in this verification flow uses "TIS Account" and "school workspace setup".

Scope notes:

- Payment, billing, provisioning, database schema, migrations, operational modules, and the Next.js landing website were not changed.
- Google/Microsoft login remains future work and was not implemented.
- Phase 2 customer-facing wording cleanup and Phase 3 account setup UI redesign were not implemented as part of this change.

Related files:

- `saas/router.py`
- `saas/service.py`
- `templates/saas/`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Subscription History

docs/history/subscriptions/README.md

2026-08-19 - Capacity-Based Packaging

ADR 0025 makes the normal customer feature matrix identical for Starter, Professional, and Enterprise AI while preserving 1/5/25, 5/20/100, and 25/100/500 organization capacity. The source-aware feature resolver accepts coherent active paid, promo, and demo authority only after permission and scope. Active promo operational feature snapshots are reconciled without modifying exact grant capacity, immutable promo evidence, Paddle, or expired authority. Priority support remains a separate service-level distinction.

2026-08-06 - Existing Workspace Paid Activation Through Paddle

Activation-required existing customer workspaces can now choose Professional or Enterprise AI from Organization Account, confirm a SchoolGroup billing identity, review an operational-branch quote, and launch Paddle without creating onboarding or tenant records. A dedicated activation aggregate preserves quote, branch, checkout, payment, contract, and provider lineage. Starter remains unavailable pending complete restricted-branch enforcement.

Only verified `transaction.completed` evidence creates paid commercial authority. The atomic transition preserves the existing SchoolGroup and operational data, keeps classification `customer`, activates the workspace lifecycle, and creates the paid subscription, workspace/branch entitlements, and paid tenant link. Promo redemption, demo conversion, and new PendingOrganization checkout remain unchanged.

Organization Account **Choose a Plan** reopens plan selection while the activation is `draft` or `checkout_ready`. The saved plan is highlighted but may be replaced by another eligible plan, which recalculates the authoritative branch quote without a Paddle call, PaymentAttempt, or branch mutation. Once checkout starts, payment is processing, or activation evidence requires manual review, replacement fails closed. Only verified payment completion can establish paid commercial authority.

Final PostgreSQL validation proved migration rollback/idempotency, one unresolved activation, one transaction launch, duplicate and out-of-order webhook safety, atomic drift rollback, and exactly one winner in a paid-versus-promo source race. Workspace-specific provider address/business lineage and complete transaction validation prevent stale customer defaults or incomplete provider responses from becoming activation authority.

Organization Account derives its activation badge from the current payment attempt, not from `workspace_lifecycle_status=provisioning`. No attempt means Activation required; a current unexpired checkout-started or processing attempt means Payment processing; failed, cancelled, and expired attempts use recovery presentation.

2026-08-05 - Secure Promo Definition Foundation

M2 adds Platform-managed Starter, Professional, and Enterprise AI promo definitions with exact positive capacity bounded by the established catalog. Controlled scopes, one expiry policy, validity, redemption-policy metadata, branch snapshots, Draft/Active/Paused/Revoked lifecycle, duplication, replacement linking, row locking, and durable redacted audit are available.

Raw codes are TIS-generated, use at least 100 bits of secure randomness, appear once under `Cache-Control: no-store`, and are never persisted or audited. The database stores dedicated-secret HMAC-SHA256 lookup authority plus a safe mask. Only Platform Owners activate or revoke; authorized Developers may manage non-terminal definitions. Promo redemption, tenant grants, M1 authority, Paddle, payment, onboarding, provisioning, and current subscription behavior remain unchanged.

2026-08-04 - Unified Operational Commercial Capacity Authority

One facade now resolves commercial access, capacity source, confirmed plan and quantity, current branch/staff/teacher usage, remaining capacity, violations, minimum eligible plan or Custom, and safe recovery. It composes the existing M7/M8 authorities and fails closed on missing, ambiguous, or contradictory relationships. Demo and Internal Sandbox are explicitly unmetered in M1.

Staff capacity now correctly counts every distinct active tenant operational User in the workspace regardless of role, title, position, or internal-test attribution. An operational owner counts. A teacher-position User counts as staff, and an associated active Teacher record separately counts as a teacher. Platform users, inactive users, other tenants, and account-only SaaS identities do not count. Known teacher IDs deduplicate across branches and academic data; blank legacy identities never merge.

Capacity-increasing writes lock the SchoolGroup and perform permission/scope validation, authority resolution, recount, proposed-final-state evaluation, mutation, and commit as one transaction. This covers branch creation and reactivation, bulk branch status changes, user growth/reactivation, teacher creation/year-copy, academic-year activation, and paid/demo provisioning final validation. Existing over-capacity tenants preserve records and access but may not further increase an exceeded dimension.

2026-07-29 - Per-Branch System-User And Teacher Capacity

Plan eligibility combines persisted `max_branches`, `max_system_users`, and `max_teachers`. Starter supports 1/5/25, Professional 5/20/100, and Enterprise AI 25/100/500. Required non-negative estimates are stored per onboarding branch and summed organization-wide. Legacy organization totals are assigned to the primary branch when no branch estimates exist.

Before payment, system-user and teacher authority separately uses the greater of estimated and actual same-workspace counts; paid operations use actual counts. This milestone originally excluded teacher-position and internal-test login records from system-user usage. M1 supersedes that operational rule: every active tenant operational User now consumes staff capacity, while active Teacher records separately consume teacher capacity. Inactive, platform, account-only, other-tenant, inactive-branch, and inactive-year data remains excluded as applicable.

The lowest eligible plan is recommended. The shared decision protects selection through payment reconciliation and includes both people counts in quote fingerprints. Estimate changes clear only an undersized plan and supersede stale checkout lineage. Paid system-user and teacher growth plus downgrades fail before mutation when capacity would be exceeded. The always-visible Custom card never creates Paddle checkout and is emphasized above any Enterprise AI maximum.

The current Teacher model has no inactive/reactivation state and no teacher import endpoint exists. Current post-activation enforcement covers individual teacher creation and preflights academic-year copying atomically; future import or reactivation workflows must adopt the same capacity boundary.

2026-07-28 - Authoritative Plan Branch-Capacity Enforcement

Initial self-service selection now enforces the persisted plan limits: Starter one active billable branch, Professional five, and Enterprise AI twenty-five. All higher plans remain eligible below their capacity, while organizations above twenty-five receive the custom-plan contact path. Actual active branches still determine the per-branch total.

The same fail-closed capacity decision protects selection, quote generation, checkout preparation and launch, and payment validation. If branch editing makes the selected plan undersized before payment, its selection and checkout lineage are superseded; an eligible higher plan remains selected. Tenant-scoped branch counting prevents another organization from affecting eligibility.

2026-07-28 - Pre-Payment Branch Editing And Quote Supersession

Onboarding branches remain editable until confirmed payment. The existence of an unpaid demo SchoolGroup, tenant link, Paddle customer, checkout session, or billed-but-unpaid attempt is not paid provisioning evidence. Once payment

is confirmed, branch changes move to Subscription Management.

Pre-payment branch changes invalidate the old quote and checkout lineage, supersede incomplete attempts, and force a new immutable transaction using the new active branch count. Late events for the old transaction cannot activate, convert, or reprovision the workspace.

2026-07-28 - Authoritative Fixed Branch Quantity In Paddle Checkout

Initial checkout now keeps the TIS-calculated billable branch quantity fixed through payment. The server creates an automatically collected Paddle transaction with customer, address, catalog price, and exact quantity; verifies its item, subtotal, and quote-fingerprint evidence; then marks the ready transaction billed before releasing its transaction ID to Paddle.js.

Quantities of one, two, or more remain data-driven. Branch changes continue only through TIS branch and subscription-quantity workflows. No shared catalog price is mutated and no quantity-specific price is created. Automatic payment completion remains the recurring-subscription, webhook, and conversion authority.

The public launcher accepts only the unique locally attributable transaction after Paddle confirms `billed` and automatic collection. It rejects draft, ready, canceled, past-due, unrelated, and mismatched transactions, while retry clicks reuse the existing started checkout rather than creating another transaction.

2026-08-18 - Promo-Aware Commercial Access Portal

Organization Account Billing & Subscription now resolves the selected workspace's authoritative commercial source before choosing a customer view. Promo-backed workspaces render a dedicated read-only Commercial Access page composed from M1 capacity authority and attributable immutable promo metadata. It shows the plan, safe access status and period, masked reference, and authoritative capacity without calling Paddle or exposing recurring billing, invoice, plan-change, quantity-change, or cancellation controls. Paid and demo portals are unchanged. Promo-to-paid conversion remains deferred.

2026-08-18 - Promo Expiry Recovery And Paid Continuation

Promo access is dynamically active only before `PromoGrant.effective_to`. At expiry, operations fail closed even when persisted entitlement rows still say active. `grace_period_days` identifies a recovery interval for customer presentation and paid continuation only; it never extends operational access or mutates tenant data.

Expired and recovery-period promo owners can prepare the existing-workspace paid activation quote and launch Paddle checkout without replacing the SchoolGroup. Promo remains the sole tenant source until verified `transaction.completed` evidence. Completion locks and revalidates current authority, then atomically ends promo entitlement, relinks the existing tenant source to the paid contract, establishes paid workspace/branch entitlements, and records immutable conversion history. Failed, abandoned, or conflicting attempts retain promo authority and no paid entitlement. Active promo conversion is deliberately deferred to avoid treating unused promo value as paid credit.

A shared authority-driven badge component now renders Promo, Demo, and Paid identity with source icon, lifecycle status, and plan treatment in compact and full modes.

2026-08-05 - Promo Commercial Grants

Approved promo definitions can now become non-Paddle commercial authority only through verified owner activation. The transaction creates immutable redemption/grant snapshots, promo workspace entitlement, explicit active or inactive branch entitlement, and a promo-sourced tenant link. It supports both completed onboarding and separately aligned existing tenants without creating fake payment, subscription, contract, demo, or pending-organization records.

Promo capacity is not a Paddle quantity. Branch selection determines covered operational branches; organization-wide staff and teacher overage blocks activation without mutating people records. Pending sessions grant no access, expiry fails closed, and paid subscription history and provider behavior are unchanged.

2026-08-17 - Explicit Promo Branch Evidence And Reconciliation

Promo activation now separates selectable active branches from the complete preserved workspace branch inventory. Selected branches receive one assignment and active entitlement; every unselected branch receives an inactive entitlement even when operationally inactive. Branch creation and individual or bulk reactivation consume unused promo capacity and update assignment, entitlement, and operational status atomically. A dry-run-first PostgreSQL reconciliation command repairs only deterministically missing inactive evidence and leaves contradictory states for manual review.

2026-07-28 - Expired-Demo Checkout Identity Repair

Paddle sandbox recovery now recognizes the existing active tenant owner as authoritative when the authenticated account, owned organization, demo source, tenant link, SchoolGroup, operational user, and email resolve uniquely. A stale account-user link from a deleted or inactive account is safely reassigned to that account/workspace relationship. Unrelated tenants, active former accounts, multiple identities, and live email-only recovery remain blocked. Failed checkout preparation exposes no internal diagnostics and shows a retry state; the same workspace UUID, SchoolGroup, and branches remain unchanged.

2026-07-28 - Expired Demo Subscription Continuity

Expired demo subscription selection now reads the preserved SchoolGroup and its active operational branches, presents real public plans and billing intervals, and continues through existing Paddle checkout. Confirmed payment still converts the same workspace; missing configuration produces safe support guidance rather than unavailable placeholders.

2026-07-20 - M7 Phase 5 Lifecycle Policy, Cancellation, And Reversal

The Subscription Management portal now resolves customer-visible state and allowed actions through `saas.subscription_lifecycle_service`. Upgrade, downgrade, branch increase/reduction, cancellation, and cancellation reversal controls are exposed only when authorization, provider subscription state, pending requests, effective dates, and local relationships permit them.

Authorized billing administrators can schedule cancellation at period end and reverse it before the effective boundary. Current paid access remains active until authoritative provider evidence confirms the end of the paid period. Provider conflicts, unknown outcomes, and local/provider mismatches fail closed to manual review.

2026-07-18 - M7 Payment Lifecycle And Reconciliation Protections

Initial checkout and post-activation subscription-change transactions are resolved as distinct payment lifecycles. Webhook processing remains signature-verified and idempotent, validates attributable provider subscription/transaction evidence, and prevents change transactions from replaying initial provisioning transitions.

Diagnostics and the sandbox-guarded finalized-lifecycle reconciliation script expose sanitized evidence and default to dry-run behavior. Reconciliation applies only when stored completed webhook evidence and authoritative Paddle transaction data agree; unexpected or ambiguous state is blocked rather than guessed.

2026-07-17 - M7 Phase 4 Plan Upgrades And Scheduled Downgrades

Authorized billing administrators can preview and submit plan transitions from `/saas/subscription`. Paddle supplies monetary previews and proration outcomes. Upgrades use immediate provider proration with payment-failure prevention and do not become local entitlement truth before verified completion. Downgrades are scheduled for the next billing period and preserve the current plan until provider/webhook confirmation at the effective boundary.

Customers can cancel or replace an eligible scheduled plan change before it becomes effective. TIS sends complete retained recurring items and enters manual review on provider/local ambiguity.

2026-07-20 - M7 Phase 6 Billing History and Invoice Management

The customer Subscription Management page now retrieves billing history directly from Paddle's paginated `GET /transactions` API, scoped to the customer's confirmed provider subscription. TIS displays provider transaction totals, statuses, origins, invoice numbers, and returned credit/refund adjustments without creating local financial records or calculating replacement amounts.

Eligible invoice downloads use Paddle's `GET /transactions/{transaction_id}/invoice` API. TIS reauthorizes the current billing user, resolves the invoice against the confirmed customer subscription, and requests a fresh expiring provider URL at download time. The provider URL is not stored locally. No billing-history cache, schema change, migration, or webhook behavior is introduced; temporary provider failures render customer-safe retry states.

2026-07-16 - M7 Phase 3 Active Branch Quantity Management

Authorized organization billing administrators can now preview and submit paid branch-quantity changes from `/saas/subscription`. Paddle remains the sole source of financial calculations: TIS sends the complete retained subscription item list to Paddle's subscription-update preview endpoint and stores only customer-safe charge, credit, net, recurring-total, and effective-date summaries.

Increases use `prorated_immediately` with `on_payment_failure=prevent_change`. Local paid capacity does not increase until verified `subscription.updated` and successful `transaction.completed` evidence confirms the requested quantity and subscription-update payment. Reductions use `prorated_next_billing_period`, issue no immediate refund, and remain locally scheduled until a renewal-boundary subscription webhook confirms the reduced quantity. Reductions below active operational branch usage are rejected, and scheduled reductions can be restored before their effective date.

`PaymentSubscription.quantity` remains the only entitlement-capacity authority. Branch creation and individual or bulk reactivation now fail closed for provisioned SaaS tenants when confirmed paid capacity is exhausted. Provider mismatches, unsupported state, stale previews, or ambiguous ownership enter a customer-safe blocked/manual-review path without exposing provider diagnostics.

This folder tracks meaningful changes to TIS subscription plans, pricing, billing status, payment behavior, checkout assumptions, and provider boundaries.

Related docs:

1 `docs/adr/0003-paddle-payment-architecture.md`

1 `docs/adr/0004-webhook-only-payment-confirmation.md`

1 `docs/TIS_MASTER_CONTEXT.md`

2026-07-11 - Paddle Transaction Payment Launcher

Paddle transaction checkout now uses a dedicated public SaaS payment launcher page at `/saas/payment` instead of the app root or operational login page. Server-side checkout still creates Paddle transactions through the existing payment service and still redirects to Paddle's returned `transaction.checkout.url`; `PADDLE_CHECKOUT_BASE_URL` should point to `https://app.tisplatform.com/saas/payment` so Paddle appends `_ptxn` to the launcher page.

The launcher page loads Paddle.js from the official Paddle CDN, initializes Paddle with the public `PADDLE_CLIENT_TOKEN`, uses `PADDLE_ENVIRONMENT` for sandbox/live mode, reads `_ptxn`, and opens checkout for the transaction. It does not require SaaS or operational login, does not expose `PADDLE_API_KEY`, and does not change webhook-confirmed payment state, subscription activation, provisioning, pricing, or billing transitions.

2026-06-30 - Paddle Initial Checkout Price Mapping Configuration

Initial Paddle checkout now has a script-based configuration path for mapping TIS subscription plan prices to Paddle provider price IDs. The runtime source of truth remains `subscription_plan_prices.provider_price_id`; Paddle credentials and endpoints remain environment variables.

Added configuration support:

- `scripts/sync_paddle_price_ids.py`
- `config/paddle/paddle_prices.sandbox.example.json`
- `config/paddle/paddle_prices.production.example.json`

The sync script validates the six required plan/interval mappings: Starter monthly, Starter annual, Professional monthly, Professional annual, Enterprise AI monthly, and Enterprise AI annual. Real local, sandbox, and production mapping files are ignored by Git so sandbox and live Paddle price IDs remain separated.

If a selected plan price still lacks a Paddle provider price ID, checkout remains fail-closed before calling Paddle. Customers see a support-oriented Secure Payment message while internal diagnostics retain plan code, billing interval, and currency context.

Smart Timetable Stage 2 Version Foundation

docs/history/workforce-planning/2026-08-19-smart-timetable-stage-2-version-foundation.md

Stage 2 converted timetable storage from one mutable live set into a durable versioned aggregate without adding automatic generation. It added input snapshots, timetable versions, one exact-scope active pointer, future generation-run records, version-owned placements, lock metadata, and version-scoped collision constraints.

Existing populated branch/year timetables are migrated exactly into one imported compatibility version and made operational through an active pointer. Deterministically inconsistent rows remain unchanged and produce safe stale evidence. Empty settings-only scopes are not versioned.

The current assignment experience remains available through a copy-on-write bridge. The first edit of an active imported timetable creates a working draft; later legacy edits reuse that draft. Active history remains immutable, while page views and XLSX/PDF exports resolve the operational version.

No solver, worker, generation endpoint, generation UI, room/resource model, availability model, or new timetable capacity authority was added.

Smart Timetable Stage 3 Readiness And Slot Semantics

docs/history/workforce-planning/2026-08-19-smart-timetable-stage-3-readiness-and-slot-semantics.md

Stage 3 introduced one deterministic projection from existing timetable settings. Teaching periods stay fixed. Full-period/all-day blocks disable covered slots, every-day rules expand across configured days, between-period blocks consume no lesson slot, and partial overlaps are explicit configuration errors.

The read-only `TimetableReadinessService` evaluates one selected organization, branch, and academic year against configuration, Planning demand/allocation, HRT, authoritative teacher capacity, slot sanity, locks, staleness, and active runs. Only `generation_ready` is ready, and it does not guarantee solver feasibility.

The page shows readiness and corrective links. Unavailable slots cannot be assigned through either the route or mutation service; existing placements are preserved as stale. Versions and exports remain compatible. No schema, solver, worker, generation endpoint, availability, rooms/resources, or Stage 4 publication UI was introduced.

Smart Timetable Stage 3.5 Composed Timeline

docs/history/workforce-planning/2026-08-21-smart-timetable-stage-35-composed-timeline.md

Stage 3.5 corrected the configuration engine so teaching clocks are no longer calculated independently of non-teaching blocks. One canonical service now composes each working day from shift start, teaching-period count and duration, and applicable blocks. After-period blocks insert at a selected teaching boundary and shift every later period. Fixed-time rows remain compatible only when their stored start is a live boundary; ambiguous rows remain unchanged and produce an explicit blocker.

The calculated end, configuration preview, main timetable bands, readiness, assignment validation, snapshot fingerprints, future solver-ready teaching slots, and XLSX/PDF rows consume the same projection. The migration adds placement mode, after-period boundary, and duration columns idempotently and defaults legacy rows to fixed time. Controlled block types were expanded without removing existing keys.

This stage added no solver, OR-Tools package, Generate/Regenerate endpoint, availability, room/resource, or soft-constraint configuration.

Smart Timetable Stage 4 Version Publication

docs/history/workforce-planning/2026-08-21-smart-timetable-stage-4-version-publication.md

Stage 4 made the Stage 2 lifecycle visible without adding automatic generation. The timetable page identifies the selected version, lifecycle, origin, freshness, manual changes, and active state; lists same-scope history; and permits explicit historical review without changing the active timetable.

Immutable history can be copied to a new draft. Draft lessons can be locked or unlocked under a dedicated permission, and lock changes refresh snapshot authority. Version-specific solver-independent validation checks current authority, demand completion, Planning teachers, canonical slots, collisions, stale placements, and locks. Generation readiness remains a separate scope-input evaluation.

Publishing a fresh validated draft locks and revalidates the version and revisioned active pointer, preserves the previous active version as superseded history, records the actor/time, and makes the published version immutable. Same-scope comparison, selected-version XLSX/PDF export, and safe draft/superseded archive behavior are also available. Placement permissions are aligned separately for create, replace, and clear. No schema, solver, worker, generation endpoint, availability, rooms/resources, commit, push, deployment, or production action was introduced.

Smart Timetable Stage 5.1 Generator

docs/history/workforce-planning/2026-08-22-smart-timetable-stage-51-generator.md

Stage 5.1 added real automatic timetable generation and regeneration. Google OR-Tools CP-SAT 9.15.6755 is isolated to `requirements-worker.txt` and the dedicated `python -m timetable_generation_worker` process. Web requests capture immutable schema-v3 inputs and use the existing PostgreSQL generation-run table as a durable queue with claim locking, leases, heartbeat, bounded recovery, cancellation, progress, idempotency, and one active run per SchoolGroup/Branch/Academic Year.

The problem contract preserves exact section-subject-teacher weekly demand, Planning and subject-specific HRT authority, composed Stage 3.5 teaching slots, locks, fingerprints, and regeneration source state. CP-SAT enforces exact demand, section/teacher exclusivity, locks, and explicit diversity. An OR-Tools-independent validator and a final current-input/source-revision comparison gate persistence.

Successful generation atomically creates a separate unpublished publication-ready version and all placements while leaving `TimetableActiveVersion` and published history unchanged. Generate/Regenerate actions, real phase polling, safe terminal messages, and `timetable.generate` are exposed now. Stage 5.2 simplification, Delete Working Timetable, Timetable History simplification, teacher My Timetable, and published-only teacher visibility remain deferred. No production action occurred.

On 2026-08-26, the production execution host changed to an on-demand Render Workflow task. The solver pipeline and all durable safety described here remain unchanged; `python -m timetable_generation_worker` is now only an optional local fallback and `requirements-workflow.txt` is the production task dependency set.

On-Demand Timetable Generation Workflow

docs/history/workforce-planning/2026-08-26-on-demand-timetable-generation-workflow.md

Production timetable generation now provisions compute per durable run through a Render Workflow task. Generate and Regenerate first commit the existing `TimetableGenerationRun` and immutable snapshot. The server then starts the configured task with only the run public ID; PostgreSQL remains authoritative for input, scope, progress, cancellation, failure, and result.

The focused `timetable_generation_workflow.py` task claims exactly that queued run and invokes the existing single-run executor. Existing leases and heartbeats reject duplicate or late executors, and the atomic persistence service still rechecks staleness and creates at most one unpublished candidate without changing the active pointer. The solver, validator, regeneration diversity, locks, and Stage 5.2 publication/deletion/visibility rules are unchanged.

The web service uses `render==1.0.1` without OR-Tools. The Workflow runtime installs `requirements-workflow.txt`, which adds pinned OR-Tools. Render task retries are set to zero; deterministic and infrastructure outcomes become durable TIS terminal state, and the user may Generate Again. Immediate dispatch failure ends an unclaimed run safely. Cooperative database cancellation is retained; calling Render cancellation is not required for this MVP.

Future Render setup is manual because Workflows are not currently represented by Blueprints: configure the same repository/revision, `DATABASE_URL`, the TIS solver settings, and task entry `timetable_generation_workflow:app`. Configure the web with the server-only `RENDER_API_KEY` and `TIS_TIMETABLE_WORKFLOW_TASK_SLUG`. No Render resource or production database was changed by this work.

Smart Timetable Stage 5.2 Simplified Workflow And Published Visibility

docs/history/workforce-planning/2026-08-26-smart-timetable-stage-52-simplified-workflow.md

Stage 5.2 simplified the school workflow to Configure, Generate Draft, Review/Edit Draft, and Publish to Users while preserving all Stage 2-5.1 version and solver internals. Customer language replaces technical lifecycle state on the main page; version selection, comparison, exports, and archive remain in History.

The 2026-08-28 academic-quality extension keeps this workflow intact while adding branch/year settings for mapped core daily distribution, spread preferences, ICT, grouped Swimming activities, and regeneration diversity. The settings are captured in immutable generation snapshots and do not change approval/publication semantics.

Delete Draft Timetable permanently removes an eligible current mutable unpublished candidate under scope locks. It removes version placements, preserves snapshots, runs, audit fields, official publication, and history, and uses `timetable.delete_versions`. Historical Archive remains a separate History-only action.

Timetable History permits permanent deletion of any non-active, never-published candidate regardless of origin or internal draft/archive state. Unpublished dependent versions are removed child-first, generation source/result links are nulled where optional, shared snapshots and generation audit records remain, and protected published lineage or active generation is reported rather than silently removed. The dedicated assignable `timetable.delete_versions` permission controls this action and is included in the Administrator defaults. History also provides exact-scope Delete All Unpublished Timetables.

Authorized users may drag an unlocked lesson within a mutable Draft Timetable to a canonical teaching slot. Empty destinations produce an immediate move; occupied destinations attempt one atomic swap. Non-teaching periods, active publication, locked lessons, class collisions, and teacher collisions fail without a partial edit. Successful edits increment the existing revision and return the candidate to the normal re-check workflow. Publishing now requires explicit first-publication or replacement confirmation and presents Published to Users instead of technical active version language; publication permissions and transaction semantics are unchanged.

The Published Timetable management view offers Edit This Timetable, which copies the official timetable into an editable draft, and Create New Timetable, which creates a fresh empty draft from current Planning/configuration authority. Both preserve the official timetable and active pointer until the draft is explicitly published.

Approve Draft replaces Check Timetable in customer-facing workflow language. It validates the exact selected draft and records the administrator and approval time. Generated and regenerated results begin unapproved; placement, drag/drop, swap, lock, regeneration, or stale-authority changes clear approval. Publish requires a still-current approval and repeats validation before the existing atomic pointer swap. The normal Draft workspace groups Regenerate and Approve Draft before approval, then shows Publish Timetable after approval. Destructive, comparison, archive, and version-specific export controls remain in History. Regeneration now requires 25 percent of unlocked direct-source placements to change, rounded up, and reports a controlled failure when no valid timetable can reach that diversity.

The published-only service resolves strictly from `TimetableActiveVersion`. `/my-timetable` uses exact-scope teacher identity and exposes only that teacher's official lessons. A general published page supports other view-only users. Management drafts, history, solver details, readiness, locks, and controls are not exposed there. No schema, migration, CP-SAT constraint, deployment, or production action occurred.

Explicit Planning Subject Demand Foundation

docs/history/workforce-planning/2026-08-28-explicit-planning-subject-demand-foundation.md

Stage 1 adds `planning_subject_demands` as the normalized future authority for one Planning section's subject and weekly periods. Rows are branch/year scoped, retain active or retired state, and carry normal timestamps and optional actor metadata. Composite section and Subject foreign keys prevent scope drift. A partial unique index permits one active row per section and subject while preserving retired rows.

Migration `20260828_004_planning_subject_demands_foundation` backfills only Current and New Planning sections. It joins Subjects by branch, academic year, and grade, copies the current weekly hours, and inserts only when no prior section-subject demand exists so reruns neither duplicate demand nor reactivate retirement evidence.

`planning_subject_demand_service.py` resolves explicit rows before legacy grade and weekly-hours fallback. An inactive explicit row is retirement evidence and suppresses fallback. No existing consumer changes authority in Stage 1: teacher allocation, reports, readiness, timetable snapshots and generation, drafts, publication, and UX remain unchanged. No production or local `tis.db` operation occurred.

Planning Subject Demand Consumer Transition

docs/history/workforce-planning/2026-08-28-planning-subject-demand-consumers.md

On 2026-08-28, Stage 2 moved live required-period consumers from direct grade-level `Subject.weekly_hours` inference to exact-scope, explicit-first `PlanningSubjectDemand` resolution. Planning totals, teacher workload, timetable readiness/workspace/snapshot/generation input, Subject Scheduling Rule arithmetic, and required-hours reporting now honor per-section active, inactive, and zero demand.

Sections without an explicit row retain the Stage 1 legacy fallback during the transition. Teacher assignment identity, solver behavior, timetable presentation, and published timetable history were not redesigned or mutated.

Atomic Curriculum Adjustment Apply

docs/history/workforce-planning/2026-08-29-curriculum-adjustment-apply.md

Stage 4 turns an already-reviewed preview into one exact-scope transaction. Dedicated `curriculum.adjust` authorization, preview-fingerprint revalidation, row locks, active-generation rejection, explicit teacher decisions, qualification/capacity, scheduling rules, and Draft locks all gate mutation.

Successful application retires or updates source demand, makes target demand explicit, applies only confirmed assignment choices, retires an active zero-source section rule, marks the unpublished Draft stale, clears approval, and inserts one durable audit. A unique scope/fingerprint prevents duplicate application. Every affected write rolls back together on failure.

No timetable regeneration occurs. Existing snapshots and placements are preserved, and published versions plus the active pointer are never mutated.

Read-Only Curriculum Adjustment Preview

docs/history/workforce-planning/2026-08-29-curriculum-adjustment-preview.md

Stage 3 adds a non-mutating preview for late subject-demand changes after Planning, teacher allocation, and Draft Timetable data exist. It supports one grade, selected Current/New sections, or all active source uses within one exact tenant/branch/year.

The result projects source-to-target periods, current teachers, suggested but uncommitted teacher choices, load/capacity, normalized scheduling-rule arithmetic, grouped legacy warnings, and Draft stale/regeneration consequences. A deterministic fingerprint binds the analyzed authority for future stale-confirmation checks.

No demand or assignment is written, no timetable is regenerated, and published history remains untouched. Apply and atomic rollback behavior are not part of this stage.

Curriculum Transfer and Subjects Display Correction

docs/history/workforce-planning/2026-08-29-curriculum-transfer-and-subjects-display-fix.md

The curriculum adjustment contract now preserves the administrator's requested transfer count from the guided UI through preview fingerprinting and atomic apply. Each section derives source and target results from its current effective Planning demand. Non-positive transfers are rejected, requests larger than a section's source demand block preview, partial transfers keep the source active, and only a zero result retires it.

The Subjects list now reads explicit-first `PlanningSubjectDemand` for Current/New sections in the selected branch and academic year. Uniform grade-subject demand is shown as the effective Planning weekly-period value. When sections differ, the list shows **Varies** and exposes each section's value. The catalog `Subject.weekly_hours` value remains unchanged and serves only as fallback/default context.

Guided Curriculum Adjustment UI

docs/history/workforce-planning/2026-08-29-guided-curriculum-adjustment-ui.md

Stage 5 introduced the administrator-facing workflow for late-stage curriculum changes. Permissioned users can select a Current/New Planning scope, choose source and target subjects, inspect the deployed preview, make an explicit teacher decision per section, and confirm the atomic apply request.

The UI clearly separates blockers and warnings and describes Draft regeneration in customer language. If the reviewed state changes, it refreshes the preview before any apply attempt can proceed. Successful application provides navigation to Timetable and a regeneration CTA without starting generation. Backend transaction ownership, solver behavior, and published timetable history remain unchanged.

Reduce-Only Curriculum Adjustment

docs/history/workforce-planning/2026-08-29-reduce-only-curriculum-adjustment.md

The guided curriculum workflow now distinguishes period transfer from source-only reduction. Reduce-only carries an authoritative requested reduction, requires no target subject or teacher decision, and calculates each selected section from explicit-first Planning demand. Partial results stay active; zero results retire the source demand and its invalid section-level scheduling rule.

Preview and atomic apply retain scope locking, stale fingerprint checks, Draft lock validation and invalidation, rollback, audit, and immutable published history. Subject Details presents effective Current/New Planning periods and offers a `curriculum.adjust`-gated, grade-specific prefilled reduction action. Original `Subject.weekly_hours` remains catalog/default data. Multi-grade aggregation is not added because the established transaction reviews one exact subject code and subject codes are grade-specific.

Teacher Scheduling Rules

docs/history/workforce-planning/2026-08-30-teacher-scheduling-rules.md

Implemented tenant-scoped teacher timing configuration under Timetable Settings. Administrators use the dedicated `timetable.manage_teacher_rules` permission to create, edit, and remove Must teach, Unavailable, Prefer teaching, and Prefer free rules for configured days, numbered/first/last periods, and assigned-class, grade, or Current/New section targets.

Migration `20260830_001_teacher_scheduling_rules` creates normalized rule, slot, and target tables after establishing the composite teacher scope required by PostgreSQL. The tables are deferred from baseline metadata creation so the repository's production migration order remains safe and idempotent.

Schema-v4 snapshots include canonical rules in timetable constraint authority. Required rules are hard CP-SAT constraints, preferences affect only optimization, and the independent validator repeats hard checks. Readiness/problem construction reports deterministic rule, workload, availability, target, and lock conflicts. Rule changes stale unpublished Drafts and clear approval; published timetable history, Planning demand, and teacher allocation are unchanged. No configured rules retains the previous generation behavior.

The administrator form was subsequently simplified to Teacher, Rule, Days, Periods, optional Sections, and Save. It directly displays P1?Pn, uses Current/New section labels with Select All, and removes normal grade, class-scope, and first/last controls. Schedule within these periods was added as a distinct hard semantic through migration `20260830_002_teacher_scheduling_window_semantics`: existing assigned lessons must fit inside the selected window, but selected slots do not create or imply extra lessons. Existing Must-teach, first/last, grade-target, and preference rows retain their original meaning.

Final administrator polish made Select All bidirectionally synchronize the exact visible Current/New section checkboxes, restored those targets during edit, and made saved summaries display grade-section names, All assigned sections, or All classes as appropriate. Selected-section targets remain intact through snapshots, the problem builder, CP-SAT, and independent validation.

Manual Draft edits now reject hard Unavailable placements and lessons outside an applicable Schedule-within window. Missing Must-teach occupancy remains permissible while editing, but complete Draft validation detects it along with unavailable, window, and selected-section target violations. That shared hard validation blocks approval and publication; soft preferences remain non-blocking and published history is unchanged.

An infeasibility investigation then found that multiple Schedule-within records covering the same demand were each prohibiting every slot outside their own row. That intersected the rows in CP-SAT even though readiness checked them separately, allowing a zero-blocker scope to fail as infeasible. Applicable rows now compose as a per-demand union consistently in manual edits, complete-Draft validation, problem construction, CP-SAT, and independent solution validation.

Pre-solve diagnostics now use exact teacher demand-to-slot matching over combined windows and unavailable periods, counting grouped activity teacher occupancy once. They also identify provable Must-teach/window, daily-coverage, hard maximum-per-day, minimum-teaching-day, and double-block conflicts with required-versus-available guidance. No hard rule is relaxed and no Planning, allocation, lock, Draft, or published-history authority is changed.

For infeasibility that remains solver-only, the Workflow now performs bounded diagnostic solves over isolated hard-constraint families and persists the result on the existing Generation Run. The diagnostic differentiates base collisions, locks, grouped activities/resources, subject rules, teacher rules, and combined subject/teacher-rule conflicts, while recording lock and grouped-activity counts. Relaxed diagnostic candidates are never saved and the complete hard model remains the only generation authority. A six-section, forty-slot-per-section regression with 240 periods, exact double blocks, daily coverage, hard maximum/day, and scoped teacher rules proves the full-utilization model itself remains feasible.

Production showed that the initial diagnostic's fixed ten-second cap could itself time out before proving a family. Isolation now reads the independent `TIS_TIMETABLE_DIAGNOSTIC_TIMEOUT_SECONDS` Workflow setting, defaults to 60 seconds per attempted profile, and may be raised without changing the main solve timeout. Zero-lock and zero-group scopes skip those profiles, and successful results include safe section, demand, teacher, rule, period, and selected-window counts where available.

Solver-backed feasibility gate

Timetable readiness now separates configuration completeness from global feasibility. A hard-only CP-SAT Workflow verifies and independently validates one complete timetable before Generate is enabled. Its placements are retained by the exact snapshot fingerprint for reuse as the optimization hint and timeout fallback; any teacher-rule or other timetable-authority change requires a new verification.

Advanced Timetable Release 1 Reconciliation And Solver Adapter

docs/history/workforce-planning/2026-09-01-advanced-timetable-r1-reconciliation.md

Milestone 0 reconciled the governed timetable design with the implemented versioned scheduling pipeline. The existing SchoolGroup/Branch/Academic Year scope, Planning demand authority, immutable snapshots and fingerprints, version-relative placements, revision-guarded mutation/publication, exclusive generation runs, CP-SAT execution, independent validation, permissions, and Render Workflow boundary remain the implementation foundation.

The first additive Release 1 slice introduced `timetable_solver.py`. The Workflow worker now calls a solver-neutral contract for solving and bounded infeasibility diagnosis. `CpSatTimetableSolver` delegates to the existing OR-Tools model without changing the problem or result contracts. No alternate solver, scoring authority, schema, API, UI state, migration, or service was introduced.

Repository evidence confirms that Planning's explicit-first `PlanningSubjectDemand` projection already supplies timetable demand; a separate academic-demand authority must not be added. It also confirms that locks remain placement metadata, while teacher Unavailable and Schedule-within behavior belongs to normalized teacher rules. General Room/Classroom/Campus authority and generic availability remain absent and require separate approval before implementation.

Structured Timetable Conflict Evidence

docs/history/workforce-planning/2026-09-01-structured-timetable-conflicts.md

Advanced Timetable Release 1 now uses `timetable_conflicts.py` as its canonical conflict boundary. The contract records a stable machine code, legacy source code, HARD/SOFT severity, DURABLE/RECALCULABLE/TRANSIENT evidence class, safe message, optional safe entities and slots, remediation, provenance, and internal requirement, allocation, or constraint correlation.

Public conflict serialization deliberately replaces internal correlation values with presence flags. Internal Lesson Requirement hashes and solver clauses are never emitted. Same-scope entity references may reuse information already exposed by the authorized workflow; unauthorized or cross-tenant references are generic and redacted.

Readiness retains its existing blocker/warning fields and adds canonical conflicts. Feasibility retains stored diagnostics and adds durable conflict projections. Generation-run responses derive durable conflict evidence from existing terminal status, failure category, safe message, and finish time. The independent validator retains legacy errors and adds recalculable canonical conflicts without importing CP-SAT.

Infeasibility, timeout, cancellation, stale authority, independent validation failure, and solver/execution failure remain distinct. Recalculable readiness and validation evidence is not persisted. No universal conflict table, migration, Room, generic availability, Resource, co-teaching, or frontend redesign was added.

Timetable Lesson Requirement Projection

docs/history/workforce-planning/2026-09-01-timetable-lesson-requirement-projection.md

Advanced Timetable Release 1 now has one internal domain contract between Planning demand and scheduling. `timetable_requirement_projection.py` resolves the exact SchoolGroup/Branch/Academic Year, Planning section, subject, effective weekly periods, existing teacher assignment or HRT fallback, and demand source. It consumes `PlanningSubjectDemand` first and uses `Subject.weekly_hours` only when no explicit row exists. Explicit zero and retired rows remain authoritative and never reactivate fallback.

The internal requirement identity is deterministic over scope and governing Planning source. A separate source fingerprint detects effective-period, active-state, and teacher-authority changes without changing the logical identity of an unchanged explicit source. These values are internal correlation and freshness evidence, not customer-facing business identifiers.

Schema-v5 snapshots store the requirement identity, source fingerprint, demand authority, and source ID. The existing problem builder remains compatible with schema-v3/v4 snapshots, while schema-v5 fails closed if projection provenance is missing. Readiness and snapshot creation now share the projection; feasibility and generation consume it through immutable snapshots, and the independent validator checks captured provenance without depending on CP-SAT.

Grouped Swimming remains its existing specialized equal-demand/common-teacher synchronization path over projected requirements. This work does not introduce general co-teaching, a Lesson Requirement table, a CRUD API, Room, generic availability, or generic Resource authority.

Professional Timetable Lifecycle UX

docs/history/workforce-planning/2026-09-02-professional-timetable-lifecycle-ux.md

The timetable workspace now exposes the existing Release 1 lifecycle in customer language: Draft, Validate, Approve, and Publish. This is not a new persisted state. Successful validation records approval and moves the Draft to `publication_ready`; the scoped active pointer identifies the official published version.

Version/history cards show origin/source, created or generated time, validation, approval, publication, mutability, and current-publication identity. Permissioned actions create independent working Drafts from current published or immutable history, while starting an empty Draft is separate. Generate and Regenerate are explicitly Draft operations.

Stale guidance explains that older inputs are neither corruption nor proof of infeasibility. Mutation/publication responses retain their message and status behavior while adding canonical conflicts for revision races, stale validation, approval, immutability, lifecycle errors, blocked slots, counts, and collisions. The UI offers a keyboard-accessible refresh action for stale concurrency evidence.

No migration, solver/Workflow behavior, Room, generic availability, Resource, co-teaching, partial regeneration, AI, or opaque score was added.

Planning And Subject Delete Dependency Guards

docs/history/workforce-planning/2026-09-03-planning-and-subject-delete-dependency-guards.md

Deleting a Planning section or a Subject previously either raised an unhandled `IntegrityError` (Internal Server Error) once enough dependent authoritative data existed, or in the Subject case silently succeeded for reference types no existing check covered. Both routes now perform a read-only, scope-safe dependency check before any mutation and block deletion with a specific, customer-safe explanation naming every blocking category and a real next action, rather than crashing, deleting silently, or leaving the administrator with no path forward.

Planning section delete (`routers/planning.py:delete_planning_section`) checks for `TeacherSectionAssignment`, `PlanningSubjectDemand`, `TimetableEntry`, `TeacherSchedulingRuleTarget`, `CalendarEvent`/`CalendarEventSectionTarget`, and section-scoped `SubjectDistributionRule` references. `TeacherSectionAssignment` is no longer silently deleted alongside the section - it is now a blocker like every other category, since no documented product rule required the previous cascade; the admin removes it through the existing Edit Planning Section save flow and retries. Multiple simultaneous blockers are listed individually with the specific action that resolves each one.

Correction on 2026-09-04: an inactive, zero-period `PlanningSubjectDemand` with `updated_by_user_id IS NULL` is the setup-only suppression created by Remove Subject Requirement, not operational demand or history. It no longer blocks an otherwise-empty section. The delete route removes only those exact rows, within the selected section/branch/year and in the same transaction as the section delete. If any other dependency blocks, the suppression is not cleaned. Active demands, Curriculum-Adjustment-touched rows, and every other dependency below remain blockers; no general cascade was introduced. Customer-facing remediation uses "Remove Subject Requirement", not "Remove demand".

Planning subject demand is split into two honest cases rather than being treated as uniformly permanent. Curriculum Adjustment (`curriculum_adjustment_apply_service._set_demand`) always stamps `updated_by_user_id` on a row it creates or modifies, while the one-time setup backfill migration never sets it. A demand row with no `updated_by_user_id` has therefore never been acted on by an admin - it is pure setup scaffolding - and a new, narrowly scoped action, `GET /planning/subject-demand/delete/{demand_id}` (surfaced as "Remove demand" next to the subject on the Planning page), hard-deletes exactly that row so the section can then be deleted. A row Curriculum Adjustment has ever touched, active or retired, is genuine history TIS preserves permanently (retirement never deletes the row), so it remains a permanent blocker with an honest explanation instead of a false "remove and retry" promise. Timetable placements follow the equivalent split implicitly: only entries in a mutable Draft can be removed, while published/active/superseded/archived placements are permanent history.

Subject delete and Bulk Delete (`routers/subjects.py`) consolidate what were previously two narrower checks into one `_get_subject_delete_blockers` scan covering `Teacher.subject_code`, `TeacherSubjectAllocation`, `TeacherSectionAssignment`, `PlanningSubjectDemand` (using the same removable/permanent split), `TimetableEntry.subject_code`, `CurriculumAdjustmentAudit` source/target codes, and `SubjectDistributionRule.subject_code`. The last three had no prior application-level check at all - a Subject referenced only by timetable placements, Curriculum Adjustment history, or a distribution-rule override could previously be deleted, silently orphaning that historical reference. Bulk Delete is atomic from the administrator's perspective: if any selected Subject is blocked, nothing in the batch is deleted, and every blocked Subject is named with its specific reason so the admin can resolve it (using the same Planning-page Remove-demand action where applicable) and retry. An unforeseen dependency reaching the database commit is still caught by an `IntegrityError` rollback rather than surfacing a raw error.

No archive/closed/inactive lifecycle, broad schema change, migration, or cascade deletion was introduced. The one new capability is a narrow, permission-gated hard delete of an untouched `PlanningSubjectDemand` row - using an existing, unused-until-now column (`updated_by_user_id`) as the sole eligibility signal - so every blocker shown to an admin now has either a real removal path or an honest permanent-history explanation.

Return-To Reopen-On-Load UI Pattern

docs/history/workforce-planning/2026-09-03-return-to-reopen-on-load-pattern.md

Server-rendered actions inside an expanded Planning section (native `<details>`) previously redirected to a fresh `/planning` render with no memory of which section was open, forcing the administrator to re-expand it for every follow-up action. This adds a small, reusable fix rather than an SPA conversion, by finishing wiring together capability that already existed in the codebase separately.

`redirect_utils.py` is a new shared module holding `safe_redirect_path()`, `redirect_with_notice()`, and `redirect_with_error()` - moved out of `main.py`, which now imports them under their original private names so every existing caller (`_safe_redirect_path`, `_redirect_with_notice`, `_redirect_with_error`) keeps working unchanged. `safe_redirect_path()` is the single open-redirect guard: it accepts only a path starting with exactly one `/`, rejecting `http(s)://` and protocol-relative `//` targets, and otherwise preserves the given `?query#fragment` unchanged. The notice/error helpers additionally inject their query parameter while re-appending a trailing `#fragment` after it, since a fragment must stay after the query string.

`static/js/reopen-on-load.js`, loaded globally from `base.html`, runs on `DOMContentLoaded`. It resolves a target element id from `window.location.hash` (the redirect case) or an optional inline `window.__tisReopenTargetId` a page can render for a non-redirect, in-place response, looks it up with `document.getElementById` (which never throws, unlike a raw CSS-selector lookup), opens it if it is a `<details>`, and scrolls it into view. A missing, stale, or renamed id is a silent no-op by design.

Planning is the first and, for now, only consumer. Each section's `<details class="subject-assignment-details">` now carries a stable `id="planning-section-{{ record.id }}"`. The "Remove demand" link passes `return_to=/planning#planning-section-<id>` (URL-encoded); `routers/planning.py: delete_planning_subject_demand` reads `return_to`, validates it with `safe_redirect_path(return_to, default="/planning")`, and redirects there on every outcome that returns to the list (success, and the not-found case), falling back to plain `/planning` when `return_to` is absent or unsafe. The in-place render outcomes that stay on the current request (permanent-demand refusal, the `IntegrityError` fallback, and `delete_planning_section`'s own blocked-dependency render) instead set `open_section_id` on `_render_planning_page`, which the template turns into the inline `window.__tisReopenTargetId` the shared script also understands - the same reopening behavior without a redirect. Permission-denied and not-authenticated outcomes are unchanged and never consult `return_to`.

Subjects and Teachers were not touched. This is scoped to Planning as the first concrete use case; other pages may adopt the same `return_to` + `#fragment` + shared-script convention later without needing a new mechanism.

Planning Subject Requirement Removal (Single And Bulk)

docs/history/workforce-planning/2026-09-04-planning-subject-requirement-removal.md

The prior round's "Remove demand" action (`GET /planning/subject-demand/ delete/{demand_id}`) only ever worked for a requirement with an explicit, untouched `PlanningSubjectDemand` row. A purely legacy-fallback requirement - resolved only from the Subject catalog's `weekly_hours`, with no explicit row at all - had no removal action, and the button was simply absent with no explanation. Reported live example: for one teacher assigned to three subjects in the same section, only the subject with an explicit row showed "Remove demand"; the other two, resolved by fallback, showed nothing. The condition controlling visibility was never about the teacher - it was whether an explicit `PlanningSubjectDemand` row existed at all for that section+subject pair, and if so, whether Curriculum Adjustment had ever touched it.

`_get_planning_requirement_removal_status` (`routers/planning.py`) now classifies every requirement using `resolve_section_subject_demands` - the exact same explicit-first/legacy-fallback authority already used to render the page, so classification can never diverge from what is displayed:

1 `removable`: an explicit row exists and `updated_by_user_id IS NULL`

(never touched by Curriculum Adjustment) - deletable.

1 `permanent`: an explicit row exists and has been touched - genuine

history, shown as "Protected (Curriculum Adjustment history)" instead of a hidden or missing action.

1 `fallback`: no explicit row exists; the requirement is resolved only from

the Subject catalog - now removable by creating an explicit `is_active=False`/`weekly_periods=0` suppression row. Unlike a Curriculum Adjustment zero-out retirement, this row is stamped with only the acting admin's `created_by_user_id` (audit trail); `updated_by_user_id` is left NULL. Because `updated_by_user_id IS NULL` is exactly the signal every removability check in this module treats as "never touched by Curriculum Adjustment", the suppression row stays classified `removable`/setup-only instead of becoming permanent - see the correction below.

1 `not_found`: no active requirement for that pair at all.

`POST /planning/subject-requirements/remove` handles both the single quick action ("Remove Subject Requirement", one target) and checkbox-selected bulk removal ("Bulk Remove Subject Requirements"). Targets are `"<planning_section_id>:<subject_code>"` strings; checkboxes use the `form="planningBulkRemoveForm"` attribute so selection works across every section rendered on the page in a single request, not only within one section - achievable without a frontend redesign since the whole Planning table is already server-rendered on one page load. Every target is scope-checked (section and subject must belong to the caller's branch/ academic year) and classified before any mutation; if any target is `permanent`, `not_found`, unparseable, or out of scope, nothing in the batch is removed and every blocked target is named with its specific reason, matching the same all-or-nothing contract established for Subject bulk delete. On success, a single-section batch reopens that section via the existing `return_to`/`open_section_id` reopen-on-load pattern; a multi-section batch returns to the plain Planning list, since there is no single section to reopen. Confirmation dialogs precede both single and bulk removal, naming the subject/count being removed.

`TeacherSectionAssignment`, `TimetableEntry`, and `CurriculumAdjustmentAudit` rows are never touched by this action - verified directly against a teacher assigned to all three example subjects (one explicit, two fallback), where the assignment rows remained intact after removing both fallback requirements. The prior round's `GET /planning/subject-demand/delete/ {demand_id}` route remains fully functional; it is no longer linked from the rendered page for cases the new route already covers, but it is still the supported way to fully clear a leftover setup-only suppression row (see correction below) so that Planning section/Subject deletion can proceed.

No schema or migration change: the suppression row reuses the existing `PlanningSubjectDemand` table and the previously-unused `created_by_user_id`/ `updated_by_user_id` columns. Subjects and Teachers pages were not changed. A teacher-based quick-select filter for bulk selection was considered but left out as an unnecessary frontend addition for this round; the admin selects the relevant checkboxes directly.

Correction (2026-09-04): setup-only suppression must not look like Curriculum Adjustment history

The first version of this change stamped a `fallback` suppression row with `created_by_user_id`/`updated_by_user_id` both set to the acting admin - mirroring a genuine Curriculum Adjustment retirement exactly. Since `updated_by_user_id IS NOT NULL` is the sole signal `_get_planning_requirement_removal_status`, `_get_planning_section_demand_status`, and `routers.subjects._get_subject_demand_status` use to classify a row as `permanent`/Curriculum-Adjustment-touched, that stamp made the row permanently undeletable the instant a plain admin cleanup created it - an admin removing a fallback requirement to unblock deleting the section could never actually get there, breaking the remove requirement -> remove section -> remove subject workflow this action exists for.

Fix (`_apply_planning_requirement_removal` in `routers/planning.py`): the suppression row now sets `created_by_user_id=<acting admin>` (audit trail of who suppressed it survives) but leaves `updated_by_user_id=None`. No schema change. This does not conflict with any existing invariant: `curriculum_adjustment_apply_service._set_demand` remains the only code path that ever sets `updated_by_user_id`, so `IS NULL` continues to mean exactly "not touched by Curriculum Adjustment" regardless of who created the row or why. A row becomes `permanent` only when Curriculum Adjustment itself later touches it - never merely by being suppressed from this action.

Verified end-to-end retry flow: a `fallback` requirement is removed (status becomes `not_found`, a setup-only row now exists) -> the Planning section is still blocked from deletion by that leftover row, but the row itself is still `removable` (not `permanent`) -> an admin clears it via the existing `GET /planning/subject-demand/delete/{demand_id}` route -> the Planning section deletes successfully -> the Subject deletes successfully. A row Curriculum Adjustment has genuinely touched (`updated_by_user_id` set directly) stays `permanent` throughout and is unaffected by this fix. Bulk removal of several `fallback` requirements in one request produces only setup-only rows (`updated_by_user_id IS NULL` on every one). No customer- facing wording changed: "Remove Subject Requirement" / "Bulk Remove Subject Requirements" / "Protected (Curriculum Adjustment history)" are unchanged, and no new lifecycle terminology was introduced - an admin never needs to know a "suppression row" exists.

Bulk-action UI completion (2026-09-04)

The first bulk UI exposed row checkboxes but placed its only submit button below the complete Planning table and provided no Select All control. Each expanded section now presents a section-scoped **Select all** checkbox beside the action it drives. The action uses the shared trash icon and reflects the live selection in its visible label: "Remove Subject Requirement" for one or "Remove N Subject Requirements" for several. Partial section selection gives the Select All control an indeterminate state, and removal controls are disabled with no selection. The existing cross-section form, atomic endpoint, permission boundary, confirmation, and persistence behavior are unchanged.

Empty-section deletion correction (2026-09-04)

After removing every fallback requirement, the visible section was empty but its inactive zero-period setup suppression rows still triggered the generic Planning-demand blocker. Section deletion now excludes only rows matching all safe artifact conditions (`is_active=False`, `weekly_periods=0`, and `updated_by_user_id IS NULL`) from the blocker check, then deletes those exact branch/year-scoped rows in the same transaction as the otherwise-allowed section delete. Cleanup does not run when another blocker exists. Active setup requirements, Curriculum Adjustment history, teacher assignments, timetable and calendar references, and scheduling rules remain protected. This is a narrow artifact cleanup, not cascade deletion.

Workforce Planning History

docs/history/workforce-planning/README.md

1 [2026-09-04 Planning subject requirement removal (single and bulk)](2026-09-04-planning-subject-requirement-removal.md)

1 [2026-09-03 return-to reopen-on-load UI pattern](2026-09-03-return-to-reopen-on-load-pattern.md)

1 [2026-09-03 Planning and Subject delete dependency guards](2026-09-03-planning-and-subject-delete-dependency-guards.md)

1 [2026-09-02 professional timetable lifecycle UX](2026-09-02-professional-timetable-lifecycle-ux.md)

1 [2026-09-01 structured timetable conflict evidence](2026-09-01-structured-timetable-conflicts.md)

1 [2026-09-01 timetable Lesson Requirement projection](2026-09-01-timetable-lesson-requirement-projection.md)

1 [2026-09-01 Advanced Timetable Release 1 reconciliation and solver adapter](2026-09-01-advanced-timetable-r1-reconciliation.md)

1 [2026-08-30 teacher scheduling rules](2026-08-30-teacher-scheduling-rules.md)

1 [2026-08-29 reduce-only curriculum adjustment](2026-08-29-reduce-only-curriculum-adjustment.md)

1 [2026-08-29 curriculum transfer and Subjects display correction](2026-08-29-curriculum-transfer-and-subjects-display-fix.md)

1 [2026-08-29 guided curriculum adjustment UI](2026-08-29-guided-curriculum-adjustment-ui.md)

This folder tracks meaningful changes to teacher information, workload planning, staffing gaps, planning sections, timetable relationships, and related reports.

Related files:

1 `routers/teachers.py`

1 `routers/planning.py`

1 `routers/timetable.py`

1 `teacher_capacity.py`

1 `timetable_logic.py`

History entries:

1 `2026-09-04-planning-subject-requirement-removal.md` - "Remove Subject

Requirement" and checkbox-selected, cross-section "Bulk Remove Subject Requirements" now cover legacy-fallback requirements (no explicit demand row) in addition to untouched explicit rows, fixing the inconsistency where the same teacher's subjects could differ in removability for reasons unrelated to the teacher; protected/Curriculum-Adjustment-touched rows show an explicit explanation instead of a hidden action. Same-day correction: the suppression row created for a fallback requirement sets only `created_by_user_id`, not `updated_by_user_id`, so it stays removable instead of instantly becoming permanent, preserving the remove requirement -> remove section -> remove subject workflow.

1 `2026-09-03-return-to-reopen-on-load-pattern.md` - a shared `redirect_utils.py`

open-redirect guard plus `static/js/reopen-on-load.js` finish wiring the existing `return_to`/fragment convention so a Planning section stays open and scrolled into view after "Remove demand" and related in-place actions, without converting TIS into an SPA.

1 `2026-09-03-planning-and-subject-delete-dependency-guards.md` - read-only

dependency checks before Planning section and Subject deletion, with every blocking category named; untouched setup-only Planning demand is now actually removable through a new narrow "Remove demand" action, while demand Curriculum Adjustment has touched and published timetable data stay honestly permanent instead of promising a fix that cannot work.

1 `2026-08-29-curriculum-adjustment-apply.md` - permissioned fingerprint guard,

atomic apply/audit, explicit teacher decisions, and Draft invalidation.

1 `2026-08-29-curriculum-adjustment-preview.md` - read-only scoped transfer preview,

teacher/capacity options, rule/grouped warnings, Draft impact, and stale guard.

1 `2026-08-28-planning-subject-demand-consumers.md` - live Planning, workload,

timetable, scheduling-rule, and required-hours report consumers adopt explicit per-section demand authority with missing-row legacy fallback.

1 `2026-08-28-explicit-planning-subject-demand-foundation.md` - normalized future

section-subject demand, safe backfill, scoped integrity, retirement state, and transitional legacy compatibility.

1 `2026-08-21-smart-timetable-stage-4-version-publication.md` - visible version history, draft locks/validation, comparison, explicit export, archive, and atomic publication.

1 `2026-08-21-smart-timetable-stage-35-composed-timeline.md` - automatic per-day teaching/block composition, calculated end time, explicit placement modes, and shared slot authority.

1 `2026-08-19-smart-timetable-stage-3-readiness-and-slot-semantics.md` - canonical fixed-period slot semantics, assignment protection, and solver-independent readiness.

1 `2026-08-19-smart-timetable-stage-2-version-foundation.md` - durable timetable versions, snapshots, active pointer, locks, migration, and legacy edit/export compatibility.

Workspace Classification History

docs/history/workspace-classification/README.md

This folder tracks the evolution of workspace identity, classification, lifecycle, intent, and later approved commercial separation rules.

2026-07-22 - M8B-1 Foundation

Added stable Workspace UUID, constrained Workspace Classification and Lifecycle metadata, onboarding Workspace Intent, SaaS Account Purpose, and internal-test identity attribution. Added validation-only services, a commercial-state skeleton, Platform Owner read-only display, a relationship diagnostic, and a controlled one-time backfill for confirmed existing test data.

M8B-1 deliberately leaves demo workflows, expiration, entitlements, branch entitlements, conversions, memberships, Al-Andalus migration, and commercial-state decisions unimplemented.

2026-07-22 - M8B-2 Commercial Resolution Foundation

Added normalized workspace entitlement envelopes, typed feature/limit values using the existing entitlement catalog, optional branch inherit/active/inactive records, and read-only effective workspace, branch, and commercial-state resolvers. Paid workspaces continue to depend on M7 confirmed local subscription evidence. No customer enforcement, demo lifecycle, conversion, membership, role, Paddle, onboarding, or Al-Andalus behavior changed.

Related docs:

```
docs/adr/0008-workspace-classification-foundation.md
docs/adr/0009-commercial-state-and-entitlement-resolution.md
docs/TIS_MASTER_CONTEXT.md
docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
```

TIS Landing Page Source of Truth

docs/marketing/landing_page_source_of_truth.md

Landing Page Source of Truth

Starter, Professional, and Enterprise AI pricing presentation is capacity-based. Normal customer feature availability is common across the three plans; AI consumption allowances may be configured separately. Public copy must not imply that ordinary modules or enabled AI availability require a higher plan.

The public header and relevant conversion areas expose Request a Demo, Subscribe, Sign In, and Open TIS App. All app links are constructed here from `NEXT_PUBLIC_TIS_APP_BASE_URL`; Sign In targets `/saas/login`, Open TIS App targets `/login`, and the landing site does not duplicate authentication or customer-state routing.

The hero Subscribe Now action remains within the marketing page and scrolls to `#pricing`. Starter, Professional, and Enterprise AI pricing actions use the configured application base URL and public `/saas/signup` route with distinct allowlisted preferred-plan codes. The application treats those codes as non-authoritative preferences and revalidates capacity after organization setup. Custom remains an email/contact action and never enters signup or checkout.

The official source of truth for the public TIS landing website is:

```
tis-landing-website/
```

Public Website

- **Domain:** https://tisplatform.com
- **Runtime:** Next.js / Node
- **Local testing URL:** http://localhost:3000

All future marketing landing page changes must be made inside `tis-landing-website/`.

Application Portal

The TIS application portal remains separate from the public landing website:

- **Domain:** https://app.tisplatform.com
- **Runtime:** FastAPI / Python with PostgreSQL

Legacy Landing Page Files

The former FastAPI/Jinja landing page files are now legacy:

- `templates/landing.html`
- `static/landing/landing.css`

Codex and other developers must not modify these legacy files unless explicitly instructed.

TIS Landing Page Master Content

docs/marketing/tis_landing_page_master_content.md

TIS Landing Page Master Content Approved Marketing Foundation

1 Main Positioning

TIS is a developing SaaS academic operations platform designed to connect school leadership, supervisors, teachers, branches, calendars, observations, staffing data, and decision-making inside one integrated system. The platform helps schools organize teacher information, analyze teaching loads, identify staffing gaps, coordinate shared academic calendars, manage structured observation cycles, support supervisor-teacher communication, and build clearer visibility across academic operations. As TIS continues to grow, advanced AI-powered features will be introduced through subscription-based plans, including curriculum-driven assessment generation, academic analytics, planning support, and intelligent recommendations based on verified school data.

1 Hero Section

Hero Badge Built for Schools Moving Beyond Spreadsheets Main Headline When Academic Decisions Are Scattered, Schools Lose Time, Clarity, and Control. Subheadline TIS brings teacher data, workloads, staffing gaps, academic calendars, observations, and leadership decisions into one connected SaaS platform - helping schools see what is happening, what is missing, and what needs action. CTA Buttons Request Early Access See How TIS Works Small Supporting Line A developing academic operations platform with advanced AI capabilities planned through subscription-based growth.

1 Problem Section

Section Label The Problem Main Heading Schools Are Not Short of Data. They Are Drowning in Disconnected Files. Section Text In many schools and educational organizations, every academic process becomes another spreadsheet. Teacher loads, subject coverage, observations, calendars, reports, staffing needs, and follow-up tasks are often managed in separate files, created by different people, in different formats. Over time, this creates a hidden operational burden. Staff members spend more time updating files than understanding the data. Leaders receive information late, inconsistently, or from multiple disconnected sources. In multi-branch organizations, the problem becomes even larger: every branch may work differently, every file may tell a different story, and every decision requires extra checking. Problem Cards

1 Too Many Spreadsheets

Academic work is scattered across different files, formats, and personal tracking systems.

1 Inconsistent Data

Each person or branch may structure information differently, making comparison and reporting difficult.

1 Heavy Staff Workload

Teachers, supervisors, and coordinators carry the burden of repeatedly entering and updating data.

1 Delayed Visibility

Leaders often discover staffing gaps, missing coverage, or academic issues after the problem has already grown.

1 Disconnected Decisions

Calendars, observations, staffing, reports, and academic planning are managed separately instead of working as one connected system.

1 Solution Section

Section Label The Solution Main Heading One Connected Platform to Bring Academic Operations Back Under Control. Section Text TIS brings the scattered pieces of school operations into one structured academic ecosystem. Instead of chasing separate spreadsheets, comparing different file formats, and waiting for updates from multiple sources, school leaders can work from a clearer, more connected view of what is happening across their school or organization. Teacher data, workloads, subject coverage, academic calendars, observations, supervision follow-up, branch structures, and leadership decisions become part of one system. This allows schools to reduce manual confusion, improve visibility, standardize academic processes, and make decisions with greater confidence. For multi-branch organizations, TIS helps create consistency across campuses while still allowing each branch to manage its own academic operations within a secure, organized, and tenant-isolated platform. Solution Cards

1 One Source of Academic Truth

Bring teacher data, workloads, observations, calendars, and staffing information into one connected system.

1 Clearer Leadership Visibility

Help principals, academic leaders, and supervisors see what is happening, what is missing, and what requires action.

1 Standardized Academic Processes

Reduce personal spreadsheet formats and create a more consistent way to manage academic operations across branches.

1 Smarter Staffing Decisions

Identify teaching load issues, uncovered hours, subject gaps, and staffing needs before they become larger problems.

1 Connected School Communication

Support better coordination between leadership, supervisors, teachers, and branches through shared academic workflows.

1 Ready for Future Intelligence

Build the structured data foundation needed for advanced AI-powered features as the platform continues to grow.

1 Core Platform Capabilities

Section Label Core Platform Capabilities Main Heading Everything Your Academic Operation Needs - Connected in One Platform. Section Intro TIS is designed to bring the most important academic operations into one structured SaaS environment. From teacher planning and branch management to observations, calendars, dashboards, and future AI-powered intelligence, the platform helps schools move from scattered work to connected academic control. Capability Cards

1 Academic Workforce Planning

Plan teacher assignments, analyze weekly loads, identify uncovered teaching hours, and understand staffing needs with greater clarity.

1 Academic Calendars & Shared Coordination

Coordinate academic events, timelines, school priorities, and branch-level planning through shared calendars that keep teams aligned.

1 Observation & Supervision Workflows

Manage structured teacher observations, supervisor feedback, shared records, and signing workflows inside one organized process.

1 Multi-Branch Organization Management

Support school groups, campuses, branches, users, roles, and tenant-isolated structures from one scalable SaaS platform.

1 Dashboards & Decision Visibility

Give leaders a clearer view of coverage, staffing gaps, teacher utilization, subject sufficiency, and academic follow-up needs.

1 Future AI-Powered Intelligence

Prepare for advanced subscription-based AI capabilities, including curriculum-driven assessment generation, answer keys, planning support, analytics, and intelligent recommendations based on verified academic data.

1 Teacher Workforce Planning Engine

Section Label Teacher Workforce Planning Engine Main Heading Know Exactly Who Is Teaching What - and Where Support Is Needed. Section Text One of the strongest current capabilities of TIS is its ability to help schools understand their teaching workforce with clarity. Instead of manually calculating teacher loads, subject coverage, and staffing needs across separate files, TIS brings this information into one structured planning engine. School leaders can track teacher qualifications, majors, subject specialties, weekly teaching loads, assigned subjects, uncovered hours, and additional staffing requirements. This helps academic teams identify gaps earlier, use existing teachers more effectively, and make staffing decisions based on clear data rather than assumptions. For schools and multi-branch organizations, this creates a more reliable way to plan academic coverage, compare staffing needs, and understand whether each subject, grade, and branch has the right teaching support. Key Capabilities

1 Teacher Load Visibility

See teacher workloads clearly and understand whether teaching hours are balanced, underused, or overloaded.

1 Subject Coverage Analysis

Identify which subjects are fully covered and which still have uncovered teaching hours.

1 Staffing Gap Detection

Detect shortages early before they affect schedules, instruction, or academic quality.

1 Qualification-Based Planning

Use teacher degrees, majors, and subject specialties to support smarter assignment decisions.

1 New Teacher Requirement Insights

Understand how many additional teachers may be needed based on uncovered hours and school workload requirements.

1 Multi-Branch Workforce Clarity

Support better staffing visibility across branches, campuses, and larger educational organizations.

1 Observation & Supervision Workflows

Section Label Observation & Supervision Workflows Main Heading Turn Teacher Observation into a Clear, Shared, and Accountable Process. Section Text TIS is designed to help schools move teacher observation away from scattered forms, disconnected files, and informal follow-up. Observation records, supervisor feedback, teacher responses, signatures, and approval steps can become part of one structured academic workflow. This gives supervisors a clearer way to document classroom visits, teachers a more transparent way to receive and respond to feedback, and school leaders a stronger view of supervision quality across departments and branches. As the platform continues to grow, advanced AI-assisted supervision features are planned to support supervisors by suggesting improvement plans based on observation notes, evaluation results, teacher performance evidence, and school-approved criteria. Key Capabilities

1 Structured Observation Records

Create organized observation forms and records that follow a clear school-approved process.

1 Shared Teacher-Supervisor Visibility

Allow teachers and supervisors to access relevant observation records, feedback, and follow-up actions.

1 Signing and Approval Workflow

Support formal documentation through review, acknowledgement, signing, and approval steps.

1 Supervision Follow-Up

Track feedback, recommendations, and improvement actions after classroom observations.

1 AI-Suggested Improvement Plans

Future AI-assisted tools may help supervisors generate suggested teacher improvement plans based on observation notes, evaluation data, and approved academic criteria.

1 Leadership Oversight

Give academic leaders better visibility over observation activity, supervision quality, and follow-up completion.

1 Reduced Observation Paperwork

Replace scattered observation files with a more consistent, traceable, and connected supervision system.

1 Academic Calendars & Shared Coordination

Section Label Academic Calendars & Shared Coordination Main Heading Turn the Academic Calendar into a Shared Planning Engine. Section Text TIS is designed to help schools move beyond static calendars and disconnected event planning. Academic leaders, supervisors, teachers, and branches can work from a shared academic calendar connected to the selected academic year, helping the whole school coordinate priorities, events, activities, observations, deadlines, and academic follow-up. Future calendar capabilities can support default worldwide and international academic events, allowing schools to plan meaningful activities around important global occasions. This gives international, national, private, and public schools a stronger way to connect school events with classroom learning, curriculum objectives, student engagement, and whole-school awareness. As TIS continues to grow,

AI-assisted calendar planning can help schools generate suggested activities for selected events, including classroom activities, outdoor activities, entertainment activities, curriculum-integrated tasks, and lesson-based ideas. The calendar vision also includes support for Sustainable Development Goals awareness planning, helping schools design year-round activities connected to SDGs, global citizenship, and curriculum integration. Key Capabilities

1 Academic-Year-Based Calendar

Build and manage school calendars according to the selected academic year, such as 2026-2027.

1 Shared School Coordination

Connect leaders, supervisors, teachers, and branches through one shared academic planning calendar.

1 International Event Awareness

Support worldwide and international academic events that schools can use for activities, campaigns, and curriculum connections.

1 AI-Suggested Activity Planning

Future AI-assisted tools may suggest activity plans based on selected events, school needs, grade levels, and curriculum links.

1 Curriculum-Integrated Events

Help schools connect calendar events with classroom learning, lesson planning, and academic objectives.

1 SDG Awareness Planning

Support year-round planning for Sustainable Development Goals awareness, student engagement, and global citizenship activities.

1 Branch-Level Visibility

Give school groups and multi-branch organizations a clearer view of academic events, priorities, and planning across campuses.

1 Multi-Branch Organization Management

Section Label Multi-Branch Organization Management Main Heading Built for Schools That Operate Across Branches, Campuses, and Teams. Section Text TIS is designed to support schools and educational organizations that need structure beyond a single location. Whether an organization manages one school, several branches, multiple campuses, or different academic pathways, TIS provides a scalable environment where users, roles, branches, and academic operations can be organized with greater clarity. School leadership and organization administrators can manage branches, users, permissions, branding, academic structures, and operational workflows inside a tenant-isolated SaaS environment. This helps every branch work within a shared organizational framework while still maintaining its own operational identity and academic needs. For educational groups, TIS can support stronger management-level visibility across branches. Leaders can monitor branch progress, compare completion levels, identify delayed tasks, and recognize where follow-up is needed across areas such as curriculum planning, exams, observations, calendars, reporting, and academic operations. Key Capabilities

1 Organization and Branch Structure

Manage organizations, schools, branches, campuses, and academic pathways inside one scalable platform.

1 Role-Based User Management

Assign users according to their responsibilities, such as organization administrator, branch leader, academic leader, supervisor, teacher, or future custom roles.

1 Tenant-Isolated Environment

Keep each organization's data, users, branding, and academic operations securely separated.

1 Branch Progress Intelligence

Give management-level users a clearer view of each branch's completion, progress, delays, and operational follow-up needs.

1 Branch Comparison Indicators

Compare branch performance across connected academic areas such as curriculum tasks, exams, observations, calendars, planning, and reports.

1 Custom Branding by Organization

Support organization and branch branding, including logos, visual identity, and school-specific presentation.

1 Scalable SaaS Structure

Prepare schools and educational groups for growth, future modules, subscription plans, and advanced AI-powered capabilities.

1 Dashboards & Decision Visibility

Section Label Dashboards & Decision Visibility Main Heading See Academic Progress, Risks, and Priorities Before They Become Problems. Section Text TIS dashboards are designed to give school leaders more than static reports. They provide a connected view of academic operations, helping leadership understand what has been completed, what is delayed, what needs follow-up, and where intervention may be required. As the platform continues to grow, dashboards can reflect progress across annual plans, weekly plans, curriculum coverage, assessment blueprints, quizzes, exams, academic calendar deadlines, teacher observations, supervision follow-up, staffing gaps, and branch-level performance. For branch management, dashboards can show the academic health of one school or campus. For organization-level leadership, dashboards can compare branches, highlight delayed tasks, identify performance gaps, and support faster, evidence-based decisions. Key Capabilities

1 Planning Completion Indicators

Track progress for annual plans, weekly plans, lesson planning, curriculum coverage, and completed academic tasks.

1 Assessment Progress Visibility

Monitor assessment blueprints, quizzes, exams, term-based assessment requirements, and calendar-linked due dates.

1 Observation & Supervision Insights

See observation progress, teacher follow-up status, supervisor actions, and areas requiring improvement plans.

1 Academic Calendar Integration

Connect events, deadlines, assessments, activities, and required tasks to dashboard indicators.

1 Teacher & Academic Performance Indicators

Support leadership with visual indicators related to teacher follow-up, academic progress, and school-level performance trends.

1 Branch and Organization Dashboards

Provide branch-level dashboards for school management and organization-level dashboards for comparing progress across multiple campuses.

1 Risk and Delay Alerts

Highlight delayed tasks, incomplete requirements, missing academic evidence, and branches or departments that need attention. Supporting Line: From reports to real-time academic visibility.

1 Future AI-Powered Intelligence

Section Label Future AI-Powered Intelligence Main Heading AI That Works Inside the Academic Workflow - Not Outside It. Section Text TIS is being developed with a future AI vision that goes beyond simple content generation. Instead of using AI as a separate tool, future AI-powered capabilities are planned to work inside the platform's academic workflows, supporting schools through the data already stored, reviewed, and approved in TIS. This future intelligence layer can support academic calendars, observation follow-up, curriculum planning, teacher support, assessment generation, dashboards, branch progress indicators, and leadership decision-making. At the center of this vision is a curriculum-to-classroom workflow. Uploaded curriculum data, standards, learning outcomes, annual plans, weekly plans, lesson plans, and confirmed instructional coverage can become the foundation for guided planning, AI-assisted recommendations, assessment generation, answer keys, and academic analytics. Future AI capabilities will be introduced progressively across the normal TIS customer product, subject to permissions, workspace scope, configured consumption allowances, and release readiness. Key Capabilities

1 Curriculum-to-Classroom AI Support

Assist teachers and academic leaders in moving from uploaded curriculum to annual plans, weekly plans, lesson plans, and verified coverage.

1 AI-Assisted Assessment Generation

Support future generation of exams, quizzes, assessment blueprints, and answer keys based on approved curriculum data and confirmed instructional coverage.

1 AI-Supported Observation Follow-Up

Help supervisors generate suggested teacher improvement plans based on observation notes, evaluation results, and school-approved criteria.

1 AI-Powered Calendar Planning

Suggest activities for international events, SDG awareness, curriculum links, classroom activities, and school-wide academic events.

1 Academic Analytics and Recommendations

Provide future intelligent insights related to planning progress, teacher follow-up, assessment readiness, curriculum coverage, and branch performance.

1 Managed AI Growth

Introduce advanced AI features progressively while controlling provider consumption, safety, and release readiness independently from plan capacity.

1 From Curriculum to Classroom - With Guided AI Support

Section Label From Curriculum to Classroom Main Heading Support Teachers Without Adding Another Burden. Section Text TIS is being developed to support teachers, not overload them. Instead of asking teachers to build annual plans, weekly plans, lesson plans, and curriculum tracking documents from scratch, future AI-assisted planning tools will help guide them through structured steps using approved templates, uploaded curriculum data, learning outcomes, standards, objectives, and school-specific requirements. Teachers will be able to review, adjust, complete, and submit their plans through the system. Academic leaders can then review and approve the work, turning planning documents into reliable, official academic data inside TIS. This helps reduce teacher workload, improve planning consistency, and give school leadership clearer visibility over what has been planned, approved, taught, and covered.

1 Subscription Plans Preview

Section Label Subscription Plans Main Heading Choose the Organization Capacity Your School Needs. Section Text TIS is a subscription-based organization SaaS platform. Starter, Professional, and Enterprise AI share the normal customer product baseline and primarily represent different organization capacities. Permissions, workspace scope, operational prerequisites, and separately configured AI consumption policy continue to control safe use. Plan Cards

1 Starter

For organizations within the Starter branch, staff-user, and teacher capacity ceilings.

1 Professional

For organizations that require the Professional branch, staff-user, and teacher capacity ceilings.

1 Enterprise AI

For larger organizations within the Enterprise AI branch, staff-user, and teacher capacity ceilings. Supporting Note AI-powered capabilities will be introduced progressively and remain subject to permissions, workspace scope, configured usage allowances, and platform development stage. CTA Request Early Access Request Pricing

1 Final Call-to-Action

Section Label Start Your TIS Journey Main Heading Ready to Bring Clarity, Control, and Intelligence to Your Academic Operations? Section Text TIS is being built for schools and educational organizations that want to move beyond scattered files, disconnected workflows, and delayed decisions. Start exploring how one connected academic operations platform can support your leadership team, supervisors, teachers, branches, and future AI-powered growth. CTA Buttons Request Early Access Book a Demo Supporting Line TIS is currently in active development, with selected features and advanced AI capabilities planned for progressive release through subscription-based plans.

TIS Location Data Roadmap

docs/location-data-roadmap.md

Phase 1: Global Form Usability

Status: implemented in the current working tree.

Phase 1 intentionally keeps location data attached directly to an organization or branch. It does not create shared locality records.

Included:

- Controlled country selection from the local dataset.
- Controlled region/state/province selection when dataset coverage exists.
- Dataset-backed city/locality selection.
- `Other / manual entry` fallback for missing regions and cities.
- Optional district and neighborhood fields, stored separately from city.
- Local cached APIs; no external location API dependency at runtime.
- Existing text location fields remain compatible with legacy Saudi data.

Manual region and city values are record-level values. They are not published to other tenants and are not added to the imported dataset.

Deferred Architecture

The following phases are roadmap items only. They are not part of Phase 1 and must not be inferred from the current text-based storage model.

Locality Registry

Create stable TIS locality identifiers and canonical locality records that are independent from vendor dataset identifiers. Organizations and branches would eventually reference these records while retaining display-name snapshots for audit and migration compatibility.

Locality Aliases

Support native names, translated names, transliterations, spelling variants, and historical names. Alias matching should be region-aware and must not merge places solely because their normalized names match.

GeoNames and Official Dataset Enrichment

Build an offline enrichment process using GeoNames country dumps and, where available, official national gazetteers. Source priority, licensing, attribution, place classification, and duplicate resolution must be defined before combining records.

Tenant-Private City Additions

Allow an authorized tenant user to suggest a missing locality. New records should initially be visible only to that tenant and must be constrained to the tenant's organization scope.

Platform Moderation

Provide a Platform Owner moderation queue for reviewing proposed localities, potential duplicates, coordinates, place type, aliases, and source evidence.

City Verification and Promotion

Support explicit states such as tenant-private, pending, verified, deprecated, and merged. Promotion to the global catalog must be an audited platform action, never an automatic consequence of a tenant entering free text.

Dataset Refresh and ETL Workflow

Introduce versioned, offline imports with source checksums, staging tables, coverage metrics, diff review, attribution records, and rollback support. Referenced localities should be deprecated or merged rather than deleted.

Guardrails for Future Work

1 Preserve tenant isolation for all tenant-created locality data.

1 Keep stable internal IDs separate from external source IDs.

1 Retain source and license provenance for every imported record.

1 Do not call public geocoding services from normal page requests.

1 Do not modify the vendor JSON to store tenant-entered values.

1 Migrate existing organization and branch text values without data loss.

Students + Talent & Potential Update Package Release Handoff (M1-M13)

docs/releases/2026-09-23-students-talent-m1-m13-release-handoff.md

Superseded in part by M14 (2026-09-23): the "Learning Style" bullet below described the M1-M13 as-shipped four-percentage model, which the M14 owner correction determined was a misinterpretation. See `docs/PROJECT_STATE.md`'s M14 entry and ADR 0031/0042/0044's M14 amendment sections for the corrected model (one categorical field, eight values; aggregate-only percentage semantics). This document's text below is left unmodified as the accurate historical record of what M1-M13 actually shipped.

Milestone range: M1-M13, 2026-09-23. This is a documentation-only closeout record for a bounded package on the `dev` branch. It does not itself deploy anything; deployment remains a separately authorized decision. It does not declare the entire Talent product roadmap complete - only this specific Students + Talent & Potential update package (Student ID, Learning Style, Al-Andalus Section display, Student roster, Talent frontend cleanup, Evaluation Progress, Results & Analytics Branch comparison, and their supporting permissions/migrations).

What Changed (User-Visible)

1 **Student ID**: New Students require a canonical `STD` + 10-digit Student

ID (user enters the 10 digits; leading zeros preserved). Same-tenant duplicates disclose identity through a privacy-safe projection; cross-tenant conflicts stay generic. Legacy Students without a number remain editable and may receive one later via `students.manage_identifiers`.

1 **Learning Style**: Student create/edit/profile expose four independent,

nullable 0-100 Verbal/Non-verbal/Quantitative/Spatial percentages. No normalization, no sum-to-100 rule, and no automatic conversion from the deprecated legacy categorical field.

1 **Al-Andalus Section display**: One authorized workspace

(`72e52eb2-3844-447b-92a8-c55015f73257`) sees a presentation-only Section label projection; the canonical `PlanningSection` record and every other tenant are unchanged.

1 **Student roster**: `.xlsx`-only export and stateless-preview/atomic-apply

import, gated by `students.import`/`students.export`. Create-only in this release; no update/merge/upsert.

1 **Talent frontend cleanup**: Reload Saved Rubric and Educator Input are no

longer exposed in the normal Talent assessment UI. Historical evidence and the underlying APIs/permissions are unchanged and still reachable through existing authorized paths.

1 **Evaluation Progress**: Student Profile Talent tab shows per-Period

results (Pending distinguished from a real 0%) and a backend-authoritative combined Overall Result, blocked when the Program's Framework changed across contributing active Periods.

1 **Results & Analytics**: One primary Organization Overview Branch

comparison chart over the seven approved metric families, including the four Learning Style dimensions, with server-authoritative privacy suppression and no client-side reconstruction or averaging.

Permission Changes

No new permission keys were introduced by this closeout task. The package's existing governed keys remain as documented in `docs/PROJECT_STATE.md`: `students.import`, `students.export`, `students.manage_identifiers`, and the pre-existing `students.*`/`talent_*` CRUD and analytics permission families (including `talent_learner_profiles.view` for individually authorized Student Evaluation Progress, and `talent_analytics.view` / `talent_analytics.view_students` for Branch/Organization aggregates). Permission grants remain backend-authoritative; frontend visibility is advisory only.

Migration Requirements (Not Migration-Free)

This package includes real schema migrations. A target PostgreSQL environment that has not yet applied them **requires running the migration ledger** as part of deployment - this release is not migration-free:

1 `20260922_001_student_learning_style_four_dimension_profile` - adds the

four Learning Style columns/CHECK constraints to `Student`.

1 `20260922_002_student_tis_number_identifier_integrity` - installs the

managed `tis_student_number` global-uniqueness and one-active-per-Student partial unique indexes (ADR 0043), now creating the canonical <=63-character active-index name directly on any fresh database.

1 `20260923_001_student_tis_number_index_identifier_length_remediation`

(M12.1) - PostgreSQL-only forward remediation that renames an already-migrated database's truncated 63-character physical index (`uq_student_external_identifiers_tis_student_number_active_stude`) to the canonical `uq_student_external_identifiers_tis_number_active_student` (57 characters). No-op if the canonical name is already present or on a fresh database. Fails closed (never guesses) on an unexpected index definition or if both names exist.

All three are additive/idempotent and run through the existing ordered migration ledger (`db_migrations.py` / `scripts/run_migrations.py`); none rewrites, merges, or deletes `Student`/`StudentExternalIdentifier` rows. SQLite (including the checked-in `tis.db`) never truncated the index name and is unaffected by the M12.1 remediation.

PostgreSQL Remediation Requirement

Any PostgreSQL environment where `20260922_002` has already run (i.e. any already-migrated PostgreSQL deployment, including a hypothetical prior partial production rollout of this package) **must** apply `20260923_001` before or during the same deployment that brings this package live, or a subsequent full-ledger re-apply against that database risks a `DuplicateTable` failure on the truncated legacy index name. A fresh PostgreSQL database (no prior run of `20260922_002`) does not need special handling - the canonical name is created directly.

Compatibility Notes

1 No CSV or `.xls` roster support - `.xlsx` only.

1 Roster import is create-only in this release; no update, merge, sync, or

upsert semantics exist.

1 Learning Style values are never snapshotted into history and never

auto-derived from the deprecated legacy categorical field.

1 No Student or Talent schema/data deletion is part of this package. The

M12.1 remediation only renames a PostgreSQL index in place.

1 `Student.id` is unchanged by any Student ID or replacement operation.

Privacy / Tenant-Isolation Invariants (Reconfirmed)

1 Permission possession is distinct from tenant/Branch scope; backend

authorization remains authoritative and frontend visibility is advisory only.

1 Tenant isolation is enforced at the query/service layer; the roster

workflow cannot create or match Students across a different tenant.

1 Same-tenant Student-number duplicate disclosure is privacy-safe;

cross-tenant conflicts remain generic (no cross-tenant identity leak).

1 Results & Analytics privacy suppression (primary + complementary) is

server-authoritative; the browser performs no reconstruction, averaging, or normalization of protected values.

1 Frozen historical Branch/Grade/Section attribution used by Evaluation

Progress and Talent evidence is preserved and not reinterpreted by current Placement.

Known Pre-Existing Issues (Not Introduced By This Package, Not Fixed Here)

1 Five PostgreSQL migration-transaction test failures, reconfirmed present

and unchanged in this closeout task's own test run (2026-09-23): `test_planning_subject_demand_migration_creates_fk_target_constraints_first`, `test_teacher_rule_migration_is_safe_after_baseline_metadata_and_idempotent`, `test_student_talent_prerequisite_unblocks_full_fresh_chain_and_is_idempotent`, `test_postgresql_failed_preledger_create_all_rolls_back_partial_student_tables`, `test_student_learning_style_profile_and_tis_number_migrations_apply_and_are_idempotent_under_postgresql` in `tests/test_postgresql_migration_transactions.py`. Root cause: a pre-existing, unrelated Talent baseline-metadata foreign-key ordering defect (`UndefinedTable` for `talent_framework_competencies` when creating `talent_grade_competency_rubric_descriptors`), first documented during M12.1's own verification. These do not exercise this package's own Student-ID/Learning-Style migrations in isolation (the dedicated Student/Learning-Style PostgreSQL migration test failing here fails only because it shares the same broader fixture chain) and are tracked as an open, out-of-scope risk against the broader migration test fixture, not against this package's own schema changes.

1 A small number of previously-documented stale test-fixture/wording issues

(for example the M12-corrected `tests/test_permission_registry_matrix.py` consumer list) remain recorded historically in `docs/CHANGE_HISTORY.md` and `docs/PROJECT_STATE.md` and are not restated as open here because M12 already fixed that specific fixture.

None of the above blocks this package's own migrations, which were verified directly (see Verification Performed).

Deployment Surfaces

1 **Web Service (Render)**: owns `main.py`/FastAPI, all Students/Talent

routers, templates, and static JS/CSS in this package. Requires the migration ledger above to be applied via the existing Pre-Deploy Command (`python scripts/run_migrations.py`, per ADR 0016) before the new application version activates.

1 **Database migrations**: applied by the same Pre-Deploy Command, not by

the web process at runtime or import time (ADR 0016). This package's three migrations are additive/idempotent and safe to re-run.

1 `tis-timetable-workflow` **(separate Render revision)**: this package is

Students/Talent schema and Web Service application code only. It does not touch timetable solver/worker tables, contracts, or code, and does not require a coordinated Web + Workflow deploy.

Migration / Deployment Sequence (Plan Only - Not Executed)

1 Confirm target environment and take/confirm the platform's standard

PostgreSQL backup/snapshot before deploying (per existing operational practice; no new backup mechanism is introduced by this package).

1 Deploy the reviewed `dev`-equivalent build so Render's Pre-Deploy Command

runs `python scripts/run_migrations.py` against the target database.

1 Migration runner applies any still-pending entries from the ordered

ledger, including (if not already applied) the three migrations listed above, in ledger order. Each migration and its ledger marker share one transaction; failure blocks activation with a nonzero exit and no partial marker/data write.

1 Render activates the new Web Service version only after the Pre-Deploy

Command exits zero.

1 No `tis-timetable-workflow` deployment step is required by this package.

Rollback Considerations

1 Application rollback: redeploy the prior Web Service revision. The

package's migrations are additive (new columns/indexes) and do not remove or rename any column or table the prior revision depends on, so a prior application revision continues to function against the migrated schema.

1 Migration rollback: no down-migration is provided (consistent with this

repository's existing forward-only migration convention). If the M12.1 index rename must be reversed, it would require a new forward migration performing `ALTER INDEX ... RENAME TO` back to the truncated name - not a destructive drop/recreate - but no such reversal is expected to be needed since both names satisfy the same uniqueness/partial-predicate invariant.

1 If a migration fails mid-ledger, the failing migration's own transaction

rolls back cleanly (no partial marker or partial DDL); the prior application revision remains safe to keep running against the not-yet-fully-migrated database until the defect is fixed and redeployed.

Post-Deploy Smoke Test Matrix (To Run After A Separately Authorized Deploy)

Create Student with Student ID; leading-zero Student ID; duplicate ID privacy-safe same-tenant disclosure; duplicate ID generic cross-tenant conflict; edit Student Learning Style; Learning Style null vs 0; Al-Andalus Section display on the authorized workspace; non-Al-Andalus Section display unchanged; roster export; roster preview; roster apply; atomic rejection of an invalid roster; Talent Student profile without Reload Saved Rubric; Talent Student profile without Educator Input; Evaluation Progress Pending state; valid Overall Result; framework-mismatch comparability boundary; Branch comparison chart renders; privacy-suppressed Branch cell renders categorically; permission-denied import/export is blocked; cross-tenant isolation sanity check (no Student/roster/analytics leakage across tenant).

Verification Performed (This Closeout Task, 2026-09-23)

1 `tests/test_permission_registry_matrix.py`: 13 passed.

1 `tests/test_student_managed_number_service.py`: 46 passed.

1 `tests/test_postgresql_migration_transactions.py` (live PostgreSQL via

`TIS_TEST_POSTGRESQL_URL`): 11 passed, 5 failed - the exact five pre-existing failures listed above, reconfirmed unchanged.

1 Code-level spot checks (not exhaustive) confirmed KMS claims against

current implementation: `models.py` Learning Style columns/constraints and the canonical <=63-character managed-index name; `db_migrations.py` migration ledger ordering (`20260922_001` -> `20260922_002` -> `20260923_001`); the Al-Andalus workspace UUID in `academic_grade.py` and ADR 0045; `routers/students.py` permission checks for `students.import`/`students.export`; `static/js/talent-operations.js` confirming both the `button('reload', ...)` no-op and the `talent_educator_inputs.*` permission short-circuit that remove Reload Saved Rubric and Educator Input from the normal UI; and `static/js/student-evaluation-progress.js`/`static/js/talent.js` confirming backend-authoritative Overall Result and framework-mismatch comparability handling.

1 `git diff --check`: no whitespace errors in this task's changes.

1 `tis.db` SHA-256 confirmed unchanged before and after this task

(`01e1a3065d92280ee9228db921f67545c8b837d4ecd787a5aeeddc6d128fc136`).

Full 1,700+ test suite was intentionally not rerun for this closeout, per task scope; M12/M12.1 test evidence is current and only this bounded documentation closeout happened since.

Deployment Readiness Statement

This documentation-only closeout does not itself authorize or perform a deployment. Based on the verification above, the package is internally consistent, documented, tested to the extent proportional to a documentation closeout, migration-safe (including the M12.1 PostgreSQL remediation), and has no newly discovered release-blocking defect. Actual deployment remains a separate, explicitly authorized decision by the repository owner.